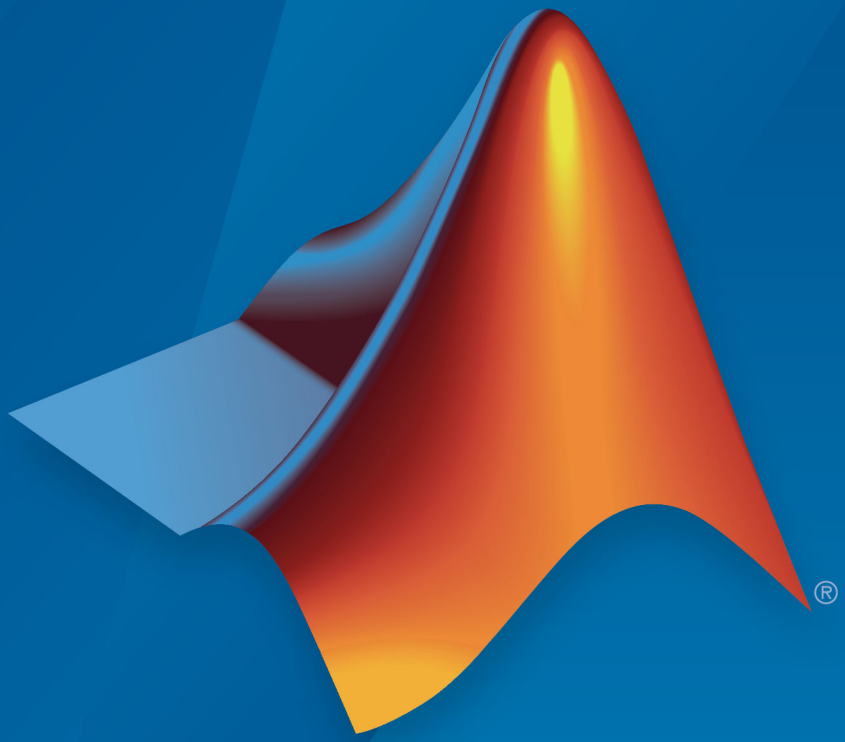
Signal Processing Toolbox™

User's Guide



MATLAB®

R2018b

MathWorks®

How to Contact MathWorks

Latest news: `www.mathworks.com`

Sales and services: `www.mathworks.com/sales_and_services`

User community: `www.mathworks.com/matlabcentral`

Technical support: `www.mathworks.com/support/contact_us`

Phone: 508-647-7000

The MathWorks, Inc.
3 Apple Hill Drive
Natick, MA 01760-2098

Signal Processing Toolbox™ User's Guide

© COPYRIGHT 1988–2018 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See `www.mathworks.com/trademarks` for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see `www.mathworks.com/patents` for more information.

Revision History

1988	First printing	New
November 1997	Second printing	Revised
January 1998	Third printing	Revised
September 2000	Fourth printing	Revised for Version 5.0 (Release 12)
July 2002	Fifth printing	Revised for Version 6.0 (Release 13)
December 2002	Online only	Revised for Version 6.1 (Release 13+)
June 2004	Online only	Revised for Version 6.2 (Release 14)
October 2004	Online only	Revised for Version 6.2.1 (Release 14SP1)
March 2005	Online only	Revised for Version 6.2.1 (Release 14SP2)
September 2005	Online only	Revised for Version 6.4 (Release 14SP3)
March 2006	Sixth printing	Revised for Version 6.5 (Release 2006a)
September 2006	Online only	Revised for Version 6.6 (Release 2006b)
March 2007	Online only	Revised for Version 6.7 (Release 2007a)
September 2007	Online only	Revised for Version 6.8 (Release 2007b)
March 2008	Online only	Revised for Version 6.9 (Release 2008a)
October 2008	Online only	Revised for Version 6.10 (Release 2008b)
March 2009	Online only	Revised for Version 6.11 (Release 2009a)
September 2009	Online only	Revised for Version 6.12 (Release 2009b)
March 2010	Online only	Revised for Version 6.13 (Release 2010a)
September 2010	Online only	Revised for Version 6.14 (Release 2010b)
April 2011	Online only	Revised for Version 6.15 (Release 2011a)
September 2011	Online only	Revised for Version 6.16 (Release 2011b)
March 2012	Online only	Revised for Version 6.17 (Release 2012a)
September 2012	Online only	Revised for Version 6.18 (Release 2012b)
March 2013	Online only	Revised for Version 6.19 (Release 2013a)
September 2013	Online only	Revised for Version 6.20 (Release 2013b)
March 2014	Online only	Revised for Version 6.21 (Release 2014a)
October 2014	Online only	Revised for Version 6.22 (Release 2014b)
March 2015	Online only	Revised for Version 7.0 (Release 2015a)
September 2015	Online only	Revised for Version 7.1 (Release 2015b)
March 2016	Online only	Revised for Version 7.2 (Release 2016a)
September 2016	Online only	Revised for Version 7.3 (Release 2016b)
March 2017	Online only	Revised for Version 7.4 (Release 2017a)
September 2017	Online only	Revised for Version 7.5 (Release 2017b)
March 2018	Online only	Revised for Version 8.0 (Release 2018a)
September 2018	Online only	Revised for Version 8.1 (Release 2018b)

Contents

1 Filtering, Linear Systems and Transforms Overview

Filter Implementation and Analysis 1-2
- Filtering Overview 1-2
- Convolution and Filtering 1-2
- Filters and Transfer Functions 1-3
- Filtering with the filter Function 1-4

The filter Function 1-6

Multirate Filter Bank Implementation 1-8

Frequency Domain Filter Implementation 1-9

Anti-Causal, Zero-Phase Filter Implementation 1-10

Impulse Response 1-13

Frequency Response 1-17
- Digital Domain 1-17
- Analog Domain 1-24

Phase Response 1-27

Delay 1-31

Zero-Pole Analysis 1-33

Discrete-Time System Models 1-37
- Transfer Function 1-37
- Zero-Pole-Gain 1-37
- State Space 1-38
- Partial Fraction Expansion (Residue Form) 1-39
- Second-Order Sections (SOS) 1-40

Lattice Structure 1-41
Convolution Matrix 1-43

Continuous-Time System Models 1-45

Linear System Transformations 1-47

Discrete Fourier Transform 1-49

2 Filter Design and Implementation

Filter Requirements and Specification 2-2

IIR Filter Design 2-5
IIR vs. FIR Filters 2-5
Classical IIR Filters 2-5
Other IIR Filters 2-5
IIR Filter Method Summary 2-6
Classical IIR Filter Design Using Analog Prototyping 2-7
Comparison of Classical IIR Filter Types 2-10

FIR Filter Design 2-19
FIR vs. IIR Filters 2-19
FIR Filter Summary 2-20
Linear Phase Filters 2-20
Windowing Method 2-21
Multiband FIR Filter Design with Transition Bands 2-25
Constrained Least Squares FIR Filter Design 2-31
Arbitrary-Response Filter Design 2-36

Special Topics in IIR Filter Design 2-42
Classic IIR Filter Design 2-42
Analog Prototype Design 2-42
Frequency Transformation 2-43
Filter Discretization 2-45

Filtering Data With Signal Processing Toolbox Software 2-51

Selected Bibliography 2-70

3 Designing a Filter in fdesign — Process Overview

Process Flow Diagram and Filter Design Methodology **3-2**
- Exploring the Process Flow Diagram **3-2**
- Selecting a Response **3-4**
- Selecting a Specification **3-4**
- Selecting an Algorithm **3-5**
- Customizing the Algorithm **3-7**
- Designing the Filter **3-7**
- Design Analysis **3-8**
- Realize or Apply the Filter to Input Data **3-9**

4 Designing a Filter in the Filter Builder GUI

Filter Builder Design Process **4-2**
- Introduction to Filter Builder **4-2**
- Design a Filter Using Filter Builder **4-2**
- Select a Response **4-2**
- Select a Specification **4-5**
- Select an Algorithm **4-5**
- Customize the Algorithm **4-6**
- Analyze the Design **4-8**
- Realize or Apply the Filter to Input Data **4-8**

Designing a FIR Filter Using filterBuilder **4-10**
- FIR Filter Design **4-10**

Compensate for Delay and Distortion Introduced by Filters **4-13**

Comparison of Analog IIR Lowpass Filters **4-20**

Frequency Response of a Lowpass Bessel Filter **4-22**

Speaker Crossover Filters **4-24**

5 Filter Designer: A Filter Design and Analysis App

Filter Design Methods **5-2**
Advanced Filter Design Methods **5-2**

Using the Filter Designer App **5-4**

Analyzing Filter Responses **5-5**

Filter Designer App Panels **5-6**

Getting Help **5-7**

Getting Started with Filter Designer **5-8**
Choosing a Response Type **5-10**
Choosing a Filter Design Method **5-10**
Setting the Filter Design Specifications **5-10**
Computing the Filter Coefficients **5-12**
Analyzing the Filter **5-13**
Editing the Filter Using the Pole/Zero Editor **5-15**
Converting the Filter Structure **5-16**
Exporting a Filter Design **5-18**
Generating a C Header File **5-22**
Generating MATLAB Code **5-23**
Managing Filters in the Current Session **5-24**
Saving and Opening Filter Design Sessions **5-25**

Importing a Filter Design **5-26**
Import Filter Panel **5-26**
Filter Structures **5-27**

FIR Bandpass Filter with Asymmetric Attenuation **5-29**

Arbitrary Magnitude Filter **5-32**

6 Statistical Signal Processing

Correlation and Covariance **6-2**
Background Information **6-2**
Using xcorr and xcov Functions **6-3**
Bias and Normalization **6-3**
Multiple Channels **6-4**

Spectral Analysis **6-5**
Background Information **6-5**
Spectral Estimation Method **6-6**

Nonparametric Methods **6-9**
Periodogram **6-9**
Performance of the Periodogram **6-11**
The Modified Periodogram **6-18**
Welch's Method **6-21**
Bias and Normalization in Welch's Method **6-24**
Multitaper Method **6-25**
Cross-Spectral Density Function **6-28**
Transfer Function Estimate **6-29**
Coherence Function **6-31**

Parametric Methods **6-34**
Yule-Walker AR Method **6-36**
Burg Method **6-38**
Covariance and Modified Covariance Methods **6-42**
MUSIC and Eigenvector Analysis Methods **6-44**
Eigenanalysis Overview **6-44**

Selected Bibliography **6-47**

7 Special Topics

Windows **7-2**
Why Use Windows? **7-2**
Available Window Functions **7-2**

Graphical User Interface Tools 7-3
Basic Shapes 7-3

Getting Started with Window Designer 7-8
Window Parameters 7-10
Window Designer Menus 7-11

Generalized Cosine Windows 7-13

Kaiser Window 7-16
Kaiser Windows in FIR Design 7-22

Chebyshev Window 7-24

Parametric Modeling 7-26
What is Parametric Modeling 7-26
Available Parametric Modeling Functions 7-26
Time-Domain Based Modeling 7-27
Frequency-Domain Based Modeling 7-30

Resampling 7-33
Available Resampling Functions 7-33
resample Function 7-33
decimate and interp Functions 7-35
upfirdn Function 7-35
spline Function 7-35

Cepstrum Analysis 7-36

FFT-Based Time-Frequency Analysis 7-40

Cross-Spectrogram of Complex Signals 7-44

Find Interference Using Persistence Spectrum 7-47

Find and Track Ridges Using Reassigned Spectrogram 7-52

Extract Regions of Interest from Whale Song 7-57

Median Filtering 7-62

Communications Applications 7-63
Modulation 7-63

Demodulation 7-64
Voltage Controlled Oscillator 7-67

Deconvolution 7-68

Chirp Z-Transform 7-69

Discrete Cosine Transform 7-72

Hilbert Transform 7-77

Walsh-Hadamard Transform 7-80

Walsh-Hadamard Transform for Spectral Analysis and Compression of ECG Signals 7-83

Eliminate Outliers Using Hampel Identifier 7-86

Selected Bibliography 7-89

8 SPTool: A Signal Processing GUI Suite

SPTool: An Interactive Signal Processing Environment 8-2
SPTool Overview 8-2
SPTool Data Structures 8-2

Opening SPTool 8-4

Getting Context-Sensitive Help 8-6

Signal Browser 8-7
Overview of the Signal Browser 8-7
Opening the Signal Browser 8-7

Filter Visualization Tool 8-10
Connection between FVTool and SPTool 8-10
Opening the Filter Visualization Tool 8-10
Analysis Parameters 8-11

Spectrum Viewer **8-12**
Spectrum Viewer Overview **8-12**
Opening the Spectrum Viewer **8-12**

Filtering and Analysis of Noise **8-15**
Overview **8-15**
Importing a Signal into SPTool **8-15**
Designing a Filter **8-17**
Applying a Filter to a Signal **8-19**
Analyzing a Signal **8-21**
Spectral Analysis in the Spectrum Viewer **8-23**

Exporting Signals, Filters, and Spectra **8-26**
Opening the Export Dialog Box **8-26**
Exporting a Filter to the MATLAB Workspace **8-27**

Accessing Filter Parameters **8-28**
Accessing Filter Parameters in a Saved Filter **8-28**
Accessing Parameters in a Saved Spectrum **8-29**

Importing Filters and Spectra **8-30**
Similarities to Other Procedures **8-30**
Importing Filters **8-30**
Importing Spectra **8-32**

Loading Variables from the Disk **8-34**

Saving and Loading Sessions **8-35**
SPTool Sessions **8-35**
Filter Formats **8-35**

Selecting Signals, Filters, and Spectra **8-37**

Editing Signals, Filters, or Spectra **8-38**

Making Signal Measurements with Markers **8-39**

Setting Preferences **8-41**
Overview of Setting Preferences **8-41**
Summary of Settable Preferences **8-42**

9 Code Generation from MATLAB Support in Signal Processing Toolbox

Supported Functions **9-2**

Specifying Inputs in Code Generation from MATLAB **9-6**
Defining Input Size and Type **9-6**
Inputs must be Constants **9-7**

Code Generation Examples **9-10**
Apply Window to Input Signal **9-10**
Apply Lowpass Filter to Input Signal **9-12**
Cross Correlate or Autocorrelate Input Data **9-12**
freqz With No Output Arguments **9-13**
Zero Phase Filtering **9-14**

10 Convolution and Correlation

Linear and Circular Convolution **10-2**

Confidence Intervals for Sample Autocorrelation **10-5**

Residual Analysis with Autocorrelation **10-7**

Autocorrelation of Moving Average Process **10-17**

Cross-Correlation of Two Moving Average Processes **10-21**

Cross-Correlation of Delayed Signal in Noise **10-23**

Cross-Correlation of Phase-Lagged Sine Wave **10-26**

11 Multirate Signal Processing

Downsampling — Signal Phases 11-2

Downsampling — Aliasing 11-6

Filtering Before Downsampling 11-13

Upsampling — Imaging Artifacts 11-16

Filtering After Upsampling — Interpolation 11-19

Simulate a Sample-and-Hold System 11-22

Changing Signal Sample Rate 11-28

12 Spectral Analysis

Power Spectral Density Estimates Using FFT 12-2

Bias and Variability in the Periodogram 12-11

Cross Spectrum and Magnitude-Squared Coherence 12-22

Amplitude Estimation and Zero Padding 12-25

Significance Testing for Periodic Component 12-29

Frequency Estimation by Subspace Methods 12-32

Frequency-Domain Linear Regression 12-35

Measure Total Harmonic Distortion 12-46

Measure Mean Frequency, Power, Bandwidth 12-49

Periodogram of Data Set with Missing Samples 12-55

Welch Spectrum Estimates 12-59

13 Linear Prediction

Prediction Polynomial 13-2

Formant Estimation with LPC Coefficients 13-6

AR Order Selection with Partial Autocorrelation Sequence 13-10

14 Transforms

Complex Cepstrum -- Fundamental Frequency Estimation 14-2

Analytic Signal for Cosine 14-7

Envelope Extraction 14-11

Analytic Signal and Hilbert Transform 14-20

Hilbert Transform and Instantaneous Frequency 14-26

Detect Closely Spaced Sinusoids 14-33

Instantaneous Frequency of Complex Chirp 14-42

Single-Sideband Amplitude Modulation 14-46

DCT for Speech Signal Compression 14-55

15 Signal Generation

Display Time-Domain Data in Signal Browser **15-2**
Import and Display Signals **15-3**
Configure the Signal Browser Properties **15-6**
Modify the Signal Browser Display **15-9**
Inspect Your Data (Scaling the Axes and Zooming) **15-11**

16 Signal Measurement

RMS Value of Periodic Waveforms **16-2**

Slew Rate of Triangular Waveform **16-6**

Duty Cycle of Rectangular Pulse Waveform **16-10**

Estimate State for Digital Clock **16-14**

Calculate Settling Time with Signal Browser **16-17**

Find Peak Amplitudes in Signal Browser **16-22**

Distortion Measurements **16-26**

Prominence **16-31**

Determine Peak Widths **16-34**

Human Activity Recognition Simulink Model for Smartphone Deployment **16-38**

17 Spectrum Object to Function Replacement

Nonparametric Spectrum Object to Function Replacement **17-2**
Periodogram PSD Object to Function Replacement Syntax . . **17-2**
Periodogram MSSPECTRUM Object to Function Replacement Syntax **17-3**
Welch PSD Object to Function Replacement Syntax **17-4**
Welch MSSPECTRUM Object to Function Replacement Syntax **17-6**
Multitaper PSD Object to Function Replacement Syntax **17-9**

Autoregressive PSD Object to Function Replacement Syntax **17-11**

Subspace Pseudospectrum Object to Function Replacement Syntax **17-13**

18 Vibration Analysis

Frequency-RPM Map of Helicopter Vibration Data **18-2**

Find Ridge of Noisy Signal **18-6**

Modal Parameters of MIMO System **18-10**

Compute and Display Order-RPM Map **18-15**

MIMO Stabilization Diagram **18-18**

Modal Analysis of Identified Models **18-23**

19 Signal Analyzer App

Using Signal Analyzer App **19-2**
Select Signals to Analyze **19-4**
Preprocess Signals **19-7**
Explore Signals **19-12**
Share Analysis **19-19**
Customize Signal Analyzer **19-21**

Find Delay Between Correlated Signals **19-27**

Plot Signals from the Command Line **19-32**

Resolve Tones by Varying Window Leakage **19-36**

Resolve Tones by Varying Window Leakage **19-42**

Analyze Signals with Inherent Time Information **19-44**

Spectrogram View of Dial Tone Signal **19-47**

Find Interference Using Persistence Spectrum **19-50**

Extract Regions of Interest from Whale Song **19-56**

Find and Track Ridges Using Reassigned Spectrogram **19-62**

Scalogram of Hyperbolic Chirp **19-68**

Extract Voices from Music Signal **19-72**

Resample and Filter a Nonuniformly Sampled Signal **19-79**

Declip Saturated Signals Using Your Own Function **19-87**

Edit Sample Rate and Other Time Information **19-93**

Data Types Supported by Signal Analyzer **19-99**
Numeric Data **19-99**
MATLAB Timetables **19-99**
timeseries Objects **19-100**

Nonuniformly Sampled Signals 19-100

Spectrum Computation in Signal Analyzer 19-102
Spectral Windowing 19-102
Parameter and Algorithm Selection 19-103
Zooming 19-105

Persistence Spectrum in Signal Analyzer 19-107

Spectrogram Computation in Signal Analyzer 19-110
Divide Signal into Segments 19-112
Window the Segments and Compute Spectra 19-116
Display Spectrum Power 19-116

Scalogram Computation in Signal Analyzer 19-119
Divide the Signal into Segments 19-120
Compute the Continuous Wavelet Transform 19-121
Display the Scalogram 19-122

Keyboard Shortcuts for Signal Analyzer 19-125
General Actions 19-125
Zooming 19-125
Data Cursors 19-125

Signal Analyzer Tips and Limitations 19-128
Select Signals to Analyze 19-128
Preprocess Signals 19-130
Explore Signals 19-130
Share Analysis 19-130

20 Common Applications

Create Uniform and Nonuniform Time Vectors 20-2

Remove Trends from Data 20-5

Remove the 60 Hz Hum from a Signal 20-9

Remove Spikes from a Signal 20-14

Process a Signal with Missing Samples **20-17**

Reconstruct a Signal from Irregularly Sampled Data **20-23**

Align Signals with Different Start Times **20-28**

Align Signals Using Cross-Correlation **20-33**

Align Two Simple Signals **20-39**

Find Peaks in Data **20-45**

Find a Signal in a Measurement **20-51**

Find Periodicity Using Autocorrelation **20-59**

Extract Features of a Clock Signal **20-64**

Find Periodicity in a Categorical Time Series **20-72**

Compensate for the Delay Introduced by an FIR Filter **20-79**

Compensate for the Delay Introduced by an IIR Filter **20-84**

Take Derivatives of a Signal **20-88**

Find Periodicity Using Frequency Analysis **20-96**

Detect a Distorted Signal in Noise **20-100**

Measure the Power of a Signal **20-106**

Compare the Frequency Content of Two Signals **20-110**

Detect Periodicity in a Signal with Missing Samples **20-114**

A Technical Conventions

Filtering, Linear Systems and Transforms Overview

- “Filter Implementation and Analysis” on page 1-2
- “The filter Function” on page 1-6
- “Multirate Filter Bank Implementation” on page 1-8
- “Frequency Domain Filter Implementation” on page 1-9
- “Anti-Causal, Zero-Phase Filter Implementation” on page 1-10
- “Impulse Response” on page 1-13
- “Frequency Response” on page 1-17
- “Phase Response” on page 1-27
- “Delay” on page 1-31
- “Zero-Pole Analysis” on page 1-33
- “Discrete-Time System Models” on page 1-37
- “Continuous-Time System Models” on page 1-45
- “Linear System Transformations” on page 1-47
- “Discrete Fourier Transform” on page 1-49

Filter Implementation and Analysis

In this section...
“Filtering Overview” on page 1-2
“Convolution and Filtering” on page 1-2
“Filters and Transfer Functions” on page 1-3
“Filtering with the filter Function” on page 1-4

Filtering Overview

This section describes how to filter discrete signals using the MATLAB® `filter` function and other Signal Processing Toolbox functions. It also discusses how to use the toolbox functions to analyze filter characteristics, including impulse response, magnitude and phase response, group delay, and zero-pole locations.

Convolution and Filtering

The mathematical foundation of filtering is convolution. The MATLAB `conv` function performs standard one-dimensional convolution, convolving one vector with another:

```
conv([1 1 1],[1 1 1])
ans =
     1     2     3     2     1
```

Note Convolve rectangular matrices for two-dimensional signal processing using the `conv2` function.

A digital filter's output $y(k)$ is related to its input $x(k)$ by convolution with its impulse response $h(k)$.

$$y(k) = \sum_{l=-\infty}^{\infty} h(l)x(k-l)$$

If a digital filter's impulse response $h(k)$ is finite in length, and the input $x(k)$ is also of finite length, you can implement the filter using `conv`. Store $x(k)$ in a vector `x`, $h(k)$ in a vector `h`, and convolve the two:

```
x = randn(5,1);     % A random vector of length 5
h = [1 1 1 1]/4;    % Length 4 averaging filter
y = conv(h,x);
```

The length of the output is the sum of the finite-length input vectors minus 1.

Filters and Transfer Functions

In general, the *z*-transform $Y(z)$ of a discrete-time filter's output $y(n)$ is related to the *z*-transform $X(z)$ of the input by

$$Y(z) = H(z)X(z) = \frac{b(1) + b(2)z^{-1} + \ldots + b(n+1)z^{-n}}{a(1) + a(2)z^{-1} + \ldots + a(m+1)z^{-m}} X(z)$$

where $H(z)$ is the filter's *transfer function*. Here, the constants $b(i)$ and $a(i)$ are the filter coefficients and the order of the filter is the maximum of n and m.

Note The filter coefficients start with subscript 1, rather than 0. This reflects the standard indexing scheme used for MATLAB vectors.

MATLAB filter functions store the coefficients in two vectors, one for the numerator and one for the denominator. By convention, it uses row vectors for filter coefficients.

Filter Coefficients and Filter Names

Many standard names for filters reflect the number of `a` and `b` coefficients present:

- When `n = 0` (that is, `b` is a scalar), the filter is an Infinite Impulse Response (IIR), all-pole, recursive, or autoregressive (AR) filter.
- When `m = 0` (that is, `a` is a scalar), the filter is a Finite Impulse Response (FIR), all-zero, nonrecursive, or moving-average (MA) filter.
- If both `n` and `m` are greater than zero, the filter is an IIR, pole-zero, recursive, or autoregressive moving-average (ARMA) filter.

The acronyms AR, MA, and ARMA are usually applied to filters associated with filtered stochastic processes.

Filtering with the filter Function

It is simple to work back to a difference equation from the Z-transform relation shown earlier. Assume that $a(1) = 1$. Move the denominator to the left side and take the inverse Z-transform.

$$y(k) + a(2)y(k-1) + \ldots + a(m+1)y(k-m) = b(1)x(k) + b(2)x(k-1) + \ldots + b(n+1)x(k-n)$$

In terms of current and past inputs, and past outputs, $y(k)$ is

$$y(k) = b(1)x(k) + b(2)x(k-1) + \ldots + b(n+1)x(k-n) - a(2)y(k-1) - \ldots - a(m+1)y(k-m)$$

This is the standard time-domain representation of a digital filter, computed starting with $y(1)$ and assuming a causal system with zero initial conditions. This representation's progression is

$$\begin{aligned} y(1) &= b(1)x(1) \\ y(2) &= b(1)x(2) + b(2)x(1) - a(2)y(1) \\ y(3) &= b(1)x(3) + b(2)x(2) + b(3)x(1) - a(2)y(2) - a(3)y(1) \\ \vdots \quad &= \vdots \end{aligned}$$

A filter in this form is easy to implement with the `filter` function. For example, a simple single-pole filter (lowpass) is

```
B = 1;          % Numerator
A = [1 -0.9];   % Denominator
```

where the vectors `B` and `A` represent the coefficients of a filter in transfer function form. Note that the `A` coefficient vectors are written as if the output and input terms are separated in the difference equation. For the example, the previous coefficient vectors represent a linear constant-coefficient difference equation of

$$y(n) - 0.9y(n-1) = x(n)$$

Changing the sign of the `A(2)` coefficient, results in the difference equation

$$y(n) + 0.9y(n-1) = x(n)$$

The previous coefficients are represented as:

```
B = 1; %Numerator
A = [1 0.9]; %Denominator
```

and results in a highpass filter.

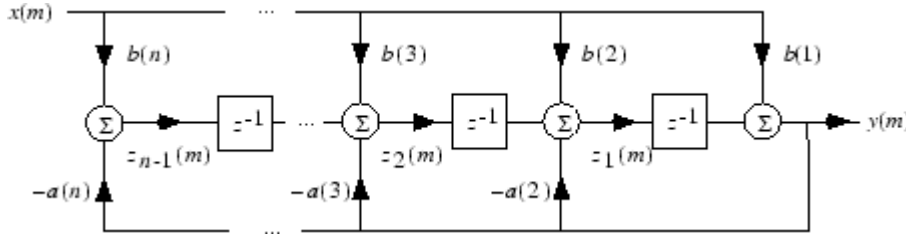
To apply this filter to your data, use

```
y = filter(B,A,x);
```

`filter` gives you as many output samples as there are input samples, that is, the length of `y` is the same as the length of `x`. If the first element of `a` is not 1, `filter` divides the coefficients by `a(1)` before implementing the difference equation.

The filter Function

`filter` is implemented as the transposed direct-form II structure, where *n*–1 is the filter order. This is a canonical form that has the minimum number of delay elements.



At sample *m*, `filter` computes the difference equations

$$
\begin{aligned}
y(m) &= b(1)x(m) + z_1(m-1) \\
z_1(m) &= b(2)x(m) + z_2(m-1) - a(2)y(m) \\
\vdots &\quad = \vdots \\
z_{n-2}(m) &= b(n-1)x(m) + z_{n-1}(m-1) - a(n-1)y(m) \\
z_{n-1}(m) &= b(n)x(m) - a(n)y(m)
\end{aligned}
$$

In its most basic form, `filter` initializes the delay outputs $z_i(1)$, i = 1, ..., n-1 to 0. This is equivalent to assuming both past inputs and outputs are zero. Set the initial delay outputs using a fourth input parameter to `filter`, or access the final delay outputs using a second output parameter:

```
[y,zf] = filter(b,a,x,zi)
```

Access to initial and final conditions is useful for filtering data in sections, especially if memory limitations are a consideration. Suppose you have collected data in two segments of 5000 points each:

```
x1 = randn(5000,1);  % Generate two random data sequences.
x2 = randn(5000,1);
```

Perhaps the first sequence, `x1`, corresponds to the first 10 minutes of data and the second, `x2`, to an additional 10 minutes. The whole sequence is `x = [x1;x2]`. If there is not sufficient memory to hold the combined sequence, filter the subsequences `x1` and `x2` one at a time. To ensure continuity of the filtered sequences, use the final conditions from `x1` as initial conditions to filter `x2`:

```
[y1,zf] = filter(b,a,x1);
y2 = filter(b,a,x2,zf);
```

The `filtic` function generates initial conditions for `filter`. `filtic` computes the delay vector to make the behavior of the filter reflect past inputs and outputs that you specify. To obtain the same output delay values `zf` as above using `filtic`, use

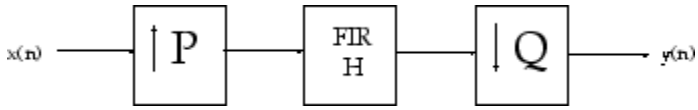
```
zf = filtic(b,a,flipud(y1),flipud(x1));
```

This can be useful when filtering short data sequences, as appropriate initial conditions help reduce transient startup effects.

Multirate Filter Bank Implementation

The `upfirdn` function alters the sampling rate of a signal by an integer ratio P/Q. It computes the result of a cascade of three systems that performs the following tasks:

- Upsampling (zero insertion) by integer factor `p`
- Filtering by FIR filter `h`
- Downsampling by integer factor `q`



For example, to change the sample rate of a signal from 44.1 kHz to 48 kHz, we first find the smallest integer conversion ratio `p/q`. Set

```
d = gcd(48000,44100);
p = 48000/d;
q = 44100/d;
```

In this example, `p = 160` and `q = 147`. Sample rate conversion is then accomplished by typing

```
y = upfirdn(x,h,p,q)
```

This cascade of operations is implemented in an efficient manner using polyphase filtering techniques, and it is a central concept of multirate filtering. Note that the quality of the resampling result relies on the quality of the FIR filter `h`.

Filter banks may be implemented using `upfirdn` by allowing the filter `h` to be a matrix, with one FIR filter per column. A signal vector is passed independently through each FIR filter, resulting in a matrix of output signals.

Other functions that perform multirate filtering (with fixed filter) include `resample`, `interp`, and `decimate`.

Frequency Domain Filter Implementation

Duality between the time domain and the frequency domain makes it possible to perform any operation in either domain. Usually one domain or the other is more convenient for a particular operation, but you can always accomplish a given operation in either domain.

To implement general IIR filtering in the frequency domain, multiply the discrete Fourier transform (DFT) of the input sequence with the quotient of the DFT of the filter:

```
n = length(x);
y = ifft(fft(x).*fft(b,n)./fft(a,n));
```

This computes results that are identical to `filter`, but with different startup transients (edge effects). For long sequences, this computation is very inefficient because of the large zero-padded FFT operations on the filter coefficients, and because the FFT algorithm becomes less efficient as the number of points `n` increases.

For FIR filters, however, it is possible to break longer sequences into shorter, computationally efficient FFT lengths. The function

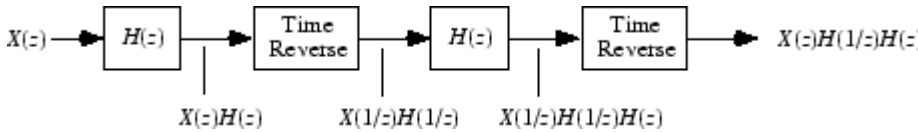
```
y = fftfilt(b,x)
```

uses the overlap add method to filter a long sequence with multiple medium-length FFTs. Its output is equivalent to `filter(b,1,x)`.

Anti-Causal, Zero-Phase Filter Implementation

In the case of FIR filters, it is possible to design linear phase filters that, when applied to data (using `filter` or `conv`), simply delay the output by a fixed number of samples. For IIR filters, however, the phase distortion is usually highly nonlinear. The `filtfilt` function uses the information in the signal at points before and after the current point, in essence "looking into the future," to eliminate phase distortion.

To see how `filtfilt` does this, recall that if the Z-transform of a real sequence $x(n)$ is $X(z)$, then the Z-transform of the time-reversed sequence $x(-n)$ is $X(1/z)$. Consider the following processing scheme:



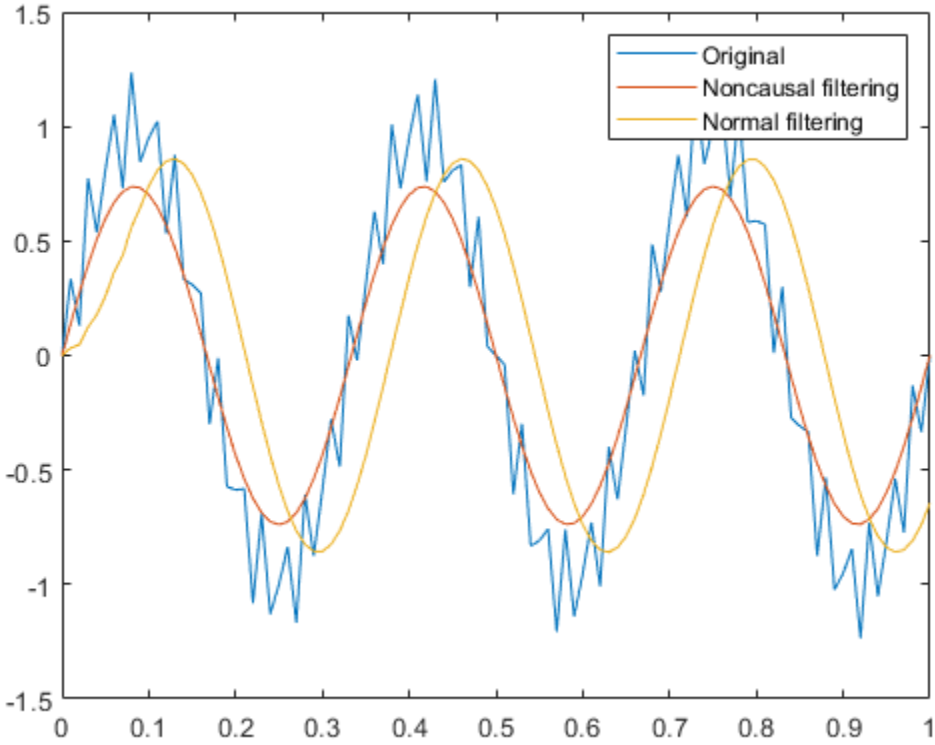
When $|z| = 1$, that is $z = e^{j\omega}$, the output reduces to $X(e^{j\omega})|H(e^{j\omega})|^2$. Given all the samples of the sequence $x(n)$, a doubly filtered version of x that has zero-phase distortion is possible.

For example, a 1-second duration signal sampled at 100 Hz, composed of two sinusoidal components at 3 Hz and 40 Hz, is

```
fs = 100;
t = 0:1/fs:1;
x = sin(2*pi*t*3)+.25*sin(2*pi*t*40);
```

Now create a 10-point averaging FIR filter. Filter `x` using both `filter` and `filtfilt` for comparison:

```
b = ones(1,10)/10;

y = filtfilt(b,1,x);
yy = filter(b,1,x);

plot(t,x,t,y,t,yy)
legend('Original','Noncausal filtering','Normal filtering')
```



Both filtered versions eliminate the 40 Hz sinusoid evident in the original, solid line. The plot also shows how `filter` and `filtfilt` differ; the dashed (`filtfilt`) line is in phase with the original 3 Hz sinusoid, while the dotted (`filter`) line is delayed by about five samples. Also, the amplitude of the dashed line is smaller due to the magnitude squared effects of `filtfilt`.

`filtfilt` reduces filter startup transients by carefully choosing initial conditions, and by prepending onto the input sequence a short, reflected piece of the input sequence. For best results, make sure the sequence you are filtering has length at least three times the filter order and tapers to zero on both edges.

See Also

`conv` | `filter` | `filtfilt`

Impulse Response

The impulse response of a digital filter is the output arising from the unit impulse sequence defined as

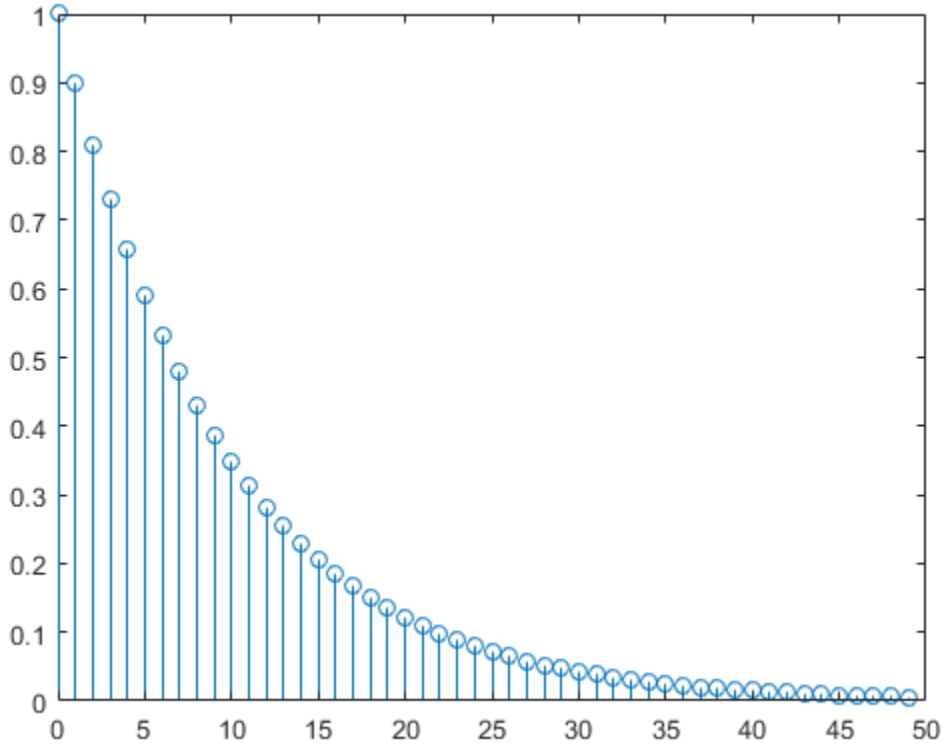
$$\delta(n) = \begin{cases} 1, & n = 0, \\ 0, & n \neq 0. \end{cases}$$

You can generate an impulse sequence a number of ways; one straightforward way is

```
imp = [1; zeros(49,1)];
```

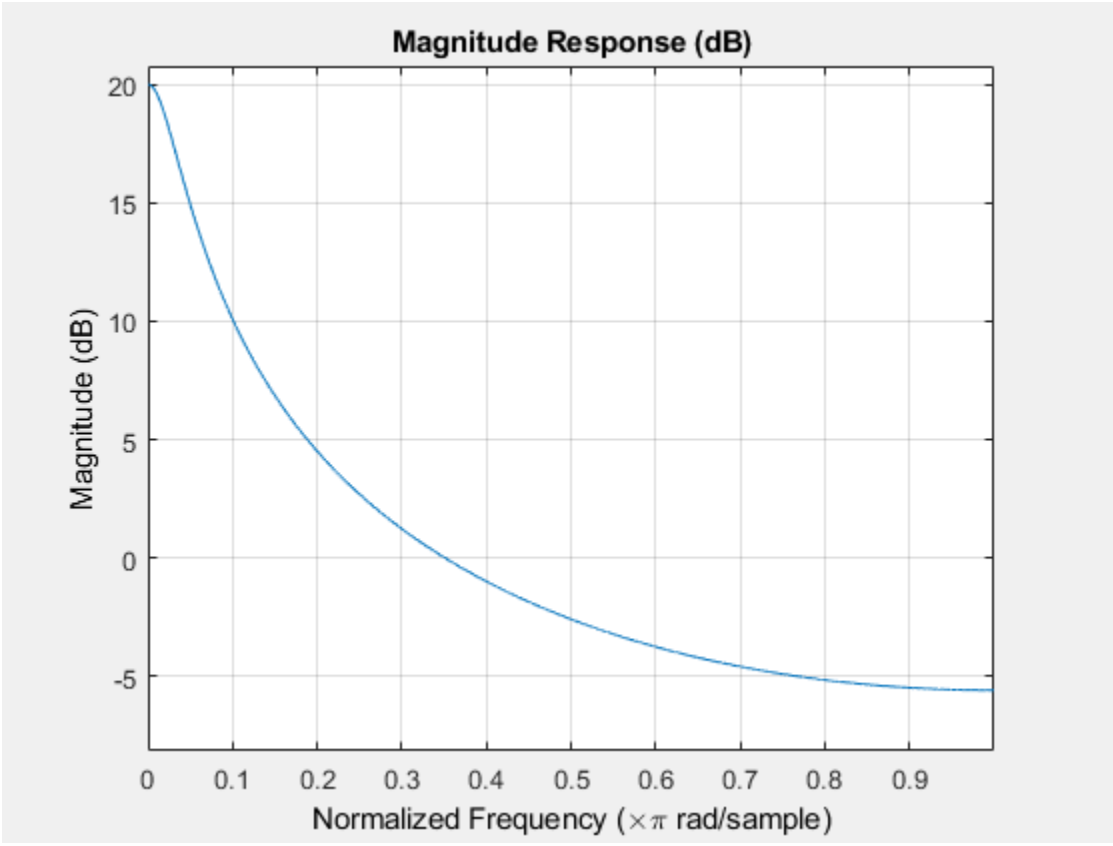
The impulse response of the simple filter with $b = 1$ and $a = [1 \; -0.9]$ is $h(n) = 0.9^n$, which decays exponentially.

```
b = 1;
a = [1 -0.9];

h = filter(b,a,imp);

stem(0:49,h)
```



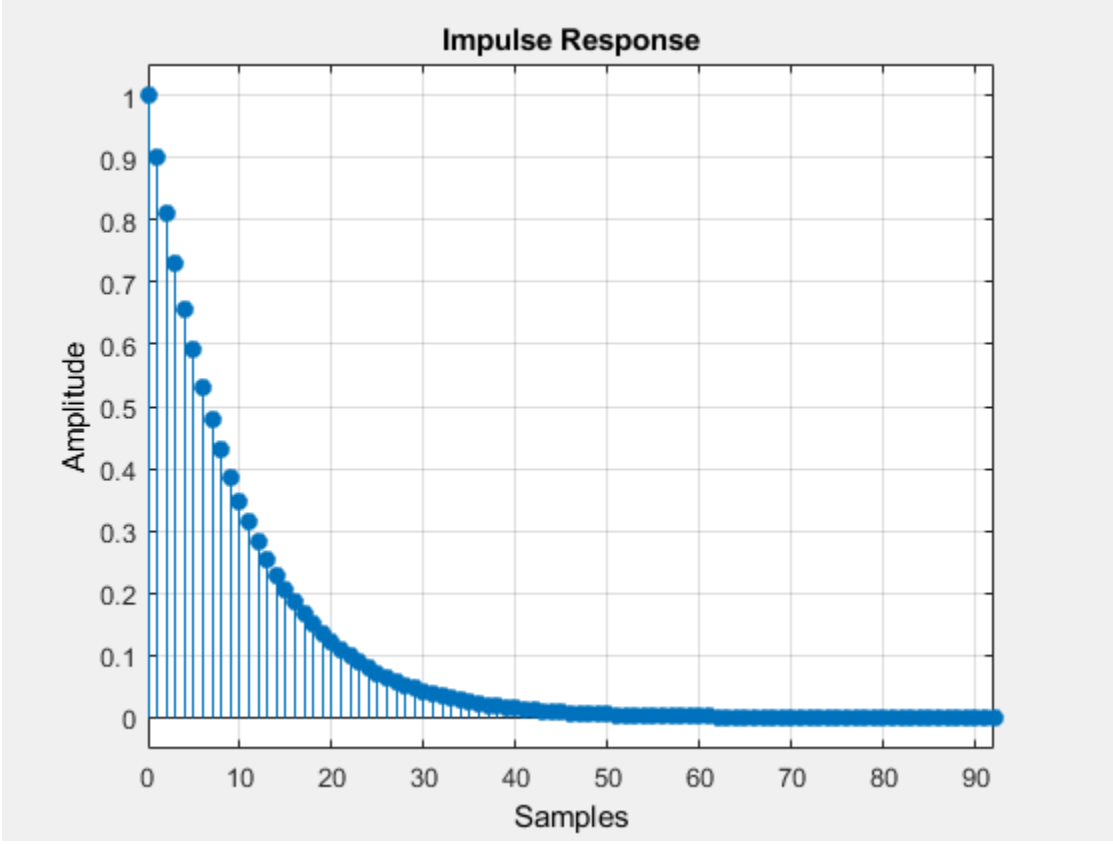
A simple way to display the impulse response is with the Filter Visualization Tool, `fvtool`.

```
fvtool(b,a)
```



Click the **Impulse Response** button, , on the toolbar, select **Analysis > Impulse Response** from the menu, or type the following code to obtain the exponential decay of the single-pole system.

```
fvtool(b,a,'Analysis','impulse')
```


Impulse Response
Amplitude
Samples
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
0
10
20
30
40
50
60
70
80
90

Frequency Response

In this section...
“Digital Domain” on page 1-17
“Analog Domain” on page 1-24

Digital Domain

`freqz` uses an FFT-based algorithm to calculate the Z-transform frequency response of a digital filter. Specifically, the statement

```
[h,w] = freqz(b,a,p)
```

returns the p-point complex frequency response, $H(e^{j\omega})$, of the digital filter.

$$H(e^{j\omega}) = \frac{b(1) + b(2)e^{-j\omega} + \ldots + b(n+1)e^{-j\omega n}}{a(1) + a(2)e^{-j\omega} + \ldots + a(m+1)e^{-j\omega m}}$$

In its simplest form, `freqz` accepts the filter coefficient vectors `b` and `a`, and an integer `p` specifying the number of points at which to calculate the frequency response. `freqz` returns the complex frequency response in vector `h`, and the actual frequency points in vector `w` in rad/s.

`freqz` can accept other parameters, such as a sampling frequency or a vector of arbitrary frequency points. The example below finds the 256-point frequency response for a 12th-order Chebyshev Type I filter. The call to `freqz` specifies a sampling frequency `fs` of 1000 Hz:

```
[b,a] = cheby1(12,0.5,200/500);
[h,f] = freqz(b,a,256,1000);
```

Because the parameter list includes a sampling frequency, `freqz` returns a vector `f` that contains the 256 frequency points between 0 and `fs/2` used in the frequency response calculation.

Note This toolbox uses the convention that unit frequency is the Nyquist frequency, defined as half the sampling frequency. The cutoff frequency parameter for all basic filter design functions is normalized by the Nyquist frequency. For a system with a 1000 Hz

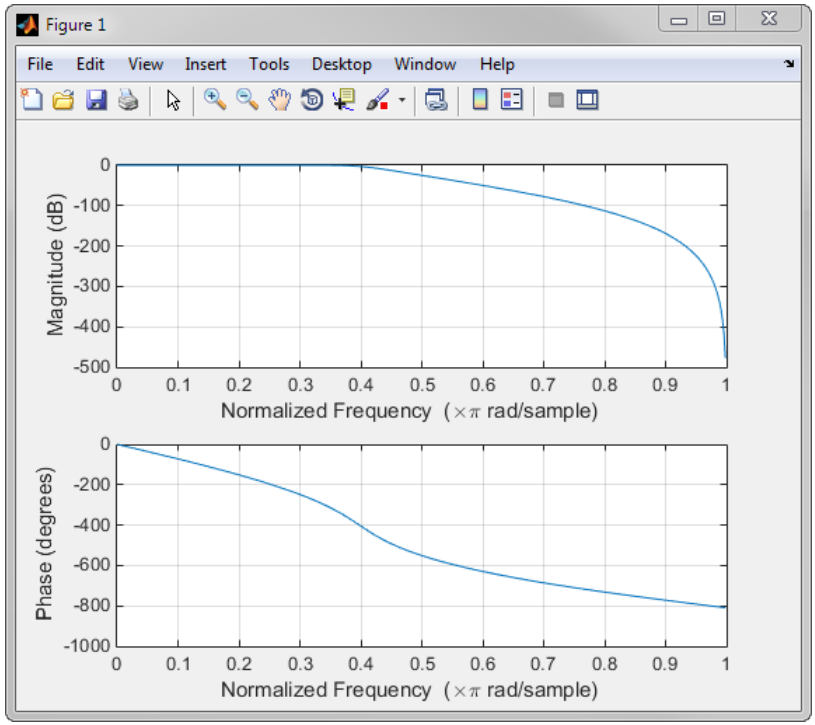
sampling frequency, for example, 300 Hz is 300/500 = 0.6. To convert normalized frequency to angular frequency around the unit circle, multiply by π. To convert normalized frequency back to hertz, multiply by half the sample frequency.

If you call `freqz` with no output arguments, it plots both magnitude versus frequency and phase versus frequency. For example, a ninth-order Butterworth lowpass filter with a cutoff frequency of 400 Hz, based on a 2000 Hz sampling frequency, is

```
[b,a] = butter(9,400/1000);
```

To calculate the 256-point complex frequency response for this filter, and plot the magnitude and phase with `freqz`, use

```
freqz(b,a,256,2000)
```



`freqz` can also accept a vector of arbitrary frequency points for use in the frequency response calculation. For example,

```
w = linspace(0,pi);
h = freqz(b,a,w);
```

calculates the complex frequency response at the frequency points in `w` for the filter defined by vectors `b` and `a`. The frequency points can range from 0 to 2π. To specify a frequency vector that ranges from zero to your sampling frequency, include both the frequency vector and the sampling frequency value in the parameter list.

These examples show how to compute and display digital frequency responses.

Frequency Response from Transfer Function

Compute and display the magnitude response of the third-order IIR lowpass filter described by the following transfer function:

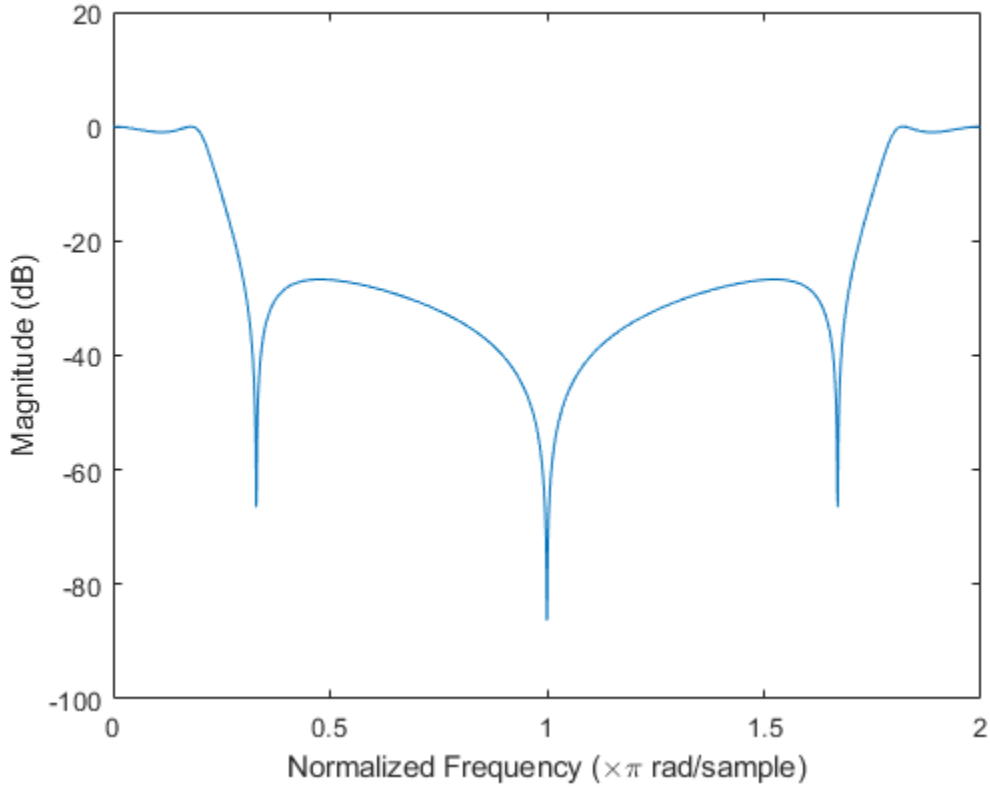
$$H(z) = \frac{0.05634(1+z^{-1})(1-1.0166z^{-1}+z^{-2})}{(1-0.683z^{-1})(1-1.4461z^{-1}+0.7957z^{-2})}.$$

Express the numerator and denominator as polynomial convolutions. Find the frequency response at 2001 points spanning the complete unit circle.

```
b0 = 0.05634;
b1 = [1  1];
b2 = [1 -1.0166 1];
a1 = [1 -0.683];
a2 = [1 -1.4461 0.7957];

b = b0*conv(b1,b2);
a = conv(a1,a2);

[h,w] = freqz(b,a,'whole',2001);
```

Plot the magnitude response expressed in decibels.

```
plot(w/pi,20*log10(abs(h)))
ax = gca;
ax.YLim = [-100 20];
ax.XTick = 0:.5:2;
xlabel('Normalized Frequency (\times\pi rad/sample)')
ylabel('Magnitude (dB)')
```

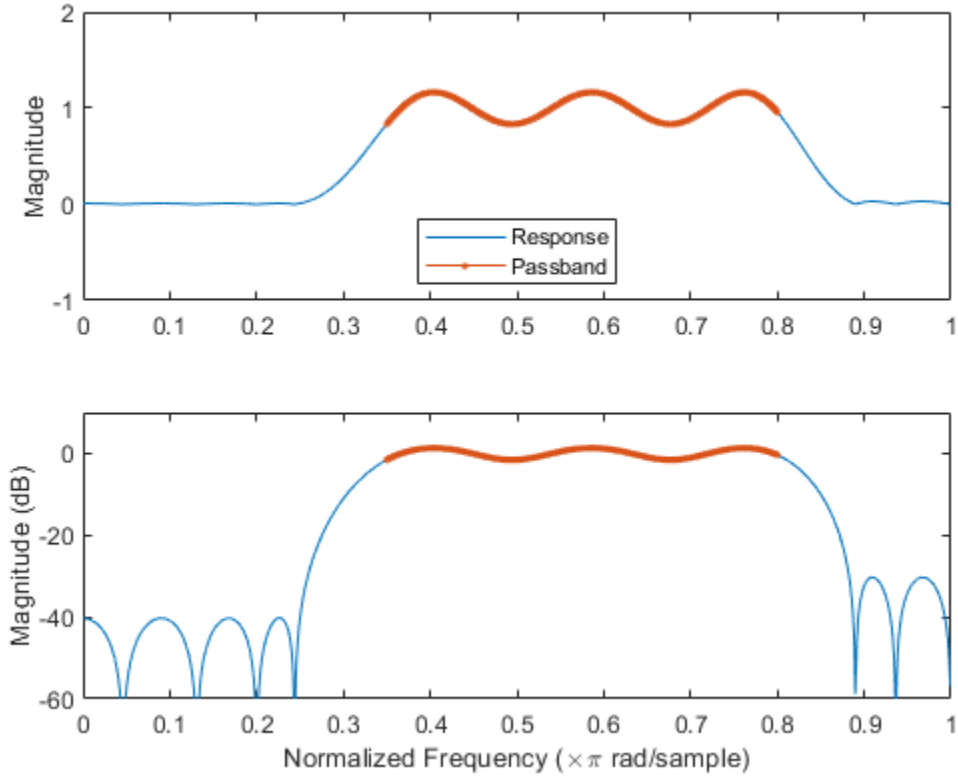


Frequency Response of an FIR Bandpass Filter

Design an FIR bandpass filter with passband between 0.35π and 0.8π rad/sample and 3 dB of ripple. The first stopband goes from 0 to 0.1π rad/sample and has an attenuation of 40 dB. The second stopband goes from 0.9π rad/sample to the Nyquist frequency and has an attenuation of 30 dB. Compute the frequency response. Plot its magnitude in both linear units and decibels. Highlight the passband.

```
sf1 = 0.1;
pf1 = 0.35;
pf2 = 0.8;
```

```
sf2 = 0.9;
pb = linspace(pf1,pf2,1e3)*pi;

bp = designfilt('bandpassfir', ...
    'StopbandAttenuation1',40, 'StopbandFrequency1',sf1,...
    'PassbandFrequency1',pf1,'PassbandRipple',3,'PassbandFrequency2',pf2, ...
    'StopbandFrequency2',sf2,'StopbandAttenuation2',30);

[h,w] = freqz(bp,1024);
hpb = freqz(bp,pb);

subplot(2,1,1)
plot(w/pi,abs(h),pb/pi,abs(hpb),'.-')
axis([0 1 -1 2])
legend('Response','Passband','Location','South')
ylabel('Magnitude')

subplot(2,1,2)
plot(w/pi,db(h),pb/pi,db(hpb),'.-')
axis([0 1 -60 10])
xlabel('Normalized Frequency (\times\pi rad/sample)')
ylabel('Magnitude (dB)')
```

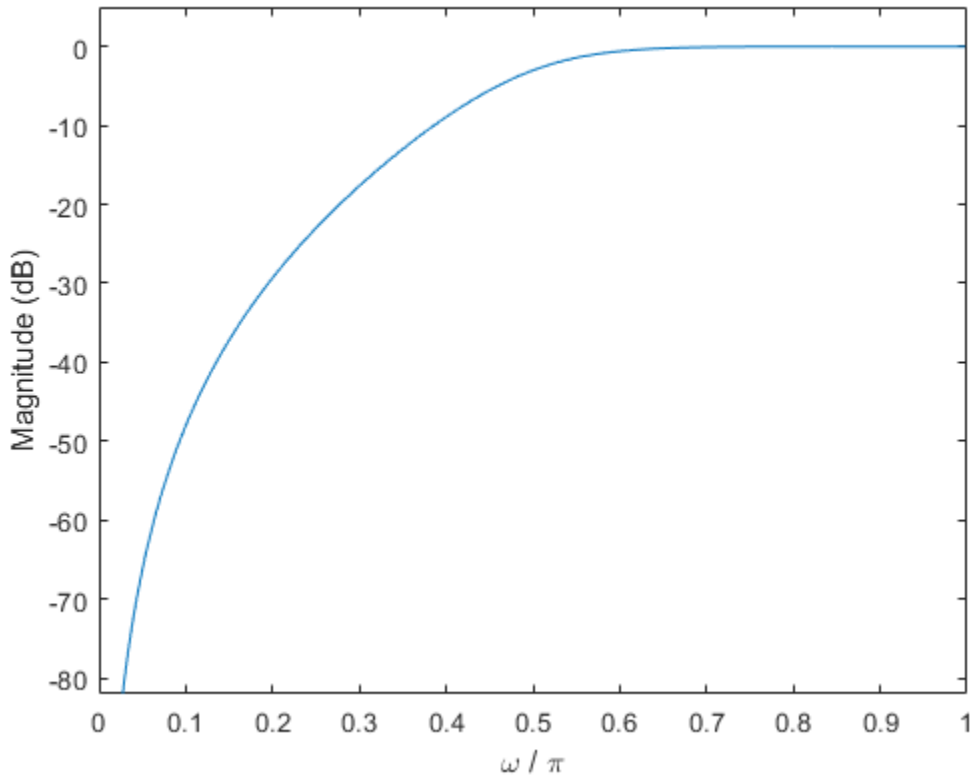


Magnitude Response of a Highpass Filter

Design a 3rd-order highpass Butterworth filter having a normalized 3-dB frequency of 0.5π rad/sample. Compute its frequency response. Express the magnitude response in decibels and plot it.

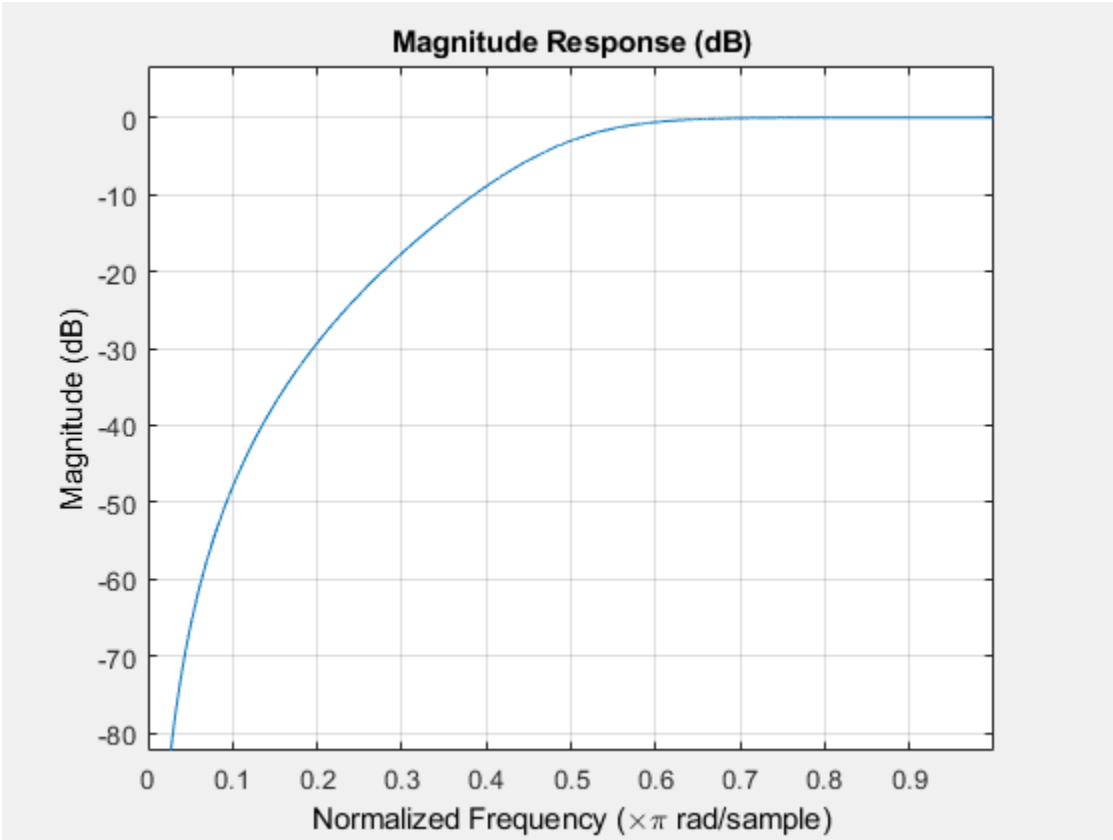
```
[b,a] = butter(3,0.5,'high');
[h,w] = freqz(b,a);

dB = mag2db(abs(h));

plot(w/pi,dB)
xlabel('\omega / \pi')
```

```
ylabel('Magnitude (dB)')
ylim([-82 5])
```



Repeat the computation using `fvtool`.

```
fvtool(b,a)
```



Analog Domain

`freqs` evaluates frequency response for an analog filter defined by two input coefficient vectors, `b` and `a`. Its operation is similar to that of `freqz`; you can specify a number of frequency points to use, supply a vector of arbitrary frequency points, and plot the magnitude and phase response of the filter. This example shows how to compute and display analog frequency responses.

Comparison of Analog IIR Lowpass Filters

Design a 5th-order analog Butterworth lowpass filter with a cutoff frequency of 2 GHz.

Multiply by 2π to convert the frequency to radians per second. Compute the frequency response of the filter at 4096 points.

```
n = 5;
f = 2e9;

[zb,pb,kb] = butter(n,2*pi*f,'s');
[bb,ab] = zp2tf(zb,pb,kb);
[hb,wb] = freqs(bb,ab,4096);
```

Design a 5th-order Chebyshev Type I filter with the same edge frequency and 3 dB of passband ripple. Compute its frequency response.

```
[z1,p1,k1] = cheby1(n,3,2*pi*f,'s');
[b1,a1] = zp2tf(z1,p1,k1);
[h1,w1] = freqs(b1,a1,4096);
```

Design a 5th-order Chebyshev Type II filter with the same edge frequency and 30 dB of stopband attenuation. Compute its frequency response.

```
[z2,p2,k2] = cheby2(n,30,2*pi*f,'s');
[b2,a2] = zp2tf(z2,p2,k2);
[h2,w2] = freqs(b2,a2,4096);
```

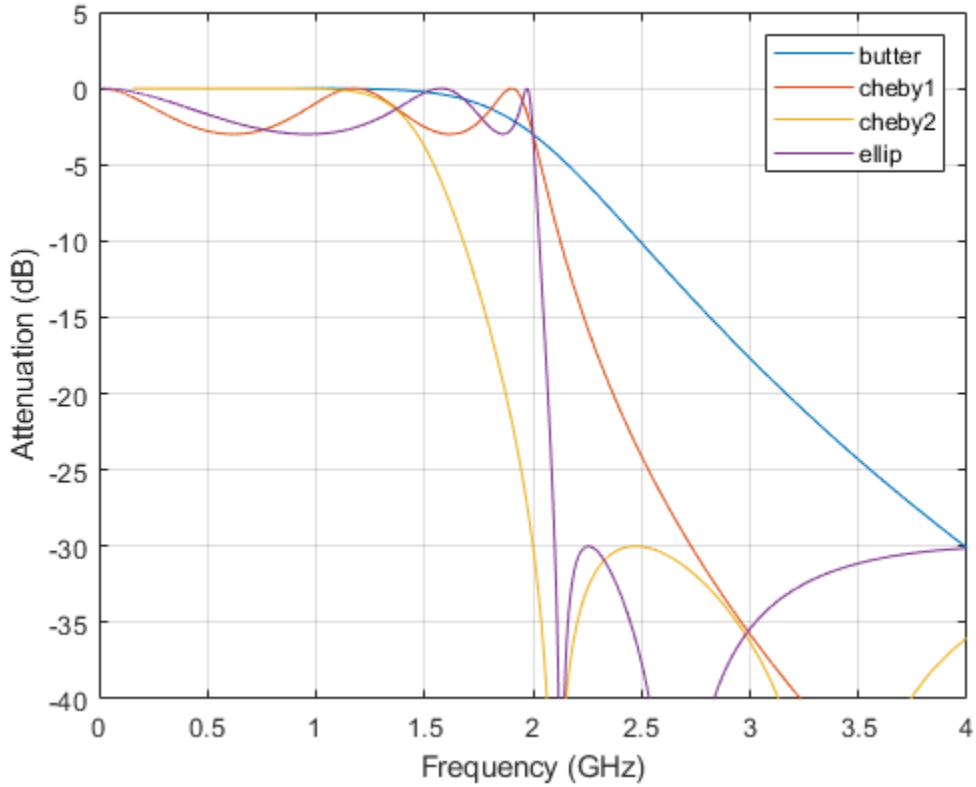
Design a 5th-order elliptic filter with the same edge frequency, 3 dB of passband ripple, and 30 dB of stopband attenuation. Compute its frequency response.

```
[ze,pe,ke] = ellip(n,3,30,2*pi*f,'s');
[be,ae] = zp2tf(ze,pe,ke);
[he,we] = freqs(be,ae,4096);
```

Plot the attenuation in decibels. Express the frequency in gigahertz. Compare the filters.

```
plot(wb/(2e9*pi),mag2db(abs(hb)))
hold on
plot(w1/(2e9*pi),mag2db(abs(h1)))
plot(w2/(2e9*pi),mag2db(abs(h2)))
plot(we/(2e9*pi),mag2db(abs(he)))
axis([0 4 -40 5])
grid
xlabel('Frequency (GHz)')
```

```
ylabel('Attenuation (dB)')
legend('butter','cheby1','cheby2','ellip')
```

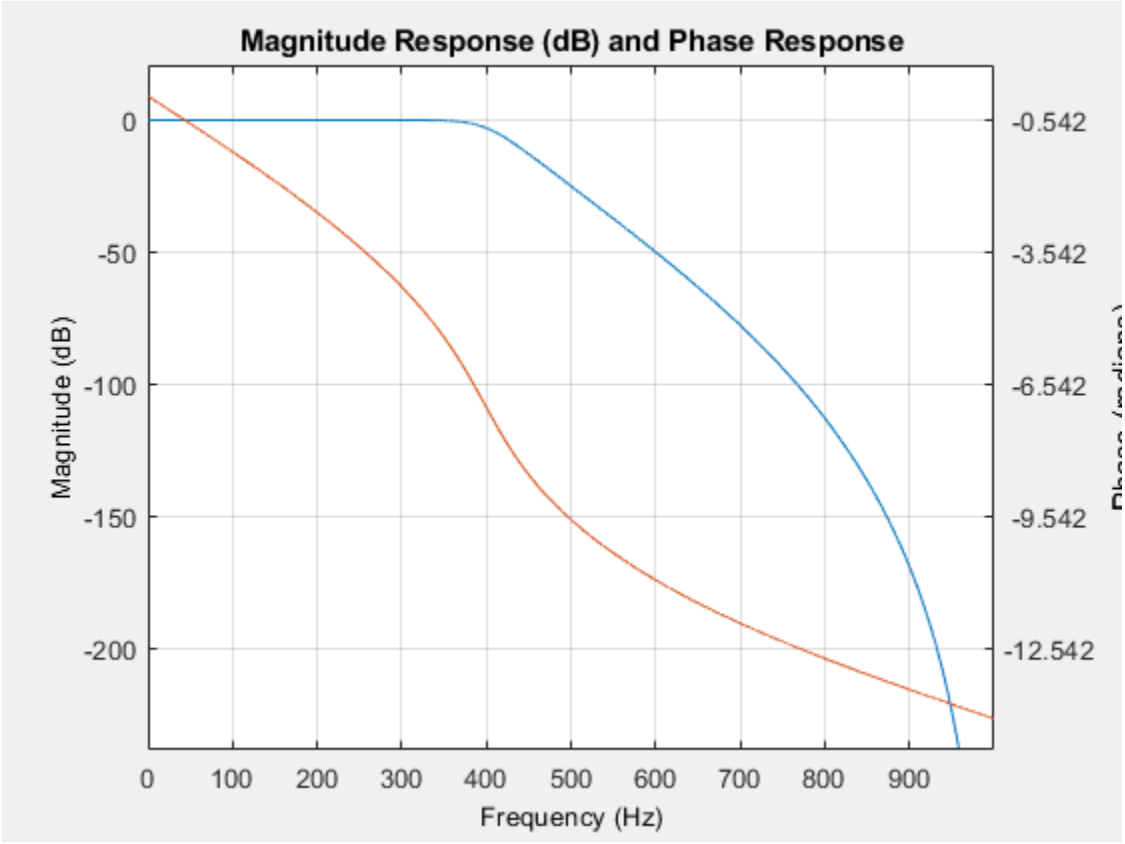


The Butterworth and Chebyshev Type II filters have flat passbands and wide transition bands. The Chebyshev Type I and elliptic filters roll off faster but have passband ripple. The frequency input to the Chebyshev Type II design function sets the beginning of the stopband rather than the end of the passband.

Phase Response

MATLAB® functions are available to extract the phase response of a filter. Given a frequency response, the function `abs` returns the magnitude and `angle` returns the phase angle in radians. To view the magnitude and phase of a Butterworth filter using `fvtool`:

```
d = designfilt('lowpassiir','FilterOrder',9, ...
    'HalfPowerFrequency',400,'SampleRate',2000);
fvtool(d,'Analysis','freq')
```



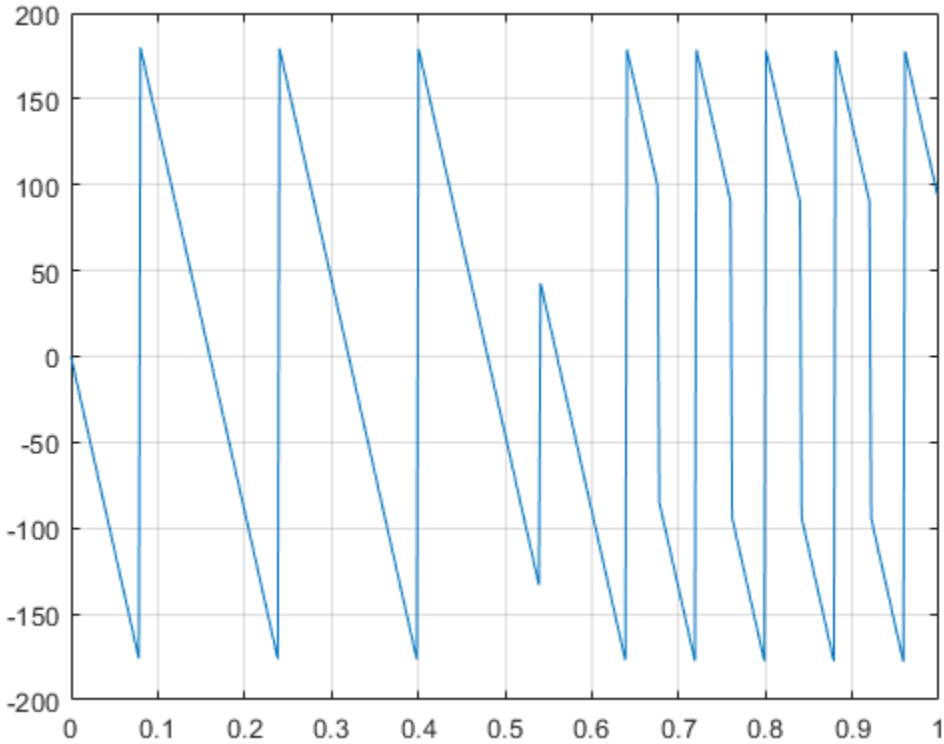
You can also click the **Magnitude and Phase Response** button on the toolbar or select **Analysis > Magnitude and Phase Response** to display the plot.

The `unwrap` function is also useful in frequency analysis. `unwrap` unwraps the phase to make it continuous across 360° phase discontinuities by adding multiples of ±360°, as needed. To see how `unwrap` is useful, design a 25th-order lowpass FIR filter:

```
h = fir1(25,0.4);
```

Obtain the frequency response with `freqz` and plot the phase in degrees:

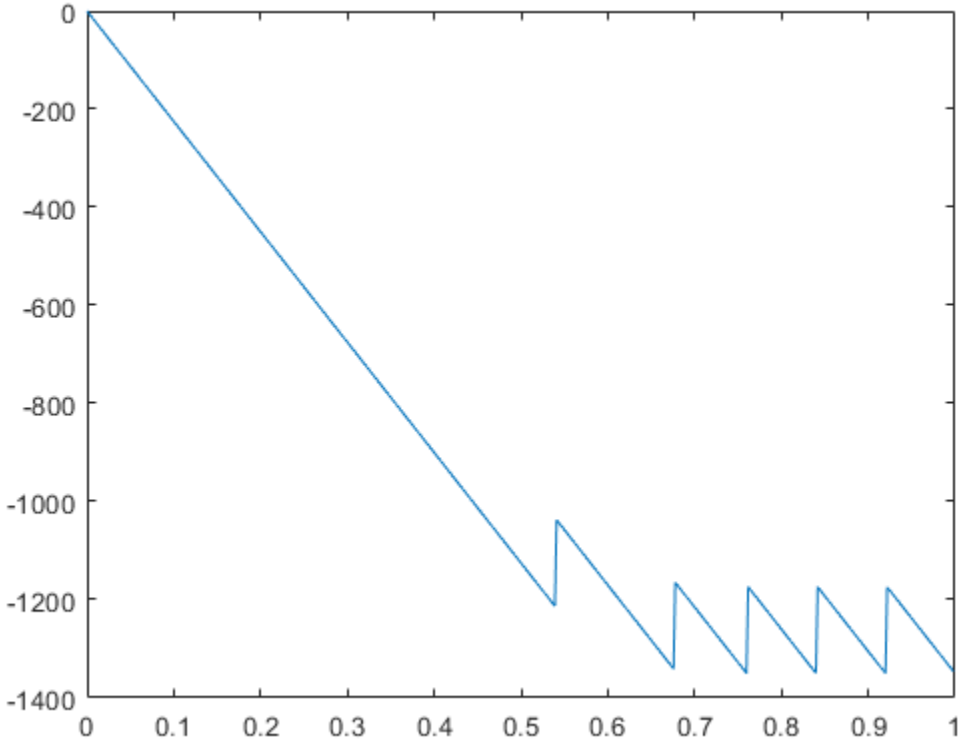
```
[H,f] = freqz(h,1,512,2);
plot(f,angle(H)*180/pi)
grid
```



It is difficult to distinguish the 360° jumps (an artifact of the arctangent function inside `angle`) from the 180° jumps that signify zeros in the frequency response.

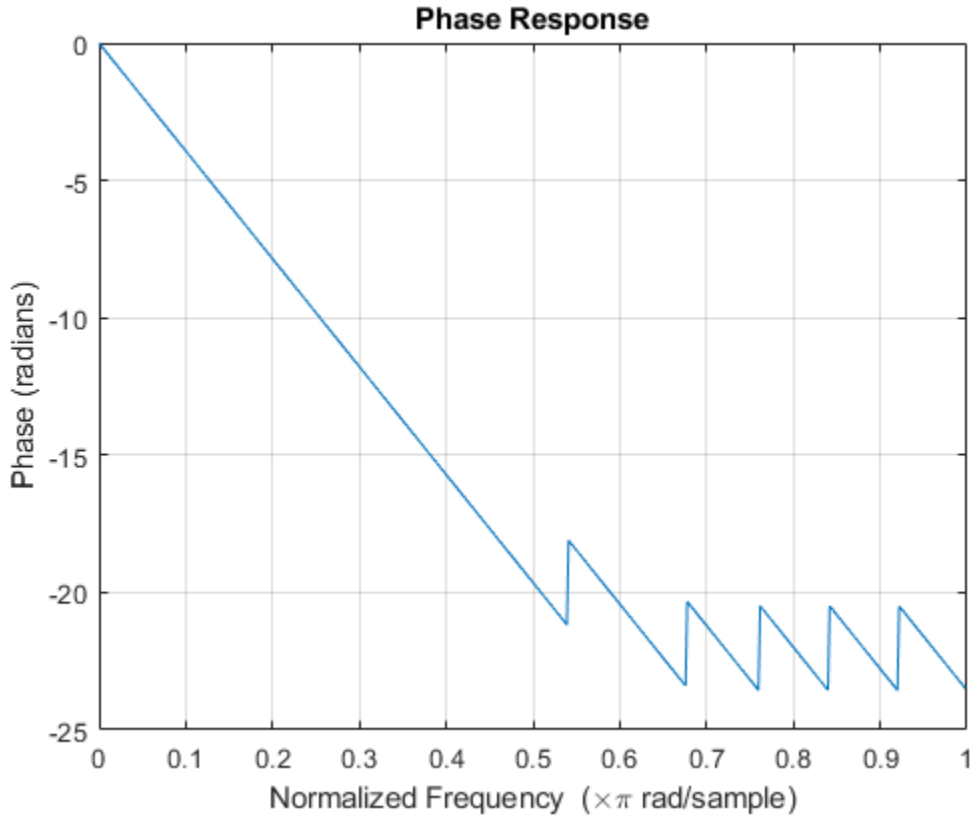
`unwrap` eliminates the 360° jumps:

```
plot(f,unwrap(angle(H))*180/pi)
```



Alternatively, you can use `phasez` to see the unwrapped phase:

```
phasez(h,1)
```



See Also

`abs` | `angle` | `freqz` | `fvtool` | `phasez` | `unwrap`

Delay

The *group delay* of a filter is a measure of the average time delay of the filter as a function of frequency. It is defined as the negative first derivative of a filter's phase response. If the complex frequency response of a filter is $H(e^{j\omega})$, then the group delay is

$$\tau_g(\omega) = -\frac{d\theta(\omega)}{d\omega}$$

where $\theta(\omega)$ is the phase, or argument of $H(e^{j\omega})$. Compute group delay with

```
[gd,w] = grpdelay(b,a,n)
```

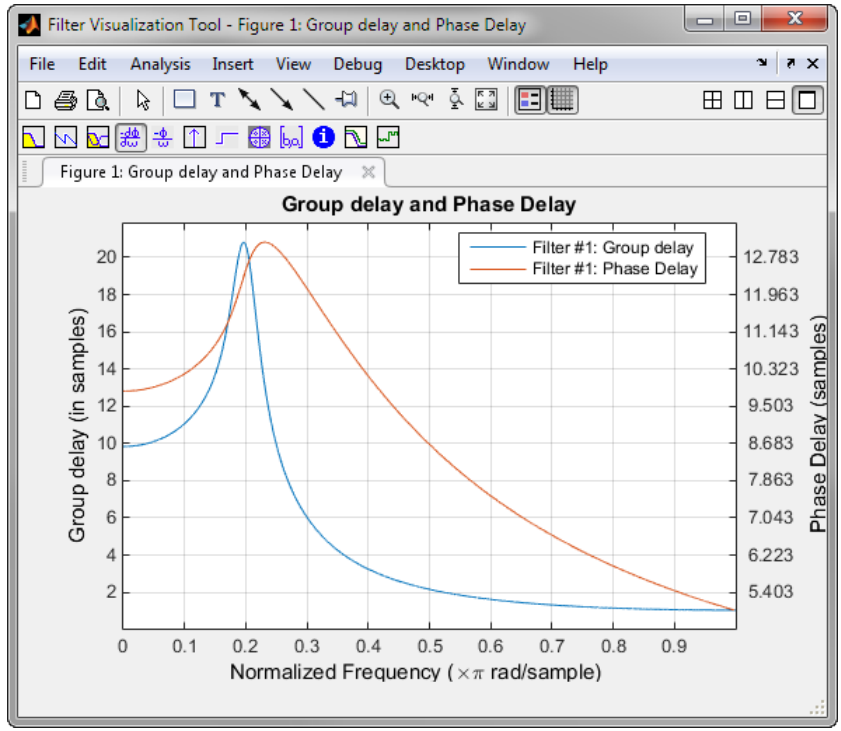
which returns the `n`-point group delay, $\tau_g(\omega)$, of the digital filter specified by `b` and `a`, evaluated at the frequencies in vector `w`.

The *phase delay* of a filter is the negative of phase divided by frequency:

$$\tau_p(\omega) = -\frac{\theta(\omega)}{\omega}$$

To plot both the group and phase delays of a system on the same FVTool graph, type

```
[z,p,k] = butter(10,200/1000);
fvtool(zp2sos(z,p,k),'Analysis','grpdelay', ...
   'OverlayedAnalysis','phasedelay','Legend','on')
```


Filter Visualization Tool - Figure 1: Group delay and Phase Delay
File Edit Analysis Insert View Debug Desktop Window Help
Figure 1: Group delay and Phase Delay
Group delay and Phase Delay
Filter #1: Group delay
Filter #1: Phase Delay
Group delay (in samples)
Phase Delay (samples)
Normalized Frequency (×π rad/sample)
0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
2 4 6 8 10 12 14 16 18 20
5.403 6.223 7.043 7.863 8.683 9.503 10.323 11.143 11.963 12.783

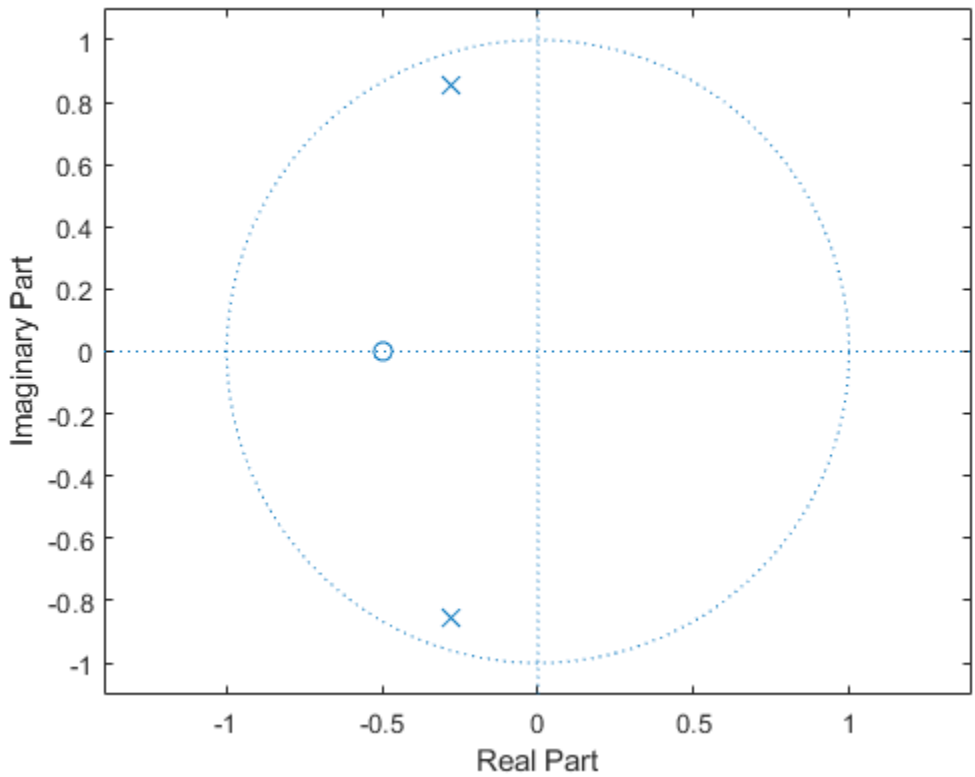
Zero-Pole Analysis

The `zplane` function plots poles and zeros of a linear system. For example, a simple filter with a zero at -1/2 and a complex pole pair at $0.9e^{-j2\pi 0.3}$ and $0.9e^{j2\pi 0.3}$ is

```
zer = -0.5;
pol = 0.9*exp(j*2*pi*[-0.3 0.3]');
```

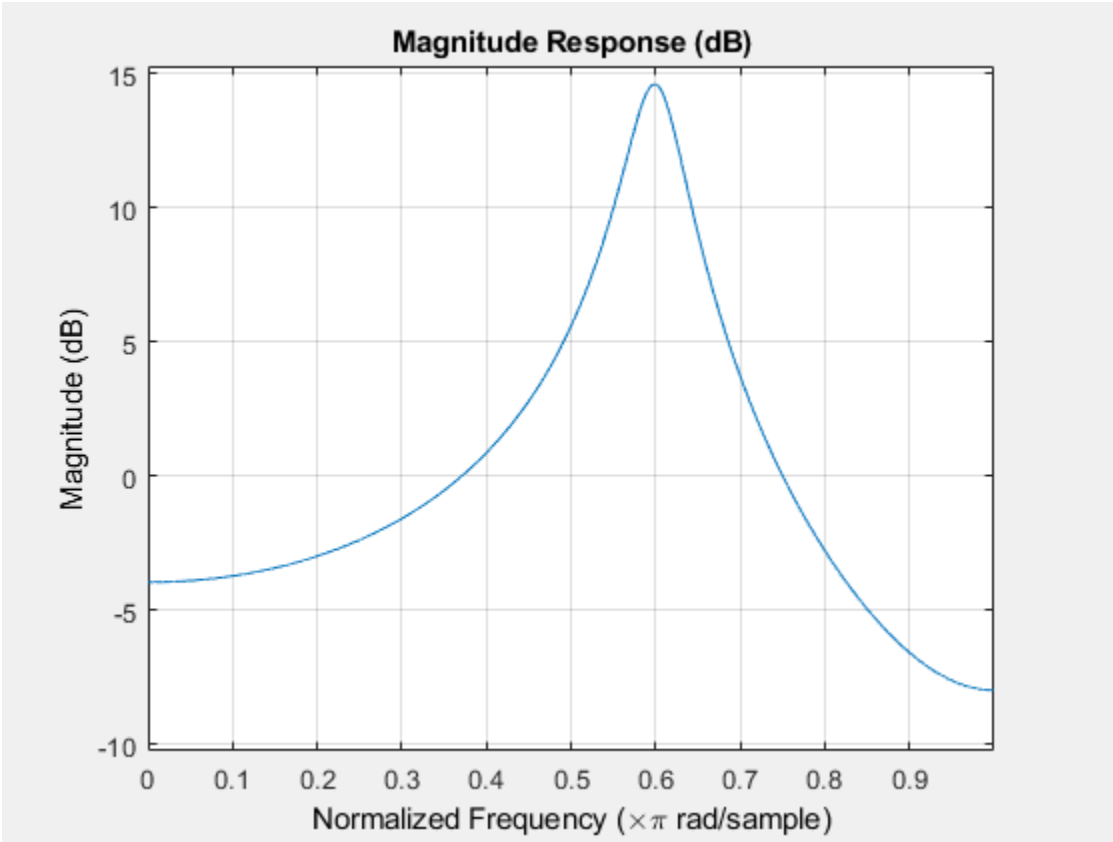
To view the pole-zero plot for this filter you can use `zplane`. Supply column vector arguments when the system is in pole-zero form.

```
zplane(zer,pol)
```



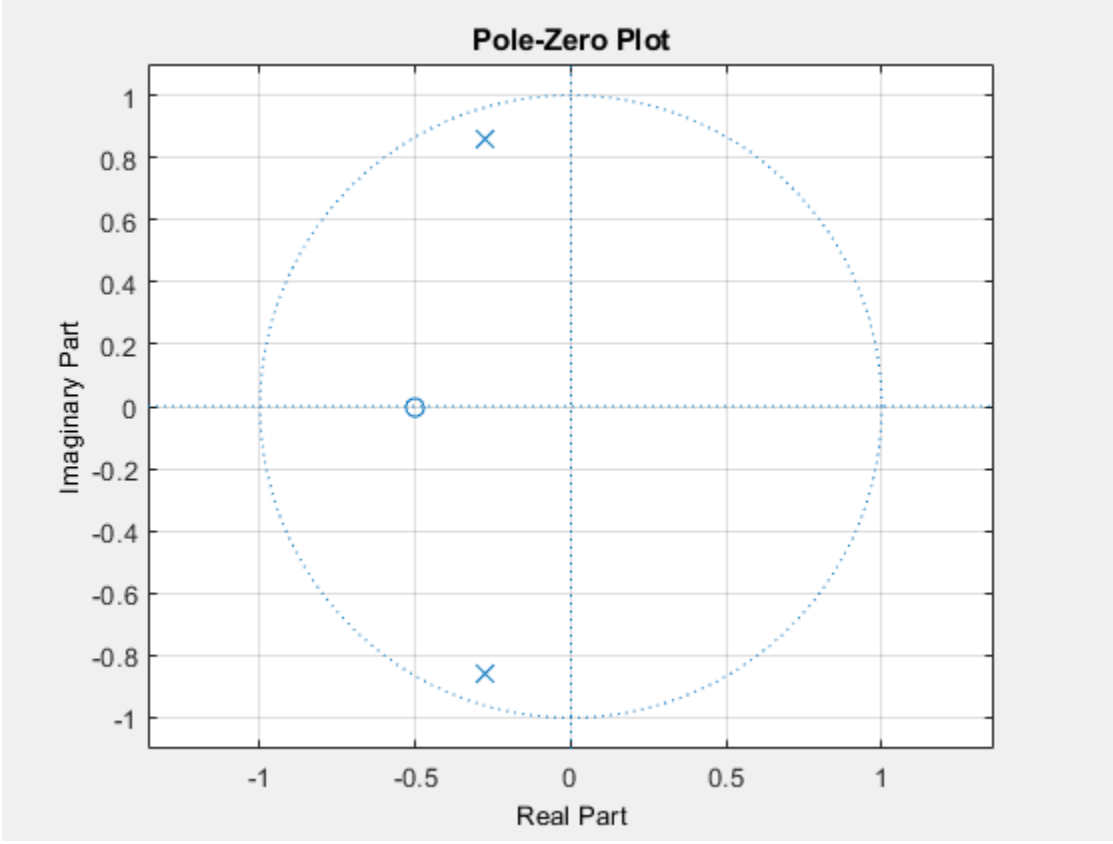
For access to additional tools, use `fvtool`. First convert the poles and zeros to transfer function form, then call `fvtool`.

```
[b,a] = zp2tf(zer,pol,1);
fvtool(b,a)
```



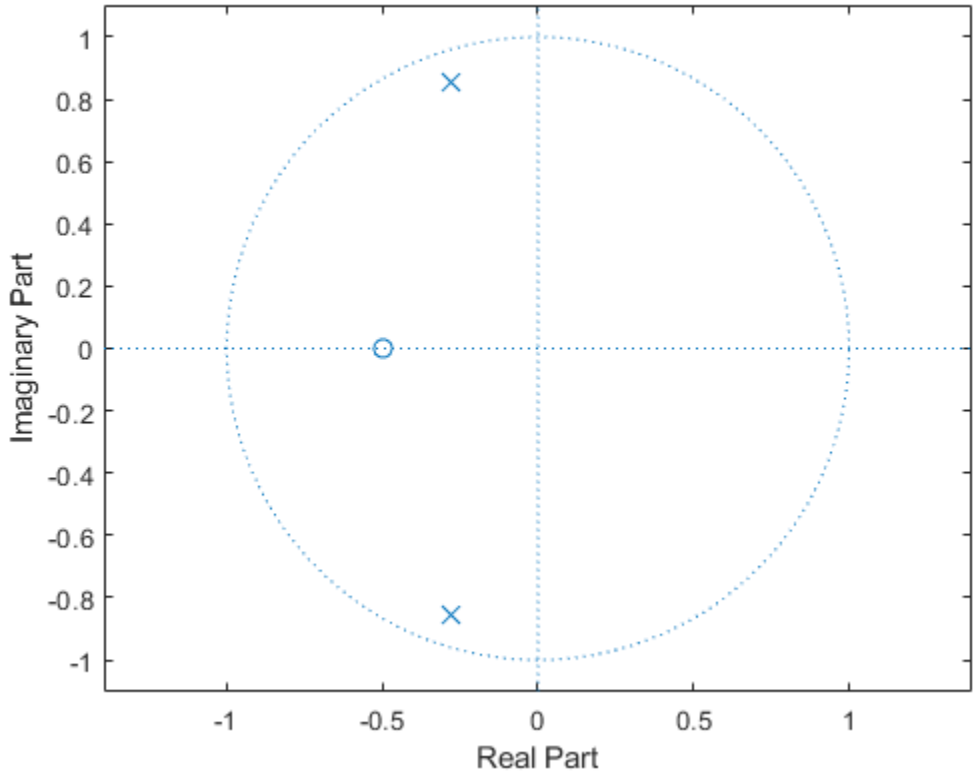
Click the **Pole/Zero Plot** toolbar button, select **Analysis > Pole/Zero Plot** from the menu, or type the following code to see the plot.

```
fvtool(b,a,'Analysis','polezero')
```



To use `zplane` for a system in transfer function form, supply row vector arguments. In this case, `zplane` finds the roots of the numerator and denominator using the `roots` function and plots the resulting zeros and poles.

```
zplane(b,a)
```



See Linear System Models for details on zero-pole and transfer function representation of systems.

Discrete-Time System Models

The discrete-time system models are representational schemes for digital filters. The MATLAB technical computing environment supports several discrete-time system models, which are described in the following sections:

- “Transfer Function” on page 1-37
- “Zero-Pole-Gain” on page 1-37
- “State Space” on page 1-38
- “Partial Fraction Expansion (Residue Form)” on page 1-39
- “Second-Order Sections (SOS)” on page 1-40
- “Lattice Structure” on page 1-41
- “Convolution Matrix” on page 1-43

Transfer Function

The *transfer function* is a basic *z*-domain representation of a digital filter, expressing the filter as a ratio of two polynomials. It is the principal discrete-time model for this toolbox. The transfer function model description for the *z*-transform of a digital filter's difference equation is

$$Y(z) = \frac{b(1) + b(2)z^{-1} + \ldots + b(n+1)z^{-n}}{a(1) + a(2)z^{-1} + \ldots + a(m+1)z^{-m}} X(z)$$

Here, the constants $b(i)$ and $a(i)$ are the filter coefficients, and the order of the filter is the maximum of n and m. In the MATLAB environment, you store these coefficients in two vectors (row vectors by convention), one row vector for the numerator and one for the denominator. See “Filters and Transfer Functions” on page 1-3 for more details on the transfer function form.

Zero-Pole-Gain

The factored or *zero-pole-gain* form of a transfer function is

$$H(z) = \frac{q(z)}{p(z)} = k \frac{(z - q(1))(z - q(2))...(z - q(n))}{(z - p(1))(z - p(2))...(z - p(n))}$$

By convention, polynomial coefficients are stored in row vectors and polynomial roots in column vectors. In zero-pole-gain form, therefore, the zero and pole locations for the numerator and denominator of a transfer function reside in column vectors. The factored transfer function gain k is a MATLAB scalar.

The `poly` and `roots` functions convert between polynomial and zero-pole-gain representations. For example, a simple IIR filter is

```
b = [2 3 4];
a = [1 3 3 1];
```

The zeros and poles of this filter are

```
q = roots(b)
p = roots(a)
% Gain factor
k = b(1)/a(1)
```

Returning to the original polynomials,

```
bb = k*poly(q)
aa = poly(p)
```

Note that `b` and `a` in this case represent the transfer function:

$$H(z) = \frac{2 + 3z^{-1} + 4z^{-2}}{1 + 3z^{-1} + 3z^{-2} + z^{-3}} = \frac{2z^2 + 3z + 4}{z^3 + 3z^2 + 3z + 1}$$

For `b = [2 3 4]`, the `roots` function misses the zero for z equal to 0. In fact, it misses poles and zeros for z equal to 0 whenever the input transfer function has more poles than zeros, or vice versa. This is acceptable in most cases. To circumvent the problem, however, simply append zeros to make the vectors the same length before using the `roots` function; for example, `b = [b 0]`.

State Space

It is always possible to represent a digital filter, or a system of difference equations, as a set of first-order difference equations. In matrix or *state-space* form, you can write the equations as

$$\begin{aligned} x(n+1) &= Ax(n) + Bu(n) \\ y(n) &= Cx(n) + Du(n) \end{aligned}$$

where `u` is the input, `x` is the state vector, and `y` is the output. For single-channel systems, `A` is an `m`-by-`m` matrix where `m` is the order of the filter, `B` is a column vector, `C` is a row vector, and `D` is a scalar. State-space notation is especially convenient for multichannel systems where input `u` and output `y` become vectors, and `B`, `C`, and `D` become matrices.

State-space representation extends easily to the MATLAB environment. `A`, `B`, `C`, and `D` are rectangular arrays; MATLAB functions treat them as individual variables.

Taking the Z-transform of the state-space equations and combining them shows the equivalence of state-space and transfer function forms:

$$Y(z) = H(z)U(z), \text{ where } H(z) = C(zI - A)^{-1}B + D$$

Don't be concerned if you are not familiar with the state-space representation of linear systems. Some of the filter design algorithms use state-space form internally but do not require any knowledge of state-space concepts to use them successfully. If your applications use state-space based signal processing extensively, however, see the Control System Toolbox™ product for a comprehensive library of state-space tools.

Partial Fraction Expansion (Residue Form)

Each transfer function also has a corresponding *partial fraction expansion* or *residue* form representation, given by

$$\frac{b(z)}{a(z)} = \frac{r(1)}{1 - p(1)z^{-1}} + \ldots + \frac{r(n)}{1 - p(n)z^{-1}} + k(1) + k(2)z^{-1} + \ldots + k(m - n + 1)z^{-(m-n)}$$

provided $H(z)$ has no repeated poles. Here, n is the degree of the denominator polynomial of the rational transfer function $b(z)/a(z)$. If r is a pole of multiplicity s_r, then $H(z)$ has terms of the form:

$$\frac{r(j)}{1 - p(j)z^{-1}} + \frac{r(j+1)}{(1 - p(j)z^{-1})^2} \ldots + \frac{r(j + s_r - 1)}{(1 - p(j)z^{-1})^{s_r}}$$

The Signal Processing Toolbox `residuez` function in converts transfer functions to and from the partial fraction expansion form. The "`z`" on the end of `residuez` stands for *z*-domain, or discrete domain. `residuez` returns the poles in a column vector `p`, the residues corresponding to the poles in a column vector `r`, and any improper part of the original transfer function in a row vector `k`. `residuez` determines that two poles are the

same if the magnitude of their difference is smaller than 0.1 percent of either of the poles' magnitudes.

Partial fraction expansion arises in signal processing as one method of finding the inverse Z-transform of a transfer function. For example, the partial fraction expansion of

$$H(z) = \frac{-4 + 8z^{-1}}{1 + 6z^{-1} + 8z^{-2}}$$

is

```
b = [-4 8];
a = [1 6 8];
[r,p,k] = residuez(b,a)
```

which corresponds to

$$H(z) = \frac{-12}{1 + 4z^{-1}} + \frac{8}{1 + 2z^{-1}}$$

To find the inverse Z-transform of $H(z)$, find the sum of the inverse Z-transforms of the two addends of $H(z)$, giving the causal impulse response:

$$h(n) = -12(-4)^n + 8(-2)^n, \quad n = 0,1,2,\ldots$$

To verify this in the MATLAB environment, type

```
imp = [1 0 0 0 0];
resptf = filter(b,a,imp)
respres = filter(r(1),[1 -p(1)],imp)+...
  filter(r(2),[1 -p(2)],imp)
```

Second-Order Sections (SOS)

Any transfer function $H(z)$ has a second-order sections representation

$$H(z) = \prod_{k=1}^{L} H_k(z) = \prod_{k=1}^{L} \frac{b_{0k} + b_{1k}z^{-1} + b_{2k}z^{-2}}{a_{0k} + a_{1k}z^{-1} + a_{2k}z^{-2}}$$

where L is the number of second-order sections that describe the system. The MATLAB environment represents the second-order section form of a discrete-time system as an L-by-6 array `sos`. Each row of `sos` contains a single second-order section, where the row elements are the three numerator and three denominator coefficients that describe the second-order section.

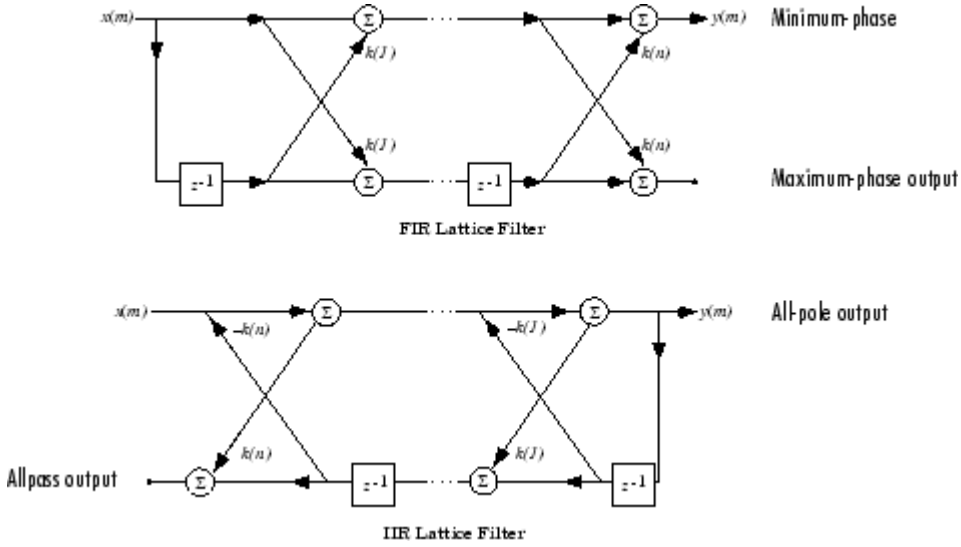
$$\text{sos} = \begin{pmatrix} b_{01} & b_{11} & b_{21} & a_{01} & a_{11} & a_{21} \\ b_{02} & b_{12} & b_{22} & a_{02} & a_{12} & a_{22} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ b_{0L} & b_{1L} & b_{2L} & a_{0L} & a_{1L} & a_{2L} \end{pmatrix}$$

There are many ways to represent a filter in second-order section form. Through careful pairing of the pole and zero pairs, ordering of the sections in the cascade, and multiplicative scaling of the sections, it is possible to reduce quantization noise gain and avoid overflow in some fixed-point filter implementations. The functions `zp2sos` and `ss2sos`, described in “Linear System Transformations” on page 1-47, perform pole-zero pairing, section scaling, and section ordering.

Note All Signal Processing Toolbox second-order section transformations apply only to digital filters.

Lattice Structure

For a discrete Nth order all-pole or all-zero filter described by the polynomial coefficients $a(n)$, $n = 1, 2, \ldots, N+1$, there are N corresponding lattice structure coefficients $k(n)$, $n = 1, 2, \ldots, N$. The parameters $k(n)$ are also called the *reflection coefficients* of the filter. Given these reflection coefficients, you can implement a discrete filter as shown below.



FIR and IIR Lattice Filter structure diagrams

For a general pole-zero IIR filter described by polynomial coefficients *a* and *b*, there are both lattice coefficients *k*(*n*) for the denominator *a* and ladder coefficients *v*(*n*) for the numerator *b*. The lattice/ladder filter may be implemented as

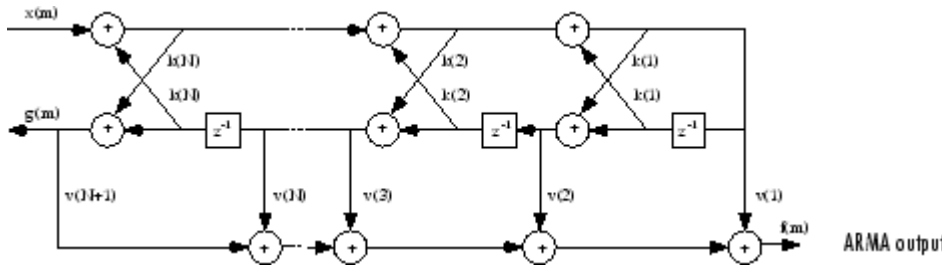


Diagram of lattice/ladder filter

The toolbox function `tf2latc` accepts an FIR or IIR filter in polynomial form and returns the corresponding reflection coefficients. An example FIR filter in polynomial form is

```
b = [1.0000   0.6149   0.9899   0.0000   0.0031  -0.0082];
```

This filter's lattice (reflection coefficient) representation is

```
k = tf2latc(b)
```

For IIR filters, the magnitude of the reflection coefficients provides an easy stability check. If all the reflection coefficients corresponding to a polynomial have magnitude less than 1, all of that polynomial's roots are inside the unit circle. For example, consider an IIR filter with numerator polynomial `b` from above and denominator polynomial:

```
a = [1 1/2 1/3];
```

The filter's lattice representation is

```
[k,v] = tf2latc(b,a);
```

Because `abs(k) < 1` for all reflection coefficients in `k`, the filter is stable.

The function `latc2tf` calculates the polynomial coefficients for a filter from its lattice (reflection) coefficients. Given the reflection coefficient vector `k`, the corresponding polynomial form is

```
b = latc2tf(k);
```

The lattice or lattice/ladder coefficients can be used to implement the filter using the function `latcfilt`.

Convolution Matrix

In signal processing, convolving two vectors or matrices is equivalent to filtering one of the input operands by the other. This relationship permits the representation of a digital filter as a *convolution matrix*.

Given any vector, the toolbox function `convmtx` generates a matrix whose inner product with another vector is equivalent to the convolution of the two vectors. The generated matrix represents a digital filter that you can apply to any vector of appropriate length; the inner dimension of the operands must agree to compute the inner product.

The convolution matrix for a vector `b`, representing the numerator coefficients for a digital filter, is

```
b = [1 2 3];
x = randn(3,1);
C = convmtx(b',3);
```

Two equivalent ways to convolve `b` with `x` are as follows.

```
y1 = C*x;
y2 = conv(b,x);
```

Continuous-Time System Models

The continuous-time system models are representational schemes for analog filters. Many of the discrete-time system models described earlier are also appropriate for the representation of continuous-time systems:

- State-space form
- Partial fraction expansion
- Transfer function
- Zero-pole-gain form

It is possible to represent any system of linear time-invariant differential equations as a set of first-order differential equations. In matrix or *state-space* form, you can express the equations as

$$\dot{x} = Ax + Bu$$
$$y = Cx + Du$$

where u is a vector of nu inputs, x is an nx-element state vector, and y is a vector of ny outputs. In the MATLAB environment, `A`, `B`, `C`, and `D` are stored in separate rectangular arrays.

An equivalent representation of the state-space system is the Laplace transform transfer function description

$$Y(s) = H(s)U(s)$$

where

$$H(s) = C(sI - A)^{-1}B + D$$

For single-input, single-output systems, this form is given by

$$H(s) = \frac{b(s)}{a(s)} = \frac{b(1)s^n + b(2)s^{n-1} + \ldots + b(n+1)}{a(1)s^m + a(2)s^{m-1} + \ldots + a(m+1)}$$

Given the coefficients of a Laplace transform transfer function, `residue` determines the partial fraction expansion of the system. See the description of `residue` for details.

The factored zero-pole-gain form is

$$H(s) = \frac{z(s)}{p(s)} = k\frac{(s - z(1))(s - z(2))\ldots(s - z(n))}{(s - p(1))(s - p(2))\ldots(s - p(m))}$$

As in the discrete-time case, the MATLAB environment stores polynomial coefficients in row vectors in descending powers of s. It stores polynomial roots, or zeros and poles, in column vectors.

Linear System Transformations

A number of Signal Processing Toolbox functions are provided to convert between the various linear system models. You can use the following chart to find an appropriate transfer function: find the row of the model to convert *from* on the left side of the chart and the column of the model to convert *to* on the top of the chart and read the function name(s) at the intersection of the row and column. Note that some cells of this table are empty.

To → From ↓	Transfer Function	State-Space	Zero-Pole- Gain	Partial Fraction	Lattice Filter	Second-Order Sections	Convolution Matrix
Transfer Function		`tf2ss`	`tf2zp` `roots`	`residuez`	`tf2latc`	none	`convmtx`
State-Space	`ss2tf`		`ss2zp`	none	none	`ss2sos`	none
Zero-Pole-Gain	`zp2tf` `poly`	`zp2ss`		none	none	`zp2sos`	none
Partial Fraction	`residuez`	none	none		none	none	none
Lattice Filter	`latc2tf`	none	none	none		none	none
SOS	`sos2tf`	`sos2ss`	`sos2zp`	none	none		none

Note Converting from one filter structure or model to another may produce a result with different characteristics than the original. This is due to the computer's finite-precision arithmetic and the variations in the conversion's round-off computations.

Many of the toolbox filter design functions use these functions internally. For example, the `zp2ss` function converts the poles and zeros of an analog prototype into the state-space form required for creation of a Butterworth, Chebyshev, or elliptic filter. Once in state-space form, the filter design function performs any required frequency transformation, that is, it transforms the initial lowpass design into a bandpass, highpass, or bandstop filter, or a lowpass filter with the desired cutoff frequency.

Note All Signal Processing Toolbox second-order section transformations apply only to digital filters.

Discrete Fourier Transform

The discrete Fourier transform, or DFT, is the primary tool of digital signal processing. The foundation of the product is the fast Fourier transform (FFT), a method for computing the DFT with reduced execution time. Many of the toolbox functions (including *Z*-domain frequency response, spectrum and cepstrum analysis, and some filter design and implementation functions) incorporate the FFT.

The MATLAB® environment provides the functions `fft` and `ifft` to compute the discrete Fourier transform and its inverse, respectively. For the input sequence *x* and its transformed version *X* (the discrete-time Fourier transform at equally spaced frequencies around the unit circle), the two functions implement the relationships

$$X(k+1) = \sum_{n=0}^{N-1} x(n+1) W_N^{kn}$$

and

$$x(n+1) = \frac{1}{N} \sum_{k=0}^{N-1} X(k+1) W_N^{-kn}.$$

In these equations, the series subscripts begin with 1 instead of 0 because of the MATLAB vector indexing scheme, and

$$W_N = e^{-j2\pi/N}.$$

Note The MATLAB convention is to use a negative *j* for the `fft` function. This is an engineering convention; physics and pure mathematics typically use a positive *j*.

`fft`, with a single input argument, `x`, computes the DFT of the input vector or matrix. If `x` is a vector, `fft` computes the DFT of the vector; if `x` is a rectangular array, `fft` computes the DFT of each array column.

For example, create a time vector and signal:

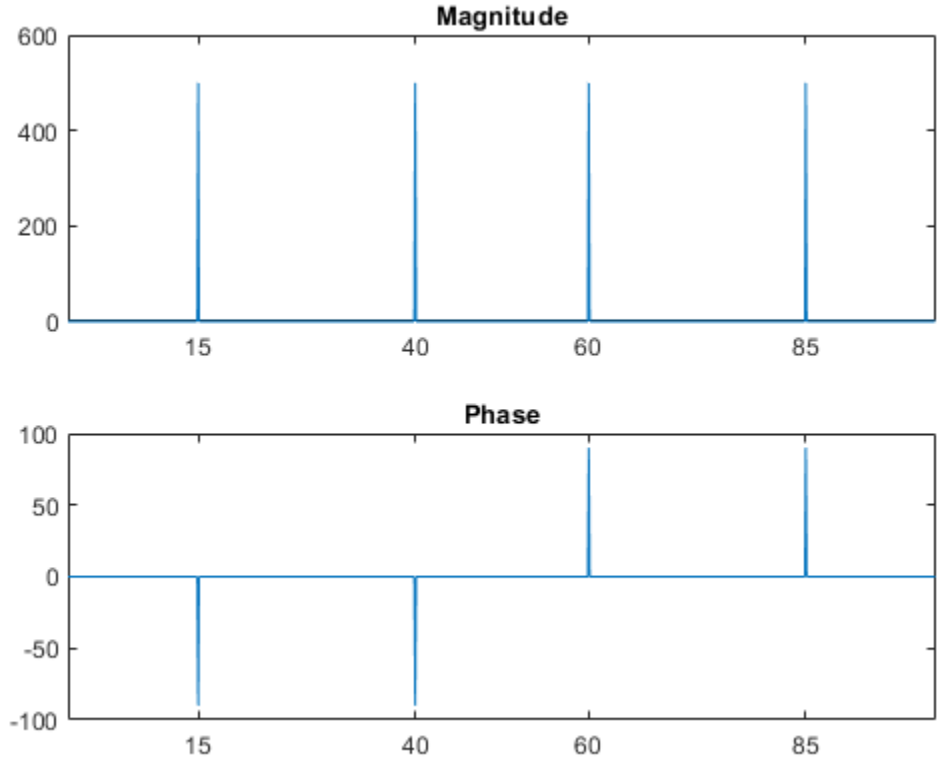
```
t = 0:1/100:10-1/100;                    % Time vector
x = sin(2*pi*15*t) + sin(2*pi*40*t);     % Signal
```

Compute the DFT of the signal and the magnitude and phase of the transformed sequence. Decrease round-off error when computing the phase by setting small-magnitude transform values to zero.

```
y = fft(x);                                 % Compute DFT of x
m = abs(y);                                 % Magnitude
y(m<1e-6) = 0;
p = unwrap(angle(y));                       % Phase
```

To plot the magnitude and phase in degrees, type the following commands:

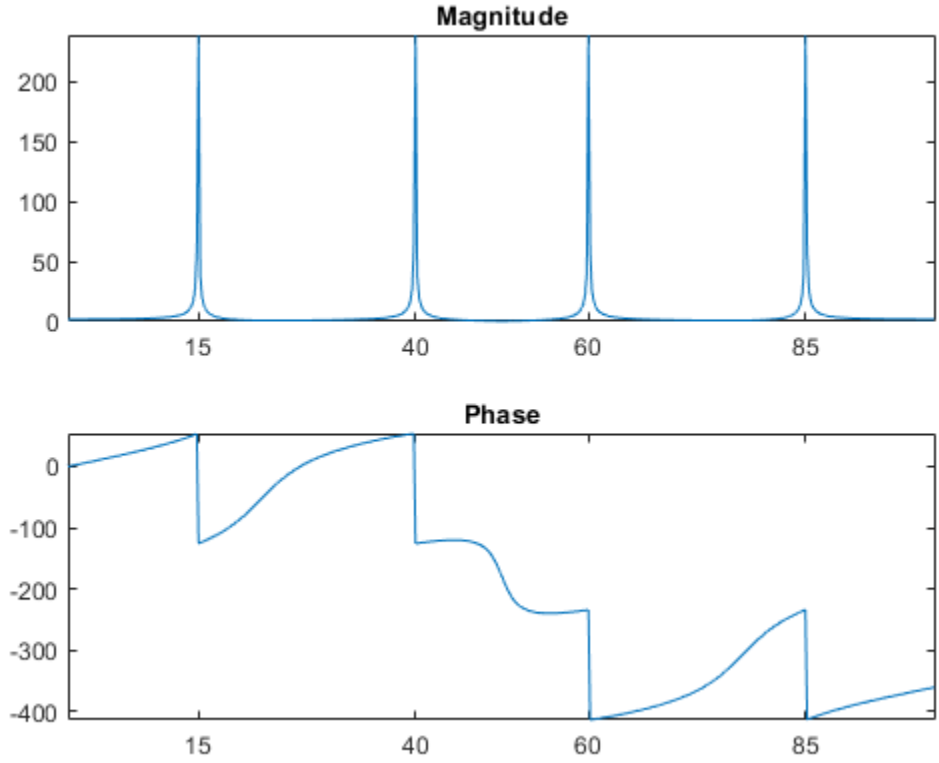
```
f = (0:length(y)-1)*100/length(y);          % Frequency vector

subplot(2,1,1)
plot(f,m)
title('Magnitude')
ax = gca;
ax.XTick = [15 40 60 85];

subplot(2,1,2)
plot(f,p*180/pi)
title('Phase')
ax = gca;
ax.XTick = [15 40 60 85];
```



A second argument to `fft` specifies a number of points `n` for the transform, representing DFT length:

```
n = 512;
y = fft(x,n);
m = abs(y);
p = unwrap(angle(y));
f = (0:length(y)-1)*100/length(y);

subplot(2,1,1)
plot(f,m)
title('Magnitude')
ax = gca;
ax.XTick = [15 40 60 85];
```

```
subplot(2,1,2)
plot(f,p*180/pi)
title('Phase')
ax = gca;
ax.XTick = [15 40 60 85];
```



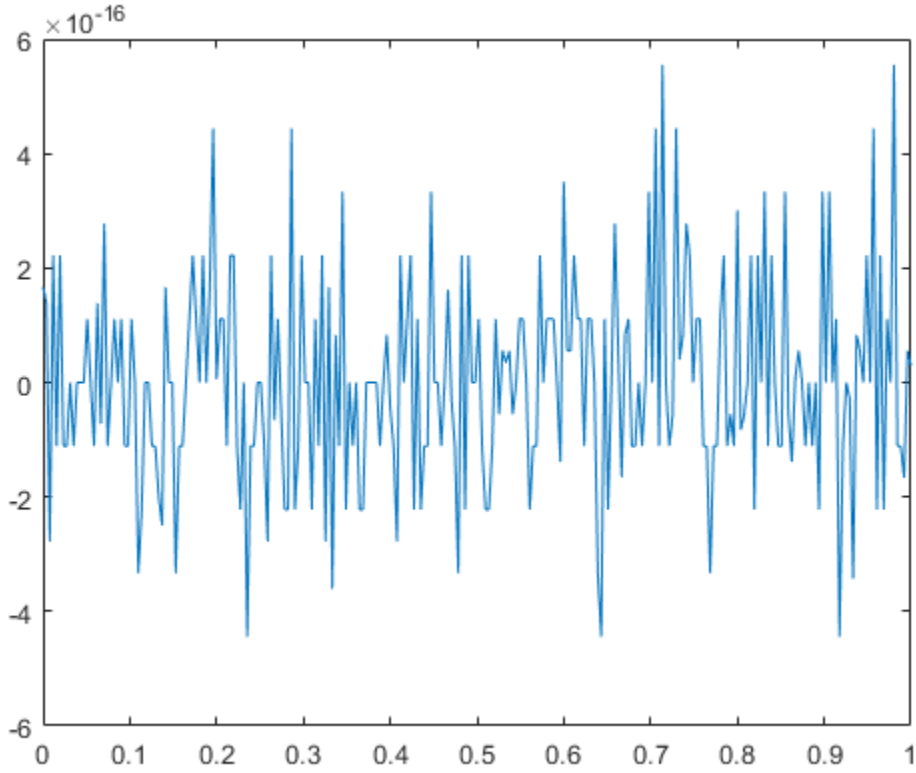
In this case, `fft` pads the input sequence with zeros if it is shorter than `n`, or truncates the sequence if it is longer than `n`. If `n` is not specified, it defaults to the length of the input sequence. Execution time for `fft` depends on the length, `n`, of the DFT it performs; see the `fft` reference page for details about the algorithm.

Note The resulting FFT amplitude is `A*n/2`, where `A` is the original amplitude and `n` is the number of FFT points. This is true only if the number of FFT points is greater than or

equal to the number of data samples. If the number of FFT points is less, the FFT amplitude is lower than the original amplitude by the above amount.

The inverse discrete Fourier transform function `ifft` also accepts an input sequence and, optionally, the number of desired points for the transform. Try the example below; the original sequence `x` and the reconstructed sequence are identical (within rounding error).

```
t = 0:1/255:1;
x = sin(2*pi*120*t);
y = real(ifft(fft(x)));

figure
plot(t,x-y)
```



This toolbox also includes functions for the two-dimensional FFT and its inverse, `fft2` and `ifft2`. These functions are useful for two-dimensional signal or image processing. The goertzel function, which is another algorithm to compute the DFT, also is included in the toolbox. This function is efficient for computing the DFT of a portion of a long signal.

It is sometimes convenient to rearrange the output of the `fft` or `fft2` function so the zero frequency component is at the center of the sequence. The function `fftshift` moves the zero frequency component to the center of a vector or matrix.

See Also

`fft` | `fft2` | `fftshift` | `goertzel` | `ifft` | `ifft2`

Filter Design and Implementation

- “Filter Requirements and Specification” on page 2-2
- “IIR Filter Design” on page 2-5
- “FIR Filter Design” on page 2-19
- “Special Topics in IIR Filter Design” on page 2-42
- “Filtering Data With Signal Processing Toolbox Software” on page 2-51
- “Selected Bibliography” on page 2-70

Filter Requirements and Specification

Filter design is the process of creating the filter coefficients to meet specific filtering requirements. Filter implementation involves choosing and applying a particular filter structure to those coefficients. Only after both design and implementation have been performed can data be filtered. The following chapter describes filter design and implementation in Signal Processing Toolbox™ software.

The goal of filter design is to perform frequency dependent alteration of a data sequence. A possible requirement might be to remove noise above 200 Hz from a data sequence sampled at 1000 Hz. A more rigorous specification might call for a specific amount of passband ripple, stopband attenuation, or transition width. A very precise specification could ask to achieve the performance goals with the minimum filter order, or it could call for an arbitrary magnitude shape, or it might require an FIR filter. Filter design methods differ primarily in how performance is specified.

To design a filter, the Signal Processing Toolbox software offers two approaches. The first approach uses the `designfilt` function. As an example, design and implement a 5th-order lowpass Butterworth filter with a 3-dB frequency of 200 Hz. Assume a sample rate of 1 kHz. Apply the filter to input data.

```
Fs = 1000;
fc = 200;
time = 0:1/Fs:1;
x = cos(2*pi*60*time)+sin(2*pi*120*time)+randn(size(time));

d = designfilt('lowpassiir','FilterOrder',5, ...
    'HalfPowerFrequency',fc,'DesignMethod','butter', ...
    'SampleRate',Fs);
yd = filter(d,x);
```

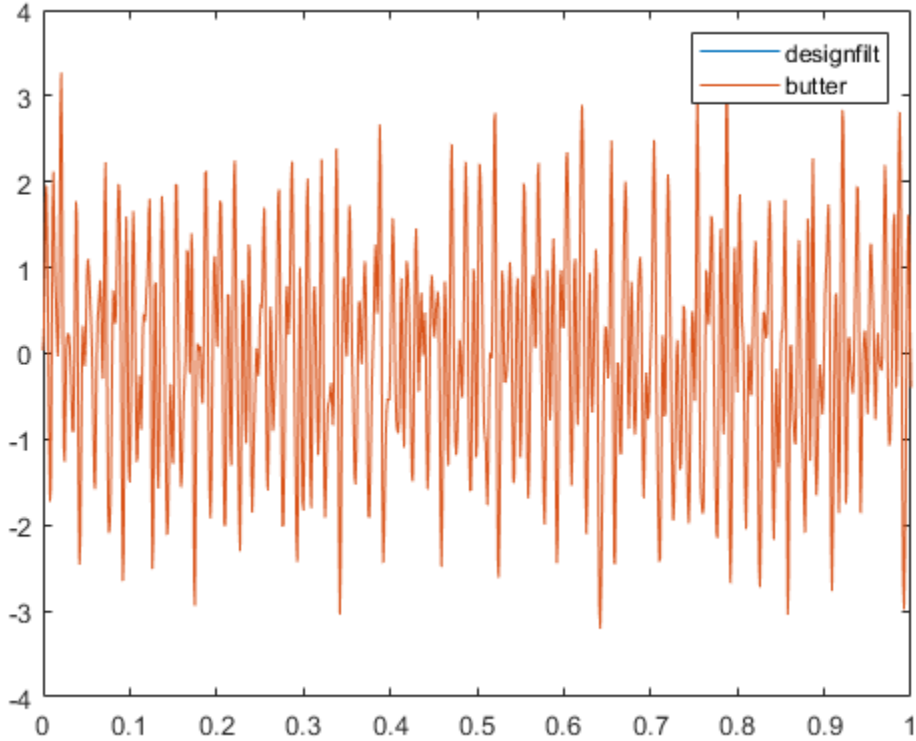
The other approach implements the filter using a function such as `butter` or `firpm`. All of these "classic" filter design functions operate with normalized frequencies. Convert frequency specifications in Hz to normalized frequency to use these functions. The Signal Processing Toolbox software defines normalized frequency to be in the closed interval [0,1], with 1 denoting π rad/sample. For example, to specify a normalized frequency of $\pi/2$ rad/sample, enter 0.5.

To convert from Hz to normalized frequency, multiply the frequency in Hz by two and divide by the sampling frequency. For example, design a 5th-order lowpass Butterworth filter with a 3-dB frequency of 200 Hz using `butter`.

```
Wn = fc/(Fs/2);

[b,a] = butter(5,Wn,'low');
yb = filter(b,a,x);
```

Plot the two filtered signals.

```
plot(time,yd,time,yb)
legend('designfilt','butter')
```



See Also

`butter` | `designfilt` | `filter`

IIR Filter Design

In this section...
“IIR vs. FIR Filters” on page 2-5
“Classical IIR Filters” on page 2-5
“Other IIR Filters” on page 2-5
“IIR Filter Method Summary” on page 2-6
“Classical IIR Filter Design Using Analog Prototyping” on page 2-7
“Comparison of Classical IIR Filter Types” on page 2-10

IIR vs. FIR Filters

The primary advantage of IIR filters over FIR filters is that they typically meet a given set of specifications with a much lower filter order than a corresponding FIR filter. Although IIR filters have nonlinear phase, data processing within MATLAB software is commonly performed “offline,” that is, the entire data sequence is available prior to filtering. This allows for a noncausal, zero-phase filtering approach (via the `filtfilt` function), which eliminates the nonlinear phase distortion of an IIR filter.

Classical IIR Filters

The classical IIR filters, Butterworth, Chebyshev Types I and II, elliptic, and Bessel, all approximate the ideal “brick wall” filter in different ways.

This toolbox provides functions to create all these types of classical IIR filters in both the analog and digital domains (except Bessel, for which only the analog case is supported), and in lowpass, highpass, bandpass, and bandstop configurations. For most filter types, you can also find the lowest filter order that fits a given filter specification in terms of passband and stopband attenuation, and transition width(s).

Other IIR Filters

The direct filter design function `yulewalk` finds a filter with magnitude response approximating a specified frequency-response function. This is one way to create a multiband bandpass filter.

You can also use the parametric modeling or system identification functions to design IIR filters. These functions are discussed in “Parametric Modeling” on page 7-26.

The generalized Butterworth design function `maxflat` is discussed in the section “Generalized Butterworth Filter Design” on page 2-17.

IIR Filter Method Summary

The following table summarizes the various filter methods in the toolbox and lists the functions available to implement these methods.

Toolbox Filters Methods and Available Functions

Filter Method	Description	Filter Functions

Analog Prototyping	Using the poles and zeros of a classical lowpass prototype filter in the continuous (Laplace) domain, obtain a digital filter through frequency transformation and filter discretization.	Complete design functions:
		`besself`, `butter`, `cheby1`, `cheby2`, `ellip`
		
		Order estimation functions:
		`buttord`, `cheb1ord`, `cheb2ord`, `ellipord`
		
		Lowpass analog prototype functions:
		`besselap`, `buttap`, `cheb1ap`, `cheb2ap`, `ellipap`
		
		Frequency transformation functions:
		`lp2bp`, `lp2bs`, `lp2hp`, `lp2lp`
		
		Filter discretization functions:
		`bilinear`, `impinvar`

Direct Design	Design digital filter directly in the discrete time-domain by approximating a piecewise linear magnitude response.	`yulewalk`
Generalized Butterworth Design	Design lowpass Butterworth filters with more zeros than poles.	`maxflat`

Parametric Modeling	Find a digital filter that approximates a prescribed time or frequency domain response. (See System Identification Toolbox™ documentation for an extensive collection of parametric modeling tools.)	Time-domain modeling functions:
		`lpc`, `prony`, `stmcb`
		
		Frequency-domain modeling functions:
		`invfreqs`, `invfreqz`

Classical IIR Filter Design Using Analog Prototyping

The principal IIR digital filter design technique this toolbox provides is based on the conversion of classical lowpass analog filters to their digital equivalents. The following sections describe how to design filters and summarize the characteristics of the

supported filter types. See “Special Topics in IIR Filter Design” on page 2-42 for detailed steps on the filter design process.

Complete Classical IIR Filter Design

You can easily create a filter of any order with a lowpass, highpass, bandpass, or bandstop configuration using the filter design functions.

Filter Design Functions

Filter Type	Design Function

Bessel (analog only)	`[b,a] = besself(n,Wn,options)`
	`[z,p,k] = besself(n,Wn,options)`
	`[A,B,C,D] = besself(n,Wn,options)`

Butterworth	`[b,a] = butter(n,Wn,options)`
	`[z,p,k] = butter(n,Wn,options)`
	`[A,B,C,D] = butter(n,Wn,options)`

Chebyshev Type I	`[b,a] = cheby1(n,Rp,Wn,options)`
	`[z,p,k] = cheby1(n,Rp,Wn,options)`
	`[A,B,C,D] = cheby1(n,Rp,Wn,options)`

Chebyshev Type II	`[b,a] = cheby2(n,Rs,Wn,options)`
	`[z,p,k] = cheby2(n,Rs,Wn,options)`
	`[A,B,C,D] = cheby2(n,Rs,Wn,options)`

Elliptic	`[b,a] = ellip(n,Rp,Rs,Wn,options)`
	`[z,p,k] = ellip(n,Rp,Rs,Wn,options)`
	`[A,B,C,D] = ellip(n,Rp,Rs,Wn,options)`

By default, each of these functions returns a lowpass filter; you need to specify only the cutoff frequency that you want, `Wn`, in normalized units such that the Nyquist frequency is 1 Hz). For a highpass filter, append `'high'` to the function's parameter list. For a

bandpass or bandstop filter, specify `Wn` as a two-element vector containing the passband edge frequencies. Append `'stop'` for the bandstop configuration.

Here are some example digital filters:

```
[b,a] = butter(5,0.4);                    % Lowpass Butterworth
[b,a] = cheby1(4,1,[0.4 0.7]);            % Bandpass Chebyshev Type I
[b,a] = cheby2(6,60,0.8,'high');          % Highpass Chebyshev Type II
[b,a] = ellip(3,1,60,[0.4 0.7],'stop');   % Bandstop elliptic
```

To design an analog filter, perhaps for simulation, use a trailing `'s'` and specify cutoff frequencies in rad/s:

```
[b,a] = butter(5,0.4,'s');      % Analog Butterworth filter
```

All filter design functions return a filter in the transfer function, zero-pole-gain, or state-space linear system model representation, depending on how many output arguments are present. In general, you should avoid using the transfer function form because numerical problems caused by roundoff errors can occur. Instead, use the zero-pole-gain form which you can convert to a second-order section (SOS) form using `zp2sos` and then use the SOS form to analyze or implement your filter.

Note All classical IIR lowpass filters are ill-conditioned for extremely low cutoff frequencies. Therefore, instead of designing a lowpass IIR filter with a very narrow passband, it can be better to design a wider passband and decimate the input signal.

Designing IIR Filters to Frequency Domain Specifications

This toolbox provides order selection functions that calculate the minimum filter order that meets a given set of requirements.

Filter Type	Order Estimation Function
Butterworth	`[n,Wn] = buttord(Wp,Ws,Rp,Rs)`
Chebyshev Type I	`[n,Wn] = cheb1ord(Wp,Ws,Rp,Rs)`
Chebyshev Type II	`[n,Wn] = cheb2ord(Wp,Ws,Rp,Rs)`
Elliptic	`[n,Wn] = ellipord(Wp,Ws,Rp,Rs)`

These are useful in conjunction with the filter design functions. Suppose you want a bandpass filter with a passband from 1000 to 2000 Hz, stopbands starting 500 Hz away on either side, a 10 kHz sampling frequency, at most 1 dB of passband ripple, and at least

60 dB of stopband attenuation. You can meet these specifications by using the `butter` function as follows.

```
[n,Wn] = buttord([1000 2000]/5000,[500 2500]/5000,1,60)
[b,a] = butter(n,Wn);

n =
    12
Wn =
    0.1951    0.4080
```

An elliptic filter that meets the same requirements is given by

```
[n,Wn] = ellipord([1000 2000]/5000,[500 2500]/5000,1,60)
[b,a] = ellip(n,1,60,Wn);

n =
    5
Wn =
    0.2000    0.4000
```

These functions also work with the other standard band configurations, as well as for analog filters.

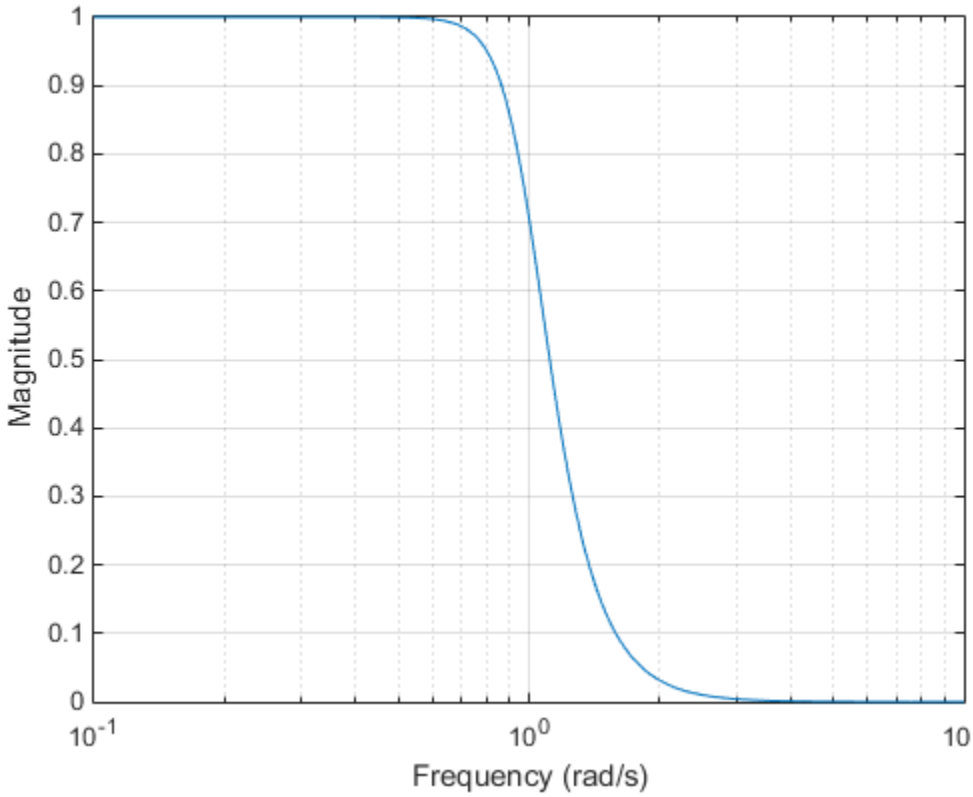
Comparison of Classical IIR Filter Types

The toolbox provides five different types of classical IIR filters, each optimal in some way. This section shows the basic analog prototype form for each and summarizes major characteristics.

Butterworth Filter

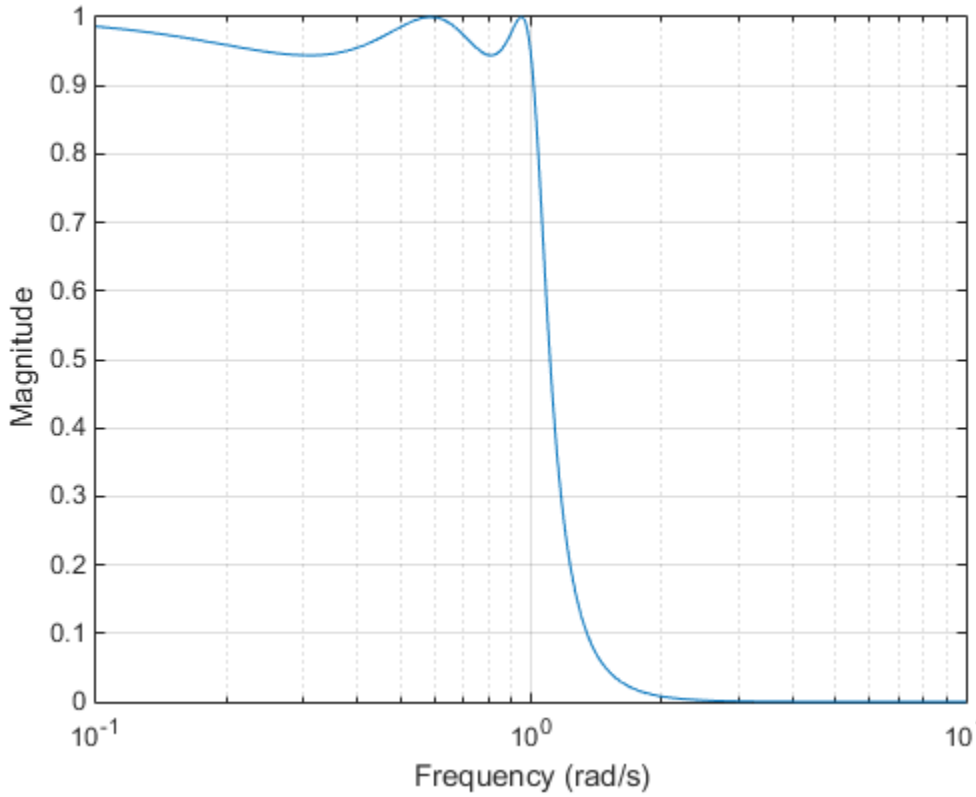
The Butterworth filter provides the best Taylor series approximation to the ideal lowpass filter response at analog frequencies $\Omega = 0$ and $\Omega = \infty$; for any order N, the magnitude squared response has $2N - 1$ zero derivatives at these locations (*maximally flat* at $\Omega = 0$ and $\Omega = \infty$). Response is monotonic overall, decreasing smoothly from $\Omega = 0$ to $\Omega = \infty$.

$|H(j\Omega)| = 1/\sqrt{2}$ at $\Omega = 1$.



Chebyshev Type I Filter

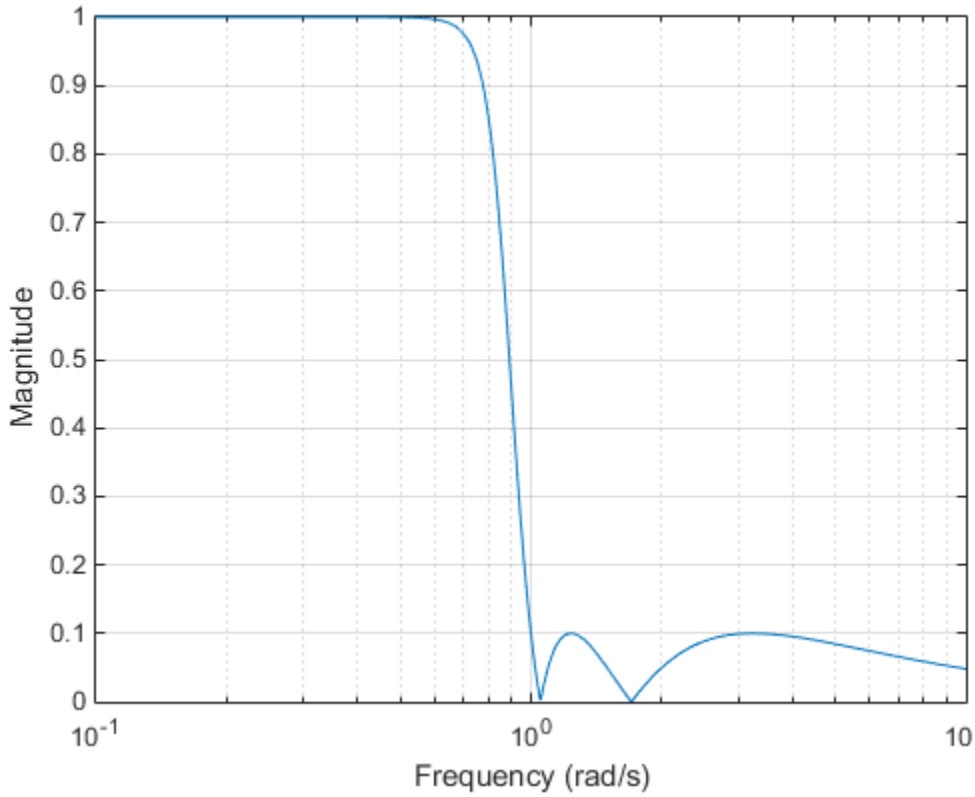
The Chebyshev Type I filter minimizes the absolute difference between the ideal and actual frequency response over the entire passband by incorporating an equal ripple of Rp dB in the passband. Stopband response is maximally flat. The transition from passband to stopband is more rapid than for the Butterworth filter. $|H(j\Omega)| = 10^{-\mathrm{Rp}/20}$ at $\Omega = 1$.



Chebyshev Type II Filter

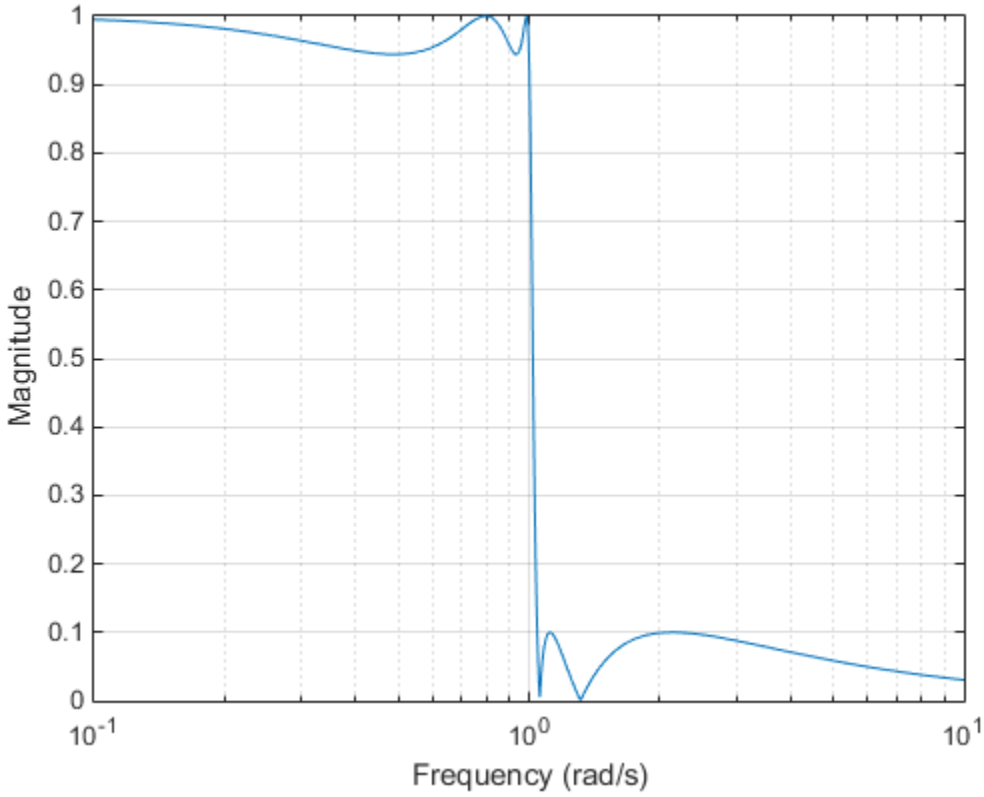
The Chebyshev Type II filter minimizes the absolute difference between the ideal and actual frequency response over the entire stopband by incorporating an equal ripple of `Rs` dB in the stopband. Passband response is maximally flat.

The stopband does not approach zero as quickly as the type I filter (and does not approach zero at all for even-valued filter order n). The absence of ripple in the passband, however, is often an important advantage. $|H(j\Omega)| = 10^{-\mathrm{Rs}/20}$ at $\Omega = 1$.



Elliptic Filter

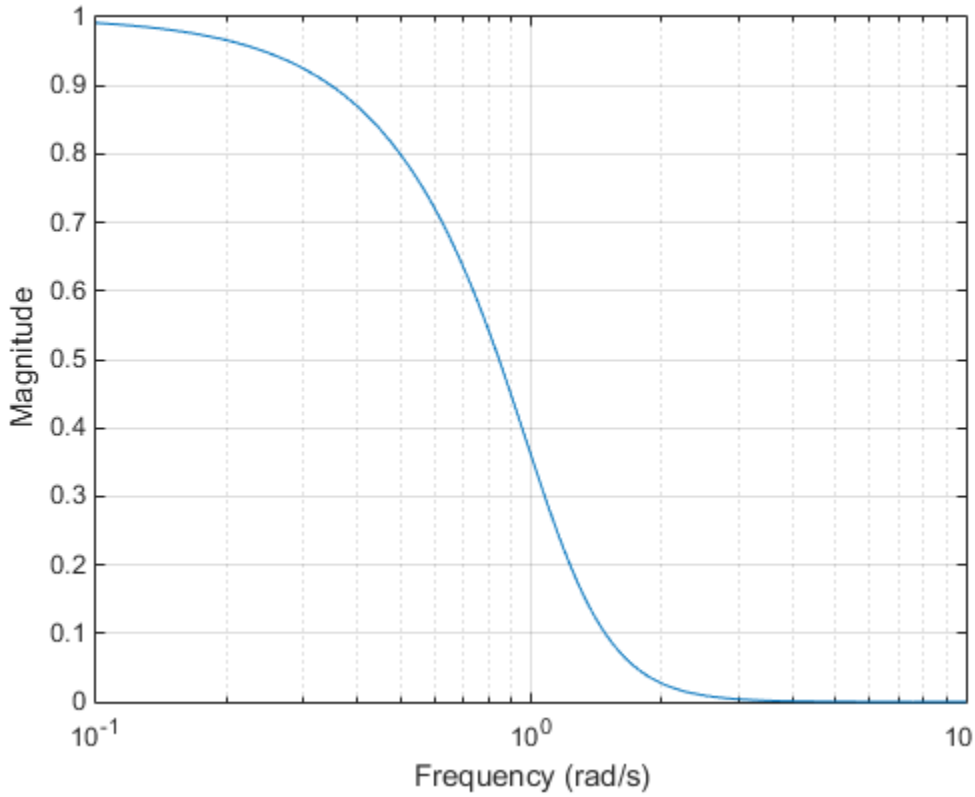
Elliptic filters are equiripple in both the passband and stopband. They generally meet filter requirements with the lowest order of any supported filter type. Given a filter order n, passband ripple `Rp` in decibels, and stopband ripple `Rs` in decibels, elliptic filters minimize transition width. $|H(j\Omega)| = 10^{-\mathrm{Rp}/20}$ at $\Omega = 1$.



Bessel Filter

Analog Bessel lowpass filters have maximally flat group delay at zero frequency and retain nearly constant group delay across the entire passband. Filtered signals therefore maintain their waveshapes in the passband frequency range. Frequency mapped and digital Bessel filters, however, do not have this maximally flat property; this toolbox supports only the analog case for the complete Bessel filter design function.

Bessel filters generally require a higher filter order than other filters for satisfactory stopband attenuation. $|H(j\Omega)| < 1/\sqrt{2}$ at $\Omega = 1$ and decreases as filter order n increases.



Note The lowpass filters shown above were created with the analog prototype functions `besselap`, `buttap`, `cheb1ap`, `cheb2ap`, and `ellipap`. These functions find the zeros, poles, and gain of an `n`th-order analog filter of the appropriate type with a cutoff frequency of 1 rad/s. The complete filter design functions (`besself`, `butter`, `cheby1`, `cheby2`, and `ellip`) call the prototyping functions as a first step in the design process. See “Special Topics in IIR Filter Design” on page 2-42 for details.

To create similar plots, use `n = 5` and, as needed, `Rp = 0.5` and `Rs = 20`. For example, to create the elliptic filter plot:

```
[z,p,k] = ellipap(5,0.5,20);
w = logspace(-1,1,1000);
h = freqs(k*poly(z),poly(p),w);
semilogx(w,abs(h)), grid
xlabel('Frequency (rad/s)')
ylabel('Magnitude')
```

Direct IIR Filter Design

This toolbox uses the term *direct methods* to describe techniques for IIR design that find a filter based on specifications in the discrete domain. Unlike the analog prototyping method, direct design methods are not constrained to the standard lowpass, highpass, bandpass, or bandstop configurations. Rather, these functions design filters with an arbitrary, perhaps multiband, frequency response. This section discusses the `yulewalk` function, which is intended specifically for filter design; “Parametric Modeling” on page 7-26 discusses other methods that may also be considered direct, such as Prony's method, Linear Prediction, the Steiglitz-McBride method, and inverse frequency design.

The `yulewalk` function designs recursive IIR digital filters by fitting a specified frequency response. `yulewalk`'s name reflects its method for finding the filter's denominator coefficients: it finds the inverse FFT of the ideal specified magnitude-squared response and solves the modified Yule-Walker equations using the resulting autocorrelation function samples. The statement

```
[b,a] = yulewalk(n,f,m)
```

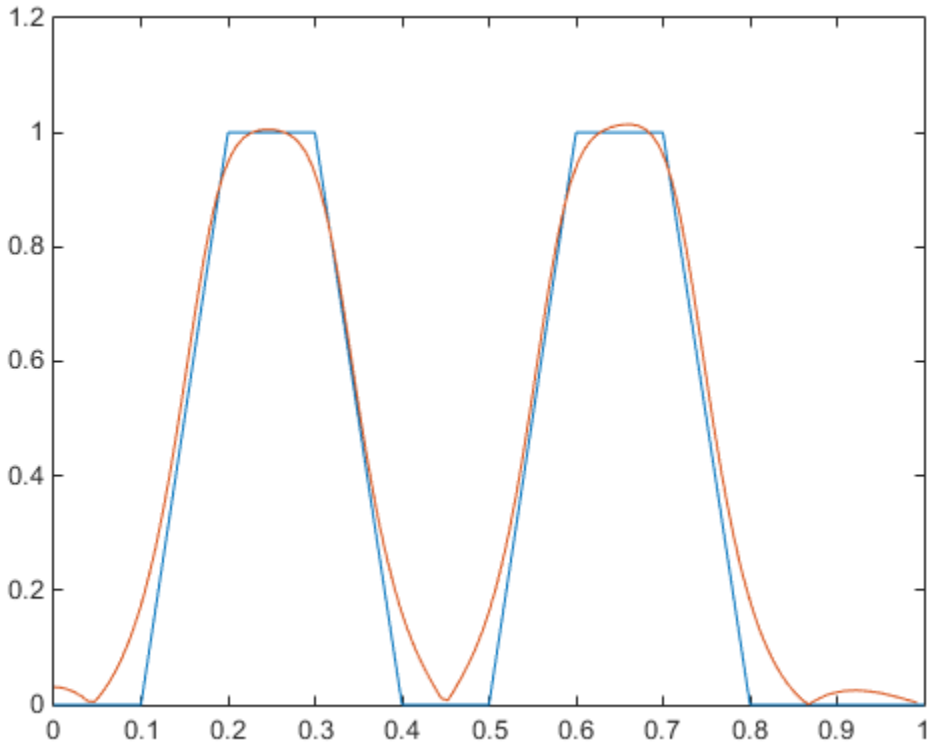
returns row vectors `b` and `a` containing the `n+1` numerator and denominator coefficients of the `n`th-order IIR filter whose frequency-magnitude characteristics approximate those given in vectors `f` and `m`. `f` is a vector of frequency points ranging from 0 to 1, where 1 represents the Nyquist frequency. `m` is a vector containing the specified magnitude response at the points in `f`. `f` and `m` can describe any piecewise linear shape magnitude response, including a multiband response. The FIR counterpart of this function is `fir2`, which also designs a filter based on an arbitrary piecewise linear magnitude response. See “FIR Filter Design” on page 2-19 for details.

Note that `yulewalk` does not accept phase information, and no statements are made about the optimality of the resulting filter.

Design a multiband filter with `yulewalk` and plot the specified and actual frequency response:

```
m = [0   0   1   1   0   0   1   1   0 0];
f = [0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 1];
```

```
[b,a] = yulewalk(10,f,m);
[h,w] = freqz(b,a,128)
plot(f,m,w/pi,abs(h))
```



Generalized Butterworth Filter Design

The toolbox function `maxflat` enables you to design generalized Butterworth filters, that is, Butterworth filters with differing numbers of zeros and poles. This is desirable in some implementations where poles are more expensive computationally than zeros. `maxflat` is just like the `butter` function, except that it you can specify *two* orders (one for the numerator and one for the denominator) instead of just one. These filters are *maximally flat*. This means that the resulting filter is optimal for any numerator and denominator

orders, with the maximum number of derivatives at 0 and the Nyquist frequency $\omega = \pi$ both set to 0.

For example, when the two orders are the same, `maxflat` is the same as `butter`:

```
[b,a] = maxflat(3,3,0.25)

b =
    0.0317    0.0951    0.0951    0.0317
a =
    1.0000   -1.4590    0.9104   -0.1978

[b,a] = butter(3,0.25)

b =
    0.0317    0.0951    0.0951    0.0317
a =
    1.0000   -1.4590    0.9104   -0.1978
```

However, `maxflat` is more versatile because it allows you to design a filter with more zeros than poles:

```
[b,a] = maxflat(3,1,0.25)

b =
    0.0950    0.2849    0.2849    0.0950
a =
    1.0000   -0.2402
```

The third input to `maxflat` is the *half-power frequency*, a frequency between 0 and 1 with a magnitude response of $1/\sqrt{2}$.

You can also design linear phase filters that have the maximally flat property using the `'sym'` option:

```
maxflat(4,'sym',0.3)

ans =
    0.0331    0.2500    0.4337    0.2500    0.0331
```

For complete details of the `maxflat` algorithm, see Selesnick and Burrus [2].

FIR Filter Design

In this section...
“FIR vs. IIR Filters” on page 2-19
“FIR Filter Summary” on page 2-20
“Linear Phase Filters” on page 2-20
“Windowing Method” on page 2-21
“Multiband FIR Filter Design with Transition Bands” on page 2-25
“Constrained Least Squares FIR Filter Design” on page 2-31
“Arbitrary-Response Filter Design” on page 2-36

FIR vs. IIR Filters

Digital filters with finite-duration impulse response (all-zero, or FIR filters) have both advantages and disadvantages compared to infinite-duration impulse response (IIR) filters.

FIR filters have the following primary advantages:

- They can have exactly linear phase.
- They are always stable.
- The design methods are generally linear.
- They can be realized efficiently in hardware.
- The filter startup transients have finite duration.

The primary disadvantage of FIR filters is that they often require a much higher filter order than IIR filters to achieve a given level of performance. Correspondingly, the delay of these filters is often much greater than for an equal performance IIR filter.

FIR Filter Summary

FIR Filters

Filter Design Method	Description	Filter Functions
Windowing	Apply window to truncated inverse Fourier transform of specified "brick wall" filter	`fir1`, `fir2`, `kaiserord`
Multiband with Transition Bands	Equiripple or least squares approach over sub-bands of the frequency range	`firls`, `firpm`, `firpmord`
Constrained Least Squares	Minimize squared integral error over entire frequency range subject to maximum error constraints	`fircls`, `fircls1`
Arbitrary Response	Arbitrary responses, including nonlinear phase and complex filters	`cfirpm`
Raised Cosine	Lowpass response with smooth, sinusoidal transition	`rcosdesign`

Linear Phase Filters

Except for `cfirpm`, all of the FIR filter design functions design linear phase filters only. The filter coefficients, or “taps,” of such filters obey either an even or odd symmetry relation. Depending on this symmetry, and on whether the order n of the filter is even or odd, a linear phase filter (stored in length $n+1$ vector `b`) has certain inherent restrictions on its frequency response.

Linear Phase Filter Type	Filter Order	Symmetry of Coefficients	Response H(f), f = 0	Response H(f), f = 1 (Nyquist)
Type I	Even	even: $b(k) = b(n+2-k), \quad k = 1,\ldots,n+1$	No restriction	No restriction
Type II	Odd	even: $b(k) = b(n+2-k), \quad k = 1,\ldots,n+1$	No restriction	$H(1) = 0$

Linear Phase Filter Type	Filter Order	Symmetry of Coefficients	Response H(f), f = 0	Response H(f), f = 1 (Nyquist)
Type III	Even	odd: $b(k) = -b(n+2-k), \quad k = 1,\ldots,n+1$	$H(0) = 0$	$H(1) = 0$
Type IV	Odd	odd: $b(k) = -b(n+2-k), \quad k = 1,\ldots,n+1$	$H(0) = 0$	No restriction

The phase delay and group delay of linear phase FIR filters are equal and constant over the frequency band. For an order n linear phase FIR filter, the group delay is $n/2$, and the filtered signal is simply delayed by $n/2$ time steps (and the magnitude of its Fourier transform is scaled by the filter's magnitude response). This property preserves the wave shape of signals in the passband; that is, there is no phase distortion.

The functions `fir1`, `fir2`, `firls`, `firpm`, `fircls`, and `fircls1` all design type I and II linear phase FIR filters by default. `rcosdesign` designs only type I filters. Both `firls` and `firpm` design type III and IV linear phase FIR filters given a `'hilbert'` or `'differentiator'` flag. `cfirpm` can design any type of linear phase filter, and nonlinear phase filters as well.

Note Because the frequency response of a type II filter is zero at the Nyquist frequency (“high” frequency), `fir1` does not design type II highpass and bandstop filters. For odd-valued `n` in these cases, `fir1` adds 1 to the order and returns a type I filter.

Windowing Method

Consider the ideal, or “brick wall,” digital lowpass filter with a cutoff frequency of ω_0 rad/s. This filter has magnitude 1 at all frequencies with magnitude less than ω_0, and magnitude 0 at frequencies with magnitude between ω_0 and π. Its impulse response sequence $h(n)$ is

$$h(n) = \frac{1}{2\pi}\int_{-\pi}^{\pi} H(\omega)e^{j\omega n}d\omega = \frac{1}{2\pi}\int_{-\omega_0}^{\omega_0} e^{j\omega n}d\omega = \frac{\sin\omega_0 n}{\pi n}$$

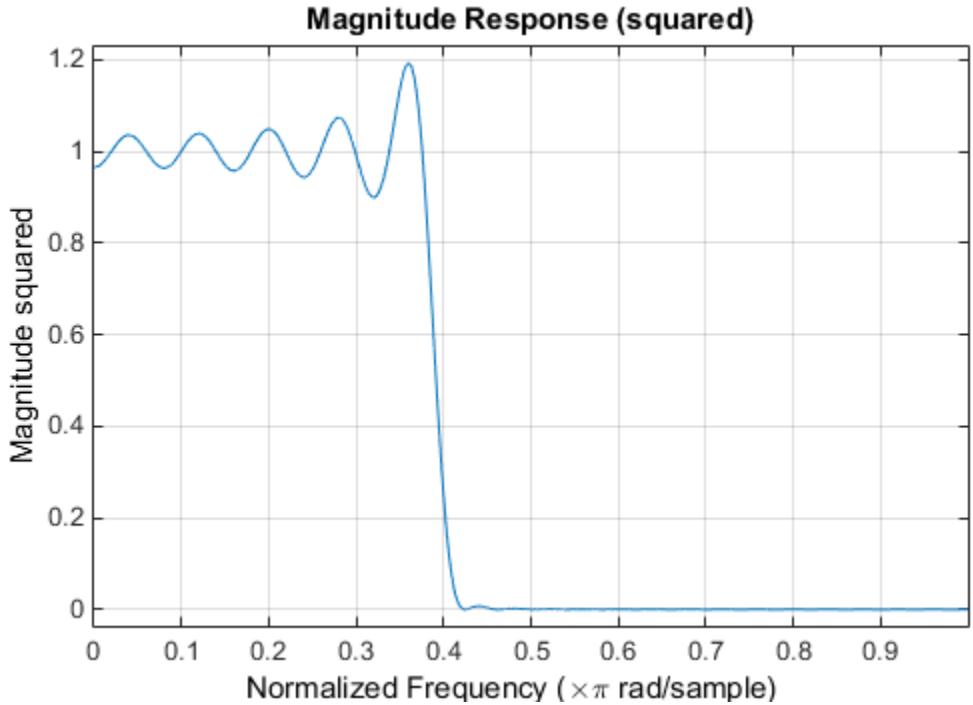
This filter is not implementable since its impulse response is infinite and noncausal. To create a finite-duration impulse response, truncate it by applying a window. By retaining the central section of impulse response in this truncation, you obtain a linear phase FIR filter. For example, a length 51 filter with a lowpass cutoff frequency ω_0 of 0.4 π rad/s is

```
b = 0.4*sinc(0.4*(-25:25));
```

The window applied here is a simple rectangular window. By Parseval's theorem, this is the length 51 filter that best approximates the ideal lowpass filter, in the integrated least squares sense. The following command displays the filter's frequency response in FVTool:

```
fvtool(b,1)
```

Note that the *y*-axis shown in the figure below is in Magnitude Squared. You can set this by right-clicking on the axis label and selecting **Magnitude Squared** from the menu.

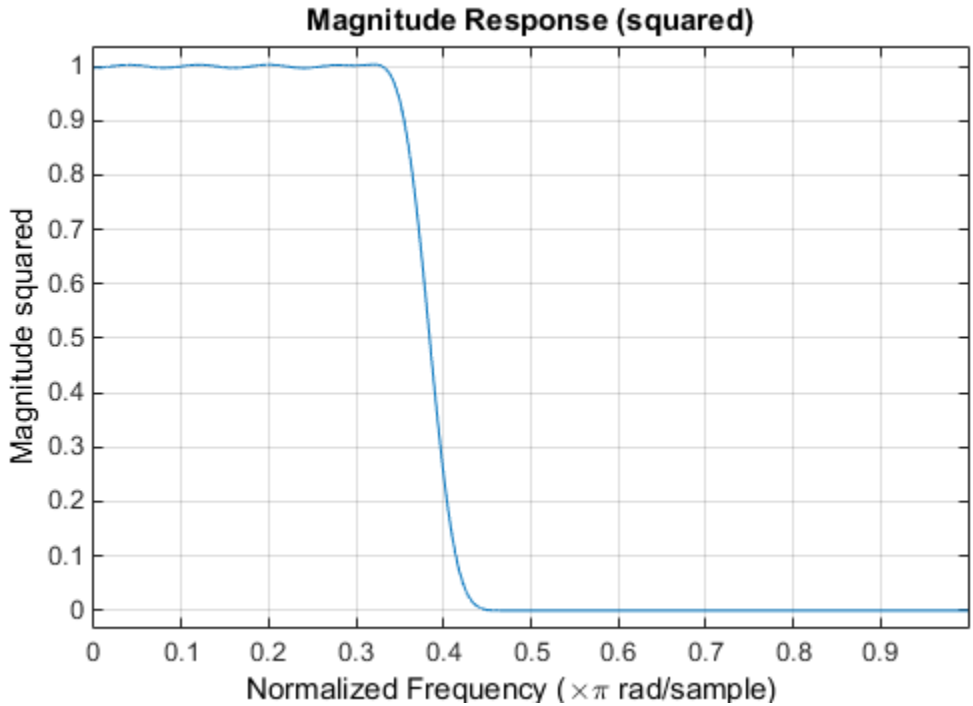


Ringing and ripples occur in the response, especially near the band edge. This "Gibbs effect" does not vanish as the filter length increases, but a nonrectangular window

reduces its magnitude. Multiplication by a window in the time domain causes a convolution or smoothing in the frequency domain. Apply a length 51 Hamming window to the filter and display the result using FVTool:

```
b = 0.4*sinc(0.4*(-25:25));
b = b.*hamming(51)';
fvtool(b,1)
```

Note that the *y*-axis shown in the figure below is in Magnitude Squared. You can set this by right-clicking on the axis label and selecting **Magnitude Squared** from the menu.



Using a Hamming window greatly reduces the ringing. This improvement is at the expense of transition width (the windowed version takes longer to ramp from passband to stopband) and optimality (the windowed version does not minimize the integrated squared error).

The functions `fir1` and `fir2` are based on this windowing process. Given a filter order and description of an ideal filter, these functions return a windowed inverse Fourier

transform of that ideal filter. Both use a Hamming window by default, but they accept any window function. See “Windows” on page 7-2 for an overview of windows and their properties.

Standard Band FIR Filter Design: fir1

`fir1` implements the classical method of windowed linear phase FIR digital filter design. It resembles the IIR filter design functions in that it is formulated to design filters in standard band configurations: lowpass, bandpass, highpass, and bandstop.

The statements

```
n = 50;
Wn = 0.4;
b = fir1(n,Wn);
```

create row vector `b` containing the coefficients of the order `n` Hamming-windowed filter. This is a lowpass, linear phase FIR filter with cutoff frequency `Wn`. `Wn` is a number between 0 and 1, where 1 corresponds to the Nyquist frequency, half the sampling frequency. (Unlike other methods, here `Wn` corresponds to the 6 dB point.) For a highpass filter, simply append `'high'` to the function's parameter list. For a bandpass or bandstop filter, specify `Wn` as a two-element vector containing the passband edge frequencies. Append `'stop'` for the bandstop configuration.

`b = fir1(n,Wn,window)` uses the window specified in column vector `window` for the design. The vector `window` must be `n+1` elements long. If you do not specify a window, `fir1` applies a Hamming window.

Kaiser Window Order Estimation

The `kaiserord` function estimates the filter order, cutoff frequency, and Kaiser window beta parameter needed to meet a given set of specifications. Given a vector of frequency band edges and a corresponding vector of magnitudes, as well as maximum allowable ripple, `kaiserord` returns appropriate input parameters for the `fir1` function.

Multiband FIR Filter Design: fir2

The `fir2` function also designs windowed FIR filters, but with an arbitrarily shaped piecewise linear frequency response. This is in contrast to `fir1`, which only designs filters in standard lowpass, highpass, bandpass, and bandstop configurations.

The commands

```
n = 50;
f = [0 .4 .5 1];
m = [1  1  0 0];
b = fir2(n,f,m);
```

return row vector `b` containing the `n+1` coefficients of the order `n` FIR filter whose frequency-magnitude characteristics match those given by vectors `f` and `m`. `f` is a vector of frequency points ranging from 0 to 1, where 1 represents the Nyquist frequency. `m` is a vector containing the specified magnitude response at the points specified in `f`. (The IIR counterpart of this function is `yulewalk`, which also designs filters based on arbitrary piecewise linear magnitude responses. See “IIR Filter Design” on page 2-5 for details.)

Multiband FIR Filter Design with Transition Bands

The `firls` and `firpm` functions provide a more general means of specifying the ideal specified filter than the `fir1` and `fir2` functions. These functions design Hilbert transformers, differentiators, and other filters with odd symmetric coefficients (type III and type IV linear phase). They also let you include transition or “don't care” regions in which the error is not minimized, and perform band dependent weighting of the minimization.

The `firls` function is an extension of the `fir1` and `fir2` functions in that it minimizes the integral of the square of the error between the specified frequency response and the actual frequency response.

The `firpm` function implements the Parks-McClellan algorithm, which uses the Remez exchange algorithm and Chebyshev approximation theory to design filters with optimal fits between the specified and actual frequency responses. The filters are optimal in the sense that they minimize the maximum error between the specified frequency response and the actual frequency response; they are sometimes called *minimax* filters. Filters designed in this way exhibit an equiripple behavior in their frequency response, and hence are also known as *equiripple* filters. The Parks-McClellan FIR filter design algorithm is perhaps the most popular and widely used FIR filter design methodology.

The syntax for `firls` and `firpm` is the same; the only difference is their minimization schemes. The next example shows how filters designed with `firls` and `firpm` reflect these different schemes.

Basic Configurations

The default mode of operation of `firls` and `firpm` is to design type I or type II linear phase filters, depending on whether the order you want is even or odd, respectively. A

lowpass example with approximate amplitude 1 from 0 to 0.4 Hz, and approximate amplitude 0 from 0.5 to 1.0 Hz is

```
n = 20;                     % Filter order
f = [0 0.4 0.5 1];          % Frequency band edges
a = [1  1  0 0];            % Amplitudes
b = firpm(n,f,a);
```

From 0.4 to 0.5 Hz, `firpm` performs no error minimization; this is a transition band or “don't care” region. A transition band minimizes the error more in the bands that you do care about, at the expense of a slower transition rate. In this way, these types of filters have an inherent trade-off similar to FIR design by windowing.

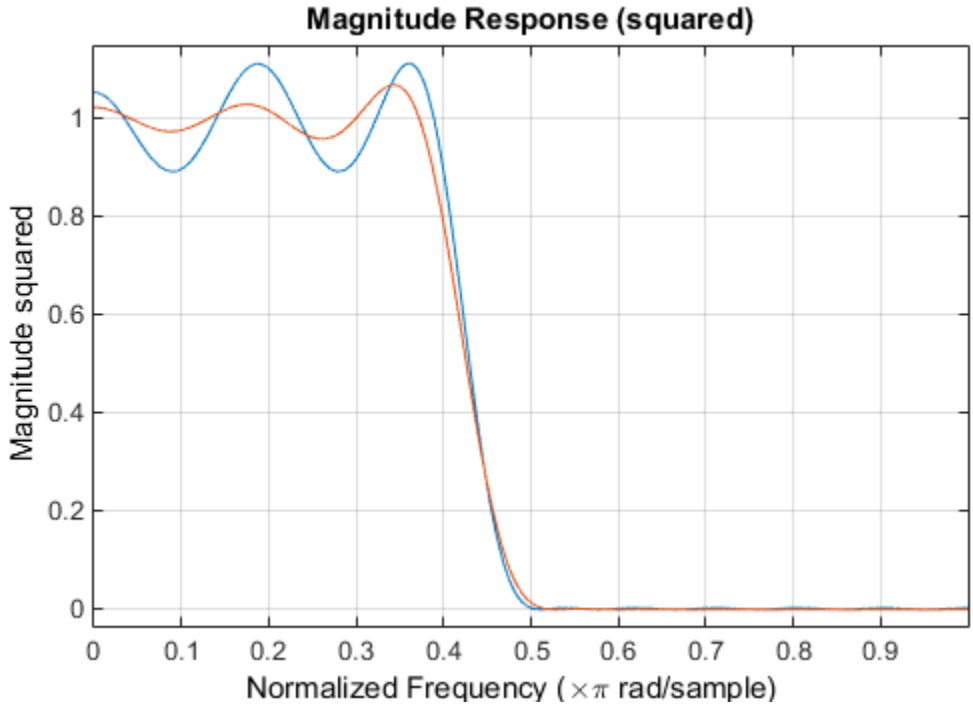
To compare least squares to equiripple filter design, use `firls` to create a similar filter. Type

```
bb = firls(n,f,a);
```

and compare their frequency responses using FVTool:

```
fvtool(b,1,bb,1)
```

Note that the *y*-axis shown in the figure below is in Magnitude Squared. You can set this by right-clicking on the axis label and selecting **Magnitude Squared** from the menu.



The filter designed with `firpm` exhibits equiripple behavior. Also note that the `firls` filter has a better response over most of the passband and stopband, but at the band edges (`f = 0.4` and `f = 0.5`), the response is further away from the ideal than the `firpm` filter. This shows that the `firpm` filter's *maximum* error over the passband and stopband is smaller and, in fact, it is the smallest possible for this band edge configuration and filter length.

Think of frequency bands as lines over short frequency intervals. `firpm` and `firls` use this scheme to represent any piecewise linear frequency-response function with any transition bands. `firls` and `firpm` design lowpass, highpass, bandpass, and bandstop filters; a bandpass example is

```
f = [0 0.3  0.4  0.7  0.8  1];   % Band edges in pairs
a = [0  0    1    1    0   0];   % Bandpass filter amplitude
```

Technically, these `f` and `a` vectors define five bands:

- Two stopbands, from 0.0 to 0.3 and from 0.8 to 1.0
- A passband from 0.4 to 0.7
- Two transition bands, from 0.3 to 0.4 and from 0.7 to 0.8

Example highpass and bandstop filters are

```
f = [0 0.7  0.8  1];            % Band edges in pairs
a = [0  0    1   1];            % Highpass filter amplitude
f = [0 0.3  0.4  0.5  0.8  1];  % Band edges in pairs
a = [1  1    0    0    1   1];  % Bandstop filter amplitude
```

An example multiband bandpass filter is

```
f = [0 0.1 0.15 0.25 0.3 0.4 0.45 0.55 0.6 0.7 0.75 0.85 0.9 1];
a = [1  1   0    0    1   1   0    0    1   1   0    0    1  1];
```

Another possibility is a filter that has as a transition region the line connecting the passband with the stopband; this can help control “runaway” magnitude response in wide transition regions:

```
f = [0 0.4 0.42 0.48 0.5  1];
a = [1 1 0.8 0.2 0 0];   % Passband, linear transition,
                         %  stopband
```

The Weight Vector

Both `firls` and `firpm` allow you to place more or less emphasis on minimizing the error in certain frequency bands relative to others. To do this, specify a weight vector following the frequency and amplitude vectors. An example lowpass equiripple filter with 10 times less ripple in the stopband than the passband is

```
n = 20;              % Filter order
f = [0 0.4 0.5 1];   % Frequency band edges
a = [1  1   0  0];   % Amplitudes
w = [1 10];          % Weight vector
b = firpm(n,f,a,w);
```

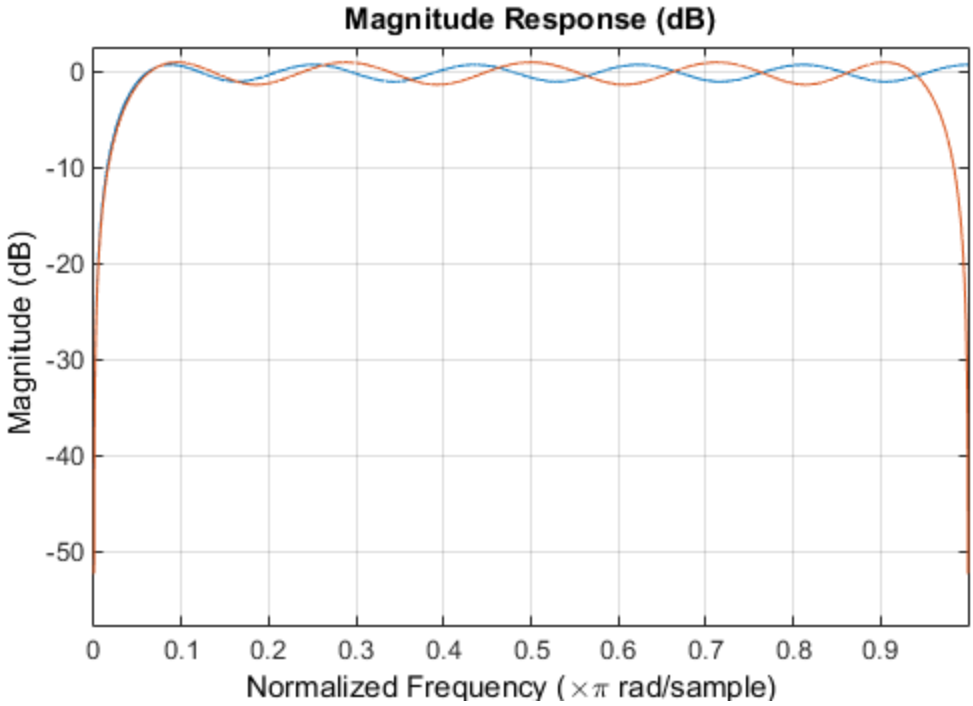
A legal weight vector is always half the length of the `f` and `a` vectors; there must be exactly one weight per band.

Anti-Symmetric Filters / Hilbert Transformers

When called with a trailing `'h'` or `'Hilbert'` option, `firpm` and `firls` design FIR filters with odd symmetry, that is, type III (for even order) or type IV (for odd order) linear

phase filters. An ideal Hilbert transformer has this anti-symmetry property and an amplitude of 1 across the entire frequency range. Try the following approximate Hilbert transformers and plot them using FVTool:

```
b = firpm(21,[0.05 1],[1 1],'h');        % Highpass Hilbert
bb = firpm(20,[0.05 0.95],[1 1],'h');    % Bandpass Hilbert
fvtool(b,1,bb,1)
```



You can find the delayed Hilbert transform of a signal `x` by passing it through these filters.

```
fs = 1000;            % Sampling frequency
t = (0:1/fs:2)';      % Two second time vector
x = sin(2*pi*300*t);  % 300 Hz sine wave example signal
xh = filter(bb,1,x);  % Hilbert transform of x
```

The analytic signal corresponding to `x` is the complex signal that has `x` as its real part and the Hilbert transform of `x` as its imaginary part. For this FIR method (an alternative to the `hilbert` function), you must delay `x` by half the filter order to create the analytic signal:

```
xd = [zeros(10,1); x(1:length(x)-10)];        % Delay 10 samples
xa = xd + j*xh;                              % Analytic signal
```

This method does not work directly for filters of odd order, which require a noninteger delay. In this case, the `hilbert` function, described in “Hilbert Transform” on page 7-77, estimates the analytic signal. Alternatively, use the `resample` function to delay the signal by a noninteger number of samples.

Differentiators

Differentiation of a signal in the time domain is equivalent to multiplication of the signal's Fourier transform by an imaginary ramp function. That is, to differentiate a signal, pass it through a filter that has a response $H(\omega) = j\omega$. Approximate the ideal differentiator (with a delay) using `firpm` or `firls` with a `'d'` or `'differentiator'` option:

```
b = firpm(21,[0 1],[0 pi],'d');
```

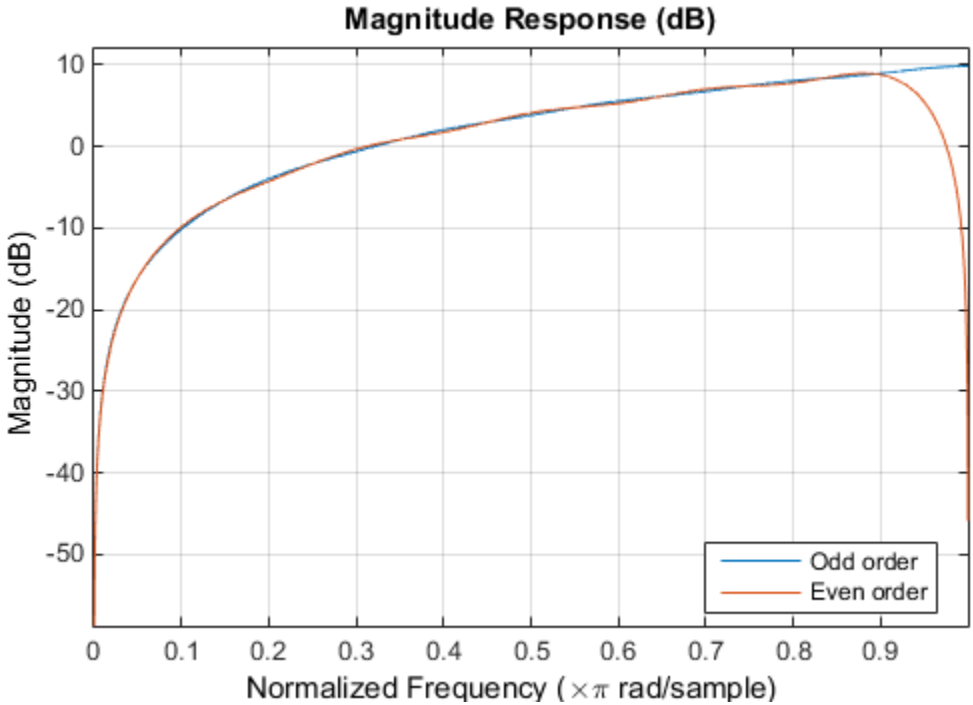
For a type III filter, the differentiation band should stop short of the Nyquist frequency, and the amplitude vector must reflect that change to ensure the correct slope:

```
bb = firpm(20,[0 0.9],[0 0.9*pi],'d');
```

In the `'d'` mode, `firpm` weights the error by $1/\omega$ in nonzero amplitude bands to minimize the maximum *relative* error. `firls` weights the error by $(1/\omega)^2$ in nonzero amplitude bands in the `'d'` mode.

The following plots show the magnitude responses for the differentiators above.

```
fvtool(b,1,bb,1)
legend('Odd order','Even order','Location','best')
```



Constrained Least Squares FIR Filter Design

The Constrained Least Squares (CLS) FIR filter design functions implement a technique that enables you to design FIR filters without explicitly defining the transition bands for the magnitude response. The ability to omit the specification of transition bands is useful in several situations. For example, it may not be clear where a rigidly defined transition band should appear if noise and signal information appear together in the same frequency band. Similarly, it may make sense to omit the specification of transition bands if they appear only to control the results of Gibbs phenomena that appear in the filter's response. See Selesnick, Lang, and Burrus [2] for discussion of this method.

Instead of defining passbands, stopbands, and transition regions, the CLS method accepts a cutoff frequency (for the highpass, lowpass, bandpass, or bandstop cases), or passband and stopband edges (for multiband cases), for the response you specify. In this way, the CLS method defines transition regions implicitly, rather than explicitly.

The key feature of the CLS method is that it enables you to define upper and lower thresholds that contain the maximum allowable ripple in the magnitude response. Given this constraint, the technique applies the least square error minimization technique over the frequency range of the filter's response, instead of over specific bands. The error minimization includes any areas of discontinuity in the ideal, "brick wall" response. An additional benefit is that the technique enables you to specify arbitrarily small peaks resulting from the Gibbs phenomenon.

There are two toolbox functions that implement this design technique.

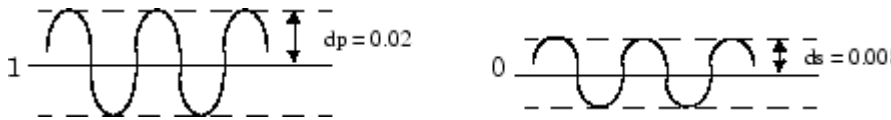
Description	Function
Constrained least square multiband FIR filter design	`fircls`
Constrained least square filter design for lowpass and highpass linear phase filters	`fircls1`

For details on the calling syntax for these functions, see their reference descriptions in the Function Reference.

Basic Lowpass and Highpass CLS Filter Design

The most basic of the CLS design functions, `fircls1`, uses this technique to design lowpass and highpass FIR filters. As an example, consider designing a filter with order 61 impulse response and cutoff frequency of 0.3 (normalized). Further, define the upper and lower bounds that constrain the design process as:

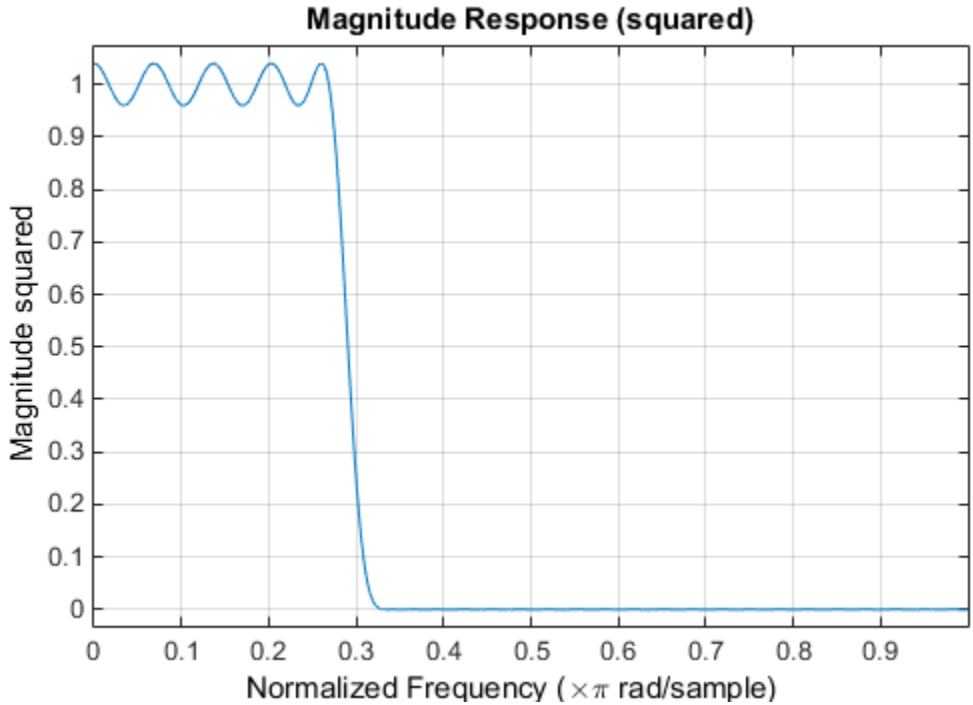
- Maximum passband deviation from 1 (passband ripple) of 0.02.
- Maximum stopband deviation from 0 (stopband ripple) of 0.008.



To approach this design problem using `fircls1`, use the following commands:

```
n = 61;
wo = 0.3;
dp = 0.02;
ds = 0.008;
h = fircls1(n,wo,dp,ds);
fvtool(h,1)
```

Note that the *y*-axis shown below is in Magnitude Squared. You can set this by right-clicking on the axis label and selecting **Magnitude Squared** from the menu.



Multiband CLS Filter Design

`fircls` uses the same technique to design FIR filters with a specified piecewise constant magnitude response. In this case, you can specify a vector of band edges and a corresponding vector of band amplitudes. In addition, you can specify the maximum amount of ripple for each band.

For example, assume the specifications for a filter call for:

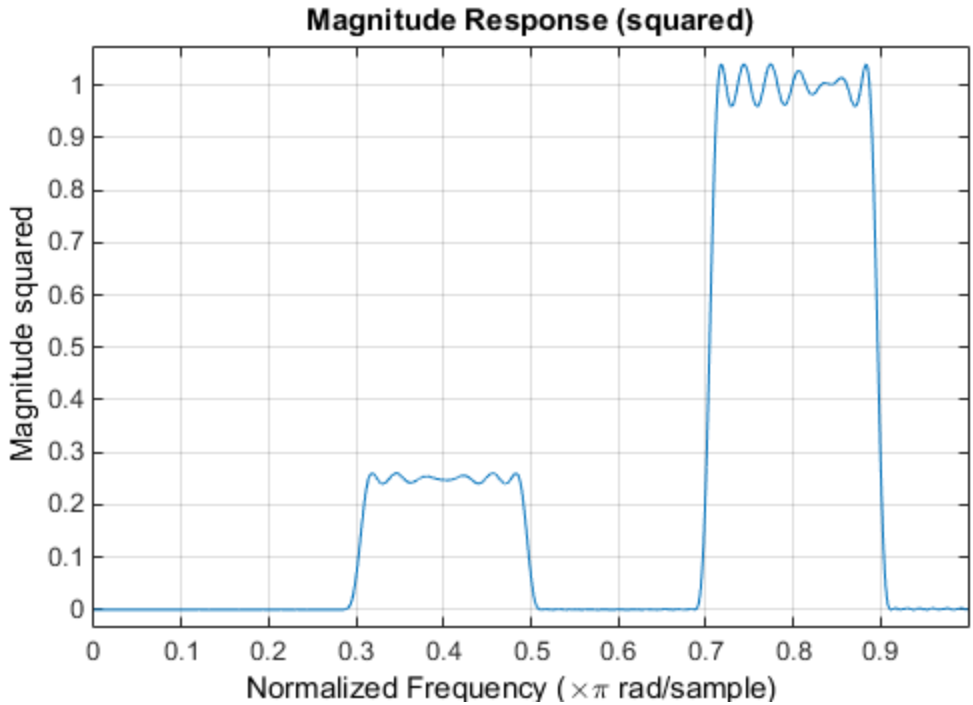
- From 0 to 0.3 (normalized): amplitude 0, upper bound 0.005, lower bound –0.005
- From 0.3 to 0.5: amplitude 0.5, upper bound 0.51, lower bound 0.49
- From 0.5 to 0.7: amplitude 0, upper bound 0.03, lower bound –0.03
- From 0.7 to 0.9: amplitude 1, upper bound 1.02, lower bound 0.98

- From 0.9 to 1: amplitude 0, upper bound 0.05, lower bound –0.05

Design a CLS filter with impulse response order 129 that meets these specifications:

```
n = 129;
f = [0 0.3 0.5 0.7 0.9 1];
a = [0 0.5 0 1 0];
up = [0.005 0.51 0.03 1.02 0.05];
lo = [-0.005 0.49 -0.03 0.98 -0.05];
h = fircls(n,f,a,up,lo);
fvtool(h,1)
```

Note that the *y*-axis shown below is in Magnitude Squared. You can set this by right-clicking on the axis label and selecting **Magnitude Squared** from the menu.



Weighted CLS Filter Design

Weighted CLS filter design lets you design lowpass or highpass FIR filters with relative weighting of the error minimization in each band. The `fircls1` function enables you to

specify the passband and stopband edges for the least squares weighting function, as well as a constant k that specifies the ratio of the stopband to passband weighting.

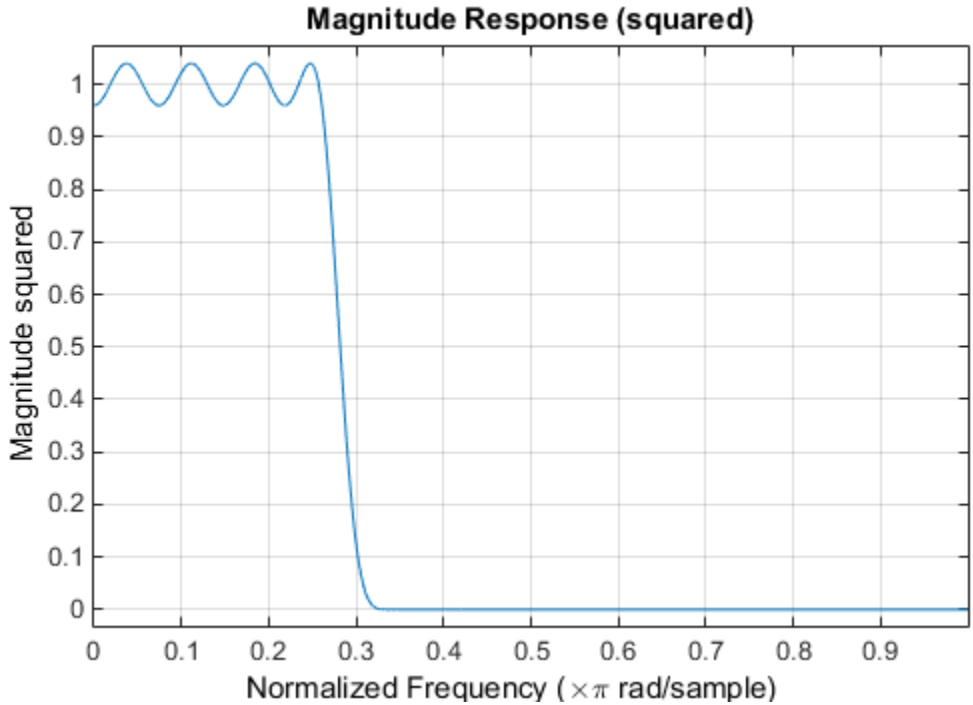
For example, consider specifications that call for an FIR filter with impulse response order of 55 and cutoff frequency of 0.3 (normalized). Also assume maximum allowable passband ripple of 0.02 and maximum allowable stopband ripple of 0.004. In addition, add weighting requirements:

- Passband edge for the weight function of 0.28 (normalized)
- Stopband edge for the weight function of 0.32
- Weight error minimization 10 times as much in the stopband as in the passband

To approach this using `fircls1`, type

```
n = 55;
wo = 0.3;
dp = 0.02;
ds = 0.004;
wp = 0.28;
ws = 0.32;
k = 10;
h = fircls1(n,wo,dp,ds,wp,ws,k);
fvtool(h,1)
```

Note that the *y*-axis shown below is in Magnitude Squared. You can set this by right-clicking on the axis label and selecting **Magnitude Squared** from the menu.



Arbitrary-Response Filter Design

The `cfirpm` filter design function provides a tool for designing FIR filters with arbitrary complex responses. It differs from the other filter design functions in how the frequency response of the filter is specified: it accepts the name of a function which returns the filter response calculated over a grid of frequencies. This capability makes `cfirpm` a highly versatile and powerful technique for filter design.

This design technique may be used to produce nonlinear-phase FIR filters, asymmetric frequency-response filters (with complex coefficients), or more symmetric filters with custom frequency responses.

The design algorithm optimizes the Chebyshev (or minimax) error using an extended Remez-exchange algorithm for an initial estimate. If this exchange method fails to obtain the optimal filter, the algorithm switches to an ascent-descent algorithm that takes over to finish the convergence to the optimal solution.

Multiband Filter Design

Consider a multiband filter with the following special frequency-domain characteristics.

Band	Amplitude	Optimization Weighting
[–1 –0.5]	[5 1]	1
[–0.4 +0.3]	[2 2]	10
[+0.4 +0.8]	[2 1]	5

A linear-phase multiband filter may be designed using the predefined frequency-response function `multiband`, as follows:

```
b = cfirpm(38, [-1 -0.5 -0.4 0.3 0.4 0.8], ...
               {'multiband', [5 1 2 2 2 1]}, [1 10 5]);
```

For the specific case of a multiband filter, we can use a shorthand filter design notation similar to the syntax for `firpm`:

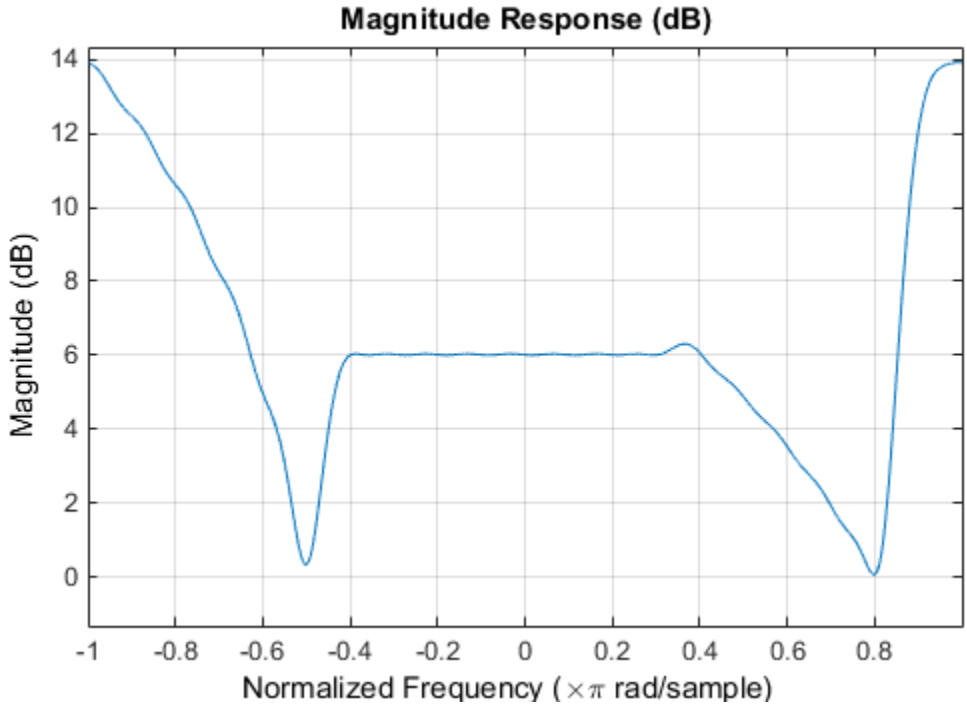
```
b = cfirpm(38,[-1 -0.5 -0.4 0.3 0.4 0.8], ...
              [5 1 2 2 2 1], [1 10 5]);
```

As with `firpm`, a vector of band edges is passed to `cfirpm`. This vector defines the frequency bands over which optimization is performed; note that there are two transition bands, from –0.5 to –0.4 and from 0.3 to 0.4.

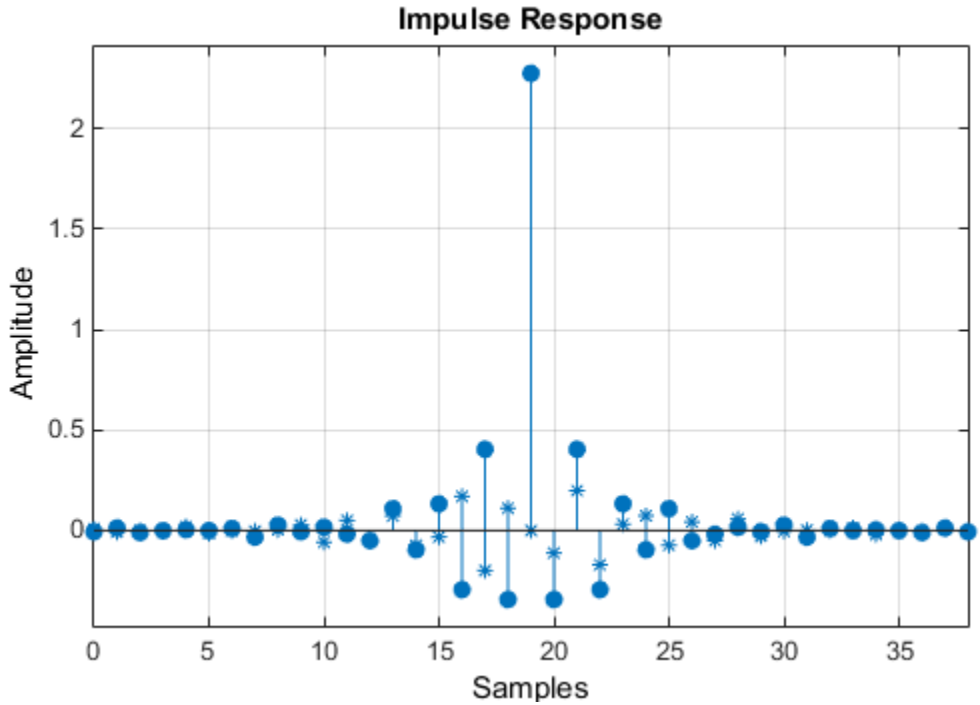
In either case, the frequency response is obtained and plotted using linear scale in FVTool:

```
fvtool(b,1)
```

Note that the range of data shown below is `(-pi,pi)`.



The filter response for this multiband filter is complex, which is expected because of the asymmetry in the frequency domain. The impulse response, which you can select from the FVTool toolbar, is shown below.



Filter Design with Reduced Delay

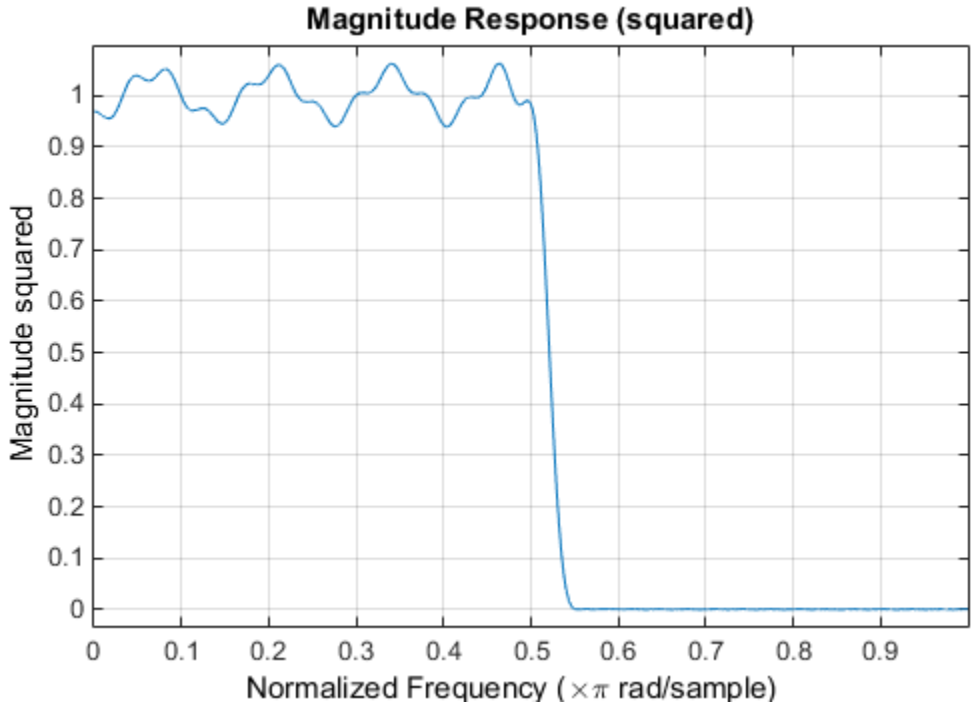
Consider the design of a 62-tap lowpass filter with a half-Nyquist cutoff. If we specify a negative offset value to the `lowpass` filter design function, the group delay offset for the design is significantly less than that obtained for a standard linear-phase design. This filter design may be computed as follows:

```
b = cfirpm(61,[0 0.5 0.55 1],{'lowpass',-16});
```

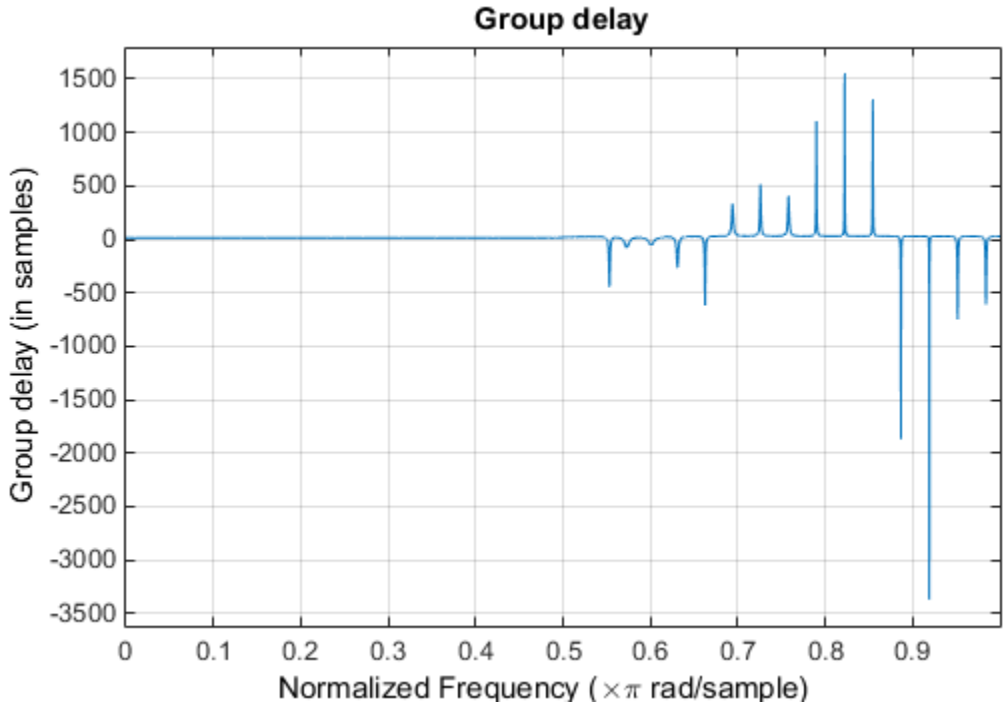
The resulting magnitude response is

```
fvtool(b,1)
```

The *y*-axis is in Magnitude Squared, which you can set by right-clicking on the axis label and selecting **Magnitude Squared** from the menu.



The group delay of the filter reveals that the offset has been reduced from `N/2` to `N/2-16` (i.e., from `30.5` to `14.5`). Now, however, the group delay is no longer flat in the passband region. To create this plot, click the **Group Delay Response** button on the toolbar.



If we compare this nonlinear-phase filter to a linear-phase filter that has exactly 14.5 samples of group delay, the resulting filter is of order 2*14.5, or 29. Using `b = cfirpm(29,[0 0.5 0.55 1],'lowpass')`, the passband and stopband ripple is much greater for the order 29 filter. These comparisons can assist you in deciding which filter is more appropriate for a specific application.

Special Topics in IIR Filter Design

In this section...
“Classic IIR Filter Design” on page 2-42
“Analog Prototype Design” on page 2-42
“Frequency Transformation” on page 2-43
“Filter Discretization” on page 2-45

Classic IIR Filter Design

The classic IIR filter design technique includes the following steps.

1. Find an analog lowpass filter with cutoff frequency of 1 and translate this prototype filter to the specified band configuration
2. Transform the filter to the digital domain.
3. Discretize the filter.

The toolbox provides functions for each of these steps.

Design Task	Available functions
Analog lowpass prototype	`buttap`, `cheb1ap`, `besselap`, `ellipap`, `cheb2ap`
Frequency transformation	`lp2lp`, `lp2hp`, `lp2bp`, `lp2bs`
Discretization	`bilinear`, `impinvar`

Alternatively, the `butter`, `cheby1`, `cheb2ord`, `ellip`, and `besself` functions perform all steps of the filter design and the `buttord`, `cheb1ord`, `cheb2ord`, and `ellipord` functions provide minimum order computation for IIR filters. These functions are sufficient for many design problems, and the lower level functions are generally not needed. But if you do have an application where you need to transform the band edges of an analog filter, or discretize a rational transfer function, this section describes the tools with which to do so.

Analog Prototype Design

This toolbox provides a number of functions to create lowpass analog prototype filters with cutoff frequency of 1, the first step in the classical approach to IIR filter design.

The table below summarizes the analog prototype design functions for each supported filter type; plots for each type are shown in “IIR Filter Design” on page 2-5.

Filter Type	Analog Prototype Function
Bessel	`[z,p,k] = besselap(n)`
Butterworth	`[z,p,k] = buttap(n)`
Chebyshev Type I	`[z,p,k] = cheb1ap(n,Rp)`
Chebyshev Type II	`[z,p,k] = cheb2ap(n,Rs)`
Elliptic	`[z,p,k] = ellipap(n,Rp,Rs)`

Frequency Transformation

The second step in the analog prototyping design technique is the frequency transformation of a lowpass prototype. The toolbox provides a set of functions to transform analog lowpass prototypes (with cutoff frequency of 1 rad/s) into bandpass, highpass, bandstop, and lowpass filters with the specified cutoff frequency.

Frequency Transformation	Transformation Function
Lowpass to lowpass $s' = s / \omega_0$	`[numt,dent] = lp2lp (num,den,Wo)` `[At,Bt,Ct,Dt] = lp2lp (A,B,C,D,Wo)`
Lowpass to highpass $s' = \frac{\omega_0}{s}$	`[numt,dent] = lp2hp (num,den,Wo)` `[At,Bt,Ct,Dt] = lp2hp (A,B,C,D,Wo)`
Lowpass to bandpass $s' = \frac{\omega_0}{B_\omega} \frac{(s/\omega_0)^2 + 1}{s/\omega_0}$	`[numt,dent] = lp2bp (num,den,Wo,Bw)` `[At,Bt,Ct,Dt] = lp2bp (A,B,C,D,Wo,Bw)`
Lowpass to bandstop $s' = \frac{B_\omega}{\omega_0} \frac{s/\omega_0}{(s/\omega_0)^2 + 1}$	`[numt,dent] = lp2bs (num,den,Wo,Bw)` `[At,Bt,Ct,Dt] = lp2bs( A,B,C,D,Wo,Bw)`

As shown, all of the frequency transformation functions can accept two linear system models: transfer function and state-space form. For the bandpass and bandstop cases

$$\omega_0 = \sqrt{\omega_1 \omega_2}$$

and

$$B_\omega = \omega_2 - \omega_1$$

where ω_1 is the lower band edge and ω_2 is the upper band edge.

The frequency transformation functions perform frequency variable substitution. In the case of `lp2bp` and `lp2bs`, this is a second-order substitution, so the output filter is twice the order of the input. For `lp2lp` and `lp2hp`, the output filter is the same order as the input.

To begin designing an order 10 bandpass Chebyshev Type I filter with a value of 3 dB for passband ripple, enter

```
[z,p,k] = cheb1ap(10,3);
```

Outputs `z`, `p`, and `k` contain the zeros, poles, and gain of a lowpass analog filter with cutoff frequency Ω_c equal to 1 rad/s. Use the function to transform this lowpass prototype to a bandpass analog filter with band edges $\Omega_1 = \pi/5$ and $\Omega_2 = \pi$. First, convert the filter to state-space form so the `lp2bp` function can accept it:

```
[A,B,C,D] = zp2ss(z,p,k);   % Convert to state-space form.
```

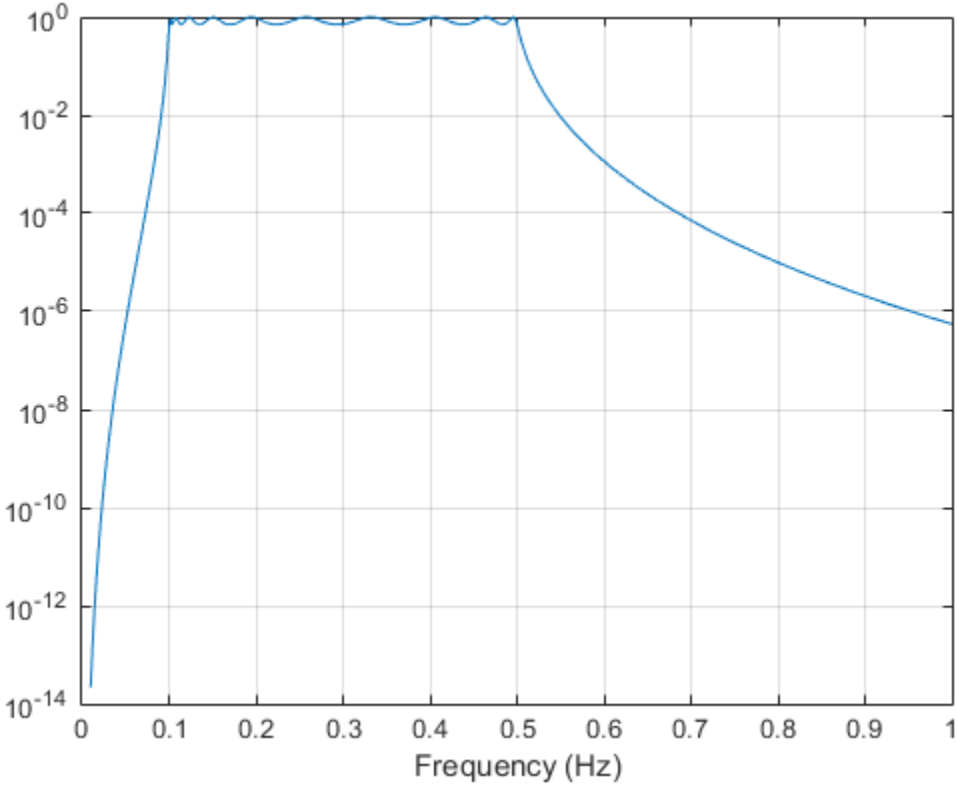
Now, find the bandwidth and center frequency, and call `lp2bp`:

```
u1 = 0.1*2*pi;
u2 = 0.5*2*pi;   % In radians per second
Bw = u2-u1;
Wo = sqrt(u1*u2);
[At,Bt,Ct,Dt] = lp2bp(A,B,C,D,Wo,Bw);
```

Finally, calculate the frequency response and plot its magnitude:

```
[b,a] = ss2tf(At,Bt,Ct,Dt);        % Convert to TF form
w = linspace(0.01,1,500)*2*pi;     % Generate frequency vector
h = freqs(b,a,w);                  % Compute frequency response
semilogy(w/2/pi,abs(h))            % Plot log magnitude vs. freq
```

```
xlabel('Frequency (Hz)')
grid
```



Filter Discretization

The third step in the analog prototyping technique is the transformation of the filter to the discrete-time domain. The toolbox provides two methods for this: the impulse invariant and bilinear transformations. The filter design functions `butter`, `cheby1`, `cheby2`, and `ellip` use the bilinear transformation for discretization in this step.

Analog to Digital Transformation	Transformation Function
Impulse invariance	`[numd,dend] = impinvar (num,den,fs)`

Bilinear transform	`[zd,pd,kd] = bilinear (z,p,k,fs,Fp)`
	`[numd,dend] = bilinear (num,den,fs,Fp)`
	`[Ad,Bd,Cd,Dd] = bilinear (At,Bt,Ct,Dt,fs,Fp)`

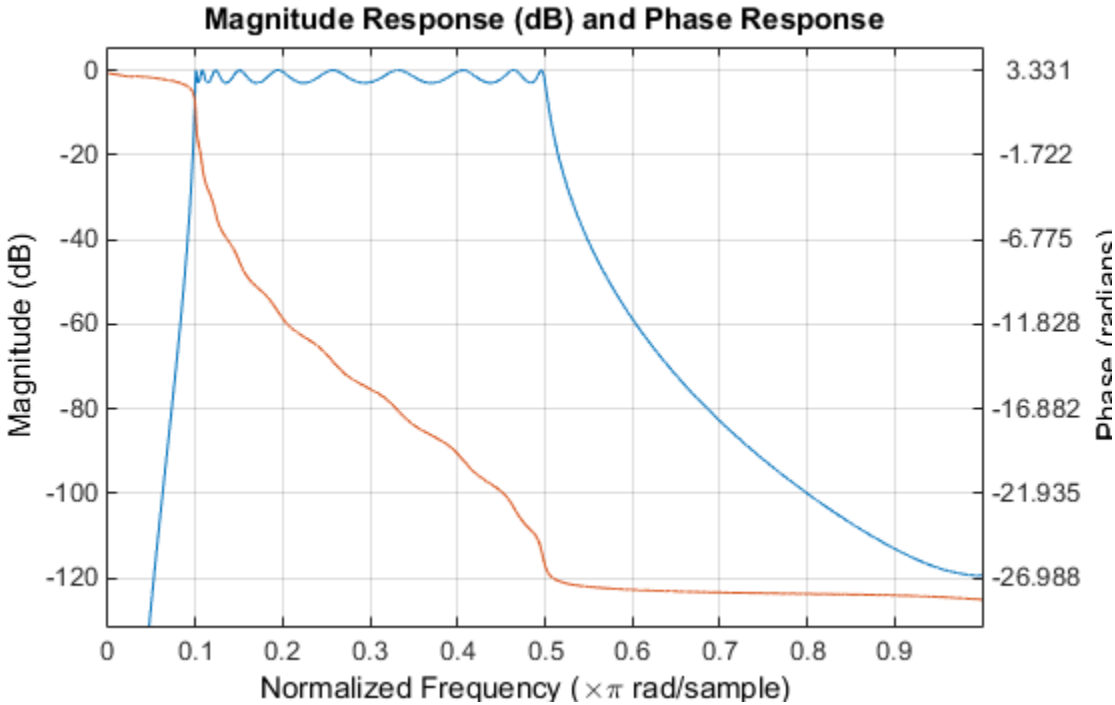
Impulse Invariance

The toolbox function `impinvar` creates a digital filter whose impulse response is the samples of the continuous impulse response of an analog filter. This function works only on filters in transfer function form. For best results, the analog filter should have negligible frequency content above half the sampling frequency, because such high-frequency content is aliased into lower bands upon sampling. Impulse invariance works for some lowpass and bandpass filters, but is not appropriate for highpass and bandstop filters.

Design a Chebyshev Type I filter and plot its frequency and phase response using FVTool:

```
[bz,az] = impinvar(b,a,2);
fvtool(bz,az)
```

Click the **Magnitude and Phase Response** toolbar button.



Impulse invariance retains the cutoff frequencies of 0.1 Hz and 0.5 Hz.

Bilinear Transformation

The bilinear transformation is a nonlinear mapping of the continuous domain to the discrete domain; it maps the *s*-plane into the *z*-plane by

$$H(z) = H(s)\Big|_{s=k\frac{z-1}{z+1}}$$

Bilinear transformation maps the $j\Omega$-axis of the continuous domain to the unit circle of the discrete domain according to

$$\omega = 2\tan^{-1}\left(\frac{\Omega}{k}\right)$$

The toolbox function `bilinear` implements this operation, where the frequency warping constant k is equal to twice the sampling frequency (`2*fs`) by default, and equal to

$2\pi f_p / \tan\left(\pi f_p / f_s\right)$ if you give `bilinear` a trailing argument that represents a "match" frequency `Fp`. If a match frequency `Fp` (in hertz) is present, `bilinear` maps the frequency $\Omega = 2\pi f_p$ (in rad/s) to the same frequency in the discrete domain, normalized to the sampling rate: $\omega = 2\pi f_p/f_s$ (in rad/sample).

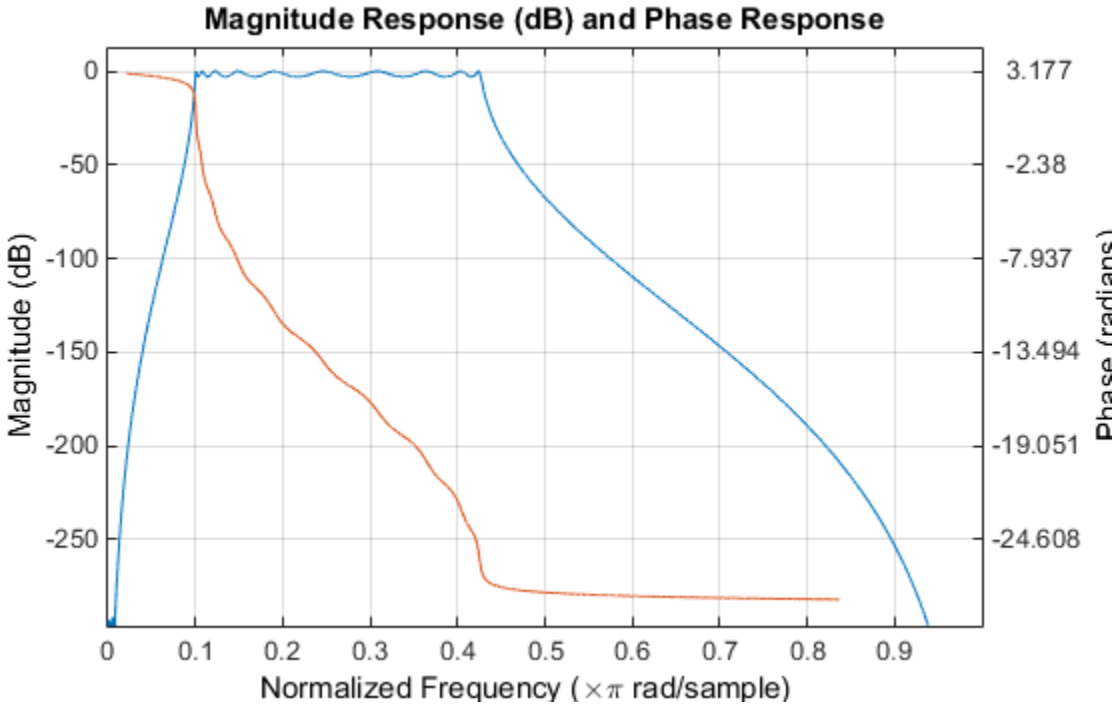
The `bilinear` function can perform this transformation on three different linear system representations: zero-pole-gain, transfer function, and state-space form. Try calling `bilinear` with the state-space matrices that describe the Chebyshev Type I filter from the previous section, using a sampling frequency of 2 Hz, and retaining the lower band edge of 0.1 Hz:

```
[Ad,Bd,Cd,Dd] = bilinear(At,Bt,Ct,Dt,2,0.1);
```

The frequency response of the resulting digital filter is

```
[bz,az] = ss2tf(Ad,Bd,Cd,Dd);       % Convert to TF
fvtool(bz,az)
```

Click the **Magnitude and Phase Response** toolbar button.



The lower band edge is at 0.1 Hz as expected. Notice, however, that the upper band edge is slightly less than 0.5 Hz, although in the analog domain it was exactly 0.5 Hz. This illustrates the nonlinear nature of the bilinear transformation. To counteract this nonlinearity, it is necessary to create analog domain filters with “prewarped” band edges, which map to the correct locations upon bilinear transformation. Here the prewarped frequencies `u1` and `u2` generate `Bw` and `Wo` for the `lp2bp` function:

```
fs = 2;                             % Sampling frequency (hertz)
u1 = 2*fs*tan(0.1*(2*pi/fs)/2);     % Lower band edge (rad/s)
u2 = 2*fs*tan(0.5*(2*pi/fs)/2);     % Upper band edge (rad/s)
Bw = u2 - u1;                       % Bandwidth
Wo = sqrt(u1*u2);                   % Center frequency
[At,Bt,Ct,Dt] = lp2bp(A,B,C,D,Wo,Bw);
```

A digital bandpass filter with correct band edges 0.1 and 0.5 times the Nyquist frequency is

```
[Ad,Bd,Cd,Dd] = bilinear(At,Bt,Ct,Dt,fs);
```

The example bandpass filters from the last two sections could also be created in one statement using the complete IIR design function `cheby1`. For instance, an analog version of the example Chebyshev filter is

```
[b,a] = cheby1(5,3,[0.1 0.5]*2*pi,'s');
```

Note that the band edges are in rad/s for analog filters, whereas for the digital case, frequency is normalized:

```
[bz,az] = cheby1(5,3,[0.1 0.5]);
```

All of the complete design functions call `bilinear` internally. They prewarp the band edges as needed to obtain the correct digital filter.

Filtering Data With Signal Processing Toolbox Software

Lowpass FIR Filter – Window Method

This example shows how to design and implement an FIR filter using two command line functions, `fir1` and `designfilt`, and the interactive **Filter Designer** app.

Create a signal to use in the examples. The signal is a 100 Hz sine wave in additive $N(0,1/4)$ white Gaussian noise. Set the random number generator to the default state for reproducible results.

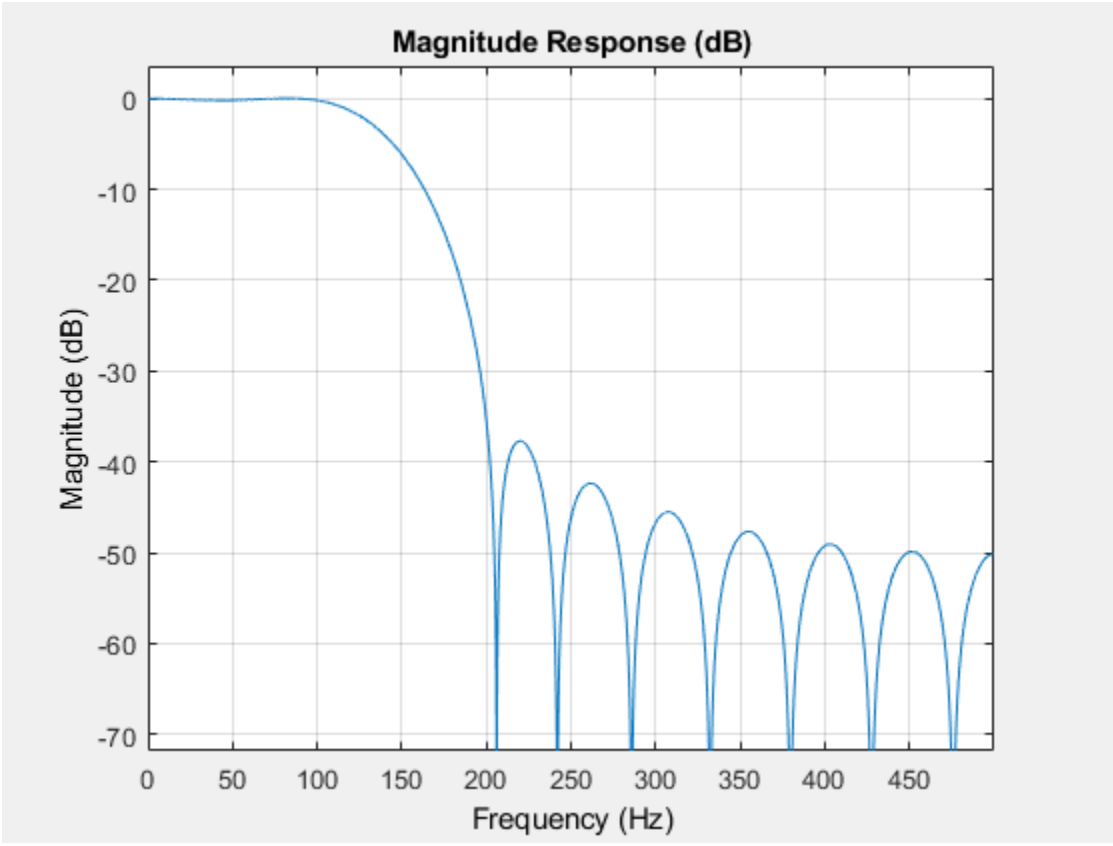
```
rng default

Fs = 1000;
t = linspace(0,1,Fs);
x = cos(2*pi*100*t)+0.5*randn(size(t));
```

The filter design is an FIR lowpass filter with order equal to 20 and a cutoff frequency of 150 Hz. Use a Kaiser window with length one sample greater than the filter order and $\beta = 3$. See `kaiser` for details on the Kaiser window.

Use `fir1` to design the filter. `fir1` requires normalized frequencies in the interval [0,1], where 1 corresponds to π rad/sample. To use `fir1`, you must convert all frequency specifications to normalized frequencies.

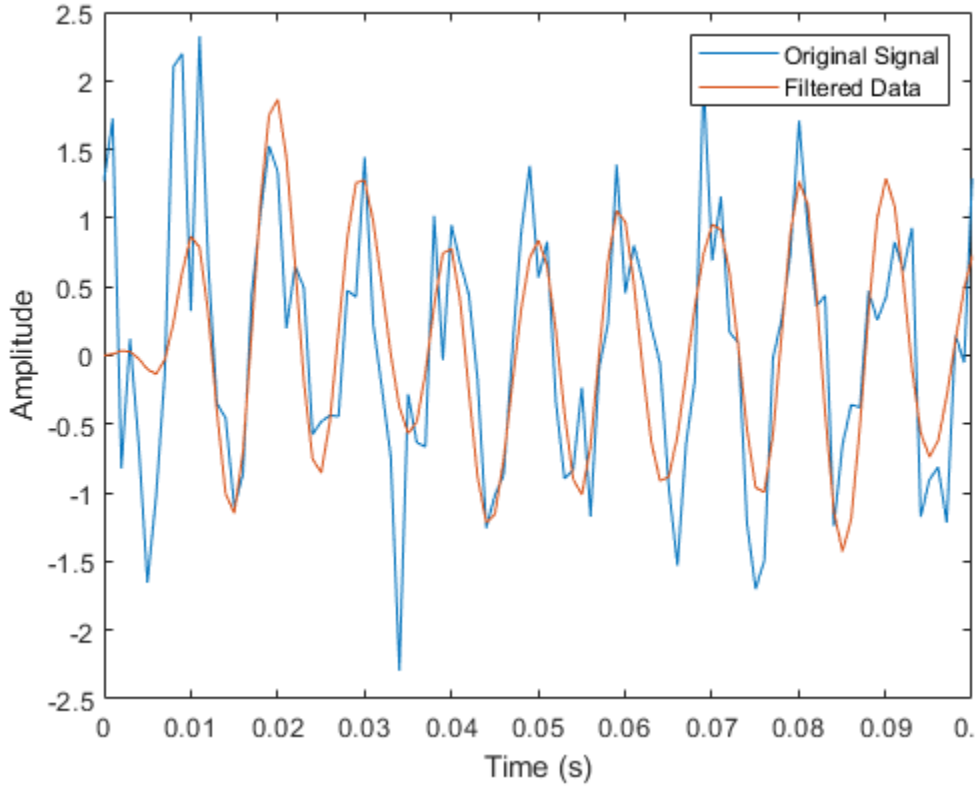
Design the filter and view the filter's magnitude response.

```
fc = 150;
Wn = (2/Fs)*fc;
b = fir1(20,Wn,'low',kaiser(21,3));

fvtool(b,1,'Fs',Fs)
```



Apply the filter to the signal and plot the result for the first ten periods of the 100 Hz sinusoid.

```
y = filter(b,1,x);

plot(t,x,t,y)
xlim([0 0.1])

xlabel('Time (s)')
ylabel('Amplitude')
legend('Original Signal','Filtered Data')
```



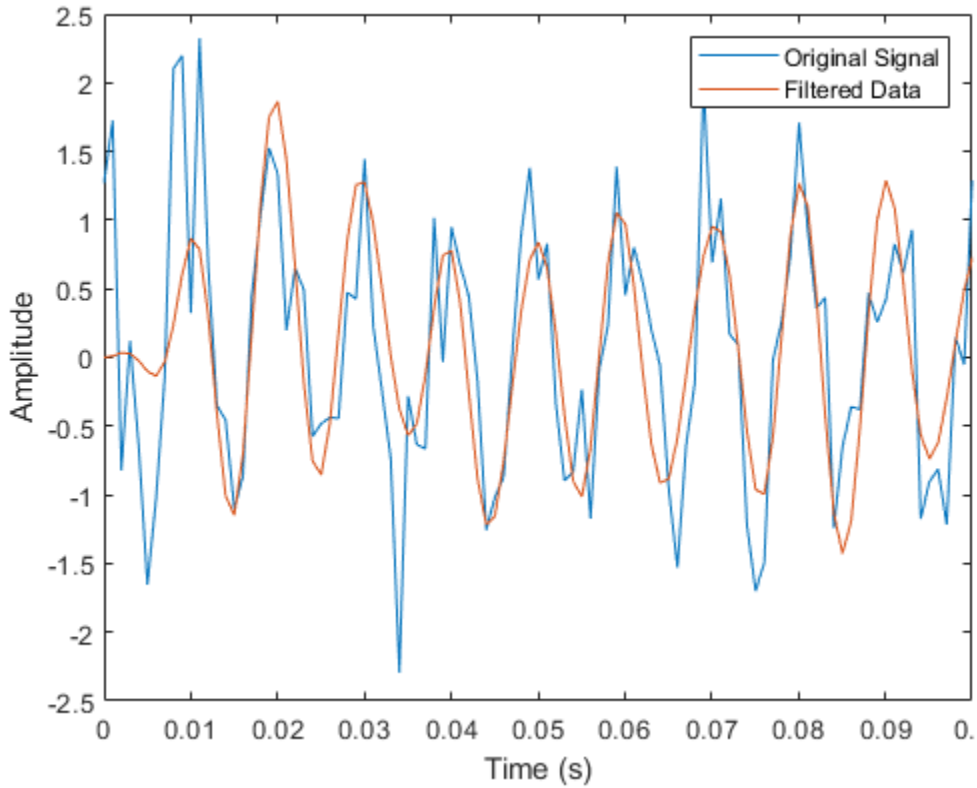
Design the same filter using `designfilt`. Set the filter response to `'lowpassfir'` and input the specifications as `Name,Value` pairs. With `designfilt`, you can specify your filter design in Hz.

```
Fs = 1000;
Hd = designfilt('lowpassfir','FilterOrder',20,'CutoffFrequency',150, ...
       'DesignMethod','window','Window',{@kaiser,3},'SampleRate',Fs);
```

Filter the data and plot the result.

```
y1 = filter(Hd,x);

plot(t,x,t,y1)
xlim([0 0.1])
```

```
xlabel('Time (s)')
ylabel('Amplitude')
legend('Original Signal','Filtered Data')
```

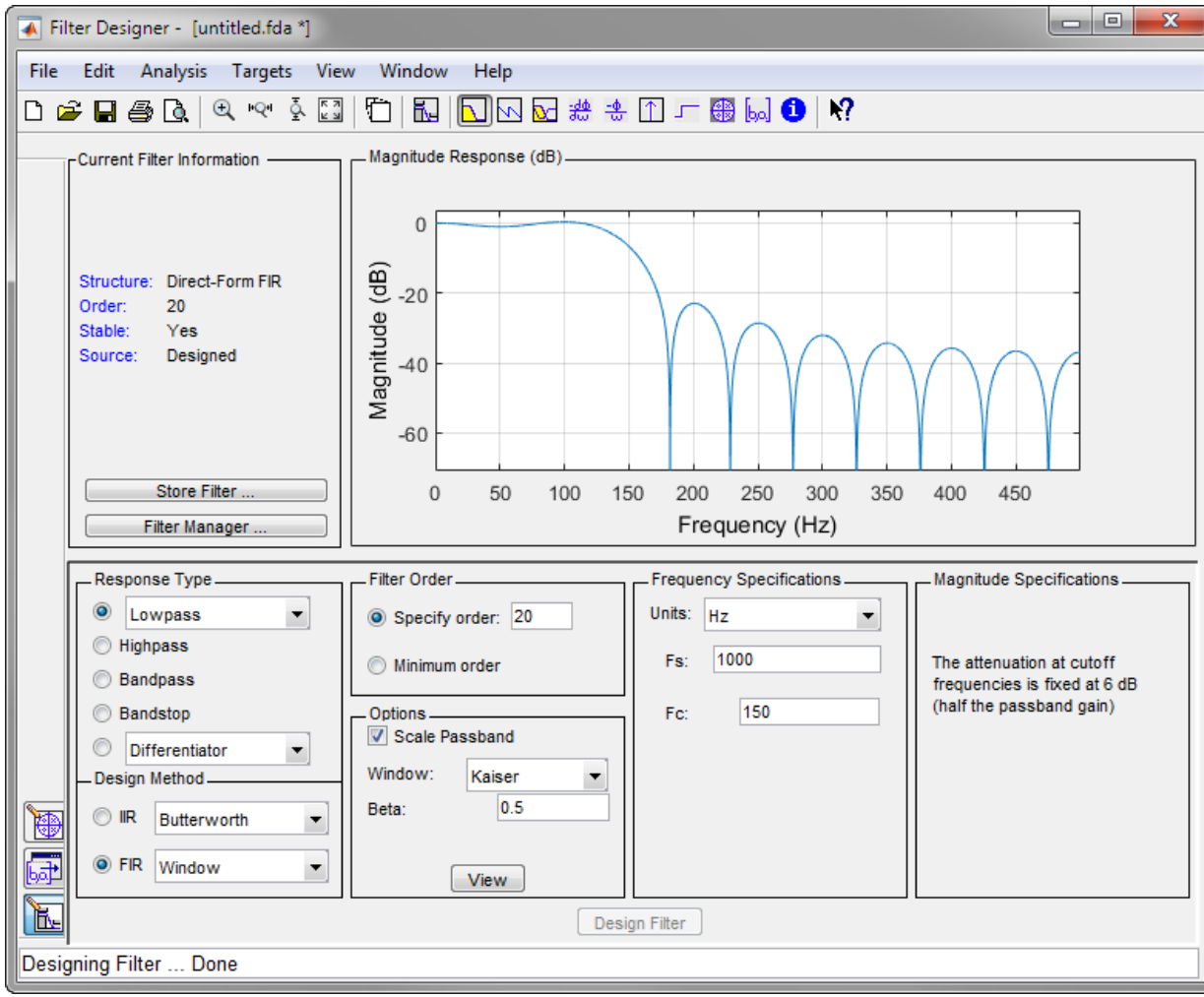


Lowpass FIR Filter with Filter Designer

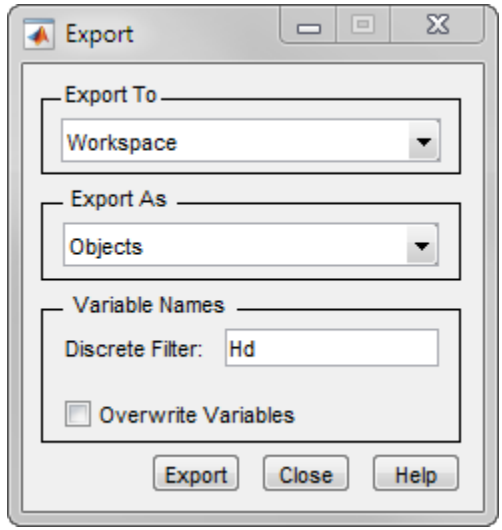
This example shows how to design and implement a lowpass FIR filter using the window method with the interactive **Filter Designer** app.

- Start the app by entering `filterDesigner` at the command line.
- Set the **Response Type** to **Lowpass**.
- Set the **Design Method** to **FIR** and select the **Window** method.

- Under **Filter Order**, select **Specify order**. Set the order to 20.
- Under **Frequency Specifications**, set **Units** to **Hz**, **Fs** to 1000, and **Fc** to 150.

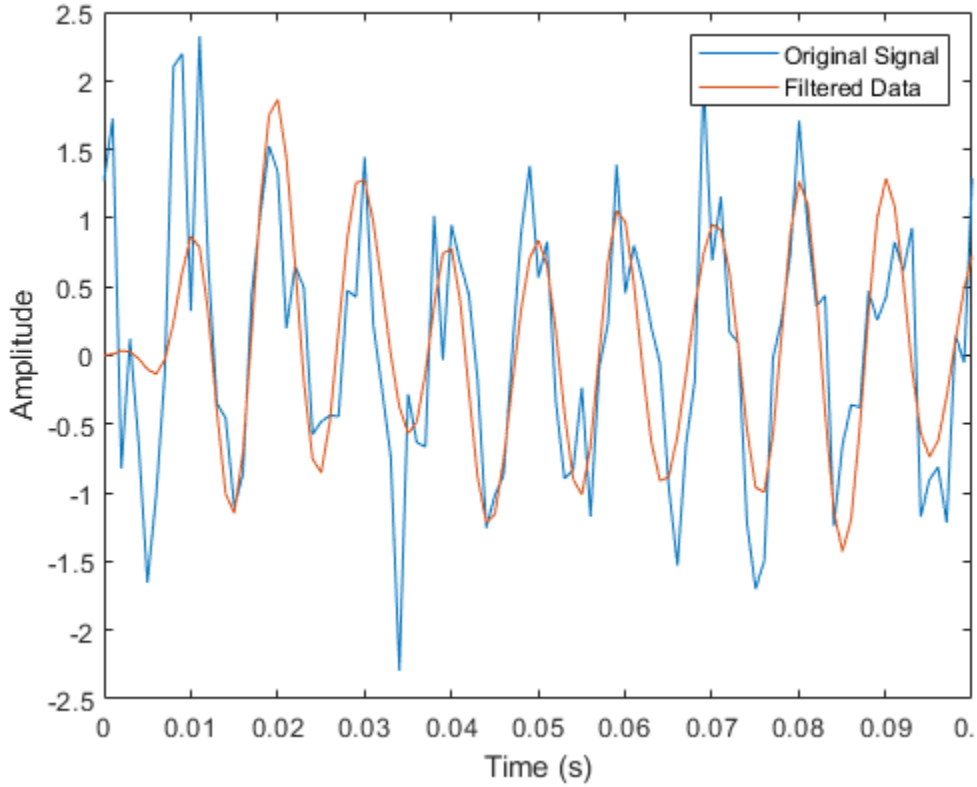


- Click **Design Filter**.
- Select **File > Export...** to export your FIR filter to the MATLAB® workspace as coefficients or a filter object. In this example, export the filter as an object. Specify the variable name as `Hd`.



- Click **Export**.
- Filter the input signal in the command window with the exported filter object. Plot the result for the first ten periods of the 100 Hz sinusoid.

```
y2 = filter(Hd,x);

plot(t,x,t,y2)
xlim([0 0.1])

xlabel('Time (s)')
ylabel('Amplitude')
legend('Original Signal','Filtered Data')
```



- Select **File > Generate MATLAB Code > Filter Design Function** to generate a MATLAB function to create a filter object using your specifications.

You can also use the interactive tool `filterBuilder` to design your filter.

Bandpass Filters – Minimum-Order FIR and IIR Systems

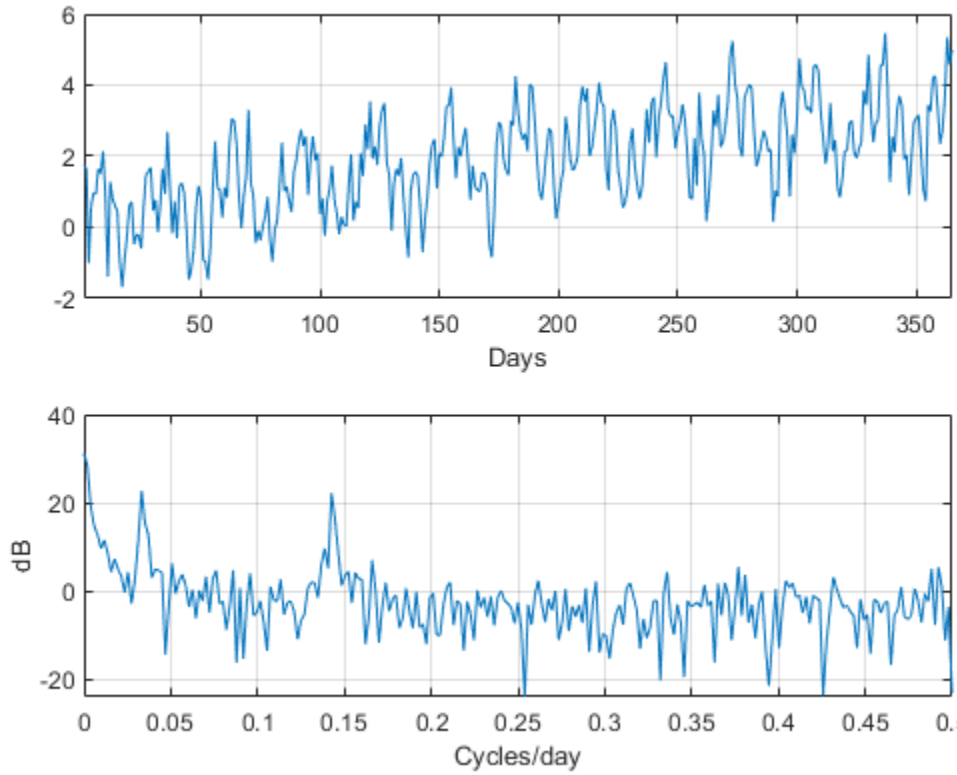
This example shows how to design a bandpass filter and filter data with minimum-order FIR equiripple and IIR Butterworth filters. You can model many real-world signals as a superposition of oscillating components, a low-frequency trend, and additive noise. For example, economic data often contain oscillations, which represent cycles superimposed on a slowly varying upward or downward trend. In addition, there is an additive noise

component, which is a combination of measurement error and the inherent random fluctuations in the process.

In these examples, assume you sample some process every day for one year. Assume the process has oscillations on approximately one-week and one-month scales. In addition, there is a low-frequency upward trend in the data and additive $N(0,1/4)$ white Gaussian noise.

Create the signal as a superposition of two sine waves with frequencies of 1/7 and 1/30 cycles/day. Add a low-frequency increasing trend term and $N(0,1/4)$ white Gaussian noise. Reset the random number generator for reproducible results. The data is sampled at 1 sample/day. Plot the resulting signal and the power spectral density (PSD) estimate.

```
rng default

Fs = 1;
n = 1:365;

x = cos(2*pi*(1/7)*n)+cos(2*pi*(1/30)*n-pi/4);
trend = 3*sin(2*pi*(1/1480)*n);

y = x+trend+0.5*randn(size(n));

[pxx,f] = periodogram(y,[],[],Fs);

subplot(2,1,1)
plot(n,y)
xlim([1 365])
xlabel('Days')
grid

subplot(2,1,2)
plot(f,10*log10(pxx))
xlabel('Cycles/day')
ylabel('dB')
grid
```



The low-frequency trend appears in the power spectral density estimate as increased low-frequency power. The low-frequency power appears approximately 10 dB above the oscillation at 1/30 cycles/day. Use this information in the specifications for the filter stopbands.

Design minimum-order FIR equiripple and IIR Butterworth filters with the following specifications: passband from [1/40,1/4] cycles/day and stopbands from [0,1/60] and [1/4,1/2] cycles/day. Set both stopband attenuations to 10 dB and the passband ripple tolerance to 1 dB.

```
Hd1 = designfilt('bandpassfir', ...
    'StopbandFrequency1',1/60,'PassbandFrequency1',1/40, ...
    'PassbandFrequency2',1/4 ,'StopbandFrequency2',1/2 , ...
    'StopbandAttenuation1',10,'PassbandRipple',1, ...
```

```
    'StopbandAttenuation2',10,'DesignMethod','equiripple','SampleRate',Fs);
Hd2 = designfilt('bandpassiir', ...
    'StopbandFrequency1',1/60,'PassbandFrequency1',1/40, ...
    'PassbandFrequency2',1/4 ,'StopbandFrequency2',1/2 , ...
    'StopbandAttenuation1',10,'PassbandRipple',1, ...
    'StopbandAttenuation2',10,'DesignMethod','butter','SampleRate',Fs);
```

Compare the order of the FIR and IIR filters and the unwrapped phase responses.

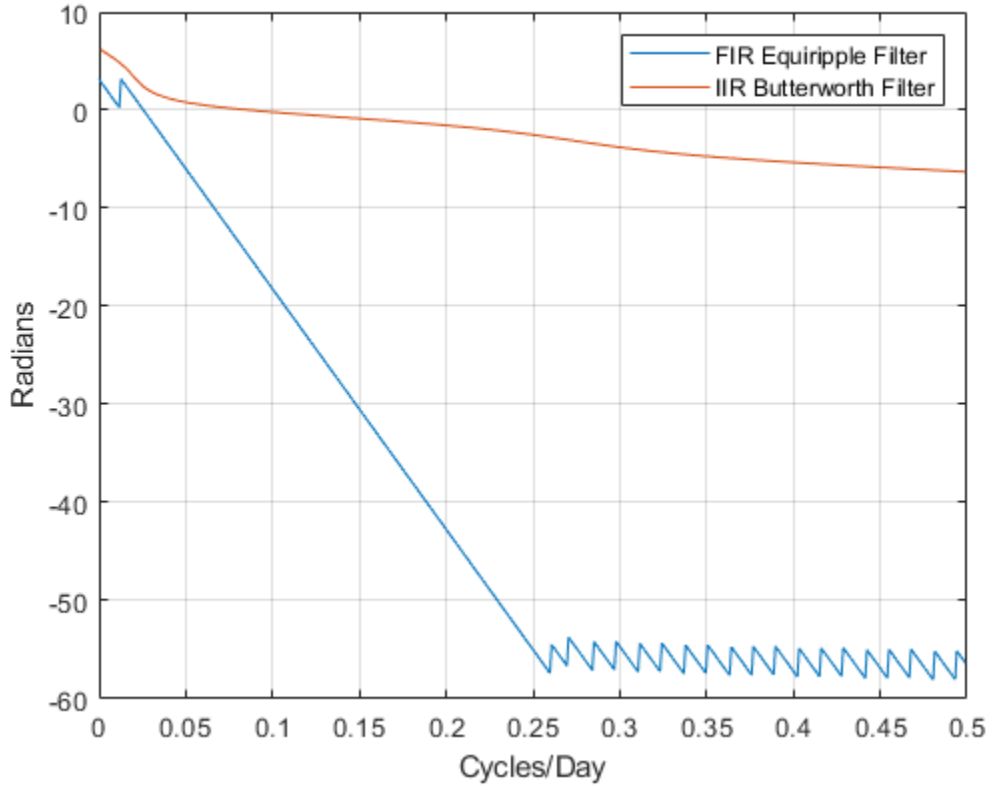
```
fprintf('The order of the FIR filter is %d\n',filtord(Hd1))
```

```
The order of the FIR filter is 78
```

```
fprintf('The order of the IIR filter is %d\n',filtord(Hd2))
```

```
The order of the IIR filter is 8
```

```
[phifir,w] = phasez(Hd1,[],1);
[phiiir,w] = phasez(Hd2,[],1);

figure
plot(w,unwrap(phifir))
hold on
plot(w,unwrap(phiiir))
hold off

xlabel('Cycles/Day')
ylabel('Radians')
legend('FIR Equiripple Filter','IIR Butterworth Filter')
grid
```



The IIR filter has a much lower order that the FIR filter. However, the FIR filter has a linear phase response over the passband, while the IIR filter does not. The FIR filter delays all frequencies in the filter passband equally, while the IIR filter does not.

Additionally, the rate of change of the phase per unit of frequency is greater in the FIR filter than in the IIR filter.

Design a lowpass FIR equiripple filter for comparison. The lowpass filter specifications are: passband [0,1/4] cycles/day, stopband attenuation equal to 10 dB, and the passband ripple tolerance set to 1 dB.

```
Hdlow = designfilt('lowpassfir', ...
    'PassbandFrequency',1/4,'StopbandFrequency',1/2, ...
```

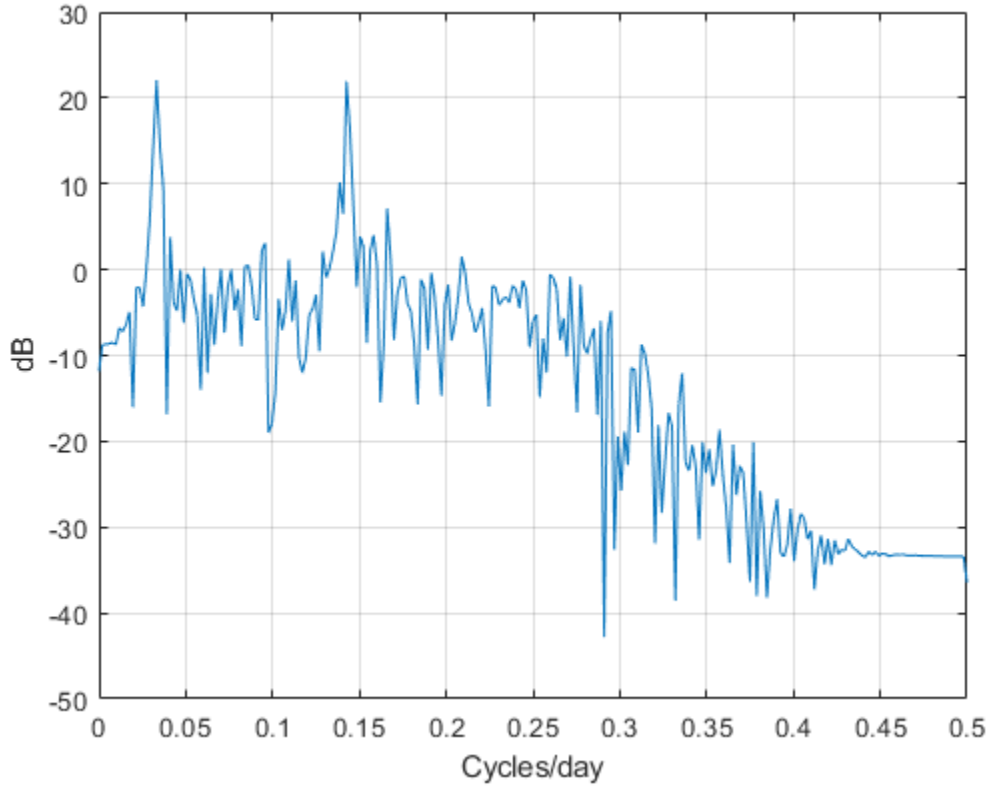
```
    'PassbandRipple',1,'StopbandAttenuation',10, ...
    'DesignMethod','equiripple','SampleRate',1);
```

Filter the data with the bandpass and lowpass filters.

```
yfir = filter(Hd1,y);
yiir = filter(Hd2,y);
ylow = filter(Hdlow,y);
```

Plot the PSD estimate of the bandpass IIR filter output. You can replace `yiir` with `yfir` in the following code to view the PSD estimate of the FIR bandpass filter output.

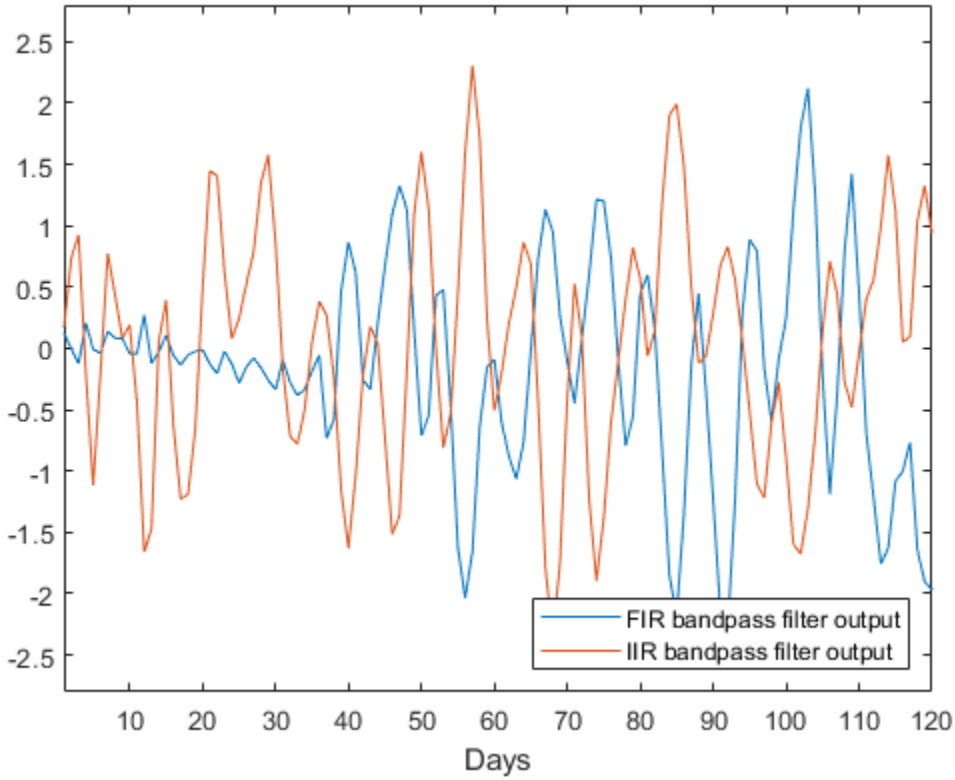
```
[pxx,f] = periodogram(yiir,[],[],Fs);

plot(f,10*log10(pxx))

xlabel('Cycles/day')
ylabel('dB')
grid
```



The PSD estimate shows the bandpass filter attenuates the low-frequency trend and high-frequency noise.

Plot the first 120 days of FIR and IIR filter output.

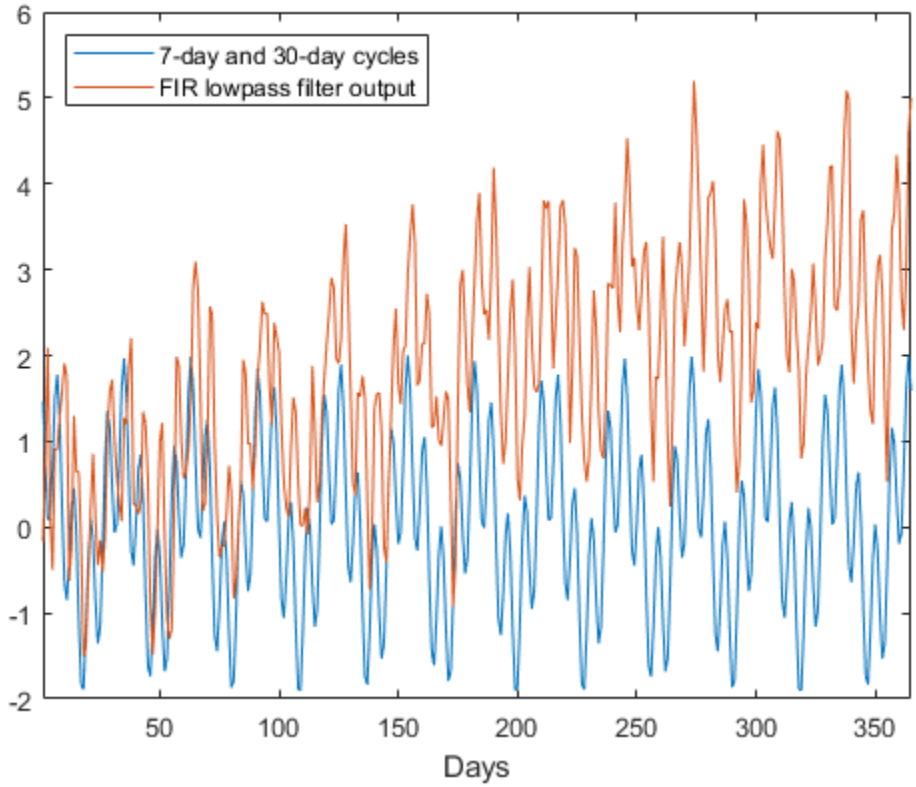
```
plot(n,yfir,n,yiir)

axis([1 120 -2.8 2.8])
xlabel('Days')
legend('FIR bandpass filter output','IIR bandpass filter output', ...
    'Location','SouthEast')
```



The increased phase delay in the FIR filter is evident in the filter output.

Plot the lowpass FIR filter output superimposed on the superposition of the 7-day and 30-day cycles for comparison.

```
plot(n,x,n,ylow)

xlim([1 365])
xlabel('Days')
legend('7-day and 30-day cycles','FIR lowpass filter output', ...
    'Location','NorthWest')
```



You can see in the preceding plot that the low-frequency trend is evident in the lowpass filter output. While the lowpass filter preserves the 7-day and 30-day cycles, the bandpass filters perform better in this example because the bandpass filters also remove the low-frequency trend.

Zero-Phase Filtering

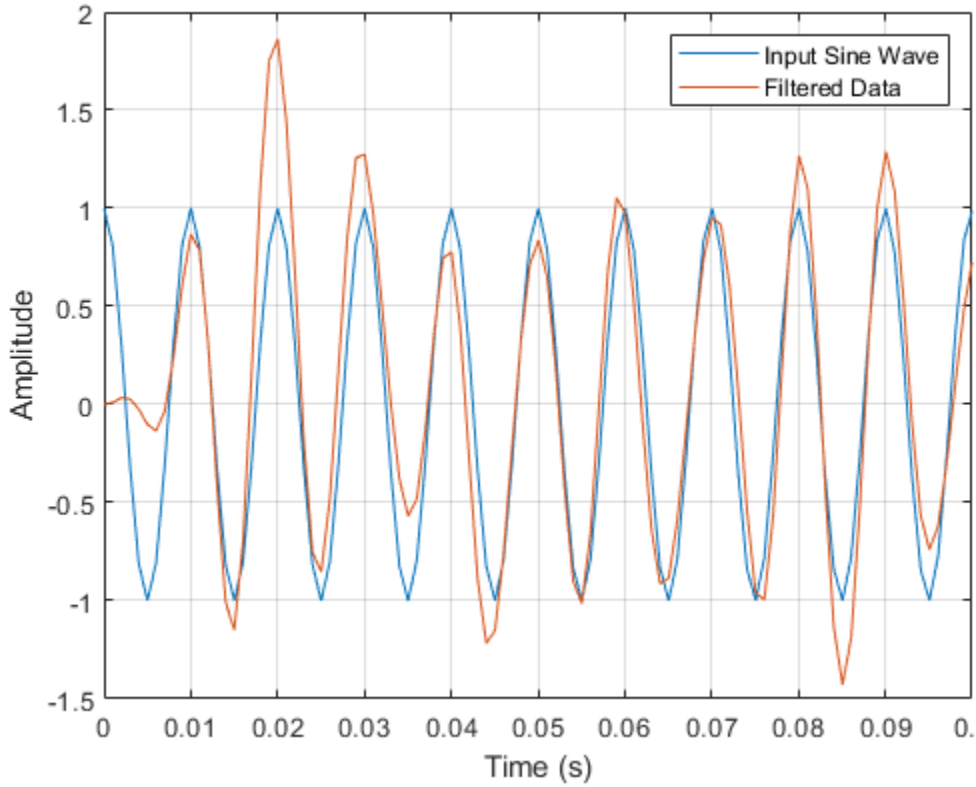
This example shows how to perform zero-phase filtering.

Repeat the signal generation and lowpass filter design with `firl` and `designfilt`. You do not have to execute the following code if you already have these variables in your workspace.

```
rng default

Fs = 1000;
t = linspace(0,1,Fs);
x = cos(2*pi*100*t)+0.5*randn(size(t));

% Using fir1
fc = 150;
Wn = (2/Fs)*fc;
b = fir1(20,Wn,'low',kaiser(21,3));

% Using designfilt
Hd = designfilt('lowpassfir','FilterOrder',20,'CutoffFrequency',150, ...
       'DesignMethod','window','Window',{@kaiser,3},'SampleRate',Fs);
```

Filter the data using `filter`. Plot the first 100 points of the filter output along with a superimposed sinusoid with the same amplitude and initial phase as the input signal.

```
yout = filter(Hd,x);
xin = cos(2*pi*100*t);

plot(t,xin,t,yout)
xlim([0 0.1])

xlabel('Time (s)')
ylabel('Amplitude')
legend('Input Sine Wave','Filtered Data')
grid
```

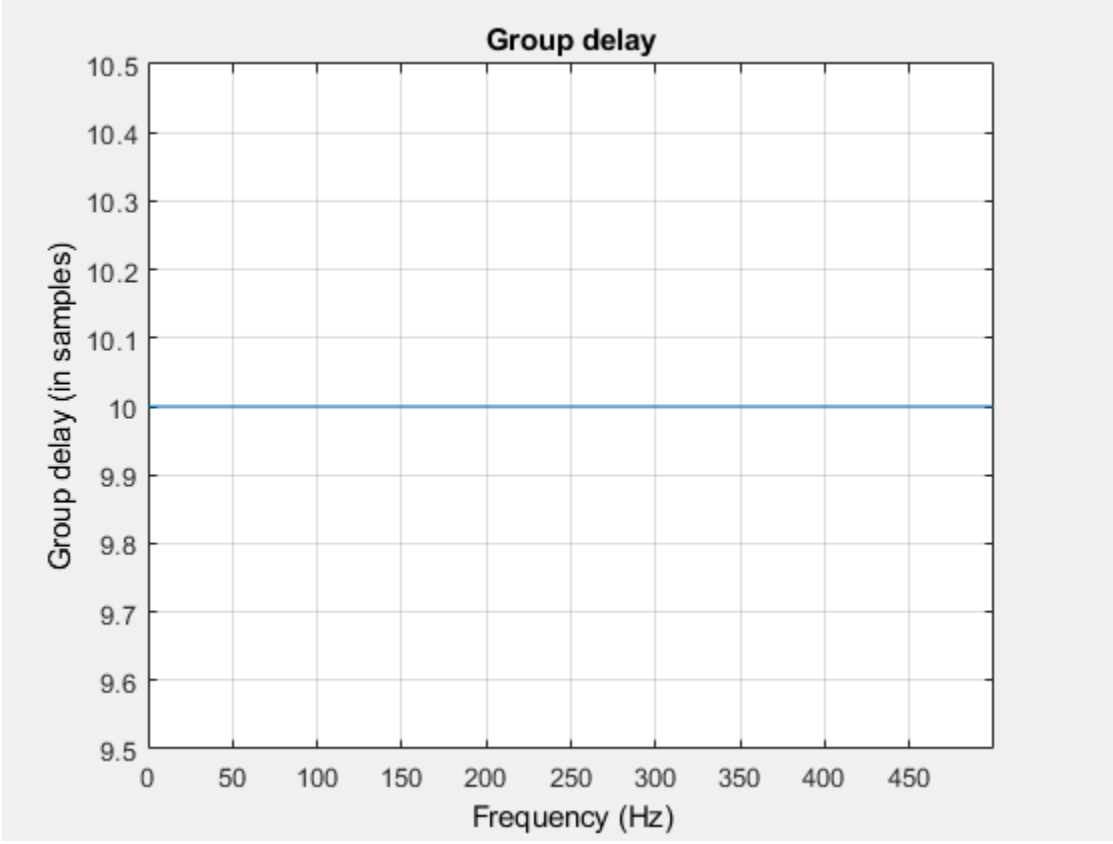


Looking at the initial 0.01 seconds of the filtered data, you see that the output is delayed with respect to the input. The delay appears to be approximately 0.01 seconds, which is almost 1/2 the length of the FIR filter in samples (10×0.001).

This delay is due to the filter's phase response. The FIR filter in these examples is a type I linear-phase filter. The group delay of the filter is 10 samples.

Plot the group delay using `fvtool`.

```
fvtool(Hd,'Analysis','grpdelay')
```

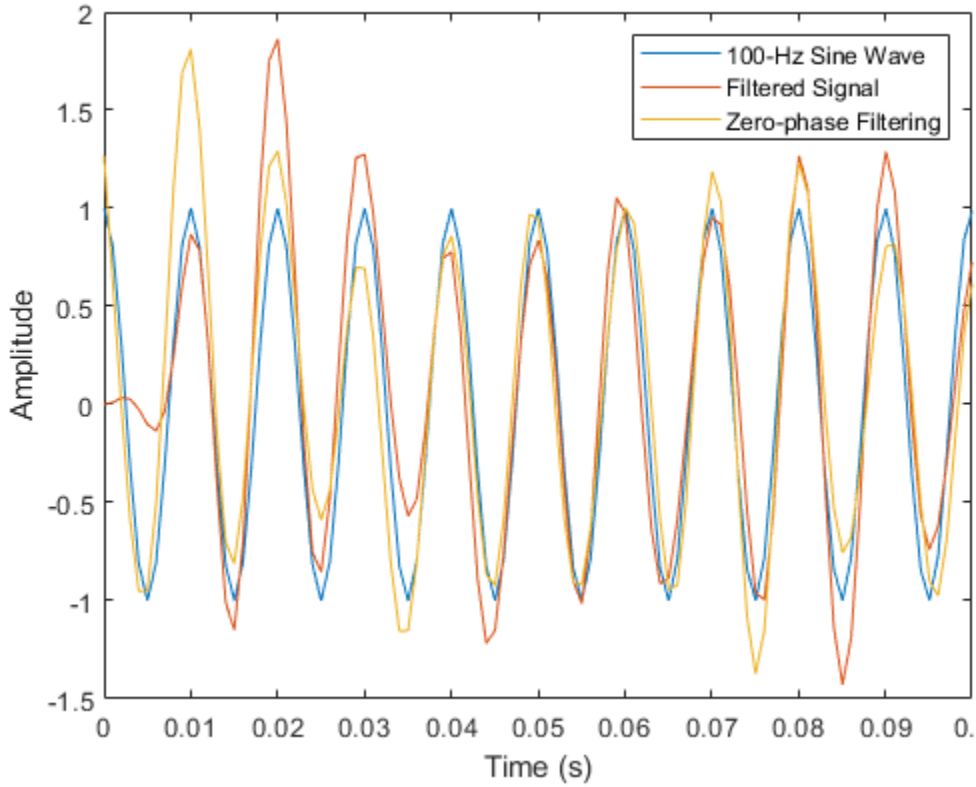


In many applications, phase distortion is acceptable. This is particularly true when phase response is linear. In other applications, it is desirable to have a filter with a zero-phase response. A zero-phase response is not technically possibly in a noncausal filter. However, you can implement zero-phase filtering using a causal filter with `filtfilt`.

Filter the input signal using `filtfilt`. Plot the responses to compare the filter outputs obtained with `filter` and `filtfilt`.

```
yzp = filtfilt(Hd,x);

plot(t,xin,t,yout,t,yzp)

xlim([0 0.1])
xlabel('Time (s)')
```

```
ylabel('Amplitude')
legend('100-Hz Sine Wave','Filtered Signal','Zero-phase Filtering',...
    'Location','NorthEast')
```



In the preceding figure, you can see that the output of `filtfilt` does not exhibit the delay due to the phase response of the FIR filter.

Selected Bibliography

[1] Karam, Lina J., and James H. McClellan. “Complex Chebyshev Approximation for FIR Filter Design.” *IEEE® Transactions on Circuits and Systems II: Analog and Digital Signal Processing*. Vol. 42, March 1995, pp. 207–216.

[2] Selesnick, Ivan W., and C. Sidney Burrus. “Generalized Digital Butterworth Filter Design.” *IEEE Transactions on Signal Processing*. Vol. 46, June 1998, pp. 1688–1694.

[3] Selesnick, Ivan W., Markus Lang, and C. Sidney Burrus. “Constrained Least Square Design of FIR Filters without Specified Transition Bands.” *IEEE Transactions on Signal Processing*. Vol. 44, August 1996, pp. 1879–1892.

3

Designing a Filter in `fdesign` — Process Overview

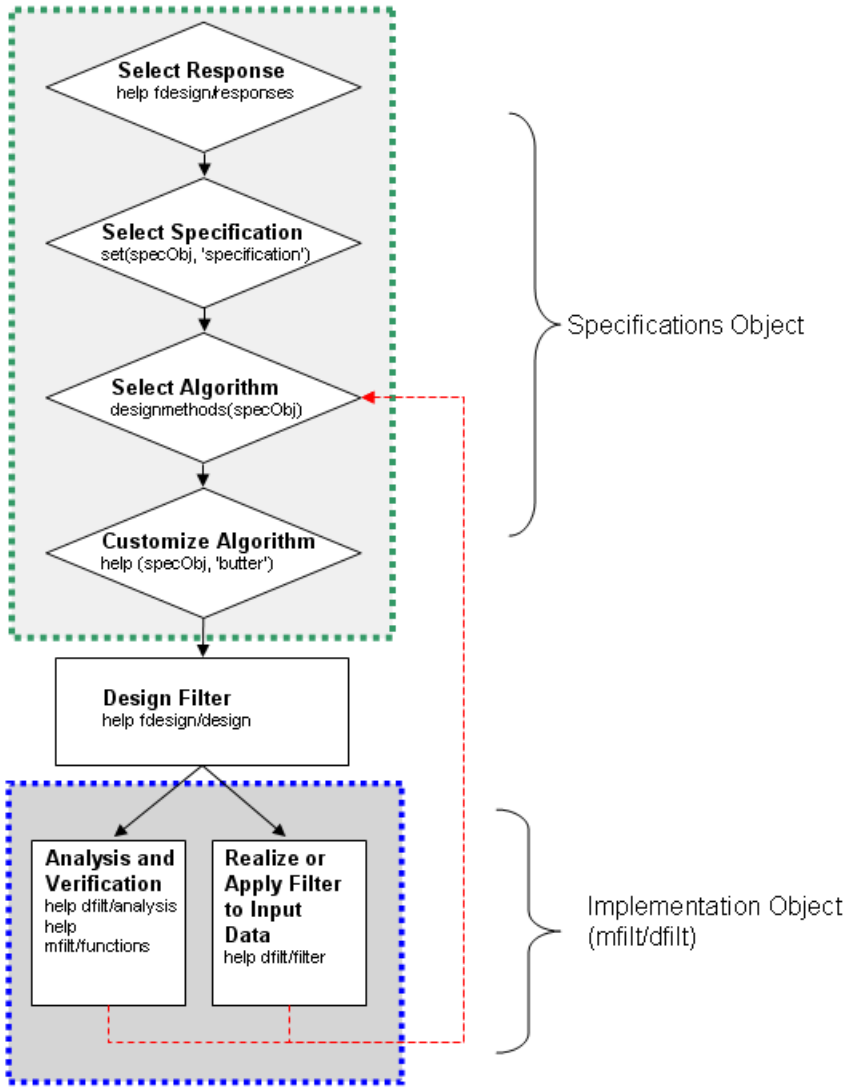
Process Flow Diagram and Filter Design Methodology

In this section...
“Exploring the Process Flow Diagram” on page 3-2 “Selecting a Response” on page 3-4 “Selecting a Specification” on page 3-4 “Selecting an Algorithm” on page 3-5 “Customizing the Algorithm” on page 3-7 “Designing the Filter” on page 3-7 “Design Analysis” on page 3-8 “Realize or Apply the Filter to Input Data” on page 3-9

Note You must minimally have the Signal Processing Toolbox installed to use `fdesign` and `design`. Some of the features described below may be unavailable if your installation does not additionally include the DSP System Toolbox™ license. The DSP System Toolbox significantly expands the functionality available for the specification, design, and analysis of filters. You can verify the presence of both toolboxes by typing `ver` at the command prompt.

Exploring the Process Flow Diagram

The process flow diagram shown in the following figure lists the steps and shows the order of the filter design process.



The first four steps of the filter design process relate to the filter Specifications Object, while the last two steps involve the filter Implementation Object. Both of these objects are discussed in more detail in the following sections. Step 5 - the design of the filter, is the transition step from the filter Specifications Object to the Implementation object. The

analysis and verification step is completely optional. It provides methods for the filter designer to ensure that the filter complies with all design criteria. Depending on the results of this verification, you can loop back to steps 3 and 4, to either choose a different algorithm, or to customize the current one. You may also wish to go back to steps 3 or 4 after you filter the input data with the designed filter (step 7), and find that you wish to tweak the filter or change it further.

The diagram shows the help command for each step. Enter the help line at the MATLAB command prompt to receive instructions and further documentation links for the particular step. Not all of the steps have to be executed explicitly. For example, you could go from step 1 directly to step 5, and the interim three steps are done for you by the software.

The following are the details for each of the steps shown above.

Selecting a Response

If you type:

```
help fdesign/responses
```

at the MATLAB command prompt, you see a list of all available filter responses. The responses marked with an asterisk require the DSP System Toolbox.

You must select a response to initiate the filter. In this example, a bandpass filter Specifications Object is created by typing the following:

```
d = fdesign.bandpass
```

Selecting a Specification

A *specification* is an array of design parameters for a given filter. The specification is a property of the Specifications Object.

Note A specification is not the same as the Specifications Object. A Specifications Object contains a specification as one of its properties.

When you select a filter response, there are a number of different specifications available. Each one contains a different combination of design parameters. After you create a filter

Specifications Object, you can query the available specifications for that response. Specifications marked with an asterisk require the DSP System Toolbox.

```
d = fdesign.bandpass;
set(d,'specification')

ans =

    'Fst1,Fp1,Fp2,Fst2,Ast1,Ap,Ast2'
    'N,F3dB1,F3dB2'
    'N,F3dB1,F3dB2,Ap'
    'N,F3dB1,F3dB2,Ast'
    'N,F3dB1,F3dB2,Ast1,Ap,Ast2'
    'N,F3dB1,F3dB2,BWp'
    'N,F3dB1,F3dB2,BWst'
    'N,Fc1,Fc2'
    'N,Fp1,Fp2,Ap'
    'N,Fp1,Fp2,Ast1,Ap,Ast2'
    'N,Fst1,Fp1,Fp2,Fst2'
    'N,Fst1,Fp1,Fp2,Fst2,Ap'
    'N,Fst1,Fst2,Ast'
    'Nb,Na,Fst1,Fp1,Fp2,Fst2'

d = fdesign.arbmag;
set(d,'specification')

ans =

    'N,F,A'
    'N,B,F,A'
```

The `set` command can be used to select one of the available specifications as follows:

```
d = fdesign.lowpass;
set(d,'specification', 'N,Fc')
```

If you do not perform this step explicitly, `fdesign` returns the default specification for the response you chose in “Select a Response” on page 4-2, and provides default values for all design parameters included in the specification.

Selecting an Algorithm

The availability of algorithms depends the chosen filter response, the design parameters, and the availability of the DSP System Toolbox. In other words, for the same lowpass

filter, changing the specification also changes the available algorithms. In the following example, for a lowpass filter and a specification of `'N, Fc'`, only one algorithm is available—`window`.

```
set (d, 'specification', 'N,Fc')
designmethods (d) %step3: get available algorithms

Design Methods for class fdesign.lowpass (N,Fc):


window
```

However, for a specification of `'Fp,Fst,Ap,Ast'`, a number of algorithms are available. If the user has only the Signal Processing Toolbox installed, the following algorithms are available:

```
set(d,'specification','Fp,Fst,Ap,Ast')
designmethods(d)

Design Methods for class fdesign.lowpass (Fp,Fst,Ap,Ast):

butter
cheby1
cheby2
ellip
equiripple
kaiserwin
```

If the user additionally has the DSP System Toolbox installed, the number of available algorithms for this response and specification increases:

```
set(d,'specification','Fp,Fst,Ap,Ast')
designmethods(d)

Design Methods for class fdesign.lowpass (Fp,Fst,Ap,Ast):

butter
cheby1
cheby2
ellip
equiripple
ifir
kaiserwin
multistage
```

The user chooses a particular algorithm and implements the filter with the `design` function.

```
Hd=design(d,'butter');
```

The preceding code actually creates the filter. If you do not perform this step explicitly, `design` automatically selects the optimum algorithm for the chosen response and specification.

Customizing the Algorithm

The customization options available for any given algorithm depend not only on the algorithm itself, selected in “Selecting an Algorithm” on page 3-5, but also on the specification selected in “Selecting a Specification” on page 3-4. To explore all the available options, type the following at the MATLAB command prompt:

```
help(d,'algorithm-name')
```

where `d` is the Filter Specification Object, and `algorithm-name` is the name of the algorithm in single quotes, such as `'butter'` or `'cheby1'`.

The application of these customization options takes place while “Designing the Filter” on page 3-7, because these options are the properties of the filter Implementation Object, not the Specification Object.

If you do not perform this step explicitly, the optimum algorithm structure is selected.

Designing the Filter

To create a filter, use the `design` command:

```
Hd = design(d);
```

where `d` is the Specifications Object. This code creates a filter without specifying the algorithm. When the algorithm is not specified, the software selects the best available one.

To apply the algorithm chosen in “Selecting an Algorithm” on page 3-5, use the same `design` command, but specify the Butterworth algorithm as follows:

```
Hd = design(d,'butter');
```

To obtain help and see all the available options, type:

```
help fdesign/design
```

This help command describes not only the options for the `design` command itself, but also options that pertain to the method or the algorithm. If you are customizing the algorithm, you apply these options in this step. In the following example, you design a bandpass filter, and then modify the filter structure:

```
Hd = design(d,'butter','FilterStructure','df2sos')
```

```
Hd =

         FilterStructure: 'Direct-Form II, Second-Order Sections'
              Arithmetic: 'double'
               sosMatrix: [13x6 double]
             ScaleValues: [14x1 double]
     OptimizeScaleValues: true
        PersistentMemory: false
```

The filter design step, just like the first task of choosing a response, must be performed explicitly. The filter is created only when `design` is called.

Design Analysis

After the filter is designed you may wish to analyze it to determine if the filter satisfies the design criteria. Filter analysis is broken into three main sections:

- Frequency domain analysis — Includes the magnitude response, group delay, and pole-zero plots.
- Time domain analysis — Includes impulse and step response
- Implementation analysis — Includes quantization noise and cost

To display help for analysis of a discrete-time filter, type:

```
>> help dfilt/analysis
```

To display help for analysis of a farrow filter, type:

```
>> help farrow/functions
```

To analyze your filter, you must explicitly perform this step.

Realize or Apply the Filter to Input Data

After the filter is designed and optimized, it can be used to filter actual input data. The basic filter command takes input data `x`, filters it through the Filter Object, and produces output `y`:

```
>> y = filter (FilterObj, x)
```

This step is never automatically performed for you. To filter your data, you must explicitly execute this step. To understand how the filtering commands work, type:

```
>> help dfilt/filter
```

Note If you have Simulink®, you have the option of exporting this filter to a Simulink block using the `realizemdl` command. To get help on this command, type:

```
>> help realizemdl
```

Designing a Filter in the Filter Builder GUI

- “Filter Builder Design Process” on page 4-2
- “Designing a FIR Filter Using filterBuilder” on page 4-10
- “Compensate for Delay and Distortion Introduced by Filters” on page 4-13
- “Comparison of Analog IIR Lowpass Filters” on page 4-20
- “Frequency Response of a Lowpass Bessel Filter” on page 4-22
- “Speaker Crossover Filters” on page 4-24

Filter Builder Design Process

In this section...
“Introduction to Filter Builder” on page 4-2
“Design a Filter Using Filter Builder” on page 4-2
“Select a Response” on page 4-2
“Select a Specification” on page 4-5
“Select an Algorithm” on page 4-5
“Customize the Algorithm” on page 4-6
“Analyze the Design” on page 4-8
“Realize or Apply the Filter to Input Data” on page 4-8

Introduction to Filter Builder

The `filterBuilder` function provides a graphical interface to the `fdesign` object-oriented filter design paradigm and is intended to reduce development time during the filter design process. `filterBuilder` uses a specification-centered approach to find the best algorithm for the desired response.

Note `filterBuilder` requires the Signal Processing Toolbox. The DSP System Toolbox product greatly expands the functionality of `filterBuilder`. Many of the features described or displayed on this page are only available if the DSP System Toolbox is installed. You may verify your installation by typing `ver` at the command prompt.

Design a Filter Using Filter Builder

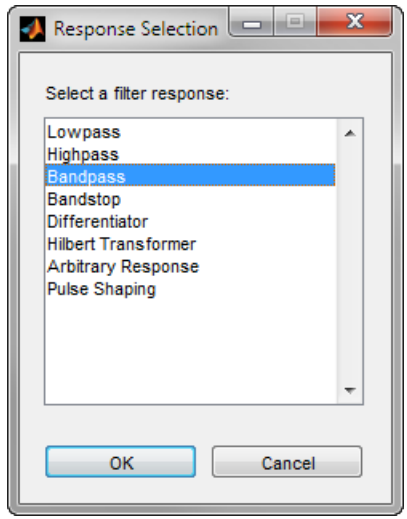
The basic workflow in using `filterBuilder` is to choose the constraints and specifications of the filter, and to use those constraints as a starting point in the design. Postponing the choice of algorithm for the filter allows the best design method to be determined automatically, based on the desired performance criteria. The following are the details of each of the steps for designing a filter with `filterBuilder`.

Select a Response

When you open the `filterBuilder` tool by typing:

```
filterBuilder
```

at the MATLAB command prompt, the **Response Selection** dialog box appears, listing all possible filter responses available in DSP System Toolbox.

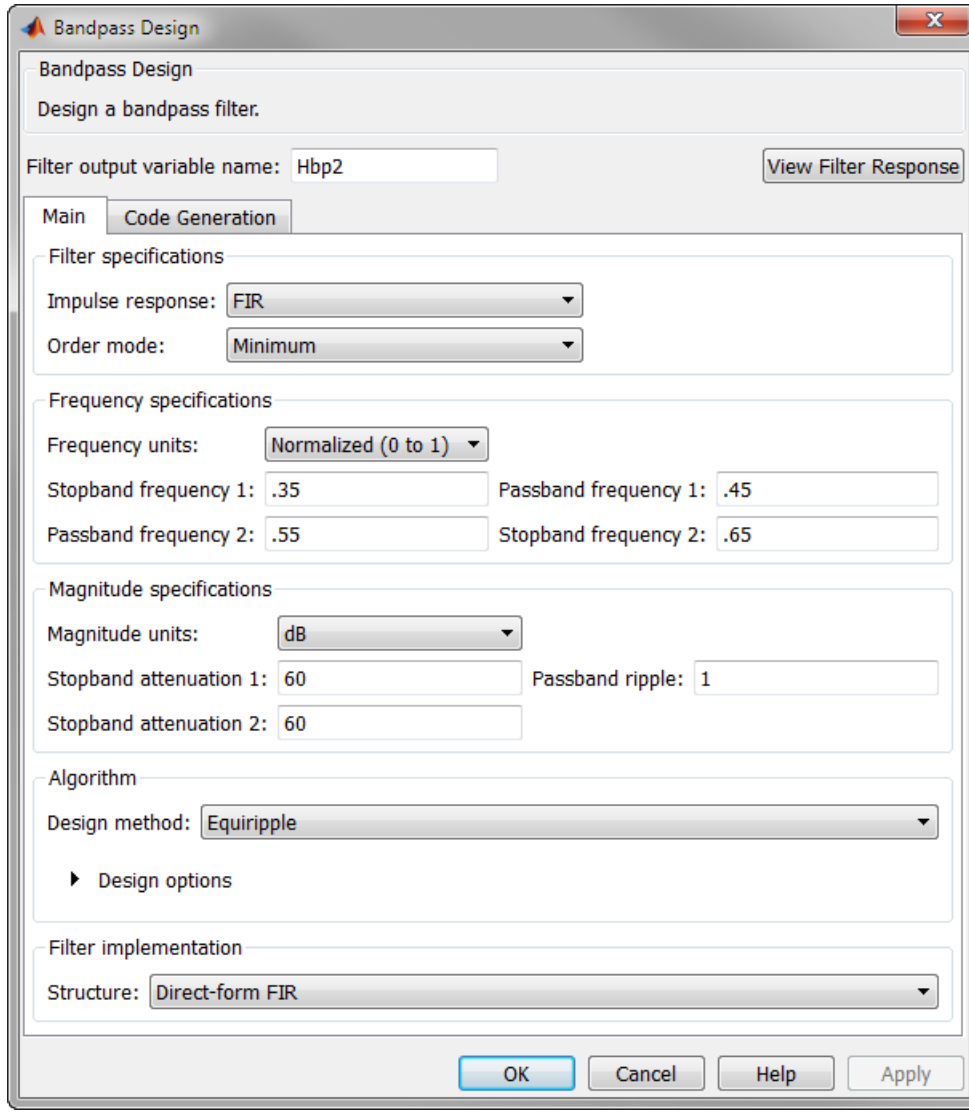


Note This step cannot be skipped because it is not automatically completed for you by the software. You must select a response to initiate the filter design process.

After you choose a response, say bandpass, you start the design of the Specifications Object, and the Bandpass Design dialog box appears. This dialog box contains a **Main** pane, a **Data Types** pane, and a **Code Generation** pane. The specifications of your filter are generally set in the **Main** pane of the dialog box.

The **Data Types** pane provides settings for precision and data types, and the **Code Generation** pane contains options for various implementations of the completed filter design.

For the initial design of your filter, you mostly use the **Main** pane.



The **Bandpass Design** dialog box contains all the parameters necessary to determine the specifications of a bandpass filter. The parameters listed in the **Main** pane depend upon the type of filter you are designing. However, no matter what type of filter you have

chosen in the **Response Selection** dialog box, the filter design dialog box contains the **Main**, **Data Types**, and **Code Generation** panes.

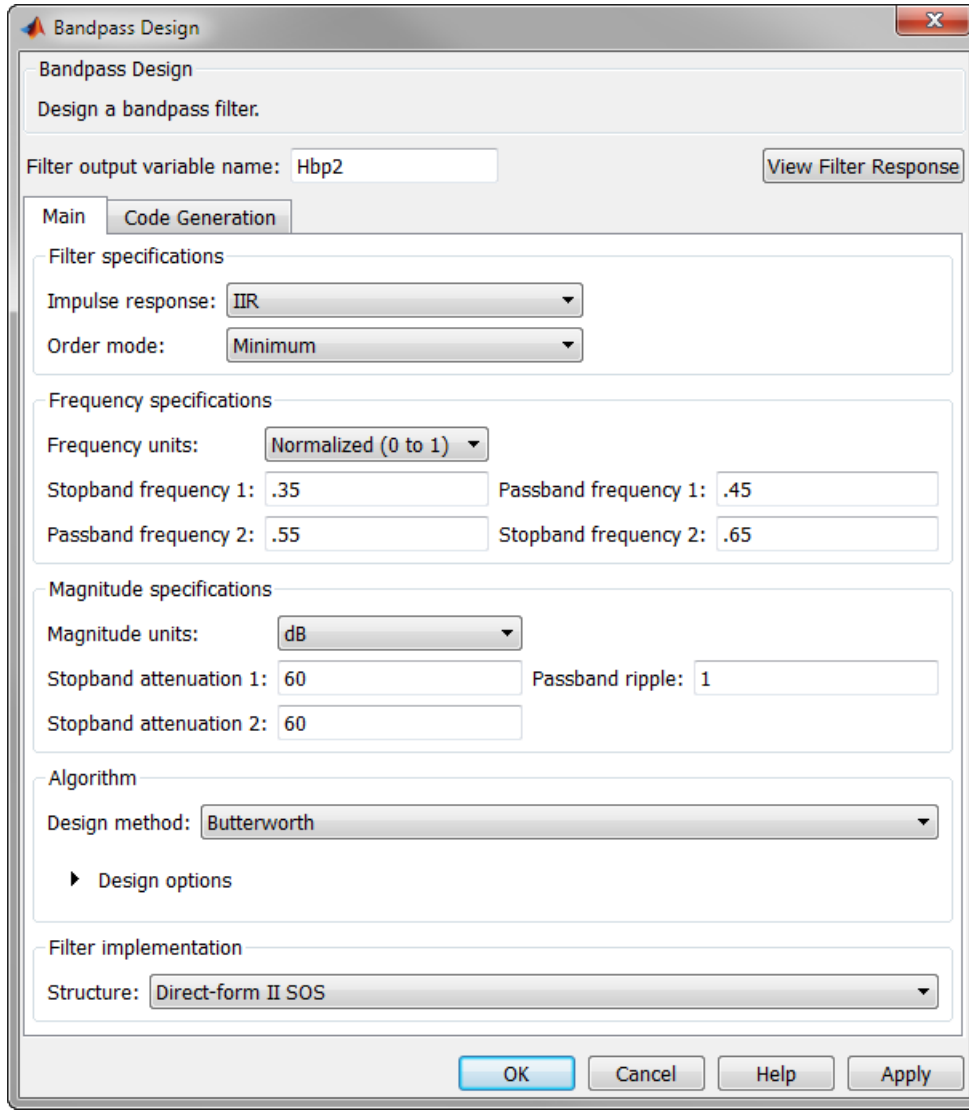
Select a Specification

To choose the specification for the bandpass filter, you can begin by selecting an **Impulse Response**, **Order Mode**, and **Filter Type** in the **Filter Specifications** frame of the **Main Pane**. You can further specify the response of your filter by setting frequency and magnitude specifications in the appropriate frames on the **Main Pane**.

Note **Frequency**, **Magnitude**, and **Algorithm** specifications are interdependent and might change based on your **Filter Specifications** selections. When choosing specifications for your filter, select your Filter Specifications first and work your way down the dialog box. This approach ensures that the best settings for dependent specifications display as available in the dialog box.

Select an Algorithm

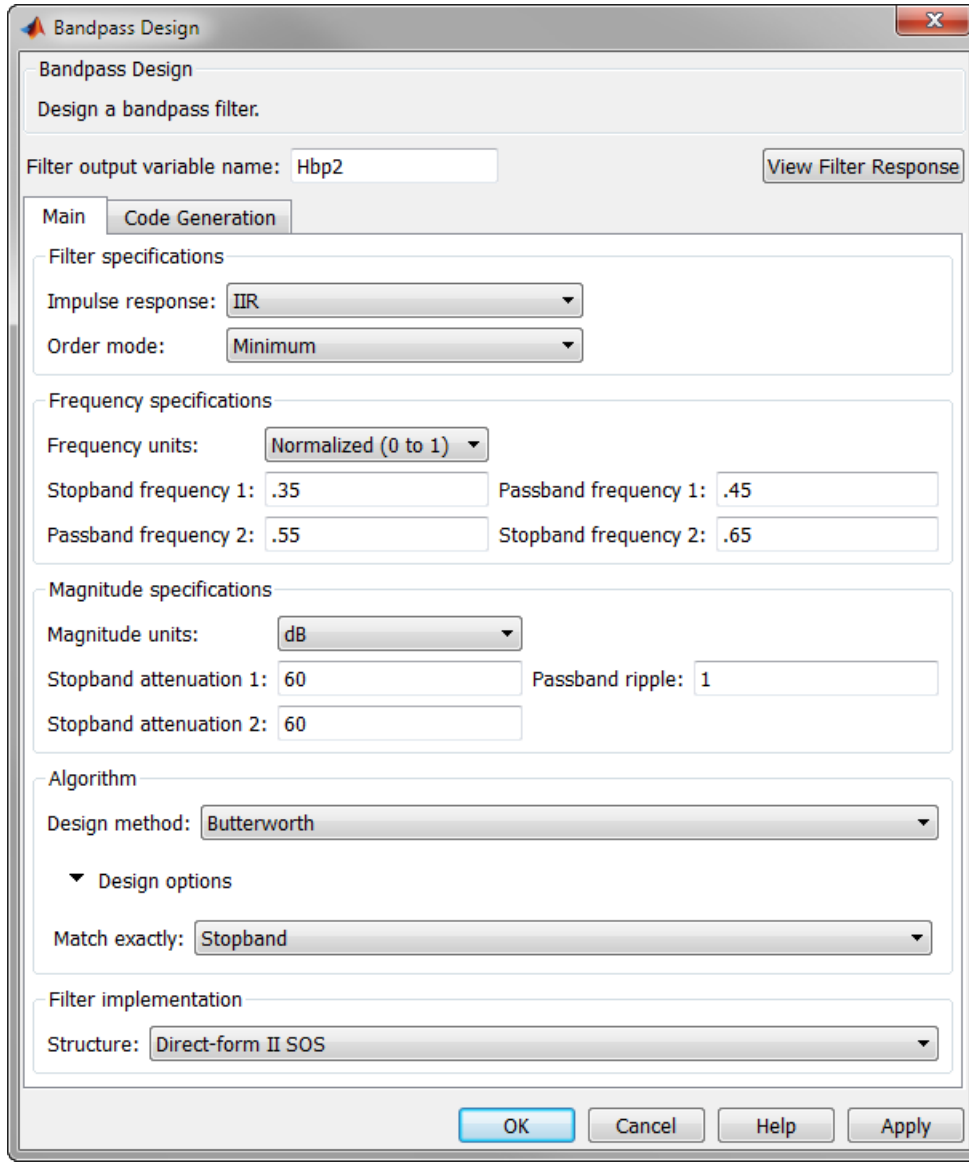
The algorithms available for your filter depend upon the filter response and design parameters you have selected in the previous steps. For example, in the case of a bandpass filter, if the impulse response selected is IIR and the **Order Mode** field is set to `Minimum`, the design methods available are `Butterworth`, `Chebyshev` type I or II, or `Elliptic`. If the **Order Mode** field is set to `Specify`, the design method available is `IIR least p-norm`.



Customize the Algorithm

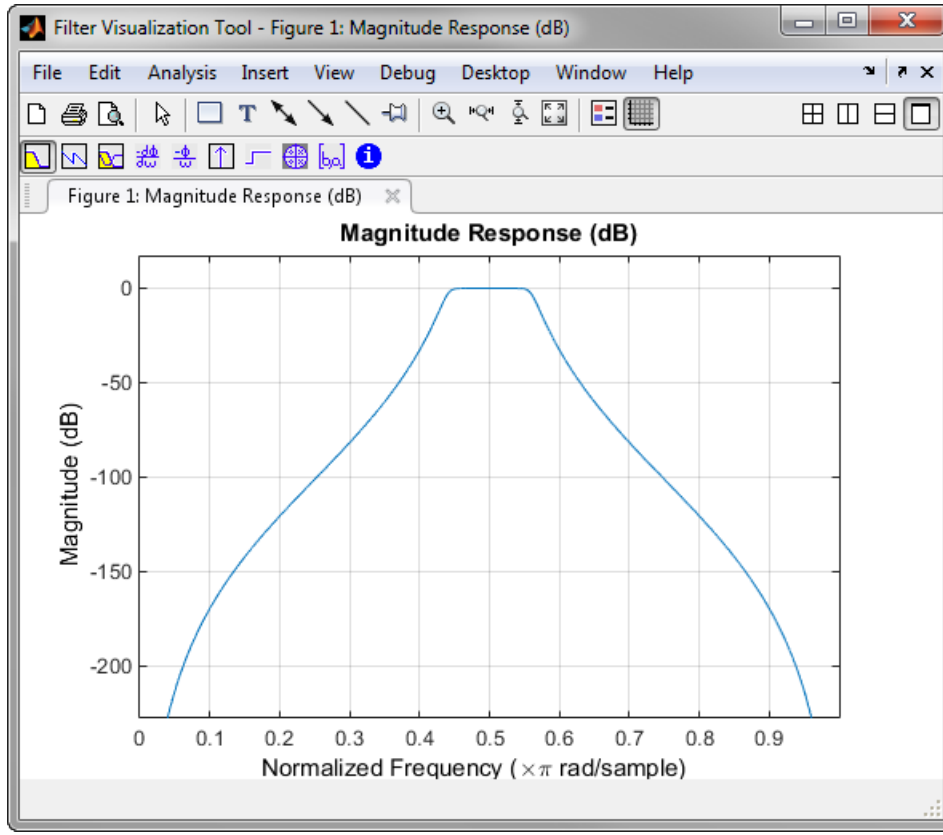
By expanding the **Design options** section of the **Algorithm** frame, you can further customize the algorithm specified. The options available depend upon the algorithm and

settings that have already been selected in the dialog box. In the case of a bandpass IIR filter using the `Butterworth` method, design options such as **Match Exactly** are available, as shown in the following figure.



Analyze the Design

To analyze the filter response, click the View Filter Response button. The Filter Visualization Tool opens displaying the magnitude plot of the filter response.



Realize or Apply the Filter to Input Data

When you have achieved the desired filter response through design iterations and analysis using the **Filter Visualization Tool**, apply the filter to the input data. Again, this step is never automatically performed for you by the software. To filter your data, you must explicitly execute this step. In the **Bandpass Design** dialog box, click OK and the Signal Processing Toolbox software creates the filter coefficients and exports it to the MATLAB workspace.

The filter is then ready to be used to filter actual input data. The basic filter command takes input data `x`, filters it through the Filter Object, and produces output `y`:

```
y = filter(Hbs,x)
```

To understand how the filtering command works, type:

```
help dfilt/filter
```

Tip If you have Simulink, you have the option of exporting this filter to a Simulink block using the `realizemdl` command. To get help on this command, type:

```
help realizemdl
```

Designing a FIR Filter Using filterBuilder

FIR Filter Design

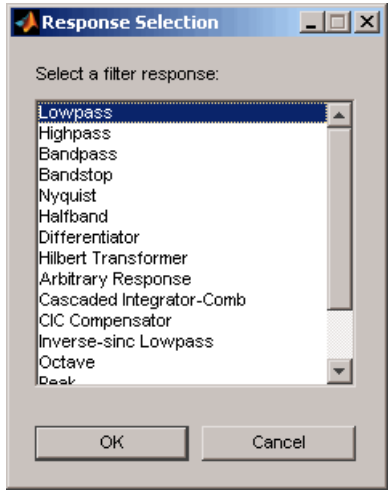
Example 4.1. Example – Using Filter Builder to Design an FIR Filter

To design a lowpass finite impulse response (FIR) filter using `filterBuilder`:

1 Open the Filter Builder GUI by typing the following at the MATLAB prompt:

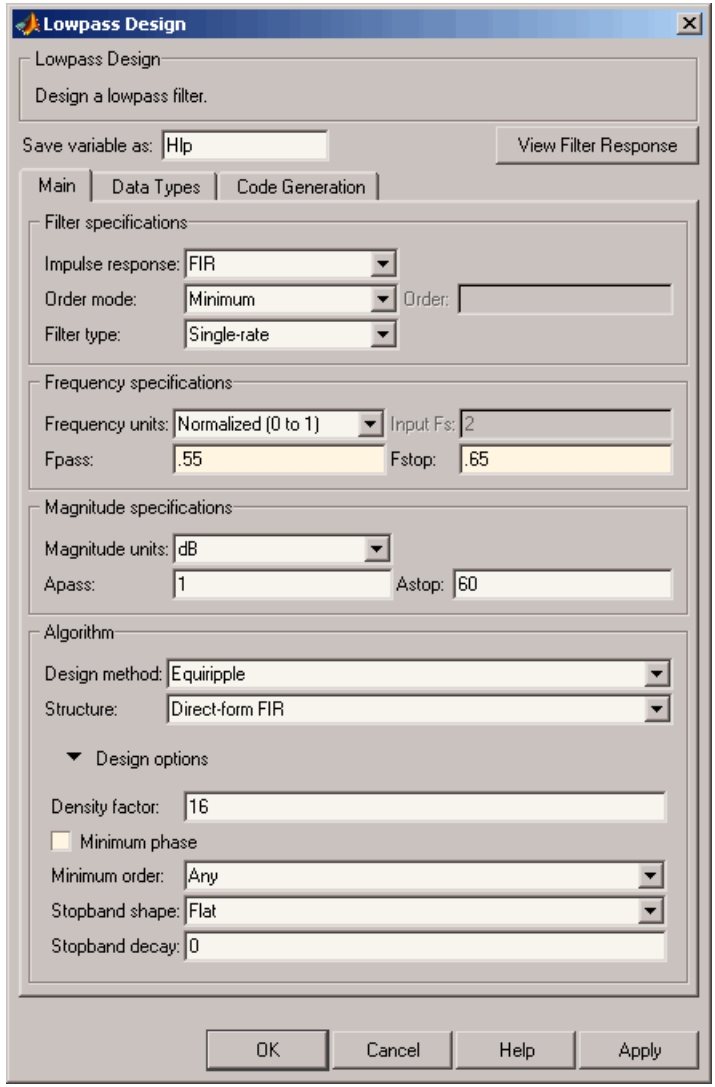
```
filterBuilder
```

The **Response Selection** dialog box appears. In this dialog box, you can select from a list of filter response types. Select `Lowpass` in the list box.



2 Hit the **OK** button. The **Lowpass Design** dialog box opens. Here you can specify the writable parameters of the Lowpass filter object. The components of the **Main** frame of this dialog box are described in the section titled *Lowpass Filter Design Dialog Box — Main Pane*. In the dialog box, make the following changes:

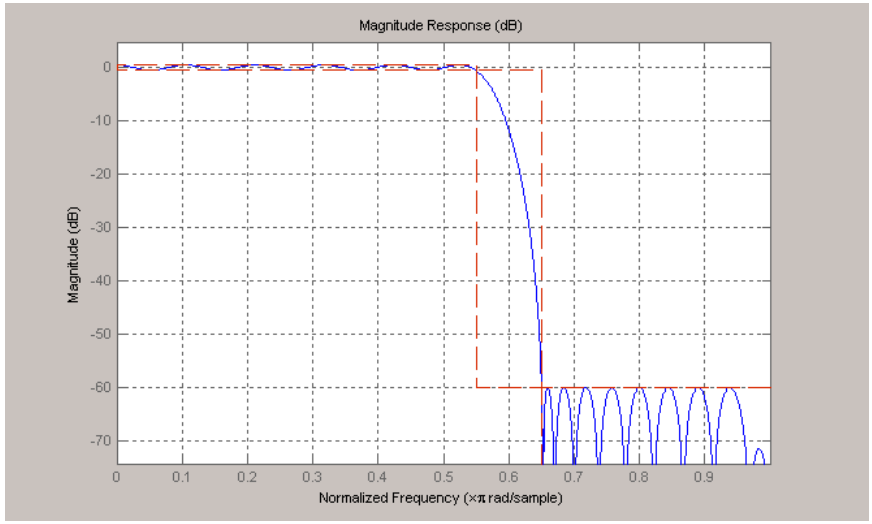
- Enter a Fpass value of 0.55.
- Enter a Fstop value of 0.65.



3 Click **Apply**, and the following message appears at the MATLAB prompt:

```
The variable 'Hlp' has been exported to the command window.
```

4 To check your design, click **View Filter Response**. The Filter Visualization tool appears, showing a plot of the magnitude response of the filter.



You can change the design and click **Apply**, followed by **View Filter Response**, as many times as needed until your design specifications are met.

Compensate for Delay and Distortion Introduced by Filters

Filtering a signal introduces a delay. This means that the output signal is shifted in time with respect to the input.

When the shift is constant, you can correct for the delay by shifting the signal in time.

Sometimes the filter delays some frequency components more than others. This phenomenon is called phase distortion. To compensate for this effect, you can perform zero-phase filtering using the `filtfilt` function.

Take an electrocardiogram reading sampled at 500 Hz for 1 s. Add random noise. Reset the random number generator for reproducible results

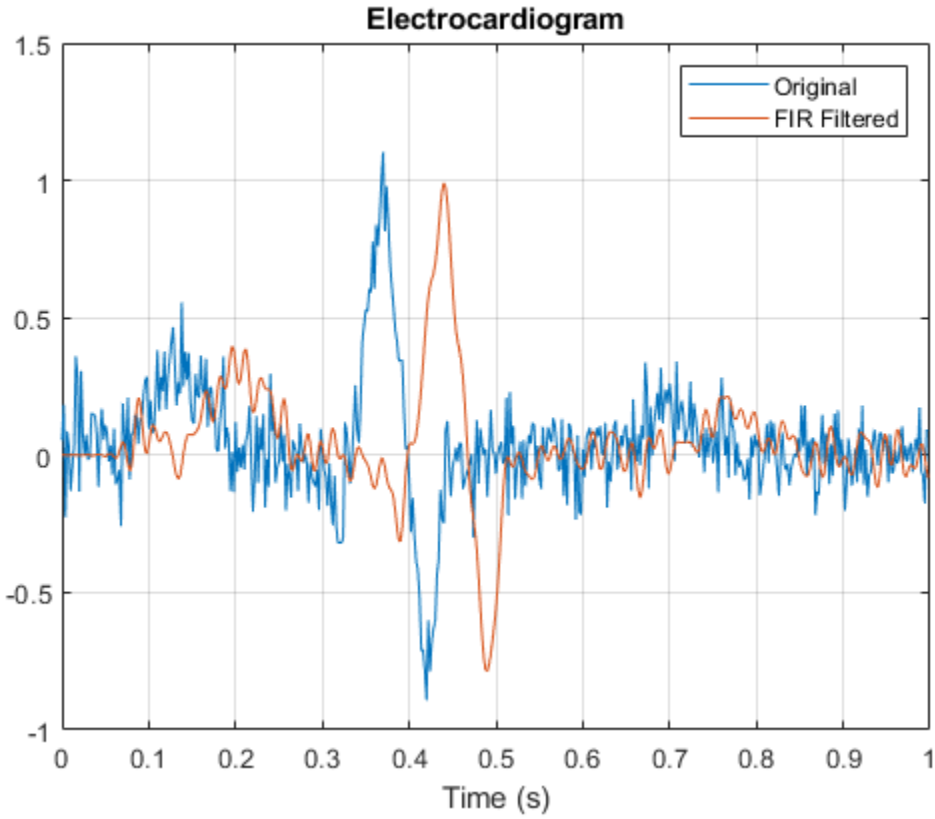
```
Fs = 500;
N = 500;

rng default

xn = ecg(N)+0.1*randn([1 N]);
tn = (0:N-1)/Fs;
```

Remove some of the noise using a filter that stops frequencies above 75 Hz. Use `designfilt` to design an FIR filter of order 70.

```
Nfir = 70;
Fst = 75;

firf = designfilt('lowpassfir','FilterOrder',Nfir, ...
    'CutoffFrequency',Fst,'SampleRate',Fs);
```

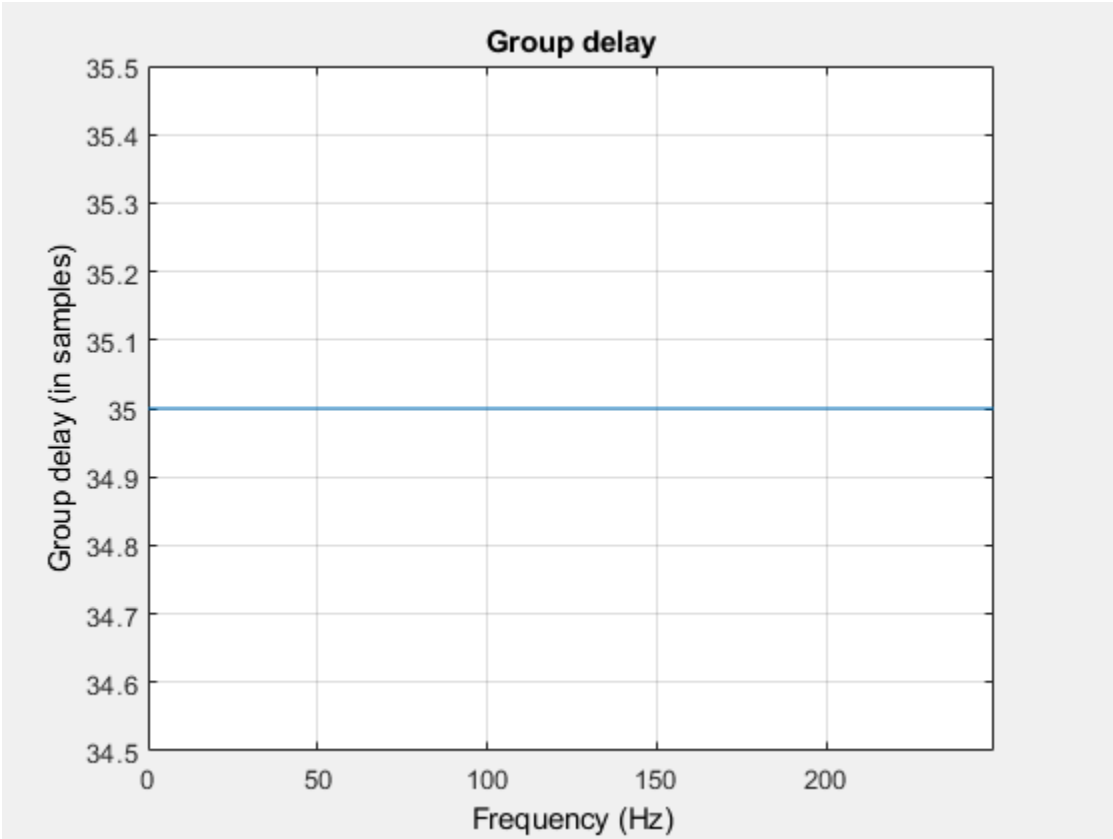
Filter the signal and plot it. The result is smoother than the original, but lags behind it.

```
xf = filter(firf,xn);

plot(tn,xn,tn,xf)
title 'Electrocardiogram'
xlabel 'Time (s)'
legend('Original','FIR Filtered')
grid
```



Use `grpdelay` to check that the delay caused by the filter equals half the filter order.

```
grpdelay(firf,N,Fs)
```




```
delay = mean(grpdelay(firf))
```

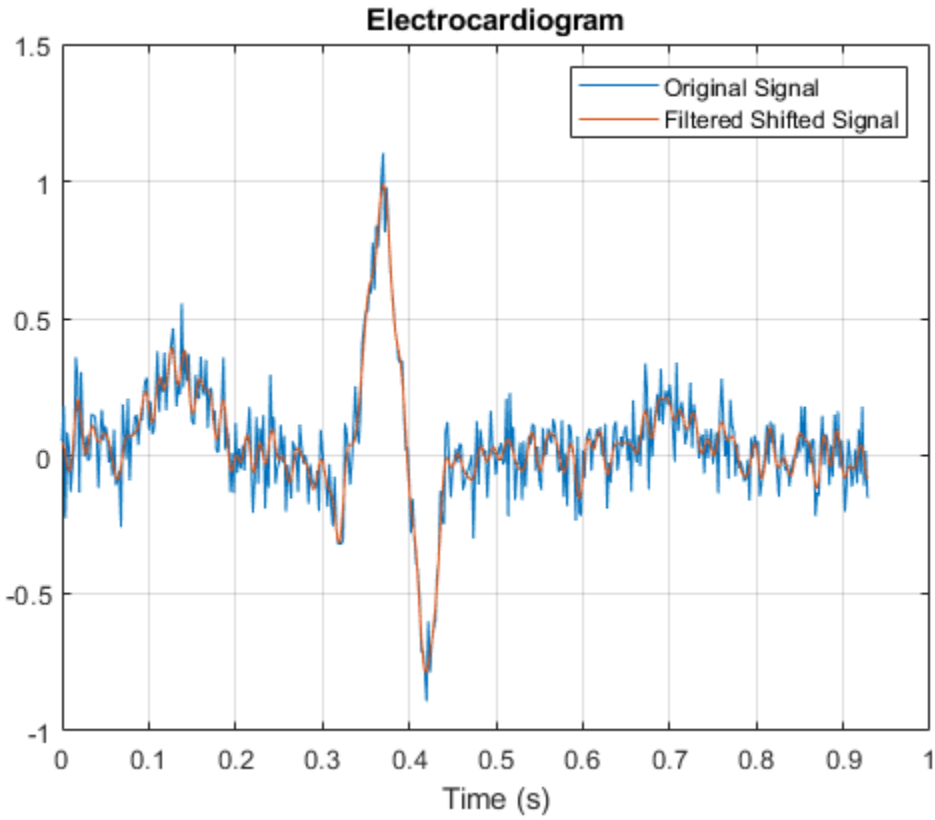
```
delay = 35
```

Line up the data. Shift the filtered signal by removing its first `delay` samples. Remove the last `delay` samples of the original and of the time vector.

```
tt = tn(1:end-delay);
sn = xn(1:end-delay);

sf = xf;
sf(1:delay) = [];
```

Plot the signals and verify that they are aligned.

```
plot(tt,sn,tt,sf)
title 'Electrocardiogram'
xlabel('Time (s)')
legend('Original Signal','Filtered Shifted Signal')
grid
```

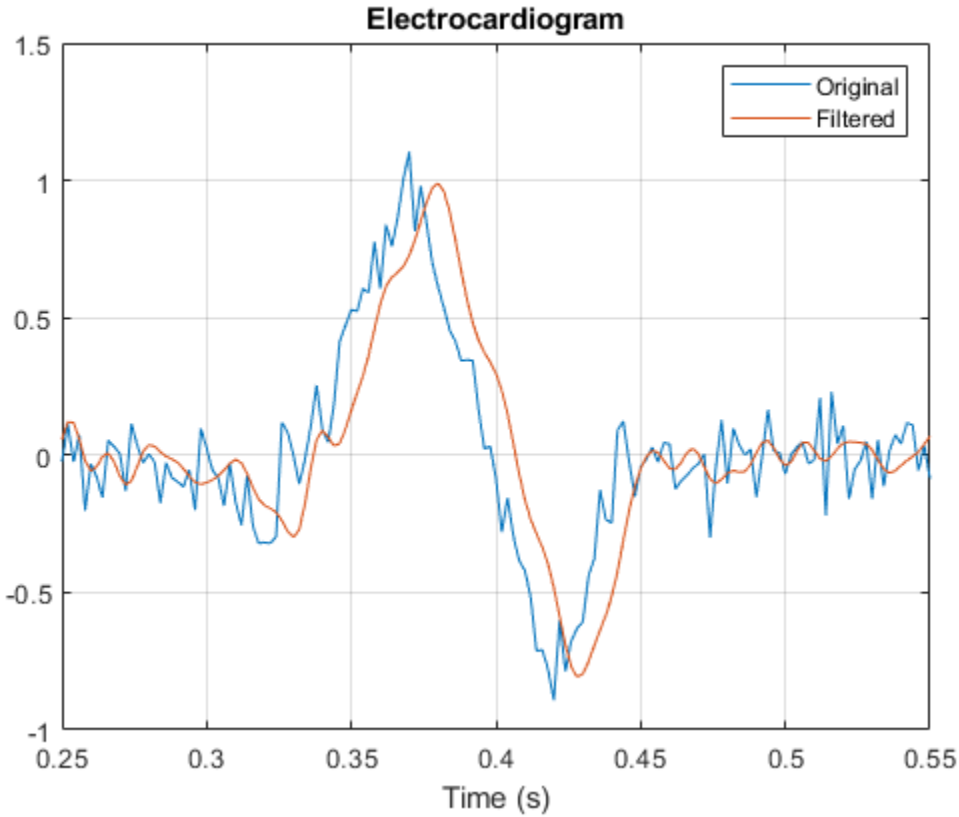


Repeat the computation using a 7th-order IIR filter.

```
Niir = 7;

iir = designfilt('lowpassiir','FilterOrder',Niir, ...
    'HalfPowerFrequency',Fst,'SampleRate',Fs);
```

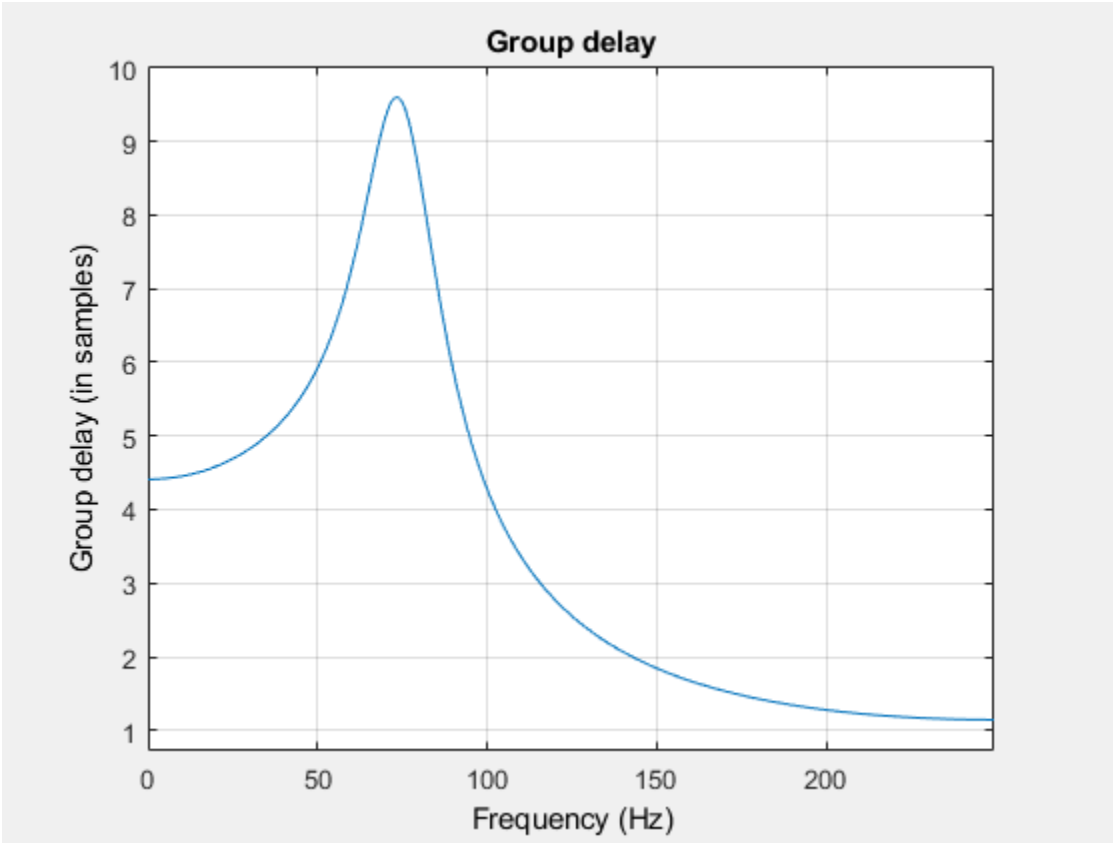
Filter the signal. The filtered signal is cleaner than the original, but lags in time with respect to it. It is also distorted due to the nonlinear phase of the filter.

```
xfilter = filter(iir,xn);

plot(tn,xn,tn,xfilter)

title 'Electrocardiogram'
xlabel 'Time (s)'
legend('Original','Filtered')
axis([0.25 0.55 -1 1.5])
grid
```



A look at the group delay introduced by the filter shows that the delay is frequency-dependent.

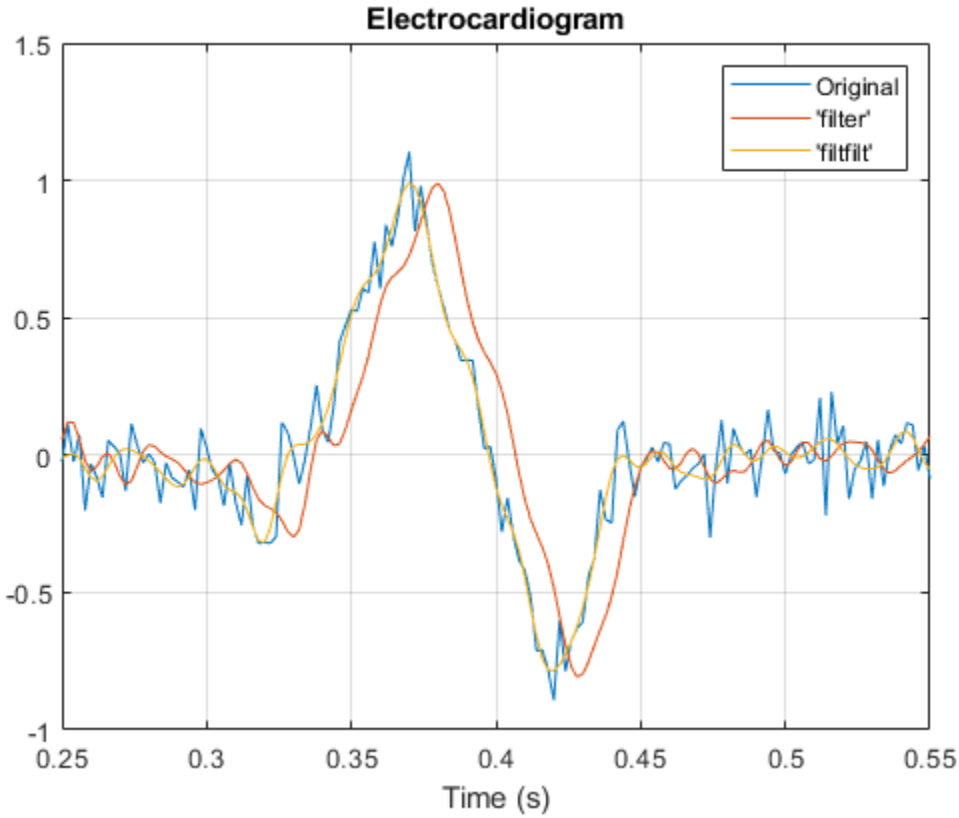
```
grpdelay(iir,N,Fs)
```



Filter the signal using `filtfilt`. The delay and distortion have been effectively removed. Use `filtfilt` when it is critical to keep the phase information of a signal intact.

```
xfiltfilt = filtfilt(iir,xn);

plot(tn,xn)
hold on
plot(tn,xfilter)
plot(tn,xfiltfilt)
```

```
title 'Electrocardiogram'
xlabel 'Time (s)'
legend('Original','''filter''','''filtfilt''')
axis([0.25 0.55 -1 1.5])
grid
```



Comparison of Analog IIR Lowpass Filters

Design a 5th-order analog Butterworth lowpass filter with a cutoff frequency of 2 GHz.

Multiply by 2π to convert the frequency to radians per second. Compute the frequency response of the filter at 4096 points.

```
n = 5;
f = 2e9;

[zb,pb,kb] = butter(n,2*pi*f,'s');
[bb,ab] = zp2tf(zb,pb,kb);
[hb,wb] = freqs(bb,ab,4096);
```

Design a 5th-order Chebyshev Type I filter with the same edge frequency and 3 dB of passband ripple. Compute its frequency response.

```
[z1,p1,k1] = cheby1(n,3,2*pi*f,'s');
[b1,a1] = zp2tf(z1,p1,k1);
[h1,w1] = freqs(b1,a1,4096);
```

Design a 5th-order Chebyshev Type II filter with the same edge frequency and 30 dB of stopband attenuation. Compute its frequency response.

```
[z2,p2,k2] = cheby2(n,30,2*pi*f,'s');
[b2,a2] = zp2tf(z2,p2,k2);
[h2,w2] = freqs(b2,a2,4096);
```

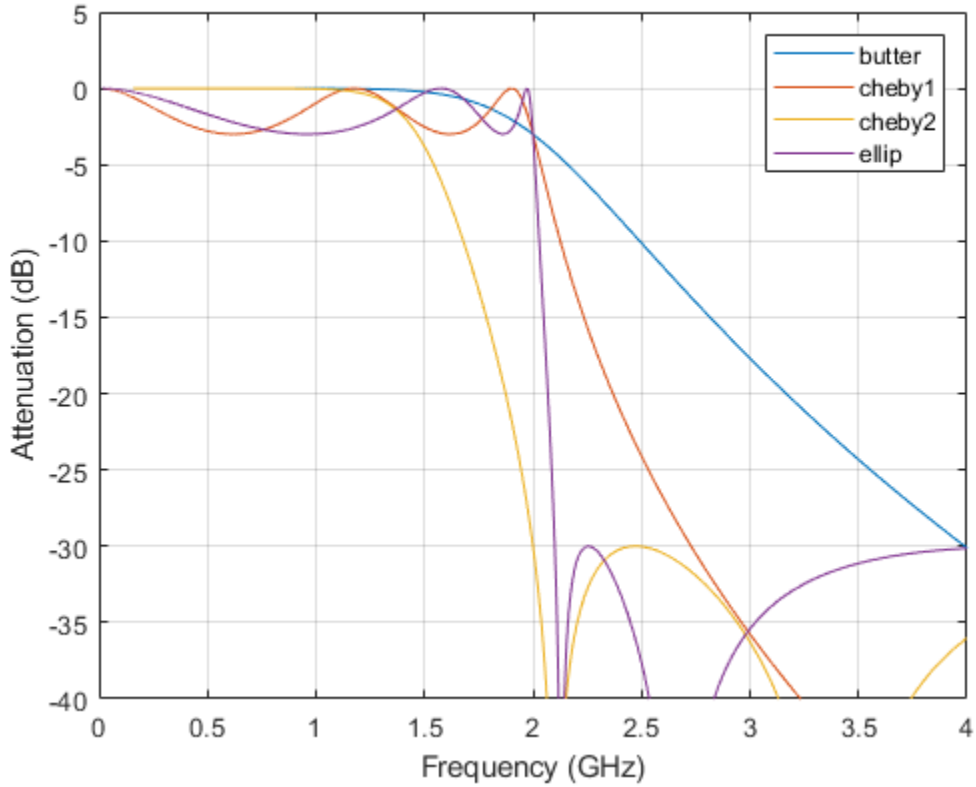
Design a 5th-order elliptic filter with the same edge frequency, 3 dB of passband ripple, and 30 dB of stopband attenuation. Compute its frequency response.

```
[ze,pe,ke] = ellip(n,3,30,2*pi*f,'s');
[be,ae] = zp2tf(ze,pe,ke);
[he,we] = freqs(be,ae,4096);
```

Plot the attenuation in decibels. Express the frequency in gigahertz. Compare the filters.

```
plot(wb/(2e9*pi),mag2db(abs(hb)))
hold on
plot(w1/(2e9*pi),mag2db(abs(h1)))
plot(w2/(2e9*pi),mag2db(abs(h2)))
plot(we/(2e9*pi),mag2db(abs(he)))
axis([0 4 -40 5])
grid
```

```
xlabel('Frequency (GHz)')
ylabel('Attenuation (dB)')
legend('butter','cheby1','cheby2','ellip')
```

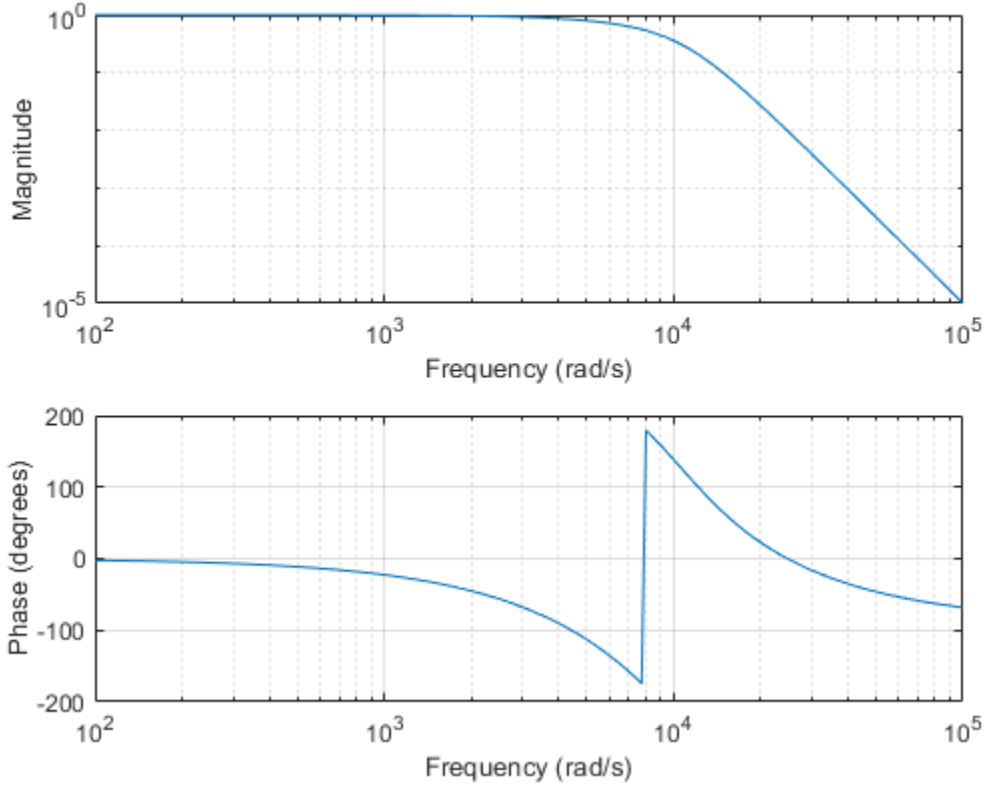


The Butterworth and Chebyshev Type II filters have flat passbands and wide transition bands. The Chebyshev Type I and elliptic filters roll off faster but have passband ripple. The frequency input to the Chebyshev Type II design function sets the beginning of the stopband rather than the end of the passband.

Frequency Response of a Lowpass Bessel Filter

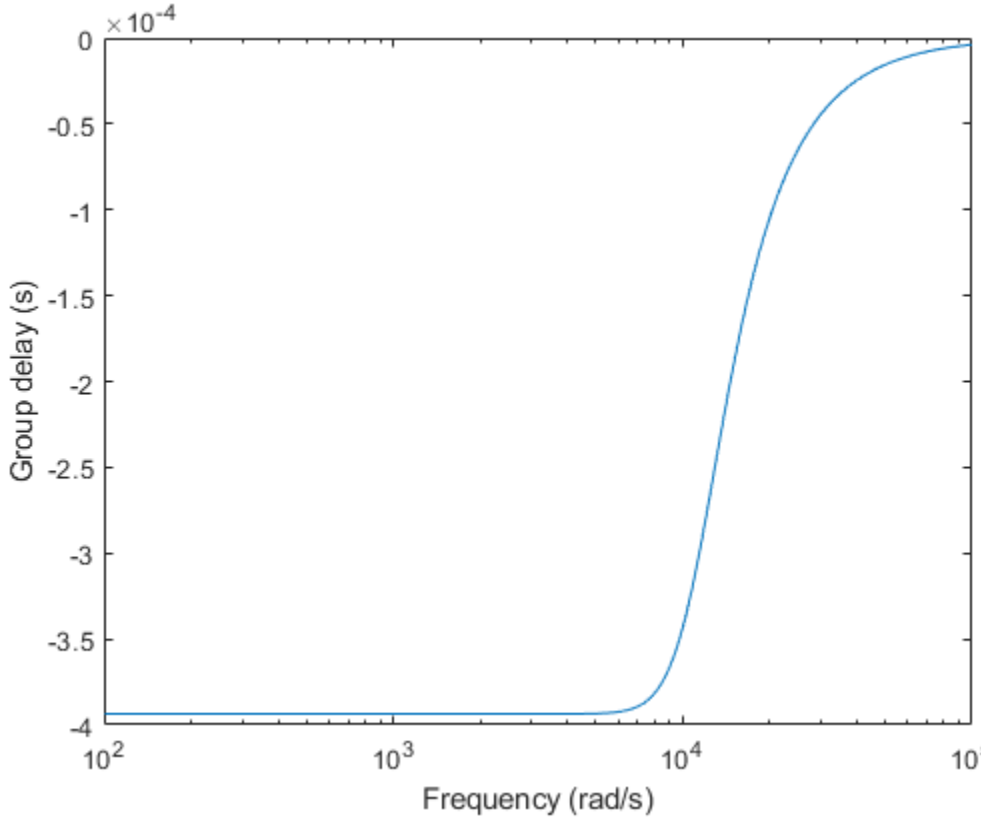
Design a 5th-order analog lowpass Bessel filter with approximately constant group delay up to 10^4 rad/s. Plot the magnitude and phase responses of the filter using `freqs`.

```
[b,a] = besself(5,10000);
freqs(b,a)
```



Compute the group delay response of the filter as the derivative of the unwrapped phase response. Plot the group delay to verify that it is approximately constant up to the cutoff frequency.

```
[h,w] = freqs(b,a,1000);
grpdel = diff(unwrap(angle(h)))./diff(w);

clf
semilogx(w(2:end),grpdel)
xlabel('Frequency (rad/s)')
ylabel('Group delay (s)')
```



Speaker Crossover Filters

This example shows how to devise a simple model of a digital three-way loudspeaker. The system splits the audio input into low-, mid-, and high-frequency bands that correspond respectively to the woofer, the midrange driver, and the tweeter. Typical values for the normalized crossover frequencies that delimit the bands are 0.136π rad/sample and 0.317π rad/sample.

Create lowpass, bandpass, and highpass filters to generate the low-frequency, mid-frequency, and high-frequency bands. Specify the frequencies.

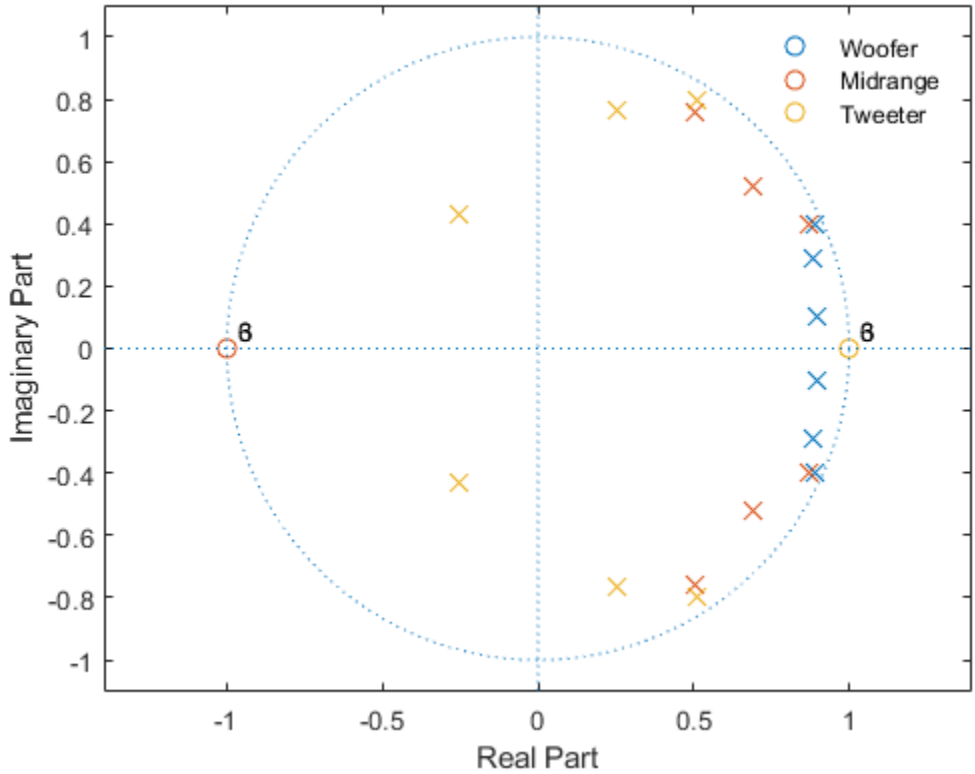
```
lo = 0.136;
hi = 0.317;
```

Use a 6th-order Chebyshev Type I design for each filter. Specify a passband ripple of 1 dB, larger than the value for real speakers. The `cheby1` function doubles the order of bandpass designs. Make all filters have the same order by halving the order of the bandpass filter. Return the zeros, poles, and gain of each filter.

```
ord = 6;
rip = 1;

[zw,pw,kw] = cheby1(ord,rip,lo);
[zm,pm,km] = cheby1(ord/2,rip,[lo hi]);
[zt,pt,kt] = cheby1(ord,rip,hi,'high');
```

Visualize the zeros and poles of the filters.

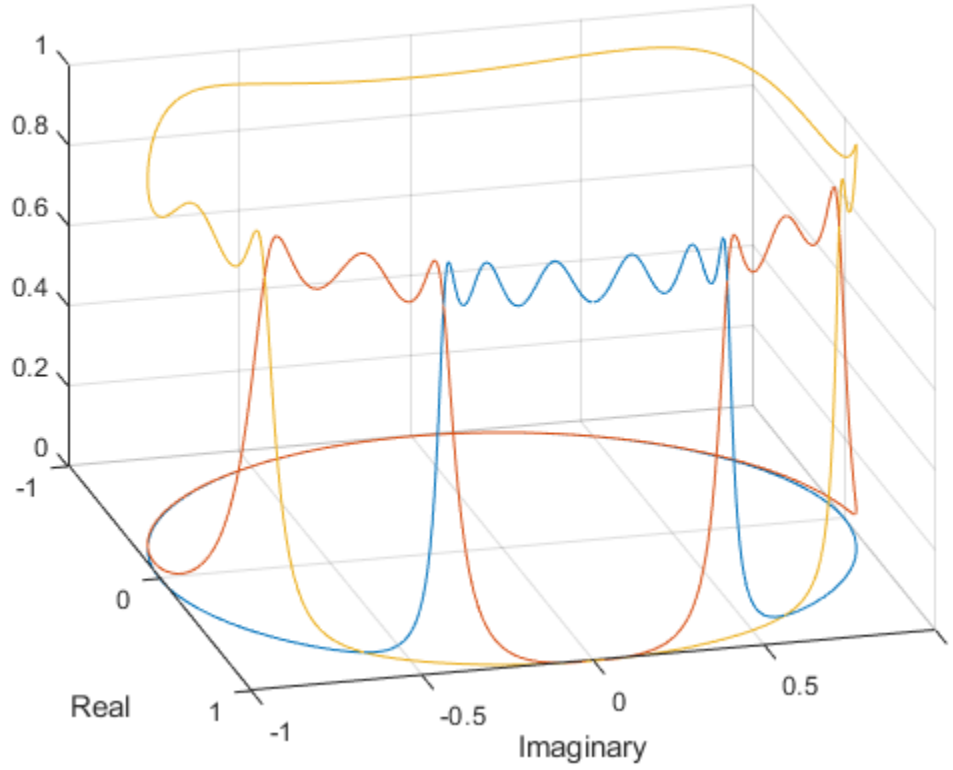
```
zplane([zw zm zt],[pw pm pt])
lg = legend('Woofer','Midrange','Tweeter');
lg.Box = 'off';
```



- *Woofer:* The zeros at $z = -1$ suppress high frequencies. The poles enhance the magnitude response between 0 and the lower crossover frequency.
- *Midrange:* The zeros at $z = 0$ and $z = 1$ suppress high and low frequencies. The poles enhance the magnitude response between the lower and higher crossover frequencies.
- *Tweeter:* The zeros at $z = 1$ suppress low frequencies. The poles enhance the magnitude response between the higher crossover frequency and π.

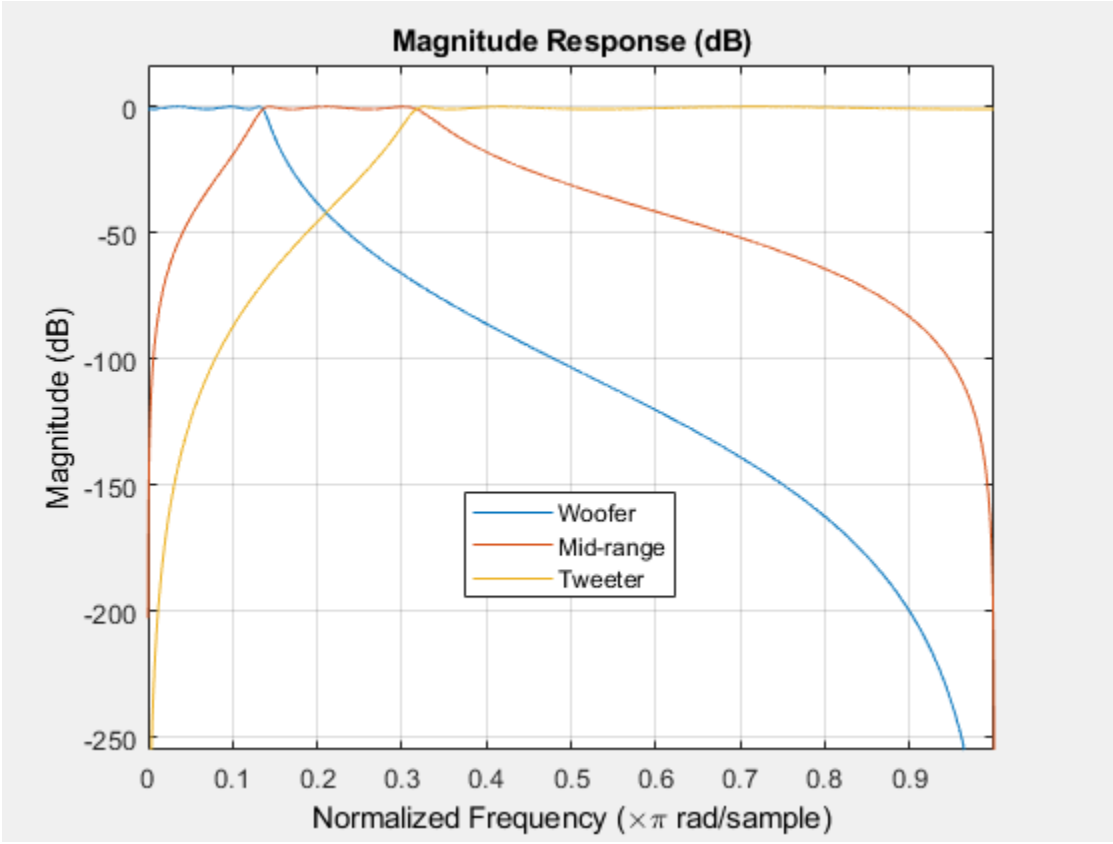
Plot the magnitude responses on the unit circle to see the effect of the different poles and zeros. Use linear units. Represent the filters as second-order sections.

```
sw = zp2sos(zw,pw,kw);
sm = zp2sos(zm,pm,km);
st = zp2sos(zt,pt,kt);

nf = 1024;
[hw,fw] = freqz(sw,nf,'whole');
hm = freqz(sm,nf,'whole');
ht = freqz(st,nf,'whole');

plot3(cos(fw),sin(fw),[abs(hw) abs(hm) abs(ht)])
xlabel('Real')
ylabel('Imaginary')
view(75,30)
grid
```



Plot the magnitude responses in dB using `fvtool`.

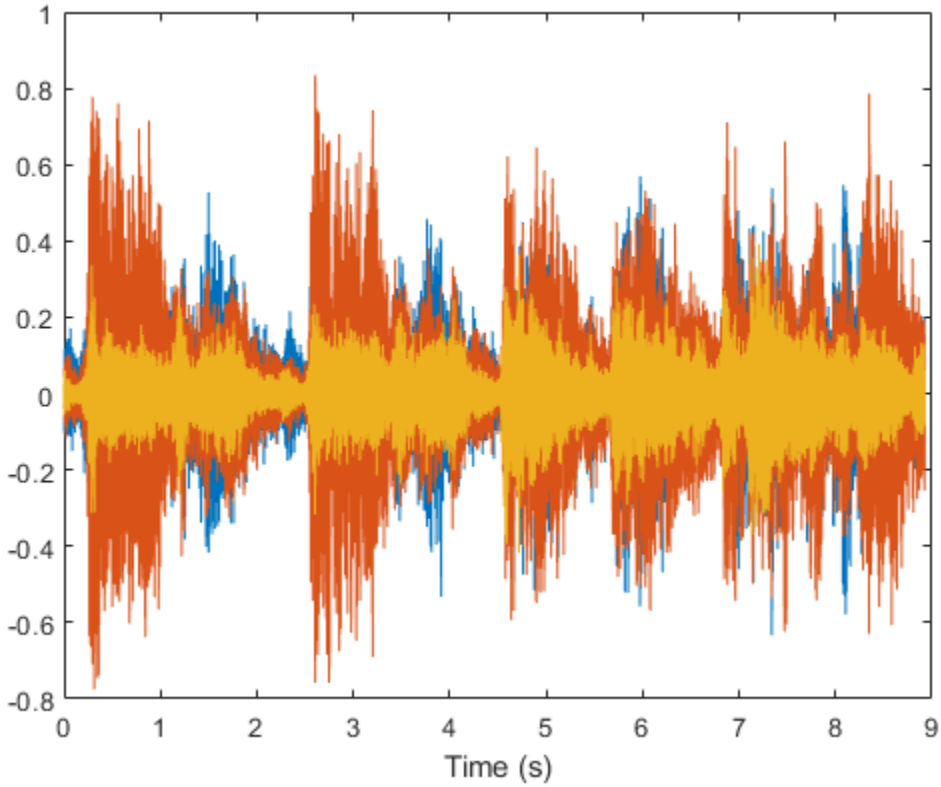
```
hfvt = fvtool(sw,sm,st);
legend(hfvt,'Woofer','Mid-range','Tweeter')
```



Load an audio file containing a fragment of Handel's "Hallelujah Chorus" sampled at 8192 Hz. Split the signal into three frequency bands by filtering. Plot the bands.

```
load handel                 % To hear, type soundsc(y,Fs)

yw = sosfilt(sw,y);         % To hear, type soundsc(yw,Fs)
ym = sosfilt(sm,y);         % To hear, type soundsc(ym,Fs)
yt = sosfilt(st,y);         % To hear, type soundsc(yt,Fs)
```

```
plot((0:length(y)-1)/Fs,[yw ym yt])
xlabel('Time (s)')
```




```
% To hear all the frequency ranges, type soundsc(yw+ym+yt,Fs)
```

References

Orfanidis, Sophocles J. *Introduction to Signal Processing*. Englewood Cliffs, NJ: Prentice Hall, 1996.

5

Filter Designer: A Filter Design and Analysis App

- “Filter Design Methods” on page 5-2
- “Using the Filter Designer App” on page 5-4
- “Analyzing Filter Responses” on page 5-5
- “Filter Designer App Panels” on page 5-6
- “Getting Help” on page 5-7
- “Getting Started with Filter Designer” on page 5-8
- “Importing a Filter Design” on page 5-26
- “FIR Bandpass Filter with Asymmetric Attenuation” on page 5-29
- “Arbitrary Magnitude Filter” on page 5-32

Filter Design Methods

The **Filter Designer** app is a user interface for designing and analyzing filters quickly. The app enables you to design digital FIR or IIR filters by setting filter specifications, by importing filters from your MATLAB workspace, or by adding, moving or deleting poles and zeros. It also provides tools for analyzing filters, such as magnitude and phase response and pole-zero plots.

The **Filter Designer** app gives you access to the following Signal Processing Toolbox filter design methods.

Design Method	Function
Butterworth	`butter`
Chebyshev Type I	`cheby1`
Chebyshev Type II	`cheby2`
Elliptic	`ellip`
Maximally Flat	`maxflat`
Equiripple	`firpm`
Least-squares	`firls`
Constrained least-squares	`fircls`
Complex equiripple	`cfirpm`
Window	`fir1`

When using the `window` method, all Signal Processing Toolbox window functions are available, and you can specify a user-defined window by entering its function name and input parameter.

Advanced Filter Design Methods

The following advanced filter design methods are available if you have DSP System Toolbox software.

Design Method	Function
Constrained equiripple FIR	`firceqrip`

Design Method	Function
Constrained-band equiripple FIR	`fircband`
Generalized remez FIR	`firgr`
Equiripple halfband FIR	`firhalfband`
Least P-norm optimal FIR	`firlpnorm`
Equiripple Nyquist FIR	`firnyquist`
Interpolated FIR	`ifir`
IIR comb notching or peaking	`iircomb`
Allpass filter (given group delay)	`iirgrpdelay`
Least P-norm optimal IIR	`iirlpnorm`
Constrained least P-norm IIR	`iirlpnormc`
Second-order IIR notch	`iirnotch`
Second-order IIR peaking (resonator)	`iirpeak`

Using the Filter Designer App

There are different ways that you can design filters using the **Filter Designer** app. For example:

- You can first choose a response type, such as bandpass, and then choose from the available FIR or IIR filter design methods.
- You can specify the filter by its type alone, along with certain frequency- or time-domain specifications such as passband frequencies and stopband frequencies. The filter you design is then computed using the default filter design method and filter order.

Analyzing Filter Responses

Once you have designed your filter, you can display the filter coefficients and detailed filter information, export the coefficients to the MATLAB workspace, create a C header file containing the coefficients, and analyze different filter responses in the app or in a separate Filter Visualization Tool (`fvtool`). The following filter responses are available:

- Magnitude response (`freqz`)
- Phase response (`phasez`)
- Group delay (`grpdelay`)
- Phase delay (`phasedelay`)
- Impulse response (`impz`)
- Step response (`stepz`)
- Pole-zero plots (`zplane`)
- Zero-phase response (`zerophase`)

Filter Designer App Panels

The **Filter Designer** app has sidebar buttons that display particular panels in the lower half. The panels are:

- Design Filter. See “Choosing a Filter Design Method” on page 5-10 for more information. You use this panel to
  - Design filters from scratch.
  - Modify existing filters designed with the app.
  - Analyze filters.
- Import filter. You use this panel to
  - Import previously saved filters or filter coefficients that you have stored in the MATLAB workspace.
  - Analyze imported filters.
- Pole/Zero Editor. See “Editing the Filter Using the Pole/Zero Editor” on page 5-15. You use this panel to add, delete, and move poles and zeros in your filter design.

If you also have DSP System Toolbox product installed, additional panels are available:

- Set quantization parameters — Use this panel to quantize double-precision filters that you design with **Filter Designer**, quantize double-precision filters that you import into the app, and analyze quantized filters.
- Transform filter — Use this panel to change a filter from one response type to another.
- Multirate filter design — Use this panel to create a multirate filter from your existing FIR design, create CIC filters, and linear and hold interpolators.

If you have Simulink installed, this panel is available:

- Realize Model — Use this panel to create a Simulink block containing the filter structure.

Getting Help

At any time, you can right-click or click the **What's this?** button, , to get information. You can also use the **Help** menu to see complete Help information.

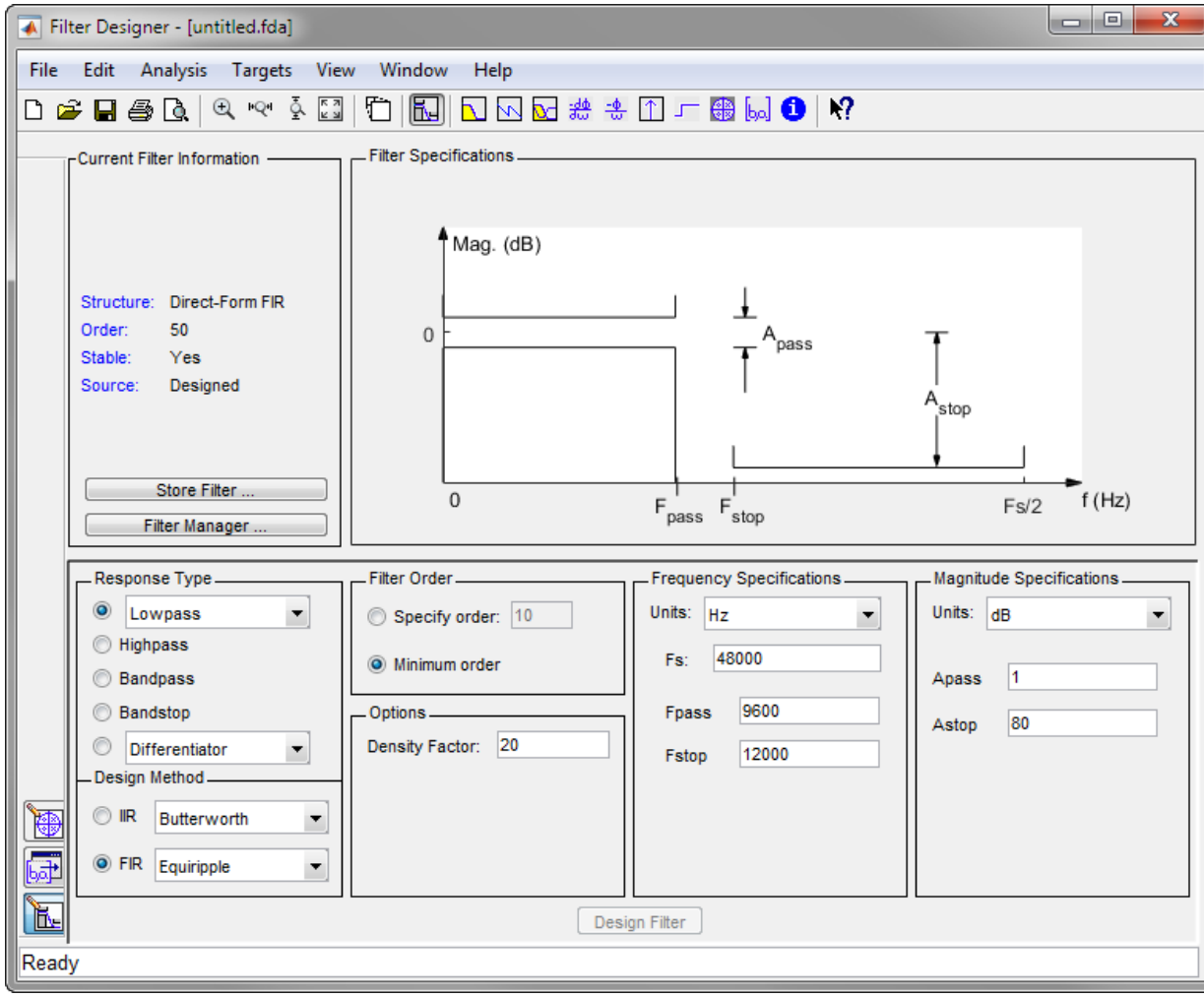
Getting Started with Filter Designer

To open the **Filter Designer** app, type

```
filterDesigner
```

at the MATLAB command prompt.

The **Filter Designer** app opens with the Design Filter panel displayed.



Note that when you open **Filter Designer**, **Design Filter** is not enabled. You must make a change to the default filter design in order to enable **Design Filter**. This is true each time you want to change the filter design. Changes to radio button items or drop down menu items such as those under **Response Type** or **Filter Order** enable **Design Filter** immediately. Changes to specifications in text boxes such as **Fs**, **Fpass**, and **Fstop** require you to click outside the text box to enable **Design Filter**.

Choosing a Response Type

You can choose from several response types:

- Lowpass
- Raised cosine
- Highpass
- Bandpass
- Bandstop
- Differentiator
- Multiband
- Hilbert transformer
- Arbitrary magnitude

Additional response types are available if you have DSP System Toolbox software installed.

Note Not all filter design methods are available for all response types. Once you choose your response type, this may restrict the filter design methods available to you. Filter design methods that are not available for a selected response type are removed from the Design Method region of the app.

Choosing a Filter Design Method

You can use the default filter design method for the response type that you've selected, or you can select a filter design method from the available FIR and IIR methods listed in the app.

To select the Remez algorithm to compute FIR filter coefficients, select the **FIR** radio button and choose `Equiripple` from the list of methods.

Setting the Filter Design Specifications

Viewing Filter Specifications

The filter design specifications that you can set vary according to response type and design method. The display region illustrates filter specifications when you select

Analysis > Filter Specifications or when you click the **Filter Specifications** toolbar button.

You can also view the filter specifications on the Magnitude plot of a designed filter by selecting **View > Specification Mask**.

Filter Order

You have two mutually exclusive options for determining the filter order when you design an equiripple filter:

- **Specify order**: You enter the filter order in a text box.
- **Minimum order**: The filter design method determines the minimum order filter.

Note that filter order specification options depend on the filter design method you choose. Some filter methods may not have both options available.

Options

The available options depend on the selected filter design method. Only the FIR Equiripple and FIR Window design methods have settable options. For FIR Equiripple, the option is a **Density Factor**. See `firpm` for more information. For FIR Window the options are **Scale Passband**, **Window** selection, and for the following windows, a settable parameter:

Window	Parameter
Chebyshev (`chebwin`)	Sidelobe attenuation
Gaussian (`gausswin`)	Alpha
Kaiser (`kaiser`)	Beta
Taylor (`taylorwin`)	Nbar and Sidelobe level
Tukey (`tukeywin`)	Alpha
User Defined	Function Name, Parameter

You can view the window in the Window Visualization Tool (`wvtool`) by clicking the **View** button.

Bandpass Filter Frequency Specifications

For a bandpass filter, you can set

- Units of frequency:
  - Hz
  - kHz
  - MHz
  - Normalized (0 to 1)
- Sampling frequency
- Passband frequencies
- Stopband frequencies

You specify the passband with two frequencies. The first frequency determines the lower edge of the passband, and the second frequency determines the upper edge of the passband.

Similarly, you specify the stopband with two frequencies. The first frequency determines the upper edge of the first stopband, and the second frequency determines the lower edge of the second stopband.

Bandpass Filter Magnitude Specifications

For a bandpass filter, you can specify the following magnitude response characteristics:

- Units for the magnitude response (dB or linear)
- Passband ripple
- Stopband attenuation

Computing the Filter Coefficients

Now that you've specified the filter design, click the **Design Filter** button to compute the filter coefficients.

Note The **Design Filter** button is disabled once you've computed the coefficients for your filter design. This button is enabled again once you make any changes to the filter specifications.

Analyzing the Filter

Displaying Filter Responses

You can view the following filter response characteristics in the display region or in a separate window.

- Magnitude response
- Phase response
- Magnitude and Phase responses
- Group delay response
- Phase delay response
- Impulse response
- Step response
- Pole-zero plot
- Zero-phase response — available from the *y*-axis context menu in a Magnitude or Magnitude and Phase response plot.

Note If you have DSP System Toolbox product installed, two other analyses are available: magnitude response estimate and round-off noise power. These two analyses are the only ones that use filter internals.

For descriptions of the above responses and their associated toolbar buttons and other **Filter Designer** toolbar buttons, see `fvtool`.

You can display two responses in the same plot by selecting **Analysis > Overlay Analysis** and selecting an available response. A second *y*-axis is added to the right side of the response plot. (Note that not all responses can be overlaid on each other.)

You can also display the filter coefficients and detailed filter information in this region.

For all the analysis methods, except zero-phase response, you can access them from the **Analysis** menu, the Analysis Parameters dialog box from the context menu, or by using the toolbar buttons. For zero-phase, right-click the *y*-axis of the plot and select **Zero-phase** from the context menu.

You can overlay the filter specifications on the Magnitude plot by selecting **View > Specification Mask**.

Using Data Tips

You can click the response to add plot data tips that display information about particular points on the response.

For information on using data tips, see “Interactively Explore Plotted Data” (MATLAB).

Drawing Spectral Masks

To add spectral masks or rejection area lines to your magnitude plot, click **View > User-defined Spectral Mask**.

The mask is defined by a frequency vector and a magnitude vector. These vectors must be the same length.

- **Enable Mask** — Select to turn on the mask display.
- **Normalized Frequency** — Select to normalize the frequency between 0 and 1 across the displayed frequency range.
- **Frequency Vector** — Enter a vector of *x*-axis frequency values.
- **Magnitude Units** — Select the desired magnitude units. These units should match the units used in the magnitude plot.
- **Magnitude Vector** — Enter a vector of *y*-axis magnitude values.

Changing the Sampling Frequency

To change the sampling frequency of your filter, right-click any filter response plot and select **Sampling Frequency** from the context menu.

To change the filter name, type the new name in **Filter name**. (In `fvtool`, if you have multiple filters, select the desired filter and then enter the new name.)

To change the sampling frequency, select the desired unit from **Units** and enter the sampling frequency in **Fs**. (For each filter in `fvtool`, you can specify a different sampling frequency or you can apply the sampling frequency to all filters.)

To save the displayed parameters as the default values to use when **Filter Designer** or FVTool is opened, click **Save as Default**.

To restore the default values, click **Restore Original Defaults**.

Displaying the Response in FVTool

To display the filter response characteristics in a separate window, select **View > Filter Visualization Tool** (available if any analysis, except the filter specifications, is in the display region) or click the **Full View Analysis** button. This starts the Filter Visualization Tool (`fvtool`).

Note If Filter Specifications are shown in the display region, clicking the **Full View Analysis** toolbar button launches a MATLAB figure window instead of FVTool. The associated menu item is **Print to Figure**, which is enabled only if the filter specifications are displayed.

You can use this tool to annotate your design, view other filter characteristics, and print your filter response. You can link **Filter Designer** and FVTool so that changes made in **Filter Designer** are immediately reflected in FVTool. See `fvtool` for more information.

Editing the Filter Using the Pole/Zero Editor

Displaying the Pole-Zero Plot

You can edit a designed or imported filter's coefficients by moving, deleting, or adding poles and/or zeros using the Pole/Zero Editor panel.

Note You cannot generate MATLAB code (**File > Generate MATLAB code**) if your filter was designed or edited with the Pole/Zero Editor.

You cannot move quantized poles and zeros. You can only move the reference poles and zeros.

Click the **Pole/Zero Editor** button in the sidebar or select **Edit > Pole/Zero Editor** to display the Pole/Zero Editor panel.

Poles are shown using x symbols and zeros are shown using o symbols.

Changing the Pole-Zero Plot

Plot mode buttons are located to the left of the pole/zero plot. Select one of the buttons to change the mode of the pole/zero plot. The Pole/Zero Editor has these buttons from left to right: **Move Pole/Zero**, **Add Pole**, **Add Zero**, and **Delete Pole/Zero**.

The following plot parameters and controls are located to the left of the pole/zero plot and below the plot mode buttons.

- **Filter gain** — factor to compensate for the filter's pole(s) and zero(s) gains
- **Coordinates** — units (`Polar` or `Rectangular`) of the selected pole or zero
- **Magnitude** — if polar coordinates is selected, magnitude of the selected pole or zero
- **Angle** — if polar coordinates is selected, angle of selected pole(s) or zero(s)
- **Real** — if rectangular coordinates is selected, real component of selected pole(s) or zero(s)
- **Imaginary** — if rectangular coordinates is selected, imaginary component of selected pole or zero
- **Section** — for multisection filters, number of the current section
- **Conjugate** — creates a corresponding conjugate pole or zero or automatically selects the conjugate pole or zero if it already exists.
- **Auto update** — immediately updates the displayed magnitude response when poles or zeros are added, moved, or deleted.

The **Edit > Pole/Zero Editor** has items for selecting multiple poles/zeros, for inverting and mirroring poles/zeros, and for deleting, scaling and rotating poles/zeros.

- When you select a pole or zero from a conjugate pair, the **Conjugate** check box and the conjugate are automatically selected.

Converting the Filter Structure

Converting to a New Structure

You can use **Edit > Convert Structure** to convert the current filter to a new structure. All filters can be converted to the following representations:

- Direct-form I
- Direct-form II

- Direct-form I transposed
- Direct-form II transposed
- Lattice ARMA

Note If you have DSP System Toolbox product installed, you will see additional structures in the Convert structure dialog box.

In addition, the following conversions are available for particular classes of filters:

- Minimum phase FIR filters can be converted to Lattice minimum phase
- Maximum phase FIR filters can be converted to Lattice maximum phase
- Allpass filters can be converted to Lattice allpass
- IIR filters can be converted to Lattice ARMA

Note Converting from one filter structure to another may produce a result with different characteristics than the original. This is due to the computer's finite-precision arithmetic and the variations in the conversion's round-off computations.

For example:

- Select **Edit > Convert Structure** to open the Convert structure dialog box.
- Select `Direct-form I` in the list of filter structures.

Converting to Second-Order Sections

You can use **Edit > Convert to Second-Order Sections** to store the converted filter structure as a collection of second-order sections rather than as a monolithic higher-order structure.

Note The following options are also used for **Edit > Reorder and Scale Second-Order Sections**, which you use to modify an SOS filter structure.

The following **Scale** options are available when converting a direct-form II structure only:

- `None` (default)

- `L-2` (L^2 norm)
- `L-infinity` (L^∞ norm)

The **Direction** (`Up` or `Down`) determines the ordering of the second-order sections. The optimal ordering changes depending on the **Scale** option selected.

For example:

- Select **Edit** > **Convert to Second-Order Sections** to open the Convert to SOS dialog box.
- Select `L-infinity` from the **Scale** menu for L^∞ norm scaling.
- Leave `Up` as the **Direction** option.

Note To convert from second-order sections back to a single section, use **Edit** > **Convert to Single Section**.

Exporting a Filter Design

Exporting Coefficients or Objects to the Workspace

You can save the filter either as filter coefficients variables or as a filter object variable. To save the filter to the MATLAB workspace:

1. Select **File** > **Export**. The Export dialog box appears.
2. Select `Workspace` from the **Export To** menu.
3. Select `Coefficients` from the **Export As** menu to save the filter coefficients or select `Objects` to save the filter in a filter object.
4. For coefficients, assign variable names using the **Numerator** (for FIR filters) or **Numerator** and **Denominator** (for IIR filters), or **SOS Matrix** and **Scale Values** (for IIR filters in second-order section form) text boxes in the Variable Names region.

   For objects, assign the variable name in the **Discrete Filter** text box. If you have variables with the same names in your workspace and you want to overwrite them, select the **Overwrite Variables** check box.
5. Click the **Export** button.

Exporting Coefficients to an ASCII File

To save filter coefficients to a text file,

1. Select **File** > **Export**. The Export dialog box appears.
2. Select `Coefficients File (ASCII)` from the **Export To** menu.
3. Click the **Export** button. The Export Filter Coefficients to .FCF File dialog box appears.
4. Choose or enter a filename and click the **Save** button.

The coefficients are saved in the text file that you specified, and the MATLAB Editor opens to display the file. The text file also contains comments with the MATLAB version number, the Signal Processing Toolbox version number, and filter information.

Exporting Coefficients or Objects to a MAT-File

To save filter coefficients or a filter object as variables in a MAT-file:

1. Select **File** > **Export**. The Export dialog box appears.
2. Select `MAT-file` from the **Export To** menu.
3. Select `Coefficients` from the **Export As** menu to save the filter coefficients or select `Objects` to save the filter in a filter object.
4. For coefficients, assign variable names using the **Numerator** (for FIR filters) or **Numerator** and **Denominator** (for IIR filters), or **SOS Matrix** and **Scale Values** (for IIR filters in second-order section form) text boxes in the Variable Names region.

   For objects, assign the variable name in the **Discrete Filter (**or **Quantized Filter)** text box. If you have variables with the same names in your workspace and you want to overwrite them, select the **Overwrite Variables** check box.
5. Click the **Export** button. The Export to a MAT-File dialog box appears.
6. Choose or enter a filename and click the **Save** button.

Exporting to a Simulink Model

If you have the Simulink product installed, you can export a Simulink block of your filter design and insert it into a new or existing Simulink model.

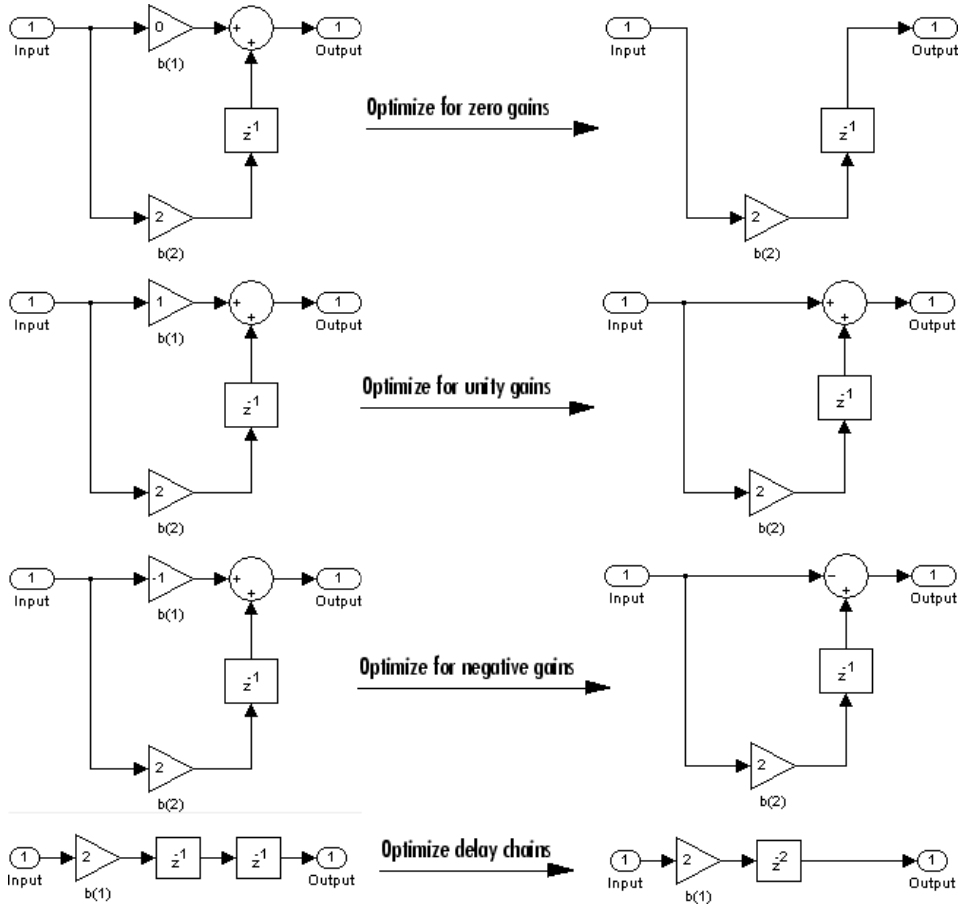
You can export a filter designed using any filter design method available in **Filter Designer**.

Note If you have the DSP System Toolbox and Fixed-Point Designer™ installed, you can export a CIC filter to a Simulink model.

1. After designing your filter, click the **Realize Model** sidebar button or select **File > Export to Simulink Model**. The Realize Model panel is displayed.
2. Specify the name to use for your block in **Block name**.
3. To insert the block into the current (most recently selected) Simulink model, set the **Destination** to `Current`. To inset the block into a new model, select `New`. To insert the block into a user-defined subsystem, select `User defined`.
4. If you want to overwrite a block previously created from this panel, check **Overwrite generated `Filter' block**.
5. If you select the **Build model using basic elements** check box, your filter is created as a subsystem (Simulink) block, which uses separate sub-elements. In this mode, the following optimization(s) are available:

   - `Optimize for zero gains` — Removes zero-valued gain paths from the filter structure.
   - `Optimize for unity gains` — Substitutes a wire (short circuit) for gains equal to 1 in the filter structure.
   - `Optimize for negative gains` — Substitutes a wire (short circuit) for gains equal to -1 and changes corresponding additions to subtractions in the filter structure.
   - `Optimize delay chains` — Substitutes delay chains composed of `n` unit delays with a single delay of `n`.
   - `Optimize for unity scale values` — Removes multiplications for scale values equal to 1 from the filter structure.

   The following illustration shows the effects of some of the optimizations:



Note The **Build model using basic elements** check box is enabled only when you have a DSP System Toolbox license and your filter can be designed using a Biquad Filter block or a Discrete FIR Filter block. For more information, see the Filter Realization Wizard topic in the DSP System Toolbox documentation.

6 Set the **Input processing** parameter to specify whether the generated filter performs sample- or frame-based processing on the input. Depending on the type of filter you design, one or both of the following options may be available:

- `Columns as channels (frame based)` — When you select this option, the block treats each column of the input as a separate channel.

- `Elements as channels (sample based)` — When you select this option, the block treats each element of the input as a separate channel.

7 Click the **Realize Model** button to create the filter block. When the **Build model using basic elements** check box is selected, **Filter Designer** implements the filter as a subsystem block using Sum, Gain, and Delay blocks.

If you double-click the Simulink Filter block, the filter structure is displayed.

Generating a C Header File

You may want to include filter information in an external C program. To create a C header file with variables that contain filter parameter data, follow this procedure:

1 Select **Targets > Generate C Header**. The Generate C Header dialog box appears.

2 Enter the variable names to be used in the C header file. The particular filter structure determines the variables that are created in the file.

Filter Structure	Variable Parameter
Direct-form I Direct-form II Direct-form I transposed Direct-form II transposed	**Numerator**, **Numerator length**, **Denominator**, **Denominator length**
Lattice ARMA	**Lattice coeff.**, **Lattice coeff. length**, **Ladder coeff.**, **Ladder coeff. length**
Lattice MA	**Lattice coeff.**, **Lattice coeff. length**, and **Number of sections** (inactive if filter has only one section)
Direct-form FIR Direct-form FIR transposed	**Numerator**, **Numerator length**, and **Number of sections** (inactive if filter has only one section)

Length variables contain the total number of coefficients of that type.

Note Variable names cannot be C language reserved words, such as “for.”

3 Select **Export Suggested** to use the suggested data type or select **Export As** and select the desired data type from the pull-down.

Note If you do not have DSP System Toolbox software installed, selecting any data type other than double-precision floating point results in a filter that does not exactly match the one you designed in the **Filter Designer**. This is due to rounding and truncating differences.

4 Click **Generate** to save the file and leave the dialog box open for additional C header file definitions. To close the dialog box, click **Close**.

Generating MATLAB Code

You can generate MATLAB code that constructs the filter you designed in **Filter Designer** from the command line. Select **File > Generate MATLAB Code > Filter Design Function** and specify the filename in the Generate MATLAB code dialog box.

Note You cannot generate MATLAB code (**File > Generate MATLAB Code > Filter Design Function**) if your filter was designed or edited with the Pole/Zero Editor.

The following is generated MATLAB code for the default lowpass filter in **Filter Designer**.

```
function Hd = ExFilter
%EXFILTER Returns a discrete-time filter object.

%
% MATLAB Code
% Generated by MATLAB(R) 7.11 and the Signal Processing Toolbox 6.14.
%
% Generated on: 17-Feb-2010 14:15:37
%

% Equiripple Lowpass filter designed using the FIRPM function.

% All frequency values are in Hz.
Fs = 48000;  % Sampling Frequency

Fpass = 9600;            % Passband Frequency
Fstop = 12000;           % Stopband Frequency
```

```
Dpass = 0.057501127785;  % Passband Ripple
Dstop = 0.0001;          % Stopband Attenuation
dens  = 20;              % Density Factor

% Calculate the order from the parameters using FIRPMORD.
[N, Fo, Ao, W] = firpmord([Fpass, Fstop]/(Fs/2), [1 0], [Dpass, Dstop]);

% Calculate the coefficients using the FIRPM function.
b  = firpm(N, Fo, Ao, W, {dens});
Hd = dfilt.dffir(b);

% [EOF]
```

Managing Filters in the Current Session

You can store filters designed in the current **Filter Designer** session for cascading together, exporting to FVTool or for recalling later in the same or future **Filter Designer** sessions.

You store and access saved filters with the **Store Filter** and **Filter Manager** buttons, respectively, in the **Current Filter Information** pane.

Store Filter — Displays the Store Filter dialog box in which you specify the filter name to use when storing the filter in the Filter Manager. The default name is the type of the filter.

Filter Manager — Opens the Filter Manager.

The current filter is listed below the listbox. To change the current filter, highlight the desired filter. If you select **Edit current filter**, **Filter Designer** displays the currently selected filter specifications. If you make any changes to the specifications, the stored filter is updated immediately.

To cascade two or more filters, highlight the desired filters and press **Cascade**. A new cascaded filter is added to the Filter Manager.

To change the name of a stored filter, press **Rename**. The Rename filter dialog box is displayed.

To remove a stored filter from the Filter Manager, press **Delete**.

To export one or more filters to FVTool, highlight the filter(s) and press **FVTool**.

Saving and Opening Filter Design Sessions

You can save your filter design session as a MAT-file and return to the same session another time.

Select the **Save Session** button to save your session as a MAT-file. The first time you save a session, a Save Filter Design Session browser opens, prompting you for a session name.

The `.fda` extension is added automatically to all filter design sessions you save.

Note You can also use **File > Save Session** and **File > Save Session As** to save a session.

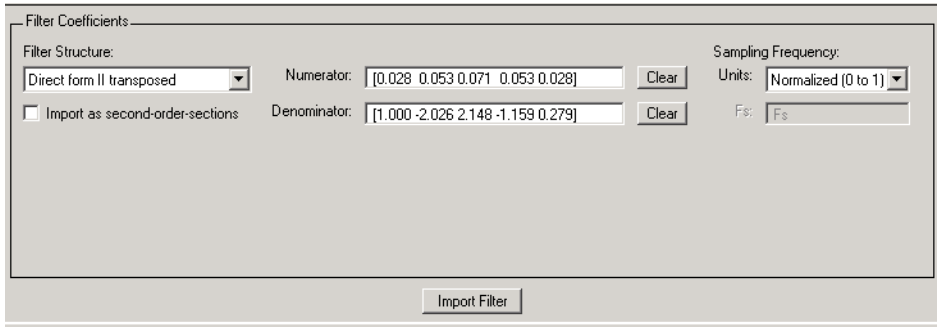
You can load existing sessions into **Filter Designer** by selecting the **Open Session** button or **File > Open Session** . A Load Filter Design Session browser opens that allows you to select from your previously saved filter design sessions.

Importing a Filter Design

In this section...
“Import Filter Panel” on page 5-26 “Filter Structures” on page 5-27

Import Filter Panel

The Import Filter panel allows you to import a filter. You can access this region by clicking the **Import Filter** button in the sidebar.



The imported filter can be in any of the representations listed in the **Filter Structure** pull-down menu. You can import a filter as second-order sections by selecting the check box.

Specify the filter coefficients in **Numerator** and **Denominator**, either by entering them explicitly or by referring to variables in the MATLAB workspace.

Select the frequency units from the following options in the **Units** menu, and for any frequency unit other than Normalized, specify the value or MATLAB workspace variable of the sampling frequency in the **Fs** field.

To import the filter, click the **Import Filter** button. The display region is automatically updated when the new filter has been imported.

You can edit the imported filter using the Pole/Zero Editor panel.

Filter Structures

The available filter structures are:

- Direct Form, which includes direct-form I, direct-form II, direct-form I transposed, direct-form II transposed, and direct-form FIR
- Lattice, which includes lattice allpass, lattice MA min phase, lattice MA max phase, and lattice ARMA
- Discrete–time Filter (`dfilt` object)

The structure that you choose determines the type of coefficients that you need to specify in the text fields to the right.

Direct-form

For direct-form I, direct-form II, direct-form I transposed, and direct-form II transposed, specify the filter by its transfer function representation

$$H(z) = \frac{b(1) + b(2)z^{-1} + b(3)z^{-2} + \ldots b(m+1)z^{-m}}{a(1) + a(2)z^{-1} + a(3)Z^{-3} + \ldots a(n+1)z^{-n}}$$

- The **Numerator** field specifies a variable name or value for the numerator coefficient vector `b`, which contains `m+1` coefficients in descending powers of z.
- The **Denominator** field specifies a variable name or value for the denominator coefficient vector `a`, which contains `n+1` coefficients in descending powers of z. For FIR filters, the **Denominator** is `1`.

Filters in transfer function form can be produced by all of the Signal Processing Toolbox filter design functions (such as `fir1`, `fir2`, `firpm`, `butter`, `yulewalk`). See “Transfer Function” on page 1-37 for more information.

Importing as second-order sections

For all direct-form structures, except direct-form FIR, you can import the filter in its second-order section representation:

$$H(z) = G \prod_{k=1}^{L} \frac{b_{0k} + b_{1k}z^{-1} + b_{2k}z^{-2}}{a_{0k} + a_{1k}z^{-1} + a_{2k}z^{-2}}$$

The **Gain** field specifies a variable name or a value for the gain G, and the **SOS Matrix** field specifies a variable name or a value for the L-by-6 SOS matrix

$$SOS = \begin{pmatrix} b_{01} & b_{11} & b_{21} & 1 & a_{11} & a_{22} \\ b_{02} & b_{12} & b_{22} & 1 & a_{12} & a_{22} \\ . & . & . & . & . & . \\ . & . & . & . & . & . \\ b_{0L} & b_{1L} & b_{2L} & 1 & a_{1L} & a_{2L} \end{pmatrix}$$

whose rows contain the numerator and denominator coefficients b_{ik} and a_{ik} of the second-order sections of $H(z)$.

Filters in second-order section form can be produced by functions such as `tf2sos`, `zp2sos`, `ss2sos`, and `sosfilt`. See “Second-Order Sections (SOS)” on page 1-40 for more information.

Lattice

For lattice allpass, lattice minimum and maximum phase, and lattice ARMA filters, specify the filter by its lattice representation:

- For lattice allpass, the **Lattice coeff** field specifies the lattice (reflection) coefficients, `k(1)` to `k(N)`, where `N` is the filter order.
- For lattice MA (minimum or maximum phase), the **Lattice coeff** field specifies the lattice (reflection) coefficients, `k(1)` to `k(N)`, where `N` is the filter order.
- For lattice ARMA, the **Lattice coeff** field specifies the lattice (reflection) coefficients, `k(1)` to `k(N)`, and the **Ladder coeff** field specifies the ladder coefficients, `v(1)` to `v(N+1)`, where `N` is the filter order.

Filters in lattice form can be produced by `tf2latc`. See “Lattice Structure” on page 1-41 for more information.

Discrete-time Filter (dfilt object)

For Discrete-time filter, specify the name of the `dfilt` object. See `dfilt` for more information.

FIR Bandpass Filter with Asymmetric Attenuation

Use the **Filter Designer** app to create a 50th-order equiripple FIR bandpass filter to be used with signals sampled at 1 kHz.

```
N = 50;
Fs = 1e3;
```

Specify that the passband spans frequencies of 200–300 Hz and that the transition region on either side has a width of 50 Hz.

```
Fstop1 = 150;
Fpass1 = 200;
Fpass2 = 300;
Fstop2 = 350;
```

Specify weights for the optimization fit:

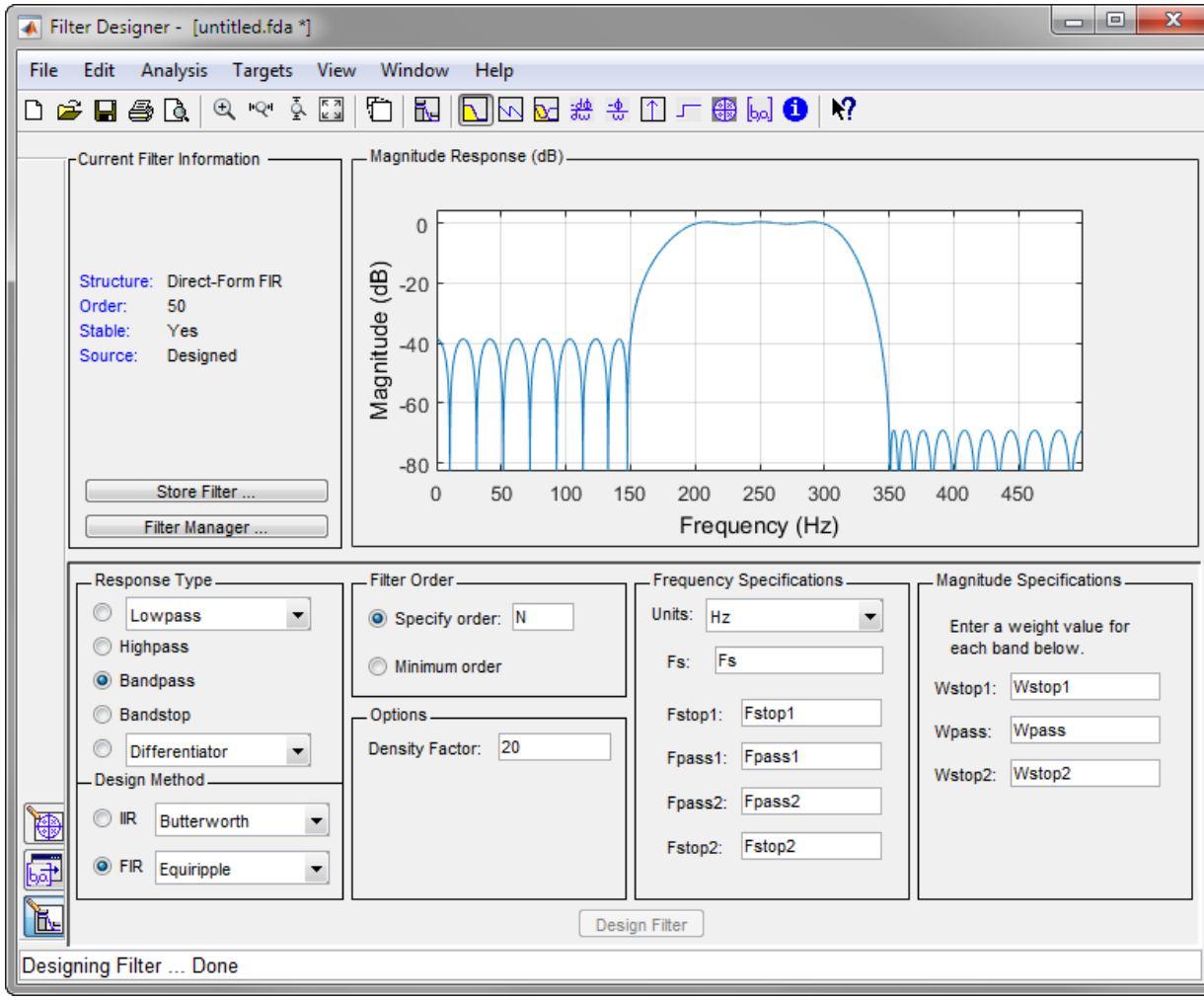
- 3 for the low-frequency stopband
- 1 for the passband
- 100 for the high-frequency stopband

Open the **Filter Designer** app.

```
Wstop1 = 3;
Wpass = 1;
Wstop2 = 100;

filterDesigner
```

Use the app to design the rest of the filter. To specify the frequency constraints and magnitude specifications, use the variables you created.

1. Set **Response Type** to `Bandpass`.
2. Set **Design Method** to `FIR`. From the drop-down list, select `Equiripple`.
3. Under **Filter Order**, specify the order as `N`.
4. Under **Frequency Specifications**, specify **Fs** as `Fs`.
5. Click **Design Filter**.



See Also

Apps
Filter Designer

Functions
`designfilt`

Arbitrary Magnitude Filter

Design an FIR filter with the following piecewise frequency response:

- A sinusoid between 0 and 0.19π rad/sample.

  ```
  F1 = 0:0.01:0.19;
  A1 = 0.5+sin(2*pi*7.5*F1)/4;
  ```

- A piecewise linear section between 0.2π rad/sample and 0.78π rad/sample.

  ```
  F2 = [0.2 0.38 0.4 0.55 0.562 0.585 0.6 0.78];
  A2 = [0.5 2.3 1 1 -0.2 -0.2 1 1];
  ```

- A quadratic section between 0.79π rad/sample and the Nyquist frequency.

  ```
  F3 = 0.79:0.01:1;
  A3 = 0.2+18*(1-F3).^2;
  ```

Specify a filter order of 50. Consolidate the frequency and amplitude vectors. To give all bands equal weights during the optimization fit, specify a weight vector of all ones. Open the **Filter Designer** app.

```
N = 50;

FreqVect = [F1 F2 F3];
AmplVect = [A1 A2 A3];
WghtVect = ones(1,N/2);

filterDesigner
```

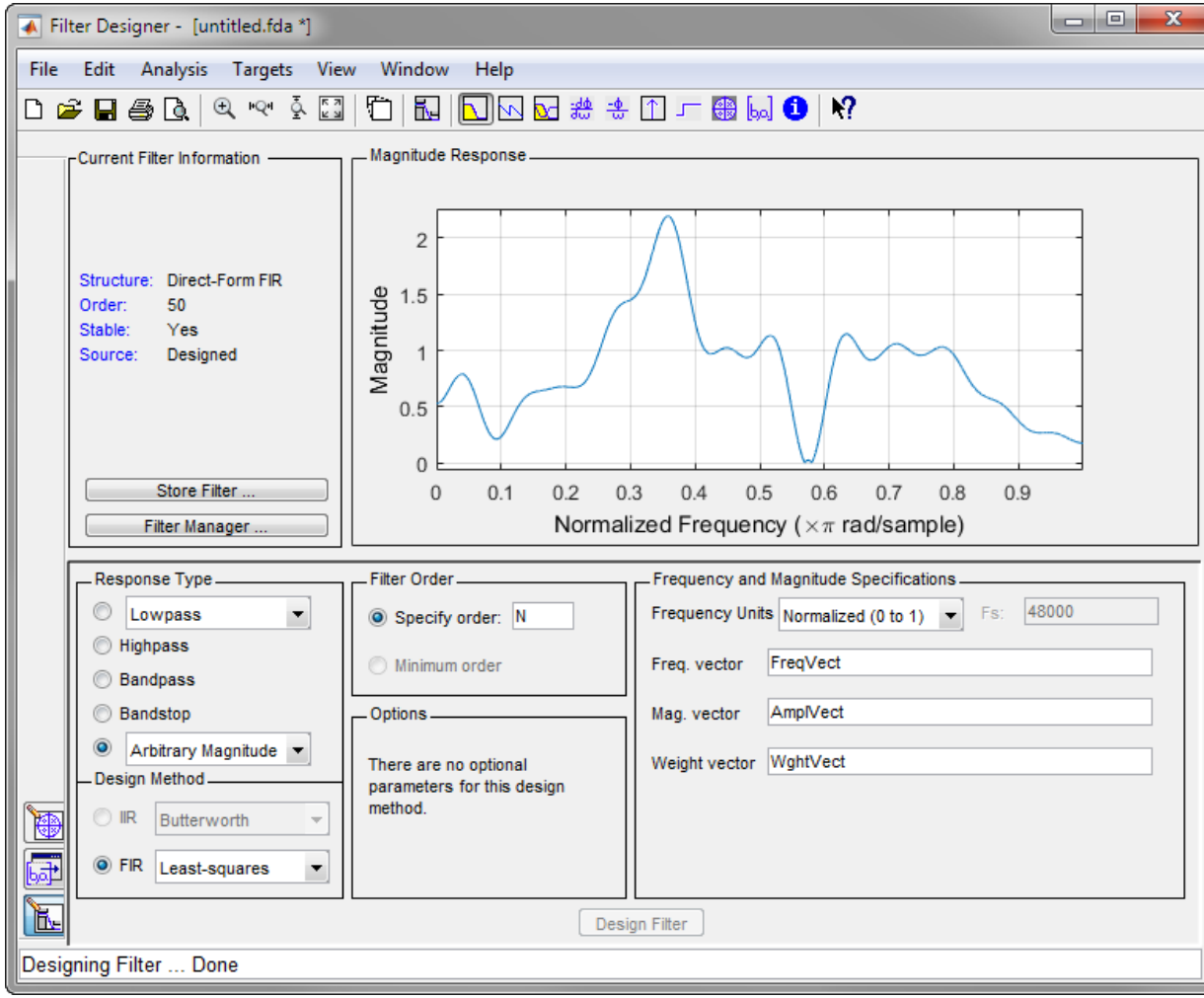
Use the app to design the filter.

1. Under **Response Type**, select the button next to `Differentiator`. From the drop-down list, choose `Arbitrary Magnitude`.
2. Set **Design Method** to `FIR`. From the drop-down list, select `Least-squares`.
3. Under **Filter Order**, specify the order as the variable `N`.
4. Under **Frequency and Magnitude Specifications**, specify the variables you created:
   - **Freq. vector** — `FreqVect`.
   - **Mag. vector** — `AmplVect`.

- **Weight vector** — `WghtVect`.

5 Click **Design Filter**.

6 Right-click the *y*-axis of the plot and select **Magnitude** to express the magnitude response in linear units.



See Also

Apps
Filter Designer

Functions
`designfilt`

Statistical Signal Processing

The following chapter discusses statistical signal processing tools and applications, including correlations, covariance, and spectral estimation.

- “Correlation and Covariance” on page 6-2
- “Spectral Analysis” on page 6-5
- “Nonparametric Methods” on page 6-9
- “Parametric Methods” on page 6-34
- “Selected Bibliography” on page 6-47

Correlation and Covariance

In this section...
“Background Information” on page 6-2
“Using xcorr and xcov Functions” on page 6-3
“Bias and Normalization” on page 6-3
“Multiple Channels” on page 6-4

Background Information

The cross-correlation sequence for two wide-sense stationary random process, $x(n)$ and $y(n)$ is

$$R_{xy}(m) = E\{x(n+m)y^*(n)\},$$

where the asterisk denotes the complex conjugate and the expectation is over the ensemble of realizations that constitute the random processes.

Note that cross-correlation is not commutative, but a Hermitian (conjugate) symmetry property holds such that:

$$R_{xy}(m) = R_{yx}^*(-m).$$

The cross-covariance between $x(n)$ and $y(n)$ is:

$$C_{xy}(m) = E\{(x(n+m) - \mu_x)(y(n) - \mu_y)^*\} = R_{xy}(m) - \mu_x\mu_y^*.$$

For zero-mean wide-sense stationary random processes, the cross-correlation and cross-covariance are equivalent.

In practice, you must estimate these sequences, because it is possible to access only a finite segment of the infinite-length random processes. Further, it is often necessary to estimate ensemble moments based on time averages because only a single realization of the random processes are available. A common estimate based on N samples of $x(n)$ and $y(n)$ is the deterministic cross-correlation sequence (also called the time-ambiguity function)

$$\hat{R}_{xy}(m) = \begin{cases} \sum_{n=0}^{N-m-1} x(n+m)y^*(n), & m \geq 0, \\ \hat{R}_{yx}^*(-m), & m < 0. \end{cases}$$

where we assume for this discussion that $x(n)$ and $y(n)$ are indexed from 0 to N – 1, and

$\hat{R}_{xy}(m)$ from –(N – 1) to N – 1.

Using xcorr and xcov Functions

The functions `xcorr` and `xcov` estimate the cross-correlation and cross-covariance sequences of random processes. They also handle autocorrelation and autocovariance as special cases. The `xcorr` function evaluates the sum shown above with an efficient FFT-based algorithm, given inputs $x(n)$ and $y(n)$ stored in length N vectors `x` and `y`. Its operation is equivalent to convolution with one of the two subsequences reversed in time.

For example:

```
x = [1 1 1 1 1]';
y = x;
xyc = xcorr(x,y)
```

Notice that the resulting sequence length is one less than twice the length of the input sequence. Thus, the Nth element is the correlation at lag 0. Also notice the triangular pulse of the output that results when convolving two square pulses.

The `xcov` function estimates autocovariance and cross-covariance sequences. This function has the same options and evaluates the same sum as `xcorr`, but first removes the means of `x` and `y`.

Bias and Normalization

An estimate of a quantity is *biased* if its expected value is not equal to the quantity it estimates. The expected value of the output of `xcorr` is

$$E\{R_{xy}(m)\} = (N - |m|)R_{xy}(m).$$

`xcorr` provides the unbiased estimate, dividing by $N - |m|$ when you specify an `'unbiased'` flag after the input sequences.

```
xcorr(x,y,'unbiased')
```

Although this estimate is unbiased, the end points (near $-(N - 1)$ and $N - 1$) suffer from large variance because `xcorr` computes them using only a few data points. A possible trade-off is to simply divide by N using the `'biased'` flag:

```
xcorr(x,y,'biased')
```

With this scheme, only the sample of the correlation at zero lag (the Nth output element) is unbiased. This estimate is often more desirable than the unbiased one because it avoids random large variations at the end points of the correlation sequence.

`xcorr` provides one other normalization scheme. The syntax

```
xcorr(x,y,'coeff')
```

divides the output by `norm(x)*norm(y)` so that, for autocorrelations, the sample at zero lag is 1.

Multiple Channels

For a multichannel signal, `xcorr` and `xcov` estimate the autocorrelation and cross-correlation and covariance sequences for all of the channels at once. If `S` is an M-by-N signal matrix representing N channels in its columns, `xcorr(S)` returns a $(2M - 1)$-by-N^2 matrix with the autocorrelations and cross-correlations of the channels of `S` in its N^2 columns. If `S` is a three-channel signal

```
S = [s1 s2 s3]
```

then the result of `xcorr(S)` is organized as

```
R = [Rs1s1 Rs1s2 Rs1s3 Rs2s1 Rs2s2 Rs2s3 Rs3s1 Rs3s2 Rs3s3]
```

Two related functions, `cov` and `corrcoef`, are available in the standard MATLAB environment. They estimate covariance and normalized covariance respectively between the different channels at lag 0 and arrange them in a square matrix.

Spectral Analysis

In this section...
“Background Information” on page 6-5 “Spectral Estimation Method” on page 6-6

Background Information

The goal of *spectral estimation* is to describe the distribution (over frequency) of the power contained in a signal, based on a finite set of data. Estimation of power spectra is useful in a variety of applications, including the detection of signals buried in wideband noise.

The *power spectral density* (PSD) of a stationary random process $x(n)$ is mathematically related to the autocorrelation sequence by the discrete-time Fourier transform. In terms of normalized frequency, this is given by

$$P_{xx}(\omega) = \frac{1}{2\pi} \sum_{m=-\infty}^{\infty} R_{xx}(m) e^{-j\omega m}.$$

This can be written as a function of physical frequency f (e.g., in hertz) by using the relation $\omega = 2\pi f / f_s$, where f_s is the sampling frequency:

$$P_{xx}(f) = \frac{1}{f_s} \sum_{m=-\infty}^{\infty} R_{xx}(m) e^{-j 2\pi m f / f_s}.$$

The correlation sequence can be derived from the PSD by use of the inverse discrete-time Fourier transform:

$$R_{xx}(m) = \int_{-\pi}^{\pi} P_{xx}(\omega) e^{j\omega m} d\omega = \int_{-f_s/2}^{f_s/2} P_{xx}(f) e^{j2\pi m f / f_s} \, df.$$

The average power of the sequence $x(n)$ over the entire Nyquist interval is represented by

$$R_{xx}(0) = \int_{-\pi}^{\pi} P_{xx}(\omega)d\omega = \int_{-f_s/2}^{f_s/2} P_{xx}(f)df.$$

The average power of a signal over a particular frequency band $[\omega_1, \omega_2]$, $0 \le \omega_1 \le \omega_2 \le \pi$, can be found by integrating the PSD over that band:

$$\bar{P}_{[\omega_1,\omega_2]} = \int_{\omega_1}^{\omega_2} P_{xx}(\omega)\,d\omega = \int_{-\omega_2}^{-\omega_1} P_{xx}(\omega)\,d\omega.$$

You can see from the above expression that $P_{xx}(\omega)$ represents the power content of a signal in an *infinitesimal* frequency band, which is why it is called the power spectral *density*.

The units of the PSD are power (e.g., watts) per unit of frequency. In the case of $P_{xx}(\omega)$, this is watts/radian/sample or simply watts/radian. In the case of $P_{xx}(f)$, the units are watts/hertz. Integration of the PSD with respect to frequency yields units of watts, as expected for the average power .

For real–valued signals, the PSD is symmetric about DC, and thus $P_{xx}(\omega)$ for $0 \le \omega \le \pi$ is sufficient to completely characterize the PSD. However, to obtain the average power over the entire Nyquist interval, it is necessary to introduce the concept of the *one-sided* PSD.

The one-sided PSD is given by

$$P_{\text{one-sided}}(\omega) = \begin{cases} 0, & -\pi \le \omega < 0, \\ 2P_{xx}(\omega), & 0 \le \omega \le \pi. \end{cases}$$

The average power of a signal over the frequency band, $[\omega_1,\omega_2]$ with $0 \le \omega_1 \le \omega_2 \le \pi$, can be computed using the one-sided PSD as

$$\bar{P}_{[\omega_1,\omega_2]} = \int_{\omega_1}^{\omega_2} P_{\text{one-sided}}(\omega)d\omega.$$

Spectral Estimation Method

The various methods of spectrum estimation available in the toolbox are categorized as follows:

- Nonparametric methods
- Parametric methods
- Subspace methods

Nonparametric methods are those in which the PSD is estimated directly from the signal itself. The simplest such method is the *periodogram*. Other nonparametric techniques such as *Welch's method* [8], the *multitaper method* (*MTM*) reduce the variance of the periodogram.

Parametric methods are those in which the PSD is estimated from a signal that is assumed to be output of a linear system driven by white noise. Examples are the *Yule-Walker autoregressive* (*AR*) *method* and the *Burg method*. These methods estimate the PSD by first estimating the parameters (coefficients) of the linear system that hypothetically generates the signal. They tend to produce better results than classical nonparametric methods when the data length of the available signal is relatively short. Parametric methods also produce smoother estimates of the PSD than nonparametric methods, but are subject to error from model misspecification.

Subspace methods, also known as *high-resolution methods* or *super-resolution methods*, generate frequency component estimates for a signal based on an eigenanalysis or eigendecomposition of the autocorrelation matrix. Examples are the *multiple signal classification* (*MUSIC*) *method* or the *eigenvector* (*EV*) *method*. These methods are best suited for line spectra — that is, spectra of sinusoidal signals — and are effective in the detection of sinusoids buried in noise, especially when the signal to noise ratios are low. The subspace methods do not yield true PSD estimates: they do not preserve process power between the time and frequency domains, and the autocorrelation sequence cannot be recovered by taking the inverse Fourier transform of the frequency estimate.

All three categories of methods are listed in the table below with the corresponding toolbox function names. More information about each function is on the corresponding function reference page. See “Parametric Modeling” on page 7-26 for details about `lpc` and other parametric estimation functions.

Spectral Estimation Methods/Functions

Method	Description	Functions
Periodogram	Power spectral density estimate	`periodogram`
Welch	Averaged periodograms of overlapped, windowed signal sections	`pwelch`, `cpsd`, `tfestimate`, `mscohere`
Multitaper	Spectral estimate from combination of multiple orthogonal windows (or “tapers”)	`pmtm`
Yule-Walker AR	Autoregressive (AR) spectral estimate of a time-series from its estimated autocorrelation function	`pyulear`
Burg	Autoregressive (AR) spectral estimation of a time-series by minimization of linear prediction errors	`pburg`
Covariance	Autoregressive (AR) spectral estimation of a time-series by minimization of the forward prediction errors	`pcov`
Modified Covariance	Autoregressive (AR) spectral estimation of a time-series by minimization of the forward and backward prediction errors	`pmcov`
MUSIC	Multiple signal classification	`pmusic`
Eigenvector	Pseudospectrum estimate	`peig`

Nonparametric Methods

The following sections discuss the periodogram on page 6-9, modified periodogram on page 6-18, Welch on page 6-21, and multitaper on page 6-25 methods of nonparametric estimation, along with the related CPSD function on page 6-28, transfer function estimate on page 6-29, and coherence function on page 6-31.

Periodogram

In general terms, one way of estimating the PSD of a process is to simply find the discrete-time Fourier transform of the samples of the process (usually done on a grid with an FFT) and appropriately scale the magnitude squared of the result. This estimate is called the *periodogram*.

The periodogram estimate of the PSD of a signal $x_L(n)$ of length L is

$$P_{xx}(f) = \frac{1}{LF_s}\left|\sum_{n=0}^{L-1} x_L(n) e^{-j2\pi fn/F_s}\right|^2,$$

where Fs is the sampling frequency.

In practice, the actual computation of $P_{xx}(f)$ can be performed only at a finite number of frequency points, and usually employs an FFT. Most implementations of the periodogram method compute the N-point PSD estimate at the frequencies

$$f_k = \frac{kF_s}{N}, \quad k = 0, 1, \ldots, N-1.$$

In some cases, the computation of the periodogram via an FFT algorithm is more efficient if the number of frequencies is a power of two. Therefore it is not uncommon to pad the input signal with zeros to extend its length to a power of two.

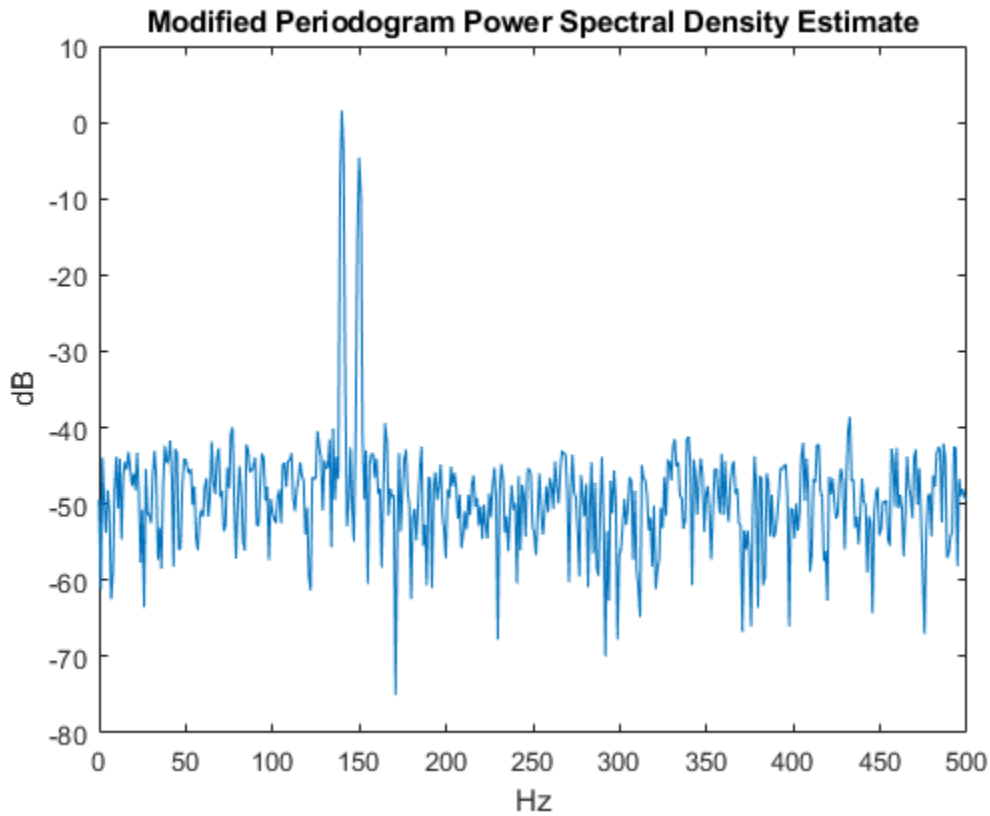
As an example of the periodogram, consider the following 1001-element signal `xn`, which consists of two sinusoids plus noise:

```
fs = 1000;                % Sampling frequency
t = (0:fs)/fs;            % One second worth of samples
```

```
A = [1 2];                  % Sinusoid amplitudes (row vector)
f = [150;140];              % Sinusoid frequencies (column vector)
xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
% The three last lines are equivalent to
% xn = sin(2*pi*150*t) + 2*sin(2*pi*140*t) + 0.1*randn(size(t));
```

The periodogram estimate of the PSD can be computed using `periodogram`. In this case, the data vector is multiplied by a Hamming window to produce a modified periodogram.

```
[Pxx,F] = periodogram(xn,hamming(length(xn)),length(xn),fs);
plot(F,10*log10(Pxx))
xlabel('Hz')
ylabel('dB')
title('Modified Periodogram Power Spectral Density Estimate')
```



Algorithm

Periodogram computes and scales the output of the FFT to produce the power vs. frequency plot as follows.

1. If the input signal is real-valued, the magnitude of the resulting FFT is symmetric with respect to zero frequency (DC). For an even-length FFT, only the first (1 + `nfft`/2) points are unique. Determine the number of unique values and keep only those unique points.
2. Take the squared magnitudes of the unique FFT values. Scale the squared magnitudes (except for DC) by $2/(F_s N)$, where N is the length of signal prior to any zero padding. Scale the DC value by $1/(F_s N)$.
3. Create a frequency vector from the number of unique points, the nfft and the sampling frequency.
4. Plot the resulting magnitude squared FFT against the frequency.

Performance of the Periodogram

The following sections discuss the performance of the periodogram with regard to the issues of leakage, resolution, bias, and variance.

Spectral Leakage

Consider the PSD of a finite-length (length L) signal $x_L(n)$. It is frequently useful to interpret $x_L(n)$ as the result of multiplying an infinite signal, $x(n)$, by a finite-length rectangular window, $w_R(n)$:

$$x_L(n) = x(n)w_R(n).$$

Because multiplication in the time domain corresponds to convolution in the frequency domain, the expected value of the periodogram in the frequency domain is

$$E\{\hat{P}_{xx}(f)\} = \frac{1}{F_s}\int_{-F_s/2}^{F_s/2} \frac{\sin^2(L\pi(f-f')/F_s)}{L\sin^2(\pi(f-f')/F_s)} P_{xx}(f')\,df',$$

showing that the expected value of the periodogram is the convolution of the true PSD with the square of the Dirichlet kernel.

The effect of the convolution is best understood for sinusoidal data. Suppose that $x(n)$ is composed of a sum of M complex sinusoids:

$$x(n) = \sum_{k=1}^{N} A_k e^{j\omega_k n}.$$

Its spectrum is

$$X(\omega) = \sum_{k=1}^{N} A_k \delta(\omega - \omega_k),$$

which for a finite-length sequence becomes

$$X(\omega) = \int_{-\pi}^{\pi} \left(\sum_{k=1}^{N} A_k \delta(\varepsilon - \omega_k) \right) W_R(\omega - \varepsilon)\, d\varepsilon.$$

The preceding equation is equal to

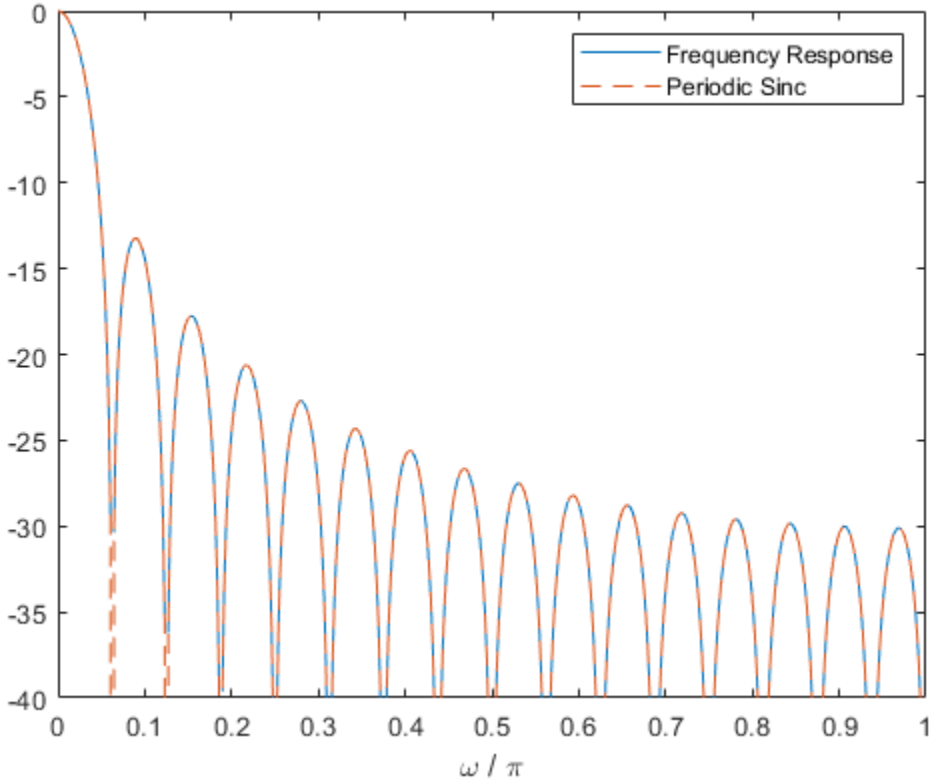
$$X(\omega) = \sum_{k=1}^{N} A_k W_R(\omega - \omega_k).$$

So in the spectrum of the finite-length signal, the Dirac deltas have been replaced by terms of the form $W_R(\omega - \omega_k)$, which corresponds to the frequency response of a rectangular window centered on the frequency ω_k.

The frequency response of a rectangular window has the shape of a periodic sinc:

```
L = 32;
[h,w] = freqz(rectwin(L)/L,1);
y = diric(w,L);

plot(w/pi,20*log10(abs(h)))
```

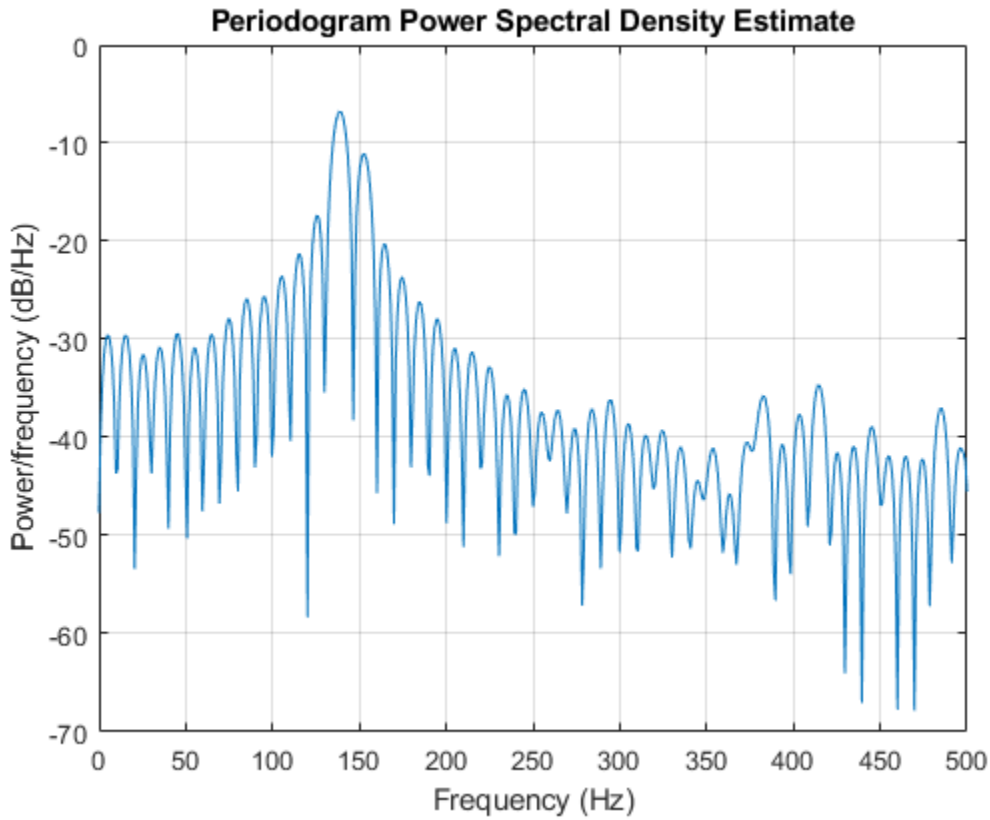
```
hold on
plot(w/pi,20*log10(abs(y)),'--')
hold off
ylim([-40,0])
legend('Frequency Response','Periodic Sinc')
xlabel('\omega / \pi')
```



The plot displays a mainlobe and several sidelobes, the largest of which is approximately 13.5 dB below the mainlobe peak. These lobes account for the effect known as spectral leakage. While the infinite-length signal has its power concentrated exactly at the discrete frequency points f_k, the windowed (or truncated) signal has a continuum of power "leaked" around the discrete frequency points f_k.

Because the frequency response of a short rectangular window is a much poorer approximation to the Dirac delta function than that of a longer window, spectral leakage is especially evident when data records are short. Consider the following sequence of 100 samples:

```
fs = 1000;                  % Sampling frequency
t = (0:fs/10)/fs;           % One-tenth second worth of samples
A = [1 2];                  % Sinusoid amplitudes
f = [150;140];              % Sinusoid frequencies
xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
periodogram(xn,rectwin(length(xn)),1024,fs)
```



It is important to note that the effect of spectral leakage is contingent solely on the length of the data record. It is not a consequence of the fact that the periodogram is computed at a finite number of frequency samples.

Resolution

Resolution refers to the ability to discriminate spectral features, and is a key concept on the analysis of spectral estimator performance.

In order to resolve two sinusoids that are relatively close together in frequency, it is necessary for the difference between the two frequencies to be greater than the width of the mainlobe of the leaked spectra for either one of these sinusoids. The mainlobe width is defined to be the width of the mainlobe at the point where the power is half the peak mainlobe power (i.e., 3 dB width). This width is approximately equal to f_s/L.

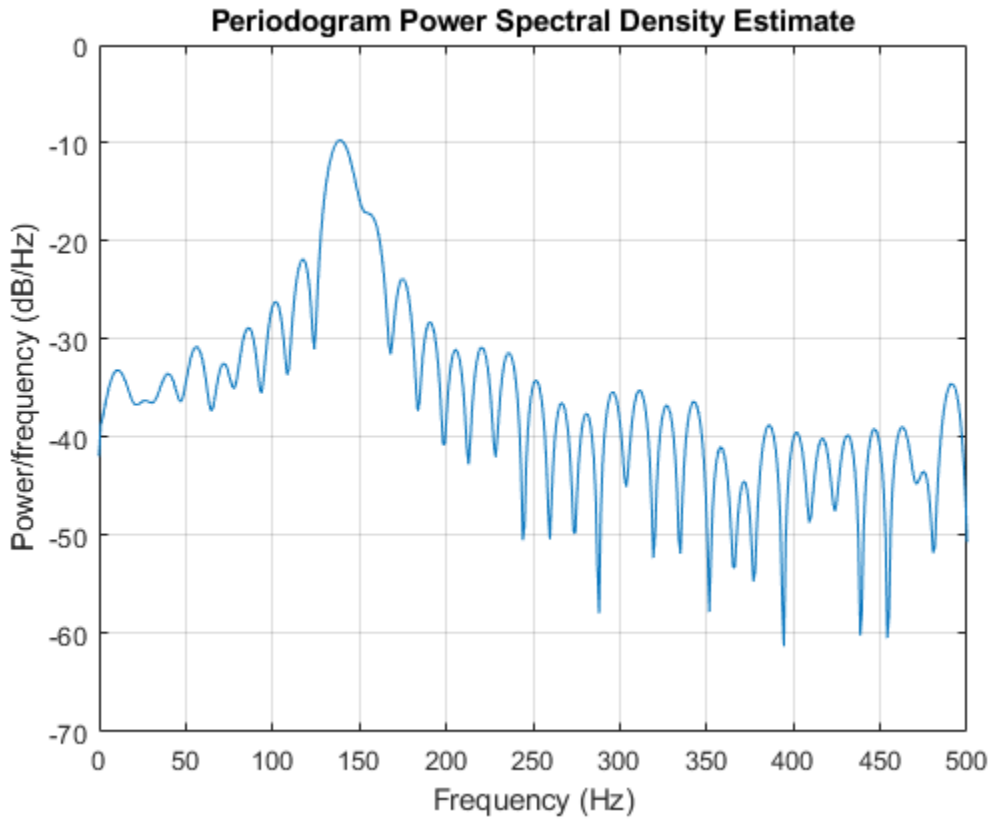
In other words, for two sinusoids of frequencies f_1 and f_2, the resolvability condition requires that

$$f_2 - f_1 > \frac{F_s}{L}.$$

In the example above, where two sinusoids are separated by only 10 Hz, the data record must be greater than 100 samples to allow resolution of two distinct sinusoids by a periodogram.

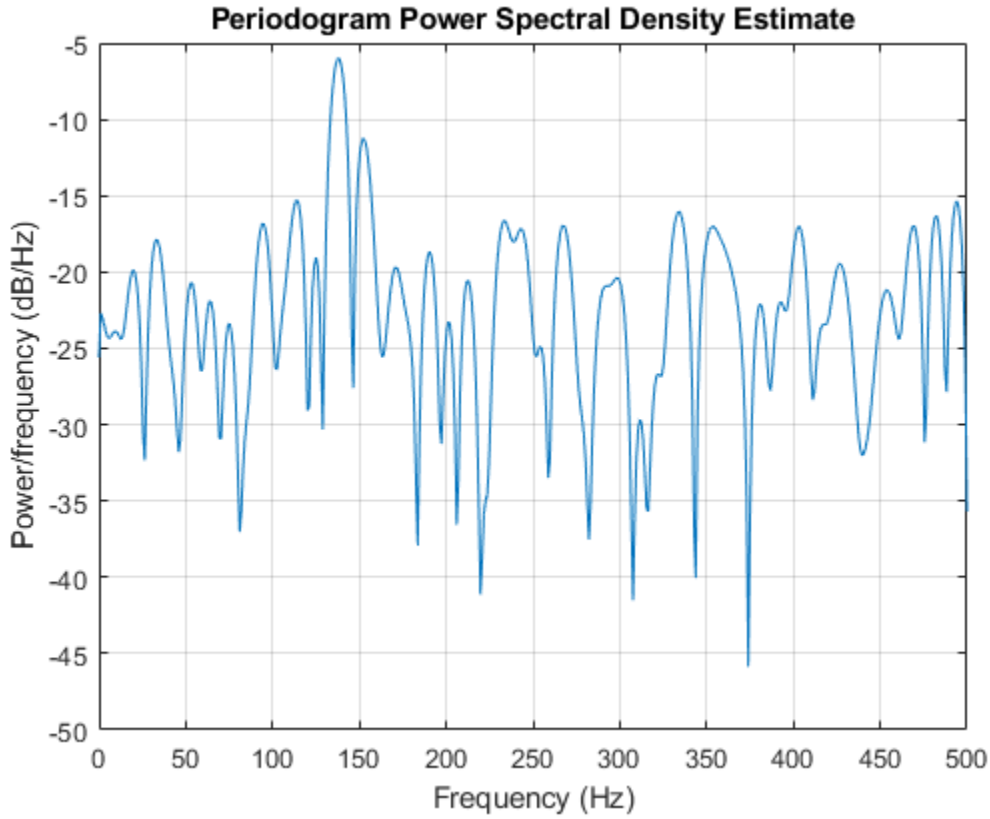
Consider a case where this criterion is not met, as for the sequence of 67 samples below:

```
fs = 1000;                  % Sampling frequency
t = (0:fs/15)/fs;           % 67 samples
A = [1 2];                  % Sinusoid amplitudes
f = [150;140];              % Sinusoid frequencies
xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
periodogram(xn,rectwin(length(xn)),1024,fs)
```



The above discussion about resolution did not consider the effects of noise since the signal-to-noise ratio (SNR) has been relatively high thus far. When the SNR is low, true spectral features are much harder to distinguish, and noise artifacts appear in spectral estimates based on the periodogram. The example below illustrates this:

```
fs = 1000;                  % Sampling frequency
t = (0:fs/10)/fs;           % One-tenth second worth of samples
A = [1 2];                  % Sinusoid amplitudes
f = [150;140];              % Sinusoid frequencies
xn = A*sin(2*pi*f*t) + 2*randn(size(t));
periodogram(xn,rectwin(length(xn)),1024,fs)
```



Bias of the Periodogram

The periodogram is a biased estimator of the PSD. Its expected value was previously shown to be

$$E\{\hat{P}_{xx}(f)\} = \frac{1}{F_s}\int_{-F_s/2}^{F_s/2} \frac{\sin^2(L\pi(f-f')/F_s)}{L\sin^2(\pi(f-f')/F_s)} P_{xx}(f')\,df'.$$

The periodogram is asymptotically unbiased, which is evident from the earlier observation that as the data record length tends to infinity, the frequency response of the rectangular window more closely approximates the Dirac delta function. However, in some cases the

periodogram is a poor estimator of the PSD even when the data record is long. This is due to the variance of the periodogram, as explained below.

Variance of the Periodogram

The variance of the periodogram can be shown to be

$$\mathrm{Var}(\hat{P}_{xx}(f)) = \begin{cases} P_{xx}^2(f), & 0 < f < F_s/2, \\ 2P_{xx}^2(f), & f = 0, F_s/2, \end{cases}$$

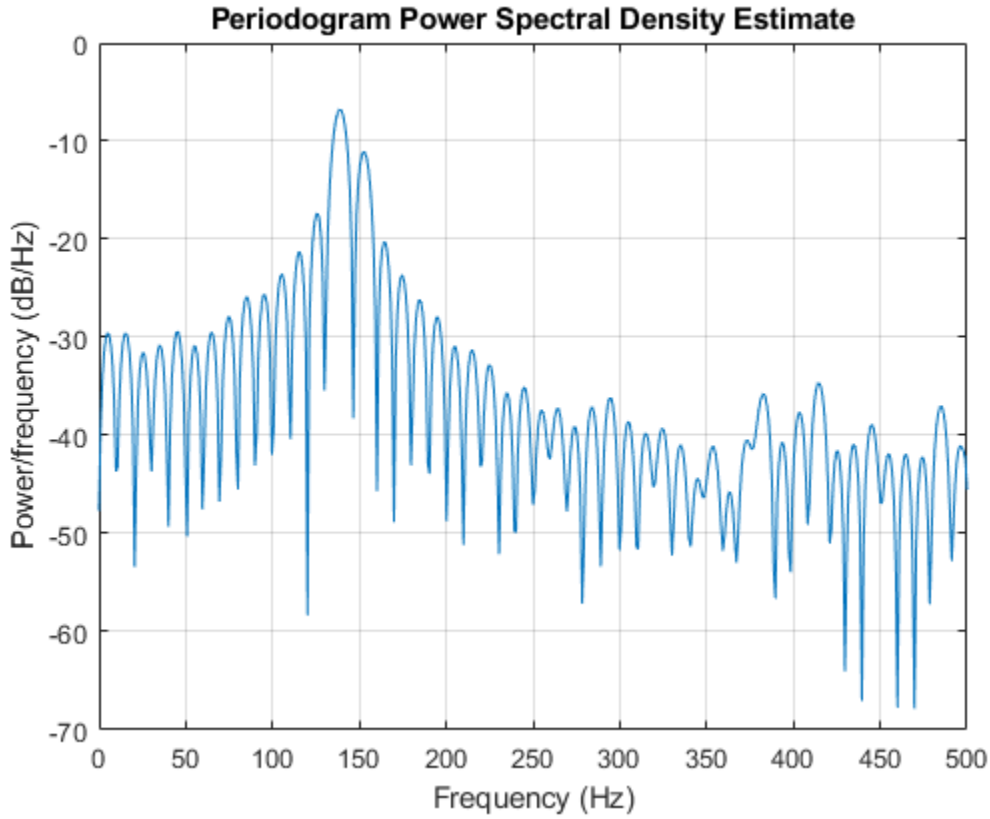
which indicates that the variance does not tend to zero as the data length L tends to infinity. In statistical terms, the periodogram is not a consistent estimator of the PSD. Nevertheless, the periodogram can be a useful tool for spectral estimation in situations where the SNR is high, and especially if the data record is long.

The Modified Periodogram

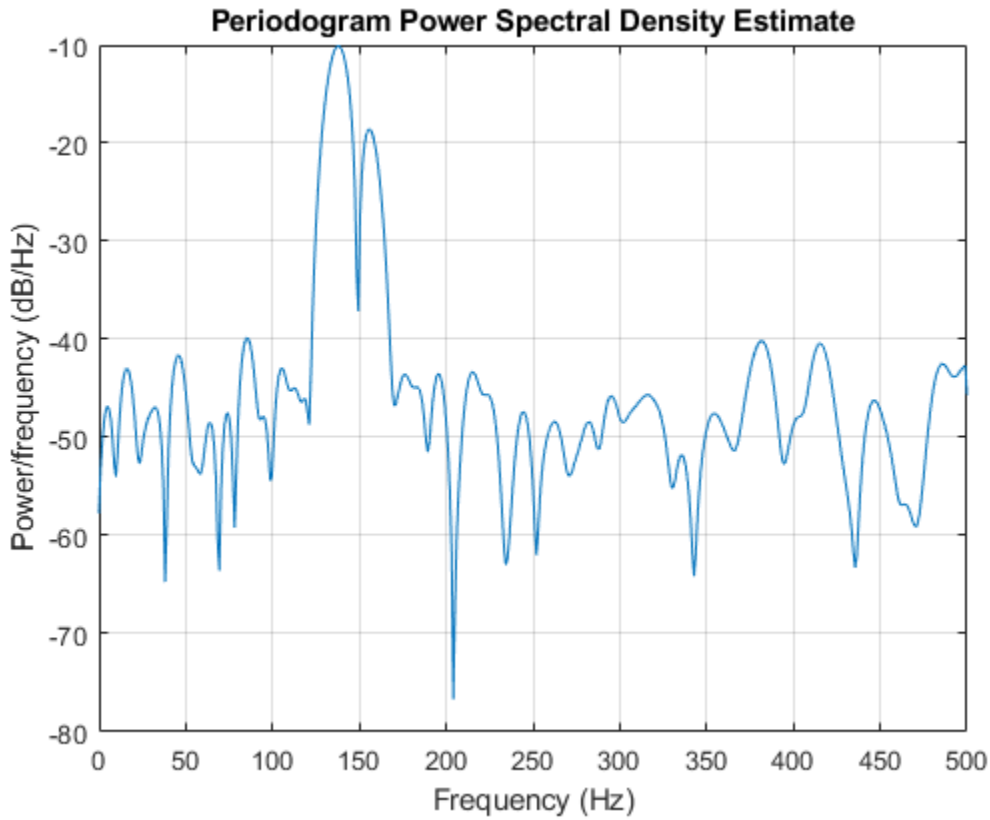
The *modified periodogram* windows the time-domain signal prior to computing the DFT in order to smooth the edges of the signal. This has the effect of reducing the height of the sidelobes or spectral leakage. This phenomenon gives rise to the interpretation of sidelobes as spurious frequencies introduced into the signal by the abrupt truncation that occurs when a rectangular window is used. For nonrectangular windows, the end points of the truncated signal are attenuated smoothly, and hence the spurious frequencies introduced are much less severe. On the other hand, nonrectangular windows also broaden the mainlobe, which results in a reduction of resolution.

The `periodogram` allows you to compute a modified periodogram by specifying the window to be used on the data. For example, compare a default rectangular window and a Hamming window. Specify the same number of DFT points in both cases.

```
fs = 1000;                  % Sampling frequency
t = (0:fs/10)/fs;           % One-tenth second worth of samples
A = [1 2];                  % Sinusoid amplitudes
f = [150;140];              % Sinusoid frequencies
nfft = 1024;

xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
periodogram(xn,rectwin(length(xn)),nfft,fs)
```




```
periodogram(xn,hamming(length(xn)),nfft,fs)
```



You can verify that although the sidelobes are much less evident in the Hamming-windowed periodogram, the two main peaks are wider. In fact, the 3 dB width of the mainlobe corresponding to a Hamming window is approximately twice that of a rectangular window. Hence, for a fixed data length, the PSD resolution attainable with a Hamming window is approximately half that attainable with a rectangular window. The competing interests of mainlobe width and sidelobe height can be resolved to some extent by using variable windows such as the Kaiser window.

Nonrectangular windowing affects the average power of a signal because some of the time samples are attenuated when multiplied by the window. To compensate for this, `periodogram` and `pwelch` normalize the window to have an average power of unity. This ensures that the measured average power is generally independent of window choice. If

the frequency components are not well resolved by the PSD estimators, the window choice does affect the average power.

The modified periodogram estimate of the PSD is

$$\hat{P}_{xx}(f) = \frac{|X(f)|^2}{F_s L U},$$

where U is the window normalization constant:

$$U = \frac{1}{L}\sum_{n=0}^{N-1} |w(n)|^2.$$

For large values of `L`, `U` tends to become independent of window length. The addition of `U` as a normalization constant ensures that the modified periodogram is asymptotically unbiased.

Welch's Method

An improved estimator of the PSD is the one proposed by Welch. The method consists of dividing the time series data into (possibly overlapping) segments, computing a modified periodogram of each segment, and then averaging the PSD estimates. The result is Welch's PSD estimate. The toolbox function `pwelch` implements Welch's method.

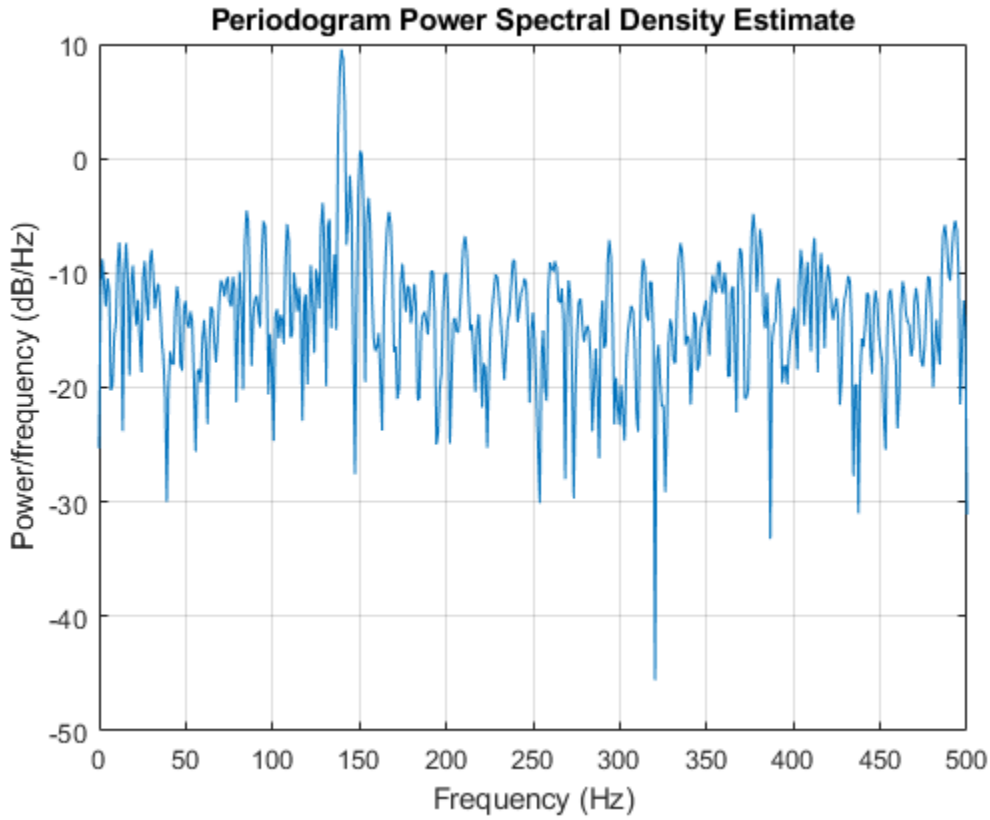
The averaging of modified periodograms tends to decrease the variance of the estimate relative to a single periodogram estimate of the entire data record. Although overlap between segments introduces redundant information, this effect is diminished by the use of a nonrectangular window, which reduces the importance or *weight* given to the end samples of segments (the samples that overlap).

However, as mentioned above, the combined use of short data records and nonrectangular windows results in reduced resolution of the estimator. In summary, there is a tradeoff between variance reduction and resolution. One can manipulate the parameters in Welch's method to obtain improved estimates relative to the periodogram, especially when the SNR is low. This is illustrated in the following example.

Consider a signal consisting of 301 samples:

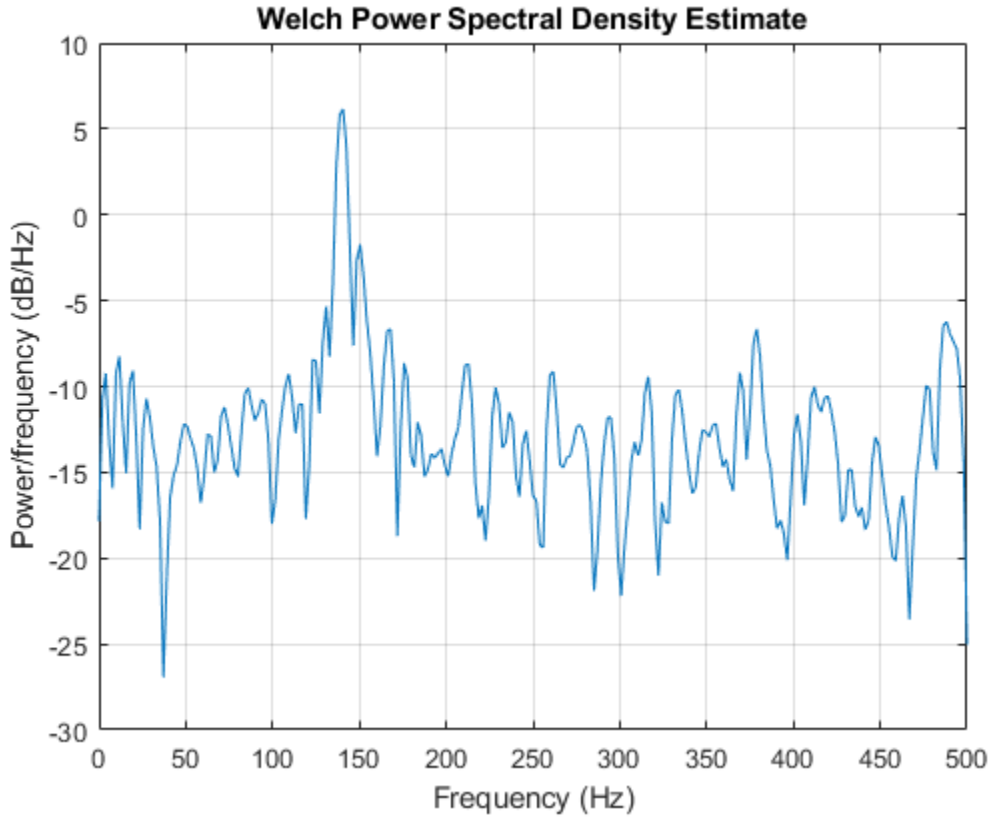
```
fs = 1000;              % Sampling frequency
t = (0:0.3*fs)/fs;      % 301 samples
```

```
A = [2 8];              % Sinusoid amplitudes (row vector)
f = [150;140];          % Sinusoid frequencies (column vector)

xn = A*sin(2*pi*f*t) + 5*randn(size(t));
periodogram(xn,rectwin(length(xn)),1024,fs)
```



We can obtain Welch's spectral estimate for 3 segments with 50% overlap using a rectangular window.

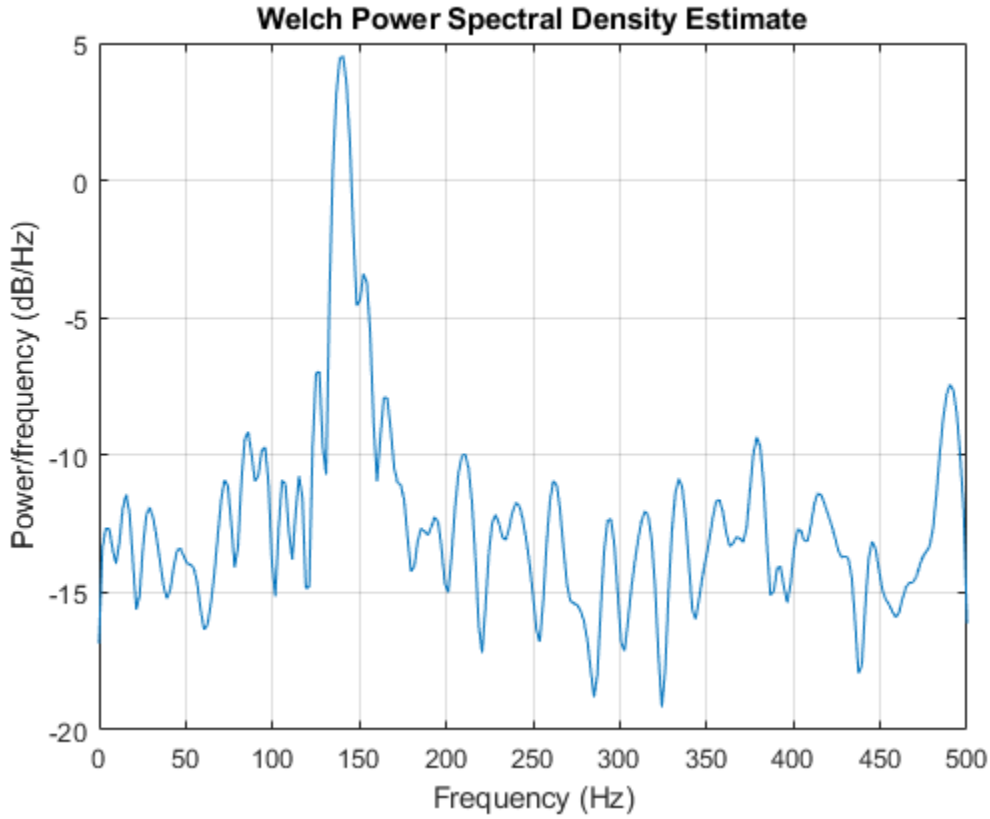
```
pwelch(xn,rectwin(150),50,512,fs)
```



In the periodogram above, noise and the leakage make one of the sinusoids essentially indistinguishable from the artificial peaks. In contrast, although the PSD produced by Welch's method has wider peaks, you can still distinguish the two sinusoids, which stand out from the "noise floor."

However, if we try to reduce the variance further, the loss of resolution causes one of the sinusoids to be lost altogether.

```
pwelch(xn,rectwin(100),75,512,fs)
```



Bias and Normalization in Welch's Method

Welch's method yields a biased estimator of the PSD. The expected value of the PSD estimate is:

$$E\{P_{\text{Welch}}(f)\} = \frac{1}{F_s LU}\int_{-F_s/2}^{F_s/2} \left|W(f-f')\right|^2 P_{xx}(f')df',$$

where L is the length of the data segments, U is the same normalization constant present in the definition of the modified periodogram, and $W(f)$ is the Fourier transform of the window function. As is the case for all periodograms, Welch's estimator is asymptotically

unbiased. For a fixed length data record, the bias of Welch's estimate is larger than that of the periodogram because the length of the segments is less than the length of the entire data sample.

The variance of Welch's estimator is difficult to compute because it depends on both the window used and the amount of overlap between segments. Basically, the variance is inversely proportional to the number of segments whose modified periodograms are being averaged.

Multitaper Method

The periodogram can be interpreted as filtering a length L signal, $x_L(n)$, through a filter bank (a set of filters in parallel) of L FIR bandpass filters. The 3 dB bandwidth of each of these bandpass filters can be shown to be approximately equal to f_s/L. The magnitude response of each one of these bandpass filters resembles that of a rectangular window. The periodogram can thus be viewed as a computation of the power of each filtered signal (i.e., the output of each bandpass filter) that uses just one sample of each filtered signal and assumes that the PSD of $x_L(n)$ is constant over the bandwidth of each bandpass filter.

As the length of the signal increases, the bandwidth of each bandpass filter decreases, making it a more selective filter, and improving the approximation of constant PSD over the bandwidth of the filter. This provides another interpretation of why the PSD estimate of the periodogram improves as the length of the signal increases. However, there are two factors apparent from this standpoint that compromise the accuracy of the periodogram estimate. First, the rectangular window yields a poor bandpass filter. Second, the computation of the power at the output of each bandpass filter relies on a single sample of the output signal, producing a very crude approximation.

Welch's method can be given a similar interpretation in terms of a filter bank. In Welch's implementation, several samples are used to compute the output power, resulting in reduced variance of the estimate. On the other hand, the bandwidth of each bandpass filter is larger than that corresponding to the periodogram method, which results in a loss of resolution. The filter bank model thus provides a new interpretation of the compromise between variance and resolution.

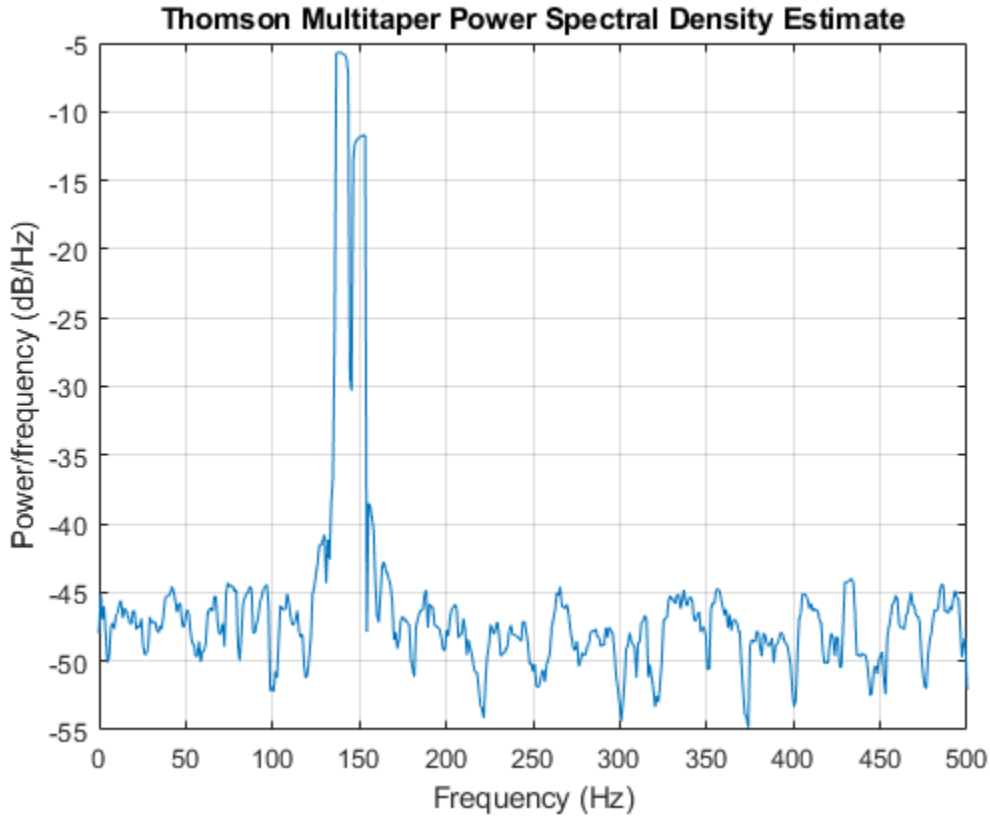
Thompson's *multitaper method* (MTM) builds on these results to provide an improved PSD estimate. Instead of using bandpass filters that are essentially rectangular windows (as in the periodogram method), the MTM method uses a bank of optimal bandpass filters

to compute the estimate. These optimal FIR filters are derived from a set of sequences known as discrete prolate spheroidal sequences (DPSSs, also known as *Slepian sequences*).

In addition, the MTM method provides a time-bandwidth parameter with which to balance the variance and resolution. This parameter is given by the time-bandwidth product, NW and it is directly related to the number of tapers used to compute the spectrum. There are always $2NW - 1$ tapers used to form the estimate. This means that, as NW increases, there are more estimates of the power spectrum, and the variance of the estimate decreases. However, the bandwidth of each taper is also proportional to NW, so as NW increases, each estimate exhibits more spectral leakage (i.e., wider peaks) and the overall spectral estimate is more biased. For each data set, there is usually a value for NW that allows an optimal trade-off between bias and variance.

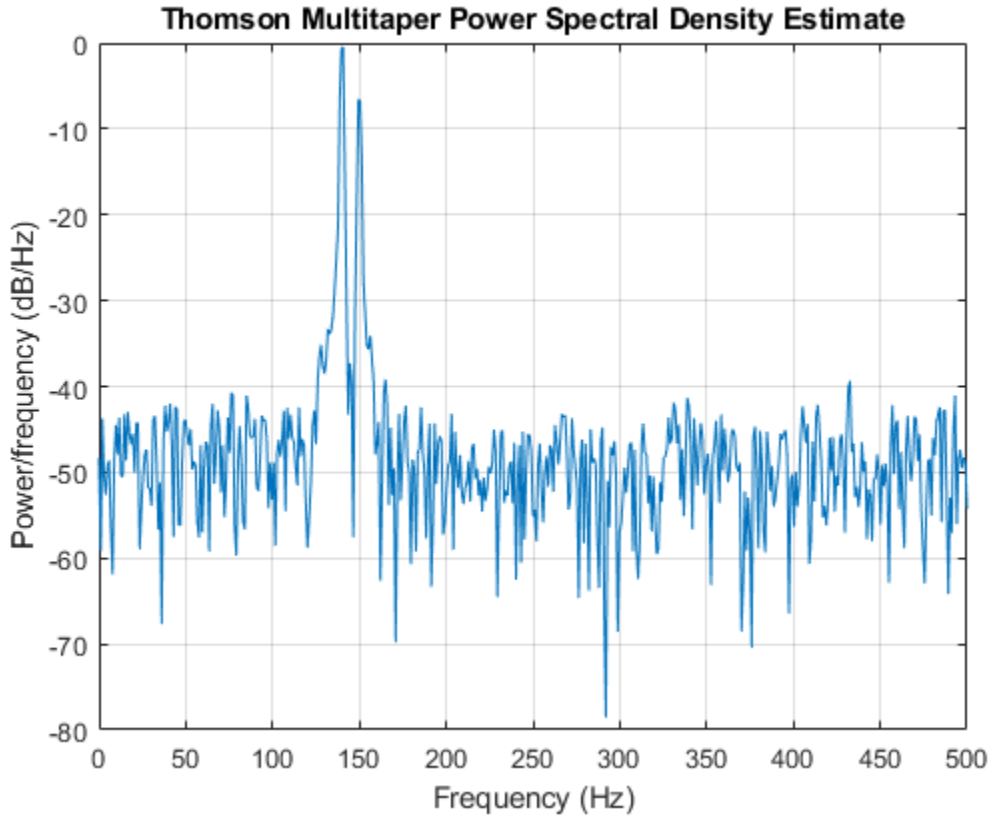
The Signal Processing Toolbox™ function that implements the MTM method is `pmtm`. Use `pmtm` to compute the PSD of a signal.

```
fs = 1000;                % Sampling frequency
t = (0:fs)/fs;            % One second worth of samples
A = [1 2];                % Sinusoid amplitudes
f = [150;140];            % Sinusoid frequencies

xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
pmtm(xn,4,[],fs)
```



By lowering the time-bandwidth product, you can increase the resolution at the expense of larger variance.

```
pmtm(xn,1.5,[],fs)
```



This method is more computationally expensive than Welch's method due to the cost of computing the discrete prolate spheroidal sequences. For long data series (10,000 points or more), it is useful to compute the DPSSs once and save them in a MAT-file. `dpsssave`, `dpssload`, `dpssdir`, and `dpssclear` are provided to keep a database of saved DPSSs in the MAT-file `dpss.mat`.

Cross-Spectral Density Function

The PSD is a special case of the *cross spectral density* (CPSD) function, defined between two signals $x(n)$ and $y(n)$ as

$$P_{xy}(\omega) = \frac{1}{2\pi} \sum_{m=-\infty}^{\infty} R_{xy}(m) e^{-j\omega m}.$$

As is the case for the correlation and covariance sequences, the toolbox *estimates* the PSD and CPSD because signal lengths are finite.

To estimate the cross-spectral density of two equal length signals `x` and `y` using Welch's method, the `cpsd` function forms the periodogram as the product of the FFT of `x` and the conjugate of the FFT of `y`. Unlike the real-valued PSD, the CPSD is a complex function. `cpsd` handles the sectioning and windowing of `x` and `y` in the same way as the `pwelch` function:

```
Sxy = cpsd(x,y,nwin,noverlap,nfft,fs)
```

Transfer Function Estimate

One application of Welch's method is nonparametric system identification. Assume that H is a linear, time invariant system, and $x(n)$ and $y(n)$ are the input to and output of H, respectively. Then the power spectrum of $x(n)$ is related to the CPSD of $x(n)$ and $y(n)$ by

$$P_{yx}(\omega) = H(\omega) P_{xx}(\omega).$$

An estimate of the transfer function between $x(n)$ and $y(n)$ is

$$\hat{H}(\omega) = \frac{\hat{P}_{yx}(\omega)}{\hat{P}_{xx}(\omega)}.$$

This method estimates both magnitude and phase information. The `tfestimate` function uses Welch's method to compute the CPSD and power spectrum, and then forms their quotient for the transfer function estimate. Use `tfestimate` the same way that you use the `cpsd` function.

Generate a signal consisting of two sinusoids embedded in white Gaussian noise.

```
rng('default')

fs = 1000;                % Sampling frequency
t = (0:fs)/fs;            % One second worth of samples
A = [1 2];                % Sinusoid amplitudes
```

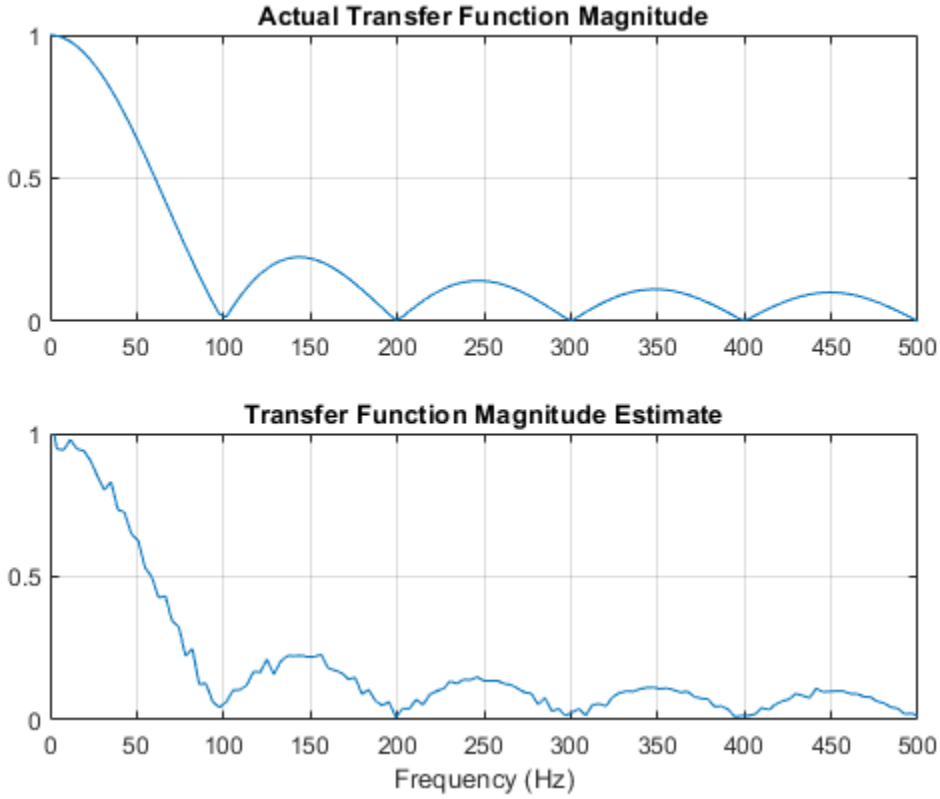
```
f = [150;140];            % Sinusoid frequencies

xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
```

Filter the signal `xn` with an FIR moving-average filter. Compute the actual magnitude response and the estimated response.

```
h = ones(1,10)/10;                % Moving-average filter
yn = filter(h,1,xn);

[HEST,f] = tfestimate(xn,yn,256,128,256,fs);
H = freqz(h,1,f,fs);
```

Plot the results.

```
subplot(2,1,1)
plot(f,abs(H))
title('Actual Transfer Function Magnitude')
yl = ylim;
grid
subplot(2,1,2)
plot(f,abs(HEST))
title('Transfer Function Magnitude Estimate')
xlabel('Frequency (Hz)')
ylim(yl)
grid
```



Coherence Function

The magnitude-squared coherence between two signals $x(n)$ and $y(n)$ is

$$C_{xy}(\omega) = \frac{|P_{xy}(\omega)|^2}{P_{xx}(\omega)P_{yy}(\omega)}.$$

This quotient is a real number between 0 and 1 that measures the correlation between $x(n)$ and $y(n)$ at the frequency ω.

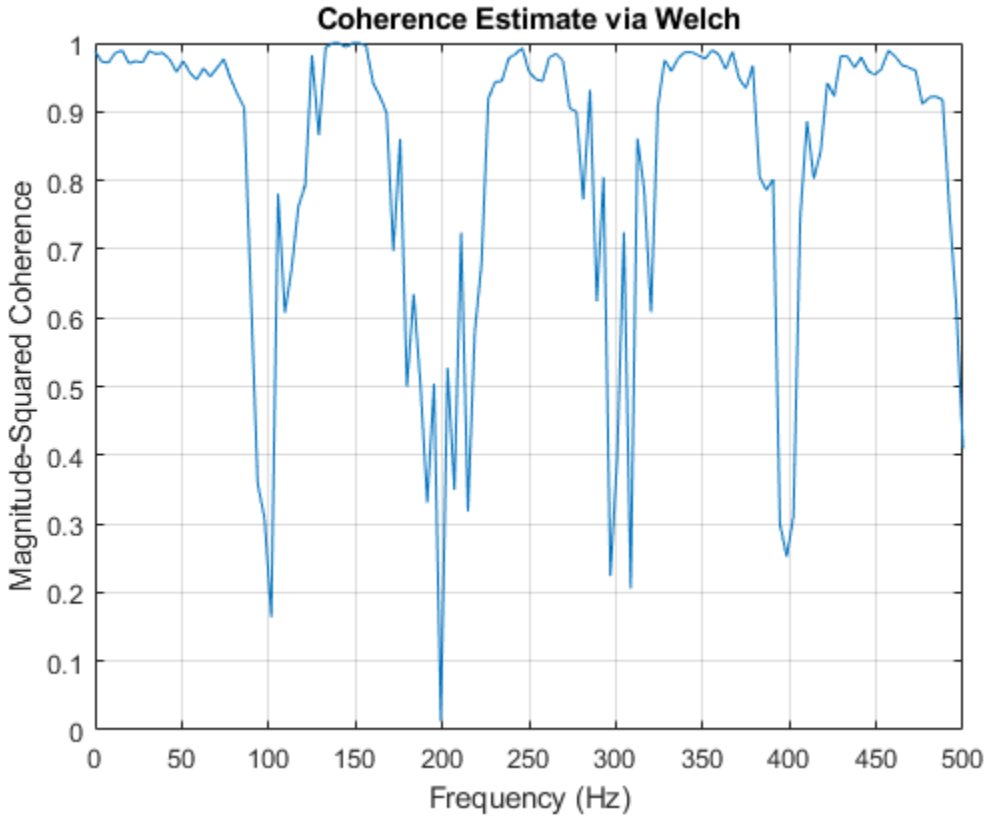
The `mscohere` function takes sequences `xn` and `yn`, computes their power spectra and CPSD, and returns the quotient of the magnitude squared of the CPSD and the product of the power spectra. Its options and operation are similar to the `cpsd` and `tfestimate` functions.

Generate a signal consisting of two sinusoids embedded in white Gaussian noise. The signal is sampled at 1 kHz for 1 second.

```
rng('default')

fs = 1000;
t = (0:fs)/fs;
A = [1 2];                 % Sinusoid amplitudes
f = [150;140];             % Sinusoid frequencies

xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
```

Filter the signal `xn` with an FIR moving-average filter. Compute and plot the coherence function of `xn` and the filter output `yn` as a function of frequency.

```
h = ones(1,10)/10;
yn = filter(h,1,xn);

mscohere(xn,yn,256,128,256,fs)
```



If the input sequence length, window length, and number of overlapping data points in a window are such that `mscohere` operates on only a single record, the function returns all ones. This is because the coherence function for linearly dependent data is one.

Parametric Methods

Parametric methods can yield higher resolutions than nonparametric methods in cases when the signal length is short. These methods use a different approach to spectral estimation; instead of trying to estimate the PSD directly from the data, they *model* the data as the output of a linear system driven by white noise, and then attempt to estimate the parameters of that linear system.

The most commonly used linear system model is the *all-pole model*, a filter with all of its zeroes at the origin in the *z*-plane. The output of such a filter for white noise input is an autoregressive (AR) process. For this reason, these methods are sometimes referred to as *AR methods* of spectral estimation.

The AR methods tend to adequately describe spectra of data that is "peaky," that is, data whose PSD is large at certain frequencies. The data in many practical applications (such as speech) tends to have "peaky spectra" so that AR models are often useful. In addition, the AR models lead to a system of linear equations which is relatively simple to solve.

Signal Processing Toolbox AR methods for spectral estimation include:

- Yule-Walker AR method (autocorrelation method) on page 6-36
- Burg method on page 6-38
- Covariance method on page 6-42
- Modified covariance method on page 6-42

All AR methods yield a PSD estimate given by

$$\hat{P}(f) = \frac{1}{F_s} \frac{\varepsilon_p}{\left| 1 - \sum_{k=1}^{p} \hat{a}_p(k) e^{-j2\pi kf/F_s} \right|^2}.$$

The different AR methods estimate the parameters slightly differently, yielding different PSD estimates. The following table provides a summary of the different AR methods.

AR Methods

	Burg	Covariance	Modified Covariance	Yule-Walker
Characteristics	Does not apply window to data	Does not apply window to data	Does not apply window to data	Applies window to data
	Minimizes the forward and backward prediction errors in the least squares sense, with the AR coefficients constrained to satisfy the L-D recursion	Minimizes the forward prediction error in the least squares sense	Minimizes the forward and backward prediction errors in the least squares sense	Minimizes the forward prediction error in the least squares sense (also called “Autocorrelation method”)
Advantages	High resolution for short data records	Better resolution than Y-W for short data records (more accurate estimates)	High resolution for short data records	Performs as well as other methods for large data records
	Always produces a stable model	Able to extract frequencies from data consisting of *p* or more pure sinusoids	Able to extract frequencies from data consisting of *p* or more pure sinusoids	Always produces a stable model
			Does not suffer spectral line-splitting	
Disadvantages	Peak locations highly dependent on initial phase	May produce unstable models	May produce unstable models	Performs relatively poorly for short data records

	Burg	Covariance	Modified Covariance	Yule-Walker
	May suffer spectral line-splitting for sinusoids in noise, or when order is very large	Frequency bias for estimates of sinusoids in noise	Peak locations slightly dependent on initial phase	Frequency bias for estimates of sinusoids in noise
	Frequency bias for estimates of sinusoids in noise		Minor frequency bias for estimates of sinusoids in noise	
Conditions for Nonsingularity		Order must be less than or equal to half the input frame size	Order must be less than or equal to 2/3 the input frame size	Because of the biased estimate, the autocorrelation matrix is guaranteed to positive-definite, hence nonsingular

Yule-Walker AR Method

The *Yule-Walker AR method* of spectral estimation computes the AR parameters by solving the following linear system, which give the Yule-Walker equations in matrix form:

$$\begin{pmatrix} r(0) & r^*(1) & \dots & r^*(p-1) \\ r(1) & r(0) & \dots & r^*(p-2) \\ r(p-1) & r(p-2) & \dots & r(0) \end{pmatrix} \begin{pmatrix} a(1) \\ a(2) \\ \vdots \\ a(p) \end{pmatrix} = \begin{pmatrix} r(1) \\ r(2) \\ \vdots \\ r(p) \end{pmatrix}.$$

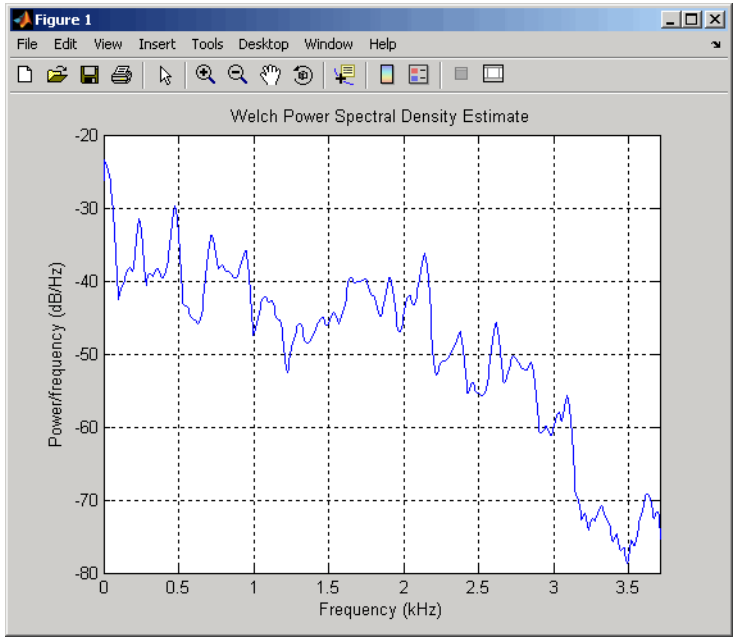
In practice, the biased estimate of the autocorrelation is used for the unknown true autocorrelation.The Yule-Walker AR method produces the same results as a maximum entropy estimator. For more information, see page 155 of item [2] in the “Selected Bibliography” on page 6-47.

The use of a biased estimate of the autocorrelation function ensures that the autocorrelation matrix above is positive definite. Hence, the matrix is invertible and a

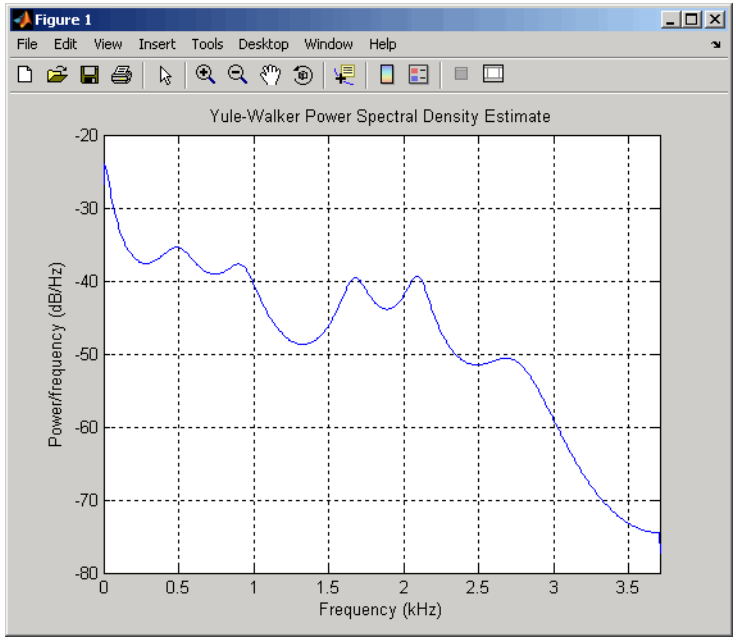
solution is guaranteed to exist. Moreover, the AR parameters thus computed always result in a stable all-pole model. The Yule-Walker equations can be solved efficiently via Levinson’s algorithm, which takes advantage of the Hermitian Toeplitz structure of the autocorrelation matrix.

The toolbox function `pyulear` implements the Yule-Walker AR method. For example, compare the spectrum of a speech signal using Welch's method and the Yule-Walker AR method:

```
load mtlb
[Pxx,F] = pwelch(mtlb,hamming(256),128,1024,Fs);
plot(F,10*log10(Pxx))
```




```
order = 14;
[Pxx,F] = pyulear(mtlb,order,1024,fs);
plot(F,10*log10(Pxx))
```



The Yule-Walker AR spectrum is smoother than the periodogram because of the simple underlying all-pole model.

Burg Method

The Burg method for AR spectral estimation is based on minimizing the forward and backward prediction errors while satisfying the Levinson-Durbin recursion (see Marple [3], Chapter 7, and Proakis [6], Section 12.3.3). In contrast to other AR estimation methods, the Burg method avoids calculating the autocorrelation function, and instead estimates the reflection coefficients directly.

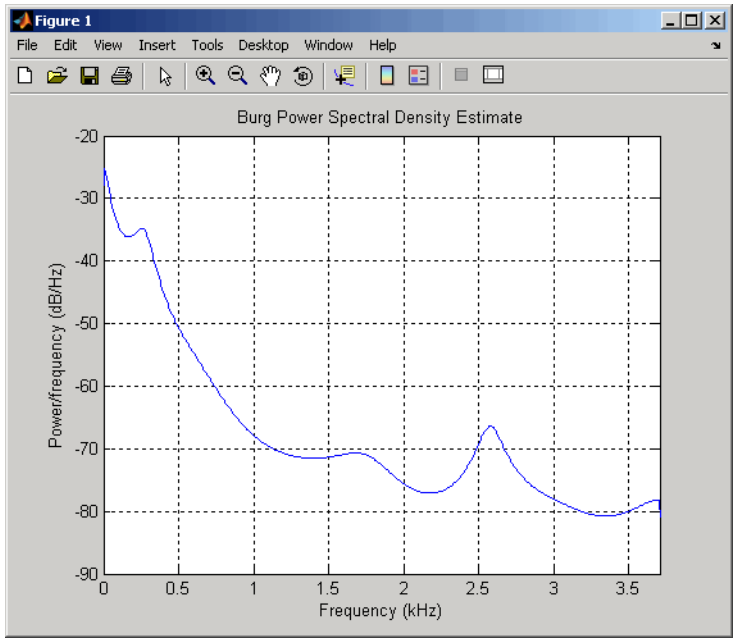
The primary advantages of the Burg method are resolving closely spaced sinusoids in signals with low noise levels, and estimating short data records, in which case the AR power spectral density estimates are very close to the true values. In addition, the Burg method ensures a stable AR model and is computationally efficient.

The accuracy of the Burg method is lower for high-order models, long data records, and high signal-to-noise ratios (which can cause *line splitting*, or the generation of extraneous peaks in the spectrum estimate). The spectral density estimate computed by the Burg

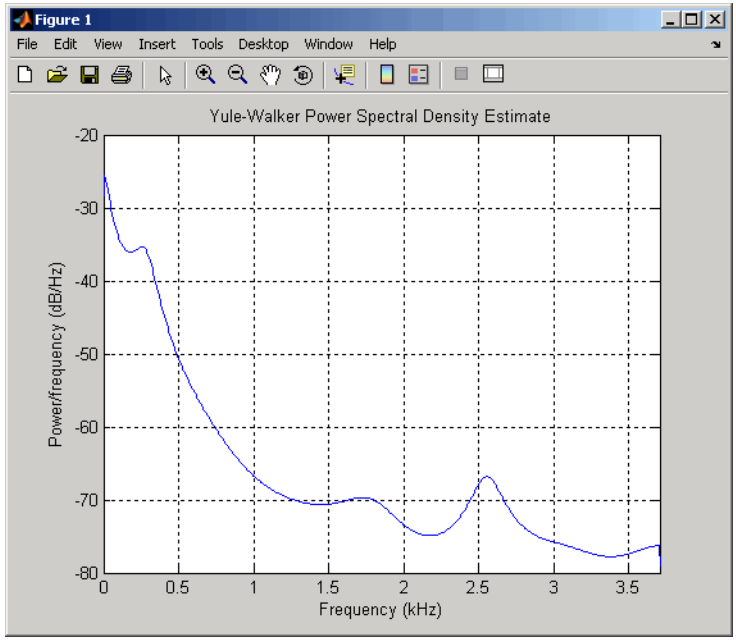
method is also susceptible to frequency shifts (relative to the true frequency) resulting from the initial phase of noisy sinusoidal signals. This effect is magnified when analyzing short data sequences.

The toolbox function `pburg` implements the Burg method. Compare the spectrum of the speech signal generated by both the Burg method and the Yule-Walker AR method. They are very similar for large signal lengths:

```
load mtlb
order = 14;
[Pburg,F] = pburg(mtlb(1:512),order,1024,fs);
plot(F,10*log10(Pburg))
```

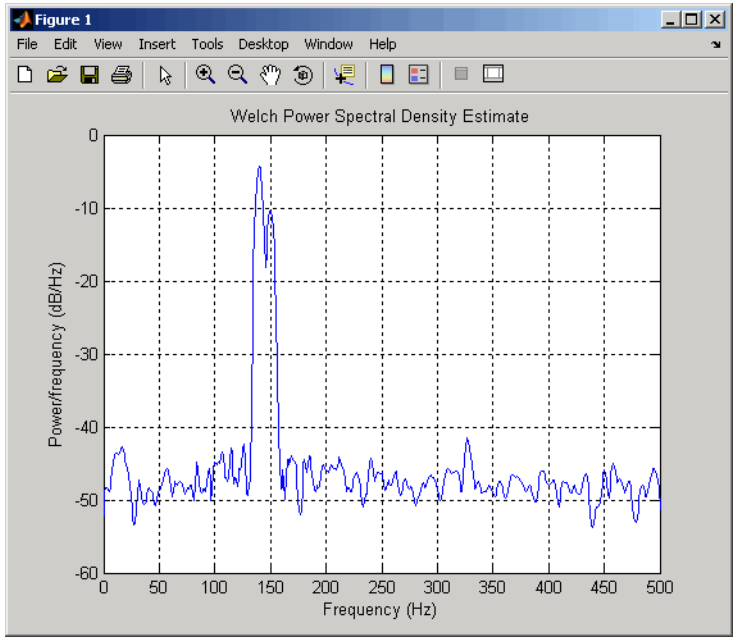



```
[Pxx,F] = pyulear(mtlb(1:512),ORDER,1024,fs);
plot(F,10*log10(Pxx))
```

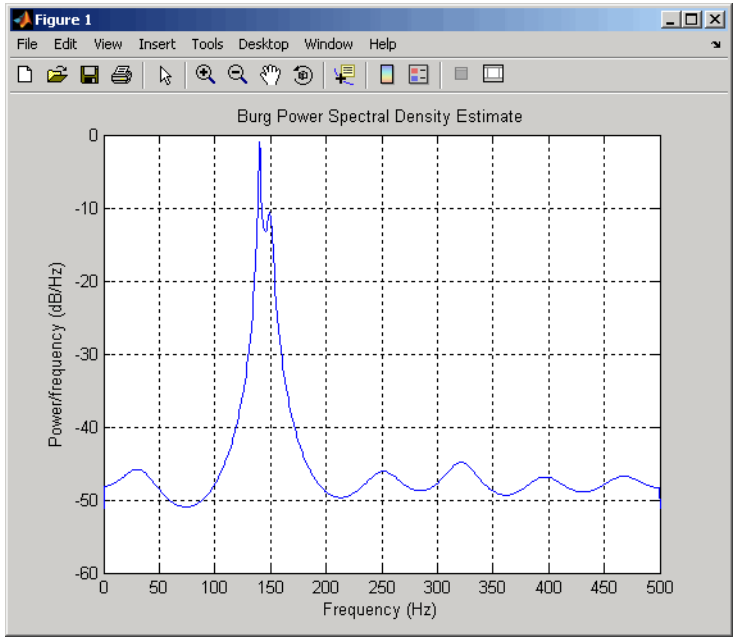


Compare the spectrum of a noisy signal computed using the Burg method and the Welch method:

```
fs = 1000;                % Sampling frequency
t = (0:fs)/fs;            % One second worth of samples
A = [1 2];                % Sinusoid amplitudes
f = [150;140];            % Sinusoid frequencies
xn = A*sin(2*pi*f*t) + 0.1*randn(size(t));
pwelch(xn,hamming(256),128,1024,fs)
```




```
pburg(xn,14,1024,fs)
```



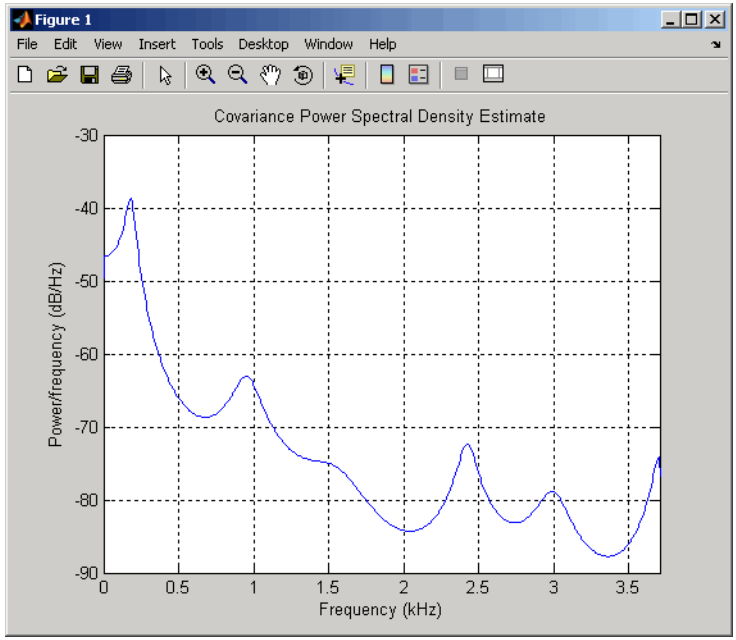
Note that, as the model order for the Burg method is reduced, a frequency shift due to the initial phase of the sinusoids will become apparent.

Covariance and Modified Covariance Methods

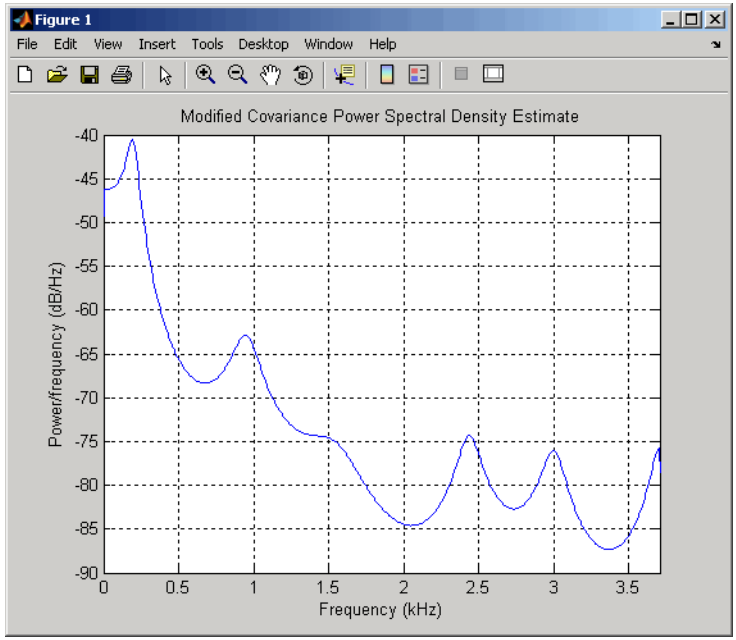
The covariance method for AR spectral estimation is based on minimizing the forward prediction error. The modified covariance method is based on minimizing the forward and backward prediction errors. The toolbox functions `pcov` and `pmcov` implement the respective methods.

Compare the spectrum of the speech signal generated by both the covariance method and the modified covariance method. They are nearly identical, even for a short signal length:

```
load mtlb
pcov(mtlb(1:64),14,1024,fs)
```




```
pmcov(mtlb(1:64),14,1024,fs)
```



MUSIC and Eigenvector Analysis Methods

The `pmusic` function and `peig` functions provide two related spectral analysis methods:

- `pmusic` provides the multiple signal classification (MUSIC) method developed by Schmidt
- `peig` provides the eigenvector (EV) method developed by Johnson

Both of these methods are frequency estimator techniques based on eigenanalysis of the autocorrelation matrix. This type of spectral analysis categorizes the information in a correlation or data matrix, assigning information to either a signal subspace or a noise subspace.

Eigenanalysis Overview

Consider a number of complex sinusoids embedded in white noise. You can write the autocorrelation matrix R for this system as the sum of the signal autocorrelation matrix (S) and the noise autocorrelation matrix (W):

$R = S + W$. There is a close relationship between the eigenvectors of the signal autocorrelation matrix and the signal and noise subspaces. The eigenvectors v of S span the same signal subspace as the signal vectors. If the system contains M complex sinusoids and the order of the autocorrelation matrix is p, eigenvectors v_{M+1} through v_{p+1} span the noise subspace of the autocorrelation matrix.

Frequency Estimator Functions

To generate their frequency estimates, eigenanalysis methods calculate functions of the vectors in the signal and noise subspaces. Both the MUSIC and EV techniques choose a function that goes to infinity (denominator goes to zero) at one of the sinusoidal frequencies in the input signal. Using digital technology, the resulting estimate has sharp peaks at the frequencies of interest; this means that there might not be infinity values in the vectors.

The MUSIC estimate is given by the formula

$$\hat{P}_{\mathrm{MUSIC}}(f) = \frac{1}{\sum_{k=p+1}^{M} \left| e(f)^H v_k \right|^2},$$

where the v_k are the eigenvectors of the noise subspace and $e(f)$ is a vector of complex sinusoids:

$$e(f) = [1 \quad e^{j2\pi f} \quad e^{j4\pi f} \quad \dots \quad e^{j2(M-1)\pi f}]^T.$$

Here v represents the eigenvectors of the input signal's correlation matrix; v_k is the kth eigenvector. H is the conjugate transpose operator. The eigenvectors used in the sum correspond to the smallest eigenvalues and span the noise subspace (p is the size of the signal subspace).

The expression $e(f)^H v_k$ is equivalent to a Fourier transform (the vector $e(f)$ consists of complex exponentials). This form is useful for numeric computation because the FFT can be computed for each v_k and then the squared magnitudes can be summed.

The EV method weights the summation by the eigenvalues of the correlation matrix:

$$\hat{P}_{\mathrm{EV}}(f) = \frac{\lambda_k}{\sum_{k=p+1}^{M} \left| e^H(f) v_k \right|^2}.$$

The `pmusic` and `peig` functions in this toolbox interpret their first input either as a signal matrix or as a correlation matrix (if the `'corr'` input flag is set). In the former case, the singular value decomposition of the signal matrix is used to determine the signal and noise subspaces. In the latter case, the eigenvalue decomposition of the correlation matrix is used to determine the signal and noise subspaces.

Selected Bibliography

[1] Hayes, Monson H. *Statistical Digital Signal Processing and Modeling*. New York: John Wiley & Sons, 1996.

[2] Kay, Steven M. *Modern Spectral Estimation*. Englewood Cliffs, NJ: Prentice Hall, 1988.

[3] Marple, S. Lawrence *Digital Spectral Analysis*. Englewood Cliffs, NJ: Prentice Hall, 1987.

[4] Orfanidis, Sophocles J. *Introduction to Signal Processing*. Upper Saddle River, NJ: Prentice Hall, 1996.

[5] Percival, D. B., and A. T. Walden. *Spectral Analysis for Physical Applications: Multitaper and Conventional Univariate Techniques*. Cambridge: Cambridge University Press, 1993.

[6] Proakis, John G., and Dimitris G. Manolakis. *Digital Signal Processing: Principles, Algorithms, and Applications*. Englewood Cliffs, NJ: Prentice Hall, 1996.

[7] Stoica, Petre, and Randolph Moses. *Spectral Analysis of Signals*. Upper Saddle River, NJ: Prentice Hall, 1997.

[8] Welch, Peter D. “The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms.” *IEEE Trans. Audio Electroacoust.*. Vol. AU-15, 1967, pp. 70–73.

Special Topics

- “Windows” on page 7-2
- “Getting Started with Window Designer” on page 7-8
- “Generalized Cosine Windows” on page 7-13
- “Kaiser Window” on page 7-16
- “Chebyshev Window” on page 7-24
- “Parametric Modeling” on page 7-26
- “Resampling” on page 7-33
- “Cepstrum Analysis” on page 7-36
- “FFT-Based Time-Frequency Analysis” on page 7-40
- “Cross-Spectrogram of Complex Signals” on page 7-44
- “Find Interference Using Persistence Spectrum” on page 7-47
- “Find and Track Ridges Using Reassigned Spectrogram” on page 7-52
- “Extract Regions of Interest from Whale Song” on page 7-57
- “Median Filtering” on page 7-62
- “Communications Applications” on page 7-63
- “Deconvolution” on page 7-68
- “Chirp Z-Transform” on page 7-69
- “Discrete Cosine Transform” on page 7-72
- “Hilbert Transform” on page 7-77
- “Walsh-Hadamard Transform” on page 7-80
- “Walsh-Hadamard Transform for Spectral Analysis and Compression of ECG Signals” on page 7-83
- “Eliminate Outliers Using Hampel Identifier” on page 7-86
- “Selected Bibliography” on page 7-89

Windows

In this section...
“Why Use Windows?” on page 7-2
“Available Window Functions” on page 7-2
“Graphical User Interface Tools” on page 7-3
“Basic Shapes” on page 7-3

Why Use Windows?

In both digital filter design and spectral estimation, the choice of a windowing function can play an important role in determining the quality of overall results. The main role of the window is to damp out the effects of the Gibbs phenomenon that results from truncation of an infinite series.

Available Window Functions

Window	Function
Bartlett-Hann window	`barthannwin`
Bartlett window	`bartlett`
Blackman window	`blackman`
Blackman-Harris window	`blackmanharris`
Bohman window	`bohmanwin`
Chebyshev window	`chebwin`
Flat Top window	`flattopwin`
Gaussian window	`gausswin`
Hamming window	`hamming`
Hann window	`hann`
Kaiser window	`kaiser`
Nuttall's Blackman-Harris window	`nuttallwin`
Parzen (de la Vallée-Poussin) window	`parzenwin`

Window	Function
Rectangular window	`rectwin`
Tapered cosine window	`tukeywin`
Triangular window	`triang`

Graphical User Interface Tools

Two graphical user interface tools are provided for working with windows in the Signal Processing Toolbox product:

- **Window Designer** app
- Window Visualization Tool (`wvtool`)

Refer to the reference pages for detailed information.

Basic Shapes

The basic window is the *rectangular window*, a vector of ones of the appropriate length. A rectangular window of length 50 is

```
n = 50;
w = rectwin(n);
```

This toolbox stores windows in column vectors by convention, so an equivalent expression is

```
w = ones(50,1);
```

To use the Window Designer app to create this window, type

```
windowDesigner
```

The app opens with a default Hamming window. To visualize the rectangular window, set **Type = Rectangular** and **Length = 50** in the Current Window Information panel and then press **Apply**.

The *Bartlett* (or triangular) *window* is the convolution of two rectangular windows. The functions `bartlett` and `triang` compute similar triangular windows, with three important differences. The `bartlett` function always returns a window with two zeros on

the ends of the sequence, so that for `n` odd, the center section of `bartlett(n+2)` is equivalent to `triang(n)`:

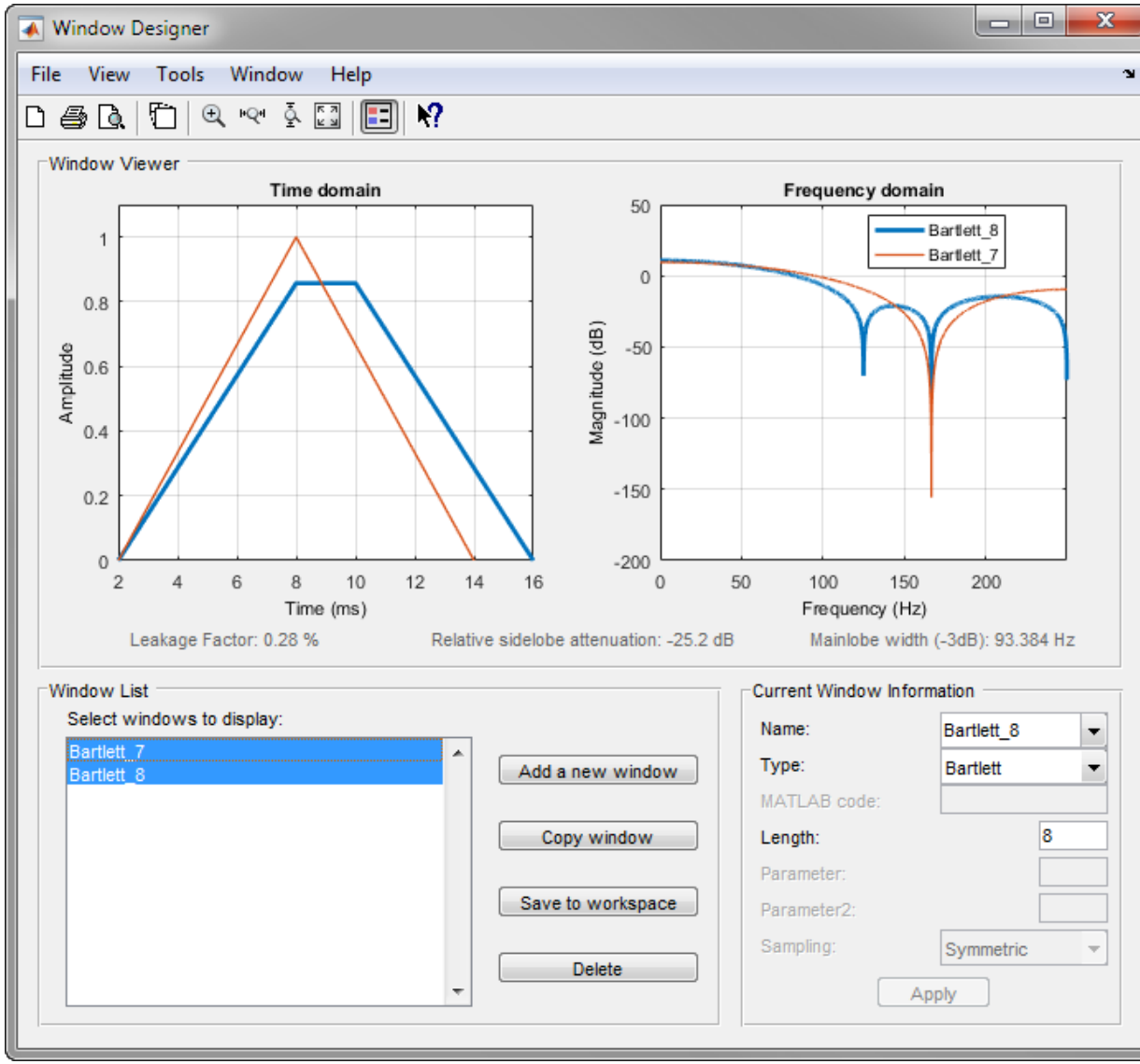
```
Bartlett = bartlett(7);
isequal(Bartlett(2:end-1),triang(5))
```

```
ans =
     1
```

For `n` even, `bartlett` is still the convolution of two rectangular sequences. There is no standard definition for the triangular window for `n` even; the slopes of the line segments of the `triang` result are slightly steeper than those of `bartlett` in this case:

```
w = bartlett(8);
[w(2:7) triang(6)]
```

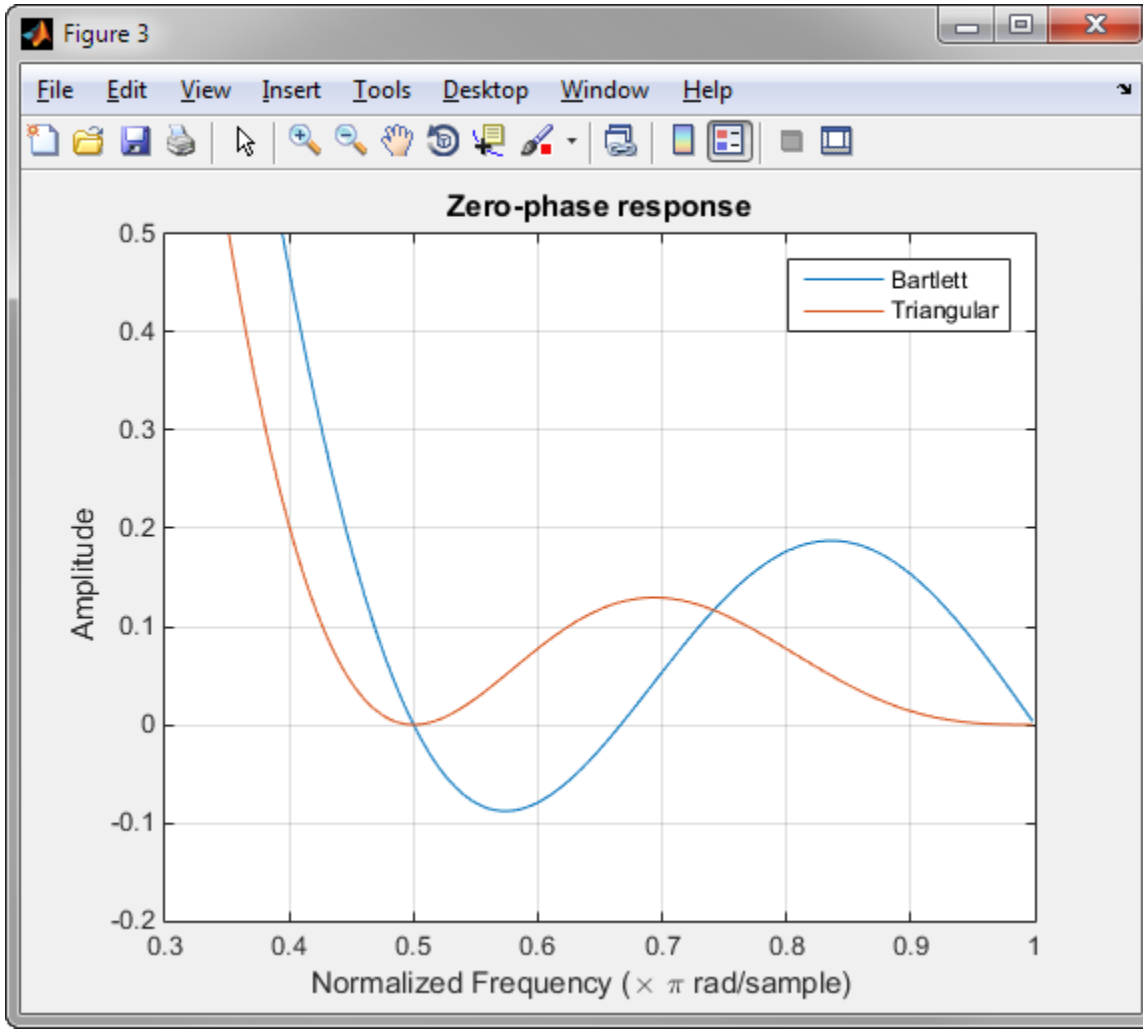
You can see the difference between odd and even Bartlett windows in Window Designer.



The final difference between the Bartlett and triangular windows is evident in the Fourier transforms of these functions. The Fourier transform of a Bartlett window is negative for n even. The Fourier transform of a triangular window, however, is always nonnegative.

The following figure, which plots the zero-phase responses of 8-point Bartlett and Triangular windows, illustrates the difference.

```
zerophase(bartlett(8))
hold on
zerophase(triang(8))
legend('Bartlett','Triangular')
axis([0.3 1 -0.2 0.5])
```



This difference can be important when choosing a window for some spectral estimation techniques, such as the Blackman-Tukey method. Blackman-Tukey forms the spectral estimate by calculating the Fourier transform of the autocorrelation sequence. The resulting estimate might be negative at some frequencies if the window's Fourier transform is negative.

See Also

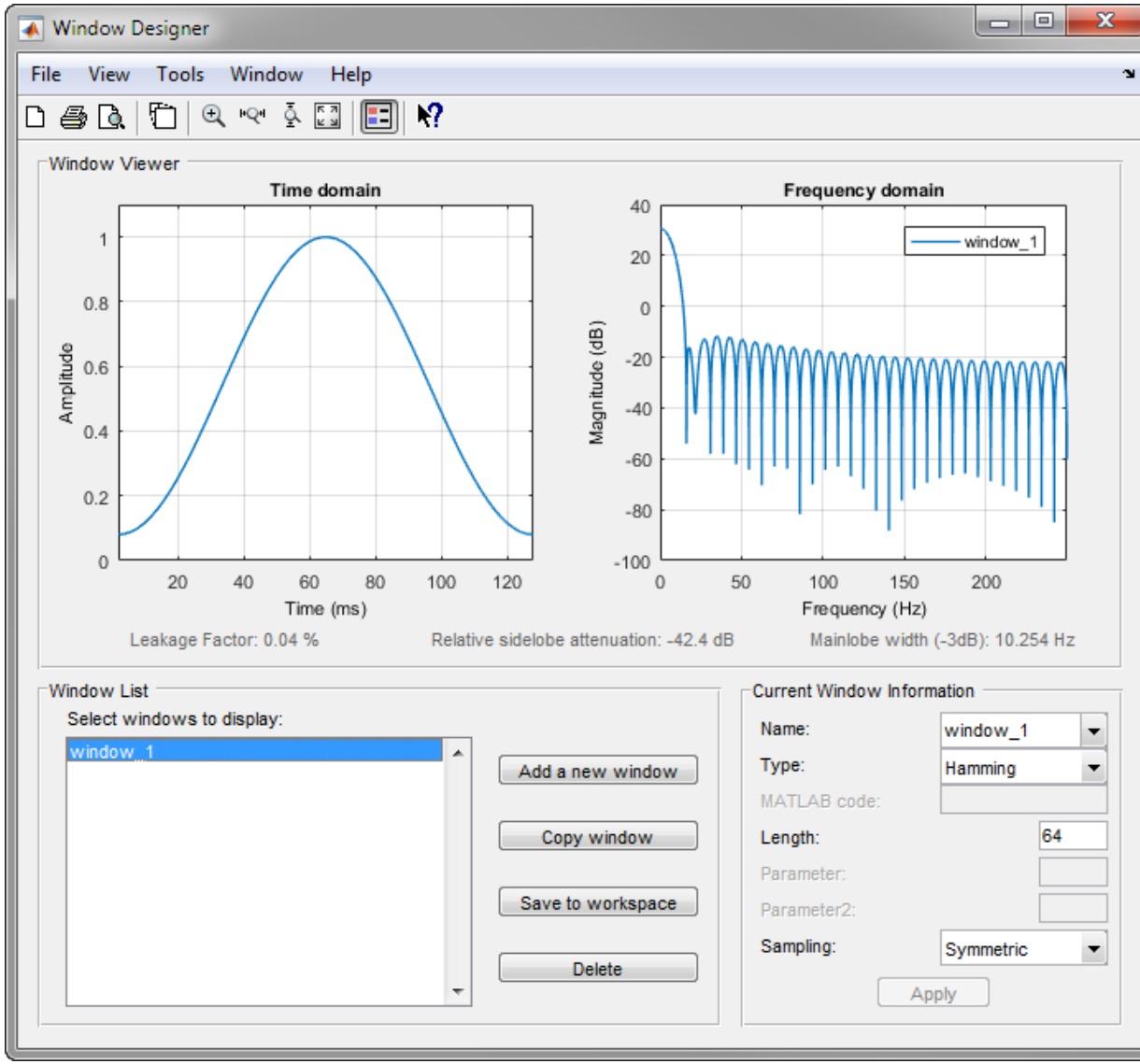
Apps
Window Designer

Functions
`bartlett` | `rectwin` | `triang` | `wvtool`

Getting Started with Window Designer

Typing `windowDesigner` at the command line opens the **Window Designer** app for designing and analyzing spectral windows. The app opens with a default 64-point Hamming window.

Note A related tool, `wvtool`, is available for displaying, annotating, or printing windows.



The app has three panels:

- Window Viewer displays the time domain and frequency domain representations of the selected window(s). The currently active window is shown in bold. Three window measurements are shown below the plots.
  - Leakage factor — ratio of power in the sidelobes to the total window power
  - Relative sidelobe attenuation — difference in height from the mainlobe peak to the highest sidelobe peak
  - Mainlobe width (–3dB) — width of the mainlobe at 3 dB below the mainlobe peak
- Window List lists the windows available for display in the Window Viewer. Highlight one or more windows to display them. The Window List buttons are:
  - **Add a new window** — Adds a default Hamming window with length 64 and symmetric sampling. You can change the information for this window by applying changes made in the **Current Window Information** panel.
  - **Copy window** — Copies the selected window(s).
  - **Save to workspace** — Saves the selected window(s) as vector(s) to the MATLAB workspace. The name of the window is used as the vector name.
  - **Delete** — Removes the selected window(s) from the window list.
- *Current Window Information* displays information about the currently active window. The active window name is shown in the `Name` field. To make another window active, select its name from the **Name** menu.

Window Parameters

Each window is defined by the parameters in the Current Window Information panel. You can change the current window's characteristics by changing its parameters and clicking **Apply**. The parameters of the current window are

- **Name** — Name of the window. The name is used for the legend in the Window Viewer, in the Window List, and for the vector saved to the workspace. You can either select a name from the menu or type the desired name in the edit box.
- **Type** — Algorithm for the window. Select the type from the menu. All Signal Processing Toolbox windows are available.
- **MATLAB code** — Any valid MATLAB expression that returns a vector defining the window if `Type = User Defined`.
- **Length** — Number of samples.

- **Parameter** — Additional parameter for windows that require it, such as Chebyshev, which requires you to specify the sidelobe attenuation. Note that the title “Parameter” changes to the appropriate parameter name.
- **Sampling** — Type of sampling to use for generalized cosine windows (Hamming, Hann, and Blackman) — `Periodic` or `Symmetric`. `Periodic` computes a length `n+1` window and returns the first `n` points, and `Symmetric` computes and returns the `n` points specified in `Length`.

Window Designer Menus

In addition to the usual menu items, **Window Designer**contains these menu commands:

File menu:

- **Export** — Exports window coefficient vectors to the MATLAB workspace, a text file, or a MAT-file.

  In the **Window List** in, highlight the window(s) you want to export and then select **File > Export**. For exporting to the workspace or a MAT-file, specify the variable name for each set of window coefficients. To overwrite variables in the workspace, select the Overwrite variables check box.
- **Full View Analysis** — Copies the windows shown in both plots to a separate `wvtool` figure window. This is useful for printing and annotating. This option is also available with the Full View Analysis toolbar button.

View menu:

- **Time domain** — Select to show the time domain plot in the Window Viewer panel.
- **Frequency domain** — Select to show the frequency domain plot in the Window Viewer panel.
- **Legend** — Toggles the window name legend on and off. This option is also available with the Legend toolbar button.
- **Analysis Parameters** — Controls the response plot parameters, including number of points, range, *x*- and *y*-axis units, sampling frequency, and normalized magnitude.

  You can also access the Analysis Parameters by right-clicking the *x*-axis label of a plot in the Window Viewer panel. The *x*-axis units for the time domain plot depend on the selected `Sampling Frequency` units.

Frequency Domain	Time Domain
Hz	s
kHz	ms
MHz	µs
GHz	ns

Tools menu:

- **Zoom In** — Zooms in along both *x*- and *y*-axes.
- **Zoom X** — Zooms in along the *x*-axis only. Drag the mouse in the *x* direction to select the zoom area.
- **Zoom Y** — Zooms in along the y-axis only. Drag the mouse in the *y* direction to select the zoom area.
- **Full View** — Returns to full view.

See Also

Functions
`window` | `wvtool`

Generalized Cosine Windows

Blackman, flat top, Hamming, Hann, and rectangular windows are all special cases of the *generalized cosine window*. These windows are combinations of sinusoidal sequences with frequencies that are multiples of $2\pi/(N-1)$, where N is the window length. One special case is the Blackman window:

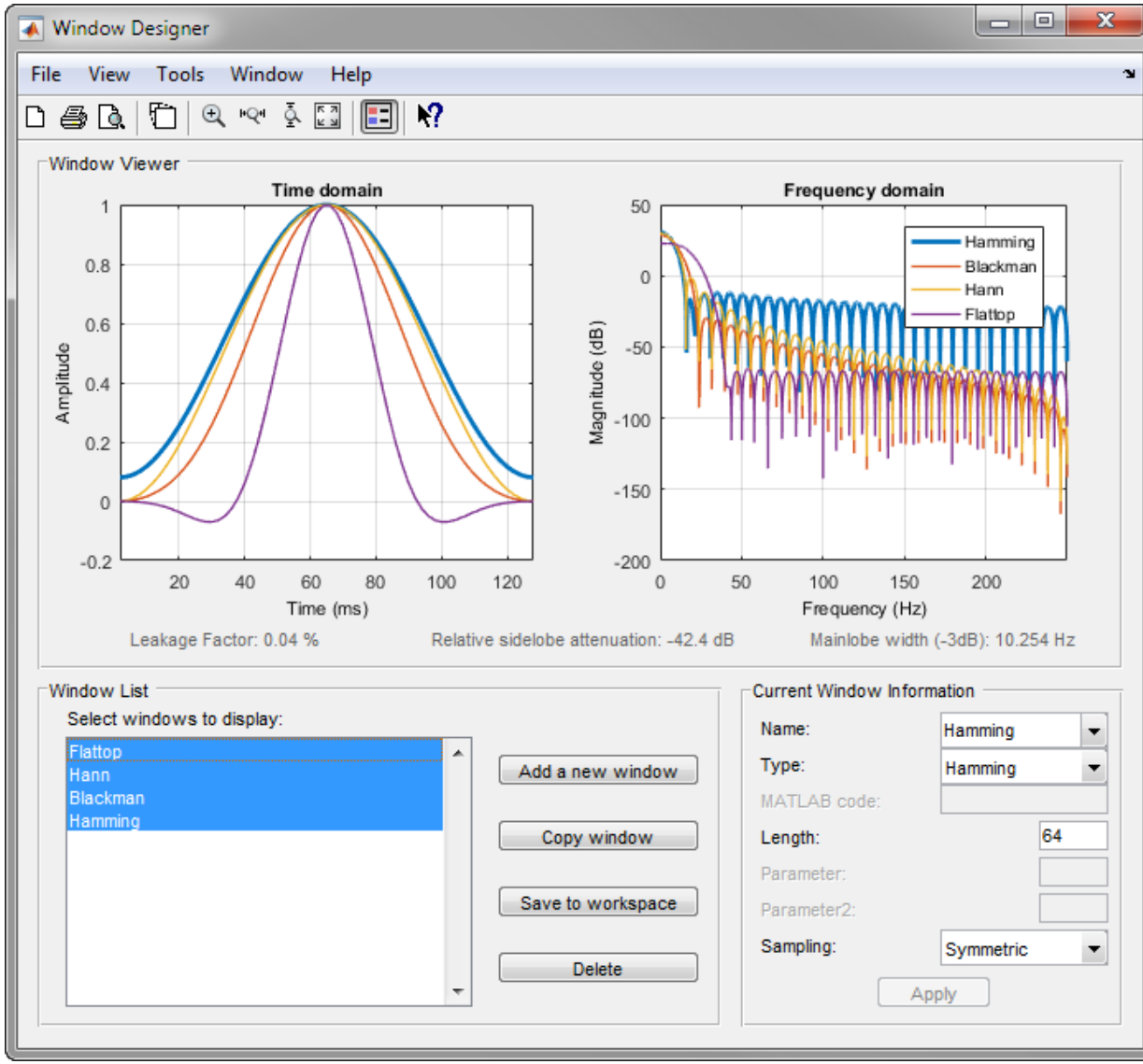
```
N = 128;
A = 0.42;
B = 0.5;
C = 0.08;
ind = (0:N-1)'*2*pi/(N-1);
w = A - B*cos(ind) + C*cos(2*ind);
```

Changing the values of the constants `A`, `B`, and `C` in the previous expression generates different generalized cosine windows like the Hamming and Hann windows. Adding additional cosine terms of higher frequency generates the flat top window. The concept behind these windows is that by summing the individual terms to form the window, the low frequency peaks in the frequency domain combine in such a way as to decrease sidelobe height. This has the side effect of increasing the mainlobe width.

The Hamming and Hann windows are two-term generalized cosine windows, given by `A = 0.54`, `B = 0.46` for the Hamming and `A = 0.5`, `B = 0.5` for the Hann.

Note that the definition of the generalized cosine window shown in the earlier MATLAB code yields zeros at samples 1 and `n` for `A = 0.5` and `B = 0.5`.

This **Window Designer** screen shot compares Blackman, Hamming, Hann, and Flat Top windows.


Window Designer
File View Tools Window Help
Window Viewer
Time domain
Amplitude
Time (ms)
Frequency domain
Magnitude (dB)
Frequency (Hz)
Hamming
Blackman
Hann
Flattop
Leakage Factor: 0.04 %
Relative sidelobe attenuation: -42.4 dB
Mainlobe width (-3dB): 10.254 Hz
Window List
Select windows to display:
Flattop
Hann
Blackman
Hamming
Add a new window
Copy window
Save to workspace
Delete
Current Window Information
Name: Hamming
Type: Hamming
MATLAB code:
Length: 64
Parameter:
Parameter2:
Sampling: Symmetric
Apply

See Also

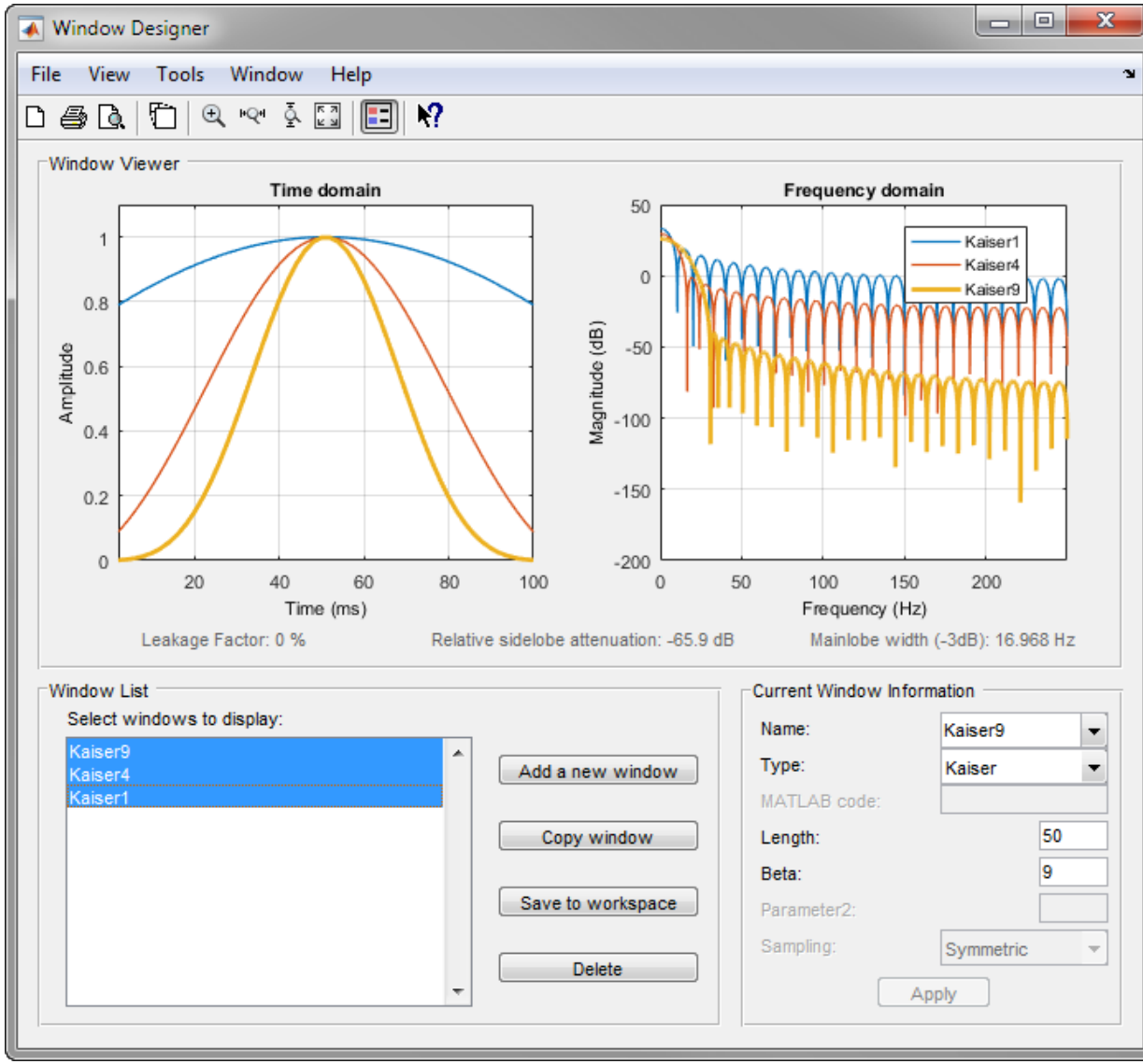
Apps
Window Designer

Functions
`blackman` | `flattopwin` | `hamming` | `hann` | `wvtool`

Kaiser Window

The Kaiser window is an approximation to the prolate spheroidal window, for which the ratio of the mainlobe energy to the sidelobe energy is maximized. For a Kaiser window of a particular length, the parameter β controls the sidelobe height. For a given β, the sidelobe height is fixed with respect to window length. The statement `kaiser(n,beta)` computes a length `n` Kaiser window with parameter `beta`.

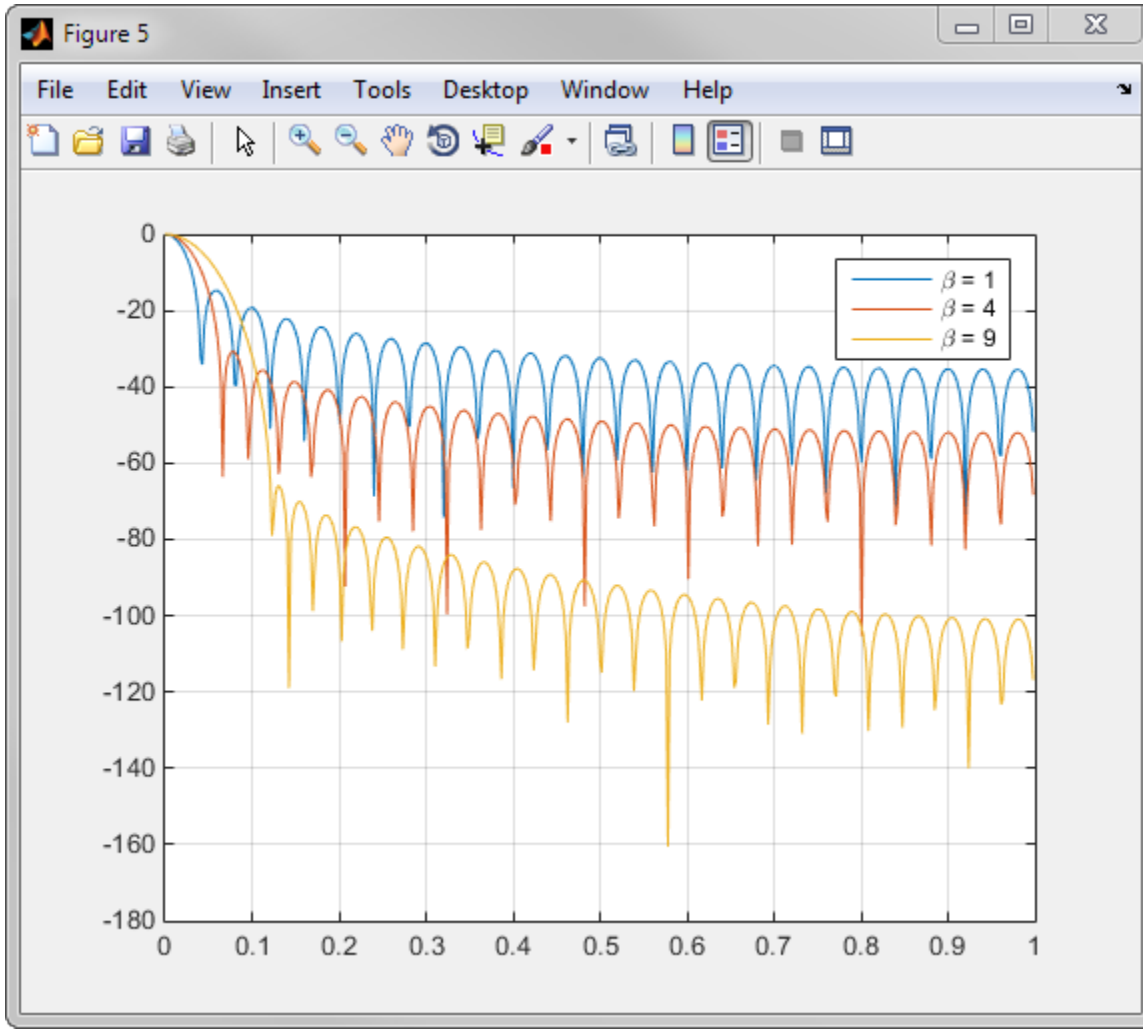
Examples of Kaiser windows with length 50 and β parameters of 1, 4, and 9 are shown in this example.



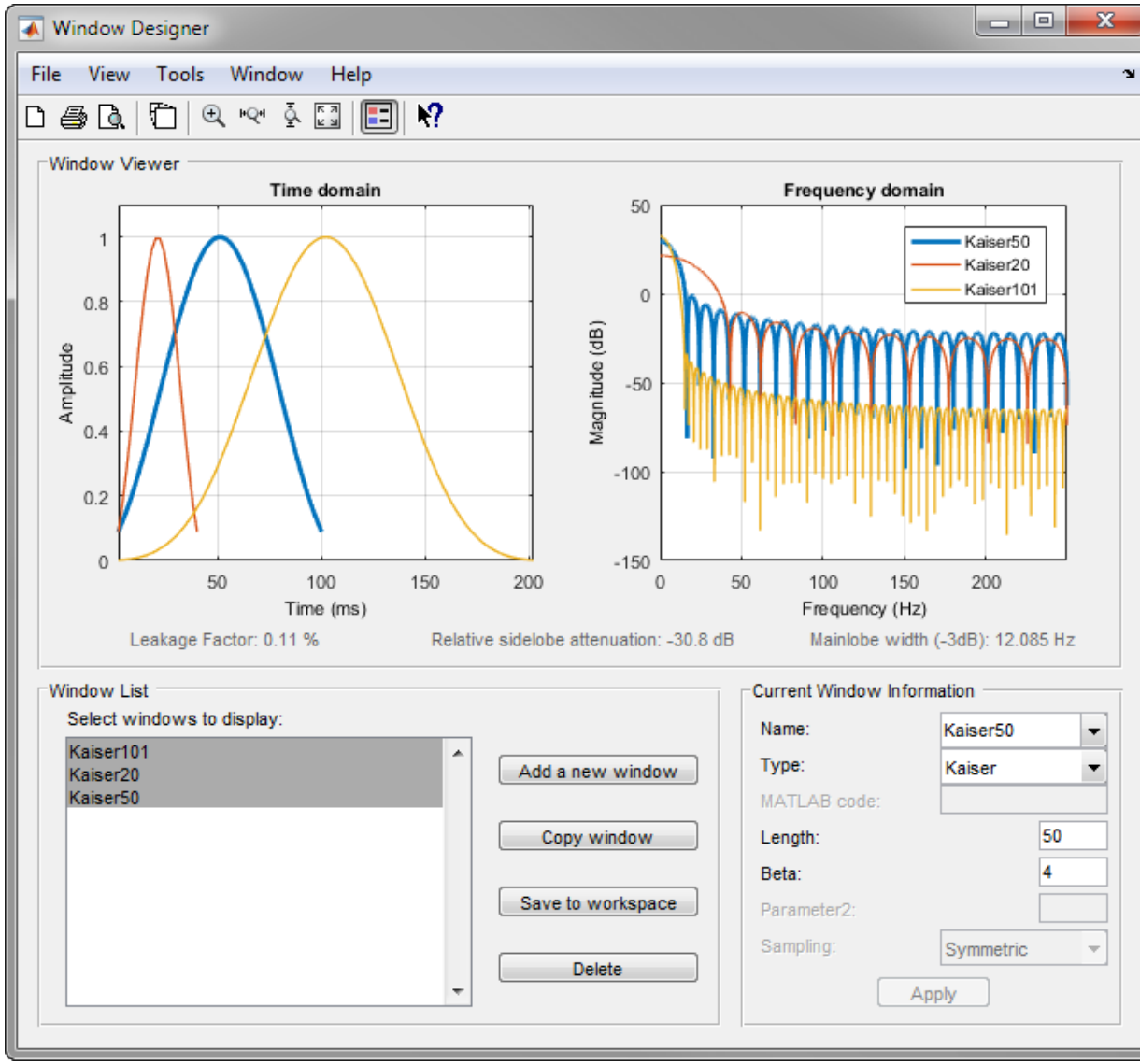
To create these Kaiser windows using the MATLAB command line, type the following:

```
n = 50;
w1 = kaiser(n,1);
```

```
w2 = kaiser(n,4);
w3 = kaiser(n,9);
[W1,f] = freqz(w1/sum(w1),1,512,2);
[W2,f] = freqz(w2/sum(w2),1,512,2);
[W3,f] = freqz(w3/sum(w3),1,512,2);
plot(f,20*log10(abs([W1 W2 W3])))
grid
legend('\beta = 1','\beta = 4','\beta = 9')
```



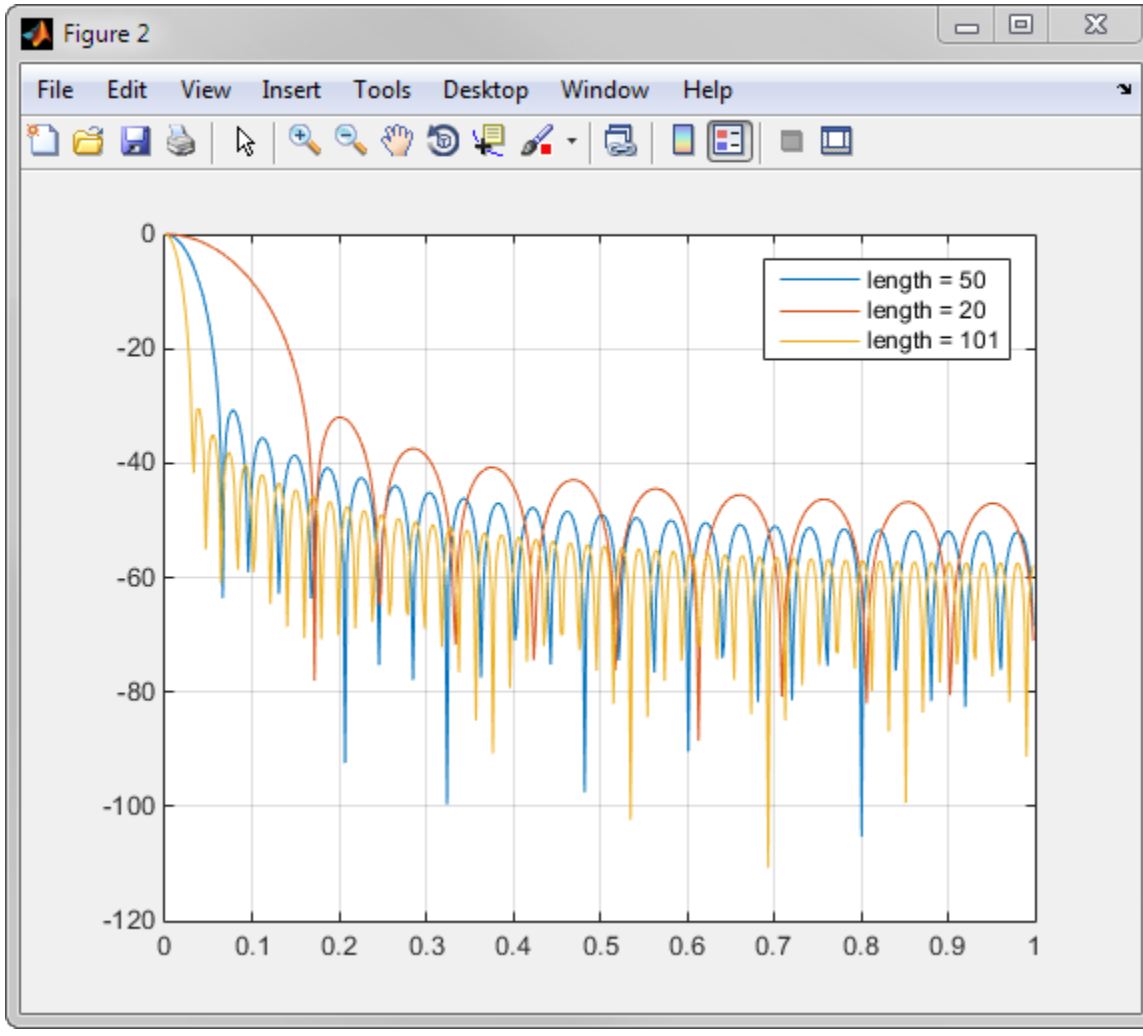
As β increases, the sidelobe height decreases and the mainlobe width increases. This screen shot shows how the sidelobe height stays the same for a fixed β parameter as the length is varied.



To create these Kaiser windows using the MATLAB command line, type the following:

```
w1 = kaiser(50,4);
w2 = kaiser(20,4);
```

```
w3 = kaiser(101,4);
[W1,f] = freqz(w1/sum(w1),1,512,2);
[W2,f] = freqz(w2/sum(w2),1,512,2);
[W3,f] = freqz(w3/sum(w3),1,512,2);
plot(f,20*log10(abs([W1 W2 W3])))
grid
legend('length = 50','length = 20','length = 101')
```



Kaiser Windows in FIR Design

There are two design formulas that can help you design FIR filters to meet a set of filter specifications using a Kaiser window. To achieve a sidelobe height of –α dB, the β (`beta`) parameter is

$$\beta = \begin{cases} 0.1102(\alpha - 8.7), & \alpha > 50, \\ 0.5842(\alpha - 21)^{0.4} + 0.7886(\alpha - 21), & 50 \geq \alpha \geq 21, \\ 0, & \alpha < 21. \end{cases}$$

For a transition width of $\Delta\omega$ rad/sample, use the length

$$n = \frac{\alpha - 8}{2.285\Delta\omega} + 1$$

.

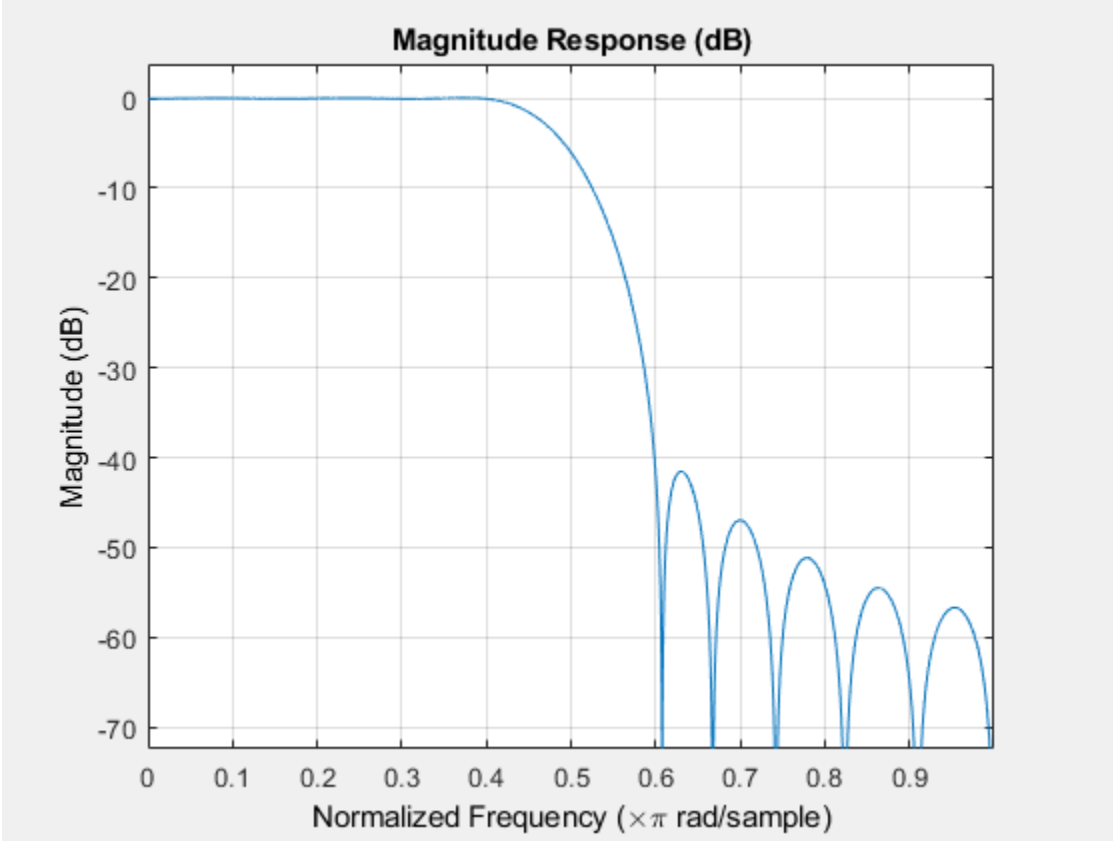
Filters designed using these heuristics will meet the specifications approximately, but you should verify this. To design a lowpass filter with cutoff frequency 0.5π rad/sample, transition width 0.2π rad/sample, and 40 dB of attenuation in the stopband, try

```
[n,wn,beta] = kaiserord([0.4 0.6]*pi,[1 0],[0.01 0.01],2*pi);
h = fir1(n,wn,kaiser(n+1,beta),'noscale');
```

The `kaiserord` function estimates the filter order, cutoff frequency, and Kaiser window beta parameter needed to meet a given set of frequency domain specifications.

The ripple in the passband is roughly the same as the ripple in the stopband. As you can see from the frequency response, this filter nearly meets the specifications:

```
fvtool(h,1)
```



See Also

Apps
Window Designer

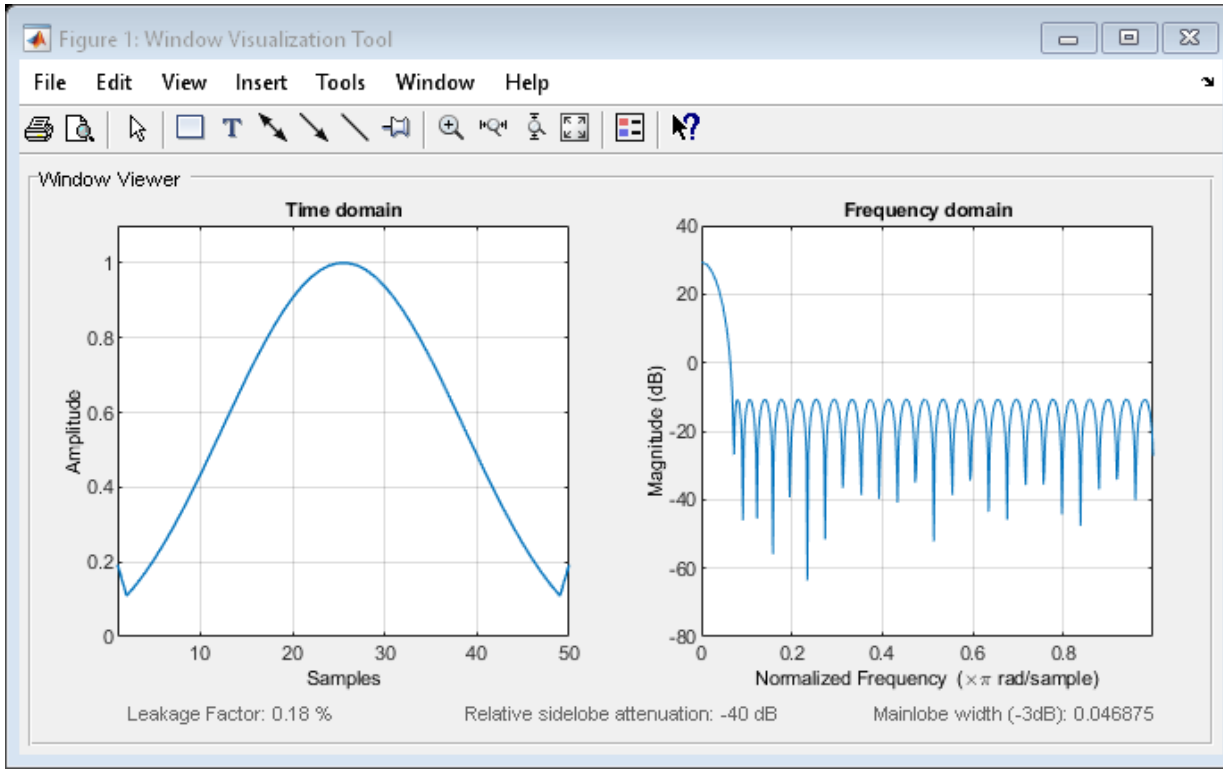
Functions
`freqz` | `kaiser` | `kaiserord` | `wvtool`

Chebyshev Window

The Chebyshev window minimizes the mainlobe width, given a particular sidelobe height. It is characterized by an equiripple behavior. Its sidelobes all have the same height.

Generate and display a 50-point Chebyshev window with a sidelobe attenuation of 40 dB.

```
w = chebwin(50,40);
wvtool(w)
```



As shown in the time-domain plot, the Chebyshev window has large spikes at its outer samples.

See Also

Apps
Window Designer

Functions
`chebwin` | `wvtool`

Parametric Modeling

In this section...
“What is Parametric Modeling” on page 7-26
“Available Parametric Modeling Functions” on page 7-26
“Time-Domain Based Modeling” on page 7-27
“Frequency-Domain Based Modeling” on page 7-30

What is Parametric Modeling

Parametric modeling techniques find the parameters for a mathematical model describing a signal, system, or process. These techniques use known information about the system to determine the model. Applications for parametric modeling include speech and music synthesis, data compression, high-resolution spectral estimation, communications, manufacturing, and simulation.

Available Parametric Modeling Functions

The toolbox parametric modeling functions operate with the rational transfer function model. Given appropriate information about an unknown system (impulse or frequency response data, or input and output sequences), these functions find the coefficients of a linear system that models the system.

One important application of the parametric modeling functions is in the design of filters that have a prescribed time or frequency response.

Here is a summary of the parametric modeling functions in this toolbox.

Domain	Functions	Description
Time	`arburg`	Generate all-pole filter coefficients that model an input data sequence using the Levinson-Durbin algorithm.
	`arcov`	Generate all-pole filter coefficients that model an input data sequence by minimizing the forward prediction error.

Domain	Functions	Description
	`armcov`	Generate all-pole filter coefficients that model an input data sequence by minimizing the forward and backward prediction errors.
	`aryule`	Generate all-pole filter coefficients that model an input data sequence using an estimate of the autocorrelation function.
	`lpc`, `levinson`	Linear Predictive Coding. Generate all-pole recursive filter whose impulse response matches a given sequence.
	`prony`	Generate IIR filter whose impulse response matches a given sequence.
	`stmcb`	Find IIR filter whose output, given a specified input sequence, matches a given output sequence.
Frequency	`invfreqz`, `invfreqs`	Generate digital or analog filter coefficients given complex frequency response data.

Time-Domain Based Modeling

The `lpc`, `prony`, and `stmcb` functions find the coefficients of a digital rational transfer function that approximates a given time-domain impulse response. The algorithms differ in complexity and accuracy of the resulting model.

Linear Prediction

Linear prediction modeling assumes that each output sample of a signal, `x(k)`, is a linear combination of the past `n` outputs (that is, it can be linearly predicted from these outputs), and that the coefficients are constant from sample to sample:

$$x(k) = -a(2)x(k-1) - a(3)x(k-2) - \ldots - a(n+1)x(k-n)$$

An `n`th-order all-pole model of a signal `x` is

```
a = lpc(x,n)
```

To illustrate `lpc`, create a sample signal that is the impulse response of an all-pole filter with additive white noise:

```
x = impz(1,[1 0.1 0.1 0.1 0.1],10) + randn(10,1)/10;
```

The coefficients for a fourth-order all-pole filter that models the system are

```
a = lpc(x,4)
```

`lpc` first calls `xcorr` to find a biased estimate of the correlation function of `x`, and then uses the Levinson-Durbin recursion, implemented in the `levinson` function, to find the model coefficients `a`. The Levinson-Durbin recursion is a fast algorithm for solving a system of symmetric Toeplitz linear equations. `lpc`'s entire algorithm for `n = 4` is

```
r = xcorr(x);
r(1:length(x)-1) = [];      % Remove corr. at negative lags
a = levinson(r,4)
```

You could form the linear prediction coefficients with other assumptions by passing a different correlation estimate to `levinson`, such as the biased correlation estimate:

```
r = xcorr(x,'biased');
r(1:length(x)-1) = [];      % Remove corr. at negative lags
a = levinson(r,4)
```

Prony's Method (ARMA Modeling)

The `prony` function models a signal using a specified number of poles and zeros. Given a sequence `x` and numerator and denominator orders `n` and `m`, respectively, the statement

```
[b,a] = prony(x,n,m)
```

finds the numerator and denominator coefficients of an IIR filter whose impulse response approximates the sequence `x`.

The `prony` function implements the method described in [4] Parks and Burrus (pgs. 226-228). This method uses a variation of the covariance method of AR modeling to find the denominator coefficients `a`, and then finds the numerator coefficients `b` for which the resulting filter's impulse response matches exactly the first `n + 1` samples of `x`. The filter is not necessarily stable, but it can potentially recover the coefficients exactly if the data sequence is truly an autoregressive moving-average (ARMA) process of the correct order.

Note The functions `prony` and `stmcb` (described next) are more accurately described as ARX models in system identification terminology. ARMA modeling assumes noise only at the inputs, while ARX assumes an external input. `prony` and `stmcb` know the input signal: it is an impulse for `prony` and is arbitrary for `stmcb`.

A model for the test sequence `x` (from the earlier `lpc` example) using a third-order IIR filter is

```
[b,a] = prony(x,3,3)
```

The `impz` command shows how well this filter's impulse response matches the original sequence:

```
format long
[x impz(b,a,10)]
```

Notice that the first four samples match exactly. For an example of exact recovery, recover the coefficients of a Butterworth filter from its impulse response:

```
[b,a] = butter(4,.2);
h = impz(b,a,26);
[bb,aa] = prony(h,4,4);
```

Try this example; you'll see that `bb` and `aa` match the original filter coefficients to within a tolerance of 10^{-13}.

Steiglitz-McBride Method (ARMA Modeling)

The `stmcb` function determines the coefficients for the system $b(z)/a(z)$ given an approximate impulse response `x`, as well as the desired number of zeros and poles. This function identifies an unknown system based on both input and output sequences that describe the system's behavior, or just the impulse response of the system. In its default mode, `stmcb` works like `prony`.

```
[b,a] = stmcb(x,3,3)
```

`stmcb` also finds systems that match given input and output sequences:

```
y = filter(1,[1 1],x);     % Create an output signal.
[b,a] = stmcb(y,x,0,1)
```

In this example, `stmcb` correctly identifies the system used to create `y` from `x`.

The Steiglitz-McBride method is a fast iterative algorithm that solves for the numerator and denominator coefficients simultaneously in an attempt to minimize the signal error between the filter output and the given output signal. This algorithm usually converges rapidly, but might not converge if the model order is too large. As for `prony`, `stmcb`'s resulting filter is not necessarily stable due to its exact modeling approach.

`stmcb` provides control over several important algorithmic parameters; modify these parameters if you are having trouble modeling the data. To change the number of iterations from the default of five and provide an initial estimate for the denominator coefficients:

```
n = 10;             % Number of iterations
a = lpc(x,3);       % Initial estimates for denominator
[b,a] = stmcb(x,3,3,n,a);
```

The function uses an all-pole model created with `prony` as an initial estimate when you do not provide one of your own.

To compare the functions `lpc`, `prony`, and `stmcb`, compute the signal error in each case:

```
a1 = lpc(x,3);
[b2,a2] = prony(x,3,3);
[b3,a3] = stmcb(x,3,3);
[x-impz(1,a1,10)  x-impz(b2,a2,10)  x-impz(b3,a3,10)]
```

In comparing modeling capabilities for a given order IIR model, the last result shows that for this example, `stmcb` performs best, followed by `prony`, then `lpc`. This relative performance is typical of the modeling functions.

Frequency-Domain Based Modeling

The `invfreqs` and `invfreqz` functions implement the inverse operations of `freqs` and `freqz`; they find an analog or digital transfer function of a specified order that matches a given complex frequency response. Though the following examples demonstrate `invfreqz`, the discussion also applies to `invfreqs`.

To recover the original filter coefficients from the frequency response of a simple digital filter:

```
[b,a] = butter(4,0.4)         % Design Butterworth lowpass
[h,w] = freqz(b,a,64);        % Compute frequency response
[b4,a4] = invfreqz(h,w,4,4)   % Model: n = 4, m = 4
```

The vector of frequencies `w` has the units in rad/sample, and the frequencies need not be equally spaced. `invfreqz` finds a filter of any order to fit the frequency data; a third-order example is

```
[b4,a4] = invfreqz(h,w,3,3)   % Find third-order IIR
```

Both `invfreqs` and `invfreqz` design filters with real coefficients; for a data point at positive frequency `f`, the functions fit the frequency response at both `f` and `-f`.

By default `invfreqz` uses an equation error method to identify the best model from the data. This finds *b* and *a* in

$$\min_{b,a} \sum_{k=1}^{n} wt(k)\left|h(k)A(w(k)) - B(w(k))\right|^2$$

by creating a system of linear equations and solving them with the MATLAB `\` operator. Here $A(w(k))$ and $B(w(k))$ are the Fourier transforms of the polynomials `a` and `b` respectively at the frequency $w(k)$, and n is the number of frequency points (the length of `h` and `w`). $wt(k)$ weights the error relative to the error at different frequencies. The syntax

```
invfreqz(h,w,n,m,wt)
```

includes a weighting vector. In this mode, the filter resulting from `invfreqz` is not guaranteed to be stable.

`invfreqz` provides a superior ("output-error") algorithm that solves the direct problem of minimizing the weighted sum of the squared error between the actual frequency response points and the desired response

$$\min_{b,a} \sum_{k=1}^{n} wt(k)\left|h(k) - \frac{B(w(k))}{A(w(k))}\right|^2$$

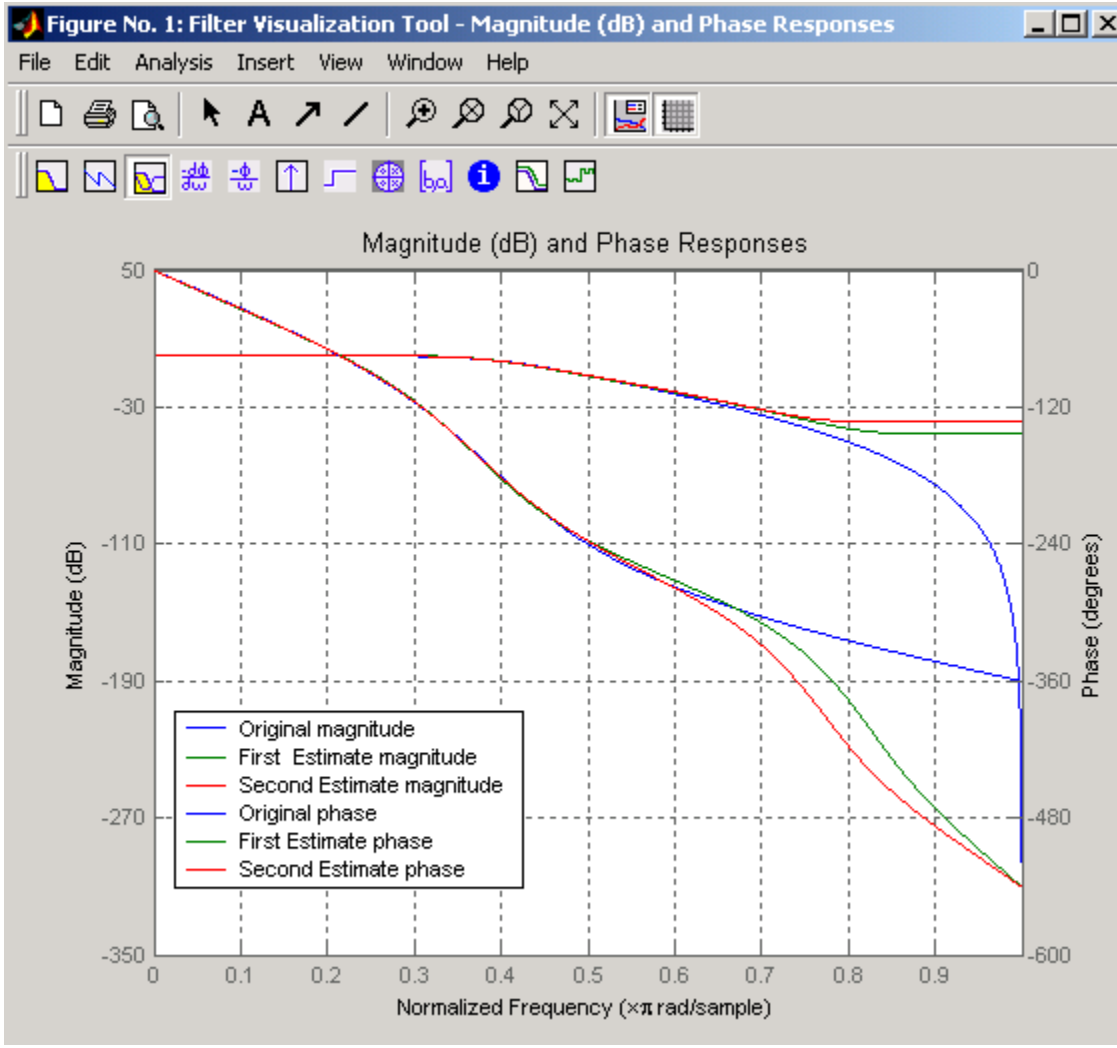
To use this algorithm, specify a parameter for the iteration count after the weight vector parameter:

```
wt = ones(size(w));   % Create unity weighting vector
[b30,a30] = invfreqz(h,w,3,3,wt,30)  % 30 iterations
```

The resulting filter is always stable.

Graphically compare the results of the first and second algorithms to the original Butterworth filter with FVTool (and select the Magnitude and Phase Responses):

```
fvtool(b,a,b4,a4,b30,a30)
```



To verify the superiority of the fit numerically, type

```
sum(abs(h-freqz(b4,a4,w)).^2)    % Total error, algorithm 1
sum(abs(h-freqz(b30,a30,w)).^2)  % Total error, algorithm 2
```

Resampling

In this section...
“Available Resampling Functions” on page 7-33
“resample Function” on page 7-33
“decimate and interp Functions” on page 7-35
“upfirdn Function” on page 7-35
“spline Function” on page 7-35

Available Resampling Functions

The toolbox provides a number of functions that resample a signal at a higher or lower rate.

Operation	Function
Apply FIR filter with resampling	`upfirdn`
Cubic spline interpolation	`spline`
Decimation	`decimate`
Interpolation	`interp`
Other 1-D interpolation	`interp1`
Resample at new rate	`resample`

resample Function

The `resample` function changes the sampling rate for a sequence to any rate that is a ratio of two integers. The basic syntax for `resample` is

```
y = resample(x,p,q)
```

where the function resamples the sequence `x` at `p/q` times the original sampling rate. The length of the result `y` is `p/q` times the length of `x`.

One resampling application is the conversion of digitized audio signals from one sampling rate to another, such as from 48 kHz (the digital audio tape standard) to 44.1 kHz (the compact disc standard).

The example file contains a length 4001 vector of speech sampled at 7418 Hz:

```
clear
load mtlb
whos
```

```
Name         Size            Bytes  Class     Attributes

Fs           1x1                 8  double
mtlb      4001x1             32008  double
```

```
Fs
```

```
Fs =

        7418
```

To play this speech signal on a workstation that can only play sound at 8192 Hz, use the `rat` function to find integers `p` and `q` that yield the correct resampling factor:

```
[p,q] = rat(8192/Fs,0.0001)
```

```
p =
   127
q =
   115
```

Since `p/q*Fs = 8192.05` Hz, the tolerance of 0.0001 is acceptable; to resample the signal at very close to 8192 Hz:

```
y = resample(mtlb,p,q);
```

`resample` applies a lowpass filter to the input sequence to prevent aliasing during resampling. It designs this filter using the `firls` function with a Kaiser window. The syntax

```
resample(x,p,q,l,beta)
```

controls the filter's length and the beta parameter of the Kaiser window. Alternatively, use the function `intfilt` to design an interpolation filter `b` and use it with

```
resample(x,p,q,b)
```

decimate and interp Functions

The `decimate` and `interp` functions do the same thing as `resample` with `p = 1` and `q = 1`, respectively. These functions provide different anti-alias filtering options, and they incur a slight signal delay due to filtering. The `interp` function is significantly less efficient than the `resample` function with `q = 1`.

upfirdn Function

The toolbox also contains a function, `upfirdn`, that applies an FIR filter to an input sequence and outputs the filtered sequence at a sample rate different than its original. See “Multirate Filter Bank Implementation” on page 1-8.

spline Function

The standard MATLAB environment contains a function, `spline`, that works with irregularly spaced data. The MATLAB function `interp1` performs interpolation, or table lookup, using various methods including linear and cubic interpolation.

Cepstrum Analysis

What Is a Cepstrum?

Cepstrum analysis is a nonlinear signal processing technique with a variety of applications in areas such as speech and image processing.

The *complex cepstrum* of a sequence x is calculated by finding the complex natural logarithm of the Fourier transform of x, then the inverse Fourier transform of the resulting sequence:

$$\hat{x} = \frac{1}{2\pi}\int_{-\pi}^{\pi} \log[X(e^{j\omega})]e^{j\omega n}\,d\omega.$$

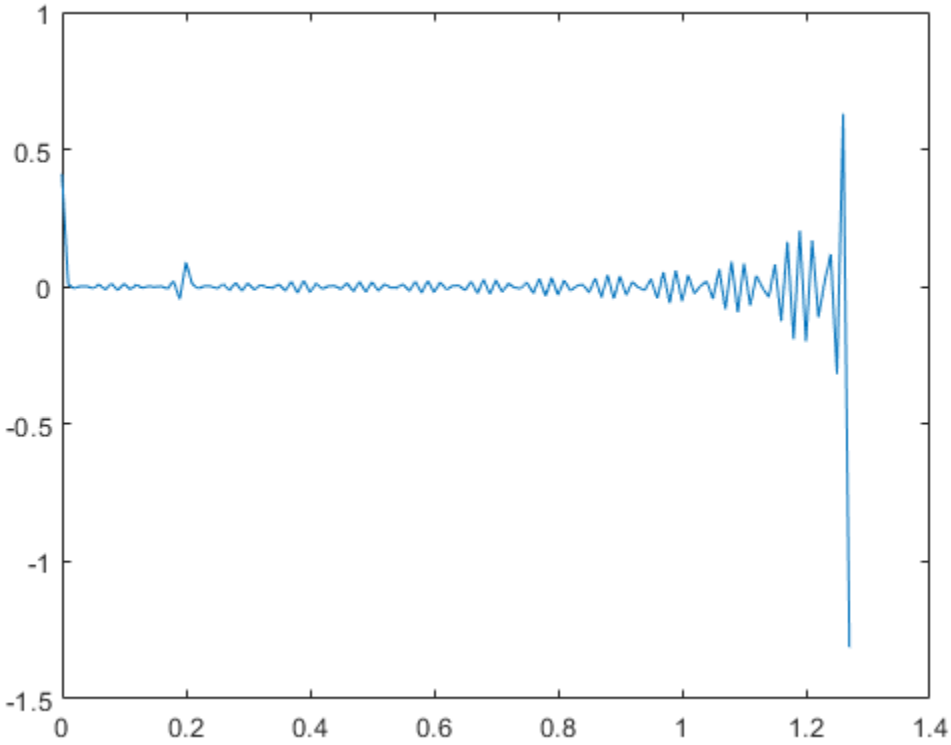
The toolbox function `cceps` performs this operation, estimating the complex cepstrum for an input sequence. It returns a real sequence the same size as the input sequence.

Try using cceps in an echo detection application. First, create a 45 Hz sine wave sampled at 100 Hz. Add an echo of the signal, with half the amplitude, 0.2 seconds after the beginning of the signal.

```
t = 0:0.01:1.27;
s1 = sin(2*pi*45*t);
s2 = s1 + 0.5*[zeros(1,20) s1(1:108)];
```

Compute and plot the complex cepstrum of the new signal.

```
c = cceps(s2);
plot(t,c)
```



The complex cepstrum shows a peak at 0.2 seconds, indicating the echo.

The *real cepstrum* of a signal x, sometimes called simply the cepstrum, is calculated by determining the natural logarithm of magnitude of the Fourier transform of x, then obtaining the inverse Fourier transform of the resulting sequence:

$$c_x = \frac{1}{2\pi}\int_{-\pi}^{\pi} \log|X(e^{j\omega})|e^{j\omega n}d\omega.$$

The toolbox function `rceps` performs this operation, returning the real cepstrum for a sequence. The returned sequence is a real-valued vector the same size as the input vector.

The `rceps` function also returns a unique minimum-phase sequence that has the same real cepstrum as the input. To obtain both the real cepstrum and the minimum-phase reconstruction for a sequence, use `[y,ym] = rceps(x)`, where `y` is the real cepstrum and `ym` is the minimum phase reconstruction of `x`. The following example shows that one output of `rceps` is a unique minimum-phase sequence with the same real cepstrum as `x`.

```
y = [4 1 5];                  % Non-minimum phase sequence
[xhat,yhat] = rceps(y);
xhat2 = rceps(yhat);
[xhat' xhat2']
```

```
ans = 3×2

    1.6225    1.6225
    0.3400    0.3400
    0.3400    0.3400
```

Inverse Complex Cepstrum

To invert the complex cepstrum, use the `icceps` function. Inversion is complicated by the fact that the `cceps` function performs a data-dependent phase modification so that the unwrapped phase of its input is continuous at zero frequency. The phase modification is equivalent to an integer delay. This delay term is returned by `cceps` if you ask for a second output:

```
x = 1:10;
[xhat,delay] = cceps(x)
```

```
xhat = 1×10

    2.2428   -0.0420   -0.0210    0.0045    0.0366    0.0788    0.1386    0.2327    0.4

delay = 1
```

To invert the complex cepstrum, use `icceps` with the original delay parameter:

```
icc = icceps(xhat,2)
```

```
icc = 1×10

    2.0000    3.0000    4.0000    5.0000    6.0000    7.0000    8.0000    9.0000   10.0
```

As shown in the above example, with any modification of the complex cepstrum, the original delay term may no longer be valid. You will not be able to invert the complex cepstrum exactly.

See Also

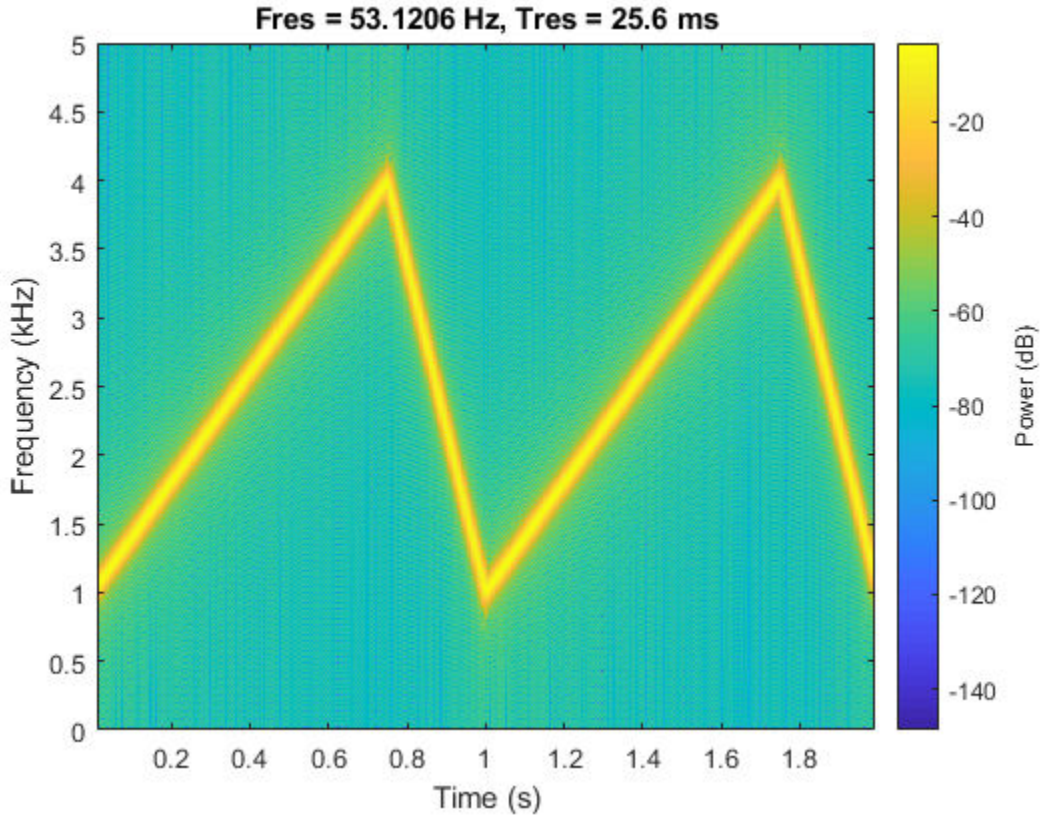
`cceps` | `icceps` | `rceps`

FFT-Based Time-Frequency Analysis

The Signal Processing Toolbox™ product provides functions that return the time-dependent Fourier transform for a sequence, or displays this information as a spectrogram. The *time-dependent Fourier transform* is the discrete-time Fourier transform for a sequence, computed using a sliding window. This form of the Fourier transform, also known as the short-time Fourier transform (STFT), has numerous applications in speech, sonar, and radar processing. The *spectrogram* of a sequence is the magnitude of the time-dependent Fourier transform versus time.

To display the spectrogram of a signal, you can use the `pspectrum` function. For example, generate two seconds of a signal sampled at 10 kHz. Specify the instantaneous frequency of the signal as a triangular function of time. To compute the spectrogram, divide the signal into segments of duration 0.0256 second and specify 86% segment-to-segment overlap. The *leakage* measures the ability of the sliding window to detect a weak tone from noise in the presence of a neighboring strong tone. Specify a leakage of 0.875.

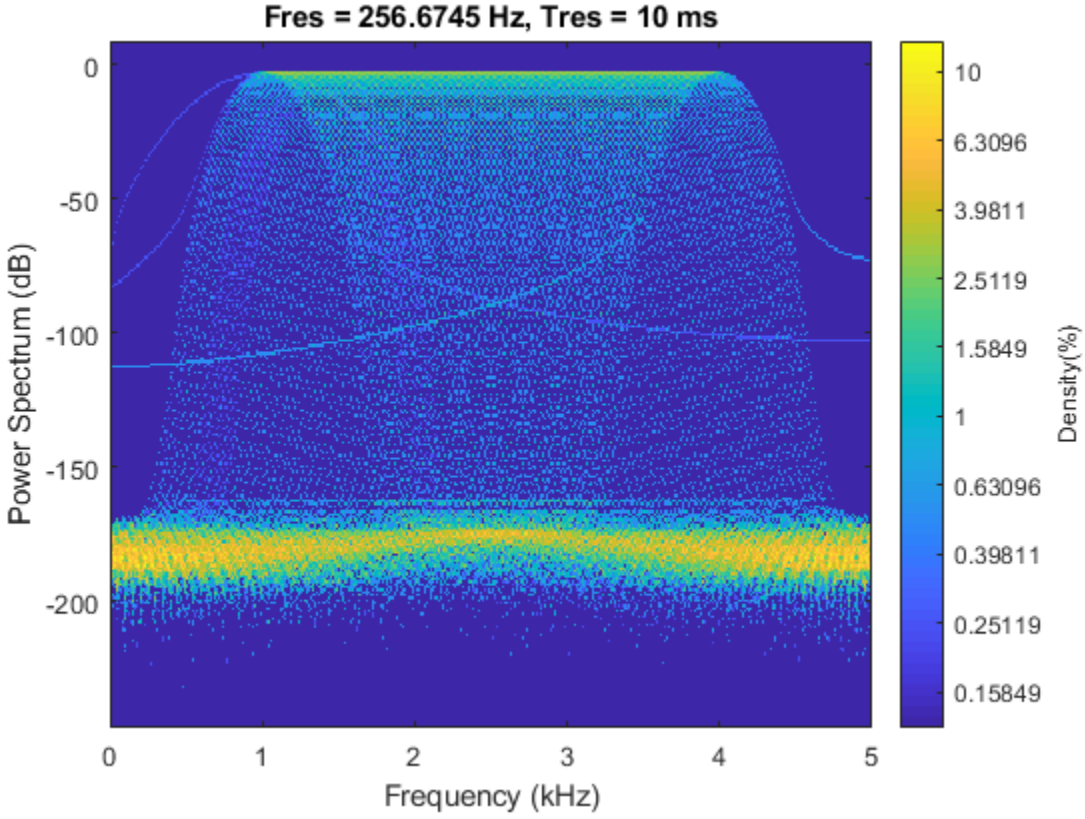
```
fs = 10000;
t = 0:1/fs:2;
x = vco(sawtooth(2*pi*t,0.75),[0.1 0.4]*fs,fs);

pspectrum(x,fs,'spectrogram', ...
    'TimeResolution',0.0256,'Overlap',86,'Leakage',0.875)
```



The *persistence spectrum* of a signal is a time-frequency view that shows the percentage of the time that a given frequency is present in a signal. The persistence spectrum is a histogram in power-frequency space. The longer a particular frequency persists in a signal as the signal evolves, the higher its time percentage and thus the brighter or "hotter" its color in the display.

Compute and display the persistence spectrum of the signal. Specify a time resolution of 0.01 second, 50% overlap between adjoining segments, and a leakage of 0.5.

```
pspectrum(x,fs,'persistence', ...
    'TimeResolution',0.01,'Overlap',50,'Leakage',0.5)
```



See Also

Apps
Signal Analyzer

Functions
`fsst` | `ifsst` | `pspectrum` | `spectrogram` | `tfridge` | `xspectrogram`

Related Examples

- “Practical Introduction to Time-Frequency Analysis”
- “Detect Closely Spaced Sinusoids” on page 14-33
- “Hilbert Transform and Instantaneous Frequency” on page 14-26

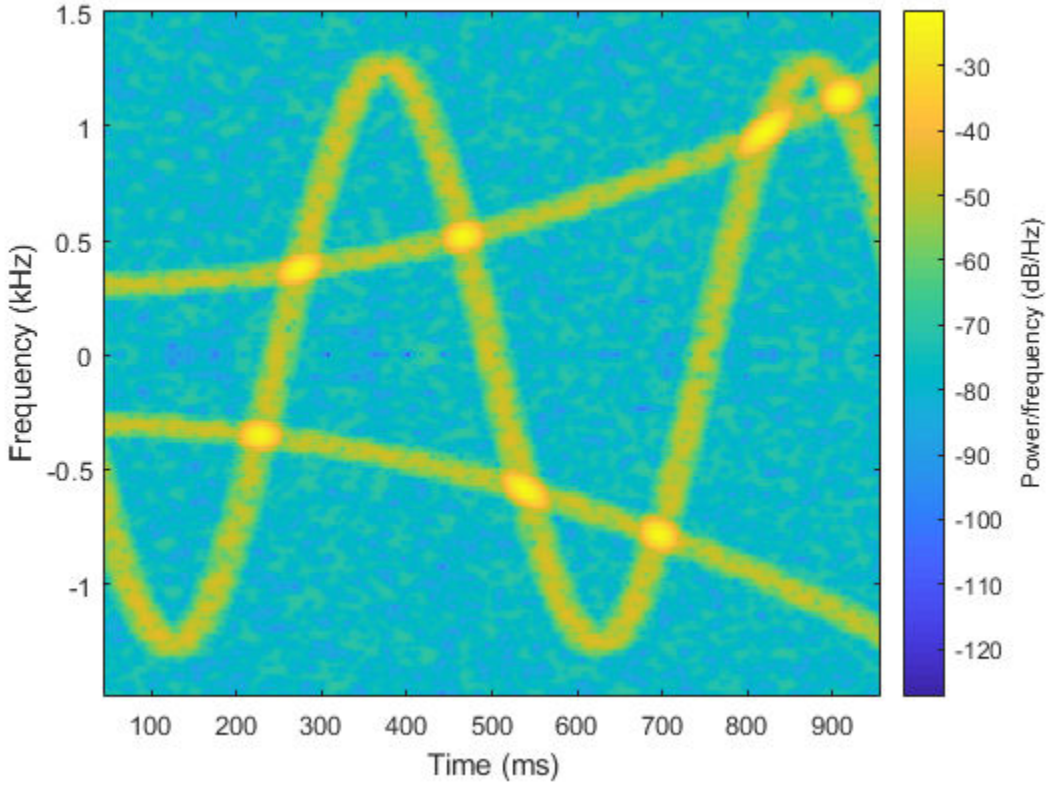
Cross-Spectrogram of Complex Signals

Generate two signals, each sampled at 3 kHz for 1 second. The first signal is a quadratic chirp whose frequency increases from 300 Hz to 1300 Hz during the measurement. The chirp is embedded in white Gaussian noise. The second signal, also embedded in white noise, is a chirp with sinusoidally varying frequency content.

```
fs = 3000;
t = 0:1/fs:1-1/fs;

x1 = chirp(t,300,t(end),1300,'quadratic')+randn(size(t))/100;

x2 = exp(2j*pi*100*cos(2*pi*2*t))+randn(size(t))/100;
```

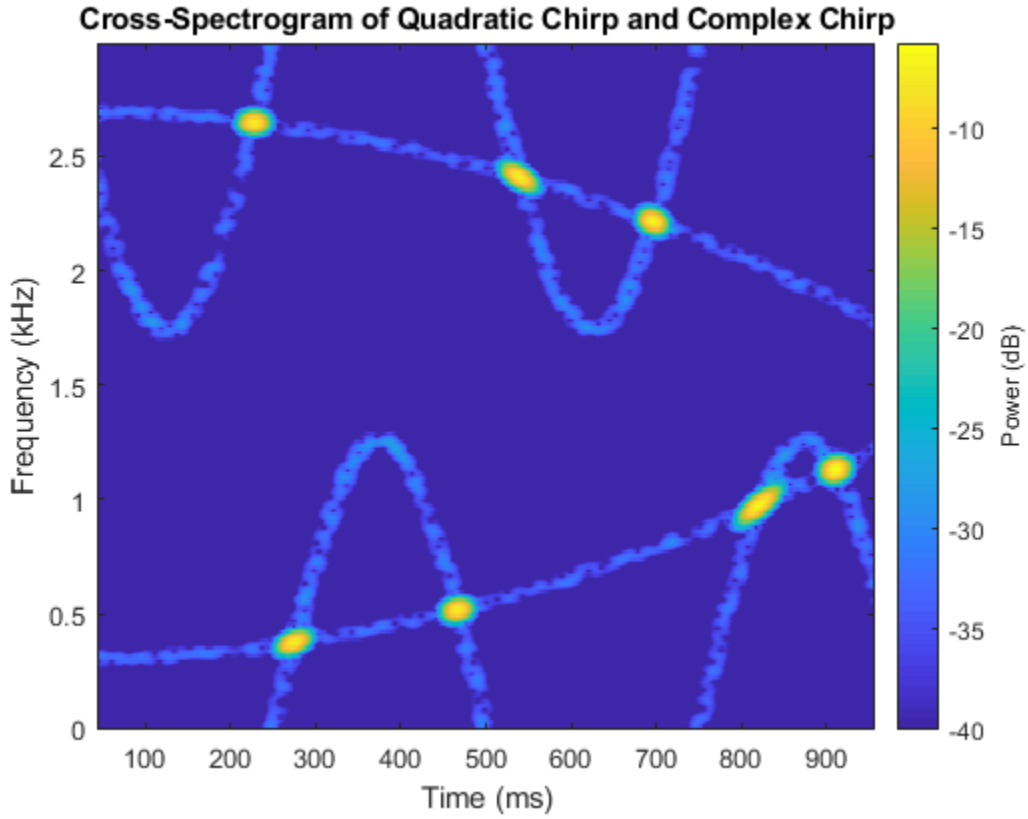
Compute and plot the cross-spectrogram of the two signals. Divide the signals into 256-sample segments with 255 samples of overlap between adjoining segments. Use a Kaiser window with shape factor $\beta = 30$ to window the segments. Use the default number of DFT points. Center the cross-spectrogram at zero frequency.

```
nwin = 256;

xspectrogram(x1,x2,kaiser(nwin,30),nwin-1,[],fs,'centered','yaxis')
```



Compute the power spectrum instead of the power spectral density. Set to zero the values smaller than –40 dB. Center the plot at the Nyquist frequency.

```
xspectrogram(x1,x2,kaiser(nwin,30),nwin-1,[],fs, ...
    'power','MinThreshold',-40,'yaxis')
title('Cross-Spectrogram of Quadratic Chirp and Complex Chirp')
```



The thresholding further highlights the regions of common frequency.

See Also

`spectrogram` | `xspectrogram`

Find Interference Using Persistence Spectrum

Visualize an interference narrowband signal embedded within a broadband signal.

Generate a chirp sampled at 1 kHz for 500 seconds. The frequency of the chirp increases from 180 Hz to 220 Hz during the measurement.

```
fs = 1000;
t = (0:1/fs:500)';
x = chirp(t,180,t(end),220) + 0.15*randn(size(t));
```

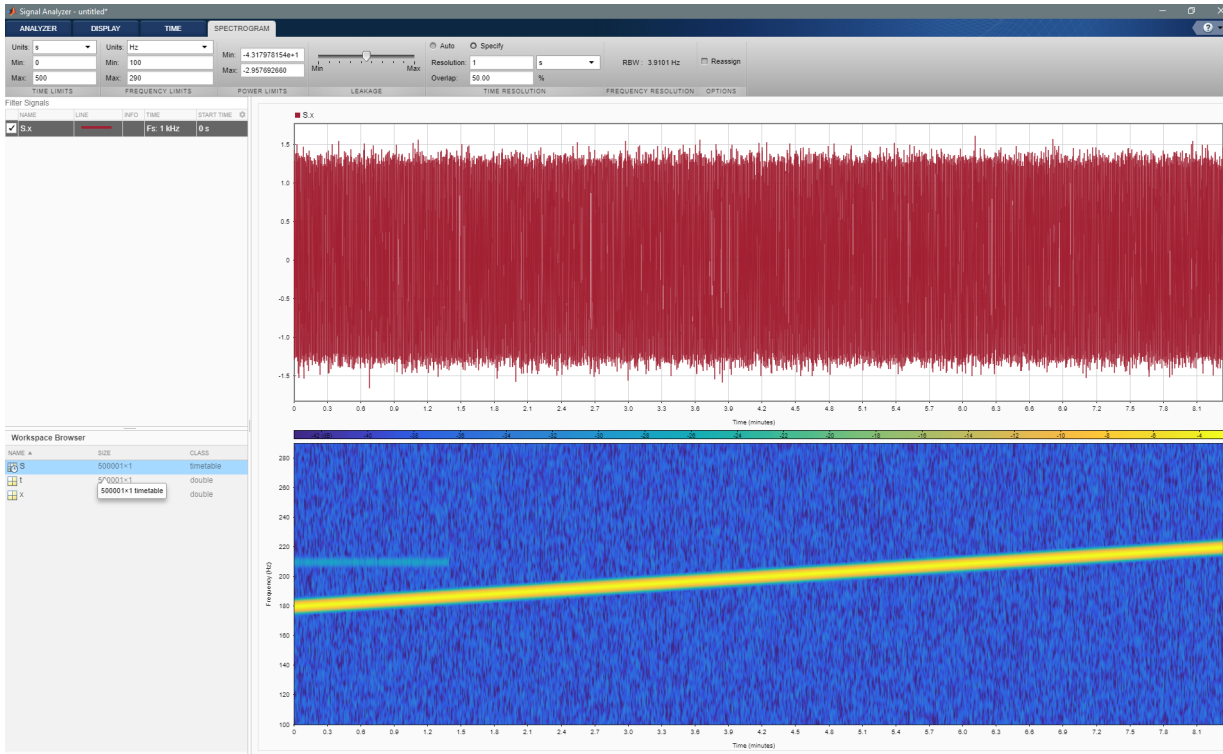
The signal also contains a 210 Hz sinusoid. The sinusoid has an amplitude of 0.05 and is present only for 1/6 of the total signal duration.

```
idx = floor(length(x)/6);
x(1:idx) = x(1:idx) + 0.05*cos(2*pi*t(1:idx)*210);
```

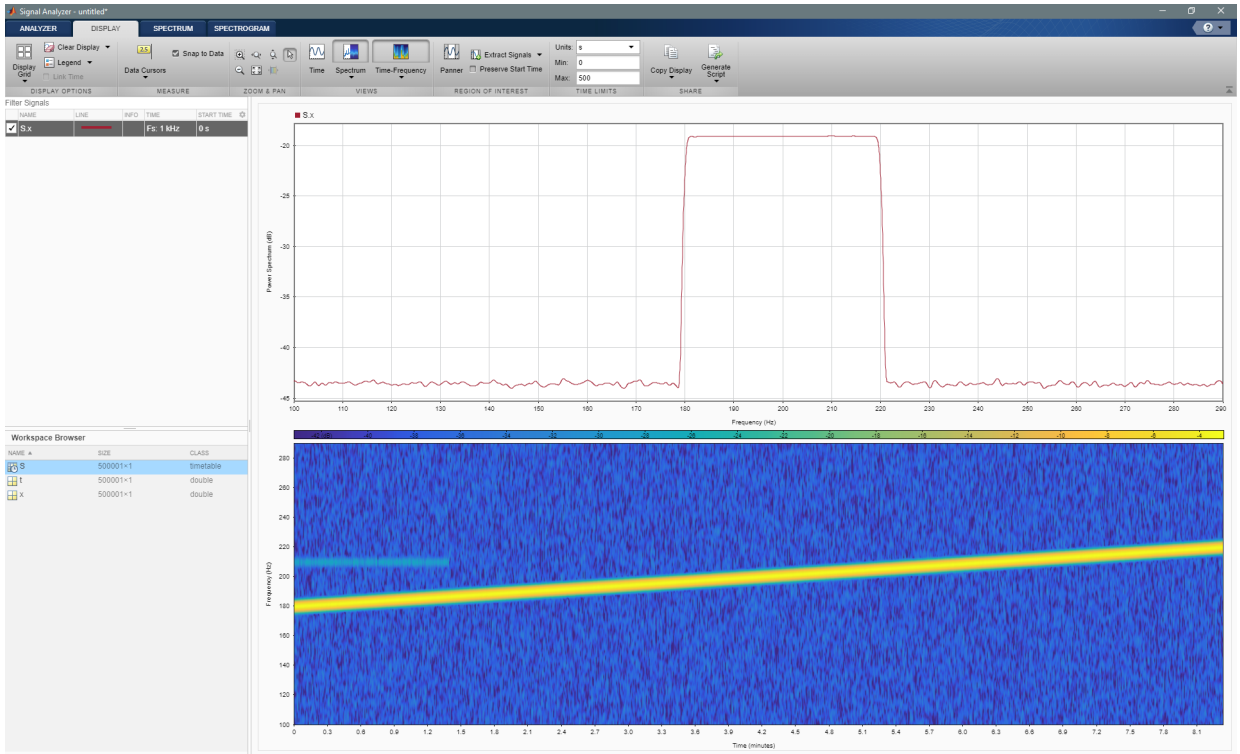
Save the signal as a MATLAB® timetable.

```
S = timetable(seconds(t),x);
```

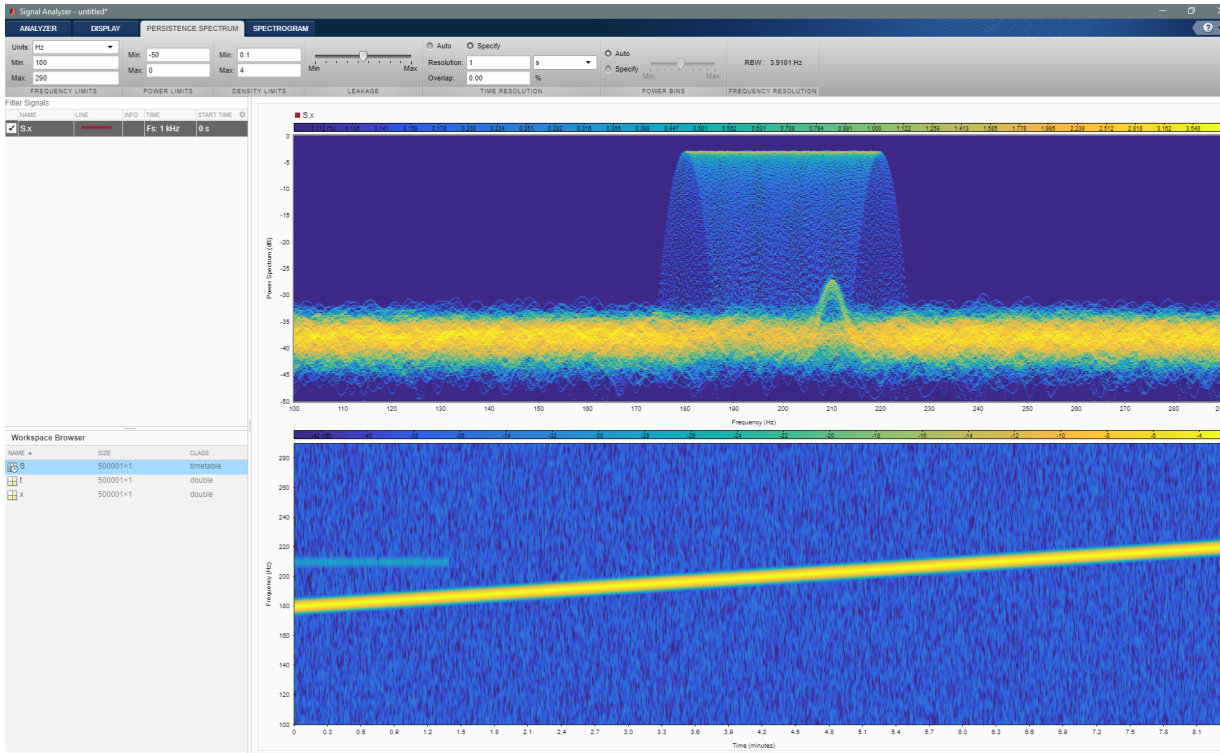
Open **Signal Analyzer** and drag the timetable from the Workspace browser to a display. Click the **Time-Frequency** button to add a spectrogram view. On the **Spectrogram** tab, under **Time Resolution**, select **Specify** and enter a time resolution of 1 second. Set the **Frequency Limits** to 100 Hz and 290 Hz. Both signal components are visible.



Go back to the **Display** tab. Click the **Time** button to remove the time view and click the **Spectrum** button to add a power spectrum view. The frequency range continues to be from 100 Hz to 290 Hz. The weak sinusoid is obscured by the chirp.



Click the **Spectrum ▼** button to change the `Spectrum` view to a `Persistence Spectrum` view. On the **Persistence Spectrum** tab, under **Time Resolution**, select **Specify** and enter a time resolution of 1 second. Specify zero overlap between adjoining segments. Set the **Power Limits** to –50 dB and 0 dB and the **Density Limits** to 0.1 and 4. Now both signal components are clearly visible.



On the **Display** tab, under **Share**, click **Generate Script ▼** and select `Persistence Spectrum Script`. The script appears in the MATLAB Editor.

```
% Compute persistence spectrum

% Generated by MATLAB(R) 9.3 and Signal Processing Toolbox 7.5.
% Generated on: 15-Jul-2017 18:43:37

% Parameters
timeLimits = seconds([0 500]); % seconds
frequencyLimits = [100 290]; % Hz
timeResolution = 1; % seconds
overlapPercent = 0;

% Index into signal time region of interest
S_x_ROI = S(:,'x');
S_x_ROI = S_x_ROI(timerange(timeLimits(1),timeLimits(2)),1);
```

```
% Compute spectral estimate
% Run the function call below without output arguments to plot the results
[P,F,PWR] = pspectrum(S_x_ROI, ...
    'persistence', ...
    'FrequencyLimits',frequencyLimits, ...
    'TimeResolution',timeResolution, ...
    'OverlapPercent',overlapPercent);
```

See Also

Apps
Signal Analyzer

Functions
pspectrum

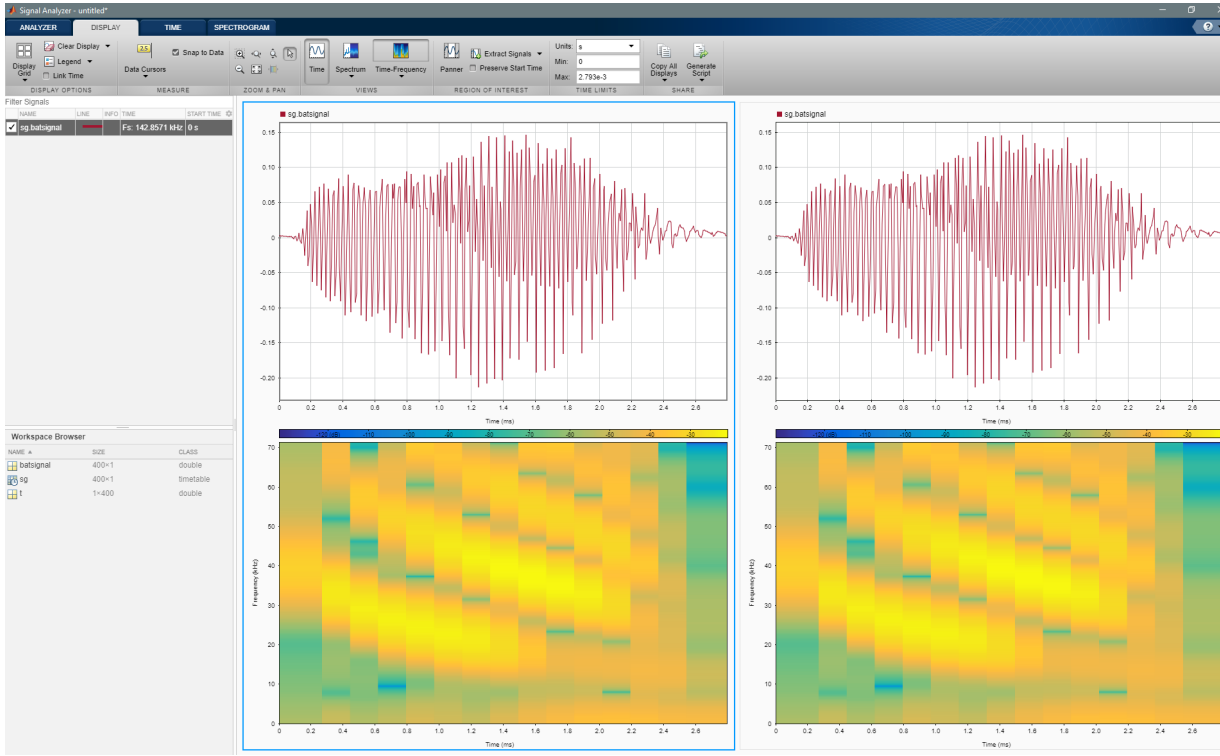
Find and Track Ridges Using Reassigned Spectrogram

Load a datafile containing an echolocation pulse emitted by a big brown bat (*Eptesicus fuscus*) and measured with a sampling interval of 7 microseconds. Create a MATLAB® timetable using the signal and the time information.

```
load batsignal

t = (0:length(batsignal)-1)*DT;
sg = timetable(seconds(t)',batsignal);
```

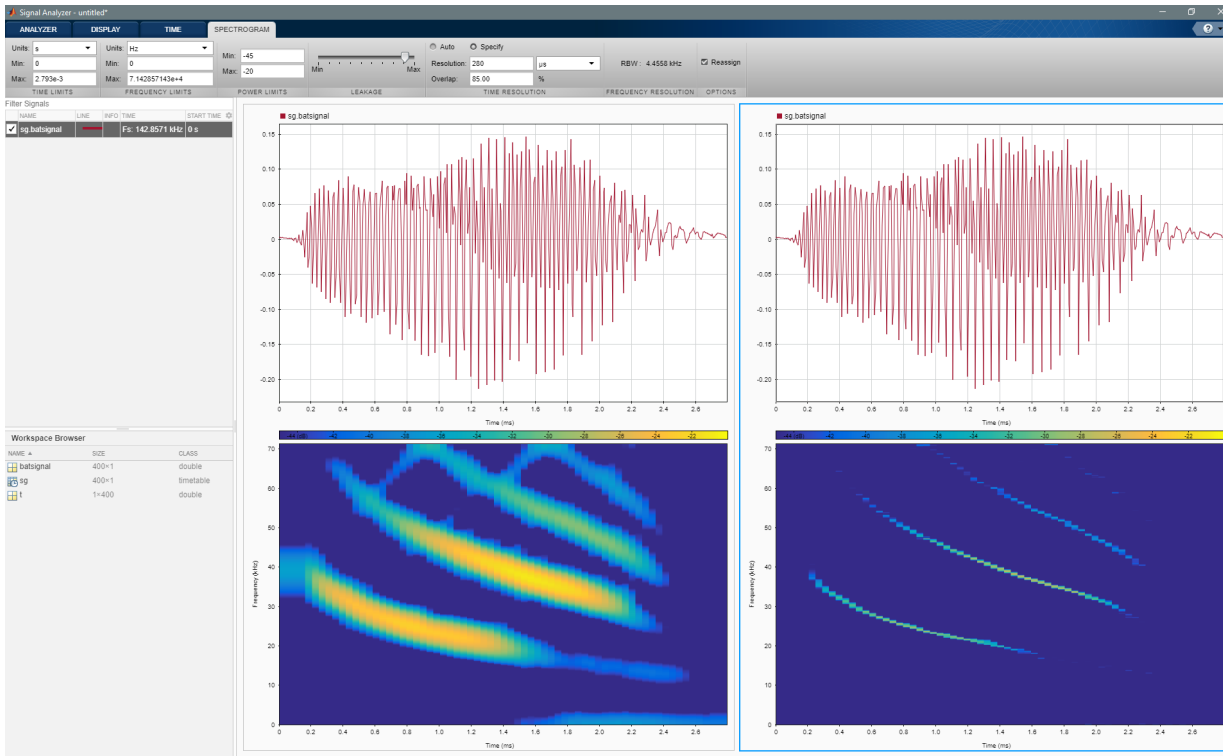
Open **Signal Analyzer** and drag the timetable from the Workspace browser to the Signal table. Click **Display Grid ▼** to create two side-by-side displays. Select each display and click the **Time-Frequency** button to add a spectrogram view.

Drag the timetable to both displays.



Select the **Spectrogram** tab. On the display at right, check **Reassign**. For each display:

- Set the time resolution to 280 microseconds and specify 85% overlap between adjoining segments.
- Use the **Leakage** slider to increase the leakage until the RBW is about 4.5 kHz.
- Set the power limits to –45 dB and –20 dB.



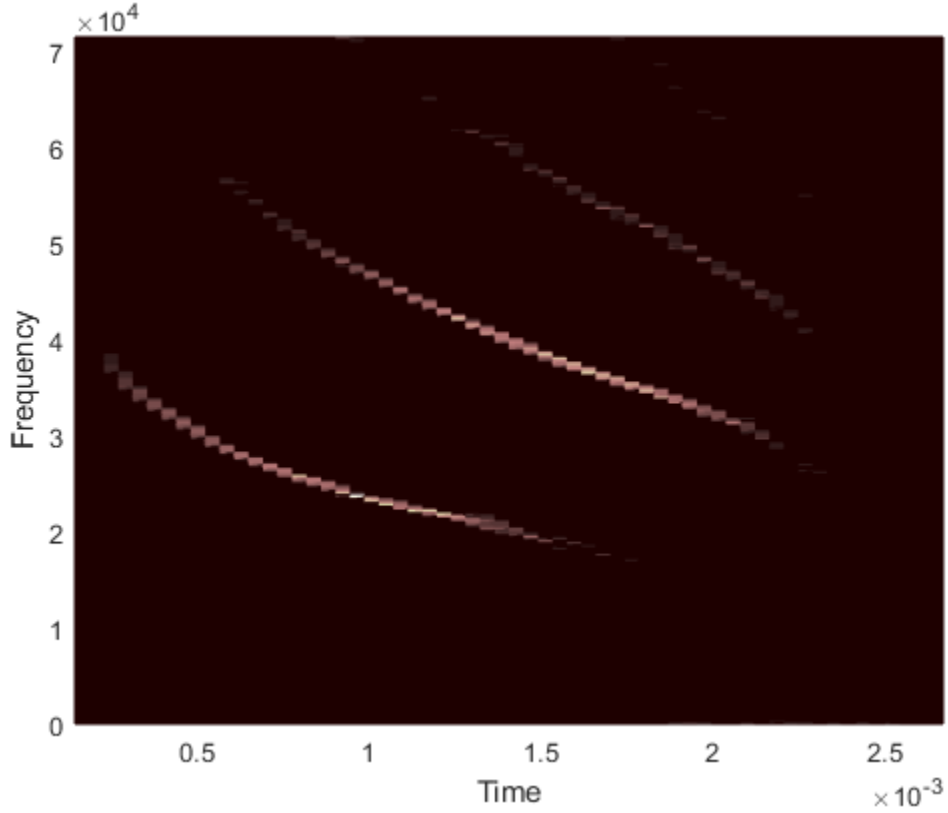
The reassigned spectrogram clearly shows three time-frequency ridges. To track the ridges, select the display at right. On the **Display** tab, click **Generate Script** and select `Spectrogram Script`. The script appears in the Editor.

```
% Compute spectrogram

% Generated by MATLAB(R) 9.3 and Signal Processing Toolbox 7.5.
% Generated on: 13-Jul-2017 19:32:31
```

```
% Parameters
timeLimits = seconds([3.805177e-06 0.002796805]); % seconds
frequencyLimits = [0 71428.57]; % Hz
leakage = 0.9;
timeResolution = 0.00028; % seconds
overlapPercent = 85;
reassignFlag = true;

% Index into signal time region of interest
sg_batsignal_ROI = sg(:,'batsignal');
sg_batsignal_ROI = sg_batsignal_ROI(timerange(timeLimits(1),timeLimits(2)),1);

% Compute spectral estimate
% Run the function call below without output arguments to plot the results
[P,F,T] = pspectrum(sg_batsignal_ROI, ...
    'spectrogram', ...
    'FrequencyLimits',frequencyLimits, ...
    'Leakage',leakage, ...
    'TimeResolution',timeResolution, ...
    'OverlapPercent',overlapPercent, ...
    'Reassign',reassignFlag);
```

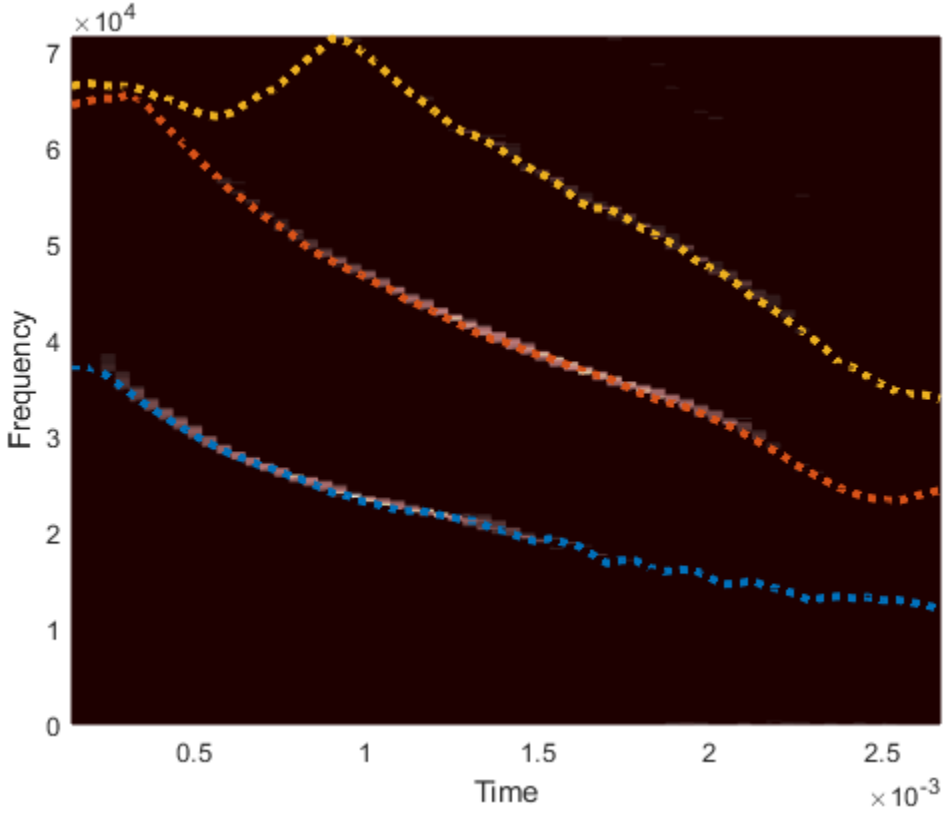
Run the script. Plot the reassigned spectrogram.

```
mesh(seconds(T),F,P)
xlabel('Time')
ylabel('Frequency')
axis tight
view(2)
colormap pink
```



Use the `tfridge` function to track the ridges.

```
[fridge,~,lridge] = tfridge(P,F,0.01,'NumRidges',3,'NumFrequencyBins',10);

hold on
plot3(seconds(T),fridge,P(lridge),':','linewidth',3)
hold off
```



Thanks to Curtis Condon, Ken White, and Al Feng of the Beckman Center at the University of Illinois for the bat data and permission to use it in this example.

See Also

Apps
Signal Analyzer

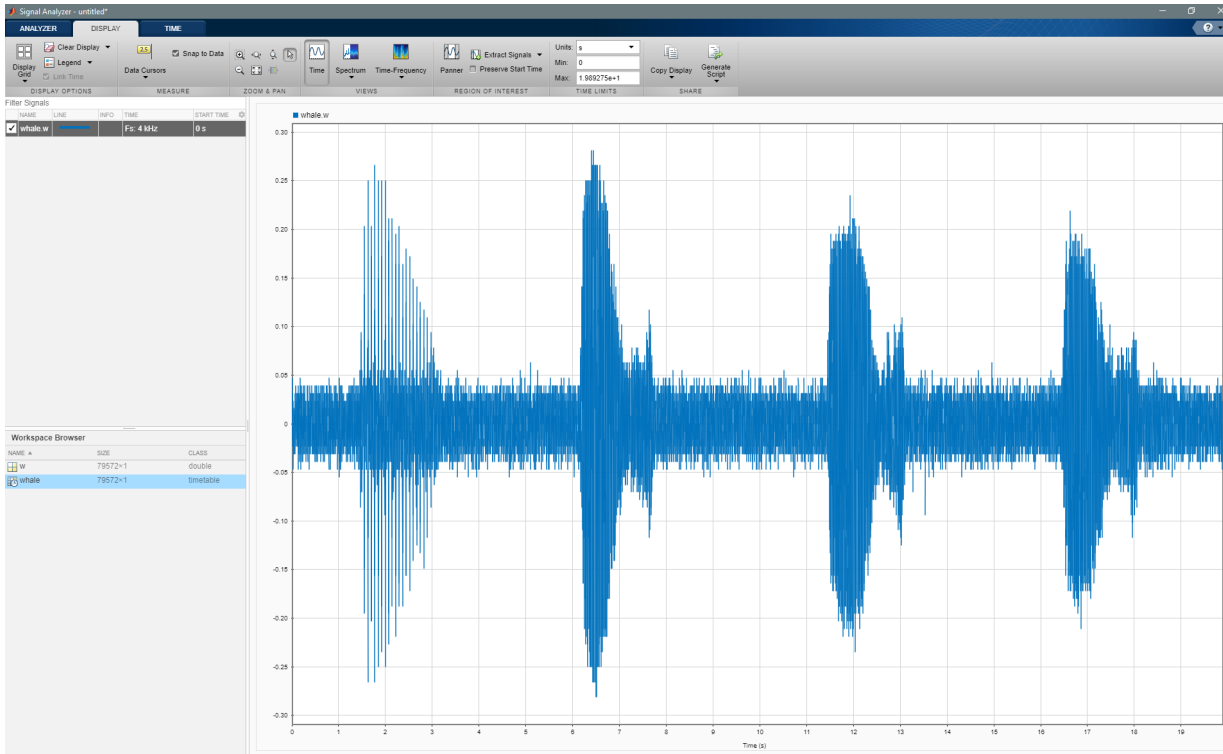
Functions
`pspectrum`

Extract Regions of Interest from Whale Song

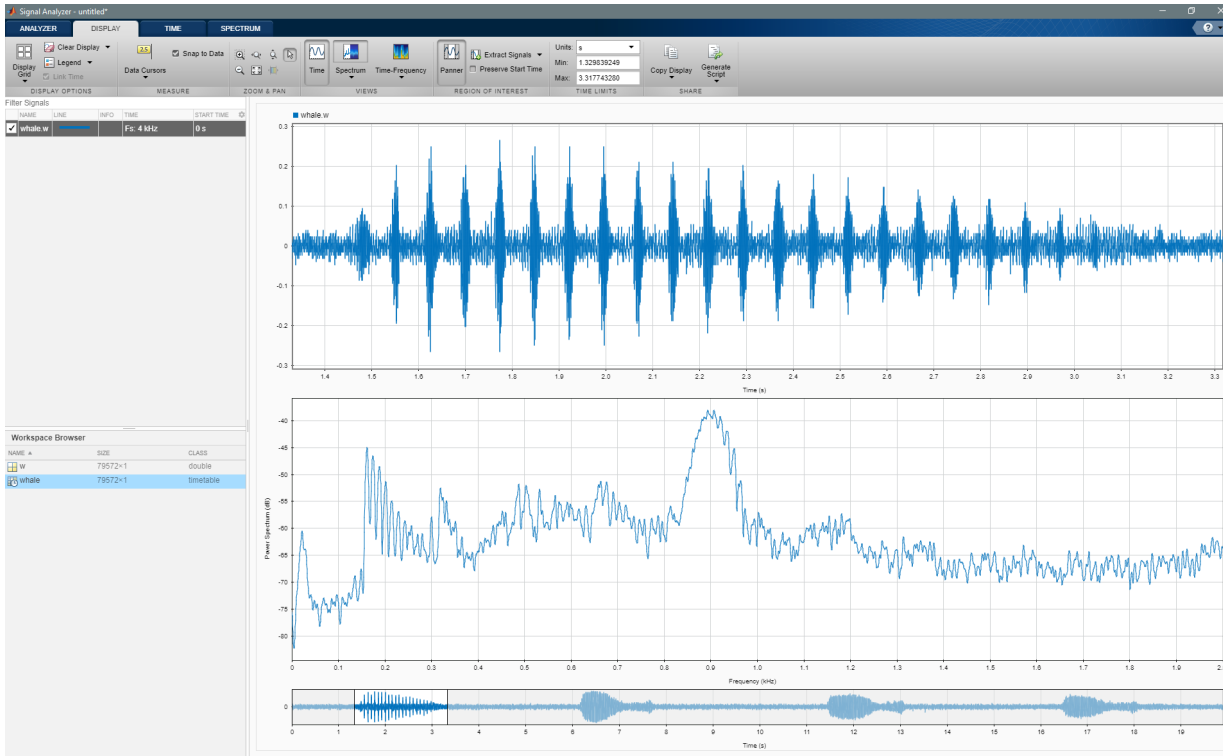
Load a file that contains audio data from a Pacific blue whale, sampled at 4 kHz. The file is from the library of animal vocalizations maintained by the Cornell University Bioacoustics Research Program. The time scale in the data is compressed by a factor of 10 to raise the pitch and make the calls more audible. Convert the signal to a MATLAB® timetable.

```
whaleFile = fullfile(matlabroot,'examples','matlab','bluewhale.au');
[w,fs] = audioread(whaleFile);

whale = timetable(seconds((0:length(w)-1)'/fs),w);

% To hear, type soundsc(w,fs)
```

Open **Signal Analyzer** and drag the timetable to a display. Four features stand out from the noise. The first is known as a *trill*, and the other three are known as *moans*.

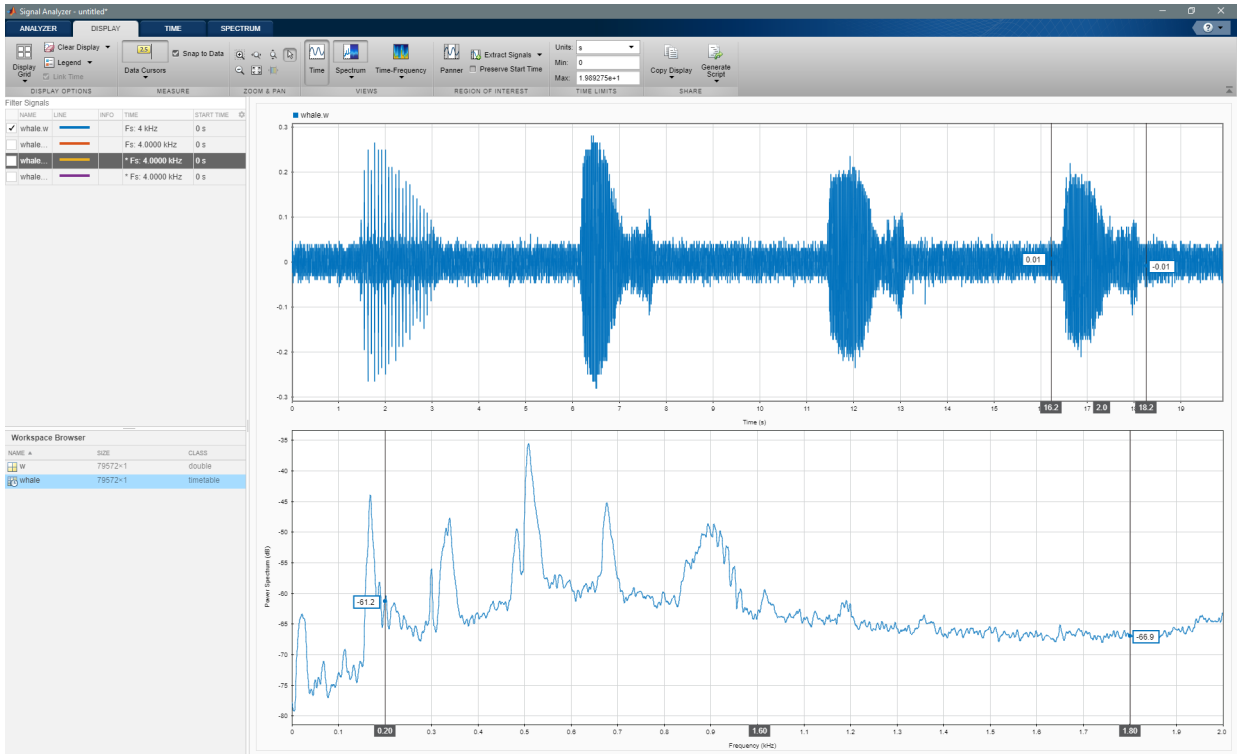


On the **Display** tab, click **Spectrum** to open a spectrum view and click **Panner** to activate the panner. Use the panner to create a zoom window with a width of about 2 seconds. Drag the zoom window so that it is centered on the trill. The spectrum shows a noticeable peak at around 900 Hz.

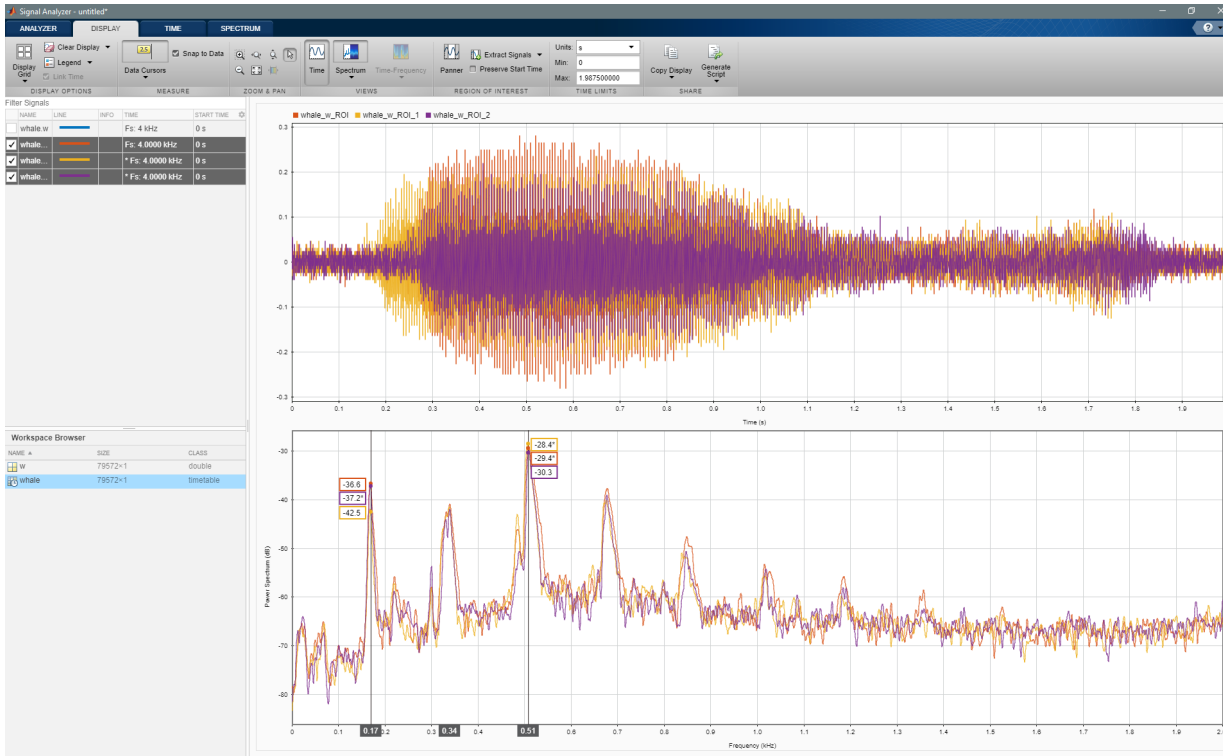


Extract the three moans to compare their spectra:

1. Center the panner zoom window on the first moan. The spectrum has eight clearly defined peaks, located very close to multiples of 170 Hz. Click **Extract Signals ▼** and select `Between Time Limits`.
2. Click **Panner** to hide the panner. Press the space bar to see the full signal. Click **Zoom in X** and zoom in on a 2-second interval of the time view centered on the second moan. The spectrum again has peaks at multiples of 170 Hz. Click **Extract Signals ▼** and select `Between Time Limits`.
3. Press the space bar to see the full signal. Click **Data Cursors ▼** and select `Two`. Place the time-domain cursors in a 2-second interval around the third moan. Again, there are peaks at multiples of 170 Hz. Click **Extract Signals ▼** and select `Between Time Cursors`.



Remove the original signal from the display by clearing the check box next to its name in the Signal table. Display the three regions of interest you just extracted. Their spectra lie approximately on top of each other. Move the frequency-domain cursors to the locations of the first and third spectral peaks. Asterisks in cursor labels indicate interpolated signal values.



See Also

Apps
Signal Analyzer

Functions
`pspectrum`

Median Filtering

The function `medfilt1` implements one-dimensional median filtering, a nonlinear technique that applies a sliding window to a sequence. The median filter replaces the center value in the window with the median value of all the points within the window [5]. In computing this median, `medfilt1` assumes zeros beyond the input points.

When the number of elements `n` in the window is even, `medfilt1` sorts the numbers, then takes the average of the `n/2` and `n/2 + 1` elements.

Two simple examples with fourth- and third-order median filters are

```
medfilt1([4 3 5 2 8 9 1],4)
ans =
    1.500 3.500 3.500 4.000 6.500 5.000 4.500
medfilt1([4 3 5 2 8 9 1],3)
ans =
     3     4     3     5     8     8     1
```

See the `medfilt2` function in the Image Processing Toolbox™ for information on two-dimensional median filtering.

Communications Applications

In this section...
“Modulation” on page 7-63 “Demodulation” on page 7-64 “Voltage Controlled Oscillator” on page 7-67

Modulation

Modulation varies the amplitude, phase, or frequency of a *carrier signal* with reference to a *message signal*. The `modulate` function modulates a message signal with a specified modulation method.

The basic syntax for the `modulate` function is

```
y = modulate(x,fc,fs,'method',opt)
```

where:

- `x` is the message signal.
- `fc` is the carrier frequency.
- `fs` is the sampling frequency.
- *`method`* is a flag for the desired modulation method.
- `opt` is any additional argument that the method requires. (Not all modulation methods require an option argument.)

The table below summarizes the modulation methods provided; see the documentation for `modulate`, `demod`, and `vco` for complete details on each.

Method	Description
`amdsb-sc` or `am`	Amplitude modulation, double sideband, suppressed carrier
`amdsb-tc`	Amplitude modulation, double sideband, transmitted carrier
`amssb`	Amplitude modulation, single sideband
`fm`	Frequency modulation
`pm`	Phase modulation

Method	Description
ppm	Pulse position modulation
pwm	Pulse width modulation
qam	Quadrature amplitude modulation

If the input x is an array rather than a vector, modulate modulates each column of the array.

To obtain the time vector that modulate uses to compute the modulated signal, specify a second output parameter:

```
[y,t] = modulate(x,fc,fs,'method',opt)
```

Demodulation

The demod function performs *demodulation*, that is, it obtains the original message signal from the modulated signal:

The syntax for demod is

```
x = demod(y,fc,fs,'method',opt)
```

demod uses any of the methods shown for modulate, but the syntax for quadrature amplitude demodulation requires two output parameters:

```
[X1,X2] = demod(y,fc,fs,'qam')
```

If the input y is an array, demod demodulates all columns.

Try modulating and demodulating a signal. A 50 Hz sine wave sampled at 1000 Hz is

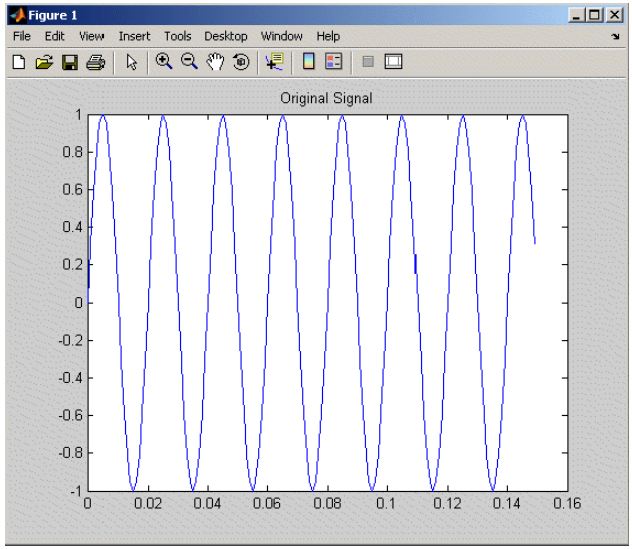
```
t = (0:1/1000:2);
x = sin(2*pi*50*t);
```

With a carrier frequency of 200 Hz, the modulated and demodulated versions of this signal are

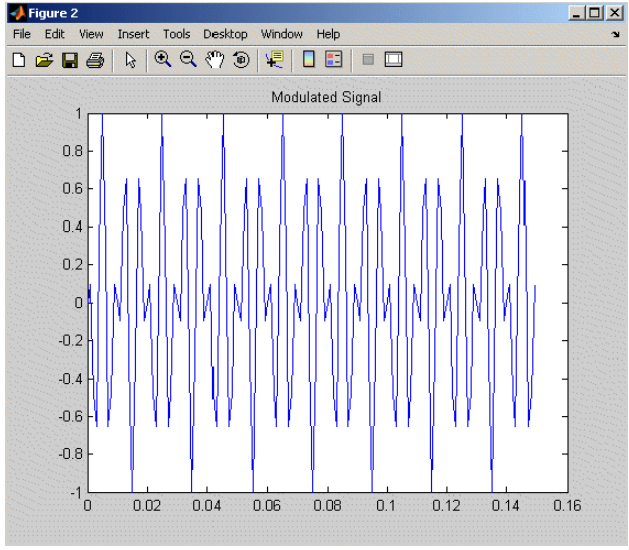
```
y = modulate(x,200,1000,'am');
z = demod(y,200,1000,'am');
```

To plot portions of the original, modulated, and demodulated signal:

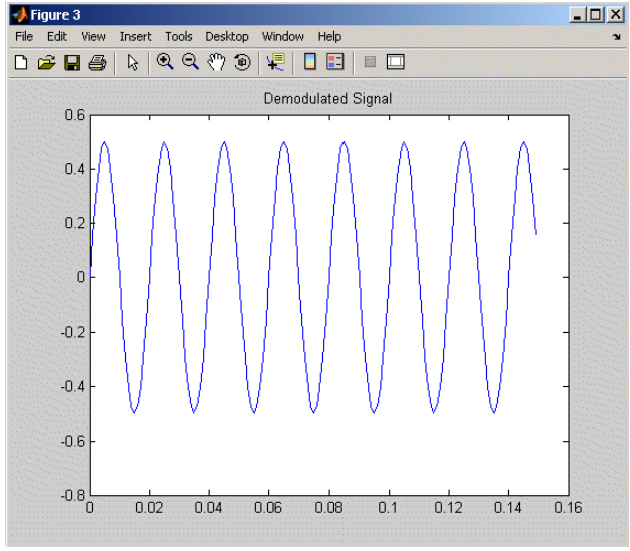
```
figure; plot(t(1:150),x(1:150)); title('Original Signal');
figure; plot(t(1:150),y(1:150)); title('Modulated Signal');
figure; plot(t(1:150),z(1:150)); title('Demodulated Signal');
```



Original Signal



Modulated Signal



Demodulated Signal

Note The demodulated signal is attenuated because demodulation includes two steps: multiplication and lowpass filtering. The multiplication produces a component with frequency centered at 0 Hz and a component with frequency at twice the carrier frequency. The filtering removes the higher frequency component of the signal, producing the attenuated result.

Voltage Controlled Oscillator

The voltage controlled oscillator function `vco` creates a signal that oscillates at a frequency determined by the input vector. The basic syntax for `vco` is

```
y = vco(x,fc,fs)
```

where `fc` is the carrier frequency and `fs` is the sampling frequency.

To scale the frequency modulation range, use

```
y = vco(x,[Fmin Fmax],fs)
```

In this case, `vco` scales the frequency modulation range so values of `x` on the interval `[-1 1]` map to oscillations of frequency on `[Fmin Fmax]`.

If the input `x` is an array, `vco` produces an array whose columns oscillate according to the columns of `x`.

See “FFT-Based Time-Frequency Analysis” on page 7-40 for an example using the `vco` function.

Deconvolution

Deconvolution, or polynomial division, is the inverse operation of convolution. Deconvolution is useful in recovering the input to a known filter, given the filtered output. This method is very sensitive to noise in the coefficients, however, so use caution in applying it.

The syntax for `deconv` is

```
[q,r] = deconv(b,a)
```

where `b` is the polynomial dividend, `a` is the divisor, `q` is the quotient, and `r` is the remainder.

To try `deconv`, first convolve two simple vectors `a` and `b` .

```
a = [1 2 3];
b = [4 5 6];
c = conv(a,b)
```

```
c =
    4   13   28   27   18
```

Now use `deconv` to deconvolve `b` from `c`:

```
[q,r] = deconv(c,a)
```

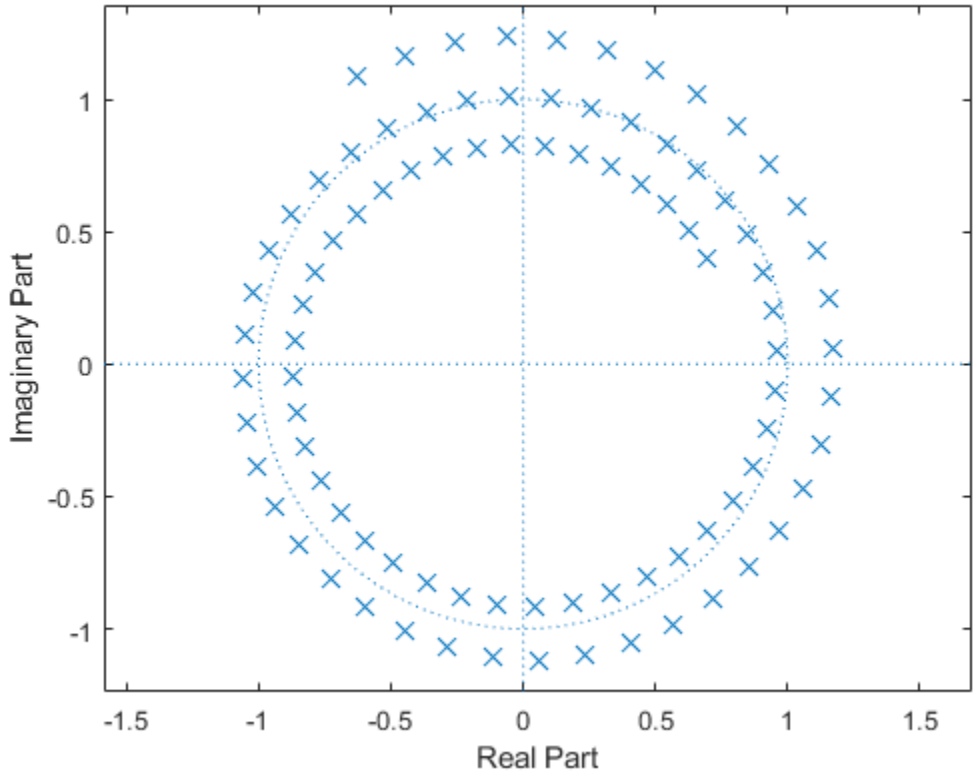
```
q =
    4    5    6
r =
    0    0    0    0    0
```

Chirp Z-Transform

The chirp Z-transform (CZT) is useful in evaluating the Z-transform along contours other than the unit circle. The chirp Z-transform is also more efficient than the DFT algorithm for the computation of prime-length transforms, and it is useful in computing a subset of the DFT for a sequence. The chirp Z-transform, or CZT, computes the Z-transform along spiral contours in the z-plane for an input sequence. Unlike the DFT, the CZT is not constrained to operate along the unit circle, but can evaluate the Z-transform along contours described by $z_\ell = AW^{-\ell}, \ell = 0, \cdots, M-1$, where A is the complex starting point, W is a complex scalar describing the complex ratio between points on the contour, and M is the length of the transform.

One possible spiral is

```
A = 0.8*exp(1j*pi/6);
W = 0.995*exp(-1j*pi*.05);
M = 91;
z = A*(W.^(-(0:M-1)));
zplane([],z.')
```

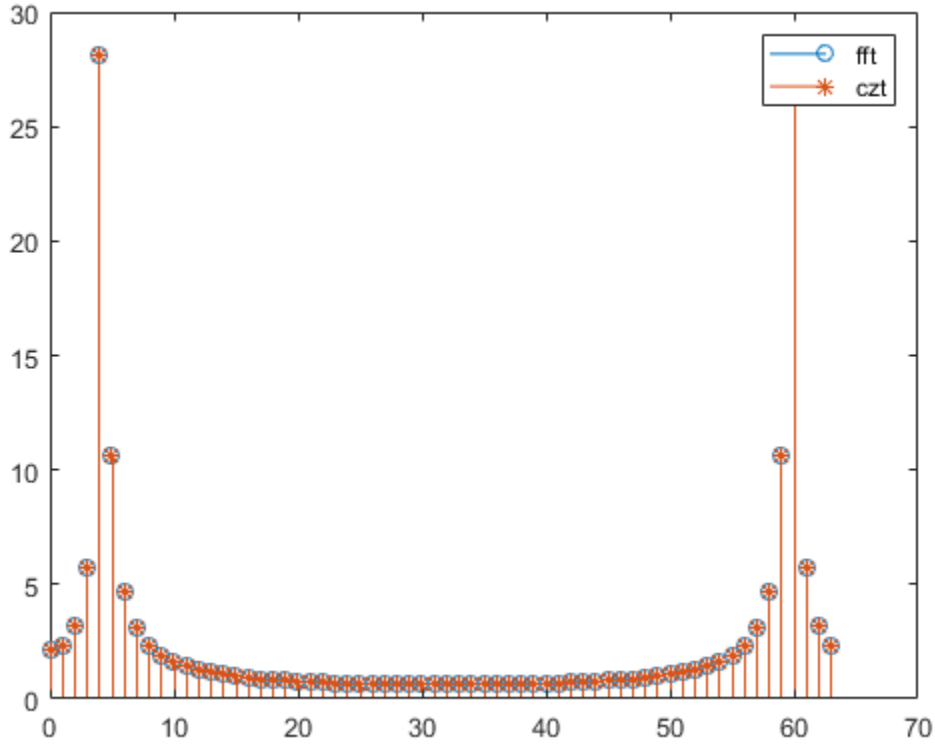


`czt(x,M,W,A)` computes the Z-transform of `x` on these points.

An interesting and useful spiral set is *m* evenly spaced samples around the unit circle, parameterized by $A = 1$ and $W = \exp(-j\pi/M)$. The Z-transform on this contour is simply the DFT, obtained by `czt`:

```
M = 64;
m = 0:M-1;

x = sin(2*pi*m/15);
FFT = fft(x);
CZT = czt(x,M,exp(-2j*pi/M),1);
```

```
stem(m,abs(FFT))
hold on
stem(m,abs(CZT),'*')
hold off
legend('fft','czt')
```



`czt` may be faster than the `fft` function for computing the DFT of sequences with certain odd lengths, particularly long prime-length sequences.

See Also

`czt` | `fft`

Discrete Cosine Transform

The discrete cosine transform (DCT) is closely related to the discrete Fourier transform (DFT). The DFT is actually one step in the computation of the DCT for a sequence. The DCT, however, has better *energy compaction* than the DFT, with just a few of the transform coefficients representing the majority of the energy in the sequence. This property of the DCT makes it useful in applications such as data communications and signal coding.

DCT Variants

The DCT has four standard variants. For a signal x of length N, and with $\delta_{k\ell}$ the Kronecker delta, the transforms are defined by:

- DCT-1:

$$y(k) = \sqrt{\frac{2}{N}} \sum_{n=1}^{N} x(n) \frac{1}{\sqrt{1+\delta_{n1}+\delta_{nN}}} \frac{1}{\sqrt{1+\delta_{k1}+\delta_{kN}}} \cos\left(\frac{\pi}{N-1}(n-1)(k-1)\right)$$

- DCT-2:

$$y(k) = \sqrt{\frac{2}{N}} \sum_{n=1}^{N} x(n) \frac{1}{\sqrt{1+\delta_{k1}}} \cos\left(\frac{\pi}{2N}(2n-1)(k-1)\right)$$

- DCT-3:

$$y(k) = \sqrt{\frac{2}{N}} \sum_{n=1}^{N} x(n) \frac{1}{\sqrt{1+\delta_{n1}}} \cos\left(\frac{\pi}{2N}(n-1)(2k-1)\right)$$

- DCT-4:

$$y(k) = \sqrt{\frac{2}{N}} \sum_{n=1}^{N} x(n) \cos\left(\frac{\pi}{4N}(2n-1)(2k-1)\right)$$

The Signal Processing Toolbox function `dct` computes the unitary DCT of an input array.

Inverse DCT Variants

All variants of the DCT are *unitary* (or, equivalently, *orthogonal*): To find their inverses, switch *k* and *n* in each definition. In particular, DCT-1 and DCT-4 are their own inverses, and DCT-2 and DCT-3 are inverses of each other:

- Inverse of DCT-1:

$$x(n) = \sqrt{\frac{2}{N}} \sum_{k=1}^{N} y(k) \frac{1}{\sqrt{1+\delta_{k1}+\delta_{kN}}} \frac{1}{\sqrt{1+\delta_{n1}+\delta_{nN}}} \cos\left(\frac{\pi}{N-1}(k-1)(n-1)\right)$$

- Inverse of DCT-2:

$$x(n) = \sqrt{\frac{2}{N}} \sum_{k=1}^{N} y(k) \frac{1}{\sqrt{1+\delta_{k1}}} \cos\left(\frac{\pi}{2N}(k-1)(2n-1)\right)$$

- Inverse of DCT-3:

$$x(n) = \sqrt{\frac{2}{N}} \sum_{k=1}^{N} y(k) \frac{1}{\sqrt{1+\delta_{n1}}} \cos\left(\frac{\pi}{2N}(2k-1)(n-1)\right)$$

- Inverse of DCT-4:

$$x(n) = \sqrt{\frac{2}{N}} \sum_{k=1}^{N} y(k) \cos\left(\frac{\pi}{4N}(2k-1)(2n-1)\right)$$

The function `idct` computes the inverse DCT for an input sequence, reconstructing a signal from a complete or partial set of DCT coefficients.

Signal Reconstruction Using DCT

Because of the energy compaction property of the DCT, you can reconstruct a signal from only a fraction of its DCT coefficients. For example, generate a 25 Hz sinusoidal sequence sampled at 1000 Hz.

```
t = 0:1/1000:1;
x = sin(2*pi*25*t);
```

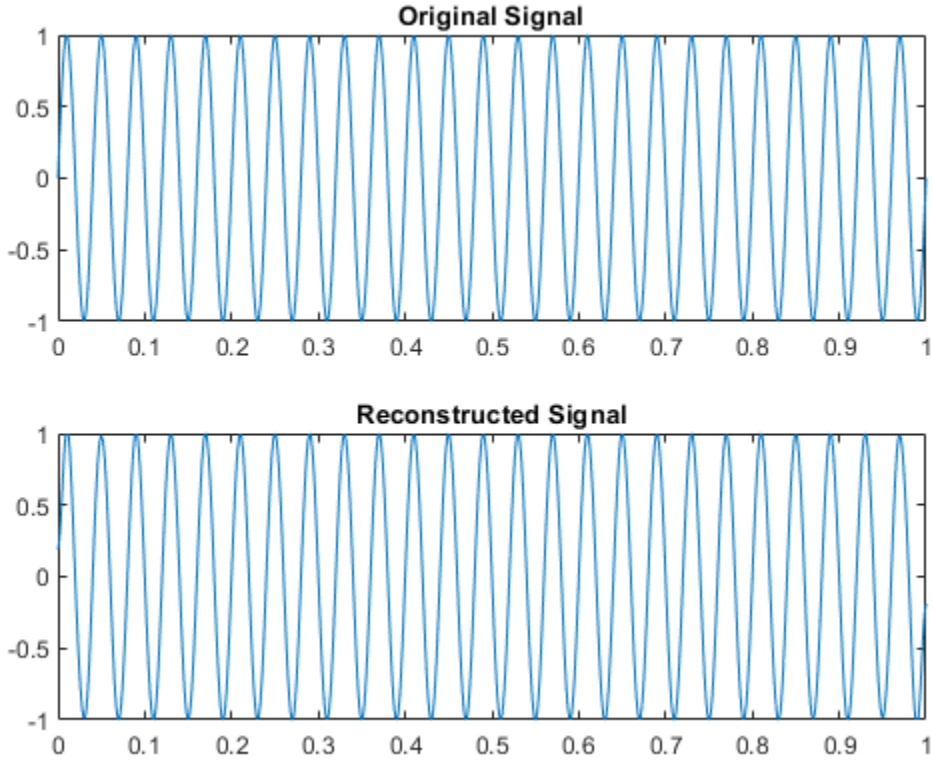
Compute the DCT of this sequence and reconstruct the signal using only those components with value greater than 0.1. Determine how many coefficients out of the original 1000 satisfy the requirement.

```
y = dct(x);
y2 = find(abs(y) < 0.1);
y(y2) = zeros(size(y2));
z = idct(y);

howmany = length(find(y))
```

```
howmany = 64
```

Plot the original and reconstructed sequences.

```
subplot(2,1,1)
plot(t,x)
ax = axis;
title('Original Signal')

subplot(2,1,2)
plot(t,z)
axis(ax)
title('Reconstructed Signal')
```



One measure of the accuracy of the reconstruction is the norm of the difference between the original and reconstructed signals, divided by the norm of the original signal. Compute this estimate and express it as a percentage.

```
norm(x-z)/norm(x)*100
```

```
ans = 1.9437
```

The reconstructed signal retains approximately 98% of the energy in the original signal.

See Also

`dct` | `idct`

Related Examples

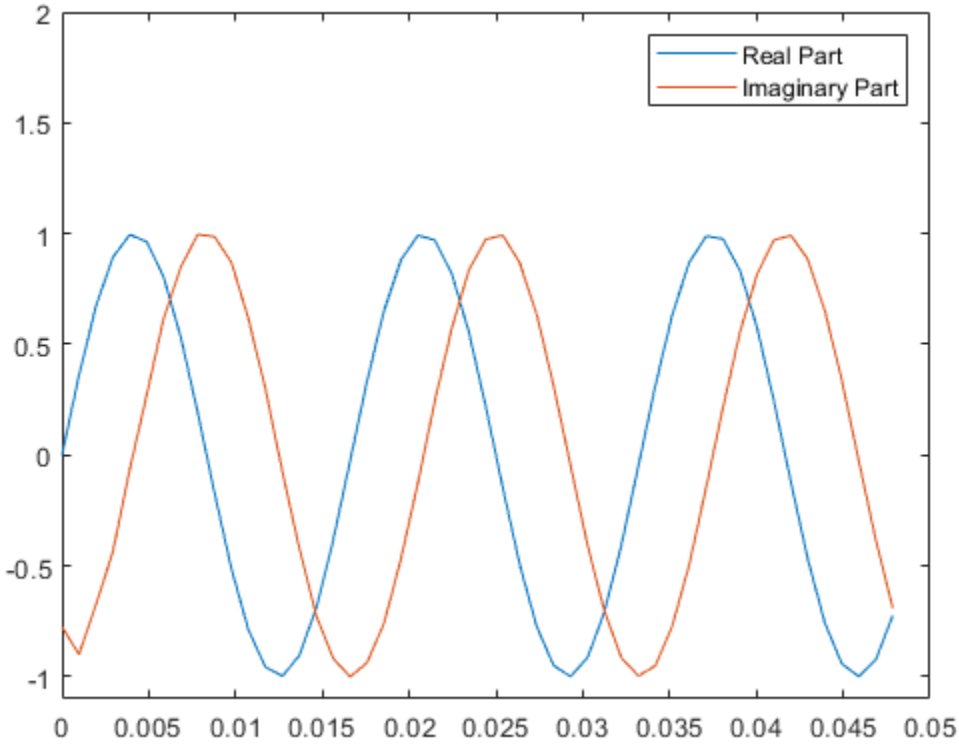
- “DCT for Speech Signal Compression” on page 14-55

Hilbert Transform

The Hilbert transform facilitates the formation of the analytic signal. The analytic signal is useful in the area of communications, particularly in bandpass signal processing. The toolbox function `hilbert` computes the Hilbert transform for a real input sequence `x` and returns a complex result of the same length, `y = hilbert(x)`, where the real part of `y` is the original real data and the imaginary part is the actual Hilbert transform. `y` is sometimes called the *analytic signal*, in reference to the continuous-time analytic signal. A key property of the discrete-time analytic signal is that its Z-transform is 0 on the lower half of the unit circle. Many applications of the analytic signal are related to this property; for example, the analytic signal is useful in avoiding aliasing effects for bandpass sampling operations. The magnitude of the analytic signal is the complex envelope of the original signal.

The Hilbert transform is related to the actual data by a 90-degree phase shift; sines become cosines and vice versa. To plot a portion of data and its Hilbert transform, use

```
t = 0:1/1024:1;
x = sin(2*pi*60*t);
y = hilbert(x);

plot(t(1:50),real(y(1:50)))
hold on
plot(t(1:50),imag(y(1:50)))
hold off
axis([0 0.05 -1.1 2])
legend('Real Part','Imaginary Part')
```



The analytic signal is useful in calculating instantaneous attributes of a time series, the attributes of the series at any point in time. The procedure requires that the signal be monocomponent.

See Also

`hilbert`

Related Examples

- “Analytic Signal for Cosine” on page 14-7

- “Envelope Extraction” on page 14-11
- “Analytic Signal and Hilbert Transform” on page 14-20
- “Hilbert Transform and Instantaneous Frequency” on page 14-26

Walsh-Hadamard Transform

The Walsh-Hadamard transform is a non-sinusoidal, orthogonal transformation technique that decomposes a signal into a set of basis functions. These basis functions are Walsh functions, which are rectangular or square waves with values of +1 or –1. Walsh-Hadamard transforms are also known as Hadamard (see the `hadamard` function in the MATLAB software), Walsh, or Walsh-Fourier transforms.

The first eight Walsh functions have these values:

Index	Walsh Function Values
0	1 1 1 1 1 1 1 1
1	1 1 1 1 -1 -1 -1 -1
2	1 1 -1 -1 -1 -1 1 1
3	1 1 -1 -1 1 1 -1 -1
4	1 -1 -1 1 1 -1 -1 1
5	1 -1 -1 1 -1 1 1 -1
6	1 -1 1 -1 -1 1 -1 1
7	1 -1 1 -1 1 -1 1 -1

The Walsh-Hadamard transform returns sequency values. Sequency is a more generalized notion of frequency and is defined as one half of the average number of zero-crossings per unit time interval. Each Walsh function has a unique sequency value. You can use the returned sequency values to estimate the signal frequencies in the original signal.

Three different ordering schemes are used to store Walsh functions: sequency, Hadamard, and dyadic. Sequency ordering, which is used in signal processing applications, has the Walsh functions in the order shown in the table above. Hadamard ordering, which is used in controls applications, arranges them as 0, 4, 6, 2, 3, 7, 5, 1. Dyadic or gray code ordering, which is used in mathematics, arranges them as 0, 1, 3, 2, 6, 7, 5, 4.

The Walsh-Hadamard transform is used in a number of applications, such as image processing, speech processing, filtering, and power spectrum analysis. It is very useful for reducing bandwidth storage requirements and spread-spectrum analysis. Like the FFT, the Walsh-Hadamard transform has a fast version, the fast Walsh-Hadamard transform (`fwht`). Compared to the FFT, the FWHT requires less storage space and is faster to calculate because it uses only real additions and subtractions, while the FFT requires

complex values. The FWHT is able to represent signals with sharp discontinuities more accurately using fewer coefficients than the FFT. Both the FWHT and the inverse FWHT (`ifwht`) are symmetric and thus, use identical calculation processes. The FWHT and IFWHT for a signal $x(t)$ of length N are defined as:

$$y_n = \frac{1}{N}\sum_{i=0}^{N-1} x_i \,\mathrm{WAL}(n,i),$$
$$x_i = \sum_{i=0}^{N-1} y_n \,\mathrm{WAL}(n,i),$$

where $i = 0,1, \ldots, N - 1$ and $\mathrm{WAL}(n,i)$ are Walsh functions. Similar to the Cooley-Tukey algorithm for the FFT, the N elements are decomposed into two sets of $N/2$ elements, which are then combined using a butterfly structure to form the FWHT. For images, where the input is typically a 2-D signal, the FWHT coefficients are calculated by first evaluating across the rows and then evaluating down the columns.

For the following simple signal, the resulting FWHT shows that `x` was created using Walsh functions with sequency values of 0, 1, 3, and 6, which are the nonzero indices of the transformed `x`. The inverse FWHT recreates the original signal.

```
x = [4 2 2 0 0 2 -2 0]
y = fwht(x)

x =

     4     2     2     0     0     2    -2     0

y =

     1     1     0     1     0     0     1     0

x1 = ifwht(y)


x1 =

     4     2     2     0     0     2    -2     0
```

See Also

`fwht` | `ifwht`

Related Examples

- “Walsh-Hadamard Transform for Spectral Analysis and Compression of ECG Signals” on page 7-83

Walsh-Hadamard Transform for Spectral Analysis and Compression of ECG Signals

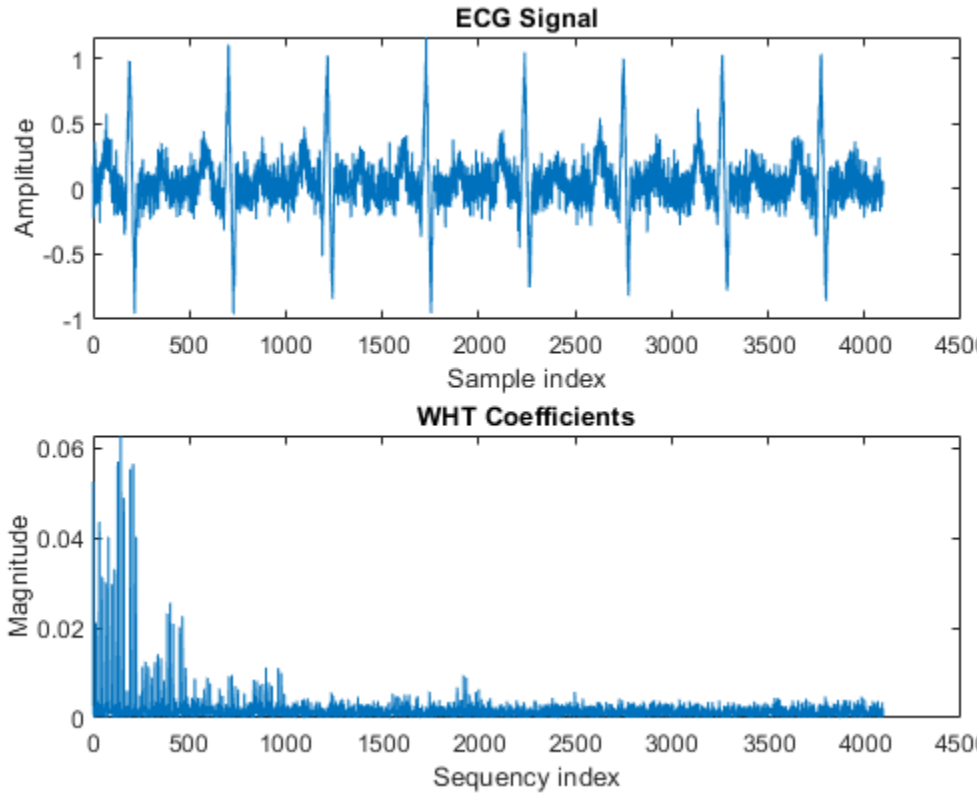
Use an electrocardiogram (ECG) signal to illustrate working with the Walsh-Hadamard transform. ECG signals typically are very large and need to be stored for analysis and retrieval at a future time. Walsh-Hadamard transforms are particularly well-suited to this application because they provide compression and thus require less storage space. They also provide rapid signal reconstruction.

Start with an ECG signal. Replicate it to create a longer signal and insert some additional random noise.

```
xe = ecg(512);
xr = repmat(xe,1,8);
x = xr + 0.1.*randn(1,length(xr));
```

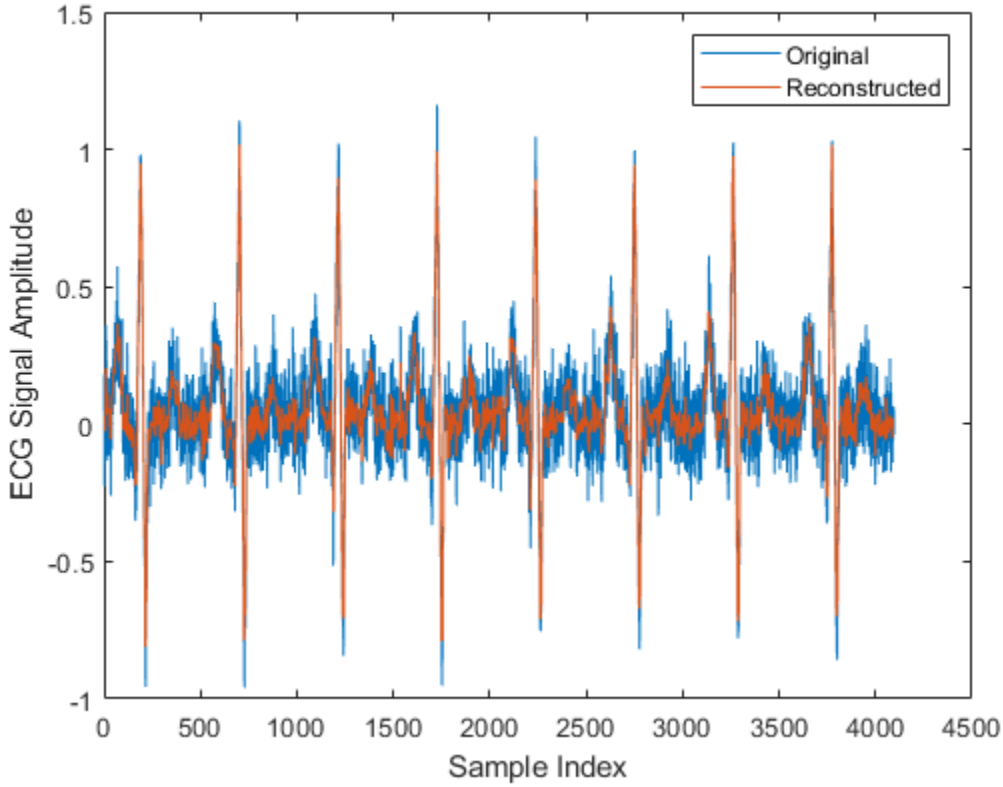
Transform the signal using the fast Walsh-Hadamard transform. Plot the original signal and the transformed signal.

```
y = fwht(x);

subplot(2,1,1)
plot(x)
xlabel('Sample index')
ylabel('Amplitude')
title('ECG Signal')

subplot(2,1,2)
plot(abs(y))
xlabel('Sequency index')
ylabel('Magnitude')
title('WHT Coefficients')
```



The plot shows that most of the signal energy is in the lower sequency values, below approximately 1100. Store only the first 1024 coefficients (out of 4096). Try to reconstruct the signal accurately from only these stored coefficients.

```
y(1025:length(x)) = 0;
xHat = ifwht(y);

figure
plot(x)
hold on
plot(xHat)
xlabel('Sample Index')
ylabel('ECG Signal Amplitude')
legend('Original','Reconstructed')
```



The reproduced signal is very close to the original but has been compressed to a quarter of the size. Storing more coefficients is a tradeoff between increased resolution and increased noise, while storing fewer coefficients can cause loss of peaks.

See Also

`fwht` | `ifwht`

Eliminate Outliers Using Hampel Identifier

This example shows a naive implementation of the procedure used by `hampel` to detect and remove outliers. The actual function is much faster.

Generate a random signal, `x`, containing 24 samples. Reset the random number generator for reproducible results.

```
rng default

lx = 24;
x = randn(1,lx);
```

Generate an observation window around each element of `x`. Take `k = 2` neighbors at either side of the sample. The moving window that results has a length of $2 \times 2 + 1 = 5$ samples.

```
k = 2;

iLo = (1:lx)-k;
iHi = (1:lx)+k;
```

Truncate the window so that the function computes medians of smaller segments as it reaches the signal edges.

```
iLo(iLo<1) = 1;
iHi(iHi>lx) = lx;
```

Record the median of each surrounding window. Find the median of the absolute deviation of each element with respect to the window median.

```
for j = 1:lx
    w = x(iLo(j):iHi(j));
    medj = median(w);
    mmed(j) = medj;
    mmad(j) = median(abs(w-medj));
end
```

Scale the median absolute deviation with

$$\frac{1}{\sqrt{2}\,\mathrm{erf}^{-1}(1/2)} \approx 1.4826$$

to obtain an estimate of the standard deviation of a normal distribution.

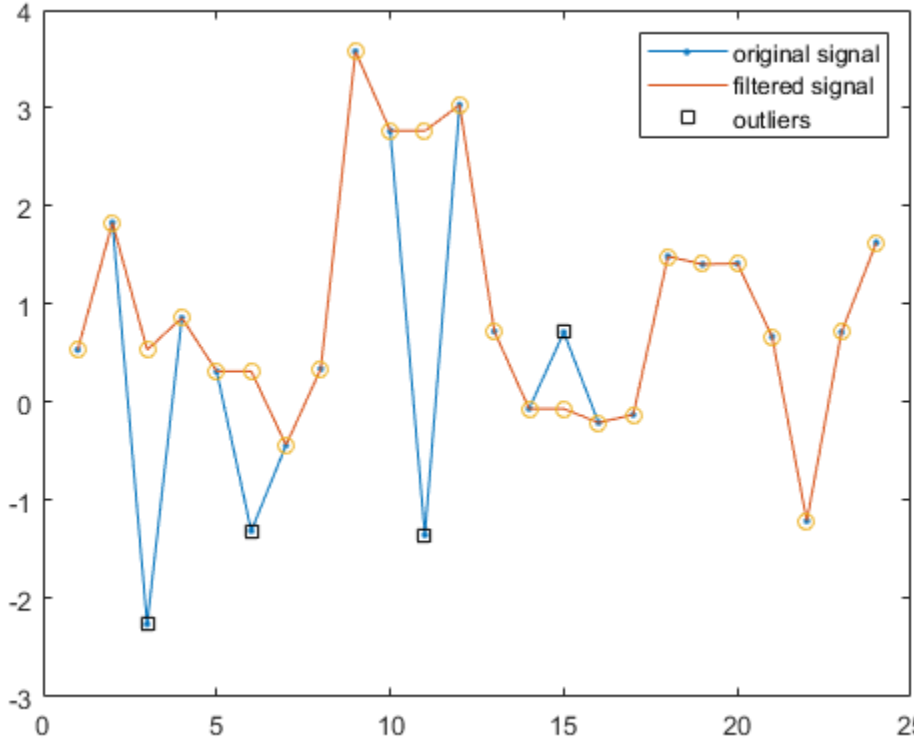
```
sd = mmad/(erfinv(1/2)*sqrt(2));
```

Find the samples that differ from the median by more than `nd` = 2 standard deviations. Replace each of those outliers by the value of the median of its surrounding window. This is the essence of the Hampel algorithm.

```
nd = 2;
ki = abs(x-mmed) > nd*sd;

yu = x;
yu(ki) = mmed(ki);
```

Use the `hampel` function to compute the filtered signal and annotate the outliers. Overlay the filtered values computed in this example.

```
hampel(x,k,nd)

hold on
plot(yu,'o','HandleVisibility','off')
hold off
```



See Also

`hampel`

Selected Bibliography

[1] Kay, Steven M. *Modern Spectral Estimation*. Englewood Cliffs, NJ: Prentice Hall, 1988.

[2] Oppenheim, Alan V., and Ronald W. Schafer. *Discrete-Time Signal Processing*. Englewood Cliffs, NJ: Prentice Hall, 1989.

[3] Oppenheim, Alan V., and Ronald W. Schafer. *Discrete-Time Signal Processing*. Englewood Cliffs, NJ: Prentice Hall, 1975.

[4] Parks, Thomas W., and C. Sidney Burrus. *Digital Filter Design*. New York: John Wiley & Sons, 1987.

[5] Pratt, W. K. *Digital Image Processing*. New York: John Wiley & Sons, 1991.

SPTool: A Signal Processing GUI Suite

- “SPTool: An Interactive Signal Processing Environment” on page 8-2
- “Opening SPTool” on page 8-4
- “Getting Context-Sensitive Help” on page 8-6
- “Signal Browser” on page 8-7
- “Filter Visualization Tool” on page 8-10
- “Spectrum Viewer” on page 8-12
- “Filtering and Analysis of Noise” on page 8-15
- “Exporting Signals, Filters, and Spectra” on page 8-26
- “Accessing Filter Parameters” on page 8-28
- “Importing Filters and Spectra” on page 8-30
- “Loading Variables from the Disk” on page 8-34
- “Saving and Loading Sessions” on page 8-35
- “Selecting Signals, Filters, and Spectra” on page 8-37
- “Editing Signals, Filters, or Spectra” on page 8-38
- “Making Signal Measurements with Markers” on page 8-39
- “Setting Preferences” on page 8-41

SPTool: An Interactive Signal Processing Environment

In this section...
“SPTool Overview” on page 8-2 “SPTool Data Structures” on page 8-2

SPTool Overview

SPTool is an interactive GUI for digital signal processing used to

- Analyze signals
- Design filters
- Analyze (view) filters
- Filter signals
- Analyze signal spectra

You can accomplish these tasks using four GUIs that you access from within SPTool:

- The “Signal Browser” on page 8-7 is for analyzing signals. You can also play signals using your computer's audio hardware.
- **Filter Designer** is available for designing or editing FIR and IIR digital filters. Most Signal Processing Toolbox filter design methods available at the command line are also available in **Filter Designer**.
- The “Filter Visualization Tool” on page 8-10 (FVTool) is for analyzing filter characteristics.
- The “Spectrum Viewer” on page 8-12 is for spectral analysis. You can use Signal Processing Toolbox spectral estimation methods to estimate the power spectral density of a signal.

SPTool Data Structures

You can use SPTool to analyze signals, filters, or spectra that you create at the MATLAB command line.

You can bring signals, filters, or spectra from the MATLAB workspace into the SPTool workspace using **File > Import**. For more information, see “Importing Filters and

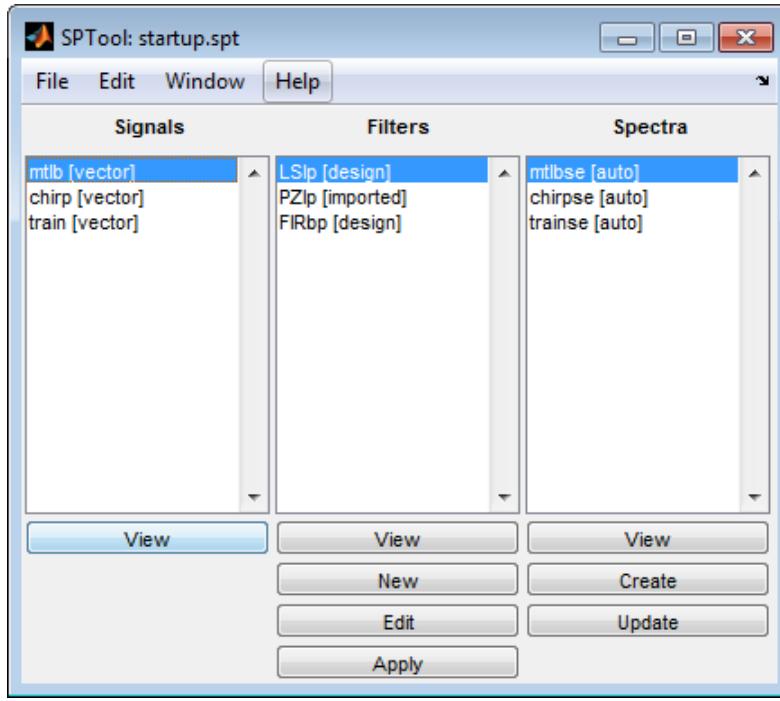
Spectra” on page 8-30. Signals, filters, or spectra that you create in (or import into) the SPTool workspace exist as MATLAB structures. See the MATLAB documentation for more information on MATLAB structures.

When you use **File > Export** to save signals, filters, and spectra that you create or modify in SPTool, these are also saved as MATLAB structures. For more information on exporting, see “Exporting Signals, Filters, and Spectra” on page 8-26.

Opening SPTool

To open SPTool, type

```
sptool
```



When you first open SPTool, it contains a collection of default signals, filters, and spectra. To specify your own preferences for what signals, filters, and spectra to see when SPTool opens see “Setting Preferences” on page 8-41.

You can access these three GUIs from SPTool by selecting a signal, filter, or spectrum and clicking the appropriate **View** button:

- Signal Browser on page 8-7
- Filter Visualization Tool on page 8-10
- Spectrum Viewer on page 8-12

You can access **Filter Designer** by clicking **New** to create a new filter or `Edit` to edit a selected filter. Clicking **Apply** applies a selected filter to a selected signal.

Create opens the Spectrum Viewer and creates the power spectral density of the selected signal. **Update** opens the Spectrum Viewer for the selected spectrum.

Getting Context-Sensitive Help

To find information on a particular feature or setting of the “Signal Browser” on page 8-7:

- In any Measurements panel, right-click anywhere on the panel and select **What's this?**.
- In any dialog box where you see the icon in the lower left corner, right-click on any parameter and select **What's this?**.

To find information on a particular region of **Filter Designer** or “Spectrum Viewer” on page 8-12:

1. Click **What's this?** .
2. Click on the region of the GUI you want information on.

You can also use **Help > What's This?** to launch context-sensitive help.

Signal Browser

In this section...
“Overview of the Signal Browser” on page 8-7 “Opening the Signal Browser” on page 8-7

Overview of the Signal Browser

You can use the Signal Browser to display and analyze signals listed in the **Signals** list box in SPTool.

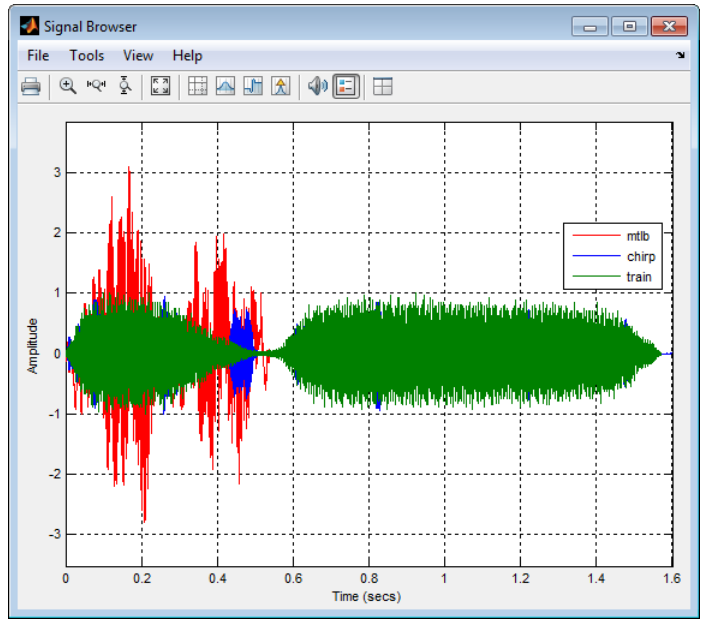
Using the Signal Browser, you can:

- Analyze and compare vector or array (matrix) signals.
- Zoom in on portions of signal data.
- Measure a variety of characteristics of signal data.
- Compare multiple signals.
- Play portions of signal data on audio hardware.
- Print signal plots.

Opening the Signal Browser

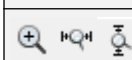
To open the Signal Browser from SPTool:

1. Select one or more signals in the **Signals** list in SPTool.
2. Click **View** under the **Signals** list.



The Signal Browser has the following components:

- A display region for analyzing signals
- A panels section on the right side of the scope window, which shows statistics and information about your signals
- A toolbar with buttons for convenient access to frequently used functions

Icon	Description
	Print the current scope window.
	Play an audio signal. The function `soundsc` is used to play the signal.
	Show multiple displays of signals.
	Zoom the signal in and out.
	Scale the axes.

Icon	Description
	Toggle the legends on and off.
	Toggle the Cursor Measurements panel. This panel allows you to see screen cursors and get measurements of time and amplitude values at the cursors.
	Toggle the Signal Statistics, Bilevel Measurements, and Peak Finder panels, which display various measurements about the selected signal.

For more information on the Signal Browser, see the `sptool` function reference page.

Filter Visualization Tool

In this section...
“Connection between FVTool and SPTool” on page 8-10 “Opening the Filter Visualization Tool” on page 8-10 “Analysis Parameters” on page 8-11

Connection between FVTool and SPTool

You can use the Filter Visualization Tool to analyze response characteristics of the selected filter(s). See `fvtool` for detailed information about FVTool.

If you start FVTool by clicking the SPTool **Filter View** button, that FVTool is linked to SPTool. Any changes made in SPTool to the filter are immediately reflected in FVTool. The FVTool title bar includes "SPTool" to indicate the link.

If you start an FVTool by clicking the **New** button or by selecting **File > New** from within FVTool, that FVTool is a standalone version and is not linked to SPTool.

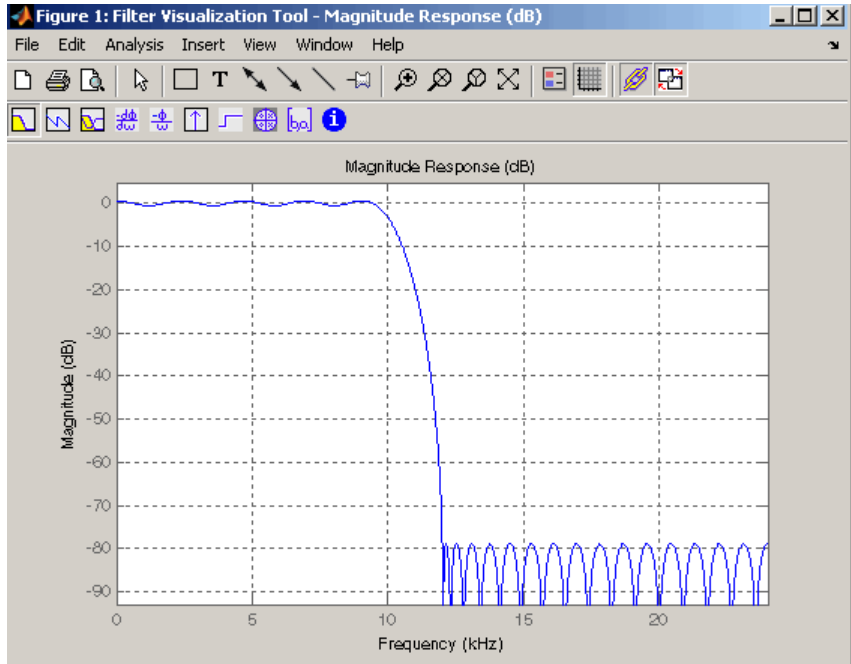
Note Every time you click the **Filter View** button a new, linked FVTool starts. This allows you to view multiple analyses simultaneously.

Opening the Filter Visualization Tool

You open FVTool from SPTool as follows.

1. Select one or more filters in the **Filters** list in SPTool.
2. Click the **View** button under the **Filters** list.

When you first open FVTool, it displays the selected filter's magnitude plot.



Analysis Parameters

In the plot area of any filter response plot, right-click and select **Analysis Parameters** to display details about the displayed plot. See “Analysis Parameters” in the **Filter Designer** online help for more information.

You can change any parameter in a linked FVTool, except the sampling frequency. You can only change the sampling frequency using the SPTool **Edit > Sampling Frequency** or the SPTool **Filters Edit** button.

Spectrum Viewer

In this section...
“Spectrum Viewer Overview” on page 8-12 “Opening the Spectrum Viewer” on page 8-12

Spectrum Viewer Overview

You can use the Spectrum Viewer for estimating and analyzing a signal's power spectral density (PSD). You can use the PSD estimates to understand a signal's frequency content.

The Spectrum Viewer provides the following functionality.

- Analyze and compare spectral density plots.
- Use different spectral estimation methods to create spectra:
  - Burg (`pburg`)
  - Covariance (`pcov`)
  - FFT (`fft`)
  - Modified covariance (`pmcov`)
  - MTM (multitaper method) (`pmtm`)
  - MUSIC (`pmusic`)
  - Welch (`pwelch`)
  - Yule-Walker AR (`pyulear`)
- Modify power spectral density parameters such as FFT length, window type, and sample frequency.
- Print spectral plots.

Opening the Spectrum Viewer

To open the Spectrum Viewer and create a PSD estimate from SPTool:

1. Select a signal from the **Signal** list box in SPTool.
2. Click **Create** in the **Spectra** list.

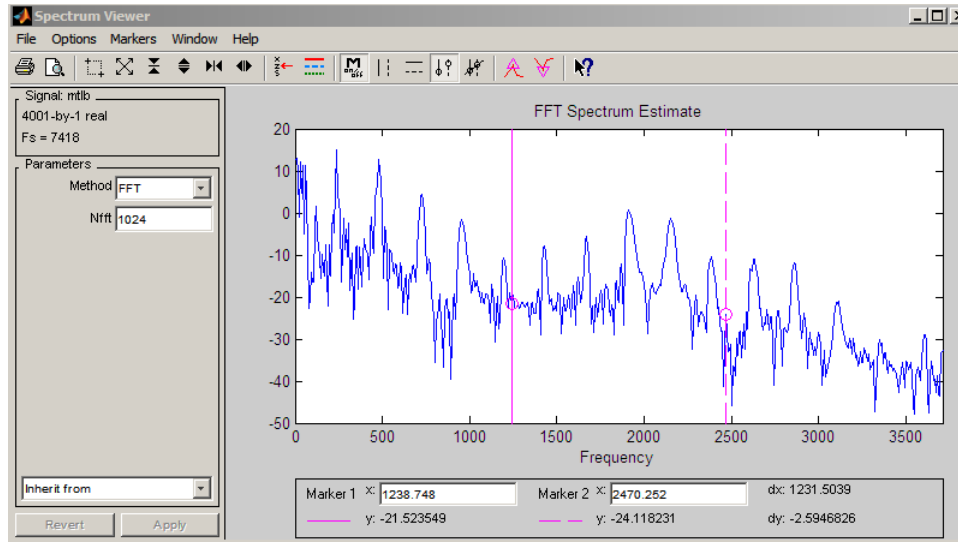
3 Click **Apply** in the Spectrum Viewer.

To open the Spectrum Viewer with a PSD estimate already listed in SPTool:

1 Select a PSD estimate from the **Spectra** list box in SPTool.
2 Click **View** in the Spectra list.

For example:

1 Select `mtlb` in the default **Signals** list in SPTool.
2 Click **Create** in SPTool to open the Spectrum Viewer.
3 Click **Apply** in the Spectrum Viewer to plot the spectrum.



The Spectrum Viewer has the following components:

- A signal identification region that provides information about the signal whose power spectral density estimate is displayed
- A Parameters region for modifying the PSD parameters
- A display region for analyzing spectra and an **Options** menu for modifying display characteristics
- Spectrum management controls

  - **Inherit from** menu to inherit PSD specifications from another PSD object listed in the menu
  - Revert button to revert to the named PSD's original specifications
  - **Apply** button for creating or updating PSD estimates
- A toolbar with buttons for convenient access to frequently used functions

Icon	Description
	Print and print preview
	Zoom the signal in and out
	Select one of several loaded signals
	Set the display color and line style of a signal
	Toggle the markers on and off
	Set marker types
	Turn on the What's This help

Filtering and Analysis of Noise

In this section...
“Overview” on page 8-15
“Importing a Signal into SPTool” on page 8-15
“Designing a Filter” on page 8-17
“Applying a Filter to a Signal” on page 8-19
“Analyzing a Signal” on page 8-21
“Spectral Analysis in the Spectrum Viewer” on page 8-23

Overview

The following sections provide an example of using the GUI-based interactive tools to:

- Design and implement an FIR bandpass digital filter
- Apply the filter to a noisy signal
- Analyze signals and their spectra

The steps include:

1. “Importing a Signal into SPTool” on page 8-15
2. Designing a bandpass filter using Filter Designer on page 8-17
3. Applying the filter to the original noise signal to create a bandlimited noise signal on page 8-19
4. Comparing the time domain information of the original and filtered signals using the Signal Browser on page 8-21
5. Comparing the spectra of both signals using the Spectrum Viewer on page 8-23

Importing a Signal into SPTool

To import a signal into SPTool from the workspace or disk, the signal must be either:

- A special MATLAB signal structure, such as that saved from a previous SPTool session
- A signal created as a variable (vector or matrix) in the MATLAB workspace

For this example, create a new signal at the command line and then import it as a structure into SPTool:

1. Create a random signal in the MATLAB workspace by typing

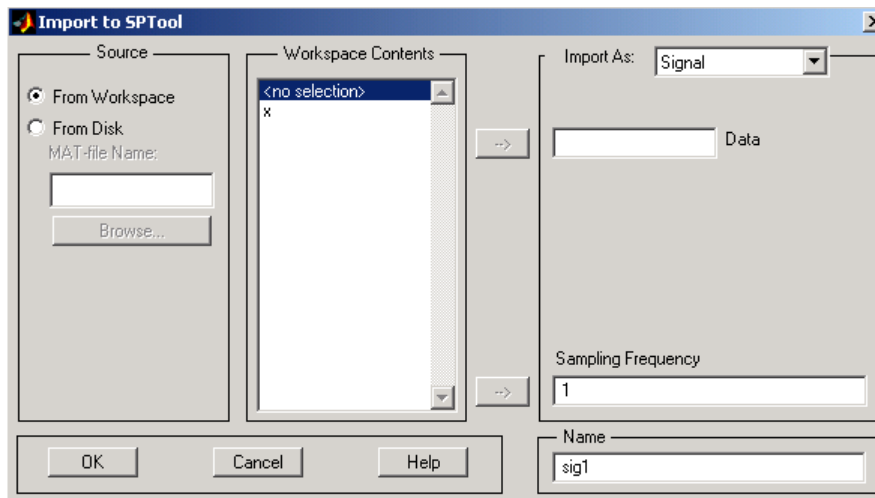
   ```
   x = randn(5000,1);
   ```

2. If SPTool is not already open, open SPTool by typing

   ```
   sptool
   ```

   The SPTool window is displayed.

3. Select **File > Import**. The Import to SPTool dialog opens.



   The variable `x` is displayed in the **Workspace Contents** list. (If it is not, select the **From Workspace** radio button to display the contents of the workspace.)

4. Select the signal and import it into the **Data** field:

   a. Select the signal variable `x` in the **Workspace Contents** list.

   b. Make sure that `Signal` is selected in the **Import As** pull-down menu.

   c. Click on the arrow to the left of the **Data** field or type `x` in the **Data** field.

   d. Type `5000` in the **Sampling Frequency** field.

   e. Name the signal by typing `noise` in the **Name** field.

f Click **OK**.

The signal `noise[vector]` appears and is selected in SPTool's **Signals** list.

Note You can import filters on page 8-30 and spectra on page 8-32 into SPTool in much the same way as you import signals. See “Importing Filters and Spectra” on page 8-30 for specific details.

You can also import signals from MAT-files on your disk, rather than from the workspace. See “Loading Variables from the Disk” on page 8-34 for more information.

Type `help sptool` for information about importing from the command line.

Designing a Filter

You can import an existing filter into SPTool, or you can design and edit a new filter using **Filter Designer**.

In this example, you

1. Open a default filter in **Filter Designer**.
2. Specify an equiripple bandpass FIR filter.

Opening Filter Designer

To open **Filter Designer**, click **New** in SPTool. **Filter Designer** opens with a default filter named `filt1`.

Specifying the Bandpass Filter

Design an equiripple bandpass FIR filter with the following characteristics:

- Sampling frequency of 5000 Hz
- Stopband frequency ranges of [0 500] Hz and [1500 2500] Hz
- Passband frequency range of [750 1250] Hz
- Ripple in the passband of 0.01 dB
- Stopband attenuation of 75 dB

To modify the filter in **Filter Designer** to meet these specifications, you need to

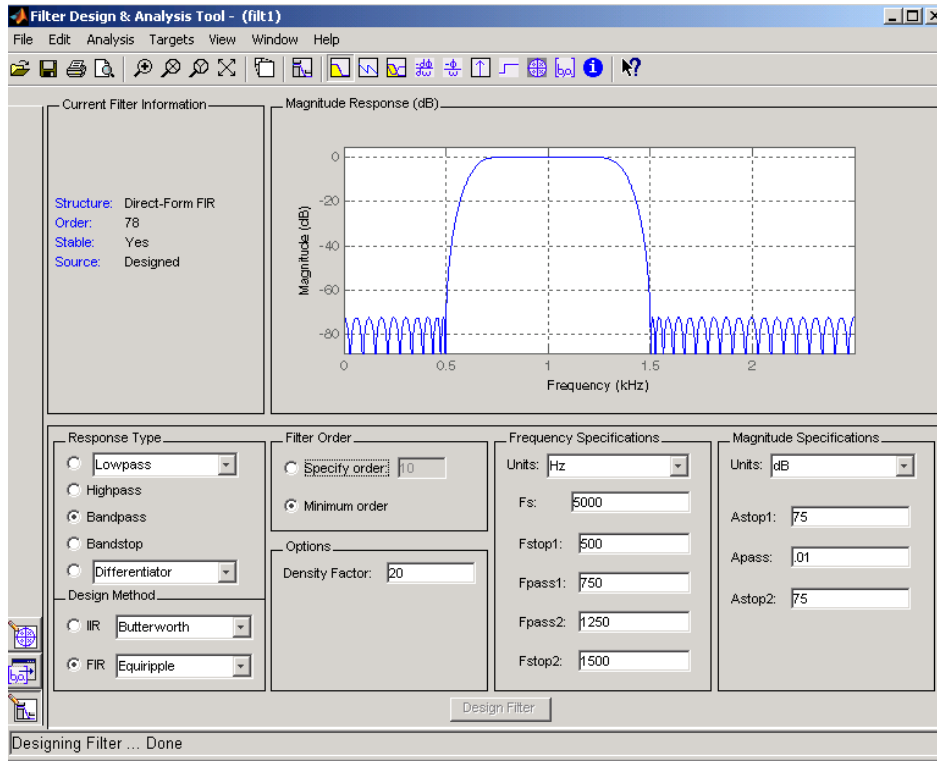
1. Select `Bandpass` from the **Response Type** list.
2. Verify that **FIR Equiripple** is selected as the **Design Method**.
3. Verify that **Minimum order** is selected as the **Filter Order** and that the **Density Factor** is set to `20`.
4. Under **Frequency Specifications**, set the sampling frequency (**Fs**) and the passband (**Fpass1**, **Fpass2**) and stopband (**Fstop1**, **Fstop2**) edges:

Units	**Hz**
Fs	`5000`
Fstop1	`500`
Fpass1	`750`
Fpass2	`1250`
Fstop2	`1500`

5. Under **Magnitude Specifications**, set the stopband attenuation (**Astop1**, **Astop2**) and the maximum passband ripple (**Apass**):

Units	**dB**
Astop1	`75`
Apass	`0.01`
Astop2	`75`

6. Click **Design Filter** to design the new filter. When the new filter is designed, the magnitude response of the filter is displayed.



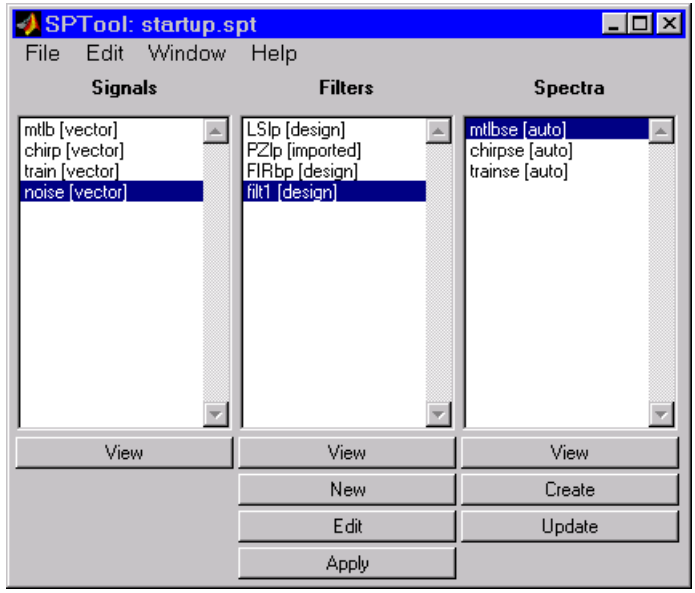
The resulting filter is an order-78 bandpass equiripple filter.

Applying a Filter to a Signal

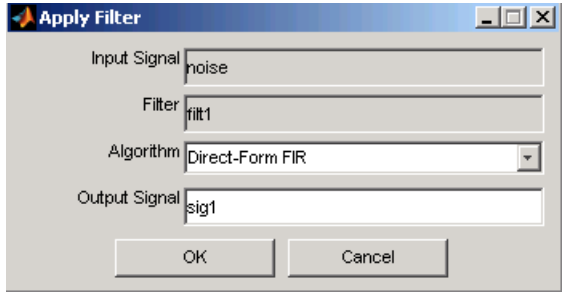
When you apply a filter to a signal, you create a new signal in SPTool representing the filtered signal.

To apply the filter `filt1` you just created to the signal `noise`,

1. In SPTool, select the signal `noise[vector]` from the **Signals** list and select the filter (named `filt1[design]`) from the **Filters** list.



2 Click **Apply** under the **Filters** list.



3 Leave the **Algorithm** as `Direct-Form FIR`.

Note You can apply one of two filtering algorithms to FIR filters. The default algorithm is specific to the filter structure, which is shown in the **Filter Designer** Current Filter Info frame. Alternately for FIR filters, `FFT based FIR (fftfilt)` uses the algorithm described in `fftfilt`.

For IIR filters, the alternate algorithm is a zero-phase IIR that uses the algorithm described in `filtfilt`.

---

4. Enter `blnoise` as the **Output Signal** name.
5. Click **OK** to close the Apply Filter dialog box.

   The filter is applied to the selected signal, and the filtered signal `blnoise[vector]` is listed in the **Signals** list in SPTool.

Analyzing a Signal

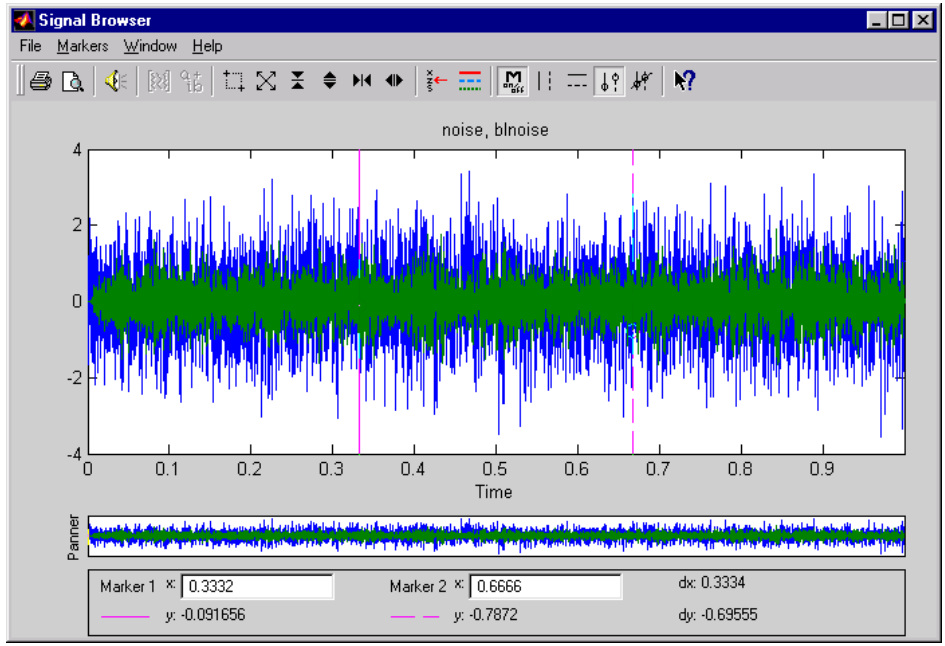
You can analyze and print signals using the Signal Browser. You can also play the signals if your computer has audio output capabilities.

For example, compare the signal `noise on page 8-15` to the filtered signal `blnoise on page 8-19`:

1. **Shift**+click on the `noise` and `blnoise` signals in the **Signals** list of SPTool to select both signals.
2. Click **View** under the **Signals** list.

   The Signal Browser is activated, and both signals are displayed in the display region. (The names of both signals are shown above the display region.) Initially, the original `noise` signal covers up the bandlimited `blnoise` signal.
3. Push the selection button on the toolbar to select the `blnoise` signal.

   The display area is updated. Now you can see the `blnoise` signal superimposed on top of the `noise` signal. The signals are displayed in different colors in both the display region and the panner. You can change the color of the selected signal using the **Line Properties** button on the toolbar, .



Playing a Signal

When you click **Play** in the Signal Browser toolbar, , the active signal is played on the computer's audio hardware.

1. To hear a portion of the active (selected) signal
   a. Use the vertical markers on page 8-39to select a portion of the signal you want to play. Vertical markers are enabled by the and buttons.
   b. Click **Play**.
2. To hear the other signal
   a. Select the signal using the selection button on the toolbar. You can also select the signal directly in the display region.
   b. Click **Play** again.

Printing a Signal

You can print from the Signal Browser using the **Print** button, .

You can use the line display buttons to maximize the visual contrast between the signals by setting the line color for noise on page 8-15 to gray and the line color for blnoise on page 8-15 to white. Do this before printing two signals together.

Note You can follow the same rules to print spectra, but you can't print filter responses directly from SPTool.

Use the Signal Browser region in the Preferences dialog box in SPTool to suppress printing of both the panner and the marker settings.

To print both signals, click **Print** in the Signal Browser toolbar.

Spectral Analysis in the Spectrum Viewer

You can analyze the frequency content of a signal using the Spectrum Viewer, which estimates and displays a signal's power spectral density.

For example, to analyze and compare the spectra of `noise` and `blnoise:`

1. Create a power spectral density (PSD) object on page 8-23, `spect1`, that is associated with the signal `noise`, and a second PSD object, spect2, that is associated with the signal `blnoise`.
2. Open the Spectrum Viewer to analyze both of these spectra on page 8-23.
3. Print both spectra on page 8-23.

Creating a PSD Object From a Signal

1. Click on SPTool, or select **Window > SPTool** in any active open GUI. SPTool is now the active window.
2. Select the `noise[vector]` on page 8-15 signal in the **Signals** list of SPTool.
3. Click **Create** in the **Spectra** list.

   The Spectrum Viewer is activated, and a PSD (`spect1`) corresponding to the `noise` signal is created in the **Spectra** list. The PSD is not computed or displayed yet.

4 Click **Apply** in the Spectrum Viewer to compute and display the PSD estimate `spect1` using the default parameters.

The PSD of the `noise` signal is displayed in the display region. The identifying information for the PSD's associated signal (`noise`) is displayed above the Parameters region.

The PSD estimate `spect1` is within 2 or 3 dB of 0, so the noise has a fairly "flat" power spectral density.

5 Follow steps 1 through 4 for the bandlimited noise signal blnoise on page 8-19 to create a second PSD estimate `spect2`.

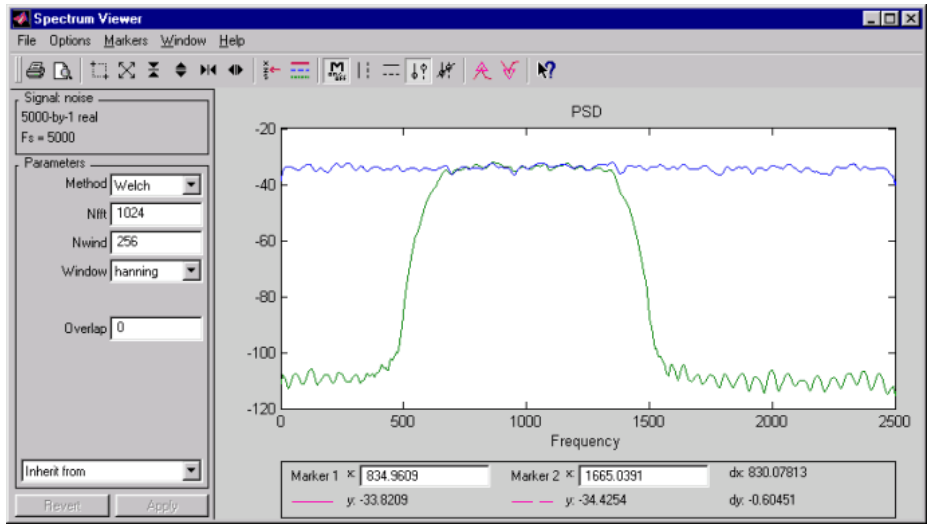
The PSD estimate `spect2` is flat between 750 and 1250 Hz and has 75 dB less power in the stopband regions of `filt1`.

Opening the Spectrum Viewer with Two Spectra

1 Reactivate SPTool again, as in step 1 above.

2 **Shift**+click on `spect1` and `spect2` in the **Spectra** list to select them both.

3 Click **View** in the **Spectra** list to reactivate the Spectrum Viewer and display both spectra together.



Printing the Spectra

Before printing the two spectra together, use the color and line style selection button, , to differentiate the two plots by line style, rather than by color.

To print both spectra:

1. Click **Print Preview** in the toolbar on the Spectrum Viewer.
2. From the Spectrum Viewer Print Preview window, drag the legend out of the display region so that it doesn't obscure part of the plot.
3. Click **Print** in the Spectrum Viewer Print Preview window.

Exporting Signals, Filters, and Spectra

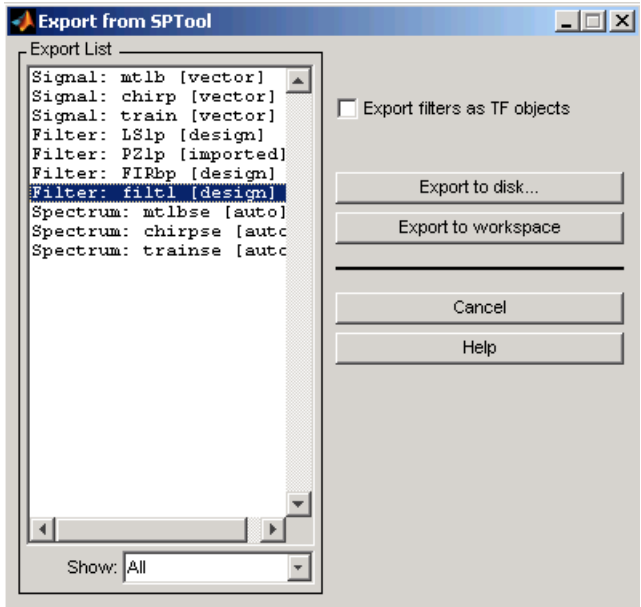
In this section...
“Opening the Export Dialog Box” on page 8-26 “Exporting a Filter to the MATLAB Workspace” on page 8-27

Opening the Export Dialog Box

To save the filter `filt1` you just created in this example, open the Export dialog box with `filt1` preselected:

1. Select `filt1` in the SPTool **Filters** list.
2. Select **File > Export**.

The Export dialog box opens with `filt1` preselected.



Exporting a Filter to the MATLAB Workspace

To export the filter `filt1` to the MATLAB workspace:

1. Select `filt1` from the **Export List** and deselect all other items using **Ctrl**+click.
2. Click **Export to Workspace**.

Accessing Filter Parameters

In this section...
“Accessing Filter Parameters in a Saved Filter” on page 8-28
“Accessing Parameters in a Saved Spectrum” on page 8-29

Accessing Filter Parameters in a Saved Filter

The MATLAB structures created by SPTool have several associated fields, many of which are also MATLAB structures. See the MATLAB documentation for general information about MATLAB structures.

For example, after exporting on page 8-27 a filter filt1 to the MATLAB workspace, type

```
filt1
```

to display the fields of the MATLAB filter structure. The `tf` field of the structure contains information that describes the filter.

The tf Field: Accessing Filter Coefficients

The `tf` field is a structure containing the transfer function representation of the filter. Use this field to obtain the filter coefficients;

- `filt1.tf.num` contains the numerator coefficients.
- `filt1.tf.den` contains the denominator coefficients.

The vectors contained in these structures represent polynomials in descending powers of z. The numerator and denominator polynomials are used to specify the transfer function

$$H(z) = \frac{B(z)}{A(z)} = \frac{b(1) + b(2)z^{-1} + \cdots + b(nb+1)z^{-m}}{a(1) + a(2)z^{-1} + \cdots + a(na+1)z^{-n}}$$

where:

- b is a vector containing the coefficients from the `tf.num` field.
- a is a vector containing the coefficients from the `tf.den` field.
- m is the numerator order.

- *n* is the denominator order.

You can change the filter representation from the default transfer function to another form by using the `tf2ss` or `tf2zp` functions.

Note The `FDAspecs` field of your filter contains internal information about **Filter Designer** and should not be changed.

Accessing Parameters in a Saved Spectrum

The following structure fields describe the spectra saved by SPTool.

Field	Description
`P`	The spectral power vector.
`f`	The spectral frequency vector.
`confid`	A structure containing the confidence intervals data • The `confid.level` field contains the chosen confidence level. • The `confid.Pc` field contains the spectral power data for the confidence intervals. • The `confid.enable` field contains a `1` if confidence levels are enabled for the power spectral density.
`signalLabel`	The name of the signal from which the power spectral density was generated.
`Fs`	The associated signal's sample rate.

You can access the information in these fields as you do with every MATLAB structure.

For example, if you export an SPTool PSD estimate `spect1` to the workspace, type

```
spect1.P
```

to obtain the vector of associated power values.

Importing Filters and Spectra

In this section...
“Similarities to Other Procedures” on page 8-30
“Importing Filters” on page 8-30
“Importing Spectra” on page 8-32

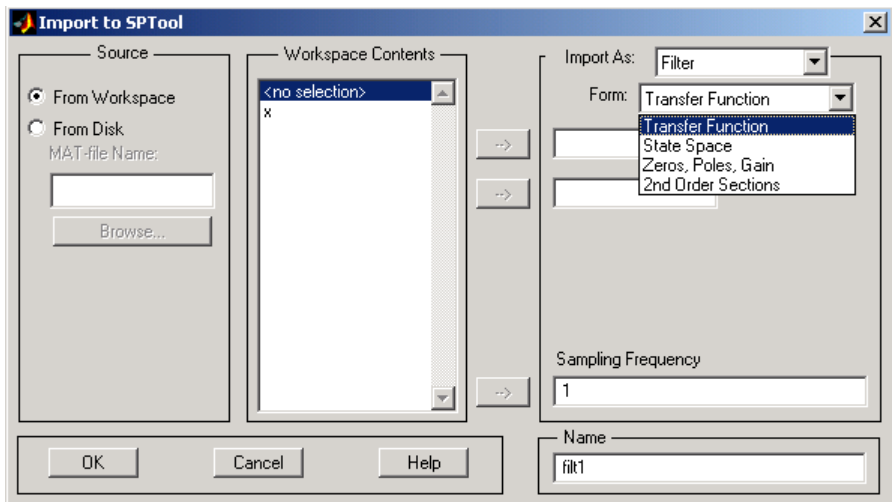
Similarities to Other Procedures

The procedures are very similar to those explained in

- “Importing a Signal into SPTool” on page 8-15 for loading variables from the workspace
- “Loading Variables from the Disk” on page 8-34 for loading variables from your disk

Importing Filters

When you import filters, first select the appropriate filter form from the **Form** list. SPTool does not currently support the import of filter objects.



For every filter you specify a variable name or a value for the filter's sampling frequency in the **Sampling Frequency** field. Each filter form requires different variables.

Transfer Function

For `Transfer Function`, you specify the filter by its transfer function representation:

$$H(z) = \frac{B(z)}{A(z)} = \frac{b(1) + b(2)z^{-1} + \cdots + b(m+1)z^{-m}}{a(1) + a(2)z^{-1} + \cdots + a(n+1)z^{-n}}$$

- The **Numerator** field specifies a variable name or value for the numerator coefficient vector b, which contains $m+1$ coefficients in descending powers of z.
- The **Denominator** field specifies a variable name or value for the denominator coefficient vector a, which contains $n+1$ coefficients in descending powers of z.

State Space

For `State Space`, you specify the filter by its state-space representation:

$$\dot{x} = Ax + Bu$$
$$y = Cx + Du$$

The **A-Matrix**, **B-Matrix**, **C-Matrix**, and **D-Matrix** fields specify a variable name or a value for each matrix in this system.

Zeros, Poles, Gain

For `Zeros`, `Poles`, `Gain`, you specify the filter by its zero-pole-gain representation:

$$H(z) = \frac{Z(z)}{P(z)} = k\frac{(z - z(1))(z - z(2))\cdots(z - z(m))}{(z - p(1))(z - p(2))\cdots(z - p(n))}$$

- The **Zeros** field specifies a variable name or value for the zeros vector z, which contains the locations of m zeros.
- The **Poles** field specifies a variable name or value for the zeros vector p, which contains the locations of n poles.
- The **Gain** field specifies a variable name or value for the gain k.

Second Order Sections

For `2nd Order Sections` you specify the filter by its second-order section representation:

$$H(z) = \prod_{k=1}^{L} H_k(z) = \prod_{k=1}^{L} \frac{b_{0k} + b_{1k} z^{-1} + b_{2k} z^{-2}}{1 + a_{1k} z^{-1} + a_{2k} z^{-2}}$$

The **SOS Matrix** field specifies a variable name or a value for the L-by-6 SOS matrix

$$\text{sos} = \begin{bmatrix} b_{01} & b_{11} & b_{21} & 1 & a_{11} & a_{21} \\ b_{02} & b_{12} & b_{22} & 1 & a_{12} & a_{22} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ b_{0L} & b_{1L} & b_{2L} & 1 & a_{1L} & a_{2L} \end{bmatrix}$$

whose rows contain the numerator and denominator coefficients b_{ik} and a_{ik} of the second-order sections of $H(z)$.

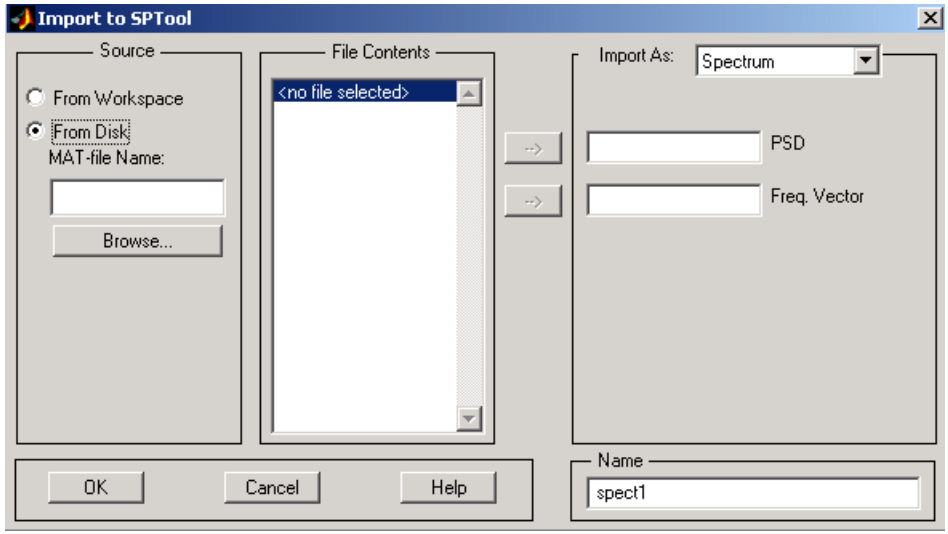
Note If you import a filter that was not created in SPTool, you can only edit that filter using the Pole/Zero Editor.

Importing Spectra

When you import a power spectral density (PSD), you specify:

- A variable name or a value for the PSD vector in the **PSD** field
- A variable name or a value for the frequency vector in the **Freq. Vector** field

The PSD values in the **PSD** vector correspond to the frequencies contained in the **Freq. Vector** vector; the two vectors must have the same length.


Import to SPTool
Source
From Workspace
From Disk
MAT-file Name:
Browse...
File Contents
<no file selected>
Import As:
Spectrum
PSD
Freq. Vector
OK
Cancel
Help
Name
spect1

Loading Variables from the Disk

To import variables representing signals, filters, or spectra from a MAT-file on your disk;

1. Select the **From Disk** radio button and do either of the following:

   - Type the name of the file you want to import into the **MAT-file Name** field and press either the **Tab** or the **Enter** key on your keyboard.
   - Select **Browse**, and then find and select the file you want to import using **Select > File to Open**. Click **OK** to close that dialog.

   In either case, all variables in the MAT-file you selected are displayed in the **File Contents** list.
2. Select the variables to be imported into SPTool.

You can now import one or more variables from the **File Contents** list into SPTool, as long as these variables are scalars, vectors, or matrices.

Saving and Loading Sessions

In this section...
“SPTool Sessions” on page 8-35
“Filter Formats” on page 8-35

SPTool Sessions

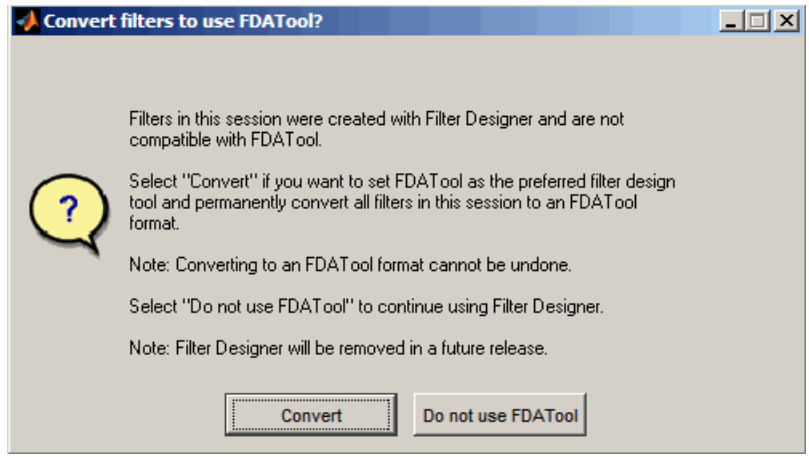
When you start SPTool, the default `startup.spt` session is loaded. To save your work in the startup SPTool session, use **File > Save Session** or to specify a session name, use **File > Save Session As**.

To recall a previously saved session, use **File > Open Session**.

Filter Formats

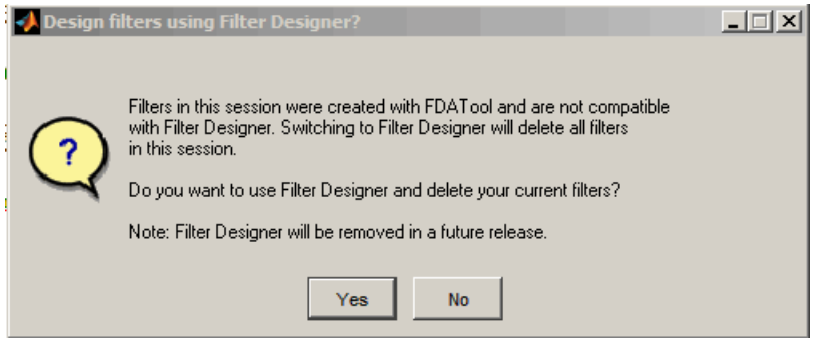
When you start SPTool or open a session, the current filter design format preference is compared to the filter formats in the session. See “Setting Preferences” on page 8-41.

- If the formats match, the session opens.
- If the filter preference is FDATool, but the session contains Filter Designer filters, this warning displays:



Click **Convert** to convert the filters to FDATool format. Click **Don't Use FDATool** to leave the filters in Filter Designer format and change the preference to **Use Filter Designer**.

- If the filter preference is **Use Filter Designer**, but the session contains FDATool filters, this warning displays:



Click **Yes** to remove the current filters. Click **No** to leave the filters in FDATool.

Selecting Signals, Filters, and Spectra

All signals, filters, or spectra listed in SPTool exist as special MATLAB structures. You can bring data representing signals, filters, or spectra into SPTool from the MATLAB workspace. In general, you can select one or several items in a given list box. An item is selected when it is highlighted.

The **Signals** list shows all vector and array signals in the current SPTool session.

The **Filters** list shows all designed and imported filters in the current SPTool session.

The **Spectra** list shows all spectra in the current SPTool session.

You can select a single data object in a list, a range of data objects in a list, or multiple separate data objects in a list. You can also have data objects simultaneously selected in different lists:

- To select a single item, click it. All other items in that list box become deselected.
- To add or remove a range of items, **Shift**+click on the items at the top and bottom of the section of the list that you want to add. You can also drag your mouse pointer to select these items.
- To add a single data object to a selection or remove a single data object from a multiple selection, **Ctrl**+click on the object.

Editing Signals, Filters, or Spectra

You can edit selected items in SPTool by

1. Selecting the names of the signals, filters, or spectra you want to edit.
2. Selecting the appropriate **Edit** menu item:

   - **Duplicate** to copy an item in an SPTool list
   - **Clear** to delete an item in an SPTool list
   - **Name** to rename an item in an SPTool list
   - **Sampling Frequency** to modify the sampling frequency associated with either a signal (and its associated spectra) or filter in an SPTool list

   The pull-down menu next to each menu item shows the names of all selected items.

You can also edit the following signal characteristics by right-clicking in the display region of the Signal Browser, the Filter Visualization Tool, or the Spectrum Viewer:

- The signal name
- The sampling frequency
- The line style properties

Note If you modify the sampling frequency associated with a signal's spectrum using the right-click menu on the Spectrum Viewer display region, the sampling frequency of the associated signal is automatically updated.

Making Signal Measurements with Markers

You can use the markers on the Signal Browser on page 8-7 or the Spectrum Viewer on page 8-7 to make measurements on either of the following:

- A signal in the Signal Browser
- A power spectral density plotted in the Spectrum Viewer

The following marker buttons are included

Icon	Description
	Toggle markers on/off
	Vertical markers
	Horizontal markers
	Vertical markers with tracking
	Vertical markers with tracking and slope
	Display peaks (local maxima) You can find peaks in a signal from the command line with `findpeaks`
	Display valleys (local minima)

To make a measurement:

1. Select a line to measure (or play, if you are in the Signal Browser).
2. Select one of the marker buttons to apply a marker to the displayed signal.
3. Position a marker in the main display area by grabbing it with your mouse and dragging:

a Select a marker setting. If you choose the **Vertical**, **Track**, or **Slope** buttons, you can drag a marker to the right or left. If you choose the **Horizontal** button, you can drag a marker up or down.

b Move the mouse over the marker (1 or 2) that you want to drag.

The hand cursor with the marker number inside it is displayed when your mouse passes over a marker.

c Drag the marker to where you want it on the signal

As you drag a marker, the bottom of the Signal Browser shows the current position of both markers. Depending on which marker setting you select, some or all of the following fields are displayed — **x1**, **y1**, **x2**, **y2**, **dx**, **dy**, **m**. These fields are also displayed when you print from the Signal Browser, unless you suppress them.

You can also position a marker by typing its **x1** and **x2** or **y1** and **y2** values in the region at the bottom.

Setting Preferences

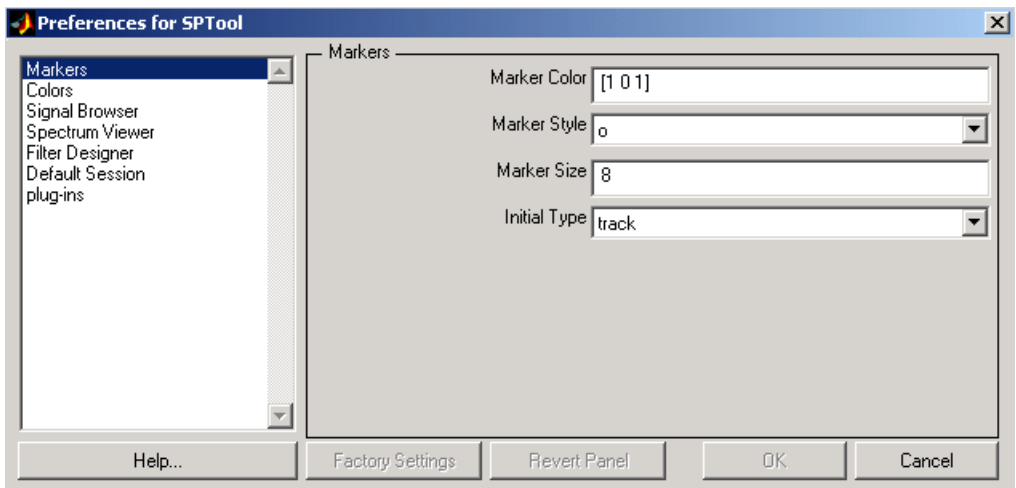
In this section...
“Overview of Setting Preferences” on page 8-41 “Summary of Settable Preferences” on page 8-42

Overview of Setting Preferences

Use **File > Preferences** to customize displays and certain parameters for SPTool and its four component GUIs. If you change any preferences, a dialog box displays when you close SPTool asking if you want to save those changes. If you click **Yes**, the new settings are saved on disk and are used when you restart SPTool from the MATLAB workspace.

Note You can set MATLAB preferences that affect the Filter Visualization Tool only from within FVTool by selecting **File > Preferences**. You can set FVTool-specific preferences using **Analysis > Analysis Parameters**.

When you first select **Preferences**, the Preferences dialog box opens with **Markers** selected by default.



Change any marker settings, if desired. To change settings for another category, click its name in the category list to display its settings. Most of the fields are self-explanatory. Details of the Filter Design options are described below.

Summary of Settable Preferences

In the Preferences regions, you can

- Select colors and markers for all displays.
- Select colors and line styles for displayed signals.
- Configure labels, and enable/disable markers, panner, and zoom in the Signal Browser on page 8-7.
- Configure display parameters, and enable/disable markers and zoom in the Spectrum Viewer on page 8-12.
- Enable/disable use of a default session file.
- Export filters for use with Control System Toolbox software.
- Enable/disable search for plug-ins at startup.

Code Generation from MATLAB Support in Signal Processing Toolbox

- “Supported Functions” on page 9-2
- “Specifying Inputs in Code Generation from MATLAB” on page 9-6
- “Code Generation Examples” on page 9-10

Supported Functions

Code generation from MATLAB is a restricted subset of the MATLAB language that provides optimizations for:

- Generating efficient, production-quality C/C++ code and MEX files for deployment in desktop and embedded applications. For embedded targets, the subset restricts MATLAB semantics to meet the memory and data type requirements of the target environments.

Depending on which feature you wish to use, there are additional required products. For a comprehensive list, see “Installing Prerequisite Products” (MATLAB Coder).

Code generation from MATLAB supports Signal Processing Toolbox functions listed in the table. To generate C code, you must have the MATLAB Coder™ software. If you have the Fixed-Point Designer software, you can use `fiaccel` to generate MEX code for fixed-point applications.

To follow the examples in this documentation:

- To generate C/C++ code and MEX files with `codegen`, install the MATLAB Coder software, the Signal Processing Toolbox, and a C compiler. For the Windows® platform, MATLAB supplies a default C compiler. Run `mex -setup` at the MATLAB command prompt to set up the C compiler.
- Change to a folder where you have write permission.

Note Many Signal Processing Toolbox functions require constant inputs in generated code. To specify a constant input for `codegen`, use `coder.Constant`.

An asterisk (*) indicates that the reference page has usage notes and limitations for C/C++ code generation.

`alignsignals`
`barthannwin*`
`bartlett*`
`besselap*`
`bitrevorder`

blackman
blackmanharris*
bohmanwin*
buttap*
butter*
buttord*
cconv
cfirpm*
cheb1ap*
cheb2ap*
cheb1ord*
cheb2ord*
chebwin*
cheby1*
cheby2*
convmtx
corrmtx
db2pow
dct*
downsample
dpss*
ellip*
ellipap*
ellipord*
emd
envelope*
filtfilt*
finddelay
findpeaks

fir1*
fir2*
fircls*
fircls1*
firls*
firpm*
firpmord*
flattopwin
freqz*
gausswin*
hamming
hann
hilbert
idct*
intfilt*
kaiser
kaiserord
levinson*
maxflat*
nuttallwin*
parzenwin*
peak2peak
peak2rms
pow2db
rcosdesign*
rectwin*
resample*
rms
sgolay

sgolayfilt
sinc
sosfilt
taylorwin*
triang*
tukeywin*
upfirdn*
upsample*
xcorr*
xcorr2
xcov
yulewalk*
zp2tf*

Specifying Inputs in Code Generation from MATLAB

In this section...
“Defining Input Size and Type” on page 9-6 “Inputs must be Constants” on page 9-7

Defining Input Size and Type

When you use Signal Processing Toolbox functions for code generation, you must define the size and type of the function inputs. One way to do this is with the `-args` compilation option. The size and type of inputs must be defined because C is a statically typed language. To illustrate the need to define input size and type, consider the simplest call to `xcorr` requiring two input arguments. The following demonstrates the differences in the use of `xcorr` in MATLAB and in Code Generation from MATLAB.

Cross correlate two white noise vectors in MATLAB:

```
x = randn(512,1); %real valued white noise
y = randn(512,1); %real valued white noise
[C,lags] = xcorr(x,y);
x_circ = randn(256,1)+1j*randn(256,1); %circular white noise
y_circ = randn(256,1)+1j*randn(256,1); %circular white noise
[C1,lags1] = xcorr(x_circ,y_circ);
```

`xcorr` does not require the size and type of the input arguments. `xcorr` obtains this information at runtime. Contrast this behavior with a MEX-file created with `codegen`. Create the file `myxcorr.m` in a folder where you have read and write permission. Ensure that this folder is in the MATLAB search path. Copy and paste the following two lines of code into `myxcorr.m` and save the file. The compiler tag `%#codegen` must be included in the file.

```
function [C,Lags]=myxcorr(x,y)  %#codegen
[C,Lags]=xcorr(x,y);
```

Enter the following command at the MATLAB command prompt:

```
codegen myxcorr -args {zeros(512,1),zeros(512,1)} -o myxcorr
```

Run the MEX-file:

```
x = randn(512,1); %real valued white noise
y = randn(512,1); %real valued white noise
[C,Lags] = myxcorr(x,y);
```

Define two new inputs `x1` and `y1` by transposing `x` and `y`.

```
x1 = x'; %x1 is 1x512
y1 = y'; %y1 is 1x512
```

Attempt to rerun the MEX-file with the transposed inputs.

```
[C,Lags] = myxcorr(x1,y1); %Errors
```

The preceding program errors with the message `??? MATLAB expression 'x' is not of the correct size: expected [512x1] found [1x512]`.

The error results because the inputs are specified to be `512x1` real-valued column vectors at compilation. For complex-valued inputs, you must specify that the input is complex valued. For example:

```
codegen myxcorr -o ComplexXcorr ...
-args {complex(zeros(512,1)),complex(zeros(512,1))}
```

Run the MEX-file at the MATLAB command prompt with complex-valued inputs of the correct size:

```
x_circ = randn(512,1)+1j*randn(512,1); %circular white noise
y_circ = randn(512,1)+1j*randn(512,1); %circular white noise
[C,Lags] = ComplexXcorr(x_circ,y_circ);
```

Attempting to run `ComplexXcorr` with real valued inputs results in the error: `??? MATLAB expression 'x' is not of the correct complexness`.

Inputs must be Constants

For a number of supported Signal Processing Toolbox functions, the inputs or a subset of the inputs must be specified as constants at compilation time. Functions with this behavior are noted in the right column of the table “Supported Functions” on page 9-2. Use `coder.Type` with the `-args` compilation option, or enter the constants directly in the source code.

Specifying inputs as constants at compilation time results in significant advantages in the speed and efficiency of the generated code. For example, storing filter coefficients or

window function values as vectors in the C source code improves performance by avoiding costly computation at runtime. Because a primary purpose of Code Generation from MATLAB is to generate optimized C code for desktop and embedded systems, emphasis is placed on providing the user with computational savings at runtime whenever possible.

To illustrate the constant input requirement with `butter`, create the file `myLowpassFilter.m` in a folder where you have read and write permission. Ensure that this folder is in the MATLAB search path. Copy and paste the following lines of code into `myLowpassFilter.m` and save the file.

```
function output = myLowpassFilter(input,N,Wn) %#codegen
[B,A] = butter(N,Wn,'low');
output = filter(B,A,input);
```

If you have the MATLAB Coder software, enter the following command at the MATLAB command prompt:

```
codegen myLowpassFilter -o myLowpassFilter ...
-args {zeros(512,1),coder.newtype('constant',5),coder.newtype('constant',0.1)} -report
```

Once the program compiles successfully, the following message appears in the command window: `Code generation successful: View report`.

Click on `View report`. Click on the `C code` tab on the top left and open the target source file `myLowpassFilter.c`.

Note that the numerator and denominator filter coefficients are included in the source code.

```
   static real_T dv0[6] = { 5.9795780369978346E-5, 0.00029897890184989173, ...
   static real_T dv1[6] = { 1.0, -3.9845431196123373, 6.4348670902758709, ...
```

Run the MEX-file without entering the constants:

```
output = myLowpassFilter(randn(512,1));
```

If you attempt to run the MEX-file by inputting the constants, you receive the error `??? Error using ==> myLowpassFilter 1 input required for entry-point 'myLowpassFilter'`.

You may also enter the constants in the MATLAB source code directly. Edit the `myLowPassFilter.m` file and replace the MATLAB code with the lines:

```
function output = myLowpassFilter(input) %#codegen
[B,A] = butter(5,0.1,'low');
output = filter(B,A,input);
```

Enter the following command at the MATLAB command prompt:

```
codegen myLowpassFilter -args {zeros(512,1)} -o myLowpassFilter
```

Run the MEX-file by entering the following at the MATLAB command prompt:

```
output = myLowpassFilter(randn(512,1));
```

See “Apply Window to Input Signal” on page 9-10, “Apply Lowpass Filter to Input Signal” on page 9-12, and “Zero Phase Filtering” on page 9-14 for additional examples of the constant input requirement.

Code Generation Examples

In this section...
“Apply Window to Input Signal” on page 9-10
“Apply Lowpass Filter to Input Signal” on page 9-12
“Cross Correlate or Autocorrelate Input Data” on page 9-12
“freqz With No Output Arguments” on page 9-13
“Zero Phase Filtering” on page 9-14

Apply Window to Input Signal

In this example, apply a Hamming window to an input data vector of size `512x1`.

Create a file called `window_data.m` by typing

```
>>edit window_data
```

at the MATLAB command prompt.

Copy and paste the code provided into the editor and save the file.

```
function output_data=window_data(input_data,N) %#codegen
Win=hamming(N);
output_data=input_data.*Win;
```

Use `codegen` to generate a MEX–file `window_data.m`.

```
codegen window_data -args {zeros(512,1),coder.newtype('constant',512)} -o window_data
```

The `-args` option defines the input specifications for the MEX –file. `input_data` is a `512x1` real valued vector. Because the input to `hamming` must be a constant, `coder.newtype` is used to specify the window length. In a conventional MATLAB program, you can read the input data length at runtime and construct a Hamming window of the corresponding length.

Alternatively, edit the code for `window_data.m` as follows:

```
function output_data=window_data(input_data) %#codegen
Win=hamming(512);
output_data=input_data.*Win;
```

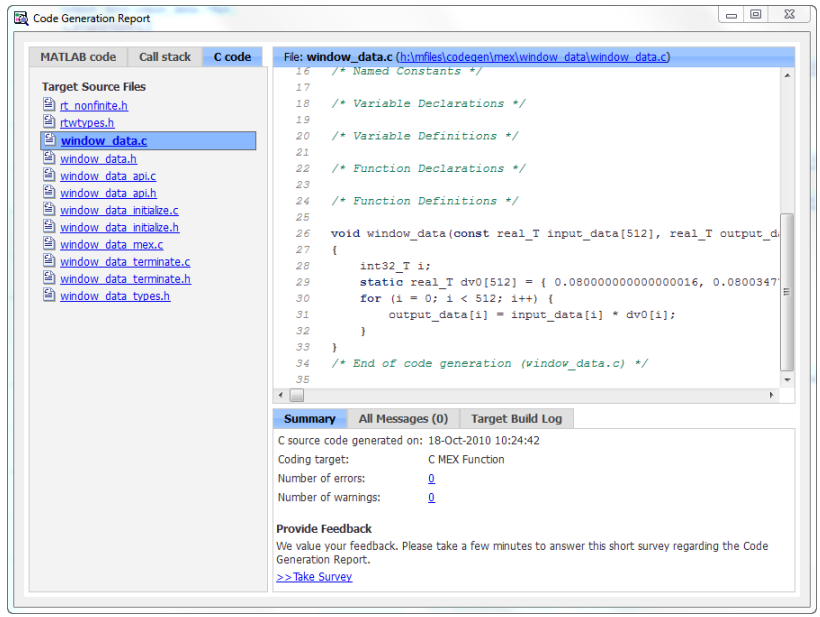
The preceding code specifies the length of the Hamming window in the source code as opposed to using `coder.newtype`. Use `codegen` to generate a MEX–file and C code:

```
codegen window_data -args {zeros(512,1)} -o window_data -report
```

The `-report` flag generates a compilation report. If the `codegen` operation is successful, you obtain: `Code generation successful: View report`.

Click on `View report` to view the Code Generation Report.

Select the `C-code` tab and select `window_data.c` as the `Target Source File`.



Note from the location bar that the C source code is in the `codegen/mex/<FUNCTION_NAME>` folder. Running `codegen` creates this folder and places the C source code, C header files, and MEX files in the folder. Each function that you create produces a `codegen/mex/<FUNCTION_NAME>` folder.

Scroll through the C code to see that the values of the Hamming window are included directly in the C source code.

Run the MEX-file on a white noise input:

```
% Window white noise input
output_data=window_data(randn(512,1));
```

Apply Lowpass Filter to Input Signal

Assuming a sampling frequency of 20 kHz, create a fourth-order Butterworth filter with a 3–dB frequency of 2.5 kHz. Use the Butterworth filter to lowpass filter a 10000x1 input data vector.

Create a file called `ButterFilt.m`. Copy and paste the following code into the file.

```
function output_data=ButterFilt(input_data) %#codegen
[b,a]=butter(4,0.25);
output_data=filter(b,a,input_data);
```

Run the `codegen` command to obtain the C source code `ButterFilt.c` and MEX file:

```
codegen ButterFilt -args {zeros(10000,1)} -o ButterFilt -report
```

The C source code includes the five numerator and denominator coefficients of the fourth-order Butterworth filter as constants.

```
static real_T dv0[5] = { 0.010209480791203124, 0.040837923164812495, 0.061256884747218
static real_T dv1[5] = { 1.0, -1.9684277869385174, 1.7358607092088851, -0.724470829507
```

Apply the filter using the MEX-file:

```
Fs=20000;
%Create 10000x1 input signal
t=0:(1/Fs):0.5-(1/Fs);
input_data=(cos(2*pi*1000*t)+sin(2*pi*500*t)+0.2*randn(size(t)))';
%Filter data
output_data=ButterFilt(input_data);
```

Cross Correlate or Autocorrelate Input Data

Estimate the cross correlation or autocorrelation of two real-valued input vectors to lag 50. Output the estimate at the nonnegative lags.

Create a file called `myxcorr.m`. Copy and paste the following code into the file:

```
function [C,Lags]=myxcorr(x,y) %#codegen
[c,lags]=xcorr(x,y,50,'coeff');
```

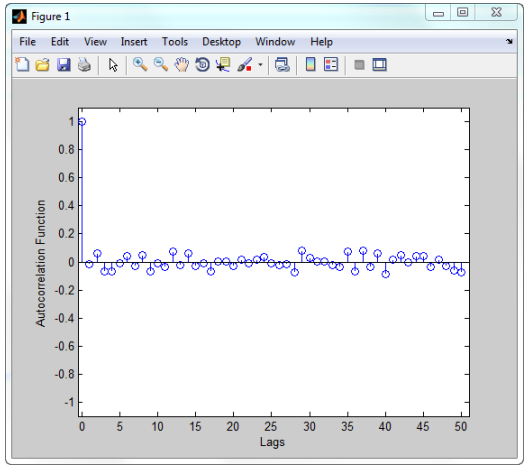
```
C=c(51:end);
Lags=lags(51:end);
```

Run the `codegen` command at the MATLAB command prompt:

```
codegen myxcorr -args {zeros(512,1), zeros(512,1)} -o myxcorr -report
```

Use the MEX-file to compute and plot the autocorrelation of a white noise input:

```
rng(0,'twister')
%White noise input
input_data=randn(512,1);
%Compute autocorrelation with MEX-file
[C,Lags]=myxcorr(input_data,input_data);
% Plot the result
stem(Lags,C); axis([-0.5 51 -1.1 1.1])
xlabel('Lags'); ylabel('Autocorrelation Function');
```



freqz With No Output Arguments

In Code Generation from MATLAB, `freqz` with no output arguments behaves differently than in the standard MATLAB language. In standard MATLAB, `freqz` with no output arguments produces a plot of the magnitude and phase response of the input filter. The plot is produced regardless of whether the call to `freqz` terminates in a semicolon or not. No frequency response or phase vectors are returned.

`freqz` with no output arguments and no terminating semicolon:

```
B = [0.05 0.9 0.05]; %Numerator coefficients
freqz(B,1) %no semicolon. Plot is produced
```

`freqz` with no output arguments and terminating in a semicolon:

```
B = [0.05 0.9 0.05]; %Numerator coefficients
freqz(B,1); %semicolon. Plot is produced
```

The behavior shown in the preceding examples differs from the expected behavior of a MEX-file using `freqz` with code generation support. To illustrate this difference create a program called `myfreqz.m`.

Copy and paste the following code into the file:

```
function myfreqz(B,A) %#codegen
freqz(B,A)
```

Run the following command at the MATLAB command prompt:

```
codegen myfreqz -args {zeros(1,3), zeros(1,1)} -o myfreqz
```

Calling the MEX-file writes a `512x1` complex-valued vector to the workspace and displays the output. The vector is the frequency response. No plot is produced.

```
myfreqz([0.05 0.9 0.05],1);
```

Change the code in `myfreqz.m` by adding a terminating semicolon:

```
function myfreqz(B,A) %#codegen
freqz(B,A);
```

Run the following command at the MATLAB command prompt:

```
codegen myfreqz -args {zeros(1,3), zeros(1,1)} -o myfreqz
```

Calling the MEX-file produces a plot of the magnitude and phase response of the filter. The output of the complex-valued frequency response is suppressed.

```
myfreqz([0.05 0.9 0.05],1);
```

Zero Phase Filtering

Design a lowpass Butterworth filter with a 1 kHz 3–dB frequency to implement zero phase filtering on data with a sampling frequency of 20 kHz.

```
[B,A] = butter(20,0.314,'low');
```

Create the program `myZerophaseFilt.m`.

```
function output  = myZerophaseFilt(input) %#codegen
B=1e-3 *[
    0.0000
    0.0001
    0.0010
    0.0060
    0.0254
    0.0814
    0.2035
    0.4071
    0.6615
    0.8820
    0.9702
    0.8820
    0.6615
    0.4071
    0.2035
    0.0814
    0.0254
    0.0060
    0.0010
    0.0001
    0.0000];
A=[1.0000
   -7.4340
   28.2476
  -71.6333
  134.6222
 -197.9575
  235.1628
 -230.2286
  188.0901
 -129.1746
   74.8284
  -36.5623
   15.0197
   -5.1525
    1.4599
   -0.3361
    0.0613
   -0.0085
```

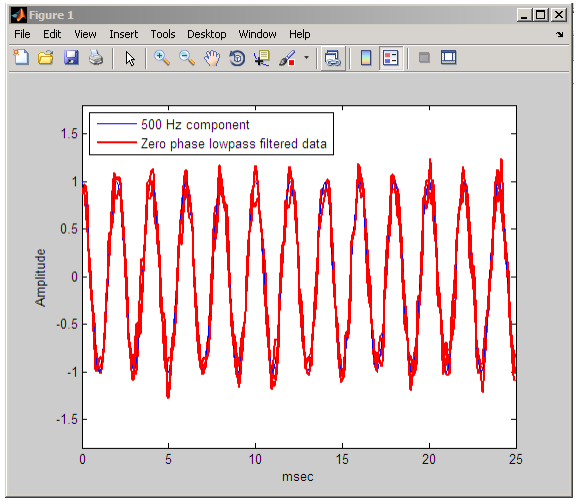
```
    0.0009
   -0.0001
    0.0000];
output = filtfilt(B,A,input);
```

Run the following command at the MATLAB command prompt:

```
codegen myZerophaseFilt -args {zeros(1,20001)} -o myZerophaseFilt
```

Filter input data with `myZerophaseFilt`:

```
Fs = 20000;
t = 0:(1/Fs):1;
Comp500Hz = cos(2*pi*500*t);
Signal = Comp500Hz+sin(2*pi*4000*t)+0.2*randn(size(t));
FilteredData = myZerophaseFilt(Signal);
plot(t(1:500).*1000,Comp500Hz(1:500));
xlabel('msec'); ylabel('Amplitude');
axis([0 25 -1.8 1.8]); hold on;
plot(t(1:500).*1000,FilteredData(1:500),'r','linewidth',2);
legend('500 Hz component','Zero phase lowpass filtered data',...
'Location','NorthWest');
```



Convolution and Correlation

- “Linear and Circular Convolution” on page 10-2
- “Confidence Intervals for Sample Autocorrelation” on page 10-5
- “Residual Analysis with Autocorrelation” on page 10-7
- “Autocorrelation of Moving Average Process” on page 10-17
- “Cross-Correlation of Two Moving Average Processes” on page 10-21
- “Cross-Correlation of Delayed Signal in Noise” on page 10-23
- “Cross-Correlation of Phase-Lagged Sine Wave” on page 10-26

Linear and Circular Convolution

This example shows how to establish an equivalence between linear and circular convolution.

Linear and circular convolution are fundamentally different operations. However, there are conditions under which linear and circular convolution are equivalent. Establishing this equivalence has important implications. For two vectors, `x` and `y`, the circular convolution is equal to the inverse discrete Fourier transform (DFT) of the product of the vectors' DFTs. Knowing the conditions under which linear and circular convolution are equivalent allows you to use the DFT to efficiently compute linear convolutions.

The linear convolution of an N-point vector, `x`, and an L-point vector, `y`, has length $N + L - 1$.

For the circular convolution of x and y to be equivalent, you must pad the vectors with zeros to length at least $N + L - 1$ before you take the DFT. After you invert the product of the DFTs, retain only the first $N + L - 1$ elements.

Create two vectors, `x` and `y`, and compute the linear convolution of the two vectors.

```
x = [2 1 2 1];
y = [1 2 3];
clin = conv(x,y);
```

The output has length 4+3-1.

Pad both vectors with zeros to length 4+3-1. Obtain the DFT of both vectors, multiply the DFTs, and obtain the inverse DFT of the product.

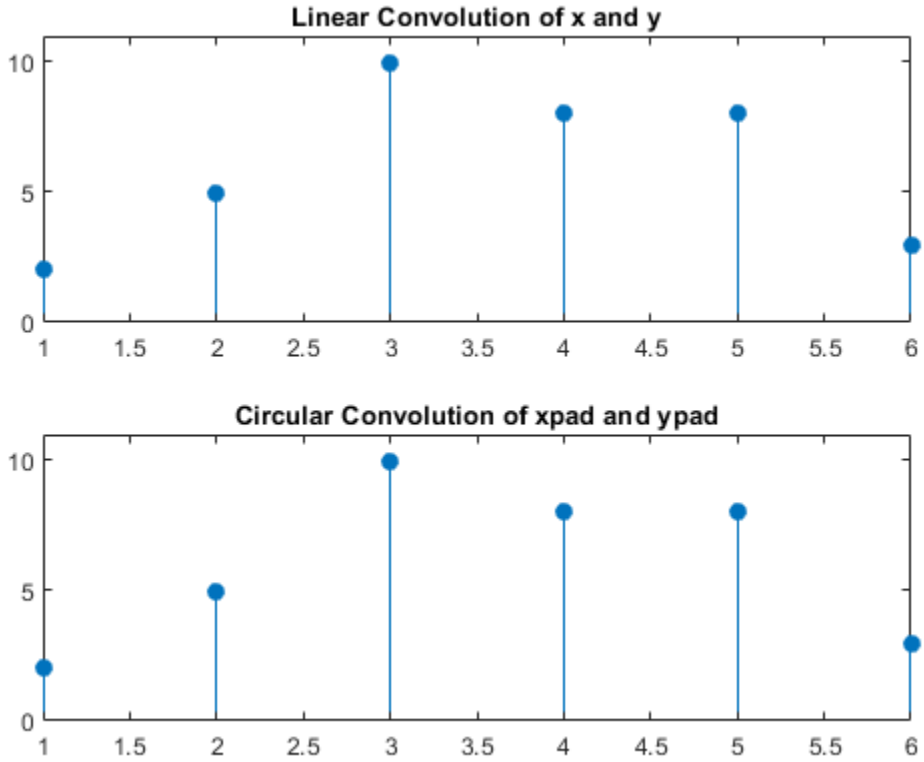
```
xpad = [x zeros(1,6-length(x))];
ypad = [y zeros(1,6-length(y))];
ccirc = ifft(fft(xpad).*fft(ypad));
```

The circular convolution of the zero-padded vectors, `xpad` and `ypad`, is equivalent to the linear convolution of `x` and `y`. You retain all the elements of `ccirc` because the output has length 4+3-1.

Plot the output of linear convolution and the inverse of the DFT product to show the equivalence.

```
subplot(2,1,1)
stem(clin,'filled')
```

```
ylim([0 11])
title('Linear Convolution of x and y')

subplot(2,1,2)
stem(ccirc,'filled')
ylim([0 11])
title('Circular Convolution of xpad and ypad')
```



Pad the vectors to length 12 and obtain the circular convolution using the inverse DFT of the product of the DFTs. Retain only the first 4+3-1 elements to produce an equivalent result to linear convolution.

```
N = length(x)+length(y)-1;
xpad = [x zeros(1,12-length(x))];
```

```
ypad = [y zeros(1,12-length(y))];
ccirc = ifft(fft(xpad).*fft(ypad));
ccirc = ccirc(1:N);
```

The Signal Processing Toolbox™ software has a function, `cconv`, that returns the circular convolution of two vectors. You can obtain the linear convolution of `x` and `y` using circular convolution with the following code.

```
ccirc2 = cconv(x,y,6);
```

`cconv` internally uses the same DFT-based procedure illustrated in the previous example.

Confidence Intervals for Sample Autocorrelation

This example shows how to create confidence intervals for the autocorrelation sequence of a white noise process. Create a realization of a white noise process with length $L = 1000$ samples. Compute the sample autocorrelation to lag 20. Plot the sample autocorrelation along with the approximate 95%-confidence intervals for a white noise process.

Create the white noise random vector. Set the random number generator to the default settings for reproducible results. Obtain the normalized sampled autocorrelation to lag 20.

```
rng default
L = 1000;
x = randn(L,1);
[xc,lags] = xcorr(x,20,'coeff');
```

Create the lower and upper 95% confidence bounds for the normal distribution $N(0, 1/L)$, whose standard deviation is $1/\sqrt{L}$. For a 95%-confidence interval, the critical value is $\sqrt{2}\,\mathrm{erf}^{-1}(0.95) \approx 1.96$ and the confidence interval is

$$\Delta = 0 \pm \frac{1.96}{\sqrt{L}}.$$

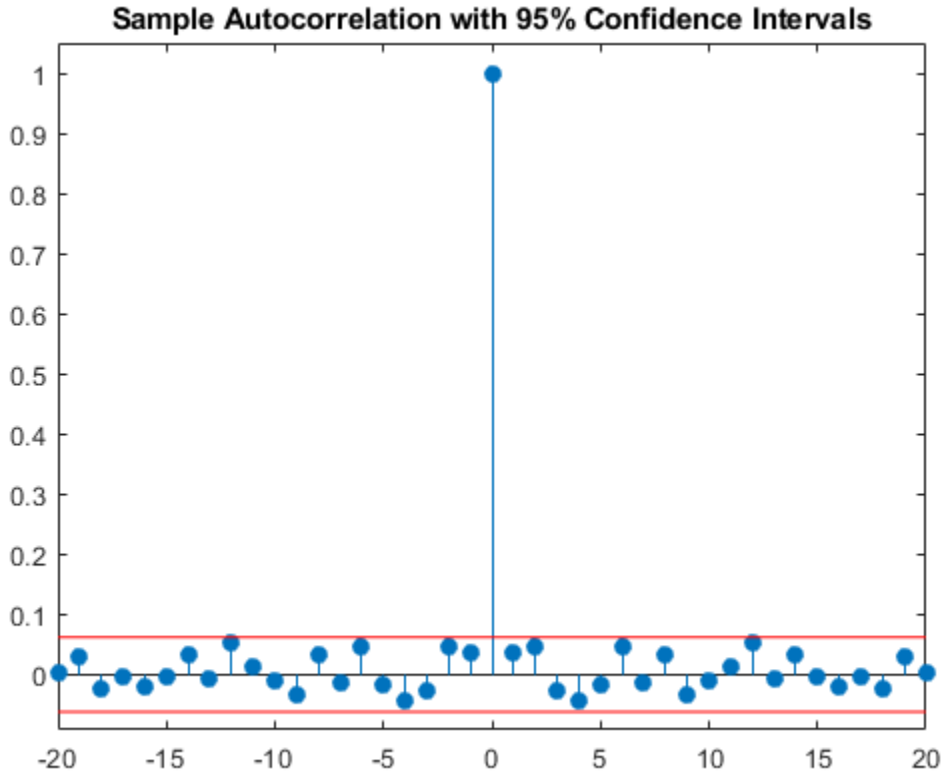
```
vcrit = sqrt(2)*erfinv(0.95)
```

```
vcrit = 1.9600
```

```
lconf = -vcrit/sqrt(L);
upconf = vcrit/sqrt(L);
```

Plot the sample autocorrelation along with the 95%-confidence interval.

```
stem(lags,xc,'filled')
hold on
plot(lags,[lconf;upconf]*ones(size(lags)),'r')
hold off
ylim([lconf-0.03 1.05])
title('Sample Autocorrelation with 95% Confidence Intervals')
```



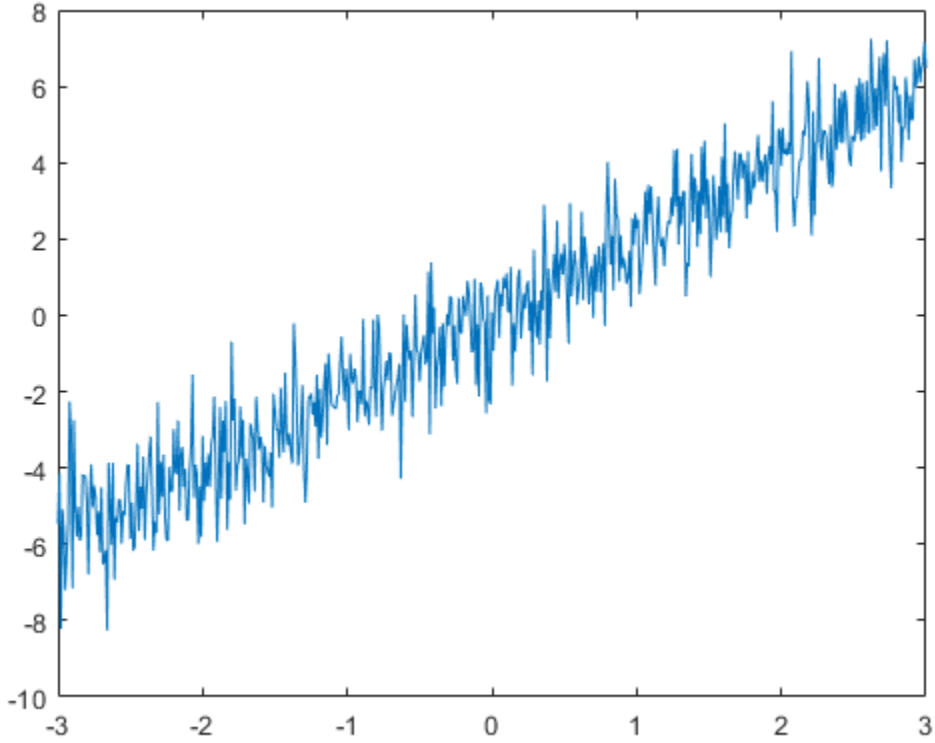
You see in the above figure that the only autocorrelation value outside of the 95%-confidence interval occurs at lag 0 as expected for a white noise process. Based on this result, you can conclude that the data are a realization of a white noise process.

Residual Analysis with Autocorrelation

This example shows how to use autocorrelation with a confidence interval to analyze the residuals of a least-squares fit to noisy data. The residuals are the differences between the fitted model and the data. In a signal-plus-white noise model, if you have a good fit for the signal, the residuals should be white noise.

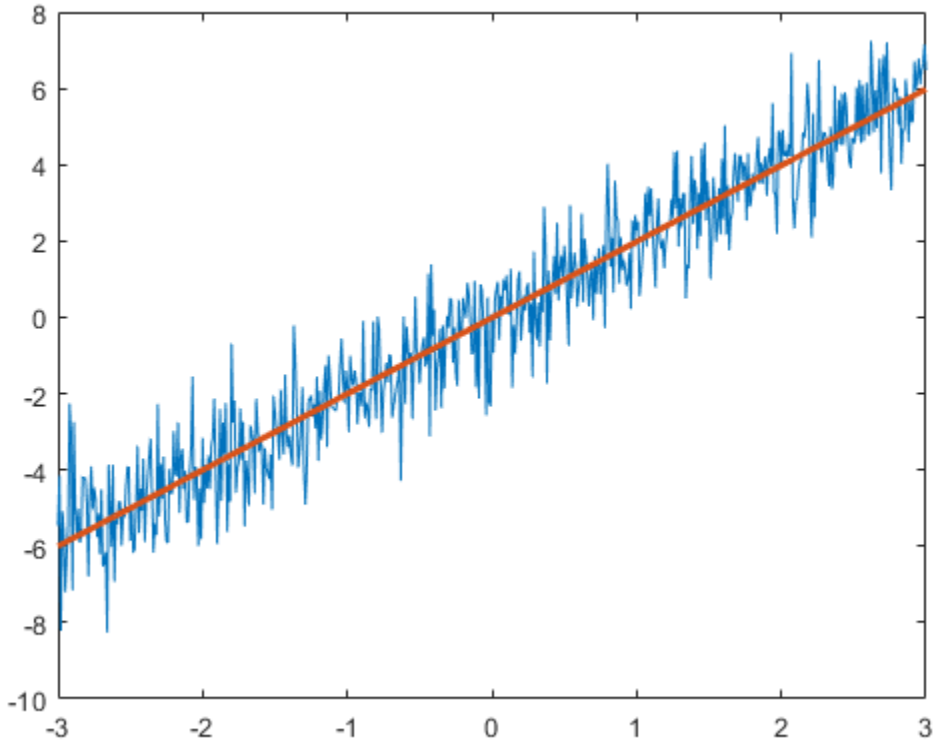
Create a noisy data set consisting of a 1st-order polynomial (straight line) in additive white Gaussian noise. The additive noise is a sequence of uncorrelated random variables following a $N(0,1)$ distribution. This means that all the random variables have mean zero and unit variance. Set the random number generator to the default settings for reproducible results.

```
x = -3:0.01:3;
rng default
y = 2*x+randn(size(x));
plot(x,y)
```



Use `polyfit` to find the least-squares line for the noisy data. Plot the original data along with the least-squares fit.

```
coeffs = polyfit(x,y,1);
yfit = coeffs(2)+coeffs(1)*x;

plot(x,y)
hold on
plot(x,yfit,'linewidth',2)
```



Find the residuals. Obtain the autocorrelation sequence of the residuals to lag 50.

```
residuals = y - yfit;
[xc,lags] = xcorr(residuals,50,'coeff');
```

When you inspect the autocorrelation sequence, you want to determine whether or not there is evidence of autocorrelation. In other words, you want to determine whether the sample autocorrelation sequence looks like the autocorrelation sequence of white noise. If the autocorrelation sequence of the residuals looks like the autocorrelation of a white noise process, you are confident that none of the signal has escaped your fit and ended up in the residuals. In this example, use a 99%-confidence interval. To construct the confidence interval, you need to know the distribution of the sample autocorrelation values. You also need to find the critical values on the appropriate distribution between

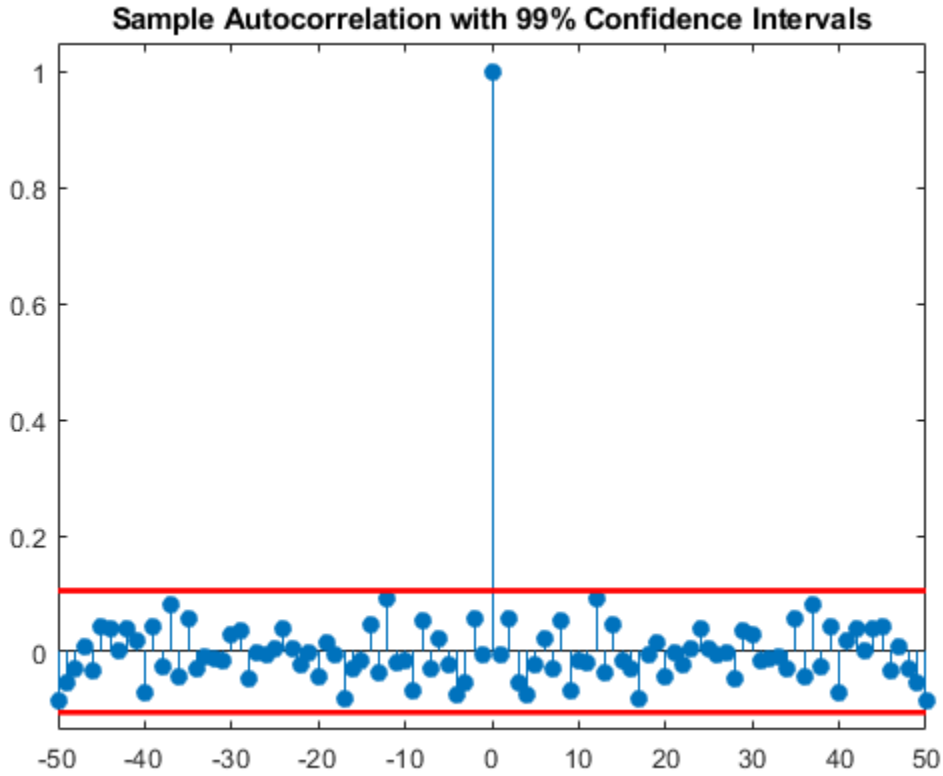
which lie 0.99 of the probability. Because the distribution in this case is Gaussian, you can use complementary inverse error function, `erfcinv`. The relationship between this function and the inverse of the Gaussian cumulative distribution function is described on the reference page for `erfcinv`.

Find the critical value for the 99%-confidence interval. Use the critical value to construct the lower and upper confidence bounds.

```
conf99 = sqrt(2)*erfcinv(2*.01/2);
lconf = -conf99/sqrt(length(x));
upconf = conf99/sqrt(length(x));
```

Plot the autocorrelation sequence along with the 99%-confidence intervals.

```
figure

stem(lags,xc,'filled')
ylim([lconf-0.03 1.05])
hold on
plot(lags,lconf*ones(size(lags)),'r','linewidth',2)
plot(lags,upconf*ones(size(lags)),'r','linewidth',2)
title('Sample Autocorrelation with 99% Confidence Intervals')
```



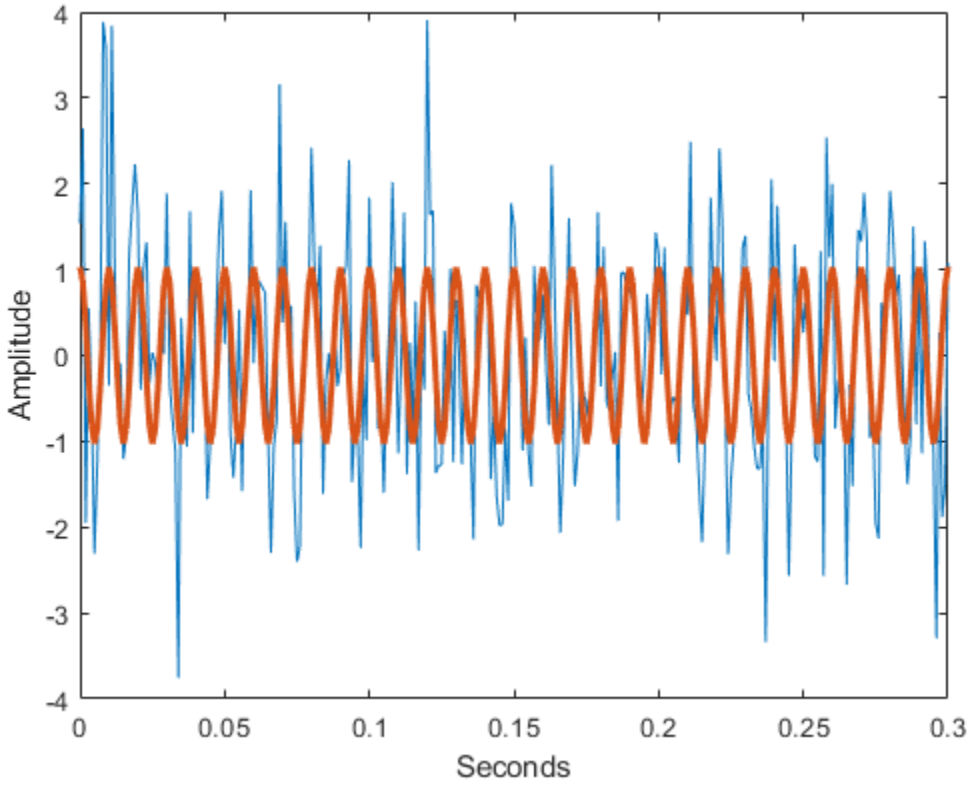
Except at zero lag, the sample autocorrelation values lie within the 99%-confidence bounds for the autocorrelation of a white noise sequence. From this, you can conclude that the residuals are white noise. More specifically, you cannot reject that the residuals are a realization of a white noise process.

Create a signal consisting of a sine wave plus noise. The data are sampled at 1 kHz. The frequency of the sine wave is 100 Hz. Set the random number generator to the default settings for reproducible results.

```
Fs = 1000;
t = 0:1/Fs:1-1/Fs;
rng default
x = cos(2*pi*100*t)+randn(size(t));
```

Use the discrete Fourier transform (DFT) to obtain the least-squares fit to the sine wave at 100 Hz. The least-squares estimate of the amplitude is 2 / *N* times the DFT coefficient corresponding to 100 Hz, where *N* is the length of the signal. The real part is the amplitude of a cosine at 100 Hz and the imaginary part is the amplitude of a sine at 100 Hz. The least-squares fit is the sum of the cosine and sine with the correct amplitude. In this example, DFT bin 101 corresponds to 100 Hz.

```
xdft = fft(x);
ampest = 2/length(x)*xdft(101);
xfit = real(ampest)*cos(2*pi*100*t)+imag(ampest)*sin(2*pi*100*t);

figure

plot(t,x)
hold on
plot(t,xfit,'linewidth',2)
axis([0 0.30 -4 4])
xlabel('Seconds')
ylabel('Amplitude')
```

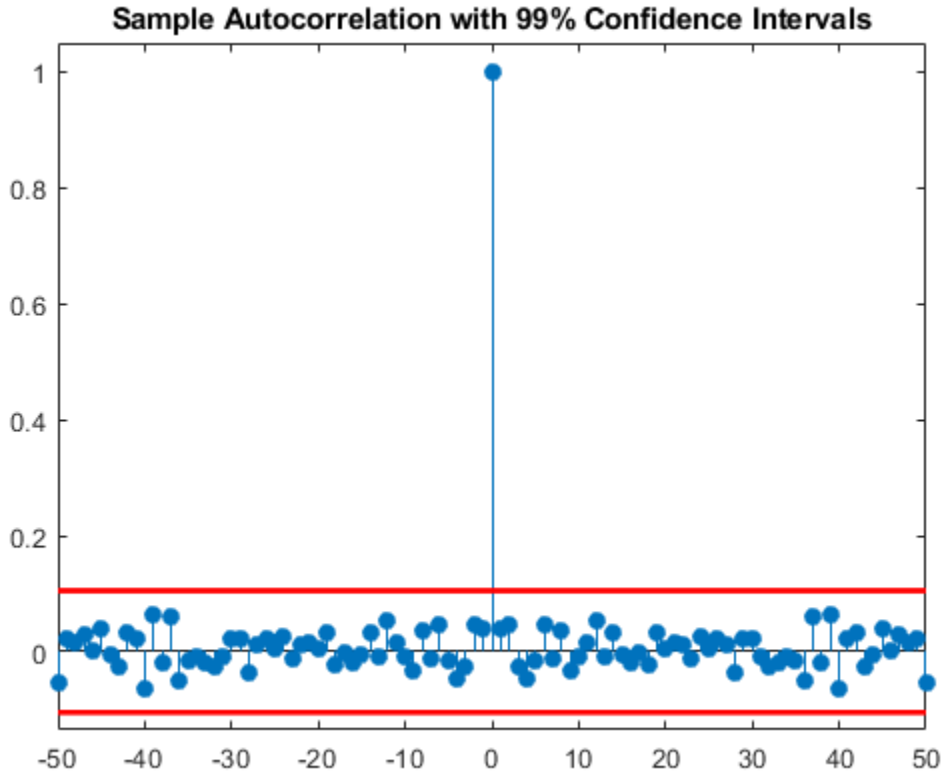


Find the residuals and determine the sample autocorrelation sequence to lag 50.

```
residuals = x-xfit;
[xc,lags] = xcorr(residuals,50,'coeff');
```

Plot the autocorrelation sequence with the 99%-confidence intervals.

```
figure

stem(lags,xc,'filled')
ylim([lconf-0.03 1.05])
hold on
plot(lags,lconf*ones(size(lags)),'r','linewidth',2)
plot(lags,upconf*ones(size(lags)),'r','linewidth',2)
title('Sample Autocorrelation with 99% Confidence Intervals')
```



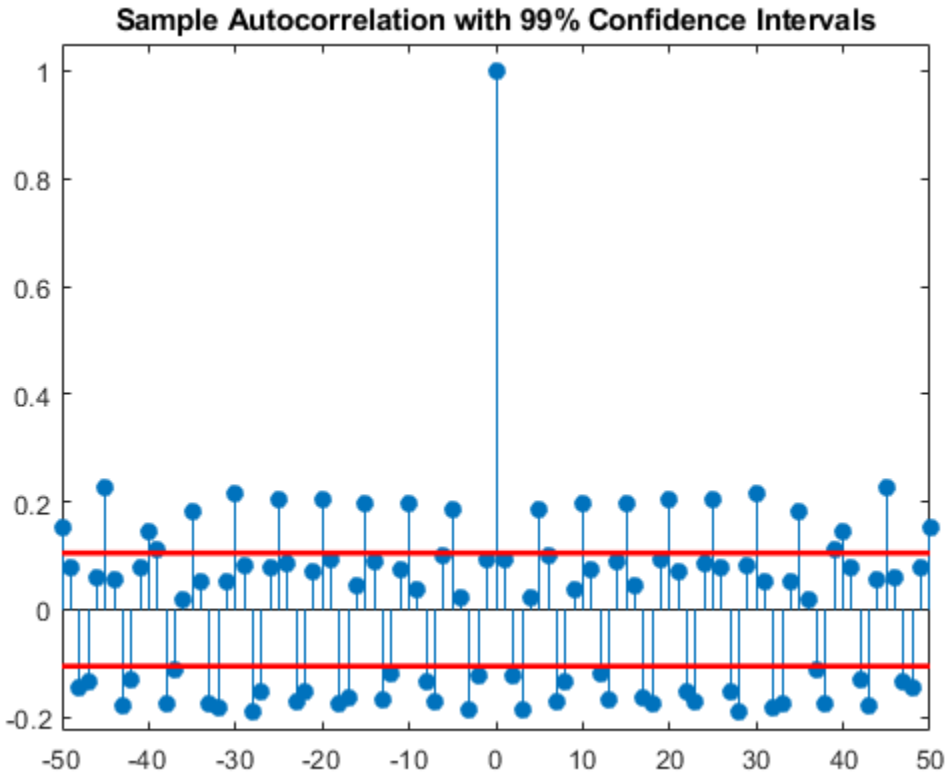
Again, you see that except at zero lag, the sample autocorrelation values lie within the 99%-confidence bounds for the autocorrelation of a white noise sequence. From this, you can conclude that the residuals are white noise. More specifically, you cannot reject that the residuals are a realization of a white noise process.

Finally, add another sine wave with a frequency of 200 Hz and an amplitude of 3/4. Fit only the sine wave at 100 Hz and find the sample autocorrelation of the residuals.

```
x = x+3/4*sin(2*pi*200*t);
xdft = fft(x);
ampest = 2/length(x)*xdft(101);
xfit = real(ampest)*cos(2*pi*100*t)+imag(ampest)*sin(2*pi*100*t);
residuals = x-xfit;
[xc,lags] = xcorr(residuals,50,'coeff');
```

Plot the sample autocorrelation along with the 99%-confidence intervals.

```
figure

stem(lags,xc,'filled')
ylim([lconf-0.12 1.05])
hold on
plot(lags,lconf*ones(size(lags)),'r','linewidth',2)
plot(lags,upconf*ones(size(lags)),'r','linewidth',2)
title('Sample Autocorrelation with 99% Confidence Intervals')
```



In this case, the autocorrelation values clearly exceed the 99%-confidence bounds for a white noise autocorrelation at many lags. Here you can reject the hypothesis that the

residuals are a white noise sequence. The implication is that the model has not accounted for all the signal and therefore the residuals consist of signal plus noise.

Autocorrelation of Moving Average Process

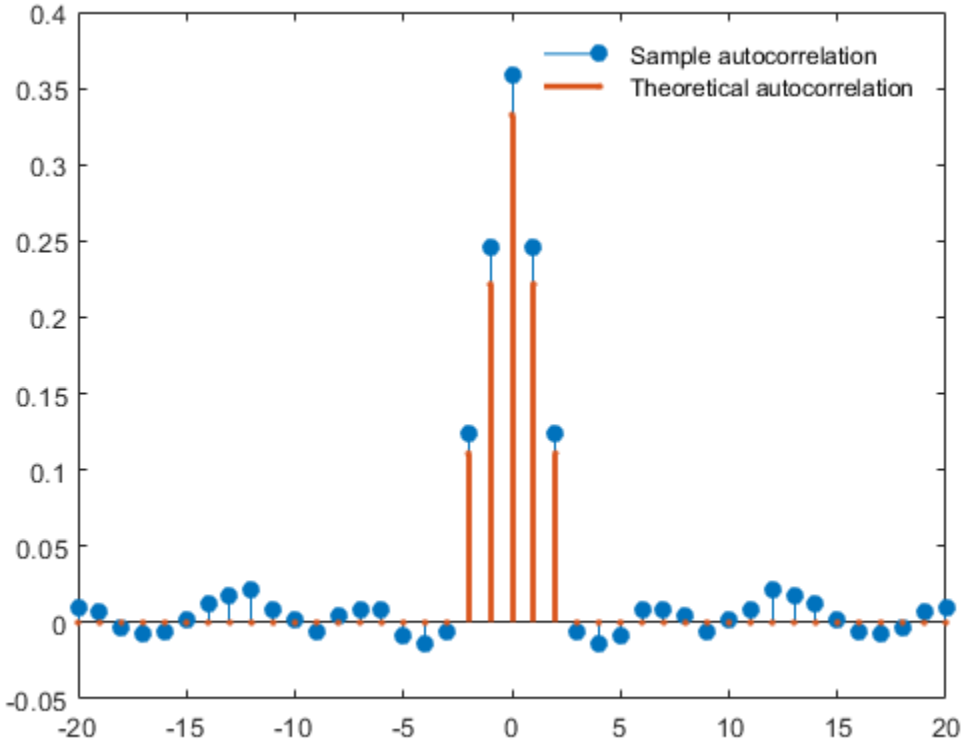
This example shows how to introduce autocorrelation into a white noise process by filtering. When we introduce autocorrelation into a random signal, we manipulate its frequency content. A moving average filter attenuates the high-frequency components of the signal, effectively smoothing it.

Create the impulse response for a 3-point moving average filter. Filter an $N(0,1)$ white noise sequence with the filter. Set the random number generator to the default settings for reproducible results.

```
h = 1/3*ones(3,1);
rng default
x = randn(1000,1);
y = filter(h,1,x);
```

Obtain the biased sample autocorrelation out to 20 lags. Plot the sample autocorrelation along with the theoretical autocorrelation.

```
[xc,lags] = xcorr(y,20,'biased');

Xc = zeros(size(xc));
Xc(19:23) = [1 2 3 2 1]/9*var(x);

stem(lags,xc,'filled')
hold on
stem(lags,Xc,'.','linewidth',2)

lg = legend('Sample autocorrelation','Theoretical autocorrelation');
lg.Location = 'NorthEast';
lg.Box = 'off';
```



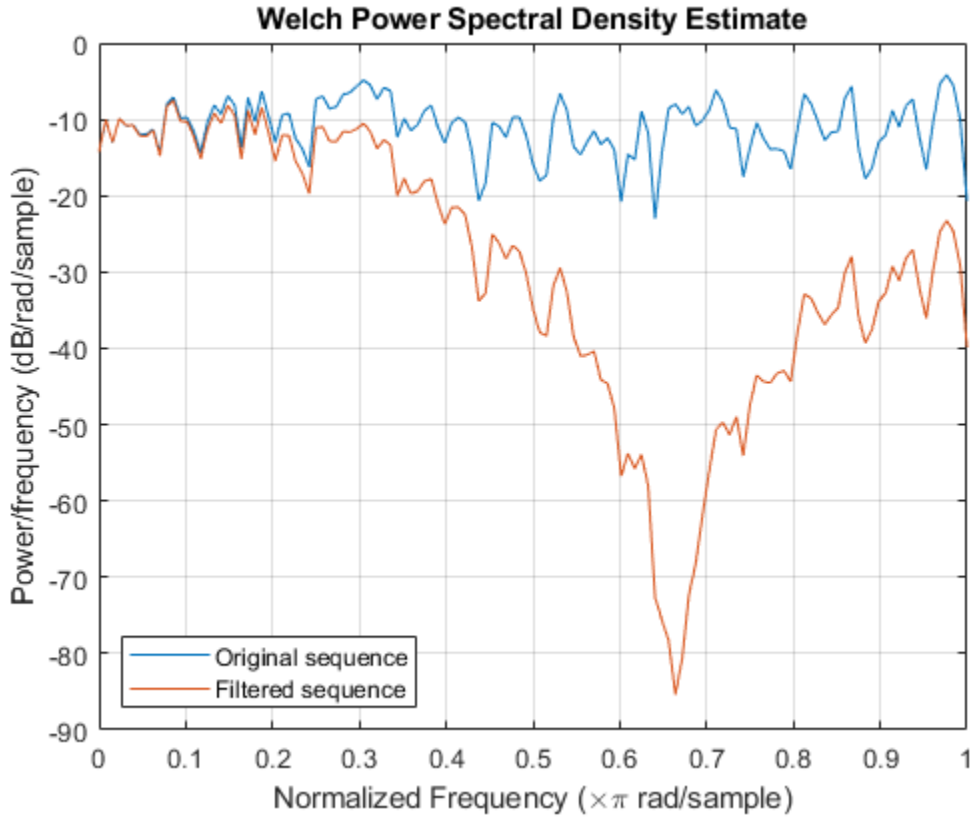
The sample autocorrelation captures the general form of the theoretical autocorrelation, even though the two sequences do not agree in detail.

In this case, it is clear that the filter has introduced significant autocorrelation only over lags [-2,2]. The absolute value of the sequence decays quickly to zero outside of that range.

To see that the frequency content has been affected, plot Welch estimates of the power spectral densities of the original and filtered signals.

```
[pxx,wx] = pwelch(x);
[pyy,wy] = pwelch(y);

figure
```

```
plot(wx/pi,20*log10(pxx),wy/pi,20*log10(pyy))

lg = legend('Original sequence','Filtered sequence');
lg.Location = 'SouthWest';

xlabel('Normalized Frequency (\times\pi rad/sample)')
ylabel('Power/frequency (dB/rad/sample)')
title('Welch Power Spectral Density Estimate')
grid
```



The white noise has been "colored" by the moving average filter.

See Also

External Websites

- Ellis, Dan. *About Colored Noise*. https://www.ee.columbia.edu/~dpwe/noise/

Cross-Correlation of Two Moving Average Processes

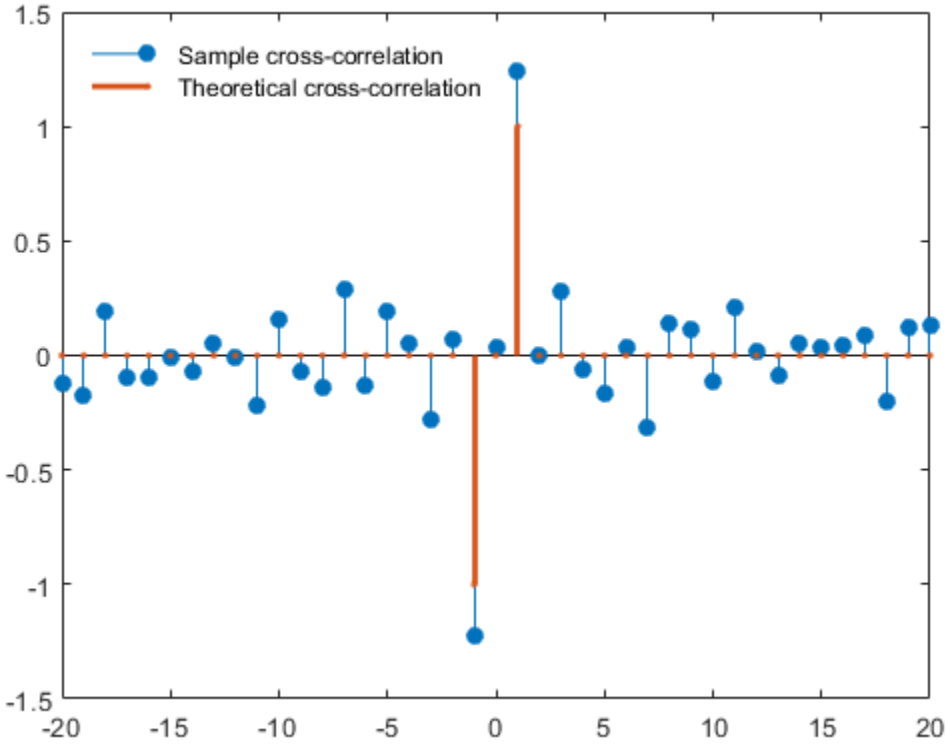
This example shows how to find and plot the cross-correlation sequence between two moving average processes. The example compares the sample cross-correlation with the theoretical cross-correlation. Filter an $N(0,1)$ white noise input with two different moving average filters. Plot the sample and theoretical cross-correlation sequences.

Create an $N(0,1)$ white noise sequence. Set the random number generator to the default settings for reproducible results. Create two moving average filters. One filter has impulse response $\delta(n)+\delta(n-1)$. The other filter has impulse response $\delta(n)-\delta(n-1)$.

```
rng default

w = randn(100,1);
x = filter([1 1],1,w);
y = filter([1 -1],1,w);
```

Obtain the sample cross-correlation sequence up to lag 20. Plot the sample cross-correlation along with the theoretical cross-correlation.

```
[xc,lags] = xcorr(x,y,20,'biased');

Xc = zeros(size(xc));
Xc(20) = -1;
Xc(22) = 1;

stem(lags,xc,'filled')
hold on
stem(lags,Xc,'.','linewidth',2)

q = legend('Sample cross-correlation','Theoretical cross-correlation');
q.Location = 'NorthWest';
q.FontSize = 9;
q.Box = 'off';
```



The theoretical cross-correlation is -1 at lag -1, 1 at lag 1, and zero at all other lags. The sample cross-correlation sequence approximates the theoretical cross-correlation.

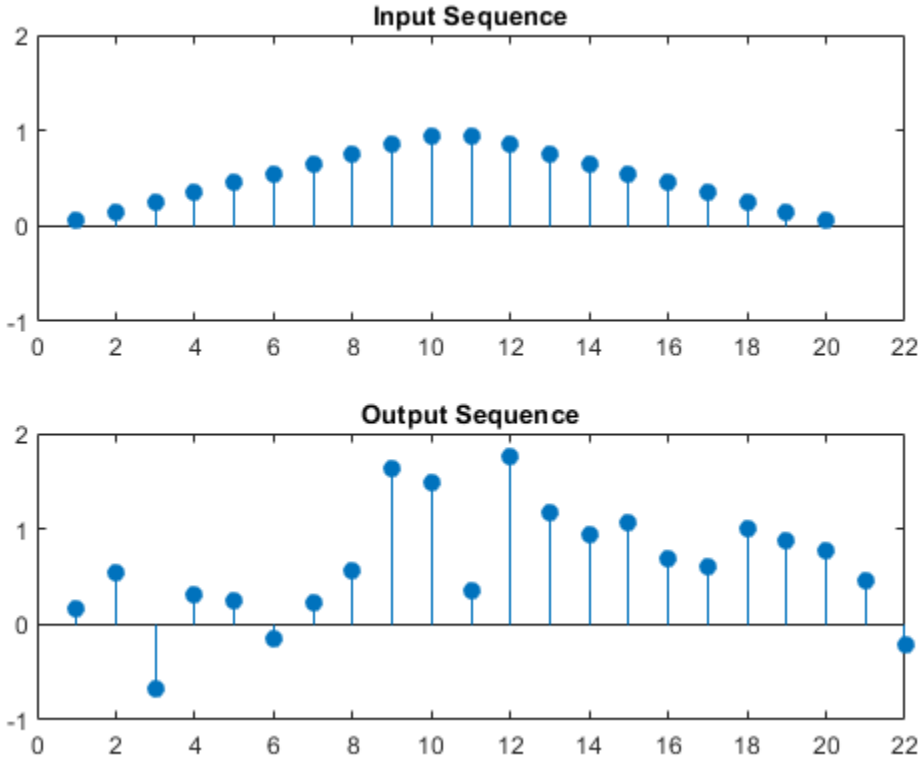
As expected, there is not perfect agreement between the theoretical cross-correlation and sample cross-correlation. The sample cross-correlation does accurately represent both the sign and magnitude of the theoretical cross-correlation sequence values at lag -1 and lag 1.

Cross-Correlation of Delayed Signal in Noise

This example shows how to use the cross-correlation sequence to detect the time delay in a noise-corrupted sequence. The output sequence is a delayed version of the input sequence with additive white Gaussian noise. Create two sequences. One sequence is a delayed version of the other. The delay is 3 samples. Add $N(0,0.3^2)$ white noise to the delayed signal. Use the sample cross-correlation sequence to detect the lag.

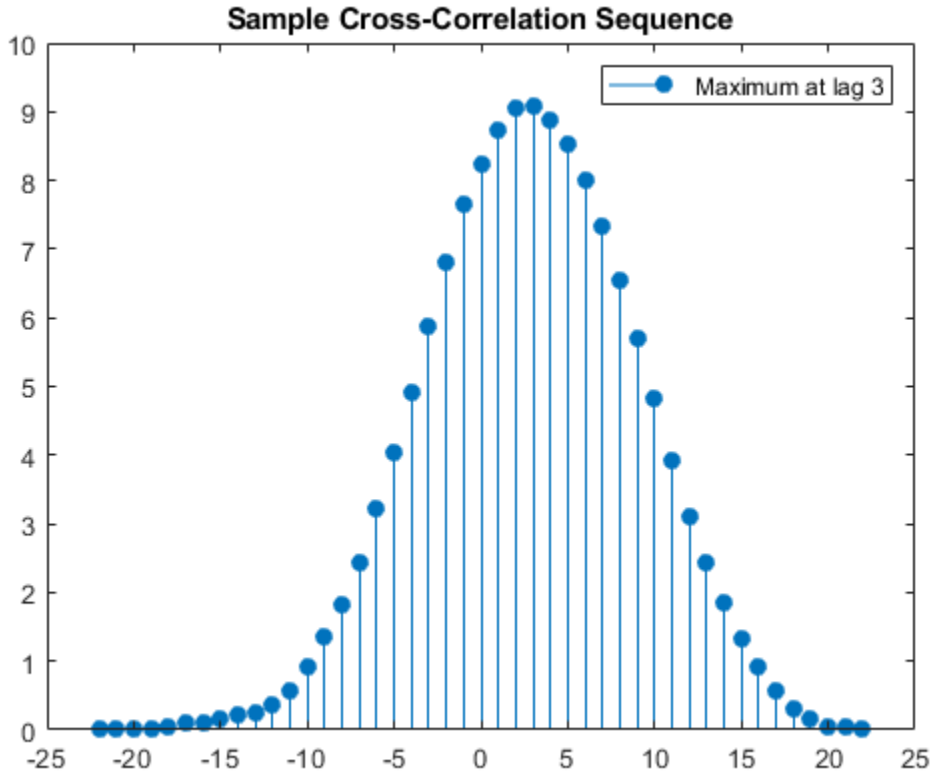
Create and plot the signals. Set the random number generator to the default settings for reproducible results.

```
rng default

x = triang(20);
y = [zeros(3,1);x]+0.3*randn(length(x)+3,1);

subplot(2,1,1)
stem(x,'filled')
axis([0 22 -1 2])
title('Input Sequence')

subplot(2,1,2)
stem(y,'filled')
axis([0 22 -1 2])
title('Output Sequence')
```



Obtain the sample cross-correlation sequence and use the maximum absolute value to estimate the lag. Plot the sample cross-correlation sequence.

```
[xc,lags] = xcorr(y,x);
[~,I] = max(abs(xc));

figure
stem(lags,xc,'filled')
legend(sprintf('Maximum at lag %d',lags(I)))
title('Sample Cross-Correlation Sequence')
```



The maximum cross correlation sequence value occurs at lag 3 as expected.

Cross-Correlation of Phase-Lagged Sine Wave

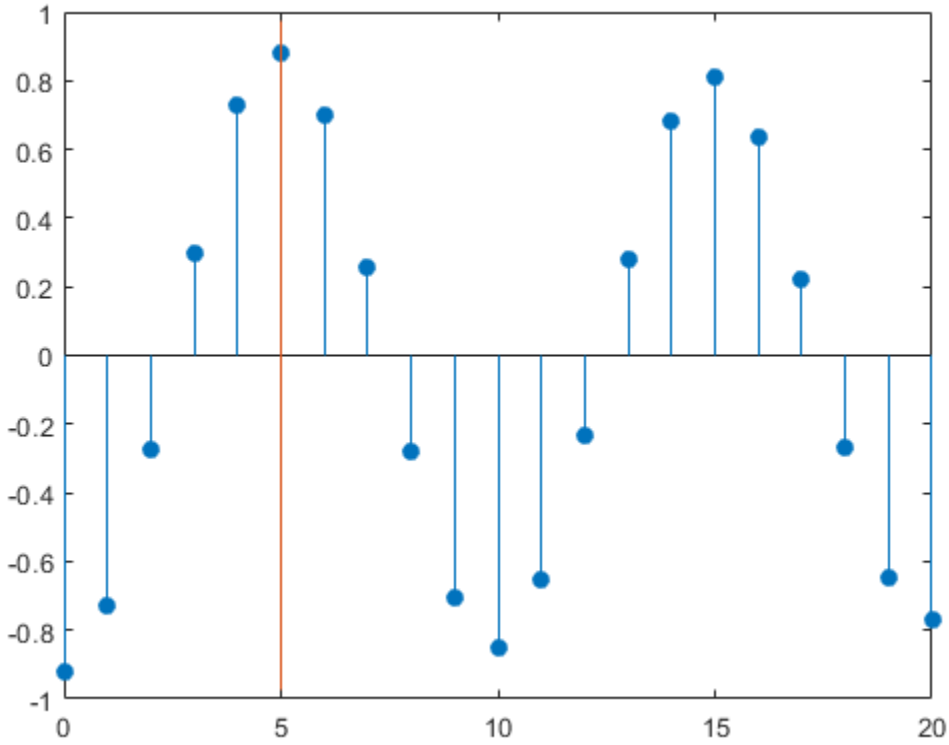
This example shows how to use the cross-correlation sequence to estimate the phase lag between two sine waves. The theoretical cross-correlation sequence of two sine waves at the same frequency also oscillates at that frequency. Because the sample cross-correlation sequence uses fewer and fewer samples at larger lags, the sample cross-correlation sequence also oscillates at the same frequency, but the amplitude decays as the lag increases.

Create two sine waves with frequencies of $2\pi/10$ rad/sample. The starting phase of one sine wave is 0, while the starting phase of the other sine wave is $-\pi$ radians. Add $N(0,0.25^2)$ white noise to the sine wave with the phase lag of π radians. Set the random number generator to the default settings for reproducible results.

```
rng default

t = 0:99;
x = cos(2*pi*1/10*t);
y = cos(2*pi*1/10*t-pi)+0.25*randn(size(t));
```

Obtain the sample cross-correlation sequence for two periods of the sine wave (10 samples). Plot the cross-correlation sequence and mark the known lag between the two sine waves (5 samples).

```
[xc,lags] = xcorr(y,x,20,'coeff');

stem(lags(21:end),xc(21:end),'filled')

hold on
plot([5 5],[-1 1])

ax = gca;
ax.XTick = 0:5:20;
```



You see that the cross-correlation sequence peaks at lag 5 as expected and oscillates with a period of 10 samples.

Multirate Signal Processing

- “Downsampling — Signal Phases” on page 11-2
- “Downsampling — Aliasing” on page 11-6
- “Filtering Before Downsampling” on page 11-13
- “Upsampling — Imaging Artifacts” on page 11-16
- “Filtering After Upsampling — Interpolation” on page 11-19
- “Simulate a Sample-and-Hold System” on page 11-22
- “Changing Signal Sample Rate” on page 11-28

Downsampling — Signal Phases

This example shows how to use `downsample` to obtain the *phases* of a signal. Downsampling a signal by M can produce M unique phases. For example, if you have a discrete-time signal, x, with $x(0)$ $x(1)$ $x(2)$ $x(3)$, ..., the M phases of x are $x(nM + k)$ with $k = 0,1, ..., M-1$.

The M signals are referred to as the *polyphase* components of x.

Create a white noise vector and obtain the 3 polyphase components associated with downsampling by 3.

Reset the random number generator to the default settings to produce a repeatable result. Generate a white noise random vector and obtain the 3 polyphase components associated with downsampling by 3.

```
rng default
x = randn(36,1);
x0 = downsample(x,3,0);
x1 = downsample(x,3,1);
x2 = downsample(x,3,2);
```

The polyphase components have length equal to 1/3 the original signal.

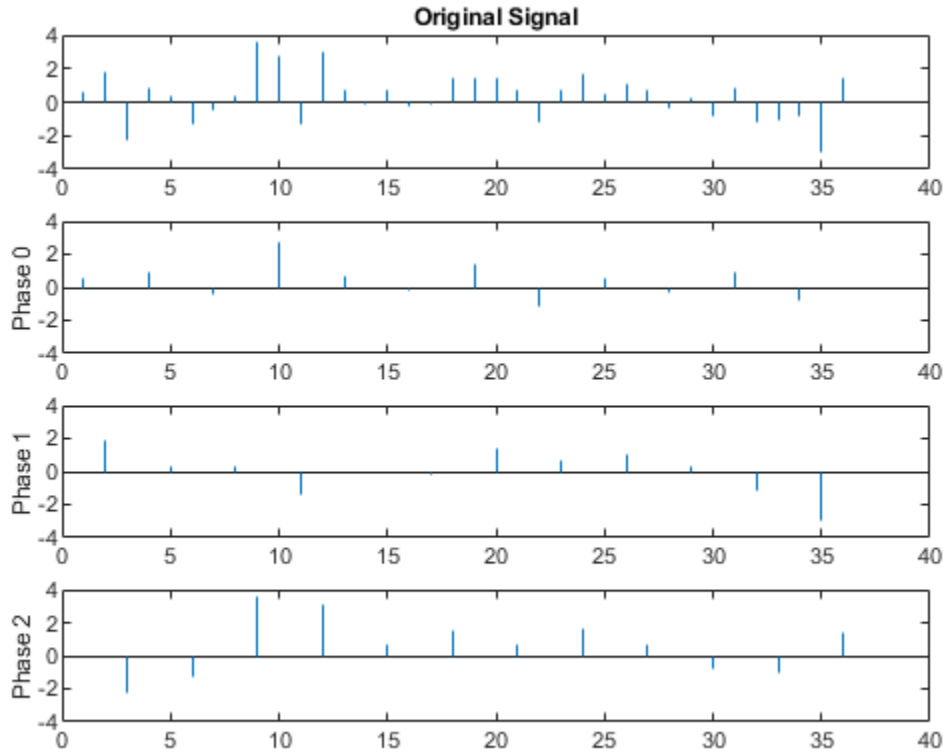
Upsample the polyphase components by 3 using `upsample`.

```
y0 = upsample(x0,3,0);
y1 = upsample(x1,3,1);
y2 = upsample(x2,3,2);
```

Plot the result.

```
subplot(4,1,1)
stem(x,'Marker','none')
title('Original Signal')
ylim([-4 4])

subplot(4,1,2)
stem(y0,'Marker','none')
ylabel('Phase 0')
ylim([-4 4])

subplot(4,1,3)
```

```
stem(y1,'Marker','none')
ylabel('Phase 1')
ylim([-4 4])

subplot(4,1,4)
stem(y2,'Marker','none')
ylabel('Phase 2')
ylim([-4 4])
```



If you sum the upsampled polyphase components you obtain the original signal.

Create a discrete-time sinusoid and obtain the 2 polyphase components associated with downsampling by 2.

Create a discrete-time sine wave with an angular frequency of $\pi/4$ rad/sample. Add a DC offset of 2 to the sine wave to help with visualization of the polyphase components. Downsample the sine wave by 2 to obtain the even and odd polyphase components.

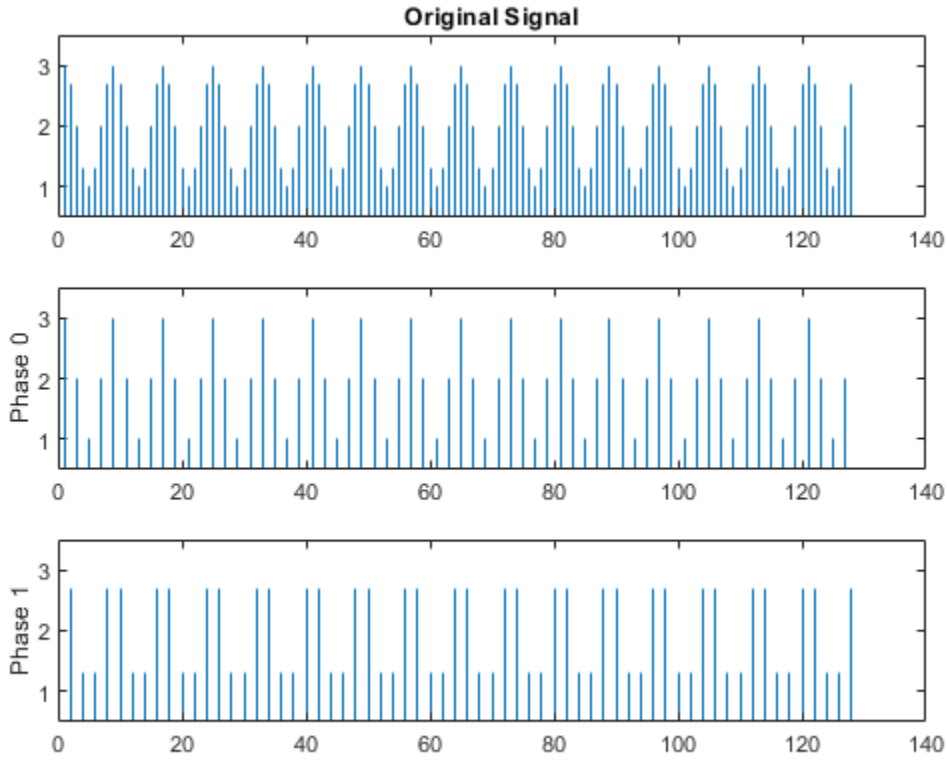
```
n = 0:127;
x = 2+cos(pi/4*n);
x0 = downsample(x,2,0);
x1 = downsample(x,2,1);
```

Upsample the two polyphase components.

```
y0 = upsample(x0,2,0);
y1 = upsample(x1,2,1);
```

Plot the upsampled polyphase components along with the original signal for comparison.

```
subplot(3,1,1)
stem(x,'Marker','none')
ylim([0.5 3.5])
title('Original Signal')

subplot(3,1,2)
stem(y0,'Marker','none')
ylim([0.5 3.5])
ylabel('Phase 0')

subplot(3,1,3)
stem(y1,'Marker','none')
ylim([0.5 3.5])
ylabel('Phase 1')
```



If you sum the two upsampled polyphase components (Phase 0 and Phase 1), you obtain the original sine wave.

See Also

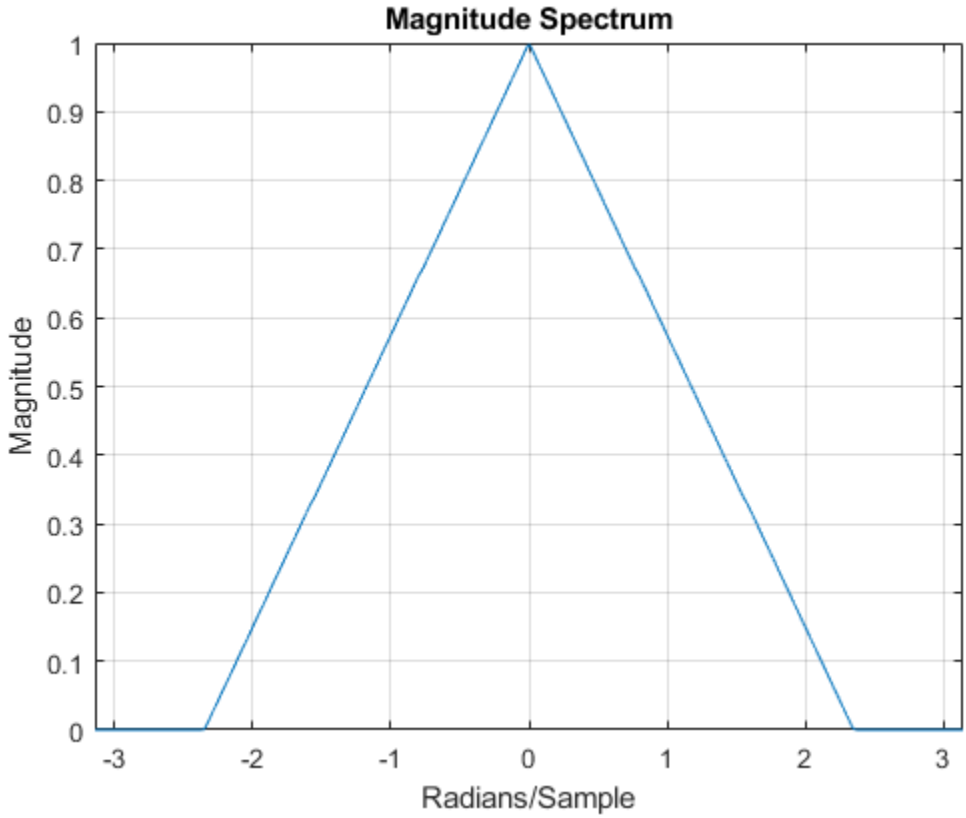
`downsample` | `upsample`

Downsampling — Aliasing

This example shows how to avoid aliasing when downsampling a signal. If a discrete-time signal's baseband spectral support is not limited to an interval of width $2\pi/M$ radians, downsampling by M results in aliasing. Aliasing is the distortion that occurs when overlapping copies of the signal's spectrum are added together. The more the signal's baseband spectral support exceeds $2\pi/M$ radians, the more severe the aliasing. Demonstrate aliasing in a signal downsampled by two. The signal's baseband spectral support exceed π radians in width.

Create a signal with baseband spectral support equal to $3\pi/2$ radians. Use `fir2` to design the signal. Plot the signal's spectrum.

```
F = [0 0.2500 0.5000 0.7500 1.0000];
A = [1.00 0.6667 0.3333 0 0];
Order = 511;
B1 = fir2(Order,F,A);
[Hx,W] = freqz(B1,1,8192,'whole');
Hx = [Hx(4098:end) ; Hx(1:4097)];
omega = -pi+(2*pi/8192):(2*pi)/8192:pi;

plot(omega,abs(Hx))
xlim([-pi pi])
grid
title('Magnitude Spectrum')
xlabel('Radians/Sample')
ylabel('Magnitude')
```

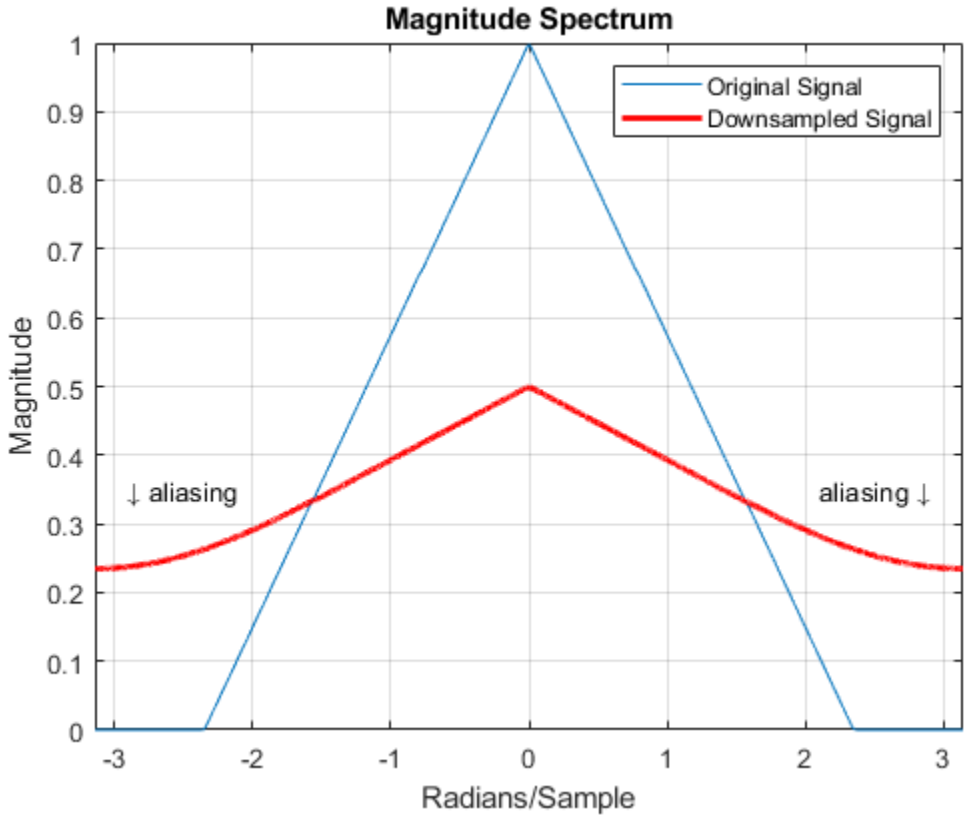


You see that the signal's baseband spectral support exceeds $[-\pi/2, \pi/2]$.

Downsample the signal by a factor of 2 and plot the downsampled signal's spectrum with the spectrum of the original signal.

```
y = downsample(B1,2,0);
[Hy,W] = freqz(y,1,8192,'whole');
Hy = [Hy(4098:end) ; Hy(1:4097)];

hold on
plot(omega,abs(Hy),'r','linewidth',2)
legend('Original Signal','Downsampled Signal')
text(-2.5,0.35,'\downarrow aliasing','HorizontalAlignment','center')
```

```
text(2.5,0.35,'aliasing \downarrow','HorizontalAlignment','center')
hold off
```

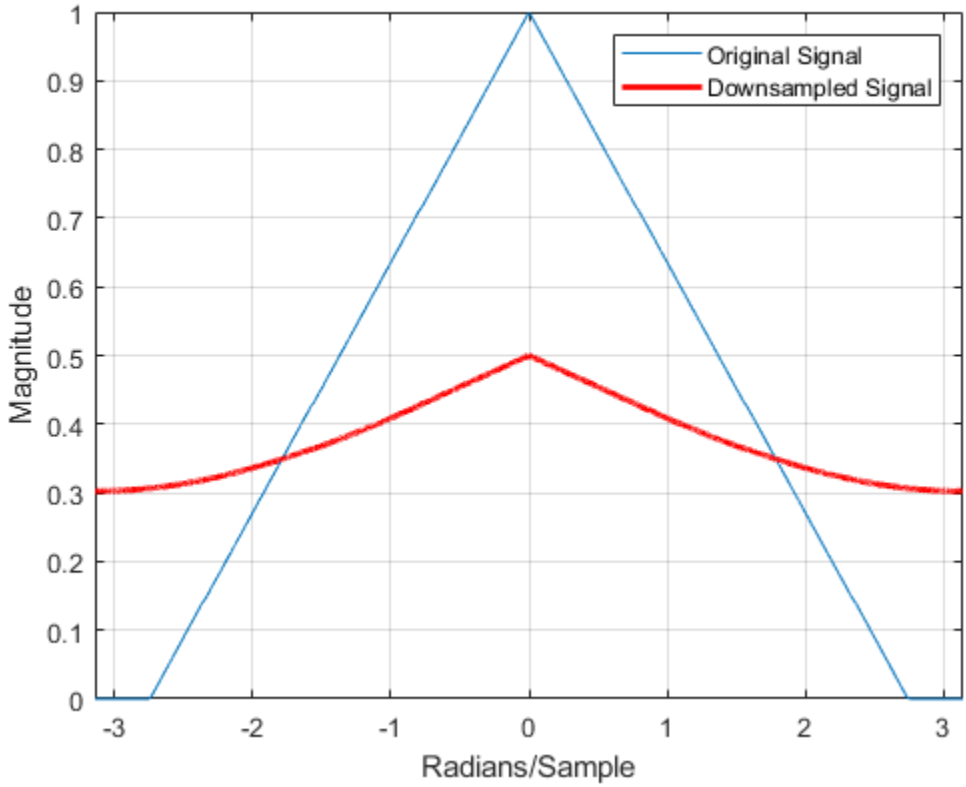


In addition to an amplitude scaling of the spectrum, the superposition of overlapping spectral replicas causes distortion of the original spectrum for $|\omega| > \pi/2$.

Increase the baseband spectral support of the signal to $[-7\pi/8, 7\pi/8]$ and downsample the signal by 2. Plot the original spectrum along with the spectrum of the downsampled signal.

```
F = [0 0.2500 0.5000 0.7500 7/8 1.0000];
A = [1.00 0.7143 0.4286 0.1429 0 0];
Order = 511;
```

```
B2 = fir2(Order,F,A);

[Hx,W] = freqz(B2,1,8192,'whole');
Hx = [Hx(4098:end) ; Hx(1:4097)];
omega = -pi+(2*pi/8192):(2*pi)/8192:pi;

plot(omega,abs(Hx))
xlim([-pi pi])

y = downsample(B2,2,0);
[Hy,W] = freqz(y,1,8192,'whole');
Hy = [Hy(4098:end) ; Hy(1:4097)];

hold on
plot(omega,abs(Hy),'r','linewidth',2)
grid
legend('Original Signal','Downsampled Signal')
xlabel('Radians/Sample')
ylabel('Magnitude')
hold off
```

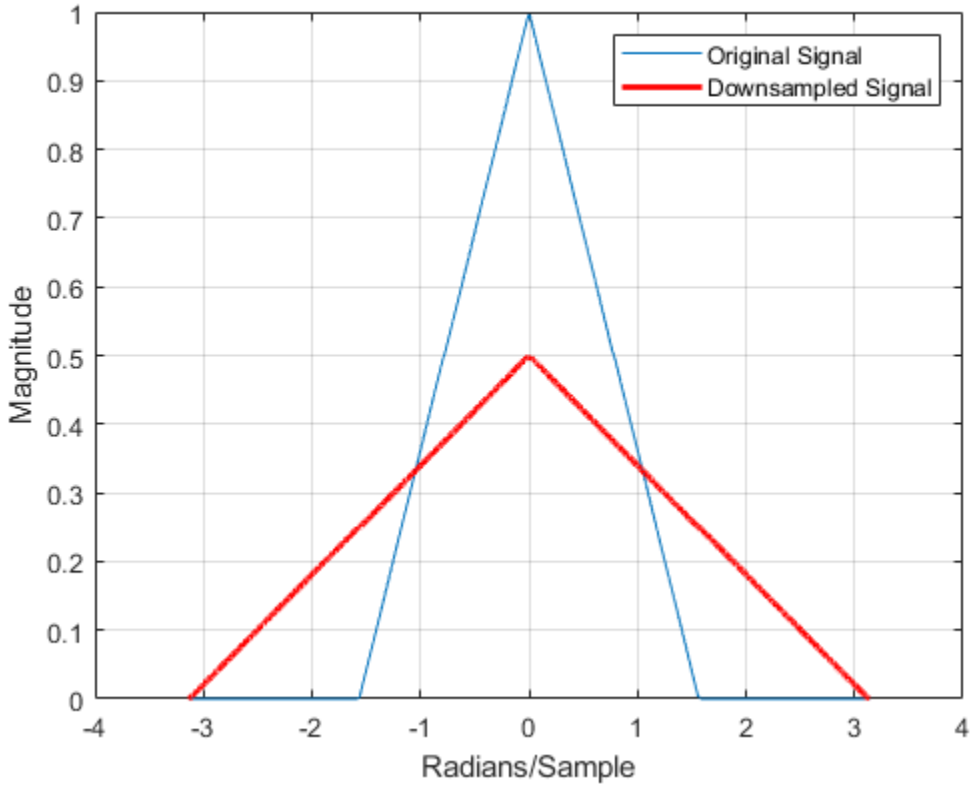


The increased spectral width results in more pronounced aliasing in the spectrum of the downsampled signal because more signal energy is outside $[-\pi/2, \pi/2]$.

Finally, construct a signal with baseband spectral support limited to $[-\pi/2, \pi/2]$. Downsample the signal by a factor of 2 and plot the spectrum of the original and downsampled signals. The downsampled signal is full band, but the shape of the spectrum is preserved because the spectral copies do not overlap. There is no aliasing.

```
F = [0 0.250 0.500 0.7500 1];
A = [1.0000 0.5000 0 0 0];
Order = 511;
B3 = fir2(Order,F,A);
[Hx,W] = freqz(B3,1,8192,'whole');
```

```
Hx = [Hx(4098:end) ; Hx(1:4097)];
omega = -pi+(2*pi/8192):(2*pi)/8192:pi;

plot(omega,abs(Hx))
xlim([-pi pi])

y = downsample(B3,2,0);
[Hy,W] = freqz(y,1,8192,'whole');
Hy = [Hy(4098:end) ; Hy(1:4097)];

plot(omega,abs(Hx))
hold on
plot(omega,abs(Hy),'r','linewidth',2)
grid
legend('Original Signal','Downsampled Signal')
xlabel('Radians/Sample')
ylabel('Magnitude')
hold off
```



You see in the preceding figure that the shape of the spectrum is preserved. The spectrum of the downsampled signal is a stretched and scaled version of the original signal's spectrum, but there is no aliasing.

See Also

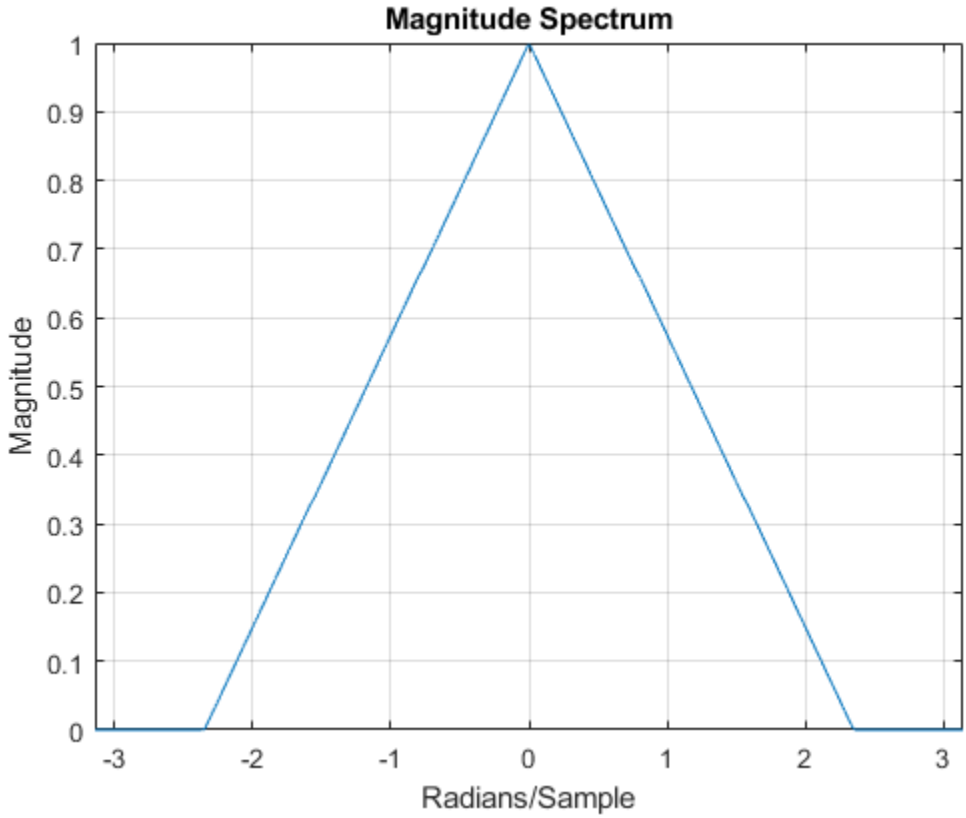
`downsample` | `fir2` | `freqz`

Filtering Before Downsampling

This example shows how to filter before downsampling to mitigate the distortion caused by aliasing. You can use `decimate` or `resample` to filter and downsample with one function. Alternatively, you can lowpass filter your data and then use `downsample`. Create a signal with baseband spectral support greater than π radians. Use `decimate` to filter the signal with a 10th-order Chebyshev type I lowpass filter prior to downsampling.

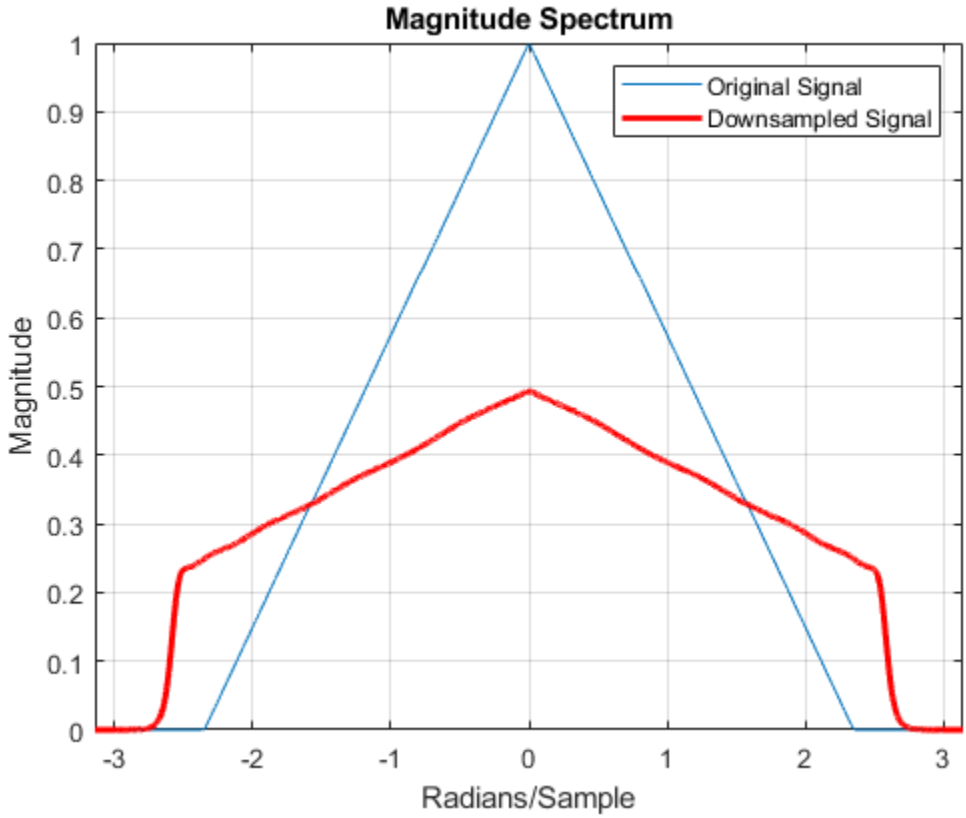
Create the signal and plot the magnitude spectrum.

```
F = [0 0.2500 0.5000 0.7500 1.0000];
A = [1.00 0.6667 0.3333 0 0];
Order = 511;
B = fir2(Order,F,A);
[Hx,W] = freqz(B,1,8192,'whole');
Hx = [Hx(4098:end) ; Hx(1:4097)];
omega = -pi+(2*pi/8192):(2*pi)/8192:pi;

plot(omega,abs(Hx))
xlim([-pi pi])
grid
title('Magnitude Spectrum')
xlabel('Radians/Sample')
ylabel('Magnitude')
```



Filter the signal with a 10th-order type I Chebyshev lowpass filter and downsample by 2. Plot the magnitude spectra of the original signal along with the filtered and downsampled signal.

```
y = decimate(B,2,10);
[Hy,W] = freqz(y,1,8192,'whole');
Hy = [Hy(4098:end) ; Hy(1:4097)];

hold on
plot(omega,abs(Hy),'r','linewidth',2)
legend('Original Signal','Downsampled Signal')
```



The lowpass filter reduces the amount of aliasing distortion outside the interval $[-\pi/2, \pi/2]$.

See Also

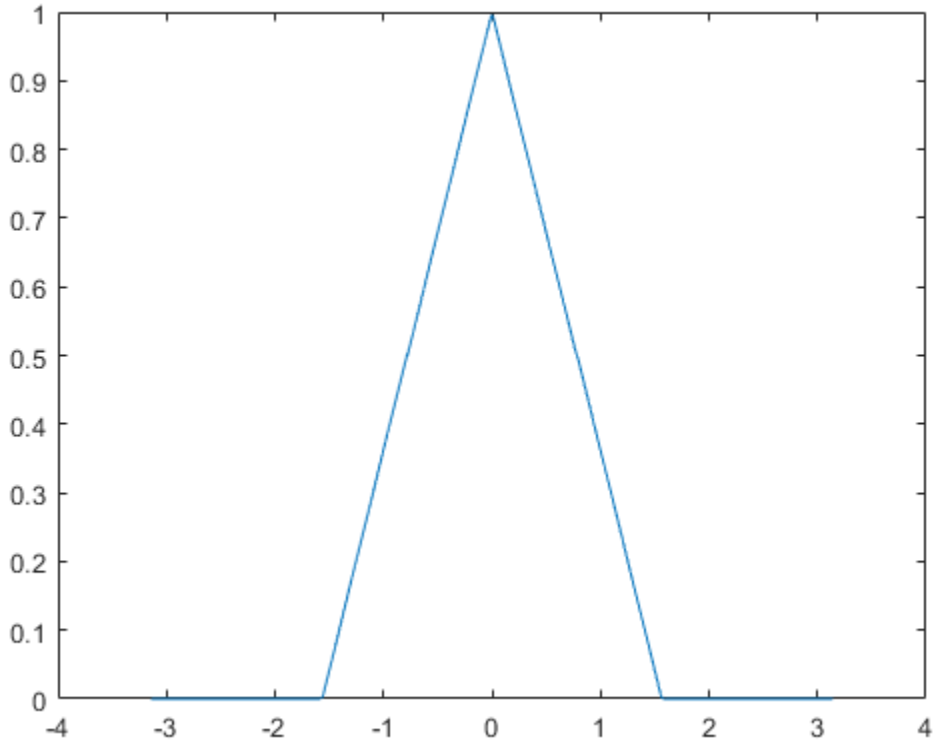
`decimate` | `fir2` | `freqz`

Upsampling — Imaging Artifacts

This example shows how to upsample a signal and how upsampling can result in images. Upsampling a signal contracts the spectrum. For example, upsampling a signal by 2 results in a contraction of the spectrum by a factor of 2. Because the spectrum of a discrete-time signal is 2π-periodic, contraction can cause replicas of the spectrum normally outside of the baseband to appear inside the interval $[-\pi,\pi]$.

Create a discrete-time signal whose baseband spectral support is $[-\pi,\pi]$. Plot the magnitude spectrum.

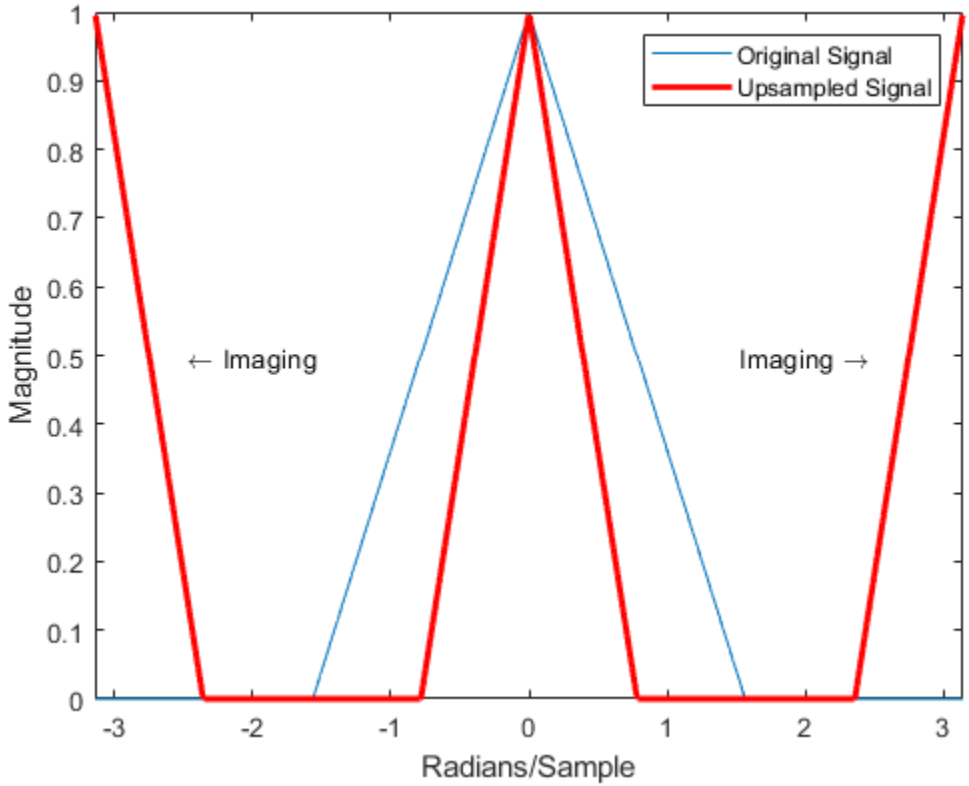
```
F = [0 0.250 0.500 0.7500 1];
A = [1.0000 0.5000 0 0 0];
Order = 511;
B = fir2(Order,F,A);
[Hx,W] = freqz(B,1,8192,'whole');
Hx = [Hx(4098:end) ; Hx(1:4097)];
omega = -pi+(2*pi/8192):(2*pi)/8192:pi;

plot(omega,abs(Hx))
```



Upsample the signal by 2. Plot the spectrum of the upsampled signal.

```
y = upsample(B,2);
[Hy,W] = freqz(y,1,8192,'whole');
Hy = [Hy(4098:end) ; Hy(1:4097)];

hold on
plot(omega,abs(Hy),'r','linewidth',2)
xlim([-pi pi])
legend('Original Signal','Upsampled Signal')
xlabel('Radians/Sample')
ylabel('Magnitude')
text(-2,0.5,'\leftarrow Imaging','HorizontalAlignment','center')
```

```
text(2,0.5,'Imaging \rightarrow','HorizontalAlignment','center')
hold off
```



You can see in the preceding figure that the contraction of the spectrum has drawn subsequent periods of the spectrum into the interval $[-\pi,\pi]$.

See Also

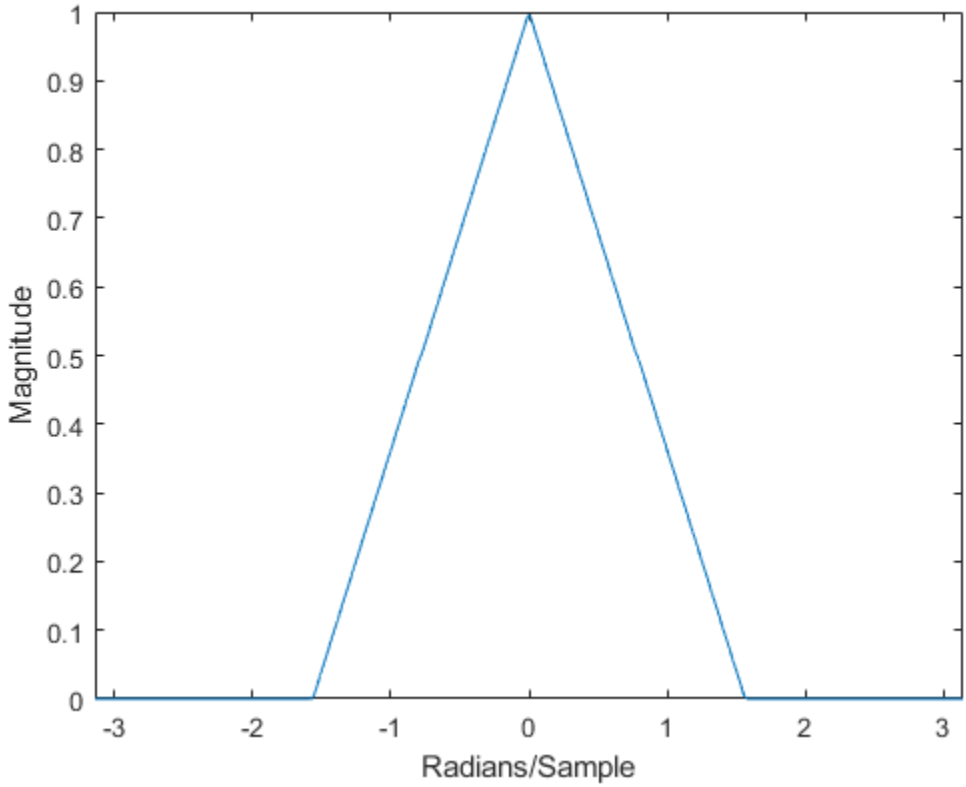
`fir2` | `freqz` | `upsample`

Filtering After Upsampling — Interpolation

This example shows how to upsample a signal and apply a lowpass interpolation filter with `interp`. Upsampling by L inserts L - 1 zeros between every element of the original signal. Upsampling can create imaging artifacts. Lowpass filtering following upsampling can remove these imaging artifacts. In the time domain, lowpass filtering interpolates the zeros inserted by upsampling.

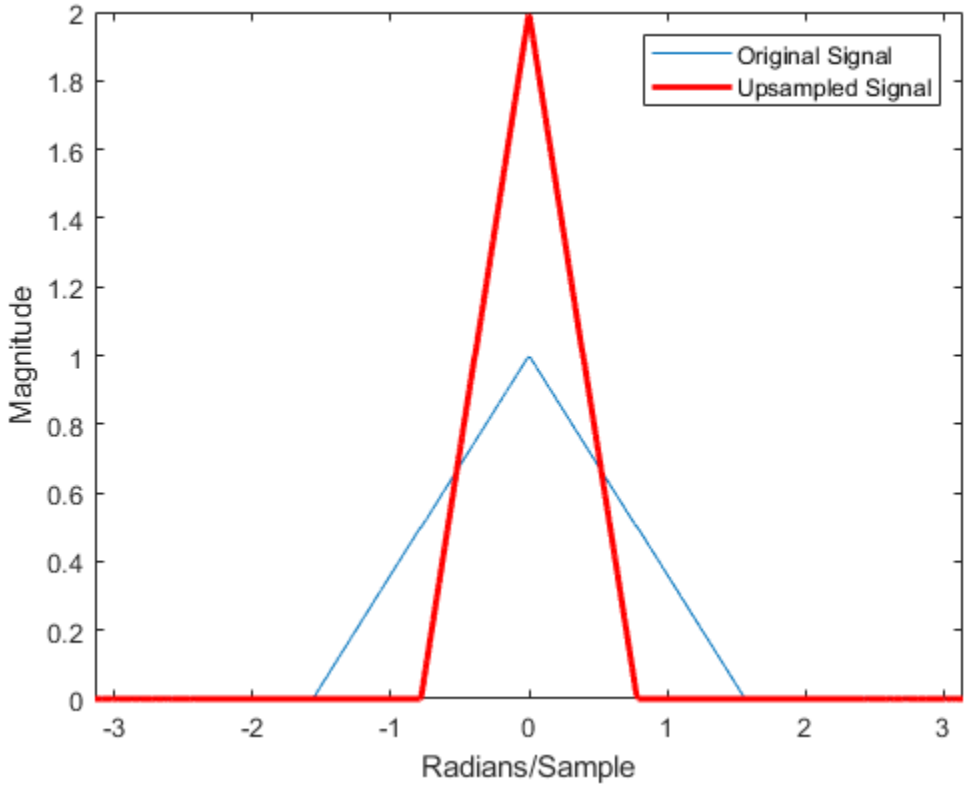
Create a discrete-time signal whose baseband spectral support is $[-\pi/2, \pi/2]$. Plot the magnitude spectrum.

```
F = [0 0.250 0.500 0.7500 1];
A = [1.0000 0.5000 0 0 0];
Order = 511;
B = fir2(Order,F,A);
[Hx,W] = freqz(B,1,8192,'whole');
Hx = [Hx(4098:end) ; Hx(1:4097)];
omega = -pi+(2*pi/8192):(2*pi)/8192:pi;

plot(omega,abs(Hx))
xlim([-pi pi])
xlabel('Radians/Sample')
ylabel('Magnitude')
```



Upsample the signal and apply a lowpass filter to remove the imaging artifacts. Plot the magnitude spectrum.

```
y = interp(B,2);
[Hy,W] = freqz(y,1,8192,'whole');
Hy = [Hy(4098:end) ; Hy(1:4097)];

hold on
plot(omega,abs(Hy),'r','linewidth',2)
legend('Original Signal','Upsampled Signal')
```



Upsampling still contracts the spectrum, but the imaging artifacts are removed by the lowpass filter.

See Also

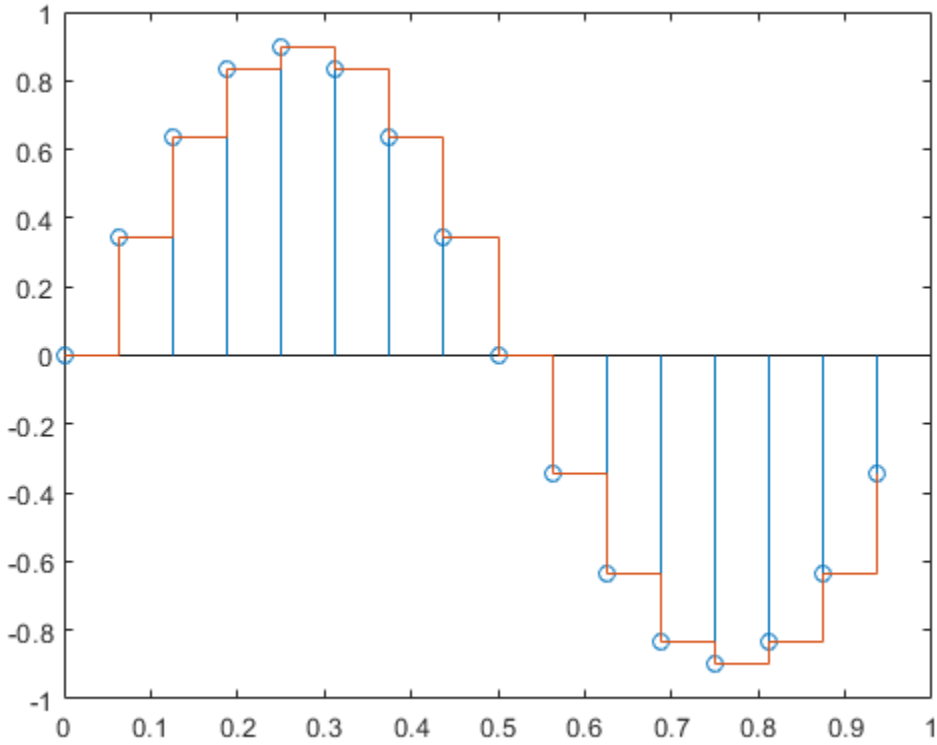
`fir2` | `freqz` | `interp`

Simulate a Sample-and-Hold System

This example shows several ways to simulate the output of a sample-and-hold system by upsampling and filtering a signal.

Construct a sinusoidal signal. Specify a sample rate such that 16 samples correspond to exactly one signal period. Draw a stem plot of the signal. Overlay a stairstep graph for sample-and-hold visualization.

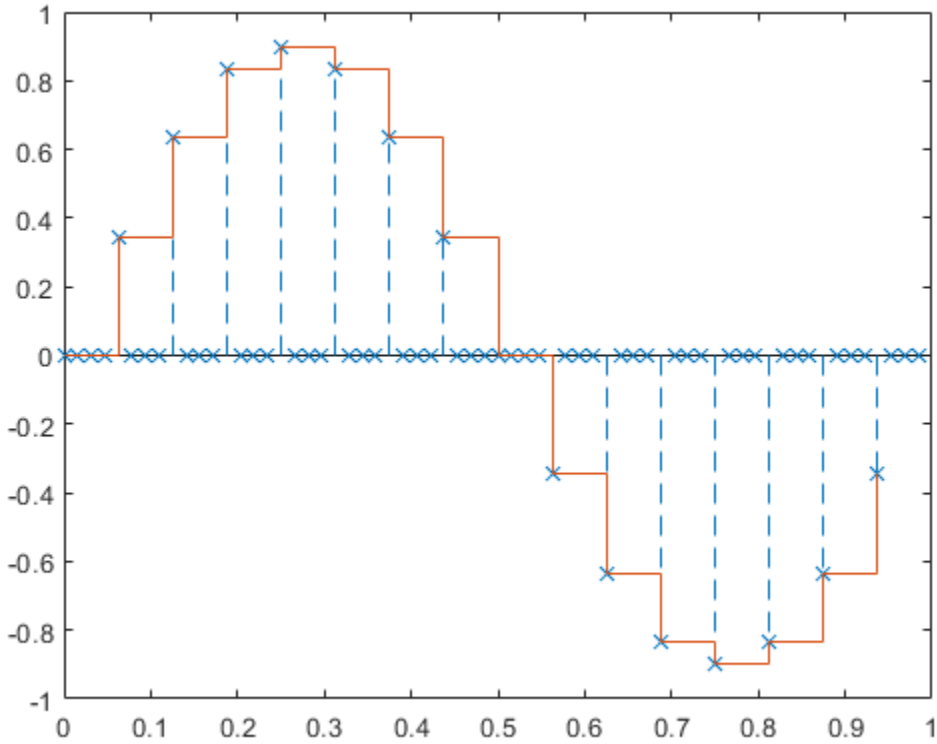
```
fs = 16;
t = 0:1/fs:1-1/fs;

x = .9*sin(2*pi*t);

stem(t,x)
hold on
stairs(t,x)
hold off
```



Upsample the signal by a factor of four. Plot the result alongside the original signal. `upsample` increases the sample rate of the signal by adding zeros between the existing samples.

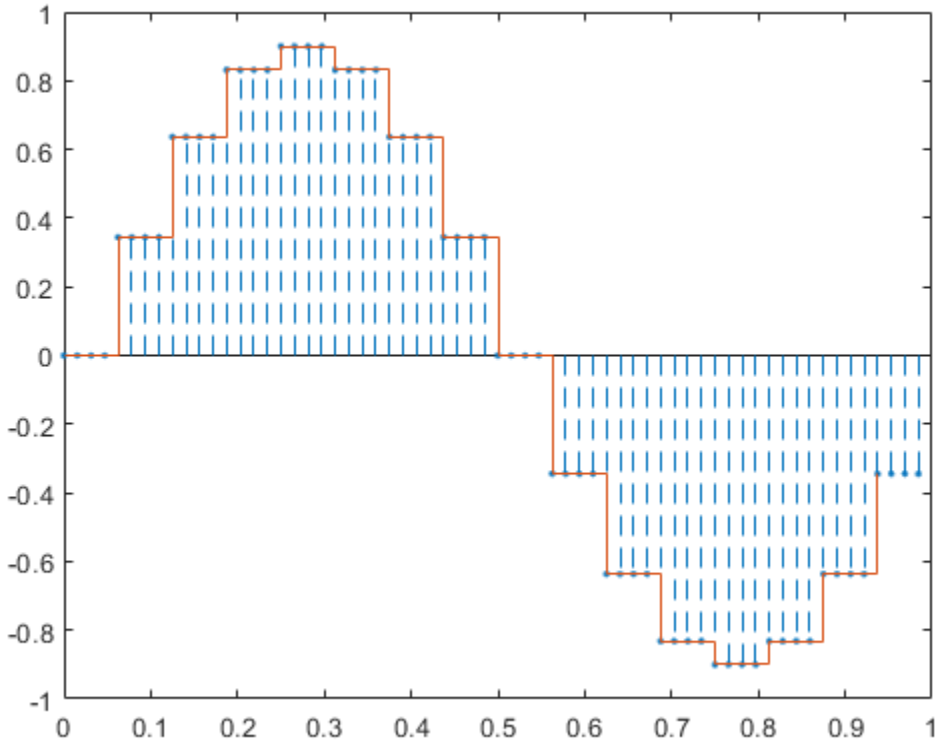
```
ups = 4;

fu = fs*ups;
tu = 0:1/fu:1-1/fu;

y = upsample(x,ups);

stem(tu,y,'--x')

hold on
```

```
stairs(t,x)
hold off
```



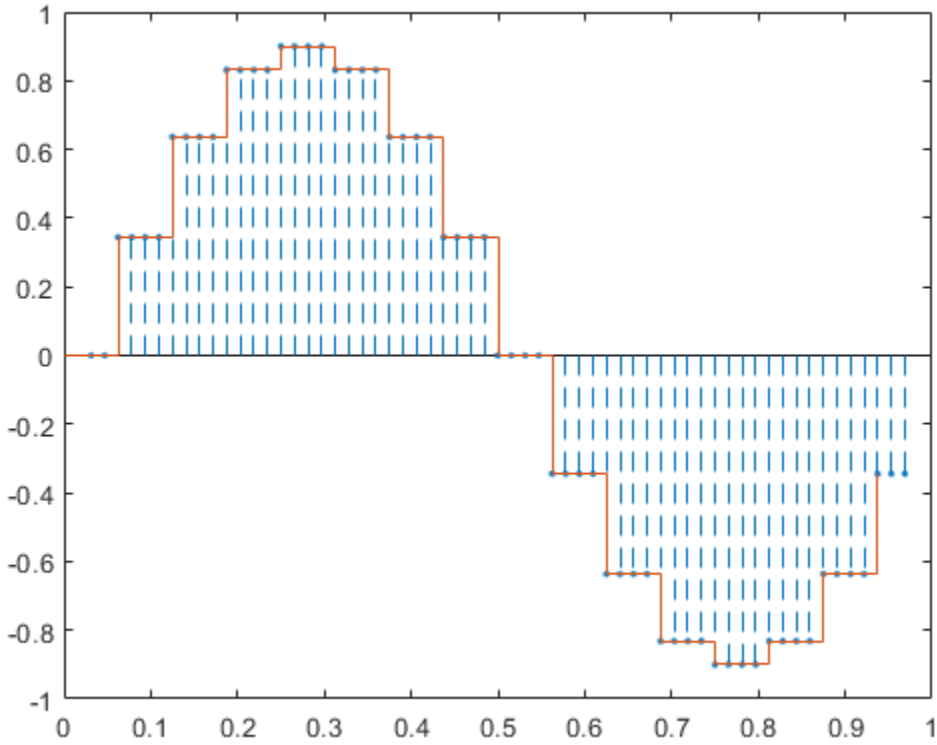
Filter with a moving-average FIR filter to fill in the zeros with sample-and-hold values.

```
h = ones(ups,1);

z = filter(h,1,y);

stem(tu,z,'--.')
hold on
stairs(t,x)
hold off
```



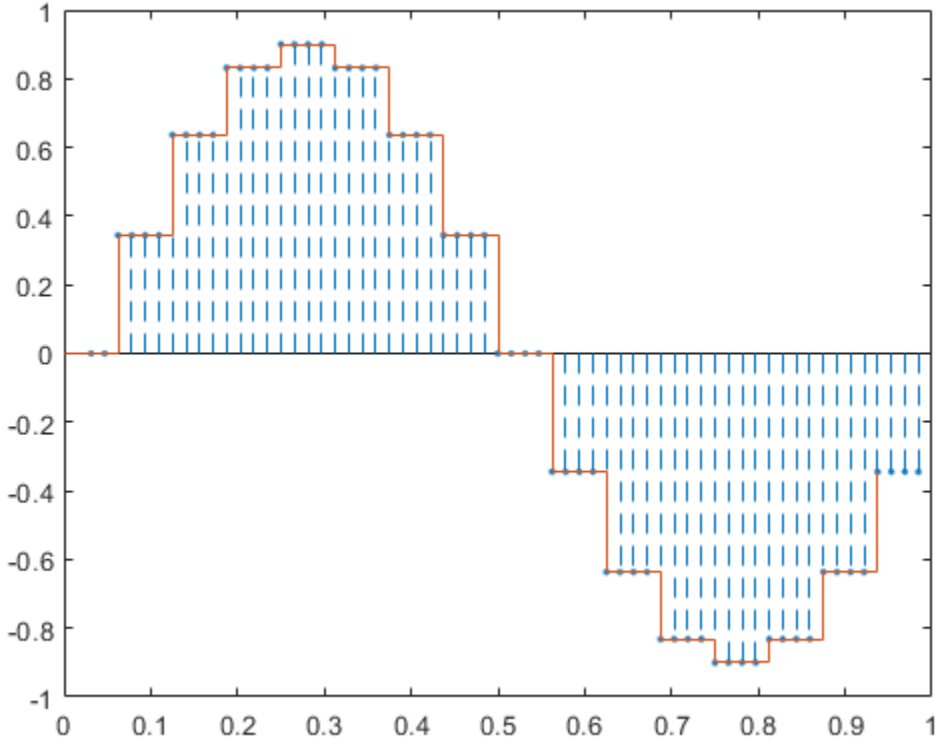
You can obtain the same behavior using the MATLAB® function `interp1` with nearest-neighbor interpolation. In that case, you must shift the origin to line up the sequence.

```
zi = interp1(t,x,tu,'nearest');

dl = floor(ups/2);

stem(tu(1+dl:end),zi(1:end-dl),'--.')
hold on
stairs(t,x)
hold off
```



The function `resample` produces the same result when you set the last input argument to zero.

```
q = resample(x,ups,1,0);

stem(tu(1+dl:end),q(1:end-dl),'--.')
hold on
stairs(t,x)
hold off
```



See Also

`resample` | `upsample`

Changing Signal Sample Rate

This example shows how to change the sample rate of a signal. The example has two parts. Part one changes the sample rate of a sinusoidal input from 44.1 kHz to 48 kHz. This workflow is common in audio processing. The sample rate used on compact discs is 44.1 kHz, while the sample rate used on digital audio tape is 48 kHz. Part two changes the sample rate of a recorded speech sample from 7418 Hz to 8192 Hz.

Create an input signal consisting of a sum of sine waves sampled at 44.1 kHz. The sine waves have frequencies of 2, 4, and 8 kHz.

```
Fs = 44.1e3;
t = 0:1/Fs:1-1/Fs;
x = cos(2*pi*2000*t)+1/2*sin(2*pi*4000*(t-pi/4))+1/4*cos(2*pi*8000*t);
```

To change the sample rate from 44.1 to 48 kHz, you have to determine a rational number (ratio of integers), `P/Q`, such that `P/Q` times the original sample rate, 44100, is equal to 48000 within some specified tolerance.

To determine these factors, use `rat`. Input the ratio of the new sample rate, 48000, to the original sample rate, 44100.

```
[P,Q] = rat(48e3/Fs);
abs(P/Q*Fs-48000)
```

```
ans = 7.2760e-12
```

You see that `P/Q*Fs` only differs from the desired sample rate, 48000, on the order of 10^{-12}.

Use the numerator and denominator factors obtained with `rat` as inputs to `resample` to output a waveform sampled at 48 kHz.

```
xnew = resample(x,P,Q);
```

If your computer can play audio, you can play the two waveforms. Set the volume to a comfortable level before you play the signals. Execute the `play` commands separately so that you can hear the signal with the two different sample rates.

```
% P44_1 = audioplayer(x,44100);
% P48 = audioplayer(xnew,48000);
% play(P44_1)
% play(P48)
```

Change the sample rate of a speech sample from 7418 Hz to 8192 Hz. The speech signal is a recording of a speaker saying "MATLAB®".

Load the speech sample.

```
load mtlb
```

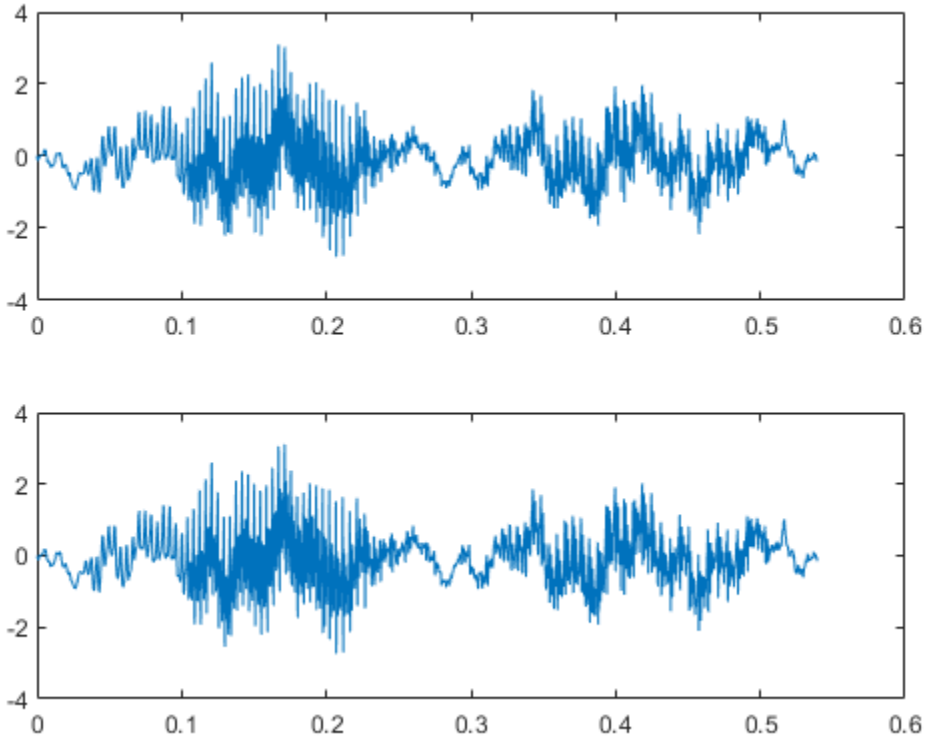
Loading the file `mtlb.mat` brings the speech signal, `mtlb`, and the sample rate, `Fs`, into the MATLAB workspace.

Determine a rational approximation to the ratio of the new sample rate, 8192, to the original sample rate. Use `rat` to determine the approximation.

```
[P,Q] = rat(8192/Fs);
```

Resample the speech sample at the new sample rate. Plot the two signals.

```
mtlb_new = resample(mtlb,P,Q);

subplot(2,1,1)
plot((0:length(mtlb)-1)/Fs,mtlb)
subplot(2,1,2)
plot((0:length(mtlb_new)-1)/(P/Q*Fs),mtlb_new)
```



If your computer has audio output capability, you can play the two waveforms at their respective sample rates for comparison. Set the volume on your computer to a comfortable listening level before playing the sounds. Execute the `play` commands separately to compare the speech samples at the different sample rates.

```
% Pmtlb = audioplayer(mtlb,Fs);
% Pmtlb_new = audioplayer(mtlb_new,8192);
% play(Pmtlb)
% play(Pmtlb_new)
```

See Also

`resample`

Spectral Analysis

- “Power Spectral Density Estimates Using FFT” on page 12-2
- “Bias and Variability in the Periodogram” on page 12-11
- “Cross Spectrum and Magnitude-Squared Coherence” on page 12-22
- “Amplitude Estimation and Zero Padding” on page 12-25
- “Significance Testing for Periodic Component” on page 12-29
- “Frequency Estimation by Subspace Methods” on page 12-32
- “Frequency-Domain Linear Regression” on page 12-35
- “Measure Total Harmonic Distortion” on page 12-46
- “Measure Mean Frequency, Power, Bandwidth” on page 12-49
- “Periodogram of Data Set with Missing Samples” on page 12-55
- “Welch Spectrum Estimates” on page 12-59

Power Spectral Density Estimates Using FFT

This example shows how to obtain nonparametric power spectral density (PSD) estimates equivalent to the periodogram using `fft`. The examples show you how to properly scale the output of `fft` for even-length inputs, for normalized frequency and hertz, and for one- and two-sided PSD estimates.

Even-Length Input with Sample Rate

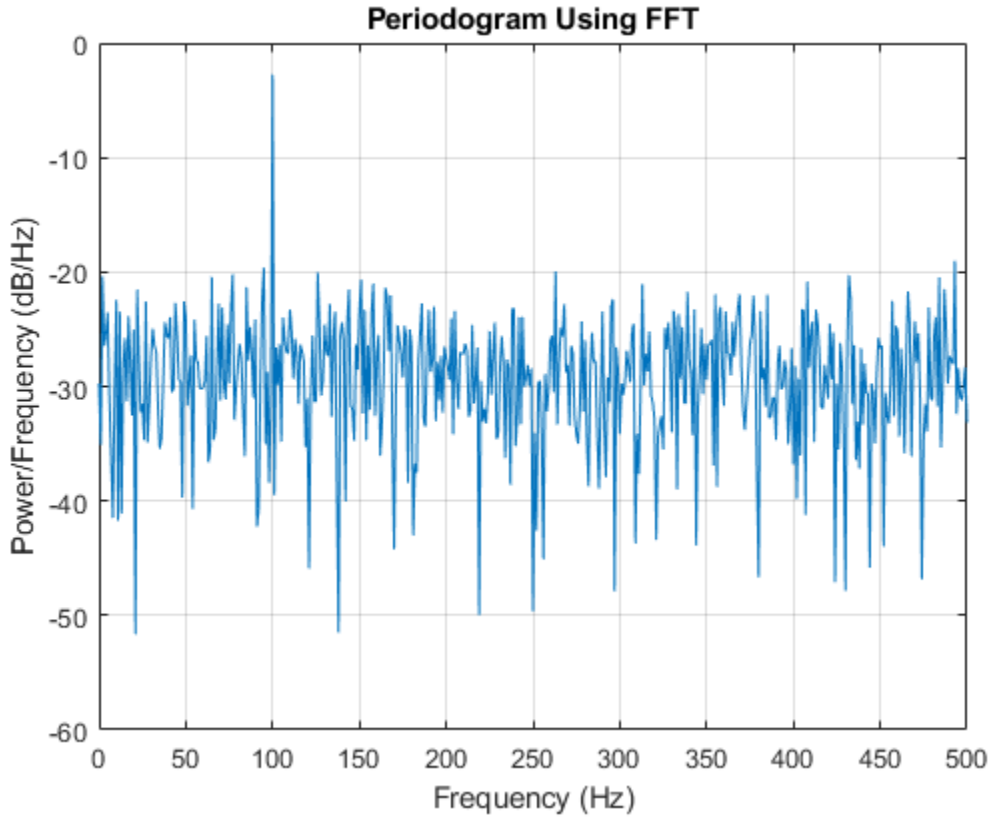
Obtain the periodogram for an even-length signal sampled at 1 kHz using both `fft` and `periodogram`. Compare the results.

Create a signal consisting of a 100 Hz sine wave in $N(0,1)$ additive noise. The sampling frequency is 1 kHz. The signal length is 1000 samples. Use the default settings of the random number generator for reproducible results.

```
rng default
Fs = 1000;
t = 0:1/Fs:1-1/Fs;
x = cos(2*pi*100*t) + randn(size(t));
```

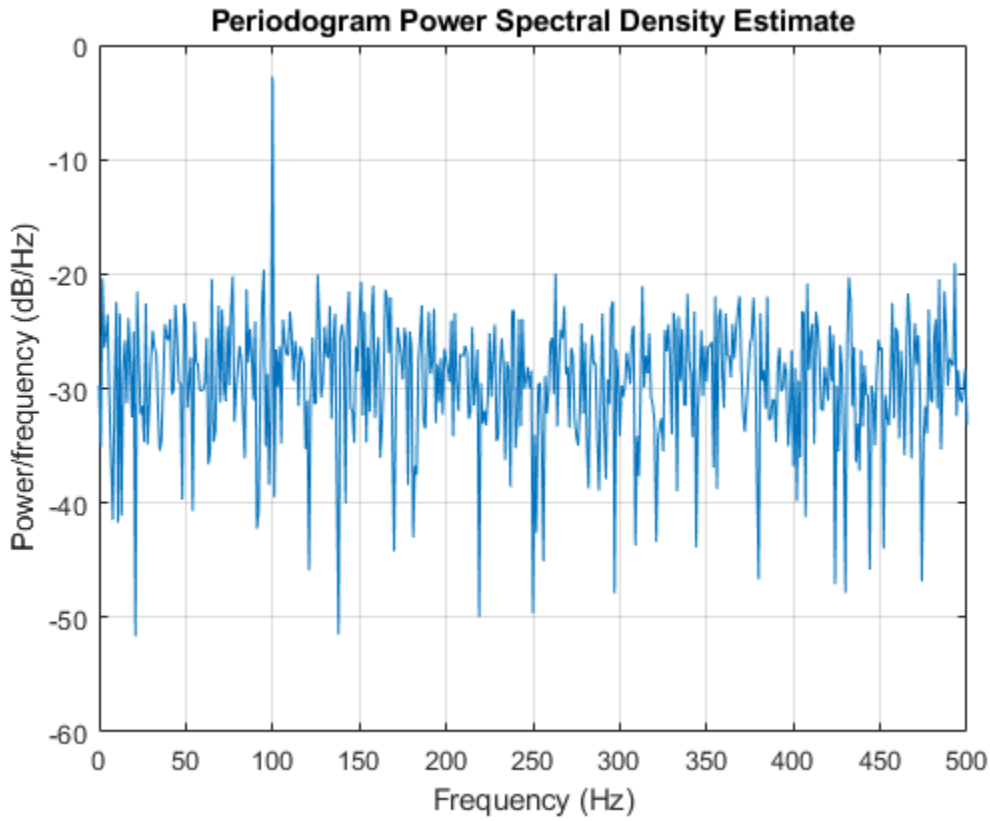
Obtain the periodogram using `fft`. The signal is real-valued and has even length. Because the signal is real-valued, you only need power estimates for the positive or negative frequencies. In order to conserve the total power, multiply all frequencies that occur in both sets -- the positive and negative frequencies -- by a factor of 2. Zero frequency (DC) and the Nyquist frequency do not occur twice. Plot the result.

```
N = length(x);
xdft = fft(x);
xdft = xdft(1:N/2+1);
psdx = (1/(Fs*N)) * abs(xdft).^2;
psdx(2:end-1) = 2*psdx(2:end-1);
freq = 0:Fs/length(x):Fs/2;

plot(freq,10*log10(psdx))
grid on
title('Periodogram Using FFT')
xlabel('Frequency (Hz)')
ylabel('Power/Frequency (dB/Hz)')
```



Compute and plot the periodogram using `periodogram`. Show that the two results are identical.

```
periodogram(x,rectwin(length(x)),length(x),Fs)
```




```
mxerr = max(psdx'-periodogram(x,rectwin(length(x)),length(x),Fs))
```

```
mxerr = 3.4694e-18
```

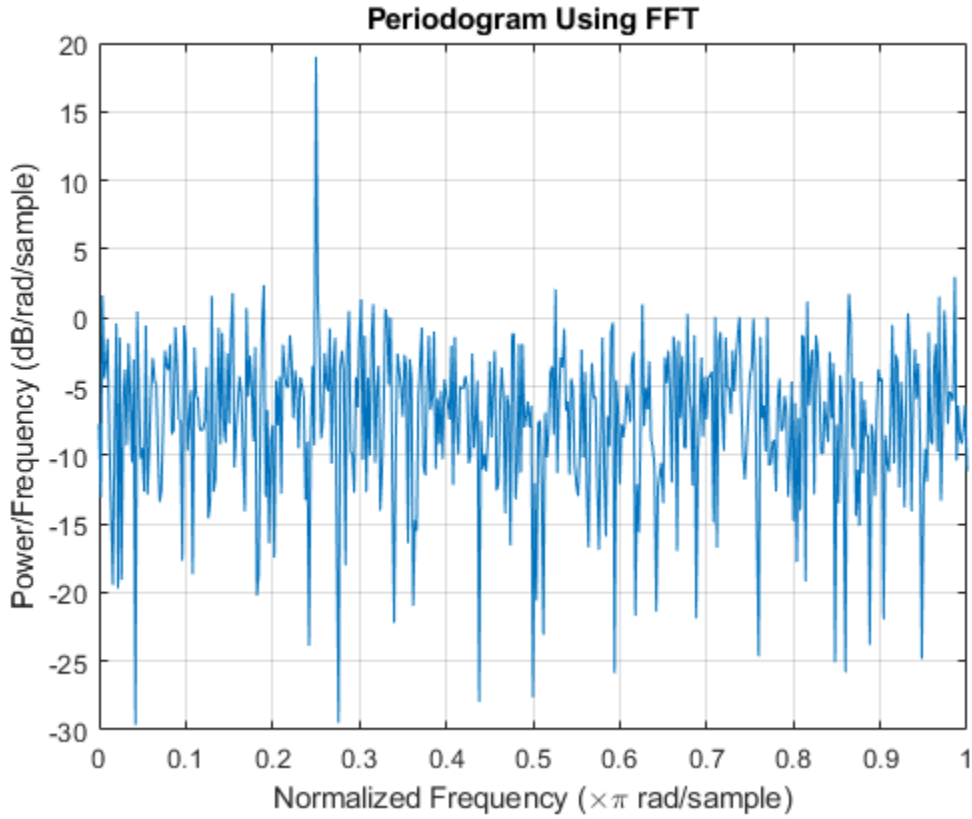
Input with Normalized Frequency

Use `fft` to produce a periodogram for an input using normalized frequency. Create a signal consisting of a sine wave in $N(0,1)$ additive noise. The sine wave has an angular frequency of $\pi/4$ rad/sample. Use the default settings of the random number generator for reproducible results.

```
rng default
n = 0:999;
x = cos(pi/4*n) + randn(size(n));
```

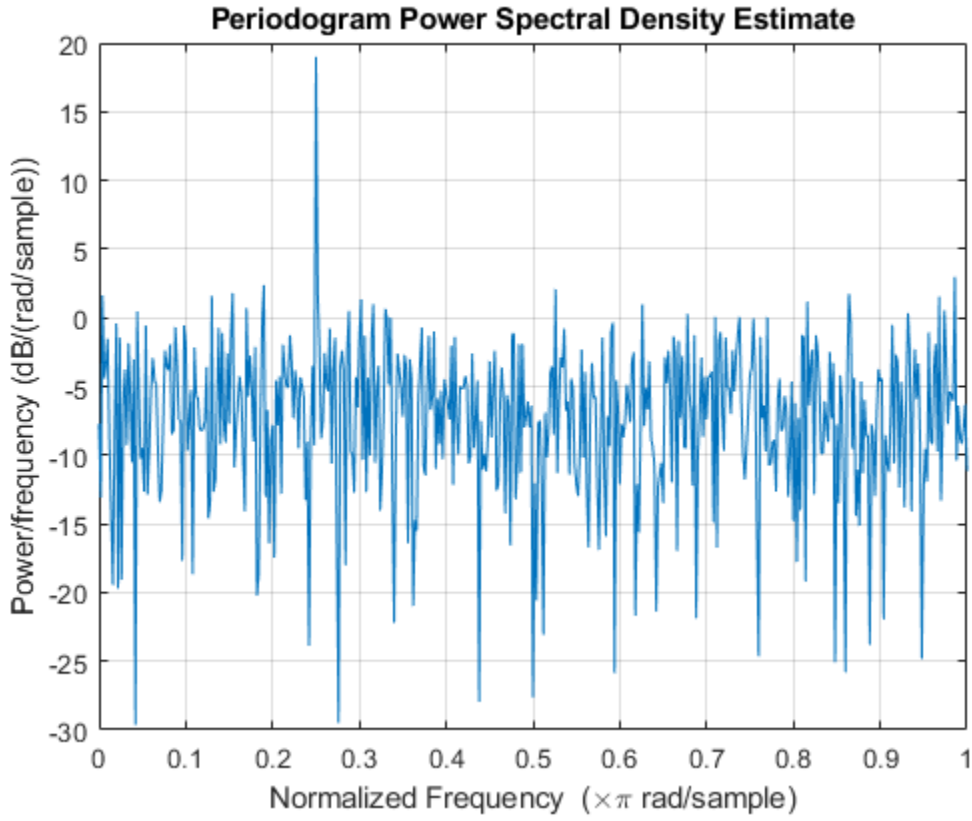
Obtain the periodogram using `fft`. The signal is real-valued and has even length. Because the signal is real-valued, you only need power estimates for the positive or negative frequencies. In order to conserve the total power, multiply all frequencies that occur in both sets -- the positive and negative frequencies -- by a factor of 2. Zero frequency (DC) and the Nyquist frequency do not occur twice. Plot the result.

```
N = length(x);
xdft = fft(x);
xdft = xdft(1:N/2+1);
psdx = (1/(2*pi*N)) * abs(xdft).^2;
psdx(2:end-1) = 2*psdx(2:end-1);
freq = 0:(2*pi)/N:pi;

plot(freq/pi,10*log10(psdx))
grid on
title('Periodogram Using FFT')
xlabel('Normalized Frequency (\times\pi rad/sample)')
ylabel('Power/Frequency (dB/rad/sample)')
```



Compute and plot the periodogram using `periodogram`. Show that the two results are identical.

```
periodogram(x,rectwin(length(x)),length(x))
```




```
mxerr = max(psdx'-periodogram(x,rectwin(length(x)),length(x)))
```

```
mxerr = 1.4211e-14
```

Complex-Valued Input with Normalized Frequency

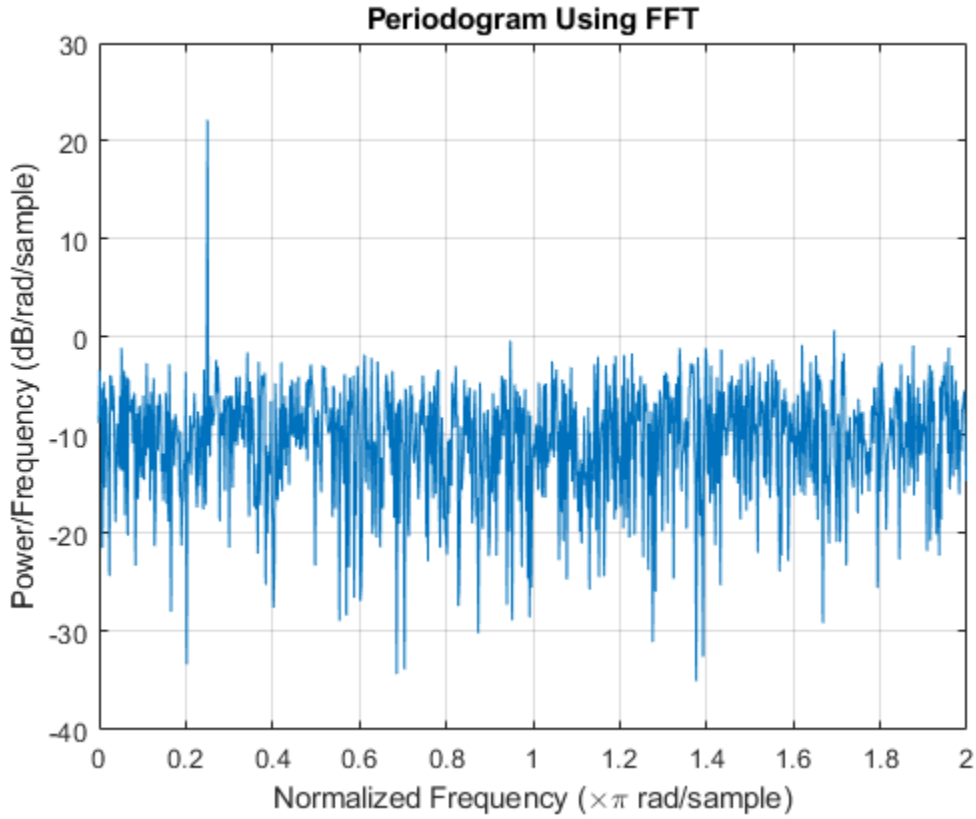
Use fft to produce a periodogram for a complex-valued input with normalized frequency.

The signal is a complex exponential with an angular frequency of $\pi/4$ rad/sample in complex-valued $N(0,1)$ noise. Set the random number generator to the default settings for reproducible results.

```
rng default
n = 0:999;
x = exp(1j*pi/4*n) + [1 1j]*randn(2,length(n))/sqrt(2);
```

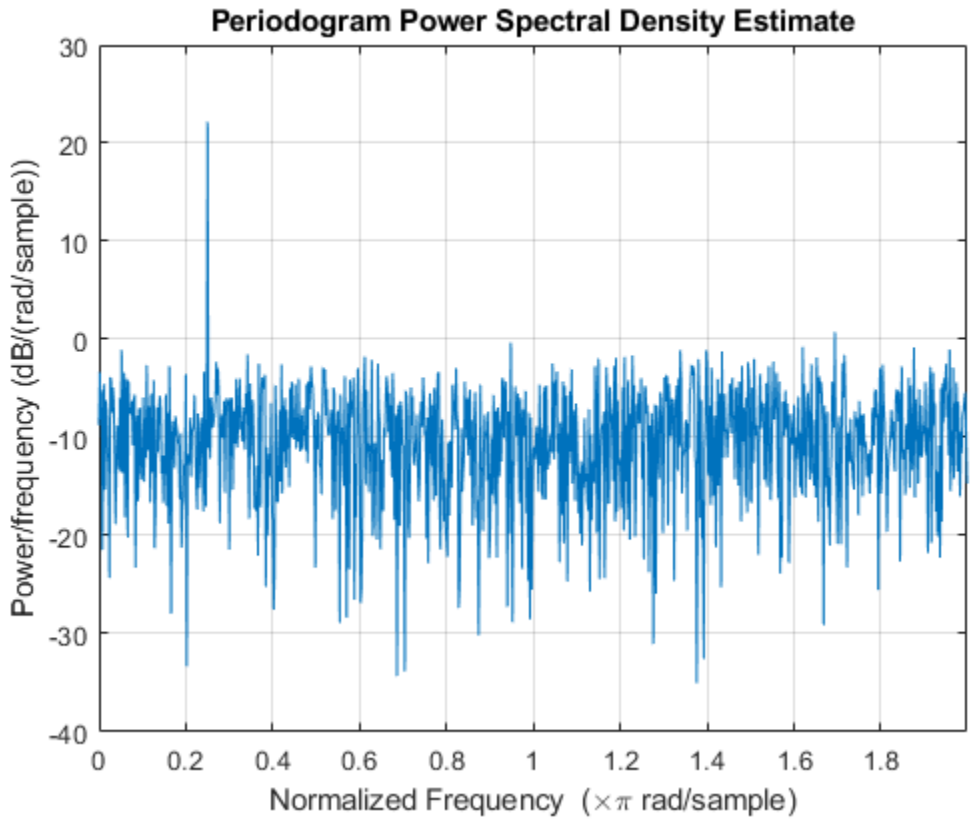
Use `fft` to obtain the periodogram. Because the input is complex-valued, obtain the periodogram from $(-\pi,\pi]$ rad/sample. Plot the result.

```
N = length(x);
xdft = fft(x);
psdx = (1/(2*pi*N)) * abs(xdft).^2;
freq = 0:(2*pi)/N:2*pi-(2*pi)/N;

plot(freq/pi,10*log10(psdx))
grid on
title('Periodogram Using FFT')
xlabel('Normalized Frequency (\times\pi rad/sample)')
ylabel('Power/Frequency (dB/rad/sample)')
```



Use `periodogram` to obtain and plot the periodogram. Compare the PSD estimates.

```
periodogram(x,rectwin(length(x)),length(x),'twosided')
```




```
mxerr = max(psdx'-periodogram(x,rectwin(length(x)),length(x),'twosided'))
```

```
mxerr = 4.4409e-16
```

See Also

Apps
Signal Analyzer

Functions
`fft` | `periodogram` | `pspectrum`

Bias and Variability in the Periodogram

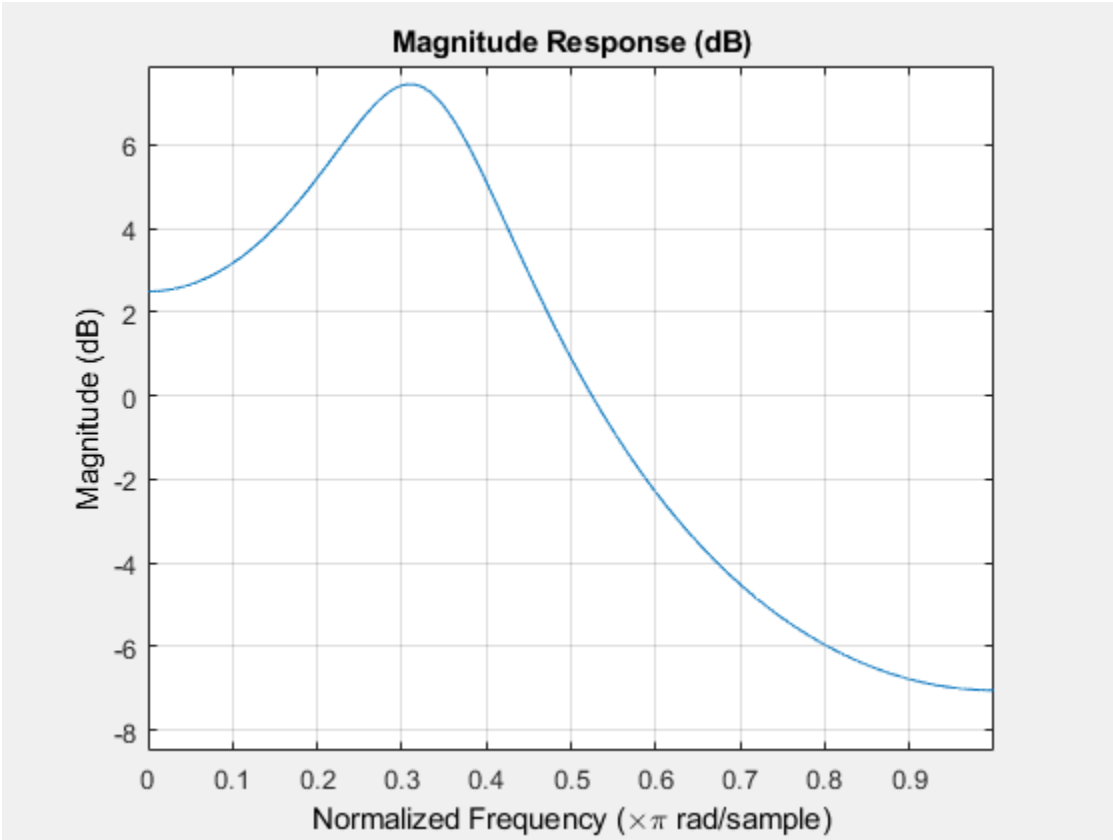
This example shows how to reduce bias and variability in the periodogram. Using a window can reduce the bias in the periodogram, and using windows with averaging can reduce variability.

Use wide-sense stationary autoregressive (AR) processes to show the effects of bias and variability in the periodogram. AR processes present a convenient model because their PSDs have closed-form expressions. Create an AR(2) model of the following form:

$$y(n) - 0.75y(n-1) + 0.5y(n-2) = \varepsilon(n),$$

where $\varepsilon(n)$ is a zero mean white noise sequence with some specified variance. In this example, assume the variance and the sampling period to be 1. To simulate the preceding AR(2) process, create an all-pole (IIR) filter. View the filter's magnitude response.

```
B2 = 1;
A2 = [1 -0.75 0.5];
fvtool(B2,A2)
```



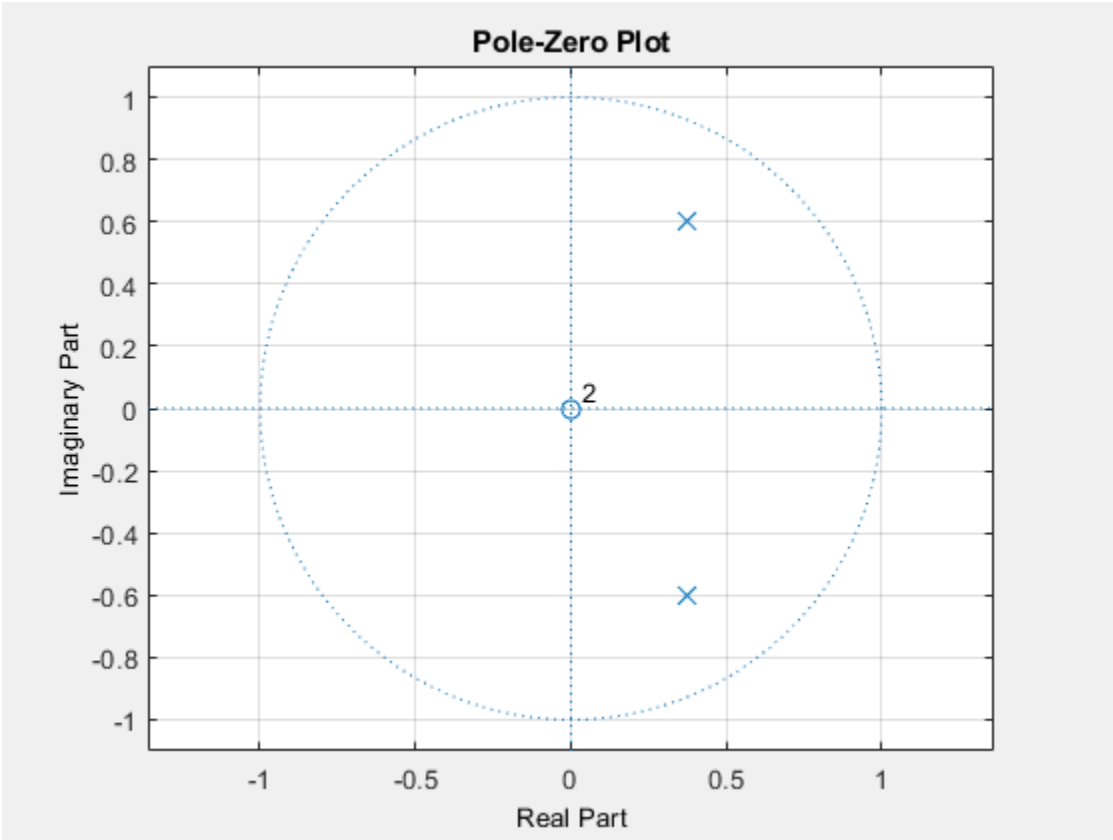
This process is bandpass. The dynamic range of the PSD is approximately 14.5 dB, as you can determine with the following code.

```
[H2,W2] = freqz(B2,A2,1e3,1);
dr2 = max(20*log10(abs(H2)))-min(20*log10(abs(H2)))
```

```
dr2 = 14.4984
```

By examining the placement of the poles, you see that this AR(2) process is stable. The two poles are inside the unit circle.

```
fvtool(B2,A2,'Analysis','polezero')
```

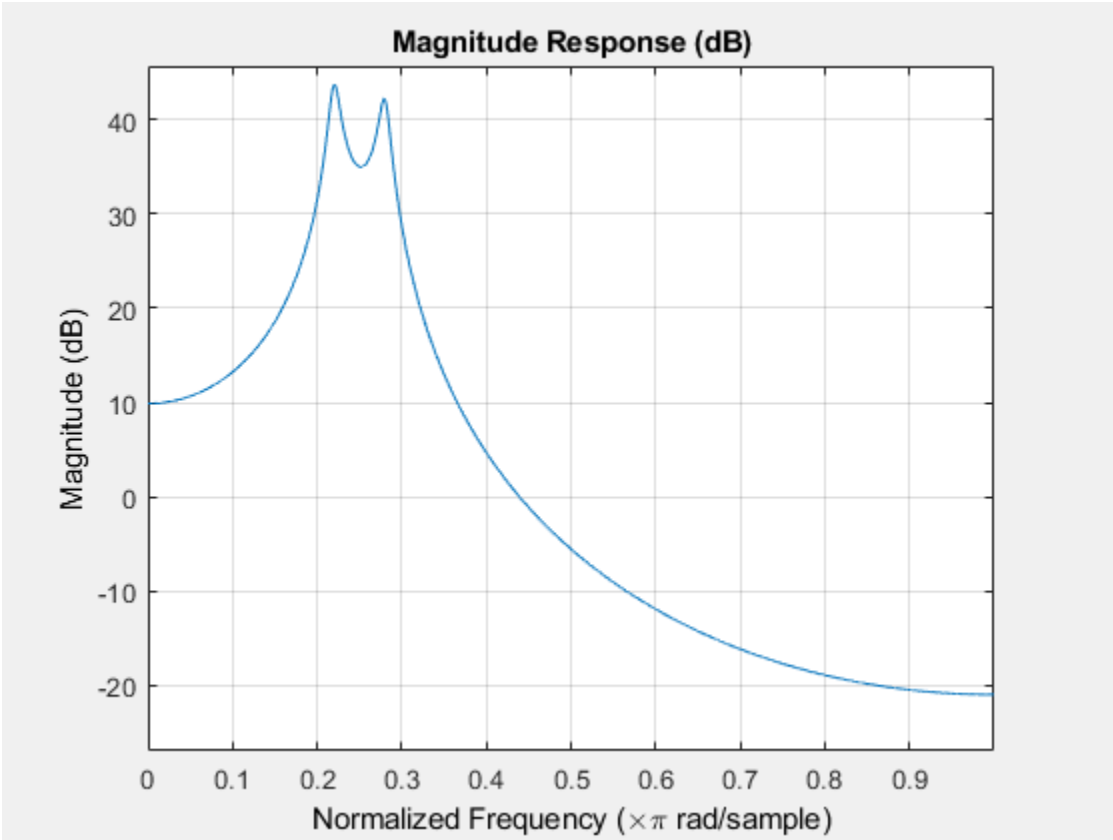


Next, create an AR(4) process described by the following equation:

$$y(n) - 2.7607y(n-1) + 3.8106y(n-2) - 2.6535y(n-3) + 0.9238y(n-4) = \varepsilon(n).$$

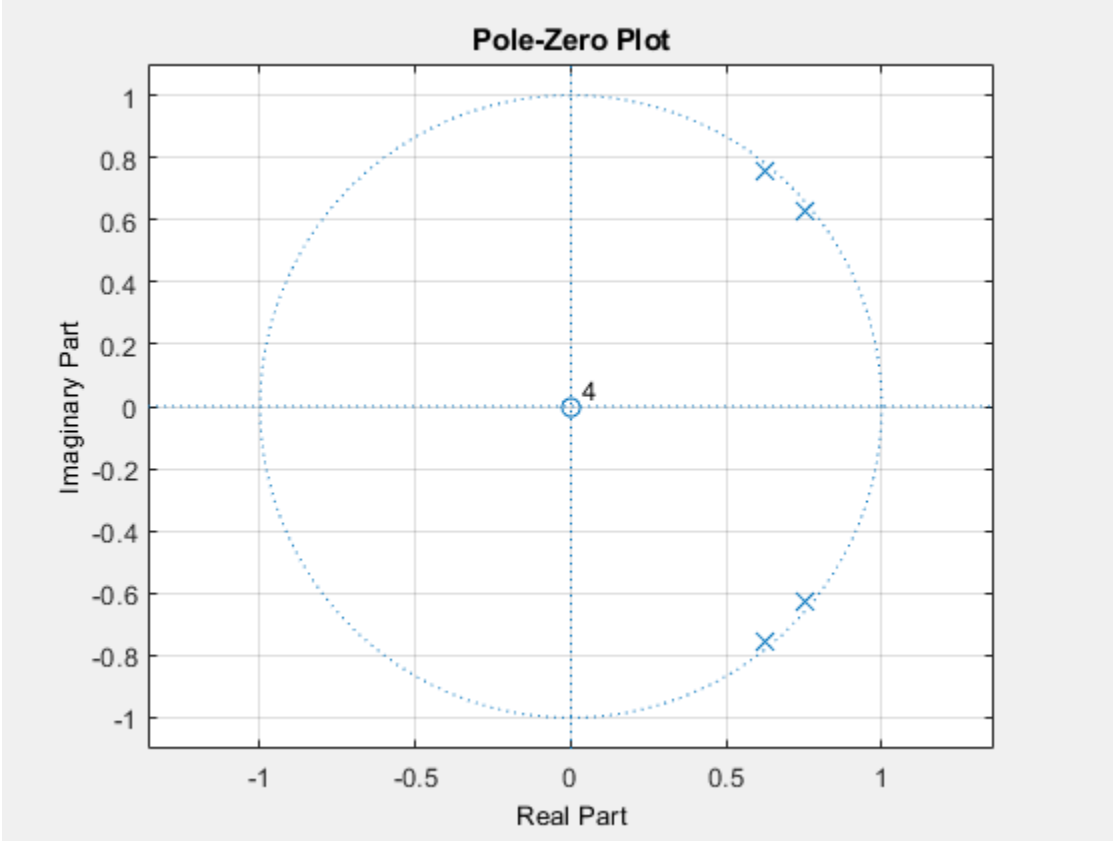
Use the following code to view the magnitude response of this IIR system.

```
B4 = 1;
A4 = [1 -2.7607 3.8106 -2.6535 0.9238];
fvtool(B4,A4)
```



Examining the placement of the poles, you can see this AR(4) process is also stable. The four poles are inside the unit circle.

```
fvtool(B4,A4,'Analysis','polezero')
```



The dynamic range of this PSD is approximately 65 dB, much larger than the AR(2) model.

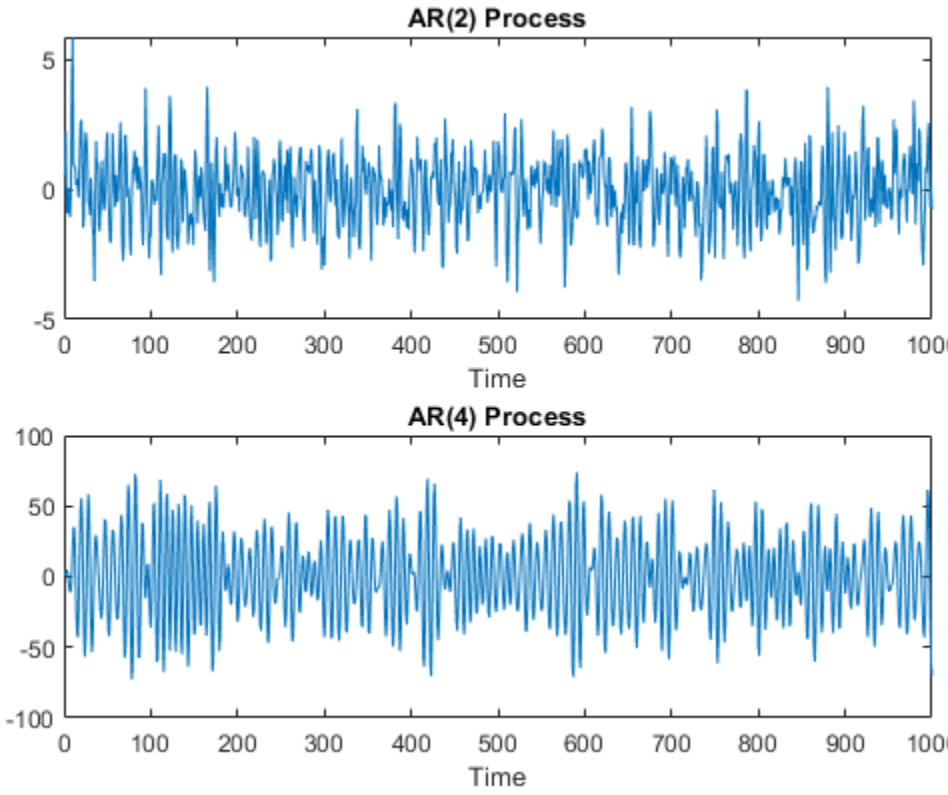
```
[H4,W4] = freqz(B4,A4,1e3,1);
dr4 = max(20*log10(abs(H4)))-min(20*log10(abs(H4)))
```

```
dr4 = 64.6213
```

To simulate realizations from these AR(*p*) processes, use `randn` and `filter`. Set the random number generator to the default settings to produce repeatable results. Plot the realizations.

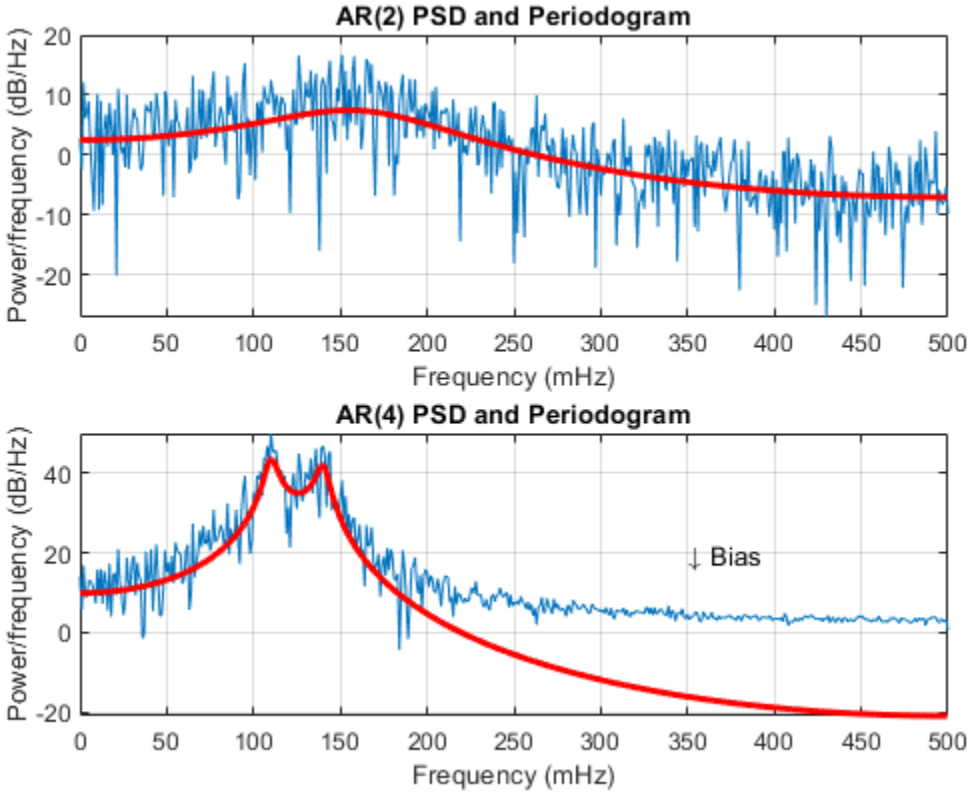
```
rng default
x = randn(1e3,1);
```

```
y2 = filter(B2,A2,x);
y4 = filter(B4,A4,x);

subplot(2,1,1)
plot(y2)
title('AR(2) Process')
xlabel('Time')

subplot(2,1,2)
plot(y4)
title('AR(4) Process')
xlabel('Time')
```



Compute and plot the periodograms of the AR(2) and AR(4) realizations. Compare the results against the true PSD. Note that `periodogram` converts the frequencies to millihertz for plotting.

```
Fs = 1;
NFFT = length(y2);

subplot(2,1,1)
periodogram(y2,rectwin(NFFT),NFFT,Fs)
hold on
plot(1000*W2,20*log10(abs(H2)),'r','linewidth',2)
title('AR(2) PSD and Periodogram')

subplot(2,1,2)
periodogram(y4,rectwin(NFFT),NFFT,Fs)
hold on
plot(1000*W4,20*log10(abs(H4)),'r','linewidth',2)
title('AR(4) PSD and Periodogram')
text(350,20,'\downarrow Bias')
```

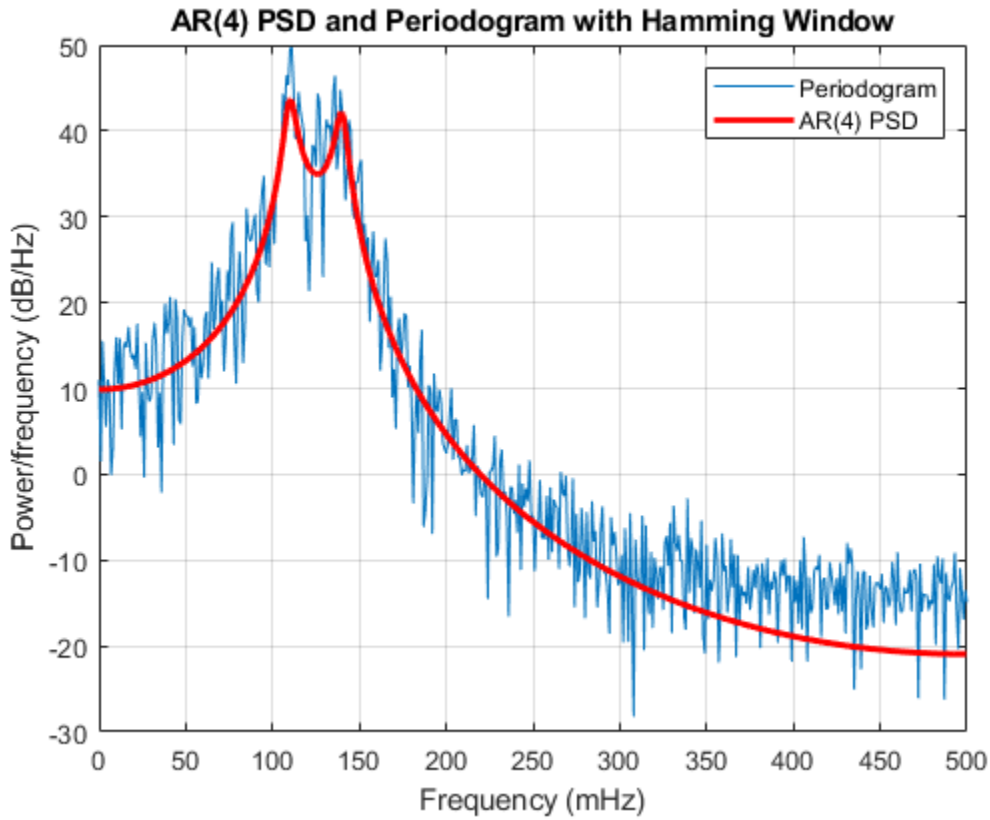


In the case of the AR(2) process, the periodogram estimate follows the shape of the true PSD but exhibits considerable variability. This is due to the low degrees of freedom. The pronounced negative deflections (in dB) in the periodogram are explained by taking the log of a chi-square random variable with two degrees of freedom.

In the case of the AR(4) process, the periodogram follows the shape of the true PSD at low frequencies but deviates from the PSD in the high frequencies. This is the effect of the convolution with Fejer's kernel. The large dynamic range of the AR(4) process compared to the AR(2) process is what makes the bias more pronounced.

Mitigate the bias demonstrated in the AR(4) process by using a taper, or window. In this example, use a Hamming window to taper the AR(4) realization before obtaining the periodogram.

```
figure
periodogram(y4,hamming(length(y4)),NFFT,Fs)
hold on
plot(1000*W4,20*log10(abs(H4)),'r','linewidth',2)
title('AR(4) PSD and Periodogram with Hamming Window')
legend('Periodogram','AR(4) PSD')
```

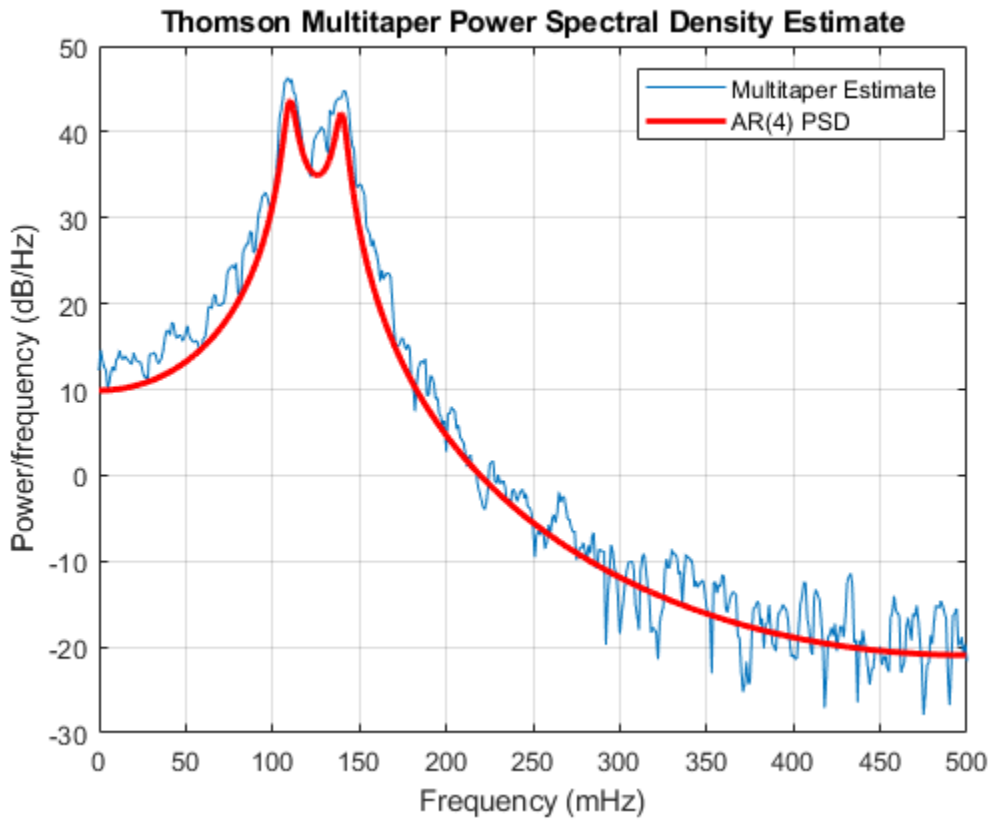


Note that the periodogram estimate now follows the true AR(4) PSD over the entire Nyquist frequency range. The periodogram estimates still only have two degrees of freedom so the use of a window does not reduce the variability of periodogram, but it does address bias.

In nonparametric spectral estimation, two methods for increasing the degrees of freedom and reducing the variability of the periodogram are Welch's overlapped segment averaging and multitaper spectral estimation.

Obtain a multitaper estimate of the AR(4) time series using a time half bandwidth product of 3.5. Plot the result.

```
NW = 3.5;

figure
pmtm(y4,NW,NFFT,Fs)
hold on
plot(1000*W4,20*log10(abs(H4)),'r','linewidth',2)
legend('Multitaper Estimate','AR(4) PSD')
```



The multitaper method produces a PSD estimate with significantly less variability than the periodogram. Because the multitaper method also uses windows, you see that the bias of the periodogram is also addressed.

See Also

`periodogram` | `pmtm`

Cross Spectrum and Magnitude-Squared Coherence

This example shows how to use the cross spectrum to obtain the phase lag between sinusoidal components in a bivariate time series. The example also uses the magnitude-squared coherence to identify significant frequency-domain correlation at the sine wave frequencies.

Create the bivariate time series. The individual series consist of two sine waves with frequencies of 100 and 200 Hz. The series are embedded in additive white Gaussian noise and sampled at 1 kHz. The sine waves in the *x*-series both have amplitudes equal to 1. The 100 Hz sine wave in the *y*-series has amplitude 0.5, and the 200 Hz sine wave in the *y*-series has amplitude 0.35. The 100 Hz and 200 Hz sine waves in the *y*-series are phase-lagged by $\pi/4$ radians and $\pi/2$ radians, respectively. You can think of the *y*-series as the noise-corrupted output of a linear system with input *x*. Set the random number generator to the default settings for reproducible results.

```
rng default

Fs = 1000;
t = 0:1/Fs:1-1/Fs;

x = cos(2*pi*100*t) + sin(2*pi*200*t) + 0.5*randn(size(t));
y = 0.5*cos(2*pi*100*t - pi/4) + 0.35*sin(2*pi*200*t - pi/2) + 0.5*randn(size(t));
```

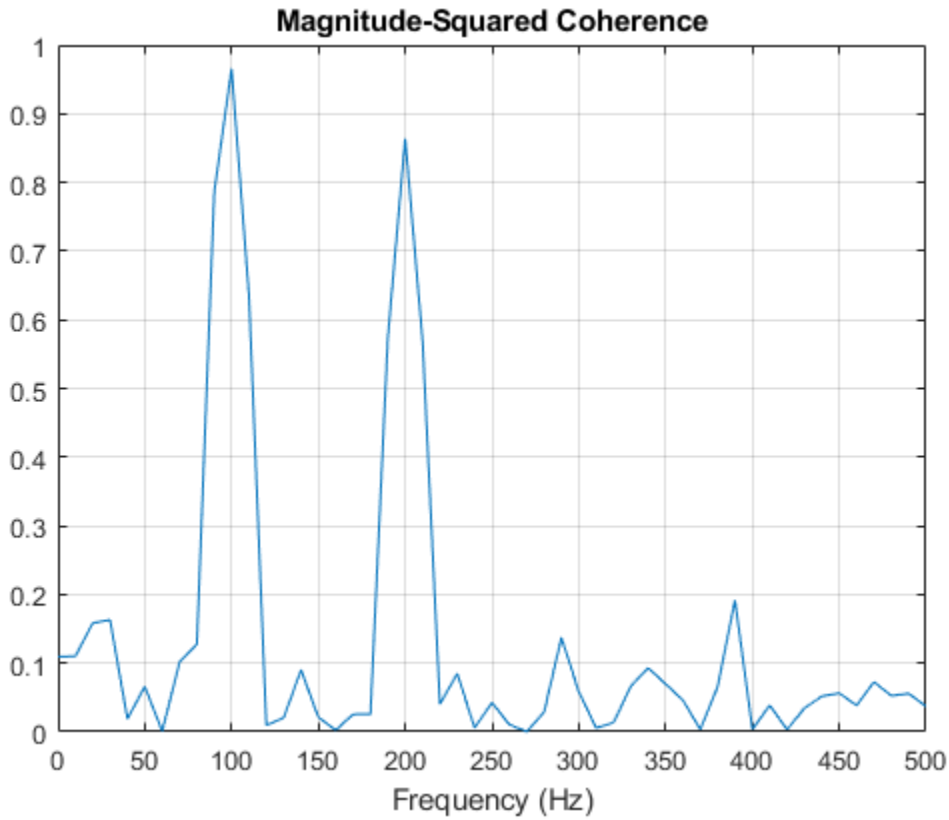
Obtain the magnitude-squared coherence estimate for the bivariate time series. The magnitude-squared coherence enables you to identify significant frequency-domain correlation between the two time series. Phase estimates in the cross spectrum are only useful where significant frequency-domain correlation exists.

To prevent obtaining a magnitude-squared coherence estimate that is identically 1 for all frequencies, you must use an averaged coherence estimator. Both Welch's overlapped segment averaging (WOSA) and multitaper techniques are appropriate. `mscohere` implements a WOSA estimator.

Set the window length to 100 samples. This window length contains 10 periods of the 100 Hz sine wave and 20 periods of the 200 Hz sine wave. Use an overlap of 80 samples with the default Hamming window. Input the sample rate explicitly to get the output frequencies in Hz. Plot the magnitude-squared coherence. The magnitude-squared coherence is greater than 0.8 at 100 and 200 Hz.

```
[Cxy,F] = mscohere(x,y,hamming(100),80,100,Fs);
```

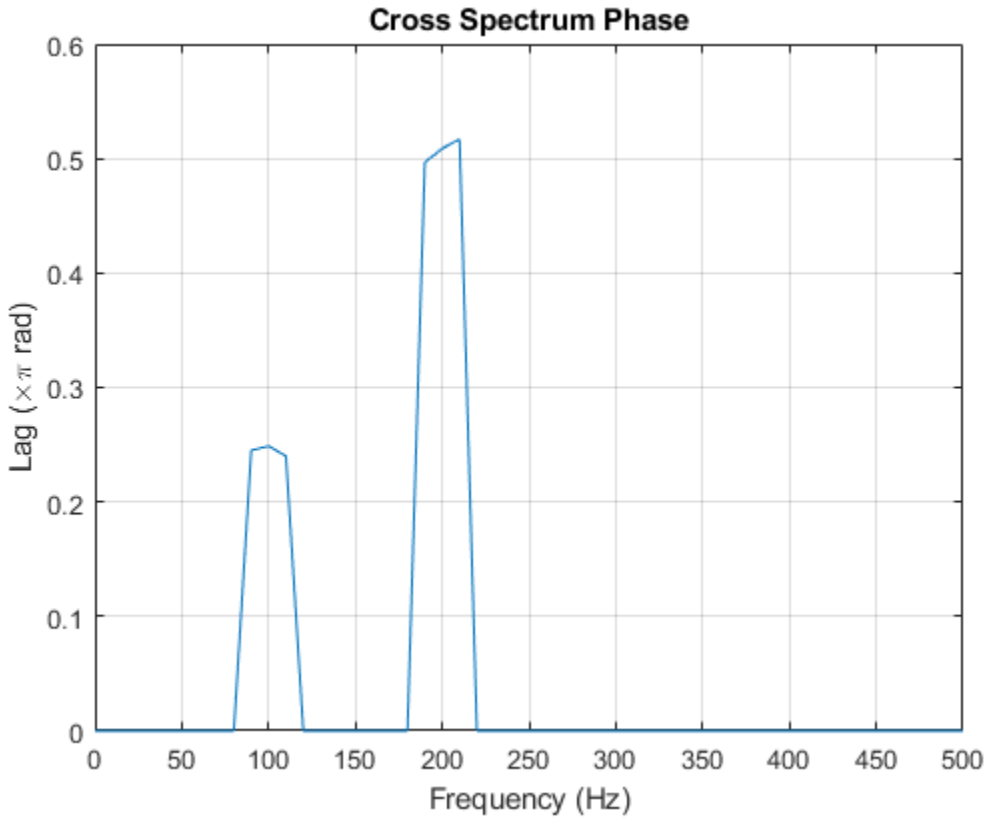
```
plot(F,Cxy)
title('Magnitude-Squared Coherence')
xlabel('Frequency (Hz)')
grid
```



Obtain the cross spectrum of *x* and *y* using `cpsd`. Use the same parameters to obtain the cross spectrum that you used in the coherence estimate. Neglect the cross spectrum when the coherence is small. Plot the phase of the cross spectrum and indicate the frequencies with significant coherence between the two times. Mark the known phase lags between the sinusoidal components. At 100 Hz and 200 Hz, the phase lags estimated from the cross spectrum are close to the true values.

```
[Pxy,F] = cpsd(x,y,hamming(100),80,100,Fs);
```

```
Pxy(Cxy < 0.2) = 0;

plot(F,angle(Pxy)/pi)
title('Cross Spectrum Phase')
xlabel('Frequency (Hz)')
ylabel('Lag (\times\pi rad)')
grid
```



See Also

cpsd | mscohere | pwelch

Amplitude Estimation and Zero Padding

This example shows how to use zero padding to obtain an accurate estimate of the amplitude of a sinusoidal signal. Frequencies in the discrete Fourier transform (DFT) are spaced at intervals of F_s/N, where F_s is the sample rate and N is the length of the input time series. Attempting to estimate the amplitude of a sinusoid with a frequency that does not correspond to a DFT *bin* can result in an inaccurate estimate. Zero padding the data before computing the DFT often helps to improve the accuracy of amplitude estimates.

Create a signal consisting of two sine waves. The two sine waves have frequencies of 100 and 202.5 Hz. The sample rate is 1000 Hz and the signal is 1000 samples in length.

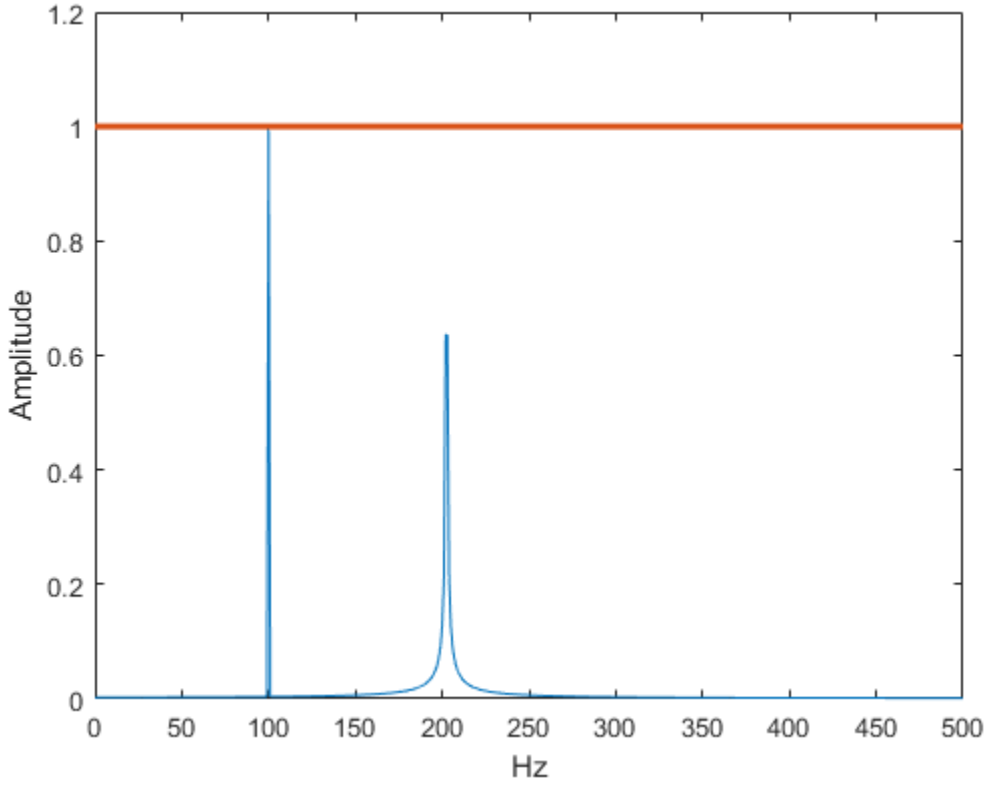
```
Fs = 1e3;
t = 0:0.001:1-0.001;
x = cos(2*pi*100*t)+sin(2*pi*202.5*t);
```

Obtain the DFT of the signal. The DFT bins are spaced at 1 Hz. Accordingly, the 100 Hz sine wave corresponds to a DFT bin, but the 202.5 Hz sine wave does not.

Because the signal is real-valued, use only the positive frequencies from the DFT to estimate the amplitude. Scale the DFT by the length of the input signal and multiply all frequencies except 0 and the Nyquist by 2.

Plot the result with the known amplitudes for comparison.

```
xdft = fft(x);
xdft = xdft(1:length(x)/2+1);
xdft = xdft/length(x);
xdft(2:end-1) = 2*xdft(2:end-1);
freq = 0:Fs/length(x):Fs/2;

plot(freq,abs(xdft))
hold on
plot(freq,ones(length(x)/2+1,1),'LineWidth',2)
xlabel('Hz')
ylabel('Amplitude')
hold off
```



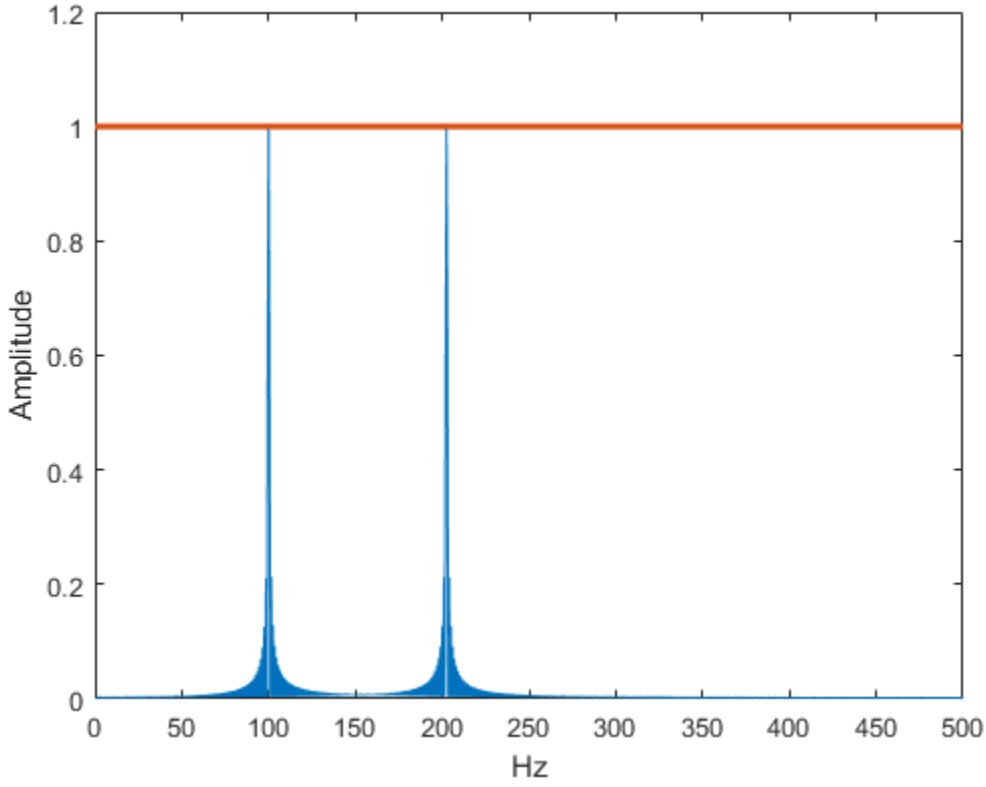
The amplitude estimate at 100 Hz is accurate because that frequency corresponds to a DFT bin. However, the amplitude estimate at 202.5 Hz is not accurate because that frequency does not correspond to a DFT bin.

You can interpolate the DFT by zero padding. Zero padding enables you to obtain more accurate amplitude estimates of resolvable signal components. On the other hand, zero padding does not improve the spectral (frequency) resolution of the DFT. The resolution is determined by the number of samples and the sample rate.

Pad the DFT out to 2000, or twice the original length of `x`. With this length, the spacing between DFT bins is $F_s/2000 = 0.5$ Hz. In this case, the energy from the 202.5 Hz sine

wave falls directly in a DFT bin. Obtain the DFT and plot the amplitude estimates. Use zero padding out to 2000 samples.

```
lpad = 2*length(x);
xdft = fft(x,lpad);
xdft = xdft(1:lpad/2+1);
xdft = xdft/length(x);
xdft(2:end-1) = 2*xdft(2:end-1);
freq = 0:Fs/lpad:Fs/2;

plot(freq,abs(xdft))
hold on
plot(freq,ones(2*length(x)/2+1,1),'LineWidth',2)
xlabel('Hz')
ylabel('Amplitude')
hold off
```



The use of zero padding enables you to estimate the amplitudes of both frequencies correctly.

See Also

`fft`

Significance Testing for Periodic Component

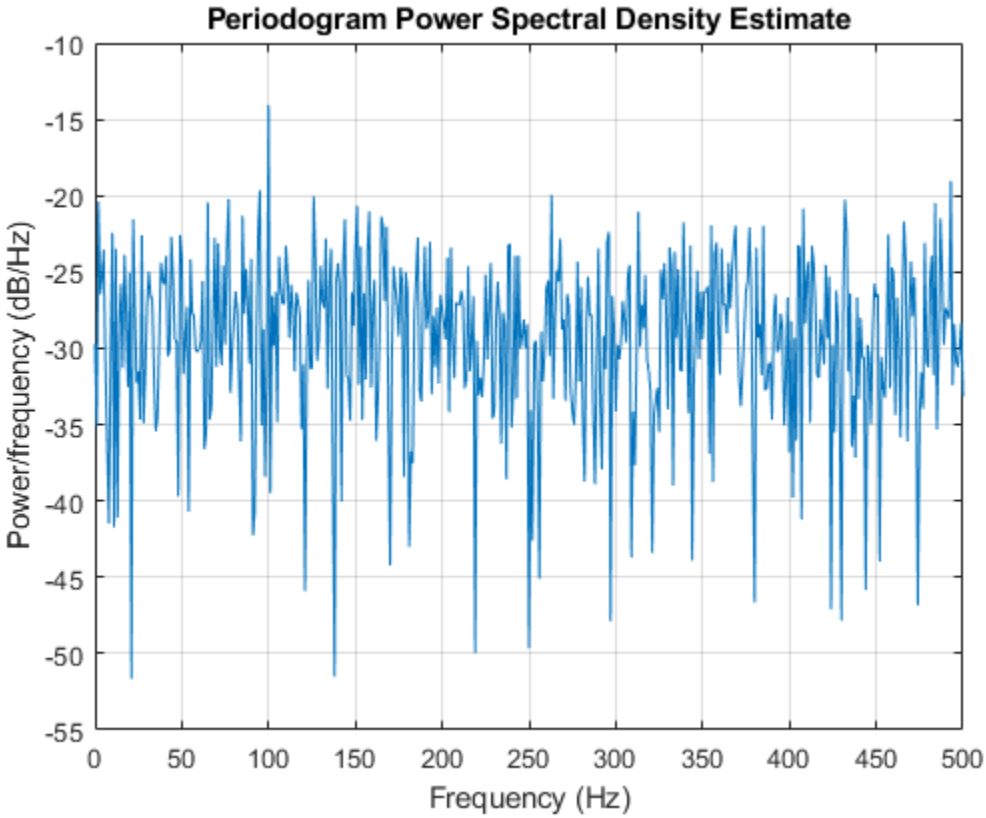
This example shows how to assess the significance of a sinusoidal component in white noise using Fisher's *g*-statistic. Fisher's *g*-statistic is the ratio of the largest periodogram value to the sum of all the periodogram values over 1/2 of the frequency interval, (0, `Fs`/2). A detailed description of the *g*-statistic and exact distribution can be found in the references.

Create a signal consisting of a 100 Hz sine wave in white Gaussian noise with zero mean and variance 1. The amplitude of the sine wave is 0.25. The sample rate is 1 kHz. Set the random number generator to the default settings for reproducible results.

```
rng default

Fs = 1e3;
t = 0:1/Fs:1-1/Fs;
x = 0.25*cos(2*pi*100*t)+randn(size(t));
```

Obtain the periodogram of the signal using `periodogram`. Exclude 0 and the Nyquist frequency (`Fs`/2). Plot the periodogram.

```
[Pxx,F] = periodogram(x,rectwin(length(x)),length(x),Fs);
Pxx = Pxx(2:length(x)/2);

periodogram(x,rectwin(length(x)),length(x),Fs)
```



Find the maximum value of the periodogram. Fisher's *g*-statistic is the ratio of the maximum periodogram value to the sum of all periodogram values.

```
[maxval,index] = max(Pxx);
fisher_g = Pxx(index)/sum(Pxx)
```

```
fisher_g = 0.0381
```

The maximum periodogram value occurs at 100 Hz, which you can verify by finding the frequency corresponding to the index of the maximum periodogram value.

```
F = F(2:end-1);
F(index)
```

```
ans = 100
```

Use the distributional results detailed in the references to determine the significance level, `pval`, of Fisher's *g*-statistic. The following MATLAB® code implements equation (6) of [2].

```
N = length(Pxx);
    upper  = floor(1/fisher_g);
    for nn = 1:3
        I(nn) = (-1)^(nn-1)*nchoosek(N,nn)*(1-nn*fisher_g)^(N-1);
    end
pval = sum(I)
```

```
pval = 2.0163e-06
```

The *p*-value is less than 0.00001, which indicates a significant periodic component at 100 Hz. The interpretation of Fisher's *g*-statistic is complicated by the presence of other periodicities. See [1] for a modification when multiple periodicities may be present.

References

[1] Percival, Donald B. and Andrew T. Walden. *Spectral Analysis for Physical Applications*. Cambridge, UK: Cambridge University Press, 1993.

[2] Wichert, Sofia, Konstantinos Fokianos, and Korbinian Strimmer. "Identifying Periodically Expressed Transcripts in Microarray Time Series Data." *Bioinformatics*. Vol. 20, 2004, pp. 5-20.

See Also

`nchoosek` | `periodogram`

Frequency Estimation by Subspace Methods

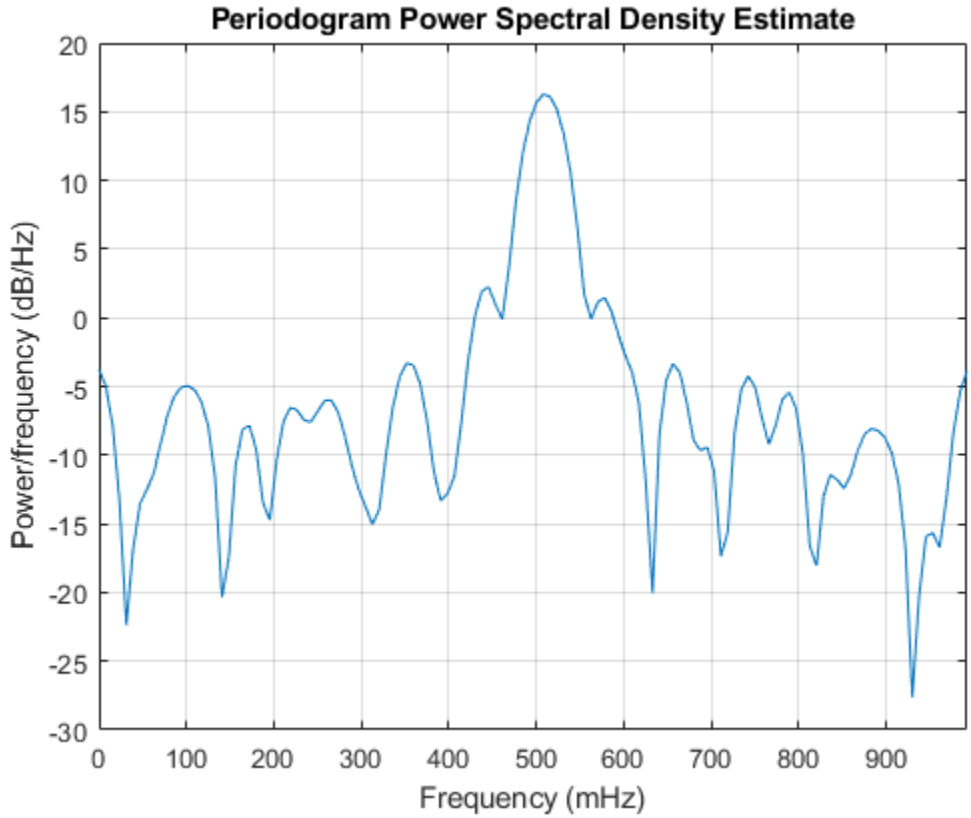
This example shows how to resolve closely spaced sine waves using subspace methods. Subspace methods assume a harmonic model consisting of a sum of sine waves, possibly complex, in additive noise. In a complex-valued harmonic model, the noise is also complex-valued.

Create a complex-valued signal 24 samples in length. The signal consists of two complex exponentials (sine waves) with frequencies of 0.50 Hz and 0.52 Hz and additive complex white Gaussian noise. The noise has zero mean and variance 0.2^2. In a complex white noise, both the real and imaginary parts have variance equal to 1/2 the overall variance.

```
n = 0:23;
rng default
x = exp(1j*2*pi*0.5*n)+exp(1j*2*pi*0.52*n)+ ...
    0.2/sqrt(2)*(randn(size(n))+1j*randn(size(n)));
```

Using `periodogram`, attempt to resolve the two sine waves.

```
periodogram(x,rectwin(length(x)),128,1)
```

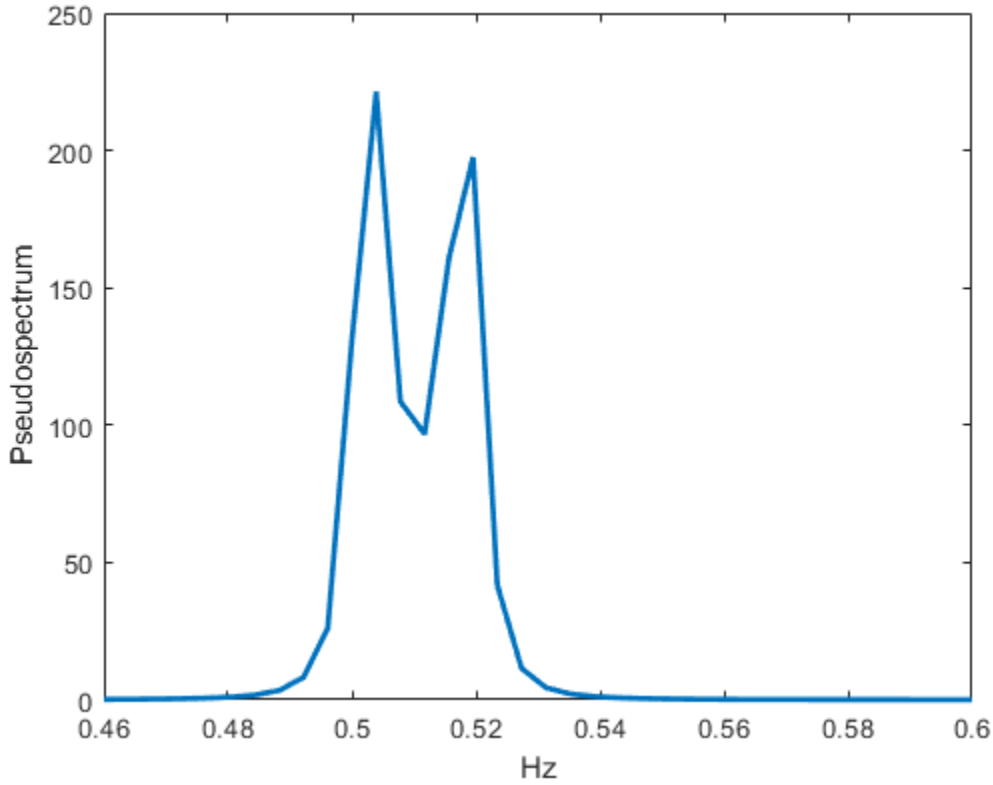


The periodogram shows a broad peak near 1/2 Hz. You cannot resolve the two separate sine waves because the frequency resolution of the periodogram is 1/_N_, where *N* is the length of the signal. In this case, 1/_N_ is greater than the separation of the two sine waves. Zero padding does not help to resolve two separate peaks.

Use a subspace method to resolve the two closely spaced peaks. In this example, use the root-MUSIC method. Estimate the autocorrelation matrix and input the autocorrelation matrix into `pmusic`. Specify a model with two sinusoidal components. Plot the result.

```
[X,R] = corrmtx(x,14,'mod');
[S,F] = pmusic(R,2,[],1,'corr');

plot(F,S,'linewidth',2)
xlim([0.46 0.60])
```

```
xlabel('Hz')
ylabel('Pseudospectrum')
```



The root-MUSIC method is able to separate the two peaks at 0.5 and 0.52 Hz. However, subspace methods do not produce power estimates like power spectral density estimates. Subspace methods are most useful for frequency identification and can be sensitive to model-order misspecification.

See Also

`corrmtx` | `periodogram` | `pmusic`

Frequency-Domain Linear Regression

This example shows how to use the discrete Fourier transform to construct a linear regression model for a time series. The time series used in this example is the monthly number of accidental deaths in the United States from 1973 to 1979. The data are published in Brockwell and Davis (2006). The original source is the U. S. National Safety Council.

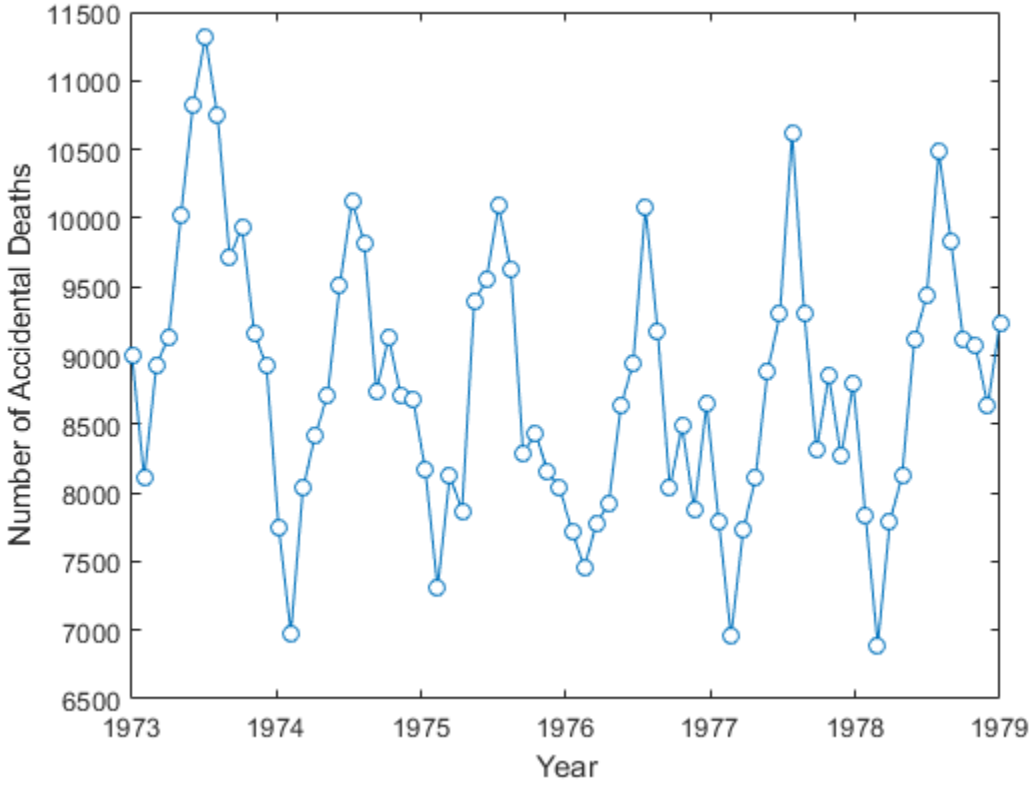
Enter the data. Copy the `exdata` matrix into the MATLAB® workspace.

```
exdata = [
        9007        7750        8162        7717        7792        7836
        8106        6981        7306        7461        6957        6892
        8928        8038        8124        7776        7726        7791
        9137        8422        7870        7925        8106        8129
       10017        8714        9387        8634        8890        9115
       10826        9512        9556        8945        9299        9434
       11317       10120       10093       10078       10625       10484
       10744        9823        9620        9179        9302        9827
        9713        8743        8285        8037        8314        9110
        9938        9129        8433        8488        8850        9070
        9161        8710        8160        7874        8265        8633
        8927        8680        8034        8647        8796        9240];
```

`exdata` is a 12-by-6 matrix. Each column of exdata contains 12 months of data. The first row of each column contains the number of U.S. accidental deaths for January of the corresponding year. The last row of each column contains the number of U.S. accidental deaths for December of the corresponding year.

Reshape the data matrix into a 72-by-1 time series and plot the data for the years 1973 to 1978.

```
ts = reshape(exdata,72,1);
years = linspace(1973,1979,72);

plot(years,ts,'o-','MarkerFaceColor','auto')
xlabel('Year')
ylabel('Number of Accidental Deaths')
```



A visual inspection of the data indicates that number of accidental deaths varies in a periodic manner. The period of the oscillation appears to be roughly 1 year (12 months). The periodic nature of the data suggests that an appropriate model may be

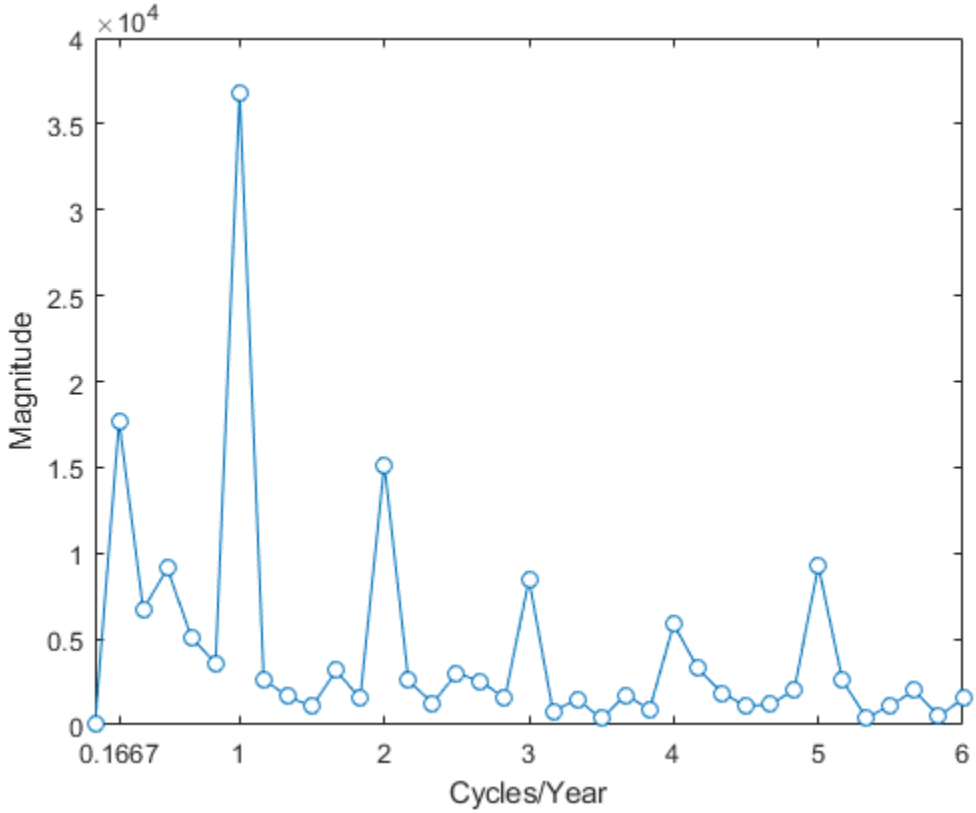
$$X(n) = \mu + \sum_k \left(A_k \cos \frac{2\pi kn}{N} + B_k \cos \frac{2\pi kn}{N} \right) + \varepsilon(n),$$

where μ is the overall mean, N is the length of the time series, and $\varepsilon(n)$ is a white noise sequence of independent and identically-distributed (iid) Gaussian random variables with zero mean and some variance. The additive noise term accounts for the randomness inherent in the data. The parameters of the model are the overall mean and the amplitudes of the cosines and sines. The model is linear in the parameters.

To construct a linear regression model in the time domain, you have to specify which frequencies to use for the cosines and sines, form the design matrix, and solve the normal equations in order to obtain the least-squares estimates of the model parameters. In this case, it is easier to use the discrete Fourier transform to detect the periodicities, retain only a subset of the Fourier coefficients, and invert the transform to obtain the fitted time series.

Perform a spectral analysis of the data to reveal which frequencies contribute significantly to the variability in the data. Because the overall mean of the signal is approximately 9,000 and is proportional to the Fourier transform at 0 frequency, subtract the mean prior to the spectral analysis. This reduces the large magnitude Fourier coefficient at 0 frequency and makes any significant oscillations easier to detect. The frequencies in the Fourier transform are spaced at an interval that is the reciprocal of the time series length, 1/72. Sampling the data monthly, the highest frequency in the spectral analysis is 1 cycle/2 months. In this case, it is convenient to look at the spectral analysis in terms of cycles/year so scale the frequencies accordingly for visualization.

```
tsdft = fft(ts-mean(ts));
freq = 0:1/72:1/2;

plot(freq.*12,abs(tsdft(1:length(ts)/2+1)),'o-', ...
    'MarkerFaceColor','auto')
xlabel('Cycles/Year')
ylabel('Magnitude')
ax = gca;
ax.XTick = [1/6 1 2 3 4 5 6];
```



Based on the magnitudes, the frequency of 1 cycle/12 months is the most significant oscillation in the data. The magnitude at 1 cycle/12 months is more than twice as large as any other magnitude. However, the spectral analysis reveals that there are also other periodic components in the data. For example, there appears to be periodic components at harmonics (integer multiples) of 1 cycle/12 months. There also appears to be a periodic component with a period of 1 cycle/72 months.

Based on the spectral analysis of the data, fit a simple linear regression model using a cosine and sine term with a frequency of the most significant component: 1 cycle/year (1 cycle/12 months).

Determine the frequency bin in the discrete Fourier transform that corresponds to 1 cycle/12 months. Because the frequencies are spaced at 1/72 and the first bin

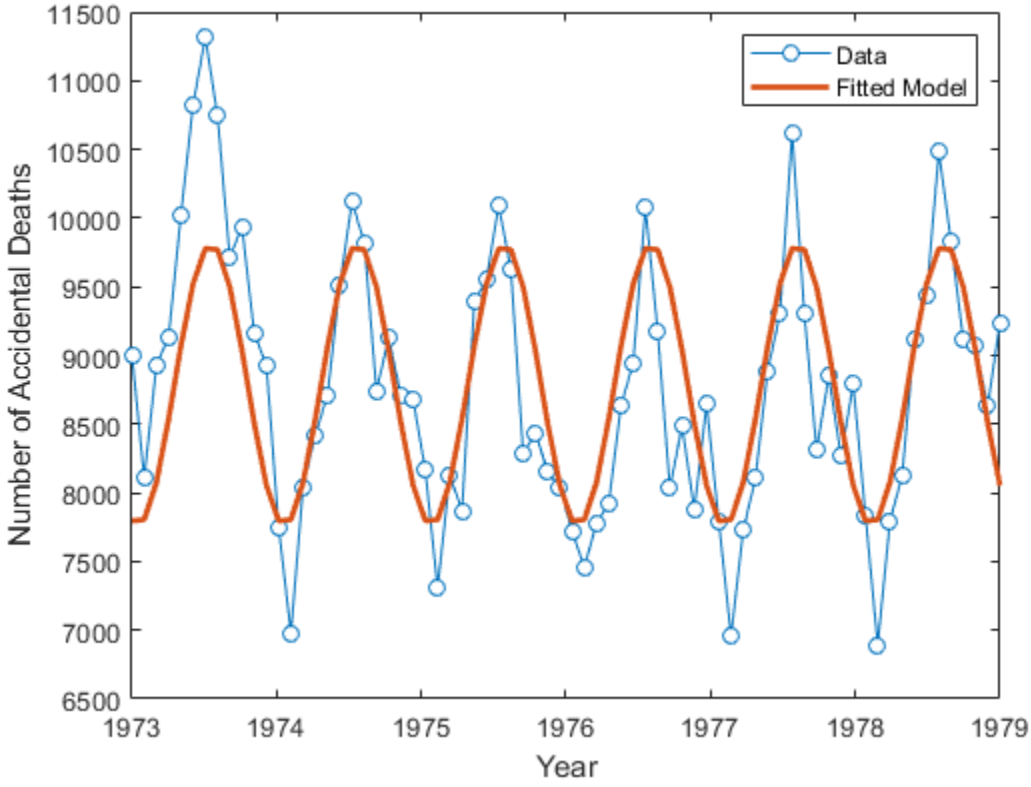
corresponds to 0 frequency, the correct bin is 72/12+1. This is the frequency bin of the *positive* frequency. You must also include the frequency bin corresponding to the *negative* frequency: -1 cycle/12 months. With MATLAB indexing, the frequency bin of the negative frequency is 72-72/12+1.

Create a 72-by-1 vector of zeros. Fill the appropriate elements of the vector with the Fourier coefficients corresponding to a positive and negative frequency of 1 cycle/12 months. Invert the Fourier transform and add the overall mean to obtain a fit to the accidental death data.

```
freqbin = 72/12;
freqbins = [freqbin 72-freqbin]+1;
tsfit = zeros(72,1);
tsfit(freqbins) = tsdft(freqbins);
tsfit = ifft(tsfit);
mu = mean(ts);
tsfit = mu+tsfit;
```

Plot the original data along with the fitted series using two Fourier coefficients.

```
plot(years,ts,'o-','MarkerFaceColor','auto')
xlabel('Year')
ylabel('Number of Accidental Deaths')
hold on
plot(years,tsfit,'linewidth',2)
legend('Data','Fitted Model')
hold off
```



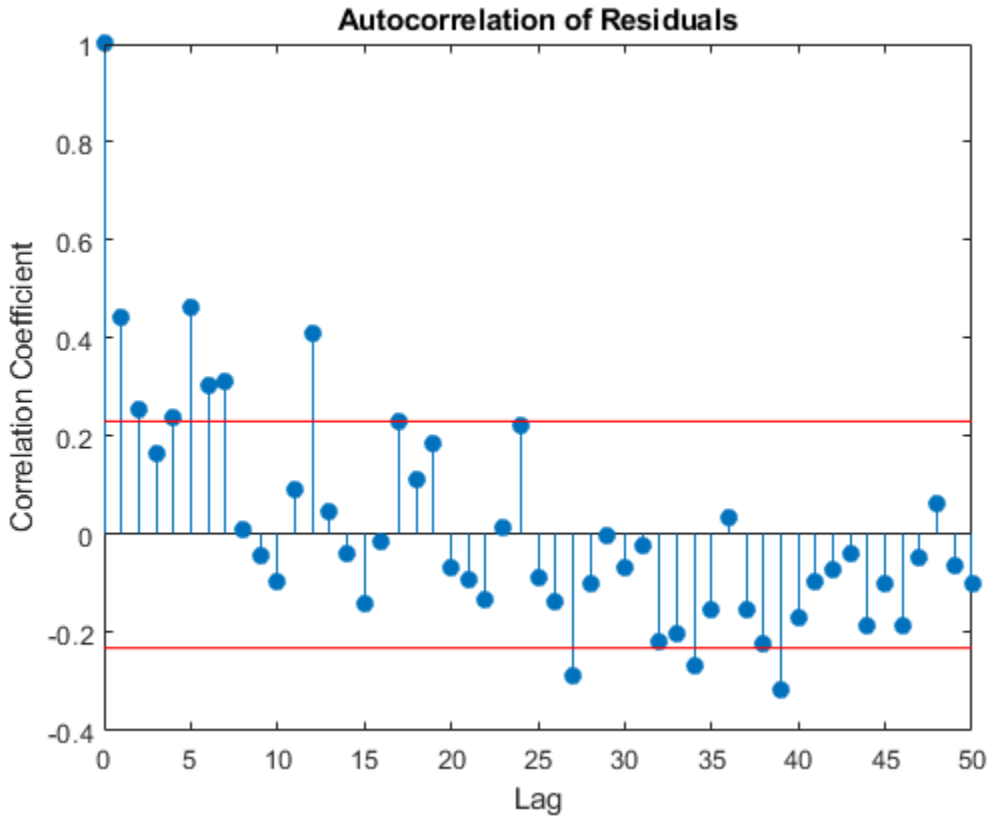
The fitted model appears to capture the general periodic nature of the data and supports the initial conclusion that data oscillate with a cycle of 1 year.

To assess how adequately the single frequency of 1 cycle/12 months accounts for the observed time series, form the residuals. If the residuals resemble a white noise sequence, the simple linear model with one frequency has adequately modeled the time series.

To assess the residuals, use the autocorrelation sequence with 95%-confidence intervals for a white noise.

```
resid = ts-tsfit;
[xc,lags] = xcorr(resid,50,'coeff');
```

```
stem(lags(51:end),xc(51:end),'filled')
hold on
lconf = -1.96*ones(51,1)/sqrt(72);
uconf = 1.96*ones(51,1)/sqrt(72);
plot(lags(51:end),lconf,'r')
plot(lags(51:end),uconf,'r')
xlabel('Lag')
ylabel('Correlation Coefficient')
title('Autocorrelation of Residuals')
hold off
```



The autocorrelation values fall outside the 95% confidence bounds at a number of lags. It does not appear that the residuals are white noise. The conclusion is that the simple linear model with one sinusoidal component does not account for all the oscillations in the

number of accidental deaths. This is expected because the spectral analysis revealed additional periodic components in addition to the dominant oscillation. Creating a model that incorporates additional periodic terms indicated by the spectral analysis will improve the fit and whiten the residuals.

Fit a model which consists of the three largest Fourier coefficient magnitudes. Because you have to retain the Fourier coefficients corresponding to both negative and positive frequencies, retain the largest 6 indices.

```
tsfit2dft = zeros(72,1);
[Y,I] = sort(abs(tsdft),'descend');
indices = I(1:6);
tsfit2dft(indices) = tsdft(indices);
```

Demonstrate that preserving only 6 of the 72 Fourier coefficients (3 frequencies) retains most of the signal's energy. First, demonstrate that retaining all the Fourier coefficients yields energy equivalence between the original signal and the Fourier transform.

```
norm(1/sqrt(72)*tsdft,2)/norm(ts-mean(ts),2)
```

```
ans = 1.0000
```

The ratio is 1. Now, examine the energy ratio where only 3 frequencies are retained.

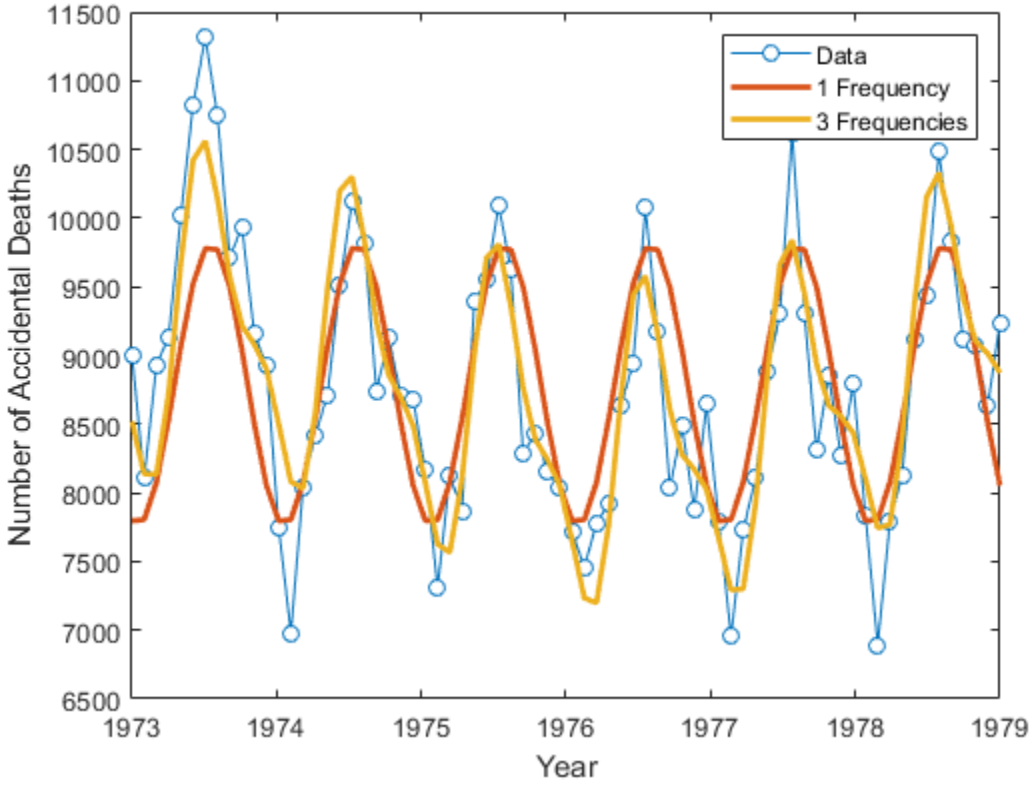
```
norm(1/sqrt(72)*tsfit2dft,2)/norm(ts-mean(ts),2)
```

```
ans = 0.8991
```

Almost 90% of the energy is retained. Equivalently, 90% of the variance of the time series is accounted for by 3 frequency components.

Form an estimate of the data based on 3 frequency components. Compare the original data, the model with one frequency, and the model with 3 frequencies.

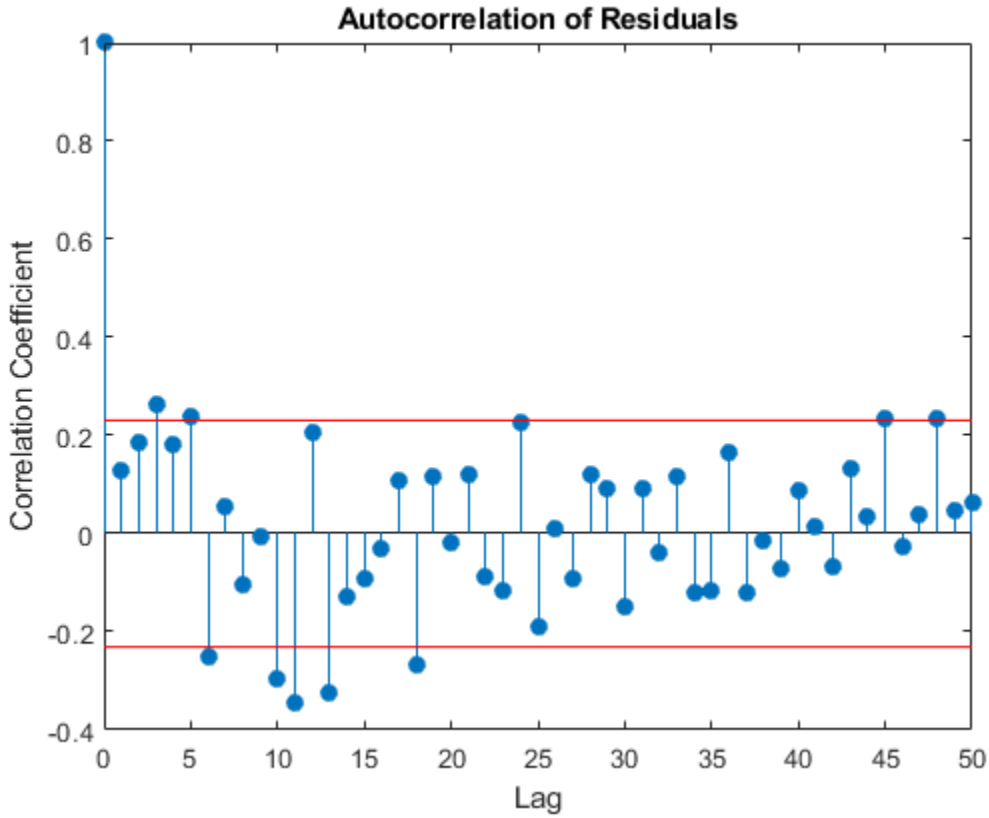
```
tsfit2 = mu+ifft(tsfit2dft,'symmetric');

plot(years,ts,'o-','markerfacecolor','auto')
xlabel('Year')
ylabel('Number of Accidental Deaths')
hold on
plot(years,tsfit,'linewidth',2)
plot(years,tsfit2,'linewidth',2)
legend('Data','1 Frequency','3 Frequencies')
hold off
```



Using 3 frequencies has improved the fit to the original signal. You can see this by examining the autocorrelation of the residuals from the 3-frequency model.

```
resid = ts-tsfit2;
[xc,lags] = xcorr(resid,50,'coeff');

stem(lags(51:end),xc(51:end),'filled')
hold on
lconf = -1.96*ones(51,1)/sqrt(72);
uconf = 1.96*ones(51,1)/sqrt(72);
plot(lags(51:end),lconf,'r')
plot(lags(51:end),uconf,'r')
xlabel('Lag')
ylabel('Correlation Coefficient')
```

```
title('Autocorrelation of Residuals')
hold off
```



Using 3 frequencies has resulted in residuals that more closely approximate a white noise process.

Demonstrate that the parameter values obtained from the Fourier transform are equivalent to a time-domain linear regression model. Find the least-squares estimates for the overall mean, the cosine amplitudes, and the sine amplitudes for the three frequencies by forming the design matrix and solving the normal equations. Compare the fitted time series with that obtained from the Fourier transform.

```
X = ones(72,7);
X(:,2) = cos(2*pi/72*(0:71))';
```

```
X(:,3) = sin(2*pi/72*(0:71))';
X(:,4) = cos(2*pi*6/72*(0:71))';
X(:,5) = sin(2*pi*6/72*(0:71))';
X(:,6) = cos(2*pi*12/72*(0:71))';
X(:,7) = sin(2*pi*12/72*(0:71))';
beta = X\ts;
tsfit_lm = X*beta;
max(abs(tsfit_lm-tsfit2))

ans = 7.2760e-12
```

The two methods yield identical results. The maximum absolute value of the difference between the two waveforms is on the order of 10-12. In this case, the frequency-domain approach was easier than the equivalent time-domain approach. You naturally use a spectral analysis to visually inspect which oscillations are present in the data. From that step, it is simple to use the Fourier coefficients to construct a model for the signal consisting of a sum cosines and sines.

For more details on spectral analysis in time series and the equivalence with time-domain regression see (Shumway and Stoffer, 2006).

While spectral analysis can answer which periodic components contribute significantly to the variability of the data, it does not explain why those components are present. If you examine these data closely, you see that the minimum values in the 12-month cycle tend to occur in February, while the maximum values occur in July. A plausible explanation for these data is that people are naturally more active in summer than in the winter. Unfortunately, as a result of this increased activity, there is an increased probability of the occurrence of fatal accidents.

References

Brockwell, Peter J., and Richard A. Davis. *Time Series: Theory and Methods*. New York: Springer, 2006.

Shumway, Robert H., and David S. Stoffer. *Time Series Analysis and Its Applications with R Examples*. New York: Springer, 2006.

See Also

`fft` | `ifft` | `xcorr`

Measure Total Harmonic Distortion

This example shows how to measure the total harmonic distortion (THD) of a sinusoidal signal. The example uses the following scenario: A manufacturer of audio speakers claims the model A speaker produces less than 0.09% harmonic distortion at 1 kHz with a 1 volt input. The harmonic distortion is measured with respect to the fundamental (THD-F).

Assume you record the following data obtained by driving the speaker with a 1 kHz tone at 1 volt. The data is sampled at 44.1 kHz for analysis.

```
Fs = 44.1e3;
t = 0:1/Fs:1;
x = cos(2*pi*1000*t)+8e-4*sin(2*pi*2000*t)+2e-5*cos(2*pi*3000*t-pi/4)+...
    8e-6*sin(2*pi*4000*t);
```

Obtain the total harmonic distortion of the input signal in dB. Specify that six harmonics are used in calculating the THD. This includes the fundamental frequency of 1 kHz. Input the sampling frequency of 44.1 kHz. Determine the frequencies of the harmonics and their power estimates.

```
nharm = 6;
[thd_db,harmpow,harmfreq] = thd(x,Fs,nharm);
```

The function `thd` outputs the total harmonic distortion in dB. Convert the measurement from dB to a percentage to compare the value against the manufacturer's claims.

```
percent_thd = 100*(10^(thd_db/20))
```

```
percent_thd = 0.0800
```

The value you obtain indicates that the manufacturer's claims about the THD for speaker model A are correct.

You can obtain further insight by examining the power (dB) of the individual harmonics.

```
T = table(harmfreq,harmpow,'VariableNames',{'Frequency','Power'})
```

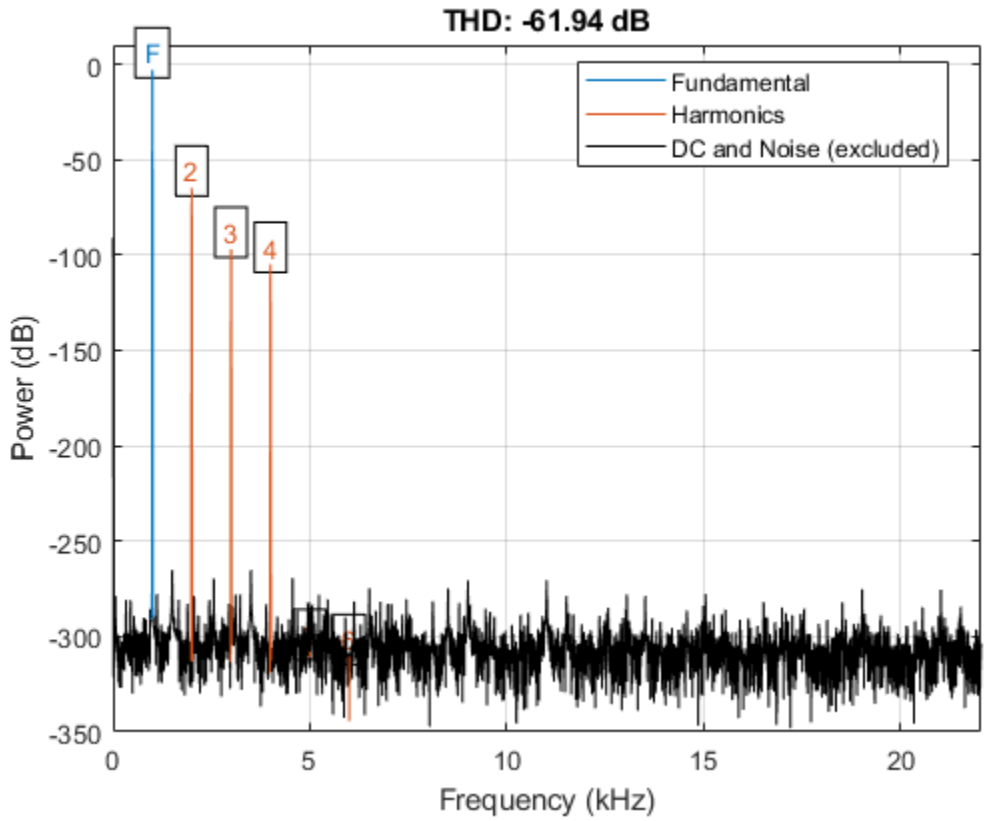
```
T=6×2 table
    Frequency     Power
    _________    _______

       1000      -3.0103
       2000      -64.949
       3000       -96.99
```

```
  4000      -104.95
4997.9      -306.11
5998.9      -310.56
```

The total harmonic distortion is approximately -62 dB. If you examine the power of the individual harmonics, you see that the major contribution comes from the harmonic at 2 kHz. The power at 2 kHz is approximately 62 dB below the power of the fundamental. The remaining harmonics do not contribute significantly to the total harmonic distortion. Additionally, the synthesized signal contains only four harmonics, including the fundamental. This is confirmed by the table, which shows a large power reduction after 4 kHz. Therefore, repeating the calculation with only four harmonics does not change the total harmonic distortion significantly.

Plot the signal spectrum, display the total harmonic distortion on the figure title, and annotate the harmonics.

```
thd(x,Fs,nharm);
```



See Also

`thd`

Related Examples

- “Analyzing Harmonic Distortion”

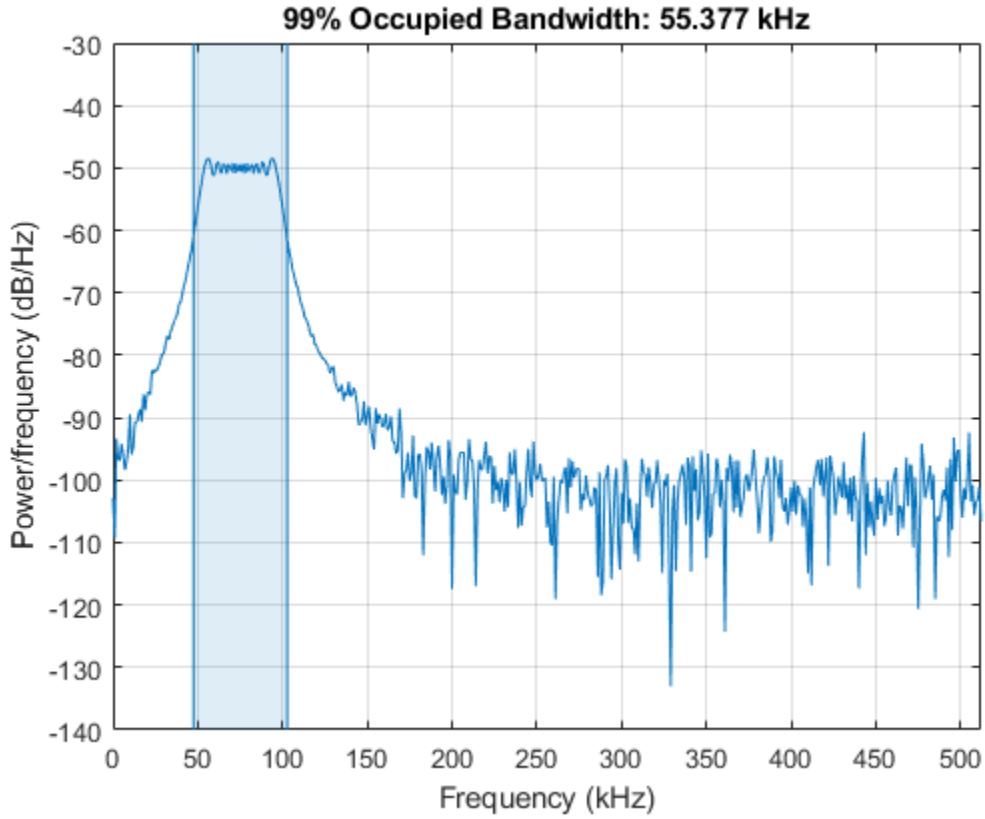
Measure Mean Frequency, Power, Bandwidth

Generate 1024 samples of a chirp sampled at 1024 kHz. The chirp has an initial frequency of 50 kHz and reaches 100 kHz at the end of the sampling. Add white Gaussian noise such that the signal-to-noise ratio is 40 dB.

```
nSamp = 1024;
Fs = 1024e3;
SNR = 40;

t = (0:nSamp-1)'/Fs;

x = chirp(t,50e3,nSamp/Fs,100e3);
x = x+randn(size(x))*std(x)/db2mag(SNR);
```

Estimate the 99% occupied bandwidth of the signal and annotate it on a plot of the power spectral density (PSD).

```
obw(x,Fs);
```



Compute the power in the band and verify that it is 99% of the total.

```
[bw,flo,fhi,powr] = obw(x,Fs);

pcent = powr/bandpower(x)*100
```

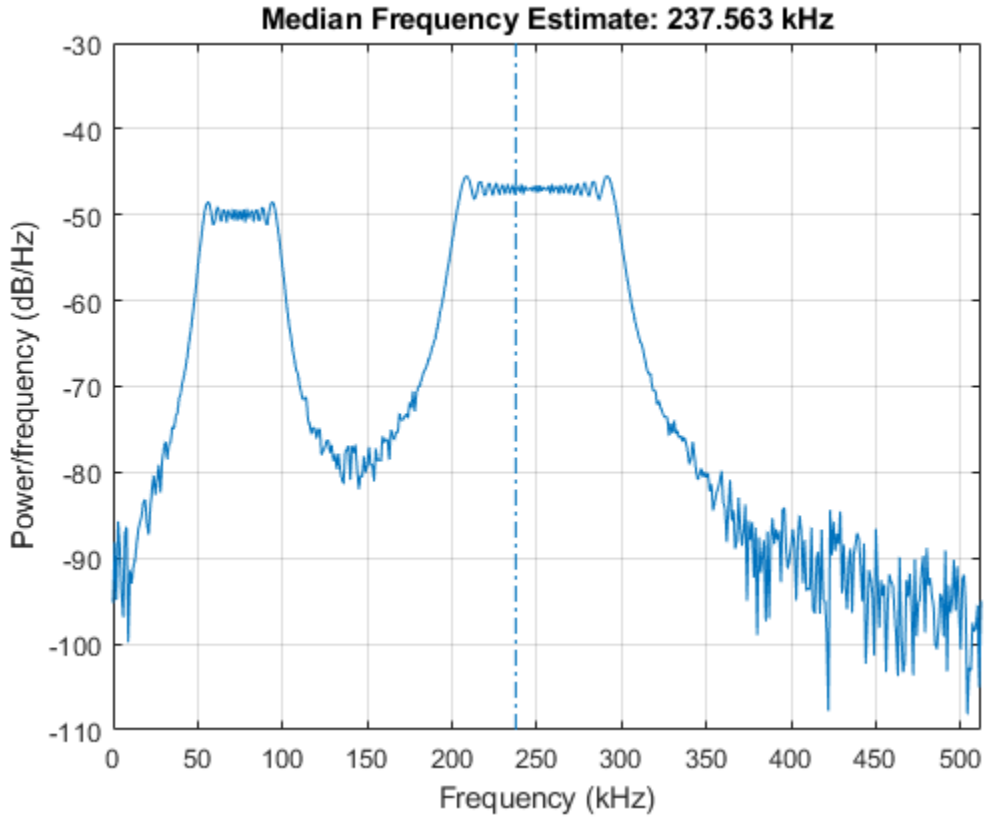
```
pcent = 99
```

Generate another chirp. Specify an initial frequency of 200 kHz, a final frequency of 300 kHz, and an amplitude that is twice that of the first signal. Add white Gaussian noise.

```
x2 = 2*chirp(t,200e3,nSamp/Fs,300e3);
x2 = x2+randn(size(x2))*std(x2)/db2mag(SNR);
```

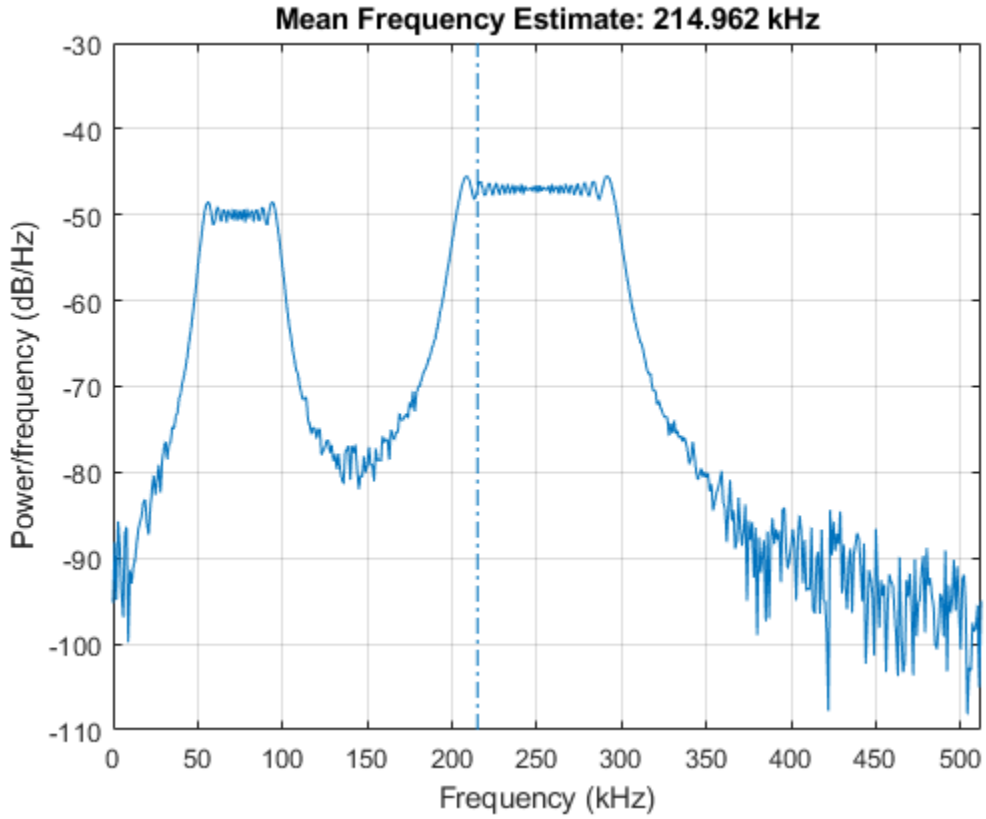
Add the two chirps to form a new signal. Plot the PSD of the signal and annotate its median frequency.

```
medfreq([x+x2],Fs);
```



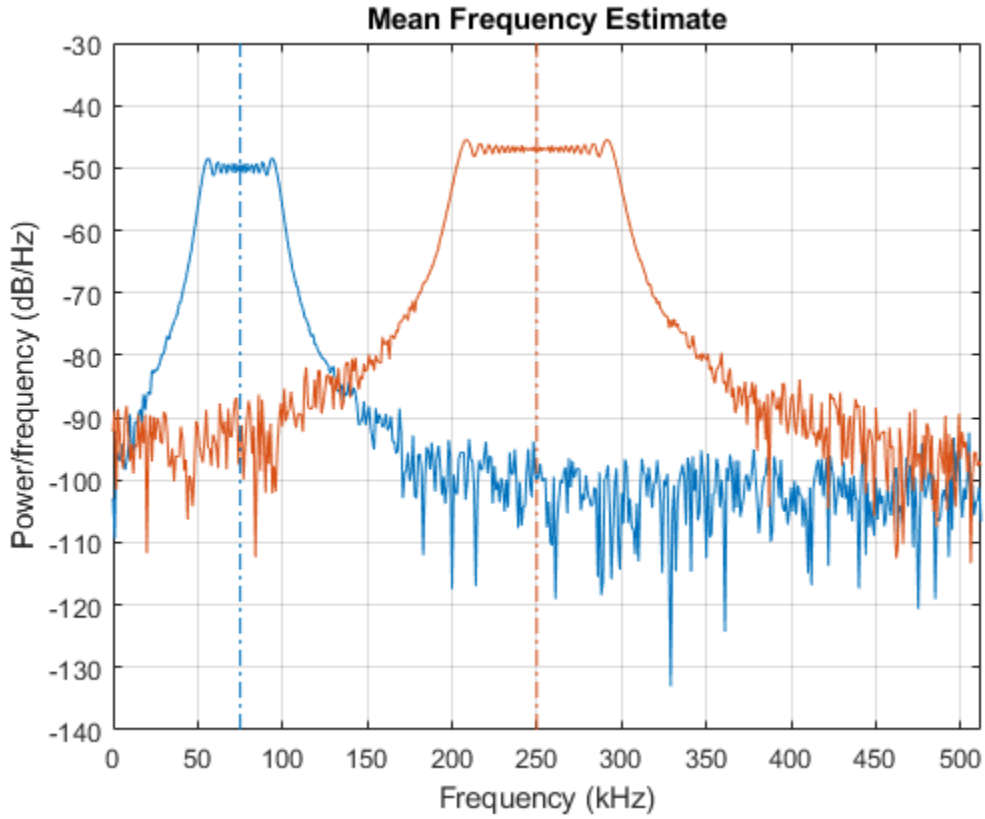
Plot the PSD and annotate the mean frequency.

```
meanfreq([x+x2],Fs);
```



Now consider each chirp to represent a separate channel. Estimate the mean frequency of each channel. Annotate the mean frequencies on a plot of the PSDs.

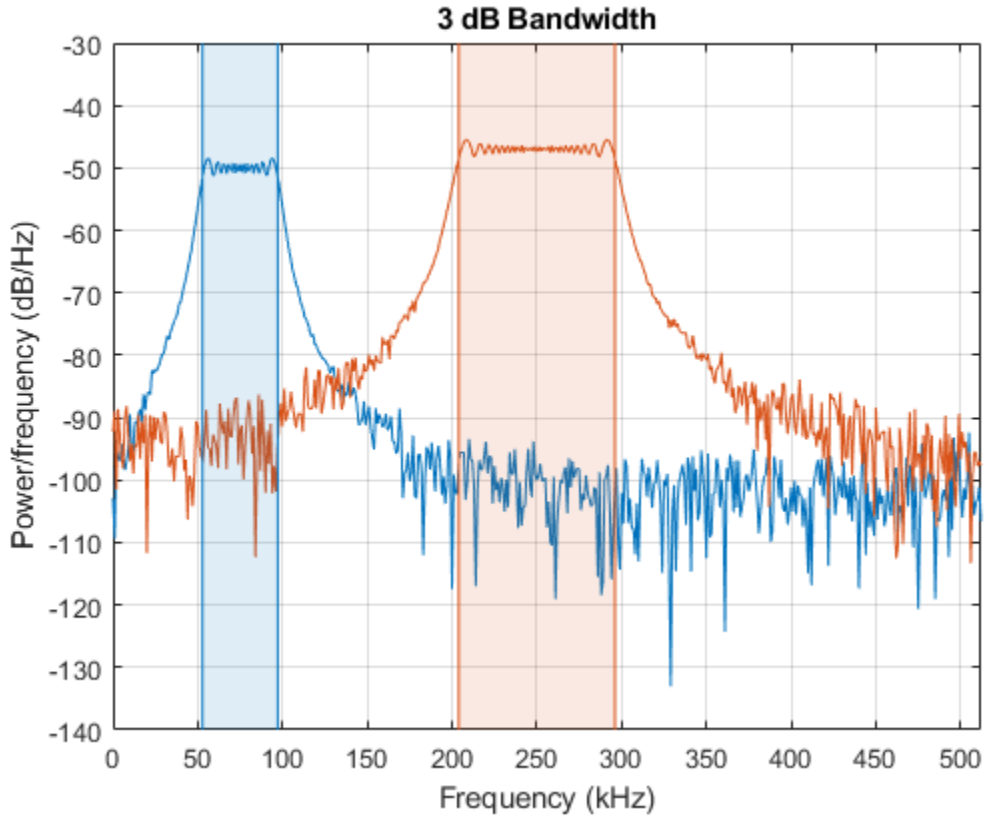
```
meanfreq([x x2],Fs)
```




```
ans = 1×2
10^5 ×

    0.7503    2.4999
```

Estimate the half-power bandwidth of each channel. Annotate the 3-dB bandwidths on a plot of the PSDs.

```
powerbw([x x2],Fs)
```




```
ans = 1×2
10^4 ×

    4.4386    9.2208
```

See Also

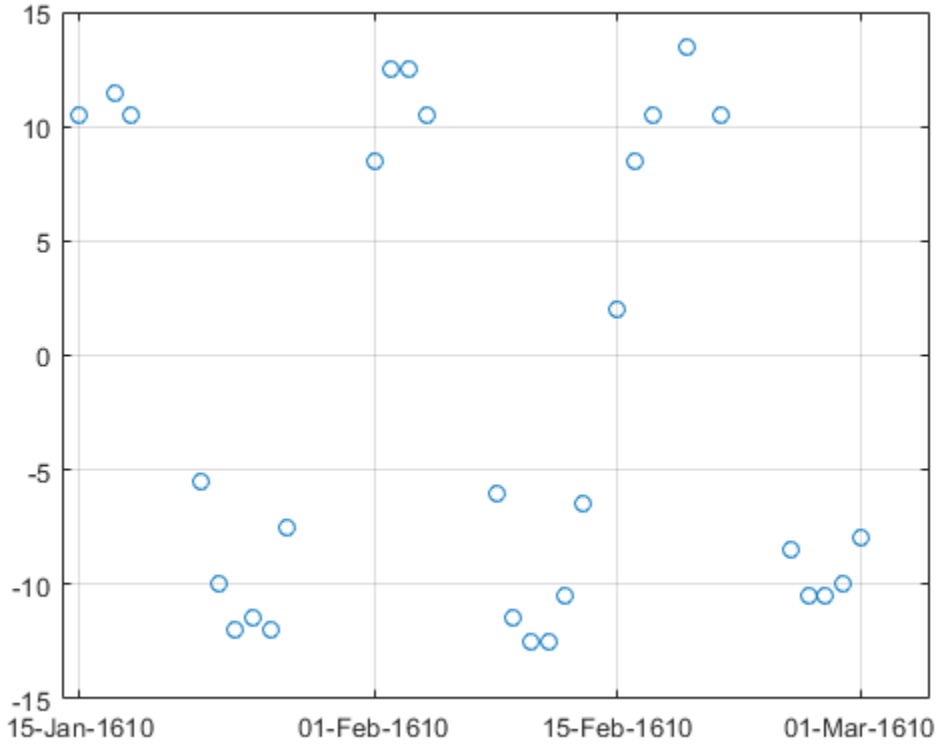
`bandpower` | `meanfreq` | `medfreq` | `obw` | `powerbw`

Periodogram of Data Set with Missing Samples

Galileo Galilei observed the motion of Jupiter's four largest satellites during the winter of 1610. When the weather allowed, Galileo recorded the satellites' locations. Use his observations to estimate the orbital period of one of the satellites, Callisto.

Callisto's angular position is measured in minutes of arc. Missing data due to cloudy conditions are specified using NaNs. The first observation is dated January 15. Generate a `datetime` array of observation times.

```
yg = [10.5 NaN 11.5 10.5 NaN NaN NaN -5.5 -10.0 -12.0 -11.5 -12.0 -7.5 ...
    NaN NaN NaN NaN 8.5 12.5 12.5 10.5 NaN NaN NaN -6.0 -11.5 -12.5 ...
    -12.5 -10.5 -6.5 NaN 2.0 8.5 10.5 NaN 13.5 NaN 10.5 NaN NaN NaN ...
    -8.5 -10.5 -10.5 -10.0 -8.0]';

obsv = datetime(1610,1,14+(1:length(yg)));

plot(yg,'o')

ax = gca;
nights = [1 18 32 46];
ax.XTick = nights;
ax.XTickLabel = char(obsv(nights));
grid
```



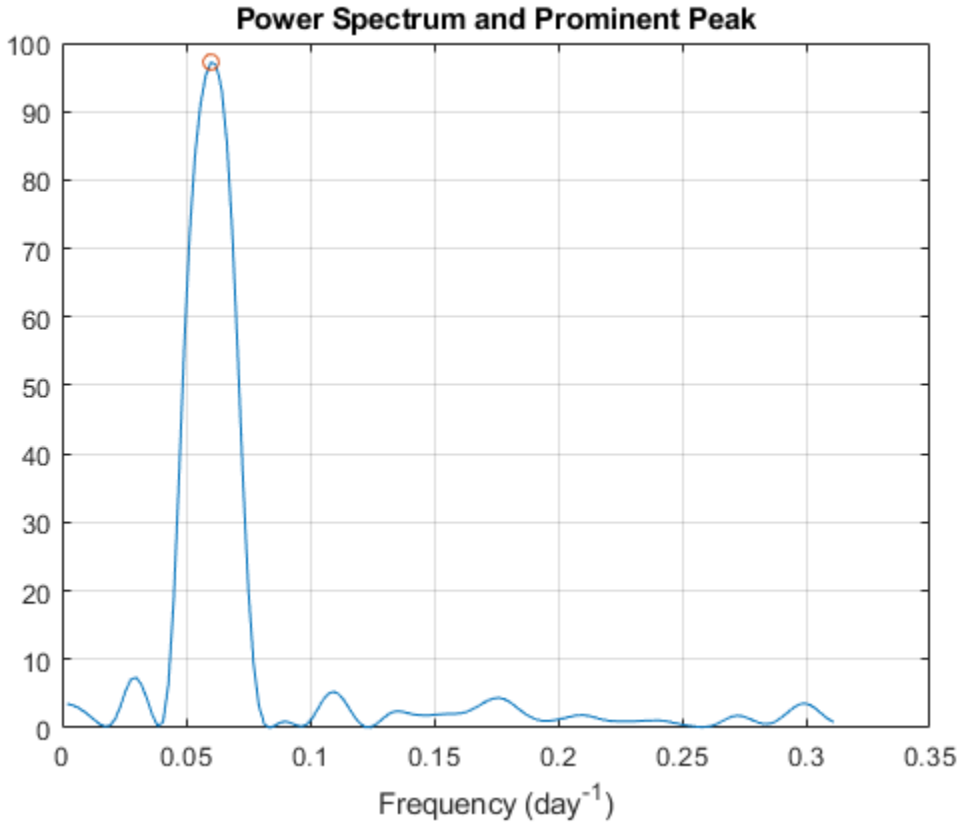
Estimate the power spectrum of the data using `plomb`. Specify an oversampling factor of 10. Express the resulting frequencies in inverse days.

```
[pxx,f] = plomb(yg,obsv,[],10,'power');
f = f*86400;
```

Use `findpeaks` to determine the location of the only prominent peak of the spectrum. Plot the power spectrum and show the peak.

```
[pk,f0] = findpeaks(pxx,f,'MinPeakHeight',10);

plot(f,pxx,f0,pk,'o')
xlabel('Frequency (day^{-1})')
```

```
title('Power Spectrum and Prominent Peak')
grid
```



Determine Callisto's orbital period (in days) as the inverse of the frequency of maximum energy. The result differs by less than 1% from the value published by NASA.

```
Period = 1/f0
```

```
Period = 16.6454
```

```
NASA = 16.6890184;
PercentDiscrep = (Period-NASA)/NASA*100
```

```
PercentDiscrep = -0.2613
```

See Also

`findpeaks` | `plomb`

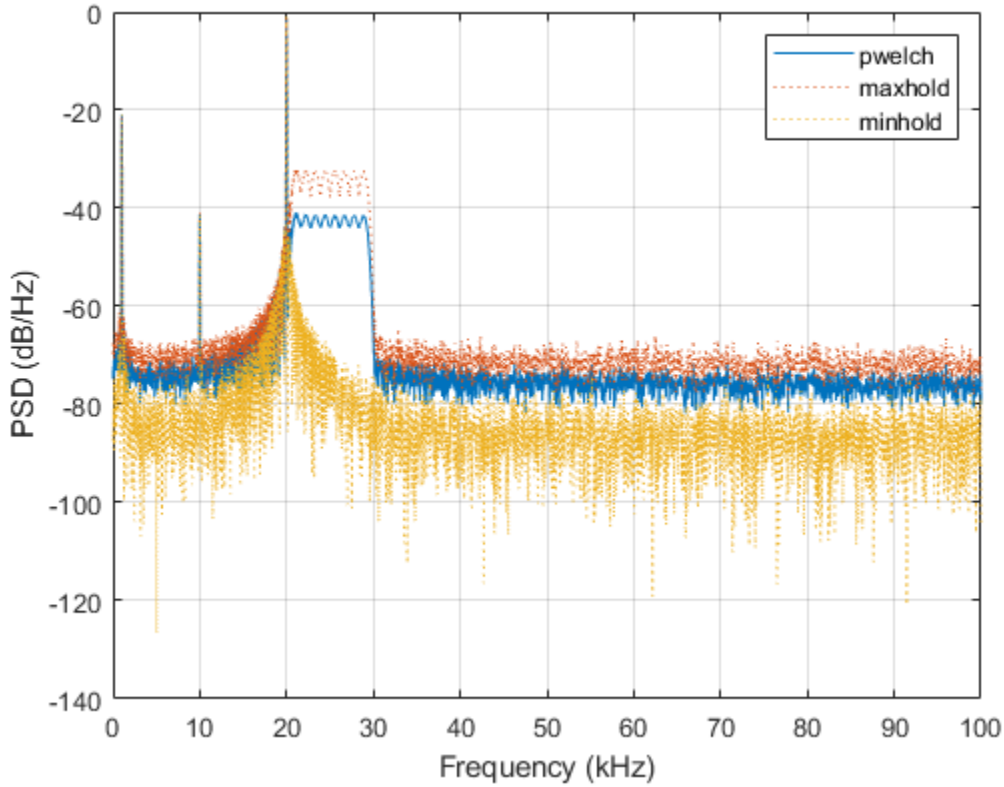
Welch Spectrum Estimates

Create a signal consisting of three noisy sinusoids and a chirp, sampled at 200 kHz for 0.1 second. The frequencies of the sinusoids are 1 kHz, 10 kHz, and 20 kHz. The sinusoids have different amplitudes and noise levels. The noiseless chirp has a frequency that starts at 20 kHz and increases linearly to 30 kHz during the sampling.

```
Fs = 200e3;
Fc = [1 10 20]'*1e3;
Ns = 0.1*Fs;

t = (0:Ns-1)/Fs;
x = [1 1/10 10]*sin(2*pi*Fc*t)+[1/200 1/2000 1/20]*randn(3,Ns);
x = x+chirp(t,20e3,t(end),30e3);
```

Compute the Welch PSD estimate and the maximum-hold and minimum-hold spectra of the signal. Plot the results.

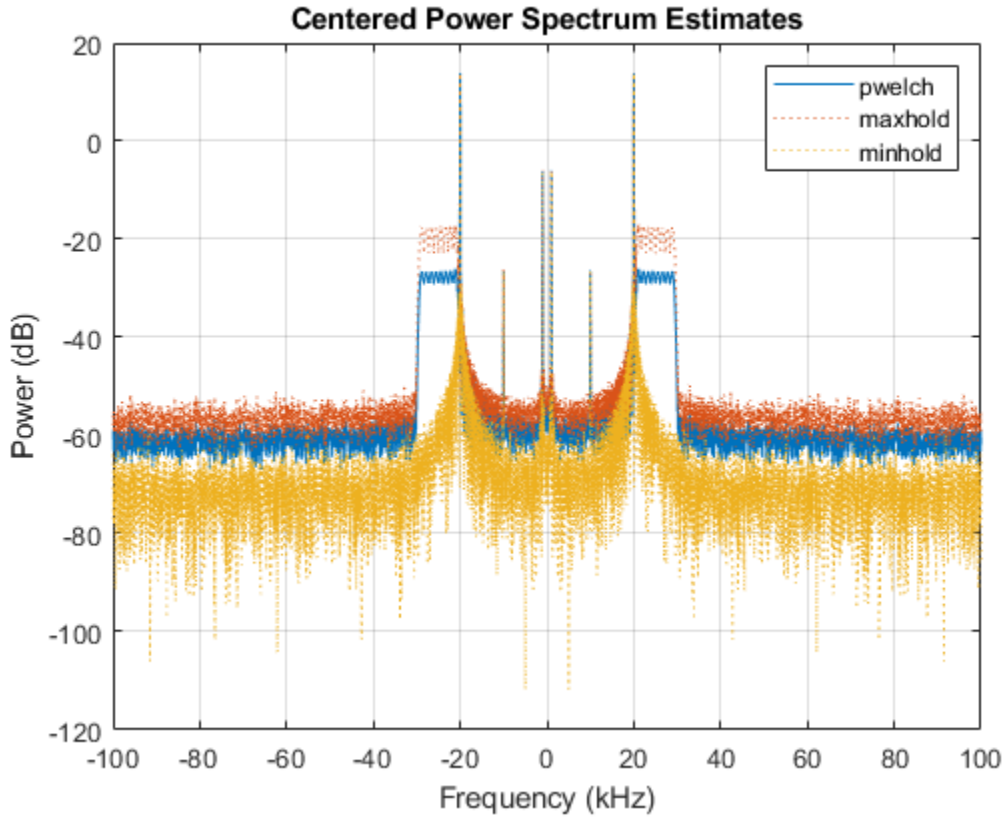
```
[pxx,f] = pwelch(x,[],[],[],Fs);
pmax = pwelch(x,[],[],[],Fs,'maxhold');
pmin = pwelch(x,[],[],[],Fs,'minhold');

plot(f/1000,pow2db(pxx))
hold on
plot(f/1000,pow2db([pmax pmin]),':')
hold off
xlabel('Frequency (kHz)')
ylabel('PSD (dB/Hz)')
legend('pwelch','maxhold','minhold')
grid
```



Repeat the procedure, this time computing centered power spectrum estimates.

```
[pxx,f] = pwelch(x,[],[],[],Fs,'centered','power');
pmax = pwelch(x,[],[],[],Fs,'maxhold','centered','power');
pmin = pwelch(x,[],[],[],Fs,'minhold','centered','power');

plot(f/1000,pow2db(pxx))
hold on
plot(f/1000,pow2db([pmax pmin]),':')
hold off
xlabel('Frequency (kHz)')
ylabel('Power (dB)')
legend('pwelch','maxhold','minhold')
```

```
title('Centered Power Spectrum Estimates')
grid
```



See Also

chirp | pow2db | pwelch

Linear Prediction

- “Prediction Polynomial” on page 13-2
- “Formant Estimation with LPC Coefficients” on page 13-6
- “AR Order Selection with Partial Autocorrelation Sequence” on page 13-10

Prediction Polynomial

This example shows how to obtain the prediction polynomial from an autocorrelation sequence. The example also shows that the resulting prediction polynomial has an inverse that produces a stable all-pole filter. You can use the all-pole filter to filter a wide-sense stationary white noise sequence to produce a wide-sense stationary autoregressive process.

Create an autocorrelation sequence defined by

$$r(k) = \frac{24}{5}2^{-|k|} - \frac{27}{10}3^{-|k|}, \quad k = 0, 1, 2.$$

```
k = 0:2;
rk = (24/5)*2.^(-k)-(27/10)*3.^(-k);
```

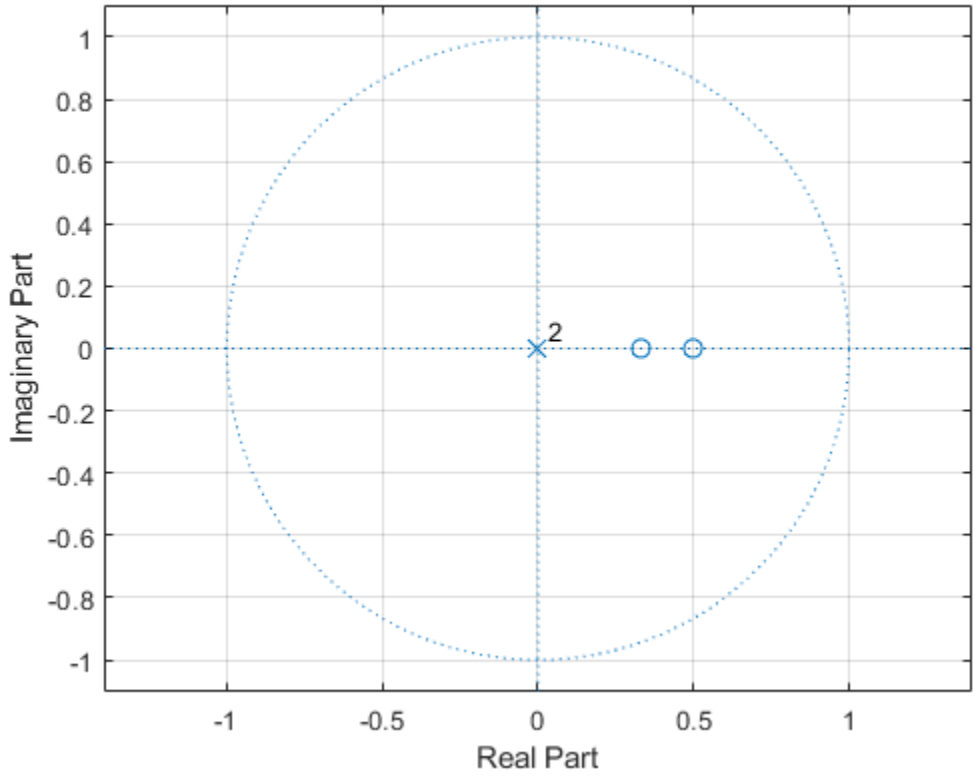
Use `ac2poly` to obtain the prediction polynomial of order 2, which is

$$A(z) = 1 - \frac{5}{6}z^{-1} + \frac{1}{6}z^{-2}.$$

```
A = ac2poly(rk);
```

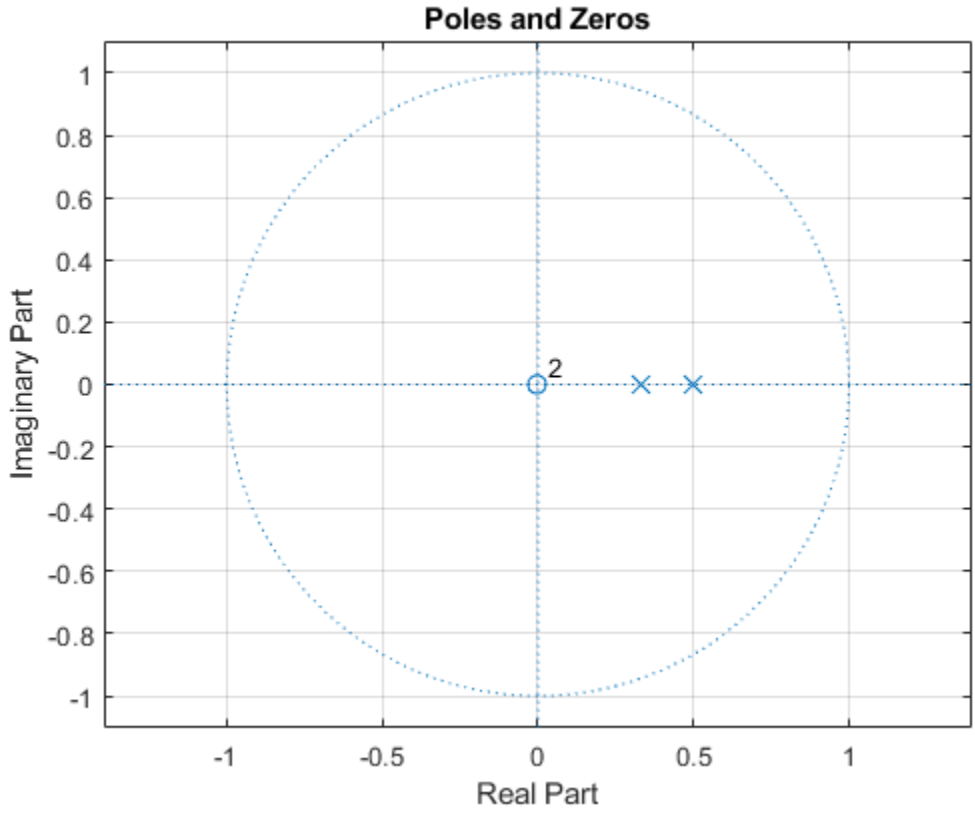
Examine the pole-zero plot of the FIR filter to see that the zeros are inside the unit circle.

```
zplane(A,1)
grid
```



The inverse all-pole filter is stable with poles inside the unit circle.

```
zplane(1,A)
grid
title('Poles and Zeros')
```



Use the all-pole filter to produce a realization of a wide-sense stationary AR(2) process from a white-noise sequence. Set the random number generator to the default settings for reproducible results.

```
rng default

x = randn(1000,1);
y = filter(1,A,x);
```

Compute the sample autocorrelation of the AR(2) realization and show that the sample autocorrelation is close to the true autocorrelation.

```
[xc,lags] = xcorr(y,2,'biased');
[xc(3:end) rk']
```

```
ans = 3×2

    2.2401    2.1000
    1.6419    1.5000
    0.9980    0.9000
```

Formant Estimation with LPC Coefficients

This example shows how to estimate vowel formant frequencies using linear predictive coding (LPC). The formant frequencies are obtained by finding the roots of the prediction polynomial.

This example uses the speech sample mtlb.mat, which is part of Signal Processing Toolbox™. The speech is lowpass-filtered. Because of the low sampling frequency, this speech sample is not optimal for this example. The low sampling frequency limits the order of the autoregressive model you can fit to the data. In spite of this limitation, the example illustrates the technique for using LPC coefficients to determine vowel formants.

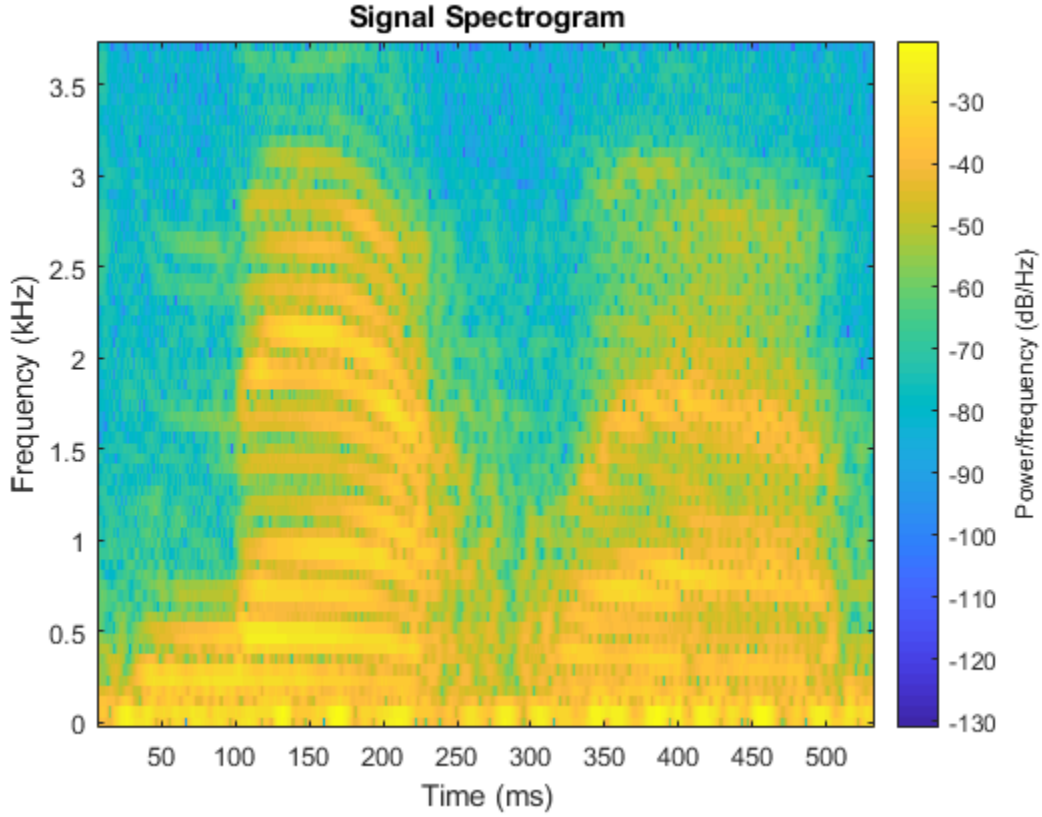
Load the speech signal. The recording is a woman saying "MATLAB". The sampling frequency is 7418 Hz.

```
load mtlb
```

The MAT file contains the speech waveform, mtlb, and the sampling frequency, `Fs`.

Use the `spectrogram` function to identify a voiced segment for analysis.

```
segmentlen = 100;
noverlap = 90;
NFFT = 128;

spectrogram(mtlb,segmentlen,noverlap,NFFT,Fs,'yaxis')
title('Signal Spectrogram')
```



Extract the segment from 0.1 to 0.25 seconds for analysis. The extracted segment corresponds roughly to the first vowel, /ae/, in "MATLAB".

```
dt = 1/Fs;
I0 = round(0.1/dt);
Iend = round(0.25/dt);
x = mtlb(I0:Iend);
```

Two common preprocessing steps applied to speech waveforms before linear predictive coding are windowing and pre-emphasis (highpass) filtering.

Window the speech segment using a Hamming window.

```
x1 = x.*hamming(length(x));
```

Apply a pre-emphasis filter. The pre-emphasis filter is a highpass all-pole (AR(1)) filter.

```
preemph = [1 0.63];
x1 = filter(1,preemph,x1);
```

Obtain the linear prediction coefficients. To specify the model order, use the general rule that the order is two times the expected number of formants plus 2. In the frequency range, [0,|Fs|/2], you expect three formants. Therefore, set the model order equal to 8. Find the roots of the prediction polynomial returned by `lpc`.

```
A = lpc(x1,8);
rts = roots(A);
```

Because the LPC coefficients are real-valued, the roots occur in complex conjugate pairs. Retain only the roots with one sign for the imaginary part and determine the angles corresponding to the roots.

```
rts = rts(imag(rts)>=0);
angz = atan2(imag(rts),real(rts));
```

Convert the angular frequencies in rad/sample represented by the angles to Hz and calculate the bandwidths of the formants.

The bandwidths of the formants are represented by the distance of the prediction polynomial zeros from the unit circle.

```
[frqs,indices] = sort(angz.*(Fs/(2*pi)));
bw = -1/2*(Fs/(2*pi))*log(abs(rts(indices)));
```

Use the criterion that formant frequencies should be greater than 90 Hz with bandwidths less than 400 Hz to determine the formants.

```
nn = 1;
for kk = 1:length(frqs)
    if (frqs(kk) > 90 && bw(kk) <400)
        formants(nn) = frqs(kk);
        nn = nn+1;
    end
end
formants
```

```
formants = 1×3
10^3 ×
```

```
    0.8697    2.0265    2.7380
```

The first three formants are 869.70, 2026.49, and 2737.95 Hz.

References

[1] Snell, Roy C., and Fausto Milinazzo. "Formant location from LPC analysis data." IEEE® Transactions on Speech and Audio Processing. Vol. 1, Number 2, 1993, pp. 129-134.

[2] Loizou, Philipos C. "COLEA: A MATLAB Software Tool for Speech Analysis."

AR Order Selection with Partial Autocorrelation Sequence

This example shows how to assess the order of an autoregressive model using the partial autocorrelation sequence. For these processes, you can use the partial autocorrelation sequence to help with model order selection. For a stationary time series with values $X(1), X(2), X(3), \ldots, X(k+1)$, the partial autocorrelation sequence at lag k is the correlation between $X(1)$ and $X(k+1)$ after regressing $X(1)$ and $X(k+1)$ on the intervening observations, $X(2), X(3), X(4), \ldots, X(k)$. For a moving average process, you can use the autocorrelation sequence to assess the order. However, for an autoregressive (AR) or autoregressive moving average (ARMA) process, the autocorrelation sequence does not help in order selection. Consider the AR(2) process defined by

$$X(n) + 1.5X(n-1) + 0.75X(n-2) = \varepsilon(n),$$

where $\varepsilon(n)$ is an $N(0,1)$ Gaussian white noise process. The following example:

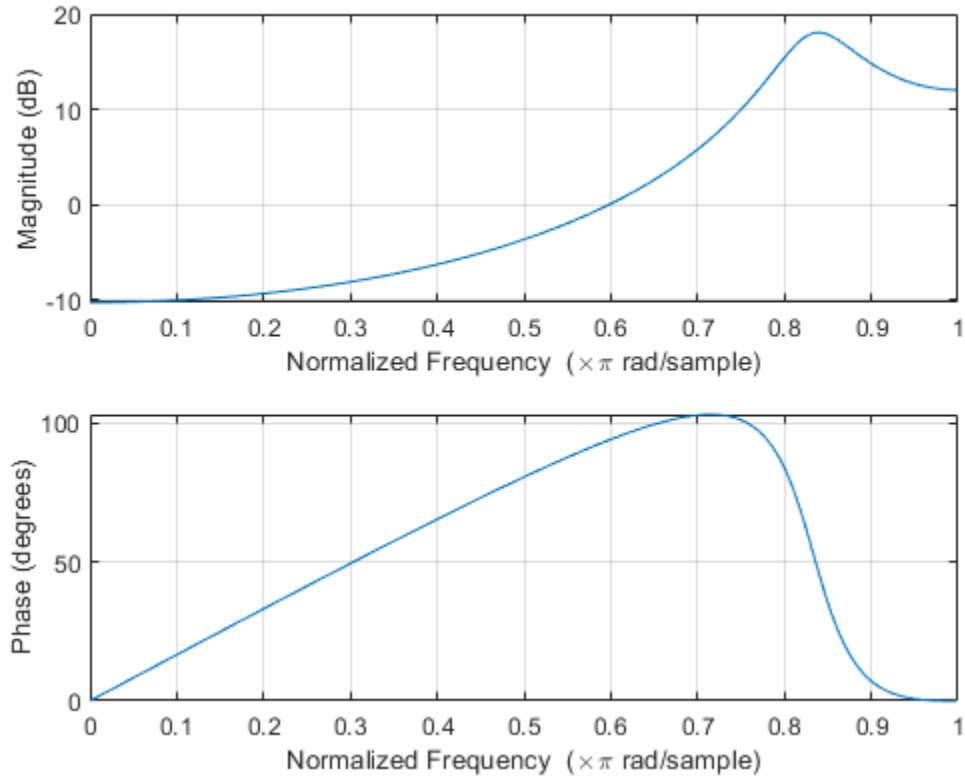
- Simulates a realization of the AR(2) process
- Graphically explores the correlation between lagged values of the time series
- Examines the sample autocorrelation sequence of the time series
- Fits an AR(15) model to the time series by solving the Yule-Walker equations (`aryule`)
- Uses the reflection coefficients returned by `aryule` to compute the partial autocorrelation sequence
- Examines the partial autocorrelation sequence to select the model order

Simulate a 1000-sample time series from the AR(2) process defined by the difference equation. Set the random number generator to the default settings for reproducible results.

```
A = [1 1.5 0.75];
rng default
x = filter(1,A,randn(1000,1));
```

View the frequency response of the AR(2) process.

```
freqz(1,A)
```

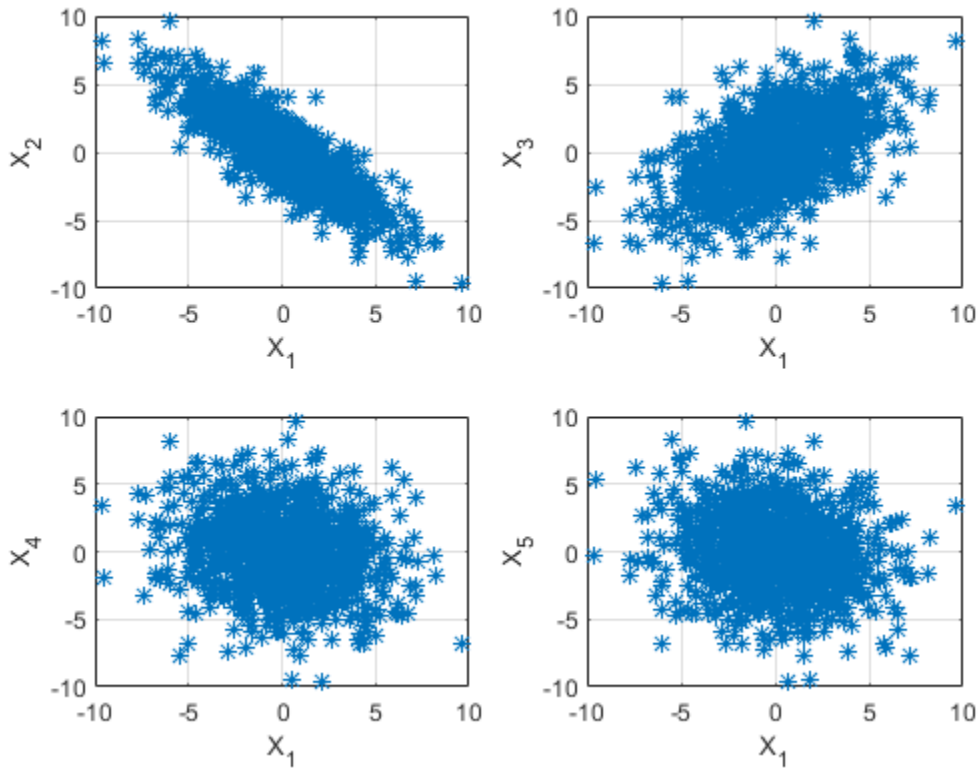


The AR(2) process acts like a highpass filter in this case.

Graphically examine the correlation in x by producing scatter plots of $X(n)$ vs. $X(1)$ for $n = 2, 3, 4, 5$.

```
x12 = x(1:end-1);
x21 = x(2:end);
subplot(2,2,1)
plot(x12,x21,'*')
xlabel('X_1')
ylabel('X_2')
grid

x13 = x(1:end-2);
```

```
x31 = x(3:end);
subplot(2,2,2)
plot(x13,x31,'*')
xlabel('X_1')
ylabel('X_3')
grid

x14 = x(1:end-3);
x41 = x(4:end);
subplot(2,2,3)
plot(x14,x41,'*')
xlabel('X_1')
ylabel('X_4')
grid

x15 = x(1:end-4);
x51 = x(5:end);
subplot(2,2,4)
plot(x15,x51,'*')
xlabel('X_1')
ylabel('X_5')
grid
```



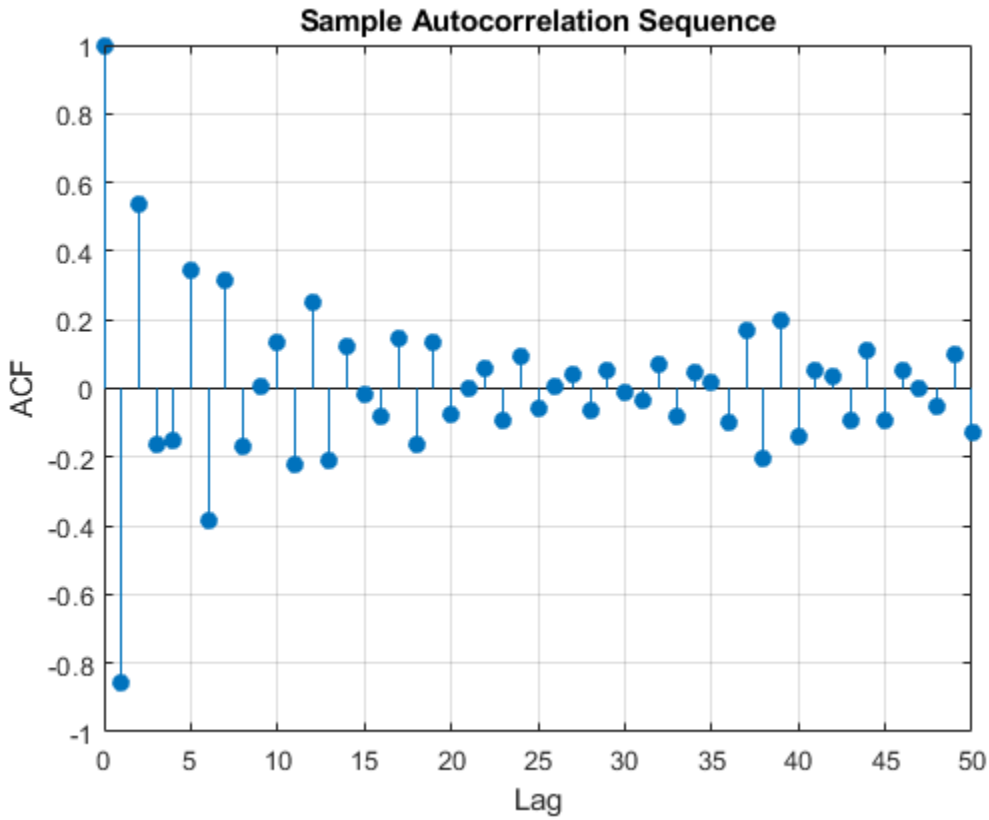
In the scatter plot, you see there is a linear relationship between $X(1)$ and $X(2)$ and between $X(1)$ and $X(3)$, but not between $X(1)$ and either $X(4)$ or $X(5)$.

The points in the top row scatter plots fall approximately on a line with a negative slope in the top left panel and positive slope in the top right panel. The scatter plots in the bottom two panels do not show any apparent linear relationship.

The negative correlation between $X(1)$ and $X(2)$ and the positive correlation between $X(1)$ and $X(3)$ are explained by the highpass-filter behavior of the AR(2) process.

Find the sample autocorrelation sequence out to lag 50 and plot the result.

```
[xc,lags] = xcorr(x,50,'coeff');

figure
stem(lags(51:end),xc(51:end),'filled')
xlabel('Lag')
ylabel('ACF')
title('Sample Autocorrelation Sequence')
grid
```



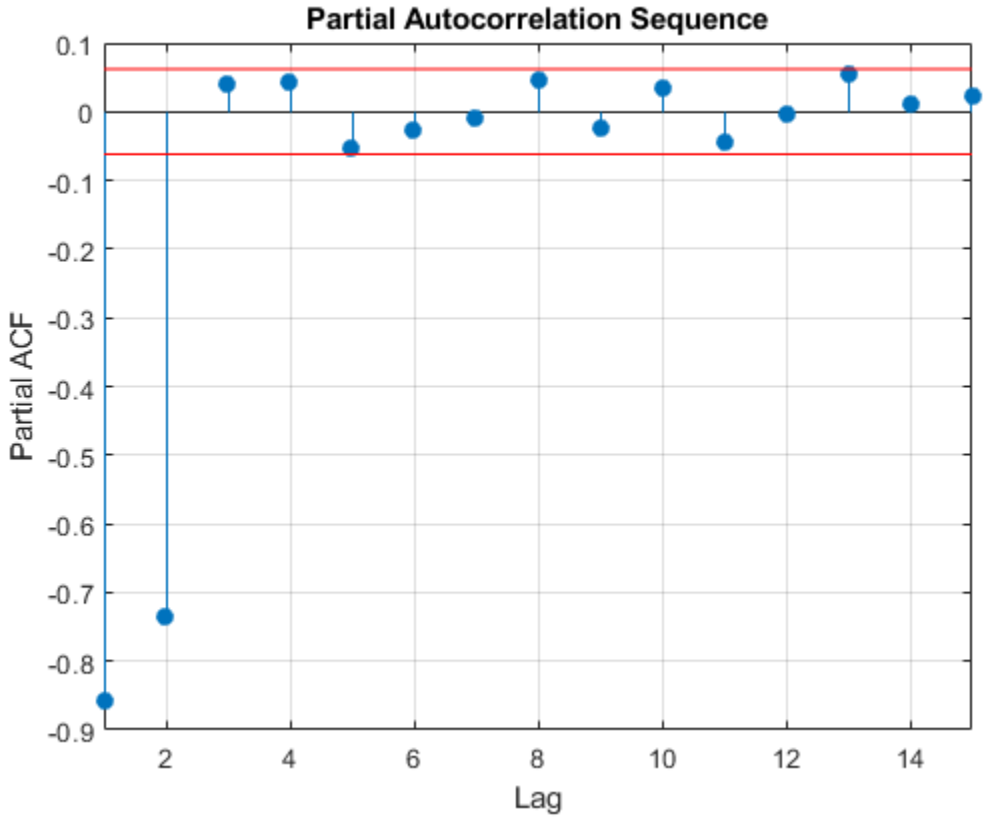
The sample autocorrelation sequence shows a negative value at lag 1 and a positive value at lag 2. Based on the scatter plot, this is the expected result. However, you cannot determine from the sample autocorrelation sequence what order is appropriate for the AR model.

Fit an AR(15) model using `aryule`. Return the reflection coefficients. The negative of the reflection coefficients is the partial autocorrelation sequence.

```
[arcoefs,E,K] = aryule(x,15);
pacf = -K;
```

Plot the partial autocorrelation sequence along with the large-sample 95% confidence intervals. If the data are generated by an autoregressive process of order p, the values of the sample partial autocorrelation sequence for lags greater than p follow a $N(0,1/N)$ distribution, where N is the length of the time series.

```
stem(pacf,'filled')
xlabel('Lag')
ylabel('Partial ACF')
title('Partial Autocorrelation Sequence')
xlim([1 15])
uconf = 1.96/sqrt(1000);
lconf = -uconf;
hold on
plot([1 15],[1 1]'*[lconf uconf],'r')
grid
```



The only values of the partial autocorrelation sequence outside the 95% confidence bounds occur at lags 1 and 2. This indicates that the correct model order for the AR process is 2.

In this example, you generated the time series to simulate an AR(2) process. The partial autocorrelation sequence only confirms that result. In practice, you have only the observed time series without any prior information about model order. In a realistic scenario, the partial autocorrelation is an important tool for appropriate model order selection in stationary autoregressive time series.

Transforms

- “Complex Cepstrum -- Fundamental Frequency Estimation” on page 14-2
- “Analytic Signal for Cosine” on page 14-7
- “Envelope Extraction” on page 14-11
- “Analytic Signal and Hilbert Transform” on page 14-20
- “Hilbert Transform and Instantaneous Frequency” on page 14-26
- “Detect Closely Spaced Sinusoids” on page 14-33
- “Instantaneous Frequency of Complex Chirp” on page 14-42
- “Single-Sideband Amplitude Modulation” on page 14-46
- “DCT for Speech Signal Compression” on page 14-55

Complex Cepstrum -- Fundamental Frequency Estimation

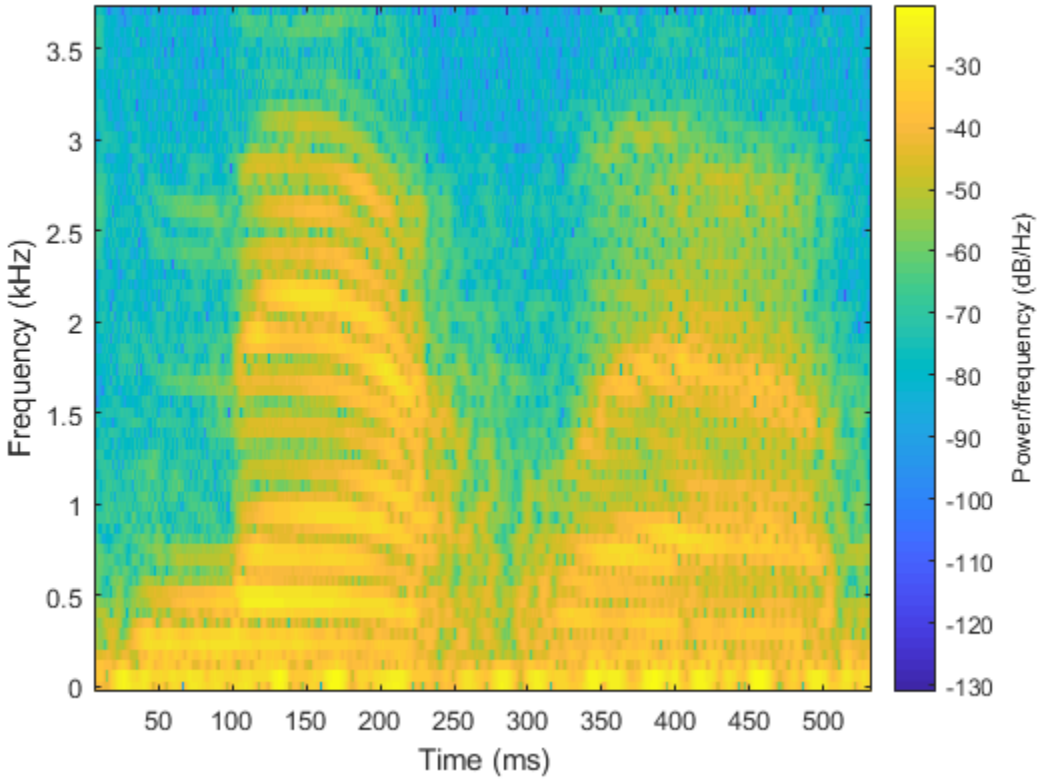
This example shows how to estimate a speaker's fundamental frequency using the complex cepstrum. The example also estimates the fundamental frequency using a zero-crossing method and compares the results.

Load the speech signal. The recording is of a woman saying "MATLAB". The sampling frequency is 7418 Hz. The following code loads the speech waveform, `mtlb`, and the sampling frequency, `Fs`, into the MATLAB® workspace.

```
load mtlb
```

Use the spectrogram to identify a voiced segment for analysis.

```
segmentlen = 100;
noverlap = 90;
NFFT = 128;

spectrogram(mtlb,segmentlen,noverlap,NFFT,Fs,'yaxis')
```



Extract the segment from 0.1 to 0.25 seconds for analysis. The extracted segment corresponds roughly to the first vowel, /æ/, in "MATLAB".

```
dt = 1/Fs;
I0 = round(0.1/dt);
Iend = round(0.25/dt);
x = mtlb(I0:Iend);
```

Obtain the complex cepstrum.

```
c = cceps(x);
```

Select a time range between 2 and 10 ms, corresponding to a frequency range of approximately 100 to 500 Hz. Identify the tallest peak of the cepstrum in the selected

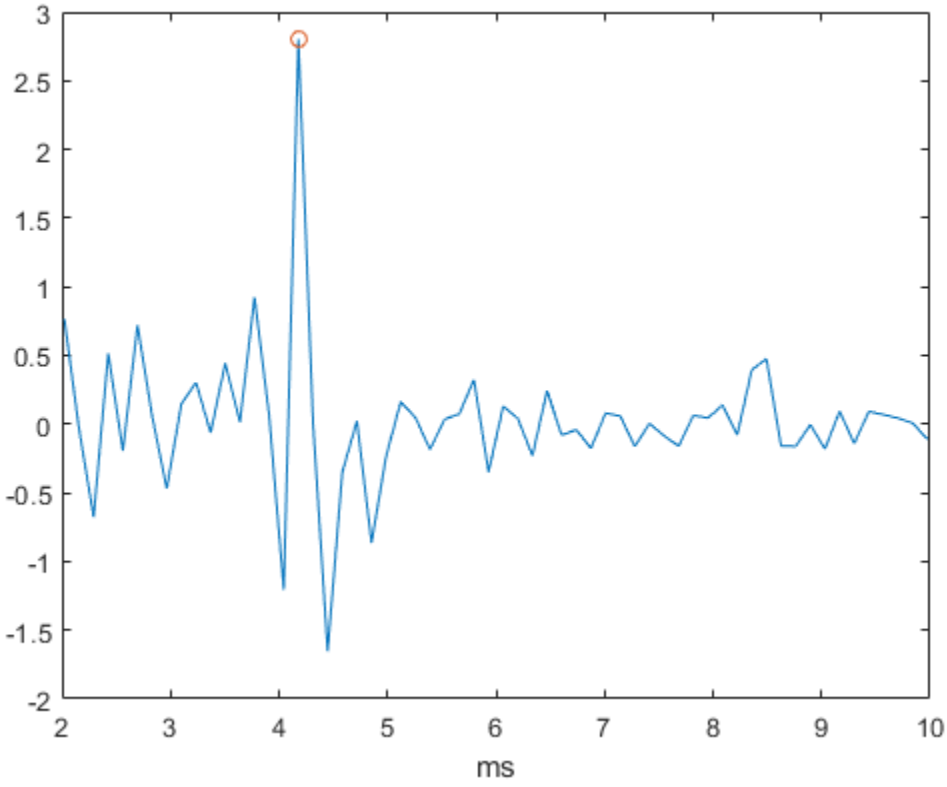
range. Find the frequency corresponding to the peak. Use the peak as the estimate of the fundamental frequency.

```
t = 0:dt:length(x)*dt-dt;

trng = t(t>=2e-3 & t<=10e-3);
crng = c(t>=2e-3 & t<=10e-3);

[~,I] = max(crng);

fprintf('Complex cepstrum F0 estimate is %3.2f Hz.\n',1/trng(I))
```

```
Complex cepstrum F0 estimate is 239.29 Hz.
```

Plot the cepstrum in the selected time range and overlay the peak.

```
plot(trng*1e3,crng)
xlabel('ms')

hold on
plot(trng(I)*1e3,crng(I),'o')
hold off
```



Use a zero-crossing detector on a lowpass-filtered and rectified form of the vowel to estimate the fundamental frequency.

```
[b0,a0] = butter(2,325/(Fs/2));
xin = abs(x);
xin = filter(b0,a0,xin);
xin = xin-mean(xin);
x2 = zeros(length(xin),1);
x2(1:length(x)-1) = xin(2:length(x));
zc = length(find((xin>0 & x2<0) | (xin<0 & x2>0)));
F0 = 0.5*Fs*zc/length(x);
fprintf('Zero-crossing F0 estimate is %3.2f Hz.\n',F0)
```

```
Zero-crossing F0 estimate is 233.27 Hz.
```

The estimate of the fundamental frequency obtained with the complex cepstrum is 239.29 Hz and the estimate with the zero-crossing detector is 233.27 Hz.

See Also

`cceps` | `icceps` | `rceps`

Analytic Signal for Cosine

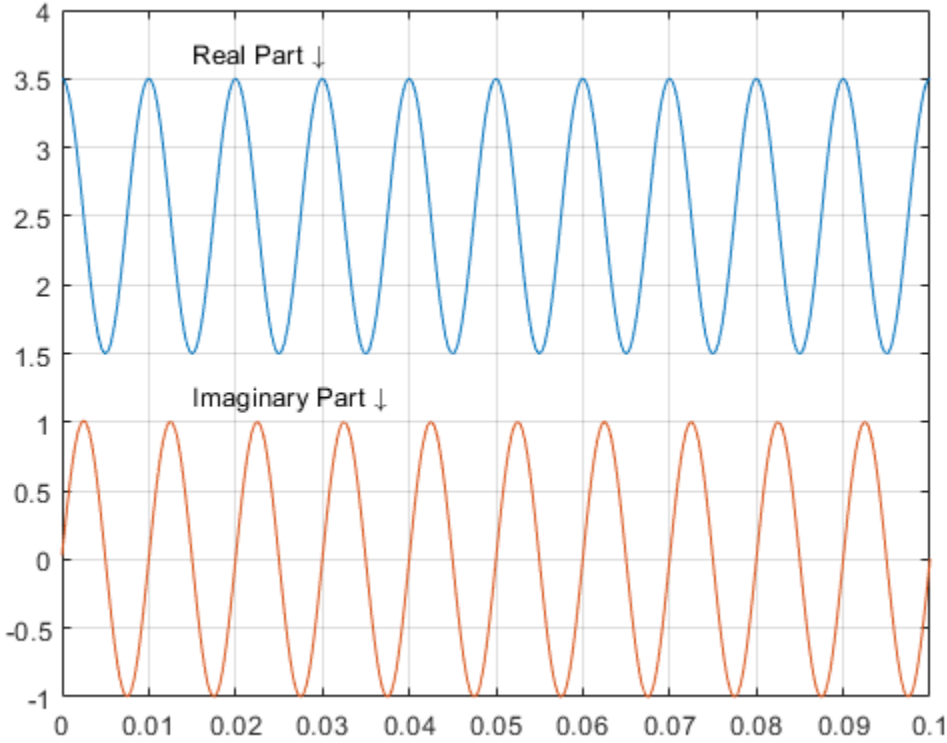
This example shows how to determine the analytic signal. The example also demonstrates that the imaginary part of the analytic signal corresponding to a cosine is a sine with the same frequency. If the cosine has a nonzero mean (DC shift), then the real part of the analytic signal is the original cosine with the same mean, but the imaginary part has zero mean.

Create a cosine with a frequency of 100 Hz. The sampling frequency is 10 kHz. Add a DC offset of 2.5 to the cosine.

```
t = 0:1e-4:1;
x = 2.5+cos(2*pi*100*t);
```

Use the `hilbert` function to obtain the analytic signal. The real part is equal to the original signal. The imaginary part is the Hilbert transform of the original signal. Plot the real and imaginary parts for comparison.

```
y = hilbert(x);

clf
plot(t,real(y))
hold on
plot(t,imag(y))
xlim([0 0.1])
grid on
text([0.015 0.015],[3.7 1.2], ...
    {'Real Part \downarrow';'Imaginary Part \downarrow'})
```



You see that the imaginary part is a sine with the same frequency as the cosine real part. However, the imaginary part has a mean of zero, while the real part has a mean of 2.5.

The original signal is

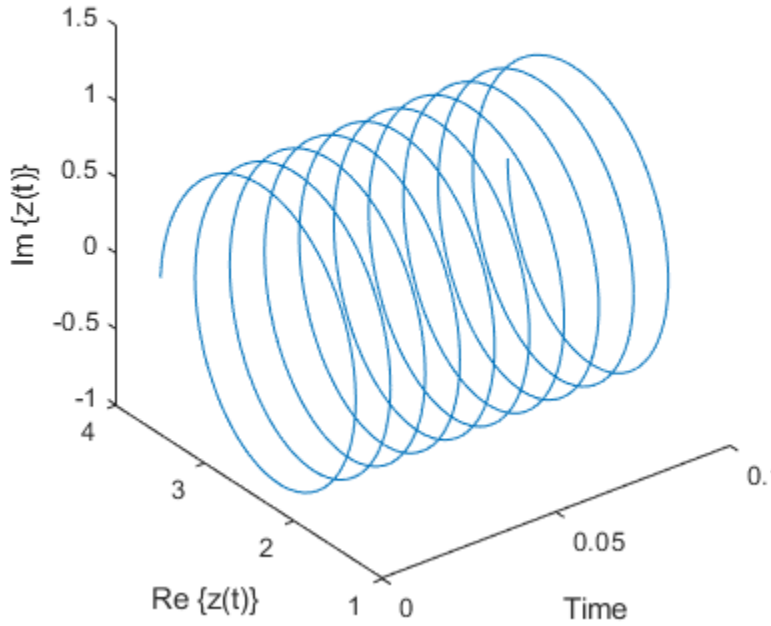
$$x(t) = 2.5 + \cos(2\pi 1000t).$$

The resulting analytic signal is

$$z(t) = 2.5 + e^{j2\pi 1000t}.$$

Plot 10 periods of the complex-valued analytic signal.

```
prds = 1:1000;

figure
plot3(t(prds),real(y(prds)),imag(y(prds)))

xlabel('Time')
ylabel('Re \{z(t)\}')
zlabel('Im \{z(t)\}')
axis square
```



See Also

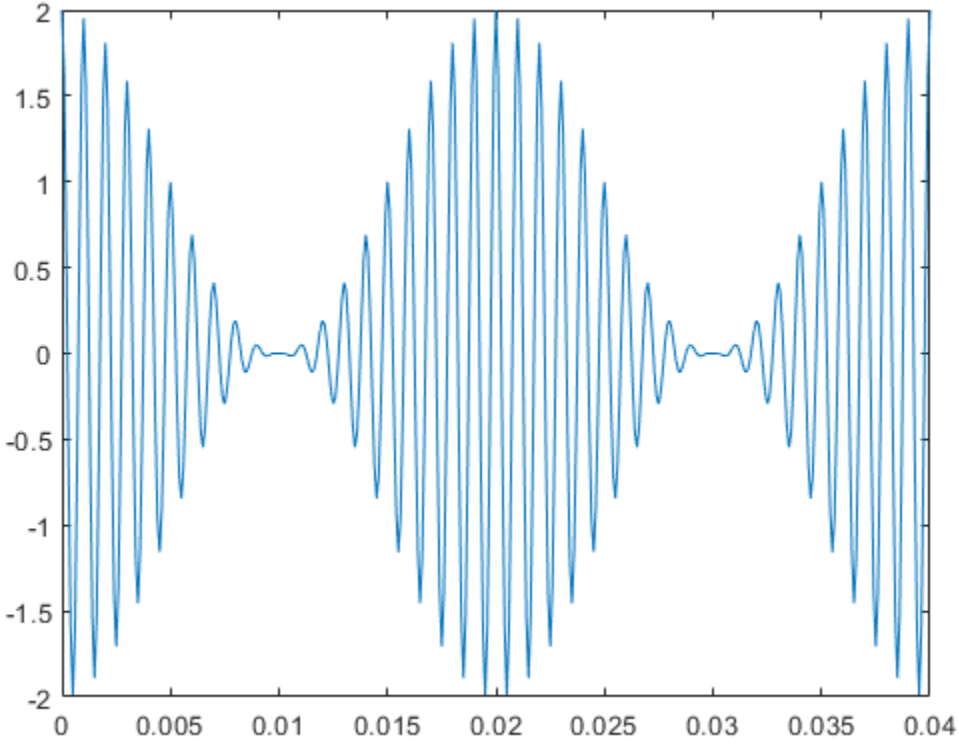
`hilbert`

Envelope Extraction

This example shows how to extract the envelope of a signal.

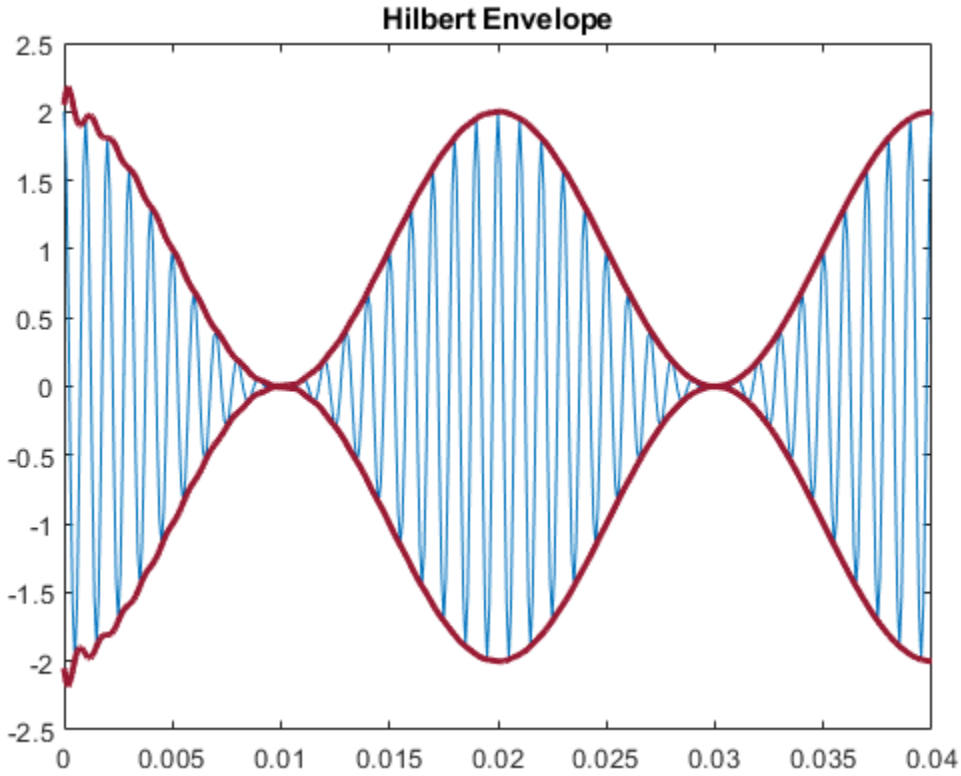
Create a double sideband amplitude-modulated signal. The carrier frequency is 1 kHz. The modulation frequency is 50 Hz. The modulation depth is 100%. The sample rate is 10 kHz.

```
t = 0:1e-4:0.1;
x = (1+cos(2*pi*50*t)).*cos(2*pi*1000*t);

plot(t,x)
xlim([0 0.04])
```



Extract the envelope using `hilbert` function. The envelope is the magnitude of the analytic signal computed by `hilbert`. Plot the envelope along with the original signal. Store the name-value pair arguments of the `plot` function in a cell array for later use. The magnitude of the analytic signal captures the slowly varying features of the signal, while the phase contains the high-frequency information.

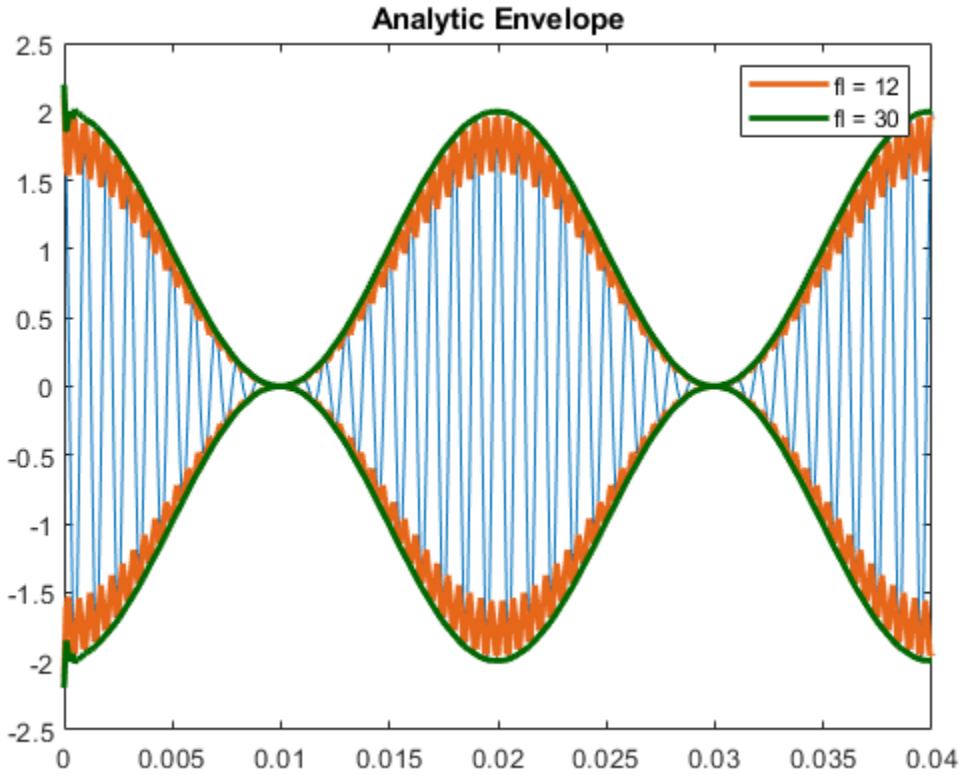
```
y = hilbert(x);
env = abs(y);
plot_param = {'Color', [0.6 0.1 0.2],'Linewidth',2};

plot(t,x)
hold on
plot(t,[-1;1]*env,plot_param{:})
hold off
xlim([0 0.04])
title('Hilbert Envelope')
```



You can also use the `envelope` function to generate the signal envelope directly and modify the way it is computed. For example, you can adjust the length of the Hilbert filter used to find the analytic envelope. Using a filter length that is too small results in a distorted envelope.

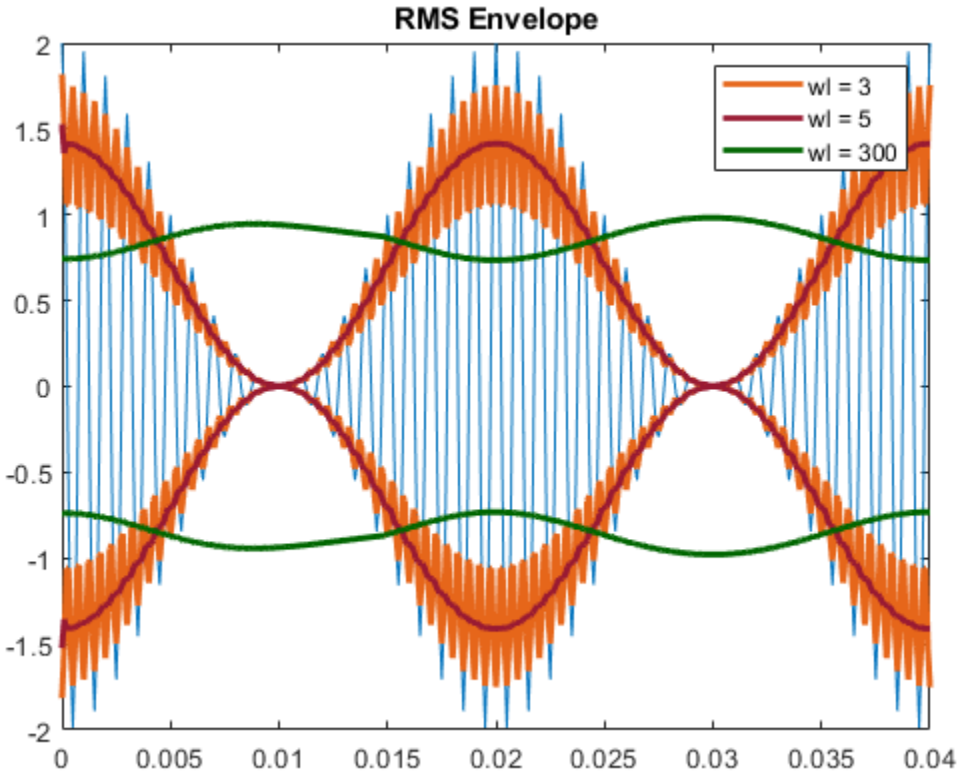
```
fl1 = 12;
[up1,lo1] = envelope(x,fl1,'analytic');
fl2 = 30;
[up2,lo2] = envelope(x,fl2,'analytic');
param_small = {'Color',[0.9 0.4 0.1],'Linewidth',2};
param_large = {'Color',[0 0.4 0],'Linewidth',2};

plot(t,x)
hold on
```

```
p1 = plot(t,up1,param_small{:});
plot(t,lo1,param_small{:});
p2 = plot(t,up2,param_large{:});
plot(t,lo2,param_large{:});
hold off

legend([p1 p2],'fl = 12','fl = 30')
xlim([0 0.04])
title('Analytic Envelope')
```



You can generate moving RMS envelopes by using a sliding window. Using a window length that is too small results in a distorted envelope. Using a window length that is too large smooths out the envelope.

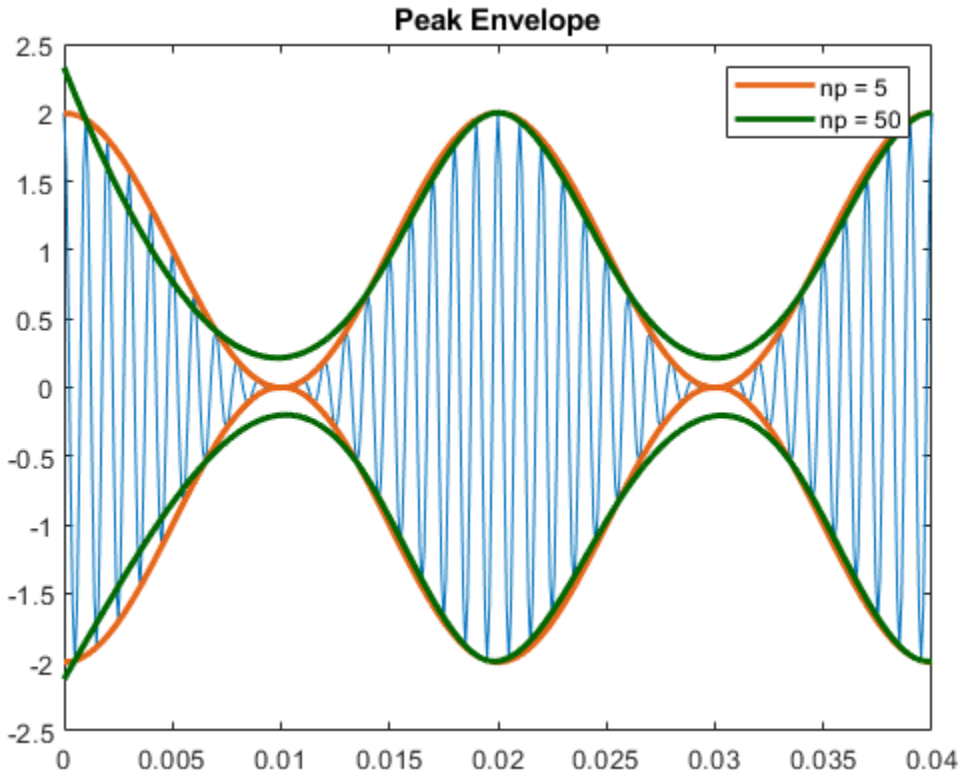
```
wl1 = 3;
[up1,lo1] = envelope(x,wl1,'rms');
wl2 = 5;
[up2,lo2] = envelope(x,wl2,'rms');
wl3 = 300;
[up3,lo3] = envelope(x,wl3,'rms');

plot(t,x)
hold on
p1 = plot(t,up1,param_small{:});
plot(t,lo1,param_small{:});
p2 = plot(t,up2,plot_param{:});
plot(t,lo2,plot_param{:});
p3 = plot(t,up3,param_large{:});
plot(t,lo3,param_large{:})
hold off

legend([p1 p2 p3],'wl = 3','wl = 5','wl = 300')
xlim([0 0.04])
title('RMS Envelope')
```



You can generate peak envelopes by using spline interpolation over local maxima separated by an adjustable number of samples. Spreading out the samples too much smooths the envelope.

```
np1 = 5;
[up1,lo1] = envelope(x,np1,'peak');
np2 = 50;
[up2,lo2] = envelope(x,np2,'peak');

plot(t,x)
hold on
p1 = plot(t,up1,param_small{:});
plot(t,lo1,param_small{:})
p2 = plot(t,up2,param_large{:});
```

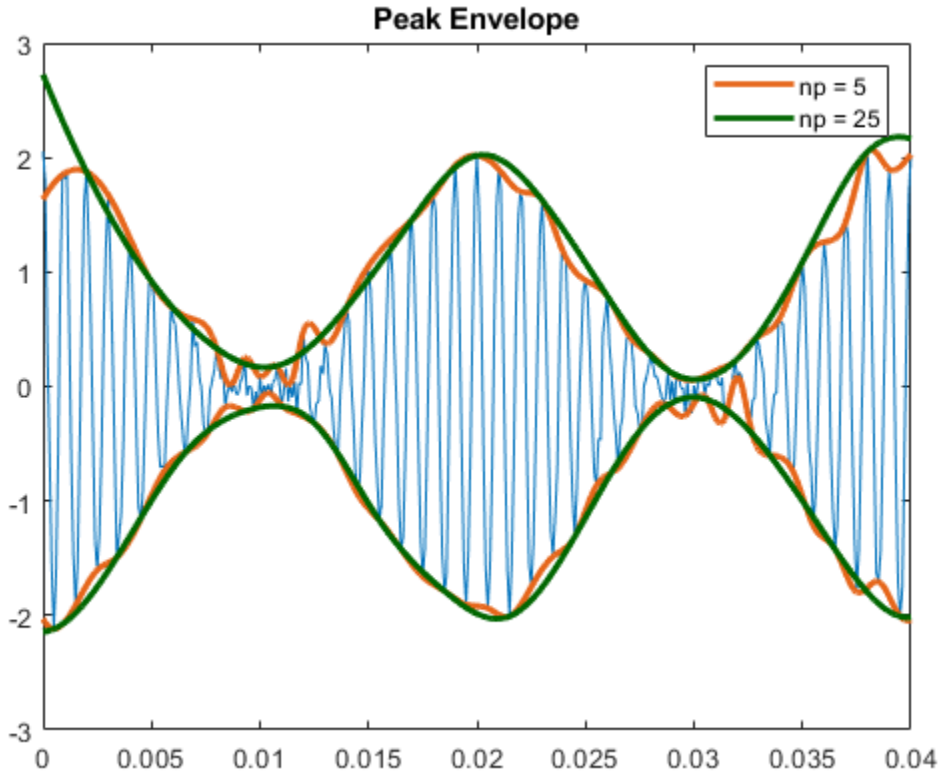
```
plot(t,lo2,param_large{:})
hold off

legend([p1 p2],'np = 5','np = 50')
xlim([0 0.04])
title('Peak Envelope')
```



Increasing the peak separation parameter can decrease the effect of spurious peaks due to noise. Introduce random noise to the signal. Use a 5-sample interval to see the effect of noise on the peak envelope. Repeat the exercise using a 25-sample interval.

```
rng default
q = x + randn(size(x))/10;
np1 = 5;
```

```
[up1,lo1] = envelope(q,np1,'peak');
np2 = 25;
[up2,lo2] = envelope(q,np2,'peak');

plot(t,q)
hold on
p1 = plot(t,up1,param_small{:});
plot(t,lo1,param_small{:})
p2 = plot(t,up2,param_large{:});
plot(t,lo2,param_large{:})
hold off

legend([p1 p2],'np = 5','np = 25')
xlim([0 0.04])
title('Peak Envelope')
```



See Also

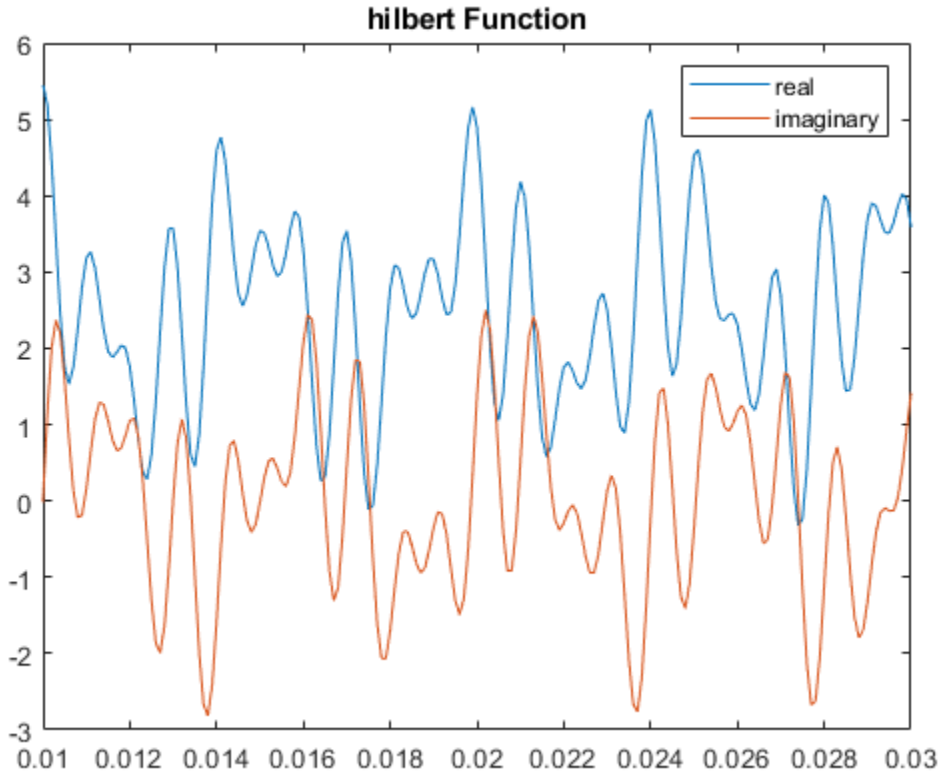
`envelope` | `hilbert`

Analytic Signal and Hilbert Transform

The `hilbert` function finds the exact analytic signal for a finite block of data. You can also generate the analytic signal by using an finite impulse response (FIR) Hilbert transformer filter to compute an approximation to the imaginary part.

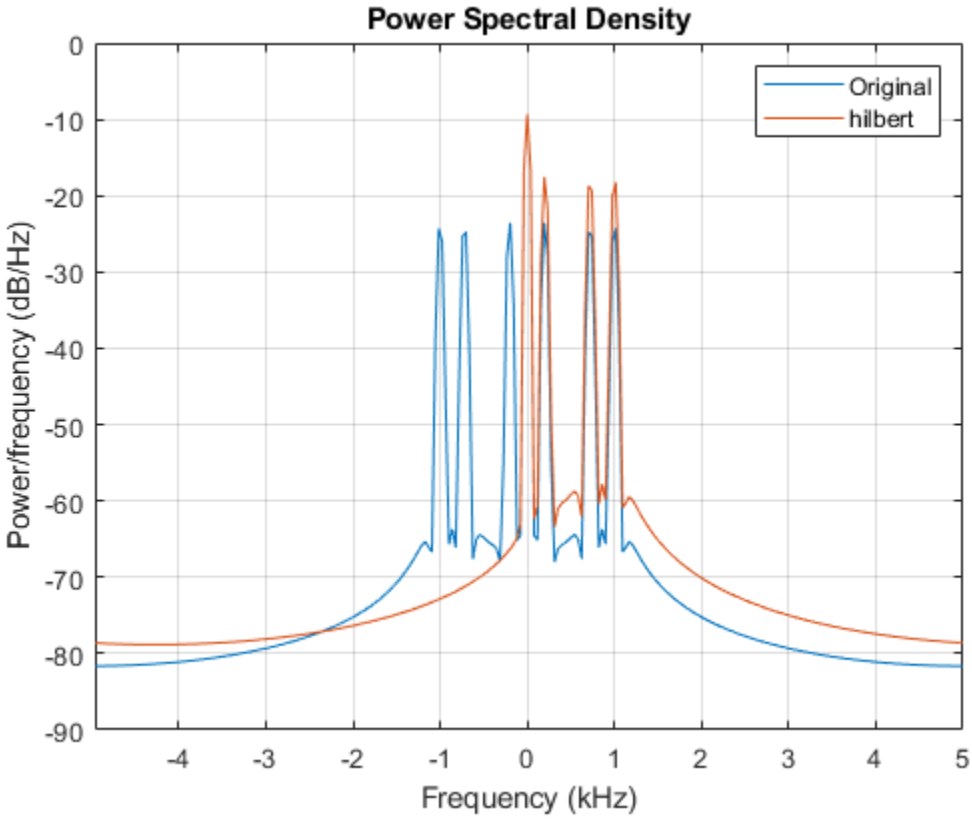
Generate a sequence composed of three sinusoids with frequencies 203, 721, and 1001 Hz. The sequence is sampled at 10 kHz for about 1 second. Use the `hilbert` function to compute the analytic signal. Plot it between 0.01 seconds and 0.03 seconds.

```
fs = 1e4;
t = 0:1/fs:1;

x = 2.5 + cos(2*pi*203*t) + sin(2*pi*721*t) + cos(2*pi*1001*t);

y = hilbert(x);

plot(t,real(y),t,imag(y))
xlim([0.01 0.03])
legend('real','imaginary')
title('hilbert Function')
```



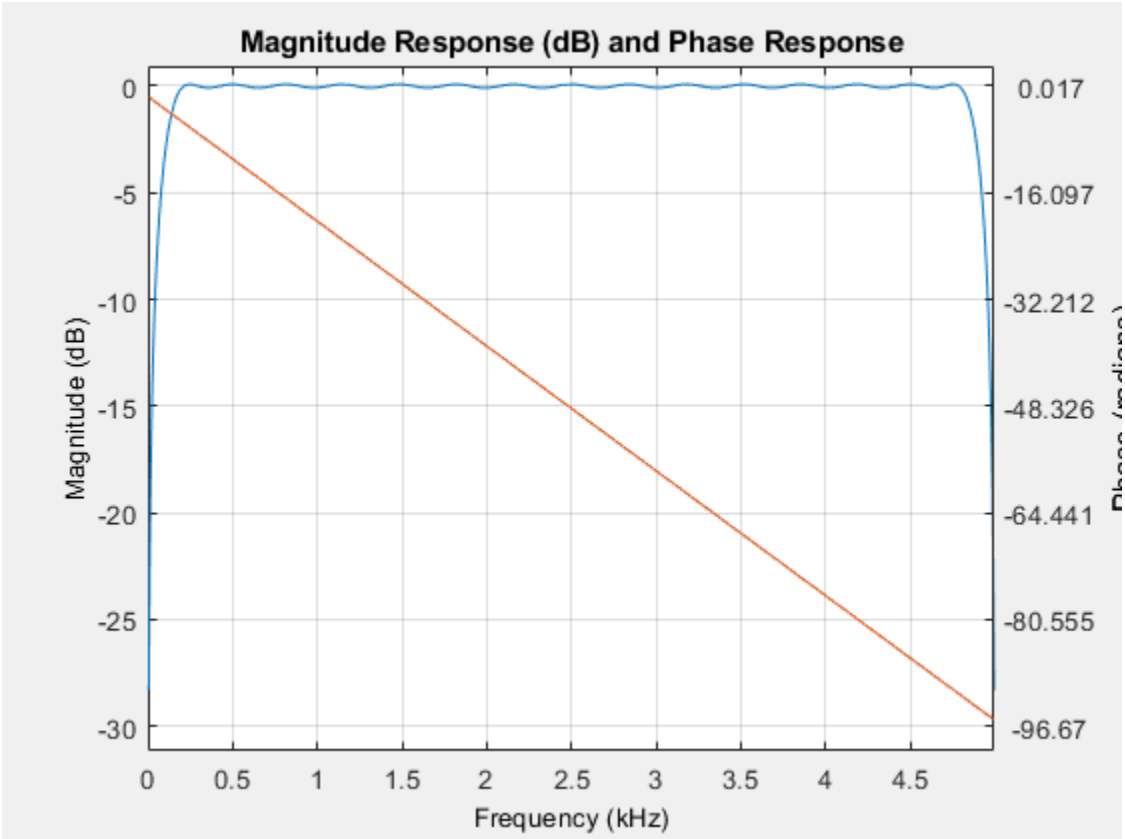
Compute Welch estimates of the power spectral densities of the original sequence and the analytic signal. Divide the sequences into Hamming-windowed, nonoverlapping sections of length 256. Verify that the analytic signal has no power at negative frequencies.

```
pwelch([x;y].',256,0,[],fs,'centered')
legend('Original','hilbert')
```



Use the `designfilt` function to design a 60th-order Hilbert transformer FIR filter. Specify a transition width of 400 Hz. Visualize the frequency response of the filter.

```
fo = 60;

d = designfilt('hilbertfir','FilterOrder',fo, ...
       'TransitionWidth',400,'SampleRate',fs);

freqz(d,1024,fs)
```



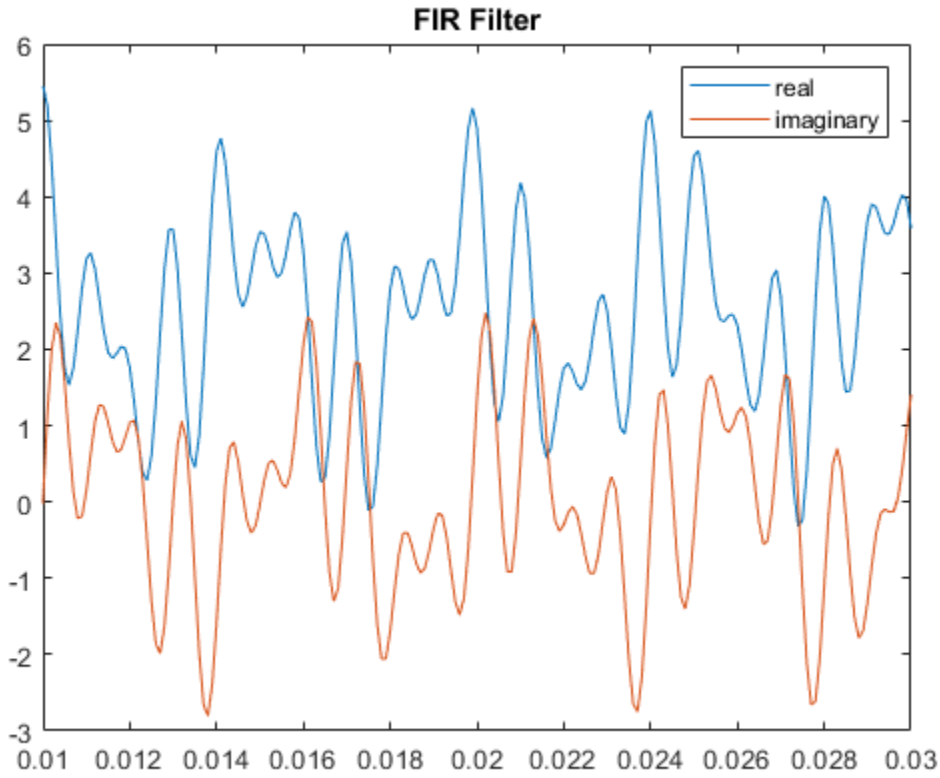
Filter the sinusoidal sequence to approximate the imaginary part of the analytic signal.

```
hb = filter(d,x);
```

The group delay of the filter, `grd`, is equal to one-half the filter order. Compensate for this delay. Remove the first `grd` samples of the imaginary part and the last `grd` samples of the real part and the time vector. Plot the result between 0.01 seconds and 0.03 seconds.

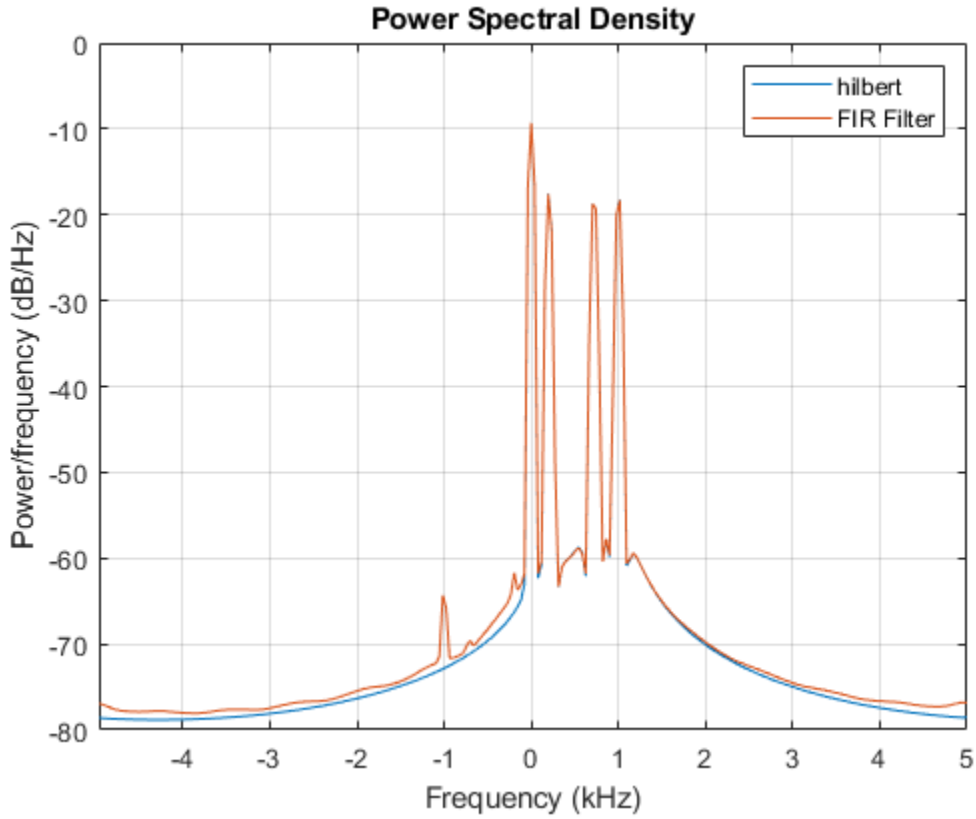
```
grd = fo/2;

y2 = x(1:end-grd) + 1j*hb(grd+1:end);
t2 = t(1:end-grd);

plot(t2,real(y2),t2,imag(y2))
```

```
xlim([0.01 0.03])
legend('real','imaginary')
title('FIR Filter')
```



Estimate the power spectral density (PSD) of the approximate analytic signal and compare it to the `hilbert` result.

```
pwelch([y;[y2 zeros(1,grd)]].',256,0,[],fs,'centered')
legend('hilbert','FIR Filter')
```



See Also

`designfilt` | `hilbert`

Hilbert Transform and Instantaneous Frequency

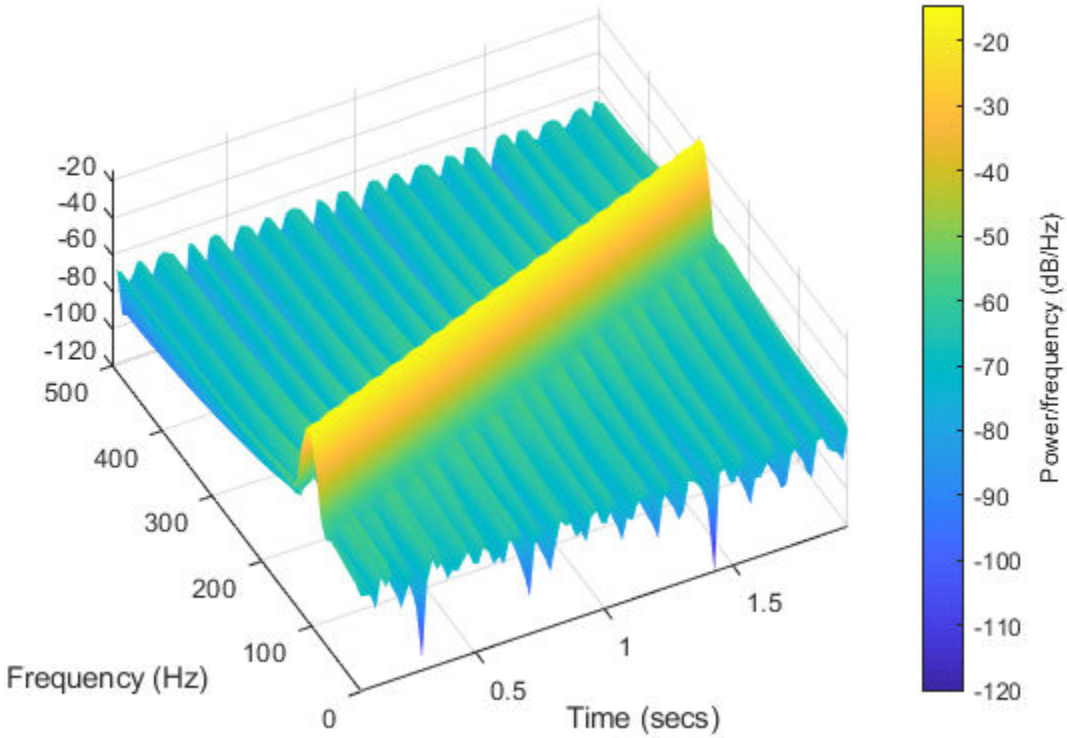
The Hilbert transform estimates the instantaneous frequency of a signal for monocomponent signals only. A monocomponent signal is described in the time-frequency plane by a single "ridge." The set of monocomponent signals includes single sinusoids, but also includes signals like chirps.

Generate a chirp sampled at 1 kHz for two seconds. Specify the chirp so its frequency is initially 100 Hz and increases to 200 Hz after one second.

```
Fs = 1000;
t = 0:1/Fs:2-1/Fs;
y = chirp(t,100,1,200);
```

Estimate the spectrum of the chirp using the short-time Fourier transform implemented in the `spectrogram` function. Divide the signal into sections of length 100, windowed with a Hamming window. Specify 80 samples of overlap between adjoining sections and evaluate the spectrum at $\lfloor 100/2+1 \rfloor = 51$ frequencies.

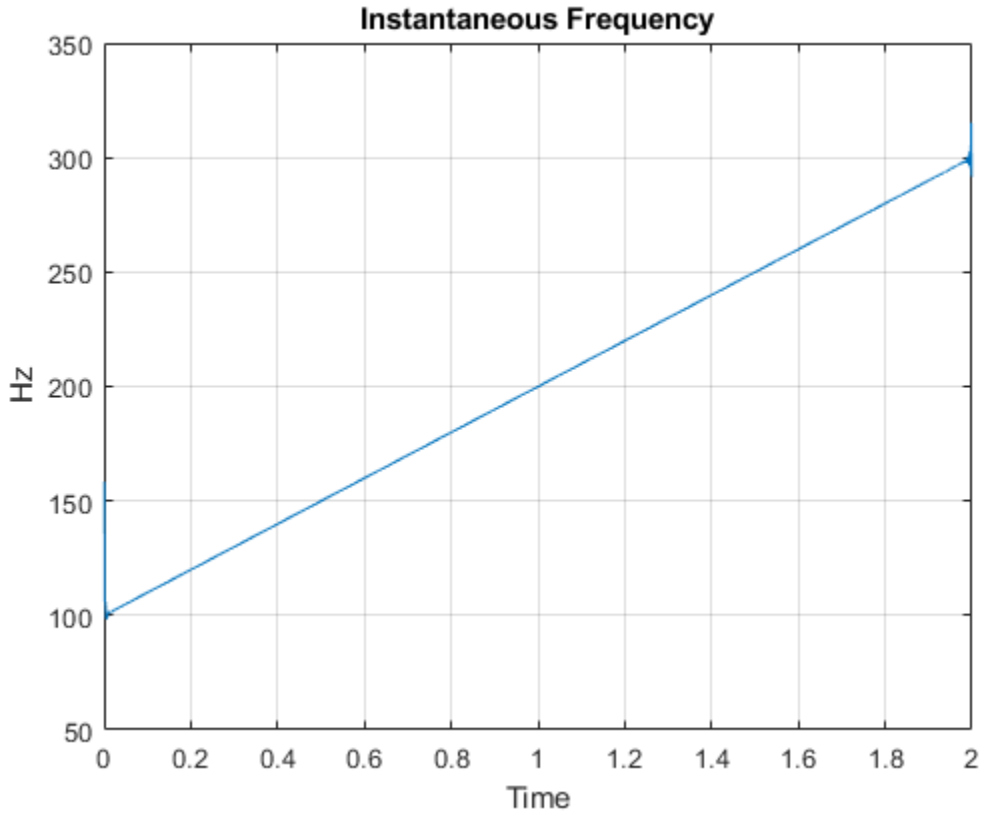
```
spectrogram(y,100,80,100,Fs,'yaxis')
view(-27,62)
shading interp
```



The signal is well described by a single peak frequency at each point in time.

Compute the analytic signal and differentiate its phase to measure the instantaneous frequency. The scaled derivative yields a meaningful estimate.

```
z = hilbert(y);
instfreq = Fs/(2*pi)*diff(unwrap(angle(z)));

plot(t(2:end),instfreq)
xlabel('Time')
ylabel('Hz')
grid on
title('Instantaneous Frequency')
```

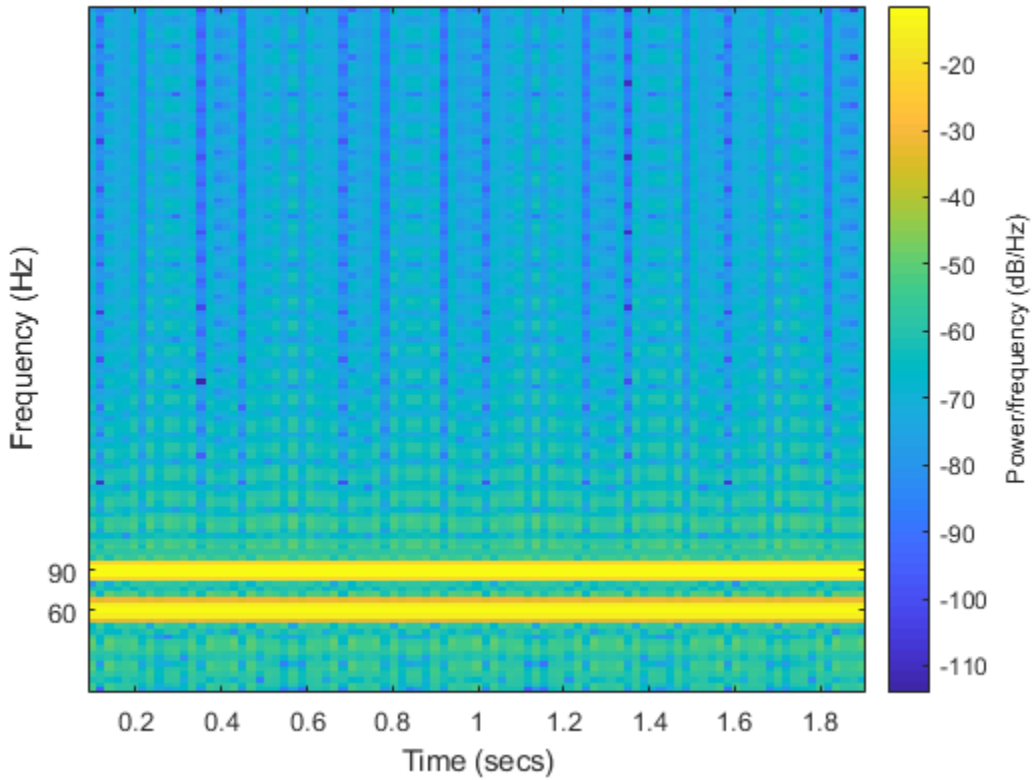


Repeat the procedure, this time for a signal that is not monocomponent.

Generate a sum of two sinusoids of frequencies 60 Hz and 90 Hz, sampled at 1023 Hz for two seconds. Compute and plot the spectrogram. Use Hamming-windowed sections that have lengths of 200 and sample overlaps of 180. Evaluate the spectrum at $\lfloor 256/2+1 \rfloor = 129$ frequencies.

```
Fs = 1023;
t = 0:1/Fs:2;
x = sin(2*pi*60*t)+sin(2*pi*90*t);

spectrogram(x,200,180,256,Fs,'yaxis')
ax = gca;
```

```
ax.YTick = [60 90];
shading interp
```

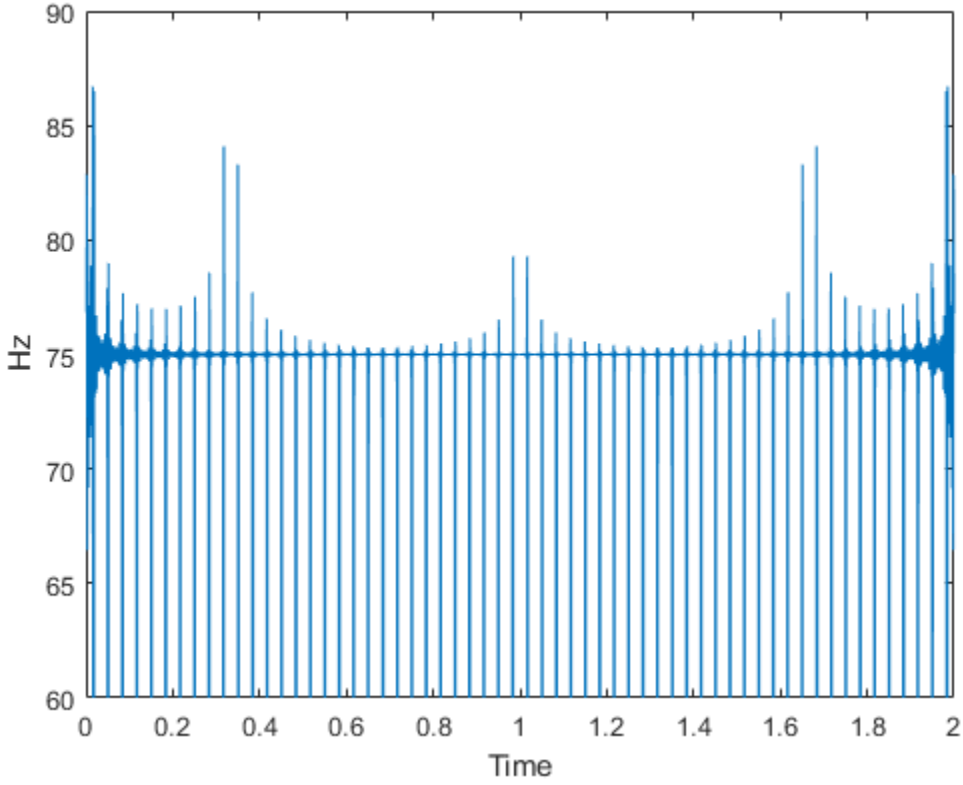


Each time point shows the presence of the two components.

Compute the analytic signal and differentiate its phase. Zoom in to the region enclosing the frequencies of the sinusoids.

```
z = hilbert(x);
instfreq = Fs/(2*pi)*diff(unwrap(angle(z)));
plot(t(2:end),instfreq)

ylim([60 90])
```

```
xlabel('Time')
ylabel('Hz')
```



The analytic signal predicts an instantaneous frequency that is the average of the sinusoid frequencies.

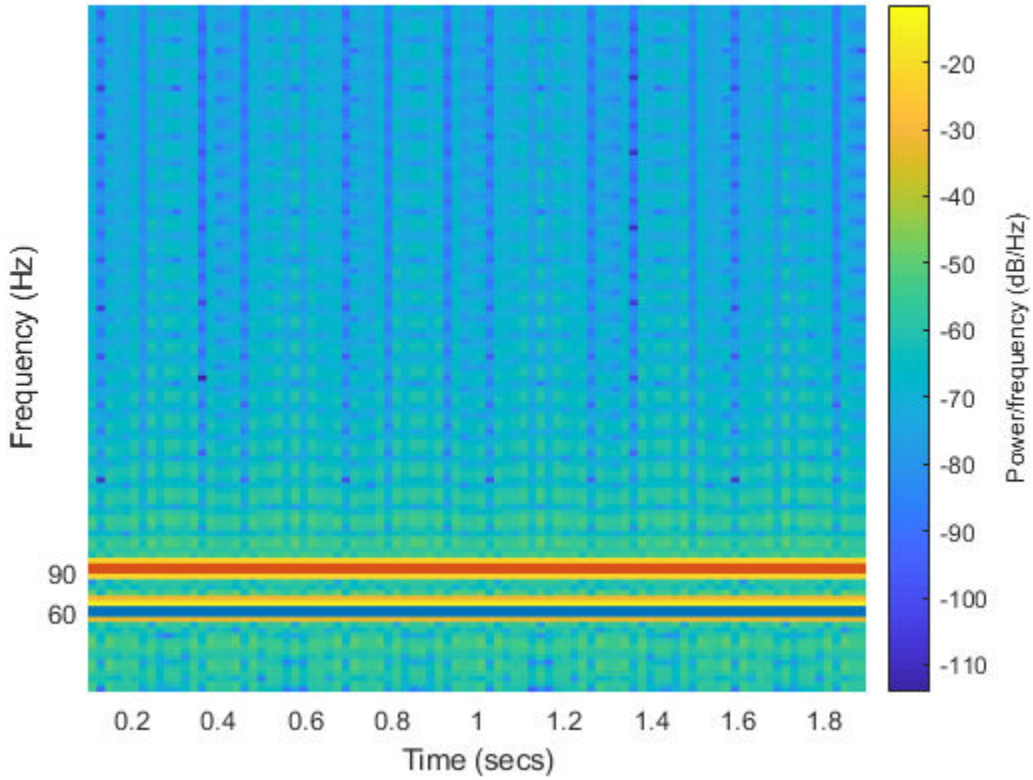
If you want to estimate both frequencies as functions of time, you can use `spectrogram`. The ridges are peaks of the power spectral density.

```
[s,f,tt,p] = spectrogram(x,200,180,256,Fs);

numcomp = 2;
nt = length(tt);
```

```
P = zeros(nt,numcomp);
K = zeros(nt,numcomp);

for k = 1:nt
    [pk,lc] = findpeaks(p(:,k),f,'SortStr','descend','NPeaks',numcomp);
    P(k,:) = pk;
    K(k,:) = lc;
end
```

Plot the results.

```
spectrogram(x,200,180,256,Fs,'yaxis')
shading interp
view(2)

hold on
for k = 1:numcomp
    plot3(tt,K(:,k),10*log10(P(:,k)),'linewidth',4)
end
hold off
ax = gca;
ax.YTick = [60 90];
```



See Also

`hilbert` | `spectrogram`

Related Examples

- “Detect Closely Spaced Sinusoids” on page 14-33

Detect Closely Spaced Sinusoids

Consider a sinusoid, $f(x) = e^{j2\pi\nu x}$, windowed with a Gaussian window, $g(t) = e^{-\pi t^2}$. The short-time transform is

$$V_g f(t,\eta) = e^{j2\pi\nu t} \int_{-\infty}^{\infty} e^{-\pi(x-t)^2} e^{-j2\pi(x-t)(\eta-\nu)}\, dx = e^{-\pi(\eta-\nu)^2} e^{j2\pi\nu t}.$$

When viewed as a function of frequency, the transform combines a constant (in time) oscillation at ν with Gaussian decay away from it. The synchrosqueezing estimate of the instantaneous frequency,

$$\Omega_g f(t,\eta) = \frac{1}{j2\pi} \frac{e^{-\pi(\eta-\nu)^2} \dfrac{\partial}{\partial t} e^{j2\pi\nu t}}{e^{-\pi(\eta-\nu)^2} e^{j2\pi\nu t}} = \nu,$$

equals the value obtained by using the standard definition, $(2\pi)^{-1} d \arg f(x)/dx$. For a superposition of sinusoids,

$$f(x) = \sum_{k=1}^{K} A_k e^{j2\pi\nu_k x},$$

the short-time transform becomes

$$V_g f(t,\eta) = \sum_{k=1}^{K} A_k e^{-\pi(\eta-\nu_k)^2} e^{j2\pi\nu_k t}.$$

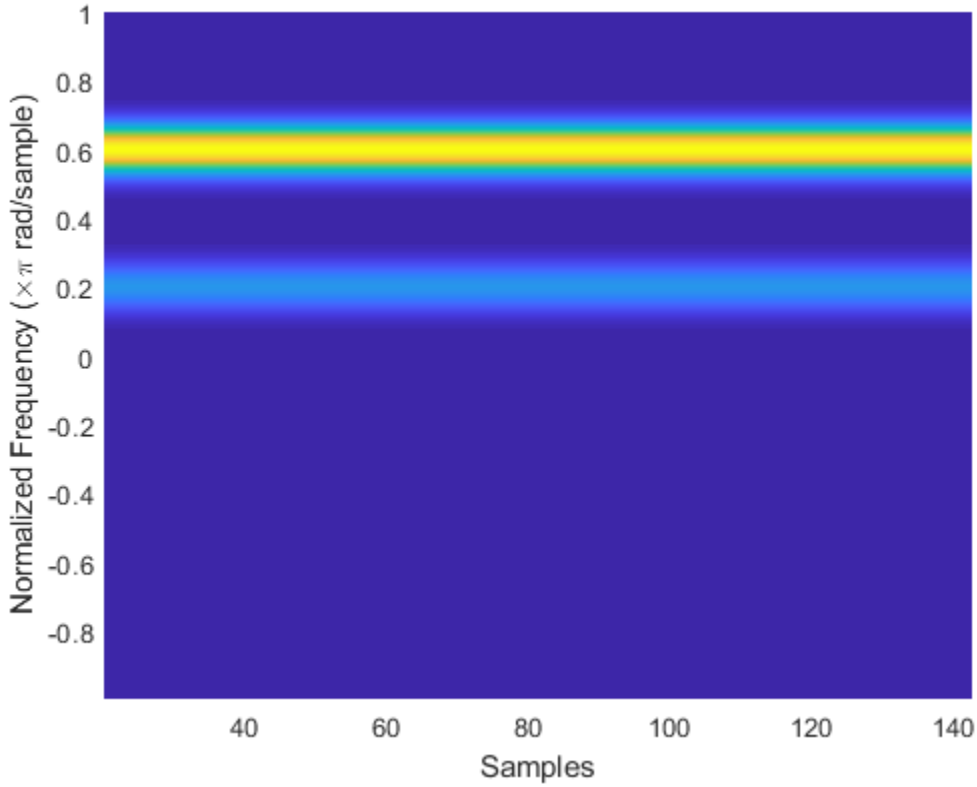
Create 1024 samples of a signal consisting of two sinusoids. One sinusoid has a normalized frequency of $\omega_0 = \pi/5$ rad/sample. The other sinusoid has three times the frequency and three times the amplitude.

```
N = 1024;
n = 0:N-1;
```

```
w0 = pi/5;
x = exp(1j*w0*n)+3*exp(1j*3*w0*n);
```

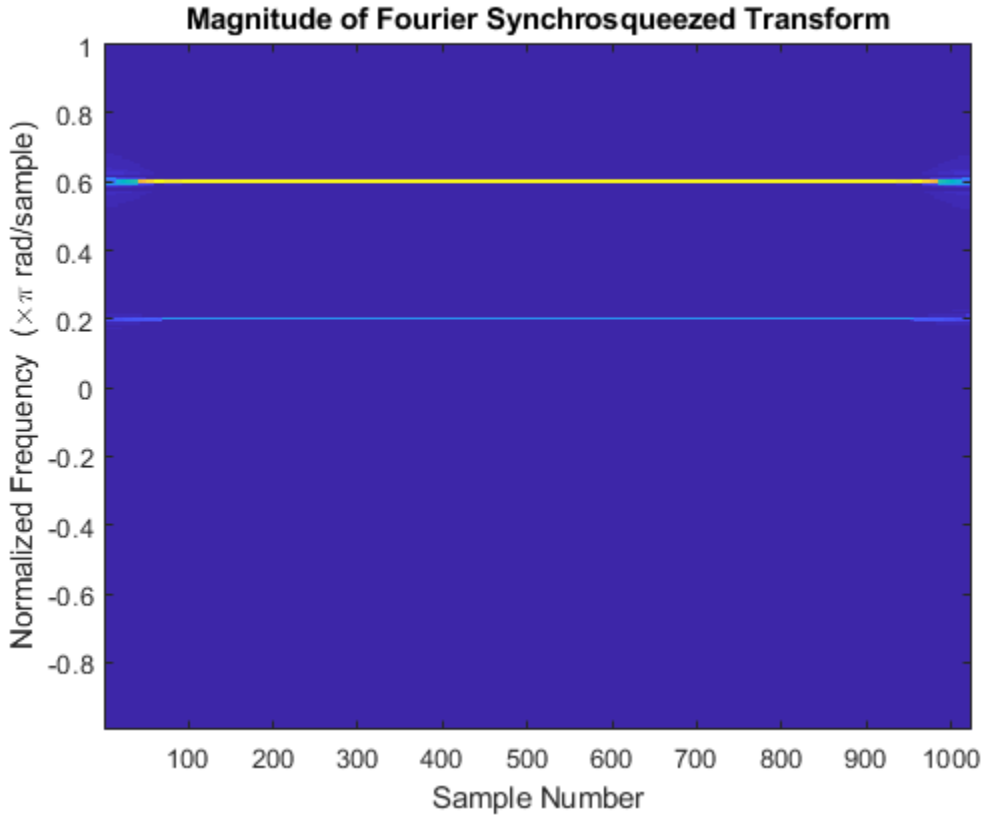
Compute the short-time Fourier transform of the signal. Use a 256-sample Gaussian window with $\alpha = 20$, 255 samples of overlap between adjoining sections, and 1024 DFT points. Plot the absolute value of the transform.

```
Nw = 256;
nfft = 1024;
alpha = 20;

[s,w,t] = spectrogram(x,gausswin(Nw,alpha),Nw-1,nfft,'centered');

surf(t,w/pi,abs(s),'EdgeColor','none')
view(2)
axis tight
xlabel('Samples')
ylabel('Normalized Frequency (\times\pi rad/sample)')
```



The Fourier synchrosqueezed transform results in a sharper, better localized estimate of the spectrum.

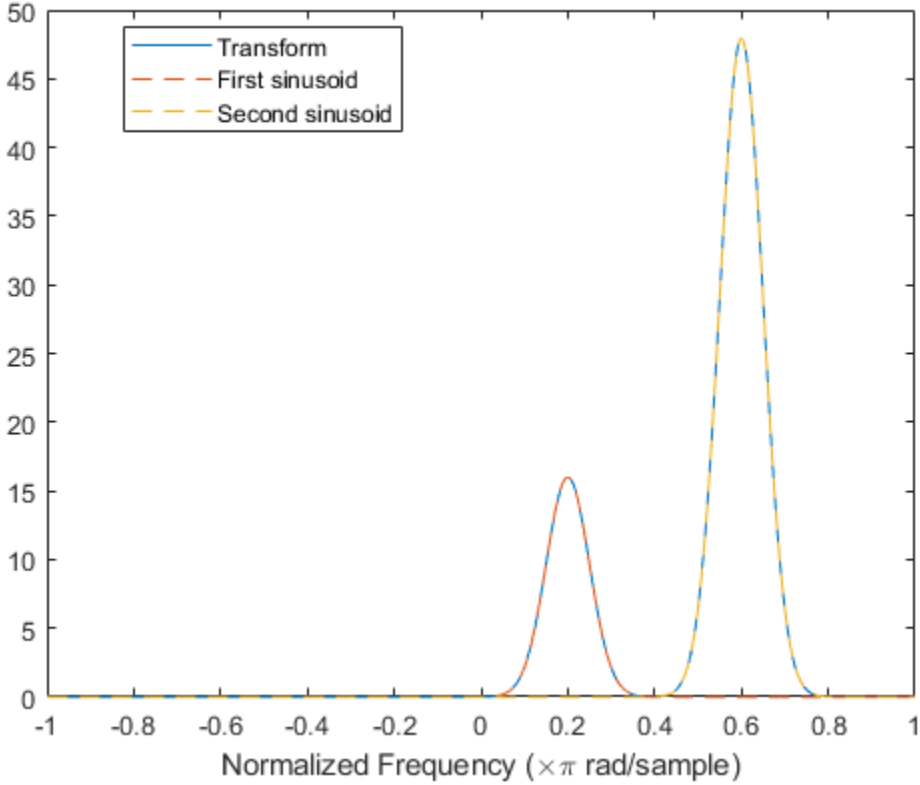
```
[ss,sw,st] = fsst(x,[],gausswin(Nw,alpha));

fsst(x,'yaxis')
```



The sinusoids are visible as constant oscillations at the expected frequency values. To see that the decay away from the ridges is Gaussian, plot an instantaneous value of the transform and overlay two instances of a Gaussian. Express the Gaussian amplitude and standard deviation in terms of α and the window length. Recall that the standard deviation of the frequency-domain window is the reciprocal of the standard deviation of the time-domain window.

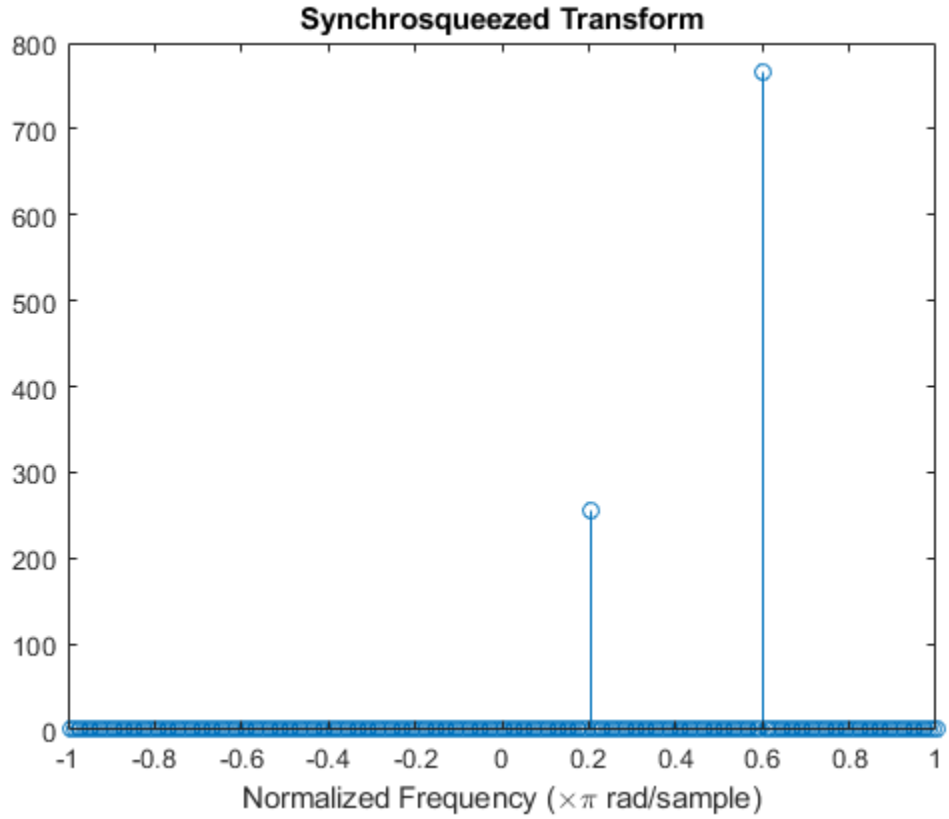
```
rstdev = (Nw-1)/(2*alpha);
amp = rstdev*sqrt(2*pi);

instransf = abs(s(:,128));

plot(w/pi,instransf)
```

```
hold on
plot(w/pi,[1 3]*amp.*exp(-rstdev^2/2*(w-[1 3]*w0).^2),'--')
hold off
xlabel('Normalized Frequency (\times\pi rad/sample)')
lg = legend('Transform','First sinusoid','Second sinusoid');
lg.Location = 'best';
```



The Fourier synchrosqueezed transform concentrates the energy content of the signal at the estimated instantaneous frequencies.

```
stem(sw/pi,abs(ss(:,128)))
xlabel('Normalized Frequency (\times\pi rad/sample)')
title('Synchrosqueezed Transform')
```



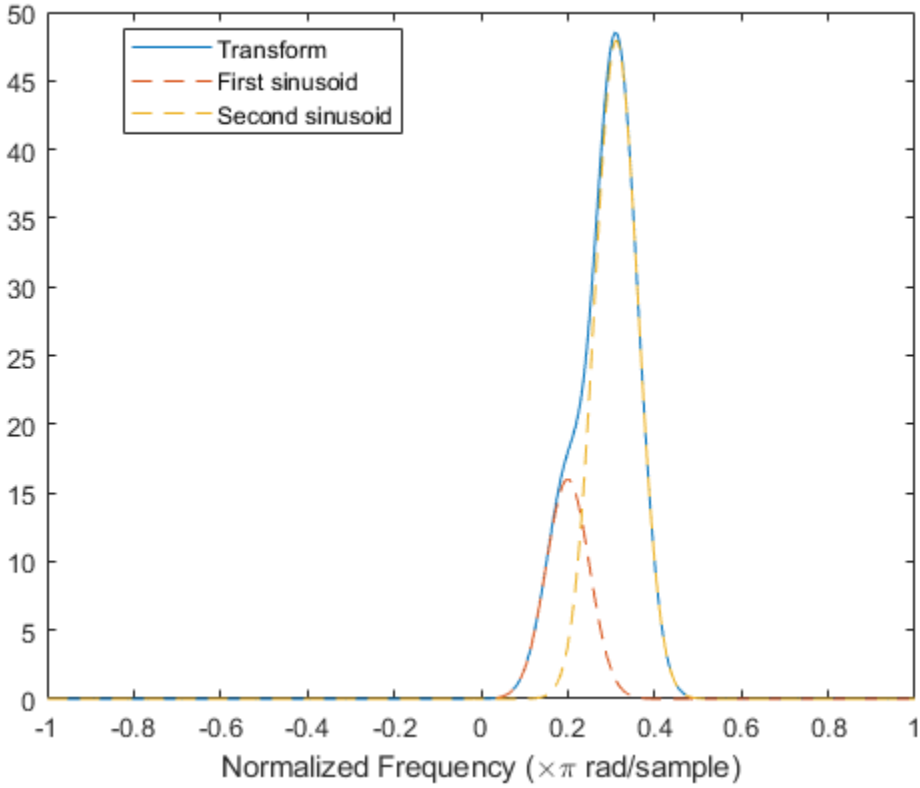
The synchrosqueezed estimates of instantaneous frequency are valid only if the sinusoids are separated by more than 2Δ, where

$$\Delta = \frac{1}{\sigma}\sqrt{2\log 2}$$

for a Gaussian window and σ is the standard deviation.

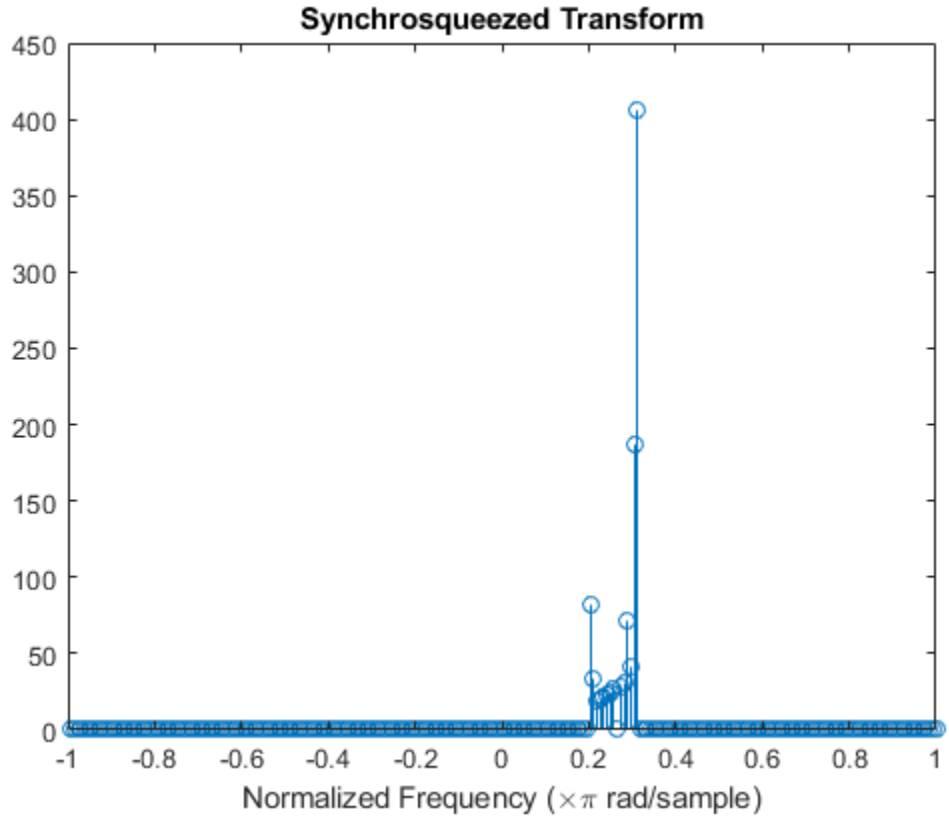
Repeat the previous calculation, but now specify that the second sinusoid has a normalized frequency of $\omega_0 + 1.9\Delta$ rad/sample.

```
D = sqrt(2*log(2))/rstdev;

w1 = w0+1.9*D;

x = exp(1j*w0*n)+3*exp(1j*w1*n);

[s,w,t] = spectrogram(x,gausswin(Nw,alpha),Nw-1,nfft,'centered');
instransf = abs(s(:,20));

plot(w/pi,instransf)
hold on
plot(w/pi,[1 3]*amp.*exp(-rstdev^2/2*(w-[w0 w1]).^2),'--')
hold off
xlabel('Normalized Frequency (\times\pi rad/sample)')
lg = legend('Transform','First sinusoid','Second sinusoid');
lg.Location = 'best';
```



The Fourier synchrosqueezed transform cannot resolve the sinusoids well because $|\omega_1 - \omega_0| < 2\Delta$. The spectral estimates decrease significantly in value.

```
[ss,sw,st] = fsst(x,[],gausswin(Nw,alpha));

stem(sw/pi,abs(ss(:,128)))
xlabel('Normalized Frequency (\times\pi rad/sample)')
title('Synchrosqueezed Transform')
```



See Also

`fsst` | `gausswin` | `ifsst` | `spectrogram`

Related Examples

- “Hilbert Transform and Instantaneous Frequency” on page 14-26
- “Instantaneous Frequency of Complex Chirp” on page 14-42
- “Practical Introduction to Time-Frequency Analysis”

Instantaneous Frequency of Complex Chirp

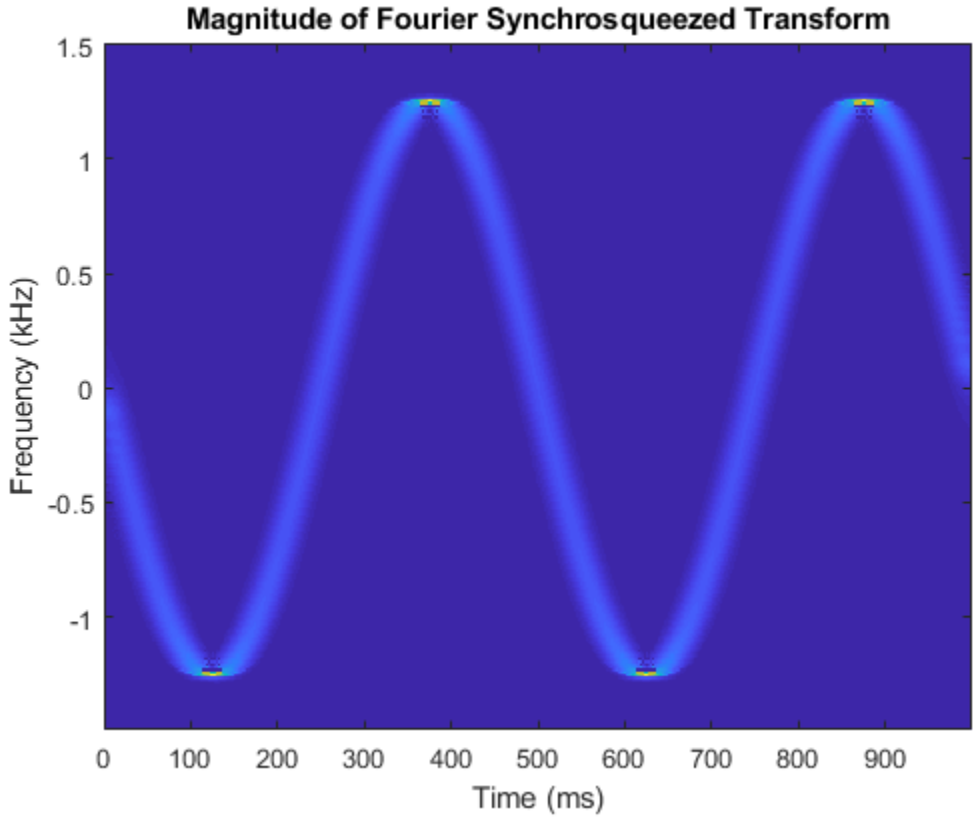
This example shows how to compute the instantaneous frequency of a signal using the Fourier synchrosqueezed transform.

Generate a chirp with sinusoidally varying frequency content. The signal is embedded in white Gaussian noise and sampled at 3 kHz for 1 second.

```
fs = 3000;
t = 0:1/fs:1-1/fs;

x = exp(2j*pi*100*cos(2*pi*2*t)) + randn(size(t))/100;
```

Compute and plot the Fourier synchrosqueezed transform of the signal. Display the time on the *x*-axis and the frequency on the *y*-axis.

```
fsst(x,fs,'yaxis')
```

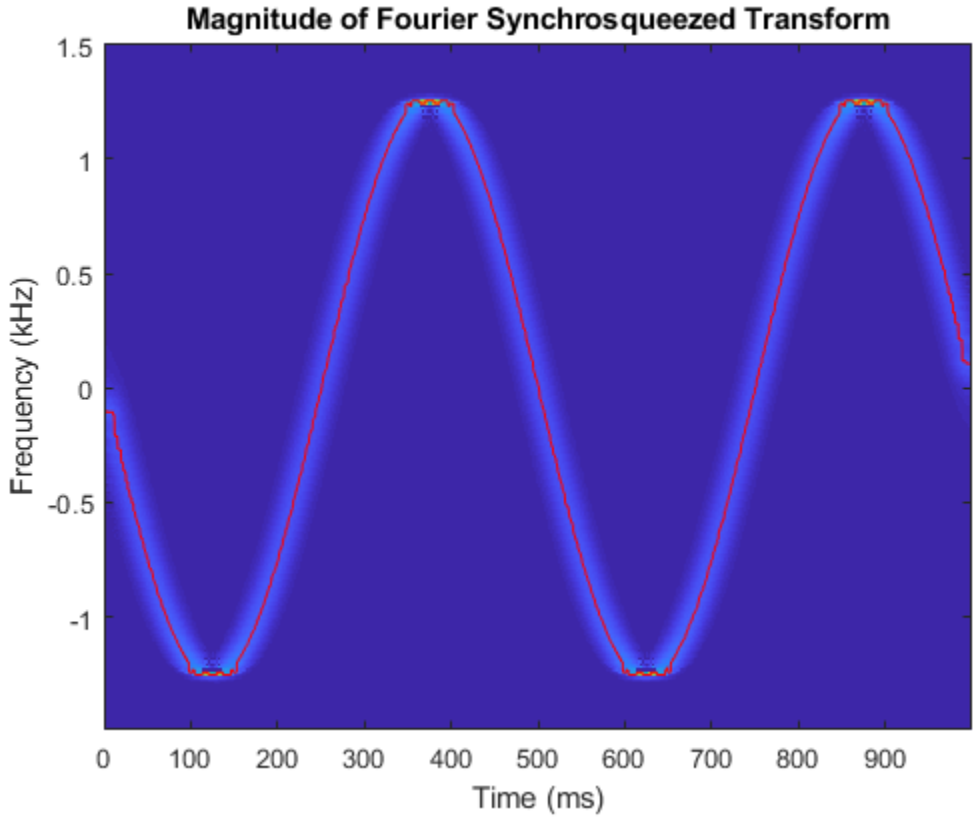


Find the instantaneous frequency of the signal by extracting the maximum-energy time-frequency ridge of the Fourier Synchrosqueezed transform.

```
[sst,f,tfs] = fsst(x,fs);

fridge = tfridge(sst,f);
```

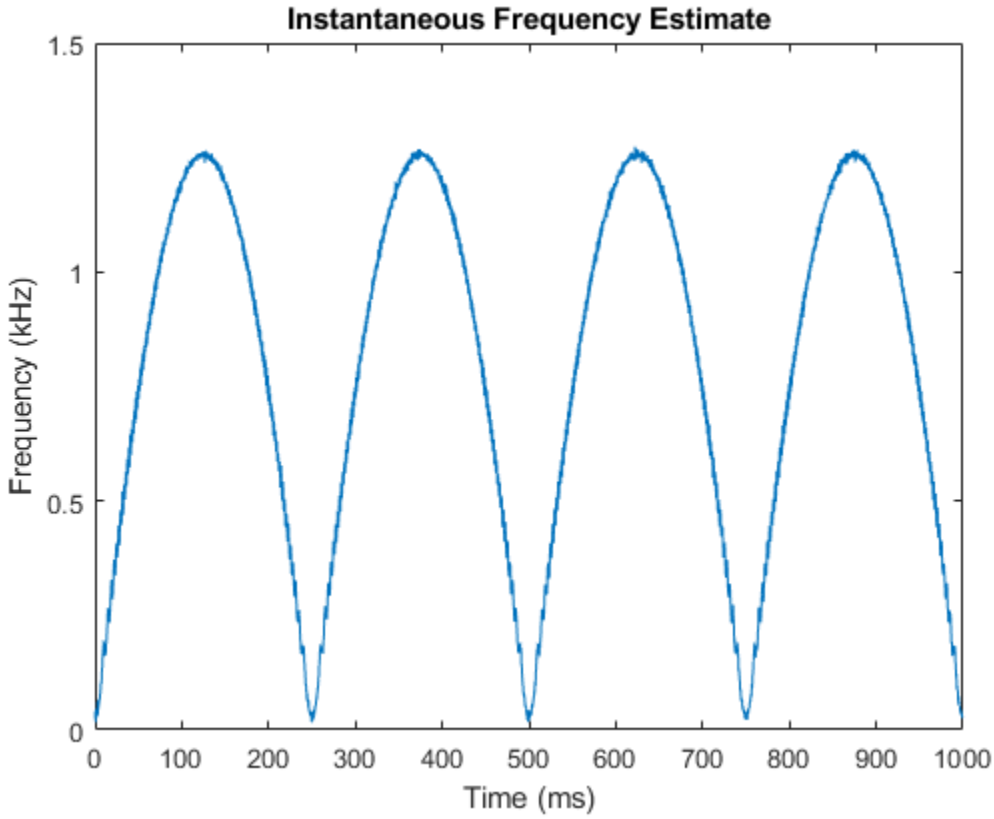
Overlay the ridge on the transform plot. Convert time to milliseconds and frequency to kHz.

```
hold on
plot(t*1000,fridge/1000,'r')
hold off
```



For a real signal, you can find the instantaneous frequency more easily using the `instfreq` function. For example, display the instantaneous frequency of the real part of the complex chirp by computing the analytic signal and differentiating its phase.

```
ax = real(x);

instfreq(ax,fs,'Method','hilbert')
```



See Also

`fsst` | `ifsst` | `instfreq` | `pspectrum` | `spectrogram` | `tfridge`

Related Examples

- “Hilbert Transform and Instantaneous Frequency” on page 14-26
- “Detect Closely Spaced Sinusoids” on page 14-33
- “Practical Introduction to Time-Frequency Analysis”

Single-Sideband Amplitude Modulation

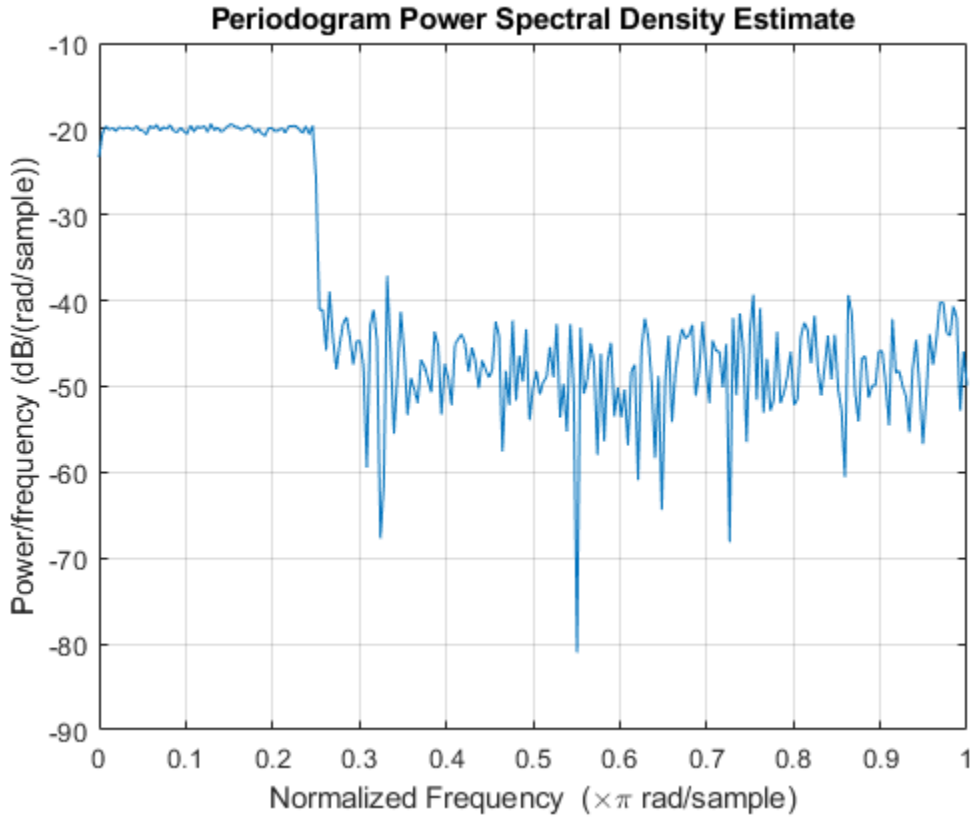
This example shows how to use the Hilbert transform to carry out single-sideband (SSB) amplitude modulation (AM) of a signal. Single-sideband AM signals have less bandwidth than normal AM signals.

Generate 512 samples of a simulated broadband signal using the `sinc` function. Specify a bandwidth of $\pi/4$ rad/sample.

```
N = 512;
n = 0:N-1;

bw = 1/4;
x = sinc((n-N/2)*bw);
```

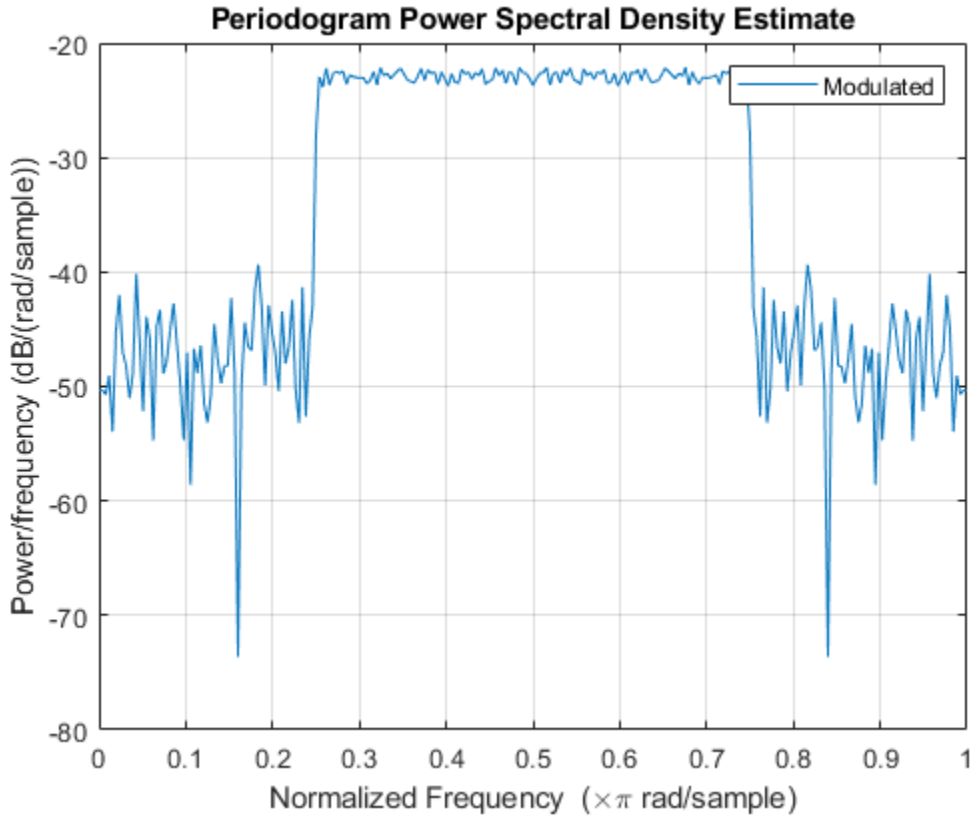
Add white Gaussian noise such that the signal-to-noise ratio is 20 dB. Reset the random number generator for reproducible results. Use the `periodogram` function to estimate the power spectral density (PSD) of the signal.

```
rng default

SNR = 20;
noise = randn(size(x))*std(x)/db2mag(SNR);
x = x + noise;

periodogram(x)
```

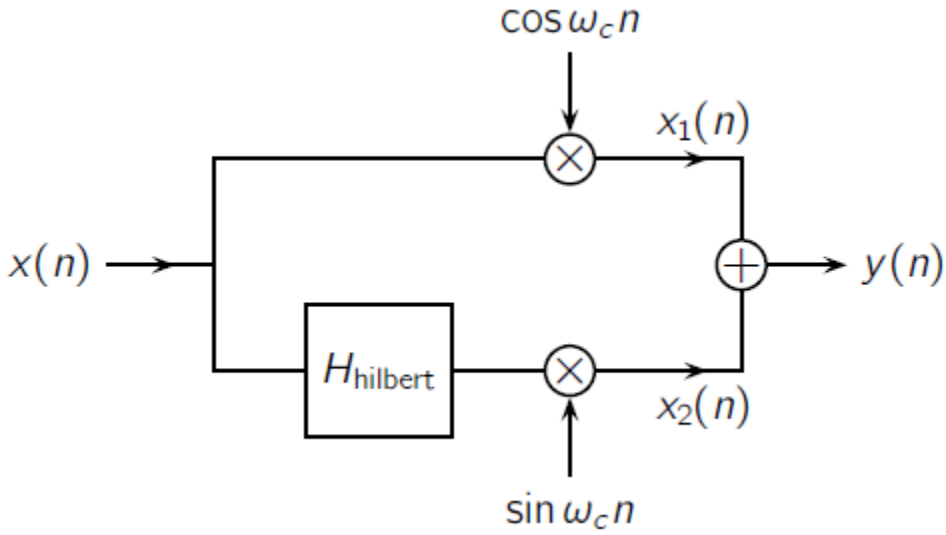


Amplitude modulate the signal using a cosine of carrier frequency $\omega_c = \pi/2$. Multiply by $\sqrt{2}$ so that the power of the modulated signal equals the power of the original signal. Estimate the PSD.

```
wc = pi/2;

x1 = x.*cos(wc*n)*sqrt(2);

periodogram(x1)
legend('Modulated')
```



SSB amplitude modulation reduces the bandwidth of the signal by half. To carry out SSB amplitude modulation, you must first compute the Hilbert transform of the signal. Then, amplitude modulate the signal using a sine with the same carrier frequency, ω_c, as before, and add it to the previous signal.

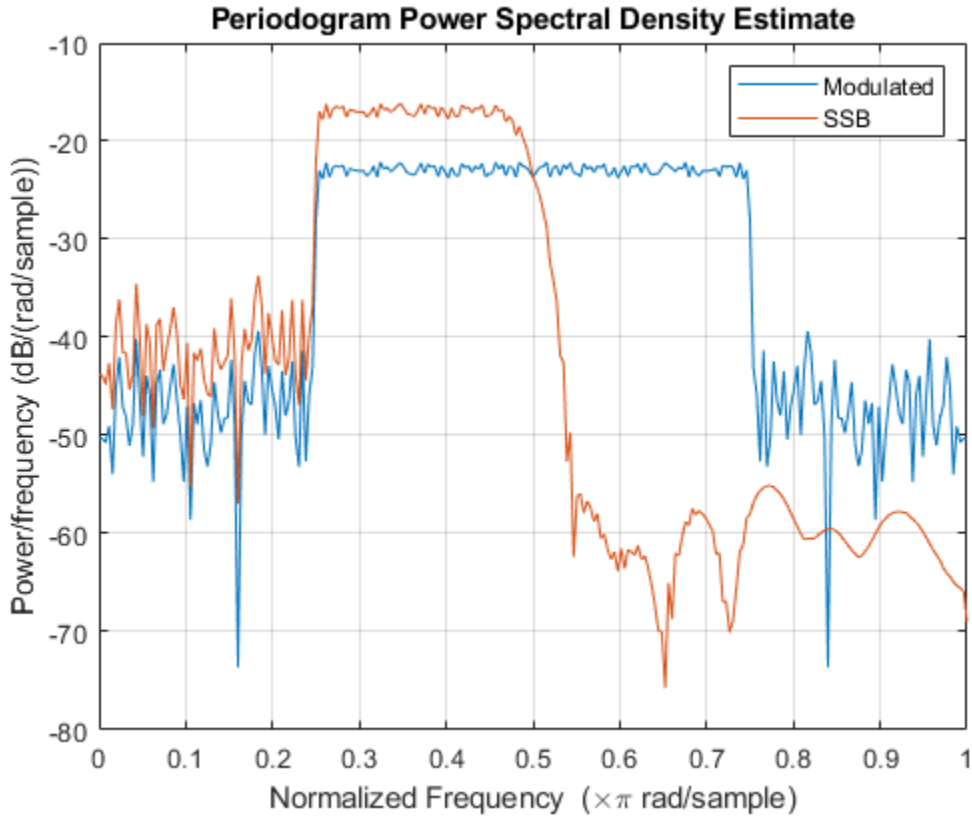


Design a Hilbert transformer using the `designfilt` function. Specify a filter order of 64 and a transition width of 0.1. Filter the signal.

```
Hhilbert = designfilt('hilbertfir','FilterOrder',64, ...
    'TransitionWidth',0.1);

xh = filter(Hhilbert,x);
```

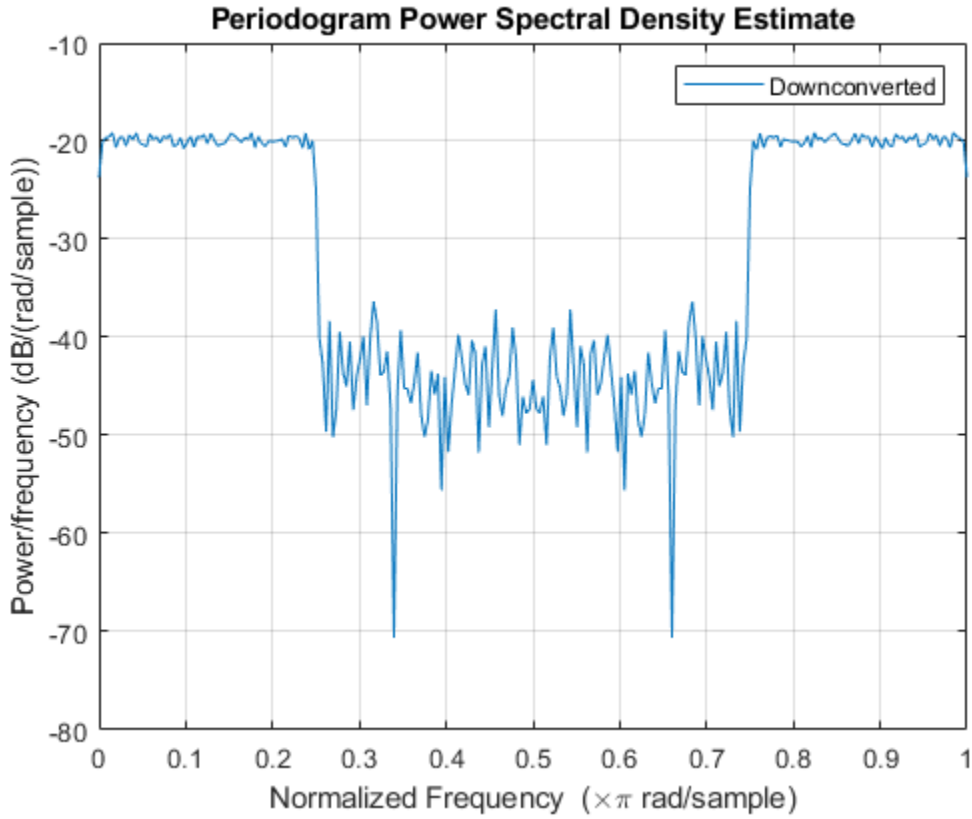
Use the `grpdelay` function to determine the delay, `gd`, introduced by the filter. Compensate for the delay by discarding the first `gd` points of the filtered signal and padding with zeros at the end. Amplitude modulate the result and add it to the original. Compare the PSDs.

```
gd = mean(grpdelay(Hhilbert));
xh = xh(gd+1:end);
eh = zeros(size(x));
eh(1:length(xh)) = xh;

x2 = eh.*sin(wc*n)*sqrt(2);

y = x1+x2;

periodogram([x1;y]')
legend('Modulated','SSB')
```



Downconvert the signal and estimate the PSD.

```
ym = y.*cos(wc*n)*sqrt(2);

periodogram(ym)
legend('Downconverted')
```

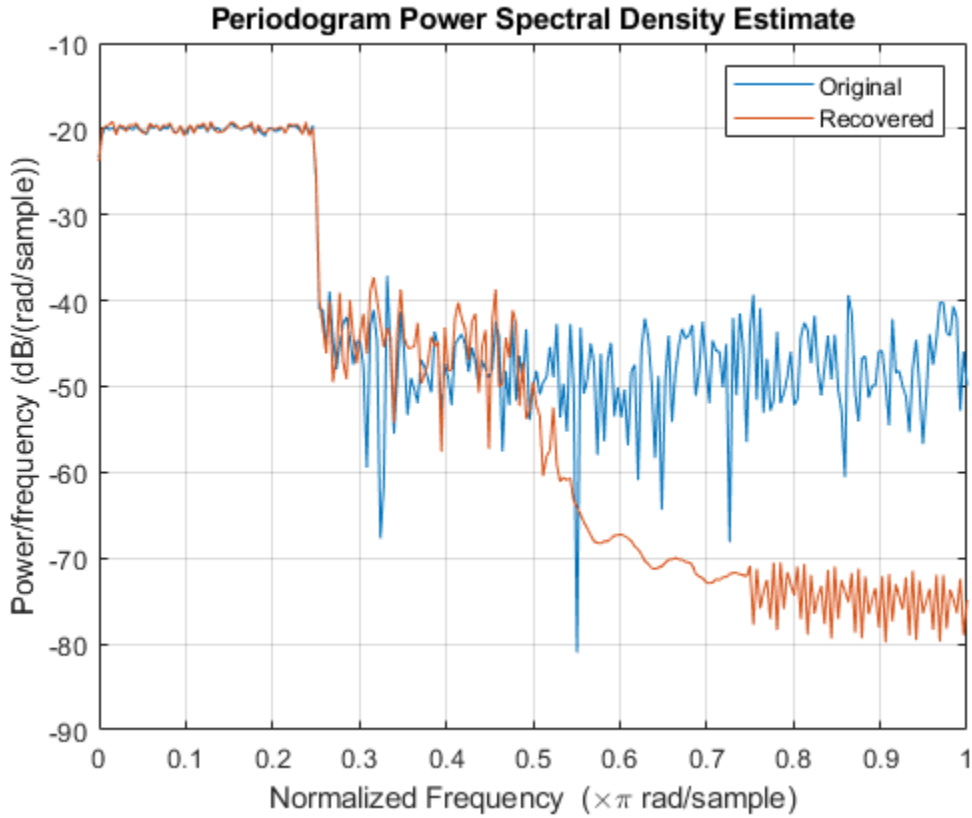


Lowpass filter the modulated signal to recover the original. Specify a 64th-order FIR lowpass filter with a cutoff frequency of $\pi/2$. Compensate for the delay introduced by the filter.

```
d = designfilt('lowpassfir','FilterOrder',64, ...
    'CutoffFrequency',0.5);
dem = filter(d,ym);

gd = mean(grpdelay(d));
dem = dem(gd+1:end);

dm = zeros(size(x));
dm(1:length(dem)) = dem;
```

Estimate the PSD of the filtered signal and compare it to that of the original.

```
periodogram([x;dm]')
legend('Original','Recovered')
```



Use the `snr` function to compare the signal-to-noise ratios of the two signals. Plot the two signals in the time domain.

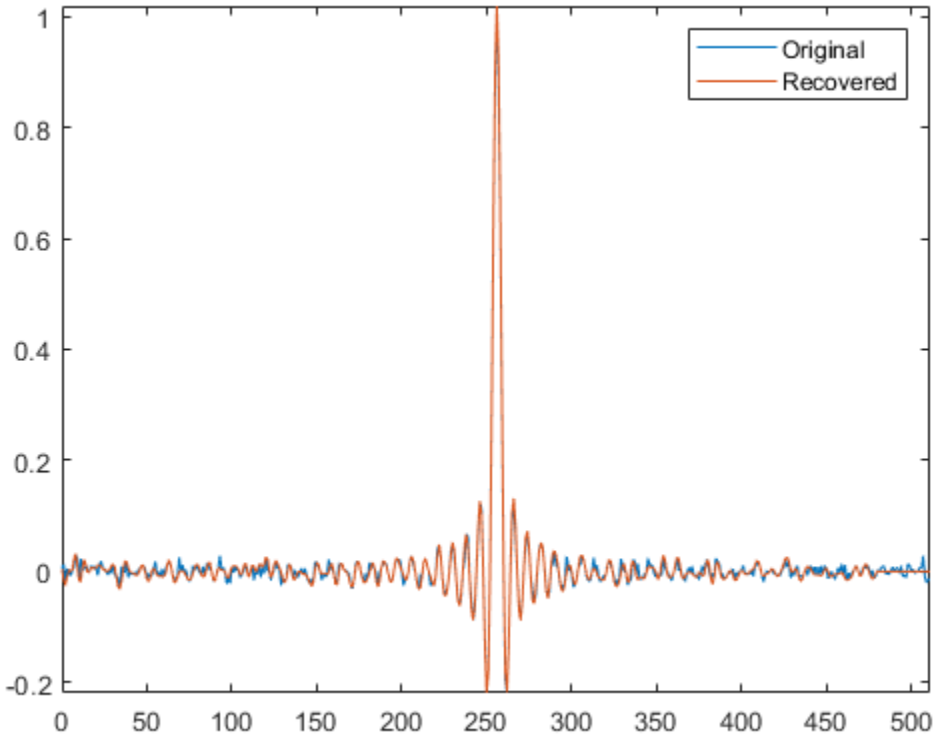
```
snrOrig = snr(x,noise)
```

```
snrOrig = 20.0259
```

```
snrRecv = snr(dm,noise)
```

```
snrRecv = 20.1373
```

```
plot(n,[x;dm]')
legend('Original','Recovered')
axis tight
```



References

Buck, John R., Michael M. Daniel, and Andrew C. Singer. *Computer Explorations in Signals and Systems Using MATLAB*. 2nd Edition. Upper Saddle River, NJ: Prentice Hall, 2002.

See Also

`designfilt` | `periodogram` | `snr`

DCT for Speech Signal Compression

This example shows how to compress a speech signal using the discrete cosine transform (DCT).

Load a file containing the word "strong," spoken by a woman and by a man. The signals are sampled at 8 kHz.

```
load(fullfile(matlabroot,'examples','signal','strong.mat'))

% To hear, type soundsc(her,fs), pause(1), soundsc(him,fs)
```

Use the discrete cosine transform to compress the female voice signal. Decompose the signal into DCT basis vectors. There are as many terms in the decomposition as there are samples in the signal. The expansion coefficients in vector X measure how much energy is stored in each of the components. Sort the coefficients from largest to smallest.

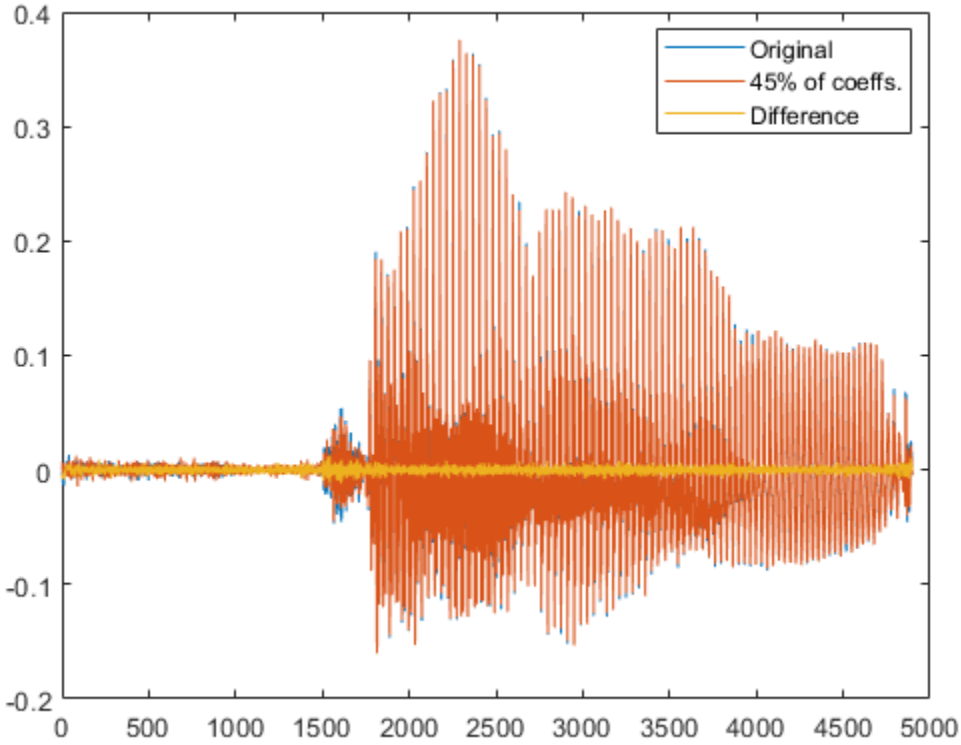
```
x = her';

X = dct(x);

[XX,ind] = sort(abs(X),'descend');
```

Find how many DCT coefficients represent 99.9% of the energy in the signal. Express the number as a percentage of the total.

```
need = 1;
while norm(X(ind(1:need)))/norm(X)<0.999
   need = need+1;
end

xpc = need/length(X)*100;
```

Set to zero the coefficients that contain the remaining 0.1% of the energy. Reconstruct the signal from the compressed representation. Plot the original signal, its reconstruction, and the difference between the two.

```
X(ind(need+1:end)) = 0;
xx = idct(X);

plot([x;xx;x-xx]')
legend('Original',[int2str(xpc) '% of coeffs.'],'Difference', ...
       'Location','best')
```




```
% To hear, type soundsc(x,fs), pause(1), soundsc(xx,fs)
```

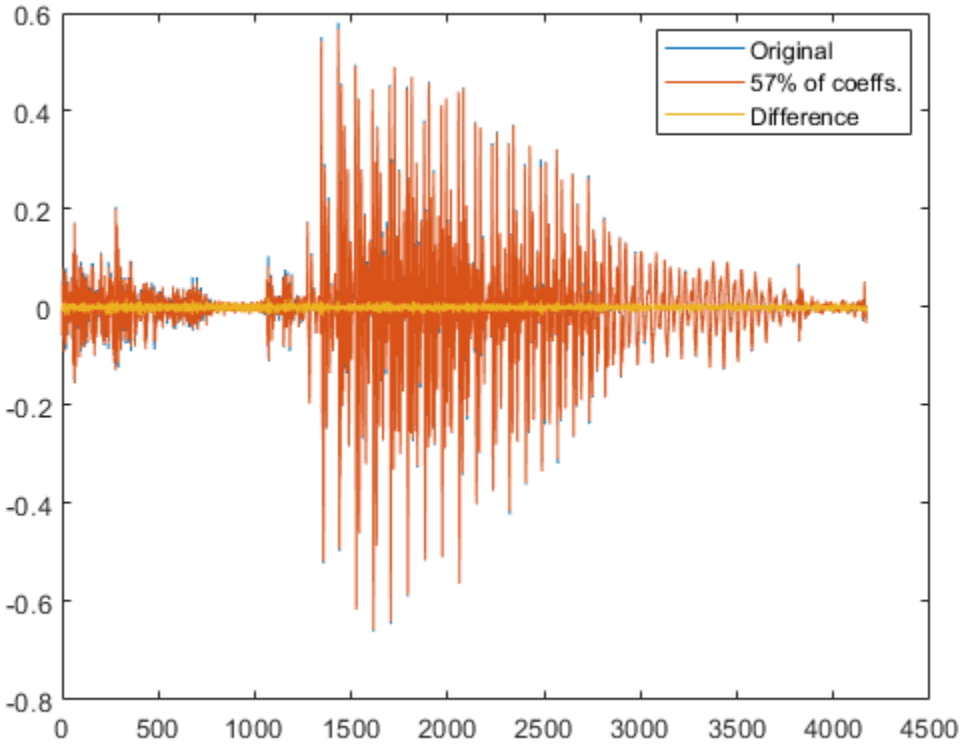
Repeat the analysis for the male voice. Find how many DCT coefficients represent 99.9% of the energy and express the number as a percentage of the total.

```
y = him';
Y = dct(y);

[YY,ind] = sort(abs(Y),'descend');

need = 1;
while norm(Y(ind(1:need)))/norm(Y)<0.999
   need = need+1;
```

```
end

ypc = need/length(Y)*100;
```

Set the rest of the coefficients to zero and reconstruct the signal from the compressed version. Plot the original signal, its reconstruction, and the difference between the two.

```
Y(ind(need+1:end)) = 0;
yy = idct(Y);

plot([y;yy;y-yy]')
legend('Original',[int2str(ypc) '% of coeffs.'],'Difference', ...
       'Location','best')
```



```
% To hear, type soundsc(y,fs), pause(1), soundsc(yy,fs)
```

In both cases, about half of the DCT coefficients suffice to reconstruct the speech signal reasonably. If the required energy fraction is 99%, the number of necessary coefficients reduces to about 20% of the total. The resulting reconstruction is inferior but still intelligible.

Analysis of these and other samples suggests that more coefficients are needed to characterize the man's voice than the woman's.

See Also

`dct` | `idct`

Signal Generation

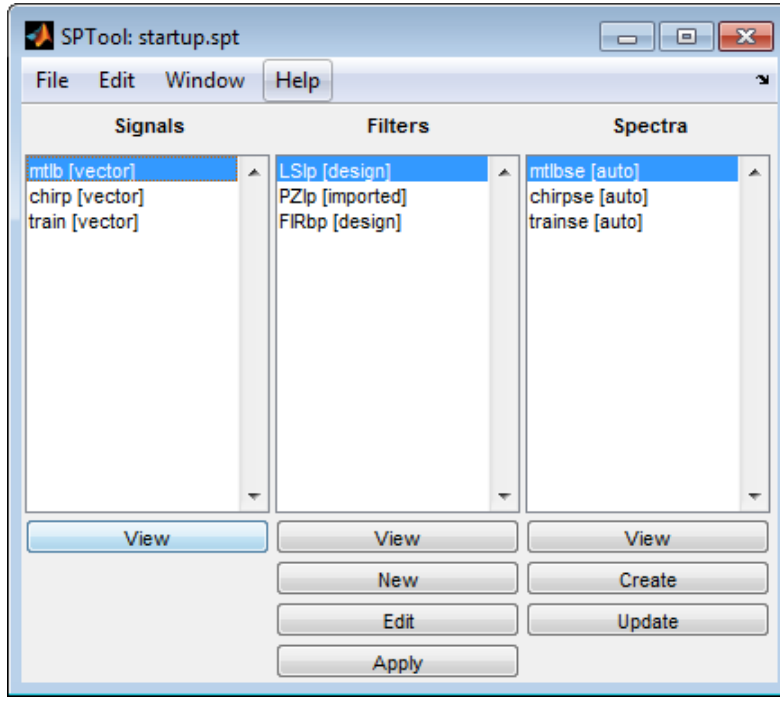
Display Time-Domain Data in Signal Browser

In this section...
“Import and Display Signals” on page 15-3
“Configure the Signal Browser Properties” on page 15-6
“Modify the Signal Browser Display” on page 15-9
“Inspect Your Data (Scaling the Axes and Zooming)” on page 15-11

This example shows how to use and configure the SPTool Signal Browser to display time-domain signals. First, open SPTool by typing

```
sptool
```

at the MATLAB command line. SPTool opens.

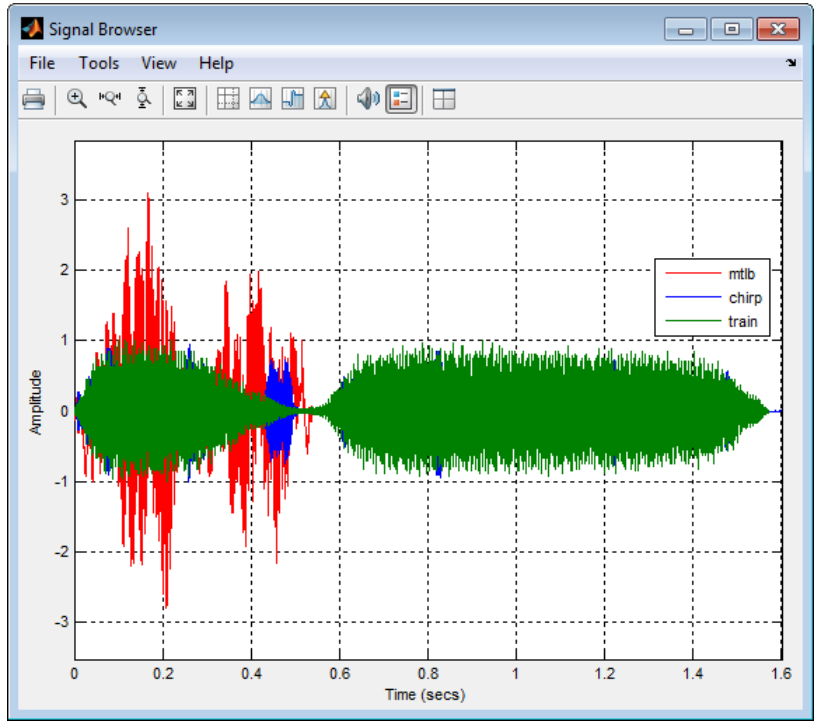


Import and Display Signals

Displaying Multiple Signals

You can display multiple signals in the Signal Browser by selecting more than one entry in the SPTool **Signals** list. In the **Signals** list, first select `mtlb [vector]`. Next, press the **Ctrl** key, and select `chirp [vector]`. Finally, click the **View** button. The Signal Browser opens, displaying both signals.

Any signals that you select in SPTool are now visible in the Signal Browser. You can also select multiple entries using the **Shift** key. In the SPTool **Signals** list, first select `mtlb [vector]`. Next, press the **Shift** key, and select `train [vector]`. The Signal Browser automatically updates to display all three signals, as shown in the following figure.

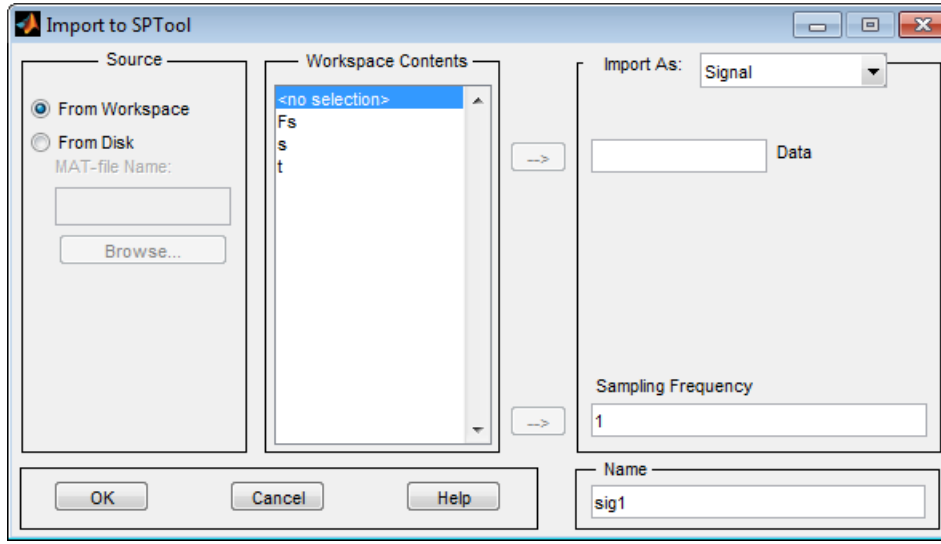


Importing New Signals

Using SPTool, you can import signals from variables in the MATLAB workspace. First, create a 3-second signal, sampled at 10 kHz, that is the sum of two sine waves. At the MATLAB command line, enter the following commands:

```
Fs = 1e4;
t = 0:1/Fs:3;
s = sum(sin(2*pi*[350;440]*t));
```

To import the signal from these variables, in the SPTool menu, select **File** > **Import**. Alternatively, you can press the **Ctrl+I** keyboard shortcut. The Import to SPTool dialog box opens.



From the **Workspace Contents** list, select `s`. Click the right arrow ( ) button to the left of the **Data** box. Next, from the **Workspace Contents** list, select `Fs`. Click the right arrow ( ) button to the left of the **Sampling Frequency** box. You can assign the signal a name in the **Name** box, but since you will rename the signal later in this example, leave it as `sig1` for now.

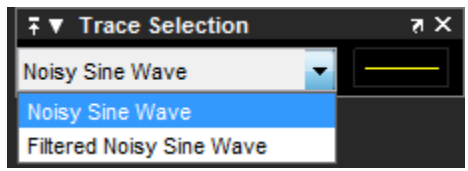
Click **OK**. The SPTool **Signals** list now contains a signal named `sig1 [vector]`.

Selecting a Signal and Playing Audio

In the SPTool **Signals** list, select `sig1 [vector]`, and click the **View** button. The Signal Browser reappears in front. To play audio for the signal, click the Play selected signal

( ) button. Signal Browser sends the audio signal to the speaker. For more information, see `sound` in the MATLAB documentation.

If you have multiple signals selected in SPTool, you can use the **Trace Selection** panel to choose which signal to make active. In the SPTool **Signals** list, press the **Ctrl** key, and select `mtlb [vector]`. The Signal Browser now displays two signals. In the Signal Browser menu, select **Tools > Measurements > Trace Selection**. The **Trace Selection** panel appears as shown in the following figure.



In the **Trace Selection** panel drop-down list, select `sig1`. Then, click the Play selected signal ( ) button. Signal Browser sends the audio signal to the speaker.

Note To hear audio when you click the Play selected signal ( ) button, your computer sound card must be able to support the sample rate of the signal. In this example, the sample rate of the signal, `sig1 [vector]`, is 10 kHz. If your sound card supports this or a greater sample rate, such as 44.1 kHz, then you can hear the audio on your speaker. For more information, see `sound` in the MATLAB documentation.

Change Signal Names from the Legend

When multiple signals are displayed, Signal Browser shows a legend by default. To turn off the legend, click the Show all legends ( ) button. Click the button again to turn the legend back on. You can modify the names of the signal directly in the legend. To do so, when the legend is visible, click and drag it to any location on the display.

You can change the name of any signal directly within the legend. In the legend, double-click the signal name `sig1`. A cursor appears, indicating that you can now change this

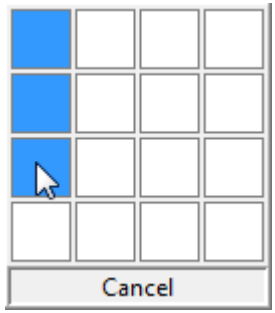
name. Highlight the text, and type `Dial Tone`. The legend now shows `Dial Tone` as the name of that signal. In the SPTool **Signals** list, the last item is now also named `Dial Tone`.

Configure the Signal Browser Properties

First, configure the appearance of the Signal Browser window. In the SPTool **Signals** list, first select `mtlb [vector]`. Next, press the **Shift** key, and select `train [vector]`. The Signal Browser automatically updates to display all three signals.

Multiple Displays

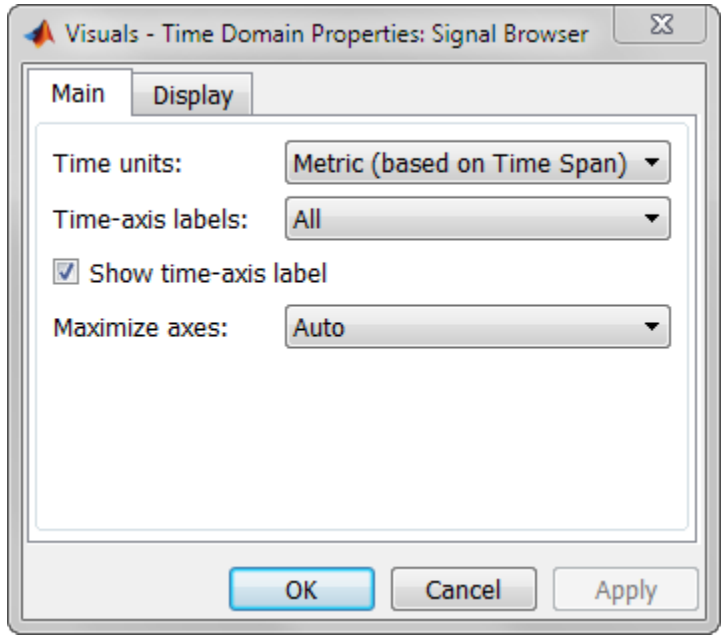
You can display multiple signals on different displays in the Signal Browser window. In the SPTool **Signals** list, first ensure all three signals are selected. In the Signal Browser toolbar, click the Layout ( ) button. Select row 3, column 1, as shown in the following figure.



After you make this selection, the Signal Browser is separated into three displays.

Configure Appearance

In the Signal Browser menu, select **View** > **Properties**. The Visuals:Time Domain Options dialog box opens, as shown in the following figure.



In the Visuals:Time Domain Options dialog box, click the **Main** tab. Choose the appropriate parameter settings for the **Main** tab, as shown in the following table.

Parameter	Setting
Time units	`Metric (based on Time Span)`
Show time-axis labels	`Bottom Displays Only`
Maximize axes	`On`

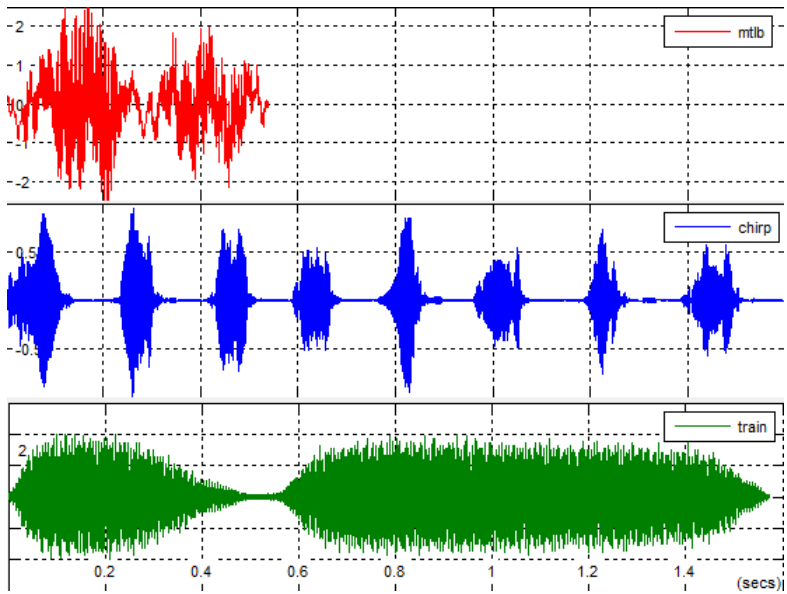
When you change the **Maximize axes** parameter to `On`, the axes are expanded to fill the entire display. To conserve space, titles and axis labels are not shown in each display. Click **Apply**.

Set Display Properties

In the Visuals:Time Domain Options dialog box, click the **Display** tab. You can change the value of the **Select display** parameter to make different settings for each display. Set the parameters to the values shown in the following table.

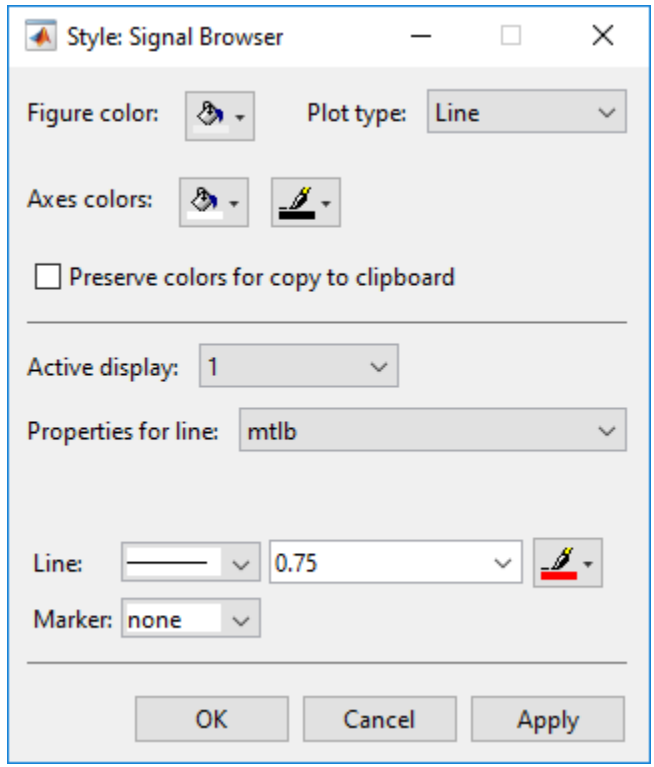
Parameter	Display 1 Setting	Display 2 Setting	Display 3 Setting
Select display	1	2	3
Title	`mtlb`	`chirp`	`train`
Show legend	Selected	Selected	Selected
Show grid	Selected	Selected	Selected
Plot signal(s) as magnitude and phase	Cleared	Cleared	Cleared
Minimum Y-limit	`-2.5`	`-1`	`-1.5`
Maximum Y-limit	`2.5`	`1`	`1.5`
Y-axis label	`Amplitude`	`Amplitude`	`Amplitude`

Click **OK** to save your changes and close the Visuals:Time Domain Options dialog box. The Signal Browser appears as shown in the following figure.



Modify the Signal Browser Display

Use the Style dialog box to modify the appearance of the axes and the lines for each of the selected signals in SPTool. In the Signal Browser menu, select **View** > **Style**. The Style dialog box opens, as shown in the following figure.



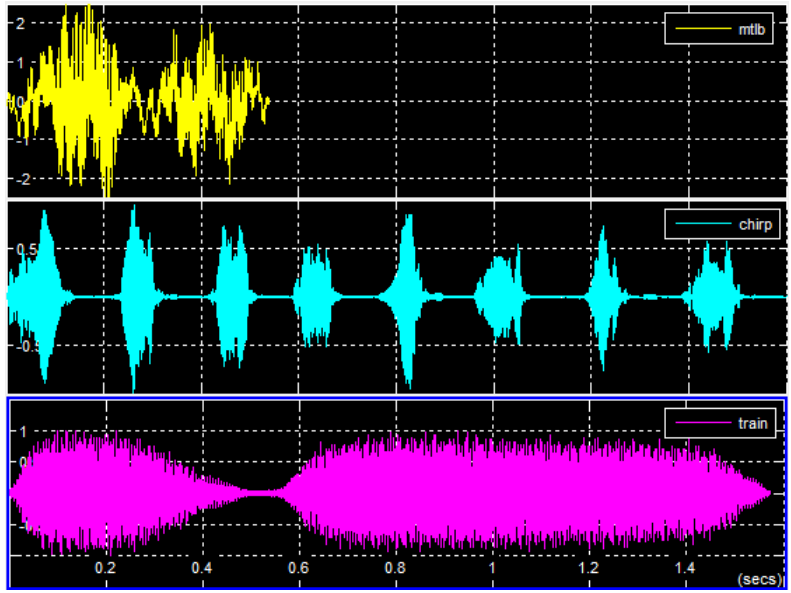
Modify Axes Colors and Line Properties

You can change the value of the **Select display** parameter to make different settings for each display. Set the parameters to the values shown in the following table.

Parameter	Display 1 Setting	Display 2 Setting	Display 3 Setting
Select display	1	2	3

Parameter	Display 1 Setting	Display 2 Setting	Display 3 Setting
Axes background color	Black	Black	Black
Ticks, labels, and grid colors	White	White	White
Line color	Yellow	Cyan	Magenta

These settings enable the Signal Browser to display line colors in the same manner as the Simulink Scope block. Click **OK** to save your changes and close the Style dialog box. The Signal Browser now appears as shown in the following figure.



Show and Hide Toolbar

To hide the toolbar, from the Signal Browser menu, select **View > Toolbar**. Doing so removes the toolbar from the Signal Browser window and also removes the check mark beside the **Toolbar** option in the **View** menu. You can choose to show the toolbar again at any time by selecting **View > Toolbar**.

Inspect Your Data (Scaling the Axes and Zooming)

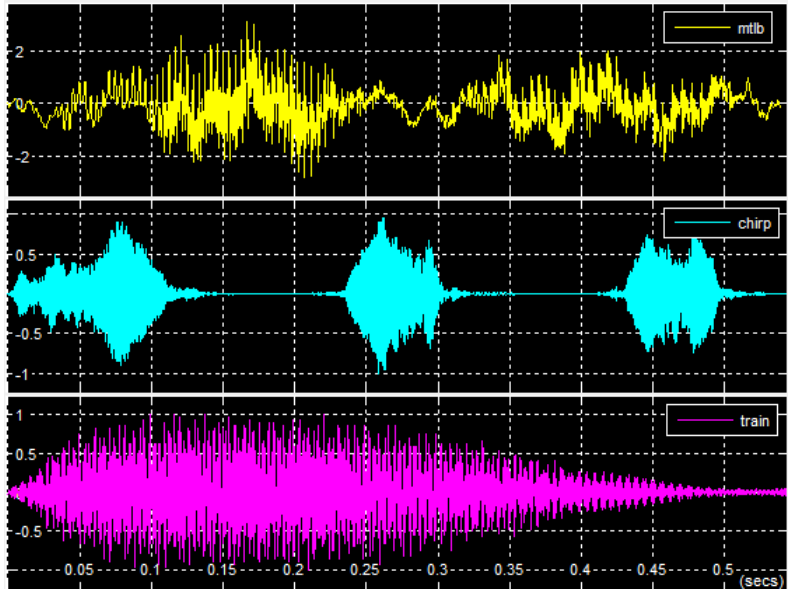
So far, you have manually set the *y*-axis limits. Use one of the following options to let Signal Browser scale the axes:

- From the Signal Browser menu, select **Tools > Scale Axes Limits**.
- From the Signal Browser toolbar, click the Scale Axes Limits ( ) button.
- With the Signal Browser as your active window, press **Ctrl + A**.

Use the Zoom Tools

The zoom tools allow you to zoom in simultaneously in the directions of both the *x*- and *y*-axes , or in either direction individually. For example, to zoom in on the signal between 0 and 0.5 seconds, you can use the **Zoom X** option.

- To activate the **Zoom X** tool, select **Tools > Zoom X**, or press the corresponding toolbar button ( ). The Signal Browser indicates that the **Zoom X** tool is active by indenting the toolbar button and placing a check mark next to the **Tools > Zoom X** menu option.
- Next, zoom in on the region between 0 and 0.5 seconds. In the Signal Browser window, click on the 0-second mark and drag to the 0.5-second mark. All three displays reflect this new *x*-axis setting, as shown in the following figure.



- To zoom out of the Signal Browser window, right-click inside the window, and select **Zoom Out**. Alternatively, you can return to the original view of your signal by right-clicking inside the Signal Browser window and selecting **Reset to Original View**.

Signal Measurement

- “RMS Value of Periodic Waveforms” on page 16-2
- “Slew Rate of Triangular Waveform” on page 16-6
- “Duty Cycle of Rectangular Pulse Waveform” on page 16-10
- “Estimate State for Digital Clock” on page 16-14
- “Calculate Settling Time with Signal Browser” on page 16-17
- “Find Peak Amplitudes in Signal Browser” on page 16-22
- “Distortion Measurements” on page 16-26
- “Prominence” on page 16-31
- “Determine Peak Widths” on page 16-34
- “Human Activity Recognition Simulink Model for Smartphone Deployment” on page 16-38

RMS Value of Periodic Waveforms

This example shows how to find the root mean square (RMS) value of a sine wave, a square wave, and a rectangular pulse train using `rms`. The waveforms in this example are discrete-time versions of their continuous-time counterparts.

Create a sine wave with a frequency of $\pi/4$ rad/sample. The length of the signal is 16 samples, which equals two periods of the sine wave.

```
n = 0:15;
x = cos(pi/4*n);
```

Compute the RMS value of the sine wave.

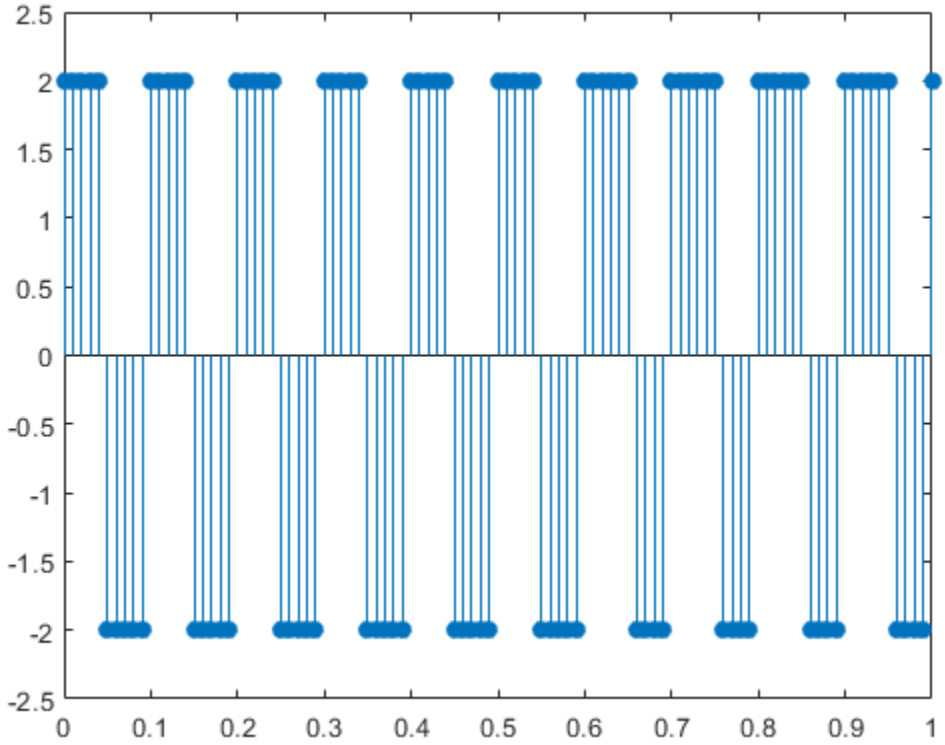
```
rmsval = rms(x)
```

```
rmsval = 0.7071
```

The RMS value is equal to $1/\sqrt{2}$, as expected.

Create a periodic square wave with a period of 0.1 seconds. The square wave values oscillate between -2 and 2.

```
t = 0:0.01:1;
x = 2*square(2*pi*10*t);

stem(t,x,'filled')
axis([0 1 -2.5 2.5])
```



Find the RMS value.

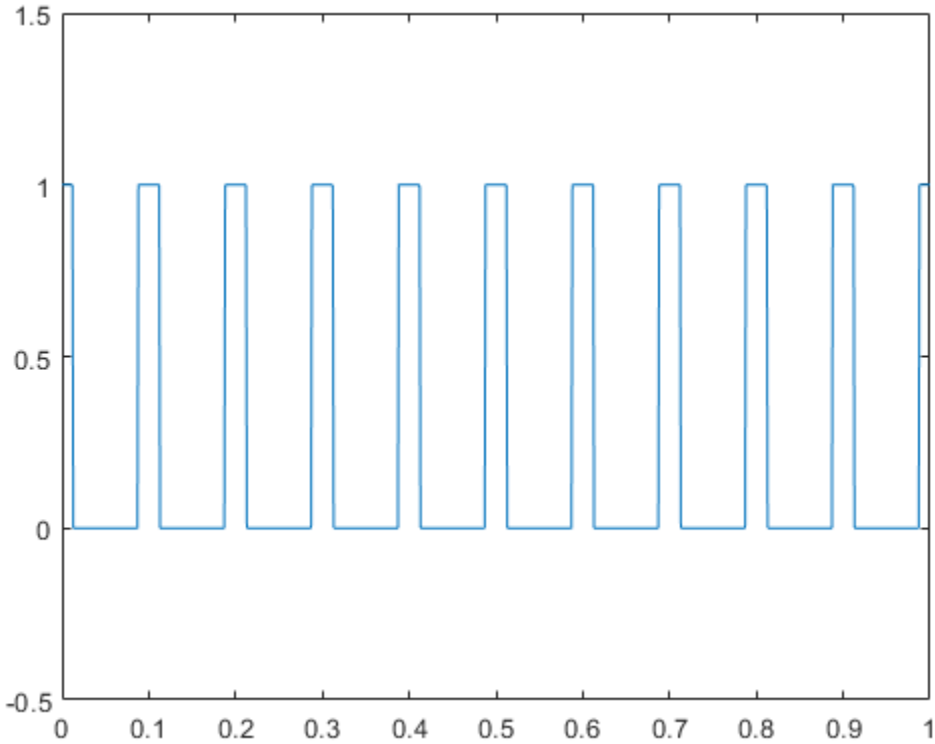
```
rmsval = rms(x)
```

```
rmsval = 2
```

The RMS value agrees with the theoretical value of 2.

Create a rectangular pulse train sampled at 1 kHz with the following parameters: the pulse is on, or equal to 1, for 0.025 seconds, and off, or equal to 0, for 0.075 seconds in each 0.1 second interval. This means the pulse period is 0.1 seconds and the pulse is on for 1/4 of that interval. This is referred to as the *duty cycle*. Use `pulstran` to create the rectangular pulse train.

```
t = 0:0.001:(10*0.1);
pulsewidth = 0.025;
pulseperiods = [0:10]*0.1;
x = pulstran(t,pulseperiods,@rectpuls,pulsewidth);

plot(t,x)
axis([0 1 -0.5 1.5])
```



Find the RMS value and compare it to the RMS of a continuous-time rectangular pulse waveform with duty cycle 1/4 and peak amplitude 1.

```
rmsval = rms(x)
```

```
rmsval = 0.5007
```

```
thrms = sqrt(1/4)
```

```
thrms = 0.5000
```

The observed RMS value and the RMS value for a continuous-time rectangular pulse waveform are in good agreement.

Slew Rate of Triangular Waveform

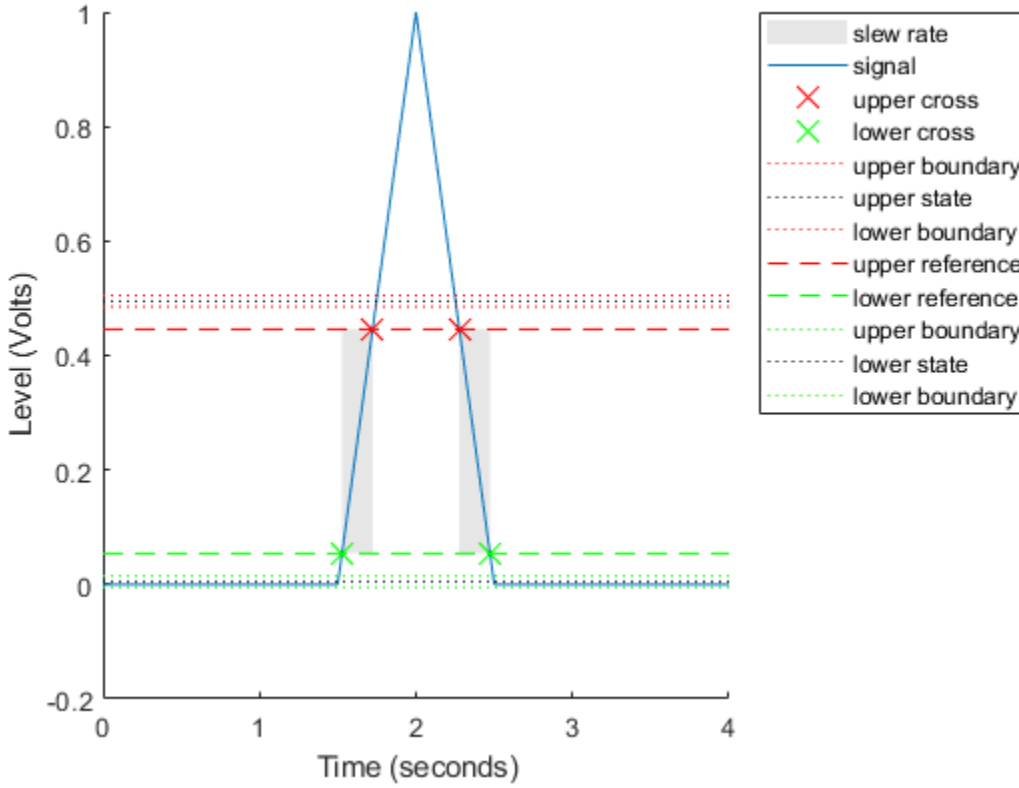
This example shows how to use the slew rate as an estimate of the rising and falling slopes of a triangular waveform. Create three triangular waveforms. One waveform has rising-falling slopes of ± 2, one waveform has rising-falling slopes of $\pm \frac{1}{2}$, and one waveform has a rising slope of $+2$ and a falling slope of $-\frac{1}{2}$. Use `slewrate` to find the slopes of the waveforms.

Use `tripuls` to create a triangular waveform with rising-falling slopes of ± 2. Set the sampling interval to 0.01 seconds, which corresponds to a sample rate of 100 hertz.

```
dt = 0.01;
t = -2:dt:2;

x = tripuls(t);
```

Compute and plot the slew rate for the triangular waveform. Input the sample rate (100 Hz) to obtain the correct positive and negative slope values.

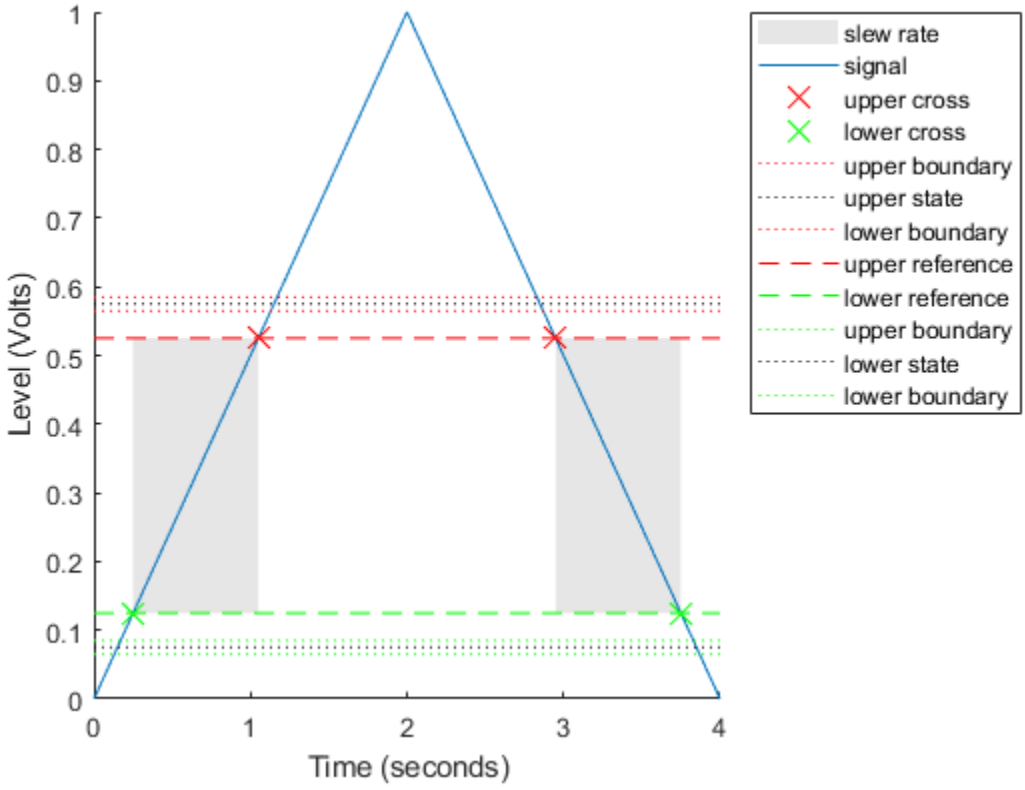
```
slewrate(x,1/dt)
```




```
ans = 1×2

    2.0000   -2.0000
```

Change the width of the triangular waveform so it has slopes of $\pm\frac{1}{2}$. Compute and plot the slew rate.

```
x = tripuls(t,4);
slewrate(x,1/dt)
```




```
ans = 1×2

    0.5000   -0.5000
```

Create a triangular waveform with a rising slope of $+2$ and a falling slope of $-\frac{1}{2}$. Compute the slew rate.

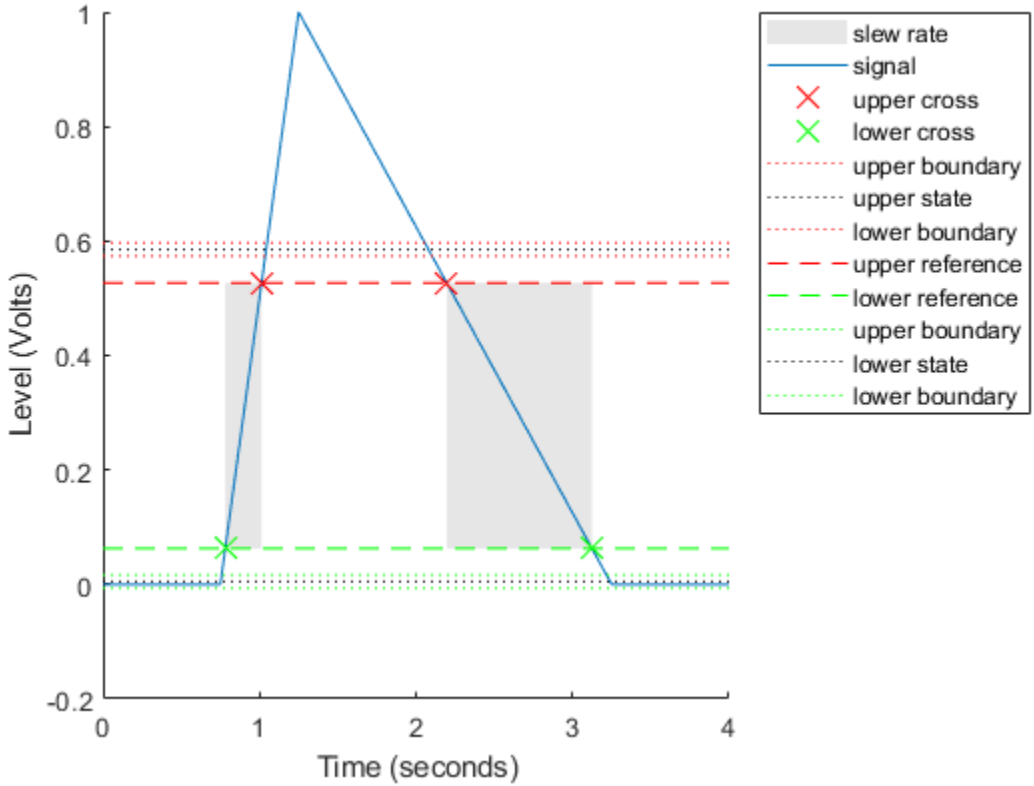
```
x = tripuls(t,5/2,-3/5);
s = slewrate(x,1/dt)
```

```
s = 1×2
```

```
    2.0000   -0.5000
```

The first element of `s` is the rising slope and the second element is the falling slope. Plot the result.

```
slewrate(x,1/dt);
```

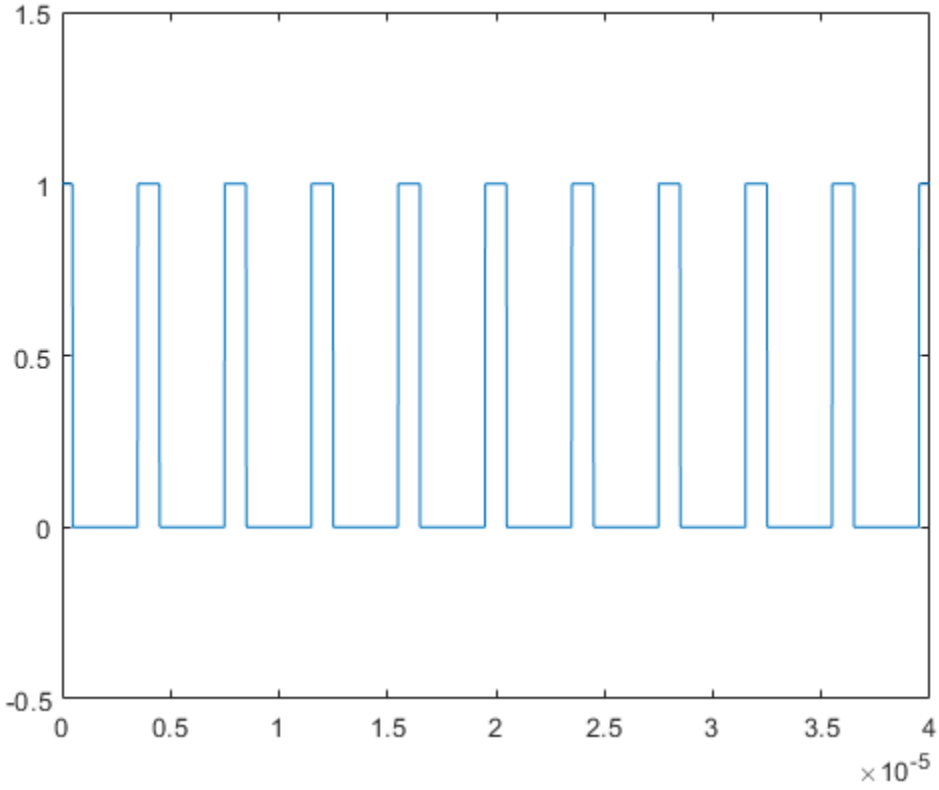


Duty Cycle of Rectangular Pulse Waveform

This example shows how to create a rectangular pulse waveform and measure its duty cycle. You can think of a rectangular pulse waveform as a sequence of *on* and *off* states. One pulse period is the total duration of an *on* and *off* state. The pulse width is the duration of the *on* state. The duty cycle is the ratio of the pulse width to the pulse period. The duty cycle for a rectangular pulse describes the fraction of time that the pulse is *on* in one pulse period.

Create a rectangular pulse sampled at 1 gigahertz. The pulse is *on*, or equal to 1, for a duration of 1 microsecond. The pulse if *off*, or equal to 0, for a duration of 3 microseconds. The pulse period is 4 microseconds. Plot the waveform.

```
Fs = 1e9;
t = 0:1/Fs:(10*4e-6);

pulsewidth = 1e-6;
pulseperiods = [0:10]*4e-6;

x = pulstran(t,pulseperiods,@rectpuls,pulsewidth);

plot(t,x)
axis([0 4e-5 -0.5 1.5])
```



Determine the duty cycle of the waveform using `dutycycle`. Input both the pulse waveform and the sample rate to output the duty cycle. `dutycycle` outputs a duty cycle value for each detected pulse.

```
D = dutycycle(x,Fs)
```

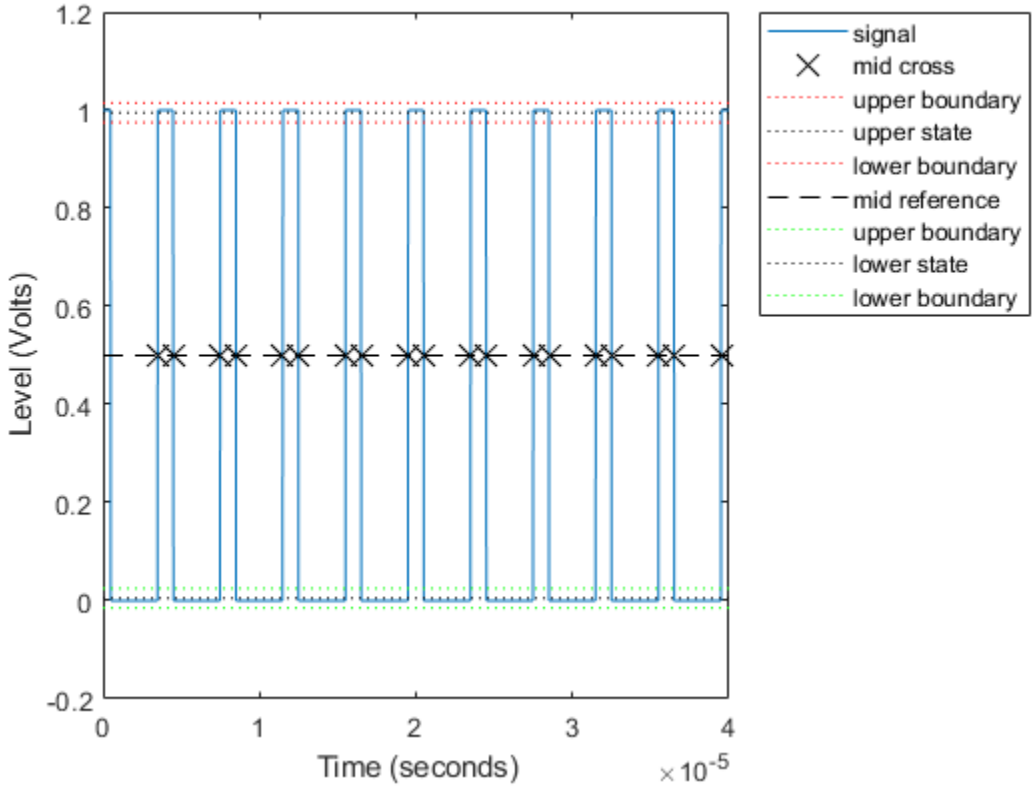
```
D = 1×9

    0.2500    0.2500    0.2500    0.2500    0.2500    0.2500    0.2500    0.2500    0.2
```

In this example, the duty cycle for each of the detected pulses is identical and equal to 0.25. This is the expected duty cycle because the pulse is on for 1 microsecond and off for

3 microseconds in each 4 microsecond period. Therefore, the pulse is on for 1/4 of each period. Expressed as a percentage, this is equal to a duty cycle of 25%.

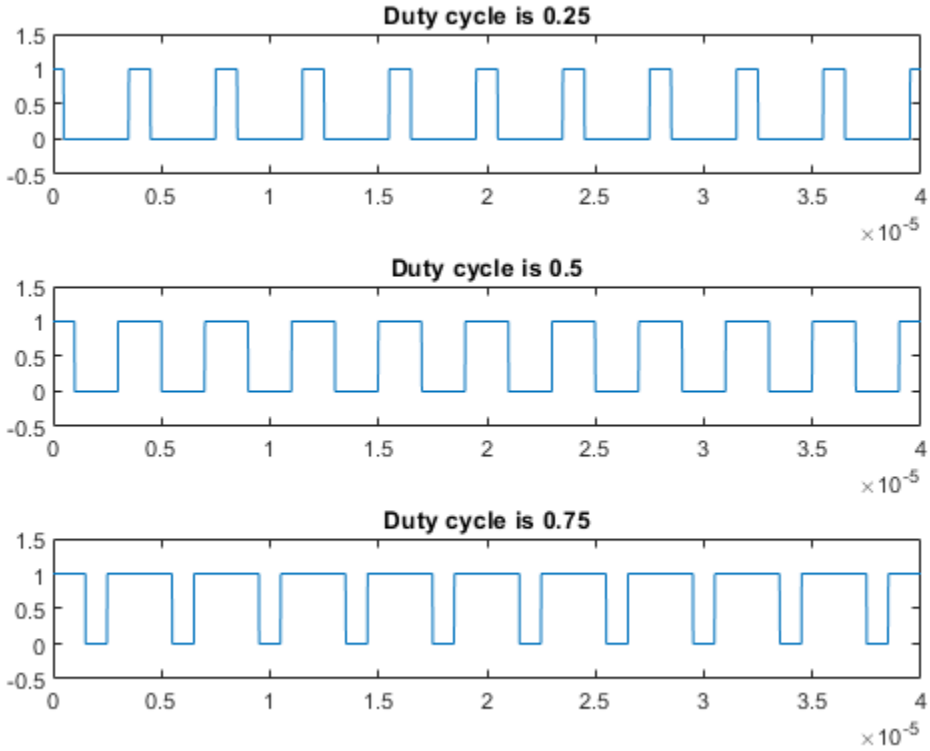
Calling `dutycycle` with no output arguments produces a plot with all the detected pulse widths marked.

```
dutycycle(x,Fs);
```



Using the same sample rate and pulse period, vary the pulse on time (pulse width) from 1 to 3 microseconds in a loop and calculate the duty cycle. Plot the pulse waveforms and display the duty cycle value in the plot title for each step through the loop. The duty cycle increases from 0.25 (1/4) to 0.75 (3/4) as the pulse width increases.

```
nwid = 3;

for nn = 1:nwid
    x = pulstran(t,pulseperiods,@rectpuls,nn*pulsewidth);

    subplot(nwid,1,nn)
    plot(t,x)
    axis([0 4e-5 -0.5 1.5])

    D = dutycycle(x,Fs);
    title(['Duty cycle is ' num2str(mean(D))])
end
```

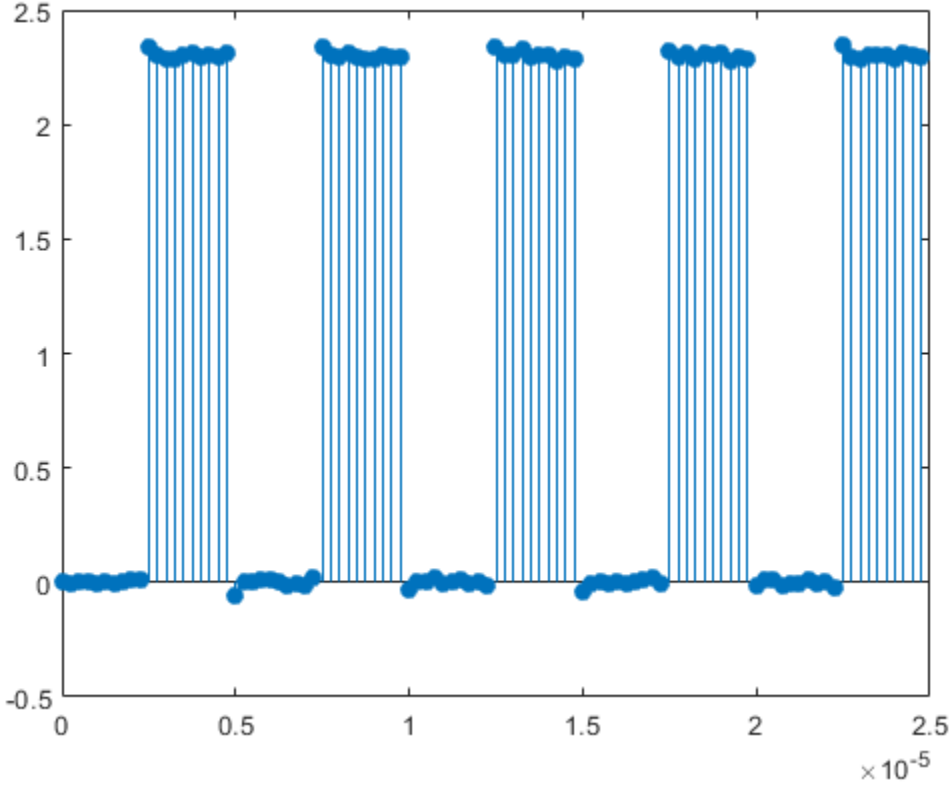


Estimate State for Digital Clock

This example shows how to estimate the high and low state levels for digital clock data. In contrast to analog voltage signals, signals in digital circuits have only two states: HIGH and LOW. Information is conveyed by the pattern of high and low state levels.

Load `clockex.mat` into the MATLAB® workspace. `clockex.mat` contains a 2.3 volt digital clock waveform sampled at 4 megahertz. Load the clock data into the variable `x` and the vector of sampling times in the variable `t`. Plot the data.

```
load('clockex.mat','x','t')
stem(t,x,'filled')
```



Determine the high and low state levels for the clock data using `statelevels`.

```
levels = statelevels(x)
```

```
levels = 1×2

    0.0027    2.3068
```

This is the expected result for the 2.3 volt clock data, where the noise-free low-state level is 0 volts and the noise-free high-state level is 2.3 volts.

Use the estimated state levels to convert the voltages into a sequence of zeros and ones. The sequence of zeros and ones is a binary waveform representation of the two states. To make the assignment, use the following decision rule:

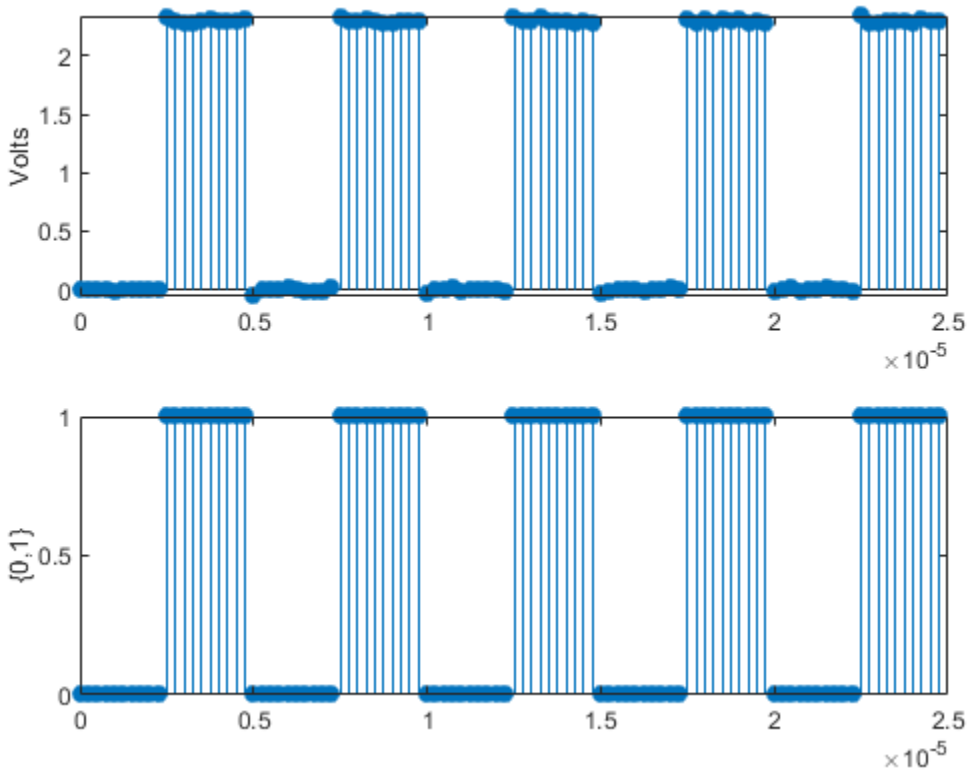
- Assign any voltage within a 3%-tolerance region of the low-state level the value 0.
- Assign any voltage within a 3%-tolerance region of the high-state level the value 1.

Determine the widths of the 3%-tolerance regions around the low- and high-state levels.

```
tolwd = 3/100*diff(levels);
```

Use logical indexing to determine the voltages within a 3%-tolerance region of the low-state level and the voltages within a 3%-tolerance region of the high-state level. Assign the value 0 to the voltages within the tolerance region of the low-state level and 1 to the voltages within the tolerance region of the high-state level. Plot the result.

```
y = zeros(size(x));
y(abs(x-min(levels))<=tolwd) = 0;
y(abs(x-max(levels))<=tolwd) = 1;

subplot(2,1,1)
stem(t,x,'filled')
ylabel('Volts')
subplot(2,1,2)
stem(t,y,'filled')
ylabel('\{0,1\}')
```



The decision rule has assigned all the voltages to the correct state.

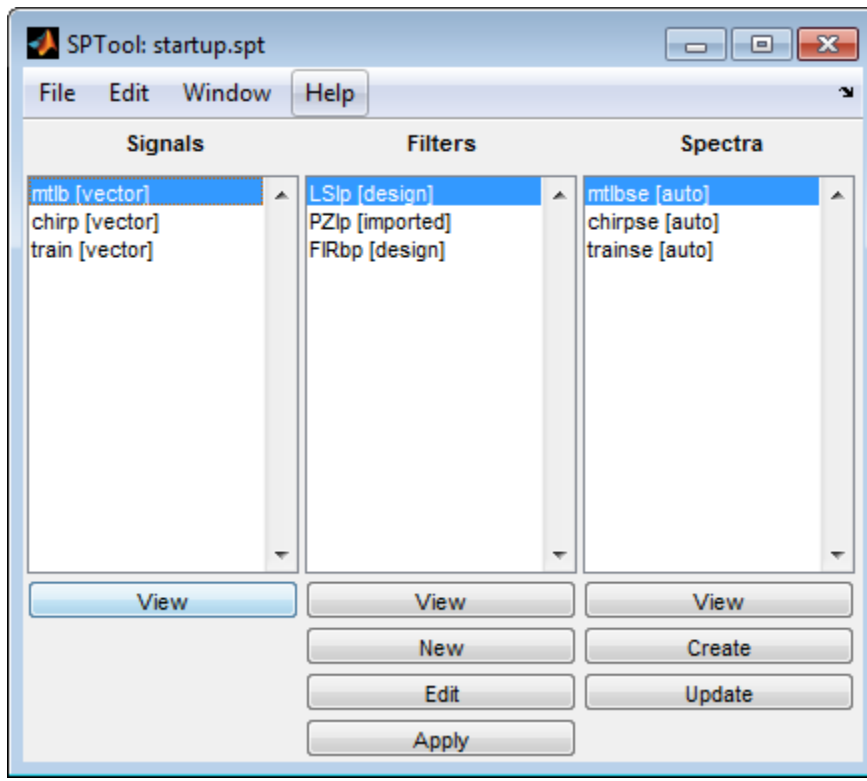
Calculate Settling Time with Signal Browser

This example shows how to use the Bilevel Measurements panel in the SPTool Signal Browser to find the settling time of a clock signal.

First, open SPTool by typing the following at the MATLAB command line.

```
sptool
```

SPTool opens.



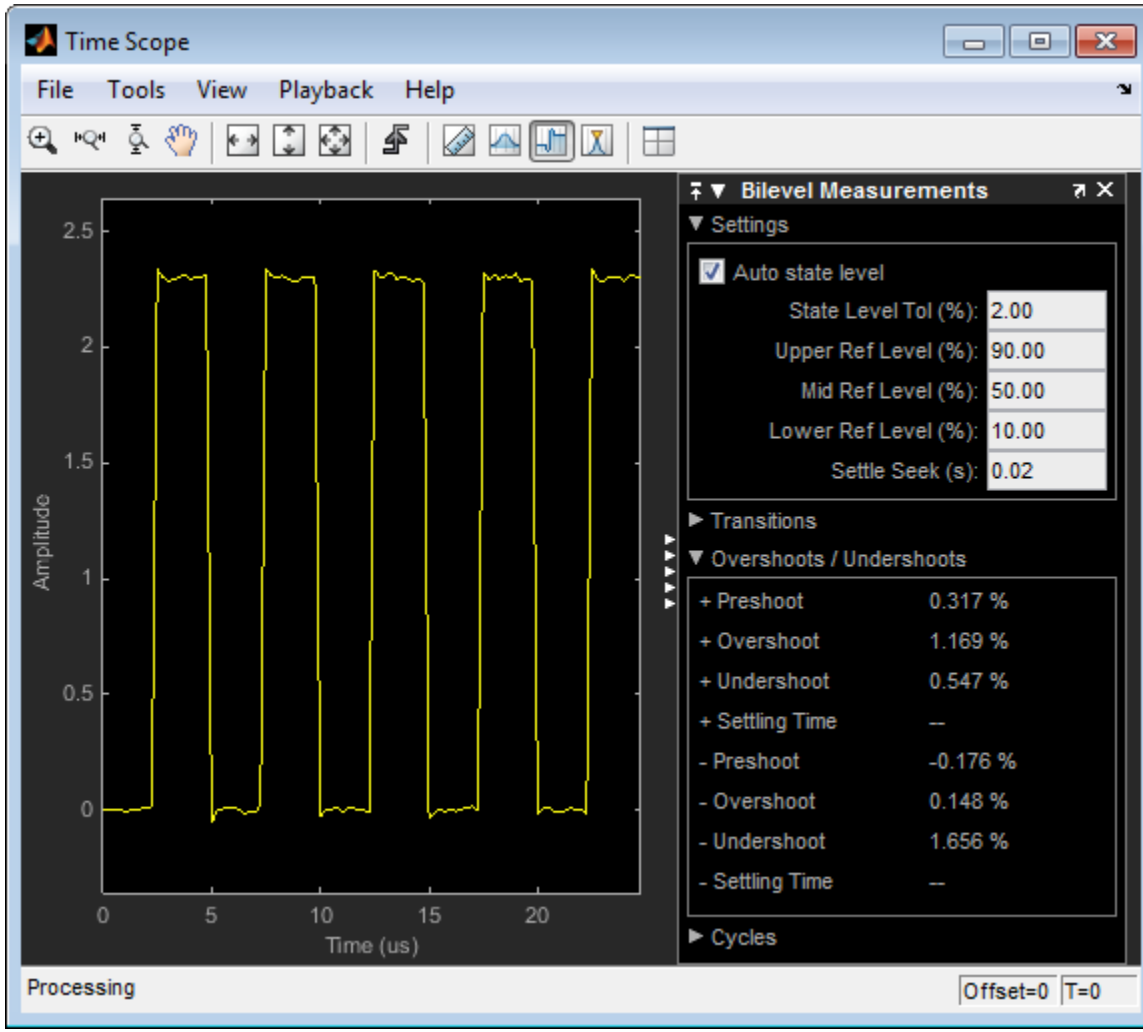
In this example, you import a clock signal from the MAT-file named `clockex.mat`. In the SPTool menu, select **File > Import**. Alternatively, you can press the **Ctrl+I** keyboard shortcut. The Import to SPTool dialog box opens.

1. Under **Source**, click **From Disk**.
2. In the **MAT-file Name** box, type `clockex`, and press **Enter**. The variables `x` and `t` appear under **File Contents**.
3. Under **File Contents**, click `x`. Click the right arrow (-->) button to the left of the **Data** box.
4. In the **Sampling Frequency** box, type `4000000`.
5. In the **Name** box, type `Clock`.
6. Click **OK**. The SPTool **Signals** list now contains a signal named `Clock [vector]`.
7. In SPTool, in the **Signals** list, select `Clock [vector]`, and click the **View** button. The Signal Browser appears and displays the clock signal.

Because you selected only one signal, the legend is not needed. Turn off the legend by clicking the Show All Legends ( ) button. Then, use the Style dialog box to modify the appearance of the axes and the lines for the signal. In the Signal Browser menu, select **View > Style**.

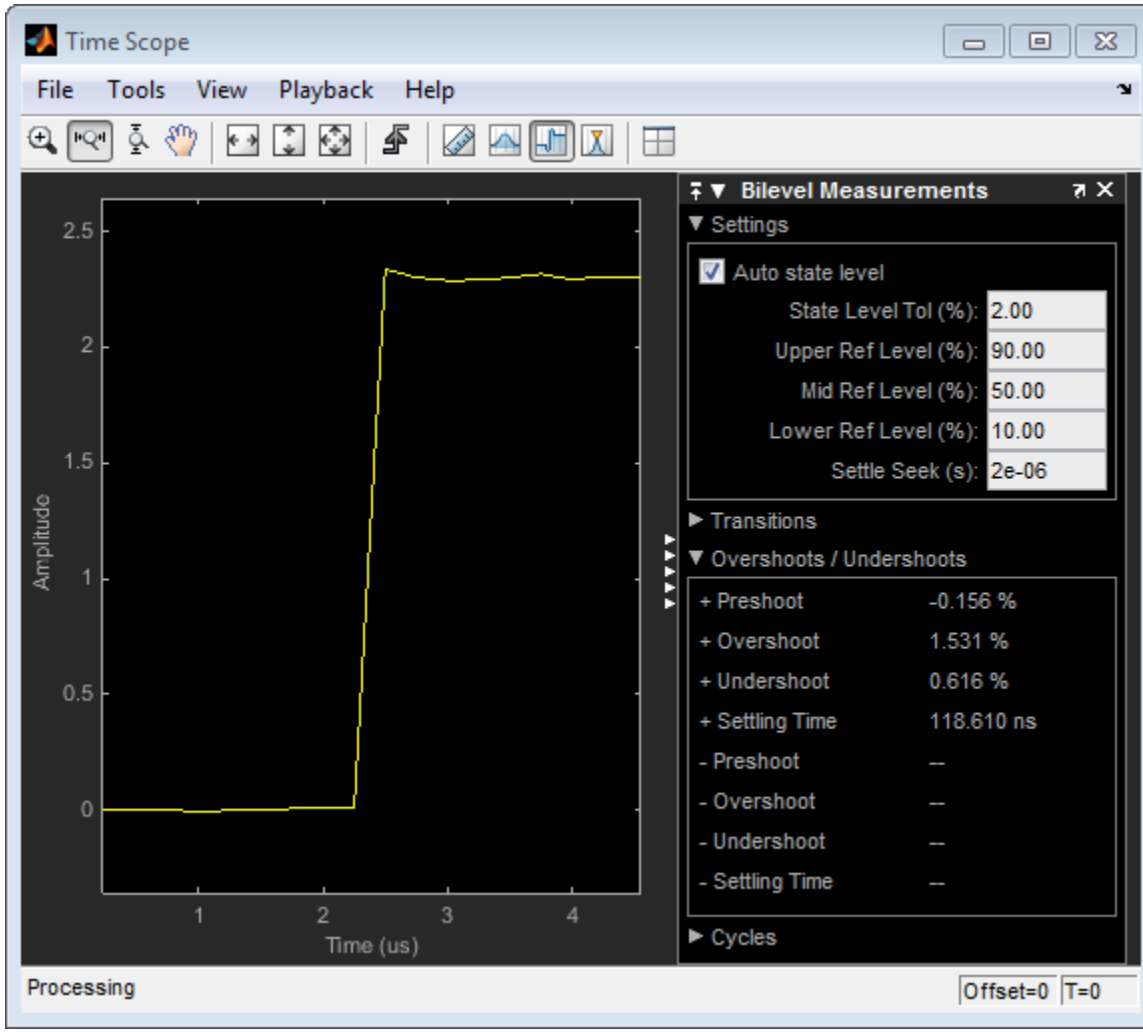
Parameter	Display 1 Setting
Axes background color	Black
Ticks, labels, and grid colors	Dark Gray
Line color	Yellow

To show the **Bilevel Measurements** panel, in the Signal Browser menu, select **Tools > Measurements > Bilevel Measurements**. To collapse the **Transitions** pane, click the pane collapse button (▼) next to that label. To expand the **Settings** pane and the **Overshoots / Undershoots** pane, click the pane expand button (▶) next to each label. The Signal Browser appears as shown in the following figure.



The value for the rising edge **Settling Time** parameter does not appear in the **Overshoots / Undershoots** pane because the **Settle Seek** parameter is too large. The **Settle Seek** value is longer than the entire simulation duration. Enter a value for settle seek of `2e-6`, and press **Enter**. Signal Browser now displays a rising edge settling time value of `118.392 ns`.

This settling time value displayed is actually the statistical average of the settling times for all five rising edges. To display the settling time for only one rising edge, you can zoom in on that transition. In the Signal Browser toolbar, click the Zoom X button ( ). Click the display near a value of 2 microseconds on the *time*-axis. Drag to the right, and release near a value of 4 microseconds on the *time*-axis. Signal Browser updates the rising edge **Settling Time** value to reflect the new time window, as shown in the following figure.



See Also

`falltime` | `overshoot` | `risetime` | `settlingtime` | `slewrate` | `undershoot`

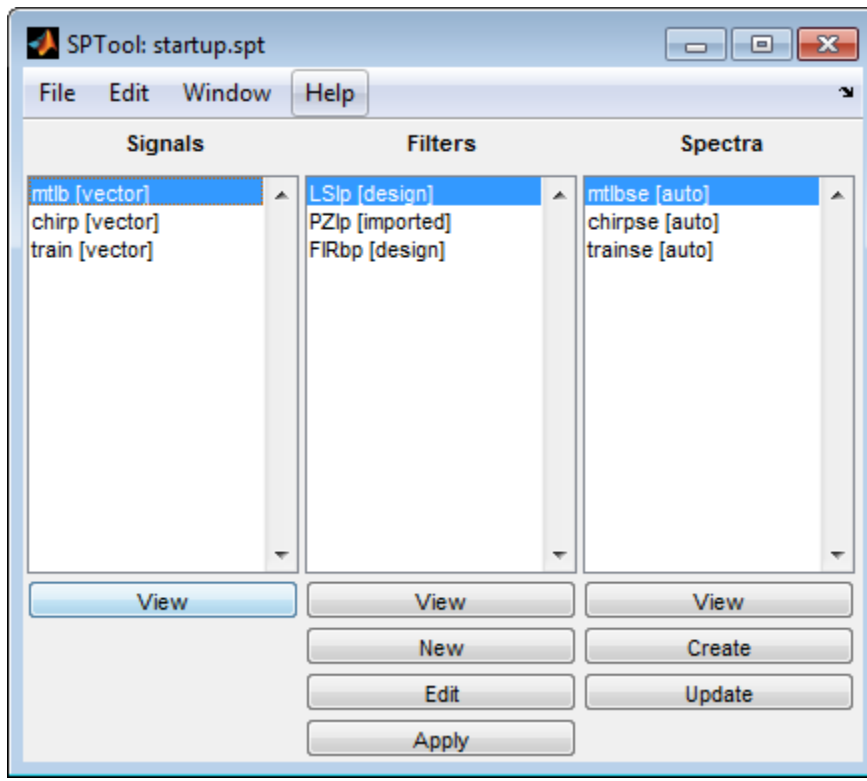
Find Peak Amplitudes in Signal Browser

This example shows how to use the Peak Finder panel in the SPTool Signal Browser to find heart rate, given an electrocardiogram (ECG) signal.

First, open SPTool by typing the following at the MATLAB command line.

```
sptool
```

SPTool opens.



Using SPTool, you can import signals from variables in the MATLAB workspace. First, create an electrocardiogram (ECG) signal, sampled at 4 kHz. To create the signal, save this function definition in a file called `ecg.m`:

```
function x = ecg(L)
a0 = [0,1,40,1,0,-34,118,-99,0,2,21,2,0,0,0];
d0 = [0,27,59,91,131,141,163,185,195,275,307,339,357,390,440];
a = a0 / max(a0);
d = round(d0 * L / d0(15));
d(15) = L;
for i = 1:14,
    m = d(i) : d(i+1) - 1
    slope = (a(i+1) - a(i)) / (d(i+1) - d(i))
    x(m+1) = a(i) + slope * (m - d(i))
end
```

Now apply the Savitzky-Golay filter to the ECG signal. At the MATLAB command line, enter the following commands:

```
x1 = 3.5*ecg(2700).';
y1 = sgolayfilt(kron(ones(1,13),x1),0,21);
n = (1:30000)';
del = round(2700*rand(1));
mhb = y1(n + del);
ts = 0.00025;
Fs = 1/ts;
```

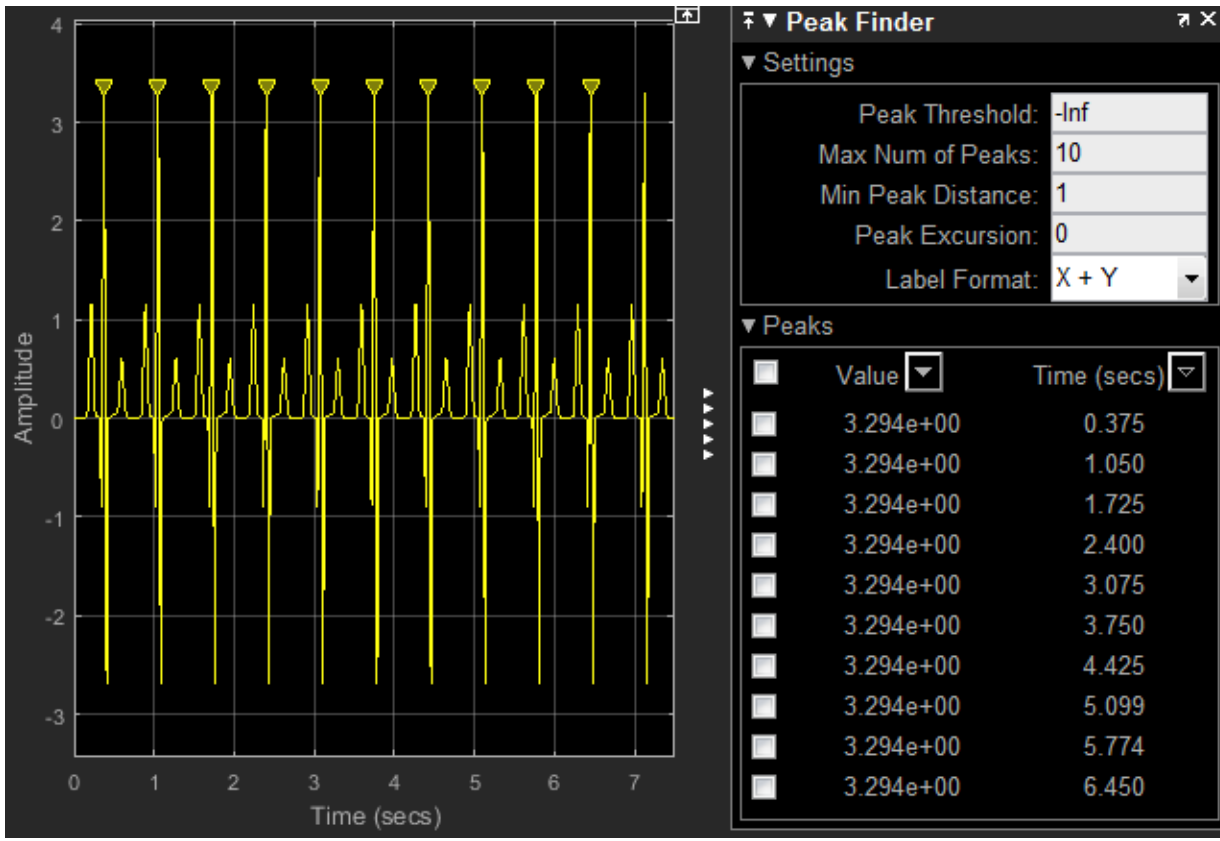
To import the signal from these variables, in the SPTool menu, select **File** > **Import**. Alternatively, you can press the **Ctrl+I** keyboard shortcut. The Import to SPTool dialog box appears.

1. Under **Workspace Contents**, click `mhb`. Click the right arrow ( ) button to the left of the **Data** box.
2. Under **Workspace Contents**, click `Fs`. Click the right arrow ( ) button to the left of the **Sampling Frequency** box.
3. In the **Name** box, type `ECG`.
4. Click **OK**. The SPTool **Signals** list now contains a signal named `ECG [vector]`.
5. In SPTool, in the **Signals** list, select `ECG [vector]`, and click the **View** button. The Signal Browser opens and displays the ECG signal.

Because you only selected one signal, the legend is not needed. Turn off the legend by clicking the Show All Legends ( ) button. Then, use the Style dialog box to modify the appearance of the axes and the lines for the signal. In the Signal Browser menu, select **View** > **Style**.

Parameter	Display 1 Setting
Axes background color	Black
Ticks, labels, and grid colors	Dark Gray
Line color	Yellow

To show the **Peak Finder** panel, in the Signal Browser menu, select **Tools** > **Measurements** > **Peak Finder**. To expand the **Settings** pane, click the pane expand button (▶) next to that label. In the **Max Num of Peaks** box, type `10` and press the **Enter** key. Signal Browser now displays in the **Peaks** pane a list of 10 peak amplitude values, and the times at which they occur, as shown in the following figure.



As you can see from the list of peak values, there is a constant time difference of 0.675 seconds between each heartbeat. Therefore, the heart rate detected by the ECG signal is given by the following equation.

$$\frac{60\frac{\text{sec}}{\text{min}}}{0.675\frac{\text{sec}}{\text{beat}}} = 88.89\frac{\text{beats}}{\text{min}}(\text{bpm})$$

See Also

`findpeaks` | `sgolay` | `sgolayfilt`

Distortion Measurements

Generate 2048 samples of a sinusoid of frequency 2.5 kHz sampled at 50 kHz. Add white Gaussian noise such that the signal-to-noise ratio (SNR) is 80 dB.

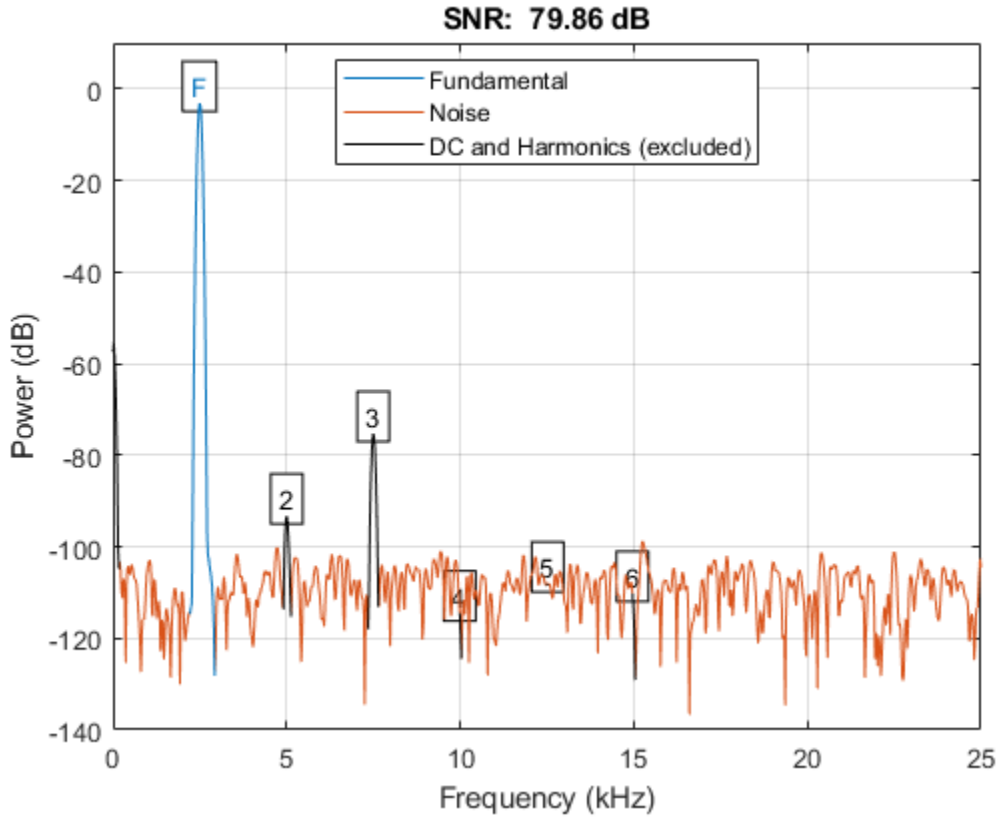
```
Fs = 5e4;
f0 = 2.5e3;
N = 2048;
t = (0:N-1)/Fs;
SNR = 80;

x = cos(2*pi*f0*t);
x = x+randn(size(x))*std(x)/db2mag(SNR);
```

Pass the result through a weakly nonlinear amplifier represented by a polynomial. The amplifier introduces spurious tones at the frequencies of the harmonics.

```
amp = [1e-5 5e-6 -1e-3 6e-5 1 25e-3];
x = polyval(amp,x);
```

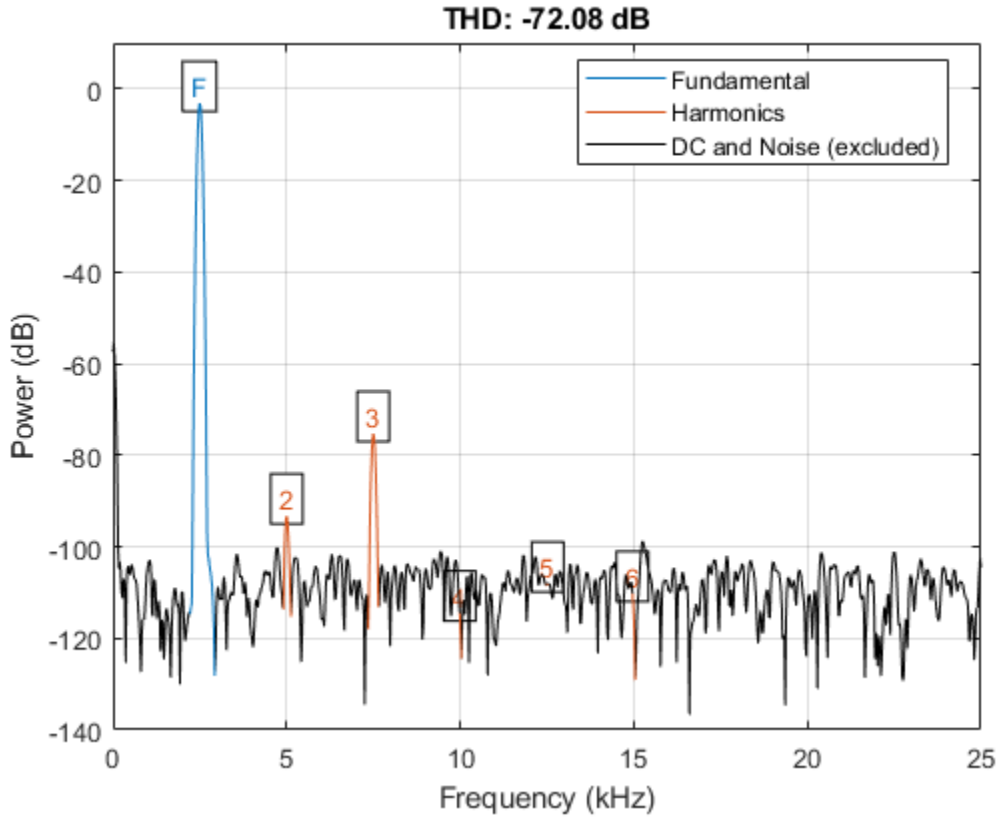
Plot the signal spectrum and annotate the SNR, verifying that it has the expected value. The `snr` function computes the power ratio of the fundamental to the noise floor and ignores the DC component and the harmonics.

```
snr(x,Fs);
```



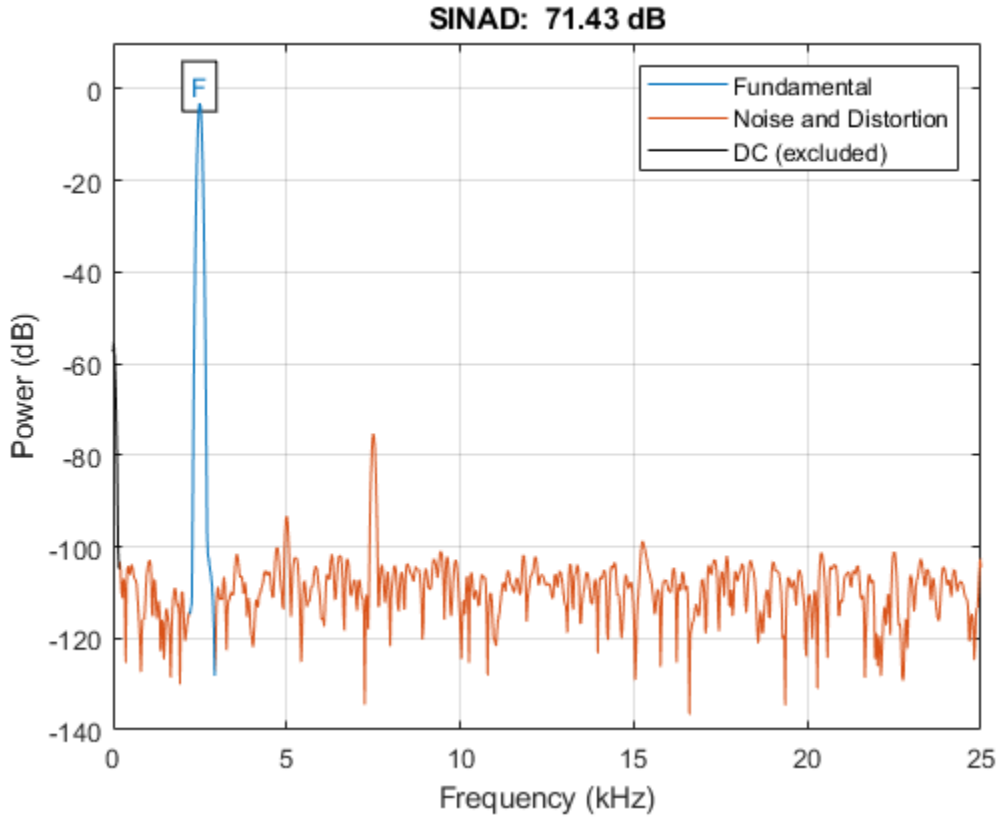
Plot the signal spectrum and annotate the total harmonic distortion (THD). The `thd` function computes the power ratio of the harmonics to the fundamental and ignores the DC component and the noise floor.

```
thd(x,Fs);
```



Plot the signal spectrum and annotate the signal to noise and distortion ratio (SINAD). The `sinad` function computes the power ratio of the fundamental to the harmonics and the noise floor. It ignores only the DC component.

```
sinad(x,Fs);
```



Verify that the SNR, THD, and SINAD obey the equation

$$10^{-\mathrm{SNR}/10} + 10^{\mathrm{THD}/10} = 10^{-\mathrm{SINAD}/10}.$$

```
lhs = 10^(-snr(x,Fs)/10)+10^(thd(x,Fs)/10)
```

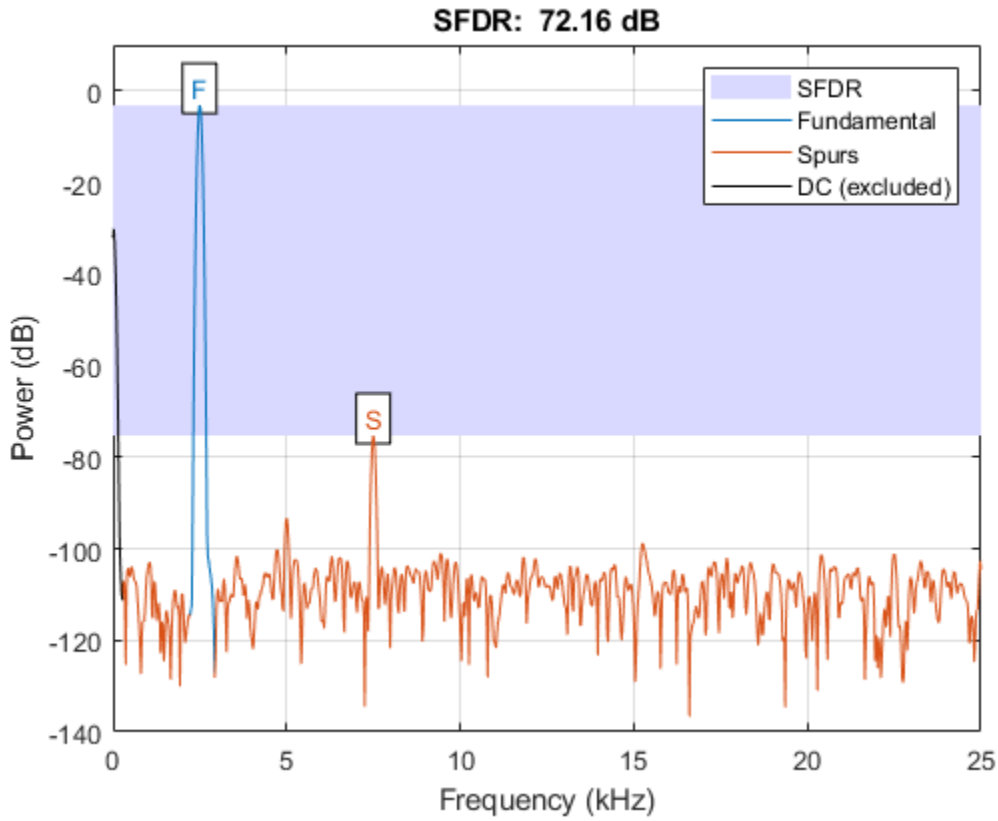
```
lhs = 7.2203e-08
```

```
rhs = 10^(-sinad(x,Fs)/10)
```

```
rhs = 7.1997e-08
```

Plot the signal spectrum and annotate the spurious-free dynamic range (SFDR). The SFDR is the power ratio of the fundamental to the strongest spurious component ("spur"). In this case, the spur corresponds to the third harmonic.

```
sfdr(x,Fs);
```



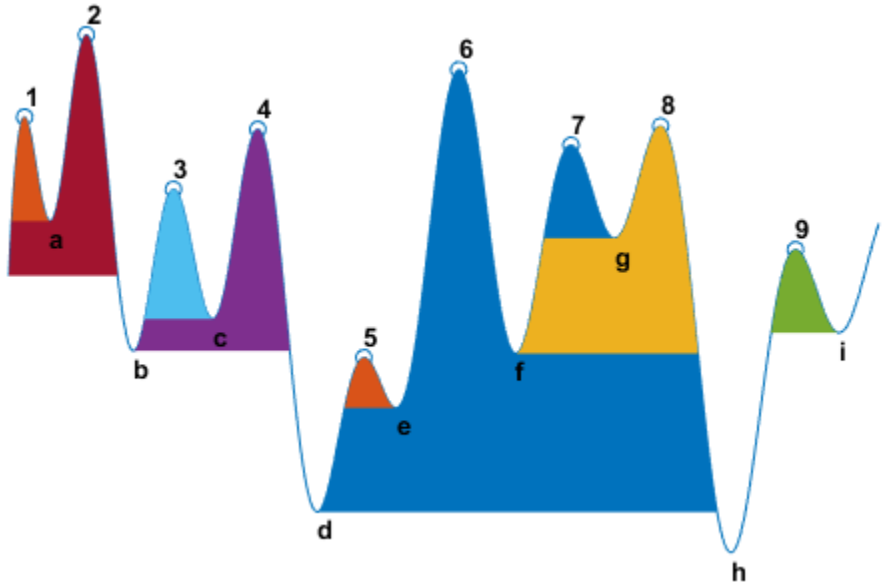
Prominence

The *prominence* of a peak measures how much the peak stands out due to its intrinsic height and its location relative to other peaks. A low isolated peak can be more prominent than one that is higher but is an otherwise unremarkable member of a tall range.

To measure the prominence of a peak:

1. Place a marker on the peak.
2. Extend a horizontal line from the peak to the left and right until the line does one of the following:
   - Crosses the signal because there is a higher peak
   - Reaches the left or right end of the signal
3. Find the minimum of the signal in each of the two intervals defined in Step 2. This point is either a valley or one of the signal endpoints.
4. The higher of the two interval minima specifies the reference level. The height of the peak above this level is its prominence.

`findpeaks` makes no assumption about the behavior of the signal beyond its endpoints, whatever their height. This is reflected in Steps 2 and 4 and often affects the value of the reference level. Consider for example the peaks of this signal:



Peak Number	Left Interval Lies Between Peak and	Right Interval Lies Between Peak and	Lowest Point on the Left Interval	Lowest Point on the Right Interval	Reference Level (Highest Minimum)
1	Left end	Crossing due to peak **2**	Left endpoint	**a**	**a**
2	Left end	Right end	Left endpoint	**h**	Left endpoint
3	Crossing due to peak **2**	Crossing due to peak **4**	**b**	**c**	**c**
4	Crossing due to peak **2**	Crossing due to peak **6**	**b**	**d**	**b**
5	Crossing due to peak **4**	Crossing due to peak **6**	**d**	**e**	**e**
6	Crossing due to peak **2**	Right end	**d**	**h**	**d**
7	Crossing due to peak **6**	Crossing due to peak **8**	**f**	**g**	**g**

Peak Number	Left Interval Lies Between Peak and	Right Interval Lies Between Peak and	Lowest Point on the Left Interval	Lowest Point on the Right Interval	Reference Level (Highest Minimum)
8	Crossing due to peak **6**	Right end	**f**	**h**	**f**
9	Crossing due to peak **8**	Crossing due to right endpoint	**h**	**i**	**i**

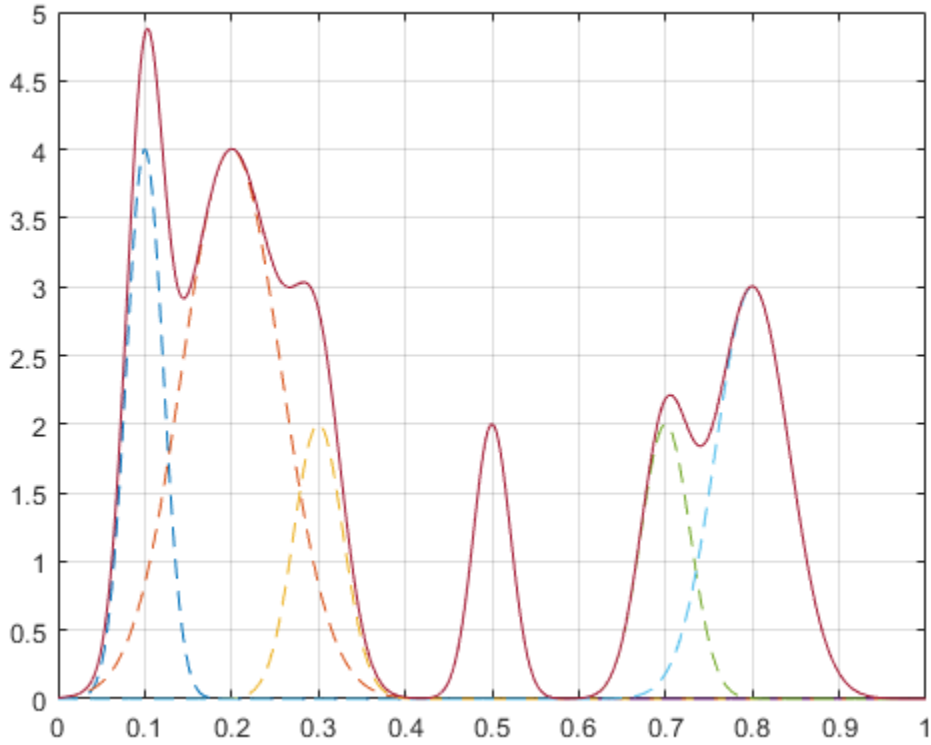
Determine Peak Widths

Create a signal that consists of a sum of bell curves. Specify the location, height, and width of each curve.

```
x = linspace(0,1,1000);

Pos = [1 2 3 5 7 8]/10;
Hgt = [4 4 2 2 2 3];
Wdt = [3 8 4 3 4 6]/100;

for n = 1:length(Pos)
    Gauss(n,:) =  Hgt(n)*exp(-((x - Pos(n))/Wdt(n)).^2);
end

PeakSig = sum(Gauss);
```

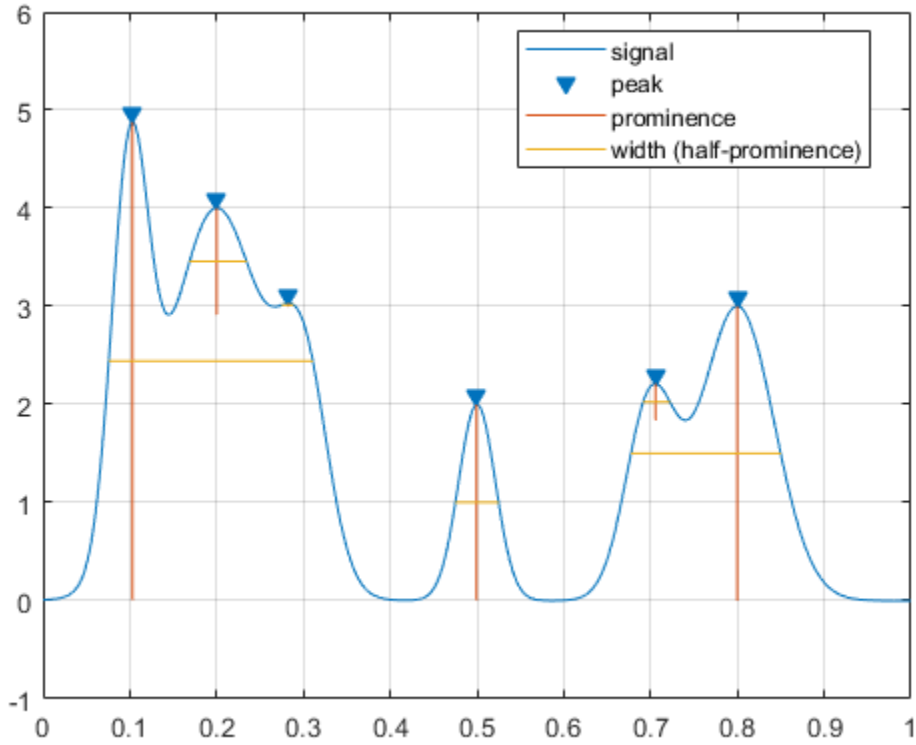
Plot the individual curves and their sum.

```
plot(x,Gauss,'--',x,PeakSig)
grid
```



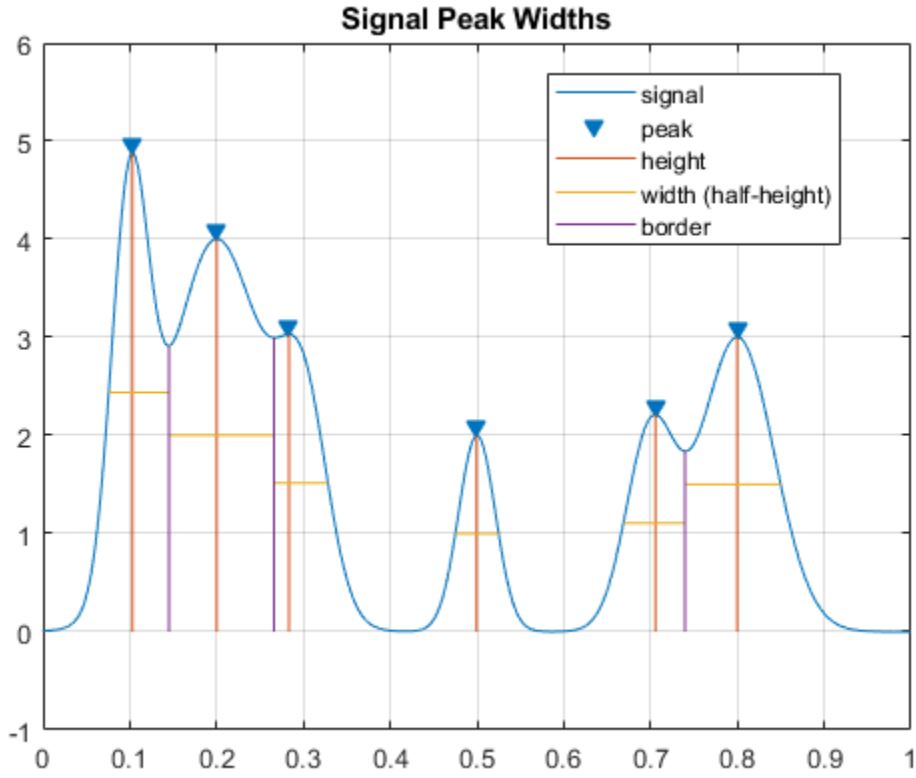
Measure the widths of the peaks using the half prominence as reference.

```
findpeaks(PeakSig,x,'Annotate','extents')
```



Measure the widths again, this time using the half height as reference.

```
findpeaks(PeakSig,x,'Annotate','extents','WidthReference','halfheight')
title('Signal Peak Widths')
```


Signal Peak Widths
signal
peak
height
width (half-height)
border
6
5
4
3
2
1
0
-1
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1

Human Activity Recognition Simulink Model for Smartphone Deployment

This example shows how to prepare a Simulink® model that classifies human activity based on smartphone sensor signals for code generation and smartphone deployment. The example provides two Simulink models that are ready for deployment to an Android device and an iOS device. After you install the required support package for a target device, train the classification model and deploy the Simulink model to the device.

Prerequisites

Simulink support packages are required for the Simulink models in this example.

- Download and Install Simulink Support Package for Android Devices (required for Android deployment)
- Download and Install Simulink Support Package for Apple iOS Devices (required for iOS deployment)

Load Sample Data Set

Load the `humanactivity` data set.

```
load humanactivity
```

The `humanactivity` data set contains 24,075 observations of five different physical human activities: Sitting, Standing, Walking, Running, and Dancing. Each observation has 60 features extracted from acceleration data measured by smartphone accelerometer sensors. The data set contains the following variables:

- `actid` — Response vector containing the activity IDs in integers: 1, 2, 3, 4, and 5 representing Sitting, Standing, Walking, Running, and Dancing, respectively
- `actnames` — Activity names corresponding to the integer activity IDs
- `feat` — Feature matrix of 60 features for 24,075 observations
- `featlabels` — Labels of the 60 features

The Sensor HAR (human activity recognition) App [1] (Statistics and Machine Learning Toolbox) was used to create the `humanactivity` data set. When measuring the raw acceleration data with this app, a person placed a smartphone in a pocket so that the smartphone was upside down and the screen faced toward the person. The software then calibrated the measured raw data accordingly and extracted the 60 features from the

calibrated data. For details about the calibration and feature extraction, see [2] (Statistics and Machine Learning Toolbox) and [3] (Statistics and Machine Learning Toolbox), respectively. The Simulink models described later also use the raw acceleration data and include blocks for calibration and feature extraction.

Prepare Data

This example uses 90% of the observations to train a model that classifies the five types of human activities and 10% of the observations to validate the trained model. Use `cvpartition` to specify a 10% holdout for the test set.

```
rng('default') % For reproducibility
Partition = cvpartition(actid,'Holdout',0.10);
trainingInds = training(Partition); % Indices for the training set
XTrain = feat(trainingInds,:);
YTrain = actid(trainingInds);
testInds = test(Partition); % Indices for the test set
XTest = feat(testInds,:);
YTest = actid(testInds);
```

Convert the feature matrix `XTrain` and the response vector `YTrain` into a table to load the training data set in the Classification Learner app.

```
tTrain = array2table([XTrain YTrain]);
```

Specify the variable name for each column of the table.

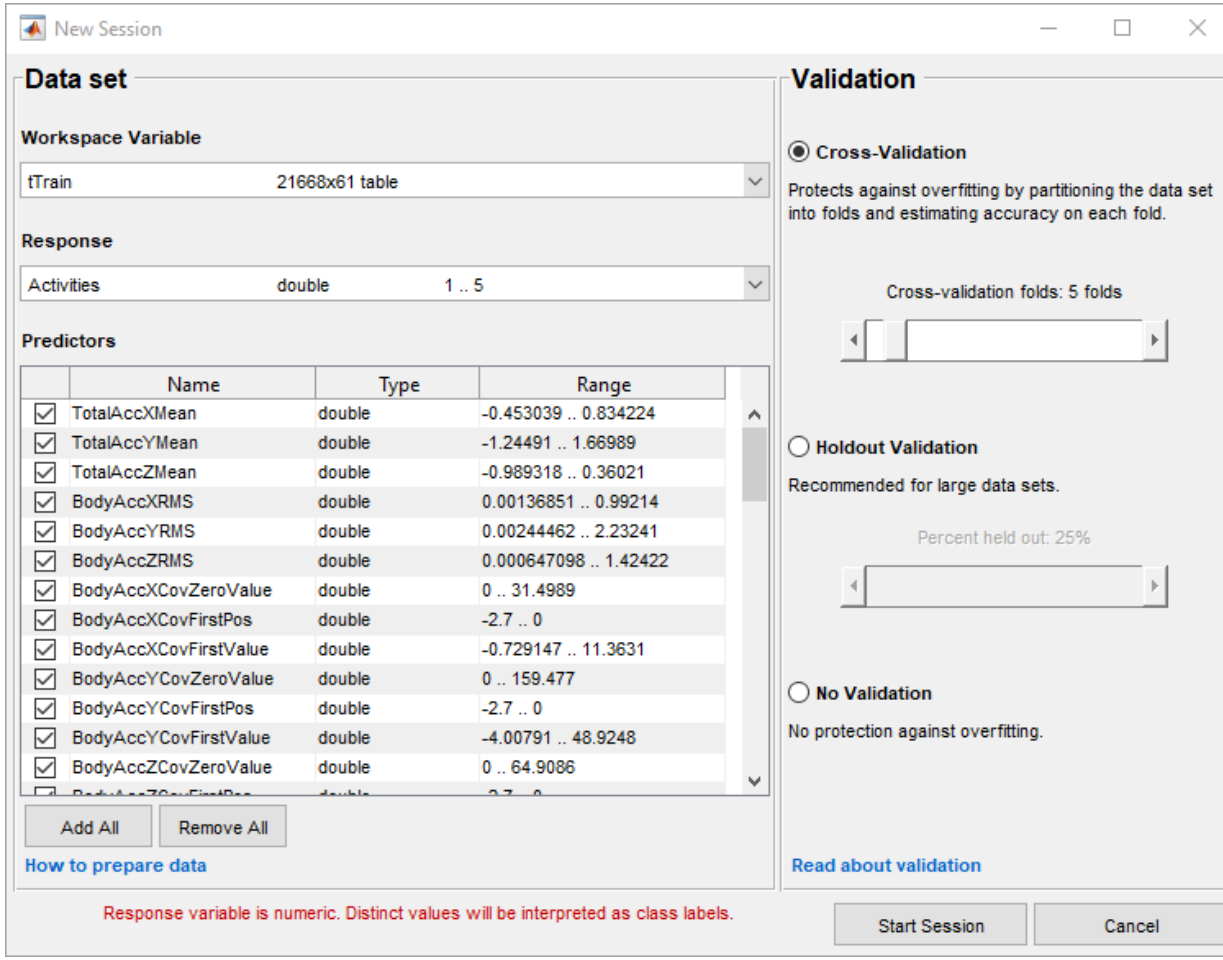
```
tTrain.Properties.VariableNames = [featlabels' 'Activities'];
```

Train Boosted Tree Ensemble Using Classification Learner App

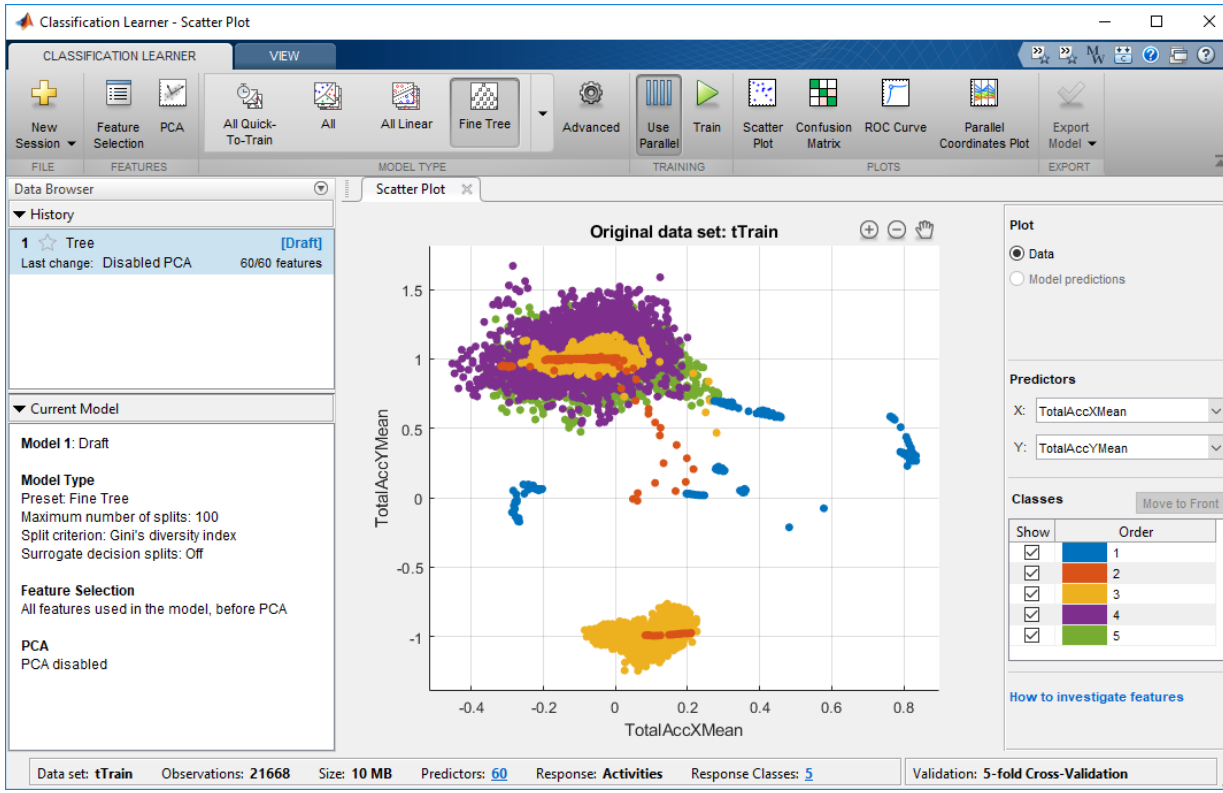
Train a classification model by using the Classification Learner app. To open the Classification Learner app, enter `classificationLearner` at the command line. Alternatively, click the **Apps** tab, and click the arrow at the right of the **Apps** section to open the gallery. Then, under **Machine Learning**, click **Classification Learner**.

On the **Classification Learner** tab, in the **File** section, click **New Session** and select **From Workspace**.

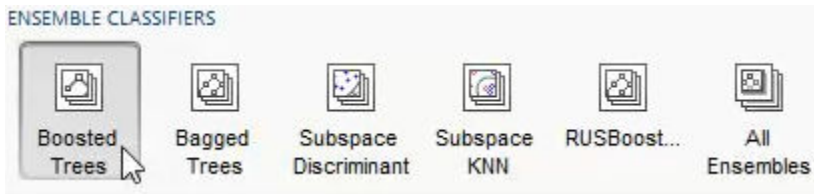
In the New Session dialog box, click the arrow for **Workspace Variable**, and then select the table `tTrain`. Classification Learner detects the predictors and the response from the table.



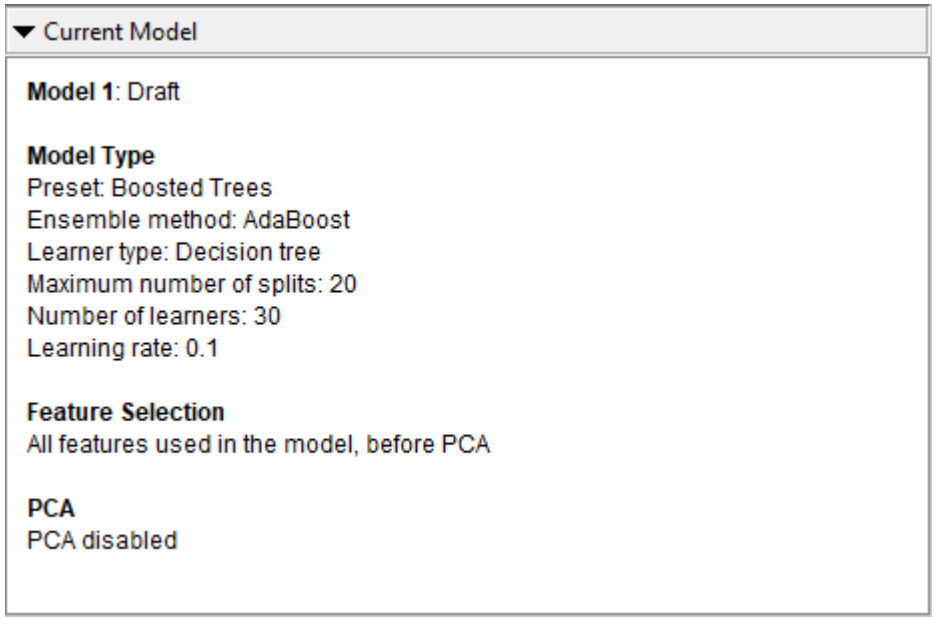
The default option is 5-fold cross-validation, which protects against overfitting. Click **Start Session**. Classification Learner loads the data set and plots a scatter plot of the first two features.



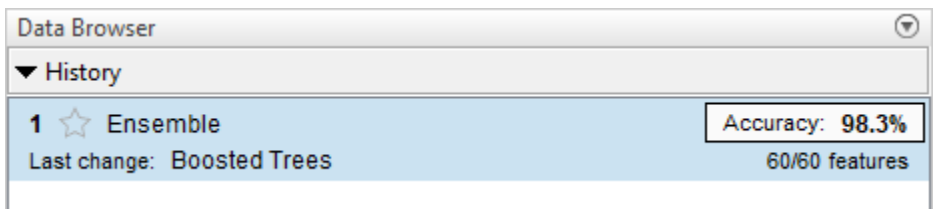
On the **Classification Learner** tab, click the arrow at the right of the **Model Type** section to open the gallery. Then, under **Ensemble Classifiers**, click **Boosted Trees**.



The Current Model pane of the Data Browser displays the default settings of the boosted tree ensemble model.



On the **Classification Learner** tab, in the **Training** section, click **Train**. When the training is complete, the History pane of the Data Browser displays the 5-fold, cross-validated classification accuracy.



On the **Classification Learner** tab, in the **Export** section, click **Export Model**, and then select **Export Compact Model**. Click **OK** in the dialog box. The structure `trainedModel` appears in the MATLAB Workspace. The field `ClassificationEnsemble` of `trainedModel` contains the compact model. Extract the trained model from the structure.

```
classificationEnsemble = trainedModel.ClassificationEnsemble;
```

Train Boosted Tree Ensemble at Command Line

Alternatively, you can train the same classification model at the command line.

```
template = templateTree('MaxNumSplits',20);
classificationEnsemble = fitcensemble(XTrain,YTrain, ...
    'Method','AdaBoostM2', ...
    'NumLearningCycles',30, ...
    'Learners',template, ...
    'LearnRate',0.1, ...
    'ClassNames',[1; 2; 3; 4; 5]);
```

Perform 5-fold cross-validation for `classificationEnsemble` and compute the validation accuracy.

```
partitionedModel = crossval(classificationEnsemble,'KFold',5);
validationAccuracy = 1-kfoldLoss(partitionedModel)
```

```
validationAccuracy = 0.9831
```

Evaluate Performance on Test Data

Evaluate performance on the test data set.

```
testAccuracy = 1-loss(classificationEnsemble,XTest,YTest)
```

```
testAccuracy = 0.9763
```

The trained model correctly classifies 97.63% of the human activities on the test data set. This result confirms that the trained model does not overfit to the training data set.

Note that the accuracy values can vary slightly depending on your operating system.

Save Trained Model

For code generation including a classification model object, use `saveCompactModel` and `loadCompactModel`.

Save the trained model by using `saveCompactModel`.

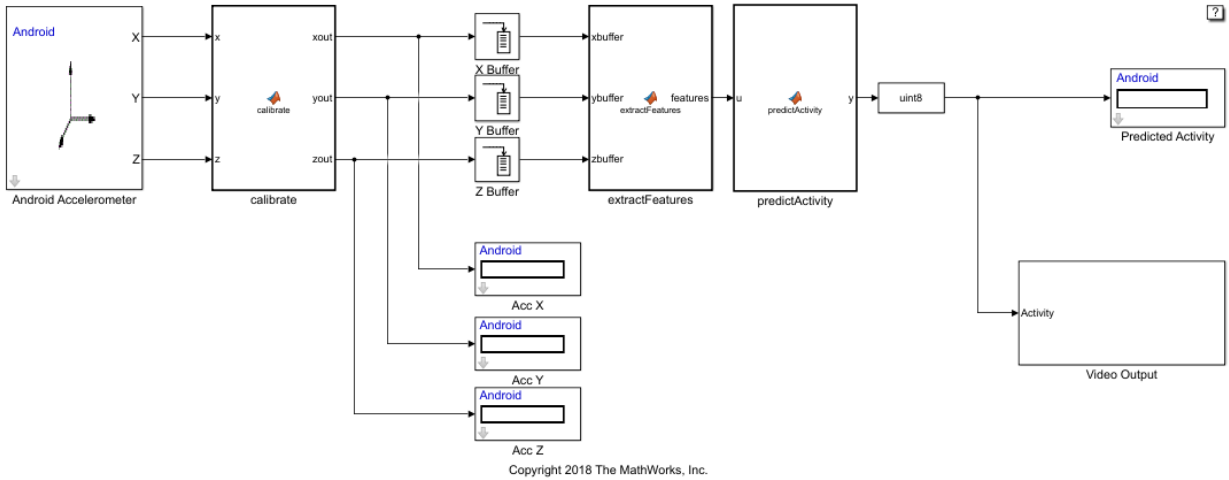
```
saveCompactModel(classificationEnsemble,'EnsembleModel.mat');
```

The function block **predictActivity** in the Simulink models loads the trained model by using `loadCompactModel` and uses the trained model to classify new data.

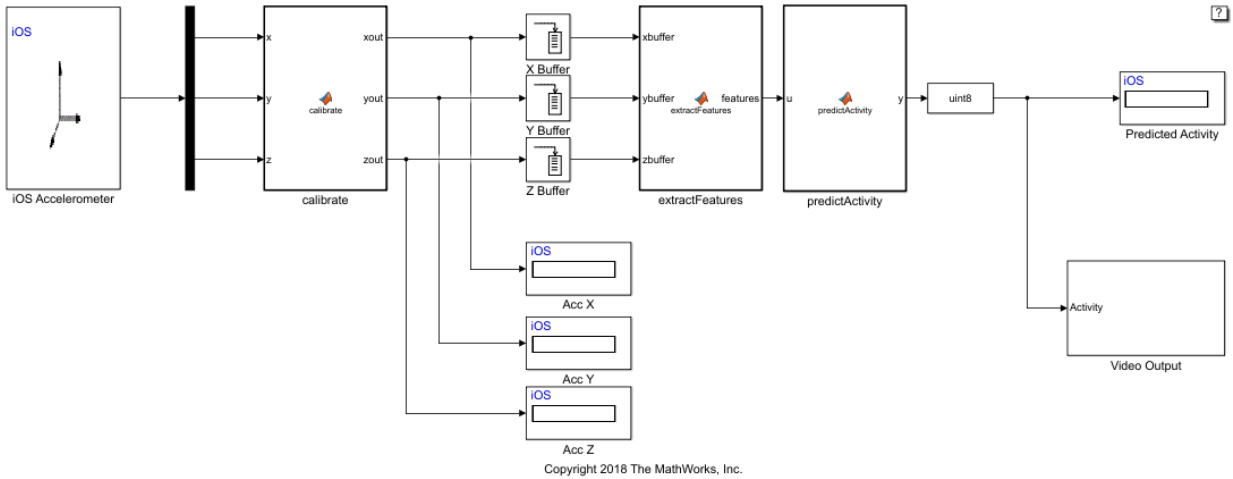
Deploy Simulink Model to Device

Now that you have prepared a classification model, you can open the Simulink model, depending on which type of smartphone you have, and deploy the model to your device. Note that the Simulink model requires the `EnsembleModel.mat` file and the calibration matrix file `slexHARAndroidCalibrationMatrix.mat` or `slexHARiOSCalibrationMatrix.mat`. If you click the button located in the upper-right section of this page and open this example in MATLAB®, then MATLAB® opens the example folder that includes these calibration matrix files.

Type `slexHARAndroidExample` to open the Simulink model for Android deployment.



Type `slexHARiOSExample` to open the Simulink model for iOS deployment.



The two Simulink models classify human activity based on acceleration data measured by a smartphone sensor. The models include the following blocks:

- The **Accelerometer** block receives raw acceleration data from accelerometer sensors on the device.
- The **calibrate** block is a MATLAB Function block that calibrates the raw acceleration data. This block uses the calibration matrix in the `slexHARAndroidCalibrationMatrix.mat` file or the `slexHARiOSCalibrationMatrix.mat` file. If you click the button located in the upper-right section of this page and open this example in MATLAB®, then MATLAB® opens the example folder that includes these files.
- The display blocks **Acc X**, **Acc Y**, and **Acc Z** are connected to the **calibrate** block and display calibrated data points for each axis on the device.
- Each of the **Buffer** blocks, **X Buffer**, **Y Buffer**, and **Z Buffer**, buffers 32 samples of an accelerometer axis with 12 samples of overlap between buffered frames. After collecting 20 samples, each **Buffer** block joins the 20 samples with 12 samples from the previous frame and passes the total 32 samples to the **extractFeatures** block. Each **Buffer** block receives an input sample every 0.1 second and outputs a buffered frame including 32 samples every 2 seconds.
- The **extractFeatures** block is a MATLAB Function block that extracts 60 features from a buffered frame of 32 accelerometer samples. This function block uses DSP System Toolbox™ and Signal Processing Toolbox™.

- The **predictActivity** block is a MATLAB Function block that loads the trained model from the `EnsembleModel.mat` file by using `loadCompactModel` and classifies the user activity using the extracted features. The output is an integer between 1 and 5, corresponding to Sitting, Standing, Walking, Running, and Dancing, respectively.
- The **Predicted Activity** block displays the classified user activity values on the device.
- The **Video Output** subsystem uses a multiport switch block to choose the corresponding user activity image data to display on the device. The **Convert to RGB** block decomposes the selected image into separate RGB vectors and passes the image to the **Activity Display** block.

To deploy the Simulink model to your device, follow the steps in Run Model on Android Devices or Run Model on iOS Devices. After the configuration process is complete, click **Deploy to Hardware** to create an app with the same name as the model. Run the app, place the device in the same way as described earlier for collecting the training data, and try the five activities. The app displays the classified activity accordingly.

To ensure the accuracy of the app, you need to place your device in the same way as described for collecting the training data. If you want to place your device in a different location or orientation, then collect the data in your own way and use your data to train the classification model.

The accuracy of the app can be different from the accuracy of the test data set (`testaccuracy`), depending on the device. To improve the app, you can consider using additional sensors and updating the calibration matrix. Also, you can add another output block for audio feedback to the output subsystem using Audio System Toolbox™. Use a ThingSpeak™ write block to publish classified activities and acceleration data from your device to the Internet of Things. For details, see https://thingspeak.com/.

Spectrum Object to Function Replacement

- “Nonparametric Spectrum Object to Function Replacement” on page 17-2
- “Autoregressive PSD Object to Function Replacement Syntax” on page 17-11
- “Subspace Pseudospectrum Object to Function Replacement Syntax” on page 17-13

Nonparametric Spectrum Object to Function Replacement

In this section...
“Periodogram PSD Object to Function Replacement Syntax” on page 17-2
“Periodogram MSSPECTRUM Object to Function Replacement Syntax” on page 17-3
“Welch PSD Object to Function Replacement Syntax” on page 17-4
“Welch MSSPECTRUM Object to Function Replacement Syntax” on page 17-6
“Multitaper PSD Object to Function Replacement Syntax” on page 17-9

Periodogram PSD Object to Function Replacement Syntax

The `spectrum.periodogram` object syntax will be removed in the future. The following table gives the equivalent recommended function syntax for `periodogram`. In the modified periodogram, you use a window other than the default rectangular window. To illustrate modified periodogram syntaxes, the table uses a specific window. In each example, `x` is the input signal.

Deprecated Syntax	Replacement Syntax
`h = spectrum.periodogram;` `psd(h,x);`	`periodogram(x);`

`% Modified periodogram with window function`
`h = spectrum.periodogram('hamming');`
`psd(h,x);`
	`win = hamming(length(x));`
	`periodogram(x,win);`

`% Window function and optional input argument`
`h = spectrum.periodogram({'Hamming','periodic'});`
`psd(h,x);`
	`win = hamming(length(x),'periodic');`
	`periodogram(x,win);`

`% Taylor window and multiple optional input arguments`
`nbar = 4;`
`sll = 30;`
`h = spectrum.periodogram({'Taylor',nbar,sll});`
`psd(h,x,'Fs',fs,'centerdc',true);`
	`nbar = 4;`
	`sll = -30;`
	`win = taylorwin(length(x),nbar,sll);`
	`periodogram(x,win,[],fs,'centered');`

`h = spectrum.periodogram(...);` `psd(h,x,'NFFT',nfft);`	`win = ...` `periodogram(x,win,nfft);`

Deprecated Syntax	Replacement Syntax
`h = spectrum.periodogram(...);` `psd(h,x,'Fs',fs);`	`win = ...` `periodogram(x,win,[],fs);`
`h = spectrum.periodogram(...);` `psd(h,x,'NFFT',nfft,'Fs',fs);`	`win = ...` `periodogram(x,win,nfft,fs);`

`h = spectrum.periodogram(...);`
`psd(h,x,...,'FreqPoints','User Defined',...`
`'FrequencyVector',w);`
	`win = ...`
	`periodogram(x,win,w);`

`h = spectrum.periodogram(...);`
`psd(h,x,'FreqPoints','User Defined',...`
`'FrequencyVector',f,'Fs',fs);`
	`win = ...`
	`periodogram(x,win,f,fs);`

`% Two-sided spectrum of a real signal`
`h = spectrum.periodogram(...);`
`psd(h,x,...,'SpectrumType','TwoSided');`
	`win = ...`
	`periodogram(x,win,...,'twosided');`

`% Two-sided spectrum with DC (0 frequency) in the center`
`h = spectrum.periodogram(...);`
`psd(h,x,...,'CenterDC',true);`
	`win = ...`
	`periodogram(x,win,...,'centered');`

`h = spectrum.periodogram(...);` `psd(h,x,...,'ConfLevel',p);`	`win = ...` `periodogram(x,win,...,'ConfidenceLevel',p);`

`h = spectrum.periodogram(...);`
`hPSD = psd(h,x,...);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
	`win = ...`
	`[Pxx,F] = periodogram(x,win,...);`

`h = spectrum.periodogram(...);`
`hPSD = psd(h,x,...,'ConfLevel',p);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
`Pxxc = hPSD.ConfInterval;`
	`win = ...`
	`[Pxx,F,Pxxc] = periodogram(x,win,...);`

Periodogram MSSPECTRUM Object to Function Replacement Syntax

The `spectrum.periodogram` MSSPECTRUM object syntax will be removed in the future. The following table gives the equivalent recommended function syntax for `periodogram`. In the modified periodogram, you use a window other than the default rectangular window. To illustrate modified periodogram syntaxes, the table uses a specific window. In each example, `x` is the input signal.

Deprecated Syntax	Recommended Syntax
`h = spectrum.periodogram;` `msspectrum(h,x);`	`periodogram(x,'power');`
`h = spectrum.periodogram('Hamming');` `msspectrum(h,x);`	`win = hamming(length(x));` `periodogram(x,win,'power');`
`h = spectrum.periodogram({'Hamming','periodic'});` `msspectrum(h,x);`	`win = hamming(length(x),'periodic');` `periodogram(x,win,'power');`

`nbar = 4;`
`sll = 30;`
`h = spectrum.periodogram({'Taylor',nbar,sll});`
`msspectrum(h,x);`
	`nbar = 4;`
	`sll = -30;`
	`win = taylorwin(length(x),nbar,sll);`
	`periodogram(x,win,'power');`

`h = spectrum.periodogram(...);` `msspectrum(h,x,'NFFT',nfft);`	`win= ...` `periodogram(x,win,nfft,'power');`
`h = spectrum.periodogram(...);` `msspectrum(h,x,'Fs',fs);`	`win = ...` `periodogram(x,win,[],fs,'power');`
`h = spectrum.periodogram(...);` `msspectrum(h,x,'NFFT',nfft,'Fs',fs);`	`win = ...` `periodogram(x,win,nfft,fs,'power');`
`h = spectrum.periodogram(...);` `msspectrum(h,x,...,'SpectrumType','TwoSided');`	`win = ...` `periodogram(x,win,..., 'twosided','power');`
`h = spectrum.periodogram(...);` `msspectrum(h, x,...,'CenterDC',true);`	`win = ...` `periodogram(x,win,...,'centered','power');`
`h = spectrum.periodogram(...);` `msspectrum(h,x,...,'ConfLevel',p);`	`win = ...` `periodogram(x,win,...,'ConfidenceLevel', p,...'power`

`h = spectrum.periodogram(...);`
`hMS = msspectrum(h,x,...);`
`Sxx = hMS.Data;`
`F = hMS.Frequencies;`
	`win = ...`
	`[Sxx,F] = periodogram(x,win,...,'power');`

`h = spectrum.periodogram(...);`
`hMS = msspectrum(h,x,...,'ConfLevel',p);`
`Sxx = hMS.Data;`
`F = hMS.Frequencies;`
`Sxxc = hMS.ConfInterval;`
	`win = ...`
	`[Sxx,F,Sxxc] = periodogram(x,win,...,'power');`

Welch PSD Object to Function Replacement Syntax

The `spectrum.welch` object syntax will be removed in the future. The following table gives the equivalent recommended function syntax for `pwelch`. To illustrate modified

periodogram syntaxes, the table uses a specific window. In each example, x is the input signal.

Deprecated Syntax	Replacement Syntax
`h = spectrum.welch;` `psd(h,x);`	`pwelch(x);`
`h = spectrum.welch('Gaussian');` `psd(h,x);`	`win = gausswin(64);` `pwelch(x,win);`

`% Welch estimate with window function and optional input arguments`
`h = spectrum.welch({'Hamming','periodic'});`
`psd(h,x);`
	`win = hamming(64,'periodic');`
	`pwelch(x,win);`

`% Taylor window and multiple optional input arguments`
`nbar = 4;`
`sll = 30;`
`h = spectrum.welch({'Taylor', nbar, sll});`
`psd(h,x);`
	`nbar = 4;`
	`sll = -30;`
	`win = taylorwin(64,nbar,sll);`
	`pwelch(x,win);`

`h = spectrum.welch('Hamming',segLen);` `psd(h,x);`	`win = hamming(segLen);` `pwelch(x,win);`

`h = spectrum.welch({'Hamming','periodic'},...`
`segLen);`
`psd(h,x);`
	`win = hamming(segLen,'periodic');`
	`pwelch(x,win);`

`nbar = 4;`
`sll = 30;`
`h = spectrum.welch({'Taylor',nbar,sll},...`
`segLen);`
`psd(h,x);`
	`nbar = 4;`
	`sll = -30;`
	`win = taylorwin(segLen,nbar,sll);`
	`pwelch(x,win);`

`h = spectrum.welch('Hamming',segLen,ovlpPct);`
`psd(h,x);`
	`win = hamming(segLen);`
	`Noverlap = ceil((ovlpPct/100)*segLen);`
	`pwelch(x,win,Noverlap);`

`h = spectrum.welch({'Hamming','periodic'},...`
`segLen,ovlpPct);`
`psd(h,x);`
	`win = hamming(segLen,'periodic');`
	`Noverlap = ceil((ovlpPct/100)*segLen);`
	`pwelch(x,win,Noverlap);`

`nbar = 4;`
`sll = 30;`
`h = spectrum.welch({'Taylor',nbar,sll},...`
`segLen,ovlpPct);`
`psd(h,x);`
	`nbar = 4;`
	`sll = -30;`
	`win = taylorwin(segLen,nbar,sll);`
	`Noverlap = ceil((ovlpPct/100)*segLen);`
	`pwelch(x,win,Noverlap);`

Deprecated Syntax	Replacement Syntax

`h = spectrum.welch(...);`
`psd(h,x,'NFFT',nfft);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,nfft);`

`h = spectrum.welch(...);`
`psd(h,x,'Fs',fs);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,[],fs);`

`h = spectrum.welch(...);`
`psd(h,x,'NFFT',nfft,'Fs',fs);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,nfft,fs);`

`h = spectrum.welch(...);`
`psd(h,x,...,'FreqPoints','User Defined',...`
`'FrequencyVector',w);`
	`win = ...`
	`periodogram(x,win,w);`

`h = spectrum.periodogram(...);`
`psd(h,x,'FreqPoints','User Defined',...`
`'FrequencyVector',f,'Fs',fs);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,f,fs);`

`% Two-sided spectrum of a real signal`
`h = spectrum.welch(...);`
`psd(h,x,...,'SpectrumType','TwoSided');`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,...,'twosided');`

`% Two-sided spectrum with DC (0 frequency) in the center`
`h = spectrum.welch(...);`
`psd(h,x,...,'CenterDC',true);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,...,'centered');`

`h = spectrum.welch(...);`
`psd(h,x,...,'ConfLevel',p);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,...'ConfidenceLevel',p);`

`h = spectrum.welch(...);`
`hPSD = psd(h,x,...);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
	`win = ...`
	`Noverlap = ...`
	`[Pxx,F] = pwelch(x,win,Noverlap,...);`

`h = spectrum.periodogram(...);`
`hPSD = psd(h,x,...,'ConfLevel',p);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
`Pxxc = hPSD.ConfInterval;`
	`win = ...`
	`Noverlap = ...`
	`[Pxx,F,Pxxc] = pwelch(x,win,Noverlap,...`
	`'ConfidenceLevel',p);`

Welch MSSPECTRUM Object to Function Replacement Syntax

The `spectrum.welch` MSSPECTRUM object syntax will be removed in the future. The following table gives the equivalent recommended function syntax for `pwelch`. In the

modified periodogram, you use a window other than the default rectangular window. To illustrate modified periodogram syntaxes, the table uses a specific window. In each example, x is the input signal.

Deprecated Syntax	Recommended Syntax
`h = spectrum.welch` `msspectrum(h,x);`	`win = hamming(64);` `pwelch(x,win,[],'power');`
`h = spectrum.welch('Gaussian');` `msspectrum(h,x);`	`win = gausswin(64);` `pwelch(x,win,[],'power');`
`h = spectrum.welch({'Hamming','periodic'});` `msspectrum(h,x);`	`win = hamming(64,'periodic');` `pwelch(x,win,[],'power');`

`nbar = 4;`
`sll = 30;`
`h = spectrum.welch({'Taylor',nbar,sll});`
`msspectrum(h,x);`
	`nbar = 4;`
	`sll = -30;`
	`win = taylorwin(64,nbar,sll);`
	`pwelch(x,win,[],'power');`

`segLen = 128;`
`h = spectrum.welch('Hamming',segLen);`
`msspectrum(h,x);`
	`win = hamming(128);`
	`pwelch(x,win,[],'power');`

`segLen = 128;`
`h = spectrum.welch({'Hamming','periodic'},...`
`segLen);`
`msspectrum(h,x);`
	`win = hamming(128,'periodic');`
	`pwelch(x,win,[],'power');`

`nbar = 4;`
`sll = 30;`
`segLen = 128;`
`h = spectrum.welch({'Taylor',nbar,sll},segLen);`
`msspectrum(h,x);`
	`nbar = 4;`
	`sll = -30;`
	`segLen = 128;`
	`win = taylorwin(segLen,nbar,sll);`
	`pwelch(x,win,[],'power');`

`segLen = 128;`
`ovlpPct = 50;`
`h = spectrum.welch('Hamming',segLen,ovlpPct);`
`msspectrum(h,x);`
	`segLen = 128;`
	`win = hamming(segLen);`
	`ovlpPct = 50;`
	`Noverlap = ceil((ovlpPct/100)*segLen);`
	`pwelch(x,win,Noverlap,'power');`

`segLen = 128;`
`ovlpPct = 50;`
`h = spectrum.welch({'Hamming','periodic'},...`
`segLen,ovlpPct);`
`msspectrum(h,x);`
	`segLen = 128;`
	`ovlpPct = 50;`
	`win = hamming(segLen,'periodic');`
	`Noverlap = ceil((ovlpPct/100)*segLen);`
	`pwelch(x,win,Noverlap,'power');`

Deprecated Syntax	Recommended Syntax

`nbar = 4;`
`sll = 30;`
`segLen = 128;`
`ovlpPct = 50;`
`h = spectrum.welch({'Taylor',nbar,sll},...`
`segLen,ovlpPct);`
`msspectrum(h,x);`
	`nbar = 4;`
	`sll = -30;`
	`segLen = 128;`
	`win = taylorwin(segLen,nbar,sll);`
	`ovlpPct = 50;`
	`Noverlap = ceil((ovlpPct/100)*segLen);`
	`pwelch(x,win,Noverlap,'power');`

`h = spectrum.welch(...);`
`msspectrum(h,x,'NFFT',nfft);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,nfft,'power');`

`h = spectrum.welch(...);`
`msspectrum(h,x,'Fs',fs);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,[],fs,'power');`

`h = spectrum.welch(...);`
`msspectrum(h,x,'NFFT',nfft,'Fs',fs);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,nfft,fs,'power');`

`h = spectrum.welch(...);`
`msspectrum(h, x,...,'FreqPoints','User Defined',`
`'FrequencyVector',w);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,f,fs,'power');`

`h = spectrum.welch(...);`
`msspectrum(h,x,...,'SpectrumType','TwoSided');`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,...,'twosided','power');`

`h = spectrum.welch(...);`
`msspectrum(h,x,...,'CenterDC',true);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,...,'centered','power');`

`h = spectrum.welch(...);`
`msspectrum(h,x,...,'ConfLevel',p);`
	`win = ...`
	`Noverlap = ...`
	`pwelch(x,win,Noverlap,...,'ConfidenceLevel',p,'power`

`h = spectrum.welch(...);`
`hMS = msspectrum(h,x,...);`
`Sxx = hMS.Data;`
`F = hMS.Frequencies;`
	`[Sxx,F] = pwelch(...,'power');`

`h = spectrum.welch(...);`
`hMS = msspectrum(h, x, …, 'ConfLevel', p);`
`Sxx = hMS.Data;`
`F = hMS.Frequencies;`
`Sxxc = hMS.ConfInterval;`
	`[Sxx,F,Sxxc] = pwelch(...,'ConfidenceLevel',p,'power`

Multitaper PSD Object to Function Replacement Syntax

The `spectrum.mtm` object syntax will be removed in the future. The following table gives the equivalent recommended function syntax for `pmtm`. In each example, `x` is the input signal.

Deprecated Syntax	Recommended Syntax
`hMTM = spectrum.mtm;` `psd(hMTM,x);`	`pmtm(x,4);`
`hMTM = spectrum.mtm(NW);` `psd(hMTM,x);`	`pmtm(x,NW);`

`[E,V] = dpss(length(x),NW);`
`hMTM = spectrum.mtm(E,V);`
`psd(hMTM,x);`
	`[E,V] = dpss(length(x),NW);`
	`pmtm(x,E,V);`

`hMTM = spectrum.mtm(NW);` `psd(hMTM,x,'Fs',fs);`	`pmtm(x,NW,fs);`
`hMTM = spectrum.mtm(E,V);` `psd(hMTM,x,'Fs',fs);`	`pmtm(x,E,V,fs);`
`hMTM = spectrum.mtm(NW);` `psd(hMTM,x,'Fs',fs,'NFFT',nfft);`	`pmtm(x,NW,nfft,fs);`
`hMTM = spectrum.mtm(E,V);` `psd(hMTM,x,'Fs',fs,'NFFT',nfft);`	`pmtm(x,E,V,nfft,fs);`

`hMTM = spectrum.mtm(NW);`
`psd(hMTM,x,'FreqPoints','User Defined',...`
`'FrequencyVector',w);`
	`pmtm(x,NW,w);`

`hMTM = spectrum.mtm(E,V);`
`psd(hMTM,x,'FreqPoints','User Defined',...`
`'FrequencyVector',w);`
	`pmtm(x,E,V,w);`

`hMTM = spectrum.mtm(NW);`
`psd(hMTM,x,'FreqPoints','User Defined',...`
`'FrequencyVector',f,'Fs',fs);`
	`pmtm(x,E,V,f,fs);`

`hMTM = spectrum.mtm(E,V);`
`psd(hMTM,x,'FreqPoints','User Defined',...`
`'FrequencyVector',f,'Fs',fs);`
	`pmtm(x,E,V,f,fs);`

`hMTM = spectrum.mtm(...,'Adaptive');` `psd(hMTM,...);`	`pmtm(...,'adapt');`
`hMTM = spectrum.mtm(...,'Eigenvalue');` `psd(hMTM,...);`	`pmtm(...,'eigen');`

Deprecated Syntax	Recommended Syntax
`hMTM = spectrum.mtm(...,'Unity');` `psd(hMTM,...);`	`pmtm(...,'unity');`
`hMTM = spectrum.mtm(...);` `psd(hMTM,...,'SpectrumType','twosided');`	`pmtm(...,'twosided');`

`hMTM = spectrum.mtm(...);`
`psd(hMTM,...,'SpectrumType','twosided',...`
`'CenterDC',true);`
	`pmtm(...,'centered');`

`hMTM = spectrum.mtm(...);` `psd(hMTM,...,'ConfLevel',p);`	`pmtm(...,'ConfidenceLevel',p);`

`hMTM = spectrum.mtm(...);`
`hPSD = psd(hMTM,...);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
	`[Pxx,F] = pmtm(...);`

`hMTM = spectrum.mtm(...);`
`hPSD = psd(hMTM,x,'ConfLevel',p);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
`Pxxc = hPSD.ConfInterval;`
	`[Pxx,F,Pxxc] = pmtm(x,'ConfidenceLevel',p);`

Autoregressive PSD Object to Function Replacement Syntax

The AR PSD object syntax will be removed in the future. The following table gives the equivalent recommended function syntax. The table uses `spectrum.burg` and `pburg` as examples, but the object-to-function replacement syntaxes are valid for all the AR spectral estimators with the appropriate substitution: `spectrum.burg` to `pburg`, `spectrum.cov` to `pcov`, `spectrum.mcov` to `pmcov`, and `spectrum.yulear` to `pyulear`. In each example, `x` is the input signal.

Deprecated Syntax	Replacement Syntax
`hBurg = spectrum.burg;` `psd(hBurg,x);`	`pburg(x,4);`
`hBurg = spectrum.burg(order);` `psd(hBurg,x);`	`pburg(x,order);`
`hBurg = spectrum.burg(order);` `psd(hBurg,x,'NFFT',nfft);`	`pburg(x,order,nfft);`
`hBurg = spectrum.burg(order);` `psd(hBurg,x,'Fs',fs);`	`pburg(x,order,[],fs);`
`hBurg = spectrum.burg(order);` `psd(hBurg,x,'NFFT',nfft,'Fs',fs);`	`pburg(x,order,nfft,fs);`

`hBurg = spectrum.burg(order);`
`psd(hBurg, x,...,'FreqPoints','User Defined',...`
`'FrequencyVector',w);`
	`pburg(x,order,w);`

`hBurg = spectrum.burg(order);`
`psd(hBurg,x,'FreqPoints','User Defined',...`
`'FrequencyVector',f,'Fs',fs);`
	`pburg(x,order,f,fs);`

`hBurg = spectrum.burg` `psd(...,'SpectrumType','TwoSided');`	`pburg(...,'twosided');`
`hBurg = spectrum.burg;` `psd(...,'CenterDC',true);`	`pburg(x,...,'centered');`
`hBurg = spectrum.burg;` `psd(...,'ConfLevel',p);`	`pburg(x,...,'ConfidenceLevel',p);`

`hBurg = spectrum.burg;`
`hPSD = psd(...);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
	`[Pxx,F] = pburg(...);`

Deprecated Syntax	Replacement Syntax

`hBurg = spectrum.burg;`
`hPSD = psd(...,'ConfLevel',p);`
`Pxx = hPSD.Data;`
`F = hPSD.Frequencies;`
`Pxxc = hPSD.ConfInterval;`
	`[Pxx,F,Pxxc] = pburg(...);`

Subspace Pseudospectrum Object to Function Replacement Syntax

The pseudospectrum object syntax will be removed in the future. The following table gives the equivalent recommended function syntax. The table uses `spectrum.music` and the functional equivalent, `pmusic`, but the syntax replacements are also valid for `spectrum.eigenvector` to `peig`. In each example, `x` is the input signal.

Deprecated Syntax	Replacement Syntax
`h = spectrum.music(nsinusoids);` `pseudospectrum(h,x);`	`pmusic(x,nsinusoids)`
`h = spectrum.music(nsinusoids);` `pseudospectrum(h,x,'Fs',fs)`	`pmusic(x,nsinusoids,[],fs);`

`h = spectrum.music(nsinusoids,segLen,ovlpPct,...`
`'Hamming');`
`pseudospectrum(h,x)`
	`win = hamming(segLen)`
	`Noverlap = ceil(ovlpPct/100*segLen);`
	`P = nsinusoids;`
	`Fs = 2*pi;`
	`pmusic(x,P,[],Fs,win,Noverlap);`

`h = spectrum.music(nsinusoids,segLen,ovlpPct,...`
`winName,thresh);`
`pseudospectrum(h,x)`
	`win = winfunc(segLen)`
	`Noverlap = ceil(ovlpPct/100*segLen);`
	`P = [nsinusoids thresh];`
	`Fs = 2 *pi;`
	`pmusic(x,P,[],Fs,win,Noverlap);`

`h = spectrum.music(nsinusoids,segLen,...`
`ovlpPct,winName,thresh);`
`pseudospectrum(h,x,'Fs',fs)`
	`win = hamming(segLen)`
	`nfft = max(256,2^nexpow2(segLen));`
	`Noverlap = ceil(ovlpPct/100*segLen);`
	`P = [nsinusoids thresh];`
	`pmusic(x,P,[],Fs,win,Noverlap);`

`h = spectrum.music(nsinusoids,segLen,...`
`ovlpPct,winName,thresh);`
`pseudospectrum(h,x,'Fs',fs,'SpectrumRange',range)`
	`win = hamming(segLen)`
	`Noverlap = ceil(ovlpPct/100*segLen);`
	`P = [nsinusoids thresh];`
	`pmusic(x,P,[],Fs,range,win,Noverlap);`

`h = spectrum.music(nsinusoids,segLen,...`
`ovlpPct,winName,thresh);`
`pseudospectrum(h,x,'Fs',fs,'SpectrumRange',range,'NFFT',nfft)`
	`win = hamming(segLen)`
	`Noverlap = ceil(ovlpPct/100*segLen);`
	`P = [nsinusoids thresh];`
	`pmusic(x,P,nfft,Fs,range,win,Noverlap);`

Deprecated Syntax	Replacement Syntax

`h = spectrum.music(nsinusoids,segLen,...`
`ovlpPct,winName,thresh);`
`pseudospectrum(h,x,...,'FreqPoints','User Defined',`
`'Frequency Vector',fVec)`
	`win = hamming(segLen)`
	`Noverlap = ceil(ovlpPct/100*segLen);`
	`P = [nsinusoids thresh];`
	`pmusic(x,P,fVec,Fs,range,win,Noverlap);`

`h = spectrum.music(...,'DataMatrix');` `pseudospectrum(...)`	`nfft = min(256,2^nextpow2(size(x,1)));` `pmusic(x,P,nfft,Fs,range,win)`

`h = spectrum.music(...,'CorrelationMatrix');`
`pseudospectrum(...)`
	`pmusic(x,P,'corr',nfft,Fs,range,win,Noverlap);`
	`% or equivalently`
	`pmusic(x,P,'corr',fVec,Fs,range,win,Noverlap);`

`h = spectrum.music(...);` `pseudospectrum(...,'CenterDC',true)`	`pmusic(...,'centered');`
`[Spec,F] = pseudospectrum(...)`	`[Spec,F] = pmusic(...);`

Vibration Analysis

- “Frequency-RPM Map of Helicopter Vibration Data” on page 18-2
- “Find Ridge of Noisy Signal” on page 18-6
- “Modal Parameters of MIMO System” on page 18-10
- “Compute and Display Order-RPM Map” on page 18-15
- “MIMO Stabilization Diagram” on page 18-18
- “Modal Analysis of Identified Models” on page 18-23

Frequency-RPM Map of Helicopter Vibration Data

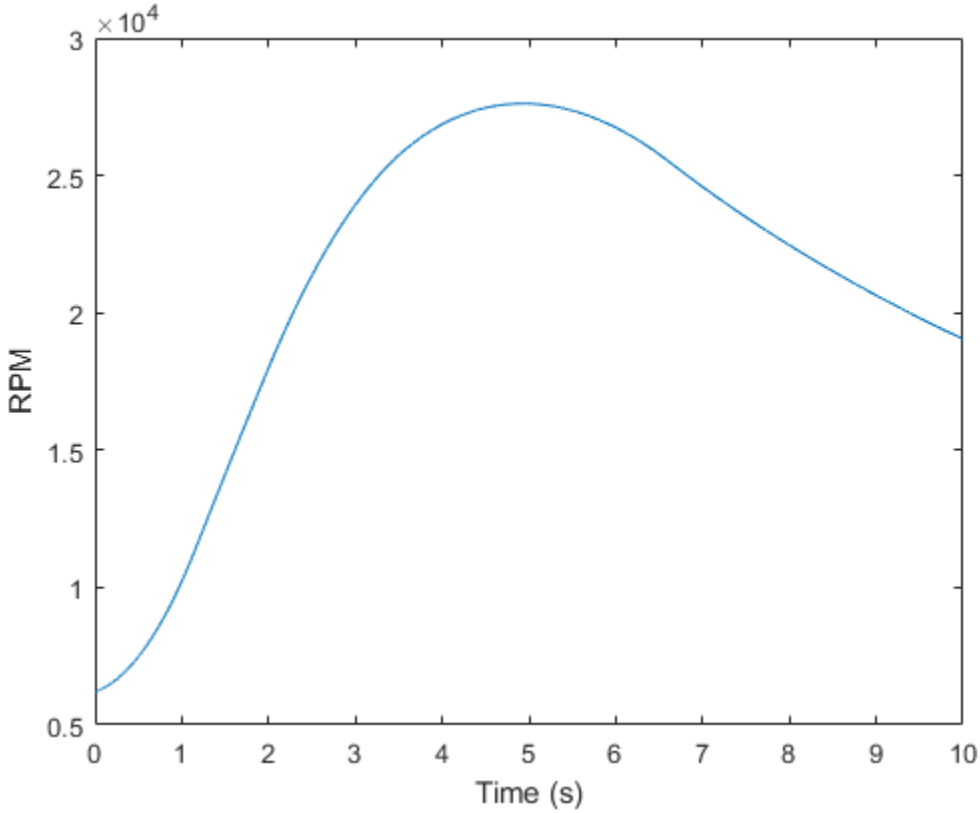
Analyze simulated data from an accelerometer placed in the cockpit of a helicopter.

Load the helicopter data. The vibrational measurements, `vib`, are sampled at a rate of 500 Hz for 10 seconds. Inspection of the data reveals that it has a linear trend. Remove the trend to prevent it from degrading the quality of the frequency estimation.

```
load('helidata.mat')

vib = detrend(vib);
```

Plot the nonlinear RPM profile. The rotor runs up until it reaches a maximum rotational speed of about 27,600 revolutions per minute and then coasts down.

```
plot(t,rpm)
xlabel('Time (s)')
ylabel('RPM')
```

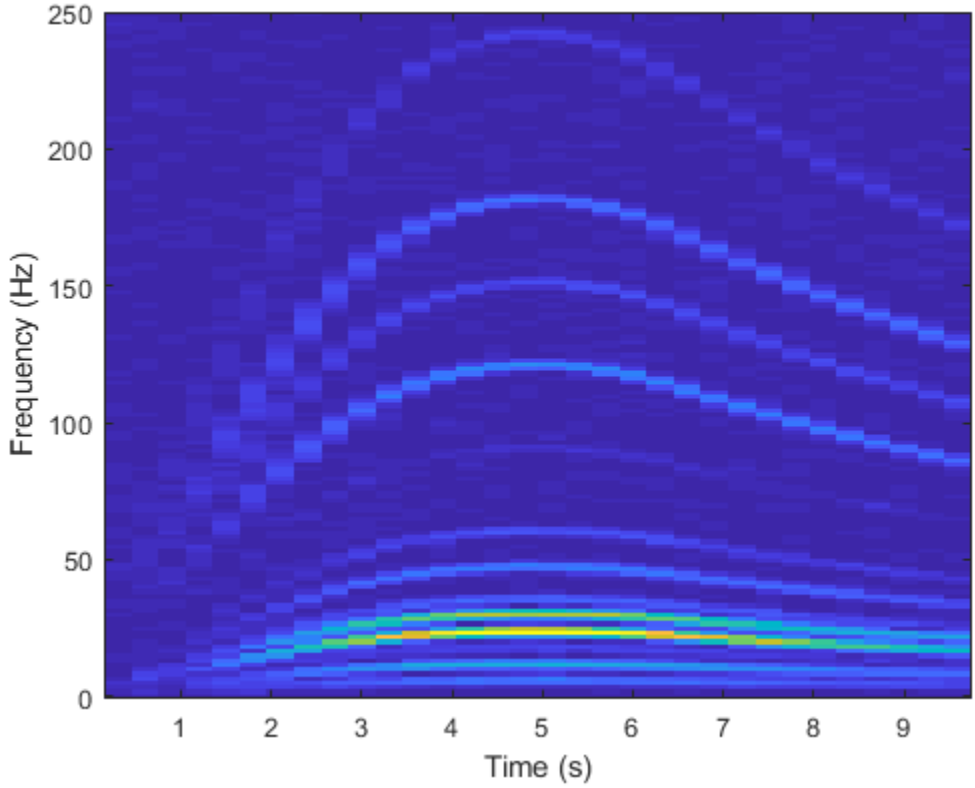


Compute the frequency-RPM map. Specify a resolution bandwidth of 2.5 Hz.

```
[map,freq,rpmOut,time] = rpmfreqmap(vib,fs,rpm,2.5);
```

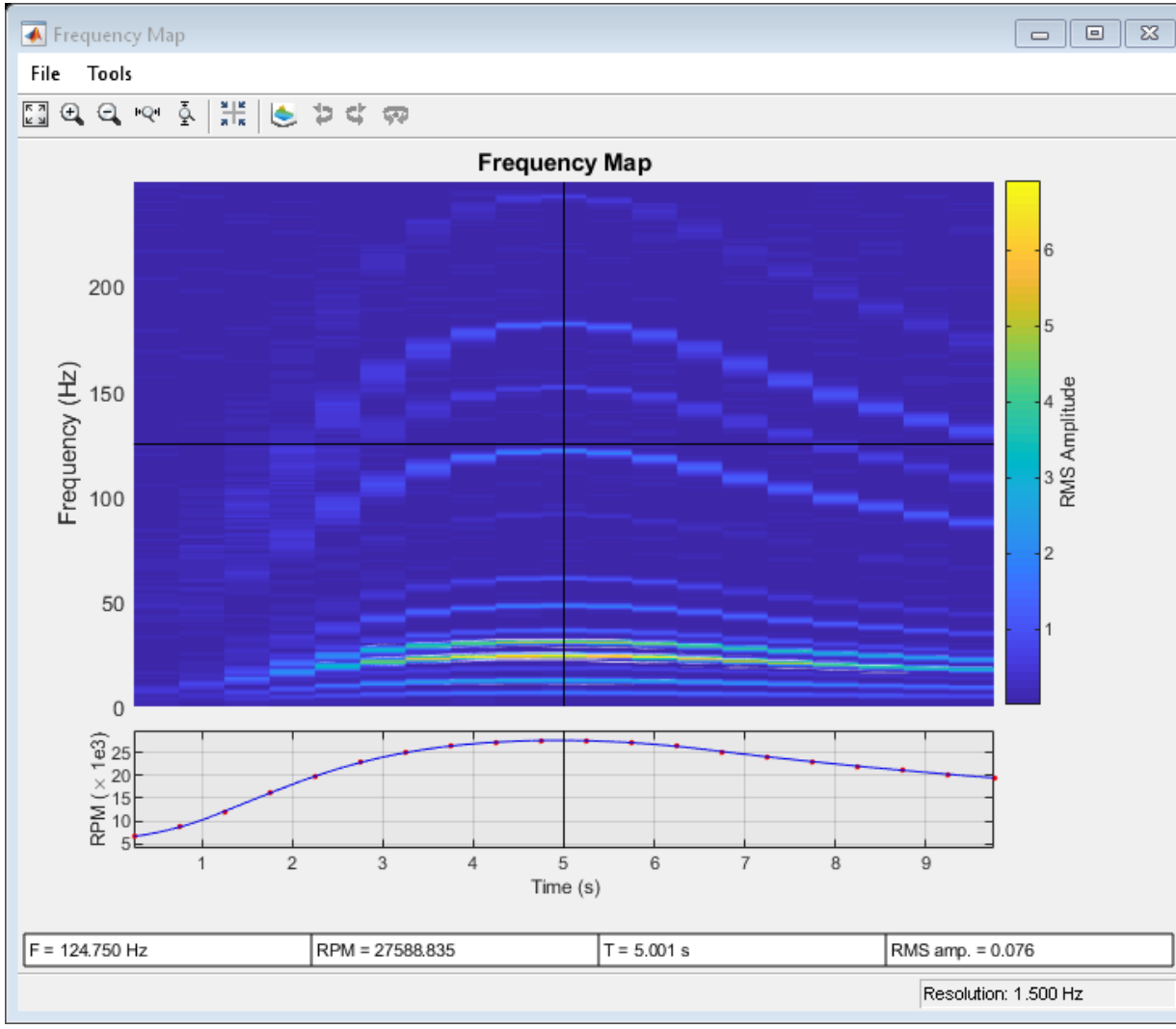
Visualize the map.

```
imagesc(time,freq,map)
ax = gca;
ax.YDir = 'normal';
xlabel('Time (s)')
ylabel('Frequency (Hz)')
```



Repeat the computation using a finer resolution bandwidth. Plot the map using the built-in functionality of `rpmfreqmap`. The gain in frequency resolution comes at the expense of time resolution.

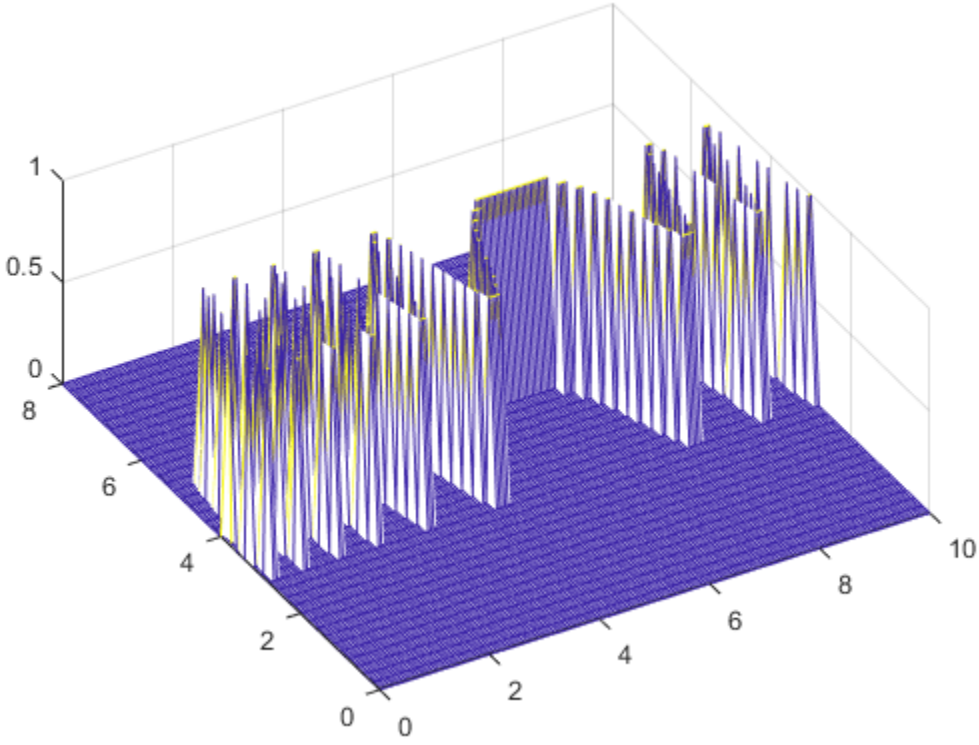
```
rpmfreqmap(vib,fs,rpm,1.5);
```


Frequency Map
File
Tools
Frequency Map
Frequency (Hz)
200
150
100
50
0
RMS Amplitude
6
5
4
3
2
1
RPM (× 1e3)
25
20
15
10
5
1
2
3
4
5
6
7
8
9
Time (s)
F = 124.750 Hz
RPM = 27588.835
T = 5.001 s
RMS amp. = 0.076
Resolution: 1.500 Hz

Find Ridge of Noisy Signal

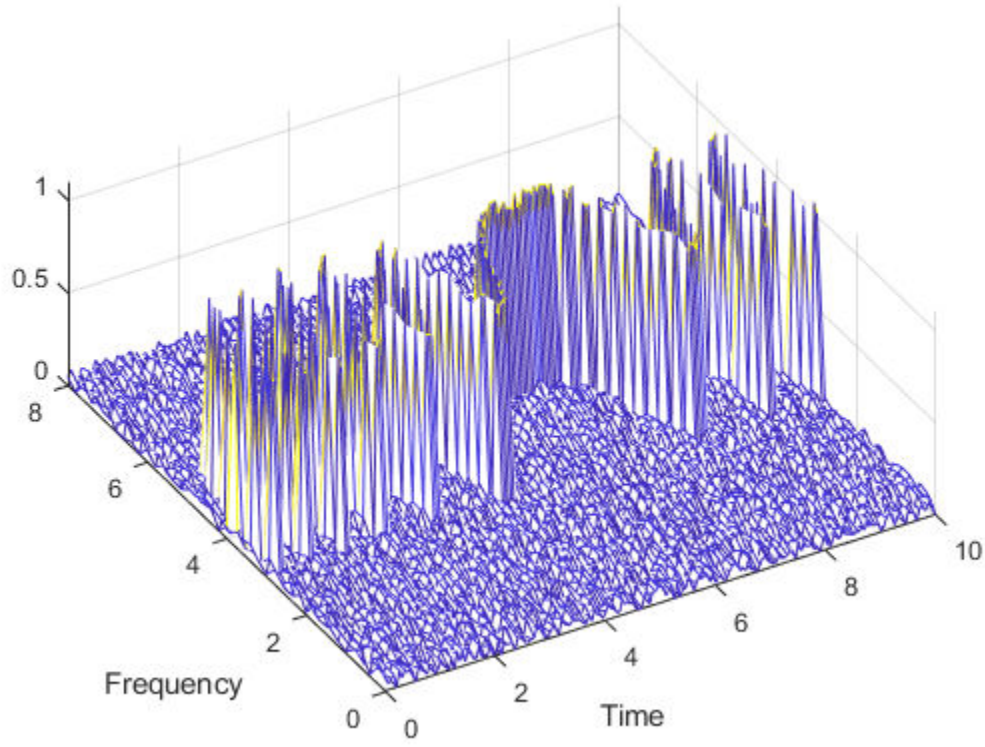
Create a matrix that resembles a time-frequency matrix with a sharp ridge. Visualize the matrix in three dimensions.

```
t = 0:0.05:10;
f = 0:0.2:8;
rv = 1;

[F,T] = ndgrid(f,t);

S = zeros(size(T));
S(abs((F-4)-cos((T-6).^2))<0.1) = rv;

mesh(T,F,S)
view(-30,60)
```



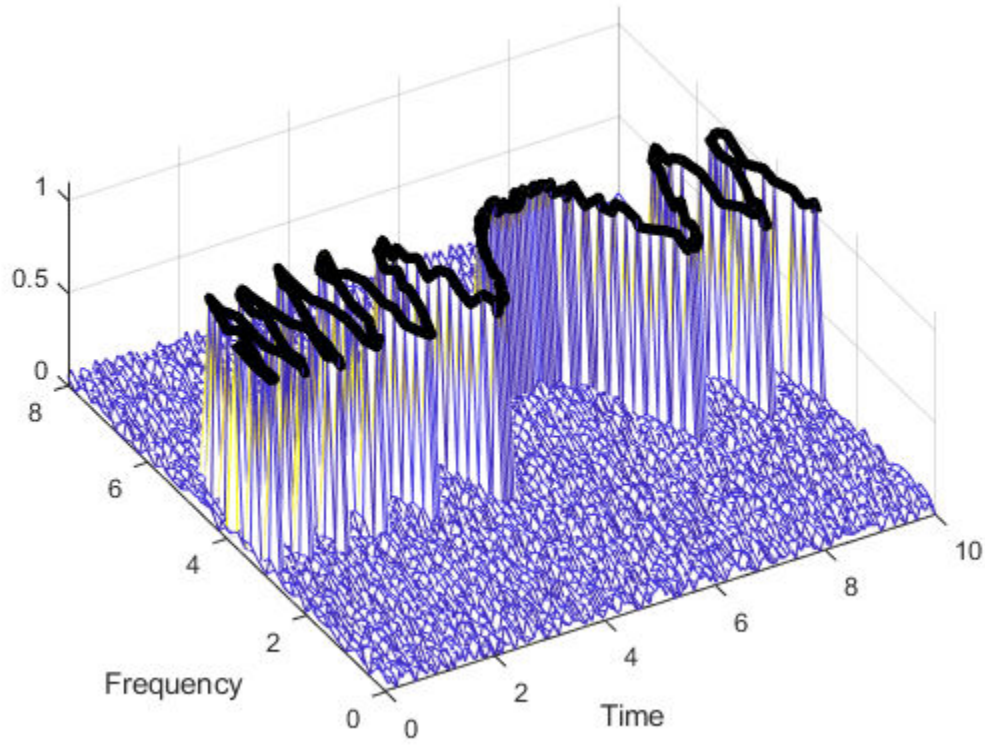
Add noise to the matrix and redisplay the plot.

```
S = S+rand(size(S))/10;

mesh(T,F,S)
view(-30,60)
xlabel('Time')
ylabel('Frequency')
```



Extract the ridge and plot the result.

```
[fridge,~,lridge] = tfridge(S,f);

rvals = S(lridge);

hold on
plot3(t,fridge,rvals,'k','linewidth',4)
hold off
```

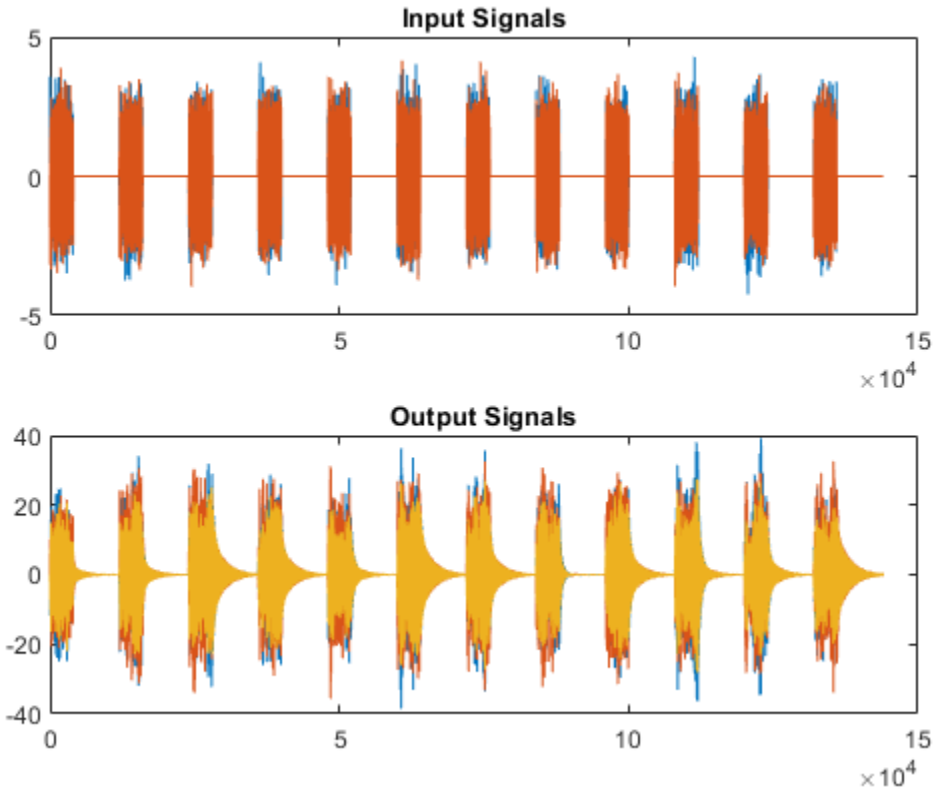

1
0.5
0
8
6
4
2
0
Frequency
0
2
4
6
8
10
Time

Modal Parameters of MIMO System

Compute the natural frequencies, the damping ratios, and the mode shapes for a two-input/three-output system excited by several bursts of random noise. Each burst lasts for 1 second, and there are 2 seconds between the end of each burst and the start of the next. The data are sampled at 4 kHz.

Load the data file. Plot the input signals and the output signals.

```
load modaldata

subplot(2,1,1)
plot(Xburst)
title('Input Signals')
subplot(2,1,2)
plot(Yburst)
title('Output Signals')
```

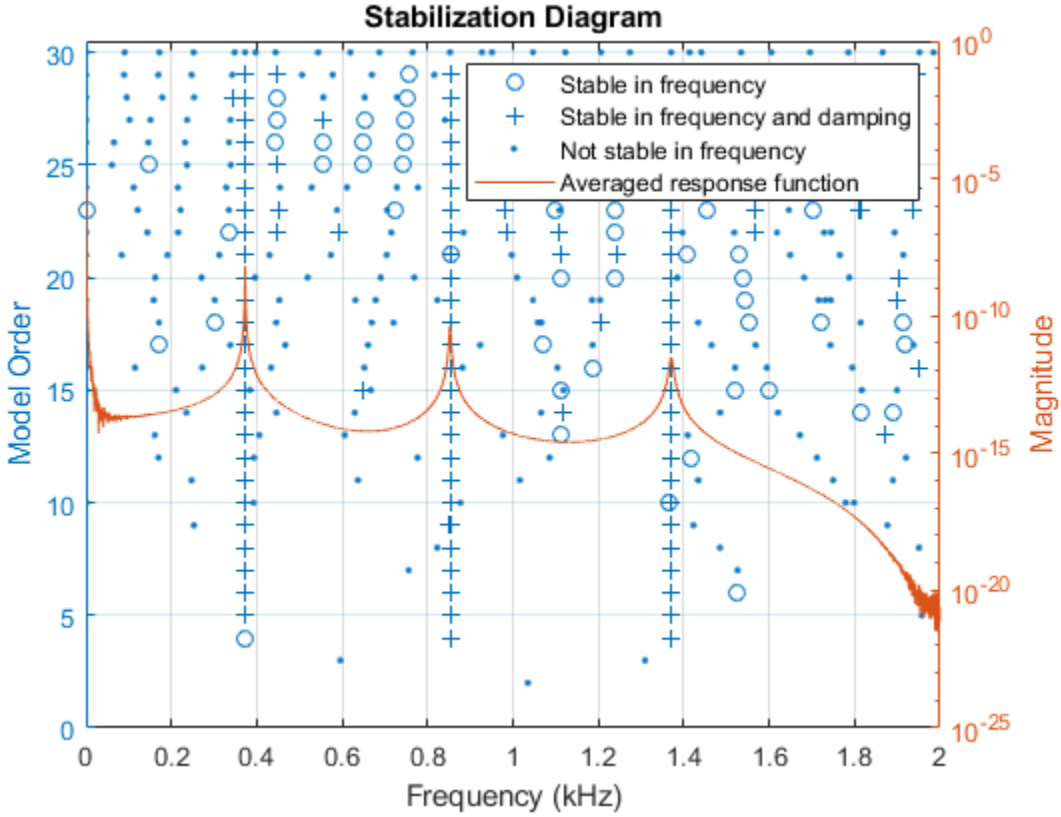


Compute the frequency-response functions. Specify a rectangular window with length equal to the burst period and no overlap between adjoining segments.

```
burstLen = 12000;
[frf,f] = modalfrf(Xburst,Yburst,fs,burstLen);
```

Visualize a stabilization diagram and return the stable natural frequencies. Specify a maximum model order of 30 modes.

```
figure
modalsd(frf,f,fs,'MaxModes',30);
```



Zoom in on the plot. The averaged response function has maxima at 373 Hz, 852 Hz, and 1371 Hz, which correspond to the physical frequencies of the system. Save the maxima to a variable.

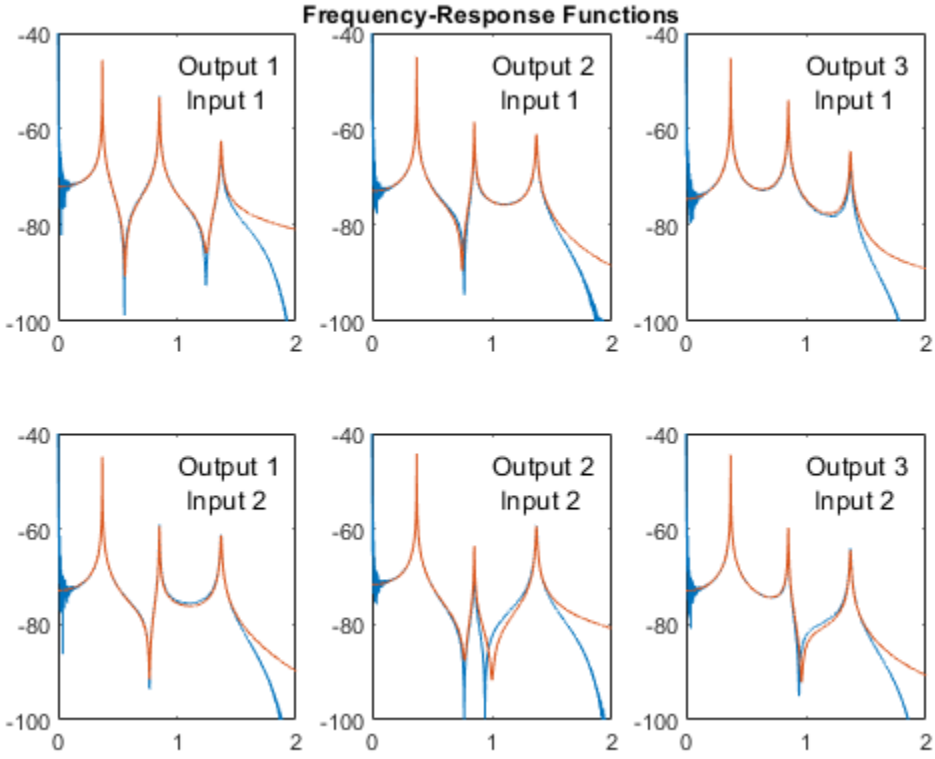
```
phfr = [373 852 1371];
```

Compute the modal parameters using the least-squares complex exponential (LSCE) algorithm. Specify a model order of 6 modes and specify physical frequencies for the 3 modes determined from the stabilization diagram. The function generates one set of natural frequencies and damping ratios for each input reference.

```
[fn,dr,ms,ofrf] = modalfit(frf,f,fs,6,'PhysFreq',phfr);
```

Plot the reconstructed frequency-response functions and compare them to the original ones.

```
for k = 1:2
    for m = 1:3
        subplot(2,3,m+3*(k-1))
        plot(f/1000,10*log10(abs(frf(:,m,k))))
        hold on
        plot(f/1000,10*log10(abs(ofrf(:,m,k))))
        hold off
        text(1,-50,[['Output ';' Input '] num2str([m k]')])
        ylim([-100 -40])
    end
end
subplot(2,3,2)
title('Frequency-Response Functions')
```


Frequency-Response Functions
Output 1
Input 1
Output 2
Input 1
Output 3
Input 1
Output 1
Input 2
Output 2
Input 2
Output 3
Input 2
-40
-60
-80
-100
0
1
2

Compute and Display Order-RPM Map

Generate a signal that consists of two linear chirps and a quadratic chirp, all sampled at 600 Hz for 5 seconds. The system that produces the signal increases its rotational speed from 10 to 40 revolutions per second during the testing period.

Generate the tachometer readings.

```
fs = 600;
t1 = 5;
t = 0:1/fs:t1;
f0 = 10;
f1 = 40;
rpm = 60*linspace(f0,f1,length(t));
```

The linear chirps have orders 1 and 2.5. The component with order 1 has twice the amplitude of the other. The quadratic chirp starts at order 6 and returns to this order at the end of the measurement. Its amplitude is 0.8. Create the signal using this information.

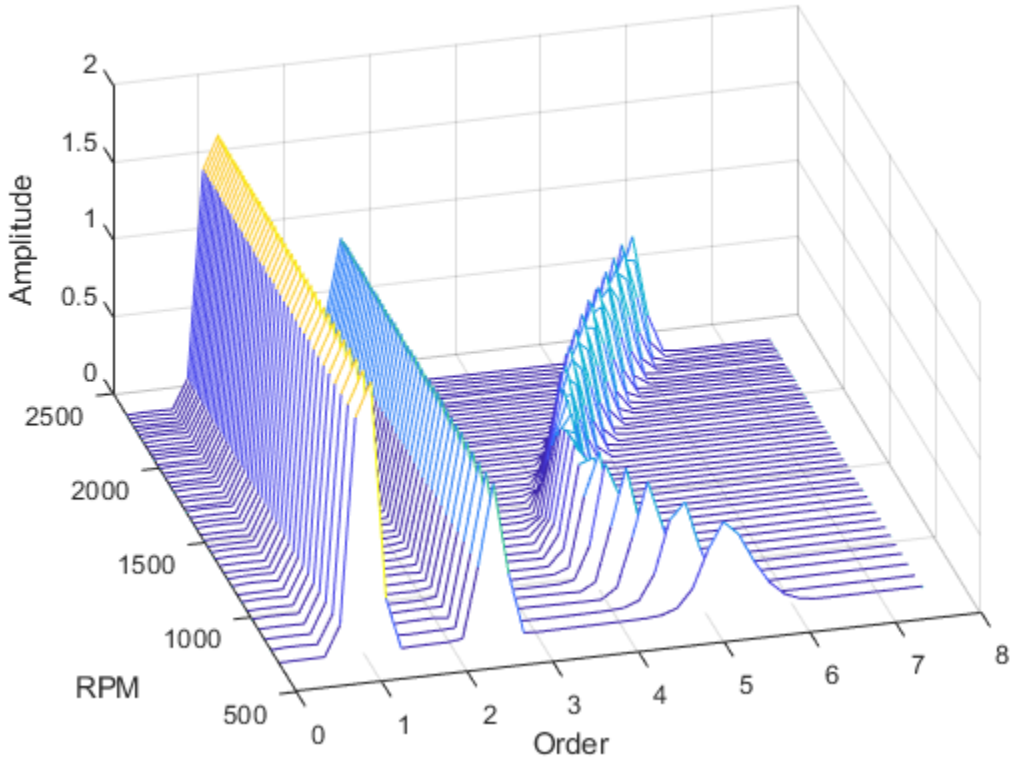
```
o1 = 1;
o2 = 2.5;
o6 = 6;
x = 2*chirp(t,o1*f0,t1,o1*f1)+chirp(t,o2*f0,t1,o2*f1) + ...
    0.8*chirp(t,o6*f0,t1,o6*f1,'quadratic');
```

Compute the order-RPM map of the signal. Use the peak amplitude at each measurement cell. Specify a resolution of 0.25 orders. Window the data with a Chebyshev window whose sidelobe attenuation is 80 dB.

```
[map,or,rp] = rpmordermap(x,fs,rpm,0.25, ...
    'Amplitude','peak','Window',{'chebwin',50});
```

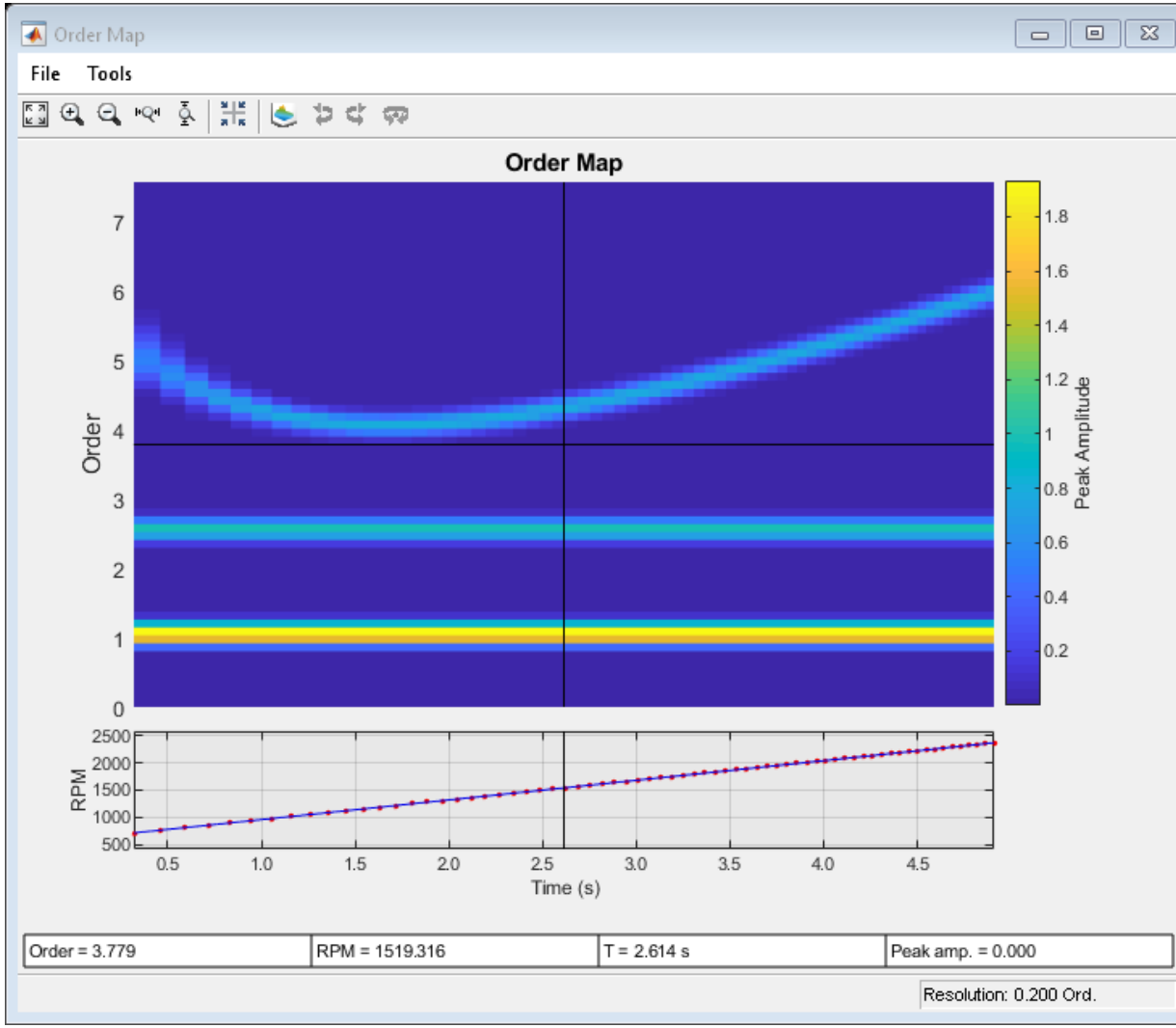
Draw the order-RPM map as a waterfall plot.

```
[OR,RP] = meshgrid(or,rp);
waterfall(OR,RP,map')
view(-15,45)
xlabel('Order')
ylabel('RPM')
zlabel('Amplitude')
```



Use the built-in functionality of `rpmordermap` to display the map. Specify a resolution of 0.2 orders and 80% overlap between adjoining segments.

```
rpmordermap(x,fs,rpm,0.2, ...
    'Amplitude','peak','OverlapPercent',80,'Window',{'chebwin',80})
```



See Also

`orderspectrum` | `ordertrack` | `orderwaveform` | `rpmfreqmap` | `rpmordermap` | `tachorpm`

MIMO Stabilization Diagram

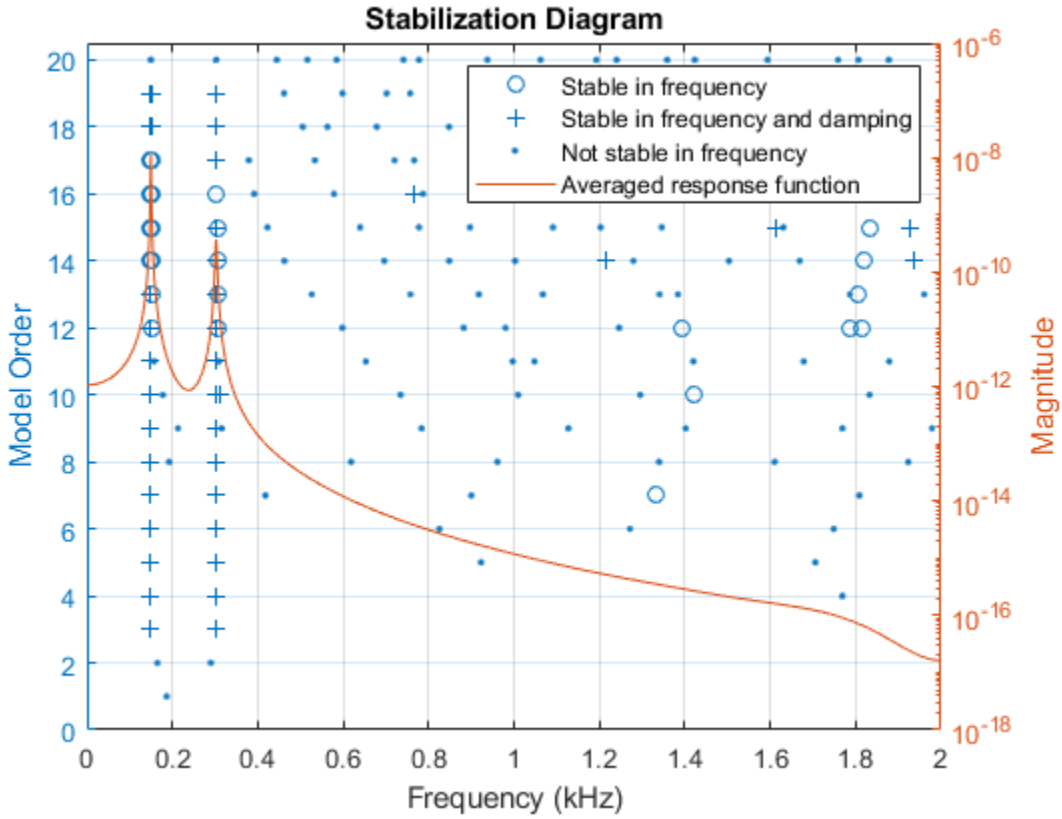
Compute the frequency-response functions for a two-input/two-output system excited by random noise.

Load the data file. Compute the frequency-response functions using a 5000-sample Hann window and 50% overlap between adjoining data segments. Specify that the output measurements are displacements.

```
load modaldata
winlen = 5000;

[frf,f] = modalfrf(Xrand,Yrand,fs,hann(winlen),0.5*winlen,'Sensor','dis');
```

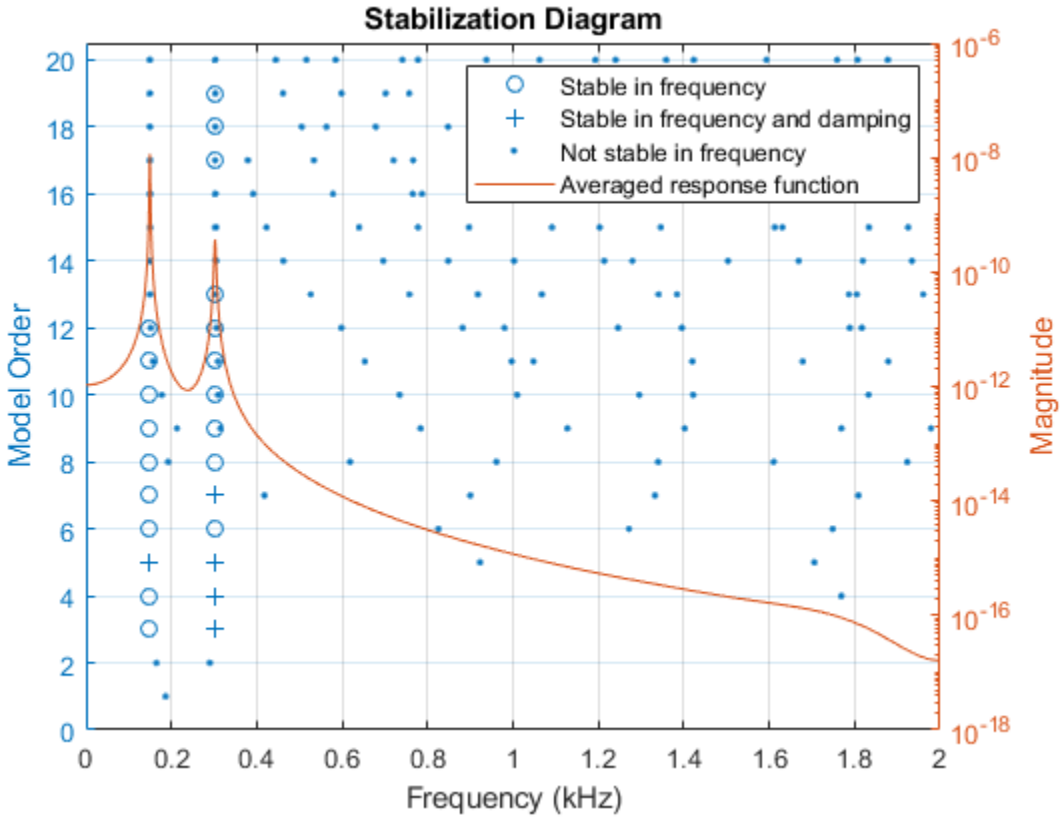
Generate a stabilization diagram to identify up to 20 physical modes.

```
modalsd(frf,f,fs,'MaxModes',20)
```



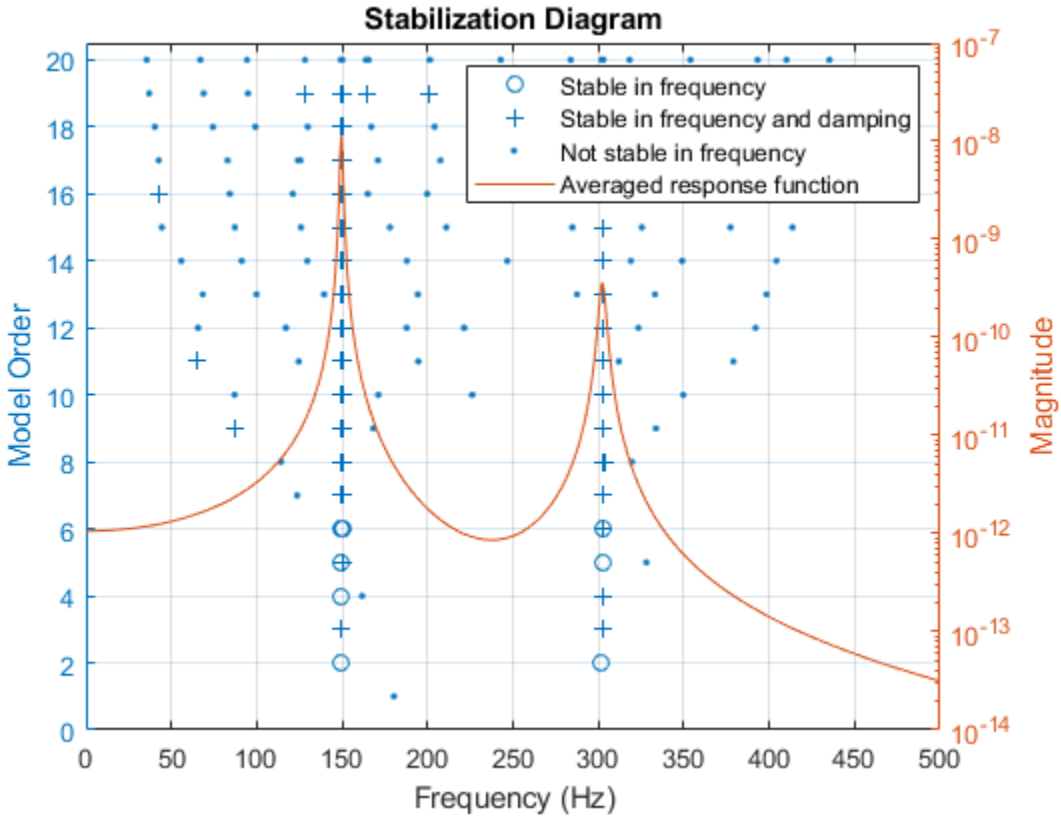
Repeat the computation, but now tighten the criteria for stability. Classify a given pole as stable in frequency if its natural frequency changes by less than 0.01% as the model order increases. Classify a given pole as stable in damping if the damping ratio estimate changes by less than 0.2% as the model order increases.

```
modalsd(frf,f,fs,'MaxModes',20,'SCriteria',[1e-4 0.002])
```



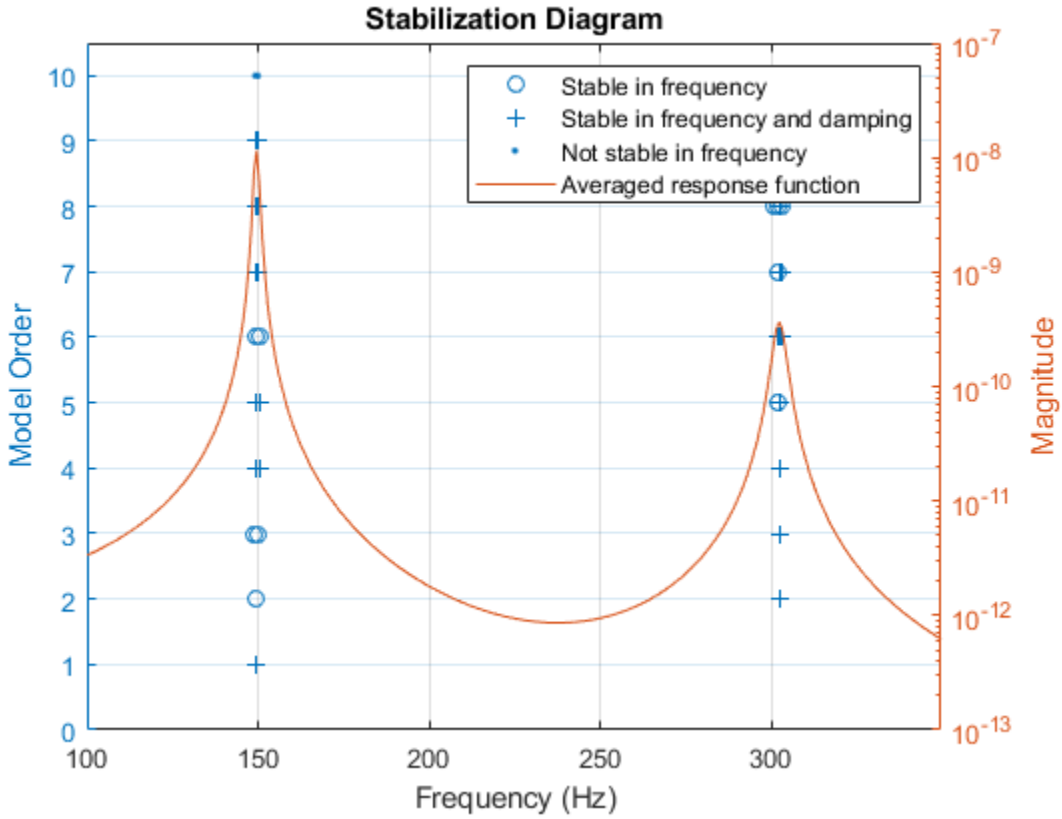
Restrict the frequency range to between 0 and 500 Hz. Relax the stability criteria to 0.5% for frequency and 10% for damping.

```
modalsd(frf,f,fs,'MaxModes',20,'SCriteria',[5e-3 0.1],'FreqRange',[0 500])
```



Repeat the computation using the least-squares rational function algorithm. Restrict the frequency range from 100 Hz to 350 Hz and identify up to 10 physical modes.

```
modalsd(frf,f,fs,'MaxModes',10,'FreqRange',[100 350],'FitMethod','lsrf')
```



See Also

`modalfit` | `modalfrf` | `modalsd`

Related Examples

- “Order Analysis of a Vibration Signal”

Modal Analysis of Identified Models

Identify state-space models of systems. Use the models to compute frequency-response functions and modal parameters. This example requires a System Identification Toolbox™ license.

Hammer Excitation

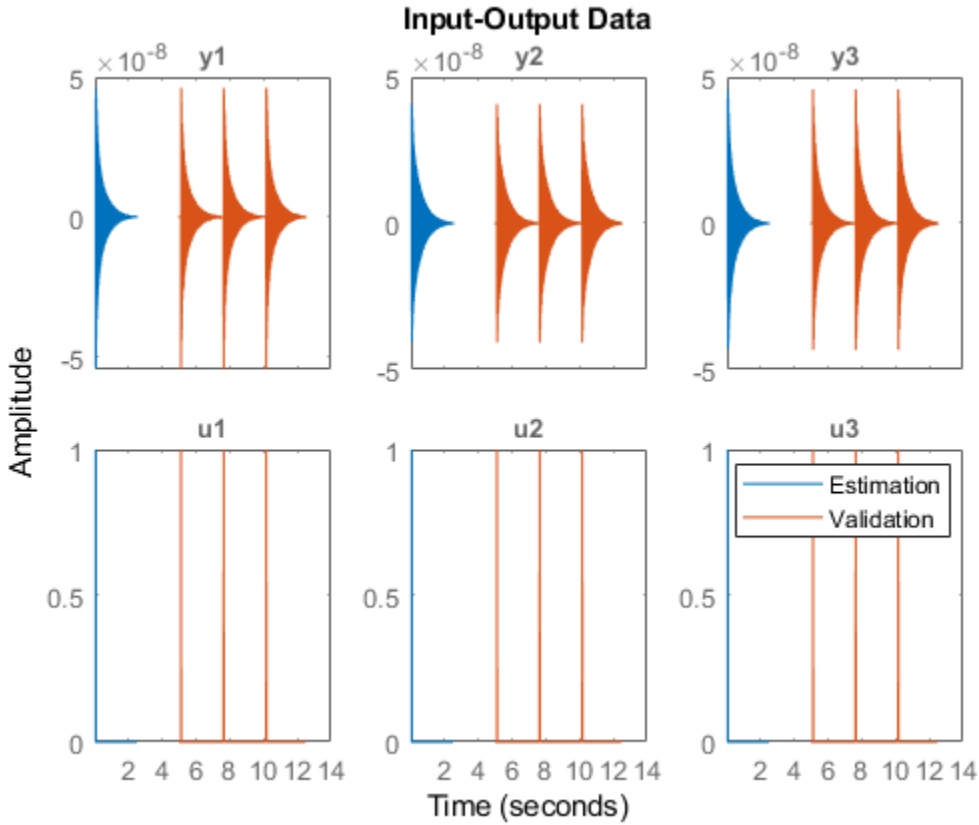
Load a file containing three-input/three-output hammer excitation data sampled at 4 kHz.

Use the first 10^4 samples for estimation and samples 2×10^4 to 5×10^4 for model quality validation. Specify the sample time as the inverse of the sample rate. Store the data as `@iddata` objects.

```
load modaldata XhammerMISO1 YhammerMISO1 fs

rest = 1:1e4;
rval = 2e4:5e4;
Ts = 1/fs;

Estimation = iddata(YhammerMISO1(rest,:),XhammerMISO1(rest,:),Ts);
Validation = iddata(YhammerMISO1(rval,:),XhammerMISO1(rval,:),Ts,'Tstart',rval(1)*Ts);
```

Plot the estimation data and the validation data.

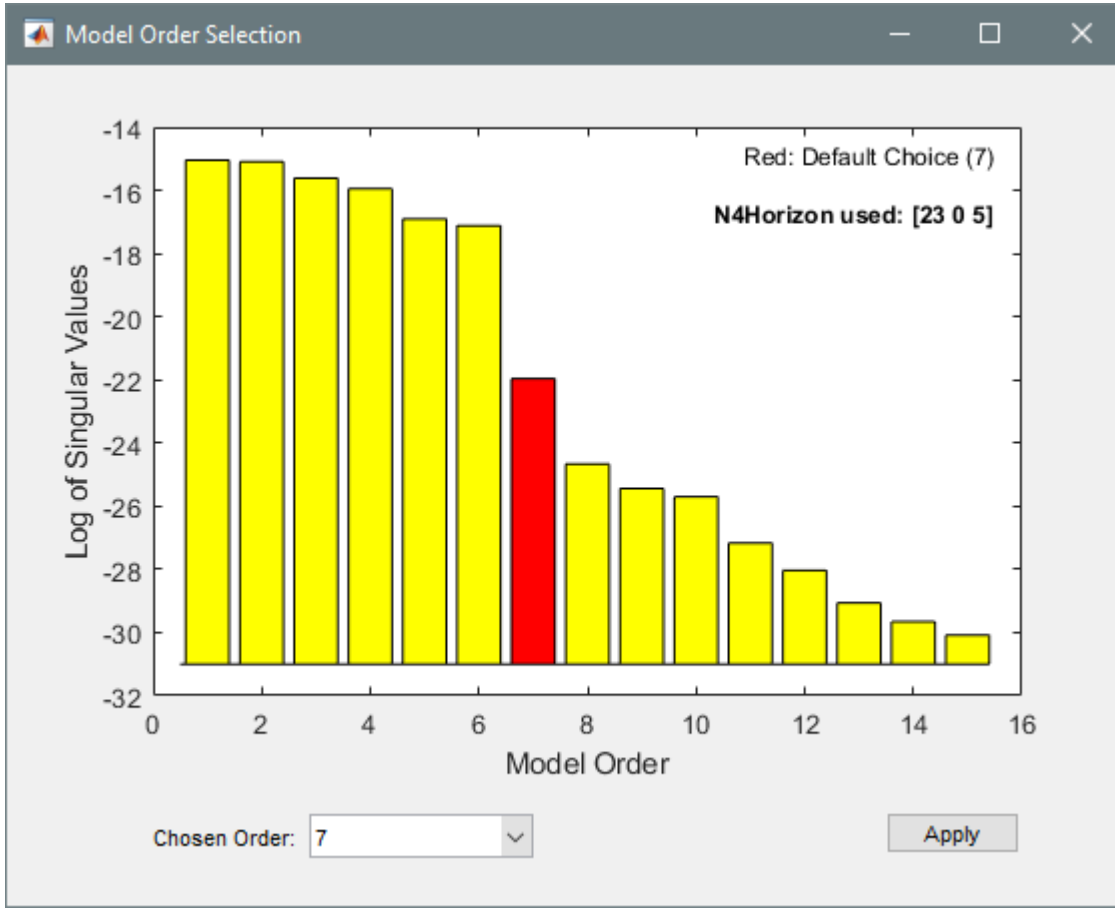
```
plot(Estimation,Validation)
legend(gca,'show')
```



Use the `ssest` function to estimate a 7th-order state-space model of the system that minimizes the simulation error between the measured outputs and the model outputs. Specify that the state-space model has feedthrough.

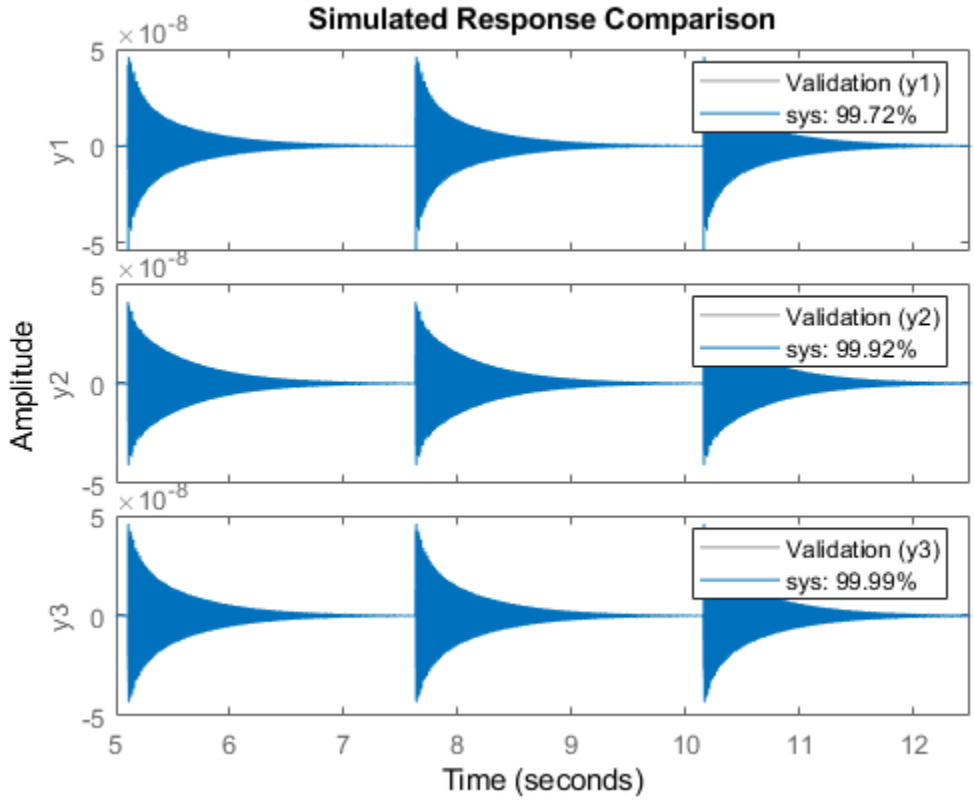
```
Orders = 7;
opt = ssestOptions('Focus','simulation');

sys = ssest(Estimation,Orders,'Feedthrough',true,'Ts',Ts,opt);
```

(To find the model order that gives the best tradeoff between accuracy and complexity, set `Orders` to `1:15` in the previous code. `ssest` outputs a log plot of singular values that lets you specify the order interactively. The function also recommends a model order of 7.)



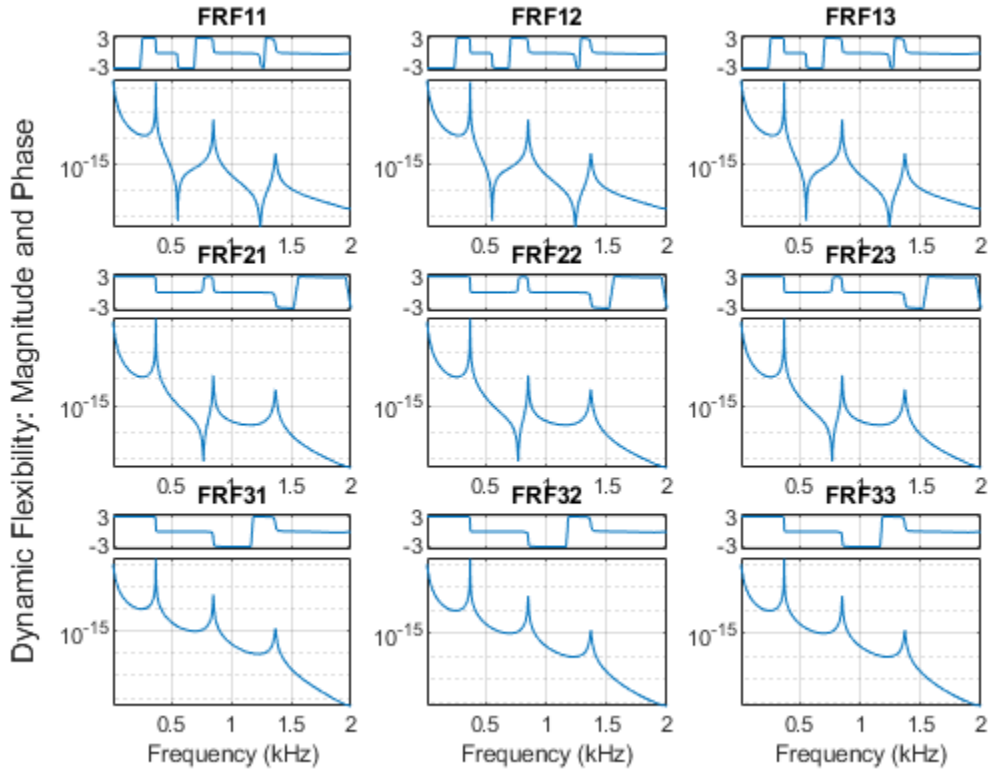
Validate the model quality on the validation dataset. Plot the normalized root mean square error (NRMSE) measure of goodness-of-fit. The model describes accurately the output signals of the validation data.

```
compare(Validation,sys)
```



Estimate the frequency-response functions of the model. Display the functions using `modalfrf` without output arguments.

```
[frf,f] = modalfrf(sys);
modalfrf(sys)
```



Assume that the system is well described using three modes. Compute the natural frequencies, damping ratios, and mode-shape vectors of the three modes.

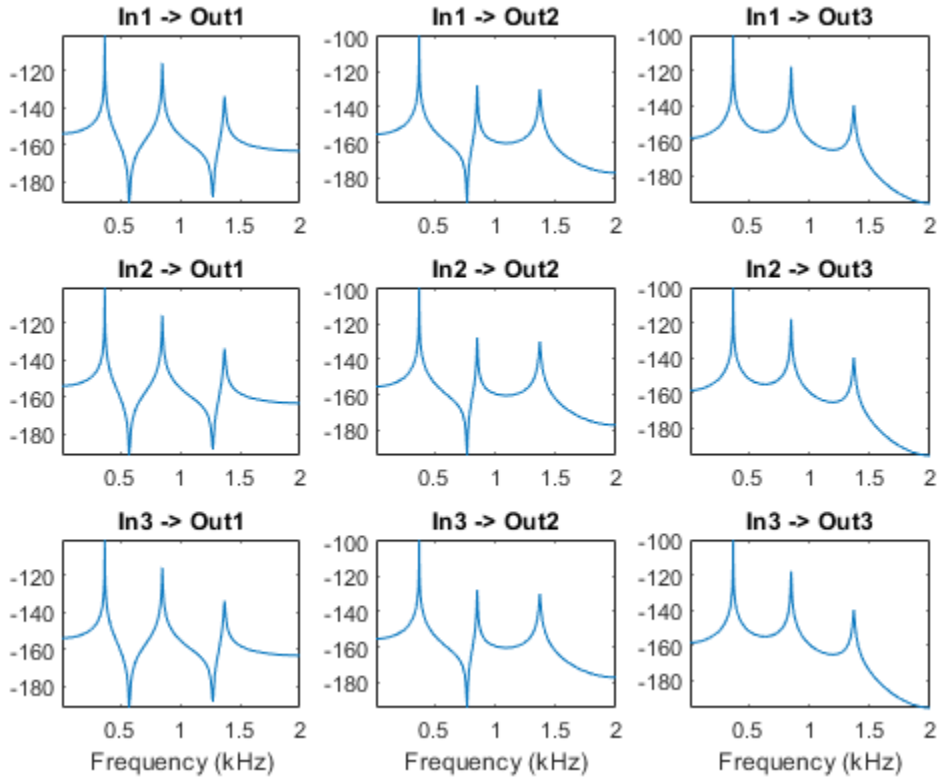
```
Modes = 3;
[fn,dr,ms] = modalfit(sys,f,Modes)
```

```
fn = 3×1
10^3 ×

    0.3727
    0.8525
    1.3706
```

```
dr = 3×1

    0.0008
    0.0018
    0.0029


ms = 3×3 complex

   0.0036 - 0.0019i   0.0039 - 0.0005i   0.0021 + 0.0006i
   0.0043 - 0.0023i   0.0010 - 0.0001i  -0.0033 - 0.0010i
   0.0040 - 0.0021i  -0.0031 + 0.0004i   0.0011 + 0.0003i
```

Compute and display the reconstructed frequency-response functions. Express the magnitudes in decibels.

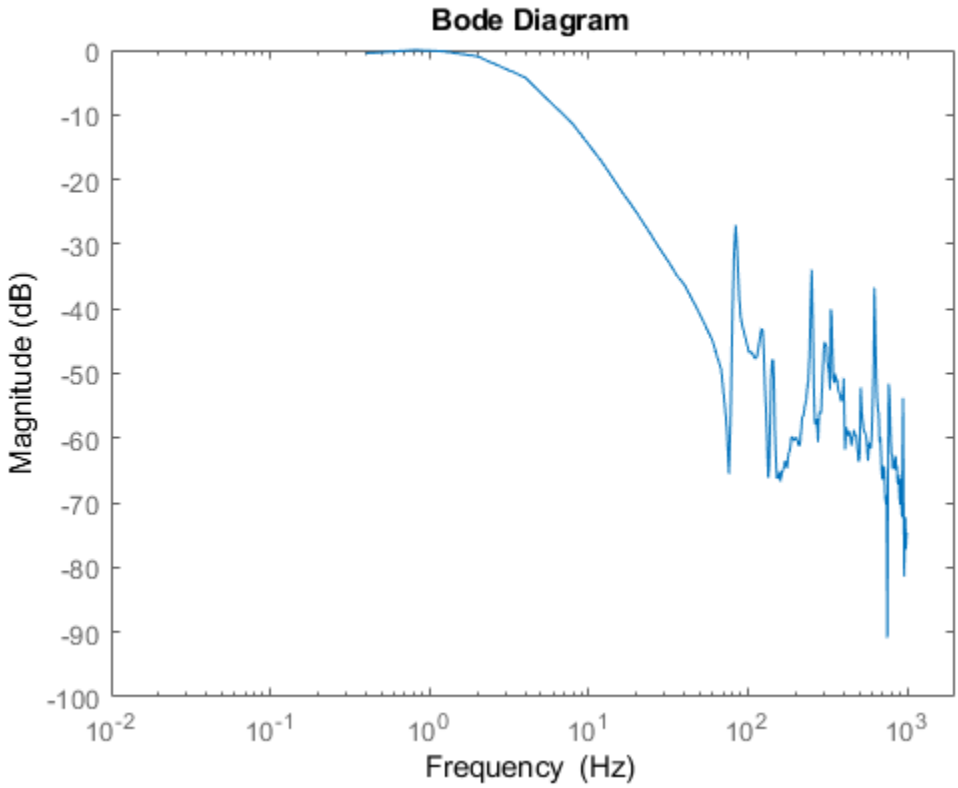
```
[~,~,~,ofrf] = modalfit(sys,f,Modes);

clf
for ij = 1:3
    for ji = 1:3
        subplot(3,3,3*(ij-1)+ji)
        plot(f/1000,20*log10(abs(ofrf(:,ji,ij))))
        axis tight
        title(sprintf('In%d -> Out%d',ij,ji))
        if ij==3
            xlabel('Frequency (kHz)')
        end
    end
end
```



Controlled Unstable Process

Load a file containing a high modal density frequency-response measurement. The data corresponds to an unstable process maintained at equilibrium using feedback control. Store the data as an `idfrd` object for identification. Plot the Bode diagram.

```
load HighModalDensData FRF f

G = idfrd(permute(FRF,[2 3 1]),f,0,'FrequencyUnit','Hz');
figure
bodemag(G)
xlim([0.01,2e3])
```

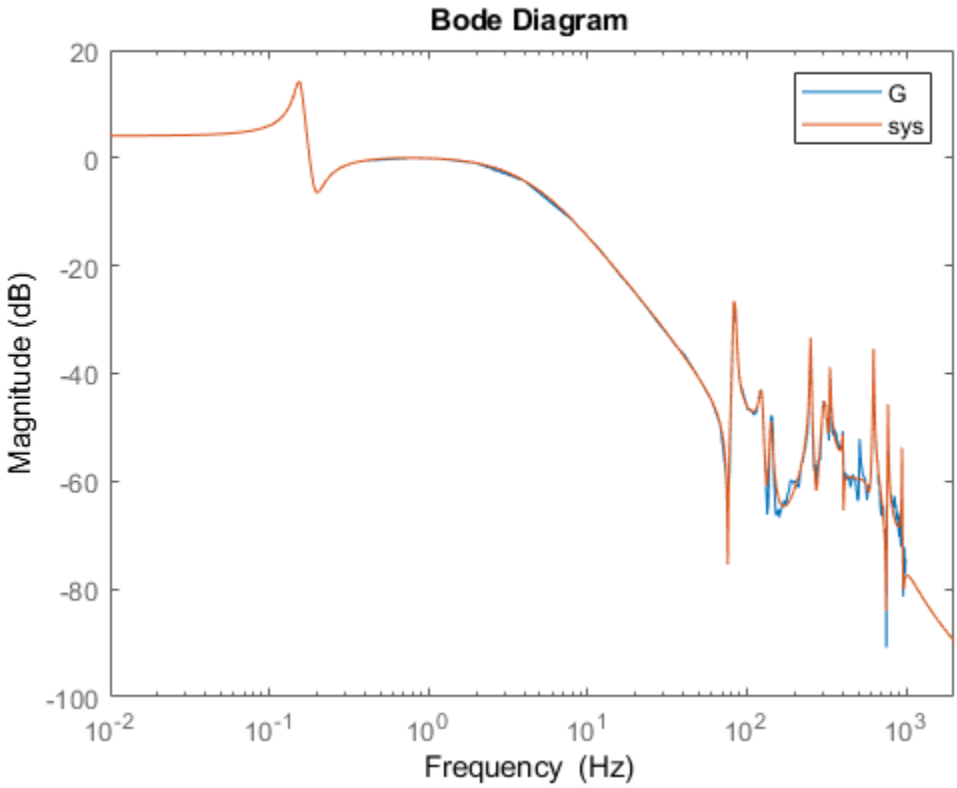


Identify a transfer function with 32 poles and 32 zeros.

```
sys = tfest(G,32,32);
```

Compare the frequency response of the model with the measured response.

```
bodemag(G,sys)
xlim([0.01,2e3])
legend(gca,'show')
```



Extract the natural frequencies and damping ratios of the first 10 least-damped oscillatory modes. Store the results in a table.

```
[fn,dr] = modalfit(sys,[],10);
T = table((1:10)',fn,dr,'VariableNames',{'Mode','Frequency','Damping'})
```

```
T=10×3 table
    Mode    Frequency     Damping
    ____    _________    _________

      1       82.764      0.011304
      2       85.013      0.015632
      3       124.04      0.025252
      4       142.04      0.017687
```

```
     5      251.46      0.0062182
     6      332.79      0.0058266
     7      401.21      0.0043645
     8      625.14      0.0039247
     9      770.49       0.002795
    10      943.64      0.0019943
```

See Also

`modalfit` | `modalfrf` | `ssest`

Signal Analyzer App

- “Using Signal Analyzer App” on page 19-2
- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Resolve Tones by Varying Window Leakage” on page 19-42
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Using Signal Analyzer App

The **Signal Analyzer** app is an interactive tool for visualizing, measuring, analyzing, and comparing signals in the time domain, in the frequency domain, and in the time-frequency domain. The app provides a way to work with many signals of varying durations at the same time and in the same view.

Start the app by choosing it from the **Apps** tab on the MATLAB toolstrip. You can also start the app by typing `signalAnalyzer` at the MATLAB command prompt.

A typical workflow for inspecting and comparing signals using the **Signal Analyzer** app is:

1. Select Signals to Analyze on page 19-4 — Select any signal available in the MATLAB workspace. The app accepts real numeric arrays and signals with inherent

time information, such as MATLAB timetables and `timeseries` objects. See “Data Types Supported by Signal Analyzer” on page 19-99 for more information.

2. “Preprocess Signals” on page 19-7 — Lowpass, highpass, bandpass, or bandstop filter signals. Smooth signals using moving averages, regression, Savitzky-Golay filters, or other methods. Change sample rates of signals or interpolate nonuniformly sampled signals onto uniform grids. Preprocess signals using your own custom functions. Generate MATLAB functions to automate preprocessing operations.

   - “Duplicate and Rename Signals” on page 19-8
   - “Filter Signals” on page 19-8
   - “Smooth Signals” on page 19-8
   - “Resample Signals” on page 19-8
   - “Add Custom Preprocessing Functions” on page 19-9
   - “Generate MATLAB Scripts and Functions” on page 19-20

3. Explore Signals on page 19-12 — Add time information to signals using sample rates, numeric vectors, `duration` arrays, or MATLAB expressions. Plot, measure, and compare data, their spectra, their spectrograms, or their scalograms. Look for features and patterns in the time domain, in the frequency domain, and in the time-frequency domain. Compute persistence spectra to analyze sporadic signals and sharpen spectrogram estimates using reassignment. Extract regions of interest from signals.

   - “Plot Signals” on page 19-12
   - “Visualize Signal Spectra” on page 19-12
   - “Visualize Persistence Spectra” on page 19-13
   - “Visualize Signal Spectrograms” on page 19-14
   - “Visualize Signal Scalograms” on page 19-15
   - “Edit Time Information and Link Displays in Time” on page 19-16
   - “Measure Signal, Spectrum, and Time-Frequency Data” on page 19-18
   - “Extract Signal Regions of Interest” on page 19-18

4. Share Analysis on page 19-19 — Copy displays from the app to the clipboard as images. Export signals to the MATLAB workspace or save them to MAT-files. Generate MATLAB scripts to automate the computation of power spectrum, spectrogram, or persistence spectrum estimates and the extraction of regions of interest. Save **Signal Analyzer** sessions to resume your analysis later or on another machine.

- “Copy Displays” on page 19-19
- “Export Signals” on page 19-19
- “Generate MATLAB Scripts and Functions” on page 19-20
- Save and Load **Signal Analyzer** Sessions on page 19-21

Select Signals to Analyze

The **Signal Analyzer** app works with vectors, matrices, MATLAB timetables, or `timeseries` objects in the MATLAB workspace. When you start the app, all usable signals in the workspace appear in the Workspace browser at the bottom-left corner. See “Data Types Supported by Signal Analyzer” on page 19-99 for more information.

Select Signals from the Workspace Browser

Select signals from the Workspace browser by clicking their names and dragging them to the Signal table at the top-left corner. To plot a signal, drag it to a display. If you select the check box next to the name of a signal in the Signal table, the signal is plotted in the selected display. You can also drag signals directly from the Workspace browser to a display. The dragged signals are plotted in the display and listed in the Signal table.

Each column of a matrix is treated as an independent signal. For example, a 100-by-3 matrix called `sgn` appears in the Signal table as three 100-sample signals, `sgn(:,1)`, `sgn(:,2)`, and `sgn(:,3)`.

Note **Signal Analyzer** does not support matrices, time series, or timetables with more than 800 channels.

There are two different ways to choose signals in the Signal table. Each way gives you access to a different set of operations.

- Selecting the signal by clicking the **Name**, **Info**, **Time**, or **Start Time** column in the Signal table enables you to perform all the operations in the **Analyzer** tab. You can change the time information and smooth, filter, or duplicate the signals. It is not necessary to plot a signal to preprocess it.
- Selecting the check box to the left of the signal name plots the signal in the currently selected display and enables you to perform all the operations in the **Display** tab. You can display the signal in the frequency domain or the time-frequency domain or you can measure the signal using cursors.

Note If you attempt to import signals with more than 100 columns, the app displays a warning. The matrix you are trying to import might be the transpose of a multichannel signal that you want to analyze. In that case, click **No** in the warning dialog box and transpose the matrix in the workspace. If you do want to import the columns as separate signals, click **Yes**. If you drag the matrix to a display and click **Yes** in the warning dialog box, then the app plots only the first 10 columns of the matrix but imports all the columns. To plot signal columns beyond the 10th, drag them to the display. Alternatively, in the Signal table, select the check boxes next to the names of the signals you want to plot.

If you modify a signal in the MATLAB workspace, the Workspace browser updates automatically. However, the app does not recognize the changes until you reimport the signal by dragging it again to the Signal table or to a display.

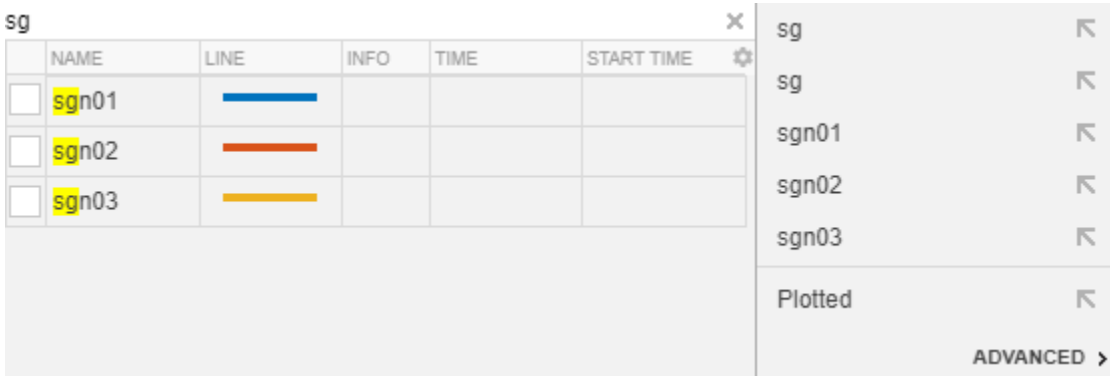
If you add or remove matrix columns, the app deletes the signals, clears any plots of them, and creates new signals with the modified matrix dimensions.

Filter Signals in the Signal Table

To help search through a large amount of data in the Signal table, you can filter signals. The filter criteria can be any text that is contained in the signal name or in other columns.

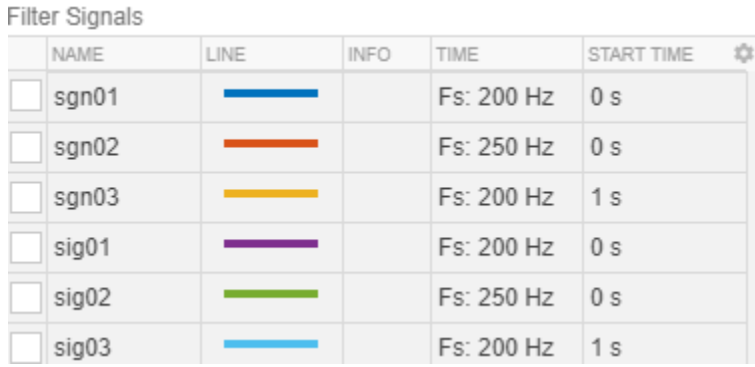
- To show signals with a given name, enter a search phrase into the **Filter Signals** text box. The matches are highlighted in the filter results.

  Suppose that you have three *sig* signals, `sig01`, `sig02`, and `sig03`, and three *sgn* signals, `sgn01`, `sgn02`, and `sgn03`. You can enter `sg` to show the three *sgn* signals, or enter `2` to show `sig02` and `sgn02`.



- You can also filter signals according to their time information. To access this functionality, click inside the search results box, and then click **Advanced**. For details on entering time information, see Edit Sample Rate and Other Time Information on page 19-93.

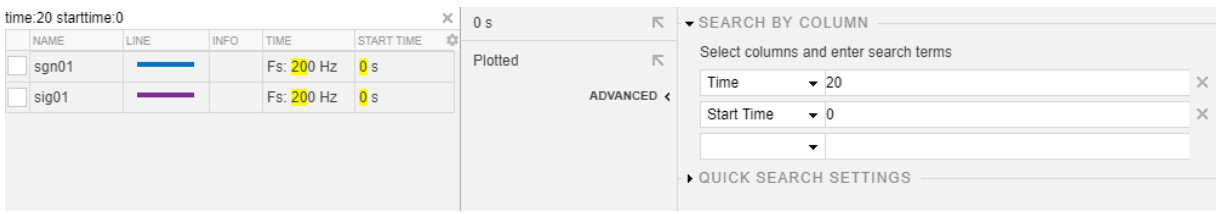
  Suppose that you have six signals with these sample times and start times:



  The **Advanced** menu lets you search signals by `Name`, `Samples`, `Start Time`, or `Time` in terms of sample rate or sample time.

  If you select the `Time` option and enter `20`, the app finds the four signals sampled at 200 Hz. In the second text box, if you select the `Start Time` option and enter `0`, the app finds `sgn01` and `sig01`.

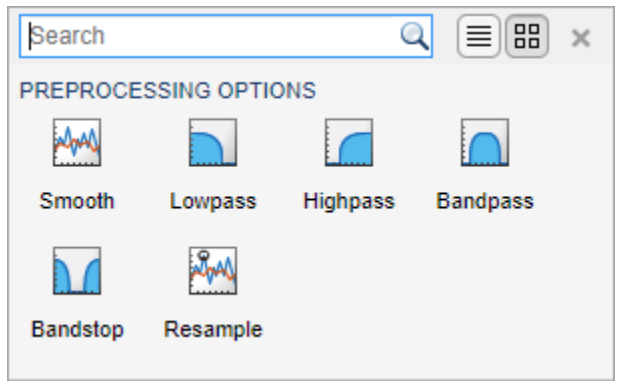
  Note The filter matches values as text, not numbers. For instance, if you choose the `Start Time` option and enter `00`, then the filter does not return any results.



- You can save and store a filter for future use. From the **Advanced** menu of the search results box, click **Quick Search Settings**. Enter a name in the **Save Search As** box, and click **Save**.

Preprocess Signals

You can use the **Signal Analyzer** app to perform several signal preprocessing tasks. The different processing options appear in the **Analyzer** tab:



Preprocessing operations, undo operations, and function generation apply to all signals currently selected in the Signal table. To select signals, click their **Name**, **Info**, **Time**, or **Start Time** column in the Signal table.

Note Preprocessing operations overwrite the signal on which they work. If you want to keep the original signal, duplicate it and operate on the duplicate.

You can perform smoothing and filtering actions any number of times and in any order.

The **Info** column in the Signal table includes an icon that indicates if any preprocessing has been performed on a signal. Clicking the icon enumerates the actions and the order in which they were performed. Preprocessing steps can be undone by clicking **Undo Preprocessing** on the **Analyzer** tab or on any tab arising from a preprocessing action. The steps are undone one at a time, starting with the most recent.

Tip To see a full summary of the preprocessing steps you took, including all settings you chose, click **Generate Function** on the **Analyzer** tab.

Duplicate and Rename Signals

Signal Analyzer enables you to duplicate and rename signals that you can then preprocess or export for further analysis.

To duplicate a signal, use the **Duplicate** button on the **Analyzer** tab or on any tab arising from a preprocessing action. Alternatively, right-click the signal in the Signal table and select **Duplicate**. The duplicate has the same name as the original signal with `_copy` appended.

To rename a signal, double-click the signal name in the Signal table and change the name. Alternatively, right-click the signal in the Signal table and select **Rename**.

Filter Signals

To filter one or more selected signals, on the **Analyzer** tab, click the **Lowpass**, **Highpass**, **Bandpass**, or **Bandstop** icon in the **Preprocessing** gallery. The app uses the `lowpass`, `highpass`, `bandpass`, and `bandstop` functions to perform the filtering. You can control the stopband attenuation, the passband frequencies, and the widths of the transition regions. See the function reference pages for additional information. Filtering does not support nonuniformly sampled signals.

Smooth Signals

To smooth one or more selected signals, on the **Analyzer** tab, click the **Smooth** icon in the **Preprocessing** gallery. The app uses the MATLAB function `smoothdata` to perform the smoothing. The following smoothing methods are available:

- Moving mean
- Moving median
- Gaussian
- Linear regression
- Quadratic regression
- Robust linear regression
- Robust quadratic regression
- Savitzky-Golay filtering

Resample Signals

To resample one or more selected signals, on the **Analyzer** tab, expand the **Preprocessing** gallery and click the **Resample** icon. **Signal Analyzer** uses the Signal

Processing Toolbox function `resample` to perform the resampling. The following options are available:

- When your signal is nonuniformly sampled, you can use the app to interpolate it onto a uniform grid. You can specify the interpolation method and the sample rate at which you want the signal to be sampled. The following interpolation methods are available:
  - Linear interpolation
  - Shape-preserving piecewise cubic interpolation
  - Cubic spline interpolation using not-a-knot end conditions

  See the `interp1` reference page for more information.
- When your signal is uniformly sampled, you can use the app to change its sample rate. You can specify either the desired sample rate or the factor by which you want to upsample or downsample the signal. In this case, the interpolation panel in the **Resample** tab is disabled because the interpolation operation does not make sense with uniformly sampled signals.

The resampling operation does not support signals in samples.

Add Custom Preprocessing Functions

To add a custom preprocessing function, on the **Analyzer** tab, click the arrow next to the **Preprocessing** gallery and then select **Add Custom Function**. The app prompts you to enter the function name and a brief description:

- If you have already written a preprocessing function, and the function is in the current folder or in the MATLAB path, the app incorporates it to the gallery. You can use tab completion to search for the function name.
- If you have not written the function yet, the app opens a blank template in the Editor.

Custom preprocessing functions have mandatory and optional arguments:

- The first input argument, `x`, is the input signal. This argument must be a vector and is treated as a single channel.
- The second input argument, `tIn`, is a vector of time values. The vector must have the same length as the signal. If the input signal has no time information, the function reads this argument as an empty array.
- Use `varargin` to specify additional input arguments. If you do not have additional input arguments, you can omit `varargin`. Enter the additional arguments as an ordered comma-separated list in the **Preprocess** tab.

- The first output argument, `y`, is the preprocessed signal.
- The second output argument, `tOut`, is a vector of output time values. If the input signal has no time information, `tOut` is returned as an empty array.
- To implement your algorithm, you can use any MATLAB or Signal Processing Toolbox function.

See “Declip Saturated Signals Using Your Own Function” on page 19-87 for more details.

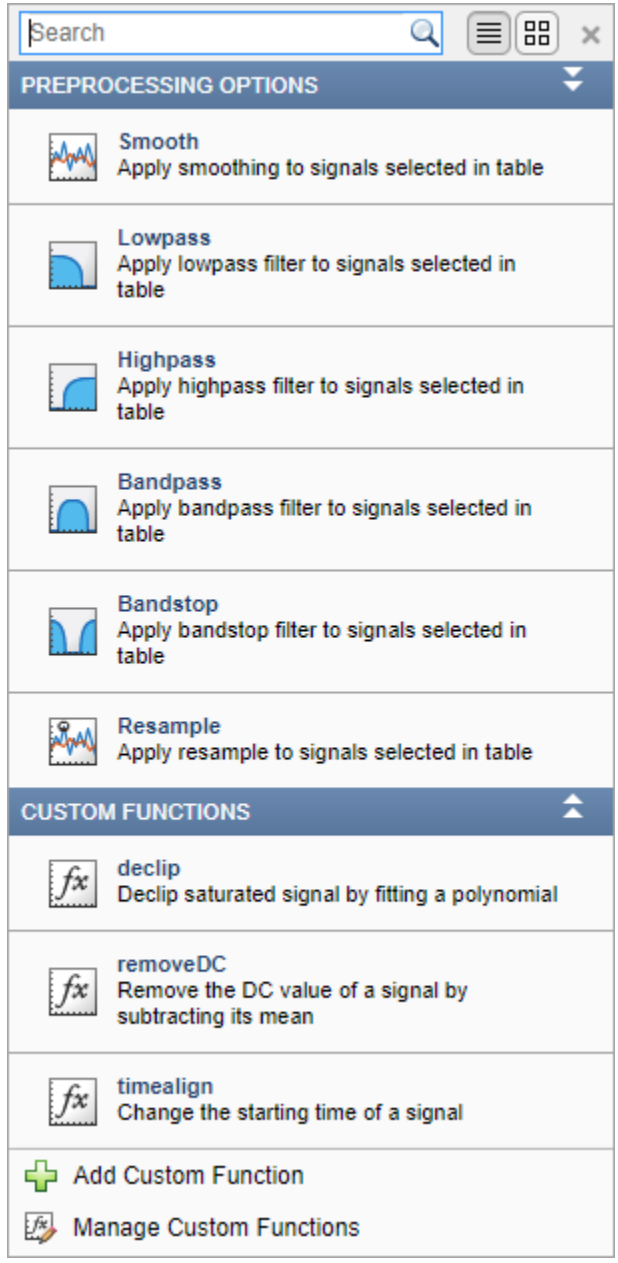
Example: The following function removes the DC value of a signal by subtracting its mean:

```
function [y,tOut] = removeDC(x,tIn)
% Remove the DC value of a signal by subtracting its mean
   y = x - mean(x);
```

Example: The following function changes the starting time of a signal to a specified value:

```
function [y,tOut] = timealign(x,tIn,startTime)
% Change the starting time of a signal
   y = x;
   t = tIn;
   if ~isempty(t)
       t = t - t(1) + startTime;
   end
   tOut = t;
```

At any time, you can edit functions, edit their descriptions, or remove them, using the **Manage Custom Functions** option in the gallery.


Search
PREPROCESSING OPTIONS
Smooth
Apply smoothing to signals selected in table
Lowpass
Apply lowpass filter to signals selected in table
Highpass
Apply highpass filter to signals selected in table
Bandpass
Apply bandpass filter to signals selected in table
Bandstop
Apply bandstop filter to signals selected in table
Resample
Apply resample to signals selected in table
CUSTOM FUNCTIONS
declip
Declip saturated signal by fitting a polynomial
removeDC
Remove the DC value of a signal by subtracting its mean
timealign
Change the starting time of a signal
Add Custom Function
Manage Custom Functions

Explore Signals

You can use the **Signal Analyzer** app to perform several tasks that help you explore your data.

Plot Signals

Select a signal by clicking its name in the Workspace browser or the Signal table. Then plot your selection by dragging it to a display. This action also selects the check box to the left of the signal **Name** on the Signal table. You can also plot a signal by selecting this check box. The app displays a set of axes with the time-domain waveform and a **Time** tab with options to control the view.

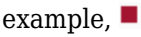
If you drag a matrix from the Workspace browser to a display, the app automatically plots each column as a separate signal, up to a maximum of 10 columns. The app creates signals in the Signal table for the remaining columns, but you must drag the additional signals to the display.

Note Signals with no time information are plotted in units of samples on the *x*-axis. Signals with time information are plotted in units of time on the *x*-axis. To plot several signals on the same display, ensure that they all have time information or are all in samples. Otherwise, you get a warning.

View Signals on Multiple Plots

Click **Display Grid** to create or remove displays.

Move Signals Between Displays

To move a signal from one display to another, click the plotted line or select its name on its **Legend**, for example,  . Click the resulting thicker line and drag it to the target display.

Visualize Signal Spectra

Use the **Signal Analyzer** app to analyze signals in the frequency domain. To activate the frequency-domain view of a signal, click **Spectrum** ▼ on the **Display** tab and select

`Spectrum`. The app displays a set of axes with the signal spectrum, and a **Spectrum** tab with options to control the view.

- If the panner is activated and is zoomed in on a particular region of interest, the spectrum in the display corresponds to the region of interest, not the whole signal.
- If you zoom in on a region of the signal in the time plot using one of the zoom actions on the **Display** tab, the spectrum in the display corresponds to the region of interest, not the whole signal.
- You cannot zoom out in frequency beyond the Nyquist range.
- To see a time plot and a spectrum plot of the same signal side-by-side, use different displays. Drag the signal to two displays. Click **Time** or **Spectrum** on the **Display** tab to control what is plotted on each display.

For more information on how **Signal Analyzer** computes spectra, see “Spectrum Computation in Signal Analyzer” on page 19-102.

If a signal is nonuniformly sampled, then **Signal Analyzer** interpolates the signal to a uniform grid to compute spectral estimates. The app uses linear interpolation and assumes a sample time equal to the median of the differences between adjacent time points. For a nonuniformly sampled signal to be supported, the median time interval and the mean time interval must obey

$$\frac{1}{100} < \frac{\text{Median time interval}}{\text{Mean time interval}} < 100.$$

Visualize Persistence Spectra

Use the **Signal Analyzer** app to visualize the persistence spectrum of a signal: The persistence spectrum contains time-dependent probabilities of occurrence of signals at given frequency locations and power levels. This type of spectrum is useful for detecting brief events.

To activate the persistence spectrum, click **Spectrum** ▼ on the **Display** tab and select `Persistence Spectrum`. The app displays a set of axes with the persistence spectrum, and a **Persistence Spectrum** tab with options to control the view. You cannot zoom out in frequency beyond the Nyquist range.

Note You can plot the persistence spectrum of only one signal per display.

For more information on how **Signal Analyzer** computes persistence spectra, see “Persistence Spectrum in Signal Analyzer” on page 19-107.

Visualize Signal Spectrograms

Use the **Signal Analyzer** app to analyze a signal in the time-frequency domain. To activate the spectrogram view of a signal, click **Time-Frequency** ▼ on the **Display** tab and select `Spectrogram`. The app displays a set of axes with the signal spectrogram, and a **Spectrogram** tab with options to control the view.

Note You can plot the spectrogram of only one signal per display.

- If the panner is activated and is zoomed in on a particular region of interest, the spectrogram in the display corresponds to the region of interest, not the whole signal.
- If you zoom in on a region of the signal in the time plot using one of the zoom actions on the **Display** tab, the spectrogram in the display corresponds to the region of interest, not the whole signal.
- You cannot zoom out in frequency beyond the Nyquist range.
- To see a time plot and a spectrogram plot of the same signal side-by-side, use different displays. Drag the signal to two displays. Click **Time** or **Time-Frequency** on the **Display** tab to control what is plotted on each display.

For more information on how **Signal Analyzer** computes spectrograms, see “Spectrogram Computation in Signal Analyzer” on page 19-110.

The reassignment technique sharpens the time and frequency localization of spectrograms by reassigning each power spectrum estimate to the location of its center of energy. If your signal contains well-localized temporal or spectral components, then this option generates a spectrogram that is easier to read and interpret. To apply reassignment to a spectrogram, check **Reassign** in the **Spectrogram** tab.

If a signal is nonuniformly sampled, then **Signal Analyzer** interpolates the signal to a uniform grid to compute spectral estimates. The app uses linear interpolation and assumes a sample time equal to the median of the differences between adjacent time points. For a nonuniformly sampled signal to be supported, the median time interval and the mean time interval must obey

$$\frac{1}{100} < \frac{\text{Median time interval}}{\text{Mean time interval}} < 100.$$

Visualize Signal Scalograms

Use the **Signal Analyzer** app to visualize the scalogram of a signal. The scalogram is useful for identifying signals with low-frequency components and for analyzing signals whose frequency content changes rapidly with time. You need a Wavelet Toolbox™ license to use the scalogram view.

To activate the scalogram view of a signal, click **Time-Frequency** ▼ on the **Display** tab and select `Scalogram`. The app displays a set of axes with the signal scalogram and a **Scalogram** tab with options to control the view.

Note You can plot the scalogram of only one signal per display.

- If the panner is activated and is zoomed in on a particular region of interest, the scalogram in the display corresponds to the whole signal, not just the region of interest. **Signal Analyzer** performs an optical zooming, using interpolation to display a smooth curve.
- If you zoom in on a region of the signal in the time plot using one of the zoom actions on the **Display** tab, the scalogram in the display corresponds to the whole signal, not just the region of interest. **Signal Analyzer** performs an optical zooming, using interpolation to display a smooth curve.
- To see a time plot and a scalogram plot of the same signal side by side, use different displays. On the **Display** tab, click **Display Grid**, create a side-by-side pair of displays, and drag-and-drop the signal on both displays. Click **Time** or **Time-Frequency** on the **Display** tab to control what is plotted on each display.

Note Scalogram view does not support nonuniformly sampled signals.

For more information on how **Signal Analyzer** computes scalograms, see “Scalogram Computation in Signal Analyzer” on page 19-119.

Zoom and Pan Through Signals

The **Signal Analyzer** app features a panner that enables you to zoom in on and navigate through signals to see how they change in frequency and time. To activate the panner, on the **Display** tab, click **Panner** .

The panner renders signals in their entire duration. To select a region of interest, click the panner and drag to create a zoom window. Use the mouse to resize or slide the zoom window along the length of the signal.

- If the spectrum of the signal is plotted, it corresponds to the region of interest, not the whole signal. See “Spectrum Computation in Signal Analyzer” on page 19-102 for more details.
- If the persistence spectrum of the signal is plotted, it corresponds to the region of interest, not the whole signal. See “Persistence Spectrum in Signal Analyzer” on page 19-107 for more details.
- If the spectrogram of the signal is plotted, it corresponds to the region of interest, not the whole signal. See “Spectrogram Computation in Signal Analyzer” on page 19-110 for more details.
- If the scalogram of the signal is plotted, it corresponds to the whole signal, not the region of interest. **Signal Analyzer** performs an optical zooming, using interpolation to display a smooth curve. See “Scalogram Computation in Signal Analyzer” on page 19-119 for more details.
- You cannot zoom out in frequency beyond the Nyquist range.

Edit Time Information and Link Displays in Time

Use the **Signal Analyzer** app to add time information to signals. In the Signal table, select the signals whose time information you want to add or modify. Add time information to the signals by clicking **Time Values** in the **Analyzer** tab.

You can express the time information in terms of a sample rate or sample time, and a start time. You can also add explicit time values using a numeric vector, a `duration` array, or a MATLAB expression. Time values must be unique and cannot be `NaN`, but they need not be uniformly spaced. The app derives a sample rate from the time values and displays it in the **Time** column of the Signal table. See “Edit Sample Rate and Other Time Information” on page 19-93 for more details.

Note Filtering and scalogram view do not support nonuniformly sampled signals.

- If a signal is nonuniformly sampled, then **Signal Analyzer** interpolates the signal to a uniform grid to compute spectral estimates. The app uses linear interpolation and assumes a sample time equal to the median of the differences between adjacent time points. The derived sample rate in the Signal table has an asterisk to indicate that the signal is nonuniformly sampled. For a nonuniformly sampled signal to be supported, the median time interval and the mean time interval must obey

  $$\frac{1}{100} < \frac{\text{Median time interval}}{\text{Mean time interval}} < 100.$$

  Note The interpolation is used only to compute spectral estimates. Time plots are not resampled.

- You can link display time spans so that plot responses are synchronized when you pan and zoom horizontally. The signals in the displays you want to link must contain time information. To link the time span of a display to the time spans of the displays linked already, select the display and, on the **Display** tab, select **Link Time**. To unlink a display, select it and clear **Link Time**.

  Note Selecting **Link Time** links the selected display to the complete collection of displays that have already been linked.

  Displays with linked time spans have the following operations synchronized:

  - Panning by selecting and dragging the plot or by using the display panner.
  - Zooming in, zooming out, or zooming on the time axis. Zooming in or out on one display affects only the time axis in the remaining linked displays.
  - Fitting data to view. The app stretches the common time axis so that it shows the span from the earliest to the latest time among all signals in the linked displays.
  - If the axes of two displays are linked in time, then the time cursors in the displays are linked.

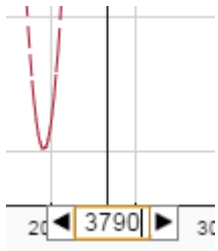
  The time axis of a linked display might update as you add or remove signals.

Note Frequency axes are never linked between displays.

Measure Signal, Spectrum, and Time-Frequency Data

Measure your data using data cursors:

1. On the **Display** tab, click **Data Cursors ▼** to add one or two data cursors to all the displays. Time-domain and frequency-domain cursors are not linked and can be moved independently.
2. The persistence spectrum, spectrogram, and scalogram views display two-dimensional crosshair cursors.
3. To move a data cursor, drag it left, right, up, or down to a point of interest. To move the cursor sample-by-sample, click the time or frequency field and use the arrow keys.
4. You can move a data cursor to a specific point without dragging it. Click the data cursor time or frequency field and enter a value.



   If the signal was not sampled at a point of interest, then the app linearly interpolates the value. If the value is interpolated, an asterisk appears in the data cursor label.
5. By default, cursors snap to the nearest data point. To change this behavior, clear the **Snap to Data** check box on the **Display** tab.
6. To toggle the cursors, click **Data Cursors**.

Extract Signal Regions of Interest

The **Signal Analyzer** app enables you to extract regions of interest from the signals you are studying and export them for further analysis. To extract regions of interest, select the display that has them. On the **Display** tab, click **Extract Signals**, or right-click the display and select **Extract Signals**

- Select `Between Time Limits` to extract a region of interest defined by the time limits of the selected display. To change the time limits, you can use the panner, select

one of the zoom actions on the **Display** tab, or change the limit values on the **Display**, **Time**, **Spectrogram**, or **Scalogram** tabs.

- Select `Between Time Cursors` to extract a region of interest defined by the locations of the time-domain cursors in the selected display.
- If a signal has time information, you can preserve the start time of the region of interest by checking **Preserve Start Time**.

The extracted regions of interest are added at the bottom of the Signal table.

Share Analysis

Copy Displays

You can share the plots that you have produced using the **Signal Analyzer** app by copying one or more displays to the clipboard as images and pasting them into another application.

To copy displays to the clipboard, on the **Display** tab, click **Copy All Displays ▼**. You can then copy either the selected display or the complete display layout.

To copy a single display to the clipboard, you can also right-click the display and select **Copy Display**.

Export Signals

You can export any signals in the **Signal Analyzer** Signal table to the MATLAB workspace or to a MAT-file.

To export signals:

1. Select one or more signals from the Signal table.
2. On the **Analyzer** tab, click **Export**

.
3. Choose whether you want to export the selected signals to the MATLAB workspace or save them to a MAT-file. If you choose to save the signals, browse to where you want to save the file, name the file, and click **Save.**

You can also select the signals, right-click, and select **Export**.

Signals are exported differently, depending on their type:

- Signals with no time information are exported or saved as numeric vectors.
- Signals stored as timetables are exported or saved as timetables.
- Signals that have time information but are not stored as timetables are exported or saved as numeric vectors. If you want to preserve the time information, you can save the signals as timetables. On the **Analyzer** tab, click **Preferences** and check `Always use timetables when signals have time information`.

Generate MATLAB Scripts and Functions

You can generate MATLAB scripts to extract signal regions of interest or automate the computation of power spectrum, persistence spectrum, spectrogram, or scalogram estimates obtained with the **Signal Analyzer** app.

To generate a MATLAB script, on the **Display** tab, click **Generate Script**. The generated script opens in the Editor.

- Select `ROI Script Between Time Limits` to generate a MATLAB script that extracts a region of interest defined by the time limits of the selected display. Depending on the preferences, the regions of interest are saved as numeric vectors or as a timetable.
- Select `ROI Script Between Time Cursors` to generate a MATLAB script that extracts a region of interest defined by the locations of the time-domain cursors in the selected display. Depending on the preferences, the regions of interest are saved as numeric vectors or as a timetable.
- Select `Spectrum Script` to generate a MATLAB script that computes the power spectrum appearing in the spectrum view of the selected display, including all current settings.
- Select `Persistence Spectrum Script` to generate a MATLAB script that computes the persistence spectrum appearing in the spectrum view of the selected display, including all current settings.
- Select `Spectrogram Script` to generate a MATLAB script that computes the spectrogram appearing in the spectrogram view of the selected display, including all current settings.
- Select `Scalogram Script` to generate a MATLAB script that computes the scalogram appearing in the scalogram view of the selected display, including all current settings. You need a Wavelet Toolbox license to use the scalogram view.

You can generate MATLAB functions to automate signal preprocessing steps performed with the **Signal Analyzer** app.

To generate a MATLAB preprocessing function, on the **Analyzer** tab, click **Generate Function**. The generated function opens in the Editor.

Save and Load Signal Analyzer Sessions

If you want to share session snapshots or archive them to view later, save the **Signal Analyzer** session to a MAT-file or MLDATX-file. Using MLDATX-files results in faster save and load times.

To save a session to a MAT-File or MLDATX-File:

1. On the **Analyzer** tab, click **Save ▼** and select `Save`.
2. Browse to where you want to save the file, name the file, choose the format, and click **Save**.

If you want to update the file, click **Save**. If you want to save the session to a different file, click **Save ▼** and select `Save as`.

To load a saved session:

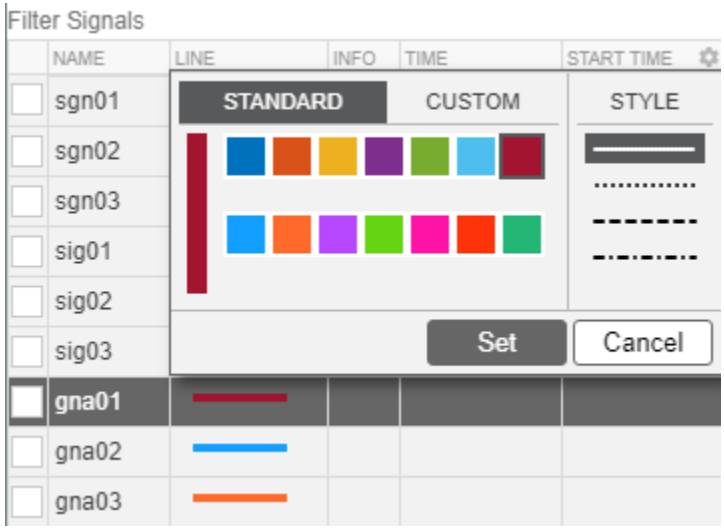
1. On the **Analyzer** tab, click **Open**.
2. Browse to the MAT-file or MLDATX-file saved from a previous session, select it, and click **Open**. The signal data and properties appear as they were when the file was last saved.

To start a new session, on the **Analyzer** tab, click **New**.

Customize Signal Analyzer

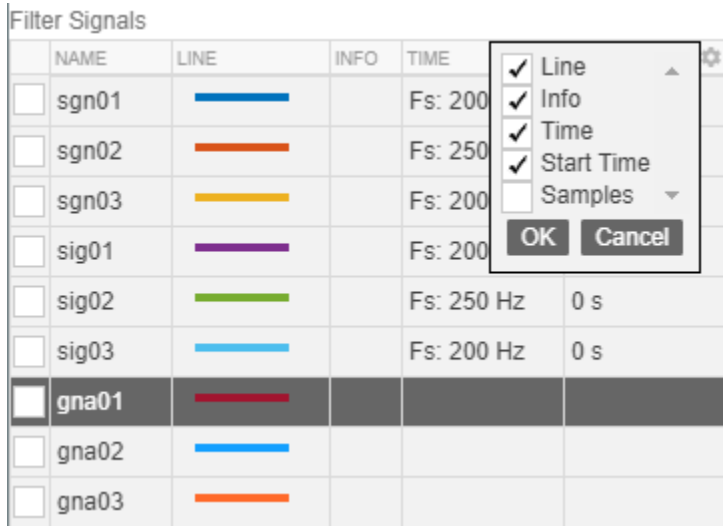
Specify Line Color and Style

To specify the line style and color, click in the **Line** column of a signal. If the line column is not shown, add the column using the column selector button . Select a color from the palette and a line style. Click **Custom** to choose custom colors for your signals. You can specify custom colors as RGB triplets or as hexadecimal codes.



Add or Remove Columns in the Signal Table

Columns in the Signal table display the plot configuration and signal properties. To add or remove a column, click the column selector button . From the list, select the columns that you want to display and click **OK**. After you select a column, the new column is added to the table in the order that it appears in the column selection list.



Modify Signal Analyzer Displays

Goal	Action
Hide the Workspace browser or the Signal table to enlarge the display area.	On the **Analyzer** tab, click one of the layout buttons.
Zoom and pan to inspect the data.	On the **Display** tab, select one of the zoom actions. Alternatively, activate the panner by clicking **Panner** .

<table>
<tr><th>Goal</th><th>Action</th></tr>
<tr><td>Fit spectrogram, scalogram, or persistence spectrum colormap to current power or density limits.</td><td>On the Display tab, click the Fit Colormap button .</td></tr>
<tr><td>Set the minimum and maximum values of the plot axes.</td><td>On the Time, Spectrum, Persistence Spectrum, Spectrogram, or Scalogram tab, enter the axes limit values. You can also change the minimum and maximum time values on the Display tab.

When setting axes for a display, you can specify time or frequency units before specifying limit values. Several engineering units are available:
<table>
<tr><th>Quantity</th><th>Units</th></tr>
<tr><td>Time</td><td>ps, ns, µs, ms, seconds, minutes, hours, days, years</td></tr>
<tr><td>Frequency</td><td>cycles/year, cycles/day, cycles/hour, cycles/minute, mHz, Hz, kHz, MHz, GHz, THz</td></tr>
</table>
Note Selecting different time or frequency units for axes limit values does not change any plots.</td></tr>
</table>

Goal	Action
Show or hide legends identifying plotted signals.	On the **Display** tab, click **Legend** . Each display gets its own legend. The legends appear either at the top of the display or to the right of the display.
Link or unlink a display.	Select a display. On the **Display** tab, select **Link Time**. **Link Time** is enabled only when there are two or more displays and at least one signal contains time information. To unlink a display, select it and clear **Link Time**. Frequency axes are never linked between displays.
Normalize the data for each signal from 0 to 1 along the *y*-axis of a time plot.	On the **Time** tab, select **Normalize Y Axis**.
Show markers at each sample point in a time plot of a signal.	On the **Time** tab, select **Show Markers**.

Signal Analyzer Preferences

- If you export or save signals that have time information but are not stored as timetables, the time information by default is not saved. If you want to preserve the time information by saving the signals as timetables, on the **Analyzer** tab, click **Preferences** and check `Always use timetables when signals have time information`.
- If you generate scripts involving signals that have time information but are not stored as timetables, the time information by default is not saved. If you want to preserve the time information by generating scripts that treat signals as timetables, on the **Analyzer** tab, click **Preferences** and check `Always use timetables when signals have time information`.

See Also

Signal Analyzer

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

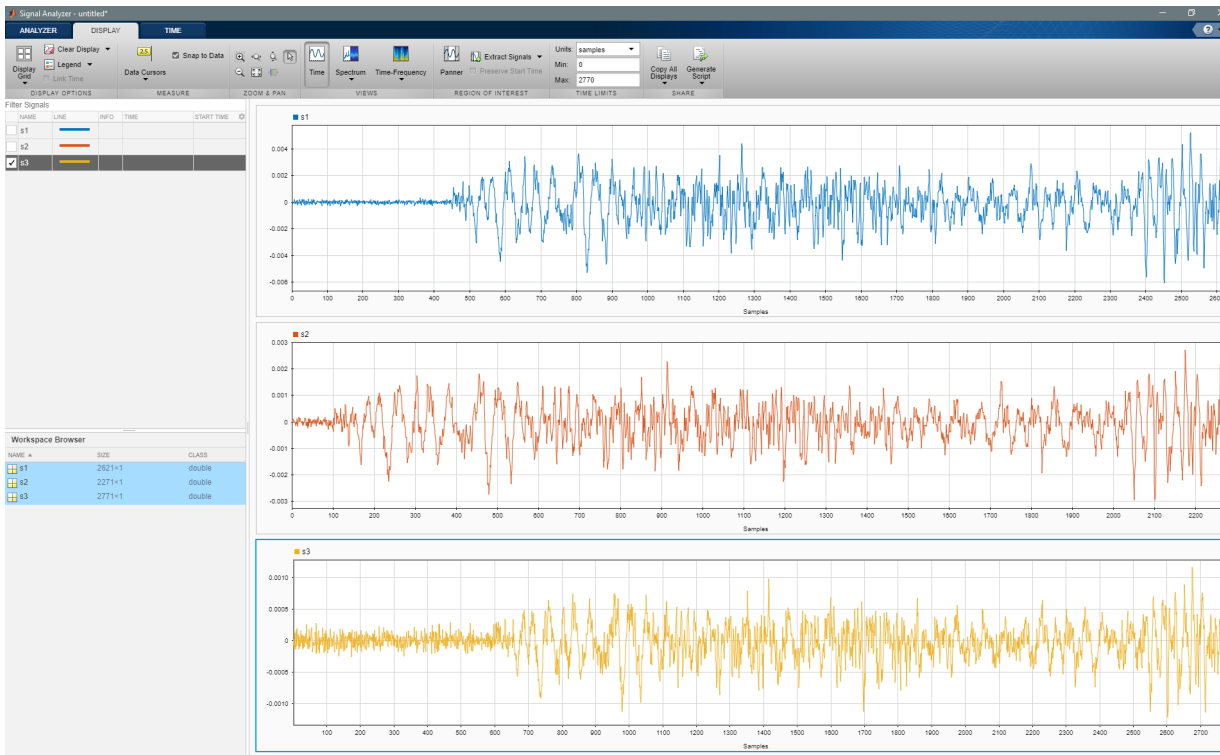
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Find Delay Between Correlated Signals

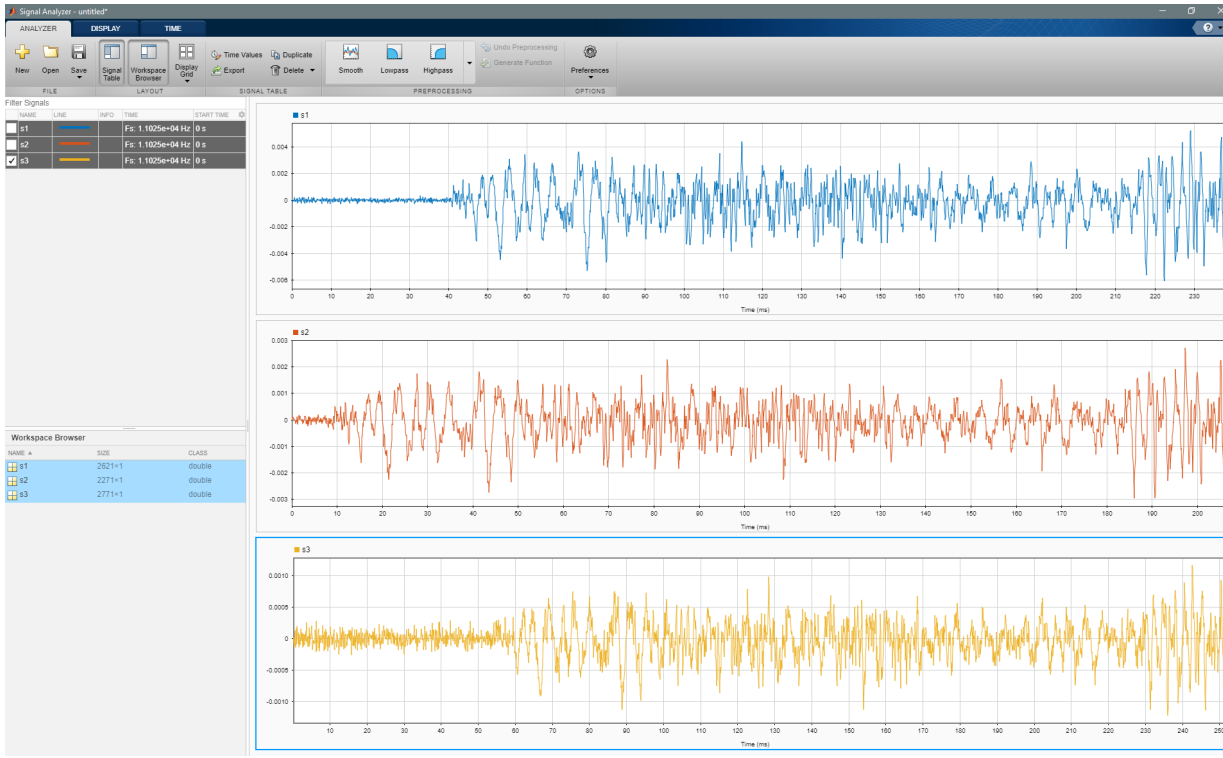
Three sensors at different locations measure vibrations caused by a car as it crosses a bridge. The signals they produce arrive at the analysis station at different times. The sample rate is 11,025 Hz. Use the **Signal Analyzer** app to determine the delays between the signals.

Load the signals into the MATLAB workspace and start the app. The name of each signal includes the number of the sensor that took it. Create three displays. Drag each signal from the Workspace browser to its own display. The signal from Sensor 2 arrives earlier than the signal from Sensor 1. The signal from Sensor 1 arrives earlier than the signal from Sensor 3.

```
load sensorData
signalAnalyzer
```

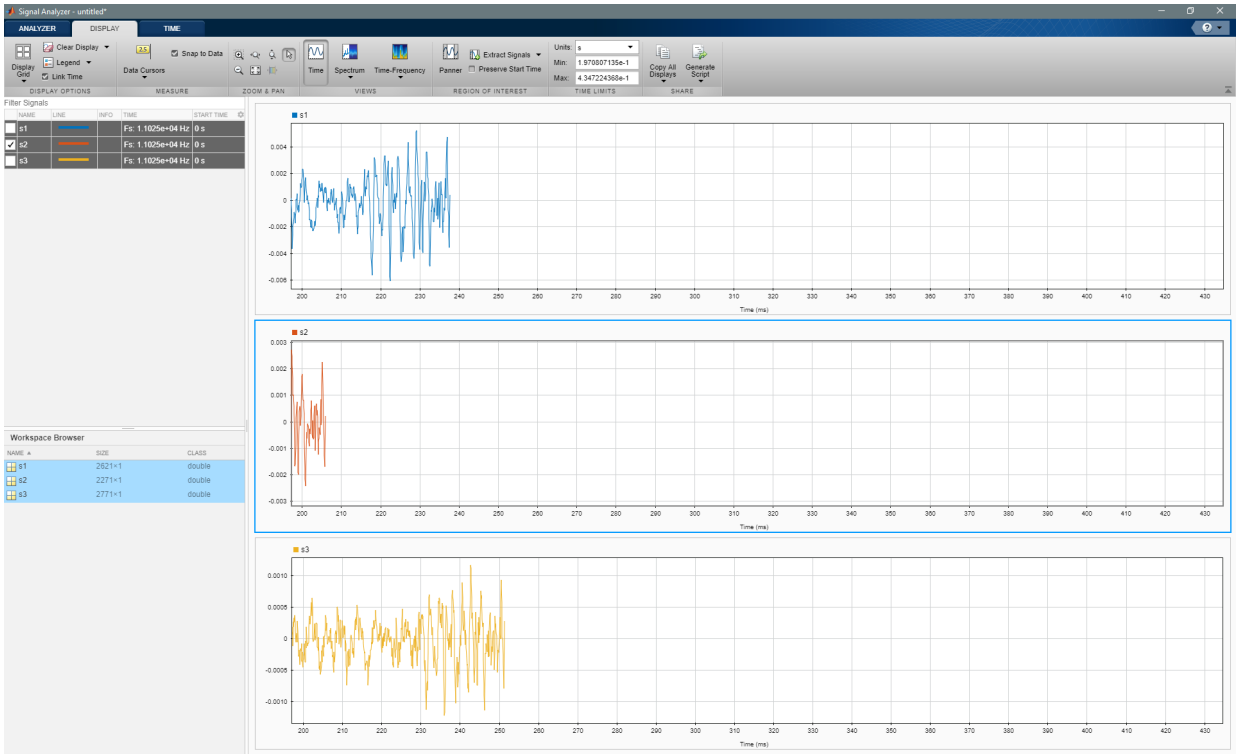


Add time information. Select the three signals in the Signal table and click the **Time Values** button on the **Analyzer** tab. Select the `Sample Rate and Start Time` option and enter the sample rate of 11,025 Hz. For more information, see “Edit Sample Rate and Other Time Information” on page 19-93.



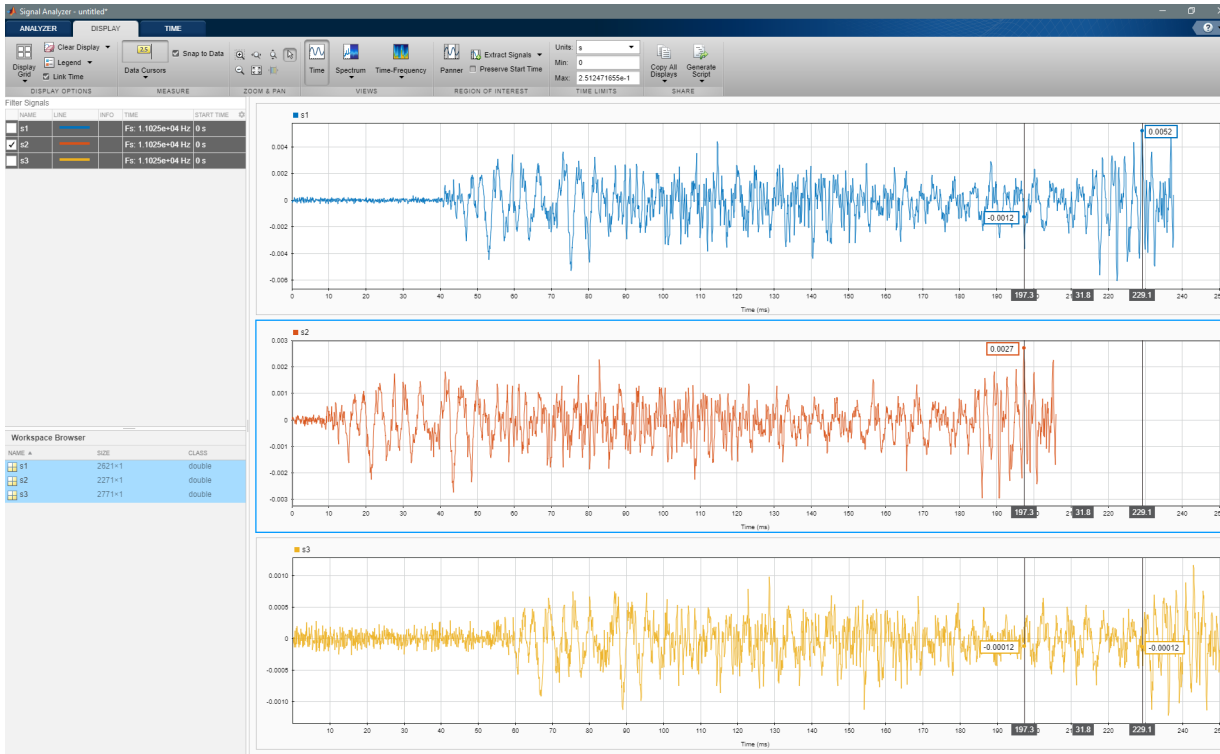
The signals share a common time axis. Link their time spans by selecting each display and selecting **Link Time** on the **Display** tab.

To estimate the delays between the signals, pan them horizontally and line up a salient feature to the end of the time axis. From the **Time** tab, read the time from the lower limit of the time-axis. Choose a region where the signal-to-noise ratio is high, such as the signal maximum toward the end of each signal. In the signal from Sensor 2, that feature occurs about 0.197 second after the clock starts.



Similarly, the signal from Sensor 1 has that feature about 0.229 second after the start, and the signal from Sensor 3 has it about 0.243 second after the start. Thus, the delays are approximately 0.032 second and 0.014 second long.

You can also use data cursors to find the delays. Press the space bar to reset the view. On the **Display** tab, click the arrow under the **Data Cursors** ▼ and select Two. Place a cursor on the maximum of each of the top two signals. You can read the lag of approximately 0.032 second directly from the app.



Similarly, the lag between the top and bottom signals is 0.014 second.

You can get similar results with the `finddelay` and `xcorr` functions.

See Also

Apps
Signal Analyzer

Functions
`alignsignals` | `finddelay` | `xcorr`

Related Examples

- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

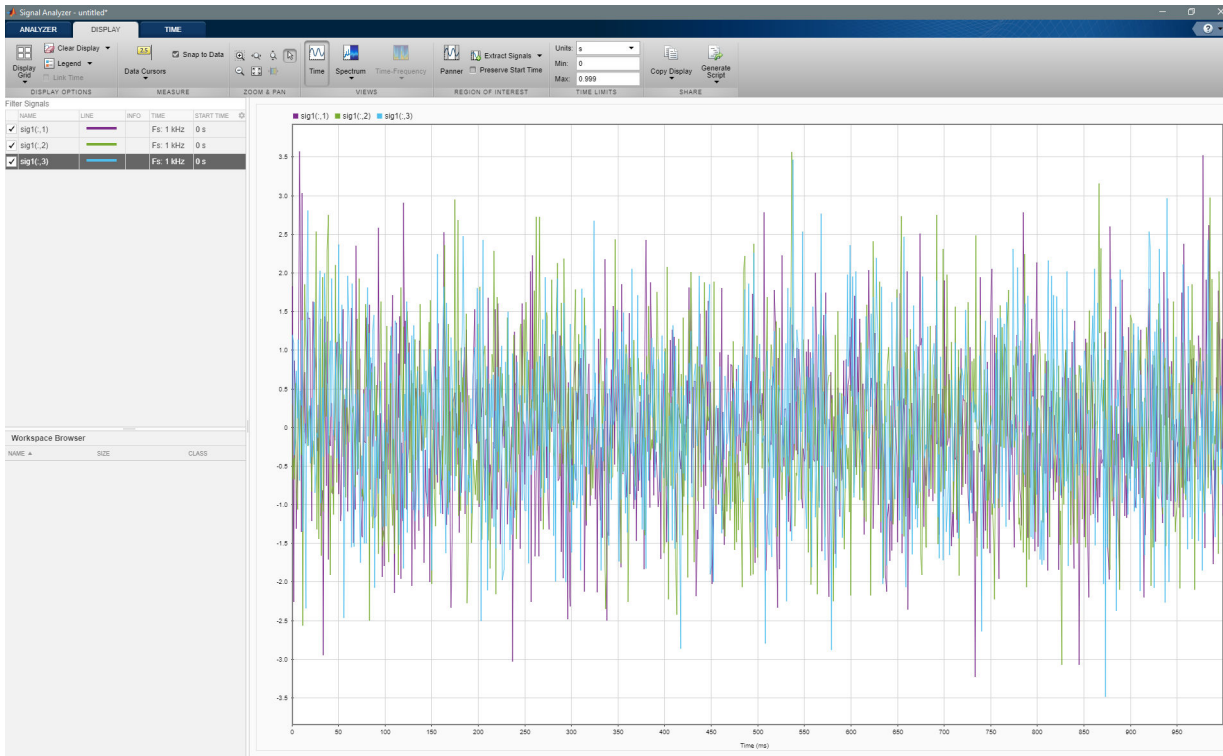
- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Plot Signals from the Command Line

You can plot signals in the **Signal Analyzer** app by calling the app in the command line.

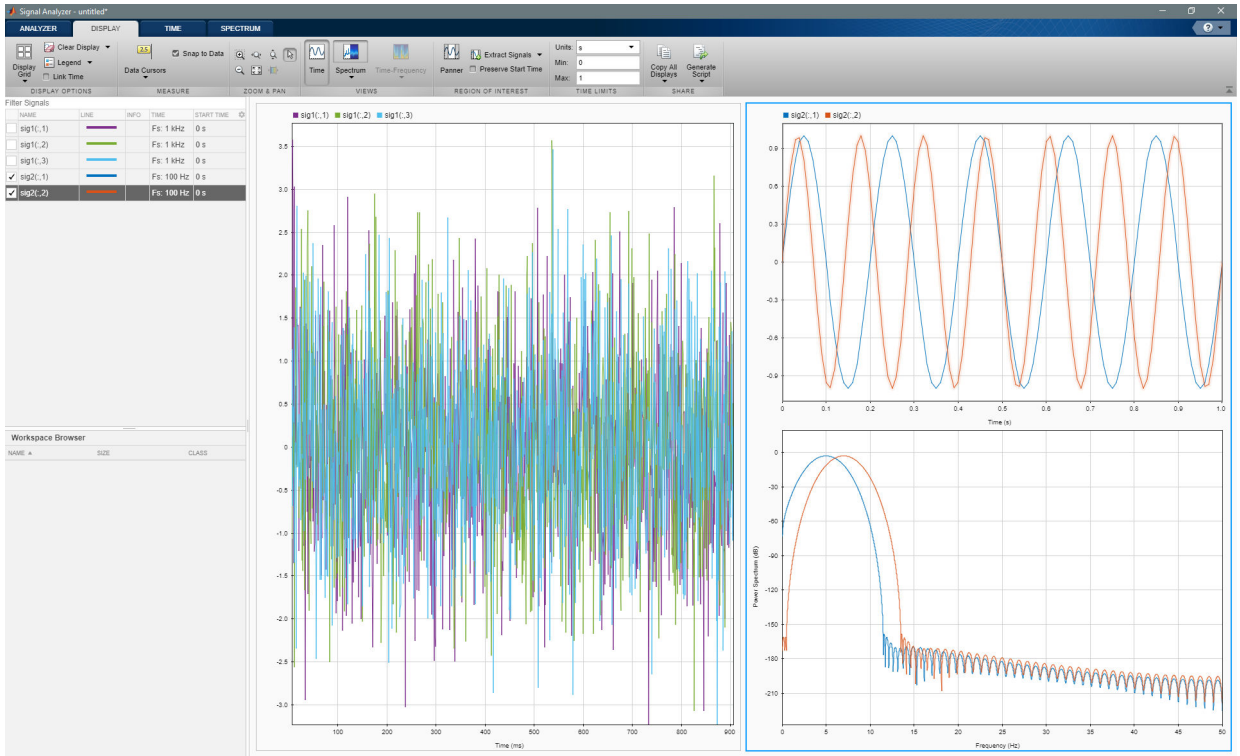
Create a three-channel random signal sampled for 1 second at 1 kHz. Plot the signal in **Signal Analyzer**.

```
signalAnalyzer(randn(1000,3),'SampleRate',1e3)
```



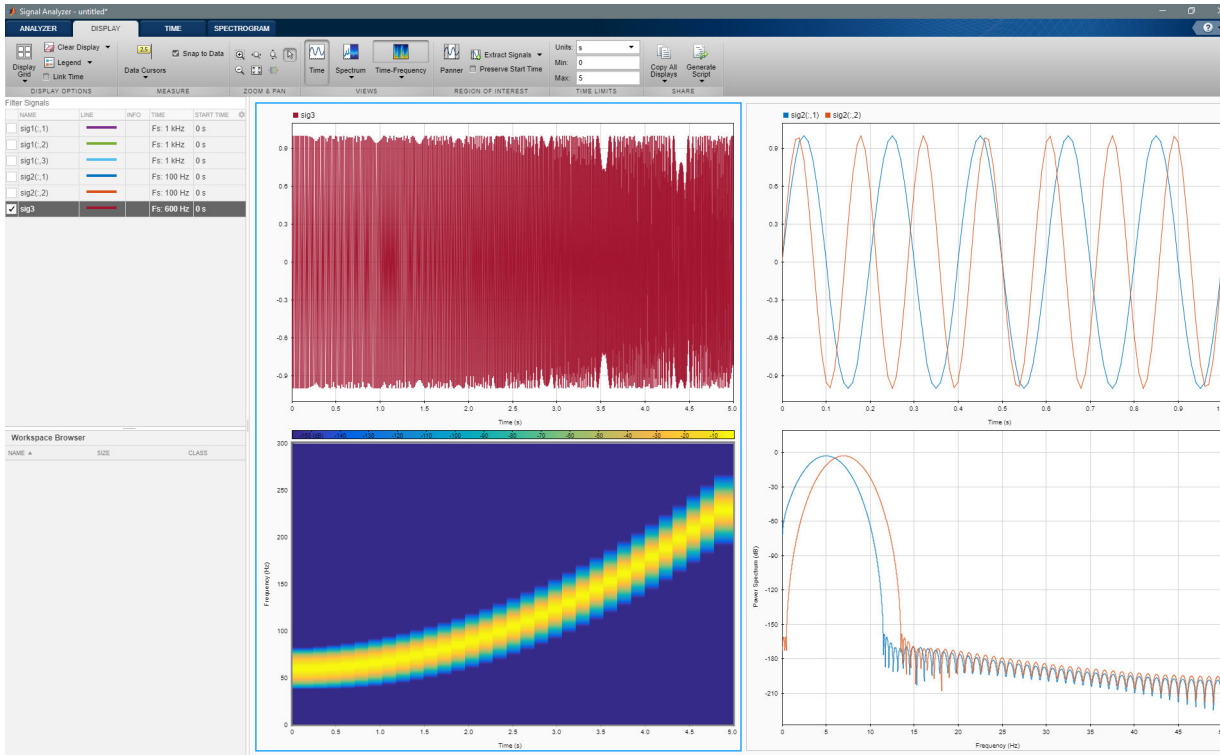
Use **Display Grid** on the **Display** tab to open an empty display. Select the new display and click **Spectrum**. Create a two-channel sinusoidal signal sampled for 1 second at 100 Hz. The sinusoids have frequencies of 5 Hz and 7 Hz. Plot the signal in the new display.

```
signalAnalyzer(sin(2*pi*[5 7].*(0:100)'/100),'SampleRate',100)
```



Select the display with the three random signals. Remove the random signals from the display by clearing the check boxes next to their names. Click **Time-Frequency** . Create a quadratic chirp sampled for 5 seconds at 600 Hz. The chirp has an initial frequency of 60 Hz that increases to 240 Hz by the end of the measurement. Plot the chirp.

```
signalAnalyzer(chirp(0:1/600:5,60,5,240,'quadratic'),'SampleRate',600)
```



See Also

Apps
Signal Analyzer

Functions
`chirp`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44

- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Resolve Tones by Varying Window Leakage

You can adjust the spectral leakage of the analysis window to resolve sinusoids in **Signal Analyzer**.

Generate a two-channel signal sampled at 100 Hz for 2 seconds.

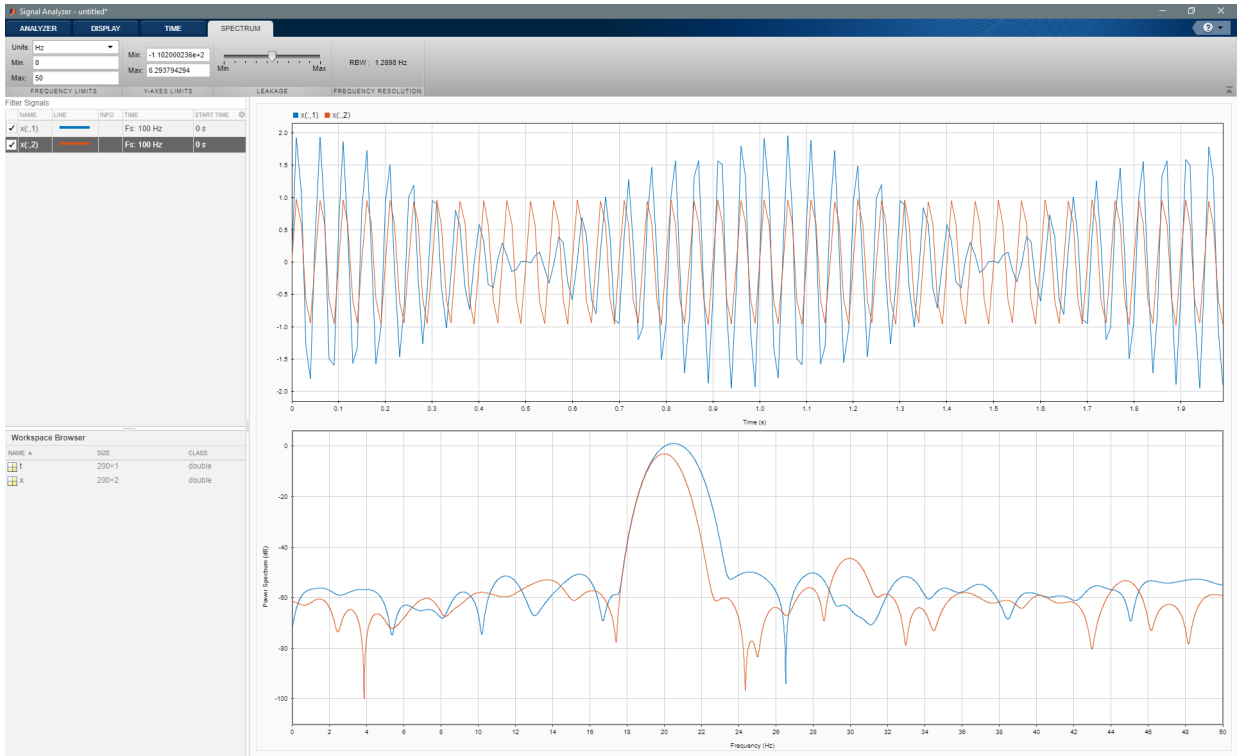
1. The first channel consists of a 20 Hz tone and a 21 Hz tone. Both tones have unit amplitude.
2. The second channel also has two tones. One tone has unit amplitude and a frequency of 20 Hz. The other tone has an amplitude of 1/100 and a frequency of 30 Hz.

```
fs = 100;

t = (0:1/fs:2-1/fs)';

x = sin(2*pi*[20 20].*t)+[1 1/100].*sin(2*pi*[21 30].*t);
```

Embed the signal in white noise. Specify a signal-to-noise ratio of 40 dB. Open **Signal Analyzer** and plot the signal.

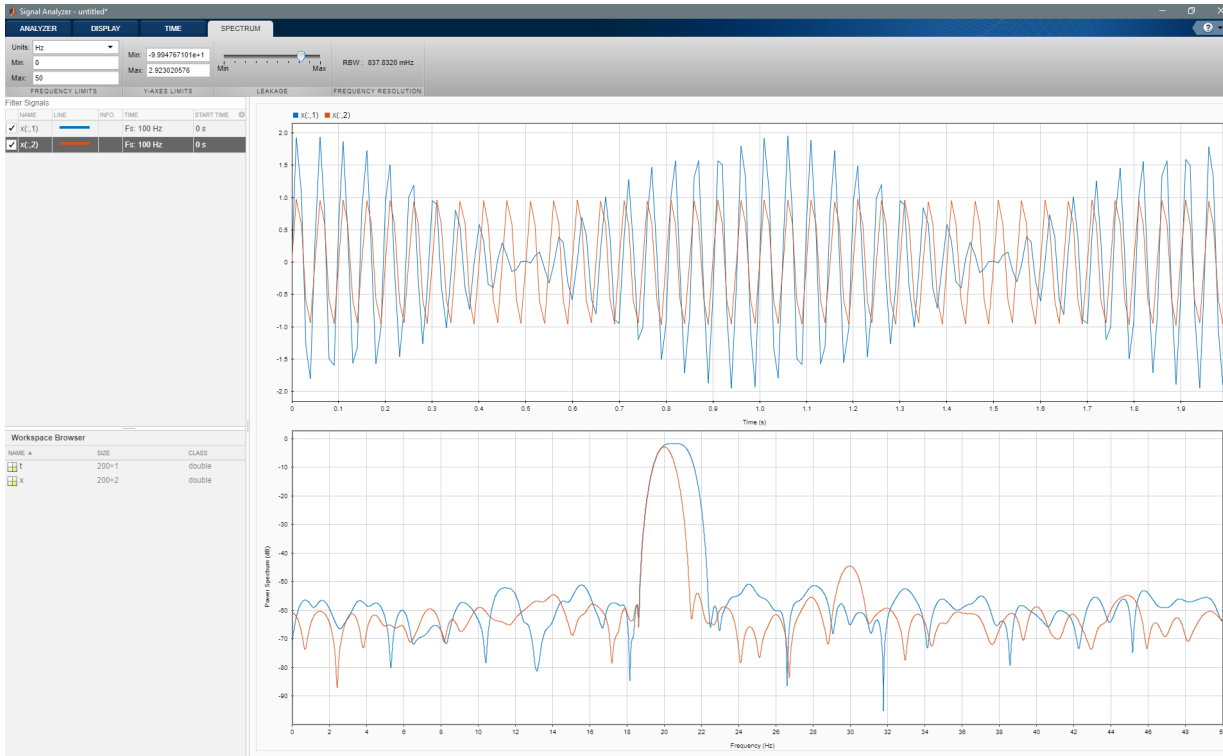
```
x = x+randn(size(x)).*std(x)/db2mag(40);

signalAnalyzer(x,'SampleRate',fs)
```

Click **Spectrum** to add spectral plots to the display. Click the **Spectrum** tab that appears.

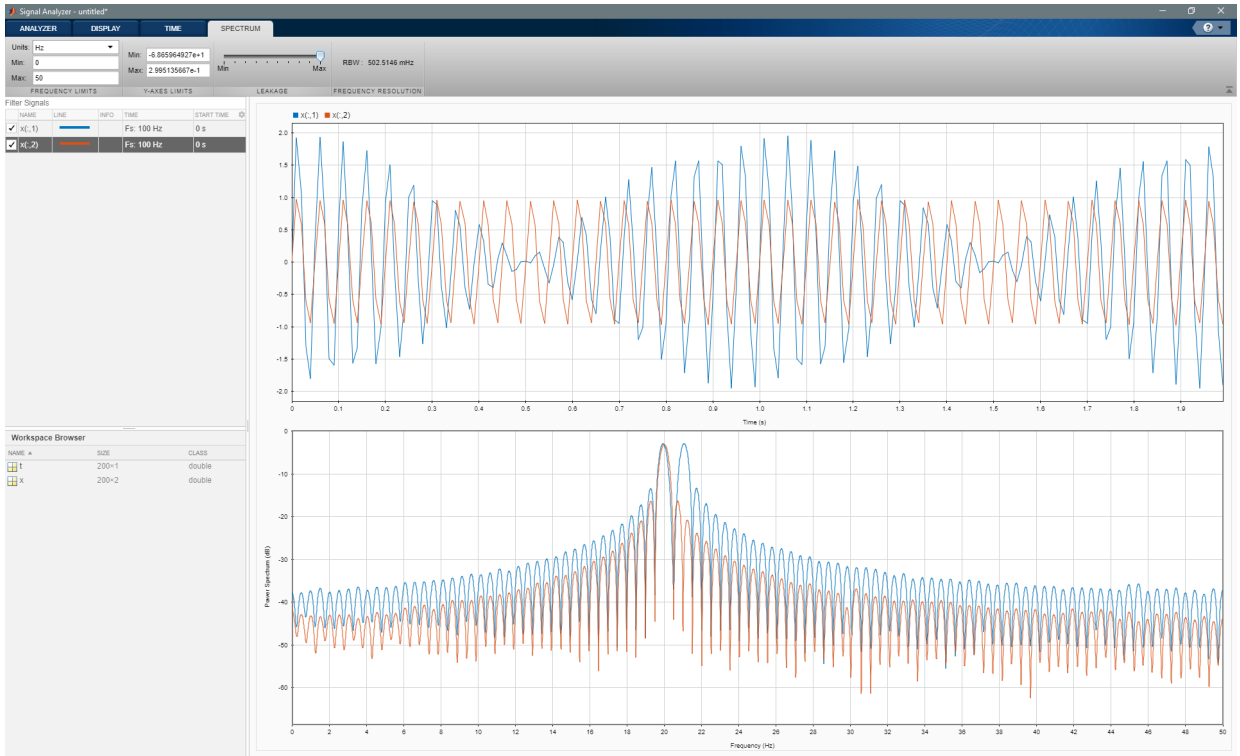


The slider that controls the spectral leakage is in the middle position, corresponding to a resolution bandwidth of about 1.29 Hz. The two tones in the first channel are not resolved. The 30 Hz tone in the second channel is visible, despite being much weaker than the other one.

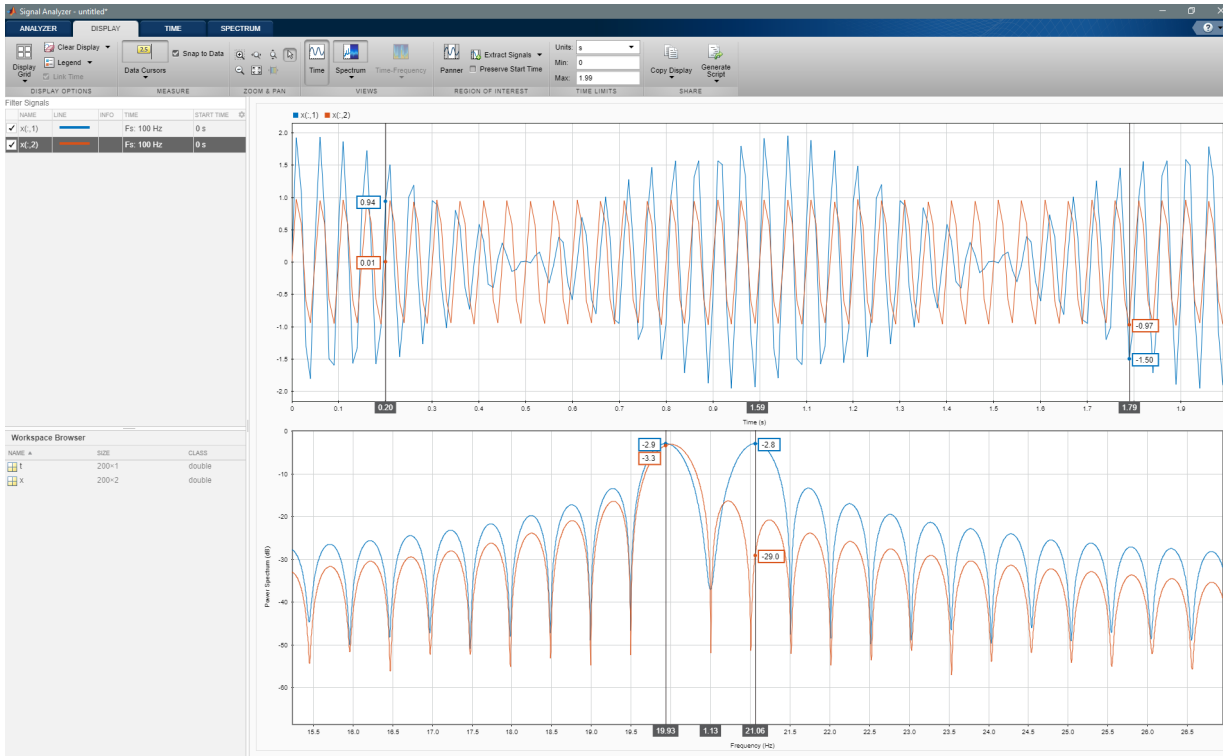
Increase the leakage so that the resolution bandwidth is approximately 0.84 Hz. The weak tone in the second channel is clearly resolved.



Move the slider to the maximum value. The resolution bandwidth is approximately 0.5 Hz. The two tones in the first channel are resolved. The weak tone in the second channel is masked by the large window sidelobes.



Click the **Display** tab. Use the horizontal zoom to magnify the frequency axis. Add frequency-domain cursors to estimate the frequencies of the tones.



See Also

Apps

Signal Analyzer

Functions

`enbw` | `pspectrum` | `pwelch`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Analyze Signals with Inherent Time Information” on page 19-44

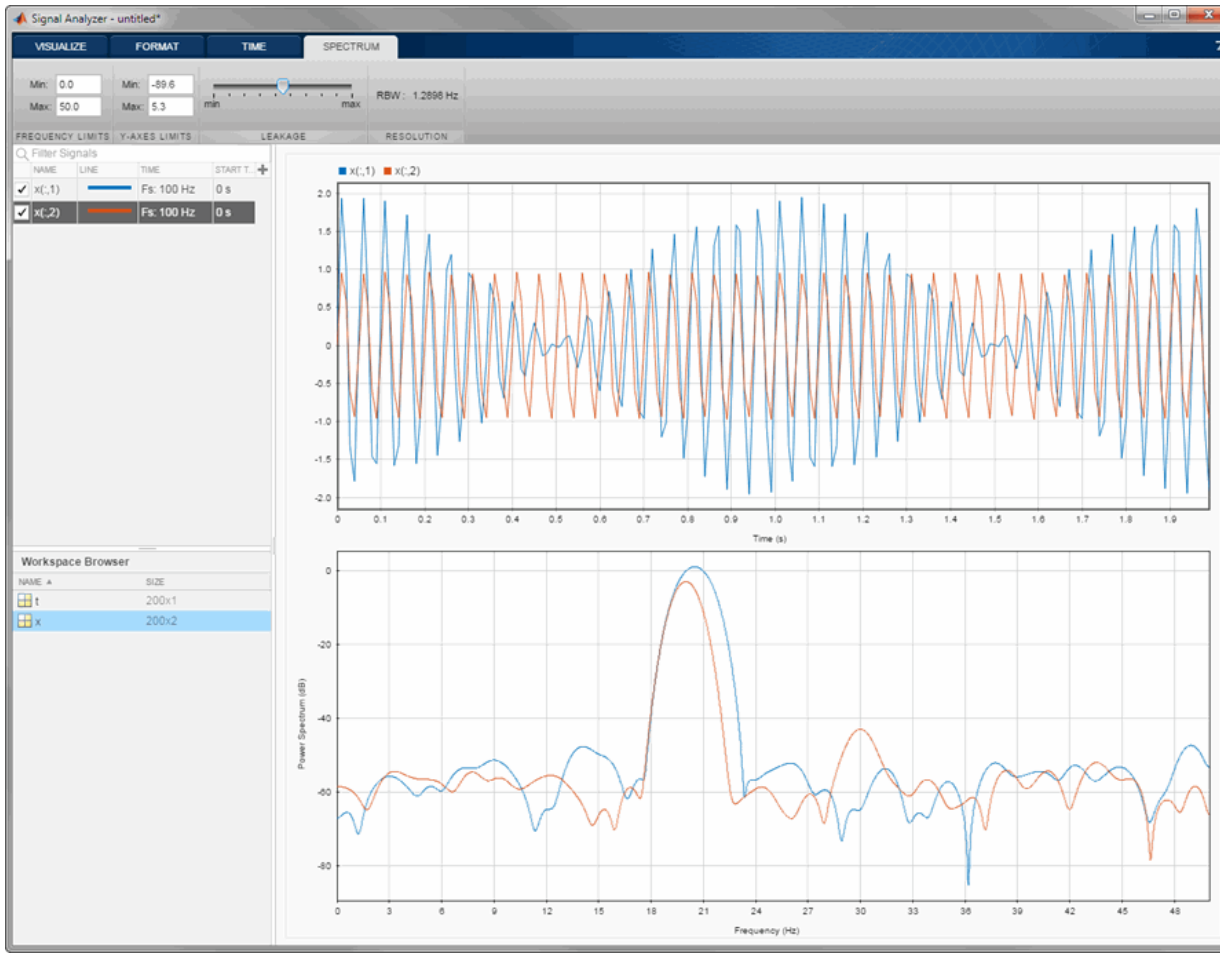
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Resolve Tones by Varying Window Leakage

Go back to example on page 19-36



See Also

Apps
Signal Analyzer

Functions
`enbw` | `pspectrum` | `pwelch`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Analyze Signals with Inherent Time Information

Load a file that contains measurements of the international normalized ratio (INR) performed on a patient over a five-year period. The INR measures the effect of warfarin, a blood thinner used to treat people predisposed to clotting.

The file includes a `datetime` array with the date and time of each measurement, and a vector of INR readings. Display the first five time points.

```
load INR

Date(1:5)
```

```
ans = 5x1 datetime array
   05/15/09 11:28 AM
   06/16/09 09:05 AM
   07/02/09 09:50 AM
   07/16/09 11:20 AM
   07/31/09 12:05 PM
```

Convert the time readings to a `duration` array by subtracting the first time point. Convert the `duration` array to seconds.

```
t = Date-Date(1);
s = seconds(t);
```

Create a timetable with the `duration` array and the INR readings.

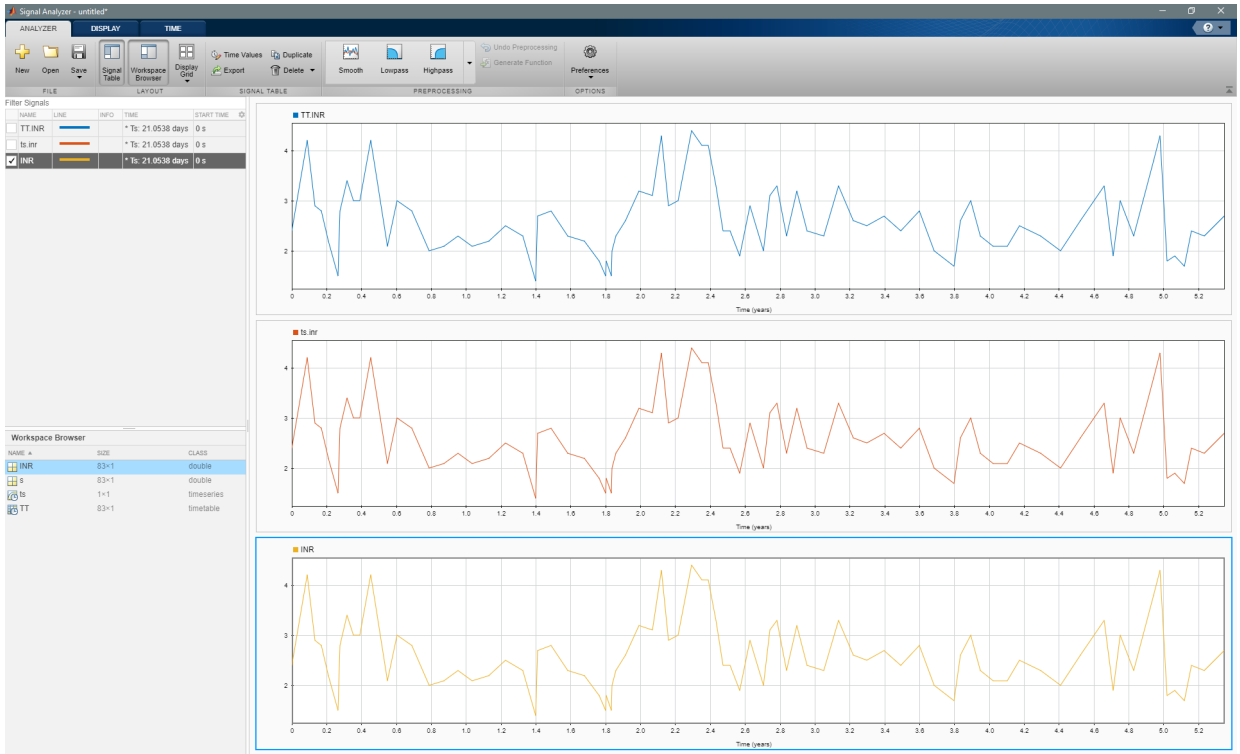
```
TT = timetable(t,INR);
```

Create a `timeseries` object with the time in seconds and the INR readings.

```
ts = timeseries(INR,s,'Name','inr');
```

Open **Signal Analyzer**. Under **Display Grid**, select three displays.

1. Drag the timetable to the first display. The app calls this signal `TT.INR`.
2. Drag the `timeseries` object to the second display. The app calls this signal `ts.inr`.
3. Drag the array of `INR` readings to the third display. Add time information. Select the signal by clicking its legend. From the **Analyzer** tab, click **Time Values**. Select the `Time Values` option and in the **Time Values** field, enter `t`.



See Also

Apps
Signal Analyzer

Functions
`timeseries` | `timetable`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32

- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

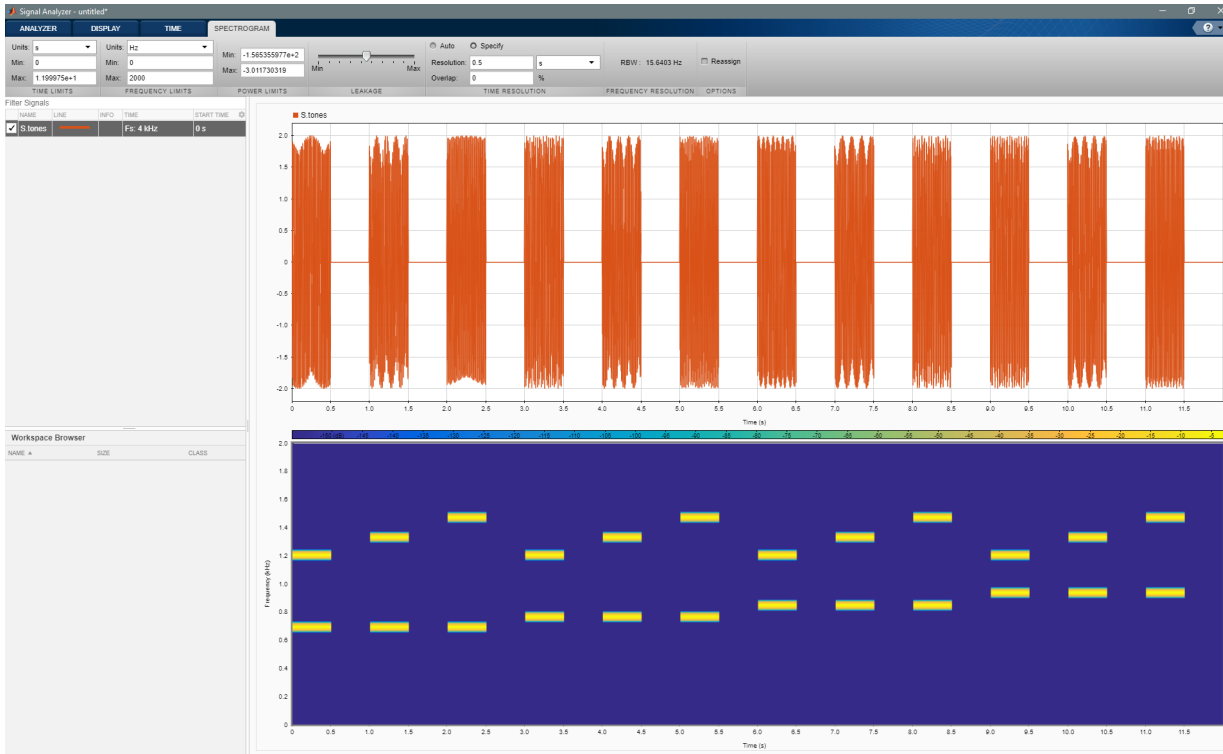
- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Spectrogram View of Dial Tone Signal

Create a signal, sampled at 4 kHz, that resembles dialing all the keys of a digital telephone. Save the signal as a MATLAB® timetable.

```
fs = 4e3;
t = 0:1/fs:0.5-1/fs;

ver = [697 770 852 941];
hor = [1209 1336 1477];

tones = [];

for k = 1:length(ver)
    for l = 1:length(hor)
        tone = sum(sin(2*pi*[ver(k);hor(l)].*t))';
        tones = [tones;tone;zeros(size(tone))];
    end
end

% To hear, type soundsc(tones,fs)

S = timetable(seconds(0:length(tones)-1)'/fs,tones);
```

Open **Signal Analyzer** and drag the timetable to a display. Click **Time-Frequency** to add a spectrogram view. On the **Spectrogram** tab, under **Time Resolution**, select **Specify**. Enter a time resolution of 0.5 second and zero overlap between adjoining segments.



The spectrogram view shows that each key is dialed for half a second, with half-second silent pauses between keys. The first tone has its frequency content concentrated around 697 Hz and 1209 Hz, corresponding to the digit `'1'` in the DTMF standard.

See Also

Apps
Signal Analyzer

Functions
`pspectrum` | `spectrogram` | `timetable`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Find Interference Using Persistence Spectrum

Visualize an interference narrowband signal embedded within a broadband signal.

Generate a chirp sampled at 1 kHz for 500 seconds. The frequency of the chirp increases from 180 Hz to 220 Hz during the measurement.

```
fs = 1000;
t = (0:1/fs:500)';
x = chirp(t,180,t(end),220) + 0.15*randn(size(t));
```

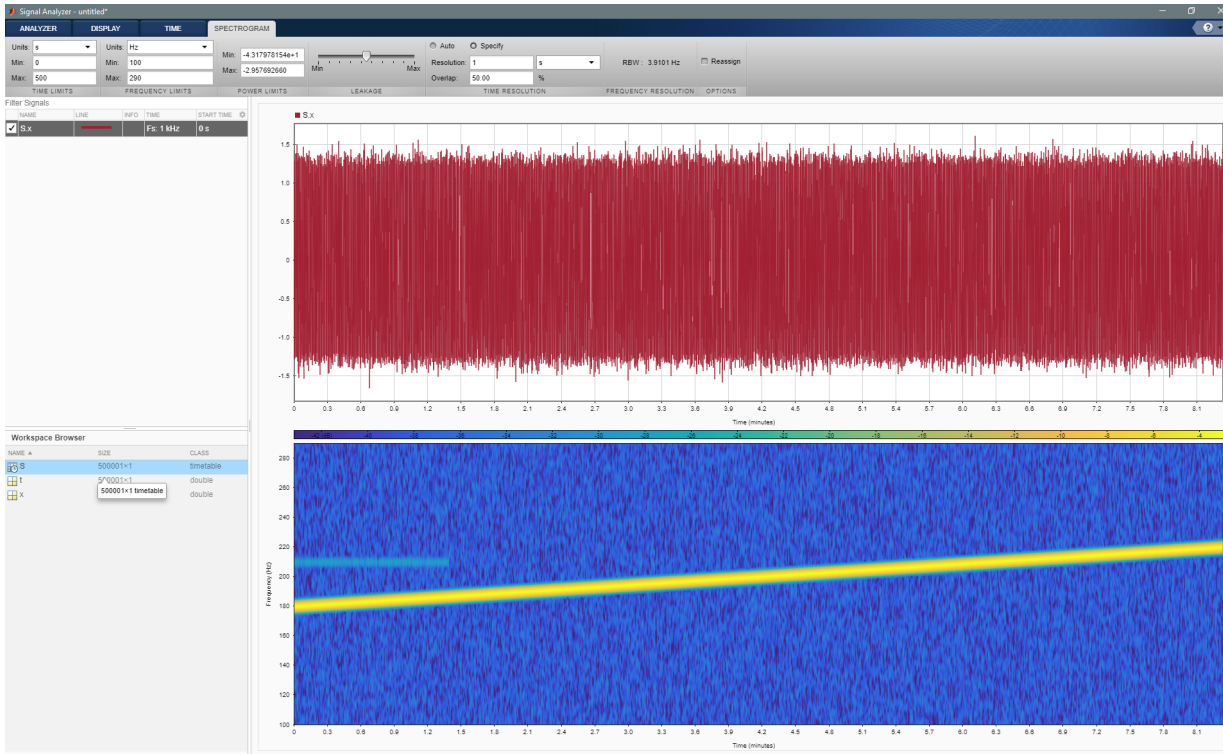
The signal also contains a 210 Hz sinusoid. The sinusoid has an amplitude of 0.05 and is present only for 1/6 of the total signal duration.

```
idx = floor(length(x)/6);
x(1:idx) = x(1:idx) + 0.05*cos(2*pi*t(1:idx)*210);
```

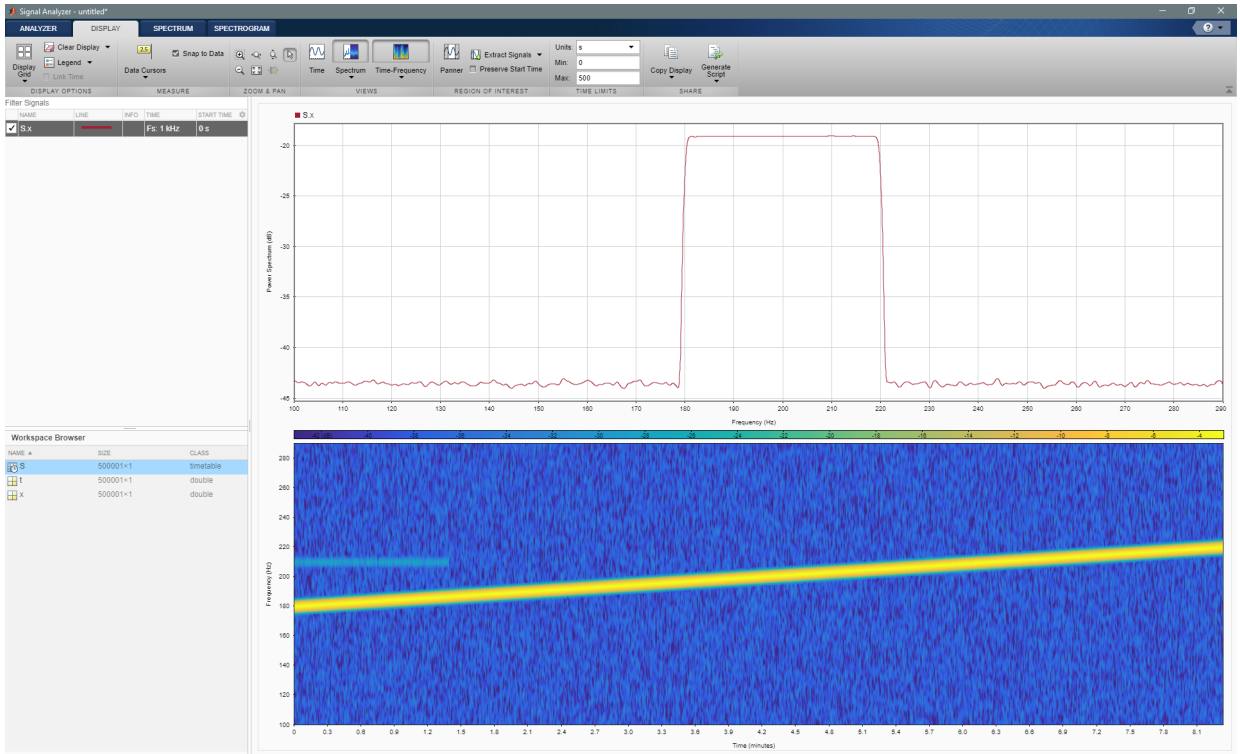
Save the signal as a MATLAB® timetable.

```
S = timetable(seconds(t),x);
```

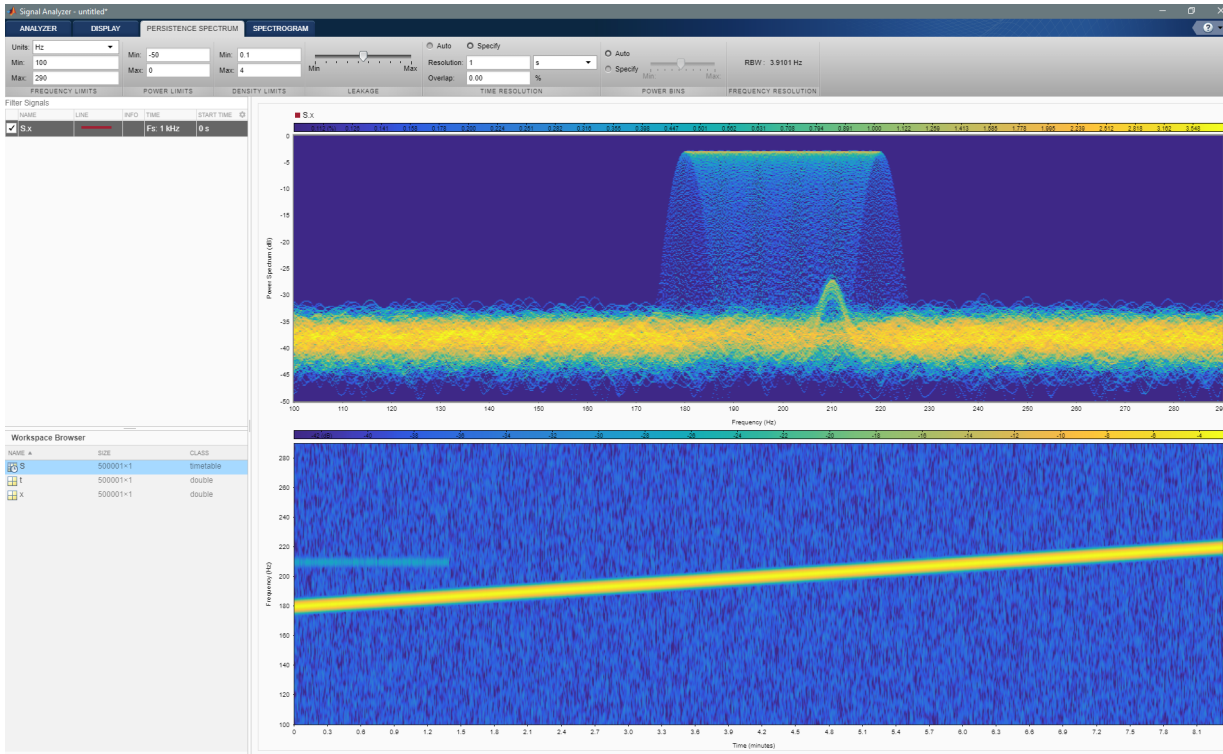
Open **Signal Analyzer** and drag the timetable from the Workspace browser to a display. Click the **Time-Frequency** button to add a spectrogram view. On the **Spectrogram** tab, under **Time Resolution**, select **Specify** and enter a time resolution of 1 second. Set the **Frequency Limits** to 100 Hz and 290 Hz. Both signal components are visible.



Go back to the **Display** tab. Click the **Time** button to remove the time view and click the **Spectrum** button to add a power spectrum view. The frequency range continues to be from 100 Hz to 290 Hz. The weak sinusoid is obscured by the chirp.



Click the **Spectrum ▼** button to change the `Spectrum` view to a `Persistence Spectrum` view. On the **Persistence Spectrum** tab, under **Time Resolution**, select **Specify** and enter a time resolution of 1 second. Specify zero overlap between adjoining segments. Set the **Power Limits** to –50 dB and 0 dB and the **Density Limits** to 0.1 and 4. Now both signal components are clearly visible.



On the **Display** tab, under **Share**, click **Generate Script ▼** and select `Persistence Spectrum Script`. The script appears in the MATLAB Editor.

```
% Compute persistence spectrum

% Generated by MATLAB(R) 9.3 and Signal Processing Toolbox 7.5.
% Generated on: 15-Jul-2017 18:43:37

% Parameters
timeLimits = seconds([0 500]); % seconds
frequencyLimits = [100 290]; % Hz
timeResolution = 1; % seconds
overlapPercent = 0;

% Index into signal time region of interest
S_x_ROI = S(:,'x');
S_x_ROI = S_x_ROI(timerange(timeLimits(1),timeLimits(2)),1);
```

```
% Compute spectral estimate
% Run the function call below without output arguments to plot the results
[P,F,PWR] = pspectrum(S_x_ROI, ...
    'persistence', ...
    'FrequencyLimits',frequencyLimits, ...
    'TimeResolution',timeResolution, ...
    'OverlapPercent',overlapPercent);
```

See Also

Apps
Signal Analyzer

Functions
`pspectrum` | `timetable`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93

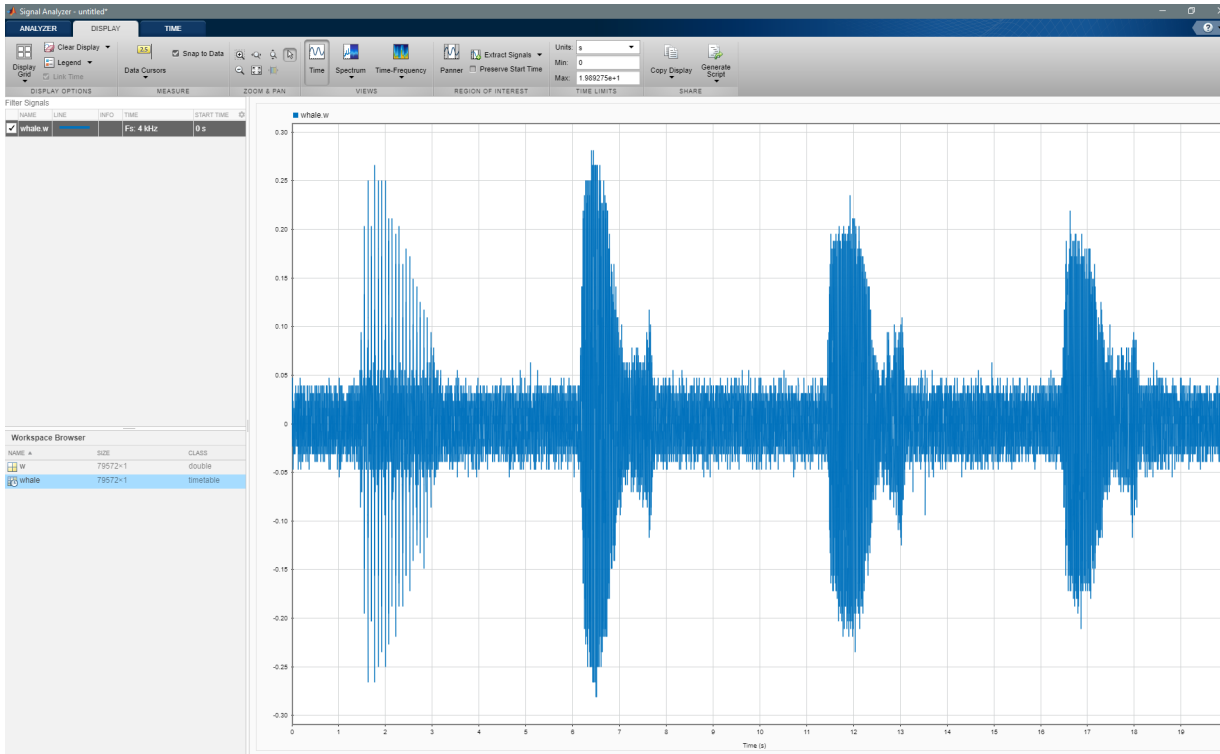
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Extract Regions of Interest from Whale Song

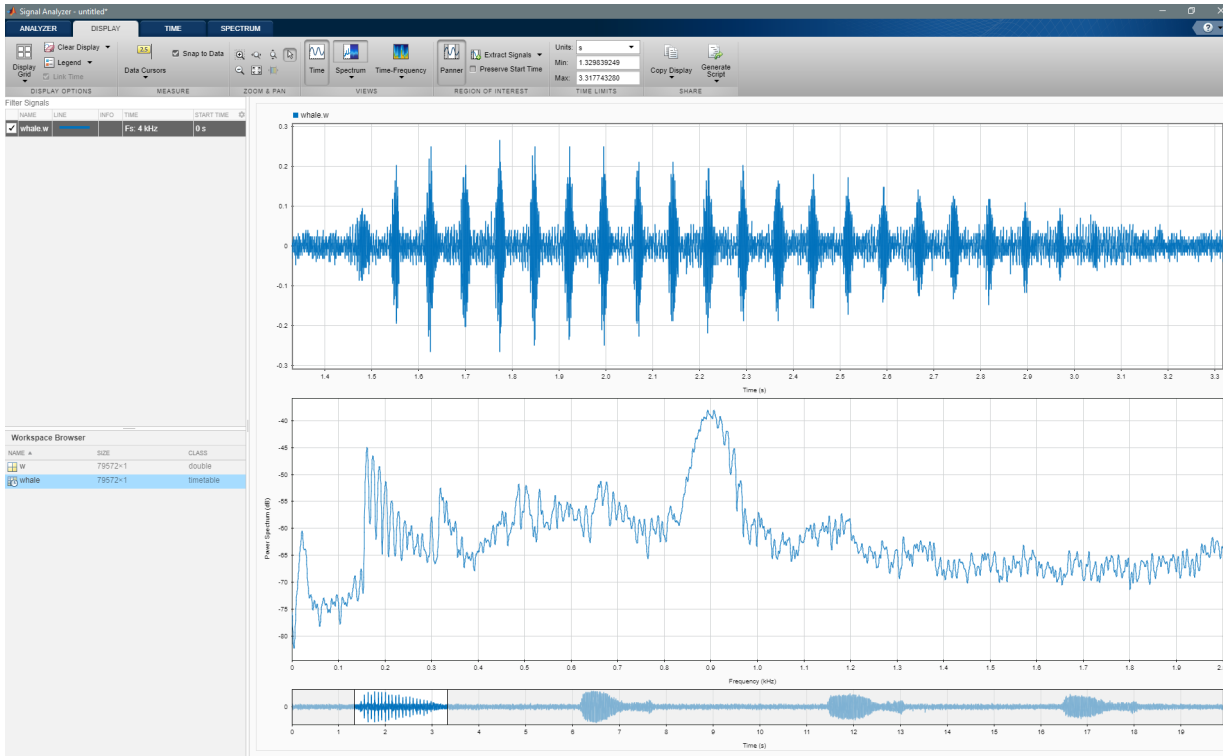
Load a file that contains audio data from a Pacific blue whale, sampled at 4 kHz. The file is from the library of animal vocalizations maintained by the Cornell University Bioacoustics Research Program. The time scale in the data is compressed by a factor of 10 to raise the pitch and make the calls more audible. Convert the signal to a MATLAB® timetable.

```
whaleFile = fullfile(matlabroot,'examples','matlab','bluewhale.au');
[w,fs] = audioread(whaleFile);

whale = timetable(seconds((0:length(w)-1)'/fs),w);

% To hear, type soundsc(w,fs)
```

Open **Signal Analyzer** and drag the timetable to a display. Four features stand out from the noise. The first is known as a *trill*, and the other three are known as *moans*.

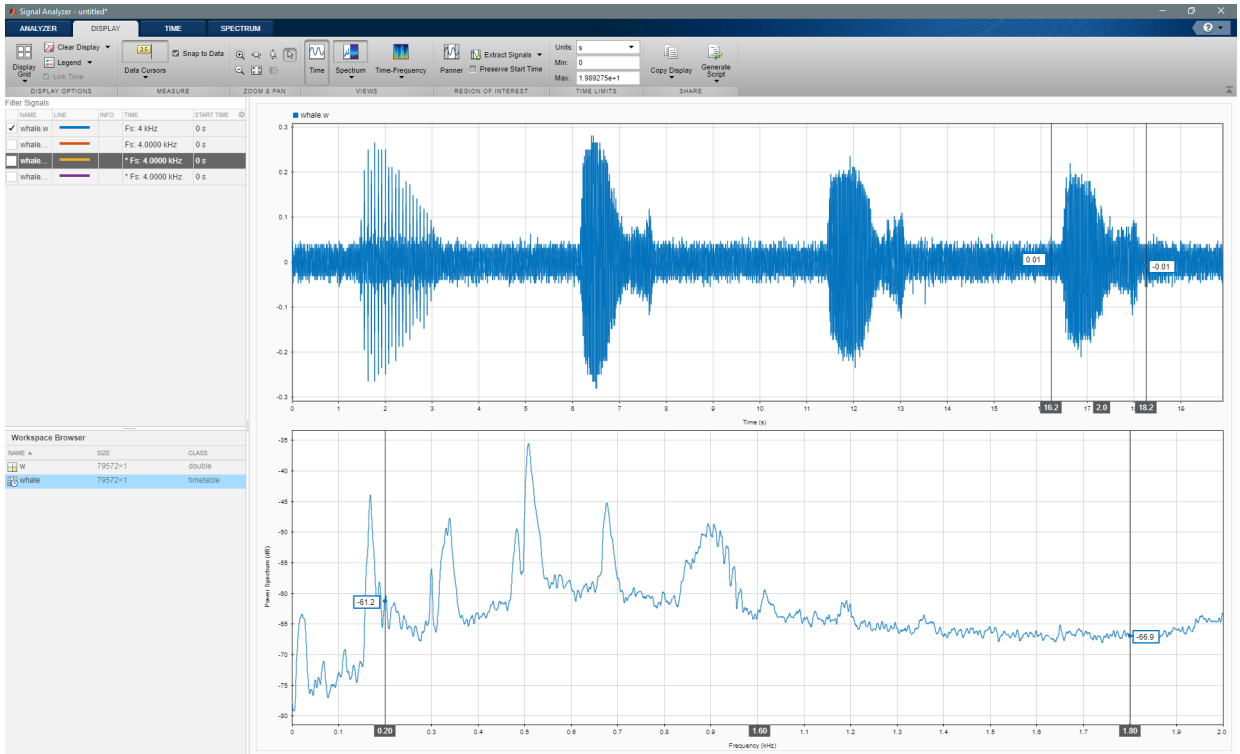


On the **Display** tab, click **Spectrum** to open a spectrum view and click **Panner** to activate the panner. Use the panner to create a zoom window with a width of about 2 seconds. Drag the zoom window so that it is centered on the trill. The spectrum shows a noticeable peak at around 900 Hz.

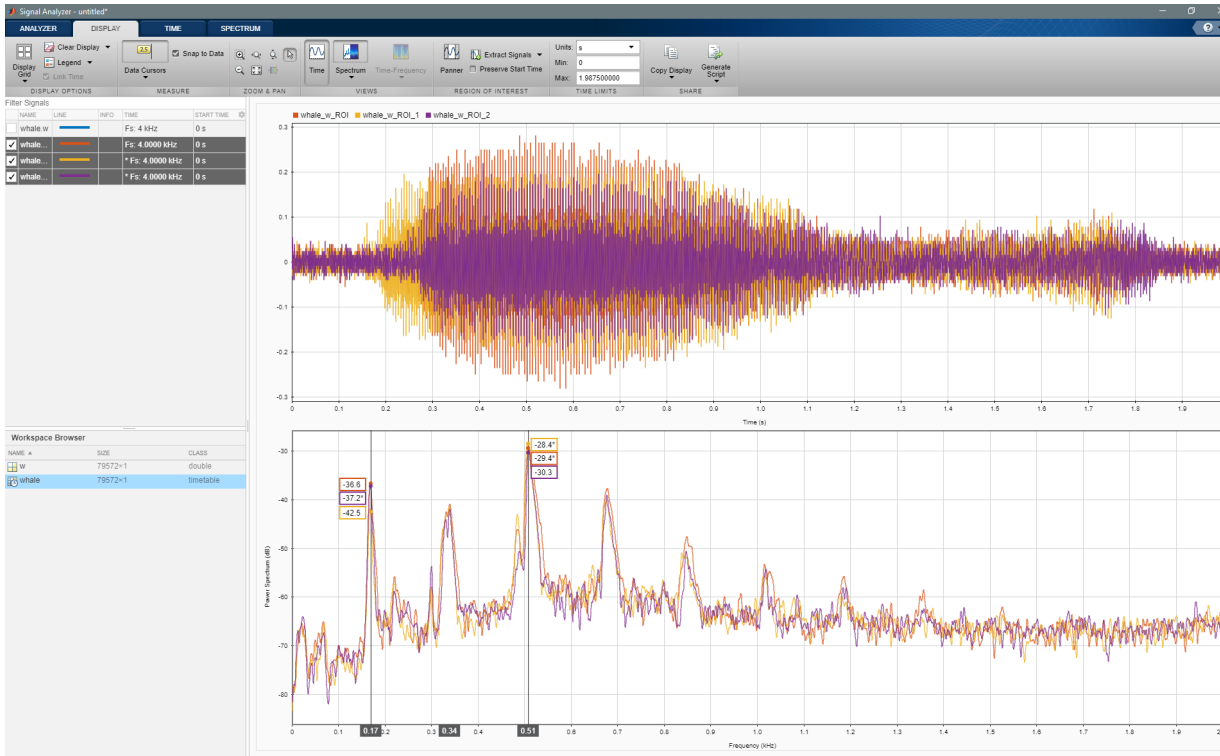


Extract the three moans to compare their spectra:

1. Center the panner zoom window on the first moan. The spectrum has eight clearly defined peaks, located very close to multiples of 170 Hz. Click **Extract Signals ▼** and select `Between Time Limits`.
2. Click **Panner** to hide the panner. Press the space bar to see the full signal. Click **Zoom in X** and zoom in on a 2-second interval of the time view centered on the second moan. The spectrum again has peaks at multiples of 170 Hz. Click **Extract Signals ▼** and select `Between Time Limits`.
3. Press the space bar to see the full signal. Click **Data Cursors ▼** and select `Two`. Place the time-domain cursors in a 2-second interval around the third moan. Again, there are peaks at multiples of 170 Hz. Click **Extract Signals ▼** and select `Between Time Cursors`.



Remove the original signal from the display by clearing the check box next to its name in the Signal table. Display the three regions of interest you just extracted. Their spectra lie approximately on top of each other. Move the frequency-domain cursors to the locations of the first and third spectral peaks. Asterisks in cursor labels indicate interpolated signal values.



See Also

Apps
Signal Analyzer

Functions
`pspectrum` | `timetable`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32

- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

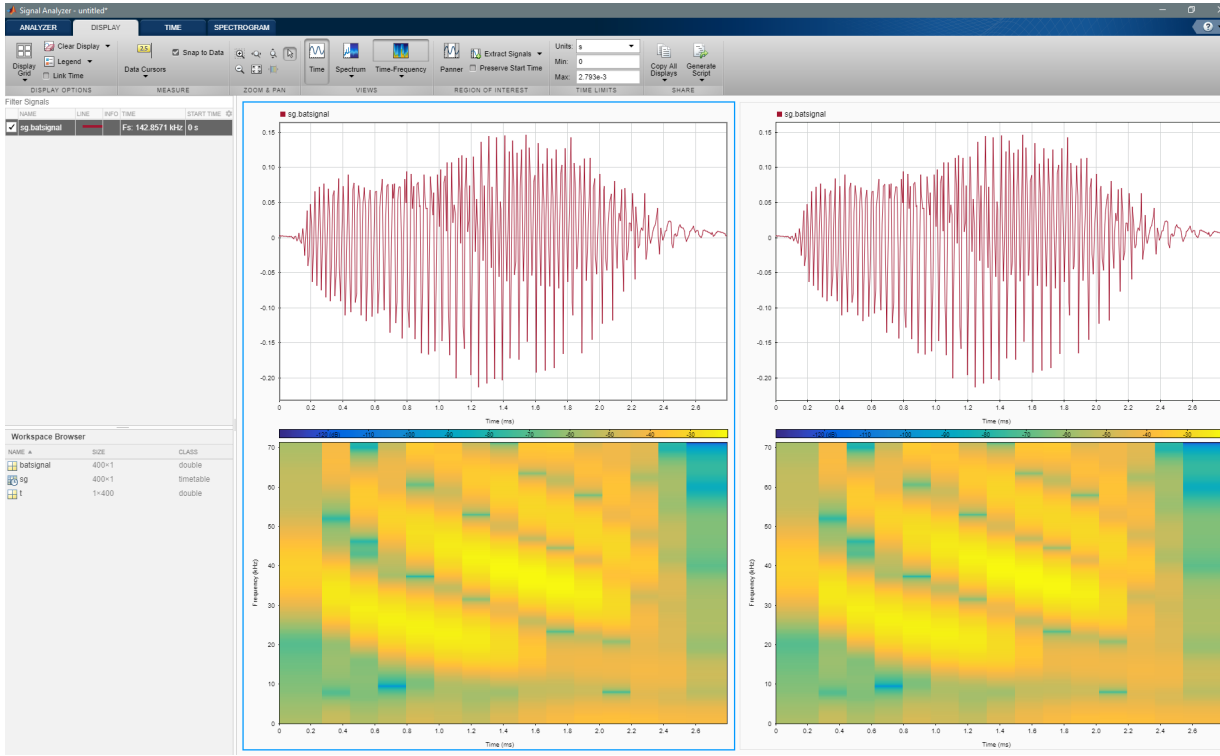
Find and Track Ridges Using Reassigned Spectrogram

Load a datafile containing an echolocation pulse emitted by a big brown bat (*Eptesicus fuscus*) and measured with a sampling interval of 7 microseconds. Create a MATLAB® timetable using the signal and the time information.

```
load batsignal

t = (0:length(batsignal)-1)*DT;
sg = timetable(seconds(t)',batsignal);
```

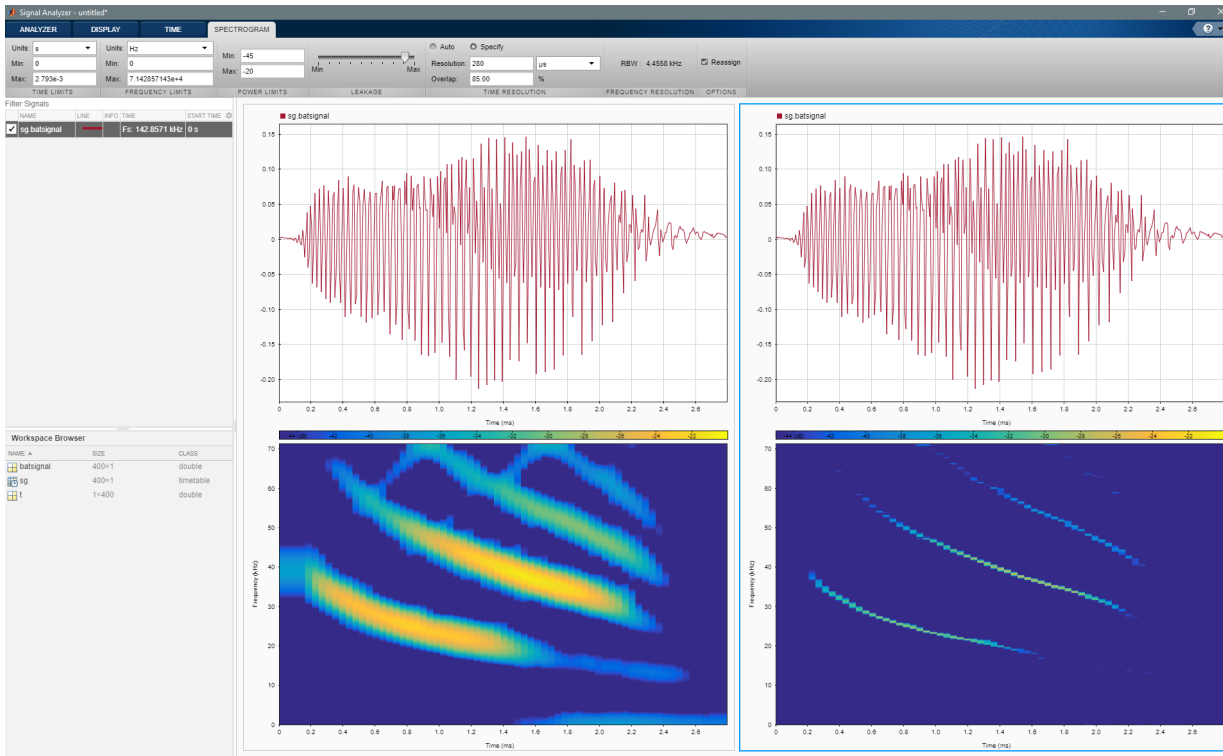
Open **Signal Analyzer** and drag the timetable from the Workspace browser to the Signal table. Click **Display Grid ▼** to create two side-by-side displays. Select each display and click the **Time-Frequency** button to add a spectrogram view.

Drag the timetable to both displays.



Select the **Spectrogram** tab. On the display at right, check **Reassign**. For each display:

- Set the time resolution to 280 microseconds and specify 85% overlap between adjoining segments.
- Use the **Leakage** slider to increase the leakage until the RBW is about 4.5 kHz.
- Set the power limits to –45 dB and –20 dB.



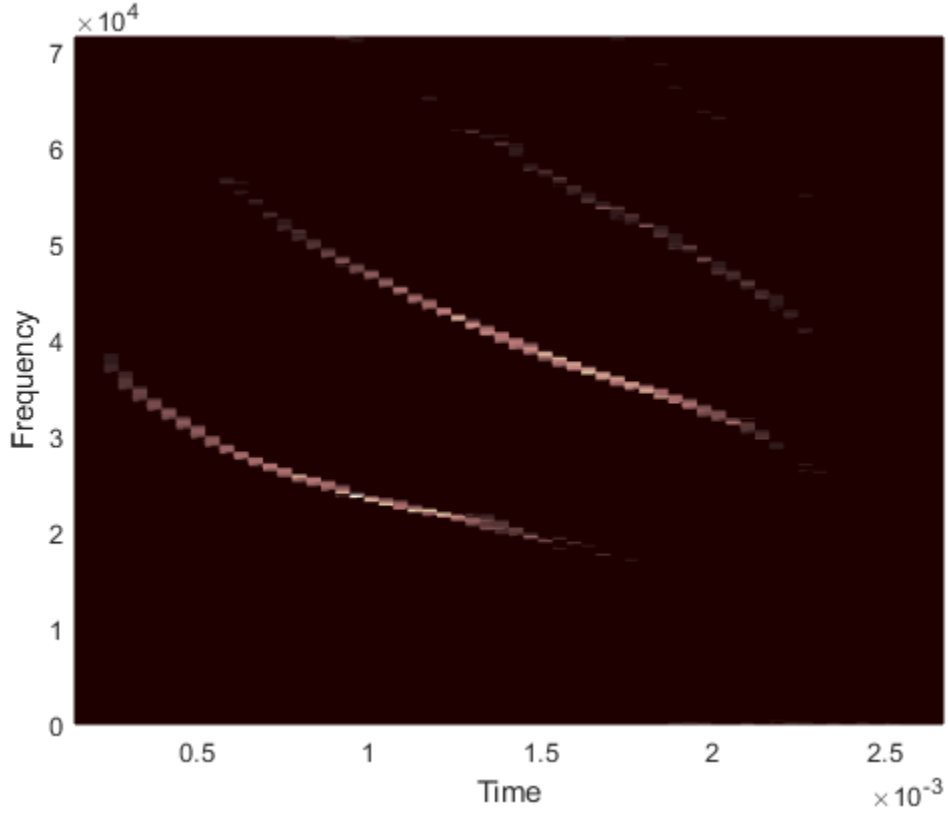
The reassigned spectrogram clearly shows three time-frequency ridges. To track the ridges, select the display at right. On the **Display** tab, click **Generate Script** and select `Spectrogram Script`. The script appears in the Editor.

```
% Compute spectrogram

% Generated by MATLAB(R) 9.3 and Signal Processing Toolbox 7.5.
% Generated on: 13-Jul-2017 19:32:31
```

```
% Parameters
timeLimits = seconds([3.805177e-06 0.002796805]); % seconds
frequencyLimits = [0 71428.57]; % Hz
leakage = 0.9;
timeResolution = 0.00028; % seconds
overlapPercent = 85;
reassignFlag = true;

% Index into signal time region of interest
sg_batsignal_ROI = sg(:,'batsignal');
sg_batsignal_ROI = sg_batsignal_ROI(timerange(timeLimits(1),timeLimits(2)),1);

% Compute spectral estimate
% Run the function call below without output arguments to plot the results
[P,F,T] = pspectrum(sg_batsignal_ROI, ...
    'spectrogram', ...
    'FrequencyLimits',frequencyLimits, ...
    'Leakage',leakage, ...
    'TimeResolution',timeResolution, ...
    'OverlapPercent',overlapPercent, ...
    'Reassign',reassignFlag);
```

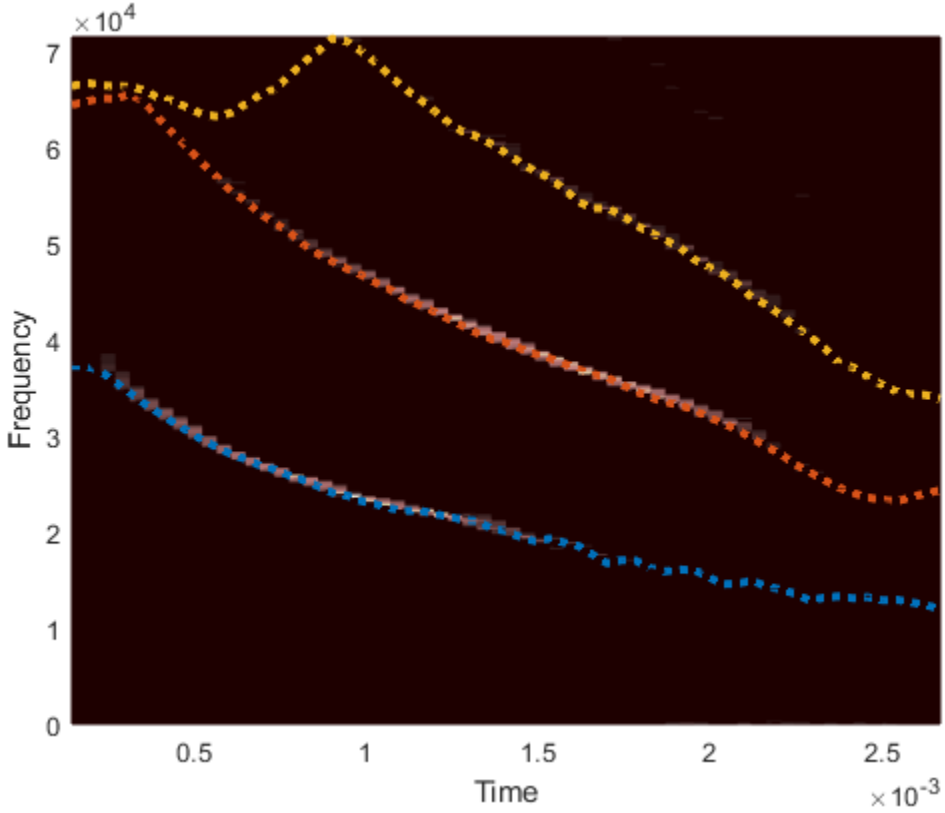
Run the script. Plot the reassigned spectrogram.

```
mesh(seconds(T),F,P)
xlabel('Time')
ylabel('Frequency')
axis tight
view(2)
colormap pink
```



Use the `tfridge` function to track the ridges.

```
[fridge,~,lridge] = tfridge(P,F,0.01,'NumRidges',3,'NumFrequencyBins',10);

hold on
plot3(seconds(T),fridge,P(lridge),':','linewidth',3)
hold off
```



Thanks to Curtis Condon, Ken White, and Al Feng of the Beckman Center at the University of Illinois for the bat data and permission to use it in this example.

See Also

Apps
Signal Analyzer

Functions
`pspectrum` | `tfridge` | `timetable`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Scalogram of Hyperbolic Chirp

Use **Signal Analyzer** to display a hyperbolic chirp and its scalogram. This example requires a Wavelet Toolbox license.

Generate a signal sampled at 1 kHz for 1 second. The signal consists of two hyperbolic chirps. One of the chirps is active between 0.1 second and 0.68 second. The other chirp has twice the amplitude and one-third of the instantaneous frequency of the first chirp. The second chirp is active between 0.1 second and 0.75 second.

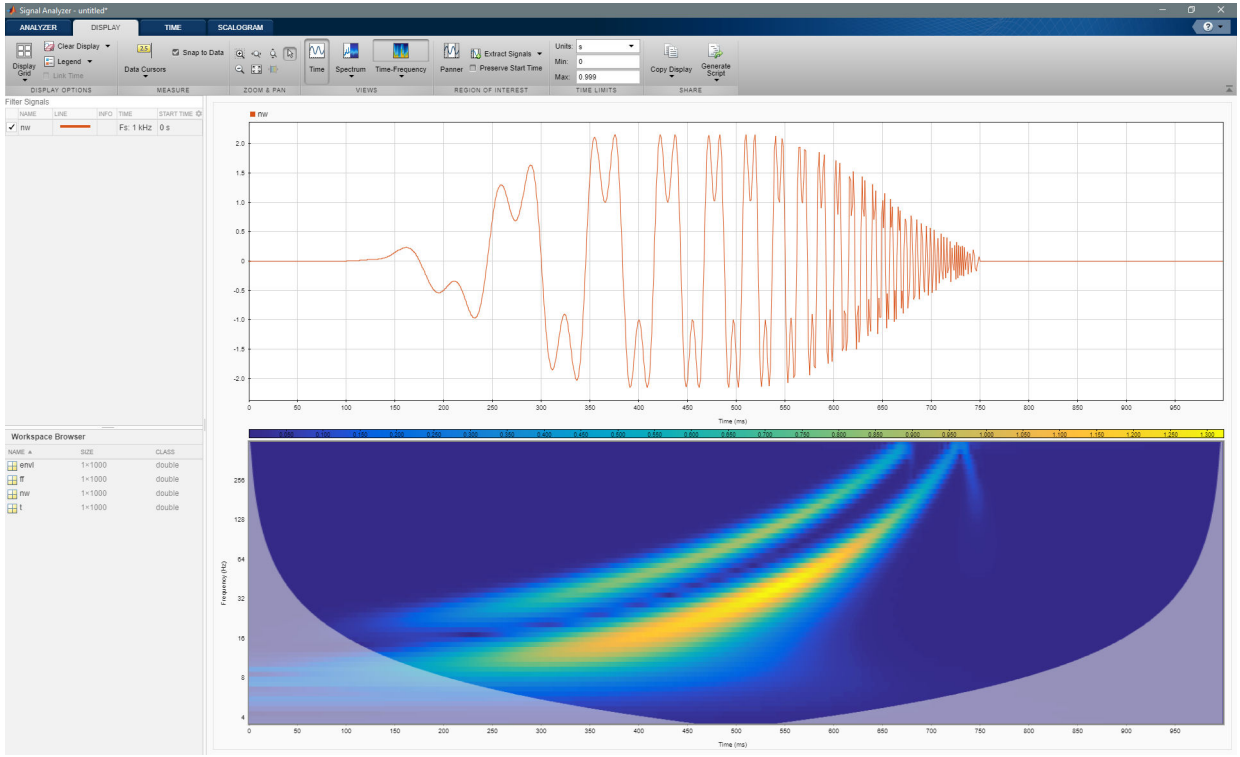
```
n = 1000;
t = (0:n-1)/n;

ff = [1 2]*(sin(pi*[15 5]'./(t-0.8001)).*(t>0.1 & t<[0.68 0.75]'));
```

The chirp is modulated by an envelope that starts at 0.1 second and is nonzero for 0.7 second.

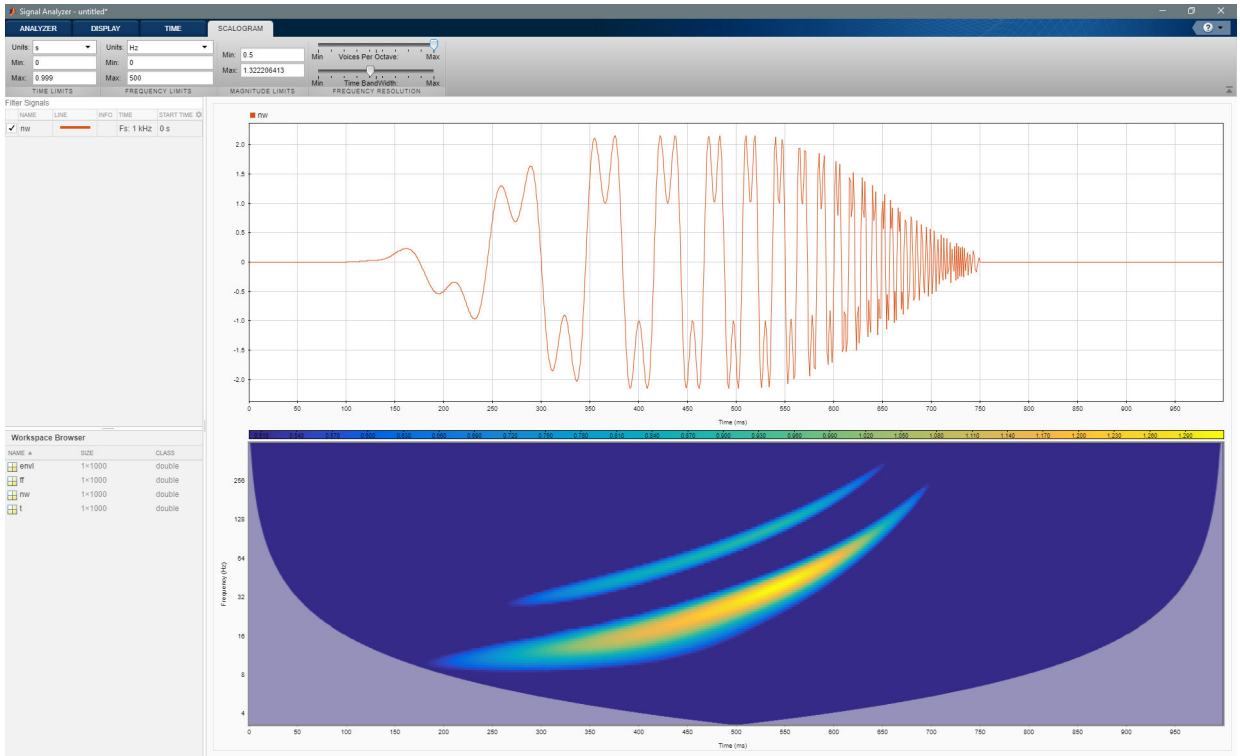
```
envl = zeros(size(t));
M = 0.7*n;
envl(n/10+(1:M)) = tukeywin(M,0.8);

nw = ff.*envl;
```

Display the signal in **Signal Analyzer**. Click **Time-Frequency** ▼ on the **Display** tab and select `Scalogram`.

```
signalAnalyzer(nw,'TimeValues',t)
```



On the **Scalogram** tab, set the **Voices Per Octave** to the maximum value. Set the lower magnitude limit to 0.5.



See Also

Apps
Signal Analyzer

Functions
`cwt` | `cwtfilterbank` | `pspectrum`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36

- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

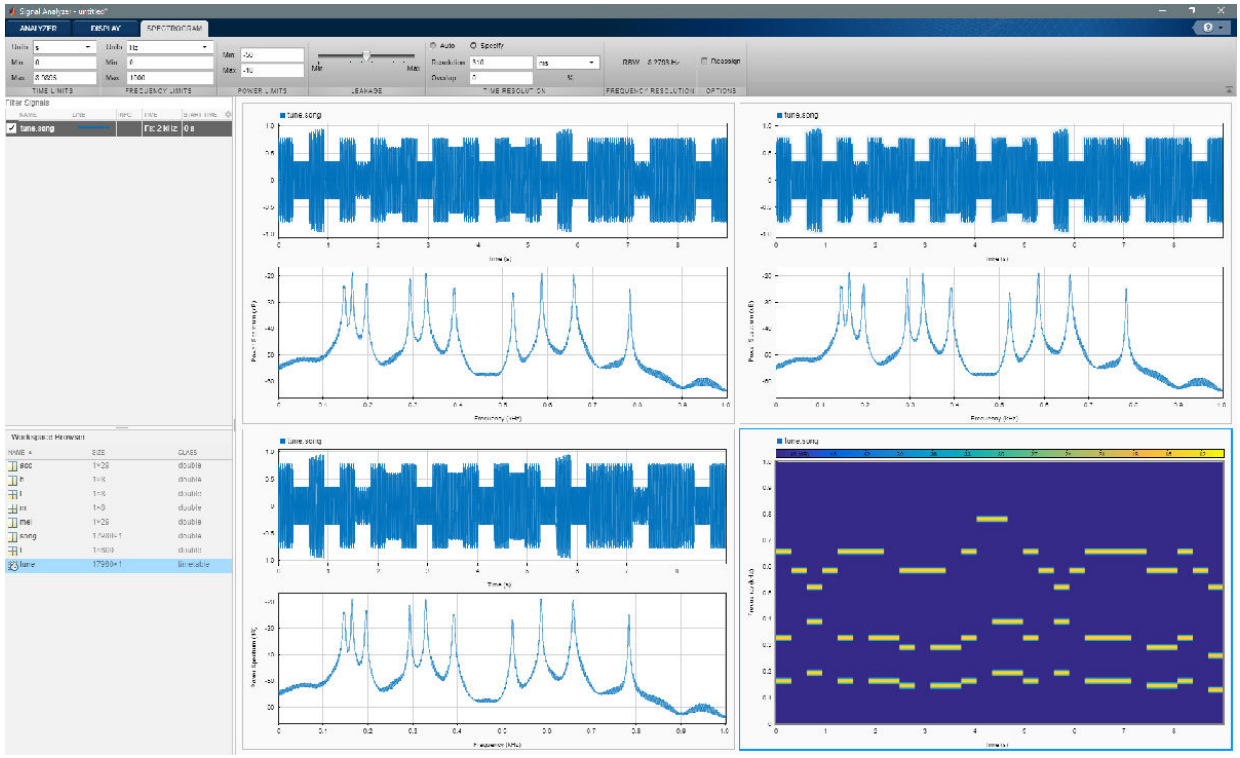
- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Extract Voices from Music Signal

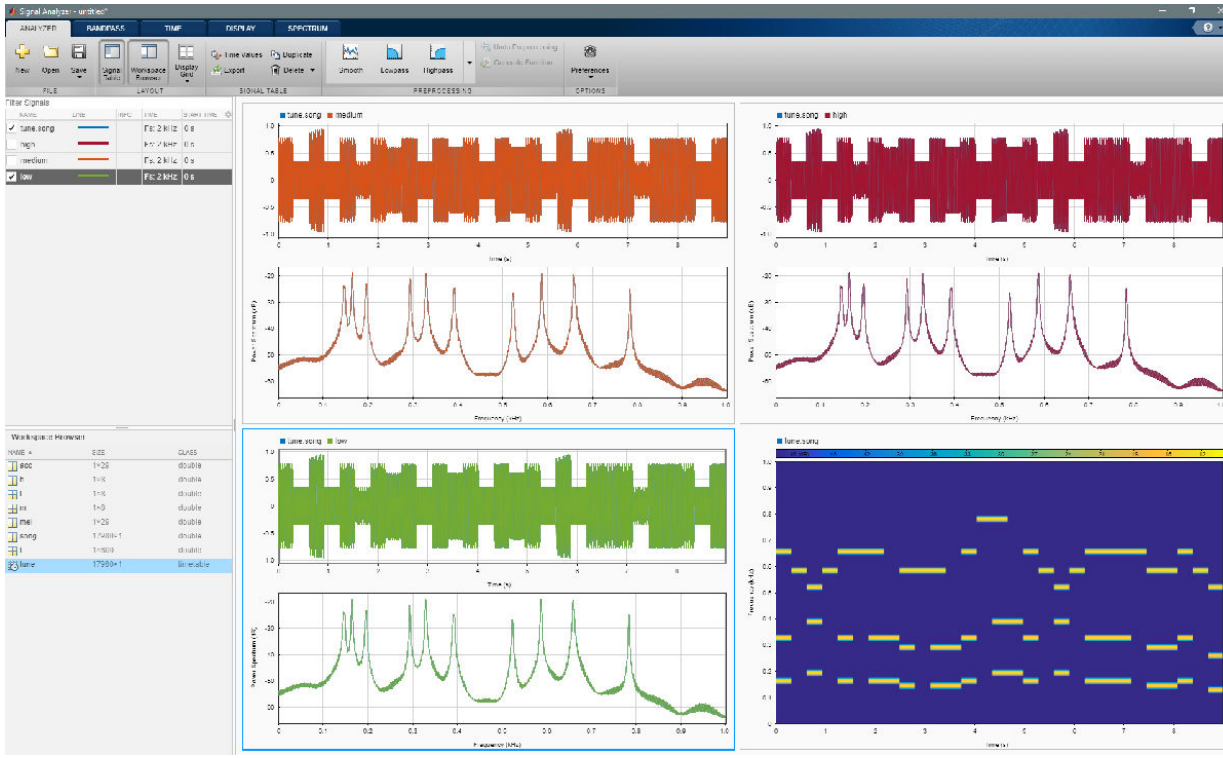
Implement a basic digital music synthesizer and use it to play a traditional song in a three-voice arrangement. Specify a sample rate of 2 kHz. Save the song as a MATLAB® timetable.

```
fs = 2e3;
t = 0:1/fs:0.3-1/fs;

l = [0 130.81 146.83 164.81 174.61 196.00 220 246.94];
m = [0 261.63 293.66 329.63 349.23 392.00 440 493.88];
h = [0 523.25 587.33 659.25 698.46 783.99 880 987.77];
note = @(f,g) [1 1 1]*sin(2*pi*[l(g) m(g) h(f)]'.*t);

mel = [3 2 1 2 3 3 3 0 2 2 2 0 3 5 5 0 3 2 1 2 3 3 3 3 2 2 3 2 1]+1;
acc = [3 0 5 0 3 0 3 3 2 0 2 2 3 0 5 5 3 0 5 0 3 3 3 0 2 2 3 0 1]+1;

song = [];
for kj = 1:length(mel)
    song = [song note(mel(kj),acc(kj)) zeros(1,0.01*fs)];
end
song = song'/(max(abs(song))+0.1);

% To hear, type sound(song,fs)

tune = timetable(seconds((0:length(song)-1)'/fs),song);
```

Open **Signal Analyzer** and drag the timetable from the Workspace browser to the Signal table. Click **Display Grid ▼** to create a two-by-two grid of displays. Select the top two displays and the lower left display and click the **Spectrum** button to add a spectrum view. Select the lower right display, click **Time-Frequency** to add a spectrogram view, and click **Time** to remove the time view. Drag the song to all four displays. Select the lower right display, and in the **Spectrogram** tab, specify a time resolution of 0.31 second (310 ms) and 0% overlap between adjoining segments. Set the **Power Limits** to -50 dB and -10 dB.

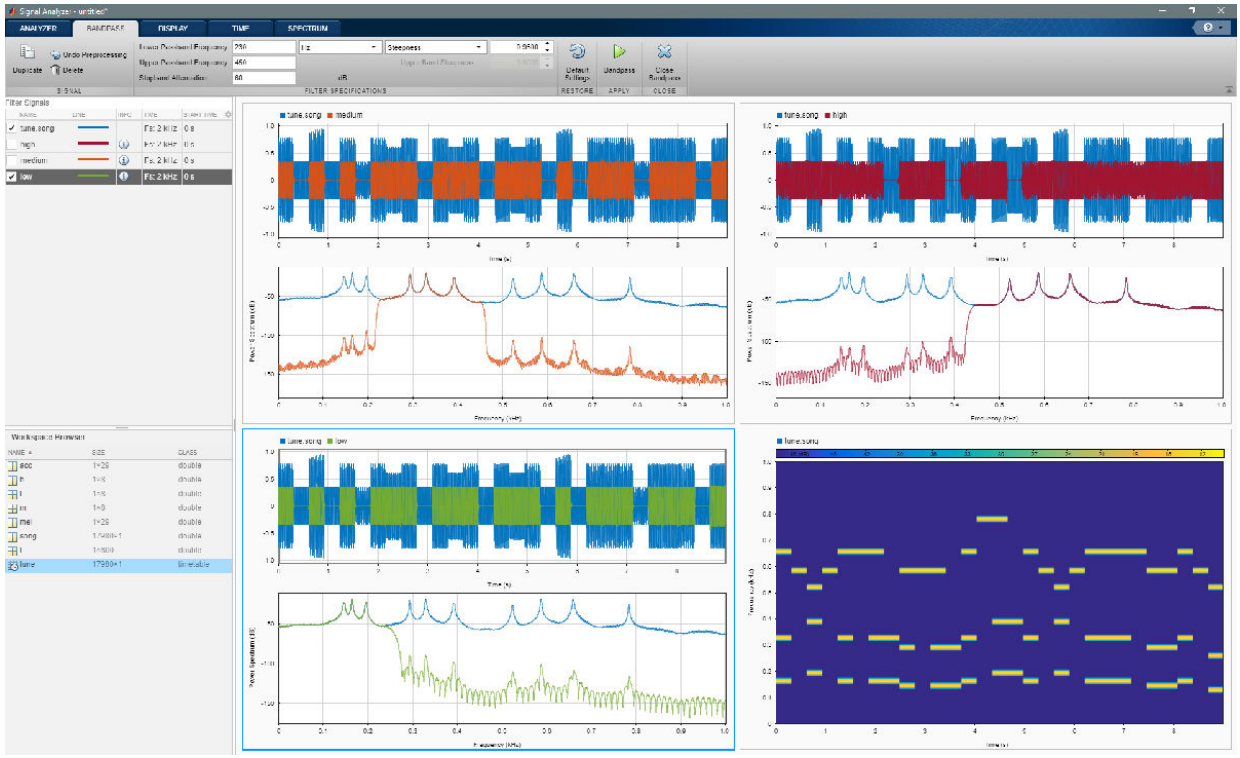


On the **Analyzer** tab, click **Duplicate** three times to create three copies of the song. Rename the copies as `high`, `medium`, and `low` by double-clicking the **Name** column in the Signal table. Move the copies to the top two and lower left displays.



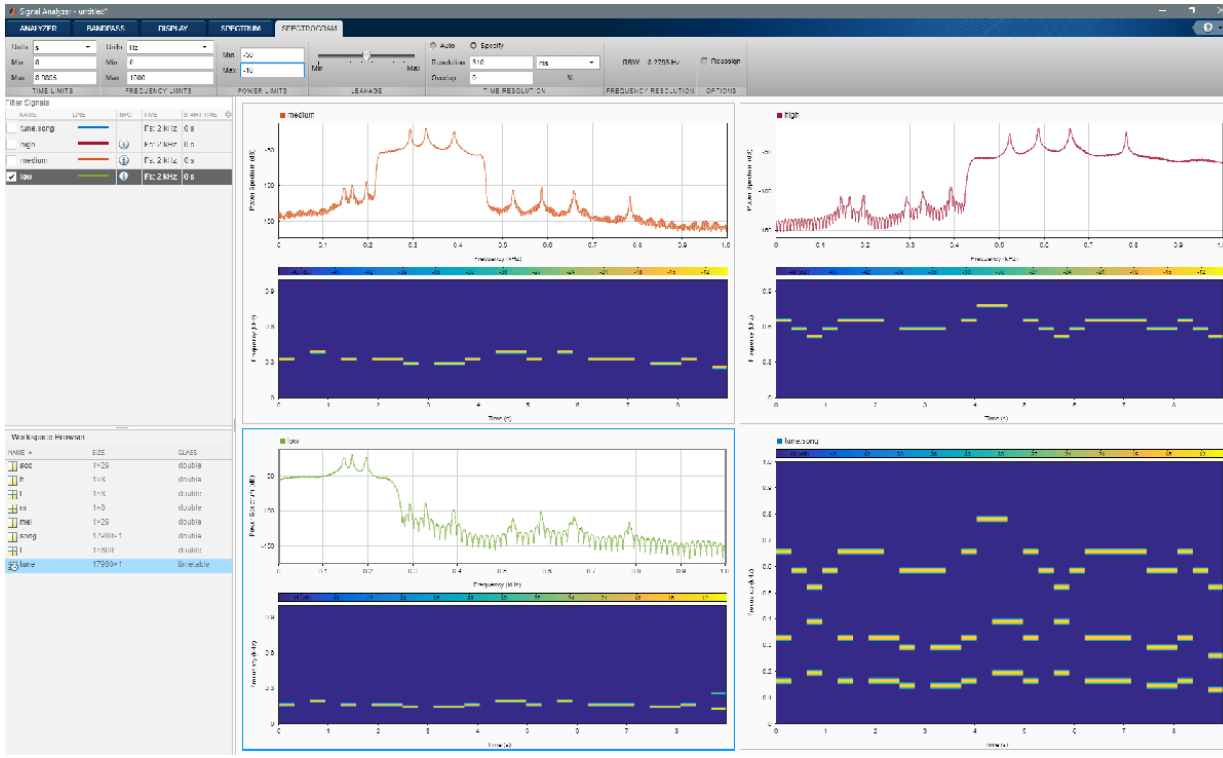
Preprocess the duplicate signals using filters.

1. Select the `high` signal by clicking its name in the Signal table. On the **Analyzer** tab, click **Highpass**. On the **Highpass** tab that appears, enter a passband frequency of 450 Hz and increase the steepness to 0.95. Click **Highpass**.
2. Select the `medium` signal by clicking its name in the Signal table. On the **Analyzer** tab, click **Preprocessing ▼** and select **Bandpass**. On the **Bandpass** tab that appears, enter 230 Hz and 450 Hz as the lower and upper passband frequencies, respectively. Increase the steepness to 0.95. Click **Bandpass**.
3. Select the `low` signal by clicking its name in the Signal table. On the **Analyzer** tab, click **Lowpass**. On the **Lowpass** tab that appears, enter a passband frequency of 230 Hz and increase the steepness to 0.95. Click **Lowpass**.



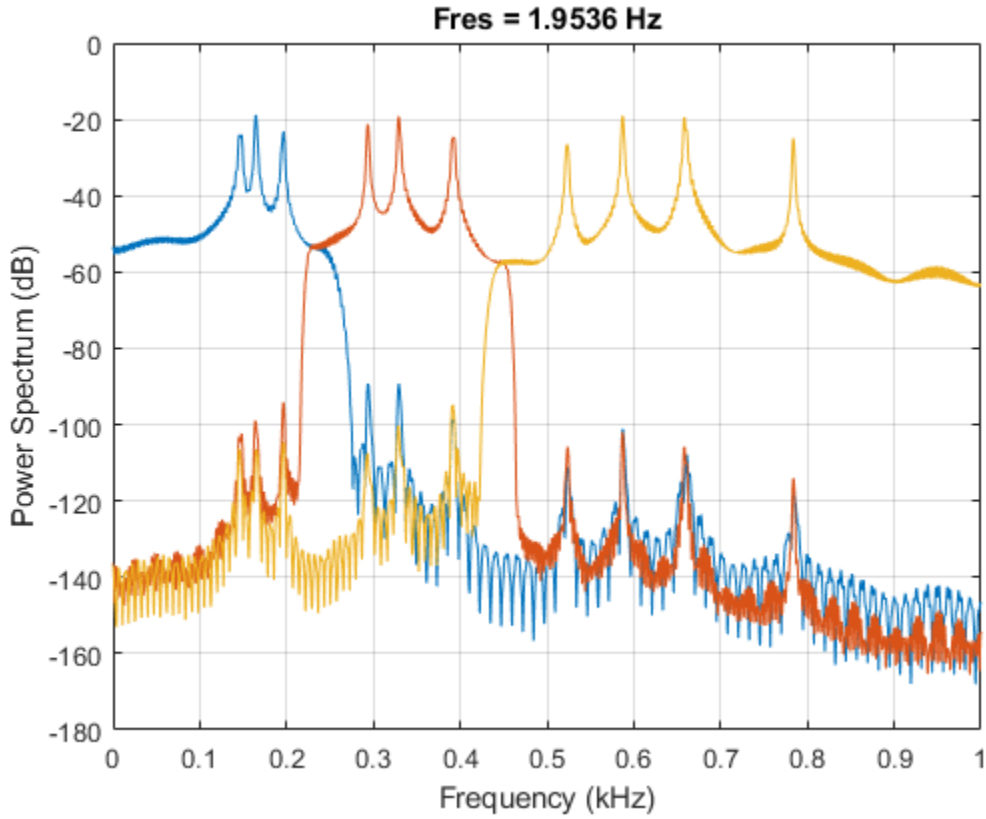
On each of the three displays containing filtered signals:

1. Remove the original signal by clearing the check box next to its name.
2. On the **Display** tab, click **Time-Frequency** to add a spectrogram view and click **Time** to remove the time view.
3. On the **Spectrogram** tab, specify a time resolution of 0.31 second and 0% overlap between adjoining segments. Set the **Power Limits** to -50 dB and -10 dB.



Select the three filtered signals by clicking their **Name** column in the Signal table. On the **Analyzer** tab, click **Export** and save the signals to a MAT-file called `music.mat`. In MATLAB, load the file to the workspace. Plot the spectra of the three signals.

```
load music

pspectrum(low)
hold on
pspectrum(medium)
pspectrum(high)
hold off
```




```
% To hear the different voices, type
% sound(low.Var1,fs), pause(5), sound(medium.Var1,fs), pause(5), sound(high.Var1,fs)
```

See Also

Apps
Signal Analyzer

Functions
`bandpass` | `bandstop` | `highpass` | `lowpass` | `pspectrum`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Resample and Filter a Nonuniformly Sampled Signal

A person recorded their weight in pounds during the leap year 2012. The person did not record their weight every day, so the data are nonuniform. Use the **Signal Analyzer** app to preprocess and study the recorded weight. The app enables you to fill in the missing data points by interpolating the signal to a uniform grid. (This procedure gives the best results if the signal has only small gaps.)

Load the data and convert the measurements to kilograms. The data file has the missing readings set to `NaN`. There are 27 data points missing, most of them during a two-week stretch in August.

```
wt = datetime(2012,1,1:366)';

load weight2012.dat
wgt = weight2012(:,2)/2.20462;

validpoints = ~isnan(wgt);
missing = wt(~validpoints);
missing(15:26)
```

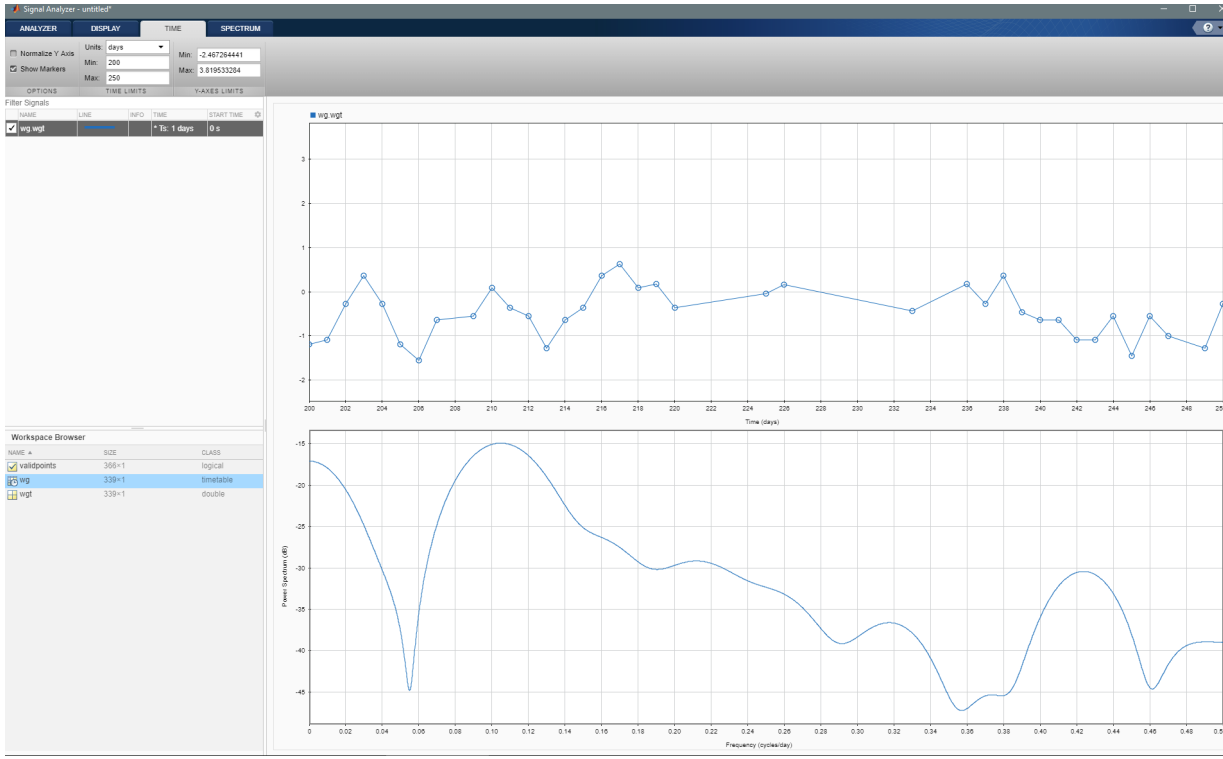
```
ans = 12x1 datetime array
   09-Aug-2012
   10-Aug-2012
   11-Aug-2012
   12-Aug-2012
   15-Aug-2012
   16-Aug-2012
   17-Aug-2012
   18-Aug-2012
   19-Aug-2012
   20-Aug-2012
   22-Aug-2012
   23-Aug-2012
```

Store the data in a MATLAB® timetable. Remove the missing points. Remove the DC value to concentrate on fluctuations. Convert the time information to a `duration` array by subtracting the first time point. For more details, see “Data Types Supported by Signal Analyzer” on page 19-99.

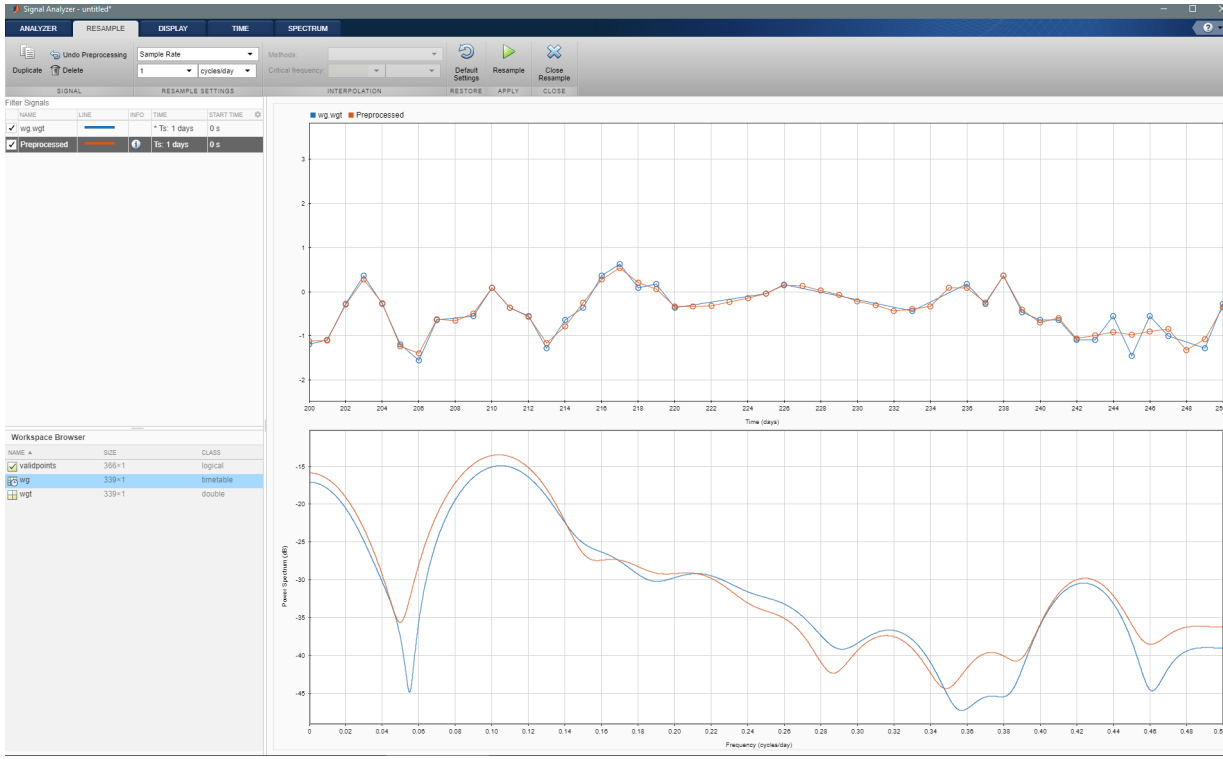
```
wgt = wgt(validpoints);
wgt = wgt - mean(wgt);
```

```
wt = wt(validpoints);
wt = wt - wt(1);

wg = timetable(wt,wgt);
```

Open **Signal Analyzer** and drag the timetable to a display. On the **Display** tab, click **Spectrum** to open a spectrum view. On the **Time** tab, select **Show Markers**. Zoom into the missing stretch by setting the **Time Limits** to 200 and 250 days.

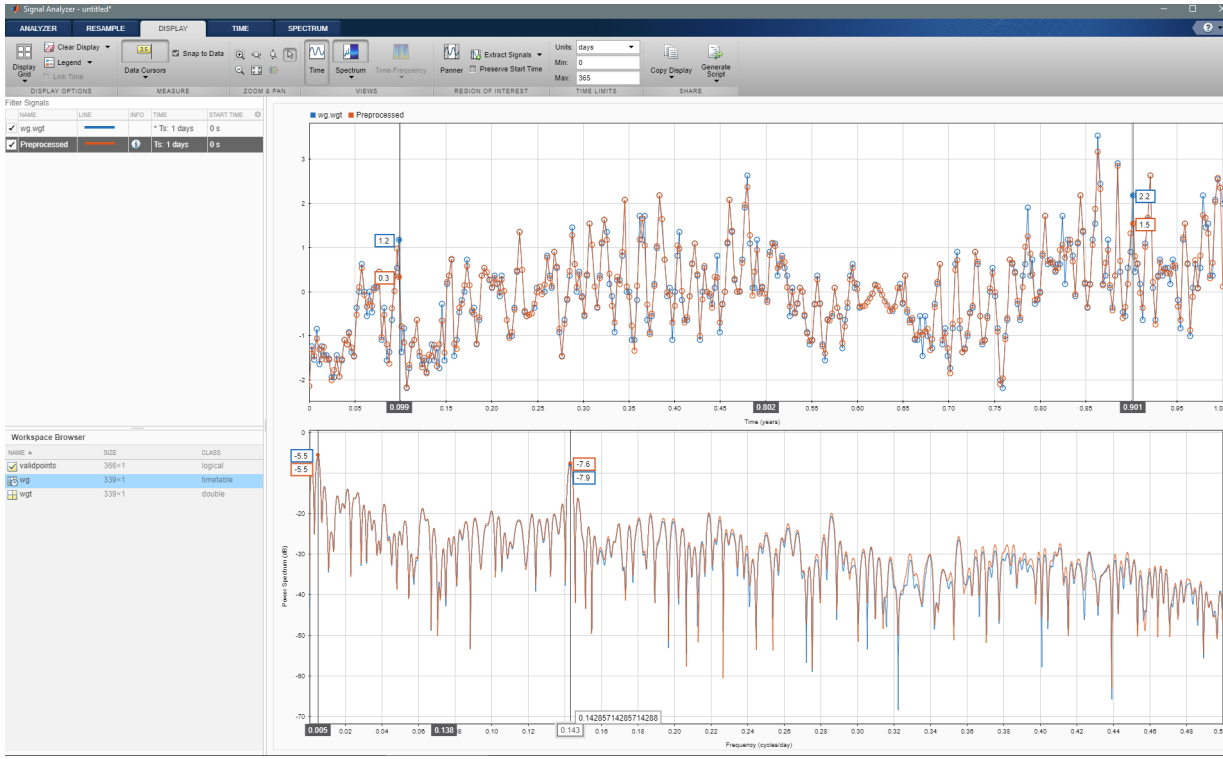


Right-click the signal in the Signal table and select `Duplicate`. Rename the copy as `Preprocessed` by double-clicking the **Name** column in the Signal table. Leave the `Preprocessed` signal selected. On the **Analyzer** tab, click **Preprocessing ▼** and select **Resample**. On the **Resample** tab that appears, enter a sample rate of `1 cycles/day` and select the `Shape Preserving Cubic` method. Click **Resample**. Overlay the resampled signal on the display by selecting the check box next to its name.



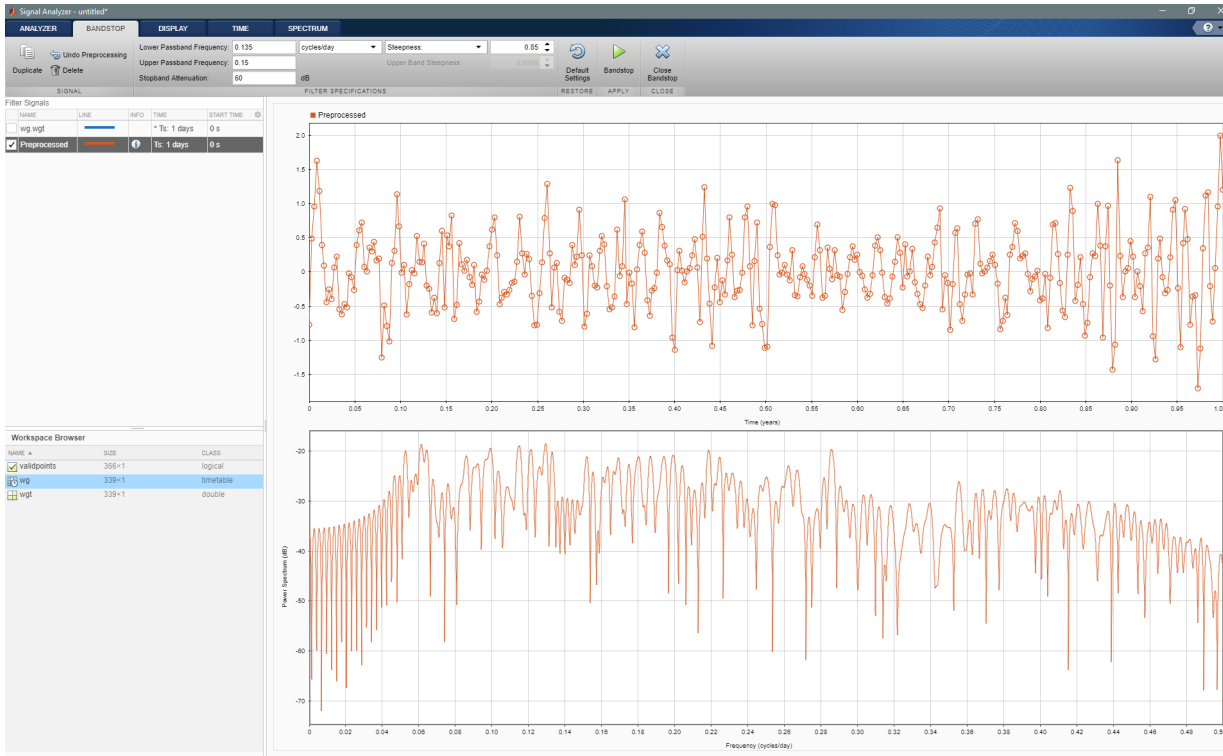
Zoom out to reveal the data for the whole year. On the **Spectrum** tab, set the leakage to the maximum value. The spectra of the original and resampled signals agree well for most frequencies. The spectrum shows two noticeable peaks, one at around 0.14 cycles/day and the other at very low frequencies. To locate the peaks better, click **Data Cursors ▼** and select `Two`. Place the cursors on the peaks. Hover over the frequency field of each cursor to get a more precise value of its location.

- The medium-frequency peak is at 0.142857 = 1/7 cycles/day, which corresponds to a one-week cycle.
- The low-frequency peak is at 0.004762 cycles/day, which corresponds to a 210-day cycle.

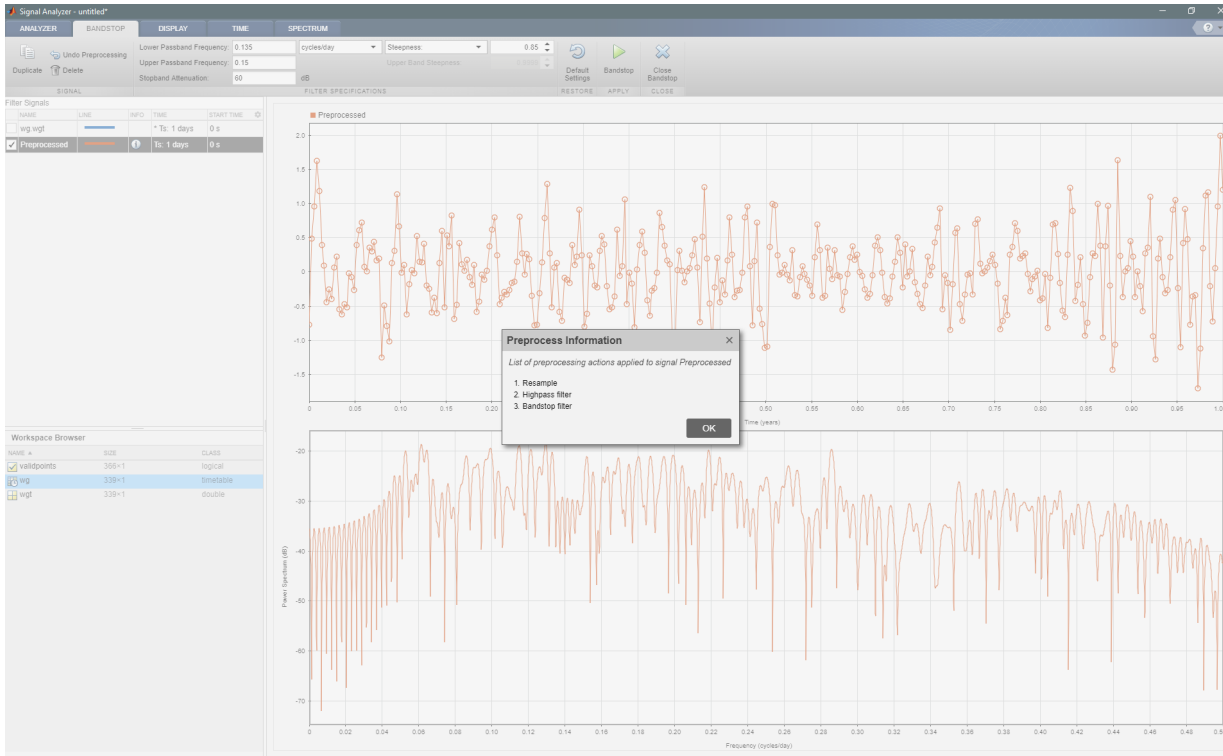


Remove the cursors by clicking the **Data Cursors** icon. Remove the original signal from the display. Filter the `Preprocessed` signal to remove the effects of the cycles.

1. To remove the low-frequency cycle, highpass-filter the signal. On the **Analyzer** tab, select **Highpass**. On the **Highpass** tab that appears, enter a passband frequency of `0.05 cycles/day`. Use the default values of the other parameters. Click **Highpass**.
2. To remove the weekly cycle, bandstop-filter the signal. On the **Analyzer** tab, click **Preprocessing ▼** and select **Bandstop**. On the **Bandstop** tab that replaces the **Highpass** tab, enter a lower passband frequency of `0.135 cycles/day` and a higher passband frequency of `0.15 cycles/day`. Use the default values of the other parameters. Click **Bandstop**.



The preprocessed signal shows less fluctuation than the original. The shape of the signal suggests the person's weight varies less in the summer months than in winter, but that may be an artifact of the resampling. Click the icon on the **Info** column in the Signal table entry for the `Preprocessed` signal to see the preprocessing steps performed on it.



To see a full summary of the preprocessing steps, including all the settings you chose, click **Generate Function** on the **Analyzer** tab. The generated function appears in the MATLAB® Editor.

```
function [y,ty] = preprocess(x,tx)
%  Preprocess input x
%    This function expects an input vector x and a vector of time values
%    tx. tx is a numeric vector in units of seconds.
%    Follow the timetable documentation (type 'doc timetable' in
%    command line) to learn how to index into a table variable and its time
%    values so that you can pass them into this function.

% Generated by MATLAB(R) 9.5 and Signal Processing Toolbox 8.1.
% Generated on: 08-Jun-2018 14:35:38

targetSampleRate = 1.1574074074074073e-05;
[y,ty] = resample(x,tx,targetSampleRate,'pchip');
```

```
Fs = 1/mean(diff(ty)); % Average sample rate
y = highpass(y,5.787e-07,Fs,'Steepness',0.85,'StopbandAttenuation',60);
y = bandstop(y,[1.5625e-06 1.73611111111111e-06],Fs,'Steepness',0.85,'StopbandAttenuat
end
```

See Also

Apps
Signal Analyzer

Functions
`bandstop` | `highpass` | `resample`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107

- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

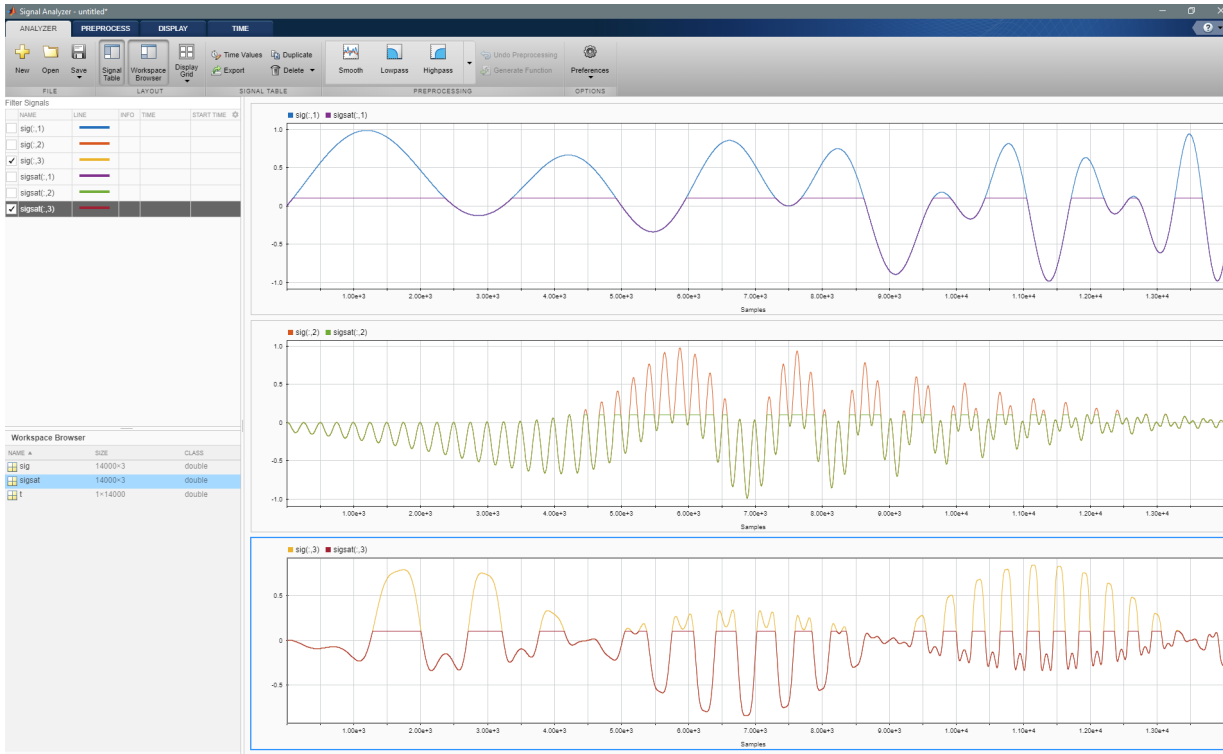
Declip Saturated Signals Using Your Own Function

Sensors can return clipped readings if the data are larger than a given saturation point. To reconstruct the readings, you can fit a polynomial through the points adjacent to the saturated intervals. Write a function that performs the reconstruction and integrate it into **Signal Analyzer**.

Generate a three-channel signal sampled at 1 kHz for 14 seconds. The signal has several peaks of varying sizes and shapes. A sensor that reads the signal saturates at 0.1 V.

```
fs = 1000;
t = 0:1/fs:14-1/fs;

sig = [chirp(t-1,0.1,17,2,'quadratic',1).*sin(2*pi*t/5);
    chirp(t-2,2,2,2.1,'quadratic',100).*exp(-(t-6.5).^2/20).*sin(2*pi*t*2);
    0.85*besselj(0,5*(sin(2*pi*(t+1.5).^2/20).^2)).*sin(2*pi*t/9)]';

sigsat = sig;
stv = 0.1;
sigsat(sigsat >= stv) = stv;
```

Open **Signal Analyzer** and drag the original signal and the saturated signal to the Signal table. Drag each original and saturated channel to its own display.



Write a function that uses a polynomial to reconstruct the signal peaks:

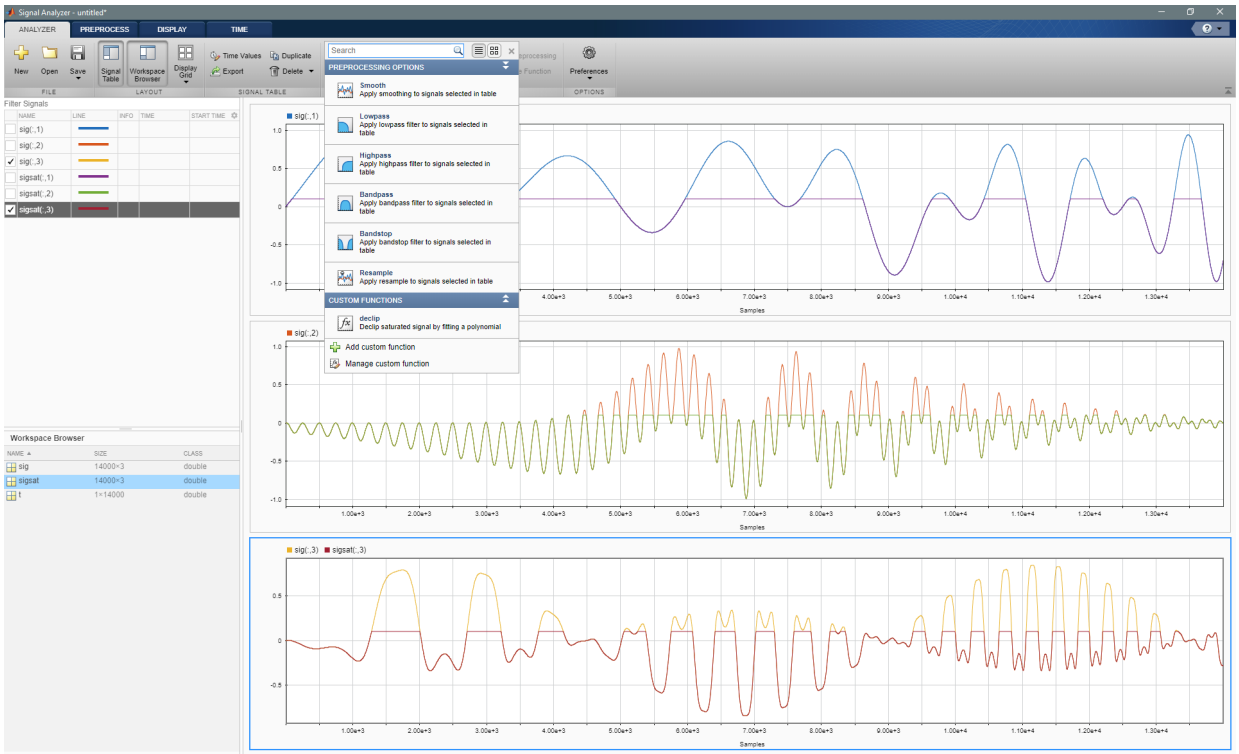
- The first input argument, `x`, is the input signal. This argument must be a vector and is treated as a single channel.
- The second input argument, `tIn`, is a vector of time values. The vector must have the same length as the signal. If the input signal has no time information, the function reads this argument as an empty array.
- Use `varargin` to specify additional input arguments. If you do not have additional input arguments, you can omit `varargin`. Enter the additional arguments as an ordered comma-separated list in the **Preprocess** tab.
- The first output argument, `y`, is the preprocessed signal.
- The second output argument, `tOut`, is a vector of output time values. If the input signal has no time information, `tOut` is returned as an empty array.

- To implement your algorithm, you can use any MATLAB® or Signal Processing Toolbox™ function.

```
function [y,tOut] = declip(x,tIn,varargin)
% Declip saturated signal by fitting a polynomial

    % Initialize the output signal

    y = x;

    % For signals with no time information, use sample numbers as abscissas

    if isempty(tIn)
        tOut = [];
        t = (1:length(x))';
    else
        t = tIn;
        tOut = t;
    end

    % Specify the degree of the polynomial as an optional input argument
    % and provide a default value of 4

    if nargin<3
        ndx = 4;
    else
        ndx = varargin{1};
    end

    % To implement your algorithm, you can use any MATLAB or Signal
    % Processing Toolbox function

    % Find the intervals where the signal is saturated and generate an
    % array containing the interval endpoints
    idx = find(x==max(x));
    fir = [true;diff(idx)~=1];
    ide = [idx(fir) idx(fir([2:end 1]))];
    % For each interval, fit a polynomial of degree ndx over the ndx+1 points
    % before the interval and the ndx+1 points after the interval
    for k = 1:size(ide,1)
        bef = ide(k,1); aft = ide(k,2);
        intv = [bef-1+(-ndx:0) aft+1+(0:ndx)];
        [pp,~,mu] = polyfit(t(intv),x(intv),ndx);
        y(bef:aft) = polyval(pp,t(bef:aft),[],mu);
```

```
    end

end
```

Add the function to **Signal Analyzer** as a custom preprocessing function. On the **Analyzer** tab, click **Preprocessing** ▼ and select **Add Custom Function**. Input the function name and description. Paste the text of your function in the editor window that appears. Save the file. The function appears in the preprocessing gallery.



Demonstrate that the function you created reconstructs the saturated regions.

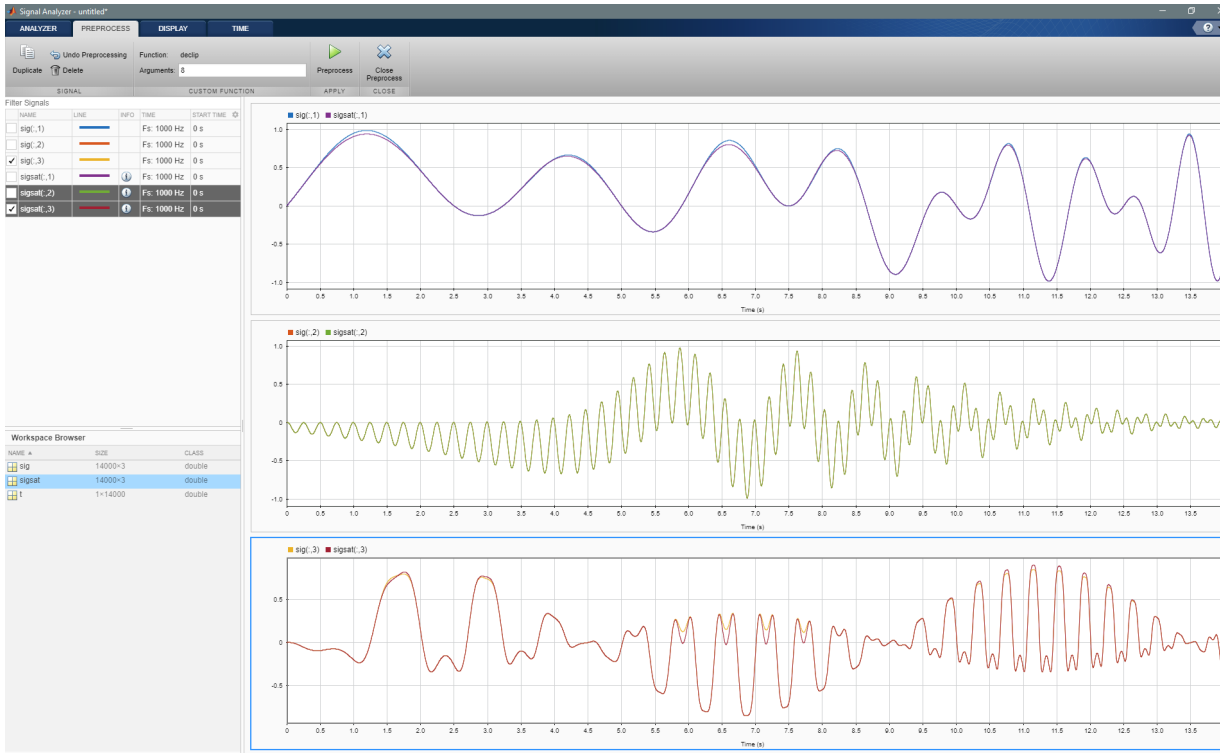
1. Select the first channel of the saturated signal in the Signal table.
2. On the **Analyzer** tab, click **Preprocessing** ▼ and select **declip**.
3. On the **Preprocessing** tab that appears, click **Preprocess**.

Verify that the preprocessing function works when the signals have time information.

1. Select all signals in the Signal table.
2. On the **Analyzer** tab, click **Time Values**, select `Sample Rate and Start Time`, and specify `fs` as the sample rate.
3. The signal in the top panel, including the reconstructed regions, has time information.

Check that the function works when you specify optional inputs.

1. Select the second and third channels of the saturated signal in the Signal table.
2. On the **Preprocessing** tab, enter 8 in the **Arguments** field and click **Preprocess**. The preprocessing function uses a polynomial of degree 8 to reconstruct the saturated regions.



See Also

Apps
Signal Analyzer

Functions
`polyfit` | `polyval` | `varargin`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Edit Sample Rate and Other Time Information

You can add and edit the time information in the **Signal Analyzer** app for any signal that is not a `timetable` or a `timeseries` object. Select one or more signals with no inherent time information in the Signal table and on the **Analyzer** tab, click **Time Values**.

Note Select a signal in the Signal table by clicking its **Name** column. The complete row is highlighted, indicating that the signal is selected. The check box next to the name of the signal indicates whether or not the signal is plotted in the active display.

In the **Time Values** dialog box, select a **Time Specification** option.

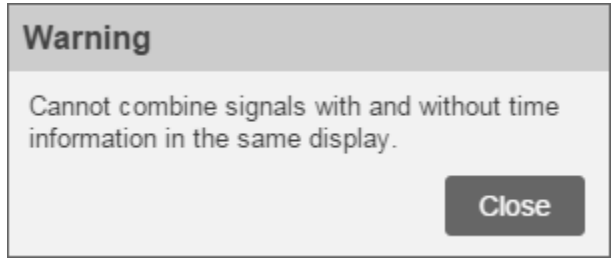
Time Specification Option	Description
`Work in Samples` (default)	This option enables you to explore signals without the need to specify a sample rate or a sample time. It is equivalent to plotting the signal in MATLAB without *x*-axis information.
`Sample Rate and Start Time`	Use this option when you know the rate at which the signal has been sampled. Specify the sample rate and the instant corresponding to the first sample. The **Sample Rate** can be expressed in Hz, kHz, MHz, or GHz. The **Start Time** can be expressed in seconds, years, days, hours, minutes, milliseconds, microseconds, or nanoseconds. Set the sample rate so that the signal is plotted in units of time on a display.

Time Specification Option	Description
`Sample Time and Start Time`	Use this option when you know the time interval between samples. Specify the sample time and the instant corresponding to the first sample. The **Sample Time** and **Start Time** each can be expressed in seconds, years, days, hours, minutes, milliseconds, microseconds, or nanoseconds. Set the sample time so that the signal is plotted in units of time on a display.

Time Specification Option	Description
`Time Values`	Use this option when you know the time value corresponding to each sample. Specify the time values using a variable name or a MATLAB expression. The **Time Values** can be stored in a numeric vector with real time values expressed in seconds. The values must be unique and cannot be `NaN`, but they need not be uniformly spaced. The vector must have the same length as the signal. The time values can also be stored in a `duration` array. The values must be unique and cannot be `NaN`, but they need not be uniformly spaced. The array must have the same length as the signal. The time values can also be entered as a MATLAB expression. The expression must specify an array with the same length as the signal. The values must be unique and cannot be `NaN`, but they need not be uniformly spaced. Valid examples include: • `(0:length(s)-1)'/Fs`, where `s` is the signal and `Fs` is a scalar in the workspace representing a sample rate. • `linspace(2,2.5,length(s))'`, where `s` is the signal. • `minutes(0:15)'`, equivalent to taking measurements every minute for 15 minutes. • `[0:10 20:30]`, equivalent to taking two sets of measurements at 1 Hz with a long pause between the sets. In all cases, the app derives a sample rate from the time values and displays it in the

Time Specification Option	Description
	Time column of the Signal table. An asterisk preceding the sample rate indicates that the signal is nonuniformly sampled.

Note Signals with no time information are plotted in units of samples on the *x*-axis. Signals with time information are plotted in units of time on the *x*-axis. To plot several signals on the same display, ensure that they all have time information or are all in samples. Otherwise, you get a warning:



- If a signal has missing or duplicate time points, you can fix it using the tips in “Clean Timetable with Missing, Duplicate, or Nonuniform Times” (MATLAB).
- If a signal is nonuniformly sampled, then **Signal Analyzer** interpolates the signal to a uniform grid to compute spectral estimates. The app uses linear interpolation and assumes a sample time equal to the median difference between adjacent time points. The derived sample rate in the Signal table has an asterisk to indicate that the signal is nonuniformly sampled.

  Note The interpolation is used only to compute spectral estimates. Time plots are not resampled.

  For a nonuniformly sampled signal to be supported, the median time interval and the mean time interval must obey

$$\frac{1}{100} < \frac{\text{Median time interval}}{\text{Mean time interval}} < 100.$$

- Filtering and scalogram view do not support nonuniformly sampled signals.

See Also

Signal Analyzer

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125

- “Signal Analyzer Tips and Limitations” on page 19-128

Data Types Supported by Signal Analyzer

Numeric Data

- Real numeric vectors and matrices with finite elements are supported.

  Example: `cos(pi./[4;2]*(0:159))'+randn(160,2)` is a two-channel signal consisting of sinusoids embedded in white noise.
- Scalars, empty arrays, multidimensional arrays, and the `ans` variable are not supported.

Note **Signal Analyzer** currently does not support arrays, timetables, or `timeseries` objects with complex values.

MATLAB Timetables

- Timetables with one or more variables are supported. Each variable can be a vector or a matrix. **Signal Analyzer** supports timetable inputs only when the time values are increasing and finite. Signals with missing, nonfinite, or duplicate time points are not imported. For some timetables, this restriction might mean that the app imports some signals but does not import others. To make sure that all signals are imported, you can fix them using the tips in "Clean Timetable with Missing, Duplicate, or Nonuniform Times" (MATLAB).

  Example: `timetable(seconds(0:4)',rand(5,2))` and `timetable(seconds(0:4)',rand(5,1),rand(5,1))` both specify a two-channel random variable sampled at 1 Hz for 4 seconds.
- Empty timetables and timetables with row times specified as `datetime` arrays are not supported.

Tip To analyze timetables with time values stored as a `datetime` array, convert the array to a `duration` array by subtracting the first time point, and then convert the `duration` array to seconds. See "Analyze Signals with Inherent Time Information" on page 19-44 for an example.

---

timeseries Objects

- Single-channel and multichannel `timeseries` objects are supported. To be supported, a `timeseries` object must have its `DataInfo.Interpolation` property set to `'linear'`. Use the `timeseries.setinterpmethod` method to change the property.

  Example: `timeseries(rand(5,2))` and `timeseries(rand(5,2),0:4)` both specify a two-channel random variable sampled at 1 Hz for 4 seconds.
- Signal Analyzer supports `timeseries` inputs only when the time values are increasing and finite. Signals with missing, nonfinite, or duplicate time points are not imported. For some `timeseries` objects, this restriction might mean that the app imports some signals but does not import others. To make sure that all signals are imported, you can fix them using the tips in "Time Series Objects" (MATLAB).
- Empty `timeseries` objects, `timeseries` objects with time vectors specified as MATLAB date strings, and `timeseries` objects whose `Name` property is not a valid MATLAB variable name are not supported. See `isvarname` for more information on valid variable names.

Note **Signal Analyzer** does not support matrices, time series, or timetables with more than 800 channels.

Nonuniformly Sampled Signals

- Filtering and scalogram view do not support nonuniformly sampled signals.
- If a signal is nonuniformly sampled, then **Signal Analyzer** interpolates the signal to a uniform grid to compute spectral estimates. The app uses linear interpolation and assumes a sample time equal to the median difference between adjacent time points. The derived sample rate in the Signal table has an asterisk to indicate that the signal is nonuniformly sampled. For a nonuniformly sampled signal to be supported, the median time interval and the mean time interval must obey

  $$\frac{1}{100} < \frac{\text{Median time interval}}{\text{Mean time interval}} < 100.$$

  Note The interpolation is used only to compute spectral estimates. Time plots are not resampled.

See Also

Apps
Signal Analyzer

Functions
`timeseries` | `timetable`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Spectrum Computation in Signal Analyzer

To compute signal spectra, **Signal Analyzer** finds a compromise between the spectral resolution achievable with the entire length of the signal and the performance limitations that result from computing large FFTs.

- If the resolution resulting from analyzing the full signal is achievable, the app computes a single modified periodogram of the whole signal using an adjustable Kaiser window.
- If the resolution resulting from analyzing the full signal is not achievable, the app computes a Welch periodogram: It divides the signal into overlapping segments, windows each segment using a Kaiser window, and averages the periodograms of the segments.

Spectral Windowing

Any real-world signal is measurable only for a finite length of time. This fact introduces nonnegligible effects into Fourier analysis, which assumes that signals are either periodic or infinitely long. *Spectral windowing*, which consists of assigning different weights to different signal samples, deals systematically with finite-size effects.

The simplest way to window a signal is to assume that it is identically zero outside of the measurement interval and that all samples are equally significant. This “rectangular window” has discontinuous jumps at both ends that result in spectral ringing. All other spectral windows taper at both ends to lessen this effect by assigning smaller weights to samples close to the signal edges.

The windowing process always involves a compromise between conflicting aims: improving resolution and decreasing leakage.

- *Resolution* is the ability to know precisely how the signal energy is distributed in the frequency space. A spectrum analyzer with ideal resolution can distinguish two different tones (pure sinusoids) present in the signal, no matter how close in frequency. Quantitatively, this ability relates to the mainlobe width of the transform of the window.
- *Leakage* is the fact that, in a finite signal, every frequency component projects energy content throughout the complete frequency span. The amount of leakage in a spectrum can be measured by the ability to detect a weak tone from noise in the presence of a neighboring strong tone. Quantitatively, this ability relates to the sidelobe level of the frequency transform of the window.

The better the resolution, the higher the leakage, and vice versa. At one end of the range, a rectangular window has the narrowest possible mainlobe and the highest sidelobes. This window can resolve closely spaced tones if they have similar energy content, but it fails to find the weaker one if they do not. At the other end, a window with high sidelobe suppression has a wide mainlobe in which close frequencies are smeared together.

Signal Analyzer uses Kaiser windows to carry out windowing. For Kaiser windows, the fraction of the signal energy captured by the mainlobe depends most importantly on an adjustable *shape factor*, β. The shape factor ranges from $\beta = 0$, which corresponds to a rectangular window, to $\beta = 40$, where a wide mainlobe captures essentially all the spectral energy representable in double precision. An intermediate value of $\beta \approx 6$ approximates a Hann window closely. To control β, use the **Leakage** slider on the **Spectrum** and **Spectrogram** tabs. If you set the leakage to ℓ using the slider, then ℓ and β are related by $\beta = 40(1 - \ell)$. See `kaiser` for more details.

Parameter and Algorithm Selection

To compute the spectra of the signals appearing on a given display, **Signal Analyzer** initially determines the *resolution bandwidth*, which measures how close two tones can be and still be resolved. The resolution bandwidth has a theoretical value of

$$\mathrm{RBW}_{\mathrm{theory}} = \frac{\mathrm{ENBW}}{t_{\max} - t_{\min}}.$$

- $t_{\max} - t_{\min}$, the *record length*, is the time-domain duration of the selected signal region.

  Use the panner to select and adjust the record length or region of interest. Equivalently, you can zoom in on the time-domain plot or change the limits on the **Time** tab.
- ENBW is the *equivalent noise bandwidth* of the spectral window. See `enbw` for more details.

  Use the **Leakage** slider in the **Spectrum** tab to control the ENBW. The minimum value in the slider range corresponds to a Kaiser window with $\beta = 40$. The maximum value corresponds to a Kaiser window with $\beta = 0$.

In practice, however, the app might coarsen the resolution. This coarsening makes it possible to compute the spectrum in a reasonable amount of time and to display it with a finite number of pixels. The coarsening criterion is that the resolution bandwidth cannot be smaller than

$$\mathrm{RBW}_{\mathrm{performance}} = \frac{\max(f_{\mathrm{Nyquist}})}{4096},$$

where $\max(f_{\mathrm{Nyquist}})$ is the highest Nyquist frequency among all the signals in the display. (If there is no aliasing, the Nyquist frequency is one-half the sample rate.) $\mathrm{RBW}_{\mathrm{performance}}$ cannot be adjusted.

To compute the spectrum of a signal, the app chooses the larger of the two values:

$$\mathrm{RBW} = \max(\mathrm{RBW}_{\mathrm{theory}}, \mathrm{RBW}_{\mathrm{performance}}).$$

This *target resolution bandwidth* is displayed on the **Spectrum** tab.

- If the resolution bandwidth is $\mathrm{RBW}_{\mathrm{theory}}$, then **Signal Analyzer** computes a single *modified periodogram* for the whole signal. The app uses a Kaiser window with the slider-controlled shape factor and applies zero-padding when the time limits on the axes exceed the signal duration. See `periodogram` for more details.
- If the resolution bandwidth is $\mathrm{RBW}_{\mathrm{performance}}$, then **Signal Analyzer** computes a *Welch periodogram* for the signal. The app:

  1. Divides the signals into overlapping segments.
  2. Windows each segment separately using a Kaiser window with the specified shape factor.
  3. Averages the periodograms of all the segments.

  Welch's procedure is designed to reduce the variance of the spectrum estimate by averaging different "realizations" of the signals, given by the overlapping sections, and using the window to remove redundant data. See `pwelch` for more details.

  - The length of each segment (or, equivalently, of the window) is computed using

    $$\text{Segment length} = \frac{\max(f_{\mathrm{Nyquist}}) \times \mathrm{ENBW}}{\mathrm{RBW}}.$$

  - The stride length is found by adjusting an initial estimate,

    $$\text{Stride length} \equiv \text{Segment length} - \text{Overlap} = \frac{\text{Segment length}}{2 \times \mathrm{ENBW} - 1},$$

so that the first window starts exactly on the first sample of the first segment and the last window ends exactly on the last sample of the last segment.

Zooming

If you zoom in on a region of a signal spectrum using one of the zoom actions on the **Display** tab, the app does not change the resolution bandwidth. Instead, **Signal Analyzer** performs an optical zooming, using bandlimited interpolation to display a smooth spectral curve.

Zooming in on a time-domain region of a signal is equivalent to setting the record length or region of interest with the panner.

If the selected time interval extends beyond the ends of a signal, the app zero-pads the signal. If a signal has no samples within the selected time interval, the app displays nothing.

References

[1] harris, fredric j. “On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform.” *Proceedings of the IEEE*. Vol. 66, January 1978, pp. 51–83.

[2] Welch, Peter D. “The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms.” *IEEE Transactions on Audio and Electroacoustics*. Vol. 15, June 1967, pp. 70–73.

See Also

Apps
Signal Analyzer

Functions
`enbw` | `kaiser` | `periodogram` | `pspectrum` | `pwelch`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27

- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128
- “Nonparametric Methods” on page 6-9

Persistence Spectrum in Signal Analyzer

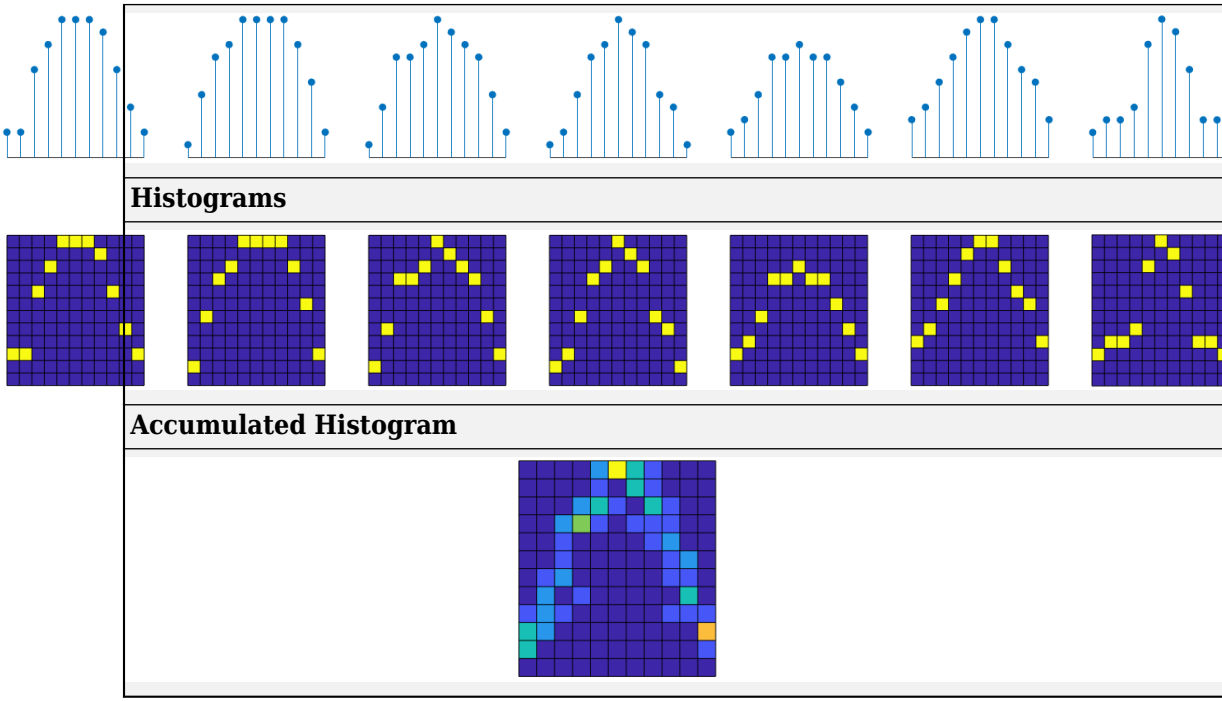
The persistence spectrum of a signal is a time-frequency view that shows the percentage of the time that a given frequency is present in a signal. The persistence spectrum is a histogram in power-frequency space. The longer a particular frequency persists in a signal as the signal evolves, the higher its time percentage and thus the brighter or "hotter" its color in the display. Use the persistence spectrum to identify signals hidden in other signals.

To compute the persistence spectrum, **Signal Analyzer** performs these steps:

1. Compute the spectrogram using the specified leakage, time resolution, and overlap. (When you zoom in time using the panner or a zoom button, the app computes and displays the persistence spectrum using the segments that fall within the visible zoomed-in region of interest, including those segments that are only partially visible. See “Spectrogram Computation in Signal Analyzer” on page 19-110 for more details.)
2. Partition the power and frequency values into 2-D bins. (To adjust the power or the frequency limits, enter the minimum and maximum **Power Limits** or **Frequency Limits** values on the **Persistence Spectrum** tab.)
3. For each time value, compute a bivariate histogram of the logarithm of the power spectrum. For every power-frequency bin where there is signal energy at that instant, increase the corresponding matrix element by 1. Sum the histograms for all the time values.
4. Plot the accumulated histogram against the power and the frequency, with the color proportional to the logarithm of the histogram counts expressed as normalized percentages. To represent zero values, use one-half of the smallest possible magnitude.

   (To adjust the range of histogram counts represented in the colormap, enter the minimum and maximum **Density Limits** on the **Persistence Spectrum** tab. To fit the colormap to the current density limits, click the **Fit colormap** button on the **Display** tab.)

Power Spectra



See Also

Apps
Signal Analyzer

Functions
`histcounts2` | `pspectrum`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47

- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

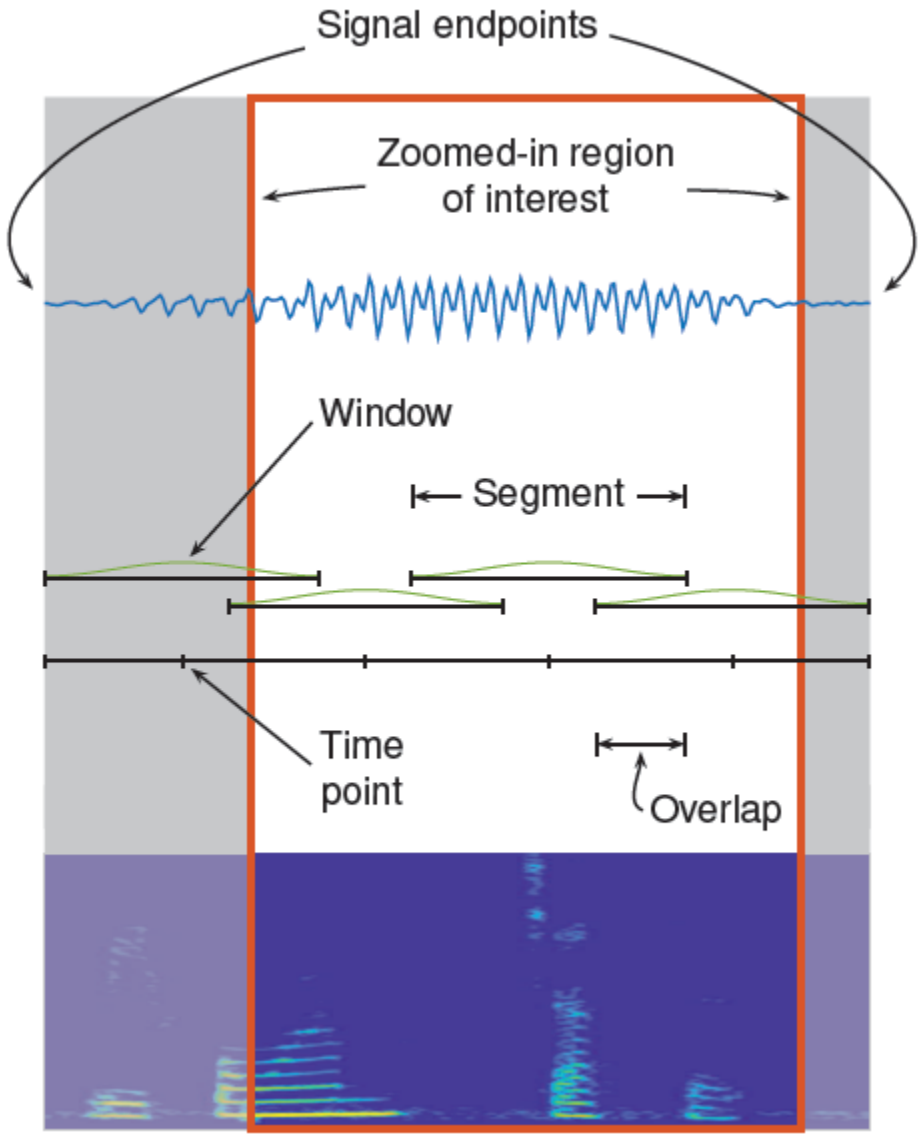
- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128
- “Nonparametric Methods” on page 6-9

Spectrogram Computation in Signal Analyzer

A nonstationary signal is a signal whose frequency content changes with time. The *spectrogram* of a nonstationary signal is an estimate of the time evolution of its frequency content. To construct the spectrogram of a nonstationary signal, **Signal Analyzer** follows these steps:

1. Divide the signal into equal-length segments. The segments must be short enough that the frequency content of the signal does not change appreciably within a segment. The segments may or may not overlap.
2. Window each segment and compute its spectrum to get the *short-time Fourier transform*.
3. Display segment-by-segment the power of each spectrum in decibels. Depict the magnitudes side-by-side as an image with magnitude-dependent colormap.

The spectrogram view is available in displays that contain only one signal.


Signal endpoints
Zoomed-in region
of interest
Window
Segment
Time
point
Overlap

Divide Signal into Segments

To construct a spectrogram, first divide the signal into possibly overlapping segments. In **Signal Analyzer**, you can control the length of the segments and the amount of overlap between adjoining segments using **Time Resolution** and **Overlap**. If you do not specify the length and overlap, **Signal Analyzer** chooses a length based on the entire length of the signal, and 50% overlap. The app aligns the time axis of the spectrogram with the axis of the time-domain plot.

Specified Time Resolution

On the **Spectrogram** tab, in the **Time Resolution** section, click **Specify**.

- If the signal does not have time information, specify the time resolution (segment length) in samples. The time resolution must be an integer greater than or equal to 1 and smaller than or equal to the signal length.

  If the signal has time information, specify the time resolution in seconds. The app converts the result into a number of samples and rounds it to the nearest integer that is less than or equal to the number but not smaller than 1. The time resolution must be smaller than or equal to the signal duration.
- Specify the overlap as a percentage of the segment length. The app converts the result into a number of samples and rounds it to the nearest integer that is less than or equal to the number.

Default Time Resolution

If you select **Auto** for the time resolution computation, then **Signal Analyzer** uses the length of the entire signal to choose the length of the segments. The app sets the time resolution as $\lceil N/d \rceil$ samples, where the brackets denote the ceiling function, N is the length of the signal, and d is a divisor that depends on N:

Signal Length (N)	Divisor (d)	Segment Length
2 samples – 63 samples	2	1 sample – 32 samples
64 samples – 255 samples	8	8 samples – 32 samples
256 samples – 2047 samples	8	32 samples – 256 samples
2048 samples – 4095 samples	16	128 samples – 256 samples

Signal Length (*N*)	Divisor (*d*)	Segment Length
4096 samples – 8191 samples	32	128 samples – 256 samples
8192 samples – 16383 samples	64	128 samples – 256 samples
16384 samples – *N* samples	128	128 samples – $\lceil N / 128 \rceil$ samples

You can still specify the overlap between adjoining segments. Specifying the overlap changes the number of segments. Segments that extend beyond the signal endpoint are zero-padded.

Consider the seven-sample signal `[s0 s1 s2 s3 s4 s5 s6]`. Because $\lceil 7/2 \rceil = \lceil 3.5 \rceil = 4$, the app divides the signal into two segments of length four when there is no overlap. The number of segments changes as the overlap increases.

Number of Overlapping Samples	Resulting Segments
0	`s0 s1 s2 s3` `s4 s5 s6 0`
1	`s0 s1 s2 s3` `s3 s4 s5 s6`

2	`s0 s1 s2 s3`
	`s2 s3 s4 s5`
	`s4 s5 s6 0`

3	`s0 s1 s2 s3`
	`s1 s2 s3 s4`
	`s2 s3 s4 s5`
	`s3 s4 s5 s6`

Time Alignment

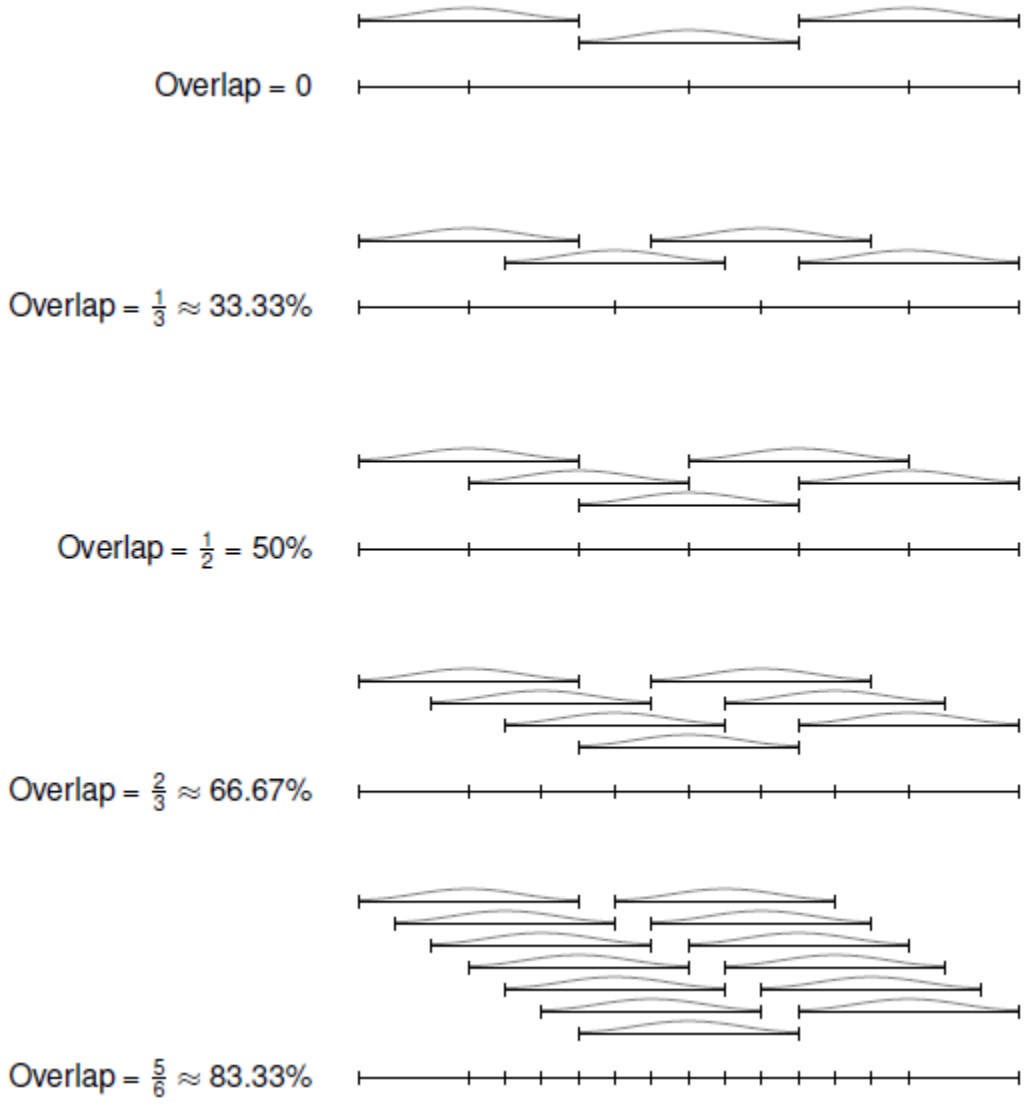
Once the segment length and overlap are set, the number of segments and their edge locations stay fixed and are independent of any zooming or panning. When you zoom and pan, the app computes and displays the spectrogram using the segments that fall within the visible zoomed-in region of interest.

The app:

- Aligns the time axis of the spectrogram with the axis of the corresponding time-domain plot. That way, the spectral content at a given time aligns with its occurrence.

- For nonzero overlap, extends the first and last segments to the signal endpoints.
- Zero-pads the signal if the last segment extends beyond the signal endpoint.

When the segments have 0% overlap, each segment is centered at the actual time of occurrence. When the overlap is nonzero, the alignment of the spectrogram time axis with the time-domain axis has the effect that the first and last time intervals are elongated. All other time intervals are of the same length. In other words, the center of each segment, except for the first and last, corresponds to the actual time of occurrence. Consider this example:


Overlap = 0
Overlap = $\frac{1}{3} \approx 33.33\%$
Overlap = $\frac{1}{2} = 50\%$
Overlap = $\frac{2}{3} \approx 66.67\%$
Overlap = $\frac{5}{6} \approx 83.33\%$

Window the Segments and Compute Spectra

After **Signal Analyzer** divides the signal into overlapping segments, the app windows each segment with a Kaiser window. The shape factor β of the window, and therefore the *leakage*, is adjustable.

Note The leakage used to compute the signal spectrum and the leakage used to window the spectrogram segments are independent of each other. You can adjust them separately.

The app then computes the spectrum of each segment, using the procedure outlined in “Spectrum Computation in Signal Analyzer” on page 19-102. In summary, **Signal Analyzer** finds a compromise between the spectral resolution achievable with the entire length of the segment and the performance limitations that result from computing large FFTs.

- If the resolution resulting from analyzing the full segment is achievable, the app computes a single modified periodogram of the whole segment using a Kaiser window with the specified shape factor.
- If the resolution resulting from analyzing the full segment is not achievable, the app computes a Welch periodogram: It divides the segment into overlapping subsegments, windows each subsegment, and averages the periodograms of the subsegments. The app chooses the subsegment size, the window, and the overlap so that the composite periodogram is equivalent to a modified periodogram of the whole segment with the specified Kaiser window.

Display Spectrum Power

The app displays the power of the short-time Fourier transform in decibels, using a color bar with the default MATLAB colormap. The color bar comprises the full power range of the spectrogram and does not change if you zoom or pan.

You can change the magnitude levels represented by a given color range. On the **Spectrogram** tab, change the minimum and maximum power values to display. You can also set the colormap so that it comprises the full power range of the zoomed-in section of the spectrogram. On the **Display** tab, click the scale color button .

See Also

Apps
Signal Analyzer

Functions
`pspectrum` | `spectrogram` | `xspectrogram`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

- “Nonparametric Methods” on page 6-9

Scalogram Computation in Signal Analyzer

The scalogram is the absolute value of the continuous wavelet transform (CWT) of a signal, plotted as a function of time and frequency. The scalogram can be more useful than the spectrogram for identifying signals with low-frequency components or rapidly changing frequency content. Use the scalogram when you want better time localization for short-duration, high-frequency events and better frequency localization for low-frequency, longer-duration events.

Unlike the spectrogram, which decomposes the input signal into sinusoids of infinite duration, the CWT decomposes the signal into *wavelets*. Wavelets are good for analyzing data that exhibit regular behavior punctuated with abrupt changes because they are localized in both frequency and time. In other words, wavelets have a beginning and an end. The finite duration of wavelets makes it possible to control not just their amplitude and shape, but also their location. This extra degree of freedom enables the CWT to detect transients and identify regions where a signal has abrupt changes in frequency.

Note You need a Wavelet Toolbox license to use the scalogram view.

To achieve localization in time, the spectrogram divides the input signal into windowed segments, which it Fourier transforms one-by-one and then displays. (See “Spectrogram Computation in Signal Analyzer” on page 19-110 for more information.) The length of the window, however, is fixed, and the uncertainty principle imposes a tradeoff between time resolution and frequency resolution:

- Improving the time resolution, for example to detect a burst, comes at the expense of the frequency resolution.
- Improving the frequency resolution, for example to characterize tones to high precision, comes at the expense of the time resolution.

The scalogram, on the other hand, can resize the wavelet as the signal evolves and adjust the wavelet when conditions change. The procedure stretches the wavelet to capture long-duration, low-frequency information and shrinks the wavelet to capture short-duration, high-frequency information. Using this procedure, the scalogram can achieve good frequency localization at low frequencies and good time localization at high frequencies.

To compute the scalogram, **Signal Analyzer** performs these steps:

1. If the signal has more than 1 million samples, divide the signal into overlapping segments.
2. Compute the CWT of each segment to get its scalogram.
3. Display the scalogram segment by segment.

Tip

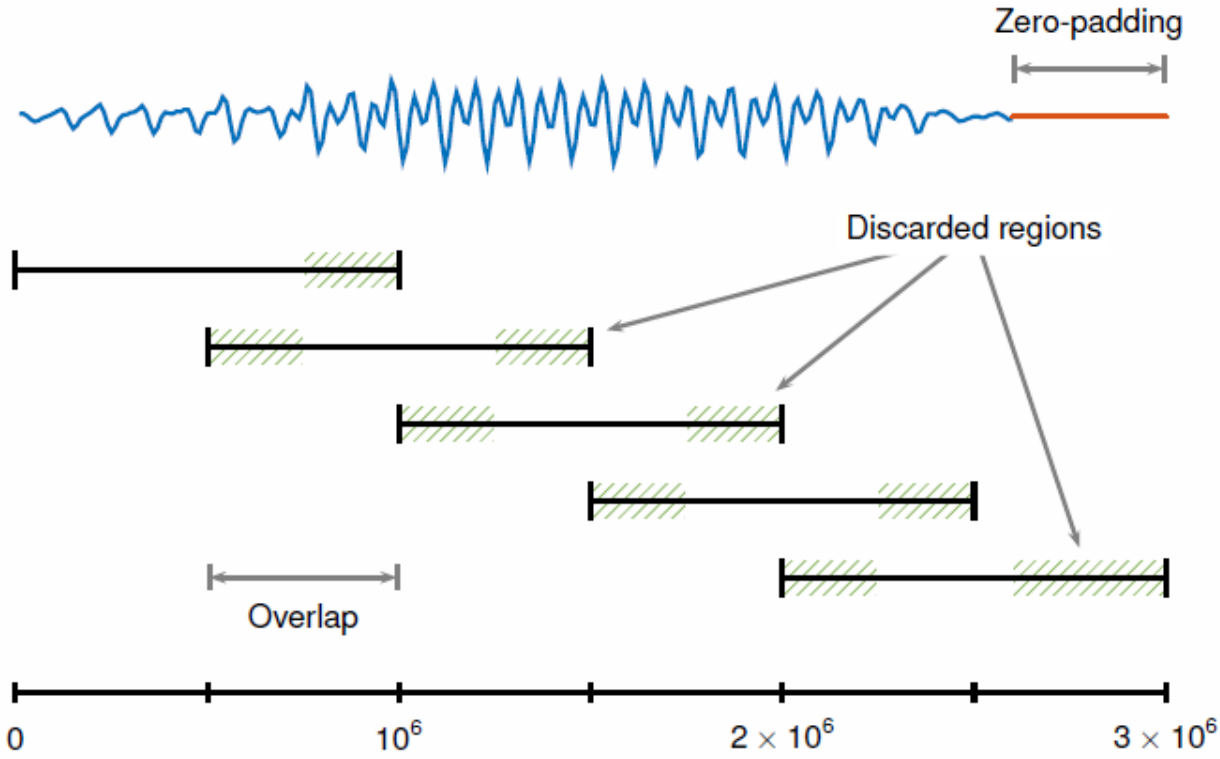
- The scalogram view does not support nonuniformly sampled signals. To compute the scalogram of a nonuniformly sampled signal, resample your signal to a uniform grid by using the `resample` function.
- The scalogram view is available in displays that contain only one signal. To compare scalograms of different signals, open separate displays and drag each signal to its own display.

Divide the Signal into Segments

If the input signal has 1 million samples or less, **Signal Analyzer** uses the `cwt` function directly. If the signal has more than 1 million samples, the app performs these steps:

1. Divide the signal into segments of 1 million samples, with 50% overlap between adjoining segments.
2. If the last segment extends beyond the signal endpoint, zero-pad the signal until the last segment has 1 million samples.
3. After computing the scalogram of each segment, remove edge effects:
    - Discard the first 250,000 and the last 250,000 scalogram samples of all segments except the first and the last.
    - Discard the last 250,000 scalogram samples of the first segment.
    - In the last segment, discard the first 250,000 scalogram samples and the samples corresponding to the zero-padded region.

Consider, for example, a signal with 2.6×10^6 samples:

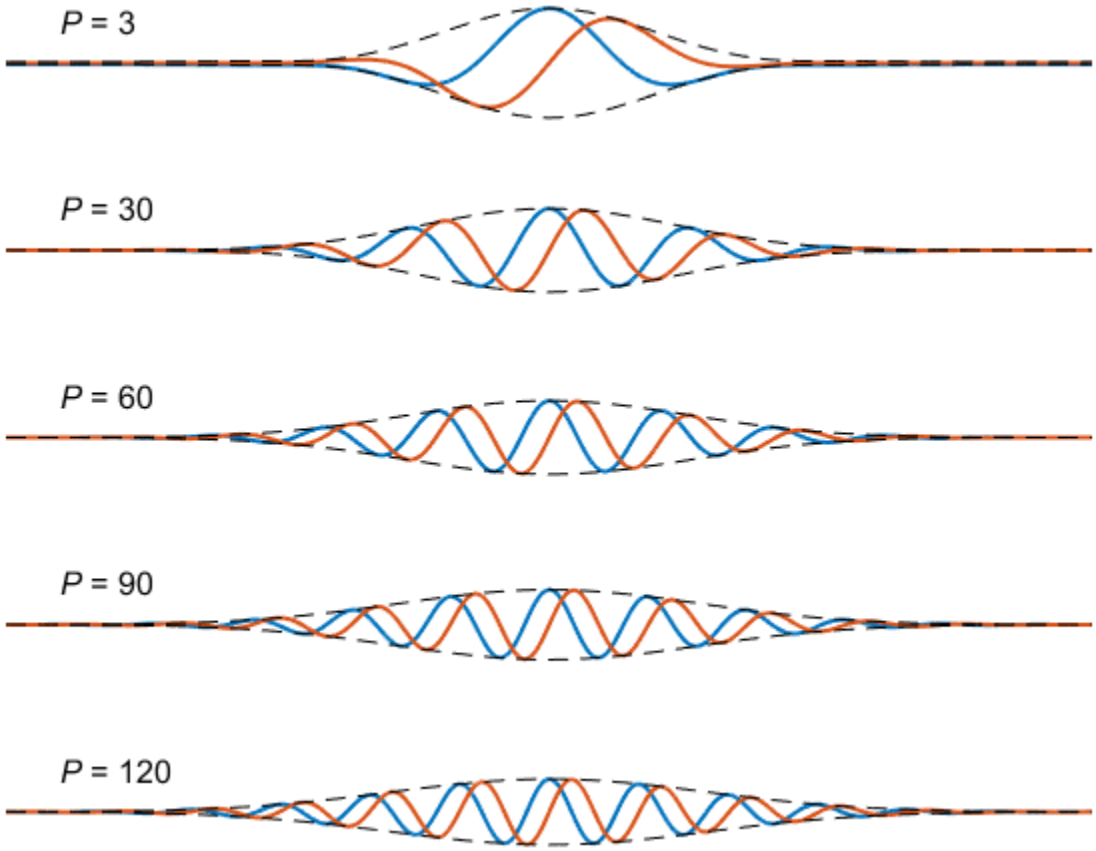


Compute the Continuous Wavelet Transform

Signal Analyzer computes the CWT using the default settings of the `cwt` function. The app uses generalized analytic Morse wavelets with gamma factor $\gamma = 3$. See "Morse Wavelets" (Wavelet Toolbox) for more information.

Signal Analyzer provides two separate controls for frequency resolution.

- The **Time-Bandwidth** slider controls the time-bandwidth product, which is proportional to the wavelet duration in the time domain. Increasing the time-bandwidth product results in wavelets with more oscillations in their central portions, larger spreads in time, and narrower spreads in frequency. The slider moves in the range from 3 to 120. The default value is 60. The figure shows some Morse wavelets with varying time-bandwidth product P. The real part is in blue, the imaginary part is in red, and the absolute value is in black.



- The **Voices Per Octave** slider controls the number of scales per octave used to discretize the CWT. As the number of voices per octave increases, the scale resolution becomes finer. The slider moves in steps of multiples of 4 in the range from 4 to 16. The default value is 8.

Display the Scalogram

Signal Analyzer plots the absolute value of the CWT coefficients as a function of time and frequency. If the signal was divided into segments, the app concatenates portions of the scalograms of the individual segments and displays them. The app also plots the cone of influence, which shows where edge effects become significant.

See Also

Apps
Signal Analyzer

Functions
`cwt` | `cwtfilterbank` | `pspectrum`

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Morse Wavelets” (Wavelet Toolbox)
- “Continuous and Discrete Wavelet Transforms” (Wavelet Toolbox)
- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110

- “Keyboard Shortcuts for Signal Analyzer” on page 19-125
- “Signal Analyzer Tips and Limitations” on page 19-128

Keyboard Shortcuts for Signal Analyzer

Note On Macintosh platforms, use the **Command** key instead of **Ctrl**.

General Actions

Task	Shortcut
Start a new session	**Ctrl+N**
Open a session	**Ctrl+O**
Save a session	**Ctrl+S**
Link or unlink a display	**Ctrl+U**
Delete a signal	**Del**
Copy a display to the clipboard	**Ctrl+C**

Zooming

Task	Shortcut
Zoom in X-axis (time or frequency)	**Ctrl+Shift+T**
Zoom in Y-axis	**Ctrl+Shift+Y**
Zoom in X and Y	**Ctrl++** (numeric keypad only)
Zoom out	**Ctrl+-** (numeric keypad only)
Fit to view	**Spacebar**
Fit colormap to current power limits	**Ctrl+Spacebar**
Cancel zoom operation or signal dragging	**Esc**

Data Cursors

Task	Shortcut
Show a data cursor	**Ctrl+I**
Hide all data cursors	**Shift+Del**

Task	Shortcut
Move a selected data cursor to the next data point	**Right arrow**
Move a selected data cursor to the previous data point	**Left arrow**
Activate first (left) cursor	**Ctrl+1**
Activate second (right) cursor	**Ctrl+2**

See Also

Signal Analyzer

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102

- “Persistence Spectrum in Signal Analyzer” on page 19-107
- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Signal Analyzer Tips and Limitations” on page 19-128

Signal Analyzer Tips and Limitations

Frequently asked questions and current limitations of the **Signal Analyzer** app.

Select Signals to Analyze

1. "I dragged a 512-by-24 matrix into a display, but the app plotted only 10 of the 24 signals. How do I plot the others?"

   By default, **Signal Analyzer** imports all the columns of a multichannel signal but plots only the first 10 columns. To plot signal columns beyond the 10th, drag them to the display. Alternatively, on the Signal table, select the check boxes next to the names of the signals you want to plot.

2. "My data is saved in structures. How can I analyze them in **Signal Analyzer**?"

   To study a structure in **Signal Analyzer**, convert it to a timetable. The easiest way to do the conversion is to convert the structure to a table and then convert the table to a timetable. The second step involves converting the time variable to a duration array. The following example creates a structure with three fields, one of them containing the time values, and converts the structure to a timetable readable by **Signal Analyzer**.

   ```
   str.st = (0:999)'/1000;
   str.s1 = randn(1000,1);
   str.s2 = sin(2*pi*20*str.st);

   T = struct2table(str);
   T.st = seconds(T.st);
   TT = table2timetable(T,'RowTimes','st');
   ```

   If your structure does not have time information, you can use other MATLAB functions. The following function takes a structure as input, extracts from it the arrays of signal values, and calls **Signal Analyzer** to plot the signals.

   ```
   function structSig(x)
      names = fieldnames(x);
      for i = 1:length(names)
         signalAnalyzer(getfield(x,names{i}))
      end
   end
   ```

3. "What does it mean when a row in the Signal table is highlighted in gray and what does the check box mean?"

There are two different ways to choose signals in the Signal table. Each way gives you access to a different set of operations.

- Selecting the signal by clicking the **Name**, **Info**, **Time**, or **Start Time** column in the Signal table enables you to perform all the operations in the **Analyzer** tab. You can change the time information and smooth, filter, or duplicate the signals. You can run preprocessing operations on a signal without plotting the signal.
- Selecting the check box to the left of the signal name plots the signal in the currently selected display and enables you to perform all the operations in the **Display** tab. You can display the signal in the frequency domain or the time-frequency domain, or you can measure the signal using cursors.

4 "I use timetables with the time values stored as `datetime` arrays. How can I analyze them?"

To analyze timetables with time values stored as a `datetime` array, convert the array to a relative `duration` array by subtracting the first element from all the others. The following example creates a timetable with `datetime` row times and converts it to a timetable readable by **Signal Analyzer**.

```
tt = timetable(datetime(2016,11,9,2,30,1:10)',randn(10,1));
dt = tt.Time-tt.Time(1);
tn = timetable(dt,tt.Var1);
```

See “Analyze Signals with Inherent Time Information” on page 19-44 for another example.

5 "I have a timetable but only some of its variables were imported. How can I import them?"

Signal Analyzer lists only the variables that it can display and process. If some variables of a timetable are not being imported, they probably are complex or have NaNs. To be able to import them to the app, you must fix them in MATLAB first. To fix timetables, you can use the tips in “Clean Timetable with Missing, Duplicate, or Nonuniform Times” (MATLAB).

6 "I changed a variable in the MATLAB workspace. Why is there no change in the **Signal Analyzer** display?"

If you modify a signal in the MATLAB workspace, the Workspace browser updates automatically. To have the app recognize the changes, reimport the signal by dragging it again to the Signal table or to a display.

Preprocess Signals

1. "How do I apply a lowpass filter to a signal that is not uniformly sampled?"

   The filtering functionality of **Signal Analyzer** supports only uniformly sampled signals. You can resample your signal to a uniform grid by using the `resample` function. The function returns the resampled signal and the uniform grid of time values.

2. "How do I know what parameters were used for a preprocessing operation?"

   To see a full summary of the preprocessing steps you took, including all settings you chose, click **Generate Function** on the **Analyzer** tab.

Explore Signals

1. "I want to view a scalogram of my signal, but I get a warning saying that I have to create a uniformly sampled signal. How do I resample my signal?"

   You can resample your signal to a uniform grid by using the `resample` function. The function returns the resampled signal and the uniform grid of time values.

2. "Why can I not zoom out beyond the Nyquist range of a scalogram?"

   If a real signal is sampled properly, then all of its frequency information is contained within the Nyquist range.

3. "How can I compare spectrograms of 10 different signals?"

   The time-frequency views of **Signal Analyzer** support only one signal per display. To compare spectrograms of 10 different signals, open 10 displays and drag each signal to its own display. You can use the same procedure for persistence spectra and scalograms.

Share Analysis

1. "I generated a script that does not run because the variable it uses does not exist. Why?"

   If you extract, duplicate, or rename a signal in **Signal Analyzer** and generate a MATLAB script without exporting the modified signal, the script will not work. Remember to export any signals you use as starting points for scripts.

2 "How do I reproduce a **Signal Analyzer** spectrum, persistence spectrum, spectrogram, or scalogram in MATLAB?"

   Click **Spectrum** or **Spectrogram** on the **Display** tab to compute and display the spectrum, persistence spectrum, spectrogram, or scalogram of a plotted signal. When you have the optimal settings for your signal, click **Generate Script** and select `Spectrum Script`, `Persistence Spectrum Script`, `Spectrogram Script`, or `Scalogram Script` to generate a script that you can use in MATLAB.

See Also

Signal Analyzer

Related Examples

- “Find Delay Between Correlated Signals” on page 19-27
- “Plot Signals from the Command Line” on page 19-32
- “Resolve Tones by Varying Window Leakage” on page 19-36
- “Analyze Signals with Inherent Time Information” on page 19-44
- “Spectrogram View of Dial Tone Signal” on page 19-47
- “Find Interference Using Persistence Spectrum” on page 19-50
- “Extract Regions of Interest from Whale Song” on page 19-56
- “Find and Track Ridges Using Reassigned Spectrogram” on page 19-62
- “Scalogram of Hyperbolic Chirp” on page 19-68
- “Extract Voices from Music Signal” on page 19-72
- “Resample and Filter a Nonuniformly Sampled Signal” on page 19-79
- “Declip Saturated Signals Using Your Own Function” on page 19-87

More About

- “Using Signal Analyzer App” on page 19-2
- “Edit Sample Rate and Other Time Information” on page 19-93
- “Data Types Supported by Signal Analyzer” on page 19-99
- “Spectrum Computation in Signal Analyzer” on page 19-102
- “Persistence Spectrum in Signal Analyzer” on page 19-107

- “Spectrogram Computation in Signal Analyzer” on page 19-110
- “Scalogram Computation in Signal Analyzer” on page 19-119
- “Keyboard Shortcuts for Signal Analyzer” on page 19-125

Common Applications

Create Uniform and Nonuniform Time Vectors

You can create uniform and nonuniform time vectors for use in computations involving time series.

Use the colon operator if you know the sampling frequency. If your system samples time at a rate of 15 Hz during one second, you get 16 readings, including the one at zero.

```
Fs = 15;
Ts = 1/Fs;
ts = 0:Ts:1;
```

Use `linspace` if you know the beginning and end of the time interval and the number of samples. Suppose you start a stopwatch and stop it one second later. If you know your instrument took 15 readings, you can generate the time vector.

```
tl = linspace(0,1,15);
```

You can compute the sample rate directly from the samples and use it to reconstruct the time vector.

```
sf = 1/(tl(2)-tl(1));

TL = (0:length(tl)-1)/sf;

ErrorTL = max(abs(tl-TL))
```

```
ErrorTL = 0
```

You can also reconstruct `ts` using `linspace`.

```
lts = length(ts);
TS = linspace(ts(1),ts(lts),lts);

ErrorTS = max(abs(ts-TS))
```

```
ErrorTS = 1.1102e-16
```

`linspace` and the colon operator create row vectors by default. Transpose them to obtain column vectors.

```
tcol = tl';
ttrans = ts';
```

Combine `linspace` and the colon operator to generate nonuniform time vectors of arbitrary characteristics.

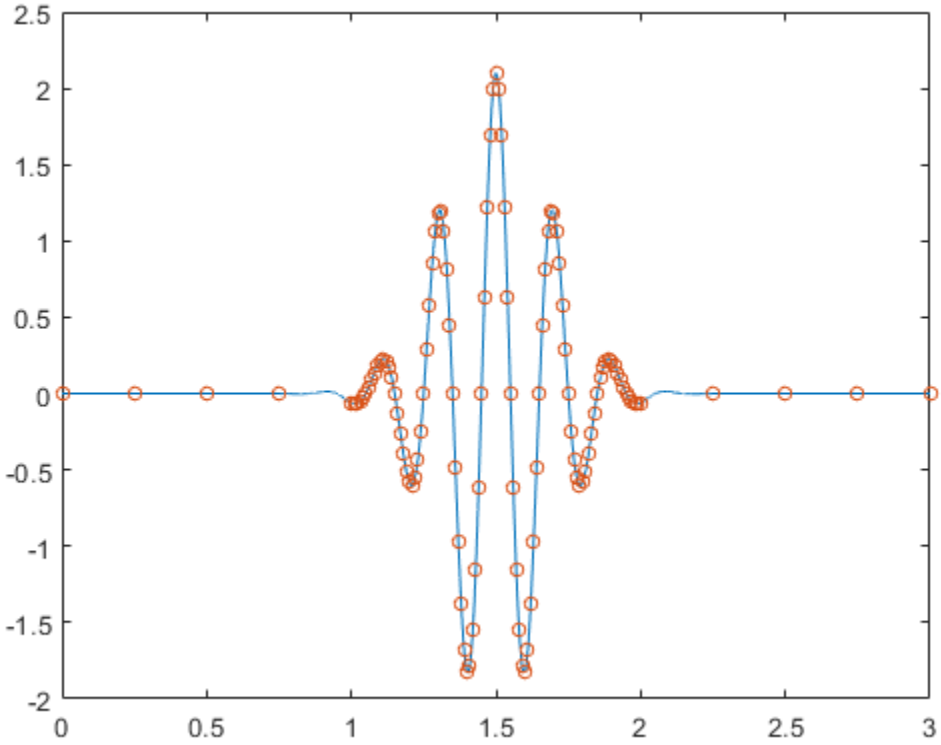
Suppose you have a Gaussian-modulated sinusoidal pulse that you must sample. The pulse changes rapidly during a one-second interval but slowly during the preceding and following seconds.

Sample the region of interest at 100 Hz and take only five samples before and after. Concatenate the vectors using square brackets.

```
gpl = @(x) 2.1*gauspuls(x-1.5,5,0.4);

Ffast = 100;
Tf = 1/Ffast;
Nslow = 5;
tdisc = [linspace(0,1,Nslow) 1+Tf:Tf:2-Tf linspace(2,3,Nslow)];
```

Generate 20001 samples of the function to simulate the continuous-time pulse. Overlay a plot of the samples defined by `tsf`.

```
Tcont = linspace(0,3,20001)';

plot(Tcont,gpl(Tcont),tdisc,gpl(tdisc),'o','markersize',5)
```



See Also

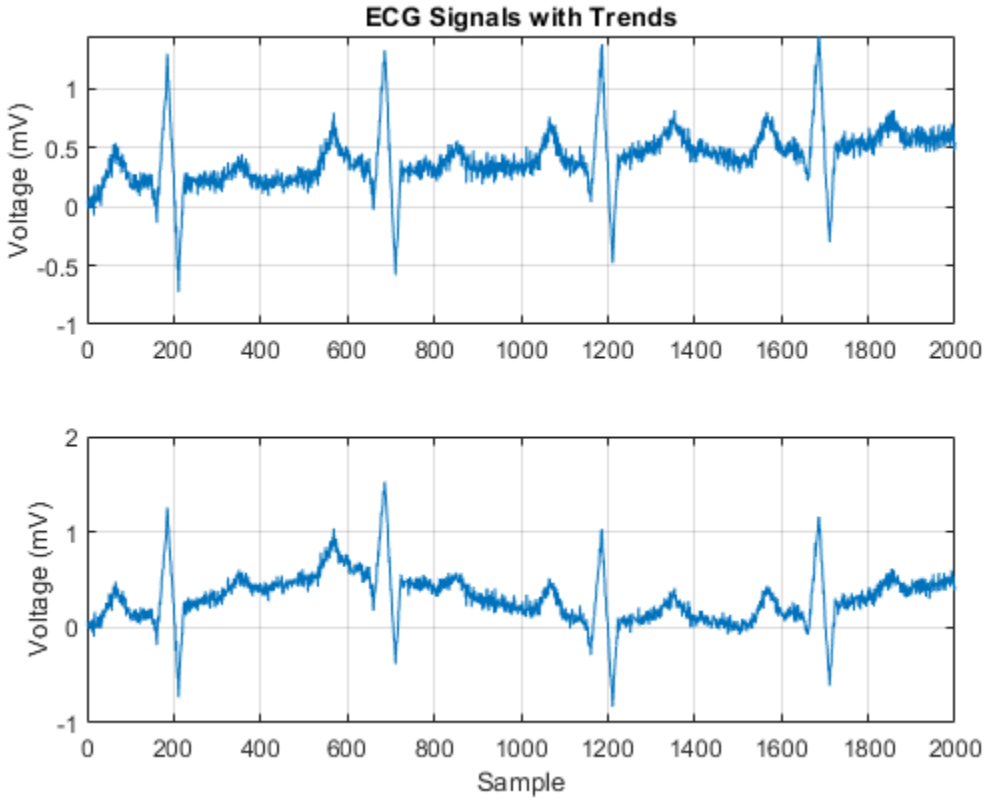
`gauspuls`

Remove Trends from Data

Measured signals can show overall patterns that are not intrinsic to the data. These trends can sometimes hinder the data analysis and must be removed.

Consider two electrocardiogram (ECG) signals with different trends. ECG signals are sensitive to disturbances such as power source interference. Load the signals and plot them.

```
load(fullfile(matlabroot,'examples','signal','ecgSignals.mat'))

t = (1:length(ecgl))';

subplot(2,1,1)
plot(t,ecgl), grid
title 'ECG Signals with Trends', ylabel 'Voltage (mV)'

subplot(2,1,2)
plot(t,ecgnl), grid
xlabel Sample, ylabel 'Voltage (mV)'
```



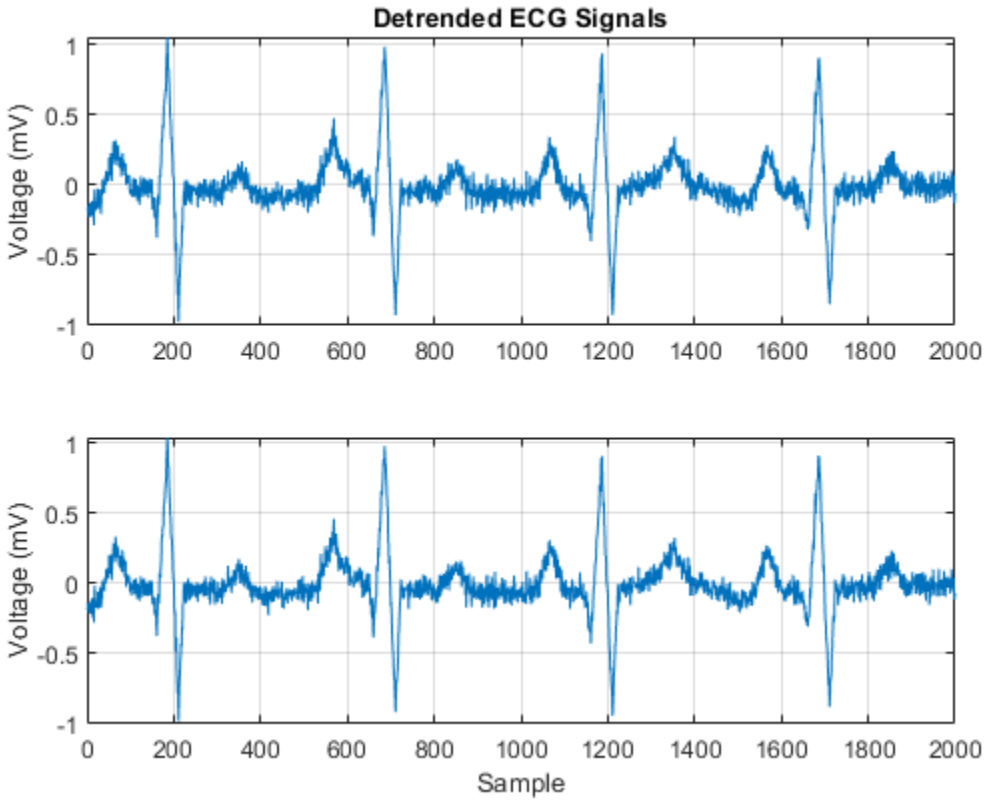
The signal on the first plot shows a linear trend. The trend on the second signal is nonlinear. To eliminate the linear trend, use the MATLAB® function `detrend`.

```
dt_ecgl = detrend(ecgl);
```

To eliminate the nonlinear trend, fit a low-order polynomial to the signal and subtract it. In this case, the polynomial is of order 6. Plot the two new signals.

```
opol = 6;
[p,s,mu] = polyfit(t,ecgnl,opol);
f_y = polyval(p,t,[],mu);

dt_ecgnl = ecgnl - f_y;
```

```
subplot(2,1,1)
plot(t,dt_ecgl), grid
title 'Detrended ECG Signals', ylabel 'Voltage (mV)'

subplot(2,1,2)
plot(t,dt_ecgnl), grid
xlabel Sample, ylabel 'Voltage (mV)'
```



The trends have been effectively removed. Observe how the signals do not show a baseline shift anymore. They are ready for further processing.

See Also

`detrend` | `polyfit` | `polyval`

Related Examples

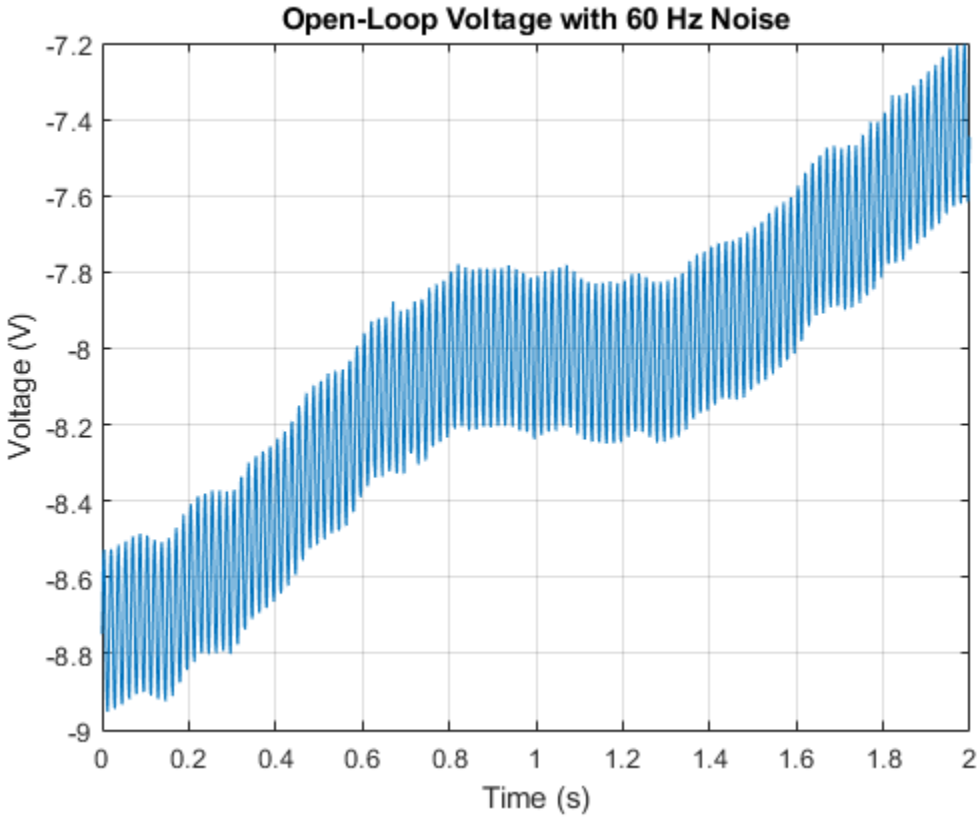
- “Peak Analysis”

Remove the 60 Hz Hum from a Signal

Alternating current in the United States and several other countries oscillates at a frequency of 60 Hz. Those oscillations often corrupt measurements and have to be subtracted.

Study the open-loop voltage across the input of an analog instrument in the presence of 60 Hz power-line noise. The voltage is sampled at 1 kHz.

```
load openloop60hertz, openLoop = openLoopVoltage;

Fs = 1000;
t = (0:length(openLoop)-1)/Fs;

plot(t,openLoop)
ylabel('Voltage (V)')
xlabel('Time (s)')
title('Open-Loop Voltage with 60 Hz Noise')
grid
```

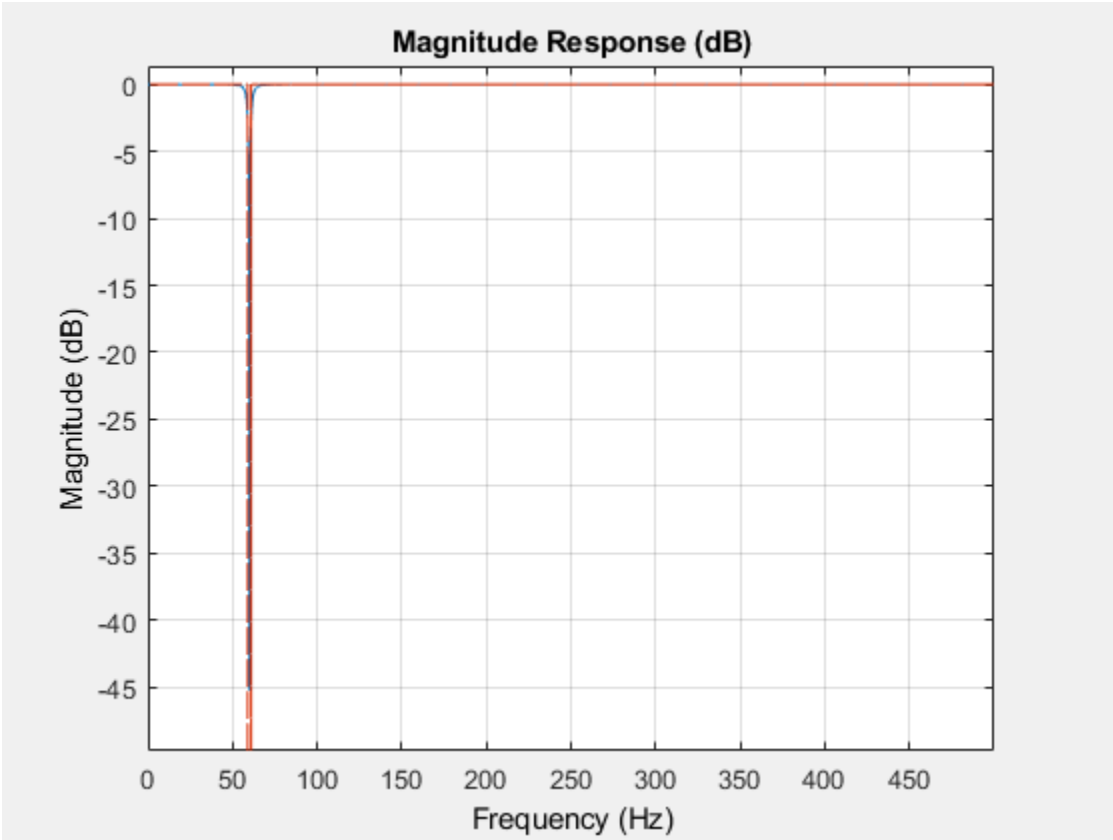


Eliminate the 60 Hz noise with a Butterworth notch filter. Use `designfilt` to design it. The width of the notch is defined by the 59 to 61 Hz frequency interval. The filter removes at least half the power of the frequency components lying in that range.

```
d = designfilt('bandstopiir','FilterOrder',2, ...
               'HalfPowerFrequency1',59,'HalfPowerFrequency2',61, ...
               'DesignMethod','butter','SampleRate',Fs);
```

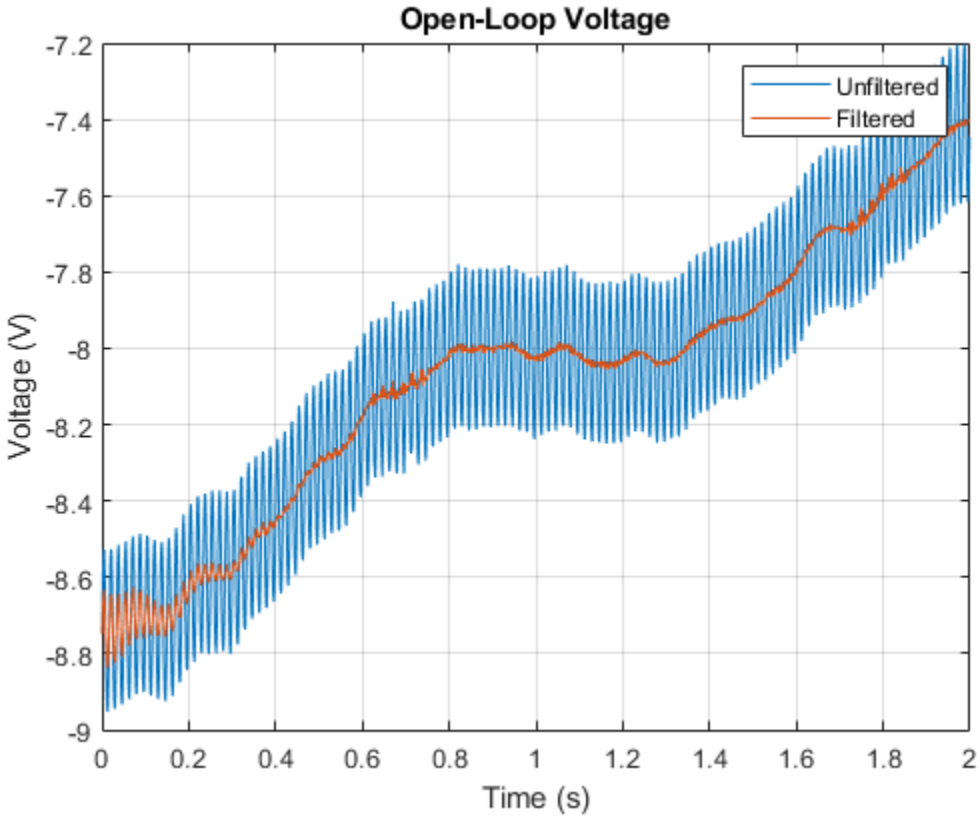
Plot the frequency response of the filter. Note that this notch filter provides up to 45 dB of attenuation.

```
fvtool(d,'Fs',Fs)
```



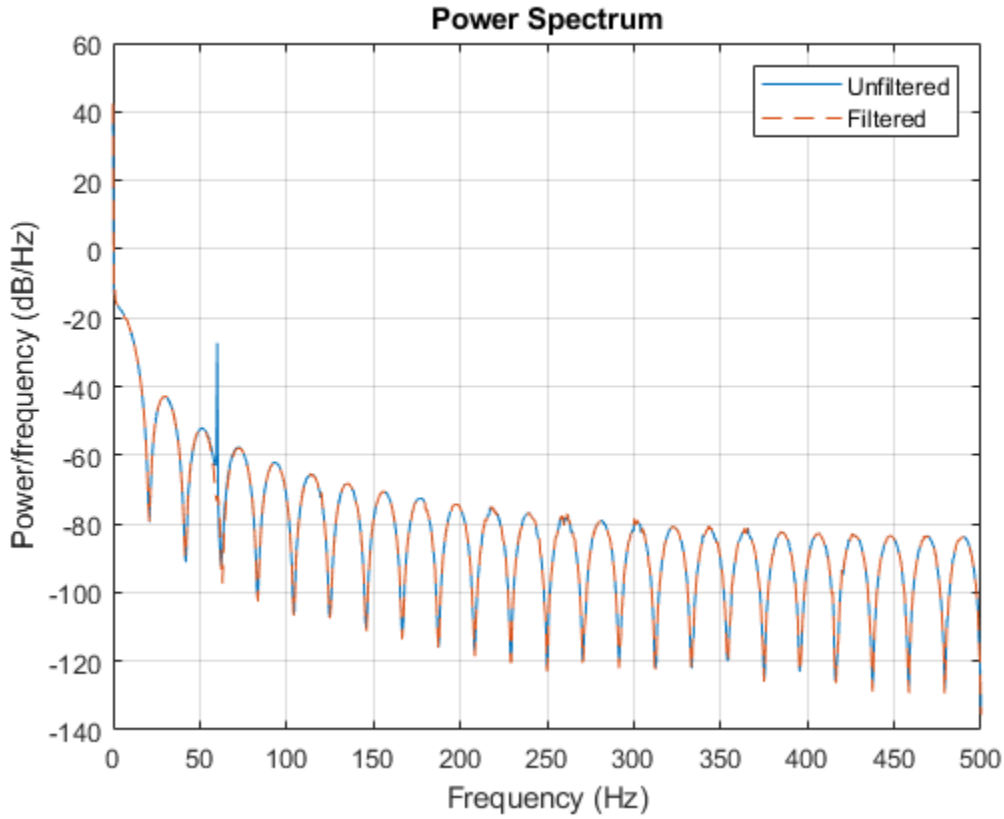
Filter the signal with `filtfilt` to compensate for filter delay. Note how the oscillations decrease significantly.

```
buttLoop = filtfilt(d,openLoop);

plot(t,openLoop,t,buttLoop)
ylabel('Voltage (V)')
xlabel('Time (s)')
title('Open-Loop Voltage')
legend('Unfiltered','Filtered')
grid
```



Use the periodogram to see that the "spike" at 60 Hz has been eliminated.

```
[popen,fopen] = periodogram(openLoop,[],[],Fs);
[pbutt,fbutt] = periodogram(buttLoop,[],[],Fs);

plot(fopen,20*log10(abs(popen)),fbutt,20*log10(abs(pbutt)),'--')
ylabel('Power/frequency (dB/Hz)')
xlabel('Frequency (Hz)')
title('Power Spectrum')
legend('Unfiltered','Filtered')
grid
```



See Also

`designfilt` | `filtfilt` | `fvtool` | `periodogram`

Related Examples

- “Signal Smoothing”

Remove Spikes from a Signal

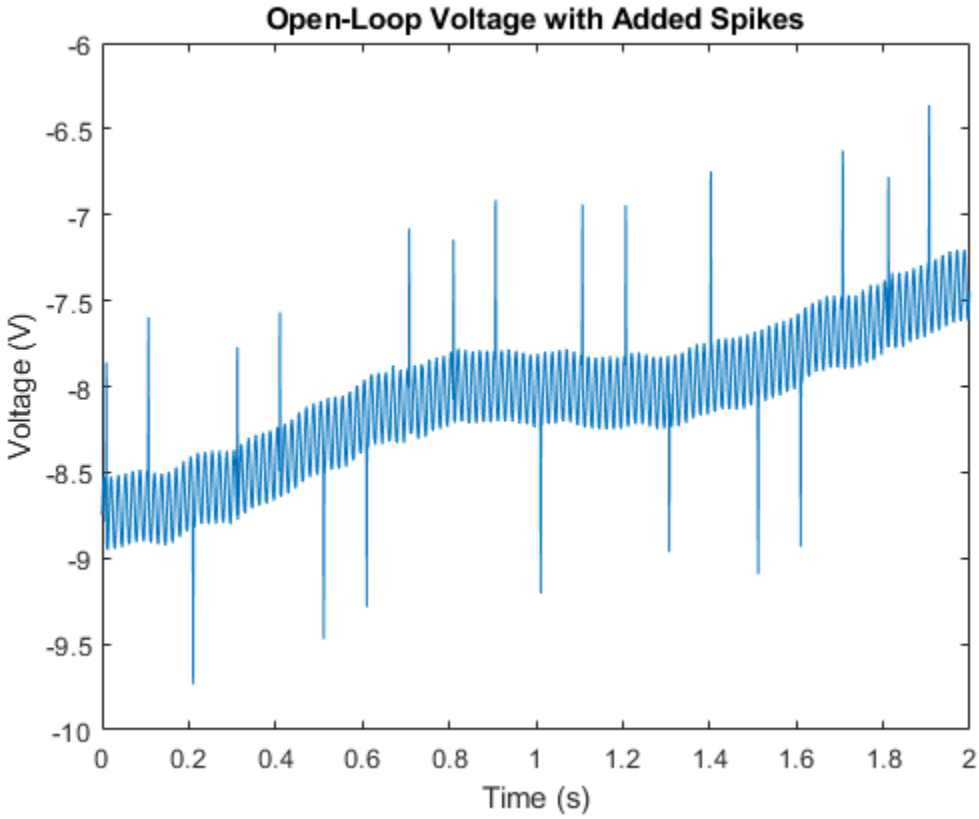
Sometimes data exhibit unwanted transients, or spikes. Median filtering is a natural way to eliminate them.

Consider the open-loop voltage across the input of an analog instrument in the presence of 60 Hz power-line noise. The sample rate is 1 kHz.

```
load openloop60hertz

fs = 1000;
t = (0:numel(openLoopVoltage) - 1)/fs;
```

Corrupt the signal by adding transients with random signs at random points. Reset the random number generator for reproducibility.

```
rng default

spikeSignal = zeros(size(openLoopVoltage));
spks = 10:100:1990;
spikeSignal(spks+round(2*randn(size(spks)))) = sign(randn(size(spks)));

noisyLoopVoltage = openLoopVoltage + spikeSignal;

plot(t,noisyLoopVoltage)

xlabel('Time (s)')
ylabel('Voltage (V)')
title('Open-Loop Voltage with Added Spikes')
```

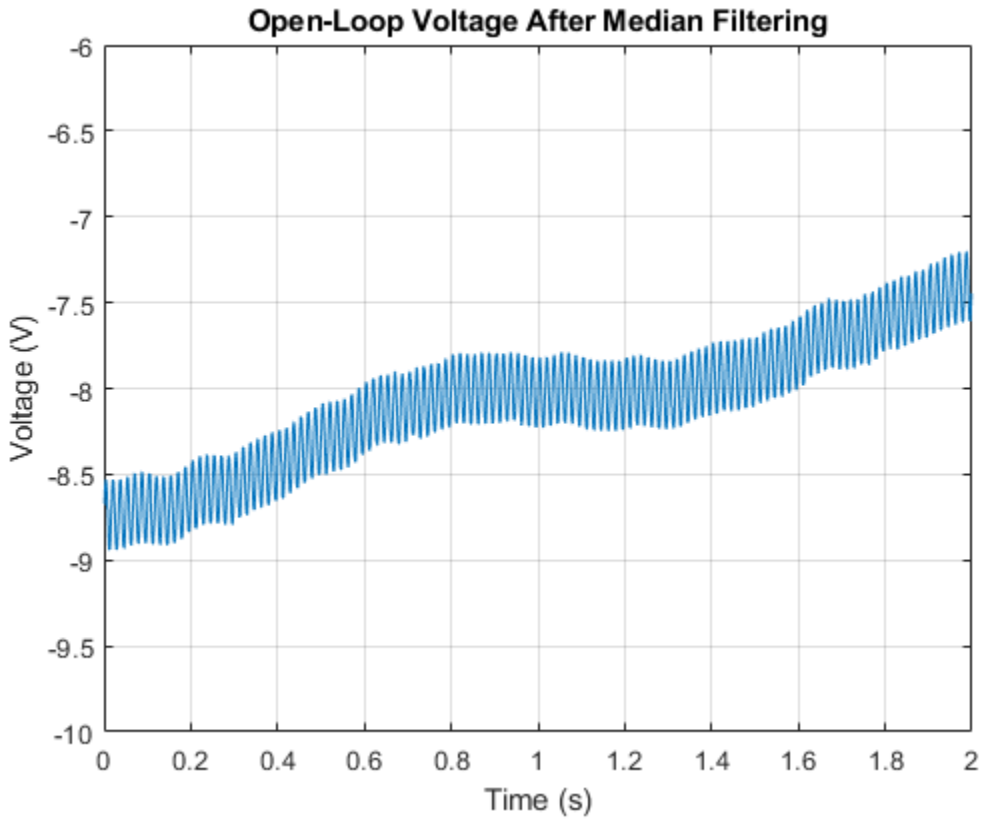



```
yax = ylim;
```

The function `medfilt1` replaces every point of a signal by the median of that point and a specified number of neighboring points. Accordingly, median filtering discards points that differ considerably from their surroundings. Filter the signal, using sets of three neighboring points to compute the medians. Note how the spikes vanish.

```
medfiltLoopVoltage = medfilt1(noisyLoopVoltage,3);

plot(t,medfiltLoopVoltage)

xlabel('Time (s)')
ylabel('Voltage (V)')
title('Open-Loop Voltage After Median Filtering')
```

```
ylim(yax)
grid
```



See Also

`medfilt1`

Related Examples

- “Signal Smoothing”

Process a Signal with Missing Samples

Consider the weight of a person as recorded (in pounds) during the leap year 2012. The person did not record their weight every day. You would like to study the periodicity of the signal, but before you can do so you must take care of the missing data.

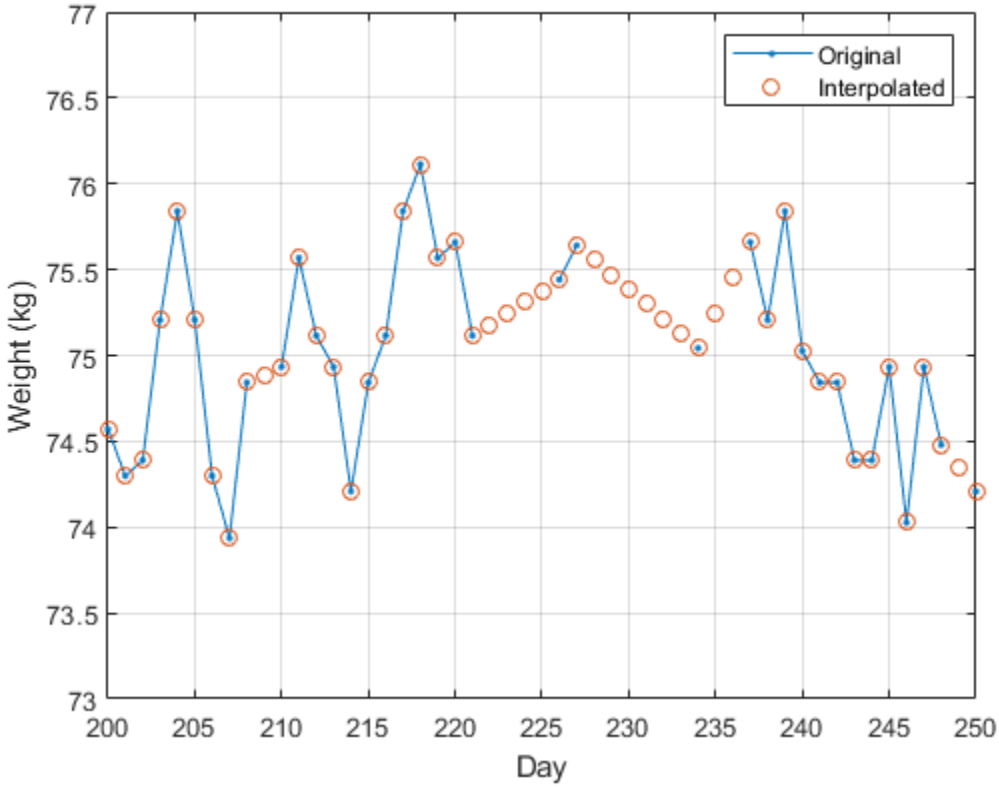
Load the data and convert the measurements to kilograms. Missed readings are set to `NaN`. Determine how many points are missing.

```
load(fullfile(matlabroot,'examples','signal','weight2012.dat'))

wgt = weight2012(:,2)/2.20462;
daynum = 1:length(wgt);
missing = isnan(wgt);

fprintf('Missing %d samples of %d\n',sum(missing),max(daynum))
```

```
Missing 27 samples of 366
```

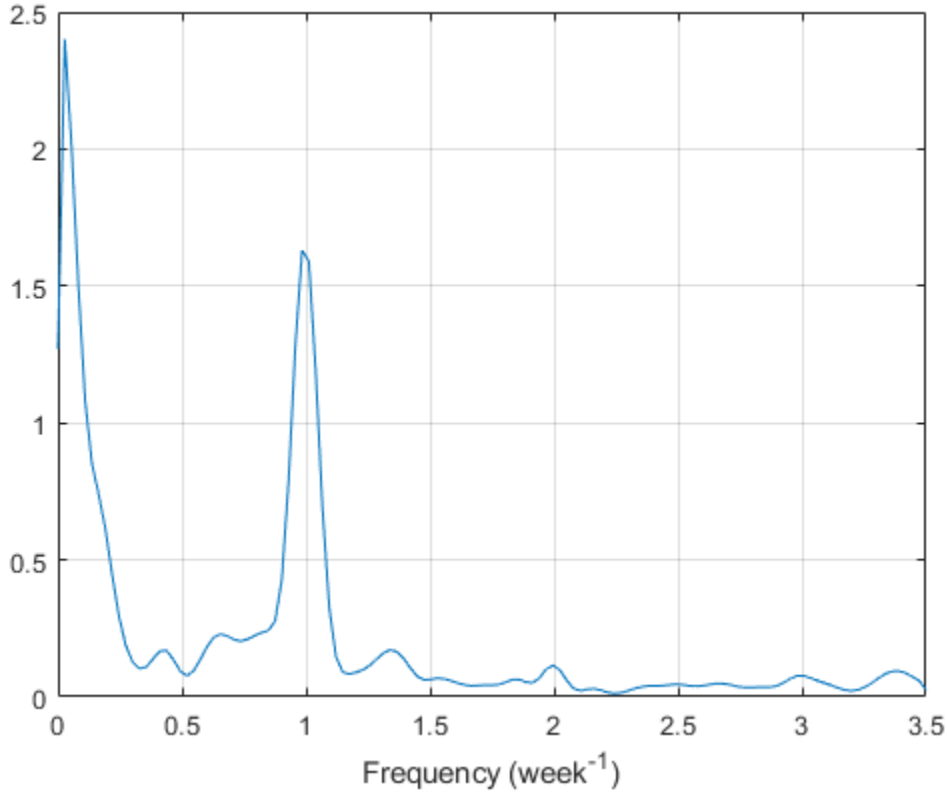
Assign values to the missing points using `resample`. By default, `resample` makes estimates using linear interpolation. Plot the original and interpolated readings. Zoom in on days 200 through 250, which contain about half of the missing points.

```
wgt_orig = wgt;
wgt = resample(wgt,daynum);

plot(daynum,wgt_orig,'.-',daynum,wgt,'o')
xlabel('Day')
ylabel('Weight (kg)')
axis([200 250 73 77])
legend('Original','Interpolated')
grid
```



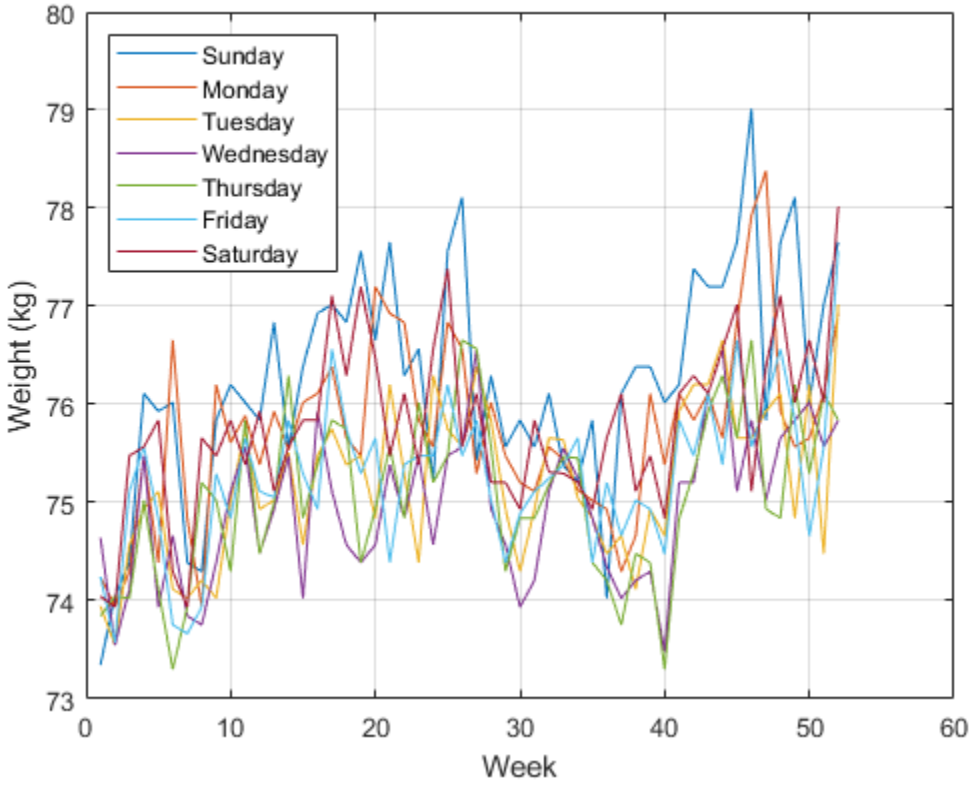
Determine if the signal is periodic by analyzing it in the frequency domain. Find the cycle duration, measuring time in weeks. Subtract the mean to concentrate on fluctuations.

```
Fs = 7;

[p,f] = pwelch(wgt-mean(wgt),[],[],[],Fs);

plot(f,p)
xlabel('Frequency (week^{-1})')
grid
```



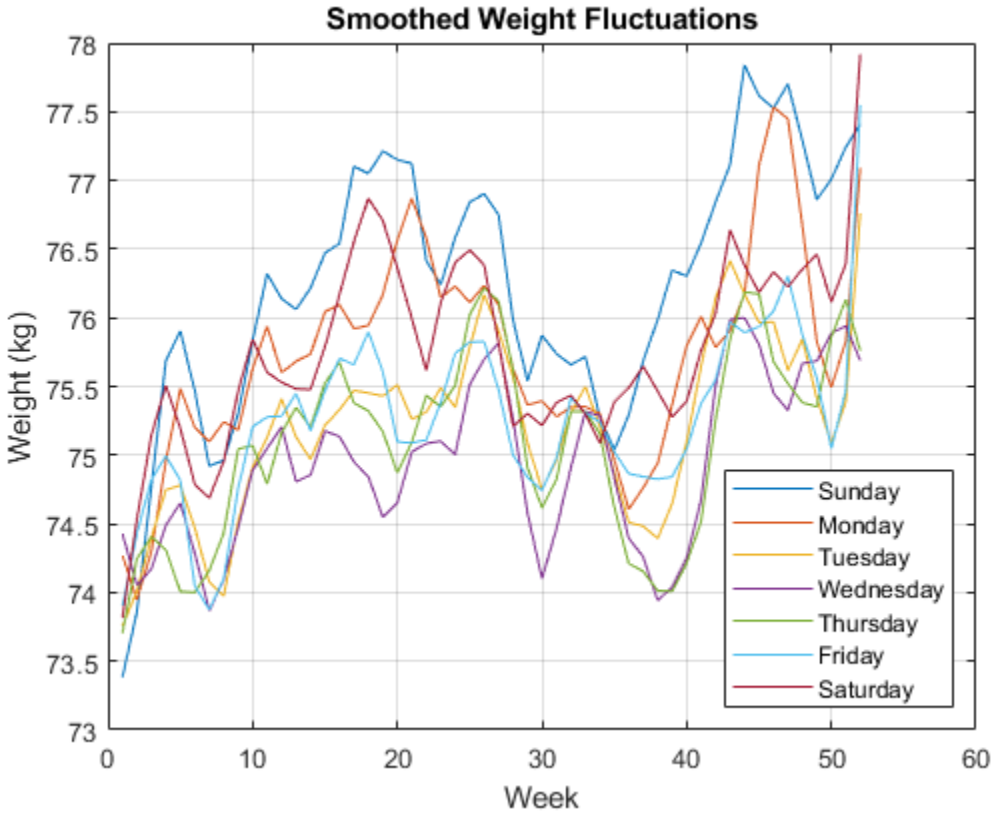
Notice how the person's weight oscillates weekly. Is there a noticeable pattern from week to week? Eliminate the last two days of the year to get 52 weeks. Reorder the measurements according to the day of the week.

```
wgd = reshape(wgt(1:7*52),[7 52]);

plot(wgd')
xlabel('Week')
ylabel('Weight (kg)')

q = legend(datestr(datenum(2012,1,1:7),'dddd'));
q.Location = 'NorthWest';
grid
```



Smooth out the fluctuations using a filter that fits low-order polynomials to subsets of the data. Specifically, set it to fit cubic polynomials to sets of seven days.

```
wgs = sgolayfilt(wgd',3,7);

plot(wgs)
xlabel('Week')
ylabel('Weight (kg)')
title('Smoothed Weight Fluctuations')

q = legend(datestr(datenum(2012,1,1:7),'dddd'));
q.Location = 'SouthEast';
grid
```



This person tends to eat more, and thus weigh more, during the weekend. Verify by computing the daily means.

```
for jk = 1:7
    fprintf('%3s mean: %5.1f kg\n', ...
        datestr(datenum(2012,1,jk),'ddd')',mean(wgd(jk,:)))
end
```

```
Sun mean:  76.2 kg
Mon mean:  75.7 kg
Tue mean:  75.2 kg
Wed mean:  74.9 kg
Thu mean:  75.1 kg
```

```
Fri mean:  75.3 kg
Sat mean:  75.8 kg
```

See Also

`pwelch` | `sgolayfilt`

Related Examples

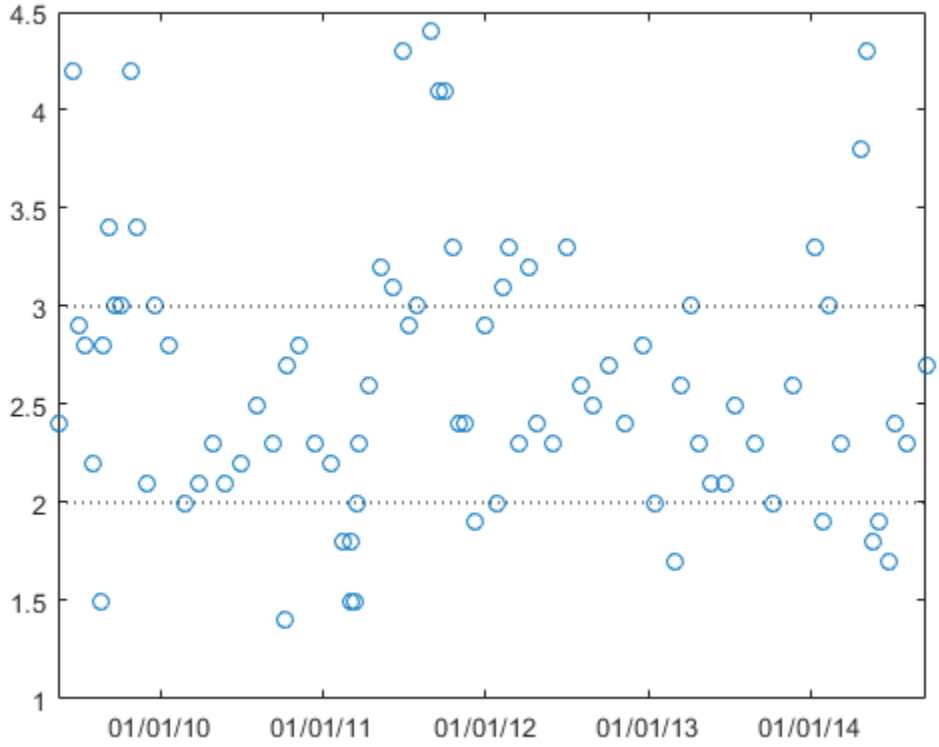
- “Signal Smoothing”

Reconstruct a Signal from Irregularly Sampled Data

People predisposed to blood clotting are treated with warfarin, a blood thinner. The international normalized ratio (INR) measures the effect of the drug. Larger doses increase the INR and smaller doses decrease it. Patients are monitored regularly by a nurse, and when their INRs fall out of the target range, their doses and the frequencies of their tests change.

The file `INR.mat` contains the INR measurements performed on a patient over a five-year period. The file includes a `datetime` array with the date and time of each measurement, and a vector with the corresponding INR readings. Load the data. Plot the INR as a function of time and overlay the target INR range.

```
load(fullfile(matlabroot,'examples','signal','INR.mat'))

plot(Date,INR,'o','DatetimeTickFormat','MM/dd/yy')

xlim([Date(1) Date(end)])
hold on
plot([xlim;xlim]',[2 3;2 3],'k:')
```

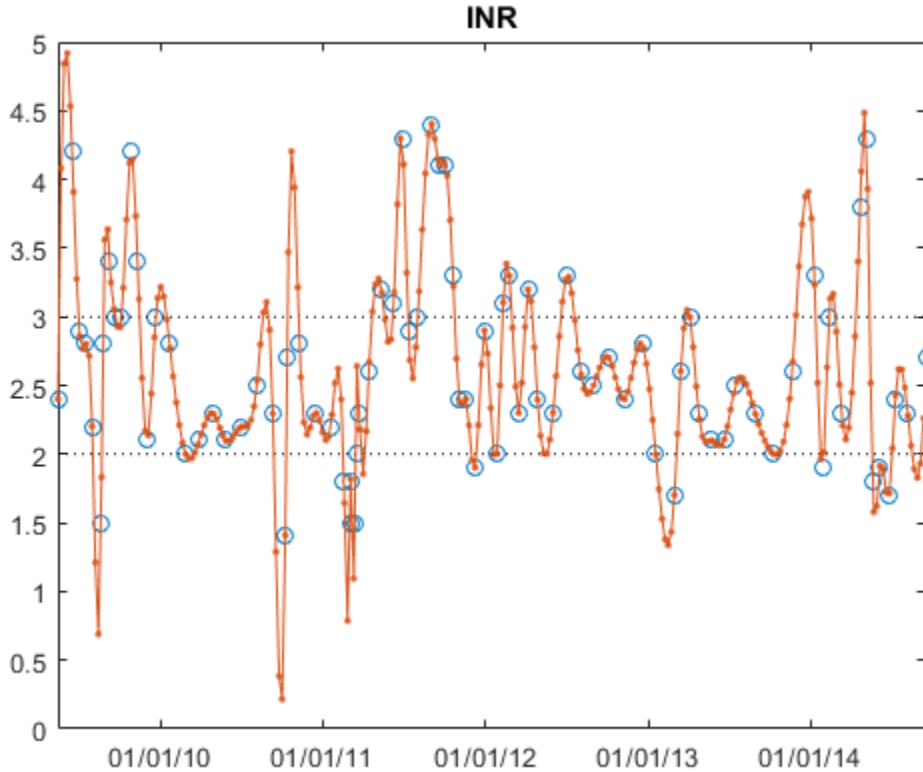


Resample the data to make the INR readings uniformly spaced. The first reading was taken at 11:28 a.m. on a Friday. Use `resample` to estimate the patient's INR at that time on every subsequent Friday. Specify a sample rate of one reading per week, or equivalently, $1/(7\times 86400)$ readings per second. Use spline interpolation for the resampling.

```
Date.Format = 'eeee, MM/dd/yy, HH:mm';
First = Date(1)
```

```
First = datetime
   Friday, 05/15/09, 11:28
```

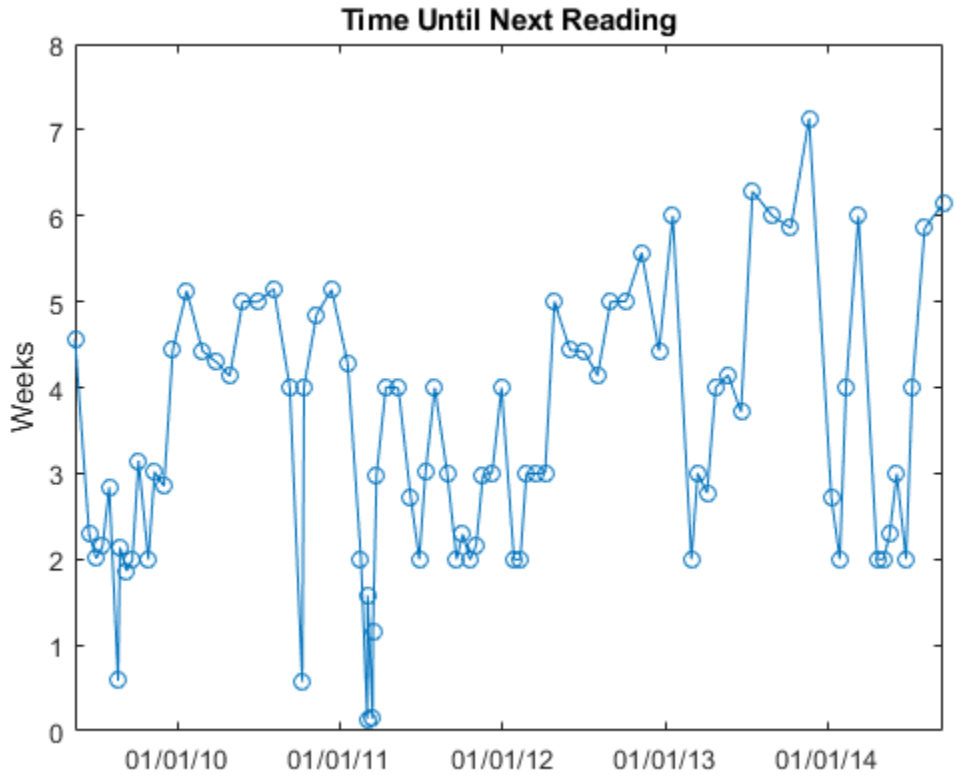
```
perweek = 1/7/86400;

[rum,tee] = resample(INR,Date,perweek,1,1,'spline');

plot(tee,rum,'.-','DatetimeTickFormat','MM/dd/yy')

title('INR')
xlim([Date(1) Date(end)])
hold off
```



Each INR reading determines when the patient must be tested next. Use `diff` to construct a vector of time intervals between measurements. Express the intervals in weeks and plot them using the same *x*-axis as before. For the last point, use the next date

prescribed by the anticoagulation nurse. The measurements are carried out in the United States.

```
nxt = datetime('10/30/2014 07:00 PM','Locale','en_US');

plot(Date,diff(datenum([Date;nxt]))/7,'o-', ...
    'DatetimeTickFormat','MM/dd/yy')

title('Time Until Next Reading')
xlim([Date(1) Date(end)])
ylabel('Weeks')
```



When the INR is out of range, the times between INR readings remain short. When the INR is too low, patients get their readings more often because the risk of thrombosis is

elevated. When the patient's INR is in range, the times between readings increase steadily until the ratio becomes too small or too large.

The large fluctuations in the resampling could be a sign of overshooting. However, warfarin has an enormous effect on the body. Small changes in warfarin dose can change the INR drastically, as can changes in diet, time spent in airplanes, or other factors. Moreover, when the ratio goes very low (as in late 2010, where the fluctuations are largest), the warfarin is supplemented by emergency injections of enoxaparin, whose effects are even greater.

See Also

`datenum` | `datestr` | `resample`

External Websites

- National Institutes of Health. *Blood Thinners*. https://www.nlm.nih.gov/medlineplus/bloodthinners.html

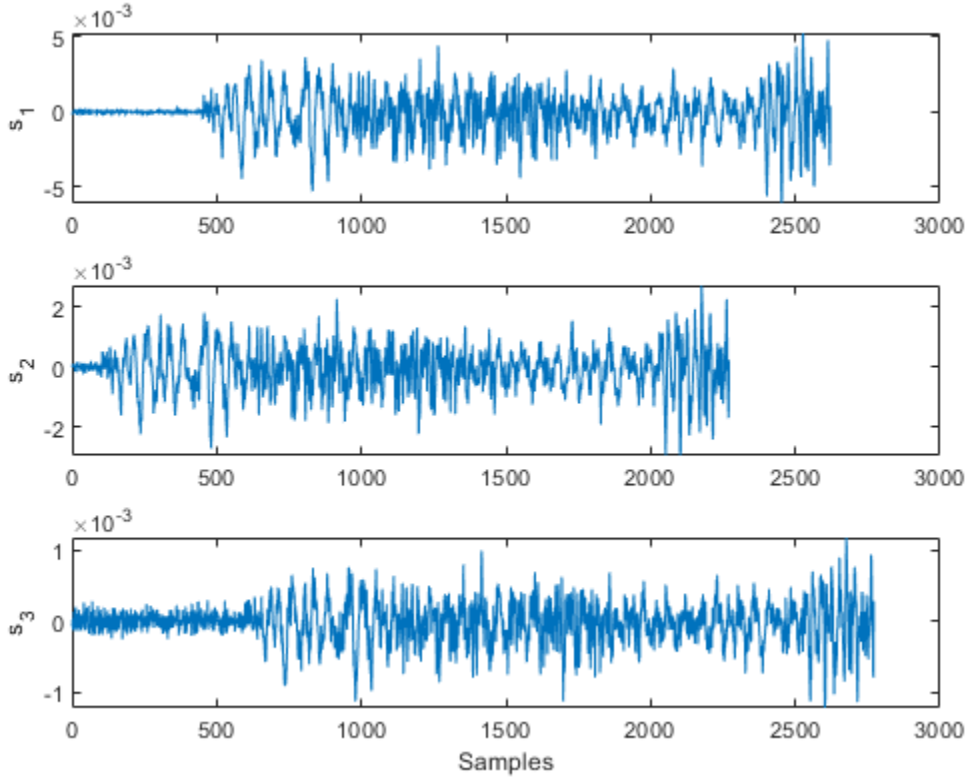
Align Signals with Different Start Times

Many measurements involve data collected asynchronously by multiple sensors. If you want to integrate the signals, you have to synchronize them. The Signal Processing Toolbox™ has functions that let you do just that.

For example, consider a car crossing a bridge. The vibrations it produces are measured by three identical sensors located at different spots. The signals have different arrival times.

Load the signals into the MATLAB® workspace and plot them.

```
load relatedsig

ax(1) = subplot(3,1,1);
plot(s1)
ylabel('s_1')

ax(2) = subplot(3,1,2);
plot(s2)
ylabel('s_2')

ax(3) = subplot(3,1,3);
plot(s3)
ylabel('s_3')
xlabel('Samples')

linkaxes(ax,'x')
```



Signal `s1` lags `s2` and in turn leads `s3`. The delays can be computed exactly using `finddelay`. You see that `s2` leads `s1` by 350 samples, `s3` lags `s1` by 150 samples, and `s2` leads `s3` by 500 samples.

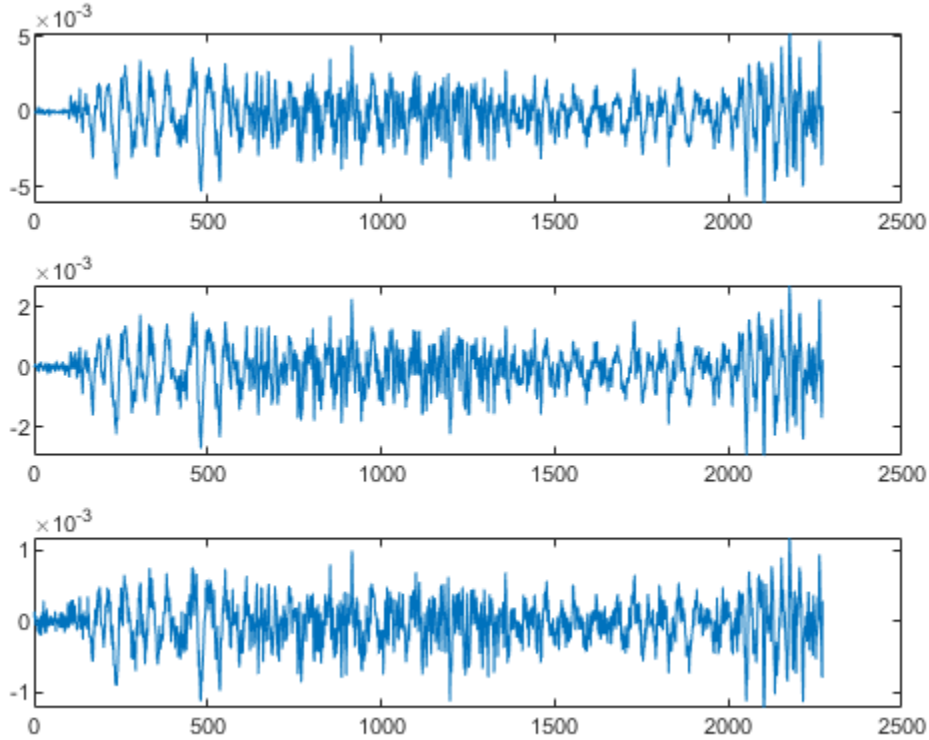
```
t21 = finddelay(s2,s1)
t31 = finddelay(s3,s1)
t32 = finddelay(s2,s3)
```

```
t21 =

   350
```

```
t31 =

  -150


t32 =

   500
```

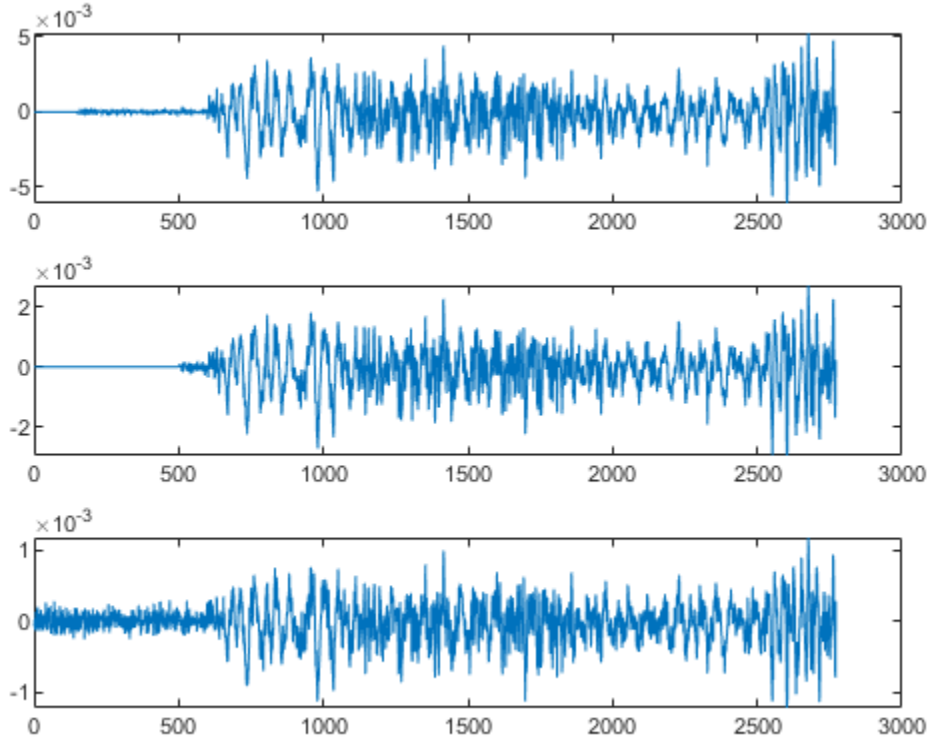
Line up the signals by leaving the earlier signal untouched and clipping the delays out of the other vectors. Add 1 to the lag differences to account for the one-based indexing used by MATLAB®. This method aligns the signals using as reference the earliest arrival time, that of `s2`.

```
axes(ax(1))
plot(s1(t21+1:end))

axes(ax(2))
plot(s2)

axes(ax(3))
plot(s3(t32+1:end))
```



Use `alignsignals` to align the signals. The function works by delaying earlier signals, so use as reference the latest arrival time, that of `s3`.

```
[x1,x3] = alignsignals(s1,s3);
x2 = alignsignals(s2,s3);

axes(ax(1))
plot(x1)

axes(ax(2))
plot(x2)

axes(ax(3))
plot(x3)
```



The signals are now synchronized and ready for further processing.

See Also

`alignsignals` | `finddelay` | `xcorr`

Related Examples

- “Measuring Signal Similarities”

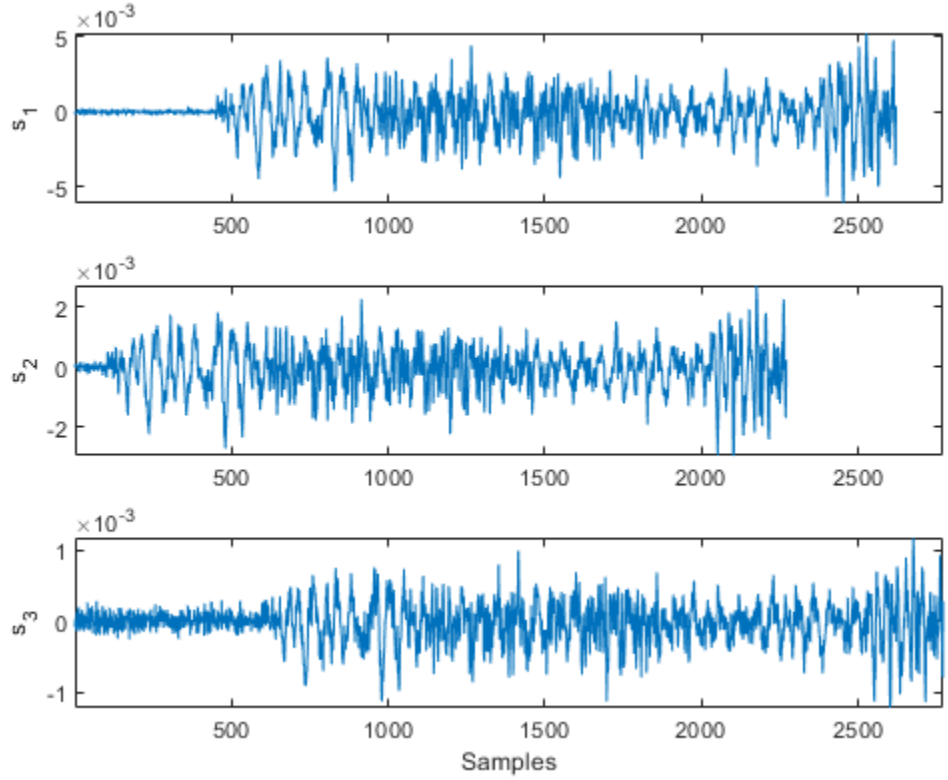
Align Signals Using Cross-Correlation

Many measurements involve data collected asynchronously by multiple sensors. If you want to integrate the signals and study them in tandem, you have to synchronize them. Use `xcorr` for that purpose.

For example, consider a car crossing a bridge. The vibrations it produces are measured by three identical sensors located at different spots. The signals have different arrival times.

Load the signals into the MATLAB® workspace and plot them.

```
load relatedsig

ax(1) = subplot(3,1,1);
plot(s1)
ylabel('s_1')
axis tight

ax(2) = subplot(3,1,2);
plot(s2)
ylabel('s_2')
axis tight

ax(3) = subplot(3,1,3);
plot(s3)
ylabel('s_3')
axis tight
xlabel('Samples')

linkaxes(ax,'x')
```



Compute the cross-correlations between the three pairs of signals. Normalize them so their maximum value is one.

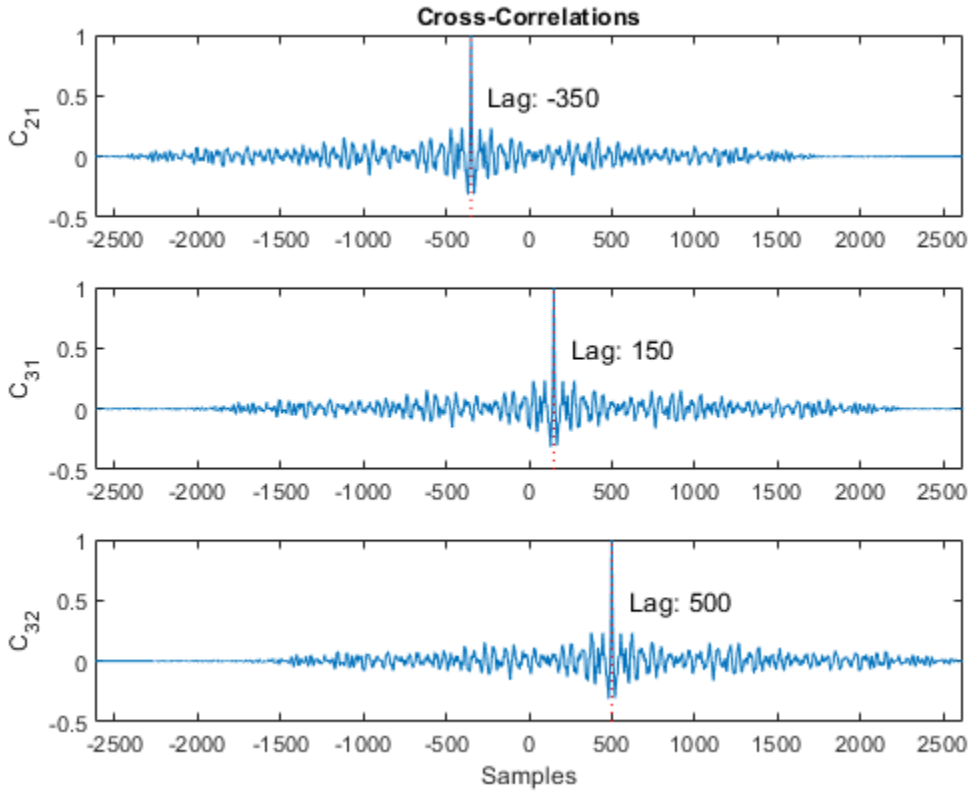
```
[C21,lag21] = xcorr(s2,s1);
C21 = C21/max(C21);

[C31,lag31] = xcorr(s3,s1);
C31 = C31/max(C31);

[C32,lag32] = xcorr(s3,s2);
C32 = C32/max(C32);
```

The locations of the maximum values of the cross-correlations indicate time leads or lags.

```
[M21,I21] = max(C21);
t21 = lag21(I21);

[M31,I31] = max(C31);
t31 = lag31(I31);

[M32,I32] = max(C32);
t32 = lag31(I32);
```

Plot the cross-correlations. In each plot display the location of the maximum.

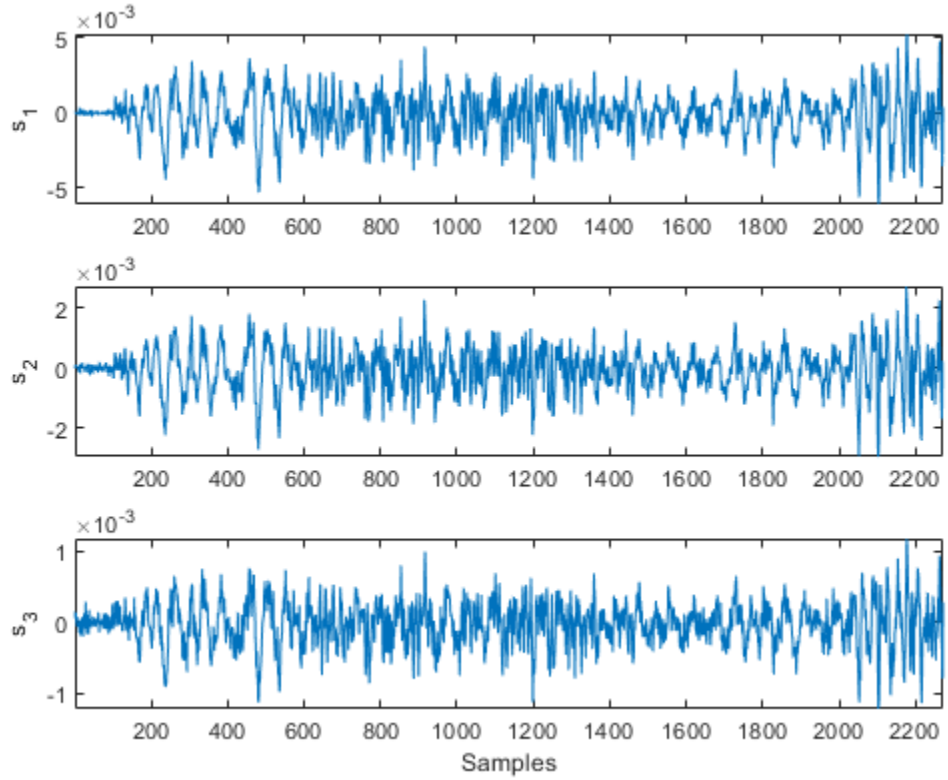
```
subplot(3,1,1)
plot(lag21,C21,[t21 t21],[-0.5 1],'r:')
text(t21+100,0.5,['Lag: ' int2str(t21)])
ylabel('C_{21}')
axis tight
title('Cross-Correlations')

subplot(3,1,2)
plot(lag31,C31,[t31 t31],[-0.5 1],'r:')
text(t31+100,0.5,['Lag: ' int2str(t31)])
ylabel('C_{31}')
axis tight

subplot(3,1,3)
plot(lag32,C32,[t32 t32],[-0.5 1],'r:')
text(t32+100,0.5,['Lag: ' int2str(t32)])
ylabel('C_{32}')
axis tight
xlabel('Samples')
```



`s2` leads `s1` by 350 samples; `s3` lags `s1` by 150 samples. Thus `s2` leads `s3` by 500 samples. Line up the signals by clipping the vectors with longer delays.

```
s1 = s1(-t21:end);
s3 = s3(t32:end);

ax(1) = subplot(3,1,1);
plot(s1)
ylabel('s_1')
axis tight

ax(2) = subplot(3,1,2);
plot(s2)
ylabel('s_2')
```

```
axis tight

ax(3) = subplot(3,1,3);
plot(s3)
ylabel('s_3')
axis tight
xlabel('Samples')

linkaxes(ax,'x')
```



The signals are now synchronized and ready for further processing.

See Also

`alignsignals` | `finddelay` | `xcorr`

Related Examples

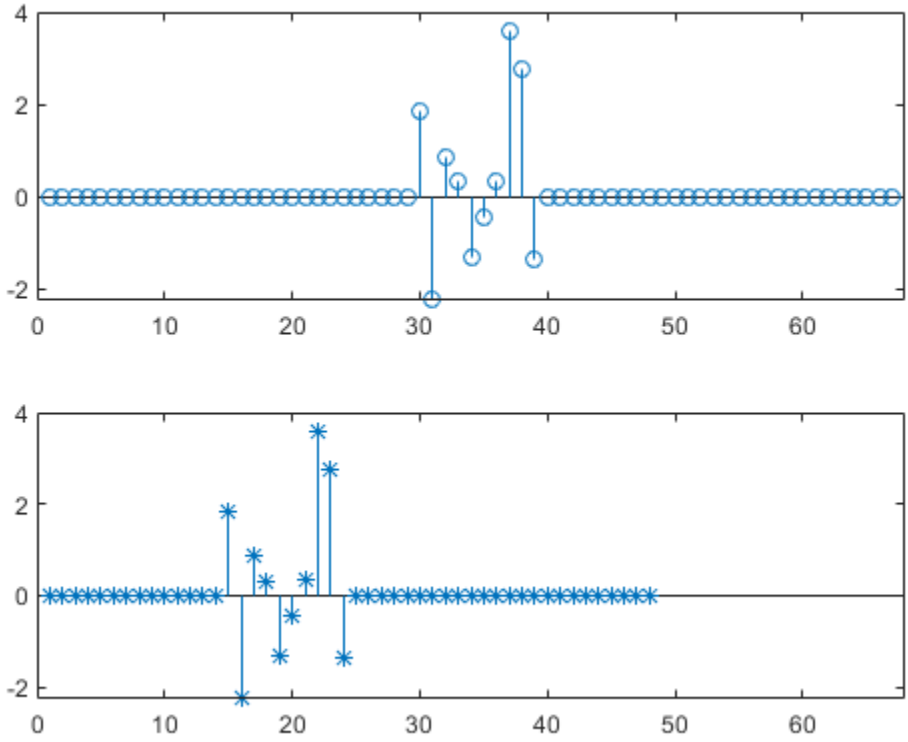
- “Measuring Signal Similarities”

Align Two Simple Signals

This example shows how to use cross-correlation to align signals. In the most general case, the signals have different lengths, and to synchronize them properly, you must take into account the lengths and the order in which you input the arguments to `xcorr`.

Consider two signals, identical except for the number of surrounding zeros and for the fact that one of them lags the other.

```
sz = 30;
sg = randn(1,randi(8)+3);
s1 = [zeros(1,randi(sz)-1) sg zeros(1,randi(sz)-1)];
s2 = [zeros(1,randi(sz)-1) sg zeros(1,randi(sz)-1)];

mx = max(numel(s1),numel(s2));

subplot(2,1,1)
stem(s1)
xlim([0 mx+1])

subplot(2,1,2)
stem(s2,'*')
xlim([0 mx+1])
```



Determine which of the two signals is longer than the other in the sense of having more elements, be they zeros or not.

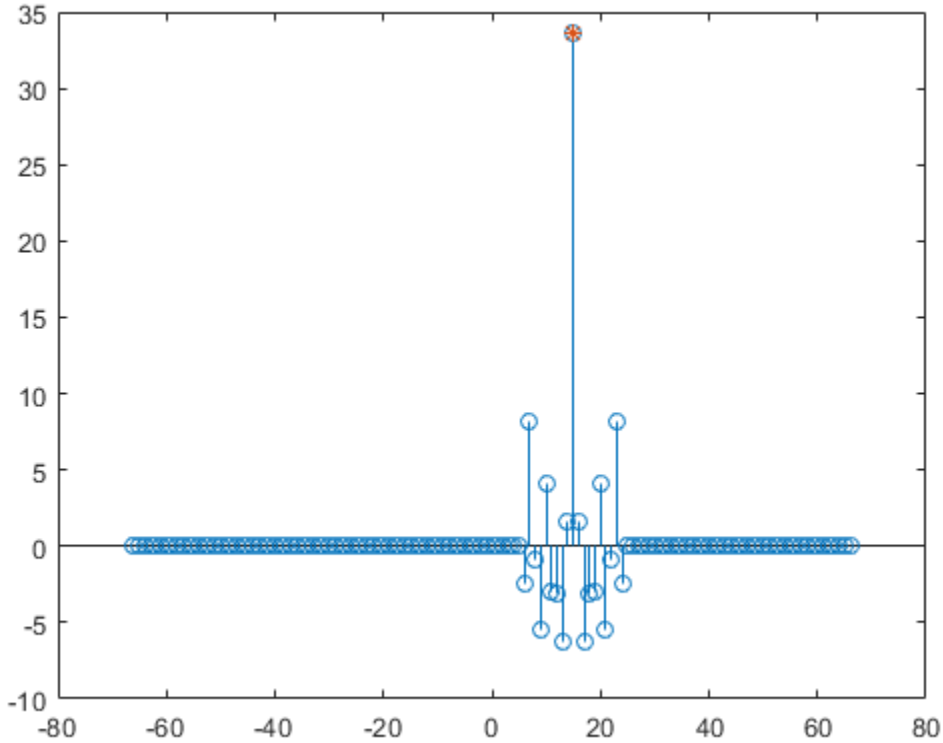
```
if numel(s1) > numel(s2)
    slong = s1;
    sshort = s2;
else
    slong = s2;
    sshort = s1;
end
```

Compute the cross-correlation of the two signals. Run `xcorr` with the longer signal as first argument and the shorter signal as second argument. Plot the result.

```
[acor,lag] = xcorr(slong,sshort);

[acormax,I] = max(abs(acor));
lagDiff = lag(I)
```

```
lagDiff = 15
```

```
figure
stem(lag,acor)
hold on
plot(lagDiff,acormax,'*')
hold off
```



Align the signals. Think of the lagging signal as being "longer" than the other, in the sense that you have to "wait longer" to detect it.

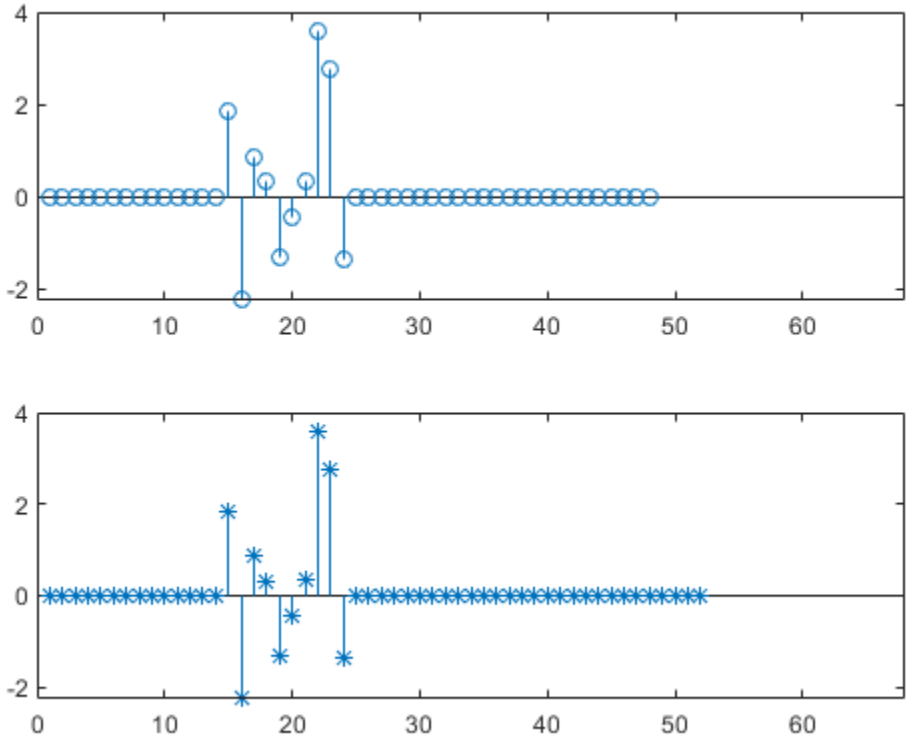
- If `lagDiff` is positive, "shorten" the long signal by considering its elements from `lagDiff+1` to the end.
- If `lagDiff` is negative, "lengthen" the short signal by considering its elements from `-lagDiff+1` to the end.

You must add 1 to the lag difference because MATLAB® uses one-based indexing.

```
if lagDiff > 0
    sorig = sshort;
    salign = slong(lagDiff+1:end);
else
    sorig = slong;
    salign = sshort(-lagDiff+1:end);
end
```

Plot the aligned signals.

```
subplot(2,1,1)
stem(sorig)
xlim([0 mx+1])

subplot(2,1,2)
stem(salign,'*')
xlim([0 mx+1])
```

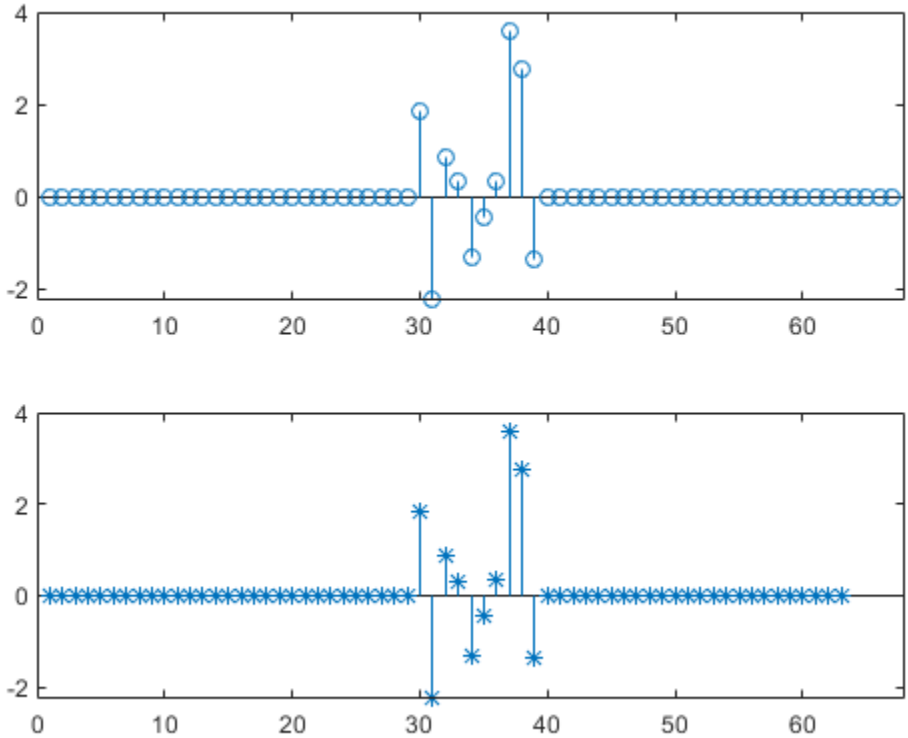


The method works because the cross-correlation operation is antisymmetric and because `xcorr` deals with signals of different lengths by adding zeros at the *end* of the shorter signal. This interpretation lets you align the signals easily using MATLAB's `end` operator without having to pad them by hand.

You can also align the signals at one stroke by invoking the `alignsignals` function.

```
[x1,x2] = alignsignals(s1,s2);

subplot(2,1,1)
stem(x1)
xlim([0 mx+1])

subplot(2,1,2)
```

```
stem(x2,'*')
xlim([0 mx+1])
```



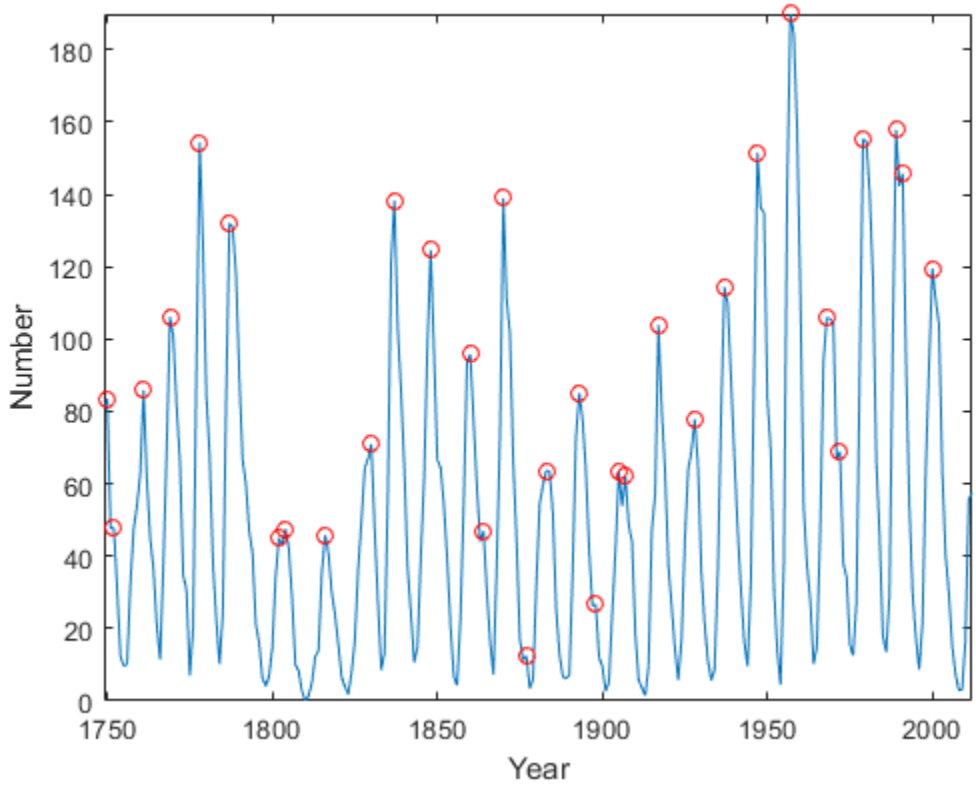
Find Peaks in Data

Use `findpeaks` to find values and locations of local maxima in a set of data.

The file `spots_num.mat` contains the average number of sunspots observed every year from 1749 to 2012. The data are available from NASA.

Find the maxima and their years of occurrence. Plot them along with the data.

```
load(fullfile(matlabroot,'examples','signal','spots_num.mat'))

[pks,locs] = findpeaks(avSpots);

plot(year,avSpots,year(locs),pks,'or')
xlabel('Year')
ylabel('Number')
axis tight
```

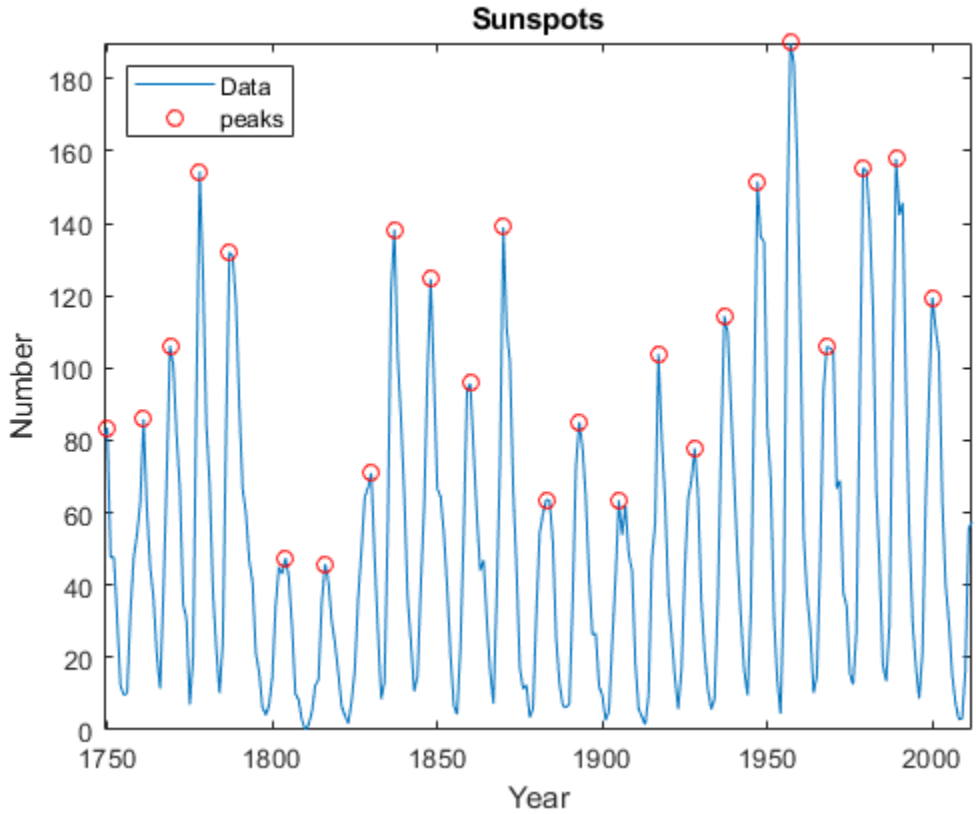


Some peaks are very close to each other. The ones that are not recur at regular intervals. There are roughly five such peaks per 50-year period.

To make a better estimate of the cycle duration, use `findpeaks` again, but this time restrict the peak-to-peak separation to at least six years. Compute the mean interval between maxima.

```
[pks,locs] = findpeaks(avSpots,'MinPeakDistance',6);

plot(year,avSpots,year(locs),pks,'or')
xlabel('Year')
ylabel('Number')
title('Sunspots')
axis tight
```

```
legend('Data','peaks','Location','NorthWest')
```

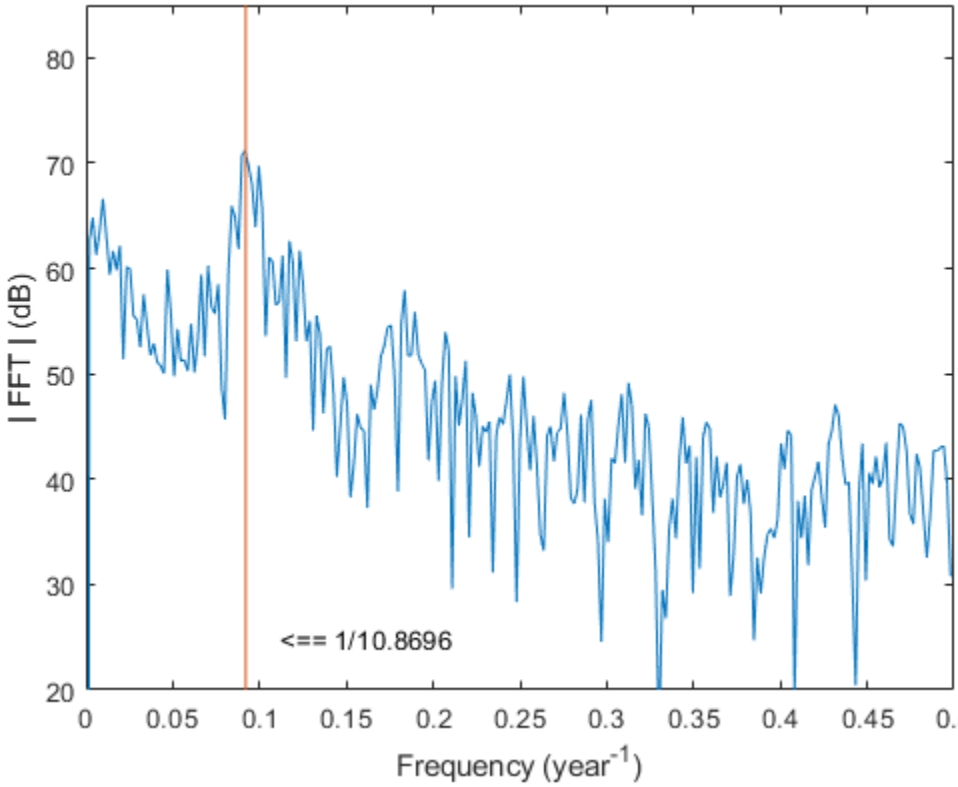



```
cycles = diff(locs);
meanCycle = mean(cycles)
```

```
meanCycle = 10.8696
```

It is well known that solar activity cycles roughly every 11 years. Check by using the Fourier transform. Remove the mean of the signal to concentrate on its fluctuations. Recall that the sample rate is measured in years. Use frequencies up to the Nyquist frequency.

```
Fs = 1;
Nf = 512;

df = Fs/Nf;
f = 0:df:Fs/2-df;

trSpots = fftshift(fft(avSpots-mean(avSpots),Nf));

dBspots = 20*log10(abs(trSpots(Nf/2+1:Nf)));

yaxis = [20 85];
plot(f,dBspots,1./[meanCycle meanCycle],yaxis)
xlabel('Frequency (year^{-1})')
ylabel('| FFT | (dB)')
axis([0 1/2 yaxis])
text(1/meanCycle + .02,25,['<== 1/' num2str(meanCycle)])
```

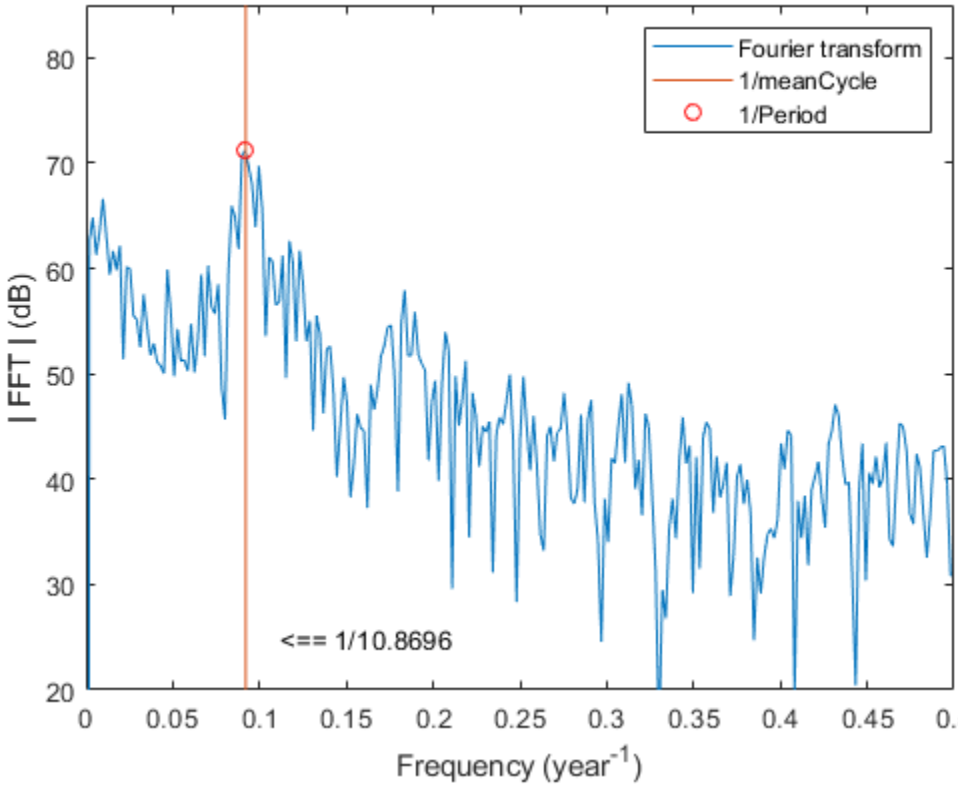


The Fourier transform indeed peaks at the expected frequency, confirming the 11-year conjecture. You also can find the period by locating the highest peak of the Fourier transform.

```
[pk,MaxFreq] = findpeaks(dBspots,'NPeaks',1,'SortStr','descend');

Period = 1/f(MaxFreq)
```

```
Period = 10.8936
```

```
hold on
plot(f(MaxFreq),pk,'or')
hold off
legend('Fourier transform','1/meanCycle','1/Period')
```



The two estimates coincide quite well.

See Also

`dlmread` | `findpeaks`

Related Examples

- “Peak Analysis”

Find a Signal in a Measurement

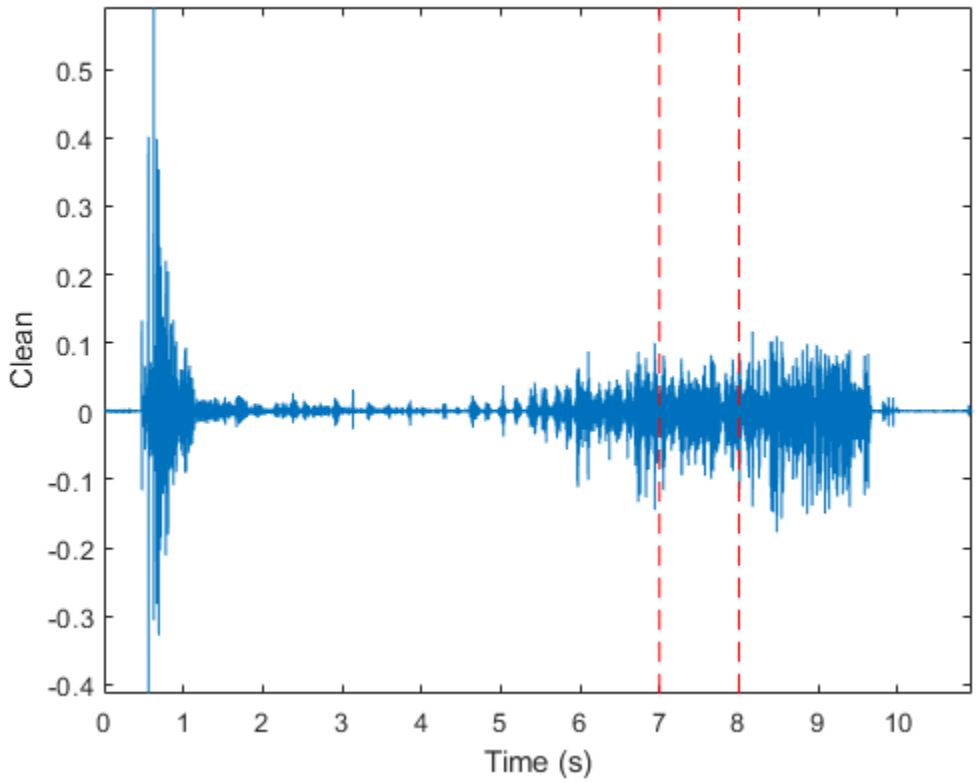
You receive some data and would like to know if it matches a longer stream you have measured. Cross-correlation allows you to make that determination, even when the data are corrupted by noise.

Load into the workspace a recording of a ring spinning on a tabletop. Crop a one-second fragment and listen to it.

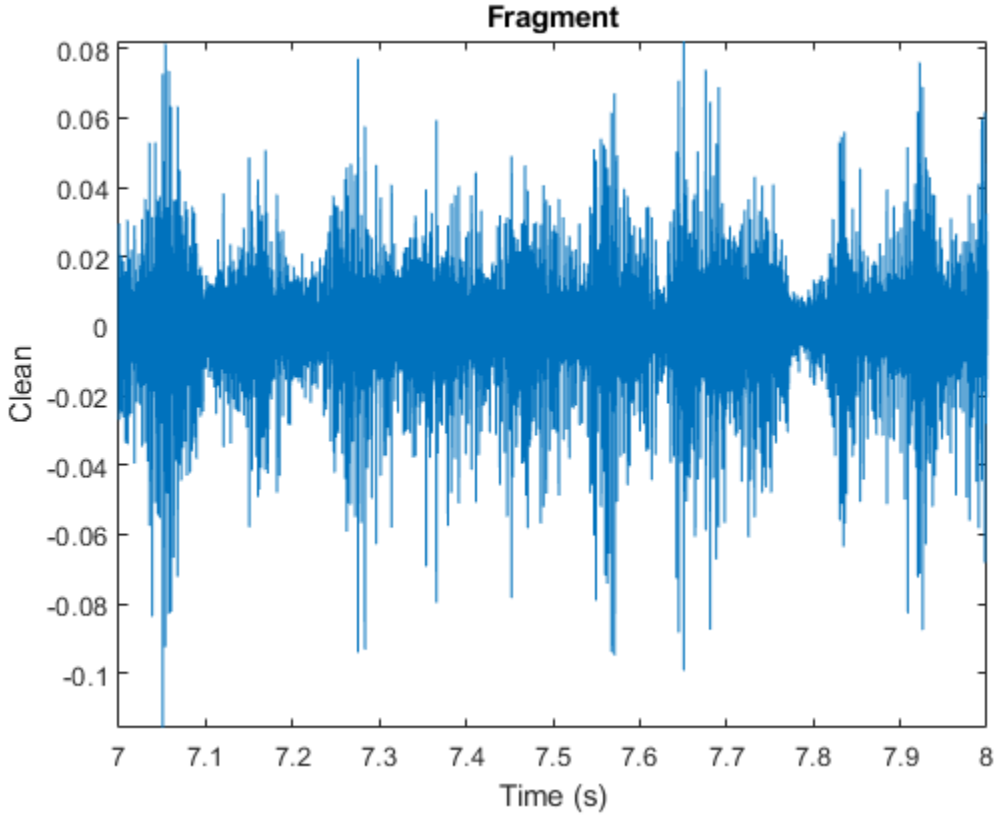
```
load(fullfile(matlabroot,'examples','signal','Ring.mat'))

Time = 0:1/Fs:(length(y)-1)/Fs;

m = min(y);
M = max(y);

Full_sig = double(y);

timeA = 7;
timeB = 8;
snip = timeA*Fs:timeB*Fs;

Fragment = Full_sig(snip);

% To hear, type soundsc(Fragment,Fs)
```

Plot the signal and the fragment. Highlight the fragment endpoints for reference.

```
plot(Time,Full_sig,[timeA timeB;timeA timeB],[m m;M M],'r--')
xlabel('Time (s)')
ylabel('Clean')
axis tight
```

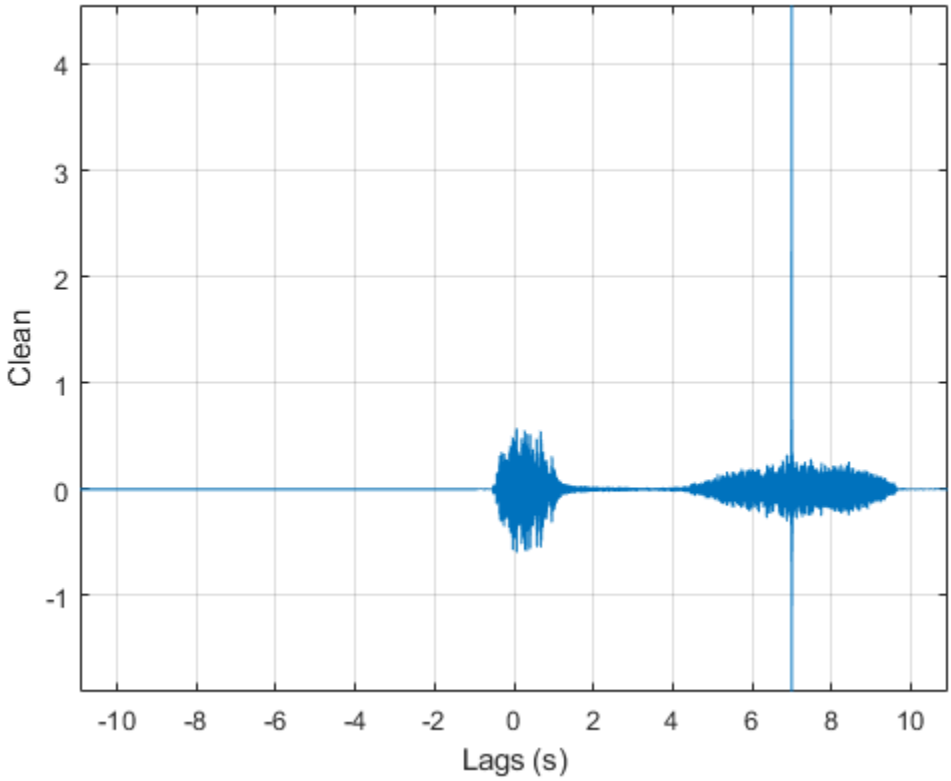



```
plot(snip/Fs,Fragment)
xlabel('Time (s)')
ylabel('Clean')
title('Fragment')
axis tight
```



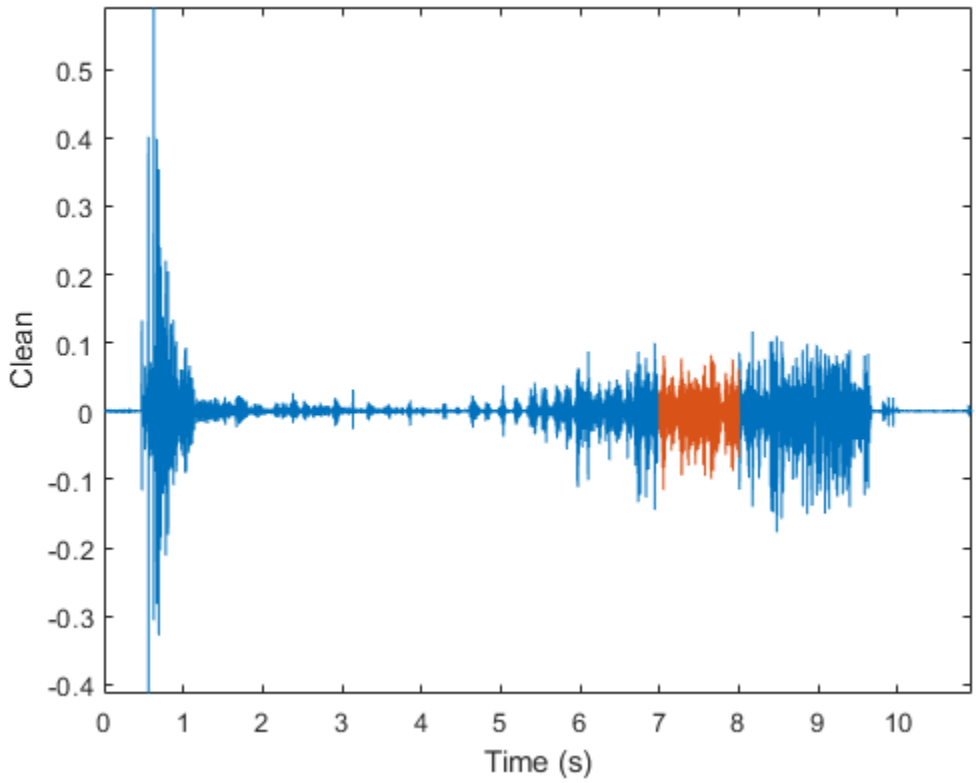
Compute and plot the cross-correlation of the full signal and the fragment.

```
[xCorr,lags] = xcorr(Full_sig,Fragment);

plot(lags/Fs,xCorr)
grid
xlabel('Lags (s)')
ylabel('Clean')
axis tight
```



The lag at which the cross-correlation is greatest is the time delay between the signals' starting points. Replot the signal and overlay the fragment.

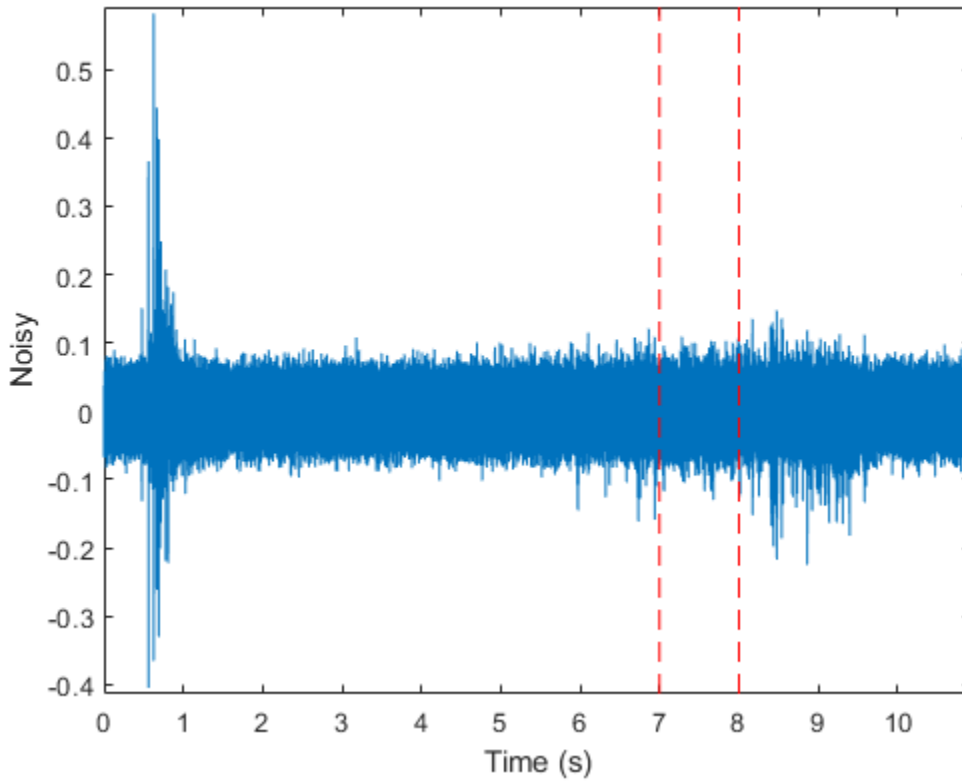
```
[~,I] = max(abs(xCorr));
maxt = lags(I);

Trial = NaN(size(Full_sig));
Trial(maxt+1:maxt+length(Fragment)) = Fragment;

plot(Time,Full_sig,Time,Trial)
xlabel('Time (s)')
ylabel('Clean')
axis tight
```



Repeat the procedure, but add noise separately to signal and fragment. The sound cannot be picked out from the noise.

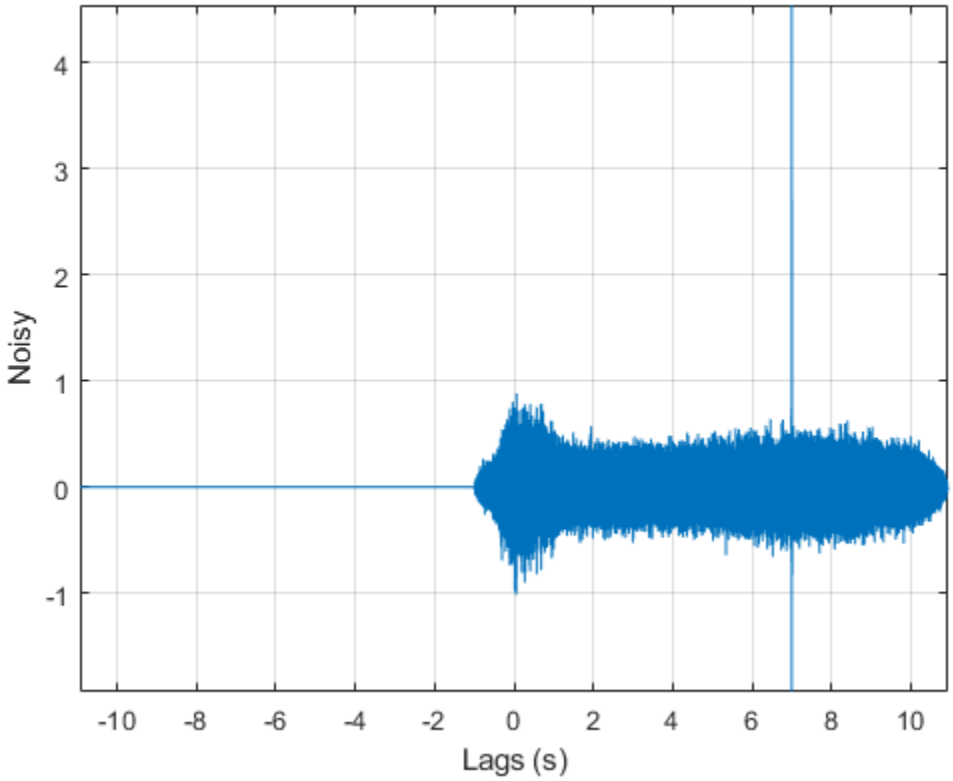
```
NoiseAmp = 0.2*max(abs(Fragment));

Fragment = Fragment+NoiseAmp*randn(size(Fragment));

Full_sig = Full_sig+NoiseAmp*randn(size(Full_sig));

% To hear, type soundsc(Fragment,Fs)

plot(Time,Full_sig,[timeA timeB;timeA timeB],[m m;M M],'r--')
xlabel('Time (s)')
```

```
ylabel('Noisy')
axis tight
```

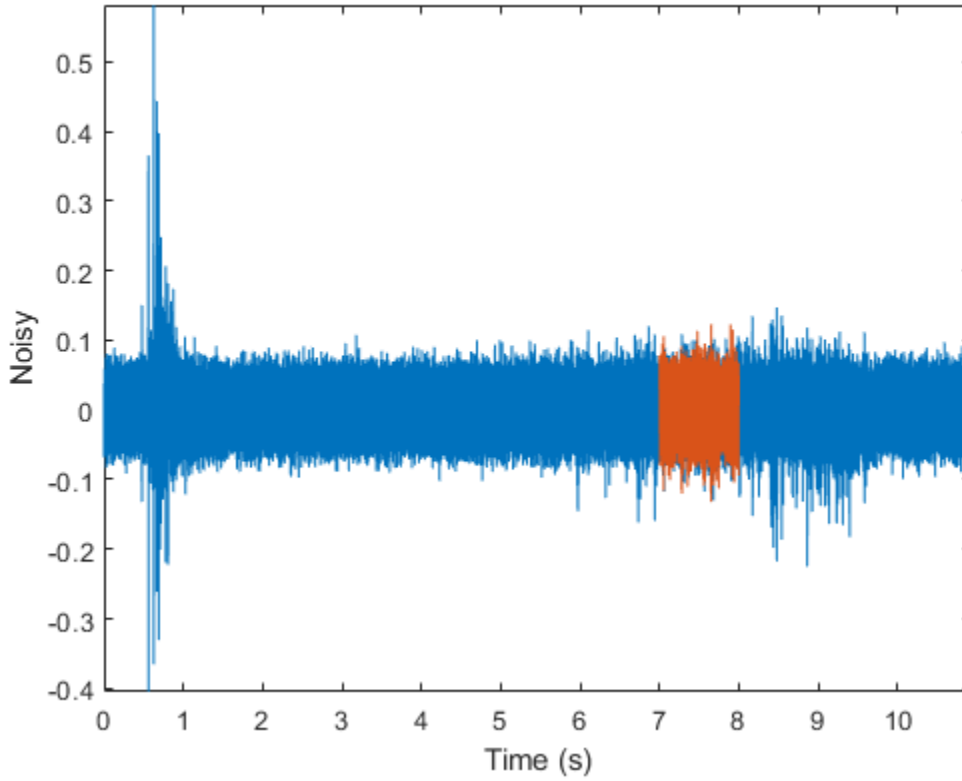


The procedure finds the missing fragment despite the high noise level.

```
[xCorr,lags] = xcorr(Full_sig,Fragment);

plot(lags/Fs,xCorr)
grid
xlabel('Lags (s)')
ylabel('Noisy')
axis tight
```




```
[~,I] = max(abs(xCorr));
maxt = lags(I);

Trial = NaN(size(Full_sig));
Trial(maxt+1:maxt+length(Fragment)) = Fragment;

figure
plot(Time,Full_sig,Time,Trial)
xlabel('Time (s)')
ylabel('Noisy')
axis tight
```



See Also

`xcorr`

Related Examples

- “Measuring Signal Similarities”

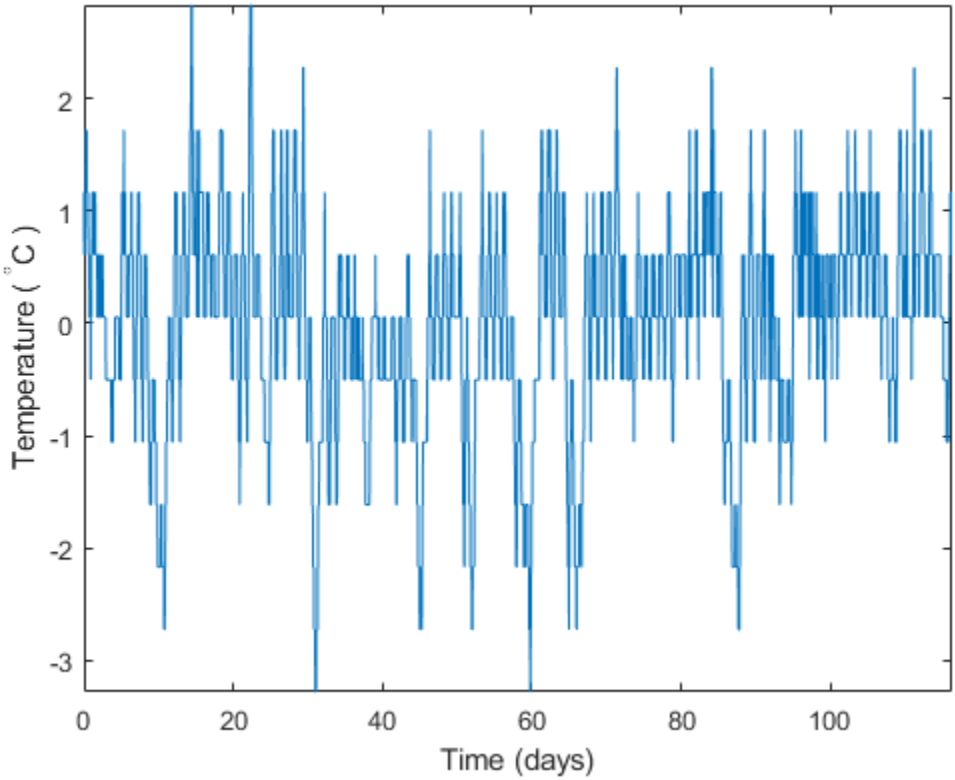
Find Periodicity Using Autocorrelation

Measurement uncertainty and noise sometimes make it difficult to spot oscillatory behavior in a signal, even if such behavior is expected.

The autocorrelation sequence of a periodic signal has the same cyclic characteristics as the signal itself. Thus, autocorrelation can help verify the presence of cycles and determine their durations.

Consider a set of temperature data collected by a thermometer inside an office building. The device takes a reading every half hour for four months. Load the data and plot it. Subtract the mean to concentrate on temperature fluctuations. Convert the temperature to degrees Celsius. Measure time in days. The sample rate is thus 2 measurements/hour × 24 hours/day = 48 measurements/day.

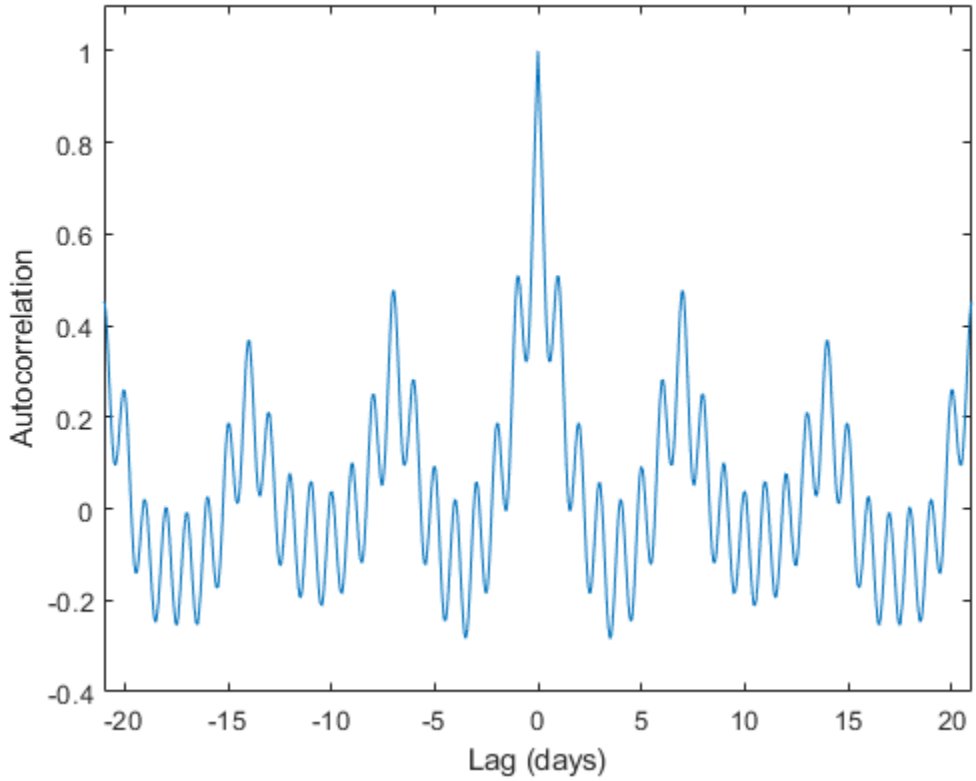
```
load officetemp

tempC = (temp-32)*5/9;

tempnorm = tempC-mean(tempC);

fs = 2*24;
t = (0:length(tempnorm) - 1)/fs;

plot(t,tempnorm)
xlabel('Time (days)')
ylabel('Temperature ( {}^\circC )')
axis tight
```



The temperature does seem to oscillate, but the lengths of the cycles cannot be read out easily.

Compute the autocorrelation of the temperature such that it is unity at zero lag. Restrict the positive and negative lags to three weeks. Note the double periodicity of the signal.

```
[autocor,lags] = xcorr(tempnorm,3*7*fs,'coeff');

plot(lags/fs,autocor)
xlabel('Lag (days)')
ylabel('Autocorrelation')
axis([-21 21 -0.4 1.1])
```



Determine the short and long periods by finding the peak locations and determining the average time differences between them.

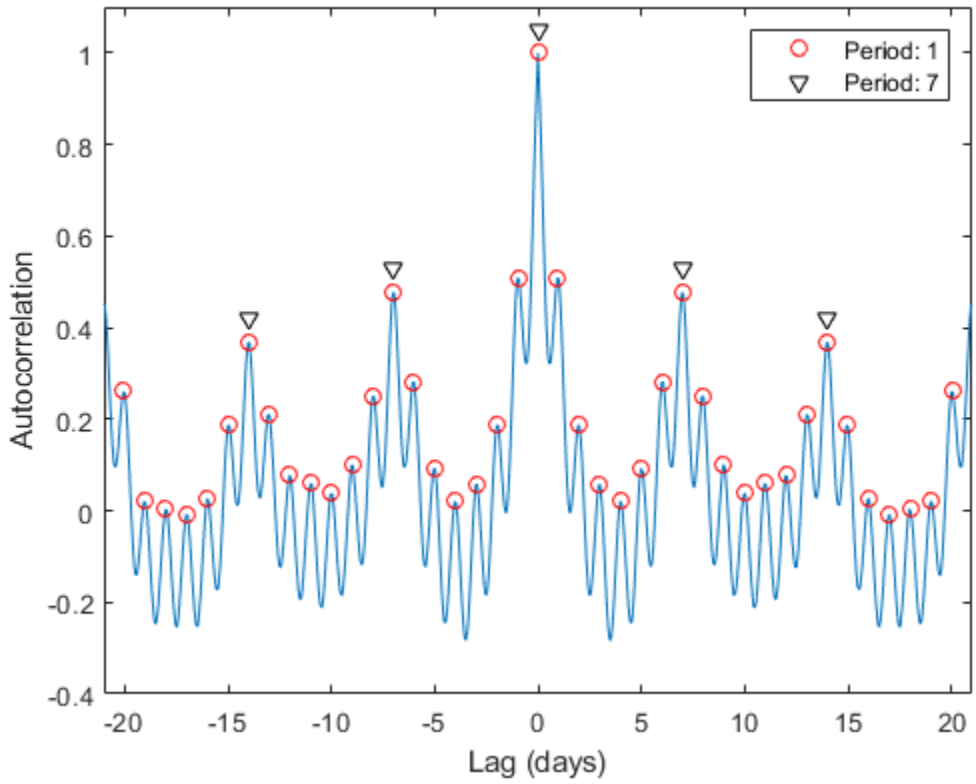
To find the long period, restrict `findpeaks` to look for peaks separated by more than the short period and with a minimum height of 0.3.

```
[pksh,lcsh] = findpeaks(autocor);
short = mean(diff(lcsh))/fs
```

```
short = 1.0021
```

```
[pklg,lclg] = findpeaks(autocor, ...
    'MinPeakDistance',ceil(short)*fs,'MinPeakheight',0.3);
long = mean(diff(lclg))/fs
```

```
long = 6.9896


hold on
pks = plot(lags(lcsh)/fs,pksh,'or', ...
    lags(lclg)/fs,pklg+0.05,'vk');
hold off
legend(pks,[repmat('Period: ',[2 1]) num2str([short;long],0)])
axis([-21 21 -0.4 1.1])
```



To a very good approximation, the autocorrelation oscillates both daily and weekly. This is to be expected, since the temperature in the building is higher when people are at work and lower at nights and on weekends.

See Also

`findpeaks` | `xcorr`

Related Examples

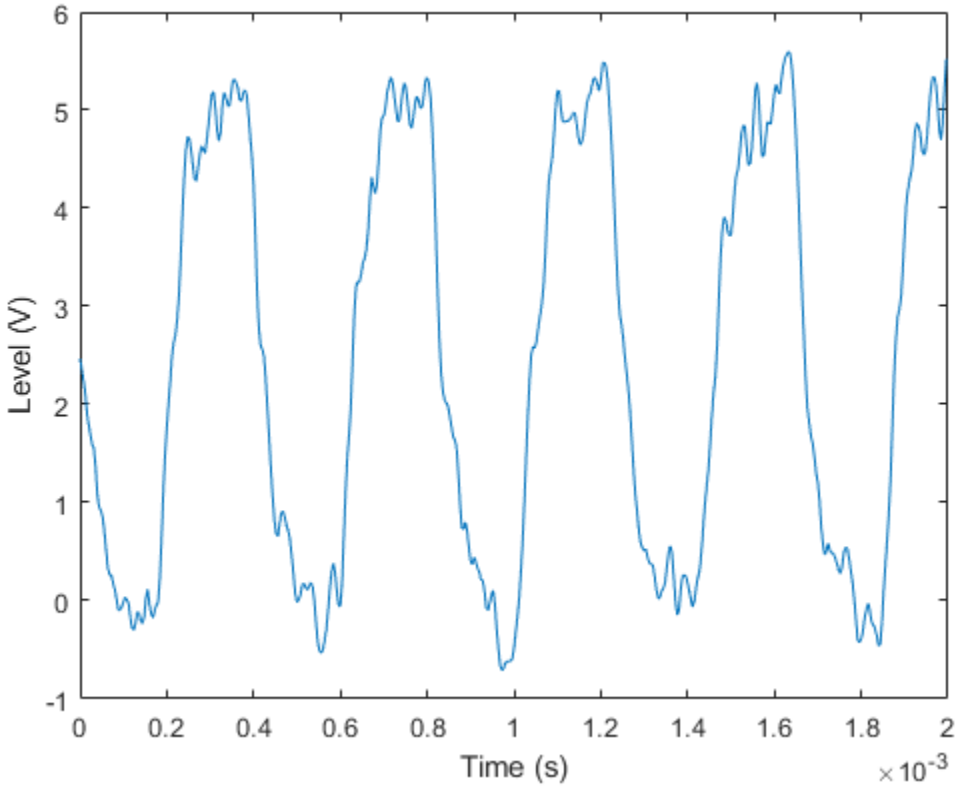
- “Find Periodicity Using Frequency Analysis” on page 20-96

Extract Features of a Clock Signal

How sharply does an on/off signal turn on and off? How often and for how long is it activated? Determine all those characteristics for the output of a clock.

Load the signal and plot it. The time is measured in seconds and the level in volts.

```
load(fullfile(matlabroot,'examples','signal','clock.mat'))

plot(tclock,clocksig)
xlabel('Time (s)')
ylabel('Level (V)')
```

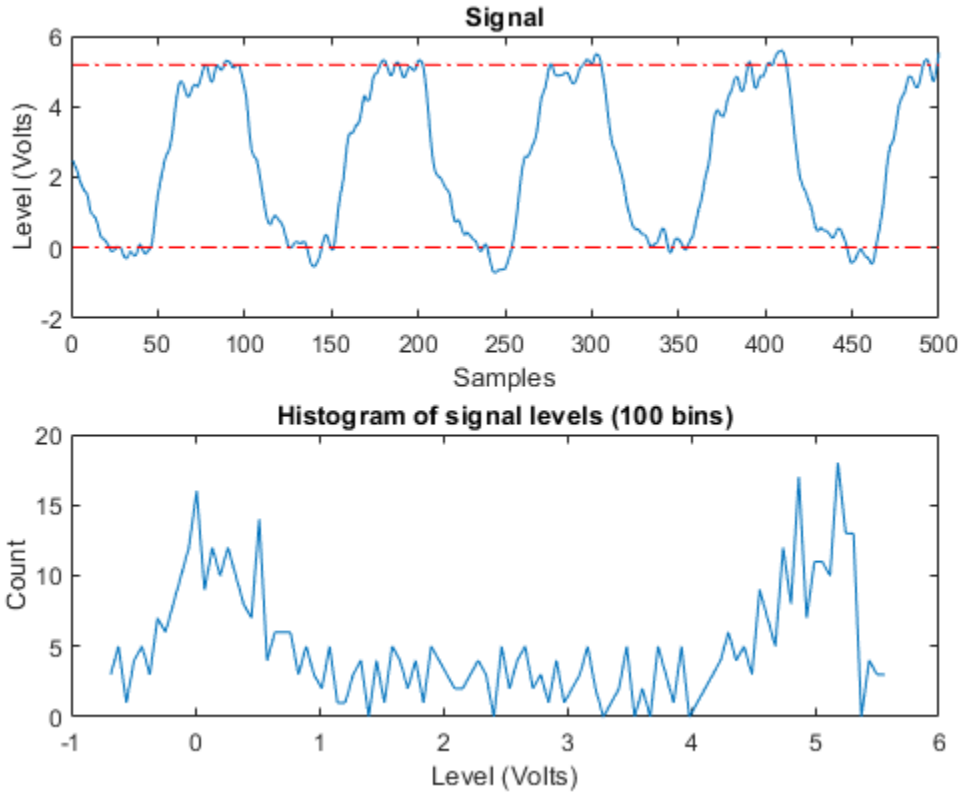


Use `statelevels` to find the lower and upper levels of the signal by means of a histogram. If you do not specify an output, the function plots the signal, marks the levels, and displays the histogram.

```
levels = statelevels(clocksig)
```

```
levels = 1×2

    0.0138    5.1848
```

```
statelevels(clocksig);
```



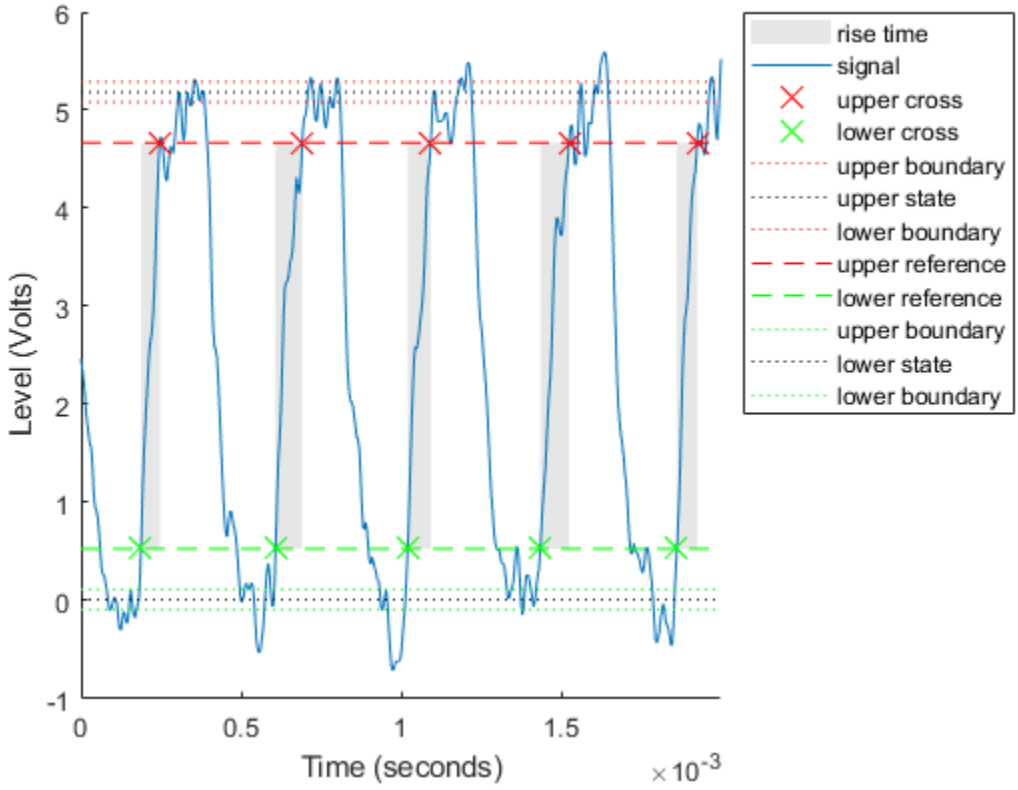
Determine how fast the signal rises at each transition. `risetime` uses the lower and upper levels found by `statelevels`. It defines the rise time as the time it takes the signal to rise from 10% to 90% of the difference between the levels.

```
[Rise,LoTime,HiTime,LoLev,HiLev] = risetime(clocksig,tclock);

Levels = [LoLev HiLev; (levels(2)-levels(1))*[0.1 0.9]+levels(1)]
```

```
Levels = 2×2

    0.5309    4.6677
    0.5309    4.6677
```

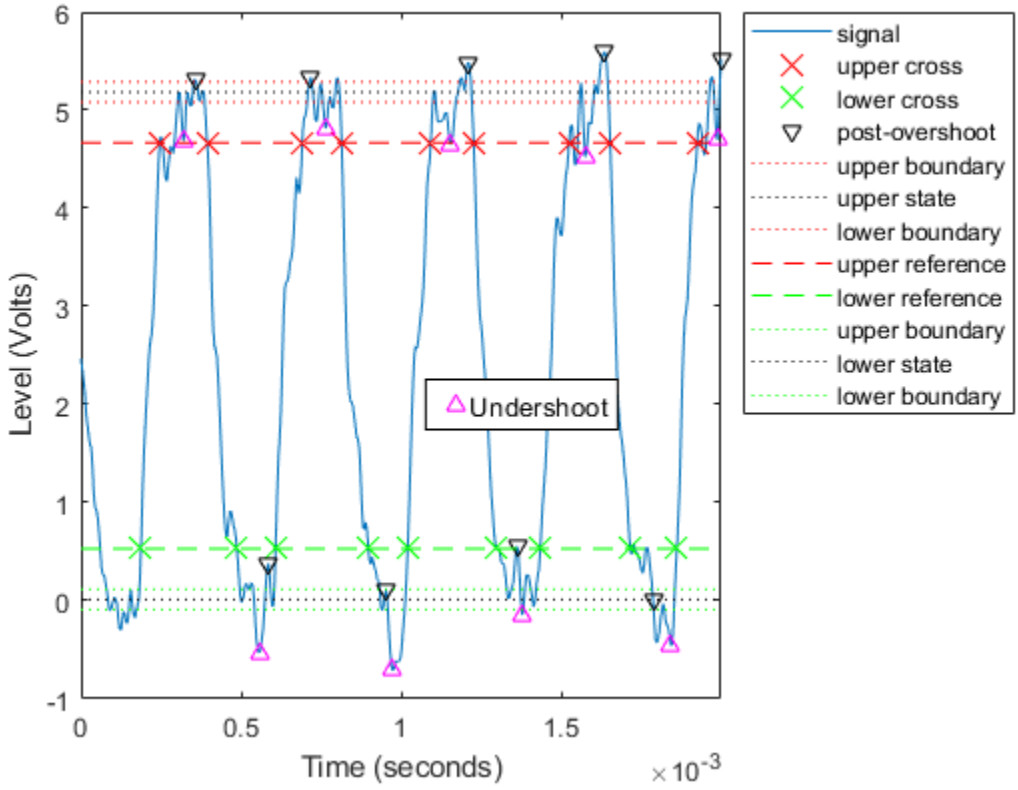
If you call `risetime` without outputs, the function draws an annotated plot of the signal. The rise times are shaded, the crossing points are marked, and the levels are displayed. You can use the time vector or the sample rate as input.

```
risetime(clocksig,Fs);
```



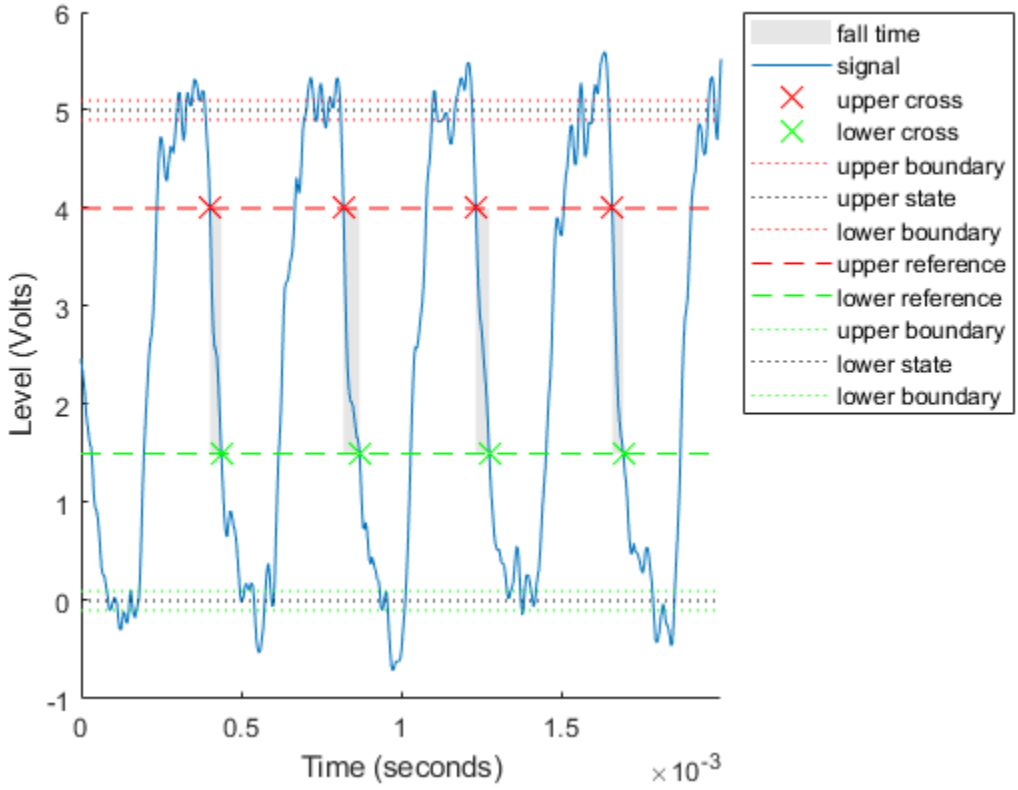
`overshoot` and `undershoot` show how far the signal deviates from the state levels at each transition. The results are expressed as percentages of the difference between the levels. Further outputs give the actual times and signal values.

```
overshoot(clocksig,Fs);

[pctgs,values,times] = undershoot(clocksig,Fs);

hold on
text(1.1e-3,2,'     Undershoot','Background','w','Edge','k')
plot([times;1.17e-3],[values;2],'^m','HandleVisibility','off')
hold off
```



Determine how fast the signal falls using `falltime`. You can set the state levels and the percentage reference levels manually. You can do the same with `risetime`.

```
falltime(clocksig,tclock, ...
    'PercentReferenceLevels',[30 80],'StateLevels',[0 5]);
```



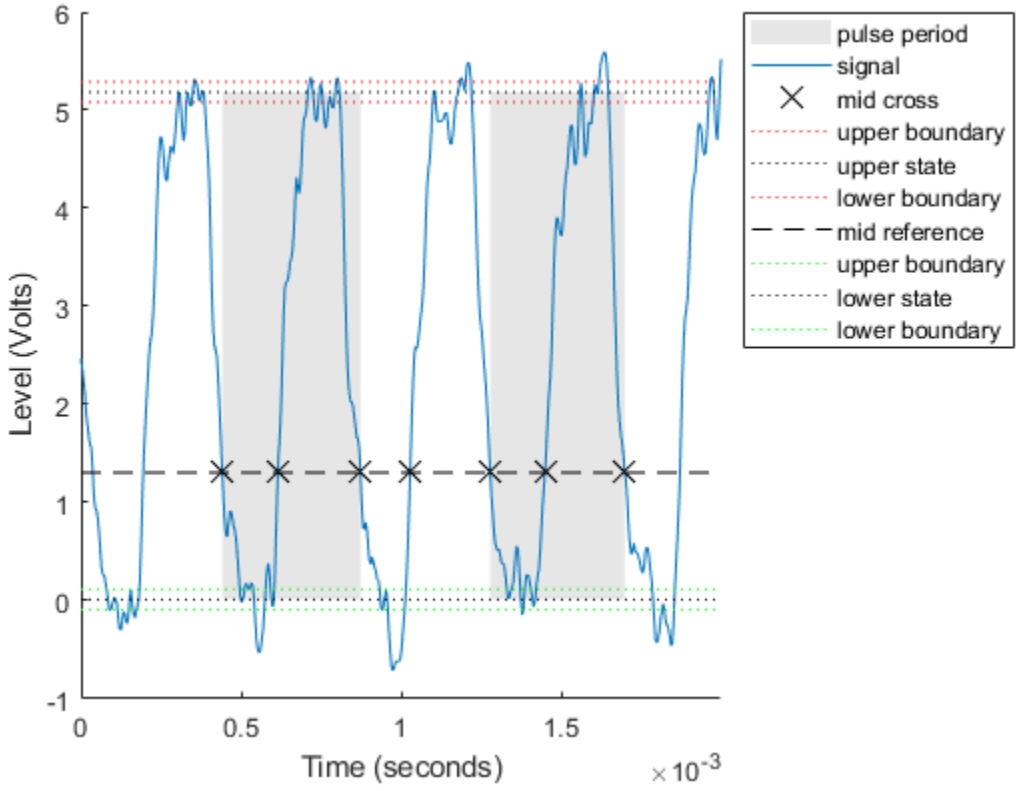
Find the period of the signal. By default, the period is defined as the time elapsed between consecutive rising crossings of the reference level halfway between the state levels. You can change the polarity of the crossings, specify the state levels, or adjust the reference level.

```
per = pulseperiod(clocksig,tclock)
```

```
per = 4×1
10^-3 ×

    0.4143
    0.4200
    0.4188
```

```
    0.4111
```

```
pulseperiod(clocksig,Fs,'Polarity','negative','MidPct',25);
```



The duty cycle is the ratio of pulse width to pulse period. Determine it directly or using a dedicated function.

```
dut = dutycycle(clocksig,Fs);

wdt = pulsewidth(clocksig,Fs);

compare = [wdt./per dut]
```

```
compare = 4×2

    0.4862    0.4862
    0.4756    0.4756
    0.4871    0.4871
    0.4886    0.4886
```

See Also

`dutycycle` | `falltime` | `overshoot` | `pulseperiod` | `pulsewidth` | `risetime` | `slewrate` | `statelevels` | `undershoot`

Related Examples

- “Measurement of Pulse and Transition Characteristics”

Find Periodicity in a Categorical Time Series

This example shows how to perform spectral analysis of categorical-valued time-series data. The spectral analysis of categorical-valued time series is useful when you are interested in cyclic behavior of data whose values are not inherently numerical. This example reproduces in part the analysis reported in Stoffer et al. (1988). The data are taken from Stoffer, Tyler, and Wendt (2000).

The data are from a study of sleep states in newborn children. A pediatric neurologist scored an infant's electroencephalographic (EEG) recording every minute for approximately two hours. The neurologist categorized the infant's sleep state into one of the following:

- `qt` - Quiet sleep, trace alternant
- `qh` - Quiet sleep, high voltage
- `tr` - Transitional sleep
- `al` - Active sleep, low voltage
- `ah` - Active sleep, high voltage
- `aw` - Awake

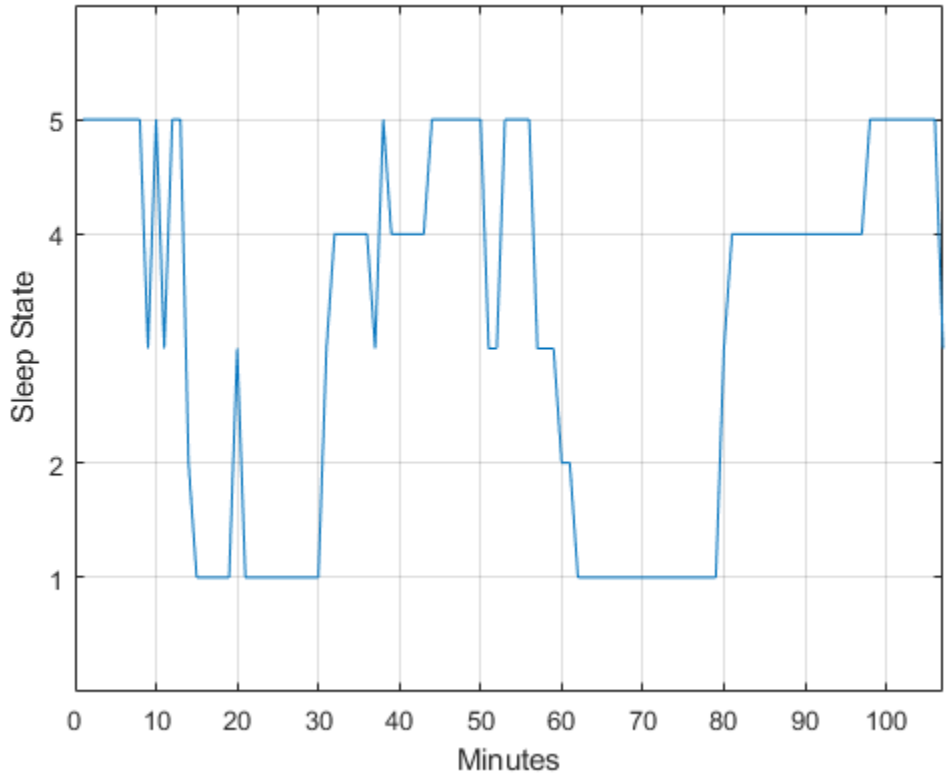
Enter the data. The infant was never awake during the EEG recording.

```
data = {'ah','ah','ah','ah','ah','ah','ah','ah','tr','ah','tr','ah', ...
   'ah','qh','qt','qt','qt','qt','qt','tr','qt','qt','qt','qt','qt', ...
   'qt','qt','qt','qt','qt','tr','al','al','al','al','al','tr','ah', ...
   'al','al','al','al','al','ah','ah','ah','ah','ah','ah','ah','tr', ...
   'tr','ah','ah','ah','ah','tr','tr','tr','qh','qh','qt','qt','qt', ...
   'qt','qt','qt','qt','qt','qt','qt','qt','qt','qt','qt','qt','qt', ...
   'qt','qt','tr','al','al','al','al','al','al','al','al','al','al', ...
   'al','al','al','al','al','al','al','ah','ah','ah','ah','ah','ah', ...
   'ah','ah','ah','tr'};

lend = length(data);
t = 1:lend;
```

The easiest way to analyze categorical-valued time series data for cyclic patterns involves assigning numerical values to the categories. There are at least two meaningful ways of assigning values to the infant's sleep states. First, note that you can order the six states from 1 to 6. This assignment makes sense along the scale of least active to most active.

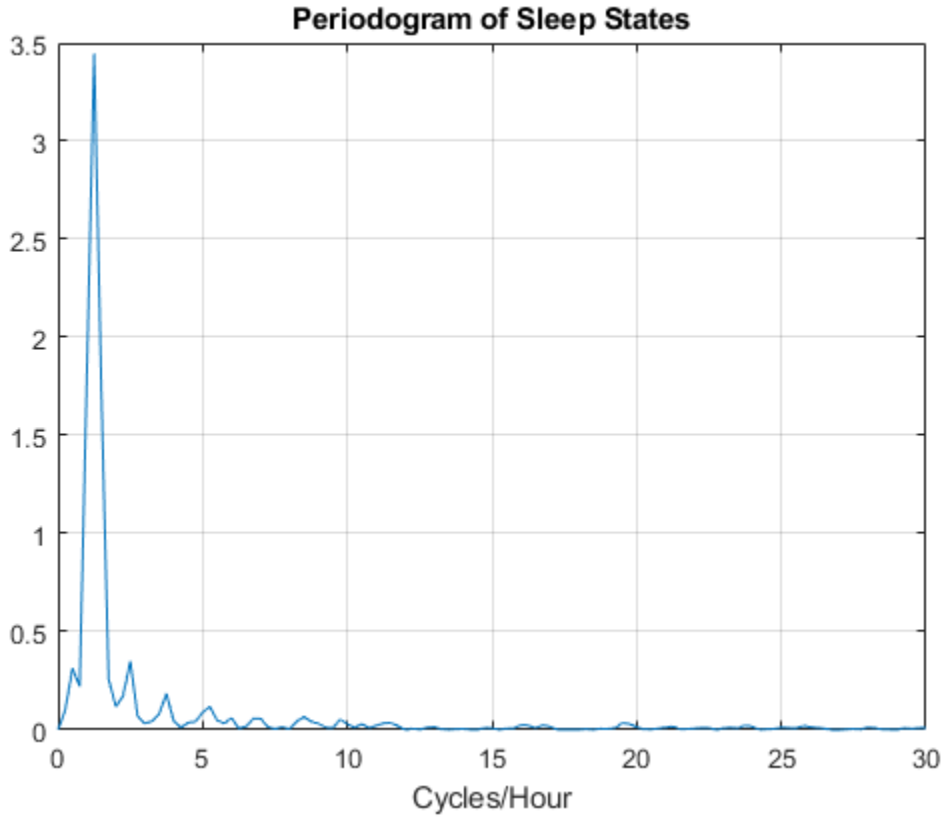
Replace the six sleep states with their numerical equivalents and plot the data.

```
states = ['qt';'qh';'tr';'al';'ah';'aw'];
levelssix = [1 2 3 4 5 6];

for nn = 1:6
    datasix(strcmp(data,states(nn,:))) = levelssix(nn);
end

plot(t,datasix)
axis([0 lend 0 6])
ax = gca;
ax.YTick = [1 2 4 5];
grid
xlabel('Minutes')
ylabel('Sleep State')
```



The data exhibit cyclic behavior when you focus on the transitions between the quietest states (1 and 2) and the most active ones (4 and 5). To determine the cycle of that behavior, use spectral analysis. Recall that the sleep states are assigned in one-minute intervals. Sampling the data in one-minute intervals is equivalent to sampling the data 60 times per hour.

```
Fs = 60;
[Pxx,F] = periodogram(detrend(datasix,0),[],240,Fs);

plot(F,Pxx)
grid
xlabel('Cycles/Hour')
title('Periodogram of Sleep States')
```



The spectral analysis shows a clear peak indicating a dominant oscillation, or cycle in the data. Determine the frequency of the peak.

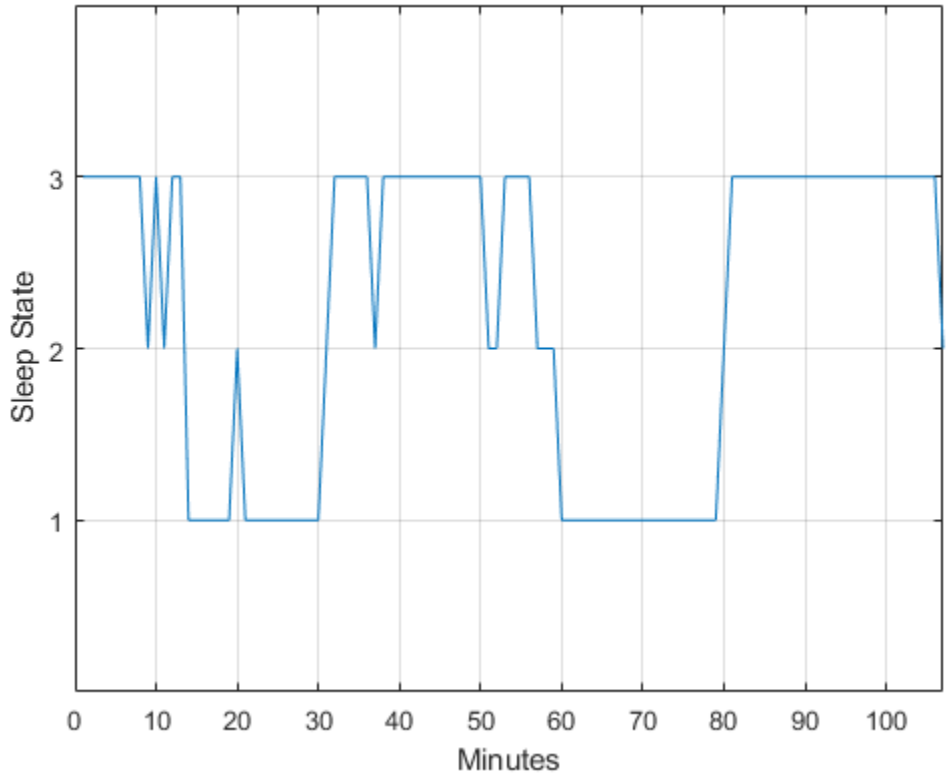
```
[~,maxidx] = max(Pxx);
Fsix = F(maxidx)
```

```
Fsix = 1.2500
```

The infant's sleep states exhibit cyclic behavior with a frequency of approximately 1.25 cycles/hour.

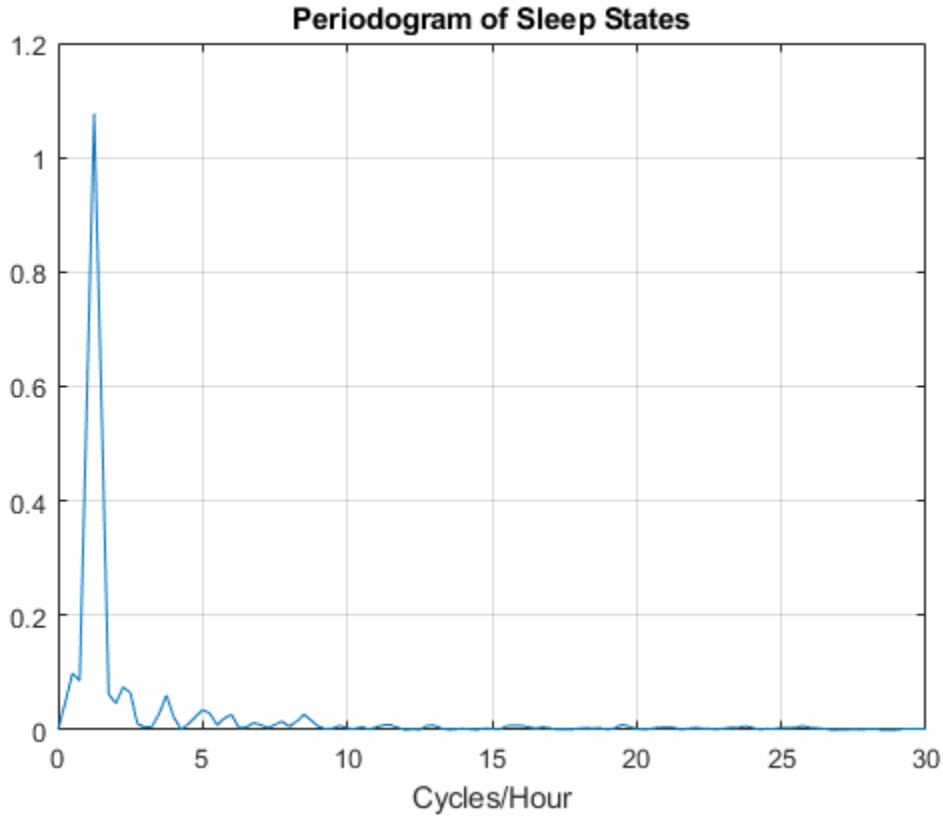
Instead of assigning the sleep states the values 1 to 6, repeat the analysis focusing only on the distinction between quiet and active sleep. Assign the quiet states, `qt` and `qh`, the value 1. Assign the transitional state, `tr`, the value 2. Finally, assign the two active sleep states, `al` and `ah`, the value 3. For completeness, assign the awake state, `aw`, the value 4, even though the state does not occur in the data.

```
states = ['qt';'qh';'tr';'al';'ah';'aw'];
levelsfou = [1 1 2 3 3 4];

for nn = 1:6
    datafou(strcmp(data,states(nn,:))) = levelsfou(nn);
end

plot(t,datafou)
axis([0 lend 0 4])
ax = gca;
ax.YTick = [1 2 3];
grid
xlabel('Minutes')
ylabel('Sleep State')
```



With this rule of assignment between the sleep states and the values 1 to 3, the cyclic behavior of the data is clearer. Repeat the spectral analysis with the new assignment.

```
[Pxx,F] = periodogram(detrend(datafou,0),[],240,Fs);

plot(F,Pxx)
grid
xlabel('Cycles/Hour')
title('Periodogram of Sleep States')
```




```
[maxval,maxidx] = max(Pxx);
F(maxidx)
```

```
ans = 1.2500
```

The new assignment has not changed the conclusion. The data show a dominant oscillation at 1.25 cycles/hour. Because the mapping between the sleep states and the integers representing those states was consistent, the analysis and conclusions were not affected. Based on a spectral analysis of this categorical data, you conclude that the infant's sleep state cycles between quiet and active sleep approximately once every hour.

References

Stoffer, David S., Mark S. Scher, Gale A. Richardson, Nancy L. Day, and Patricia A. Coble. "A Walsh-Fourier Analysis of the Effects of Moderate Maternal Alcohol Consumption on Neonatal Sleep-State Cycling." *Journal of the American Statistical Association.* Vol. 83, 1988, pp. 954-963.

Stoffer, David S., D. E. Tyler, and D. A. Wendt. "The Spectral Envelope and Its Applications." *Statistical Science.* Vol. 15, 2000, pp. 224-253.

See Also

`detrend` | `periodogram`

Compensate for the Delay Introduced by an FIR Filter

Filtering a signal introduces a delay. This means that the output signal is shifted in time with respect to the input. This example shows you how to counteract this effect.

Finite impulse response filters often delay all frequency components by the same amount. This makes it easy to correct for the delay by shifting the signal in time.

Take an electrocardiogram reading sampled at 500 Hz for 1 s. Add random noise. Reset the random number generator for reproducibility.

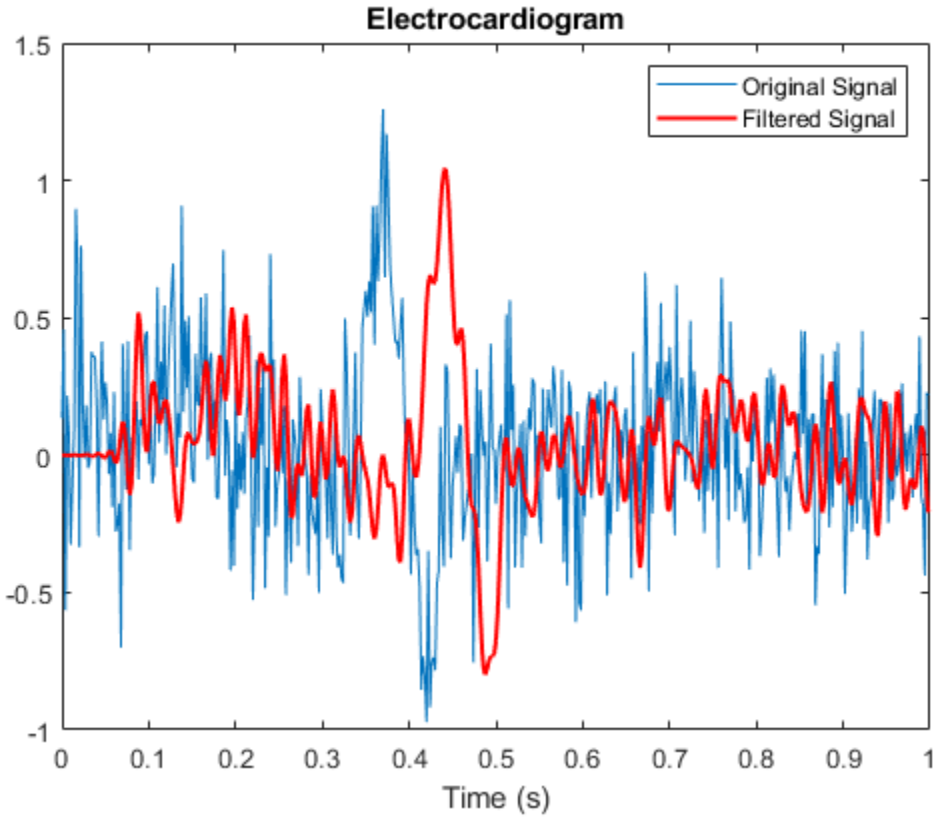
```
Fs = 500;
N = 500;
rng default

xn = ecg(N)+0.25*randn([1 N]);
tn = (0:N-1)/Fs;
```

Remove some of the noise with a filter that stops frequencies above 75 Hz. Use `designfilt` to design a filter of order 70.

```
nfilt = 70;
Fst = 75;

d = designfilt('lowpassfir','FilterOrder',nfilt, ...
               'CutoffFrequency',Fst,'SampleRate',Fs);
```

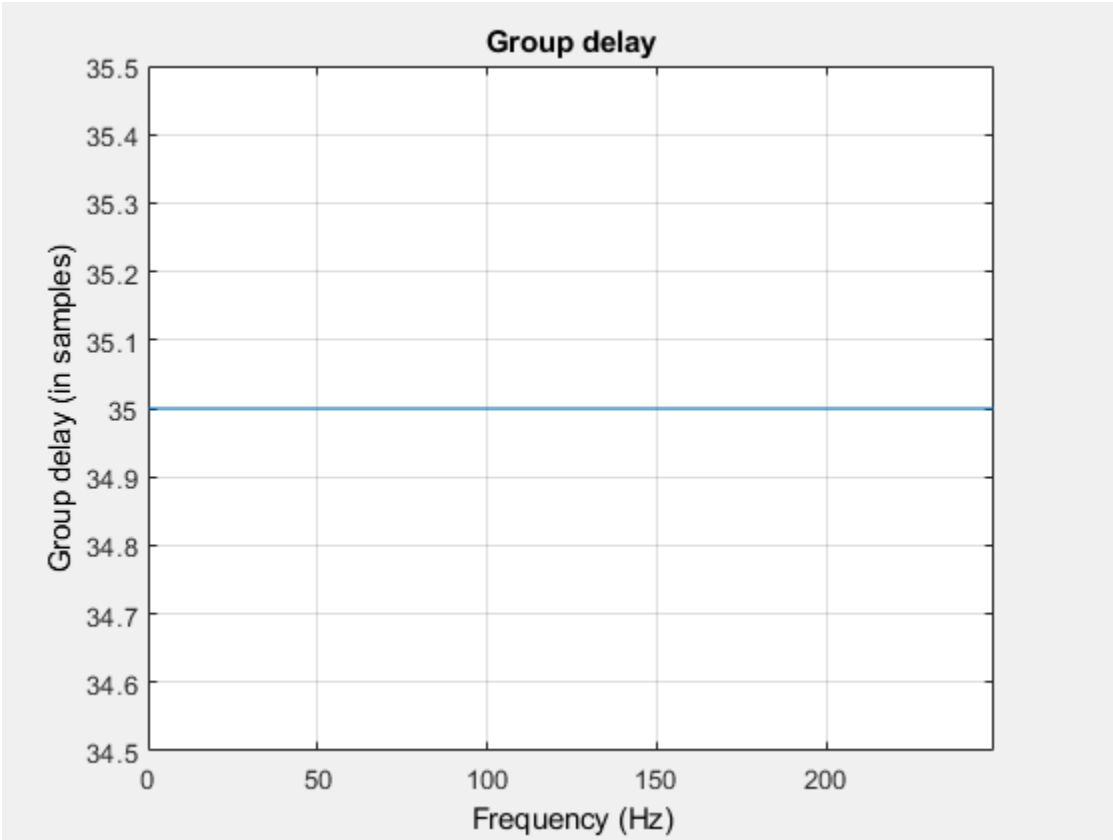
Filter the signal and plot it. The result is smoother than the original, but lags behind it.

```
xf = filter(d,xn);

plot(tn,xn)
hold on, plot(tn,xf,'-r','linewidth',1.5), hold off
title 'Electrocardiogram'
xlabel 'Time (s)', legend('Original Signal','Filtered Signal')
```



Use `grpdelay` to check that the delay caused by the filter equals half the filter order.

```
grpdelay(d,N,Fs)
```




```
delay = mean(grpdelay(d))
```

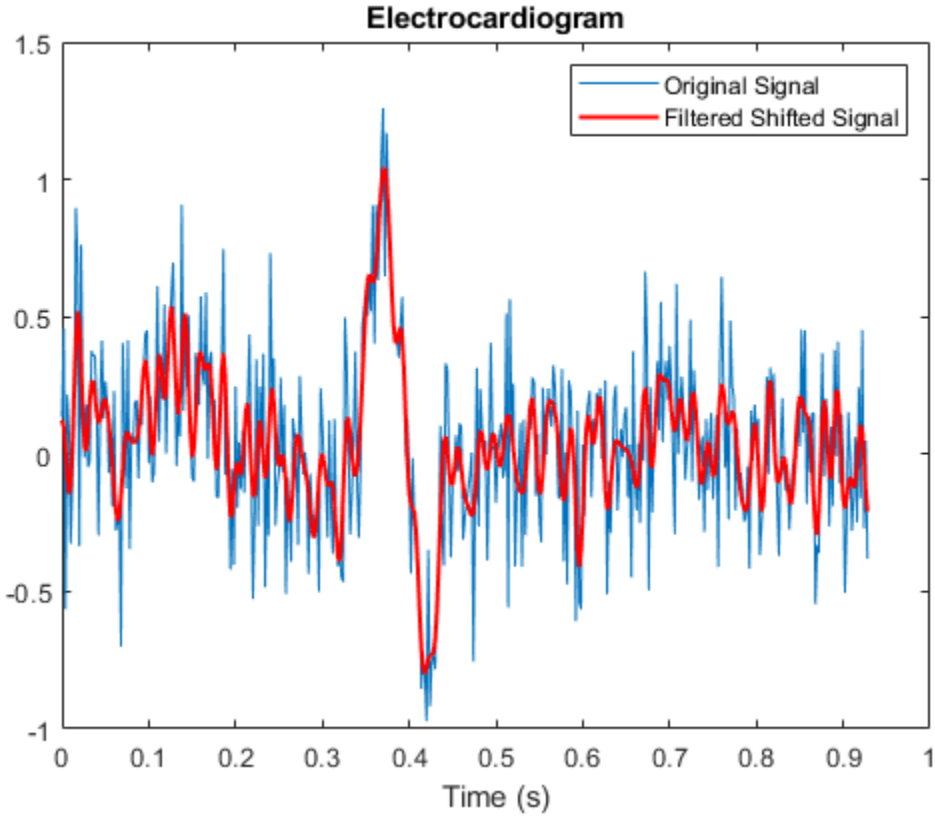
```
delay = 35
```

Shift the filtered signal to line up the data. Remove its first `delay` samples. Remove the last `delay` samples of the original and of the time vector.

```
tt = tn(1:end-delay);
sn = xn(1:end-delay);

sf = xf;
sf(1:delay) = [];
```

Plot the signals and verify that they are aligned.

```
plot(tt,sn)
hold on, plot(tt,sf,'-r','linewidth',1.5), hold off
title 'Electrocardiogram'
xlabel('Time (s)'), legend('Original Signal','Filtered Shifted Signal')
```



See Also

`designfilt` | `filter` | `filtfilt` | `grpdelay`

Related Examples

- “Compensate for the Delay Introduced by an IIR Filter” on page 20-84

- “Practical Introduction to Digital Filtering”

Compensate for the Delay Introduced by an IIR Filter

Filtering a signal introduces a delay. This means that the output signal is shifted in time with respect to the input.

Infinite impulse response filters delay some frequency components more than others. They effectively distort the input signal. The function `filtfilt` compensates for the delays introduced by such filters, and thus corrects for filter distortion. This "zero-phase filtering" results from filtering the signal in the forward and backward directions.

Take an electrocardiogram reading sampled at 500 Hz for 1 s. Add random noise.

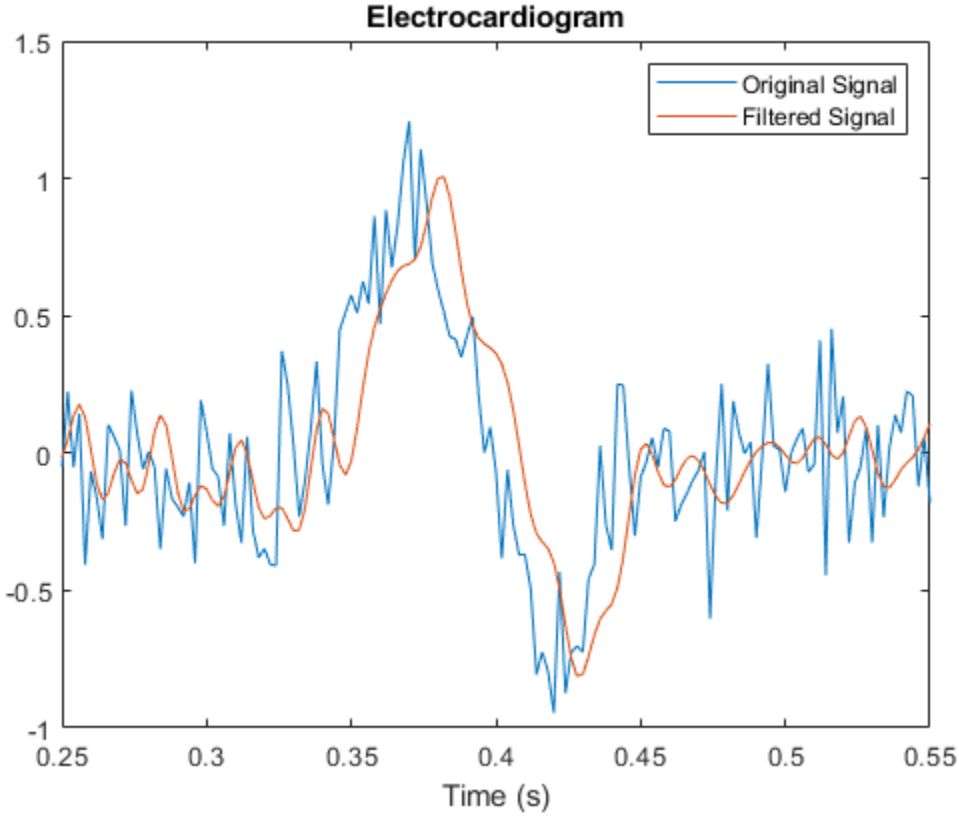
```
Fs = 500;
N = 500;

rng default
xn = ecg(N) + 0.2*randn([1 N]);
tn = (0:N-1)/Fs;
```

Remove some of the noise with a filter that stops frequencies above 75 Hz. Specify a 7th-order IIR filter with 1 dB of passband ripple and 60 dB of stopband attenuation.

```
Nf = 7;
Fp = 75;
Ap = 1;
As = 60;

d = designfilt('lowpassiir','FilterOrder',Nf,'PassbandFrequency',Fp, ...
    'PassbandRipple',Ap,'StopbandAttenuation',As,'SampleRate',Fs);
```

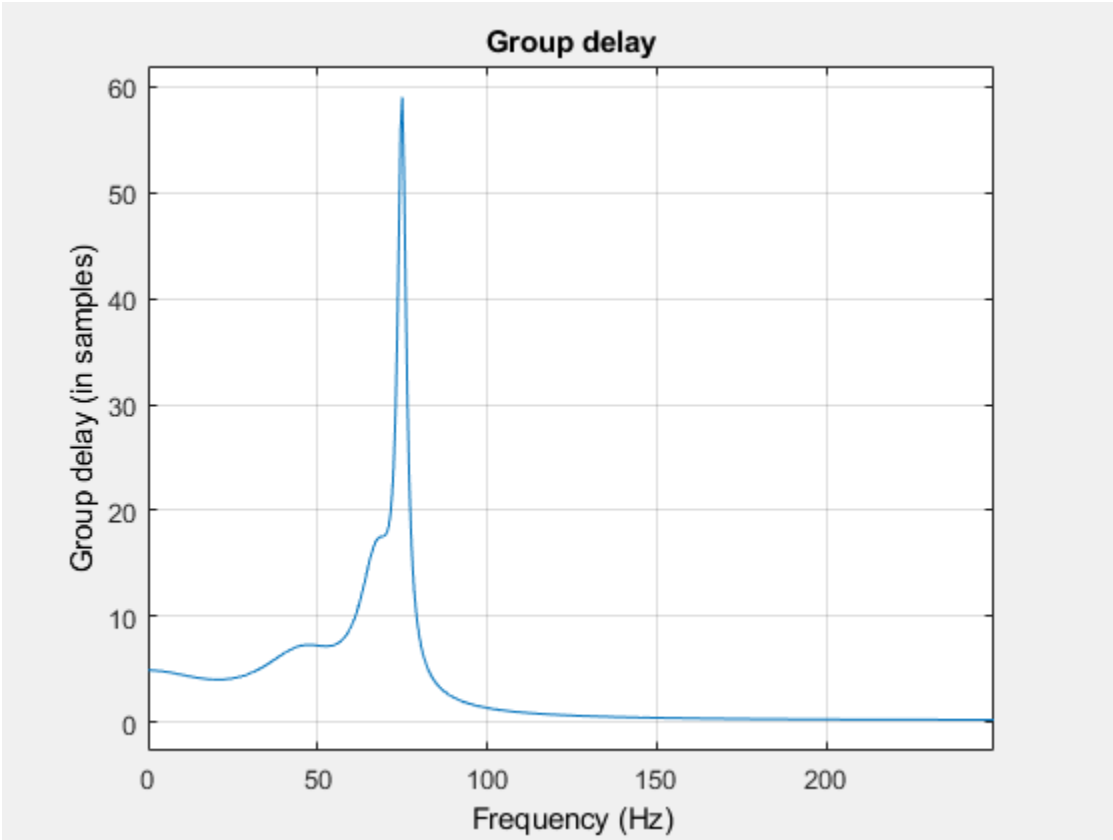
Filter the signal. The filtered signal is cleaner than the original, but lags in time with respect to it. It is also distorted due to the nonlinear phase of the filter. Zoom in close to the peak.

```
xfilter = filter(d,xn);

plot(tn,xn,tn,xfilter)

title 'Electrocardiogram'
xlabel 'Time (s)', legend('Original Signal','Filtered Signal')
axis([0.25 0.55 -1 1.5])
```



A look at the *group delay* introduced by the filter shows that the delay is frequency-dependent.

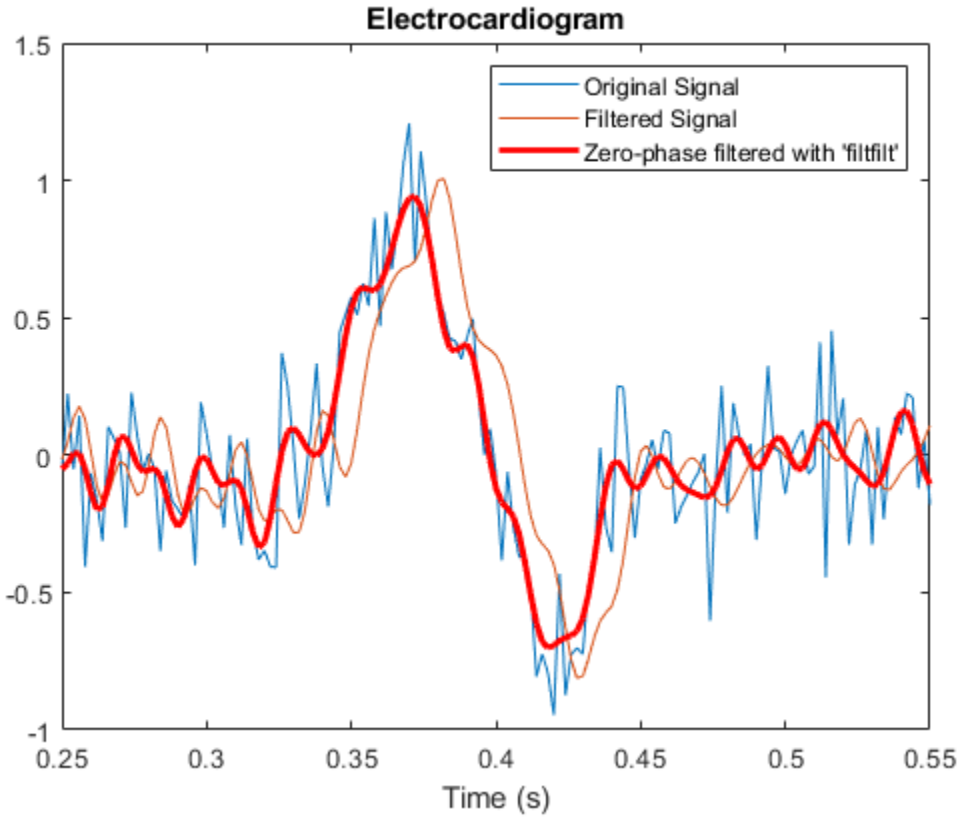
```
grpdelay(d,N,Fs)
```



Filter the signal using `filtfilt`. The delay and distortion have been effectively removed. Use `filtfilt` when it is critical to keep the phase information of a signal intact.

```
xfiltfilt = filtfilt(d,xn);

plot(tn,xn,tn,xfilter)
hold on
plot(tn,xfiltfilt,'r','linewidth',2)
hold off

title 'Electrocardiogram'
xlabel 'Time (s)'
legend('Original Signal','Filtered Signal', ...
```

```
        'Zero-phase filtered with ''filtfilt''')
axis([0.25 0.55 -1 1.5])
```



See Also

`designfilt` | `filter` | `filtfilt` | `grpdelay`

Related Examples

- “Compensate for the Delay Introduced by an FIR Filter” on page 20-79
- “Practical Introduction to Digital Filtering”

Take Derivatives of a Signal

You want to differentiate a signal without increasing the noise power. MATLAB®'s function `diff` amplifies the noise, and the resulting inaccuracy worsens for higher derivatives. To fix this problem, use a differentiator filter instead.

Analyze the displacement of a building floor during an earthquake. Find the speed and acceleration as functions of time.

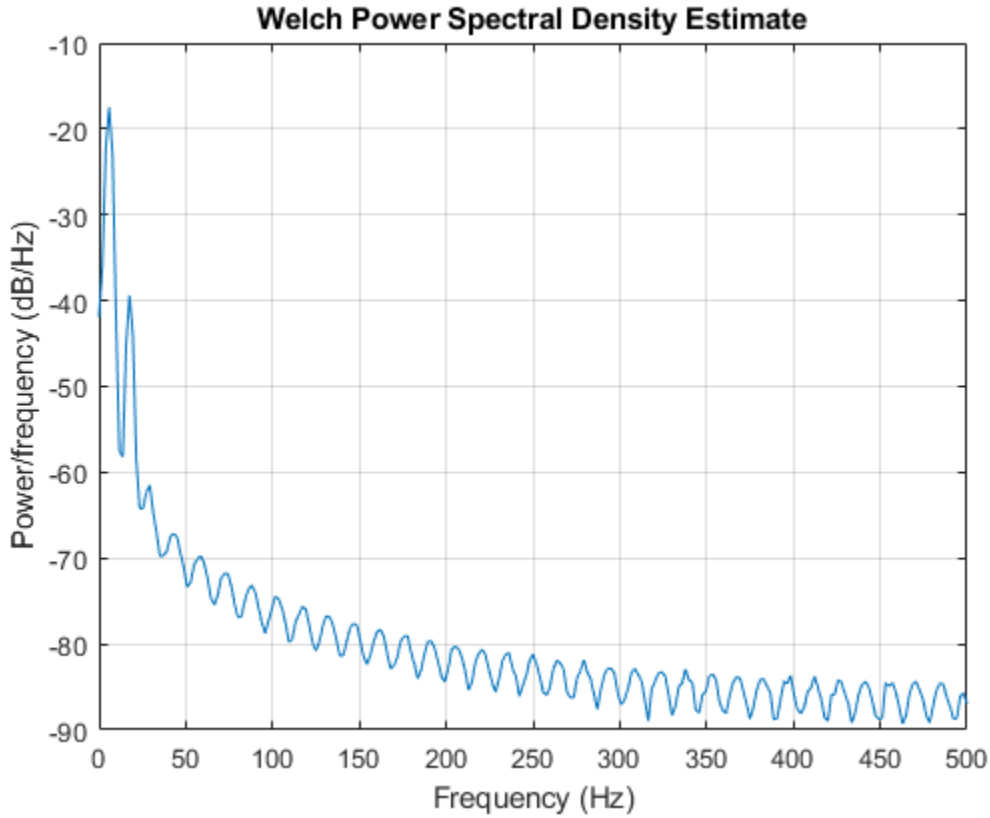
Load the file `earthquake`. The file contains the following variables:

- `drift`: Floor displacement, measured in centimeters
- `t`: Time, measured in seconds
- `Fs`: Sample rate, equal to 1 kHz

```
load(fullfile(matlabroot,'examples','signal','earthquake.mat'))
```

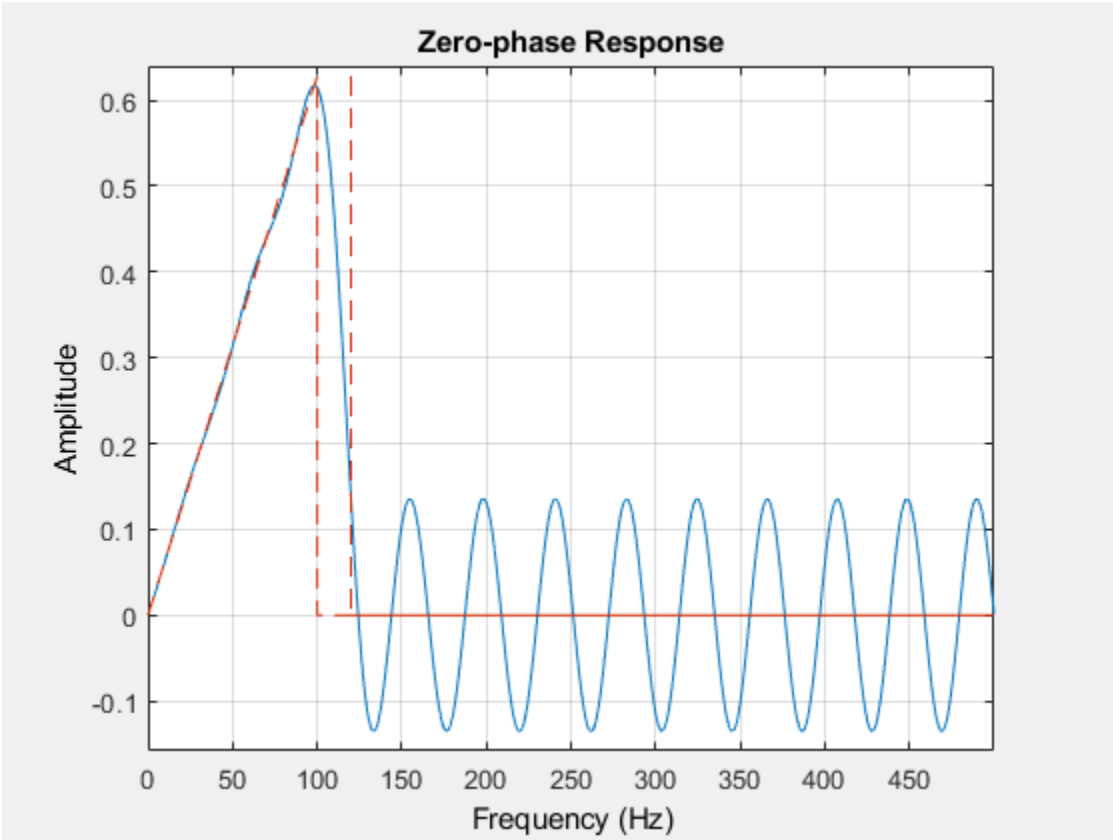
Use `pwelch` to display an estimate of the power spectrum of the signal. Note how most of the signal energy is contained in frequencies below 100 Hz.

```
pwelch(drift,[],[],[],Fs)
```



Use `designfilt` to design an FIR differentiator of order 50. To include most of the signal energy, specify a passband frequency of 100 Hz and a stopband frequency of 120 Hz. Inspect the filter with `fvtool`.

```
Nf = 50;
Fpass = 100;
Fstop = 120;

d = designfilt('differentiatorfir','FilterOrder',Nf, ...
    'PassbandFrequency',Fpass,'StopbandFrequency',Fstop, ...
    'SampleRate',Fs);

fvtool(d,'MagnitudeDisplay','zero-phase','Fs',Fs)
```



Differentiate the drift to find the speed. Divide the derivative by `dt`, the time interval between consecutive samples, to set the correct units.

```
dt = t(2)-t(1);

vdrift = filter(d,drift)/dt;
```

The filtered signal is delayed. Use `grpdelay` to determine that the delay is half the filter order. Compensate for it by discarding samples.

```
delay = mean(grpdelay(d))

delay = 25
```

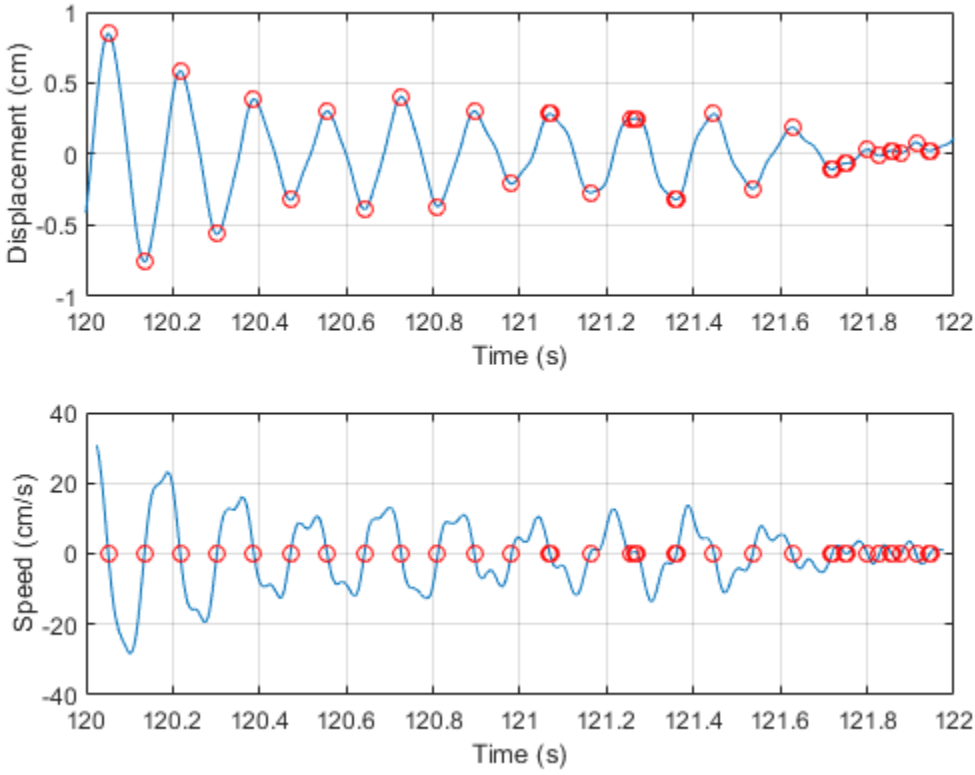
```
tt = t(1:end-delay);
vd = vdrift;
vd(1:delay) = [];
```

The output also includes a transient whose length equals the filter order, or twice the group delay. `delay` samples were discarded above. Discard `delay` more to eliminate the transient.

```
tt(1:delay) = [];
vd(1:delay) = [];
```

Plot the drift and the drift speed. Use `findpeaks` to verify that the maxima and minima of the drift correspond to the zero crossings of its derivative.

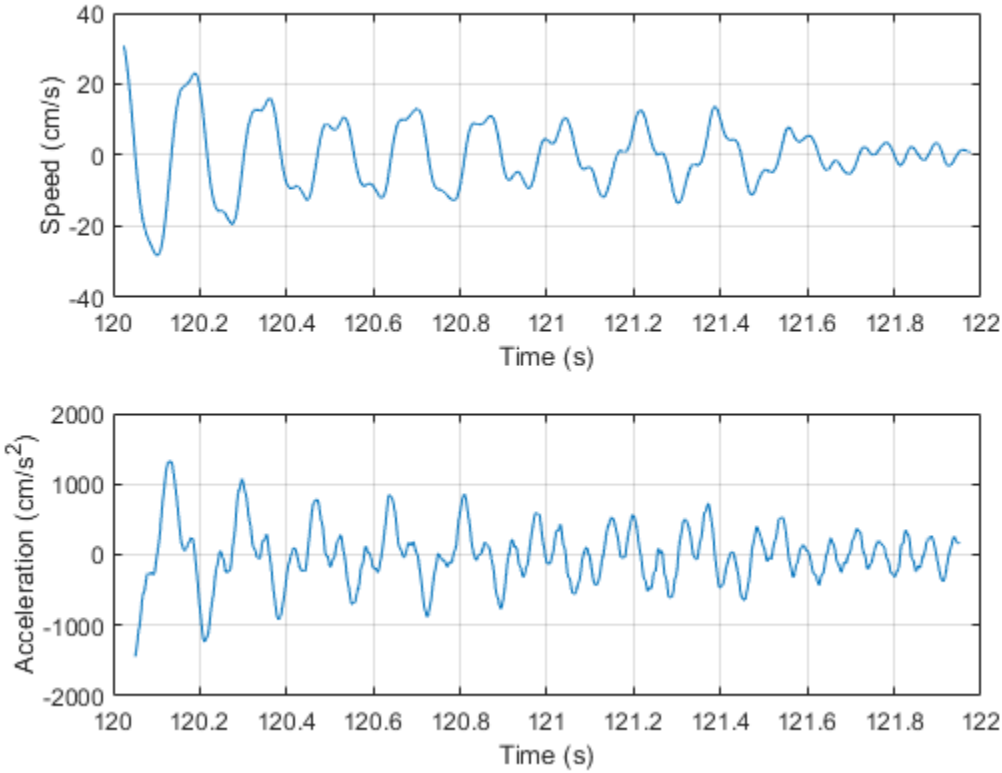
```
[pkp,lcp] = findpeaks(drift);
zcp = zeros(size(lcp));

[pkm,lcm] = findpeaks(-drift);
zcm = zeros(size(lcm));

subplot(2,1,1)
plot(t,drift,t([lcp lcm]),[pkp -pkm],'or')
xlabel('Time (s)')
ylabel('Displacement (cm)')
grid

subplot(2,1,2)
plot(tt,vd,t([lcp lcm]),[zcp zcm],'or')
xlabel('Time (s)')
ylabel('Speed (cm/s)')
grid
```



Differentiate the drift speed to find the acceleration. The lag is twice as long. Discard twice as many samples to compensate for the delay, and the same number to eliminate the transient. Plot the speed and acceleration.

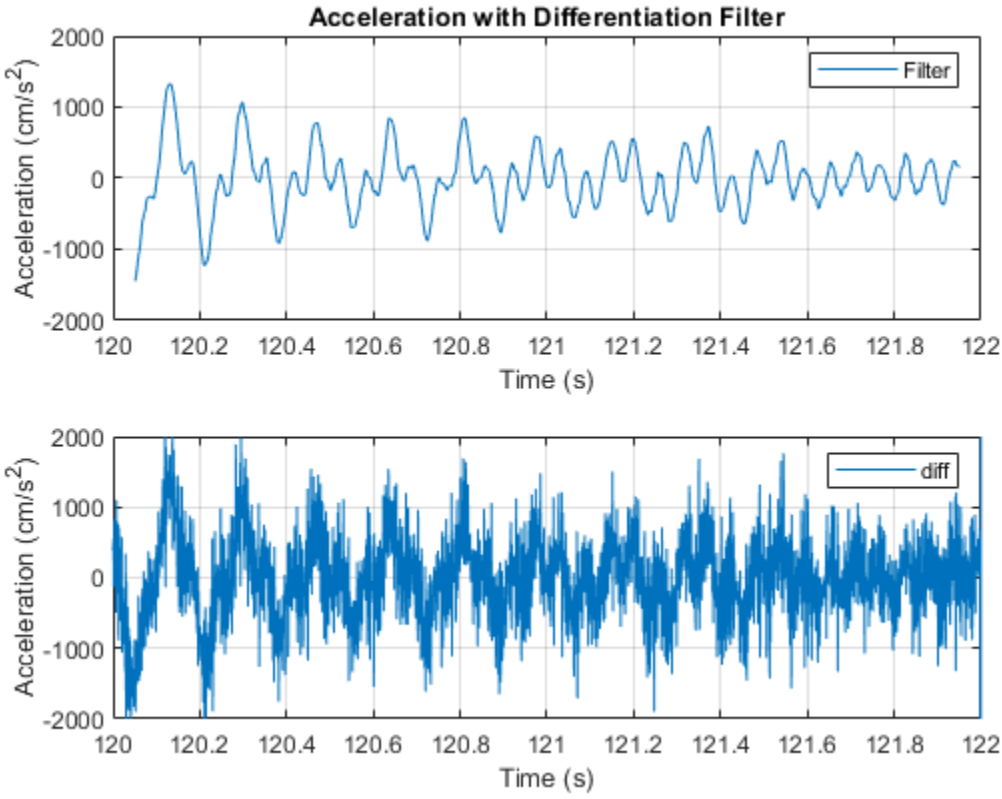
```
adrift = filter(d,vdrift)/dt;

at = t(1:end-2*delay);
ad = adrift;
ad(1:2*delay) = [];

at(1:2*delay) = [];
ad(1:2*delay) = [];

subplot(2,1,1)
```

```
plot(tt,vd)
xlabel('Time (s)')
ylabel('Speed (cm/s)')
grid

subplot(2,1,2)
plot(at,ad)
ax = gca;
ax.YLim = 2000*[-1 1];
xlabel('Time (s)')
ylabel('Acceleration (cm/s^2)')
grid
```



Compute the acceleration using `diff`. Add zeros to compensate for the change in array size. Compare the result to that obtained with the filter. Notice the amount of high-frequency noise.

```
vdiff = diff([drift;0])/dt;
adiff = diff([vdiff;0])/dt;

subplot(2,1,1)
plot(at,ad)
ax = gca;
ax.YLim = 2000*[-1 1];
xlabel('Time (s)')
ylabel('Acceleration (cm/s^2)')
grid
legend('Filter')
title('Acceleration with Differentiation Filter')

subplot(2,1,2)
plot(t,adiff)
ax = gca;
ax.YLim = 2000*[-1 1];
xlabel('Time (s)')
ylabel('Acceleration (cm/s^2)')
grid
legend('diff')
```



See Also

`designfilt` | `findpeaks` | `fvtool` | `grpdelay` | `periodogram`

Related Examples

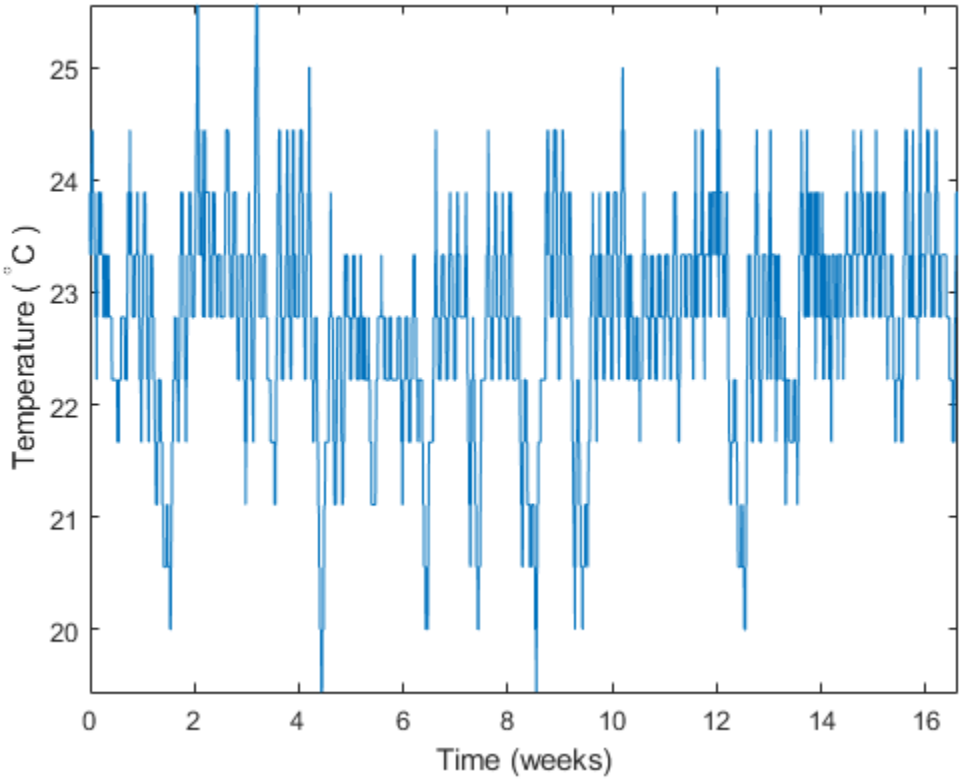
- “Practical Introduction to Digital Filtering”

Find Periodicity Using Frequency Analysis

It is often difficult to characterize oscillatory behavior in data by looking at time measurements. Spectral analysis can help determine if a signal is periodic and measure the different cycles.

A thermometer in an office building measures the inside temperature every half hour for four months. Load the data and plot it. Convert the temperature to degrees Celsius. Measure time in weeks. The sample rate is thus 2 measurements/hour × 24 hours/day × 7 days/week = 336 measurements/week.

```
load officetemp

tempC = (temp - 32)*5/9;

fs = 2*24*7;
t = (0:length(tempC) - 1)/fs;

plot(t,tempC)
xlabel('Time (weeks)')
ylabel('Temperature ( {}^\circC )')
axis tight
```

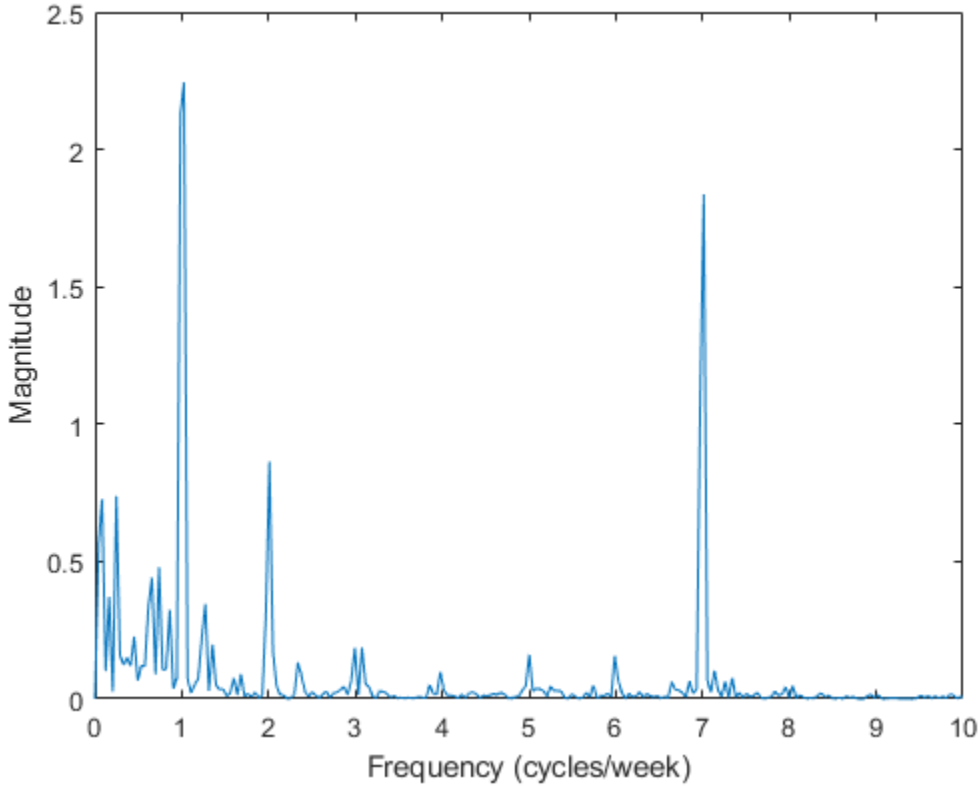


The temperature does seem to oscillate, but the lengths of the cycles cannot be determined easily. Look at the signal's frequency content instead.

Subtract the mean to concentrate on temperature fluctuations. Compute and plot the periodogram.

```
tempnorm = tempC - mean(tempC);

[pxx,f] = periodogram(tempnorm,[],[],fs);

plot(f,pxx)
ax = gca;
ax.XLim = [0 10];
```

```
xlabel('Frequency (cycles/week)')
ylabel('Magnitude')
```



The temperature clearly has a daily cycle and a weekly cycle. The result is not surprising: the temperature is higher when people are at work and lower at nights and on weekends.

See Also

`findpeaks` | `periodogram` | `xcorr`

Related Examples

- “Find Periodicity Using Autocorrelation” on page 20-59
- “Practical Introduction to Frequency-Domain Analysis”

Detect a Distorted Signal in Noise

The presence of noise often makes it difficult to determine the spectral content of a signal. Frequency analysis can help in such cases.

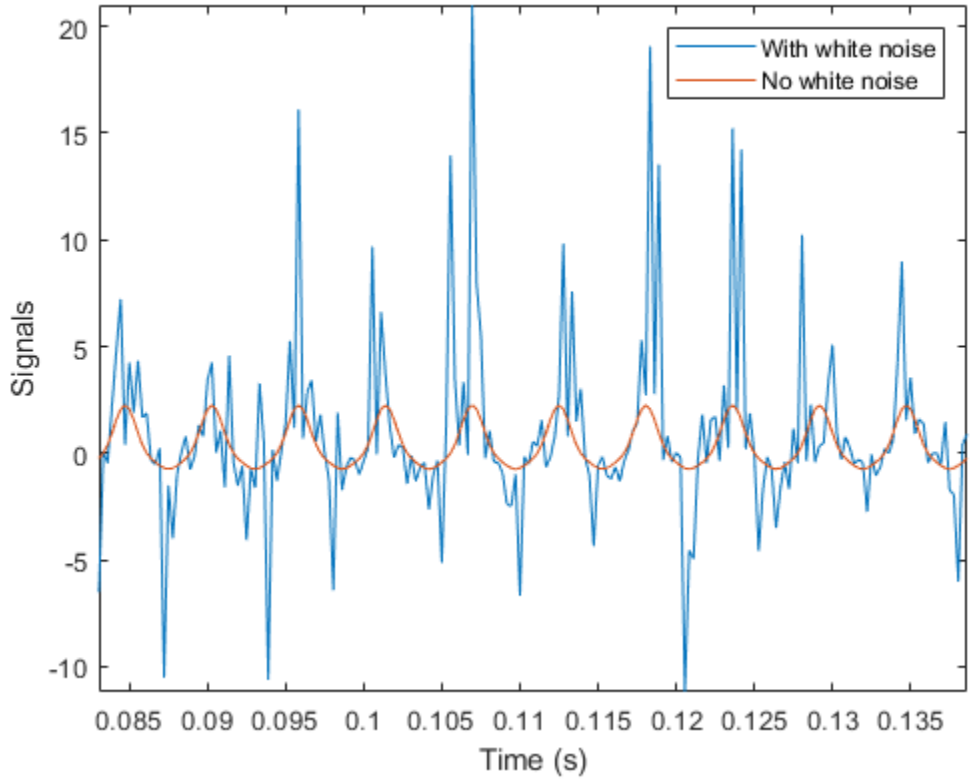
Consider for example the simulated output of a nonlinear amplifier that introduces third-order distortion.

The input signal is a 180 Hz unit-amplitude sinusoid sampled at 3.6 kHz. Generate 10000 samples.

```
N = 1e4;
n = 0:N-1;
fs = 3600;
f0 = 180;
t = n/fs;
y = sin(2*pi*f0*t);
```

Add unit-variance white noise to the input. Model the amplifier using a third-order polynomial. Pass the input signal through the amplifier using `polyval`. Plot a section of the output. For comparison plot the output of a pure sinusoid.

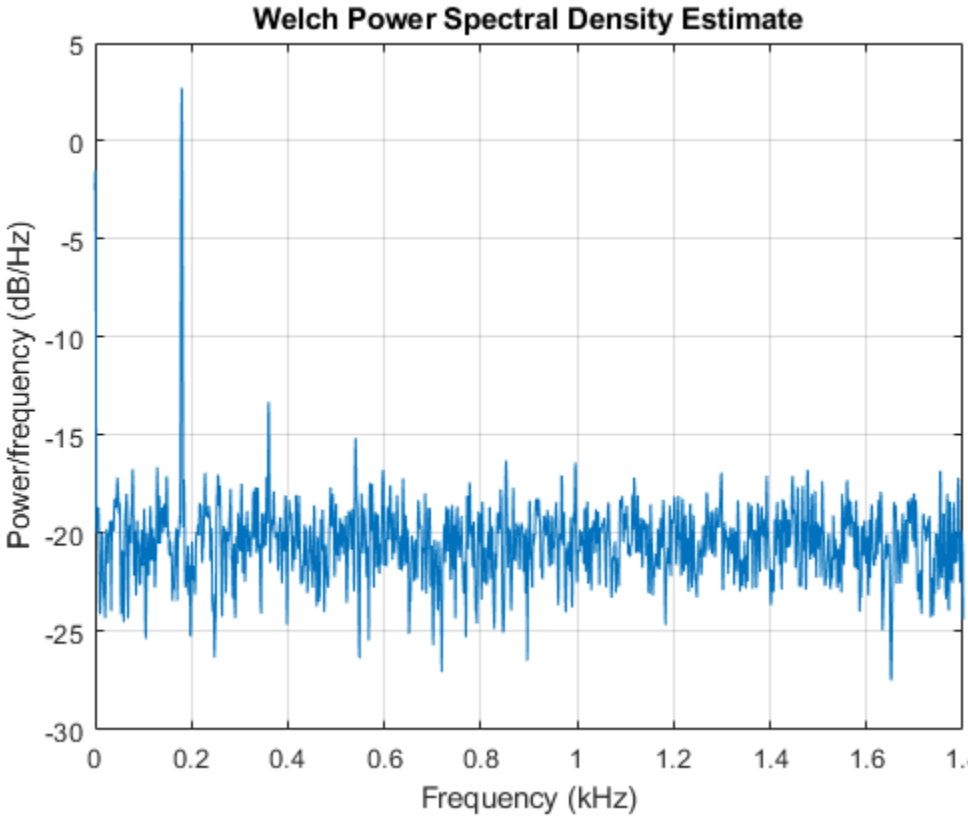
```
rng default
noise = randn(size(y));

dispol = [0.5 0.75 1 0];
out = polyval(dispol,y+noise);

ns = 300:500;

plot(t(ns),[out(ns);polyval(dispol,y(ns))])
xlabel('Time (s)')
ylabel('Signals')
axis tight
legend('With white noise','No white noise')
```



Use `pwelch` to compute and plot the power spectral density of the output.

```
[pxx,f] = pwelch(out,[],[],[],fs);
```

```
pwelch(out,[],[],[],fs)
```



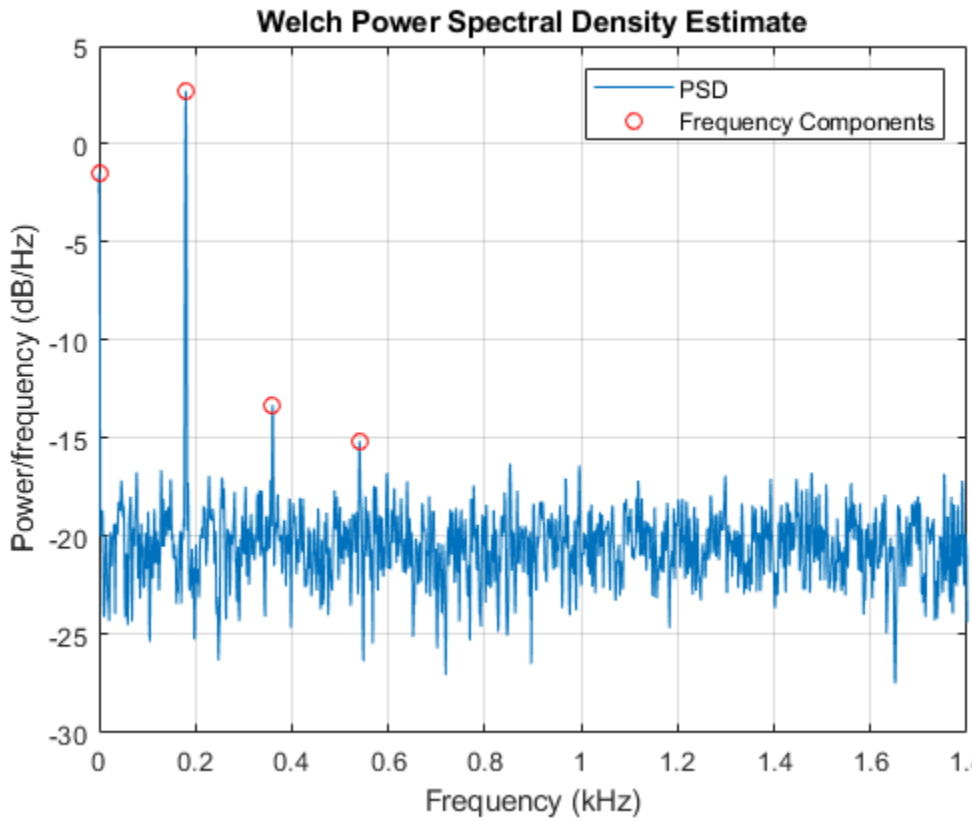
Because the amplifier introduces third-order distortion, the output signal is expected to have:

- A *DC* (zero-frequency) component;
- A *fundamental* component with the same frequency as the input, 180 Hz;
- Two *harmonics* -- frequency components at twice and three times the frequency of the input, 360 and 540 Hz.

Verify that the output is as expected for a cubic nonlinearity.

```
[pks,lox] = findpeaks(pxx,'NPeaks',4,'SortStr','descend');

hold on
```

```
plot(f(lox)/1000,10*log10(pks),'or')
hold off

legend('PSD','Frequency Components')
```



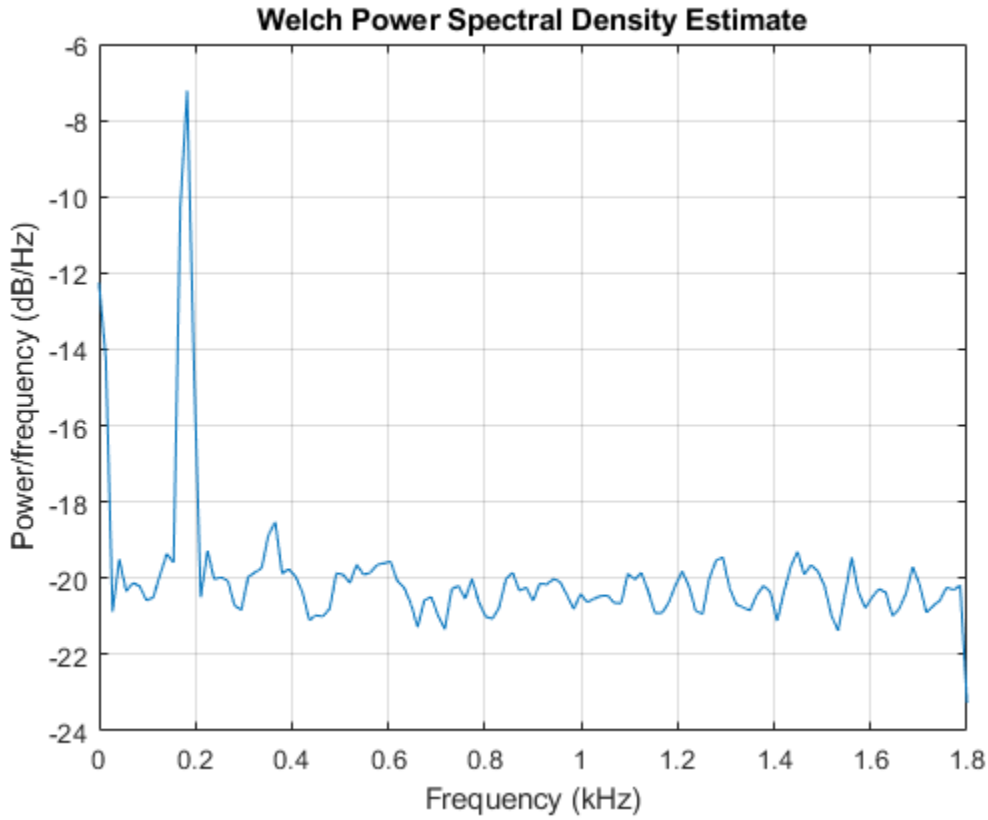

```
components = sort([f(lox) f0*(0:3)'])'
```

```
components = 2×4

    0.8789  180.1758  360.3516  540.5273
         0  180.0000  360.0000  540.0000
```

`pwelch` works by dividing the signal into overlapping segments, computing the periodogram of each segment, and averaging. By default, the function uses eight segments with 50% overlap. For 10000 samples, this corresponds to 2222 samples per segment.

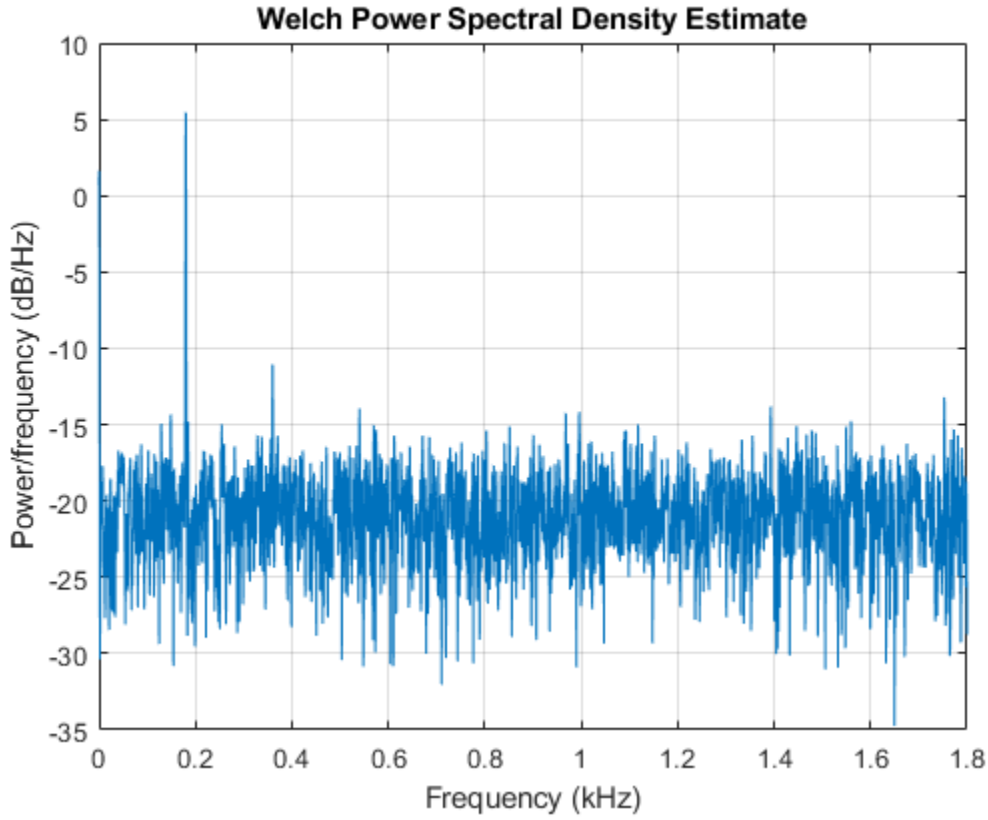
Dividing the signal into shorter segments results in more averaging. The periodogram is smoother, but has lower resolution. The higher harmonic cannot be distinguished.

```
pwelch(out,222,[],[],fs)
```



Dividing the signal into longer segments increases the resolution, but also the randomness. The signal and the harmonics are precisely at the expected locations. However, there is at least one spurious high-frequency peak with more power than the higher harmonic.

```
pwelch(out,4444,[],[],fs)
```



See Also

`findpeaks` | `pwelch`

Related Examples

- “Practical Introduction to Frequency-Domain Analysis”

Measure the Power of a Signal

The power of a signal is the sum of the absolute squares of its time-domain samples divided by the signal length, or, equivalently, the square of its RMS level. The function `bandpower` allows you to estimate signal power in one step.

Consider a unit chirp embedded in white Gaussian noise and sampled at 1 kHz for 1.2 seconds. The chirp's frequency increases in one second from an initial value of 100 Hz to 300 Hz. The noise has variance 0.01^2. Reset the random number generator for reproducible results.

```
N = 1200;
Fs = 1000;
t = (0:N-1)/Fs;

sigma = 0.01;
rng('default')

s = chirp(t,100,1,300)+sigma*randn(size(t));
```

Verify that the power estimate given by `bandpower` is equivalent to the definition.

```
pRMS = rms(s)^2
```

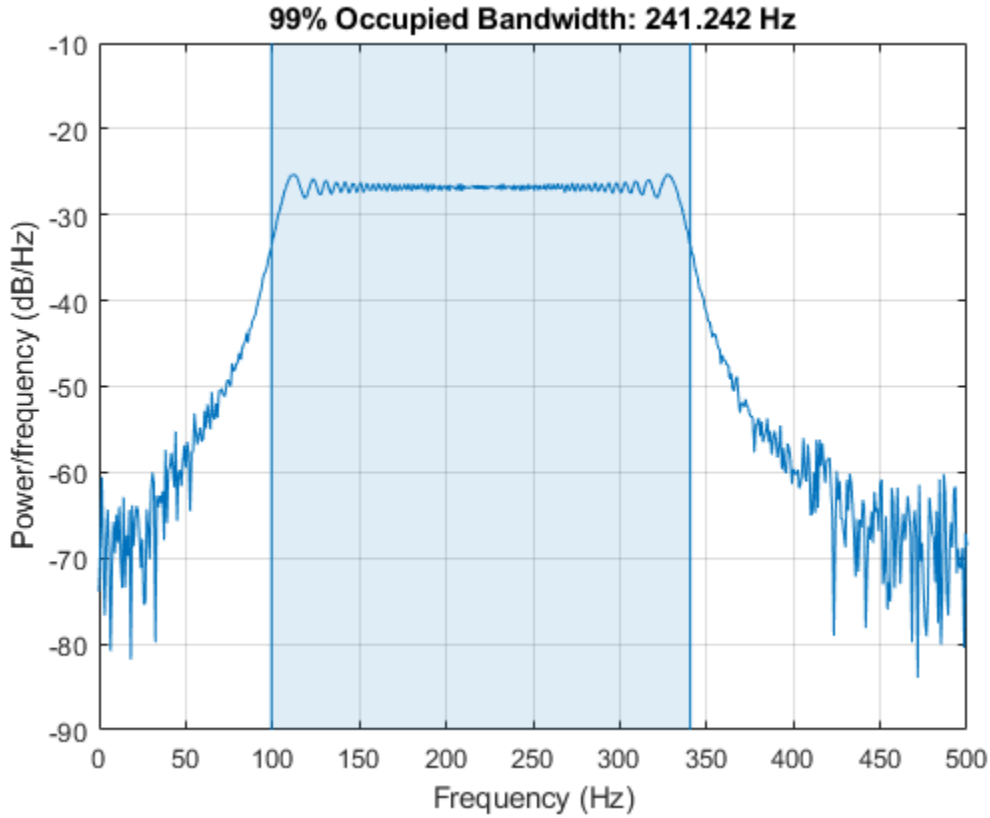
```
pRMS = 0.5003
```

```
powbp = bandpower(s,Fs,[0 Fs/2])
```

```
powbp = 0.5005
```

Use the `obw` function to estimate the width of the frequency band that contains 99% of the power of the signal, the lower and upper bounds of the band, and the power in the band. The function also plots the spectrum estimate and annotates the occupied bandwidth.

```
obw(s,Fs);
```




```
[wd,lo,hi,power] = obw(s,Fs);
powtot = power/0.99
```

```
powtot = 0.5003
```

A nonlinear power amplifier is given a 60 Hz sinusoid as input and outputs a noisy signal with third-order distortion. The sample rate is 3.6 kHz. Subtract the zero-frequency (DC) component to concentrate on the spectral content.

```
load(fullfile(matlabroot,'examples','signal','AmpOutput.mat'))
Fs = 3600;
y = y-mean(y);
```

Because the amplifier introduces third-order distortion, the output signal is expected to have

- A *fundamental* component with the same frequency as the input, 60 Hz;
- Two *harmonics* -- frequency components at twice and three times the frequency of the input, 120 and 180 Hz.

Use `bandpower` to determine the power stored in the fundamental and the harmonics. Express each value as a percentage of the total power and in decibels. Display the values as a table.

```
pwrTot = bandpower(y,Fs,[0 Fs/2]);

Harmonic = {'Fundamental';'First';'Second'};

Freqs = [60 120 180]';

Power = zeros([3 1]);
for k = 1:3
    Power(k) = bandpower(y,Fs,Freqs(k)+[-10 10]);
end

Percent = Power/pwrTot*100;

inDB = pow2db(Power);

T = table(Freqs,Power,Percent,inDB,'RowNames',Harmonic)
```

```
T=3×4 table
                   Freqs     Power      Percent      inDB
                   _____    ________    _______    ________

    Fundamental      60       1.0079    12.563     0.034136
    First           120      0.14483    1.8053      -8.3914
    Second          180     0.090023    1.1221      -10.456
```

See Also

`bandpower` | `pow2db` | `pwelch` | `snr`

Related Examples

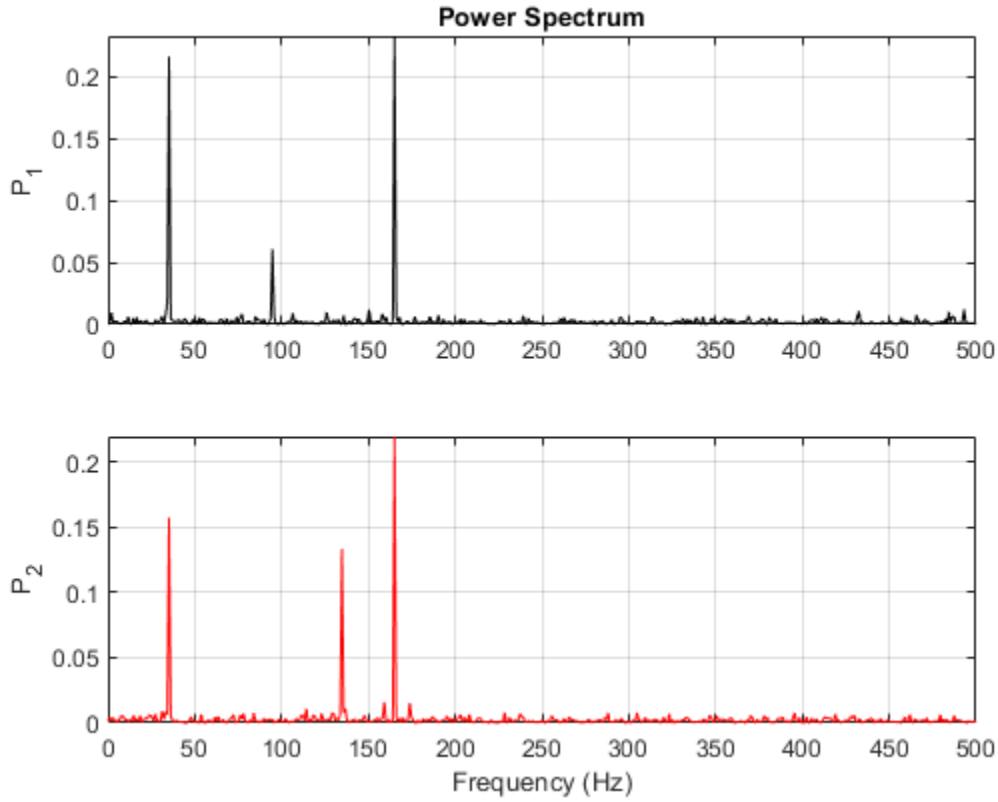
- “Practical Introduction to Frequency-Domain Analysis”

Compare the Frequency Content of Two Signals

Spectral coherence helps identify similarity between signals in the frequency domain. Large values indicate frequency components common to the signals.

Load two sound signals into the workspace. They are sampled at 1 kHz. Compute their power spectra using `periodogram` and plot them next to each other.

```
load relatedsig

Fs = FsSig;

[P1,f1] = periodogram(sig1,[],[],Fs,'power');
[P2,f2] = periodogram(sig2,[],[],Fs,'power');

subplot(2,1,1)
plot(f1,P1,'k')
grid
ylabel('P_1')
title('Power Spectrum')

subplot(2,1,2)
plot(f2,P2,'r')
grid
ylabel('P_2')
xlabel('Frequency (Hz)')
```



Each signal has three frequency components with significant energy. Two of those components appear to be shared. Find the corresponding frequencies using `findpeaks`.

```
[pk1,lc1] = findpeaks(P1,'SortStr','descend','NPeaks',3);
P1peakFreqs = f1(lc1)
```

```
P1peakFreqs = 3×1

  165.0391
   35.1563
   94.7266
```

```
[pk2,lc2] = findpeaks(P2,'SortStr','descend','NPeaks',3);
P2peakFreqs = f2(lc2)
```

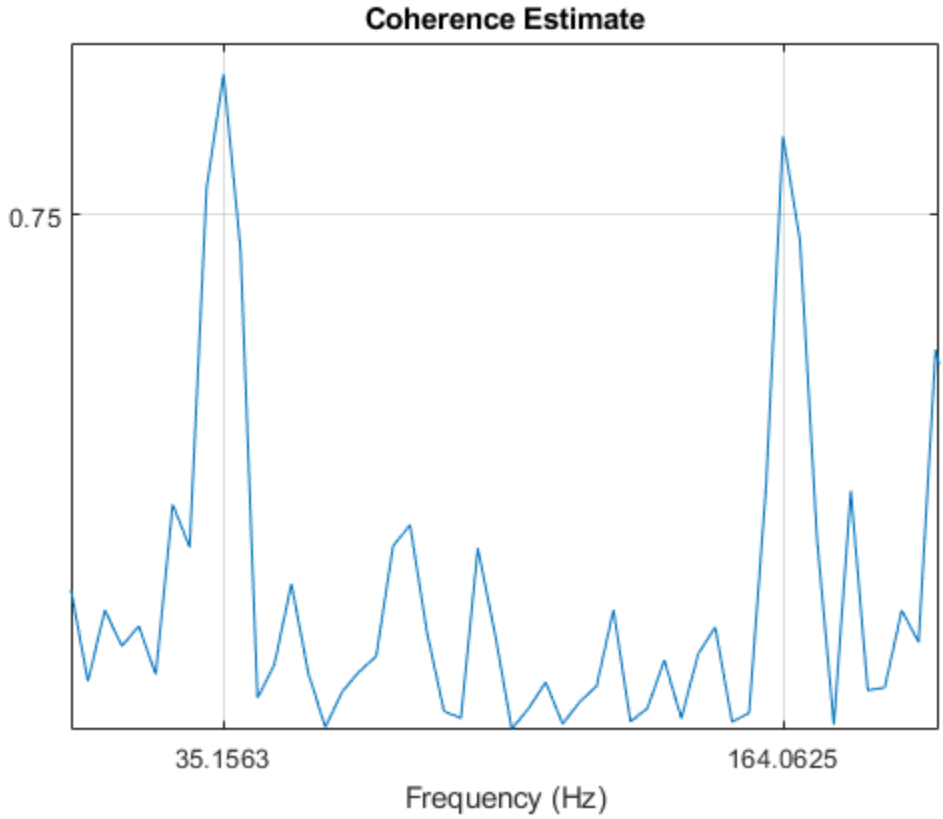
```
P2peakFreqs = 3×1

  165.0391
   35.1563
  134.7656
```

The common components are located around 165 and 35 Hz. You can use `mscohere` to find the matching frequencies directly. Plot the coherence estimate. Find the peaks above a threshold of 0.75.

```
[Cxy,f] = mscohere(sig1,sig2,[],[],[],Fs);

thresh = 0.75;
[pks,locs] = findpeaks(Cxy,'MinPeakHeight',thresh);
MatchingFreqs = f(locs)
```

```
MatchingFreqs = 2×1

   35.1563
  164.0625
```

```
figure
plot(f,Cxy)
ax = gca;
grid
xlabel('Frequency (Hz)')
title('Coherence Estimate')
ax.XTick = MatchingFreqs;
ax.YTick = thresh;
axis([0 200 0 1])
```



You get the same values as before. You can find the frequency content common to two signals without studying the two signals separately.

See Also

`findpeaks` | `mscohere` | `periodogram`

Related Examples

- “Practical Introduction to Frequency-Domain Analysis”

Detect Periodicity in a Signal with Missing Samples

Consider the weight of a person as recorded (in pounds) during the leap year 2012. The person did not record their weight every day. You would like to study the periodicity of the signal, even though some data points are missing.

Load the data and convert the measurements to kilograms. Missed readings are set to `NaN`. Determine how many points are missing.

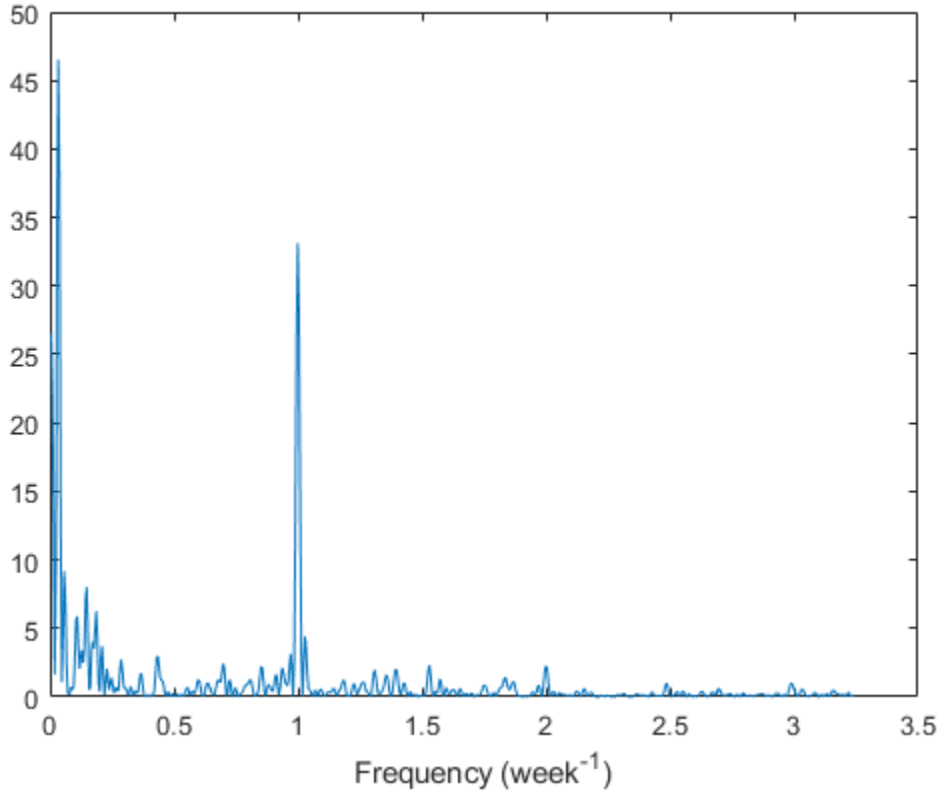
```
load(fullfile(matlabroot,'examples','signal','weight2012.dat'))

wgt = weight2012(:,2)/2.20462;

fprintf('Missing %d samples of %d\n',sum(isnan(wgt)),length(wgt))
```

```
Missing 27 samples of 366
```

Determine if the signal is periodic by analyzing it in the frequency domain. The Lomb-Scargle algorithm is designed to handle data with missing samples or data that have been sampled irregularly.

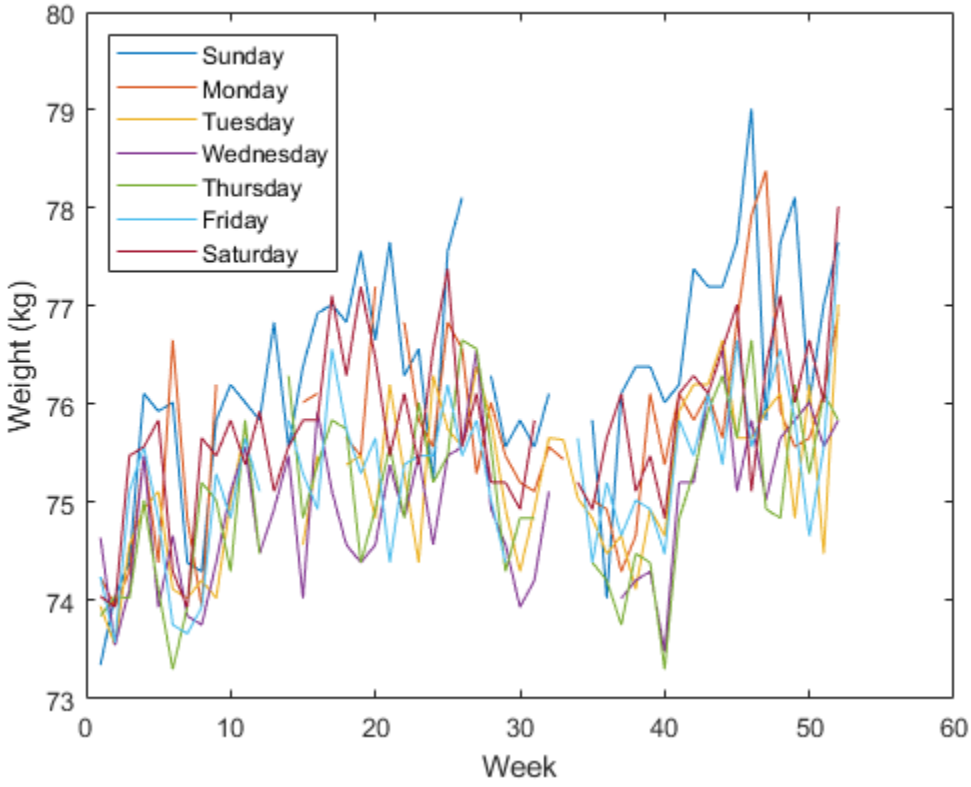
Find the cycle durations, measuring time in weeks.

```
[p,f] = plomb(wgt,7,'normalized');

plot(f,p)
xlabel('Frequency (week^{-1})')
```



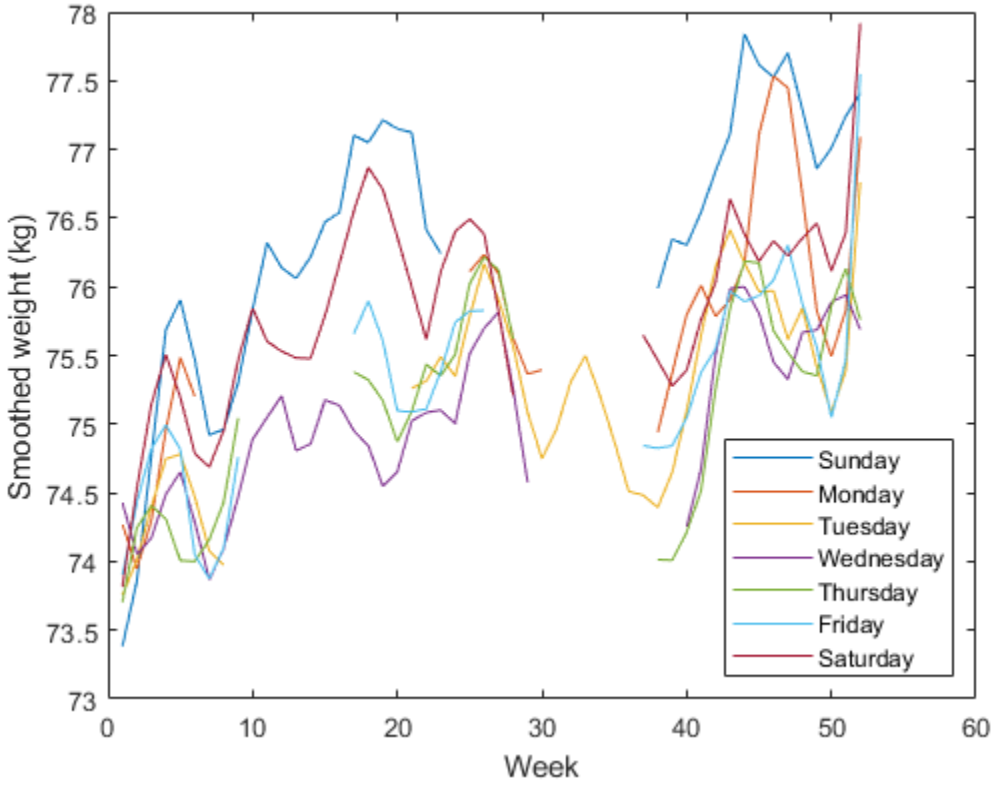
Notice how the person's weight oscillates weekly. Is there a noticeable pattern from week to week? Eliminate the last two days of the year to get 52 weeks. Reorder the measurements according to the day of the week.

```
wgd = reshape(wgt(1:7*52),[7 52])';

plot(wgd)
xlabel('Week')
ylabel('Weight (kg)')

q = legend(datestr(datenum(2012,1,1:7),'dddd'));
q.Location = 'NorthWest';
```



Smooth out the fluctuations using a filter that fits low-order polynomials to subsets of the data. Specifically, set it to fit cubic polynomials to sets of seven days.

```
wgs = sgolayfilt(wgd,3,7);

plot(wgs)
xlabel('Week')
ylabel('Smoothed weight (kg)')

q = legend(datestr(datenum(2012,1,1:7),'dddd'));
q.Location = 'SouthEast';
```



This person tends to eat more, and thus weigh more, during the weekend. Verify by computing the daily means. Exclude the missing values from the calculation.

```
for jk = 1:7
    wgm = find(~isnan(wgd(:,jk)));
    fprintf('%3s mean: %5.1f kg\n', ...
        datestr(datenum(2012,1,jk),'ddd')',mean(wgd(wgm,jk)))
end
```

```
Sun mean:  76.3 kg
Mon mean:  75.7 kg
Tue mean:  75.2 kg
Wed mean:  74.9 kg
Thu mean:  75.1 kg
```

```
Fri mean:  75.3 kg
Sat mean:  75.8 kg
```

See Also

`datestr` | `plomb` | `sgolayfilt`

Related Examples

- “Signal Smoothing”

Technical Conventions

This manual and all Signal Processing Toolbox functions use the following technical notations.

Term or Symbol	Description
Nyquist frequency	One-half the sampling frequency. Some toolbox functions normalize this value to 1.
`x(1)`	The first element of a data sequence or filter, corresponding to zero lag.
Ω or w	Analog frequency in radians per second.
ω or w	Digital frequency in radians per sample.
`f`	Digital frequency in hertz.
[*x*, *y*)	The interval from *x* to *y*, including *x* but not including *y*.
`...`	Ellipses in the argument list for a given syntax on a function reference page indicate all possible argument lists for that function appearing prior to the given syntax are valid.